

Complete Application Package

Uni-Cal Academic Dashboard

Self-Hosting Guide + Complete Source Code

Updated: March 28, 2026

132 source files • React/TypeScript + Express/PostgreSQL
Full-stack academic task management with HA integration

Complete Application Package

Document Contents

- 1. Self-Hosting Guide (setup, HA YAML, automation flows A-I)
 - 2. Complete Source Code (every file in the application)
-

PART 1: SELF-HOSTING GUIDE

Self-Hosting Guide: Dashboard App on Raspberry Pi

Table of Contents

Part 1: Setup

1. [Can I Run This on My Home Assistant Laptop?](#can-i-run-this-on-my-home-assistant-laptop)
2. [Equipment to Buy (Amazon Links)](#equipment-to-buy)
3. [Step 1: Flash the Operating System](#step-1-flash-the-operating-system)
4. [Step 2: First Boot](#step-2-first-boot)
5. [Step 3: Install System Dependencies](#step-3-install-system-dependencies)
6. [Step 4: Set Up the Database](#step-4-set-up-the-database)
7. [Step 5: Create the App Directory](#step-5-create-the-app-directory)
8. [Step 6: Get the Code Out of Replit](#step-6-get-the-code-out-of-replit)
9. [Step 7: Set Up Environment Variables](#step-7-set-up-environment-variables)
10. [Step 8: Setting Up OAuth Credentials](#step-8-setting-up-oauth-credentials-the-hard-part)
11. [Step 9: Initialize the Database](#step-9-initialize-the-database)
12. [Step 10: Build and Test](#step-10-build-and-test)
13. [Step 11: Set Up Auto-Start (systemd Service)](#step-11-set-up-auto-start-systemd-service)
14. [Step 12: Set Up Log Rotation](#step-12-set-up-log-rotation)
15. [Step 13: Update Home Assistant Webhooks](#step-13-update-home-assistant-webhooks)
16. [Step 14: Update Google Apps Script](#step-14-update-google-apps-script)
17. [Step 15: Touchscreen Setup](#step-15-touchscreen-setup)

Part 2: Ongoing Maintenance

18. [Viewing Logs](#viewing-logs)
19. [Restarting / Updating / Backing Up](#restarting-the-app)
20. [Key Differences from Replit](#key-differences-from-replit)
21. [Estimated Migration Effort](#estimated-migration-effort)

Part 3: Home Assistant YAML Reference

22. [REST Commands (configuration.yaml)](#rest-commands-configurationyaml)
23. [HA Automations (automations.yaml)](#ha-automations-automationsyaml)
24. [HA Input Booleans](#ha-input-booleans-configurationyaml)

Part 4: Cat Washroom Study Reading System — Complete Flow

25. [Devices Involved](#devices-involved)
26. [Flow A: Lights Turn ON — Full Prompt Flow](#flow-a-lights-turn-on--full-prompt-flow)
27. [Flow B: Lights Turn OFF — Stop Everything](#flow-b-lights-turn-off--stop-everything)
28. [Flow C: Confirmed Playback Flow (Reading the PDF)](#flow-c-confirmed-playback-flow-reading-the-pdf)
29. [Flow D: Shower Button — Direct Play](#flow-d-shower-button--direct-play-skip-prompt)
30. [Flow E: Volume Knob](#flow-e-volume-knob--adjust-active-speaker-volume)
31. [Flow F: Knob Press — Master STOP](#flow-f-knob-press--master-stop)
32. [Flow G: Toothbrush Auto-Stop](#flow-g-toothbrush-auto-stop)
33. [Flow H: Voice Commands (pause/resume/stop/skip/restart/reset)](#flow-h-voice-commands)
34. [Flow I: Play Urgent PDF](#flow-i-play-urgent-pdf-on-demand)
35. [TTS Fallback Chain](#tts-fallback-chain)
36. [Background Processes](#background-processes-no-webhook-needed)

Can I Run This on My Home Assistant Laptop?

No, not directly. Home Assistant OS is a locked-down, purpose-built Linux distribution — it manages its own filesystem, networking, and containers. You can't install Node.js, PostgreSQL, or run arbitrary apps alongside it. It's designed to only run HA and its add-ons. You'd need to replace HAOS with a regular Linux install to do that, which would mean rebuilding your entire HA setup.

A Raspberry Pi 5 on the same local network is the right approach — fast, cheap, always-on, and won't interfere with your HA installation.

Equipment to Buy

Item	Model / Spec	Approx. Price	Amazon Link
***Raspberry Pi 5**	***8 GB RAM**	~\$80 USD	[amazon.ca/dp/B0CK2FCG1K](https://www.amazon.ca/Raspberry-Pi-8GB-2023-Processo...)
***Power Supply**	Official Raspberry Pi 5 27W USB-C	~\$12	[amazon.ca/dp/B0CN1HP2P7](https://www.amazon.ca/dp/B0CN1HP2P7)
***microSD Card**	Samsung EVO Select 128 GB (A2-rated)	~\$13	[amazon.ca/dp/B09B1HJM9Z](https://www.amazon.ca/dp/B09B1HJM9Z)
***Active Cooler**	Official Raspberry Pi 5 Active Cooler	~\$5	[amazon.ca/dp/B0CN1GXRKQ](https://www.amazon.ca/dp/B0CN1GXRKQ)
***Case**	Official Pi 5 Case (Red/White)	~\$10	[amazon.ca/dp/B0CN1HP2RZ](https://www.amazon.ca/dp/B0CN1HP2RZ)
***Ethernet Cable**	Cat 6, 3-foot or whatever length you need	~\$5	[amazon.ca/dp/B00N2VILDM](https://www.amazon.ca/dp/B00N2VILDM)
***USB microSD Reader**	Anker USB 3.0 Card Reader	~\$8	[amazon.ca/dp/B006T9B6R2](https://www.amazon.ca/dp/B006T9B6R2)

Total: ~\$125–135 USD

Notes:

- The 8 GB Pi 5 is essential — Node.js + PostgreSQL + TTS audio processing need the memory. 4 GB would be tight.
- The official 27W power supply is important — underpowered supplies cause random crashes and SD card corruption.
- The microSD must be A2-rated for the random read/write speed needed by PostgreSQL.
- The active cooler is necessary because TTS processing pushes the CPU under sustained load. A passive heatsink alone won't cut it.
- If your PC already has a microSD slot, skip the USB reader.

Setup Instructions

Step 1: Flash the Operating System

1. On your regular computer, download **Raspberry Pi Imager** from [raspberrypi.com/software](https://www.raspberrypi.com/software/)
2. Insert the microSD card into your computer
3. Open the Imager and choose:
 - Device: Raspberry Pi 5
 - OS: Raspberry Pi OS (64-bit, Lite) — no desktop environment needed
4. Click the **gear icon** before flashing and configure:
 - Enable SSH: Yes
 - Username: pi
 - Password: (set a strong password)
 - Wi-Fi: Enter your network name and password (as backup even if using ethernet)
 - Hostname: dashboard-server
5. Flash the card and wait for verification to complete

Step 2: First Boot

1. Insert the microSD card into the Pi 5
2. Connect the ethernet cable to your router
3. Plug in the power supply
4. Wait approximately 2 minutes for the first boot to complete
5. From your computer, open a terminal and connect:

```
`bash
ssh pi@dashboard-server.local
`
```

If that doesn't resolve, check your router's admin page for the Pi's IP address and use:

```
`bash
ssh pi@<IP_ADDRESS>
`
```

6. Enter the password you set during flashing

Step 3: Install System Dependencies

Run each block of commands in order:

```
# Update the system
sudo apt update && sudo apt upgrade -y

# Install Node.js 20
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# Install PostgreSQL
sudo apt install -y postgresql postgresql-contrib

# Install build tools (required for some npm packages)
sudo apt install -y build-essential git

# Install espeak-ng (local offline TTS fallback)
sudo apt install -y espeak-ng

# Install Edge TTS (Microsoft TTS fallback)
sudo apt install -y python3-pip
pip3 install edge-tts --break-system-packages
```

Verify installations:

```
node --version      # Should show v20.x.x
psql --version      # Should show 16.x
espeak-ng --version # Should show espeak-ng version
edge-tts --version  # Should show edge-tts version
```

Step 4: Set Up the Database

```
# Create the database user
sudo -u postgres psql -c "CREATE USER dashboard WITH PASSWORD 'CHOOSE_A_STRONG_PASSWORD';"

# Create the database
sudo -u postgres psql -c "CREATE DATABASE dashboard_db OWNER dashboard;"

# Grant privileges
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE dashboard_db TO dashboard;"
```

Replace `CHOOSE_A_STRONG_PASSWORD` with an actual secure password. Write it down — you'll need it for the environment file.

Step 5: Create the App Directory

```
sudo mkdir -p /opt/dashboard
sudo chown pi:pi /opt/dashboard
cd /opt/dashboard
```

Step 6: Get the Code Out of Replit

Your entire app — every file, every line of code — is yours and can be downloaded at any time. Here are three ways to get it:

Option A — Download as ZIP (easiest, no account needed):

1. In Replit, look at the left sidebar where your files are listed
2. Click the ****three dots (...)**** at the top of the file panel
3. Click ****"Download as ZIP"****
4. A `.zip`` file will download to your computer (probably to your Downloads folder)
5. That ZIP contains everything: all the server code, frontend code, database schema, configuration files, this guide — all

of it

To transfer the ZIP to your Pi, open a terminal on your regular computer and run:

```
scp ~/Downloads/home-view.zip pi@dashboard-server.local:/opt/dashboard/
```

(If your computer is Windows, you can use WinSCP or drag-and-drop the file using FileZilla instead.)

Then on the Pi (SSH in first):

```
cd /opt/dashboard
sudo apt install -y unzip      # install unzip if not already there
unzip home-view.zip
npm install
```

Option B — Push to GitHub first (better for ongoing updates):

1. In Replit, click the ****Git**** icon in the left sidebar (the branch icon)
2. Click ****"Connect to GitHub"***** and follow the prompts to create a repository
3. Once connected, all your code is on GitHub — you can access it from anywhere, forever

Then on the Pi:

```
cd /opt/dashboard
git clone https://github.com/YOUR_USERNAME/YOUR_REPO.git .
npm install
```

Future updates become easy: edit on Replit, push to GitHub, then `git pull && npm install && npm run build && sudo systemctl restart dashboard` on the Pi.

Option C — Copy files directly via SCP (no ZIP needed):

If you just want to copy the project folder from your computer after downloading:

```
scp -r ~/Downloads/home-view/* pi@dashboard-server.local:/opt/dashboard/
```

What you're getting:

The download includes absolutely everything that makes the app work:

- ``server/routes.ts`` — the entire backend (all webhooks, TTS, Spotify, calendar, cat washroom logic)
- ``server/storage.ts`` — database operations
- ``shared/schema.ts`` — database table definitions
- ``client/src/`` — the entire frontend (dashboard, Spotify player, PDF reader, OneNote, settings)
- ``package.json`` — all the dependency definitions (npm install will download them)
- ``Self_Hosting_Guide.md`` — this guide
- ``HA_Automations_Reference.md`` — the full HA automations reference
- Everything else: configuration files, assets, styles

The code is fully self-contained. You don't need Replit to run it — it's a standard Node.js + PostgreSQL app that runs anywhere.

Step 7: Set Up Environment Variables

Create the environment file:

```
nano /opt/dashboard/.env
```

Add the following (fill in your actual values):

```
# Database
DATABASE_URL=postgresql://dashboard:CH00SE_A_STR0NG_PASSWORD@localhost:5432/dashboard_db

# Home Assistant
HOME_ASSISTANT_TOKEN=your_long_lived_access_token_from_ha

# App URL (update to Pi's address)
DEPLOYED_APP_URL=http://dashboard-server.local:5000

# Google Calendar OAuth (from Google Cloud Console)
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
GOOGLE_REFRESH_TOKEN=your_google_refresh_token

# Spotify OAuth (from developer.spotify.com)
SPOTIFY_CLIENT_ID=your_spotify_client_id
SPOTIFY_CLIENT_SECRET=your_spotify_client_secret
SPOTIFY_REFRESH_TOKEN=your_spotify_refresh_token

# Microsoft / OneDrive / Outlook (from Azure portal)
MICROSOFT_CLIENT_ID=your_azure_client_id
MICROSOFT_CLIENT_SECRET=your_azure_client_secret
MICROSOFT_REFRESH_TOKEN=your_microsoft_refresh_token

# Gmail (from Google Cloud Console — can be same project as Calendar)
GMAIL_CLIENT_ID=your_gmail_client_id
GMAIL_CLIENT_SECRET=your_gmail_client_secret
GMAIL_REFRESH_TOKEN=your_gmail_refresh_token
```

Save and exit (Ctrl+X, then Y, then Enter).

Secure the file:

```
chmod 600 /opt/dashboard/.env
```

Step 8: Setting Up OAuth Credentials (The Hard Part)

On Replit, the integration connectors handle OAuth automatically. On the Pi, you need to register apps with each service and get refresh tokens manually.

Google (Calendar + Gmail)

1. Go to console.cloud.google.com

2. Create a new project (or use existing)
3. Enable **Google Calendar API** and **Gmail API**
4. Go to **Credentials**, then Create **OAuth 2.0 Client ID**
5. Application type: **Web application**
6. Add redirect URI: `http://dashboard-server.local:5000/api/google/callback``
7. Copy the Client ID and Client Secret into your `.env``
8. To get a refresh token, you'll need to do a one-time OAuth flow — open this URL in a browser (replace YOUR_CLIENT_ID):

`https://accounts.google.com/o/oauth2/v2/auth?client_id=YOUR_CLIENT_ID&redirect_uri=http://dashboard-server.local:5000/api/google/callback&response_type=code&scope=https://www.googleapis.com/auth/calendar.readonly%20https://www.googleapis.com/auth/gmail.readonly&access_type=offline&prompt=consent``

9. After authorizing, the callback will receive an authorization code. Exchange it for tokens.

Spotify

1. Go to developer.spotify.com/dashboard
2. Create an app
3. Add redirect URI: `http://dashboard-server.local:5000/api/spotify/callback``
4. Copy Client ID and Client Secret into your `.env``
5. Do the OAuth flow similar to Google to get a refresh token

Microsoft (OneDrive + Outlook)

1. Go to portal.azure.com then Azure Active Directory, then App registrations
2. Register a new application
3. Add redirect URI: `http://dashboard-server.local:5000/api/microsoft/callback``
4. Under **Certificates & secrets**, create a new client secret
5. Under **API permissions**, add: `Files.ReadWrite``, `Mail.Read``, `Calendars.Read``
6. Copy Client ID and Client Secret into your `.env``
7. Do the OAuth flow to get a refresh token

Step 9: Initialize the Database

```
cd /opt/dashboard
npm run db:push
```

This creates all the tables. If it asks for confirmation, type yes.

Step 10: Build and Test

```
# Build the app
npm run build

# Test run (foreground — you'll see logs)
node dist/index.js
```

Open a browser on your network and go to:

`http://dashboard-server.local:5000`

Verify the dashboard loads. Press Ctrl+C to stop the test.

Step 11: Set Up Auto-Start (systemd Service)

Create the service file:

```
sudo nano /etc/systemd/system/dashboard.service
```

Paste this:

```
[Unit]
Description=Dashboard App
After=network.target postgresql.service

[Service]
Type=simple
User=pi
WorkingDirectory=/opt/dashboard
EnvironmentFile=/opt/dashboard/.env
ExecStart=/usr/bin/node dist/index.js
Restart=always
RestartSec=10
StandardOutput=append:/var/log/dashboard.log
```

```
StandardError=append:/var/log/dashboard-error.log
```

```
[Install]
WantedBy=multi-user.target
```

Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable dashboard
sudo systemctl start dashboard
sudo systemctl status dashboard
```

The app will now start automatically on every boot and restart if it crashes.

Step 12: Set Up Log Rotation

```
sudo nano /etc/logrotate.d/dashboard
```

Paste:

```
/var/log/dashboard.log /var/log/dashboard-error.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    create 0644 pi pi
}
```

Step 13: Update Home Assistant Webhooks

In your HA configuration.yaml, update all rest_command URLs from:

```
https://home-view--bkh416.replit.app/api/webhook/...
```

To:

```
http://dashboard-server.local:5000/api/webhook/...
```

Or use the Pi's static IP address instead of the hostname for reliability.

See the Home Assistant YAML Reference section below for every rest_command and automation you need.

Step 14: Update Google Apps Script

If you have a Google Apps Script pushing emails to the app, update the webhook URLs there too to point to the Pi's address.

Important: Since the Pi is on your local network, the Google Apps Script (which runs in the cloud) won't be able to reach dashboard-server.local. You'll need either:

- A static public IP with port forwarding on your router (port 5000)
- A reverse tunnel service like **Cloudflare Tunnel** (free) to expose the Pi to the internet securely
- Or keep using Replit just for the Gmail webhook endpoint

Step 15: Touchscreen Setup

On your 1920x720 touchscreen, open the browser and navigate to:

```
http://dashboard-server.local:5000
```

Bookmark it or set it as the homepage.

Ongoing Maintenance

Viewing Logs

```
# Live logs
sudo journalctl -u dashboard -f

# Or read the log files
tail -100 /var/log/dashboard.log
tail -100 /var/log/dashboard-error.log
```

Restarting the App

```
sudo systemctl restart dashboard
```

Updating the Code

```
cd /opt/dashboard
# Transfer new code (scp or git pull)
npm install
npm run build
sudo systemctl restart dashboard
```

Backing Up the Database

```
pg_dump -U dashboard dashboard_db > /opt/dashboard/backup_$(date +%Y%m%d).sql
```

Restoring a Backup

```
psql -U dashboard dashboard_db < /opt/dashboard/backup_YYYYMMDD.sql
```

Key Differences from Replit

Feature	Replit	Self-Hosted Pi
OAuth Integrations	Managed automatically by Replit connectors	You manage tokens, refresh, and re-auth manually
HTTPS	Automatic	Not included – use Cloudflare Tunnel if needed for external access
Uptime	Replit manages hosting	You manage power, updates, SD card health
External Webhooks	Public URL available immediately	Need port forwarding or tunnel for cloud services to reach the Pi
Database Backups	Replit handles it	You set up cron jobs for backups
Speed	Cloud latency to HA	Local network – faster for all HA communication

Estimated Migration Effort

Task	Time
Hardware setup + OS flash	30 minutes
System dependencies	20 minutes
Code transfer + database init	15 minutes
Google OAuth setup	1-2 hours
Spotify OAuth setup	30 minutes
Microsoft OAuth setup	1-2 hours
HA webhook URL updates	30 minutes
Testing everything	1-2 hours
Total	**4-7 hours**

The OAuth setup is the bulk of the work. Everything else is straightforward.

Home Assistant YAML Reference

All YAML below goes into your Home Assistant configuration.yaml (or split into automations.yaml / rest_commands.yaml using !include if you prefer). Replace YOUR_APP_URL with either https://home-view--bkh416.replit.app (Replit) or http://dashboard-server.local:5000 (self-hosted Pi).

REST Commands (configuration.yaml)

These define the HTTP calls HA can make to your app. Every webhook endpoint needs one.

```
rest_command:
  # Cat Washroom Lights – triggers study reading prompt or stop
  cat_lights_webhook:
    url: "YOUR_APP_URL/api/webhook/cat-lights"
    method: POST
    headers:
      Content-Type: "application/json"
    payload: '{"state": "{{ states(''light.cat_lights'') }}"}'

  # Cat Washroom Lights – confirmation ("yes, play the reading")
  cat_lights_confirm_webhook:
    url: "YOUR_APP_URL/api/webhook/cat-lights-confirm"
    method: POST
    headers:
      Content-Type: "application/json"

  # Cat Washroom Shower Button – direct play (skips the prompt)
  cat_shower_button_webhook:
    url: "YOUR_APP_URL/api/webhook/cat-shower-button"
    method: POST
    headers:
      Content-Type: "application/json"

  # Cat Washroom Stop – stops playback and saves progress
  cat_wash_stop_webhook:
    url: "YOUR_APP_URL/api/webhook/cat-wash-stop"
    method: POST
    headers:
      Content-Type: "application/json"
    payload: '{"trigger": "{{ trigger }}"}'

  # Cat Washroom Volume Knob – adjusts active speaker volume
  cat_volume_webhook:
    url: "YOUR_APP_URL/api/webhook/cat-volume"
    method: POST
    headers:
      Content-Type: "application/json"
    payload: '{"direction": "{{ direction }}", "speed": "{{ speed }}"}'

  # Cat Washroom Knob Press – master STOP button
  cat_knob_press_webhook:
    url: "YOUR_APP_URL/api/webhook/cat-knob-press"
    method: POST
    headers:
      Content-Type: "application/json"

  # Voice Commands – pause, resume, stop, skip, restart, reset
  voice_command_webhook:
    url: "YOUR_APP_URL/api/webhook/voice-command"
    method: POST
    headers:
      Content-Type: "application/json"
    payload: '{"command": "{{ command }}"}'

  # Kitchen Volume Knob – adjusts Kitchen Echo Studio volume
  kitchen_volume_webhook:
    url: "YOUR_APP_URL/api/webhook/kitchen-volume"
    method: POST
    headers:
      Content-Type: "application/json"
    payload: '{"direction": "{{ direction }}", "speed": "{{ speed }}"}'

  # Play Urgent PDF – on-demand play most important unlistened reading
  play_urgent_pdf_webhook:
    url: "YOUR_APP_URL/api/webhook/play-urgent-pdf"
    method: POST
    headers:
      Content-Type: "application/json"
      x-webhook-secret: "YOUR_SITE_PASSWORD"
    payload: '{"entity_id": "media_player.bathroom_speaker"}
```

HA Automations (automations.yaml)

Automation 1: Cat Washroom Lights Changed

Fires whenever the cat washroom lights turn on or off. The app checks the actual state and either starts the study prompt flow (on) or stops everything (off).

```

automation:
- alias: "Cat Washroom Lights Changed"
  trigger:
    - platform: state
      entity_id: light.cat_lights
  action:
    - service: rest_command.cat_lights_webhook

```

Automation 2: Cat Washroom Lights Confirmation

Fires when you confirm (button press, voice, or HA toggle) that you want to hear the reading. This resolves the 23-second wait from Automation 1.

```

automation:
- alias: "Cat Washroom Reading Confirmed"
  trigger:
    - platform: state
      entity_id: input_boolean.module_reading_confirmed
      to: "on"
  action:
    - service: rest_command.cat_lights_confirm_webhook

```

Automation 3: Cat Washroom Shower Button

Fires when the shower button entity changes. Skips the prompt and goes straight into playback.

```

automation:
- alias: "Cat Washroom Shower Button"
  trigger:
    - platform: state
      entity_id: switch.cat_shower_button # or whatever your shower button entity is
  action:
    - service: rest_command.cat_shower_button_webhook

```

Automation 4: Cat Washroom Volume Knob (Rotate)

Fires when the rotary encoder sends a rotation event. Your ESPHome or Zigbee config should set the direction variable to "up" or "down" and speed to "normal" or "fast".

```

automation:
- alias: "Cat Washroom Volume Up"
  trigger:
    - platform: state
      entity_id: sensor.cat_volume_knob_rotation # your encoder entity
  action:
    - service: rest_command.cat_volume_webhook
      data:
        direction: "{{ 'up' if trigger.to_state.state | float > trigger.from_state.state | float else 'down' }}"
        speed: "normal"

- alias: "Cat Washroom Volume Fast"
  trigger:
    - platform: state
      entity_id: sensor.cat_volume_knob_fast_rotation # fast rotation entity
  action:
    - service: rest_command.cat_volume_webhook
      data:
        direction: "{{ 'up' if trigger.to_state.state | float > trigger.from_state.state | float else 'down' }}"
        speed: "fast"

```

Automation 5: Cat Washroom Knob Press (STOP)

Fires when you press the physical knob button. Stops everything.

```

automation:
- alias: "Cat Washroom Knob Press STOP"
  trigger:
    - platform: state
      entity_id: binary_sensor.cat_volume_knob_press # your knob press entity
      to: "on"
  action:
    - service: rest_command.cat_knob_press_webhook

```

Automation 6: Kitchen Volume Knob

Same pattern as the cat washroom volume knob, but for the kitchen.

```

automation:
- alias: "Kitchen Volume Up"
  trigger:
    - platform: state
      entity_id: sensor.kitchen_volume_knob_rotation
  action:
    - service: rest_command.kitchen_volume_webhook
      data:
        direction: "{{ 'up' if trigger.to_state.state | float > trigger.from_state.state | float else 'down' }}"
        speed: "normal"

```

HA Input Booleans (configuration.yaml)

These are required for the confirmation flow. The app sets them during the lights prompt and polls them as a backup confirmation method.

Cat Washroom Study Reading System — Complete Flow

Overview

The cat washroom has a study reading system that plays your school readings aloud on the Nest speaker while displaying the text on a Fire tablet and Samsung TV. It is triggered by turning on the cat washroom lights or pressing the shower button, and can be stopped in multiple ways.

Devices Involved

Device	HA Entity	Role
Cat Washroom Lights	`light.cat_lights`	Trigger (on/off)
Google Nest Speaker	`media_player.bathroom_speaker`	Main audio playback (TTS reading)
HA Voice ESPHome Device	`media_player.home_assistant_voice_097c38_media_player`	Voice prompts ("Would you like to play.....")
Fire Tablet (Cat WR)	`media_player.tablet_cat`	Displays PDF reader page with synced text
Samsung TV (Cat WR)	`media_player.tv_cat_wr`	Displays PDF reader follow-along page
Fire Stick (Cat WR TV)	`media_player.fire_tv_172_24_0_88`	Drives Samsung TV display via HDMI
Echo Cat Left	`media_player.echo_cat_left_am`	CHUM FM playback (Echo speaker group)
Echo Cat Right	`media_player.echo_cat_right_am`	CHUM FM playback (Echo speaker group)
Echo Cat Middle	`media_player.echo_cat_washroom_middle`	CHUM FM playback (Echo speaker group)
Cat WR Media Group	`media_player.cat_washroom_media_group`	Alexa multi-room group for CHUM FM
HA Cloud TTS	`tts.home_assistant_cloud`	Nabu Casa cloud TTS (last-resort prompt voice)
Oral-B Toothbrush	`sensor.toothbrush_bryn_toothbrush_state`	Auto-stop trigger (polled, not webhook)

FLOW A: Lights Turn ON — Full Prompt Flow

Trigger: light.cat_lights state changes (HA automation calls rest_command.cat_lights_webhook)
Endpoint: POST /api/webhook/cat-lights

Step-by-step:

1. Guards (any of these abort the flow):

- Server startup cooldown (skips if server just restarted within cooldown window)
- Already playing a reading (skips if `catWashPlaybackActive` is true)
- Another prompt is already pending (skips if `catLightsPromptPending` is true)

2. Query the actual light state from HA:

- `GET /api/states/light.cat\_lights`
- If state is "off" — go to **FLOW B** (Lights Turn OFF) instead
- If state is not "on" — ignore the webhook

3. Immediate acknowledgment (plays right away while the app looks up files):

- Set volume on HA Voice device to 0.64
- Set `input\_boolean.module\_reading\_confirmed` to OFF
- Set `input\_boolean.module\_reading\_pending` to ON
- Play via HA Cloud TTS on the HA Voice device:

****"One moment, checking your readings."****

4. Look up the next unlistened file:

- Get active semester settings from the database
- Calculate the current week number (accounting for reading week)
- Search database for the next unlistened file for this week, prioritized:

1. Module files first (sorted by course priority)
2. Reading files second (sorted by course priority)

- If no cached files found, sync from OneDrive first, then search again

5. Light re-check:

- Query `light.cat\_lights` state again
- If the light turned OFF while the app was looking up files — abort silently

6. If NO unlistened files found:

- Play CHUM FM 104.5 on the Echo speaker group:
- media_player/play_media on media_player.cat_washroom_media_group
- media_content_type: "custom", media_content_id: "play 104.5 chum fm"
- Done. No prompt.

7. If an unlistened file IS found — play the voice prompt:

- Stop any leftover media on all cat washroom speakers first
- Set Nest speaker volume to 0.64
- Generate the prompt message. Example:
****"Would you like to play week 8, C.P.P.A. 1 22 module?"****
- Try to play via HA Cloud TTS on the HA Voice device
- If that fails, fall back to Edge TTS audio file played on the HA Voice device
- If ALL TTS methods fail, give up and play CHUM FM instead

8. Wait 2 seconds for the prompt to finish speaking

9. Light re-check #2:

- Query `light.cat\_lights` again
- If OFF — abort, reset booleans, done

10. Wait for confirmation (up to 23 seconds):

- Primary: wait for `POST /api/webhook/cat-lights-confirm` (resolves the promise immediately)
- Backup: poll `input\_boolean.module\_reading\_confirmed` every 10 seconds
- Timeout after 23 seconds

11. Reset HA booleans regardless of outcome:

- `input\_boolean.module\_reading\_pending` OFF
- `input\_boolean.module\_reading\_confirmed` OFF

12a. If NOT confirmed (23-second timeout):

- Play CHUM FM 104.5 on Echo speaker group
- Done.

12b. If CONFIRMED:

- Light re-check #3 (abort if OFF)
- Play confirmation voice:
****"Okay, I will now play week 8, C.P.P.A. 1 22 module."****
- Start the full playback flow (**FLOW C** below)

FLOW B: Lights Turn OFF — Stop Everything

Trigger: light.cat_lights state changes to OFF

Endpoint: POST /api/webhook/cat-lights (same endpoint, app checks state)

Step-by-step:

1. If a reading is actively playing:

- Save current chunk progress to database
- Stop Nest speaker: `media\_player/media\_stop` on `media\_player.bathroom\_speaker`
- Turn off Fire Stick: `media\_player/turn\_off` on `media\_player.fire\_tv\_172\_24\_0\_88`
- Turn off Samsung TV: `media\_player/turn\_off` on `media\_player.tv\_cat\_wr`
- Play goodbye TTS on Nest speaker:

****"Stopping. [File Name]. The file position has been saved. See you next time Bryn."****

- Send `stop\_playback` command to tablet and TV displays
- Stop toothbrush polling
- Clear playback session from database

2. If a TTS session is active:

- Stop the TTS session

3. Stop all cat washroom speakers (always, even if nothing was playing):

- `media\_player/media\_stop` on `media\_player.bathroom\_speaker`
- `media\_player/media\_stop` on all 3 Echo speakers (`echo\_cat\_left\_am`, `echo\_cat\_right\_am`, `echo\_cat\_washroom\_middle`)
- `media\_player/media\_stop` on `media\_player.cat\_washroom\_media\_group`

4. Reset state flags:

- `catLightsPromptPending = false`
- `catWashPlaybackTrigger = null`

FLOW C: Confirmed Playback Flow (Reading the PDF)

Triggered by: Flow A confirmation, shower button (Flow D), voice commands (resume/restart/reset/skip), or play-urgent-pdf webhook.

Step-by-step:

1. Calculate resume point:

- Look up `lastChunkIndex` from the database for this file
- Resume from `max(0, savedChunk - 1)` — goes back 1 chunk for context overlap

2. Build URLs:

- Tablet URL: `pdf-reader/{fileId}?catWashFollow=true&resumeChunk={N}&voice=echo&fullscreen=true`
- TV follow URL: `pdf-reader/{fileId}?catWashFollow=true&followOnly=true&...`

3. PARALLEL SETUP (all happen simultaneously):

Tablet Setup (media_player.tablet_cat):

- ADB: `input keyevent KEYCODE\_WAKEUP` — wake the tablet
- ADB: `settings put system screen\_brightness 255` — max brightness
- Wait 1.5 seconds
- ADB: `am start --activity-clear-task -a android.intent.action.VIEW -d "{readerUrl}" com.amazon.cloud9` — open PDF

reader in Silk browser

- Wait 1 second
- ADB: `settings put global policy\_control immersive.full=com.amazon.cloud9` — immersive mode (hide status/nav bars)
- ADB: `input keyevent KEYCODE\_F11` — fullscreen

TV Setup:

- `media\_player/turn\_on` on `media\_player.fire\_tv\_172\_24\_0\_88` — wake Fire Stick
- `media\_player/turn\_on` on `media\_player.tv\_cat\_wr` — turn on Samsung TV
- Wait 3 seconds (let TV boot)
- `media\_player/select\_source` on `media\_player.tv\_cat\_wr`, source: `HDMI1` — switch to Fire Stick input
- ADB on Fire Stick: wake, kill existing Silk browser, open redirect URL, set immersive mode, press center button to

dismiss dialogs

Pre-generate first audio chunk:

- Check if pre-prepared audio exists (from background 30-min prep job)
- If yes, use the cached .mp3 file
- If no, generate via Edge TTS (voice: `en-US-AndrewMultilingualNeural`)

Confirmation TTS (plays on Nest speaker):

****"Okay, I will now play [file description]."*

- Wait for estimated duration (word count / 140 wpm)

4. Set playback volume:

- `media\_player/volume\_set` on HA Voice device @ 0.75
- `media\_player/volume\_set` on Nest speaker @ 0.75

5. Start chunk-by-chunk reading loop:

For each chunk (starting from resume point):

- Check session is still valid (not cancelled, not stale)
 - Generate or use pre-prepared TTS audio for this chunk
 - Play on Nest speaker:
- media_player/media_stop on media_player.bathroom_speaker
 - media_player/volume_set @ 0.75
 - media_player/play_media with the audio URL
 - Wait for estimated chunk duration (word count / 175 wpm + 1 second buffer)
 - Poll Nest speaker state to confirm playback finished
 - Update database: `lastChunkIndex`, `checkedChunks`
 - Send progress to tablet and TV via polling endpoint (they update the highlighted text)
 - Pre-generate NEXT chunk audio in background (pipeline)

6. When ALL chunks are complete:

- Mark file as "listened" in database
 - Play goodbye TTS:
- ***"All done with [file name]. Great work Bryn!"****
- Look for the next unlistened file
- If another file exists: start playing it automatically (back to step 1)
 - If no more files: stop playback, turn off Fire Stick and Samsung TV

7. Start toothbrush polling (runs alongside playback):

- Poll `sensor.toothbrush\_bryn\_toothbrush\_state` every 3 seconds
- If state changes to "running" or "brushing", auto-stop playback (see Flow F)

FLOW D: Shower Button — Direct Play (Skip Prompt)

Trigger: Shower button entity changes in HA

Endpoint: POST /api/webhook/cat-shower-button

Step-by-step:

1. Guards:

- Server startup cooldown
- Skip if playback already active (assumes you're toggling the shower/fan)
- Skip if a lights prompt is already pending

2. Look up semester and find next file (same as Flow A, steps 4-6)

3. If no unlistened files:

- Play CHUM FM 104.5 on Echo speaker group
- Done.

4. If file found — go straight to playback (no prompt, no 23-second wait):

- Play confirmation:
- ***"Okay, I will now play [file description].****
- Start **Flow C** (confirmed playback) immediately

FLOW E: Volume Knob — Adjust Active Speaker Volume

Trigger: Rotary encoder rotation in HA

Endpoint: POST /api/webhook/cat-volume

Body: { "direction": "up" | "down", "speed": "normal" | "fast" }

Step-by-step:

1. Query which speakers are currently active:

- Check state of `media\_player.bathroom\_speaker` (Nest)
- Check state of `media\_player.cat\_washroom\_media\_group` (Echo group)
- Active = state is `"playing"`, `"paused"`, or `"buffering"`
- If none active, default to Nest speaker

2. Calculate new volume:

- Normal speed: current volume +/- 0.05 (5% per click)
- Fast speed: current volume +/- 0.15 (15% per click)
- Clamped between 0.0 and 1.0

3. Set volume on each active speaker:

- `media\_player/volume\_set` on each active entity

FLOW F: Knob Press — Master STOP

Trigger: Physical knob button press in HA

Endpoint: POST /api/webhook/cat-knob-press

Step-by-step:

1. If a reading is actively playing:

- Internally calls `POST /api/webhook/cat-wash-stop` with `{ "trigger": "knob\_press", "keepOpen": true }`
- This triggers the full stop flow: save progress, stop Nest, turn off TV/Fire Stick, play goodbye
- Goodbye TTS:

"Stopping. [File Name]. The file position has been saved. See you next time Bryn."

2. If nothing is playing:

- Stop all cat washroom speakers (kills CHUM FM, any leftover media):
- media_player/media_stop on Nest, all 3 Echos, and the media group

FLOW G: Toothbrush Auto-Stop

Trigger: App polls sensor.toothbrush_bryn_toothbrush_state every 3 seconds (NOT a webhook)

No HA automation needed — the app does this internally whenever playback starts.

Step-by-step:

1. Polling starts when any playback begins (Flow C)

2. Each poll:

- `GET /api/states/sensor.toothbrush\_bryn\_toothbrush\_state` from HA
- If state is `"running"` or `"brushing"`:
- Stop playback via internal cat-wash-stop logic
- Save progress to database
- Stop Nest speaker
- Turn off Fire Stick and Samsung TV
- Play goodbye:

"Stopping. Your reading position has been saved. See you next time Bryn."

3. Polling stops when playback ends (any stop mechanism)

FLOW H: Voice Commands

Trigger: HA voice assistant or custom intent
Endpoint: POST /api/webhook/voice-command
Body: { "command": "<command>" }

PAUSE

```
{ "command": "pause" }
```

- If nothing playing: **Nothing is playing right now.**
- Save chunk progress, stop Nest speaker, stop word advancement on tablet
- Start 10-minute auto-stop timer (if no resume within 10 min, everything shuts off):

- Auto-stop TTS: "Pause timed out. Playback has been stopped. Your progress has been saved."

- TTS response:

Paused. Say resume to continue, or I'll stop in 10 minutes.

RESUME

```
{ "command": "resume" }
```

- If nothing paused: **Nothing is paused right now.**
- Clear the 10-minute auto-stop timer
- Stop Echo speakers (clear any CHUM FM that might have started)
- TTS response:

Resuming [file description].

- Restart playback from saved chunk position (**Flow C**)

STOP

```
{ "command": "stop" }
```

- If nothing playing or paused: **Nothing is playing.**
- If actively playing: save progress, stop Nest, turn off TV/Fire Stick
- TTS response:

Stopped. Your progress has been saved. See you next time Bryn.

SKIP (next file)

```
{ "command": "skip" }
```

- If nothing playing: **Nothing is playing to skip.**
- Mark current file as "listened" (complete) in database
- Stop current playback
- Find next unlistened file

- If no more files: "Skipped. No more readings for this week. Great work Bryn!" (turn off TV/Fire Stick)

- If next file found: "Skipped. Now playing [file description]." (start Flow C with new file)

RESTART (go back one chunk)

```
{ "command": "restart" }
```

- If nothing playing: **Nothing is playing to restart.**
- Go back 1 chunk from current position (minimum chunk 0)
- Stop current playback, save new position
- TTS response:

Going back. Restarting from an earlier section.

- Restart playback from earlier chunk (**Flow C**)

RESET (start from beginning)

```
{ "command": "reset" }
```

- If nothing playing: **Nothing is playing to reset.**
- Reset file progress in database: ``lastChunkIndex = 0`, `checkedChunks = []``
- Stop current playback
- TTS response:

Resetting [file description]. Starting from the beginning.

- Start playback from chunk 0 (**Flow C**)

FLOW I: Play Urgent PDF (On-Demand)

Trigger: HA script, button, or automation

Endpoint: POST /api/webhook/play-urgent-pdf

Auth: Requires x-webhook-secret header matching your site password

Step-by-step:

1. Authenticate via header

2. Find all unlistened files, ordered by priority:

1. Module files first (sorted by course)
2. Reading files second (sorted by course)

3. If no files:

- TTS on target speaker:

"All week [N] readings are complete. Great job!"

4. If file found:

- Start **Flow C** (confirmed playback) immediately

TTS Fallback Chain

The app uses a 3-level fallback chain for generating speech audio:

Priority	Engine	Voice	When It's Used
1	OpenAI TTS	alloy	Primary (rate-limited)
2	Edge TTS (Microsoft)	en-US-AndrewMultilingualNeural	When OpenAI is rate-limited or fails
3	espeak-ng (local)	Default English	When Edge TTS has 5+ consecutive failures

For voice prompts (the "Would you like to play?" question), the app uses HA Cloud TTS (Nabu Casa) as the primary method since it's the fastest (no file generation needed), with Edge TTS file playback as the fallback.

Known Issues & Learnings (Nest Speaker)

These are hard-won lessons from debugging the cat washroom reading system. Keep them in mind when modifying the speaker playback code.

1. Nest Speaker State Is Unreliable — Always Trust the Service Call

The Google Nest speaker frequently reports its state as "unknown" even when it IS actively playing audio. The media_player entity in HA does not reliably transition to "playing" or "buffering" for Cast-based playback.

Rule: If the media_player/play_media service call returns 200 OK from Home Assistant, treat it as SUCCESS. Do NOT use the speaker's reported state (idle, unknown, unavailable) to determine whether playback actually started. The only failure case is when the HA service call itself throws an error (network failure, HA offline, etc.).

If you add a state check after play_media, treat "unknown" the same as "playing" — return success. Only retry on "idle" or "off", and even then, after max retries, still return success since the command was accepted by HA.

2. The Nest CAN Play Audio from the Deployed App URL

The Nest speaker successfully plays MP3 files served from <https://home-view--bkh416.replit.app/api/tts-audio/....> The /api/tts-audio/ endpoint is explicitly excluded from authentication in server/index.ts (line ~190), so no cookies or tokens are needed.

Do NOT add an HA local media upload step — it adds complexity without benefit. The direct URL approach works. If playback appears broken, the issue is almost certainly the state check logic (see point 1), not the URL accessibility.

3. Circuit Breaker Must Not Fire on State-Check Failures

The chunk playback loop has a circuit breaker that triggers after 3 consecutive "failures." If playOnNestSpeaker incorrectly returns {success: false} due to an "unknown" state, the circuit breaker fires and prompts the user to switch to Echo speakers — even though the Nest was playing fine.

Rule: playOnNestSpeaker should only return {success: false} when the HA service call itself fails (HTTP error, timeout, HA offline). State check results should never cause a {success: false} return.

4. Volume Levels

Context	Volume
Voice prompts ("Would you like to play?")	0.35
HA Voice confirmation prompt	0.45
Nest pre-confirm / playback	0.75
Fallback / cat-lights volume	0.35

5. Confirm TTS Plays on the Nest, Not HA Voice

The confirmation message ("Okay, I will now play your module...") should play on the Nest speaker using a generated OpenAI/Edge TTS audio file, not via HA Cloud TTS on the HA Voice speaker. The fallback to HA Voice should only happen if the Nest play_media service call actually fails (HTTP error).

6. Stale Playback Sessions

- The webhook handler checks for stale playback at the start. A session is considered stale if:
- It has been running for 3+ minutes AND is still at chunk 0 (stuck at start)
 - It has been running for 10+ minutes regardless of chunk position

Stale sessions are cleared so the next light trigger can start fresh.

Background Processes (No Webhook Needed)

These run automatically inside the app — no HA automation required:

Process	Interval	What It Does
Audio Preparation	Every 30 minutes	Pre-generates TTS audio for upcoming files. Also retries files with failed chunks.
Semester Auto-Activation	Every 6 hours + startup	Checks all semesters and activates the one whose date range includes tod...
Ticker Push to HA	Every 5 minutes	Fetches weather, news, pollen, course announcements and pushes to `sensor.dashboard_tic...
Toothbrush Polling	Every 3 seconds (only during playback)	Polls toothbrush sensor state. Auto-stops reading if brushing d...
Alexa Reminder Scheduler	Every 60 seconds	Checks for due task reminders and sends Echo voice announcements + iPhone push ...
Playback Session Persistence	On every chunk change	Saves current playback state to `app_state` table so playback can resu...

PART 2: COMPLETE SOURCE CODE

***Generated:** March 28, 2026*

***Total files:** 132*

File Index

- `server/db.ts` (395 B)
- `server/email.ts` (15.1 KB)
- `server/gmail.ts` (8.5 KB)
- `server/gmailTicker.ts` (1.5 KB)
- `server/googleCalendar.ts` (22.2 KB)
- `server/index.ts` (11.5 KB)
- `server/onedrive.ts` (22.5 KB)
- `server/outlookCalendar.ts` (11.7 KB)
- `server/reminderScheduler.ts` (14.7 KB)
- `server/replit\_integrations/audio/client.ts` (15.0 KB)
- `server/replit\_integrations/audio/index.ts` (277 B)
- `server/replit\_integrations/audio/routes.ts` (5.1 KB)
- `server/replit\_integrations/batch/index.ts` (108 B)
- `server/replit\_integrations/batch/utils.ts` (5.5 KB)
- `server/replit\_integrations/chat/index.ts` (108 B)
- `server/replit\_integrations/chat/routes.ts` (4.0 KB)
- `server/replit\_integrations/chat/storage.ts` (1.7 KB)
- `server/replit\_integrations/image/client.ts` (1.4 KB)
- `server/replit\_integrations/image/index.ts` (117 B)
- `server/replit\_integrations/image/routes.ts` (874 B)
- `server/replit\_integrations/object\_storage/index.ts` (377 B)
- `server/replit\_integrations/object\_storage/objectAcl.ts` (5.4 KB)
- `server/replit\_integrations/object\_storage/objectStorage.ts` (8.2 KB)
- `server/replit\_integrations/object\_storage/routes.ts` (3.2 KB)
- `server/routes.ts` (739.9 KB)
- `server/secondGoogleAccount.ts` (10.0 KB)
- `server/spotify.ts` (8.0 KB)
- `server/static.ts` (1.2 KB)
- `server/storage.ts` (33.1 KB)
- `server/thirdGoogleAccount.ts` (5.0 KB)
- `server/timezone.ts` (2.6 KB)
- `server/tmuCalendar.ts` (7.6 KB)
- `server/vite.ts` (1.5 KB)
- `client/index.html` (3.8 KB)
- `client/public/silk-test.html` (568 B)
- `client/replit\_integrations/audio/audio-playback-worklet.js` (3.0 KB)
- `client/replit\_integrations/audio/audio-utils.ts` (1.1 KB)
- `client/replit\_integrations/audio/index.ts` (1.3 KB)
- `client/replit\_integrations/audio/useAudioPlayback.ts` (3.4 KB)
- `client/replit\_integrations/audio/useVoiceRecorder.ts` (1.5 KB)
- `client/replit\_integrations/audio/useVoiceStream.ts` (2.8 KB)
- `client/src/App.tsx` (6.8 KB)
- `client/src/components/AddDeviceDialog.tsx` (7.4 KB)
- `client/src/components/BookAnimation.tsx` (9.4 KB)
- `client/src/components/CourseDetailDialog.tsx` (207.9 KB)
- `client/src/components/DeviceCard.tsx` (5.3 KB)

- `client/src/components/FastInput.tsx` (2.1 KB)
- `client/src/components/NewCourseWizard.tsx` (53.4 KB)
- `client/src/components/ObjectUploader.tsx` (3.2 KB)
- `client/src/components/OtherRowEditDialog.tsx` (30.9 KB)
- `client/src/components/SemesterChecklistDialog.tsx` (8.3 KB)
- `client/src/components/Sidebar.tsx` (2.9 KB)
- `client/src/components/StickyNotelItem.tsx` (19.1 KB)
- `client/src/components/access-gate.tsx` (4.8 KB)
- `client/src/components/ui/accordion.tsx` (1.9 KB)
- `client/src/components/ui/alert-dialog.tsx` (4.3 KB)
- `client/src/components/ui/alert.tsx` (1.5 KB)
- `client/src/components/ui/aspect-ratio.tsx` (140 B)
- `client/src/components/ui/avatar.tsx` (1.6 KB)
- `client/src/components/ui/badge.tsx` (1.2 KB)
- `client/src/components/ui/breadcrumb.tsx` (2.6 KB)
- `client/src/components/ui/button.tsx` (2.3 KB)
- `client/src/components/ui/calendar.tsx` (2.6 KB)
- `client/src/components/ui/card.tsx` (1.9 KB)
- `client/src/components/ui/carousel.tsx` (6.1 KB)
- `client/src/components/ui/chart.tsx` (10.2 KB)
- `client/src/components/ui/checkbox.tsx` (1.0 KB)
- `client/src/components/ui/collapsible.tsx` (329 B)
- `client/src/components/ui/command.tsx` (4.8 KB)
- `client/src/components/ui/context-menu.tsx` (7.3 KB)
- `client/src/components/ui/dialog.tsx` (3.9 KB)
- `client/src/components/ui/drawer.tsx` (3.0 KB)
- `client/src/components/ui/dropdown-menu.tsx` (8.5 KB)
- `client/src/components/ui/form.tsx` (4.0 KB)
- `client/src/components/ui/hover-card.tsx` (1.2 KB)
- `client/src/components/ui/input-otp.tsx` (2.1 KB)
- `client/src/components/ui/input.tsx` (857 B)
- `client/src/components/ui/label.tsx` (710 B)
- `client/src/components/ui/menubar.tsx` (8.4 KB)
- `client/src/components/ui/navigation-menu.tsx` (5.0 KB)
- `client/src/components/ui/pagination.tsx` (2.7 KB)
- `client/src/components/ui/popover.tsx` (1.3 KB)
- `client/src/components/ui/progress.tsx` (791 B)
- `client/src/components/ui/radio-group.tsx` (1.4 KB)
- `client/src/components/ui/resizable.tsx` (1.7 KB)
- `client/src/components/ui/scroll-area.tsx` (1.6 KB)
- `client/src/components/ui/select.tsx` (5.8 KB)
- `client/src/components/ui/separator.tsx` (756 B)
- `client/src/components/ui/sheet.tsx` (4.2 KB)
- `client/src/components/ui/sidebar.tsx` (21.3 KB)
- `client/src/components/ui/skeleton.tsx` (261 B)
- `client/src/components/ui/slider.tsx` (1.1 KB)
- `client/src/components/ui/switch.tsx` (1.1 KB)
- `client/src/components/ui/table.tsx` (2.7 KB)
- `client/src/components/ui/tabs.tsx` (1.8 KB)
- `client/src/components/ui/textarea.tsx` (689 B)
- `client/src/components/ui/toast.tsx` (4.7 KB)
- `client/src/components/ui/toaster.tsx` (788 B)
- `client/src/components/ui/toggle-group.tsx` (1.7 KB)
- `client/src/components/ui/toggle.tsx` (1.5 KB)
- `client/src/components/ui/tooltip.tsx` (1.2 KB)

- `client/src/hooks/use-devices.ts` (4.6 KB)
- `client/src/hooks/use-mobile.tsx` (565 B)
- `client/src/hooks/use-toast.ts` (3.8 KB)
- `client/src/hooks/use-upload.ts` (5.2 KB)
- `client/src/index.css` (54.7 KB)
- `client/src/lib/queryClient.ts` (1.4 KB)
- `client/src/lib/utils.ts` (166 B)
- `client/src/main.tsx` (2.6 KB)
- `client/src/pages/code-checker.tsx` (6.4 KB)
- `client/src/pages/dashboard.tsx` (1.7 MB)
- `client/src/pages/files.tsx` (103.4 KB)
- `client/src/pages/mobile-notes.tsx` (24.8 KB)
- `client/src/pages/not-found.tsx` (711 B)
- `client/src/pages/onedrive.tsx` (10.8 KB)
- `client/src/pages/onenote.tsx` (56.4 KB)
- `client/src/pages/pdf-reader.tsx` (160.1 KB)
- `client/src/pages/pdf-viewer.tsx` (13.9 KB)
- `client/src/pages/projects.tsx` (46.6 KB)
- `client/src/pages/spotify-player.tsx` (132.4 KB)
- `client/src/pages/ticker.tsx` (13.8 KB)
- `shared/electiveCourses.ts` (49.1 KB)
- `shared/models/chat.ts` (1.2 KB)
- `shared/routes.ts` (3.6 KB)
- `shared/schema.ts` (29.8 KB)
- `shared/semesterUtils.ts` (6.1 KB)
- `package.json` (4.4 KB)
- `tsconfig.json` (657 B)
- `vite.config.ts` (2.0 KB)
- `drizzle.config.ts` (325 B)
- `tailwind.config.ts` (4.0 KB)
- `postcss.config.js` (80 B)

`server/db.ts`

```
import { drizzle } from "drizzle-orm/node-postgres";
import pg from "pg";
import * as schema from "@shared/schema";

const { Pool } = pg;

if (!process.env.DATABASE_URL) {
  throw new Error(
    "DATABASE_URL must be set. Did you forget to provision a database?",
  );
}

export const pool = new Pool({ connectionString: process.env.DATABASE_URL });
export const db = drizzle(pool, { schema });
```

`server/email.ts`

```
import { Resend } from 'resend';

const resend = new Resend(process.env.RESEND_API_KEY);

const FROM_EMAIL = 'reminders@uni-cal.app';
const TO_EMAIL = 'bryn.kai-hendricks@outlook.com';
const SMS_EMAIL = '4168275455@sms.rogers.com';

export interface TaskReminder {
  id: number;
  title: string;
  dueDate: string;
  courseName?: string | null;
  type?: string | null;
}

export async function sendTaskReminder(task: TaskReminder): Promise<{ success: boolean; error?: string }> {
  try {
```

```

const dueDate = new Date(task.dueDate);
const formattedDate = dueDate.toLocaleDateString('en-GB', {
  weekday: 'long',
  day: 'numeric',
  month: 'long',
  year: 'numeric'
});

const { data, error } = await resend.emails.send({
  from: FROM_EMAIL,
  to: TO_EMAIL,
  subject: `Task Reminder: ${task.title}`,
  html: `
<div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px;">
  <h2 style="color: #333; border-bottom: 2px solid #4578B0; padding-bottom: 10px;">
    Task Reminder
  </h2>
  <div style="background: #f5f5f5; padding: 20px; border-radius: 8px; margin: 20px 0;">
    <h3 style="color: #042550; margin-top: 0;">${task.title}</h3>
    <p><strong>Due:</strong> ${formattedDate}</p>
    ${task.courseName ? `<p><strong>Course:</strong> ${task.courseName}</p>` : ''}
    ${task.type ? `<p><strong>Type:</strong> ${task.type}</p>` : ''}
  </div>
  <p style="color: #666; font-size: 14px;">
    This is an automated reminder from your Uni-Cal task manager.
  </p>
</div>
  `
});

if (error) {
  console.error('Resend error:', error);
  return { success: false, error: error.message };
}

console.log('Email sent successfully:', data);
return { success: true };
} catch (err) {
  console.error('Email send error:', err);
  return { success: false, error: err instanceof Error ? err.message : 'Unknown error' };
}
}

export async function sendDailyDigest(tasks: TaskReminder[]): Promise<{ success: boolean; error?: string }> {
  if (tasks.length === 0) {
    return { success: true };
  }

  try {
    const now = new Date();
    const taskListHtml = tasks.map(task => {
      const dueDate = new Date(task.dueDate);
      const formattedDate = dueDate.toLocaleDateString('en-GB', {
        weekday: 'short',
        day: 'numeric',
        month: 'short'
      });
    });
    const diffMs = dueDate.getTime() - now.getTime();
    const diffHours = Math.round(diffMs / (1000 * 60 * 60));
    const diffDays = Math.round(diffMs / (1000 * 60 * 60 * 24));
    let timeUntil = '';
    if (diffMs < 0) {
      const overdueDays = Math.abs(diffDays);
      timeUntil = overdueDays === 0 ? 'Overdue (today)' : `Overdue by ${overdueDays} days${overdueDays > 1 ? 's' : ''}...`;
    } else if (diffHours < 1) {
      timeUntil = 'Due in less than 1 hour';
    } else if (diffHours < 24) {
      timeUntil = `Due in ${diffHours} hour${diffHours > 1 ? 's' : ''}`;
    } else if (diffDays === 1) {
      timeUntil = 'Due tomorrow';
    } else {
      timeUntil = `Due in ${diffDays} day${diffDays > 1 ? 's' : ''}`;
    }
    return `
    <tr>
      <td style="padding: 10px; border-bottom: 1px solid #eee;">${task.title}</td>
      <td style="padding: 10px; border-bottom: 1px solid #eee;">${task.courseName || '-'}</td>
      <td style="padding: 10px; border-bottom: 1px solid #eee;">${formattedDate}<br/><span style="font-size: 12px...
    </tr>
  `;
  }).join('');

  const firstDue = new Date(tasks[0].dueDate);
  const firstDiffMs = firstDue.getTime() - now.getTime();
  const firstDiffHours = Math.round(firstDiffMs / (1000 * 60 * 60));
  const firstDiffDays = Math.round(firstDiffMs / (1000 * 60 * 60 * 24));
  let subjectTiming = '';
  if (tasks.length === 1) {
    if (firstDiffMs < 0) subjectTiming = ' (overdue)';
    else if (firstDiffHours < 24) subjectTiming = ` (in ${firstDiffHours} hour${firstDiffHours > 1 ? 's' : ''})`;
    else if (firstDiffDays === 1) subjectTiming = ' (tomorrow)';
    else subjectTiming = ` (in ${firstDiffDays} days)`;
  }

  const { data, error } = await resend.emails.send({
    from: FROM_EMAIL,
    to: TO_EMAIL,
    subject: `Daily Task Digest: ${tasks.length} task${tasks.length > 1 ? 's' : ''} due${subjectTiming}`,
    html: `
    <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px;">
      <h2 style="color: #333; border-bottom: 2px solid #4578B0; padding-bottom: 10px;">
        Daily Task Digest
      </h2>
      <p>You have <strong>${tasks.length} task${tasks.length > 1 ? 's' : ''}</strong> due:</p>
      <table style="width: 100%; border-collapse: collapse; margin: 20px 0;">
        <thead>
          <tr style="background: #042550; color: white;">
            <th style="padding: 10px; text-align: left;">Task</th>

```

```

        <th style="padding: 10px; text-align: left;">Course</th>
        <th style="padding: 10px; text-align: left;">Due</th>
    </tr>
</thead>
<tbody>
    ${taskListHtml}
</tbody>
</table>
<p style="color: #666; font-size: 14px;">
    This is an automated digest from your Uni-Cal task manager.
</p>
</div>
    ,
    '
});
if (error) {
    console.error('Resend error:', error);
    return { success: false, error: error.message };
}

console.log('Daily digest sent successfully:', data);
return { success: true };
} catch (err) {
    console.error('Email send error:', err);
    return { success: false, error: err instanceof Error ? err.message : 'Unknown error' };
}
}

export async function sendTestEmail(): Promise<{ success: boolean; error?: string }> {
    try {
        const { data, error } = await resend.emails.send({
            from: FROM_EMAIL,
            to: TO_EMAIL,
            subject: 'Uni-Cal Email Test',
            html: `
                <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px;">
                    <h2 style="color: #333; border-bottom: 2px solid #4578B0; padding-bottom: 10px;">
                        Email Test Successful!
                    </h2>
                    <p>Your Uni-Cal email reminders are now set up and working correctly.</p>
                    <p style="color: #666; font-size: 14px;">
                        You will receive task reminders at this email address.
                    </p>
                </div>
            `,
        });
    } catch (err) {
        console.error('Resend error:', error);
        return { success: false, error: error.message };
    }

    console.log('Test email sent successfully:', data);
    return { success: true };
} catch (err) {
    console.error('Email send error:', err);
    return { success: false, error: err instanceof Error ? err.message : 'Unknown error' };
}
}

export async function sendSmsReminder(task: TaskReminder): Promise<{ success: boolean; error?: string }> {
    try {
        const dueDate = new Date(task.dueDate);
        const formattedDate = dueDate.toLocaleDateString('en-GB', {
            weekday: 'short',
            day: 'numeric',
            month: 'short'
        });
    } catch (err) {
        console.error('Resend error:', error);
        return { success: false, error: error.message };
    }

    // SMS messages should be short and plain text
    const smsBody = `Uni-Cal Reminder: ${task.title}${task.courseName ? ` (${task.courseName})` : ''} - Due: ${format...`;

    const { data, error } = await resend.emails.send({
        from: FROM_EMAIL,
        to: SMS_EMAIL,
        subject: 'Task Reminder',
        text: smsBody,
    });

    if (error) {
        console.error('SMS send error:', error);
        return { success: false, error: error.message };
    }

    console.log('SMS sent successfully:', data);
    return { success: true };
} catch (err) {
    console.error('SMS send error:', err);
    return { success: false, error: err instanceof Error ? err.message : 'Unknown error' };
}
}

export async function sendTestSms(): Promise<{ success: boolean; error?: string }> {
    try {
        const { data, error } = await resend.emails.send({
            from: FROM_EMAIL,
            to: SMS_EMAIL,
            subject: 'Test',
            text: 'Uni-Cal SMS reminders are now set up!',
        });
    } catch (err) {
        console.error('SMS send error:', error);
        return { success: false, error: error.message };
    }

    console.log('Test SMS sent successfully:', data);
    return { success: true };
}
}

```

```

    } catch (err) {
      console.error('SMS send error:', err);
      return { success: false, error: err instanceof Error ? err.message : 'Unknown error' };
    }
  }

export async function sendCalendarInvite(params: {
  to: string[];
  title: string;
  description?: string;
  startDate: Date;
  endDate?: Date;
  location?: string;
  organizerName?: string;
  organizerEmail?: string;
}): Promise<{ success: boolean; error?: string }> {
  try {
    const { to, title, description, startDate, endDate, location, organizerName, organizerEmail } = params;

    const end = endDate || new Date(startDate.getTime() + 60 * 60 * 1000);
    const uid = `unical-${Date.now()}-${Math.random().toString(36).slice(2)}@uni-cal.app`;

    const fmt = (d: Date) => d.toISOString().replace(/[-:]/g, '').replace(/\\.\\d{3}/, '');

    const icsContent = [
      'BEGIN:VCALENDAR',
      'VERSION:2.0',
      'PRODID:-//Uni-Cal//Task Manager//EN',
      'CALSCALE:GREGORIAN',
      'METHOD:REQUEST',
      'BEGIN:VEVENT',
      `DTSTART:${fmt(startDate)}`,
      `DTEND:${fmt(end)}`,
      `SUMMARY:${title}`,
      `description ? `DESCRIPTION:${description.replace(/\\n/g, '\\\\n')}` : '',
      `location ? `LOCATION:${location}` : '',
      `UID:${uid}`,
      `ORGANIZER;CN=${organizerName || 'Uni-Cal'}:mailto:${organizerEmail || FROM_EMAIL}`,
      ...to.map(email => `ATTENDEE;ROLE=REQ-PARTICIPANT;RSVP=TRUE:mailto:${email}`),
      `DTSTAMP:${fmt(new Date())}`,
      'STATUS:CONFIRMED',
      'END:VEVENT',
      'END:VCALENDAR',
    ].filter(Boolean).join('\\r\\n');

    const icsBase64 = Buffer.from(icsContent).toString('base64');

    const formattedDate = startDate.toLocaleDateString('en-GB', {
      weekday: 'long',
      day: 'numeric',
      month: 'long',
      year: 'numeric',
    });
    const formattedTime = startDate.toLocaleTimeString('en-GB', {
      hour: '2-digit',
      minute: '2-digit',
    });

    const { data, error } = await resend.emails.send({
      from: FROM_EMAIL,
      to,
      subject: `Calendar Invite: ${title}`,
      html: `
        <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px;">
          <h2 style="color: #333; border-bottom: 2px solid #4578B0; padding-bottom: 10px;">
            Calendar Invite
          </h2>
          <div style="background: #f5f5f5; padding: 20px; border-radius: 8px; margin: 20px 0;">
            <h3 style="color: #042550; margin-top: 0;">${title}</h3>
            <p><strong>Date:</strong> ${formattedDate}</p>
            <p><strong>Time:</strong> ${formattedTime}</p>
            <p><strong>Location:</strong> ${location}</p> : ''
            <p><strong>Details:</strong> ${description}</p> : ''
          </div>
          <p style="color: #666; font-size: 14px;">
            Open the attached .ics file to add this event to your calendar.
          </p>
        </div>
      `,
      attachments: [
        {
          filename: 'invite.ics',
          content: icsBase64,
          content_type: 'text/calendar; method=REQUEST',
        },
      ],
    });

    if (error) {
      console.error('Calendar invite send error:', error);
      return { success: false, error: error.message };
    }

    console.log('Calendar invite sent successfully:', data);
    return { success: true };
  } catch (err) {
    console.error('Calendar invite send error:', err);
    return { success: false, error: err instanceof Error ? err.message : 'Unknown error' };
  }
}

// Home Assistant push notification functions
const HA_URL = "https://ec8ebfanqrqlsnmnggrdl4yzq2i8koah.ui.nabu.casa";
const tokenFromEnv = process.env.HOME_ASSISTANT_TOKEN || "";
const urlFromEnv = process.env.HOME_ASSISTANT_URL || "";
const HA_TOKEN = tokenFromEnv.startsWith("eyJ") ? tokenFromEnv : (urlFromEnv.startsWith("eyJ") ? urlFromEnv : tokenFr...)

export async function sendHaPushNotification(title: string, message: string): Promise<{ success: boolean; error?: str...

```

```

if (!HA_TOKEN) {
  return { success: false, error: 'Home Assistant Token not configured' };
}

const haUrl = HA_URL.replace(/\/$/, '');

try {
  const response = await fetch(`${haUrl}/api/services/notify/mobile_app_iphone_10`, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${HA_TOKEN}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      title: title,
      message: message,
    }),
  });

  if (!response.ok) {
    const errorText = await response.text();
    console.error('HA push notification error:', errorText);
    return { success: false, error: `HA API error: ${response.status}` };
  }

  console.log('HA push notification sent successfully');
  return { success: true };
} catch (err) {
  console.error('HA push notification error:', err);
  return { success: false, error: err instanceof Error ? err.message : 'Unknown error' };
}
}

export async function sendHaTaskReminder(task: TaskReminder): Promise<{ success: boolean; error?: string }> {
  const dueDate = new Date(task.dueDate);
  const formattedDate = dueDate.toLocaleDateString('en-GB', {
    weekday: 'short',
    day: 'numeric',
    month: 'short'
  });

  const title = `Uni-Cal Reminder`;
  const message = `${task.title}${task.courseName ? ` (${task.courseName})` : ''} - Due: ${formattedDate}`;

  return sendHaPushNotification(title, message);
}

export async function sendEchoVoiceAnnouncement(message: string): Promise<{ success: boolean; error?: string }> {
  if (!HA_TOKEN) {
    return { success: false, error: 'Home Assistant Token not configured' };
  }

  const haUrl = HA_URL.replace(/\/$/, '');
  const ECHO_TARGETS = [
    "media_player.cat_wr",
    "media_player.echo_cat_left_am",
    "media_player.echo_cat_right_am",
    "media_player.echo_cat_washroom_middle",
    "media_player.echo_kitchen_studio_black_am",
  ];

  try {
    const response = await fetch(`${haUrl}/api/services/notify/alexa_media`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${HA_TOKEN}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        message: message,
        data: { type: "tts" },
        target: ECHO_TARGETS,
      }),
    });

    if (!response.ok) {
      const errorText = await response.text();
      console.error('Echo voice announcement error:', errorText);
      return { success: false, error: `HA API error: ${response.status}` };
    }

    console.log('Echo voice announcement sent successfully');
    return { success: true };
  } catch (err) {
    console.error('Echo voice announcement error:', err);
    return { success: false, error: err instanceof Error ? err.message : 'Unknown error' };
  }
}

export async function sendTestHaPush(): Promise<{ success: boolean; error?: string }> {
  return sendHaPushNotification('Uni-Cal Test', 'Push notifications via Home Assistant are working!');
}

```

`server/gmail.ts`

```

// Gmail integration for D2L announcement emails
// Uses direct REST API calls with the connected Gmail access token

let connectionSettings: any;

export async function getGmailAccessToken(): Promise<string> {
  return getAccessToken();
}

```

```

async function getAccessToken(): Promise<string> {
  if (connectionSettings && connectionSettings.settings.expires_at && new Date(connectionSettings.settings.expires_at...
    return connectionSettings.settings.access_token;
  }

  const hostname = process.env.REPLIT_CONNECTORS_HOSTNAME;
  const xReplitToken = process.env.REPL_IDENTITY
    ? 'repl ' + process.env.REPL_IDENTITY
    : process.env.WEB_REPL_RENEWAL
    ? 'depl ' + process.env.WEB_REPL_RENEWAL
    : null;

  if (!xReplitToken) {
    throw new Error('X-Replit-Token not found for repl/depl');
  }

  connectionSettings = await fetch(
    'https://' + hostname + '/api/v2/connection?include_secrets=true&connector_names=google-mail',
    {
      headers: {
        'Accept': 'application/json',
        'X-Replit-Token': xReplitToken
      }
    }
  ).then(res => res.json()).then(data => data.items?.[0]);

  const accessToken = connectionSettings?.settings?.access_token || connectionSettings?.settings?.oauth?.credentials?...

  if (!connectionSettings || !accessToken) {
    throw new Error('Gmail not connected');
  }
  return accessToken;
}

const D2L_SENDER = 'NotificationEmail@toronto-mu.brightspace.com';

export interface D2LAnnouncement {
  id: string;
  subject: string;
  snippet: string;
  body: string;
  date: string;
  courseName: string;
  read: boolean;
}

function extractCourseName(subject: string): string {
  const match = subject.match(/\s*([^\s]+)\s*/);
  if (match) return match[1].trim();
  const dashMatch = subject.match(/^[A-Z]{3,4}\s\d{3}/i);
  if (dashMatch) return dashMatch[1];
  return 'University';
}

function decodeBase64Url(data: string): string {
  const base64 = data.replace(/-/g, '+').replace(/_/g, '/');
  return Buffer.from(base64, 'base64').toString('utf-8');
}

function stripHtml(html: string): string {
  return html
    .replace(/<style[^>]*>[\s\S]*?</style>/gi, '')
    .replace(/<script[^>]*>[\s\S]*?</script>/gi, '')
    .replace(/<br\s*\/?>/gi, '\n')
    .replace(/<\/p>/gi, '\n')
    .replace(/<\/div>/gi, '\n')
    .replace(/<[^>]+>/g, '')
    .replace(/&nbsp;/g, ' ')
    .replace(/&/g, '&')
    .replace(/</g, '<')
    .replace(/>/g, '>')
    .replace(/"/g, '"')
    .replace(/&#39;/g, "'")
    .replace(/\\n{3}/g, '\n\n')
    .trim();
}

async function gmailGet(endpoint: string): Promise<any> {
  const token = await getAccessToken();
  const resp = await fetch('https://gmail.googleapis.com/gmail/v1/users/me/${endpoint}', {
    headers: { 'Authorization': `Bearer ${token}` }
  });
  if (!resp.ok) {
    const errBody = await resp.text();
    throw new Error('Gmail API error ${resp.status}: ${errBody}');
  }
  return resp.json();
}

export async function sendGmail(params: { to: string; subject: string; htmlBody: string; textBody?: string }): Promis...
  try {
    const token = await getAccessToken();
    const boundary = 'boundary_' + Date.now();
    const mimeLines = [
      `To: ${params.to}`,
      `Subject: ${params.subject}`,
      `MIME-Version: 1.0`,
      `Content-Type: multipart/alternative; boundary="${boundary}"`,
      `--${boundary}`,
      `Content-Type: text/plain; charset="UTF-8"`,
      params.textBody || params.htmlBody.replace(/<[^>]+>/g, ''),
      `--${boundary}`,
      `Content-Type: text/html; charset="UTF-8"`,
      params.htmlBody,
    ];
  }

```

```

    '\n',
    `--${boundary}--`,
  ].join('\r\n');

  const raw = Buffer.from(mimeLines).toString('base64url');

  const resp = await fetch('https://gmail.googleapis.com/gmail/v1/users/me/messages/send', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({ raw }),
  });

  if (!resp.ok) {
    const errBody = await resp.text();
    console.error('[Gmail] Send error:', errBody);
    return { success: false, error: `Gmail API error ${resp.status}: ${errBody}` };
  }

  const data = await resp.json();
  console.log('[Gmail] Email sent successfully, id:', data.id);
  return { success: true };
} catch (err: any) {
  console.error('[Gmail] Send error:', err.message);
  return { success: false, error: err.message };
}
}

export async function sendGmailWithAttachment(params: { to: string; subject: string; htmlBody: string; attachments: A...
  try {
    const token = await getAccessToken();
    const boundary = 'boundary_att_' + Date.now();
    const mimeLines = [
      `To: ${params.to}`,
      `Subject: ${params.subject}`,
      `MIME-Version: 1.0`,
      `Content-Type: multipart/mixed; boundary="${boundary}"`,
      `--${boundary}`,
      `Content-Type: text/html; charset="UTF-8"`,
      params.htmlBody,
    ];

    for (const att of params.attachments) {
      mimeLines.push(`--${boundary}`);
      mimeLines.push(`Content-Type: ${att.mimeType}`);
      mimeLines.push(`Content-Transfer-Encoding: base64`);
      mimeLines.push(`Content-Disposition: attachment; filename="${att.filename}"`);
      mimeLines.push('');
      mimeLines.push(att.content.toString('base64'));
    }

    mimeLines.push(`--${boundary}--`);

    const raw = Buffer.from(mimeLines.join('\r\n')).toString('base64url');

    const resp = await fetch('https://gmail.googleapis.com/gmail/v1/users/me/messages/send', {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${token}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ raw }),
    });

    if (!resp.ok) {
      const errBody = await resp.text();
      console.error('[Gmail] Send with attachment error:', errBody);
      return { success: false, error: `Gmail API error ${resp.status}: ${errBody}` };
    }

    const data = await resp.json() as any;
    console.log('[Gmail] Email with attachment sent, id:', data.id);
    return { success: true };
  } catch (err: any) {
    console.error('[Gmail] Send with attachment error:', err.message);
    return { success: false, error: err.message };
  }
}

export async function fetchD2LAnnouncements(maxResults: number = 20): Promise<D2LAnnouncement[]> {
  try {
    const query = encodeURIComponent(`from:${D2L_SENDER}`);
    const listData = await gmailGet(`messages?q=${query}&maxResults=${maxResults}`);

    const messages = listData.messages || [];
    if (messages.length === 0) return [];

    const announcements: D2LAnnouncement[] = [];

    for (const msg of messages) {
      try {
        const detail = await gmailGet(`messages/${msg.id}?format=full`);

        const headers = detail.payload?.headers || [];
        const subject = headers.find((h: any) => h.name?.toLowerCase() === 'subject')?.value || 'No Subject';
        const dateStr = headers.find((h: any) => h.name?.toLowerCase() === 'date')?.value || '';
        const isRead = !detail.labelIds?.includes('UNREAD');

        let bodyText = '';
        const payload = detail.payload;

        if (payload?.body?.data) {
          bodyText = decodeBase64Url(payload.body.data);
        } else if (payload?.parts) {

```

```

    for (const part of payload.parts) {
      if (part.mimeType === 'text/plain' && part.body?.data) {
        bodyText = decodeBase64Url(part.body.data);
        break;
      }
      if (part.mimeType === 'text/html' && part.body?.data) {
        bodyText = stripHtml(decodeBase64Url(part.body.data));
      }
    }
  }
}

const cleanSubject = subject
  .replace(/^Fwd:\s*/i, '')
  .replace(/^Fw:\s*/i, '')
  .trim();

announcements.push({
  id: msg.id,
  subject: cleanSubject,
  snippet: detail.snippet || '',
  body: bodyText.slice(0, 500),
  date: dateStr ? new Date(dateStr).toISOString() : new Date().toISOString(),
  courseName: extractCourseName(cleanSubject),
  read: isRead,
});
} catch (e: any) {
  console.error(`[Gmail] Failed to fetch message ${msg.id}:`, e.message);
}
}

announcements.sort((a, b) => new Date(b.date).getTime() - new Date(a.date).getTime());
return announcements;
} catch (err: any) {
  console.error(`[Gmail] Error fetching D2L announcements:`, err.message);
  throw err;
}
}

```

`server/gmailTicker.ts`

```

export interface TickerEmail {
  emailId: string;
  body: string;
  receivedAt: Date;
  command?: 'delete' | 'clear' | 'expire';
  commandTarget?: string;
  expireMinutes?: number;
}

export function parseTickerCommand(body: string): { command?: 'delete' | 'clear' | 'expire'; target?: string; expireM...
  const lower = body.toLowerCase().trim();

  if (lower === 'clear all' || lower === 'delete all' || lower === 'remove all') {
    return { command: 'clear' };
  }

  const deleteMatch = lower.match(/^(?:delete|remove)\s+(.+(?:\s+(?:from\s+)?ticker)?$/i);
  if (deleteMatch) {
    return { command: 'delete', target: deleteMatch[1].trim() };
  }

  const expireMatch = lower.match(/^(?:expire|expires?)\s+(?:in\s+)?(\d+)\s*(min(?:ute)?s?|hours?|hr?s?)/i);
  if (expireMatch) {
    let minutes = parseInt(expireMatch[1]);
    const unit = expireMatch[2].toLowerCase();
    if (unit.startsWith('h')) minutes *= 60;
    return { command: 'expire', expireMinutes: minutes };
  }

  return {};
}

export function extractInlineExpiry(body: string): { cleanBody: string; expireMinutes: number | null } {
  const match = body.match(/\[expires?:\s*(\d+)\s*(min(?:ute)?s?|hours?|hr?s?|days?)\s*\]/i);
  if (!match) return { cleanBody: body, expireMinutes: null };
  let minutes = parseInt(match[1]);
  const unit = match[2].toLowerCase();
  if (unit.startsWith('h')) minutes *= 60;
  if (unit.startsWith('d')) minutes *= 1440;
  const cleanBody = body.replace(match[0], '').trim();
  return { cleanBody, expireMinutes: minutes };
}

```

`server/googleCalendar.ts`

```

// Google Calendar Integration - Replit Connector
import { google } from 'googleapis';

let connectionSettings: any;

async function getAccessToken() {
  if (connectionSettings && connectionSettings.settings.expires_at && new Date(connectionSettings.settings.expires_at...
    return connectionSettings.settings.access_token;
  }

  const hostname = process.env.REPLIT_CONNECTORS_HOSTNAME
  const xReplitToken = process.env.REPL_IDENTITY
  ? 'repl ' + process.env.REPL_IDENTITY
  : process.env.WEB_REPL_RENEWAL
  ? 'depl ' + process.env.WEB_REPL_RENEWAL

```



```

    : null;

    if (!xReplitToken) {
        throw new Error('X_REPLIT_TOKEN not found for repl/depl');
    }

    connectionSettings = await fetch(
        'https://' + hostname + '/api/v2/connection?include_secrets=true&connector_names=google-calendar',
        {
            headers: {
                'Accept': 'application/json',
                'X_REPLIT_TOKEN': xReplitToken
            }
        }
    ).then(res => res.json()).then(data => data.items?.[0]);

    const accessToken = connectionSettings?.settings?.access_token || connectionSettings?.settings?.oauth?.credentials?...

    if (!connectionSettings || !accessToken) {
        throw new Error('Google Calendar not connected');
    }
    return accessToken;
}

// WARNING: Never cache this client.
// Access tokens expire, so a new client must be created each time.
// Always call this function again to get a fresh client.
export async function getGoogleCalendarClient() {
    const accessToken = await getAccessToken();

    const oauth2Client = new google.auth.OAuth2();
    oauth2Client.setCredentials({
        access_token: accessToken
    });

    return google.calendar({ version: 'v3', auth: oauth2Client });
}

// Create a calendar event from a task
export async function createCalendarEvent(task: {
    id: number;
    title: string;
    description?: string | null;
    dueDate: Date | string;
    courseName?: string | null;
}) {
    const calendar = await getGoogleCalendarClient();

    const dueDate = new Date(task.dueDate);
    const hour = dueDate.getHours();

    // Check if this is an all-day task (midnight or 11 PM)
    const isAllDay = hour === 0 || hour === 23;

    const summary = `${task.courseName ? `[${task.courseName}]` : ''} ${task.title}`;

    const checkDateStr = dueDate.toISOString().split('T')[0];
    const existingEventId = await findExistingEventBySummary(summary, checkDateStr);
    if (existingEventId) {
        console.log(`[Calendar] Event already exists: "${summary}" on ${checkDateStr}, reusing id=${existingEventId}`);
        return { id: existingEventId };
    }

    if (task.courseName) {
        const coursePrefix = `[${task.courseName}]`;
        const titleWords = task.title.toLowerCase().replace(/^[a-z0-9\s&]/g, '').trim().split(/\s+/).filter(w => w.length...);
        try {
            const checkCal = await getGoogleCalendarClient();
            const tMin = new Date(checkDateStr + 'T00:00:00-05:00').toISOString();
            const tMax = new Date(checkDateStr + 'T23:59:59-05:00').toISOString();
            const resp = await checkCal.events.list({ calendarId: 'primary', timeMin: tMin, timeMax: tMax, q: coursePrefix... });
            const similar = (resp.data.items || []).find(e => {
                if (!e.summary.startsWith(coursePrefix)) return false;
                const existingTitle = e.summary.replace(coursePrefix, '').trim().toLowerCase().replace(/^[a-z0-9\s&]/g, '').t...
                return titleWords.some(w => existingTitle.includes(w));
            });
            if (similar?.id) {
                console.log(`[Calendar] Similar event found: "${similar.summary}" for "${summary}" on ${checkDateStr}, reusin...`);
                return { id: similar.id };
            }
        } catch (err) {
            console.error('[Calendar] Fuzzy dedup check failed:', err);
        }
    }

    let event: any;

    if (isAllDay) {
        // All-day event uses date (not dateTime)
        const dateStr = dueDate.toISOString().split('T')[0]; // YYYY-MM-DD format
        // Google Calendar uses exclusive end date, so add 1 day for single-day events
        const nextDay = new Date(dueDate);
        nextDay.setDate(nextDay.getDate() + 1);
        const endDateStr = nextDay.toISOString().split('T')[0];
        event = {
            summary,
            description: task.description || '',
            start: {
                date: dateStr,
            },
            end: {
                date: endDateStr,
            },
            reminders: {
                useDefault: false,
                overrides: [
                    { method: 'popup', minutes: 60 * 24 * 2 }, // 2 days before
                    { method: 'popup', minutes: 60 * 24 }, // 1 day before
                ],
            },
        };
    }
}

```

```

    },
  };
} else {
  // Timed event
  const startTime = dueDate.toISOString();
  const endTime = new Date(dueDate.getTime() + 60 * 60 * 1000).toISOString();
  event = {
    summary,
    description: task.description || '',
    start: {
      dateTime: startTime,
      timeZone: 'America/Toronto',
    },
    end: {
      dateTime: endTime,
      timeZone: 'America/Toronto',
    },
    reminders: {
      useDefault: false,
      overrides: [
        { method: 'popup', minutes: 60 * 24 * 2 }, // 2 days before
        { method: 'popup', minutes: 60 * 24 }, // 1 day before
        { method: 'popup', minutes: 60 * 2 }, // 2 hours before
        { method: 'popup', minutes: 30 }, // 30 minutes before
      ],
    },
  };
}

const response = await calendar.events.insert({
  calendarId: 'primary',
  requestBody: event,
});

return response.data;
}

// Create event in a specific calendar (for secondary calendar sync)
export async function createEventInCalendar(calendarId: string, task: {
  id: number;
  title: string;
  description?: string | null;
  dueDate: Date | string;
  courseName?: string | null;
}) {
  const calendar = await getGoogleCalendarClient();

  const dueDate = new Date(task.dueDate);
  const hour = dueDate.getHours();
  const isAllDay = hour === 0 || hour === 23;

  const summary = `${task.courseName ? `[${task.courseName}]` : ''} ${task.title}`;

  let event: any;

  if (isAllDay) {
    const dateStr = dueDate.toISOString().split('T')[0];
    const nextDay = new Date(dueDate);
    nextDay.setDate(nextDay.getDate() + 1);
    const endDateStr = nextDay.toISOString().split('T')[0];

    event = {
      summary,
      description: task.description || '',
      start: { date: dateStr },
      end: { date: endDateStr },
    };
  } else {
    const startTime = dueDate.toISOString();
    const endDate = new Date(dueDate.getTime() + 60 * 60 * 1000);
    const endTime = endDate.toISOString();

    event = {
      summary,
      description: task.description || '',
      start: { dateTime: startTime, timeZone: 'America/Los_Angeles' },
      end: { dateTime: endTime, timeZone: 'America/Los_Angeles' },
    };
  }

  const response = await calendar.events.insert({
    calendarId: calendarId,
    requestBody: event,
  });

  return response.data;
}

// Delete event from a specific calendar
export async function deleteEventFromCalendar(calendarId: string, eventId: string) {
  const calendar = await getGoogleCalendarClient();

  await calendar.events.delete({
    calendarId: calendarId,
    eventId: eventId,
  });
}

// Delete a calendar event
export async function deleteCalendarEvent(eventId: string) {
  const calendar = await getGoogleCalendarClient();

  await calendar.events.delete({
    calendarId: 'primary',
    eventId: eventId,
  });
}

```

```

// Helper to check if a date should be treated as all-day
function isAllDayEvent(date: Date): boolean {
  const utcHour = date.getUTCHours();
  const utcMinute = date.getUTCMinutes();
  // All-day if at midnight (00:00) or near end of day (23:00+)
  return utcHour === 0 || utcHour >= 23;
}

// Update a calendar event (deletes and recreates if switching between all-day/timed)
export async function updateCalendarEvent(eventId: string, task: {
  title: string;
  description?: string | null;
  dueDate: Date | string;
  courseName?: string | null;
}) {
  const calendar = await getGoogleCalendarClient();

  const dueDate = new Date(task.dueDate);
  const isAllDay = isAllDayEvent(dueDate);

  const summary = `${task.courseName ? `[${task.courseName}]` : ''}${task.title}`;

  let existingEvent;
  try {
    existingEvent = await calendar.events.get({
      calendarId: 'primary',
      eventId: eventId,
    });
  } catch (err: any) {
    const status = err?.status || err?.code;
    if (status === 410 || status === 404) {
      console.log(`Event ${eventId} was deleted/gone, will recreate`);
      return { id: null, deleted: true } as any;
    }
    console.log(`Existing event ${eventId} not found, will create new`);
    existingEvent = null;
  }

  const existingIsAllDay = !!(existingEvent?.data?.start?.date && !existingEvent?.data?.start?.dateTime);

  if (existingEvent && existingIsAllDay !== isAllDay) {
    console.log(`Switching event type: was all-day=${existingIsAllDay}, now all-day=${isAllDay}`);
    try {
      await calendar.events.delete({
        calendarId: 'primary',
        eventId: eventId,
      });
    } catch (delErr: any) {
      console.log(`Delete during type switch failed (likely already gone), continuing with create`);
    }

    // Create new event with correct type
    let event: any;
    if (isAllDay) {
      const dateStr = dueDate.toISOString().split('T')[0];
      const nextDay = new Date(dueDate);
      nextDay.setDate(nextDay.getDate() + 1);
      const endDateStr = nextDay.toISOString().split('T')[0];
      event = {
        summary,
        description: task.description || '',
        start: { date: dateStr },
        end: { date: endDateStr },
        reminders: { useDefault: false, overrides: [] },
      };
    } else {
      const startTime = dueDate.toISOString();
      const endTime = new Date(dueDate.getTime() + 60 * 60 * 1000).toISOString();
      event = {
        summary,
        description: task.description || '',
        start: { dateTime: startTime, timeZone: 'America/Toronto' },
        end: { dateTime: endTime, timeZone: 'America/Toronto' },
        reminders: {
          useDefault: false,
          overrides: [
            { method: 'popup', minutes: 720 },
            { method: 'popup', minutes: 360 },
            { method: 'popup', minutes: 120 },
            { method: 'popup', minutes: 30 },
          ],
        },
      };
    }
  }

  const response = await calendar.events.insert({
    calendarId: 'primary',
    requestBody: event,
  });
  return response.data;
}

// Same type, just update in place
let event: any;
if (isAllDay) {
  const dateStr = dueDate.toISOString().split('T')[0];
  const nextDay = new Date(dueDate);
  nextDay.setDate(nextDay.getDate() + 1);
  const endDateStr = nextDay.toISOString().split('T')[0];
  event = {
    summary,
    description: task.description || '',
    start: { date: dateStr },
    end: { date: endDateStr },
  };
} else {
  const startTime = dueDate.toISOString();
  const endTime = new Date(dueDate.getTime() + 60 * 60 * 1000).toISOString();

```

```

    event = {
      summary,
      description: task.description || '',
      start: { dateTime: startTime, timeZone: 'America/Toronto' },
      end: { dateTime: endTime, timeZone: 'America/Toronto' },
    };
  }

  const response = await calendar.events.update({
    calendarId: 'primary',
    eventId: eventId,
    requestBody: event,
  });

  return response.data;
}

// Create a prep/planning event for when to start working on a task
export async function createPrepCalendarEvent(task: {
  id: number;
  title: string;
  description?: string | null;
  startDate: Date | string;
  dueDate: Date | string;
  courseName?: string | null;
}) {
  const calendar = await getGoogleCalendarClient();

  const startDate = new Date(task.startDate);
  const dueDate = new Date(task.dueDate);
  const isAllDay = isAllDayEvent(startDate);

  const summary = `[PREP] ${task.courseName ? `[${task.courseName}]` : ''} ${task.title}`;
  const description = `Start preparing for: ${task.title}\nDue: ${dueDate.toLocaleDateString()}\n\n${task.description...}`;

  let event: any;

  if (isAllDay) {
    const dateStr = startDate.toISOString().split('T')[0];
    const nextDay = new Date(startDate);
    nextDay.setDate(nextDay.getDate() + 1);
    const endDateStr = nextDay.toISOString().split('T')[0];
    event = {
      summary,
      description,
      start: { date: dateStr },
      end: { date: endDateStr },
      reminders: { useDefault: false, overrides: [] },
    };
  } else {
    const startTime = startDate.toISOString();
    const endTime = new Date(startDate.getTime() + 60 * 60 * 1000).toISOString();
    event = {
      summary,
      description,
      start: { dateTime: startTime, timeZone: 'America/Toronto' },
      end: { dateTime: endTime, timeZone: 'America/Toronto' },
      reminders: {
        useDefault: false,
        overrides: [
          { method: 'popup', minutes: 60 },
          { method: 'popup', minutes: 30 },
        ],
      },
    };
  }

  const response = await calendar.events.insert({
    calendarId: 'primary',
    requestBody: event,
  });

  return response.data;
}

// Update a prep/planning event
export async function updatePrepCalendarEvent(eventId: string, task: {
  title: string;
  description?: string | null;
  startDate: Date | string;
  dueDate: Date | string;
  courseName?: string | null;
}) {
  const calendar = await getGoogleCalendarClient();

  const startDate = new Date(task.startDate);
  const dueDate = new Date(task.dueDate);
  const isAllDay = isAllDayEvent(startDate);

  const summary = `[PREP] ${task.courseName ? `[${task.courseName}]` : ''} ${task.title}`;
  const description = `Start preparing for: ${task.title}\nDue: ${dueDate.toLocaleDateString()}\n\n${task.description...}`;

  let existingEvent;
  try {
    existingEvent = await calendar.events.get({
      calendarId: 'primary',
      eventId: eventId,
    });
  } catch (err: any) {
    const status = err?.status || err?.code;
    if (status === 410 || status === 404) {
      console.log(`Prep event ${eventId} was deleted/gone, will recreate`);
      return { id: null, deleted: true } as any;
    }
    existingEvent = null;
  }

  const existingIsAllDay = !!(existingEvent?.data?.start?.date && !existingEvent?.data?.start?.dateTime);

```

```

if (existingEvent && existingIsAllDay !== isAllDay) {
  try {
    await calendar.events.delete({
      calendarId: 'primary',
      eventId: eventId,
    });
  } catch (delErr: any) {
    console.log(`Delete prep during type switch failed (likely already gone), continuing with create`);
  }

  let event: any;
  if (isAllDay) {
    const dateStr = startDate.toISOString().split('T')[0];
    const nextDay = new Date(startDate);
    nextDay.setDate(nextDay.getDate() + 1);
    const endDateStr = nextDay.toISOString().split('T')[0];
    event = {
      summary,
      description,
      start: { date: dateStr },
      end: { date: endDateStr },
      reminders: { useDefault: false, overrides: [] },
    };
  } else {
    const startTime = startDate.toISOString();
    const endTime = new Date(startDate.getTime() + 60 * 60 * 1000).toISOString();
    event = {
      summary,
      description,
      start: { dateTime: startTime, timeZone: 'America/Toronto' },
      end: { dateTime: endTime, timeZone: 'America/Toronto' },
      reminders: {
        useDefault: false,
        overrides: [{ method: 'popup', minutes: 60 }, { method: 'popup', minutes: 30 }],
      },
    };
  }

  const response = await calendar.events.insert({
    calendarId: 'primary',
    requestBody: event,
  });
  return response.data;
}

let event: any;
if (isAllDay) {
  const dateStr = startDate.toISOString().split('T')[0];
  const nextDay = new Date(startDate);
  nextDay.setDate(nextDay.getDate() + 1);
  const endDateStr = nextDay.toISOString().split('T')[0];
  event = {
    summary,
    description,
    start: { date: dateStr },
    end: { date: endDateStr },
  };
} else {
  const startTime = startDate.toISOString();
  const endTime = new Date(startDate.getTime() + 60 * 60 * 1000).toISOString();
  event = {
    summary,
    description,
    start: { dateTime: startTime, timeZone: 'America/Toronto' },
    end: { dateTime: endTime, timeZone: 'America/Toronto' },
  };
}

const response = await calendar.events.update({
  calendarId: 'primary',
  eventId: eventId,
  requestBody: event,
});
return response.data;
}

// List calendars
export async function listCalendars() {
  const calendar = await getGoogleCalendarClient();

  const response = await calendar.calendarList.list();
  return response.data.items || [];
}

// List events in a date range
export async function listEvents(timeMin: Date, timeMax: Date) {
  const calendar = await getGoogleCalendarClient();

  let allItems: any[] = [];
  let pageToken: string | undefined;

  do {
    const response = await calendar.events.list({
      calendarId: 'primary',
      timeMin: timeMin.toISOString(),
      timeMax: timeMax.toISOString(),
      singleEvents: true,
      orderBy: 'startTime',
      maxResults: 2500,
      pageToken,
    });

    const items = response.data.items || [];
    allItems = allItems.concat(items);
    pageToken = response.data.nextPageToken || undefined;
  } while (pageToken);
}

```

```

    return allItems;
}

// Create a recurring class event
export async function createRecurringClassEvent(classInfo: {
  courseName: string;
  courseCode: string;
  startDate: string; // YYYY-MM-DD
  endDate: string; // YYYY-MM-DD
  startTime: string; // HH:MM (24hr)
  endTime: string; // HH:MM (24hr)
  daysOfWeek: string[]; // ['MO', 'TU', 'WE', 'TH', 'FR']
  location?: string;
}) {
  const calendar = await getGoogleCalendarClient();

  // Build RRULE - e.g., RRULE:FREQ=WEEKLY;BYDAY=TU,TH;UNTIL=20260616T235959Z
  const untilDate = classInfo.endDate.replace(/-/g, '') + 'T235959Z';
  const byDay = classInfo.daysOfWeek.join(',');
  const recurrence = [`RRULE:FREQ=WEEKLY;BYDAY=${byDay};UNTIL=${untilDate}`];

  // Create start and end datetime
  const startDateTime = `${classInfo.startDate}T${classInfo.startTime}:00`;
  const endDateTime = `${classInfo.startDate}T${classInfo.endTime}:00`;

  const event = {
    summary: `${classInfo.courseCode} - ${classInfo.courseName}`,
    location: classInfo.location || '',
    start: {
      dateTime: startDateTime,
      timeZone: 'America/Toronto',
    },
    end: {
      dateTime: endDateTime,
      timeZone: 'America/Toronto',
    },
    recurrence,
    reminders: {
      useDefault: false,
      overrides: [
        { method: 'popup', minutes: 60 }, // 1 hour before
        { method: 'popup', minutes: 15 }, // 15 minutes before
      ],
    },
  };

  const response = await calendar.events.insert({
    calendarId: 'primary',
    requestBody: event,
  });

  return response.data;
}

export async function findExistingEventBySummary(summary: string, dateStr: string): Promise<string | null> {
  const calendar = await getGoogleCalendarClient();
  try {
    const timeMin = new Date(dateStr + 'T00:00:00-05:00').toISOString();
    const timeMax = new Date(dateStr + 'T23:59:59-05:00').toISOString();
    const response = await calendar.events.list({
      calendarId: 'primary',
      timeMin,
      timeMax,
      q: summary.substring(0, 50),
      singleEvents: true,
      maxResults: 20,
    });
    const events = response.data.items || [];
    const match = events.find(e => e.summary === summary);
    return match?.id || null;
  } catch {
    return null;
  }
}

export async function findAndDeleteDuplicateEvents(timeMin: string, timeMax: string): Promise<{ deleted: number; kept...> {
  const calendar = await getGoogleCalendarClient();
  const response = await calendar.events.list({
    calendarId: 'primary',
    timeMin,
    timeMax,
    singleEvents: true,
    maxResults: 2500,
    orderBy: 'startTime',
  });
  const events = response.data.items || [];
  const seen = new Map<string, string>();
  let deleted = 0;
  const delay = (ms: number) => new Promise(r => setTimeout(r, ms));

  for (const event of events) {
    if (!event.summary || !event.id) continue;
    const startKey = event.start?.dateTime || event.start?.date || '';
    const key = `${event.summary}|||${startKey}`;
    if (seen.has(key)) {
      try {
        await calendar.events.delete({ calendarId: 'primary', eventId: event.id });
        deleted++;
        await delay(200);
      } catch (err: any) {
        console.error('Failed to delete duplicate event ${event.id}:', err?.message);
      }
    } else {
      seen.set(key, event.id);
    }
  }
  return { deleted, kept: seen.size };
}

```

```

}

export async function createYearlyScholarshipEvent(info: {
  name: string;
  date: string;
  type: 'open' | 'deadline';
  description?: string;
  recurring?: boolean;
}): Promise<{ id: string } | null> {
  try {
    const calendar = await getGoogleCalendarClient();
    const dateObj = new Date(info.date + 'T12:00:00');
    const summary = info.type === 'open'
      ? `0=0i Scholarship Opens: ${info.name}`
      : `0=0" Scholarship Deadline: ${info.name}`;

    const dateStr = info.date;
    const endDate = new Date(dateObj);
    endDate.setDate(endDate.getDate() + 1);
    const endDateStr = endDate.toISOString().split('T')[0];

    const isRecurring = info.recurring !== false;

    const event: any = {
      summary,
      description: info.description || '',
      start: { date: dateStr },
      end: { date: endDateStr },
      reminders: {
        useDefault: false,
        overrides: [
          { method: 'popup', minutes: 60 * 24 * 7 },
          { method: 'popup', minutes: 60 * 24 * 2 },
        ],
      },
    };

    if (isRecurring) {
      const rruleUntilYear = dateObj.getFullYear() + 10;
      const rruleUntilMonth = String(dateObj.getMonth() + 1).padStart(2, '0');
      const rruleUntilDay = String(dateObj.getDate()).padStart(2, '0');
      event.recurrence = ['RRULE:FREQ=YEARLY;UNTIL=${rruleUntilYear}${rruleUntilMonth}${rruleUntilDay}T235959Z'];
    }

    const response = await calendar.events.insert({
      calendarId: 'primary',
      requestBody: event,
    });
    console.log(`[Calendar] Created yearly scholarship ${info.type} event: "${summary}" on ${dateStr}, id=${response.id}`);
    return { id: response.data.id || '' };
  } catch (error) {
    console.error(`[Calendar] Failed to create yearly scholarship ${info.type} event:`, error);
    return null;
  }
}

```

`server/index.ts`

```

process.env.TZ = 'America/Toronto';

import express, { type Request, Response, NextFunction } from "express";
import { registerRoutes } from "../routes";
import { serveStatic } from "../static";
import { createServer } from "http";
import { startReminderScheduler } from "../reminderScheduler";
import { storage } from "../storage";
import crypto from "crypto";
import cookieParser from "cookie-parser";

async function checkAndSwitchSemester() {
  try {
    const active = await storage.getActiveSemesterSettings();
    if (!active || !active.semesterEndDate) return;

    const now = new Date();
    const endDate = new Date(active.semesterEndDate);

    if (now > endDate) {
      const allSemesters = await storage.getAllSemesterSettings();
      const nextSemester = allSemesters
        .filter(s => !s.isActive && new Date(s.semesterStartDate) > endDate)
        .sort((a, b) => new Date(a.semesterStartDate).getTime() - new Date(b.semesterStartDate).getTime())[0];

      if (nextSemester) {
        await storage.updateSemesterSettings(active.id, { isActive: false });
        await storage.updateSemesterSettings(nextSemester.id, { isActive: true });
        console.log(`[Semester Switch] Switched from "${active.semesterName}" to "${nextSemester.semesterName}"`);
      } else {
        console.log(`[Semester Switch] "${active.semesterName}" has ended but no next semester found`);
      }
    }
  } catch (err) {
    console.error("[Semester Switch] Error checking semester:", err);
  }
}

const app = express();
const httpServer = createServer(app);

declare module "http" {
  interface IncomingMessage {
    rawBody: unknown;
  }
}

```

```

app.use(cookieParser());

app.use((req: any, res: any, next: any) => {
  if (req.path === '/api/uploads/direct') {
    return next();
  }
  const contentType = (req.headers['content-type'] || '').toLowerCase();
  if (req.method !== 'GET' && req.method !== 'HEAD' && contentType.includes('application/json')) {
    let chunks: Buffer[] = [];
    const origOn = req.on.bind(req);
    let rawData = '';
    express.text({ limit: '50mb', type: 'application/json' })(req, res, (err: any) => {
      if (typeof req.body === 'string') {
        const raw = req.body;
        req.rawBody = Buffer.from(raw);
        try {
          req.body = JSON.parse(raw);
        } catch {
          try {
            const fixed = raw.replace(/\bTrue\b/g, 'true').replace(/\bFalse\b/g, 'false').replace(/\bNone\b/g, 'null');
            req.body = JSON.parse(fixed);
          } catch {
            req.body = {};
          }
        }
      }
      next();
    });
  } else {
    express.json({ limit: '50mb', verify: (req: any, _res: any, buf: Buffer) => { req.rawBody = buf; } })(req, res, n...
  )
});

app.use((req: any, res: any, next: any) => {
  if (req.path === '/api/uploads/direct') return next();
  express.urlencoded({ extended: false })(req, res, next);
});

const SITE_PASSWORD = process.env.SITE_PASSWORD;
const SESSION_SECRET = process.env.SESSION_SECRET || "uni-cal-session-key";

const TOKEN_MAX_AGE_MS = 10 * 365 * 24 * 60 * 60 * 1000;

function createSessionToken(): string {
  const timestamp = Date.now().toString(36);
  const sig = crypto.createHmac("sha256", SESSION_SECRET)
    .update(`uni-cal-auth:${SITE_PASSWORD}:${timestamp}`)
    .digest("hex");
  return `${timestamp}.${sig}`;
}

function isValidToken(token: string): boolean {
  if (!SITE_PASSWORD) return true;
  if (!token || typeof token !== "string") return false;
  const dotIndex = token.indexOf(".");
  if (dotIndex === -1) return false;
  const timestamp = token.substring(0, dotIndex);
  const sig = token.substring(dotIndex + 1);
  if (!/^[a-z0-9]+$/.test(timestamp) || !/^[a-f0-9]{64}$/.test(sig)) return false;
  const created = parseInt(timestamp, 36);
  if (isNaN(created) || Date.now() - created > TOKEN_MAX_AGE_MS) return false;
  const expected = crypto.createHmac("sha256", SESSION_SECRET)
    .update(`uni-cal-auth:${SITE_PASSWORD}:${timestamp}`)
    .digest("hex");
  return sig.length === expected.length && crypto.timingSafeEqual(Buffer.from(sig), Buffer.from(expected));
}

function getAuthToken(req: Request): string | undefined {
  const cookieToken = req.cookies?.uni_cal_session;
  if (cookieToken) return cookieToken;
  const authHeader = req.headers.authorization;
  if (authHeader && authHeader.startsWith('Bearer ')) {
    return authHeader.substring(7);
  }
  const queryToken = req.query?.token as string | undefined;
  if (queryToken) return queryToken;
  return undefined;
}

function isAutoAuthRequest(req: Request): boolean {
  const autoAuth = req.query?.auth as string | undefined;
  return autoAuth === SITE_PASSWORD;
}

app.post("/api/auth/login", (req: Request, res: Response) => {
  const { password } = req.body;
  if (!SITE_PASSWORD) {
    return res.json({ success: true, token: '' });
  }
  if (password === SITE_PASSWORD) {
    const token = createSessionToken();
    res.cookie("uni_cal_session", token, {
      httpOnly: true,
      secure: process.env.NODE_ENV === "production",
      sameSite: "lax",
      maxAge: 10 * 365 * 24 * 60 * 60 * 1000,
    });
    return res.json({ success: true, token });
  }
  return res.status(401).json({ success: false, message: "Incorrect password" });
});

app.get("/api/auth/check", (req: Request, res: Response) => {
  if (!SITE_PASSWORD) {
    return res.json({ authenticated: true });
  }
  if (isAutoAuthRequest(req)) {

```



```

const token = createSessionToken();
res.cookie("uni_cal_session", token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === "production",
  sameSite: "lax",
  maxAge: 10 * 365 * 24 * 60 * 60 * 1000,
});
return res.json({ authenticated: true, token });
}
const token = getAuthToken(req);
if (token) {
  try {
    if (isValidToken(token)) {
      return res.json({ authenticated: true });
    }
  } catch (e) {}
}
return res.json({ authenticated: false });
});

app.post("/api/auth/logout", (_req: Request, res: Response) => {
  res.clearCookie("uni_cal_session");
  return res.json({ success: true });
});

app.use((req: Request, res: Response, next: NextFunction) => {
  if (!SITE_PASSWORD) return next();
  if (process.env.NODE_ENV !== "production") return next();
  if (req.path.startsWith("/api/auth/")) return next();
  if (req.path.startsWith("/api/webhook/")) return next();
  if (req.path.startsWith("/api/shower/")) return next();
  if (req.path.startsWith("/api/cat-wash/")) return next();
  if (req.path.startsWith("/api/tablet-nav/")) return next();
  if (req.path === "/api/debug-beacon") return next();
  if (req.path === "/tablet") return next();
  if (req.path.startsWith("/api/tts-audio/")) return next();
  if (req.path === "/api/ticker" || req.path.startsWith("/api/ticker-assets/")) return next();
  if (req.path === "/api/news" || req.path === "/api/weather" || req.path === "/api/pollen" || req.path === "/api/wea...") return next();
  if (req.path.startsWith("/api/webhook/")) || req.path === "/api/announcements/webhook") return next();
  if (req.path === "/api/client-error") return next();
  if (req.path === "/api/export" || req.path === "/api/import" || req.path === "/api/cleanup-duplicates") return next();
  if (req.path === "/api/shift-schedule" && req.method === "POST") return next();
  if (req.path.startsWith("/api/files/") && req.method === "PATCH") return next();
  if (req.path === "/login") return next();
  if (req.path.startsWith("/assets/") || req.path.startsWith("/favicon")) return next();

  if (isAutoAuthRequest(req)) {
    const newToken = createSessionToken();
    res.cookie("uni_cal_session", newToken, {
      httpOnly: true,
      secure: process.env.NODE_ENV === "production",
      sameSite: "lax",
      maxAge: 10 * 365 * 24 * 60 * 60 * 1000,
    });
    return next();
  }

  const token = getAuthToken(req);
  if (token) {
    try {
      if (isValidToken(token)) {
        return next();
      }
    } catch (e) {}
  }

  if (req.path.startsWith("/api/")) {
    return res.status(401).json({ message: "Not authenticated" });
  }

  return next();
});

export function log(message: string, source = "express") {
  const formattedTime = new Date().toLocaleTimeString("en-US", {
    hour: "numeric",
    minute: "2-digit",
    second: "2-digit",
    hour12: true,
  });
  console.log(`${formattedTime} [${source}] ${message}`);
}

app.use((req, res, next) => {
  const start = Date.now();
  const path = req.path;
  let capturedJsonResponse: Record<string, any> | undefined = undefined;

  const originalResJson = res.json;
  res.json = function (bodyJson, ...args) {
    capturedJsonResponse = bodyJson;
    return originalResJson.apply(res, [bodyJson, ...args]);
  };

  res.on("finish", () => {
    const duration = Date.now() - start;
    if (path.startsWith("/api/")) {
      let logLine = `${req.method} ${path} ${res.statusCode} in ${duration}ms`;
      if (capturedJsonResponse) {
        logLine += ` :: ${JSON.stringify(capturedJsonResponse)}`;
      }
      log(logLine);
    }
  });
});

next();

```

```

});

(async () => {
  await registerRoutes(httpServer, app);

  app.use((err: any, _req: Request, res: Response, next: NextFunction) => {
    const status = err.status || err.statusCode || 500;
    const message = err.message || "Internal Server Error";

    console.error("Internal Server Error:", err);

    if (res.headersSent) {
      return next(err);
    }

    return res.status(status).json({ message });
  });

  // importantly only setup vite in development and after
  // setting up all the other routes so the catch-all route
  // doesn't interfere with the other routes
  if (process.env.NODE_ENV === "production") {
    serveStatic(app);
  } else {
    const { setupVite } = await import("./vite");
    await setupVite(httpServer, app);
  }

  // ALWAYS serve the app on the port specified in the environment variable PORT
  // Other ports are firewalled. Default to 5000 if not specified.
  // this serves both the API and the client.
  // It is the only port that is not firewalled.
  const port = parseInt(process.env.PORT || "5000", 10);
  httpServer.listen(
    {
      port,
      host: "0.0.0.0",
      reusePort: true,
    },
    () => {
      log(`serving on port ${port}`);
      startRemindersScheduler();
      checkAndSwitchSemester();
      setInterval(checkAndSwitchSemester, 60 * 60 * 1000);

      async function runFileMonitor() {
        try {
          const resp = await fetch(`http://localhost:${port}/api/files/monitor-sync`, { method: 'POST', headers: { 'Content-Type': 'application/json' } });
          const data = await resp.json();
          if (data.totalSynced > 0) {
            console.log(`[File Monitor] Synced ${data.totalSynced} new Spring/Summer files`);
          }
        } catch (e: any) {
          console.error("[File Monitor] Error:", e.message);
        }
      }

      setTimeout(runFileMonitor, 10000);
      setInterval(runFileMonitor, 5 * 60 * 1000);

      async function ensureOneDrivePlaceholders() {
        try {
          const resp = await fetch(`http://localhost:${port}/api/onedrive/ensure-placeholder-folders`, { method: 'POST' });
          const data = await resp.json();
          const placeholders = data.placeholders || [];
          if (placeholders.length > 0) {
            console.log(`[OneDrive] Created ${placeholders.length} placeholder folders: ${placeholders.join(', ')}`);
          } else {
            console.log(`[OneDrive] All year/semester placeholder folders already exist`);
          }
        } catch (e: any) {
          console.error("[OneDrive] Error ensuring placeholder folders:", e.message);
        }
      }

      setTimeout(ensureOneDrivePlaceholders, 15000);
    },
  );
})();

```

`server/onedrive.ts`

```

// OneDrive integration for browsing and accessing PDF files
import { Client } from '@microsoft/microsoft-graph-client';

let connectionSettings: any;

async function getAccessToken() {
  if (connectionSettings && connectionSettings.settings.expires_at && new Date(connectionSettings.settings.expires_at...) < new Date()) {
    return connectionSettings.settings.access_token;
  }

  const hostname = process.env.REPLIT_CONNECTORS_HOSTNAME;
  const xReplitToken = process.env.REPL_IDENTITY
    ? 'repl ' + process.env.REPL_IDENTITY
    : process.env.WEB_REPL_RENEWAL
    ? 'depl ' + process.env.WEB_REPL_RENEWAL
    : null;

  if (!xReplitToken) {
    throw new Error('X_REPLIT_TOKEN not found for repl/depl');
  }

  connectionSettings = await fetch(
    'https://' + hostname + '/api/v2/connection?include_secrets=true&connector_names=onedrive',
    {
      headers: {
        'X-Replit-Token': xReplitToken,
      },
    },
  ).then((res) => res.json());
}

```

```

    headers: {
      'Accept': 'application/json',
      'X_REPLIT_TOKEN': xReplitToken
    }
  }).then(res => res.json()).then(data => data.items?.[0]);

const accessToken = connectionSettings?.settings?.access_token || connectionSettings?.settings?.oauth?.credentials?...

if (!connectionSettings || !accessToken) {
  throw new Error('OneDrive not connected');
}
return accessToken;
}

// WARNING: Never cache this client.
// Access tokens expire, so a new client must be created each time.
export async function getOneDriveClient() {
  const accessToken = await getAccessToken();

  return Client.initWithMiddleware({
    authProvider: {
      getAccessToken: async () => accessToken
    }
  });
}

// List files/folders in a directory
export async function listOneDriveItems(path: string = '/') {
  const client = await getOneDriveClient();

  try {
    let response;
    if (path === '/' || path === '') {
      // List root folder contents
      response = await client.api('/me/drive/root/children').get();
    } else {
      // List specific folder contents
      response = await client.api(`/me/drive/root:${path}/children`).get();
    }

    return response.value.map((item: any) => ({
      id: item.id,
      name: item.name,
      type: item.folder ? 'folder' : 'file',
      mimeType: item.file?.mimeType,
      size: item.size,
      lastModified: item.lastModifiedDate,
      downloadUrl: item['@microsoft.graph.downloadUrl'],
      path: item.parentReference?.path ?
        item.parentReference.path.replace('/drive/root:', '') + '/' + item.name :
        '/' + item.name
    })));
  } catch (error: any) {
    console.error('Error listing OneDrive items:', error);
    throw error;
  }
}

// Get file content/download URL
export async function getOneDriveFile(itemId: string) {
  const client = await getOneDriveClient();

  try {
    const item = await client.api(`/me/drive/items/${itemId}`).get();
    return {
      id: item.id,
      name: item.name,
      mimeType: item.file?.mimeType,
      size: item.size,
      downloadUrl: item['@microsoft.graph.downloadUrl'],
      webUrl: item.webUrl
    };
  } catch (error: any) {
    console.error('Error getting OneDrive file:', error);
    throw error;
  }
}

// Get file content as stream
export async function getOneDriveFileContent(itemId: string) {
  const client = await getOneDriveClient();

  try {
    const content = await client.api(`/me/drive/items/${itemId}/content`).get();
    return content;
  } catch (error: any) {
    console.error('Error getting OneDrive file content:', error);
    throw error;
  }
}

export async function createOneDriveFolder(parentPath: string, folderName: string): Promise<any> {
  const client = await getOneDriveClient();
  try {
    const apiPath = (!parentPath || parentPath === '/')
      ? '/me/drive/root/children'
      : `/me/drive/root:${parentPath}/children`;
    const response = await client.api(apiPath).post({
      name: folderName,
      folder: {},
      '@microsoft.graph.conflictBehavior': 'fail'
    });
    return { id: response.id, name: response.name, created: true };
  } catch (error: any) {
    if (error.statusCode === 409 || error.code === 'nameAlreadyExists') {
      return { name: folderName, created: false, exists: true };
    }
  }
}

```

```

        console.error(`Error creating folder "${folderName}" in "${parentPath}":`, error.message || error);
        throw error;
    }
}

export async function renameOneDriveFolder(folderPath: string, newName: string): Promise<{ renamed: boolean; error?: ...
const client = await getOneDriveClient();
try {
    const encodedPath = encodeURIComponent(folderPath).replace(/%2F/g, '/');
    await client.api(`/me/drive/root:${encodedPath}`).patch({ name: newName });
    return { renamed: true };
} catch (error: any) {
    if (error.statusCode === 404) {
        return { renamed: false, error: 'Folder not found' };
    }
    console.error(`Error renaming folder "${folderPath}" to "${newName}":`, error.message || error);
    return { renamed: false, error: error.message };
}
}

export async function checkOneDriveFolderExists(folderPath: string): Promise<boolean> {
const client = await getOneDriveClient();
try {
    const encodedPath = encodeURIComponent(folderPath).replace(/%2F/g, '/');
    await client.api(`/me/drive/root:${encodedPath}`).get();
    return true;
} catch {
    return false;
}
}

export async function listOneDriveFolderChildren(folderPath: string): Promise<any[]> {
const client = await getOneDriveClient();
try {
    const encodedPath = encodeURIComponent(folderPath).replace(/%2F/g, '/');
    const response = await client.api(`/me/drive/root:${encodedPath}/children`).get();
    return (response.value || []).map((item: any) => ({
        id: item.id,
        name: item.name,
        folder: !!item.folder,
    }));
} catch (error: any) {
    console.error(`Error listing children of "${folderPath}":`, error.message || error);
    return [];
}
}

export async function renameOneDriveItem(itemId: string, newName: string): Promise<void> {
const client = await getOneDriveClient();
await client.api(`/me/drive/items/${itemId}`).patch({ name: newName });
}

export async function getOneDriveFileContentAsText(itemId: string): Promise<string> {
const client = await getOneDriveClient();
try {
    const response = await client.api(`/me/drive/items/${itemId}/content`).get();
    if (typeof response === 'string') return response;
    if (response instanceof ArrayBuffer || response instanceof Buffer) {
        return Buffer.from(response).toString('utf-8');
    }
    if (response && typeof response.text === 'function') {
        return await response.text();
    }
    if (response && response.body && typeof response.body.read === 'function') {
        const chunks: Buffer[] = [];
        for await (const chunk of response.body) {
            chunks.push(Buffer.from(chunk));
        }
        return Buffer.concat(chunks).toString('utf-8');
    }
    return String(response);
} catch (error: any) {
    console.error('Error getting OneDrive file content as text:', error.message || error);
    throw error;
}
}

export async function getOneDriveItemByPath(path: string): Promise<any> {
const client = await getOneDriveClient();
try {
    const encodedPath = encodeURIComponent(path).replace(/%2F/g, '/');
    const item = await client.api(`/me/drive/root:${encodedPath}`).get();
    return {
        id: item.id,
        name: item.name,
        mimeType: item.file?.mimeType,
        size: item.size,
        lastModified: item.lastModifiedDate,
        downloadUrl: item['@microsoft.graph.downloadUrl'],
        webUrl: item.webUrl,
    };
} catch (error: any) {
    console.error(`Error getting OneDrive item at path "${path}":`, error.message || error);
    throw error;
}
}

export async function createOneDriveTextFile(parentPath: string, fileName: string, content: string): Promise<any> {
const client = await getOneDriveClient();
try {
    const encodedPath = encodeURIComponent(`${parentPath}/${fileName}`).replace(/%2F/g, '/');
    const response = await client.api(`/me/drive/root:${encodedPath}/content`)
        .header('Content-Type', 'text/plain')
        .put(content);
    return {
        id: response.id,
        name: response.name,
        lastModified: response.lastModifiedDate,
    };
}

```

```

        webUrl: response.webUrl,
    };
} catch (error: any) {
    console.error('Error creating text file "${fileName}":', error.message || error);
    throw error;
}
}

export async function uploadOneDriveFile(parentPath: string, fileName: string, fileBuffer: Buffer, contentType: string) {
    const accessToken = await getAccessToken();
    try {
        const fullPath = `${parentPath}/${fileName}`;
        const encodedPath = encodeURIComponent(fullPath).replace(/%2F/g, '/');
        const url = `https://graph.microsoft.com/v1.0/me/drive/root:${encodedPath}:content`;
        const response = await fetch(url, {
            method: 'PUT',
            headers: {
                'Authorization': `Bearer ${accessToken}`,
                'Content-Type': contentType,
            },
            body: fileBuffer,
        });
    } catch (error: any) {
        console.error('Error uploading file "${fileName}" to "${parentPath}":', error.message || error);
        throw error;
    }
}

export async function updateOneDriveFileContent(itemId: string, content: string): Promise<any> {
    const client = await getOneDriveClient();
    try {
        const response = await client.api(`/me/drive/items/${itemId}/content`)
            .header('Content-Type', 'text/plain')
            .put(content);
        return {
            id: response.id,
            name: response.name,
            lastModified: response.lastModifiedDateTime,
            webUrl: response.webUrl,
        };
    } catch (error: any) {
        console.error('Error updating file content:', error.message || error);
        throw error;
    }
}

export async function deleteOneDriveItem(itemId: string): Promise<void> {
    const client = await getOneDriveClient();
    try {
        await client.api(`/me/drive/items/${itemId}`).delete();
    } catch (error: any) {
        console.error('Error deleting OneDrive item:', error.message || error);
        throw error;
    }
}

// Search for files
export async function searchOneDriveFiles(query: string) {
    const client = await getOneDriveClient();

    try {
        const response = await client.api(`/me/drive/root/search(q='${query}')`).get();

        return response.value.map((item: any) => ({
            id: item.id,
            name: item.name,
            type: item.folder ? 'folder' : 'file',
            mimeType: item.file?.mimeType,
            size: item.size,
            lastModified: item.lastModifiedDateTime,
            downloadUrl: item['@microsoft.graph.downloadUrl'],
            path: item.parentReference?.path ?
                item.parentReference.path.replace('/drive/root:', '') + '/' + item.name :
                '/' + item.name
        }));
    } catch (error: any) {
        console.error('Error searching OneDrive:', error);
        throw error;
    }
}

function extractUTF16Strings(data: Uint8Array, minLength = 3) {
    const results: { text: string; pos: number }[] = [];
    let current = '';
    let startPos = -1;

    for (let i = 0; i < data.length - 1; i += 2) {
        const ch = data[i] | (data[i + 1] << 8);
        if (ch >= 32 && ch < 127) {
            if (current === '') startPos = i;
            current += String.fromCharCode(ch);
        } else if (ch === 10 || ch === 13) {
            if (current === '') startPos = i;
            current += '\n';
        } else {
            if (current.trim().length >= minLength) {
                results.push({ text: current, pos: startPos });
                current = '';
                startPos = -1;
            }
        }
    }
    if (current.trim().length >= minLength) {
        results.push({ text: current, pos: startPos });
    }
}

```

```

        results.push({ text: current.trim(), pos: startPos });
    }
    current = '';
    startPos = -1;
}
}
if (current.trim().length >= minLength) {
    results.push({ text: current.trim(), pos: startPos });
}
return results;
}

function isMetadata(text: string): boolean {
    const metaPatterns = [
        /^Bryn Kai-Hendricks$/,
        /^PageTitle$/,
        /^PageDateTime$/,
        /^Calibri/,
        /^var\(--/,
        /^<resolutionId/,
        /^IBM Plex/,
        /^ ÅBöA
    ];
    return metaPatterns.some(p => p.test(text));
}

export interface OneNotePage {
    title: string;
    content: string;
    position: number;
}

export async function getOneNotePages(notebookPath: string, sectionFileName: string): Promise<OneNotePage[]> {
    const client = await getOneDriveClient();

    const itemPath = `${notebookPath}/${sectionFileName}`;
    const meta = await client.api(`me/drive/root:${encodeURIComponent(itemPath)}`).get();
    const downloadUrl = meta['@microsoft.graph.downloadUrl'];
    if (!downloadUrl) throw new Error('No download URL for .one file');

    const response = await fetch(downloadUrl);
    const buf = await response.arrayBuffer();
    const bytes = new Uint8Array(buf);

    const strings = extractUTF16Strings(bytes, 3);

    const SKIP_PATTERNS = [
        /^Calibri/i, /^PageDateTime/i, /^PageTitle$/i,
        /^Bryn Kai-Hendricks$/i, /^Bryn$/i,
        /^Times New Roman$/i, /^Arial$/i, /^Segoe/i, /^Consolas$/i,
        /^var\(--font/i, /^<resolutionId/,
        /^en-US$/, /^en-CAS$/,
        /^d{1,2}:d{2}$/,
        /^Microsoft$/i, /^OneNote$/i,
        /^Windows Live$/i,
    ];

    function isSkippable(text: string): boolean {
        if (isMetadata(text)) return true;
        for (const p of SKIP_PATTERNS) {
            if (p.test(text)) return true;
        }
        const asciiCount = (text.match(/[x20-x7E]/g) || []).length;
        if (asciiCount <= text.length * 0.7) return true;
        return false;
    }

    const pageTitlePositions: number[] = [];
    for (let i = 0; i < bytes.length - 18; i++) {
        const target = 'PageTitle';
        let match = true;
        for (let j = 0; j < target.length; j++) {
            if (bytes[i + j * 2] !== target.charCodeAt(j) || bytes[i + j * 2 + 1] !== 0) {
                match = false;
                break;
            }
        }
        if (match) pageTitlePositions.push(i);
    }

    const contentStrings = strings.filter(s => !isSkippable(s.text) && s.text.length >= 3);

    if (pageTitlePositions.length === 0) {
        return [{
            title: sectionFileName.replace(/\.one$/i, ''),
            content: contentStrings.map(s => s.text).join('\n\n'),
            position: 0
        }];
    }

    const pages: OneNotePage[] = [];
    const seenTitles = new Map<string, number>();

    function isTitleLike(text: string): boolean {
        if (text.length < 2 || text.length > 80) return false;
        if (text.startsWith('- ') || text.startsWith('')) return false;
        if (text.startsWith('HYPERLINK') || text.startsWith('http')) return false;
        if (text.includes('\n')) return false;
        if (/^d+\./s/.test(text)) return false;
        if (text.includes('Weakness') || text.includes('Opportunity') || text.includes('Strength') || text.includes('...')) return false;
        if (text.startsWith('The ') || text.startsWith('Add ') || text.startsWith('Build ')) return false;
        const wordCount = text.split(/\s+/).length;
        return wordCount <= 6;
    }

    for (let p = 0; p < pageTitlePositions.length; p++) {
        const markerPos = pageTitlePositions[p];
        const nextMarkerPos = p + 1 < pageTitlePositions.length

```

```

    ? pageTitlePositions[p + 1]
    : bytes.length;

const pageContent = contentStrings.filter(
  s => s.pos > markerPos && s.pos < nextMarkerPos
);

if (pageContent.length === 0) continue;

const subPages: { title: string; segments: typeof pageContent; pos: number }[] = [];
let currentTitle = pageContent[0].text.split('\n')[0].substring(0, 100);
let currentSegments: typeof pageContent = [];
let currentPos = pageContent[0].pos;

for (let i = 1; i < pageContent.length; i++) {
  const s = pageContent[i];
  const gap = i > 0 ? s.pos - pageContent[i - 1].pos : 0;

  if (gap > 10000 && isTitleLike(s.text)) {
    subPages.push({ title: currentTitle, segments: currentSegments, pos: currentPos });
    currentTitle = s.text.split('\n')[0].substring(0, 100);
    currentSegments = [];
    currentPos = s.pos;
  } else {
    currentSegments.push(s);
  }
}
subPages.push({ title: currentTitle, segments: currentSegments, pos: currentPos });

for (const sp of subPages) {
  const content = sp.segments
    .map(s => {
      let t = s.text;
      t = t.replace(/^HYPERLINK\s+"([^\s"]+)"\s*/, (_, url) => `[${url}] `);
      return t;
    })
    .join('\n\n');

  const existingIdx = seenTitles.get(sp.title);
  if (existingIdx !== undefined) {
    if (content.length > pages[existingIdx].content.length) {
      pages[existingIdx] = { title: sp.title, content, position: sp.pos };
    }
  } else {
    seenTitles.set(sp.title, pages.length);
    pages.push({ title: sp.title, content, position: sp.pos });
  }
}
}

return pages;
}

export async function getOneNotePagesViaApi(notebookDisplayName: string, sectionName: string): Promise<OneNotePage[]> {
  const client = await getOneDriveClient();

  try {
    const nbRes = await client.api(`/me/onenote/notebooks`).select('id,displayName').get();
    const nb = (nbRes.value || []).find((n: any) => n.displayName === notebookDisplayName);
    if (!nb) {
      console.log(`[OneNote API] Notebook "${notebookDisplayName}" not found`);
      return [];
    }

    const secRes = await client.api(`/me/onenote/notebooks/${nb.id}/sections`).select('id,displayName').get();
    const sec = (secRes.value || []).find((s: any) => s.displayName === sectionName);
    if (!sec) {
      console.log(`[OneNote API] Section "${sectionName}" not found in "${notebookDisplayName}"`);
      return [];
    }

    const pagesRes = await client.api(`/me/onenote/sections/${sec.id}/pages`)
      .select('id,title,createdDateTime,lastModifiedDateTime')
      .orderBy('createdDateTime desc')
      .top(200)
      .get();

    const pages: OneNotePage[] = [];
    for (const p of (pagesRes.value || [])) {
      let content = '';
      try {
        const contentRes = await client.api(`/me/onenote/pages/${p.id}/content`).get();
        const html = typeof contentRes === 'string' ? contentRes : await contentRes.text?(). || '';
        content = html
          .replace(/<style[^>]*>[\s\S]*?</style>/gi, '')
          .replace(/<[^>+>/g, ' ')
          .replace(/&nbsp;/g, ' ')
          .replace(/&amp;/g, '&')
          .replace(/&lt;/g, '<')
          .replace(/&gt;/g, '>')
          .replace(/&#d+;/g, '')
          .replace(/&s+/g, ' ')
          .trim();
      } catch (e: any) {
        console.log(`[OneNote API] Could not fetch content for "${p.title}": ${e.message}`);
      }
      pages.push({
        title: p.title || 'Untitled Page',
        content,
        position: pages.length,
      });
    }

    console.log(`[OneNote API] Found ${pages.length} pages in ${notebookDisplayName}/${sectionName}`);
    return pages;
  } catch (err: any) {
    console.error(`[OneNote API] Error listing pages:`, err.message || err);
    return [];
  }
}

```

```

    }
  }
}

export async function createOneNotePage(notebookDisplayName: string, sectionName: string, title: string, content: str...
const client = await getOneDriveClient();

try {
  const nbRes = await client.api(`/me/onenote/notebooks`).select('id,displayName').get();
  const nb = (nbRes.value || []).find((n: any) => n.displayName === notebookDisplayName);
  if (!nb) {
    throw new Error(`Notebook "${notebookDisplayName}" not found`);
  }

  const secRes = await client.api(`/me/onenote/notebooks/${nb.id}/sections`).select('id,displayName').get();
  const sec = (secRes.value || []).find((s: any) => s.displayName === sectionName);
  if (!sec) {
    throw new Error(`Section "${sectionName}" not found in "${notebookDisplayName}"`);
  }

  const htmlContent = `
<!DOCTYPE html>
<html>
<head><title>${title.replace(/</g, '&lt;').replace(/>/g, '&gt;')}</title></head>
<body>
${content.split('\n').map(line => `<p>${line.replace(/</g, '&lt;').replace(/>/g, '&gt;')}</p>`).join('\n')}
</body>
</html>`;

  const page = await client.api(`/me/onenote/sections/${sec.id}/pages`)
    .header('Content-Type', 'text/html')
    .post(htmlContent);

  console.log(`[OneNote API] Created page "${title}" in ${notebookDisplayName}/${sectionName}`);
  return { id: page.id, title: page.title || title };
} catch (err: any) {
  console.error(`[OneNote API] Error creating page:`, err.message || err);
  throw err;
}
}

export async function deleteOneNotePage(notebookDisplayName: string, sectionName: string, pageTitle: string): Promise...
const client = await getOneDriveClient();

try {
  const nbRes = await client.api(`/me/onenote/notebooks`).select('id,displayName').get();
  const nb = (nbRes.value || []).find((n: any) =>
    n.displayName === notebookDisplayName
  );
  if (!nb) {
    console.error(`[OneNote] Notebook "${notebookDisplayName}" not found in OneNote API`);
    return false;
  }

  const secRes = await client.api(`/me/onenote/notebooks/${nb.id}/sections`).select('id,displayName').get();
  const sec = (secRes.value || []).find((s: any) =>
    s.displayName === sectionName
  );
  if (!sec) {
    console.error(`[OneNote] Section "${sectionName}" not found in notebook "${notebookDisplayName}"`);
    return false;
  }

  const pagesRes = await client.api(`/me/onenote/sections/${sec.id}/pages`).select('id,title').top(100).get();
  const page = (pagesRes.value || []).find((p: any) =>
    p.title === pageTitle
  );
  if (!page) {
    console.error(`[OneNote] Page "${pageTitle}" not found in section "${sectionName}"`);
    return false;
  }

  await client.api(`/me/onenote/pages/${page.id}`).delete();
  console.log(`[OneNote] Deleted page "${pageTitle}" from ${notebookDisplayName}/${sectionName}`);
  return true;
} catch (err: any) {
  console.error(`[OneNote] Delete page error:`, err.message || err);
  throw err;
}
}

export async function listOneNoteNotebooks(): Promise<{ name: string; path: string; sections: { name: string; id: str...
const client = await getOneDriveClient();
const notebooks: { name: string; path: string; sections: { name: string; id: string }[] }[] = [];

const knownPaths = [
  { name: "Bryn's Notebook", path: "/Documents/Bryn's Notebook" },
  { name: "Notebook", path: "/School/1. TMU/Notebook" },
];

for (const nb of knownPaths) {
  try {
    const res = await client.api(`/me/drive/root:${encodeURIComponent(nb.path)}/children`).get();
    const sections = (res.value || []).
      .filter((item: any) => item.name.endsWith('.one'))
      .map((item: any) => ({ name: item.name.replace(/\.one$/i, ''), id: item.id }));

    if (sections.length > 0) {
      notebooks.push({ name: nb.name, path: nb.path, sections });
    }
  } catch (e) {
    console.error(`[OneNote] Failed to list sections for ${nb.name}:`, e);
  }
}

return notebooks;
}

```

server/outlookCalendar.ts


```

import { Client } from '@microsoft/microsoft-graph-client';
import { storage } from './storage';

// Outlook connector - uses dedicated Outlook integration with Calendar permissions
let outlookConnectionSettings: any;

async function getOutlookAccessToken() {
  if (outlookConnectionSettings && outlookConnectionSettings.settings.expires_at && new Date(outlookConnectionSetting...
    return outlookConnectionSettings.settings.access_token;
  }

  const hostname = process.env.REPLIT_CONNECTORS_HOSTNAME;
  const xReplitToken = process.env.REPL_IDENTITY
    ? 'repl ' + process.env.REPL_IDENTITY
    : process.env.WEB_REPL_RENEWAL
    ? 'depl ' + process.env.WEB_REPL_RENEWAL
    : null;

  if (!xReplitToken) {
    throw new Error('X-Replit-Token not found for repl/depl');
  }

  outlookConnectionSettings = await fetch(
    'https://' + hostname + '/api/v2/connection?include_secrets=true&connector_names=outlook',
    {
      headers: {
        'Accept': 'application/json',
        'X-Replit-Token': xReplitToken
      }
    }
  ).then(res => res.json()).then(data => data.items?.[0]);

  const accessToken = outlookConnectionSettings?.settings?.access_token || outlookConnectionSettings?.settings?.oauth...

  if (!outlookConnectionSettings || !accessToken) {
    throw new Error('Outlook not connected');
  }
  return accessToken;
}

async function getOutlookClient() {
  const accessToken = await getOutlookAccessToken();
  return Client.initWithMiddleware({
    authProvider: {
      getAccessToken: async () => accessToken
    }
  });
}

export interface OutlookEvent {
  id: string;
  subject: string;
  bodyPreview: string;
  start: { dateTime: string; timeZone: string };
  end: { dateTime: string; timeZone: string };
  location?: { displayName?: string };
  organizer?: { emailAddress?: { name?: string; address?: string } };
  isAllDay?: boolean;
  isCancelled?: boolean;
}

export async function fetchOutlookCalendarEvents(daysAhead: number = 14): Promise<OutlookEvent[]> {
  const client = await getOutlookClient();

  const now = new Date();
  const startDateTime = now.toISOString();
  const end = new Date(now);
  end.setDate(end.getDate() + daysAhead);
  const endDateTime = end.toISOString();

  try {
    const calendarsResponse = await client.api('/me/calendars').select('id,name').get();
    const calendars = calendarsResponse.value || [];
    console.log('[Outlook Calendar] Found ${calendars.length} calendars: ${calendars.map((c: any) => c.name).join(', ...

    const allEvents: OutlookEvent[] = [];

    for (const cal of calendars) {
      try {
        const response = await client
          .api(`/me/calendars/${cal.id}/calendarview`)
          .query({
            startDateTime,
            endDateTime,
            $orderby: 'start/dateTime',
            $top: 100,
            $select: 'id,subject,bodyPreview,start,end,location,organizer,isAllDay,isCancelled',
          })
          .get();

        const events = (response.value || []).filter((ev: any) => !ev.isCancelled);
        console.log('[Outlook Calendar] "${cal.name}": ${events.length} events');
        allEvents.push(...events);
      } catch (calError: any) {
        console.error('[Outlook Calendar] Error fetching from "${cal.name}":`, calError.message || calError);
      }
    }

    allEvents.sort((a, b) => new Date(a.start.dateTime).getTime() - new Date(b.start.dateTime).getTime());
    return allEvents;
  } catch (error: any) {
    console.error('[Outlook Calendar] Error fetching events:', error.message || error);
    throw error;
  }
}

export async function syncOutlookEventsToReview(): Promise<{ added: number; skipped: number }> {

```

```

const events = await fetchOutlookCalendarEvents(30);
let added = 0;
let skipped = 0;

const allReviewItems = await storage.getPendingReviewItems();
const normalizeTitle = (t: string) => t.toLowerCase().replace(/\s+/g, ' ').replace(/^[a-z0-9- ]/g, '').trim();
const existingReviewKeys = new Set(allReviewItems.map(i => {
  const normT = normalizeTitle(i.title || '');
  const dateK = i.startDate ? new Date(i.startDate).toISOString().split('T')[0] : 'nodate';
  return `${normT}||${dateK}`;
})));

const allTasks = await storage.getTasks();
const existingTaskKeys = new Set(allTasks.map(t => {
  const normT = normalizeTitle(t.title || '');
  const dateK = t.dueDate ? new Date(t.dueDate).toISOString().split('T')[0] : 'nodate';
  return `${normT}||${dateK}`;
})));

for (const event of events) {
  const existing = await storage.getPendingReviewItemByExternalId(event.id, 'outlook_calendar');
  if (existing) {
    skipped++;
    continue;
  }

  const startDt = new Date(event.start.dateTime + (event.start.timeZone === 'UTC' ? 'Z' : ''));
  const endDt = new Date(event.end.dateTime + (event.end.timeZone === 'UTC' ? 'Z' : ''));

  const normTitle = normalizeTitle(event.subject || 'untitled event');
  const dateKey = startDt.toISOString().split('T')[0];
  const titleDateKey = `${normTitle}||${dateKey}`;
  if (existingReviewKeys.has(titleDateKey)) {
    skipped++;
    continue;
  }
  if (existingTaskKeys.has(titleDateKey)) {
    skipped++;
    continue;
  }
  existingReviewKeys.add(titleDateKey);

  const startTime = event.isAllDay ? null : `${String(startDt.getHours()).padStart(2, '0')}:${String(startDt.getMin...
  const endTime = event.isAllDay ? null : `${String(endDt.getHours()).padStart(2, '0')}:${String(endDt.getMinutes)...

  await storage.createPendingReviewItem({
    source: 'outlook_calendar',
    sourceEmail: event.organizer?.emailAddress?.address || null,
    externalId: event.id,
    title: event.subject || 'Untitled Event',
    description: event.bodyPreview || null,
    startDate: startDt,
    endDate: endDt,
    eventStartTime: startTime,
    eventEndTime: endTime,
    location: event.location?.displayName || null,
    rawData: JSON.stringify(event),
    status: 'pending',
    courseName: null,
    taskType: 'meeting',
  });
  added++;
}

console.log(`[Outlook Calendar] Synced: ${added} added, ${skipped} skipped (already exists)`);
return { added, skipped };
}

export async function findOrCreateMailFolder(folderName: string): Promise<string> {
  const client = await getOutlookClient();
  const folders = await client.api('/me/mailFolders').top(100).get();
  const existing = folders.value?.find((f: any) => f.displayName === folderName);
  if (existing) {
    console.log(`[Outlook Mail] Found folder "${folderName}" (${existing.id})`);
    return existing.id;
  }
  try {
    const created = await client.api('/me/mailFolders').post({ displayName: folderName });
    console.log(`[Outlook Mail] Created folder "${folderName}" (${created.id})`);
    return created.id;
  } catch (e: any) {
    if (e.message?.includes('already exists')) {
      const retryFolders = await client.api('/me/mailFolders').filter('displayName eq "${folderName}"').get();
      if (retryFolders.value?.[0]) {
        return retryFolders.value[0].id;
      }
    }
    throw e;
  }
}

export async function createMailRule(ruleName: string, senderDomain: string, folderId: string): Promise<any> {
  const client = await getOutlookClient();

  const existingRules = await client.api('/me/mailFolders/inbox/messageRules').get();
  const existing = existingRules.value?.find((r: any) => r.displayName === ruleName);
  if (existing) {
    console.log(`[Outlook Mail] Rule "${ruleName}" already exists`);
    return existing;
  }

  const rule = await client.api('/me/mailFolders/inbox/messageRules').post({
    displayName: ruleName,
    sequence: 1,
    isEnabled: true,
    conditions: {
      senderContains: [senderDomain]
    },
  },

```

```

    actions: {
      moveToFolder: folderId,
      stopProcessingRules: false
    }
  });

  console.log(`[Outlook Mail] Created rule "${ruleName}" !' folder ${folderId}`);
  return rule;
}

export async function moveExistingEmailsToFolder(senderDomain: string, folderId: string): Promise<number> {
  const client = await getOutlookClient();
  let moved = 0;
  let nextLink: string | null = null;

  const searchUrl = `/me/messages?$filter=contains(from/emailAddress/address,'${senderDomain}')&$top=50&$select=id,su...
  let response = await client.api(searchUrl).get();

  while (true) {
    for (const msg of (response.value || [])) {
      try {
        await client.api(`/me/messages/${msg.id}/move`).post({ destinationId: folderId });
        moved++;
      } catch (e: any) {
        console.log(`[Outlook Mail] Failed to move "${msg.subject}": ${e.message}`);
      }
    }

    nextLink = response['@odata.nextLink'];
    if (!nextLink) break;
    response = await client.api(nextLink).get();
  }

  console.log(`[Outlook Mail] Moved ${moved} existing emails from *${senderDomain} !' folder`);
  return moved;
}

export async function moveAllEmailsFromFolder(sourceFolderId: string, destFolderId: string): Promise<number> {
  const client = await getOutlookClient();
  let moved = 0;

  let response = await client.api(`/me/mailFolders/${sourceFolderId}/messages`).top(50).select('id,subject').get();

  while (true) {
    for (const msg of (response.value || [])) {
      try {
        await client.api(`/me/messages/${msg.id}/move`).post({ destinationId: destFolderId });
        moved++;
      } catch (e: any) {
        console.log(`[Outlook Mail] Failed to move "${msg.subject}": ${e.message}`);
      }
    }

    const nextLink = response['@odata.nextLink'];
    if (!nextLink) break;
    response = await client.api(nextLink).get();
  }

  return moved;
}

export async function deleteMailRulesByName(nameContains: string): Promise<number> {
  const client = await getOutlookClient();
  const rules = await client.api('/me/mailFolders/inbox/messageRules').get();
  let deleted = 0;

  for (const rule of (rules.value || [])) {
    if (rule.displayName && rule.displayName.includes(nameContains)) {
      await client.api(`/me/mailFolders/inbox/messageRules/${rule.id}`).delete();
      console.log(`[Outlook Mail] Deleted rule "${rule.displayName}"`);
      deleted++;
    }
  }
  return deleted;
}

export async function moveEmailsNotFromDomains(sourceFolderId: string, destFolderId: string, keepDomains: string[]): ...
const client = await getOutlookClient();
let moved = 0;
let kept = 0;

let response = await client.api(`/me/mailFolders/${sourceFolderId}/messages`).top(50).select('id,subject,from').get();

while (true) {
  for (const msg of (response.value || [])) {
    const senderEmail = (msg.from?.emailAddress?.address || '').toLowerCase();
    const belongsHere = keepDomains.some(d => senderEmail.includes(d.toLowerCase()));

    if (belongsHere) {
      kept++;
    } else {
      try {
        await client.api(`/me/messages/${msg.id}/move`).post({ destinationId: destFolderId });
        moved++;
      } catch (e: any) {
        console.log(`[Outlook Mail] Failed to move back "${msg.subject}": ${e.message}`);
      }
    }
  }

  const nextLink = response['@odata.nextLink'];
  if (!nextLink) break;
  response = await client.api(nextLink).get();
}

console.log(`[Outlook Mail] Sorted folder: kept ${kept} (matching domains), moved ${moved} back`);
return { moved, kept };
}

```

```

export async function getMailFolderId(folderName: string): Promise<string | null> {
  outlookConnectionSettings = null;
  const client = await getOutlookClient();
  try {
    const filtered = await client.api('/me/mailFolders').filter(`displayName eq '${folderName}'`).get();
    if (filtered.value?.[0]) return filtered.value[0].id;
  } catch (e) {}
  const folders = await client.api('/me/mailFolders').top(100).get();
  const found = folders.value?.find((f: any) => f.displayName === folderName);
  return found?.id || null;
}

```

`server/reminderScheduler.ts`

```

import { storage } from "../storage";
import { sendTaskReminder, sendHaTaskReminder, sendEchoVoiceAnnouncement, sendDailyDigest, type TaskReminder } from "...
import { getIsTravellingMode } from "../routes";
import { db } from "../db";
import { appState } from "@shared/schema";
import { eq } from "drizzle-orm";
import { syncOutlookEventsToReview } from "../outlookCalendar";
import { easternNow, easternDateStr, easternHour, easternMidnight, taskDateStr, addDays } from "../timezone";

const getEasternNow = easternNow;
const getEasternDateStr = easternDateStr;
const getEasternHour = easternHour;
const getEasternMidnight = easternMidnight;
const addDaysET = addDays;

const sentReminders = new Set<string>();

const DAILY_DIGEST_HOUR = 7;
const OUTLOOK_SYNC_HOUR = 8;
let schedulerRunning = false;
let lastCheckTime: Date | null = null;
let remindersSentCount = 0;
let sentRemindersLoaded = false;

function getReminderKey(taskId: number, reminderMinutes: number): string {
  return `${taskId}-${reminderMinutes}`;
}

async function getAppState(key: string): Promise<string | null> {
  try {
    const rows = await db.select().from(appState).where(eq(appState.key, key));
    return rows.length > 0 ? rows[0].value : null;
  } catch {
    return null;
  }
}

async function setAppState(key: string, value: string): Promise<void> {
  try {
    await db.insert(appState)
      .values({ key, value, updatedAt: new Date() })
      .onConflictDoUpdate({
        target: appState.key,
        set: { value, updatedAt: new Date() },
      });
  } catch (err) {
    console.error(`[Reminder] Failed to persist app state for key "${key}":`, err);
  }
}

async function loadSentReminders(): Promise<void> {
  if (sentRemindersLoaded) return;
  try {
    const stored = await getAppState("sent_reminders");
    if (stored) {
      const keys: string[] = JSON.parse(stored);
      for (const k of keys) {
        sentReminders.add(k);
      }
      console.log(`[Reminder] Loaded ${keys.length} sent reminder keys from database`);
    }
    sentRemindersLoaded = true;
  } catch (err) {
    console.error("[Reminder] Failed to load sent reminders:", err);
    sentRemindersLoaded = true;
  }
}

async function persistSentReminders(): Promise<void> {
  const keys = Array.from(sentReminders);
  await setAppState("sent_reminders", JSON.stringify(keys));
}

export function getSchedularStatus() {
  return {
    running: schedulerRunning,
    lastCheck: lastCheckTime?.toISOString() || null,
    remindersSent: remindersSentCount,
    trackedReminders: sentReminders.size,
  };
}

let announcementsSentThisCycle = 0;
const MAX_ANNOUNCEMENTS_PER_CYCLE = 3;
let lastAnnouncementTime = 0;
const MIN_ANNOUNCEMENT_GAP_MS = 10000;

async function digestAlreadySentToday(): Promise<boolean> {
  const todayStr = getEasternDateStr(new Date());

```

```

const lastDigestDate = await getAppState("last_digest_date");
return lastDigestDate === todayStr;
}

export async function checkReminders() {
  try {
    await loadSentReminders();
    const allTasks = await storage.getTasks({ showCompleted: false });
    const now = getEasternNow();
    lastCheckTime = now;
    let stateChanged = false;
    announcementsSentThisCycle = 0;

    for (const task of allTasks) {
      if (task.isCompleted || !task.dueDate) continue;

      const dueDate = new Date(task.dueDate);
      const reminderFields = [
        task.reminder1,
        task.reminder2,
        task.reminder3,
        task.reminder4,
      ];

      for (const reminderMinutes of reminderFields) {
        if (reminderMinutes == null) continue;

        const reminderTime = new Date(dueDate.getTime() - reminderMinutes * 60 * 1000);
        const key = getReminderKey(task.id, reminderMinutes);

        if (sentReminders.has(key)) continue;

        if (now >= reminderTime && now < dueDate) {
          if (announcementsSentThisCycle >= MAX_ANNOUNCEMENTS_PER_CYCLE) {
            console.log(`[Reminder] Rate limit: already sent ${MAX_ANNOUNCEMENTS_PER_CYCLE} announcements this cycle,...`);
            sentReminders.add(key);
            stateChanged = true;
            continue;
          }

          const timeSinceLast = Date.now() - lastAnnouncementTime;
          if (timeSinceLast < MIN_ANNOUNCEMENT_GAP_MS && lastAnnouncementTime > 0) {
            console.log(`[Reminder] Rate limit: only ${timeSinceLast}ms since last announcement, skipping "${task.title}...`);
            sentReminders.add(key);
            stateChanged = true;
            continue;
          }

          console.log(`[Reminder] Triggering reminder for task "${task.title}" (${reminderMinutes} min before due)`);

          const taskReminder: TaskReminder = {
            id: task.id,
            title: task.title,
            dueDate: task.dueDate instanceof Date ? task.dueDate.toISOString() : String(task.dueDate),
            courseName: task.courseName,
            type: task.type,
          };

          const timeLabel = reminderMinutes >= 60
            ? `${Math.round(reminderMinutes / 60)} hour${Math.round(reminderMinutes / 60) !== 1 ? 's' : ''}`
            : `${reminderMinutes} minutes`;
          const voiceMessage = `Reminder: ${task.title}${task.courseName ? ', for ${task.courseName}' : ''}, is due i...`;

          const isTravelling = getIsTravellingMode();
          const digestSentToday = await digestAlreadySentToday();
          const notifications: Promise<{ success: boolean; error?: string }>[] = [
            sendTaskReminder(taskReminder),
            sendHaTaskReminder(taskReminder),
          ];
          if (digestSentToday) {
            console.log(`[Reminder] Skipping individual Echo announcement for "${task.title}" (daily digest already a...`);
          } else if (!isTravelling) {
            notifications.push(sendEchoVoiceAnnouncement(voiceMessage));
          } else {
            console.log(`[Reminder] Skipping Echo announcement for "${task.title}" (travelling mode)`);
          }

          const results = await Promise.allSettled(notifications);

          const [emailResult, haResult] = results;
          if (emailResult.status === "fulfilled" && emailResult.value.success) {
            console.log(`[Reminder] Email sent for "${task.title}"`);
          } else {
            const err = emailResult.status === "rejected" ? emailResult.reason : emailResult.value.error;
            console.error(`[Reminder] Email failed for "${task.title}":`, err);
          }

          if (haResult.status === "fulfilled" && haResult.value.success) {
            console.log(`[Reminder] HA push sent for "${task.title}"`);
          } else {
            const err = haResult.status === "rejected" ? haResult.reason : haResult.value.error;
            console.error(`[Reminder] HA push failed for "${task.title}":`, err);
          }

          if (!isTravelling && results[2]) {
            const echoResult = results[2];
            if (echoResult.status === "fulfilled" && echoResult.value.success) {
              console.log(`[Reminder] Echo voice announcement sent for "${task.title}"`);
            } else {
              const err = echoResult.status === "rejected" ? echoResult.reason : echoResult.value.error;
              console.error(`[Reminder] Echo voice announcement failed for "${task.title}":`, err);
            }
          }

          sentReminders.add(key);
          stateChanged = true;
          remindersSentCount++;
          announcementsSentThisCycle++;
        }
      }
    }
  }
}

```

```

        lastAnnouncementTime = Date.now();
    }

    if (now > dueDate) {
        if (!sentReminders.has(key)) {
            sentReminders.add(key);
            stateChanged = true;
        }
    }
}

if (stateChanged) {
    await persistSentReminders();
}
} catch (err) {
    console.error("[Reminder] Error checking reminders:", err);
}
}

export async function checkDailyDigest() {
    try {
        const now = new Date();
        const todayStr = getEasternDateStr(now);
        const easternHour = getEasternHour(now);

        if (easternHour < DAILY_DIGEST_HOUR) return;

        const lastDigestDate = await getAppState("last_digest_date");
        if (lastDigestDate === todayStr) return;

        await setAppState("last_digest_date", todayStr);
        console.log("[Reminder] Sending daily digest (Echo announcements suppressed for individual reminders rest of toda...

        const allTasks = await storage.getTasks({ showCompleted: false });
        const threeDaysFromNow = addDaysET(now, 3);
        const todayMidnight = getEasternMidnight(now);

        const upcomingTasksRaw: TaskReminder[] = allTasks
            .filter(task => {
                if (task.isCompleted || !task.dueDate) return false;
                const dueStr = getEasternDateStr(new Date(task.dueDate));
                const todayStrCheck = getEasternDateStr(todayMidnight);
                const endStr = getEasternDateStr(threeDaysFromNow);
                return dueStr >= todayStrCheck && dueStr <= endStr;
            })
            .map(task => ({
                id: task.id,
                title: task.title,
                dueDate: task.dueDate instanceof Date ? task.dueDate.toISOString() : String(task.dueDate),
                courseName: task.courseName,
                type: task.type,
            }));
        const seenTitles = new Set<string>();
        const upcomingTasks = upcomingTasksRaw.filter(t => {
            if (seenTitles.has(t.title)) return false;
            seenTitles.add(t.title);
            return true;
        });
        if (upcomingTasks.length !== upcomingTasksRaw.length) {
            console.log(`[Reminder] Daily digest: deduplicated ${upcomingTasksRaw.length} !' ${upcomingTasks.length} tasks (...
        }

        if (upcomingTasks.length > 0) {
            const result = await sendDailyDigest(upcomingTasks);
            if (result.success) {
                console.log(`[Reminder] Daily digest sent with ${upcomingTasks.length} tasks`);
            } else {
                console.error("[Reminder] Daily digest failed:", result.error);
            }
        }

        const haMsg = upcomingTasks.map(t => t.title).join(", ");
        await sendHaTaskReminder({
            id: 0,
            title: `Daily Digest: ${upcomingTasks.length} tasks due soon`,
            dueDate: new Date().toISOString(),
            courseName: haMsg,
        });

        const isTravelling = getIsTravellingMode();
        if (!isTravelling) {
            const hour = getEasternHour(new Date());
            const greeting = hour < 12 ? "Good morning" : hour < 17 ? "Good afternoon" : "Good evening";

            const todayMid = easternMidnight();
            const tomorrowDateStr = easternDateStr(addDays(todayMid, 1));
            const day2DateStr = easternDateStr(addDays(todayMid, 2));
            const day3DateStr = easternDateStr(addDays(todayMid, 3));

            const tomorrowTasks = upcomingTasks.filter(t => taskDateStr(t.dueDate) === tomorrowDateStr);
            const day2Tasks = upcomingTasks.filter(t => taskDateStr(t.dueDate) === day2DateStr);
            const day3Tasks = upcomingTasks.filter(t => taskDateStr(t.dueDate) === day3DateStr);

            console.log(`[Reminder] Digest voice: tomorrow=${tomorrowDateStr} (${tomorrowTasks.length}), day2=${day2DateS...

            const MAX_NAMES_TO_READ = 10;

            function deduplicateTaskNames(tasks: TaskReminder[]): string[] {
                const seen = new Set<string>();
                const result: string[] = [];
                for (const t of tasks) {
                    const baseName = t.title.includes(' - ') ? t.title.split(' - ')[0].trim() : t.title.trim();
                    if (!seen.has(baseName.toLowerCase())) {
                        seen.add(baseName.toLowerCase());
                        result.push(baseName);
                    }
                }
                return result;
            }

```

```

    }

    let voiceMsg = `${greeting}.`;
    if (tomorrowTasks.length > 0) {
      const dedupedNames = deduplicateTaskNames(tomorrowTasks);
      if (dedupedNames.length <= MAX_NAMES_TO_READ) {
        const tomorrowList = dedupedNames.join(", ");
        voiceMsg += ` You have ${tomorrowTasks.length} task${tomorrowTasks.length !== 1 ? 's' : ''} due tomorrow...`;
      } else {
        const firstFew = dedupedNames.slice(0, 5).join(", ");
        voiceMsg += ` You have ${tomorrowTasks.length} tasks due tomorrow, including: ${firstFew}, and ${dedupedN...`;
      }
    } else {
      voiceMsg += ` No tasks due tomorrow.`;
    }

    if (day2Tasks.length > 0) {
      voiceMsg += ` Also, ${day2Tasks.length} task${day2Tasks.length !== 1 ? 's' : ''} the day after.`;
    }

    if (day3Tasks.length > 0) {
      voiceMsg += ` And ${day3Tasks.length} more in 3 days.`;
    }

    const echoResult = await sendEchoVoiceAnnouncement(voiceMsg);
    if (echoResult.success) {
      console.log(`[Reminder] Daily digest Echo announcement sent (${tomorrowTasks.length} tomorrow, ${day2Tasks....`);
    } else {
      console.error(`[Reminder] Daily digest Echo announcement failed:`, echoResult.error);
    }
  } else {
    console.log(`[Reminder] Skipping daily digest Echo announcement (travelling mode)`);
  }
} else {
  console.log("[Reminder] No upcoming tasks for daily digest");
}
} catch (err) {
  console.error("[Reminder] Error sending daily digest:", err);
}
}

export async function checkOutlookSync() {
  try {
    const now = new Date();
    const todayStr = getEasternDateStr(now);
    const easternHour = getEasternHour(now);

    if (easternHour < OUTLOOK_SYNC_HOUR) return;

    const lastSyncDate = await getAppState("last_outlook_sync_date");
    if (lastSyncDate === todayStr) return;

    await setAppState("last_outlook_sync_date", todayStr);
    console.log("[Reminder] Running Outlook calendar sync...");

    try {
      const result = await syncOutlookEventsToReview();
      console.log(`[Reminder] Outlook sync complete: ${result.added} added, ${result.skipped} skipped`);
    } catch (err: any) {
      console.error("[Reminder] Outlook sync failed:", err.message || err);
    }
  } catch (err) {
    console.error("[Reminder] Error in Outlook sync check:", err);
  }
}

let reminderInterval: ReturnType<typeof setInterval> | null = null;

export function startReminderScheduler() {
  if (reminderInterval) {
    clearInterval(reminderInterval);
    reminderInterval = null;
    console.log("[Reminder] Cleared existing scheduler interval before starting new one");
  }

  console.log("=== [Reminder] Starting reminder scheduler (checking every 60 seconds) ===");
  schedulerRunning = true;

  checkReminders();
  checkDailyDigest();
  checkOutlookSync();

  reminderInterval = setInterval(() => {
    checkReminders();
    checkDailyDigest();
    checkOutlookSync();
  }, 60 * 1000);
}

export function stopReminderScheduler() {
  if (reminderInterval) {
    clearInterval(reminderInterval);
    reminderInterval = null;
    schedulerRunning = false;
    console.log("[Reminder] Scheduler stopped");
  }
}
}

```

`server/replit\_integrations/audio/client.ts`

```

import OpenAI, { toFile } from "openai";
import { Buffer } from "node:buffer";
import { spawn } from "child_process";
import { writeFile, unlink, readFile } from "fs/promises";
import { randomUUID } from "crypto";
import { tmpdir } from "os";

```

```

import { join } from "path";
import { EdgeTTS } from "node-edge-tts";

export const openai = new OpenAI({
  apiKey: process.env.AI_INTEGRATIONS_OPENAI_API_KEY,
  baseURL: process.env.AI_INTEGRATIONS_OPENAI_BASE_URL,
});

let useEdgeTTSFallback = false;
let rateLimitResetTime: number | null = null;

export async function initTTSFallbackStatus() {
  try {
    const { db } = await import("../db");
    const { appState } = await import("../shared/schema");
    const { eq } = await import("drizzle-orm");
    const rows = await db.select().from(appState).where(eq(appState.key, 'tts_rate_limit_reset'));
    if (rows.length > 0) {
      const resetMs = parseInt(rows[0].value);
      if (resetMs > Date.now()) {
        useEdgeTTSFallback = true;
        rateLimitResetTime = resetMs;
        console.log(`[TTS] Rate limit active until ${new Date(resetMs).toISOString()} - using Edge TTS`);
      } else {
        useEdgeTTSFallback = false;
        rateLimitResetTime = null;
        await db.delete(appState).where(eq(appState.key, 'tts_rate_limit_reset'));
        console.log(`[TTS] Rate limit has expired - using OpenAI TTS`);
      }
    }
  } catch (e: any) {
    console.error(`[TTS] Failed to check rate limit status: ${e.message}`);
  }
}

async function saveRateLimitReset(resetMs: number) {
  try {
    const { db } = await import("../db");
    const { appState } = await import("../shared/schema");
    await db.insert(appState).values({ key: 'tts_rate_limit_reset', value: String(resetMs) })
      .onConflictDoUpdate({ target: appState.key, set: { value: String(resetMs) } });
  } catch (e: any) {
    console.error(`[TTS] Failed to save rate limit reset: ${e.message}`);
  }
}

const edgeVoiceMap: Record<string, string> = {
  echo: "en-US-AndrewMultilingualNeural",
  alloy: "en-US-AvaMultilingualNeural",
  fable: "en-GB-RyanNeural",
  onyx: "en-US-GuyNeural",
  nova: "en-US-JennyNeural",
  shimmer: "en-US-AriaNeural",
};

let edgeTTSConsecutiveFailures = 0;
const EDGE_TTS_MAX_CONSECUTIVE_FAILURES = 5;

async function edgeTTSGenerate(text: string, voice: string, retries = 2): Promise<Buffer> {
  const edgeVoice = edgeVoiceMap[voice] || edgeVoiceMap.echo;
  for (let attempt = 0; attempt <= retries; attempt++) {
    try {
      const outputPath = join(tmpdir(), `edge-tts-${randomUUID()}.mp3`);
      const tts = new EdgeTTS({ voice: edgeVoice });
      await tts.ttsPromise(text, outputPath);
      const buf = await readFile(outputPath);
      await unlink(outputPath).catch(() => {});
      if (buf.length === 0) throw new Error("Edge TTS returned empty audio");
      edgeTTSConsecutiveFailures = 0;
      return buf;
    } catch (err: any) {
      console.error(`[Edge TTS] Attempt ${attempt + 1} failed: ${err?.message || err}`);
      if (attempt < retries) {
        await new Promise(r => setTimeout(r, 1000 * (attempt + 1)));
      } else {
        edgeTTSConsecutiveFailures++;
        throw err;
      }
    }
  }
  edgeTTSConsecutiveFailures++;
  throw new Error("Edge TTS failed after retries");
}

async function localTTSGenerate(text: string): Promise<Buffer> {
  const outputPath = join(tmpdir(), `local-tts-${randomUUID()}.wav`);
  const cleanText = text.replace(/["'"]/g, "").replace(/\^\x20-\x7E\n/g, ' ').slice(0, 5000);
  try {
    await new Promise<void>((resolve, reject) => {
      const proc = spawn("espeak-ng", [
        "-v", "en-us",
        "-s", "145",
        "-p", "40",
        "-w", outputPath,
        cleanText,
      ]);
      let stderr = "";
      proc.stderr.on("data", (d) => { stderr += d.toString(); });
      proc.on("close", (code) => {
        if (code === 0) resolve();
        else reject(new Error(`espeak-ng exited with code ${code}: ${stderr}`));
      });
      proc.on("error", reject);
    });
    const wavBuf = await readFile(outputPath);
    await unlink(outputPath).catch(() => {});
    if (wavBuf.length === 0) throw new Error("espeak-ng returned empty audio");
    console.log(`[Local TTS] Generated ${wavBuf.length} bytes via espeak-ng`);
  }
}

```



```

    return wavBuf;
  } catch (err: any) {
    await unlink(outPath).catch(() => {});
    throw new Error(`Local TTS (espeak-ng) failed: ${err.message}`);
  }
}

export type AudioFormat = "wav" | "mp3" | "webm" | "mp4" | "ogg" | "unknown";

/**
 * Detect audio format from buffer magic bytes.
 * Supports: WAV, MP3, WebM (Chrome/Firefox), MP4/M4A/MOV (Safari/iOS), OGG
 */
export function detectAudioFormat(buffer: Buffer): AudioFormat {
  if (buffer.length < 12) return "unknown";

  // WAV: RIFF...WAVE
  if (buffer[0] === 0x52 && buffer[1] === 0x49 && buffer[2] === 0x46 && buffer[3] === 0x46) {
    return "wav";
  }
  // WebM: EBML header
  if (buffer[0] === 0x1a && buffer[1] === 0x45 && buffer[2] === 0xdf && buffer[3] === 0xa3) {
    return "webm";
  }
  // MP3: ID3 tag or frame sync
  if (
    (buffer[0] === 0xff && (buffer[1] === 0xfb || buffer[1] === 0xfa || buffer[1] === 0xf3)) ||
    (buffer[0] === 0x49 && buffer[1] === 0x44 && buffer[2] === 0x33)
  ) {
    return "mp3";
  }
  // MP4/M4A/MOV: ...ftyp (Safari/iOS records in these containers)
  if (buffer[4] === 0x66 && buffer[5] === 0x74 && buffer[6] === 0x79 && buffer[7] === 0x70) {
    return "mp4";
  }
  // OGG: OggS
  if (buffer[0] === 0x4f && buffer[1] === 0x67 && buffer[2] === 0x67 && buffer[3] === 0x53) {
    return "ogg";
  }
  return "unknown";
}

/**
 * Convert any audio/video format to WAV using ffmpeg.
 * Uses temp files instead of pipes because video containers (MP4/MOV)
 * require seeking to find the audio track.
 */
export async function convertToWav(audioBuffer: Buffer): Promise<Buffer> {
  const inputPath = join(tmpdir(), `input-${randomUUID()}`);
  const outputPath = join(tmpdir(), `output-${randomUUID()}.wav`);

  try {
    // Write input to temp file (required for video containers that need seeking)
    await writeFile(inputPath, audioBuffer);

    // Run ffmpeg with file paths
    await new Promise<void>((resolve, reject) => {
      const ffmpeg = spawn("ffmpeg", [
        "-i", inputPath,
        "-vn", // Extract audio only (ignore video track)
        "-f", "wav",
        "-ar", "16000", // 16kHz sample rate (good for speech)
        "-ac", "1", // Mono
        "-acodec", "pcm_s16le",
        "-y", // Overwrite output
        outputPath,
      ]);

      ffmpeg.stderr.on("data", () => {}); // Suppress logs
      ffmpeg.on("close", (code) => {
        if (code === 0) resolve();
        else reject(new Error(`ffmpeg exited with code ${code}`));
      });
      ffmpeg.on("error", reject);
    });

    // Read converted audio
    return await readFile(outputPath);
  } finally {
    // Clean up temp files
    await unlink(inputPath).catch(() => {});
    await unlink(outputPath).catch(() => {});
  }
}

/**
 * Auto-detect and convert audio to OpenAI-compatible format.
 * - WAV/MP3: Pass through (already compatible)
 * - WebM/MP4/OGG: Convert to WAV via ffmpeg
 */
export async function ensureCompatibleFormat(
  audioBuffer: Buffer
): Promise<{ buffer: Buffer; format: "wav" | "mp3" }> {
  const detected = detectAudioFormat(audioBuffer);
  if (detected === "wav") return { buffer: audioBuffer, format: "wav" };
  if (detected === "mp3") return { buffer: audioBuffer, format: "mp3" };
  // Convert WebM, MP4, OGG, or unknown to WAV
  const wavBuffer = await convertToWav(audioBuffer);
  return { buffer: wavBuffer, format: "wav" };
}

/**
 * Voice Chat: User speaks, LLM responds with audio (audio-in, audio-out).
 * Uses gpt-audio model via Replit AI Integrations.
 * Note: Browser records WebM/opus - convert to WAV using ffmpeg before calling this.
 */
export async function voiceChat(
  audioBuffer: Buffer,

```

```

voice: "alloy" | "echo" | "fable" | "onyx" | "nova" | "shimmer" = "alloy",
inputFormat: "wav" | "mp3" = "wav",
outputFormat: "wav" | "mp3" = "mp3"
): Promise<{ transcript: string; audioResponse: Buffer }> {
  const audioBase64 = audioBuffer.toString("base64");
  const response = await openai.chat.completions.create({
    model: "gpt-audio",
    modalities: ["text", "audio"],
    audio: { voice, format: outputFormat },
    messages: [{
      role: "user",
      content: [
        { type: "input_audio", input_audio: { data: audioBase64, format: inputFormat } },
      ],
    }],
  });
  const message = response.choices[0]?.message as any;
  const transcript = message?.audio?.transcript || message?.content || "";
  const audioData = message?.audio?.data ?? "";
  return {
    transcript,
    audioResponse: Buffer.from(audioData, "base64"),
  };
}

/**
 * Streaming Voice Chat: For real-time audio responses.
 * Note: Streaming only supports pcm16 output format.
 *
 * @example
 * // Converting browser WebM to WAV before calling:
 * const webmBuffer = Buffer.from(req.body.audio, "base64");
 * const wavBuffer = await convertWebmToWav(webmBuffer);
 * for await (const chunk of voiceChatStream(wavBuffer)) { ... }
 */
export async function voiceChatStream(
  audioBuffer: Buffer,
  voice: "alloy" | "echo" | "fable" | "onyx" | "nova" | "shimmer" = "alloy",
  inputFormat: "wav" | "mp3" = "wav"
): Promise<AsyncIterable<{ type: "transcript" | "audio"; data: string }>> {
  const audioBase64 = audioBuffer.toString("base64");
  const stream = await openai.chat.completions.create({
    model: "gpt-audio",
    modalities: ["text", "audio"],
    audio: { voice, format: "pcm16" },
    messages: [{
      role: "user",
      content: [
        { type: "input_audio", input_audio: { data: audioBase64, format: inputFormat } },
      ],
    }],
  }, {
    stream: true,
  });

  return (async function* () {
    for await (const chunk of stream) {
      const delta = chunk.choices?.[0]?.delta as any;
      if (!delta) continue;
      if (delta?.audio?.transcript) {
        yield { type: "transcript", data: delta.audio.transcript };
      }
      if (delta?.audio?.data) {
        yield { type: "audio", data: delta.audio.data };
      }
    }
  })();
}

/**
 * Text-to-Speech: Converts text to speech verbatim.
 * Uses gpt-audio model via Replit AI Integrations.
 */
export async function textToSpeech(
  text: string,
  voice: "alloy" | "echo" | "fable" | "onyx" | "nova" | "shimmer" = "echo",
  format: "wav" | "mp3" | "flac" | "opus" | "pcm16" = "wav",
  slowPace: boolean = false
): Promise<Buffer> {
  if (useEdgeTTSFallback) {
    if (edgeTTSTConsecutiveFailures >= EDGE_TTS_MAX_CONSECUTIVE_FAILURES) {
      console.log(`[TTS] Edge TTS has ${edgeTTSTConsecutiveFailures} consecutive failures – using local TTS (espeak-ng...)`);
      return localTTSGenerate(text);
    }
    try {
      console.log(`[TTS] Using Edge TTS fallback (voice: ${edgeVoiceMap[voice]} || edgeVoiceMap.echo)`);
      return await edgeTTSGenerate(text, voice);
    } catch (edgeErr: any) {
      console.warn(`[TTS] Edge TTS failed (${edgeTTSTConsecutiveFailures} consecutive): ${edgeErr.message} – falling b...`);
      return localTTSGenerate(text);
    }
  }

  try {
    const systemContent = slowPace
      ? "You are an assistant that performs text-to-speech. Speak at a slow, measured pace – slightly slower than nor..."
      : "You are an assistant that performs text-to-speech. Read the text naturally and clearly.";
    const response = await openai.chat.completions.create({
      model: "gpt-audio",
      modalities: ["text", "audio"],
      audio: { voice, format },
      messages: [
        { role: "system", content: systemContent },
        { role: "user", content: `Repeat the following text verbatim: ${text}` },
      ],
    });
    const audioData = (response.choices[0]?.message as any)?.audio?.data ?? "";
    return Buffer.from(audioData, "base64");
  } catch (err: any) {

```

```

    if (err?.status === 429 || err?.code === 'RATELIMIT_EXCEEDED') {
      const resetHeader = err?.headers?.get?('x-ratelimit-reset') || err?.headers?.['x-ratelimit-reset'];
      const resetMs = resetHeader ? parseInt(String(resetHeader)) : Date.now() + 30 * 24 * 60 * 60 * 1000;
      console.log(`[TTS] Rate limited – switching to Edge TTS until ${new Date(resetMs).toISOString()}`);
      useEdgeTTSFallback = true;
      rateLimitResetTime = resetMs;
      await saveRateLimitReset(resetMs);
      try {
        return await edgeTTSGenerate(text, voice);
      } catch (edgeErr: any) {
        console.warn(`[TTS] Edge TTS also failed: ${edgeErr.message} – falling back to local TTS`);
        return localTTSGenerate(text);
      }
    }
    try {
      return await edgeTTSGenerate(text, voice);
    } catch (edgeErr: any) {
      console.warn(`[TTS] OpenAI error + Edge TTS failed: ${edgeErr.message} – falling back to local TTS`);
      return localTTSGenerate(text);
    }
  }
}

/**
 * Streaming Text-to-Speech: Converts text to speech with real-time streaming.
 * Uses gpt-audio model via Replit AI Integrations.
 * Note: Streaming only supports pcm16 output format.
 */
export async function textToSpeechStream(
  text: string,
  voice: "alloy" | "echo" | "fable" | "onyx" | "nova" | "shimmer" = "echo",
  slowPace: boolean = false
): Promise<AsyncIterable<string>> {
  const systemContent = slowPace
    ? "You are an assistant that performs text-to-speech. Speak at a slow, measured pace – slightly slower than norma..."
    : "You are an assistant that performs text-to-speech. Read the text naturally and clearly.";
  const stream = await openai.chat.completions.create({
    model: "gpt-audio",
    modalities: ["text", "audio"],
    audio: { voice, format: "pcm16" },
    messages: [
      { role: "system", content: systemContent },
      { role: "user", content: `Repeat the following text verbatim: ${text}` },
    ],
    stream: true,
  });

  return (async function* () {
    for await (const chunk of stream) {
      const delta = chunk.choices?.[0]?.delta as any;
      if (!delta) continue;
      if (delta?.audio?.data) {
        yield delta.audio.data;
      }
    }
  })();
}

/**
 * Speech-to-Text: Transcribes audio using dedicated transcription model.
 * Uses gpt-4o-mini-transcribe for accurate transcription.
 */
export async function speechToText(
  audioBuffer: Buffer,
  format: "wav" | "mp3" | "webm" = "wav"
): Promise<string> {
  const file = await toFile(audioBuffer, `audio.${format}`);
  const response = await openai.audio.transcriptions.create({
    file,
    model: "gpt-4o-mini-transcribe",
  });
  return response.text;
}

/**
 * Streaming Speech-to-Text: Transcribes audio with real-time streaming.
 * Uses gpt-4o-mini-transcribe for accurate transcription.
 */
export async function speechToTextStream(
  audioBuffer: Buffer,
  format: "wav" | "mp3" | "webm" = "wav"
): Promise<AsyncIterable<string>> {
  const file = await toFile(audioBuffer, `audio.${format}`);
  const stream = await openai.audio.transcriptions.create({
    file,
    model: "gpt-4o-mini-transcribe",
    stream: true,
  });

  return (async function* () {
    for await (const event of stream) {
      if (event.type === "transcript.text.delta") {
        yield event.delta;
      }
    }
  })();
}

```

`server/replit\_integrations/audio/index.ts`

```

export { registerAudioRoutes } from "./routes";
export {
  openai,
  detectAudioFormat,

```

```

convertToWav,
ensureCompatibleFormat,
type AudioFormat,
voiceChat,
voiceChatStream,
textToSpeech,
textToSpeechStream,
speechToText,
speechToTextStream,
} from "../client";

```

`server/replit\_integrations/audio/routes.ts`

```

import express, { type Express, type Request, type Response } from "express";
import { chatStorage } from "../chat/storage";
import { openai, speechToText, ensureCompatibleFormat } from "../client";

// Body parser with 50MB limit for audio payloads
const audioBodyParser = express.json({ limit: "50mb" });

export function registerAudioRoutes(app: Express): void {
  // Get all conversations
  app.get("/api/conversations", async (req: Request, res: Response) => {
    try {
      const conversations = await chatStorage.getAllConversations();
      res.json(conversations);
    } catch (error) {
      console.error("Error fetching conversations:", error);
      res.status(500).json({ error: "Failed to fetch conversations" });
    }
  });

  // Get single conversation with messages
  app.get("/api/conversations/:id", async (req: Request, res: Response) => {
    try {
      const id = parseInt(String(req.params.id));
      const conversation = await chatStorage.getConversation(id);
      if (!conversation) {
        return res.status(404).json({ error: "Conversation not found" });
      }
      const messages = await chatStorage.getMessagesByConversation(id);
      res.json({ ...conversation, messages });
    } catch (error) {
      console.error("Error fetching conversation:", error);
      res.status(500).json({ error: "Failed to fetch conversation" });
    }
  });

  // Create new conversation
  app.post("/api/conversations", async (req: Request, res: Response) => {
    try {
      const { title } = req.body;
      const conversation = await chatStorage.createConversation(title || "New Chat");
      res.status(201).json(conversation);
    } catch (error) {
      console.error("Error creating conversation:", error);
      res.status(500).json({ error: "Failed to create conversation" });
    }
  });

  // Delete conversation
  app.delete("/api/conversations/:id", async (req: Request, res: Response) => {
    try {
      const id = parseInt(String(req.params.id));
      await chatStorage.deleteConversation(id);
      res.status(204).send();
    } catch (error) {
      console.error("Error deleting conversation:", error);
      res.status(500).json({ error: "Failed to delete conversation" });
    }
  });

  // Send voice message and get streaming audio response
  // Auto-detects audio format and converts WebM/MP4/OGG to WAV
  // Uses gpt-4o-mini-transcribe for STT, gpt-audio for voice response
  app.post("/api/conversations/:id/messages", audioBodyParser, async (req: Request, res: Response) => {
    try {
      const conversationId = parseInt(String(req.params.id));
      const { audio, voice = "alloy" } = req.body;

      if (!audio) {
        return res.status(400).json({ error: "Audio data (base64) is required" });
      }

      // 1. Auto-detect format and convert to OpenAI-compatible format
      const rawBuffer = Buffer.from(audio, "base64");
      const { buffer: audioBuffer, format: inputFormat } = await ensureCompatibleFormat(rawBuffer);

      // 2. Transcribe user audio
      const userTranscript = await speechToText(audioBuffer, inputFormat);

      // 3. Save user message
      await chatStorage.createMessage(conversationId, "user", userTranscript);

      // 4. Get conversation history
      const existingMessages = await chatStorage.getMessagesByConversation(conversationId);
      const chatHistory = existingMessages.map(m => ({
        role: m.role as "user" | "assistant",
        content: m.content,
      }));

      // 5. Set up SSE
      res.setHeader("Content-Type", "text/event-stream");
      res.setHeader("Cache-Control", "no-cache");
    }
  });
}

```

```

    res.setHeader("Connection", "keep-alive");

    res.write(`data: ${JSON.stringify({ type: "user_transcript", data: userTranscript })}\n\n`);

    // 6. Stream audio response from gpt-audio
    const stream = await openai.chat.completions.create({
      model: "gpt-audio",
      modalities: ["text", "audio"],
      audio: { voice, format: "pcm16" },
      messages: chatHistory,
      stream: true,
    });

    let assistantTranscript = "";

    for await (const chunk of stream) {
      const delta = chunk.choices?.[0]?.delta as any;
      if (!delta) continue;

      if (delta?.audio?.transcript) {
        assistantTranscript += delta.audio.transcript;
        res.write(`data: ${JSON.stringify({ type: "transcript", data: delta.audio.transcript })}\n\n`);
      }

      if (delta?.audio?.data) {
        res.write(`data: ${JSON.stringify({ type: "audio", data: delta.audio.data })}\n\n`);
      }
    }

    // 7. Save assistant message
    await chatStorage.createMessage(conversationId, "assistant", assistantTranscript);

    res.write(`data: ${JSON.stringify({ type: "done", transcript: assistantTranscript })}\n\n`);
    res.end();
  } catch (error) {
    console.error("Error processing voice message:", error);
    if (res.headersSent) {
      res.write(`data: ${JSON.stringify({ type: "error", error: "Failed to process voice message" })}\n\n`);
      res.end();
    } else {
      res.status(500).json({ error: "Failed to process voice message" });
    }
  }
});
}

```

`server/replit\_integrations/batch/index.ts`

```

export {
  batchProcess,
  batchProcessWithSSE,
  isRateLimitError,
  type BatchOptions,
} from "./utils";

```

`server/replit\_integrations/batch/utils.ts`

```

import pLimit from "p-limit";
import pRetry, { AbortError } from "p-retry";

/**
 * Batch Processing Utilities
 *
 * This module provides a generic batch processing function with built-in
 * rate limiting and automatic retries. Use it for any task that requires
 * processing multiple items through an LLM or external API.
 *
 * USAGE:
 * ```typescript
 * import { batchProcess, isRateLimitError } from "./replit_integrations/batch";
 *
 * const results = await batchProcess(
 *   artworks,
 *   async (artwork) => {
 *     // Your custom LLM logic here
 *     const response = await openai.chat.completions.create({
 *       model: "gpt-5.1",
 *       messages: [{ role: "user", content: `Categorize: ${artwork.name}` }],
 *       response_format: { type: "json_object" },
 *     });
 *     return JSON.parse(response.choices[0]?.message?.content || "{}");
 *   },
 *   { concurrency: 2, retries: 5 }
 * );
 */

export interface BatchOptions {
  /** Max concurrent requests (default: 2) */
  concurrency?: number;
  /** Max retry attempts for rate limit errors (default: 7) */
  retries?: number;
  /** Initial retry delay in ms (default: 2000) */
  minTimeout?: number;
  /** Max retry delay in ms (default: 128000) */
  maxTimeout?: number;
  /** Callback for progress updates */
  onProgress?: (completed: number, total: number, item: unknown) => void;
}

```

```

/**
 * Check if an error is a rate limit or quota violation.
 * Use this in custom error handling if needed.
 */
export function isRateLimitError(error: unknown): boolean {
  const errorMsg = error instanceof Error ? error.message : String(error);
  return (
    errorMsg.includes("429") ||
    errorMsg.includes("RATELIMIT_EXCEEDED") ||
    errorMsg.toLowerCase().includes("quota") ||
    errorMsg.toLowerCase().includes("rate limit")
  );
}

/**
 * Process items in batches with rate limiting and automatic retries.
 *
 * @param items - Array of items to process
 * @param processor - Async function to process each item (write your LLM logic here)
 * @param options - Concurrency and retry settings
 * @returns Promise resolving to array of results in the same order as input
 *
 * @example
 * // Process CSV artwork data with custom categorization
 * const categorized = await batchProcess(
 *   csvRows,
 *   async (row) => {
 *     const response = await openai.chat.completions.create({
 *       model: "gpt-5.1", // the newest OpenAI model
 *       messages: [{ role: "user", content: `Categorize artwork: ${row.name}` }],
 *       response_format: { type: "json_object" },
 *     });
 *     return { ...row, category: JSON.parse(response.choices[0]?.message?.content || "{}") };
 *   }
 * );
 */
export async function batchProcess<T, R>(
  items: T[],
  processor: (item: T, index: number) => Promise<R>,
  options: BatchOptions = {}
): Promise<R[]> {
  const {
    concurrency = 2,
    retries = 7,
    minTimeout = 2000,
    maxTimeout = 120000,
    onProgress,
  } = options;

  const limit = pLimit(concurrency);
  let completed = 0;

  const promises = items.map((item, index) =>
    limit(() =>
      pRetry(
        async () => {
          try {
            const result = await processor(item, index);
            completed++;
            onProgress?.(completed, items.length, item);
            return result;
          } catch (error: unknown) {
            if (isRateLimitError(error)) {
              throw error; // Rethrow to trigger p-retry
            }
            // For non-rate-limit errors, abort immediately
            throw new AbortError(
              error instanceof Error ? error.message : String(error)
            );
          }
        },
        { retries, minTimeout, maxTimeout, factor: 2 }
      )
    )
  );

  return Promise.all(promises);
}

/**
 * Process items sequentially with SSE progress streaming.
 * Use this when you need real-time progress updates to the client.
 *
 * @param items - Array of items to process
 * @param processor - Async function to process each item
 * @param sendEvent - Function to send SSE events to the client
 * @param options - Retry settings (concurrency is always 1 for sequential)
 */
export async function batchProcessWithSSE<T, R>(
  items: T[],
  processor: (item: T, index: number) => Promise<R>,
  sendEvent: (event: { type: string; [key: string]: unknown }) => void,
  options: Omit<BatchOptions, "concurrency" | "onProgress"> = {}
): Promise<R[]> {
  const { retries = 5, minTimeout = 1000, maxTimeout = 15000 } = options;

  sendEvent({ type: "started", total: items.length });

  const results: R[] = [];
  let errors = 0;

  for (let index = 0; index < items.length; index++) {
    const item = items[index];
    sendEvent({ type: "processing", index, item });

    try {
      const result = await pRetry(

```

```

    () => processor(item, index),
    {
      retries,
      minTimeout,
      maxTimeout,
      factor: 2,
      onFailedAttempt: (error) => {
        if (!isRateLimitError(error)) {
          throw new AbortError(
            error instanceof Error ? error.message : String(error)
          );
        }
      }
    },
  );
  results.push(result);
  sendEvent({ type: "progress", index, result });
} catch (error) {
  errors++;
  results.push(undefined as R); // Placeholder for failed items
  sendEvent({
    type: "progress",
    index,
    error: error instanceof Error ? error.message : "Processing failed",
  });
}
}

sendEvent({ type: "complete", processed: items.length, errors });
return results;
}

```

`server/replit\_integrations/chat/index.ts`

```

export { registerChatRoutes } from "../routes";
export { chatStorage, type IChatStorage } from "../storage";

```

`server/replit\_integrations/chat/routes.ts`

```

import type { Express, Request, Response } from "express";
import OpenAI from "openai";
import { chatStorage } from "../storage";

const openai = new OpenAI({
  apiKey: process.env.AI_INTEGRATIONS_OPENAI_API_KEY,
  baseURL: process.env.AI_INTEGRATIONS_OPENAI_BASE_URL,
});

export function registerChatRoutes(app: Express): void {
  // Get all conversations
  app.get("/api/conversations", async (req: Request, res: Response) => {
    try {
      const conversations = await chatStorage.getAllConversations();
      res.json(conversations);
    } catch (error) {
      console.error("Error fetching conversations:", error);
      res.status(500).json({ error: "Failed to fetch conversations" });
    }
  });

  // Get single conversation with messages
  app.get("/api/conversations/:id", async (req: Request, res: Response) => {
    try {
      const id = parseInt(String(req.params.id));
      const conversation = await chatStorage.getConversation(id);
      if (!conversation) {
        return res.status(404).json({ error: "Conversation not found" });
      }
      const messages = await chatStorage.getMessagesByConversation(id);
      res.json({ ...conversation, messages });
    } catch (error) {
      console.error("Error fetching conversation:", error);
      res.status(500).json({ error: "Failed to fetch conversation" });
    }
  });

  // Create new conversation
  app.post("/api/conversations", async (req: Request, res: Response) => {
    try {
      const { title } = req.body;
      const conversation = await chatStorage.createConversation(title || "New Chat");
      res.status(201).json(conversation);
    } catch (error) {
      console.error("Error creating conversation:", error);
      res.status(500).json({ error: "Failed to create conversation" });
    }
  });

  // Delete conversation
  app.delete("/api/conversations/:id", async (req: Request, res: Response) => {
    try {
      const id = parseInt(String(req.params.id));
      await chatStorage.deleteConversation(id);
      res.status(204).send();
    } catch (error) {
      console.error("Error deleting conversation:", error);
      res.status(500).json({ error: "Failed to delete conversation" });
    }
  });
}

```

```

});

// Send message and get AI response (streaming)
app.post("/api/conversations/:id/messages", async (req: Request, res: Response) => {
  try {
    const conversationId = parseInt(String(req.params.id));
    const { content } = req.body;

    // Save user message
    await chatStorage.createMessage(conversationId, "user", content);

    // Get conversation history for context
    const messages = await chatStorage.getMessagesByConversation(conversationId);
    const chatMessages = messages.map(m => ({
      role: m.role as "user" | "assistant",
      content: m.content,
    }));

    // Set up SSE
    res.setHeader("Content-Type", "text/event-stream");
    res.setHeader("Cache-Control", "no-cache");
    res.setHeader("Connection", "keep-alive");

    // Stream response from OpenAI
    const stream = await openai.chat.completions.create({
      model: "gpt-5.1",
      messages: chatMessages,
      stream: true,
      max_completion_tokens: 2048,
    });

    let fullResponse = "";

    for await (const chunk of stream) {
      const content = chunk.choices[0]?.delta?.content || "";
      if (content) {
        fullResponse += content;
        res.write(`data: ${JSON.stringify({ content })}\n\n`);
      }
    }

    // Save assistant message
    await chatStorage.createMessage(conversationId, "assistant", fullResponse);

    res.write(`data: ${JSON.stringify({ done: true })}\n\n`);
    res.end();
  } catch (error) {
    console.error("Error sending message:", error);
    // Check if headers already sent (SSE streaming started)
    if (res.headersSent) {
      res.write(`data: ${JSON.stringify({ error: "Failed to send message" })}\n\n`);
      res.end();
    } else {
      res.status(500).json({ error: "Failed to send message" });
    }
  }
});
}

```

`server/replit\_integrations/chat/storage.ts`

```

import { db } from "../../db";
import { conversations, messages } from "@shared/schema";
import { eq, desc } from "drizzle-orm";

export interface IChatStorage {
  getConversation(id: number): Promise<typeof conversations.$inferSelect | undefined>;
  getAllConversations(): Promise<(typeof conversations.$inferSelect)[]>;
  createConversation(title: string): Promise<typeof conversations.$inferSelect>;
  deleteConversation(id: number): Promise<void>;
  getMessagesByConversation(conversationId: number): Promise<(typeof messages.$inferSelect)[]>;
  createMessage(conversationId: number, role: string, content: string): Promise<typeof messages.$inferSelect>;
}

export const chatStorage: IChatStorage = {
  async getConversation(id: number) {
    const [conversation] = await db.select().from(conversations).where(eq(conversations.id, id));
    return conversation;
  },

  async getAllConversations() {
    return db.select().from(conversations).orderBy(desc(conversations.createdAt));
  },

  async createConversation(title: string) {
    const [conversation] = await db.insert(conversations).values({ title }).returning();
    return conversation;
  },

  async deleteConversation(id: number) {
    await db.delete(messages).where(eq(messages.conversationId, id));
    await db.delete(conversations).where(eq(conversations.id, id));
  },

  async getMessagesByConversation(conversationId: number) {
    return db.select().from(messages).where(eq(messages.conversationId, conversationId)).orderBy(messages.createdAt);
  },

  async createMessage(conversationId: number, role: string, content: string) {
    const [message] = await db.insert(messages).values({ conversationId, role, content }).returning();
    return message;
  },
};

```

`server/replit\_integrations/image/client.ts`

```
import fs from "node:fs";
import OpenAI, { toFile } from "openai";
import { Buffer } from "node:buffer";

export const openai = new OpenAI({
  apiKey: process.env.AI_INTEGRATIONS_OPENAI_API_KEY,
  baseURL: process.env.AI_INTEGRATIONS_OPENAI_BASE_URL,
});

/**
 * Generate an image and return as Buffer.
 * Uses gpt-image-1 model via Replit AI Integrations.
 */
export async function generateImageBuffer(
  prompt: string,
  size: "1024x1024" | "512x512" | "256x256" = "1024x1024"
): Promise<Buffer> {
  const response = await openai.images.generate({
    model: "gpt-image-1",
    prompt,
    size,
  });
  const base64 = response.data?.[0]?.b64_json ?? "";
  return Buffer.from(base64, "base64");
}

/**
 * Edit/combine multiple images into a composite.
 * Uses gpt-image-1 model via Replit AI Integrations.
 */
export async function editImages(
  imageFiles: string[],
  prompt: string,
  outputPath?: string
): Promise<Buffer> {
  const images = await Promise.all(
    imageFiles.map((file) =>
      toFile(fs.createReadStream(file), file, {
        type: "image/png",
      })
    )
  );
  const response = await openai.images.edit({
    model: "gpt-image-1",
    image: images,
    prompt,
  });
  const imageBase64 = response.data?.[0]?.b64_json ?? "";
  const imageBytes = Buffer.from(imageBase64, "base64");

  if (outputPath) {
    fs.writeFileSync(outputPath, imageBytes);
  }

  return imageBytes;
}
```

`server/replit\_integrations/image/index.ts`

```
export { registerImageRoutes } from "./routes";
export { openai, generateImageBuffer, editImages } from "./client";
```

`server/replit\_integrations/image/routes.ts`

```
import type { Express, Request, Response } from "express";
import { openai } from "./client";

export function registerImageRoutes(app: Express): void {
  app.post("/api/generate-image", async (req: Request, res: Response) => {
    try {
      const { prompt, size = "1024x1024" } = req.body;

      if (!prompt) {
        return res.status(400).json({ error: "Prompt is required" });
      }

      const response = await openai.images.generate({
        model: "gpt-image-1",
        prompt,
        n: 1,
        size: size as "1024x1024" | "512x512" | "256x256",
      });

      const imageData = response.data?.[0];
      res.json({
        url: imageData.url,
        b64_json: imageData.b64_json,
      });
    }
  });
}
```

```

    });
  } catch (error) {
    console.error("Error generating image:", error);
    res.status(500).json({ error: "Failed to generate image" });
  }
});
}

```

`server/replit\_integrations/object\_storage/index.ts`

```

export {
  ObjectStorageService,
  ObjectNotFoundError,
  ObjectStorageClient,
} from "./objectStorage";

export type {
  ObjectAclPolicy,
  ObjectAccessGroup,
  ObjectAccessGroupType,
  ObjectAclRule,
} from "./objectAcl";

export {
  canAccessObject,
  getObjectAclPolicy,
  setObjectAclPolicy,
} from "./objectAcl";

export { registerObjectStorageRoutes } from "./routes";

```

`server/replit\_integrations/object\_storage/objectAcl.ts`

```

import { File } from "@google-cloud/storage";

const ACL_POLICY_METADATA_KEY = "custom:aclPolicy";

// The type of the access group.
//
// Can be flexibly defined according to the use case.
//
// Examples:
// - USER_LIST: the users from a list stored in the database;
// - EMAIL_DOMAIN: the users whose email is in a specific domain;
// - GROUP_MEMBER: the users who are members of a specific group;
// - SUBSCRIBER: the users who are subscribers of a specific service / content
//   creator.
export enum ObjectAccessGroupType {}

// The logic user group that can access the object.
export interface ObjectAccessGroup {
  // The type of the access group.
  type: ObjectAccessGroupType;
  // The logic id that is enough to identify the qualified group members.
  //
  // It may have different format for different types. For example:
  // - for USER_LIST, the id could be the user list db entity id, and the
  //   user list db entity could contain a bunch of user ids. User needs
  //   to be a member of the user list to be able to access the object.
  // - for EMAIL_DOMAIN, the id could be the email domain, and the user needs
  //   to have an email with the domain to be able to access the object.
  // - for GROUP_MEMBER, the id could be the group db entity id, and the
  //   group db entity could contain a bunch of user ids. User needs to be
  //   a member of the group to be able to access the object.
  // - for SUBSCRIBER, the id could be the subscriber db entity id, and the
  //   subscriber db entity could contain a bunch of user ids. User needs to
  //   be a subscriber to be able to access the object.
  id: string;
}

export enum ObjectPermission {
  READ = "read",
  WRITE = "write",
}

export interface ObjectAclRule {
  group: ObjectAccessGroup;
  permission: ObjectPermission;
}

// The ACL policy of the object.
// This would be set as part of the object custom metadata:
// - key: "custom:aclPolicy"
// - value: JSON string of the ObjectAclPolicy object.
export interface ObjectAclPolicy {
  owner: string;
  visibility: "public" | "private";
  aclRules?: Array<ObjectAclRule>;
}

// Check if the requested permission is allowed based on the granted permission.
function isPermissionAllowed(
  requested: ObjectPermission,
  granted: ObjectPermission,
): boolean {
  // Users granted with read or write permissions can read the object.
  if (requested === ObjectPermission.READ) {
    return [ObjectPermission.READ, ObjectPermission.WRITE].includes(granted);
  }
}

```

```

    }

    // Only users granted with write permissions can write the object.
    return granted === ObjectPermission.WRITE;
}

// The base class for all access groups.
//
// Different types of access groups can be implemented according to the use case.
abstract class BaseObjectAccessGroup implements ObjectAccessGroup {
    constructor(
        public readonly type: ObjectAccessGroupType,
        public readonly id: string,
    ) {}

    // Check if the user is a member of the group.
    public abstract hasMember(userId: string): Promise<boolean>;
}

function createObjectAccessGroup(
    group: ObjectAccessGroup,
): BaseObjectAccessGroup {
    switch (group.type) {
        // Implement the case for each type of access group to instantiate.
        //
        // For example:
        // case "USER_LIST":
        //     return new UserListAccessGroup(group.id);
        // case "EMAIL_DOMAIN":
        //     return new EmailDomainAccessGroup(group.id);
        // case "GROUP_MEMBER":
        //     return new GroupMemberAccessGroup(group.id);
        // case "SUBSCRIBER":
        //     return new SubscriberAccessGroup(group.id);
        default:
            throw new Error(`Unknown access group type: ${group.type}`);
    }
}

// Sets the ACL policy to the object metadata.
export async function setObjectAclPolicy(
    objectFile: File,
    aclPolicy: ObjectAclPolicy,
): Promise<void> {
    const [exists] = await objectFile.exists();
    if (!exists) {
        throw new Error(`Object not found: ${objectFile.name}`);
    }

    await objectFile.setMetadata({
        metadata: {
            [ACL_POLICY_METADATA_KEY]: JSON.stringify(aclPolicy),
        },
    });
}

// Gets the ACL policy from the object metadata.
export async function getObjectAclPolicy(
    objectFile: File,
): Promise<ObjectAclPolicy | null> {
    const [metadata] = await objectFile.getMetadata();
    const aclPolicy = metadata?.metadata?.[ACL_POLICY_METADATA_KEY];
    if (!aclPolicy) {
        return null;
    }
    return JSON.parse(aclPolicy as string);
}

// Checks if the user can access the object.
export async function canAccessObject({
    userId,
    objectFile,
    requestedPermission,
}): {
    userId?: string;
    objectFile: File;
    requestedPermission: ObjectPermission;
}: Promise<boolean> {
    // When this function is called, the acl policy is required.
    const aclPolicy = await getObjectAclPolicy(objectFile);
    if (!aclPolicy) {
        return false;
    }

    // Public objects are always accessible for read.
    if (
        aclPolicy.visibility === "public" &&
        requestedPermission === ObjectPermission.READ
    ) {
        return true;
    }

    // Access control requires the user id.
    if (!userId) {
        return false;
    }

    // The owner of the object can always access it.
    if (aclPolicy.owner === userId) {
        return true;
    }

    // Go through the ACL rules to check if the user has the required permission.
    for (const rule of aclPolicy.aclRules || []) {
        const accessGroup = createObjectAccessGroup(rule.group);
        if (
            (await accessGroup.hasMember(userId)) &&
            isPermissionAllowed(requestedPermission, rule.permission)
        ) {
            return true;
        }
    }

    return false;
}

```

```

    } {
      return true;
    }
  }

  return false;
}

```

`server/replit\_integrations/object\_storage/objectStorage.ts`

```

import { Storage, File } from "@google-cloud/storage";
import { Response } from "express";
import { randomUUID } from "crypto";
import {
  ObjectAclPolicy,
  ObjectPermission,
  canAccessObject,
  getObjectAclPolicy,
  setObjectAclPolicy,
} from "./objectAcl";

const REPLIT_SIDE CAR_ENDPOINT = "http://127.0.0.1:1106";

// The object storage client is used to interact with the object storage service.
export const objectStorageClient = new Storage({
  credentials: {
    audience: "replit",
    subject_token_type: "access_token",
    token_url: `${REPLIT_SIDE CAR_ENDPOINT}/token`,
    type: "external_account",
    credential_source: {
      url: `${REPLIT_SIDE CAR_ENDPOINT}/credential`,
      format: {
        type: "json",
        subject_token_field_name: "access_token",
      },
    },
  },
  universe_domain: "googleapis.com",
},
projectId: "");

export class ObjectNotFoundError extends Error {
  constructor() {
    super("Object not found");
    this.name = "ObjectNotFoundError";
    Object.setPrototypeOf(this, ObjectNotFoundError.prototype);
  }
}

// The object storage service is used to interact with the object storage service.
export class ObjectStorageService {
  constructor() {}

  // Gets the public object search paths.
  getPublicObjectSearchPaths(): Array<string> {
    const pathsStr = process.env.PUBLIC_OBJECT_SEARCH_PATHS || "";
    const paths = Array.from(
      new Set(
        pathsStr
          .split(",")
          .map((path) => path.trim())
          .filter((path) => path.length > 0)
      )
    );
    if (paths.length === 0) {
      throw new Error(
        "PUBLIC_OBJECT_SEARCH_PATHS not set. Create a bucket in 'Object Storage' " +
        "tool and set PUBLIC_OBJECT_SEARCH_PATHS env var (comma-separated paths)."
      );
    }
    return paths;
  }

  // Gets the private object directory.
  getPrivateObjectDir(): string {
    const dir = process.env.PRIVATE_OBJECT_DIR || "";
    if (!dir) {
      throw new Error(
        "PRIVATE_OBJECT_DIR not set. Create a bucket in 'Object Storage' " +
        "tool and set PRIVATE_OBJECT_DIR env var."
      );
    }
    return dir;
  }

  // Search for a public object from the search paths.
  async searchPublicObject(filePath: string): Promise<File | null> {
    for (const searchPath of this.getPublicObjectSearchPaths()) {
      const fullPath = `${searchPath}/${filePath}`;

      // Full path format: <bucket_name>/<object_name>
      const { bucketName, objectName } = parseObjectPath(fullPath);
      const bucket = objectStorageClient.bucket(bucketName);
      const file = bucket.file(objectName);

      // Check if file exists
      const [exists] = await file.exists();
      if (exists) {
        return file;
      }
    }
  }
}

```

```

    return null;
}

// Downloads an object to the response.
async downloadObject(file: File, res: Response, cacheTtlSec: number = 3600) {
    try {
        // Get file metadata
        const [metadata] = await file.getMetadata();
        // Get the ACL policy for the object.
        const aclPolicy = await getObjectAclPolicy(file);
        const isPublic = aclPolicy?.visibility === "public";
        // Set appropriate headers
        res.set({
            "Content-Type": metadata.contentType || "application/octet-stream",
            "Content-Length": metadata.size,
            "Cache-Control": `${
                isPublic ? "public" : "private"
            }, max-age=${cacheTtlSec}`,
        });

        // Stream the file to the response
        const stream = file.createReadStream();

        stream.on("error", (err) => {
            console.error("Stream error:", err);
            if (!res.headersSent) {
                res.status(500).json({ error: "Error streaming file" });
            }
        });

        stream.pipe(res);
    } catch (error) {
        console.error("Error downloading file:", error);
        if (!res.headersSent) {
            res.status(500).json({ error: "Error downloading file" });
        }
    }
}

// Gets the upload URL for an object entity.
async getObjectEntityUploadURL(): Promise<string> {
    const privateObjectDir = this.getPrivateObjectDir();
    if (!privateObjectDir) {
        throw new Error(
            "PRIVATE_OBJECT_DIR not set. Create a bucket in 'Object Storage' " +
            "tool and set PRIVATE_OBJECT_DIR env var."
        );
    }
}

const objectId = randomUUID();
const fullPath = `${privateObjectDir}/uploads/${objectId}`;

const { bucketName, objectName } = parseObjectPath(fullPath);

// Sign URL for PUT method with TTL
return signObjectURL({
    bucketName,
    objectName,
    method: "PUT",
    ttlSec: 900,
});
}

// Gets the object entity file from the object path.
async getObjectEntityFile(objectPath: string): Promise<File> {
    if (!objectPath.startsWith("/objects/")) {
        throw new ObjectNotFoundError();
    }

    const parts = objectPath.slice(1).split("/");
    if (parts.length < 2) {
        throw new ObjectNotFoundError();
    }

    const entityId = parts.slice(1).join("/");
    let entityDir = this.getPrivateObjectDir();
    if (!entityDir.endsWith("/")) {
        entityDir = `${entityDir}/`;
    }
    const objectEntityPath = `${entityDir}${entityId}`;
    const { bucketName, objectName } = parseObjectPath(objectEntityPath);
    const bucket = objectStorageClient.bucket(bucketName);
    const objectFile = bucket.file(objectName);
    const [exists] = await objectFile.exists();
    if (!exists) {
        throw new ObjectNotFoundError();
    }
    return objectFile;
}

normalizeObjectEntityPath(
    rawPath: string,
): string {
    if (!rawPath.startsWith("https://storage.googleapis.com/")) {
        return rawPath;
    }

    // Extract the path from the URL by removing query parameters and domain
    const url = new URL(rawPath);
    const rawObjectPath = url.pathname;

    let objectEntityDir = this.getPrivateObjectDir();
    if (!objectEntityDir.endsWith("/")) {
        objectEntityDir = `${objectEntityDir}/`;
    }

    if (!rawObjectPath.startsWith(objectEntityDir)) {
        return rawObjectPath;
    }

```

```

    }

    // Extract the entity ID from the path
    const entityId = rawObjectPath.slice(objectEntityDir.length);
    return `/objects/${entityId}`;
}

// Tries to set the ACL policy for the object entity and return the normalized path.
async trySetObjectEntityAclPolicy(
  rawPath: string,
  aclPolicy: ObjectAclPolicy
): Promise<string> {
  const normalizedPath = this.normalizeObjectEntityPath(rawPath);
  if (!normalizedPath.startsWith("/")) {
    return normalizedPath;
  }

  const objectFile = await this.getObjectEntityFile(normalizedPath);
  await setObjectAclPolicy(objectFile, aclPolicy);
  return normalizedPath;
}

// Checks if the user can access the object entity.
async canAccessObjectEntity({
  userId,
  objectFile,
  requestedPermission,
}): {
  userId?: string;
  objectFile: File;
  requestedPermission?: ObjectPermission;
}): Promise<boolean> {
  return canAccessObject({
    userId,
    objectFile,
    requestedPermission: requestedPermission ?? ObjectPermission.READ,
  });
}

function parseObjectPath(path: string): {
  bucketName: string;
  objectName: string;
} {
  if (!path.startsWith("/")) {
    path = `/${path}`;
  }
  const pathParts = path.split("/");
  if (pathParts.length < 3) {
    throw new Error("Invalid path: must contain at least a bucket name");
  }

  const bucketName = pathParts[1];
  const objectName = pathParts.slice(2).join("/");

  return {
    bucketName,
    objectName,
  };
}

async function signObjectURL({
  bucketName,
  objectName,
  method,
  ttlSec,
}): {
  bucketName: string;
  objectName: string;
  method: "GET" | "PUT" | "DELETE" | "HEAD";
  ttlSec: number;
}): Promise<string> {
  const request = {
    bucket_name: bucketName,
    object_name: objectName,
    method,
    expires_at: new Date(Date.now() + ttlSec * 1000).toISOString(),
  };
  const response = await fetch(
    `${REPLIT_SIDE CAR_ENDPOINT}/object-storage/signed-object-url`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(request),
    }
  );
  if (!response.ok) {
    throw new Error(
      `Failed to sign object URL, errorcode: ${response.status}, ` +
      `make sure you're running on Replit`
    );
  }

  const { signed_url: signedURL } = await response.json();
  return signedURL;
}

```

`server/replit\_integrations/object\_storage/routes.ts`

```

import type { Express } from "express";
import { ObjectStorageService, ObjectNotFoundError } from "../objectStorage";

```

```

import { storage } from "../../storage";

/**
 * Register object storage routes for file uploads.
 *
 * This provides example routes for the presigned URL upload flow:
 * 1. POST /api/uploads/request-url - Get a presigned URL for uploading
 * 2. The client then uploads directly to the presigned URL
 *
 * IMPORTANT: These are example routes. Customize based on your use case:
 * - Add authentication middleware for protected uploads
 * - Add file metadata storage (save to database after upload)
 * - Add ACL policies for access control
 */
export function registerObjectStorageRoutes(app: Express): void {
  const objectStorageService = new ObjectStorageService();

  /**
   * Request a presigned URL for file upload.
   *
   * Request body (JSON):
   * {
   *   "name": "filename.jpg",
   *   "size": 12345,
   *   "contentType": "image/jpeg"
   * }
   *
   * Response:
   * {
   *   "uploadURL": "https://storage.googleapis.com/...",
   *   "objectPath": "/objects/uploads/uuid"
   * }
   *
   * IMPORTANT: The client should NOT send the file to this endpoint.
   * Send JSON metadata only, then upload the file directly to uploadURL.
   */
  app.post("/api/uploads/request-url", async (req, res) => {
    try {
      const { name, size, contentType, folder } = req.body;

      if (!name) {
        return res.status(400).json({
          error: "Missing required field: name",
        });
      }

      const uploadURL = await objectStorageService.getObjectEntityUploadURL();

      // Extract object path from the presigned URL for later reference
      const objectPath = objectStorageService.normalizeObjectEntityPath(uploadURL);

      // Save file metadata to database (including folder if provided)
      const fileData: any = {
        originalName: name,
        displayName: name,
        objectPath,
        contentType: contentType || "application/octet-stream",
        size: size || null,
      };

      if (folder) {
        fileData.folder = folder;
      }

      const createdFile = await storage.createFile(fileData);

      res.json({
        uploadURL,
        objectPath,
        fileId: createdFile?.id,
        // Echo back the metadata for client convenience
        metadata: { name, size, contentType, folder },
      });
    } catch (error) {
      console.error("Error generating upload URL:", error);
      res.status(500).json({ error: "Failed to generate upload URL" });
    }
  });

  /**
   * Serve uploaded objects.
   *
   * GET /objects/{*objectPath}
   *
   * This serves files from object storage. For public files, no auth needed.
   * For protected files, add authentication middleware and ACL checks.
   */
  app.get("/objects/{*objectPath}", async (req, res) => {
    try {
      const objectFile = await objectStorageService.getObjectEntityFile(req.path);
      await objectStorageService.downloadObject(objectFile, res);
    } catch (error) {
      console.error("Error serving object:", error);
      if (error instanceof ObjectNotFoundError) {
        return res.status(404).json({ error: "Object not found" });
      }
      return res.status(500).json({ error: "Failed to serve object" });
    }
  });
}

```

server/routes.ts

```

import type { Express } from "express";
import express from "express";
import type { Server } from "http";
import crypto from "crypto";
import fs from "fs";
import path from "path";
import { storage } from "../storage";
import { api } from "@shared/routes";
import { db } from "../db";
import { sql, eq } from "drizzle-orm";
import { getWeekDates, getWeekNumber, FIRST_WEEK, LAST_WEEK, DEFAULT_REMINDER_1, DEFAULT_REMINDER_2, COURSES, type Re...
import { z } from "zod";
import { LIBERAL_STUDIES_COURSES, OPEN_ELECTIVE_COURSES } from "@shared/electiveCourses";
import { registerObjectStorageRoutes } from "../replit_integrations/object_storage";
import { objectStorageClient } from "../replit_integrations/object_storage/objectStorage";
import { createCalendarEvent, deleteCalendarEvent, updateCalendarEvent, listEvents, listCalendars, createPrepCalendar...
import { getSecondAccountAuthUrl, exchangeCodeForTokens, isSecondAccountConnected, disconnectSecondAccount, createEve...
import { getThirdAccountAuthUrl, exchangeCodeForTokensThird, isThirdAccountConnected, disconnectThirdAccount, getEven...
import { textToSpeech, initTTSFallbackStatus } from "../replit_integrations/audio/client";
import { sendTestEmail, sendTaskReminder, sendDailyDigest, sendTestSms, sendSmsReminder, sendTestHaPush, sendHaTaskRe...
import { syncOutlookEventsToReview, fetchOutlookCalendarEvents, findOrCreateMailFolder, createMailRule, moveExistingE...
import { parseTickerViewCommand, extractInlineExpiry } from "../gmailTickerView";
// fetchD2LAnnouncements available in ./gmail but Gmail connector lacks read scope; D2L sync handled by external Apps...
import { getSchedulerStatus } from "../reminderScheduler";
import { fetchTMUCalendarEvents } from "../tmuCalendar";
import { listOneDriveItems, getOneDriveFile, searchOneDriveFiles, createOneDriveFolder, getOneDriveFileContentAsText,...
import * as spotifyApi from "../spotify";

```

```

// Helper function to generate repeated task due dates
function generateRepeatDates(

```

```

  startDueDate: Date,
  repeatType: RepeatType,
  repeatEndDate: Date | null,
  repeatInterval?: number,
  repeatIntervalUnit?: RepeatIntervalUnit
): Date[] {

```

```

  const dates: Date[] = [];
  if (repeatType === "none") return dates;

```

```

  // Default end date: 6 months from start if not specified
  const endDate = repeatEndDate || new Date(startDueDate.getTime() + 180 * 24 * 60 * 60 * 1000);
  let currentDate = new Date(startDueDate);

```

```

  // Add interval based on repeat type
  const addInterval = (date: Date): Date => {
    const newDate = new Date(date);
    switch (repeatType) {
      case "daily":
        newDate.setDate(newDate.getDate() + 1);
        break;
      case "weekly":
        newDate.setDate(newDate.getDate() + 7);
        break;
      case "monthly":
        newDate.setMonth(newDate.getMonth() + 1);
        break;
      case "custom":
        if (repeatIntervalUnit === "days") {
          newDate.setDate(newDate.getDate() + (repeatInterval || 1));
        } else if (repeatIntervalUnit === "weeks") {
          newDate.setDate(newDate.getDate() + (repeatInterval || 1) * 7);
        }
        break;
    }
    return newDate;
  };

```

```

  // Generate dates until end date (max 100 to prevent infinite loops)
  let count = 0;
  while (count < 100) {
    currentDate = addInterval(currentDate);
    if (currentDate > endDate) break;
    dates.push(new Date(currentDate));
    count++;
  }

```

```

  return dates;
}

```

```

// Dynamic import for pdf-parse v2

```

```

async function getPdfParser() {
  const { PDFParse } = await import("pdf-parse");
  return PDFParse;
}

```

```

// ===== CENTRALIZED CONFIGURATION =====

```

```

// Change these values in ONE place if URLs or devices change.

```

```

const DEPLOYED_APP_URL = process.env.DEPLOYED_APP_URL || "https://home-view-bkh416.replit.app";
const HOME_ASSISTANT_URL = process.env.HOME_ASSISTANT_URL_OVERRIDE || "https://ec8ebfanqrqlsnmggrdl4yzq2i8koah.ui.na...
const tokenFromEnv = process.env.HOME_ASSISTANT_TOKEN || "";
const urlFromEnv = process.env.HOME_ASSISTANT_URL || "";
const HOME_ASSISTANT_TOKEN = tokenFromEnv.startsWith("eyJ") ? tokenFromEnv : (urlFromEnv.startsWith("eyJ") ? urlFromEnv...

```

```

const BATHROOM_ECHO_ENTITY = "media_player.bathroom_speaker";
const KITCHEN_ECHO_ENTITY = "media_player.echo_kitchen_studio_black_am";
const NEST_SPEAKER_ENTITY = "media_player.bathroom_speaker";
const CAT_WR_HA_VOICE_ENTITY = "media_player.home_assistant_voice_097c38_media_player";
const NON_ALEXA_ENTITIES = [NEST_SPEAKER_ENTITY, CAT_WR_HA_VOICE_ENTITY];
const MODULE_READING_PENDING = "input_boolean.module_reading_pending";
const MODULE_READING_CONFIRMED = "input_boolean.module_reading_confirmed";
const PARTNER_PHONE_ENTITY = "device_tracker.y_phone_app";
const HA_CLOUD_TTS_ENTITY = "tts.home_assistant_cloud";
const CAT_LIGHTS_ENTITY = "light.cat_lights";
const CAT_TV_ENTITY = "media_player.tv_cat_wr";
const FIRE_STICK_ADB_ENTITY = "media_player.fire_tv_172_24_0_88";
const CAT_WR_MEDIA_GROUP = "media_player.cat_washroom_media_group";
const CAT_ECHO_ENTITIES = [
  "media_player.echo_cat_left_am",

```



```

    "media_player.echo_cat_right_am",
    "media_player.echo_cat_washroom_middle",
];

const SPOTIFYPLUS_ENTITY = "media_player.spotifyplus_byhomeyyz";
const EVERYWHERE_GROUP_ENTITY = "media_player.byhome";

async function haFetch(url: string, options: RequestInit = {}, maxRetries = 3, label = 'HA'): Promise<Response> {
    for (let attempt = 0; attempt < maxRetries; attempt++) {
        let timer: ReturnType<typeof setTimeout> | null = null;
        try {
            const controller = new AbortController();
            timer = setTimeout(() => controller.abort(), 12000);
            const resp = await fetch(url, { ...options, signal: controller.signal });
            clearTimeout(timer);
            timer = null;
            if (!resp.ok) {
                const errText = await resp.text().catch(() => '');
                throw new Error(`HTTP ${resp.status}: ${errText.substring(0, 200)}`);
            }
            return resp;
        } catch (e: any) {
            if (timer) clearTimeout(timer);
            const msg = e?.message || String(e);
            if (attempt < maxRetries - 1) {
                const delay = 1500 * (attempt + 1);
                console.warn(`[${label}] fetch attempt ${attempt + 1}/${maxRetries} failed: ${msg} - retrying in ${delay}ms`);
                await new Promise(r => setTimeout(r, delay));
            } else {
                throw new Error(`[${label}] All ${maxRetries} fetch attempts failed: ${msg}`);
            }
        }
    }
    throw new Error(`[${label}] Unreachable`);
}

async function haServiceCall(service: string, data: object, label = 'HA'): Promise<Response> {
    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
    return haFetch(`${haUrl}/api/services/${service}`, {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
        body: JSON.stringify(data),
    }, 3, label);
}

interface QueuedHACommand {
    service: string;
    data: object;
    label: string;
    queuedAt: number;
    attempts: number;
}

const haCommandQueue: QueuedHACommand[] = [];
const HA_QUEUE_MAX_AGE_MS = 5 * 60 * 1000;
const HA_QUEUE_MAX_SIZE = 50;
let haQueueProcessing = false;

async function processHACommandQueue(): Promise<void> {
    if (haQueueProcessing || haCommandQueue.length === 0) return;
    haQueueProcessing = true;
    try {
        let remaining = haCommandQueue.length;
        for (let processed = 0; processed < remaining; processed++) {
            const cmd = haCommandQueue.shift();
            if (!cmd) break;
            const now = Date.now();
            if (now - cmd.queuedAt > HA_QUEUE_MAX_AGE_MS) {
                console.log(`[HA Queue] Expired command dropped: ${cmd.label} ${cmd.service} (queued ${Math.round((now - cmd....
                continue;
            }
            try {
                await haServiceCall(cmd.service, cmd.data, `${cmd.label} [Replayed]`);
                console.log(`[HA Queue] Successfully replayed: ${cmd.label} ${cmd.service} (was queued ${Math.round((now - cm...
            } catch (e: any) {
                cmd.attempts++;
                if (cmd.attempts >= 3) {
                    console.warn(`[HA Queue] Giving up on: ${cmd.label} ${cmd.service} after ${cmd.attempts} attempts`);
                } else {
                    haCommandQueue.push(cmd);
                    console.warn(`[HA Queue] Replay failed (attempt ${cmd.attempts}): ${cmd.label} ${cmd.service} - moved to ba...
                }
            }
        }
    } finally {
        haQueueProcessing = false;
    }
}

async function haServiceCallSafe(service: string, data: object, label = 'HA'): Promise<boolean> {
    try {
        await haServiceCall(service, data, label);
        if (haCommandQueue.length > 0 && !haQueueProcessing) {
            processHACommandQueue().catch(() => {});
        }
        return true;
    } catch (e: any) {
        console.warn(`[HA Safe] ${label} ${service} failed: ${e.message} - queuing for retry`);
        if (haCommandQueue.length < HA_QUEUE_MAX_SIZE) {
            haCommandQueue.push({ service, data, label, queuedAt: Date.now(), attempts: 1 });
        } else {
            console.warn(`[HA Queue] Queue full (${HA_QUEUE_MAX_SIZE}) - dropping: ${label} ${service}`);
        }
        return false;
    }
}

import { easternNow as torontoDate, easternDateStr, easternHour, easternMidnight, taskDateStr, addDays } from './time...

```

```

function formatLocalDate(d: Date): string {
    return `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, '0')}-${String(d.getDate()).padStart(2, '0')}`;
}

interface FlickDevice {
    id: string;
    name: string;
    entityId: string;
    type: "tablet" | "echo" | "echo_show" | "tv" | "speaker" | "group";
    canDisplay: boolean;
    room: string;
}

interface FlickRoomGroup {
    room: string;
    icon: string;
    devices: FlickDevice[];
}

const FLICK_DEVICES: FlickRoomGroup[] = [
    {
        room: "Hallway", icon: "0=p³",
        devices: [
            { id: "hallway_tablet_entrance", name: "Tablet (Entrance)", entityId: "media_player.tablet_hallway_entrance", t...
            { id: "hallway_tablet", name: "Tablet (Main)", entityId: "media_player.tablet_hallway", type: "tablet", canDisp...
            { id: "hallway_echo_entrance", name: "Echo (Entrance)", entityId: "media_player.echo_hallway_entrance_am", type...
            { id: "hallway_group", name: "All Hallway", entityId: "media_player.hallway_media_group", type: "group", canDis...
        ]
    },
    {
        room: "Living Room", icon: "0=pÉp ",
        devices: [
            { id: "lr_tablet", name: "Fire Tablet (11)", entityId: "media_player.tablet_11", type: "tablet", canDisplay: tr...
            { id: "lr_echo_couch_l", name: "Echo (Couch L)", entityId: "media_player.echo_lr_couch_l_am", type: "echo", can...
            { id: "lr_echo_couch_r", name: "Echo (Couch R)", entityId: "media_player.echo_lr_couch_r_am", type: "echo", can...
            { id: "lr_echo_hub", name: "Echo (Hub)", entityId: "media_player.echo_lr_hub_am", type: "echo", canDisplay: fal...
            { id: "lr_echo_studio", name: "Echo Studio (White)", entityId: "media_player.echo_lr_studio_white_am", type: "e...
            { id: "lr_echo_tv_shelf", name: "Echo (TV Shelf)", entityId: "media_player.echo_lr_tv_shelf_am", type: "echo", ...
            { id: "lr_tv", name: "TV (70")", entityId: "media_player.tv_living_room_70", type: "tv", canDisplay: true, roo...
            { id: "lr_group", name: "All Living Room", entityId: "media_player.living_room_media_group", type: "group", can...
        ]
    },
    {
        room: "King Bedroom", icon: "0=pİp ",
        devices: [
            { id: "king_tablet", name: "Tablet", entityId: "media_player.bd24bb29_04a116d8_king", type: "tablet", canDispla...
            { id: "king_echo_l", name: "Echo (Left)", entityId: "media_player.echo_king_l_am", type: "echo", canDisplay: fa...
            { id: "king_echo_r", name: "Echo (Right)", entityId: "media_player.echo_king_r_am", type: "echo", canDisplay: f...
            { id: "king_echo_tv", name: "Echo (TV)", entityId: "media_player.echo_king_tv_am", type: "echo", canDisplay: fa...
            { id: "king_tv", name: "TV", entityId: "media_player.tv_king", type: "tv", canDisplay: true, room: "King Bedroo...
            { id: "king_group", name: "All King Bedroom", entityId: "media_player.king_bedroom_media_group", type: "group",...
        ]
    },
    {
        room: "Queen Bedroom", icon: "0=ÜQ",
        devices: [
            { id: "queen_tablet", name: "Tablet", entityId: "media_player.tablet_queen", type: "tablet", canDisplay: true, ...
            { id: "queen_echo_balcony", name: "Echo (Balcony)", entityId: "media_player.echo_queen_balcony_am", type: "echo...
            { id: "queen_echo_bed_l", name: "Echo (Bed L)", entityId: "media_player.echo_queen_bed_l_am", type: "echo", can...
            { id: "queen_echo_bed_r", name: "Echo (Bed R)", entityId: "media_player.echo_queen_bed_r_am", type: "echo", can...
            { id: "queen_group", name: "All Queen Bedroom", entityId: "media_player.queen_bedroom_media_group", type: "grou...
        ]
    },
    {
        room: "Kitchen", icon: "0<Bs",
        devices: [
            { id: "kitchen_tablet", name: "Tablet (Kitchen Island)", entityId: "media_player.tablet_kitchen_island", type: ...
            { id: "kitchen_echo_cupboards_l", name: "Echo (Cupboards L)", entityId: "media_player.echo_kitchen_cupboards_le...
            { id: "kitchen_echo_cupboards_r", name: "Echo (Cupboards R)", entityId: "media_player.echo_kitchen_cupboards_r_...
            { id: "kitchen_echo_fridge", name: "Echo (Fridge)", entityId: "media_player.echo_kitchen_fridge_am", type: "ech...
            { id: "kitchen_echo_hutch", name: "Echo (Hutch)", entityId: "media_player.echo_kitchen_hutch_am", type: "echo",...
            { id: "kitchen_echo_island", name: "Echo (Island Corner)", entityId: "media_player.echo_kitchen_island_corner_a...
            { id: "kitchen_echo_studio", name: "Echo Studio (Black)", entityId: "media_player.echo_kitchen_studio_black_am"...
            { id: "kitchen_tv", name: "TV", entityId: "media_player.tv_kitchen", type: "tv", canDisplay: true, room: "Kitch...
            { id: "kitchen_group", name: "All Kitchen", entityId: "media_player.kitchen_media_group", type: "group", canDis...
        ]
    },
    {
        room: "Cat Washroom", icon: "0=Ü1",
        devices: [
            { id: "cat_tablet", name: "Tablet", entityId: "media_player.tablet_cat", type: "tablet", canDisplay: true, room...
            { id: "cat_echo_middle", name: "Echo (Middle)", entityId: "media_player.echo_cat_washroom_middle", type: "echo"...
            { id: "cat_echo_left", name: "Echo (Left)", entityId: "media_player.echo_cat_left_am", type: "echo", canDisplay...
            { id: "cat_echo_right", name: "Echo (Right)", entityId: "media_player.echo_cat_right_am", type: "echo", canDisp...
            { id: "cat_nest", name: "Nest Speaker", entityId: "media_player.bathroom_speaker", type: "speaker", canDisplay:...
            { id: "cat_tv", name: "TV", entityId: CAT_TV_ENTITY, type: "tv", canDisplay: true, room: "Cat Washroom" },
            { id: "cat_group", name: "All Cat Washroom", entityId: CAT_WR_MEDIA_GROUP, type: "group", canDisplay: false, ro...
        ]
    },
    {
        room: "Pug Washroom", icon: "0=Ü6",
        devices: [
            { id: "pug_echo_show", name: "Echo Show", entityId: "media_player.echo_show_pug_am", type: "echo_show", canDisp...
            { id: "pug_group", name: "All Pug Washroom", entityId: "media_player.pug_media_group", type: "group", canDispla...
        ]
    },
    {
        room: "Closet", icon: "0=ÜT",
        devices: [
            { id: "closet_echo", name: "Echo", entityId: "media_player.echo_closet_am", type: "echo", canDisplay: false, ro...
            { id: "closet_group", name: "All Closet", entityId: "media_player.closet_media_group", type: "group", canDispla...
        ]
    },
    {
        room: "Everywhere", icon: "0<Bà",
        devices: [
            { id: "everywhere", name: "All Speakers", entityId: EVERYWHERE_GROUP_ENTITY, type: "group", canDisplay: false, ...
        ]
    }
]

```

```

    },
  ];

  // Track travelling state (synced from client) to suppress Echo announcements
  let isTravellingMode = false;
  let travelStartDate: string | null = null;
  let travelEndDate: string | null = null;
  export function getIsTravellingMode(): boolean {
    if (isTravellingMode && travelStartDate && travelEndDate) {
      const now = new Date();
      const start = new Date(travelStartDate);
      const end = new Date(travelEndDate);
      if (!isNaN(start.getTime()) && !isNaN(end.getTime())) {
        return now >= start && now <= end;
      }
    }
    return isTravellingMode;
  }

  // Track TTS reading session for resume functionality
  interface TTSSession {
    fullText: string;
    currentPosition: number;
    startTime: number;
    isPlaying: boolean;
    autoTimer: ReturnType<typeof setTimeout> | null;
    targetEntity?: string;
    consecutiveErrors: number;
    sessionCreatedAt: number;
  }
  let currentTTSSession: TTSSession | null = null;
  const MAX_CONSECUTIVE_ERRORS = 5;
  const MAX_SESSION_AGE_MS = 4 * 60 * 60 * 1000; // 4 hours max session
  // Alexa TTS at 90% speed: ~180 wpm = ~900 chars/min = ~15 chars/sec
  // Use conservative estimate to avoid cutting off speech mid-sentence
  const CHARS_PER_SECOND = 13; // Conservative: let chunk finish before sending next
  const CHUNK_SIZE = 2000; // Characters per TTS chunk

  // Helper to generate OpenAI TTS audio and save to object storage for playback
  async function generateAndSaveTTSAudio(text: string, fileId: string, voice: string = "echo", slowPace: boolean = false) {
    const publicPath = process.env.PUBLIC_OBJECT_SEARCH_PATHS?.split(',').filter(Boolean).join('');
    if (!publicPath) {
      throw new Error("PUBLIC_OBJECT_SEARCH_PATHS not configured");
    }

    // Normalize text for TTS
    let normalizedText = text
      .replace(/https?:\/\/[^\s]+/gi, '')
      .replace(/doi:[^\s]+/gi, '')
      .replace(/[\d+](?:\s*\d+)?/gi, '')
      .replace(/[\A-Z][a-z]+(?:\s+(?:&|and)\s+[\A-Z][a-z]+)*,?\s*\d{4}[a-z]?/gi, '')
      .replace(/pp?\s*\d+(?:\s*[-]\s*\d+)?/gi, '')
      .replace(/[\^]{50}/gi, '')
      .replace(/[-]/gi, '-')
      .replace(/["]/gi, '"')
      .replace(/[']/gi, "'")
      .replace(/\\s+/gi, ' ')
      .trim();

    console.log(`Generating OpenAI TTS for ${normalizedText.length} chars, file: ${fileId}`);

    const ttsStart = Date.now();
    const audioBuffer = await textToSpeech(normalizedText, voice as "alloy" | "echo" | "fable" | "onyx" | "nova" | "shi...");
    console.log(`OpenAI TTS completed in ${Date.now() - ttsStart}ms, ${audioBuffer.length} bytes`);

    if (audioBuffer.length === 0) {
      throw new Error(`TTS returned empty audio buffer for file: ${fileId}`);
    }

    const audioFileName = `tts-audio/${fileId}-${Date.now()}.mp3`;
    const { bucketName, objectName } = parsePublicObjectPath(`${publicPath}/${audioFileName}`);

    const bucket = objectStorageClient.bucket(bucketName);
    const file = bucket.file(objectName);

    await new Promise<void>((resolve, reject) => {
      const stream = file.createWriteStream({
        contentType: 'audio/mpeg',
        metadata: {
          cacheControl: 'public, max-age=3600',
        },
        resumable: false,
      });
      stream.on('finish', () => resolve());
      stream.on('error', (err: any) => reject(err));
      stream.end(audioBuffer);
    });

    // Return a proxy URL through the app (object storage public access is blocked)
    const proxyUrl = `/api/tts-audio/${encodeURIComponent(audioFileName)}`;
    console.log(`TTS audio saved, proxy path: ${proxyUrl}`);
    return proxyUrl;
  }

  // Parse public object path to bucket/object name
  function parsePublicObjectPath(path: string): { bucketName: string; objectName: string } {
    if (!path.startsWith("/")) path = `/${path}`;
    const parts = path.split("/").filter(Boolean);
    if (parts.length < 2) throw new Error("Invalid path");
    return { bucketName: parts[0], objectName: parts.slice(1).join("/") };
  }

  // Clean text for TTS - remove special characters that cause errors
  function cleanTextForTTS(text: string): string {
    console.log("CleanTextForTTS input length:", text.length);

    // First pass: Remove URLs, emails, video/audio references, timestamps
  
```



```

        .replace(/\([A-Z][a-z]+(?:\s+(?:and|&)\s+[A-Z][a-z]+)*(?:\s*(?:et\s+al\.\.?)?)?\s*\d{4}[a-z]?(?:\s*p{1,2}\.\s*\.\.\.
        .replace(/\([A-Z][a-z]+(?:\s+(?:and|&)\s+[A-Z][a-z]+)*\s*n\.\d\.(?:\s*p{1,2}\.\s*\d+(?:-\d+)??)\)/g, ' ');

// Remove standalone citation-style lines (Author, Year. Title. Journal...)
cleanedText = cleanedText
    .replace(/\^[A-Z][a-z]+\s+[A-Z]\.\.(?:\s*[A-Z]\.)*\s+\(\d{4}\)\.\.*$/gm, ' ')
    .replace(/\^[A-Z][a-z]+\s+[A-Z]\.\.(?:\s*[A-Z]\.)*\s+(?:and|&)\s+[A-Z][a-z]+\s+[A-Z]\.\.*\(\d{4}\)\.\.*$/gm, ' ')
    .replace(/\^[A-Z][a-z]+\s+[A-Z]\.\s+\(\d{4}\)\s+\w+\s+\d+\)\.\.*$/gm, ' ');

// Remove dense citation blocks that survive other filters (sequences of Author, Year patterns)
cleanedText = cleanedText
    .replace(/(?:[A-Z][a-z]+\s+[A-Z]\.\.(?:\s*[A-Z]\.)*\s*(?:\s*(?:&|and)\s+[A-Z][a-z]+\s+[A-Z]\.\.(?:\s*[A-Z]\.)*\s*\.\.\.

// Remove section headings (standalone or at start of lines followed by content)
cleanedText = cleanedText
    .replace(/\^(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Foreword|Acknowledgements?|References|Bibli...
    .replace(/\^(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Foreword|Acknowledgements?|References|Bibli...

// Remove tab/accordion UI artifacts from PDFs (e.g. "Tab 1", "tab expand", "Tab Panel", "Tab Panels", etc.)
// These are navigation elements from D2L/Brightspace LMS that get embedded when saving course pages as PDF
cleanedText = cleanedText
    .replace(/\bTab\s*Panels?\b/g, ' ')
    .replace(/\btabs\s*panels?\b/gi, ' ')
    .replace(/\btabs\d+\b/gi, ' ')
    .replace(/\btabs\+expand\b/gi, ' ')
    .replace(/\btabs\+collapse\b/gi, ' ')
    .replace(/\btabs\+selected\b/gi, ' ')
    .replace(/\btabs\+unselected\b/gi, ' ')
    .replace(/\bselected\s+tab\b/gi, ' ')
    .replace(/\bcurrent\s+tab\b/gi, ' ')
    .replace(/\bexpand\s+tab\b/gi, ' ')
    .replace(/\bcollapse\s+tab\b/gi, ' ')
    .replace(/\btabs?\s*/gi, ' ')
    .replace(/\bexpand\s+all\b/gi, ' ')
    .replace(/\bcollapse\s+all\b/gi, ' ')
    .replace(/\(expanded\)\/gi, ' ')
    .replace(/\(collapsed\)\/gi, ' ')
    .replace(/\(selected\)\/gi, ' ')
    .replace(/\(unselected\)\/gi, ' ')
    .replace(/\(active\)\/gi, ' ')
    .replace(/\(inactive\)\/gi, ' ')
    .replace(/\bTab\b/g, ' ')
    .replace(/\btabs\b/g, ' ');

cleanedText = cleanedText
    .replace(/x{3,}/gi, ' ')
    .replace(/(?:X[\s,;]+){2,}X?\b/g, ' ')
    .replace(/(?:\bX\b[\s,;]*){3,}/g, ' ')
    .replace(/\s+X(?:\s+[A-Z])\s*$/g, ' ')
    .replace(/\b(?:AB|BC|MB|NB|NL|NS|ON|PE|I)?|QC|SK(?:\s+[\s,;]+(?:AB|BC|MB|NB|NL|NS|ON|PE|I)?|QC|SK)){1,}\b/g, ' ')
    .replace(/\bMunicipal Responsibility\s+(?:NL|PEI?|NS|NB|QC|ON|MB|SK|AB|BC)[\s\w]*(?:AB|BC)\b/g, ' ')
    .replace(/\s*(?:AB|BC|MB|NB|NL|NS|ON|PE|I)?|QC|SK\s+(?:AB|BC|MB|NB|NL|NS|ON|PE|I)?|QC|SK)\.*/gm, ' ');

// Remove spaced-out headings like "ar e po l i t i c a l p a r t i e s" (single letters with spaces)
cleanedText = cleanedText
    .replace(/(?:[a-zA-Z]\s+){5,}[a-zA-Z]/g, ' ');

// Remove inline footnote numbers (superscript references like 65, 66, 67 appearing mid-sentence)
cleanedText = cleanedText
    .replace(/(?:<=w)(\d{1,3})(?:\s+[A-Z])/g, ' ')
    .replace(/\b\d{1,2}\s+(?:See|Ibid|Op\.\.?s*cit|Supra|Infra)\b.*$/gm, ' ');

// Final cleanup
let result = cleanedText
    .replace(/&/g, 'and')
    .replace(/&/g, 'and')
    .replace(/<=]/g, ' ')
    .replace(/[\^w\s.,!?:'"()-]/g, ' ')
    .replace(/\s+/g, ' ')
    .trim();

// Post-cleanup: remove heading words that ended up inline after whitespace collapse
result = result
    .replace(/(\.\s+)(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Foreword|Acknowledgements?|References...
    .replace(/\^(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Foreword|Acknowledgements?|References|Bibli...

console.log("Final cleaned length:", result.length);
return result;
}

// Get chunk with sentence boundary detection
function getChunkWithSentenceBoundary(text: string, maxLength: number): string {
    if (text.length <= maxLength) return text;

    let cutoff = maxLength;
    const lastPeriod = text.lastIndexOf('.', cutoff);
    const lastQuestion = text.lastIndexOf('?', cutoff);
    const lastExclaim = text.lastIndexOf('!', cutoff);
    const bestBreak = Math.max(lastPeriod, lastQuestion, lastExclaim);

    if (bestBreak > maxLength * 0.5) {
        cutoff = bestBreak + 1;
    }

    return text.substring(0, cutoff);
}

// Function to fully stop and clean up TTS session
function stopTTSSession(reason: string) {
    console.log(`[TTS] Stopping session: ${reason}`);
    if (currentTTSSession) {
        if (currentTTSSession.autoTimer) {
            clearTimeout(currentTTSSession.autoTimer);
            currentTTSSession.autoTimer = null;
        }
        currentTTSSession.isPlaying = false;
    }
}
}

```

```

// Function to send next TTS chunk automatically
async function sendNextChunk() {
  if (!currentTTSSession || !currentTTSSession.isPlaying) {
    console.log("[TTS] sendNextChunk: No active session or not playing");
    return;
  }

  // Safety: check session age to prevent zombie sessions
  const sessionAge = Date.now() - currentTTSSession.sessionCreatedAt;
  if (sessionAge > MAX_SESSION_AGE_MS) {
    stopTTSSession(`Session too old (${Math.round(sessionAge / 60000)} minutes)`);
    return;
  }

  // Safety: check consecutive errors
  if (currentTTSSession.consecutiveErrors >= MAX_CONSECUTIVE_ERRORS) {
    stopTTSSession(`Too many consecutive errors (${currentTTSSession.consecutiveErrors})`);
    return;
  }

  console.log("[TTS] sendNextChunk called, currentPosition:", currentTTSSession.currentPosition);

  // Check if we've finished
  if (currentTTSSession.currentPosition >= currentTTSSession.fullText.length) {
    stopTTSSession("Finished entire document");
    return;
  }

  // Get next chunk from cleaned text
  let rawChunk = currentTTSSession.fullText.substring(
    currentTTSSession.currentPosition,
    currentTTSSession.currentPosition + CHUNK_SIZE
  );

  if (rawChunk.trim().length === 0) {
    stopTTSSession("No more content");
    return;
  }

  // Clean the chunk and apply sentence boundary
  let nextChunk = cleanTextForTTS(rawChunk);
  nextChunk = getChunkWithSentenceBoundary(nextChunk, CHUNK_SIZE);

  // Update position BEFORE sending - advance by the chunk length we're about to send
  const chunkLength = nextChunk.length;
  currentTTSSession.currentPosition += chunkLength;
  currentTTSSession.startTime = Date.now();

  const targetEntity = currentTTSSession.targetEntity || NEST_SPEAKER_ENTITY;
  const isNonAlexa = NON_ALEXA_ENTITIES.includes(targetEntity);
  console.log("[TTS] Auto-continuing, chunk length:", chunkLength,
    "new position:", currentTTSSession.currentPosition,
    "remaining:", currentTTSSession.fullText.length - currentTTSSession.currentPosition,
    "to:", targetEntity, isNonAlexa ? "(non-Alexa, using play_media)" : "(Alexa)");

  const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

  try {
    let response: Response;

    if (!isNonAlexa) {
      const audioPath = await generateAndSaveTTSAudio(nextChunk, `tts-chunk-${Date.now()}`, "echo");
      const appUrl = DEPLOYED_APP_URL;
      const fullAudioUrl = `${appUrl}${audioPath}`;
      console.log("[TTS] Non-Alexa: Generated audio at ${audioPath}, playing on ${targetEntity}");

      response = await fetch(`${haUrl}/api/services/media_player/play_media`, {
        method: 'POST',
        headers: {
          'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({
          entity_id: targetEntity,
          media_content_id: fullAudioUrl,
          media_content_type: "music",
        }),
      });
    } else {
      const ssmlChunk = `<prosody rate="90%">${nextChunk}</prosody></speak>`;

      response = await fetch(`${haUrl}/api/services/notify/alexa_media`, {
        method: 'POST',
        headers: {
          'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({
          message: ssmlChunk,
          target: targetEntity,
          data: { type: "tts" }
        }),
      });
    }

    if (!response.ok) {
      const errorText = await response.text();
      console.error("[TTS] Chunk send error, status:", response.status, errorText);
      currentTTSSession.consecutiveErrors++;
      if (currentTTSSession.consecutiveErrors >= MAX_CONSECUTIVE_ERRORS) {
        stopTTSSession(`Giving up after ${MAX_CONSECUTIVE_ERRORS} consecutive errors`);
        return;
      }
      // Rewind position so the failed chunk gets retried
      currentTTSSession.currentPosition -= chunkLength;
      console.log("[TTS] Rewound position to ${currentTTSSession.currentPosition} for retry");
    } else {

```

```

    currentTTSSession.consecutiveErrors = 0;
    console.log("[TTS] Chunk sent successfully");
}

// Schedule next chunk only if session is still active and healthy
if (currentTTSSession && currentTTSSession.isPlaying &&
    currentTTSSession.currentPosition < currentTTSSession.fullText.length) {
    // On error, wait longer before retrying to let transient issues resolve
    if (currentTTSSession.consecutiveErrors > 0) {
        console.log(`[TTS] Retry in 10s due to error (attempt ${currentTTSSession.consecutiveErrors}/${MAX_CONSECUTIV...
        currentTTSSession.autoTimer = setTimeout(() => {
            console.log("[TTS] Retry timer fired, calling sendNextChunk");
            sendNextChunk();
        }, 10000);
    } else {
        scheduleNextChunk();
    }
} else {
    console.log("[TTS] Not scheduling next - session ended or no more content");
}
} catch (error) {
    console.error("[TTS] Auto-continue error:", error);
    if (currentTTSSession) {
        currentTTSSession.consecutiveErrors++;
        // Rewind position for retry
        currentTTSSession.currentPosition -= chunkLength;
        if (currentTTSSession.consecutiveErrors >= MAX_CONSECUTIVE_ERRORS) {
            stopTTSSession(`Network error, giving up after ${MAX_CONSECUTIVE_ERRORS} consecutive errors`);
            return;
        }
        // Retry after delay
        console.log(`[TTS] Network error retry in 10s (attempt ${currentTTSSession.consecutiveErrors}/${MAX_CONSECUTIVE...
        currentTTSSession.autoTimer = setTimeout(() => {
            console.log("[TTS] Network retry timer fired");
            sendNextChunk();
        }, 10000);
    }
}
}

// Calculate delay based on chunk size and speed
// At 90% speed (slightly slower than normal)
const SPEED_RATE = 0.90;

function scheduleNextChunk() {
    if (!currentTTSSession || !currentTTSSession.isPlaying) {
        console.log("[TTS] scheduleNextChunk: No active session or not playing, aborting");
        return;
    }

    // Safety: don't schedule if too many errors
    if (currentTTSSession.consecutiveErrors >= MAX_CONSECUTIVE_ERRORS) {
        stopTTSSession("Too many errors, not scheduling more chunks");
        return;
    }

    // Clear any existing timer
    if (currentTTSSession.autoTimer) {
        clearTimeout(currentTTSSession.autoTimer);
        currentTTSSession.autoTimer = null;
    }

    const baseSeconds = CHUNK_SIZE / CHARS_PER_SECOND;
    const adjustedSeconds = baseSeconds / SPEED_RATE;
    // Add 3-second buffer for Alexa processing overhead
    const delayMs = adjustedSeconds * 1000 + 3000;

    console.log(`[TTS] Scheduling next chunk in ${(delayMs / 1000).toFixed(1)}s`);

    currentTTSSession.autoTimer = setTimeout(() => {
        console.log("[TTS] Timer fired, calling sendNextChunk");
        sendNextChunk();
    }, delayMs);
}

const BUILD_VERSION = Date.now().toString();

export async function registerRoutes(
    httpServer: Server,
    app: Express
): Promise<Server> {
    await initTTSTFallbackStatus();

    const serverPort = process.env.PORT || 5000;

    setTimeout(async () => {
        try {
            const nestPlaying = await isNestSpeakerPlaying();
            if (nestPlaying) {
                console.log(`[Startup] Nest speaker is playing – starting toothbrush polling for orphaned session recovery`);
                startToothbrushPolling();
            }
        } catch (e: any) {
            console.log(`[Startup] Error checking Nest state: ${e.message}`);
        }
    }, 10000);

    (async () => {
        try {
            const allTasks = await storage.getTasks({});
            const autoCreatedIds = allTasks
                .filter(t => t.type === 'module' || t.type === 'reading')
                .filter(t => !t.isCompleted)
                .filter(t => {
                    const title = t.title || '';
                    return /^(CPPA|CFNF|CASL\d*)\s+(Module|Reading|Module & Reading)$/.test(title)
                        || /^Module( & Reading)?$/i.test(title)
                });
        }
    })();
}

```

```

    || /^Reading$/.test(title);
  })
  .map(t => t.id);
  if (autoCreatedIds.length > 0) {
    for (const id of autoCreatedIds) {
      await storage.deleteTask(id);
    }
    console.log(`[Cleanup] Removed ${autoCreatedIds.length} auto-created module/reading tasks`);
  }
} catch (e) {
  console.error('[Cleanup] Error removing auto-created tasks:', e);
}
})();

try {
  const fixResult = await db.execute(sql`UPDATE files SET folder = REPLACE(folder, 'casl101-other', 'casl101-module...
  const count = (fixResult as any)?.rowCount || (fixResult as any)?.changes || 0;
  if (count > 0) {
    console.log(`Fixed ${count} CASL101 file folder(s) from 'other' to 'module'`);
  }
} catch (e) {
  console.error("Failed to fix CASL101 file folders:", e);
}

app.get('/api/degree-tracking', async (_req, res) => {
  try {
    const rows = await db.select().from(degreeTrackingData);
    const result: Record<string, any> = {};
    for (const row of rows) {
      try { result[row.key] = JSON.parse(row.value); } catch { result[row.key] = row.value; }
    }
    res.json(result);
  } catch (e) {
    res.status(500).json({ error: 'Failed to load degree tracking data' });
  }
});

app.post('/api/degree-tracking', async (req, res) => {
  try {
    const { key, value } = req.body;
    if (!key) return res.status(400).json({ error: 'key is required' });
    const valueStr = typeof value === 'string' ? value : JSON.stringify(value);
    await db.insert(degreeTrackingData).values({ key, value: valueStr }).onConflictDoUpdate({ target: degreeTrackin...
    res.json({ ok: true });
  } catch (e) {
    res.status(500).json({ error: 'Failed to save degree tracking data' });
  }
});

app.post('/api/degree-tracking/bulk', async (req, res) => {
  try {
    const data = req.body;
    if (!data || typeof data !== 'object') return res.status(400).json({ error: 'Object required' });
    let saved = 0;
    for (const [key, value] of Object.entries(data)) {
      const valueStr = typeof value === 'string' ? value : JSON.stringify(value);
      await db.insert(degreeTrackingData).values({ key, value: valueStr }).onConflictDoUpdate({ target: degreeTrack...
      saved++;
    }
    res.json({ ok: true, saved });
  } catch (e) {
    res.status(500).json({ error: 'Failed to bulk save degree tracking data' });
  }
});

app.get('/api/ha-redirect', (req, res) => {
  const path = req.query.path ? decodeURIComponent(String(req.query.path)) : '/lovelace/test-home';
  const haUrl = `http://172.24.0.2:8123${path}`;
  res.setHeader('Content-Type', 'text/html');
  res.setHeader('Cache-Control', 'no-store');
  res.send(`<!DOCTYPE html><html><head><meta http-equiv="refresh" content="0;url=${haUrl}></script>window.location....
});

app.get('/api/version', (_req, res) => {
  res.json({ version: BUILD_VERSION });
});

app.get('/tablet', (req, res) => {
  const baseUrl = DEPLOYED_APP_URL;
  const targetUrl = req.query.target ? decodeURIComponent(String(req.query.target)) : `${baseUrl}/?auth=5747`;
  res.setHeader('Cache-Control', 'no-store, no-cache, must-revalidate');
  res.setHeader('Content-Type', 'text/html');
  res.send(`<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,initial-scale=1,maximum-scale=1,user-scalable=no">
<meta name="mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-capable" content="yes">
<title>Uni-Cal Tablet</title>
<style>
*{margin:0;padding:0;box-sizing:border-box}
html,body{width:100%;height:100%;overflow:hidden;background:#000}
iframe{width:100vw;height:100vh;border:none;position:fixed;top:0;left:0}
</style>
</head>
<body>
<iframe id="frame" src="${targetUrl}" allow="fullscreen;autoplay" allowfullscreen></iframe>
<script>
(function(){
  var frame = document.getElementById('frame');
  var lastTs = 0;

  function goFullscreen() {
    var el = document.documentElement;
    try {
      if (el.requestFullscreen) el.requestFullscreen();
      else if (el.webkitRequestFullscreen) el.webkitRequestFullscreen();
    }
  }
});

```



```

        else if (el.mozRequestFullScreen) el.mozRequestFullScreen();
    } catch(e){}
}
document.addEventListener('click', goFullscreen);
document.addEventListener('touchstart', goFullscreen);
setTimeout(goFullscreen, 500);
setTimeout(goFullscreen, 2000);
setTimeout(goFullscreen, 5000);

function poll() {
    var url = '${baseUrl}/api/tablet-nav?device=master&auth=5747&t=' + Date.now();
    fetch(url, {cache:'no-store'}).then(function(r){return r.json()}).then(function(data){
        if (!data || !data.action) return;
        if (data.timestamp && data.timestamp <= lastTs) return;
        if (data.timestamp && (Date.now() - data.timestamp > 120000)) return;
        lastTs = data.timestamp || Date.now();
        fetch('${baseUrl}/api/tablet-nav/ack', {method:'POST',headers:{'Content-Type':'application/json'},body:JSON.str...
        fetch('${baseUrl}/api/debug-beacon', {method:'POST',headers:{'Content-Type':'application/json'},body:JSON.strin...
        if (data.action === 'navigate' && data.url) {
            frame.src = data.url;
        } else if (data.action === 'go_home') {
            frame.src = '${baseUrl}/?auth=5747';
        }
    }).catch(function(){});
}
setInterval(poll, 3000);
setTimeout(poll, 500);

fetch('${baseUrl}/api/debug-beacon', {method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify...
})();
</script>
</body>
</html>`;
});

app.post('/api/client-error', (req, res) => {
    const { message, stack, userAgent, url, timestamp } = req.body || {};
    console.error(`[CLIENT ERROR] ${timestamp || new Date().toISOString()} | UA: ${userAgent || 'unknown'} | URL: ${u...
    res.json({ ok: true });
});

app.post('/api/debug-beacon', (req, res) => {
    const { step, data } = req.body || {};
    console.log(`[DEBUG-BEACON] ${step}: ${JSON.stringify(data || {})} `);
    res.json({ ok: true });
});

// GET /api/tasks
app.get(api.tasks.list.path, async (req, res) => {
    const weekNumber = req.query.weekNumber ? Number(req.query.weekNumber) : undefined;
    const type = req.query.type as string | undefined;
    const showCompleted = req.query.showCompleted !== 'false';

    const tasks = await storage.getTasks({ weekNumber, type, showCompleted });

    // Mark missed tasks and add subtask counts
    const now = new Date();
    const tasksWithExtras = await Promise.all(tasks.map(async (task) => {
        const subtasks = await storage.getSubtasksByTask(task.id);
        const subtaskCount = subtasks.length;
        const completedSubtaskCount = subtasks.filter(s => s.isCompleted).length;

        return {
            ...task,
            isMissed: !task.isCompleted && new Date(task.dueDate) < now,
            subtaskCount,
            completedSubtaskCount
        };
    }));
    res.json(tasksWithExtras);
});

// GET /api/tasks/:id
app.get(api.tasks.get.path, async (req, res) => {
    const task = await storage.getTask(Number(req.params.id));
    if (!task) {
        return res.status(404).json({ message: 'Task not found' });
    }
    res.json(task);
});

// POST /api/tasks
app.post(api.tasks.create.path, async (req, res) => {
    try {
        const input = api.tasks.create.input.parse(req.body);
        if (!input.startDate && input.dueDate) {
            const due = new Date(input.dueDate);
            const now = new Date();
            now.setHours(0, 0, 0, 0);
            if (due.getTime() > now.getTime()) {
                input.startDate = now.toISOString();
            }
        }
        const task = await storage.createTask(input);

        // Auto-sync to Google Calendar (primary account)
        try {
            const event = await createCalendarEvent({
                id: task.id,
                title: task.title,
                description: task.description,
                dueDate: task.dueDate,
                courseName: task.courseName,
            });
            // Update task with calendar event ID
            await storage.updateTask(task.id, {
                calendarEventId: event.id,
            });
        } catch (e) {
            // Ignore calendar sync errors
        }
    } catch (e) {
        // Ignore other errors
    }
});

```

```

    calendarProvider: "google",
  });

  // Also sync to secondary calendar if configured
  const activeSemester = await storage.getActiveSemesterSettings();
  if (activeSemester?.secondaryCalendarId) {
    try {
      const secondaryEvent = await createEventInCalendar(activeSemester.secondaryCalendarId, {
        id: task.id,
        title: task.title,
        description: task.description,
        dueDate: task.dueDate,
        courseName: task.courseName,
      });
      await storage.updateTask(task.id, {
        secondaryCalendarEventId: secondaryEvent.id,
      });
    } catch (secErr) {
      console.error("Auto-sync to secondary calendar failed:", secErr);
    }
  }
} catch (calErr) {
  console.error("Auto-sync to Google Calendar failed:", calErr);
}

// Also sync to second Google account if connected
try {
  const secondAccountStatus = await isSecondAccountConnected();
  if (secondAccountStatus.connected) {
    const secondEvent = await createEventInSecondAccount({
      id: task.id,
      title: task.title,
      description: task.description,
      dueDate: task.dueDate,
      courseName: task.courseName,
    });
    await storage.updateTask(task.id, {
      secondAccountCalendarEventId: secondEvent.id,
    });

    // Also sync prep event if task has startDate
    if (task.startDate) {
      const prepEvent = await createPrepEventInSecondAccount({
        id: task.id,
        title: task.title,
        description: task.description,
        startDate: task.startDate,
        dueDate: task.dueDate,
        courseName: task.courseName,
      });
      await storage.updateTask(task.id, {
        secondAccountPrepEventId: prepEvent.id,
      });
    }
  }
} catch (secAccErr) {
  console.error("Auto-sync to second Google account failed:", secAccErr);
}

// Generate repeated task instances if repeat is set
if (task.repeatType && task.repeatType !== "none") {
  const activeSemester = await storage.getActiveSemesterSettings();
  const semesterStart = activeSemester ? new Date(activeSemester.semesterStartDate) : undefined;

  const repeatDates = generateRepeatDates(
    task.dueDate,
    task.repeatType as RepeatType,
    task.repeatEndDate,
    task.repeatInterval ?? undefined,
    task.repeatIntervalUnit as RepeatIntervalUnit | undefined
  );

  // Calculate the duration of prep period (if any)
  let prepDuration = 0;
  if (task.startDate) {
    prepDuration = task.dueDate.getTime() - task.startDate.getTime();
  }

  // Create child tasks for each repeat date
  for (const repeatDueDate of repeatDates) {
    const childStartDate = prepDuration > 0
      ? new Date(repeatDueDate.getTime() - prepDuration)
      : null;

    const childTask: InsertTask = {
      title: task.title,
      description: task.description,
      type: task.type,
      courseName: task.courseName,
      startDate: childStartDate,
      dueDate: repeatDueDate,
      eventStartTime: task.eventStartTime,
      eventEndTime: task.eventEndTime,
      reminder1: task.reminder1,
      reminder2: task.reminder2,
      reminder3: task.reminder3,
      reminder4: task.reminder4,
      weekNumber: getWeekNumber(repeatDueDate, semesterStart, activeSemester?.readingWeekStart),
      priority: task.priority,
      notes: task.notes,
      referenceLink: task.referenceLink,
      attachments: task.attachments,
      repeatType: "none", // Child tasks don't repeat
      parentTaskId: task.id,
    };

    try {
      const createdChild = await storage.createTask(childTask);
    }
  }
}

```

```

    // Sync child to calendar too
    const childEvent = await createCalendarEvent({
      id: createdChild.id,
      title: createdChild.title,
      description: createdChild.description,
      dueDate: createdChild.dueDate,
      courseName: createdChild.courseName,
    });
    await storage.updateTask(createdChild.id, {
      calendarEventId: childEvent.id,
      calendarProvider: "google",
    });

    // Also sync child to second Google account
    const secondStatus = await isSecondAccountConnected();
    if (secondStatus.connected) {
      try {
        const childSecondEvent = await createEventInSecondAccount({
          id: createdChild.id,
          title: createdChild.title,
          description: createdChild.description,
          dueDate: createdChild.dueDate,
          courseName: createdChild.courseName,
        });
        await storage.updateTask(createdChild.id, {
          secondAccountCalendarEventId: childSecondEvent.id,
        });
      } catch (secErr) {
        console.error("Error syncing child task to second account:", secErr);
      }
    } catch (childErr) {
      console.error("Error creating repeated task instance:", childErr);
    }
  }
}

// Fetch the updated parent task
const updatedTask = await storage.getTask(task.id);
res.status(201).json(updatedTask);
} catch (err) {
  if (err instanceof z.ZodError) {
    return res.status(400).json({
      message: err.errors[0].message,
      field: err.errors[0].path.join('.'),
    });
  }
  throw err;
}
});

app.post("/api/tasks/bulk-import", async (req, res) => {
  try {
    const { tasks: incoming } = req.body as { tasks: any[] };
    if (!Array.isArray(incoming)) return res.status(400).json({ message: 'tasks must be an array' });

    const existing = await storage.getTasks({});
    const existingMap = new Map<string, any>();
    existing.forEach(t => {
      const key = `${t.courseName}||${t.title}||${t.type}||${t.weekNumber} || ''`;
      existingMap.set(key, t);
    });

    let created = 0, updated = 0, skipped = 0;
    for (const t of incoming) {
      const key = `${t.courseName}||${t.title}||${t.type}||${t.weekNumber} || ''`;
      const { id, isMissed, subtaskCount, completedSubtaskCount, calendarEventId, calendarProvider, prepCalendarEve...
      if (taskData.dueDate && typeof taskData.dueDate === 'string') taskData.dueDate = new Date(taskData.dueDate);
      if (taskData.startDate && typeof taskData.startDate === 'string') taskData.startDate = new Date(taskData.star...
      if (taskData.completedAt && typeof taskData.completedAt === 'string') taskData.completedAt = new Date(taskDat...
      if (taskData.repeatEndDate && typeof taskData.repeatEndDate === 'string') taskData.repeatEndDate = new Date(t...

      const existingTask = existingMap.get(key);
      if (existingTask) {
        if (taskData.gradeWeight !== undefined || taskData.gradeValue !== undefined || taskData.gradeTotal !== unde...
          await storage.updateTask(existingTask.id, {
            gradeWeight: taskData.gradeWeight,
            gradeValue: taskData.gradeValue,
            gradeTotal: taskData.gradeTotal,
            assignmentGroup: taskData.assignmentGroup,
            sortOrder: taskData.sortOrder,
            dueDate: taskData.dueDate,
          });
          updated++;
        } else {
          skipped++;
        }
      } else {
        try {
          if (!taskData.startDate && taskData.dueDate) {
            const due = new Date(taskData.dueDate);
            const now = new Date();
            now.setHours(0, 0, 0, 0);
            if (due.getTime() > now.getTime()) {
              taskData.startDate = now;
            }
          }
          await storage.createTask(taskData);
          created++;
        } catch (e: any) {
          console.error('Bulk import failed for "${t.title}":', e.message);
          skipped++;
        }
      }
    }
  }
}

res.json({ success: true, created, updated, skipped, total: incoming.length });
} catch (err: any) {

```

```

    res.status(500).json({ message: err.message });
  }
});

app.post("/api/tasks/reorder", async (req, res) => {
  try {
    const { updates } = req.body as { updates: { id: number; sortOrder: number; assignmentGroup?: string | null }[] };
    if (!Array.isArray(updates)) return res.status(400).json({ message: 'updates must be an array' });
    for (const u of updates) {
      await storage.updateTask(u.id, { sortOrder: u.sortOrder, assignmentGroup: u.assignmentGroup ?? undefined });
    }
    res.json({ success: true });
  } catch (err: any) {
    res.status(500).json({ message: err.message });
  }
});

// PATCH /api/tasks/:id
app.patch(api.tasks.update.path, async (req, res) => {
  try {
    const input = api.tasks.update.input.parse(req.body);
    const existingTask = await storage.getTask(Number(req.params.id));
    const task = await storage.updateTask(Number(req.params.id), input);
    if (!task) {
      return res.status(404).json({ message: 'Task not found' });
    }

    // Auto-sync to Google Calendar if task has calendar event
    if (task.calendarEventId) {
      try {
        await updateCalendarEvent(task.calendarEventId, {
          title: task.title,
          description: task.description,
          dueDate: task.dueDate,
          courseName: task.courseName,
        });
      } catch (calErr) {
        console.error("Auto-update Google Calendar failed:", calErr);
      }
    }

    // Also sync to second Google account if connected and has event
    if (task.secondAccountCalendarEventId) {
      try {
        await updateEventInSecondAccount(task.secondAccountCalendarEventId, {
          title: task.title,
          description: task.description,
          dueDate: task.dueDate,
          courseName: task.courseName,
        });
      } catch (secAccErr) {
        console.error("Auto-update second Google account failed:", secAccErr);
      }
    }
  }

  // If repeatType changed from "none" to something else, generate recurring instances
  if (existingTask && (!existingTask.repeatType || existingTask.repeatType === "none") && task.repeatType && task...)
  try {
    const activeSemester = await storage.getActiveSemesterSettings();
    const semesterStart = activeSemester ? new Date(activeSemester.semesterStartDate) : undefined;

    const repeatDates = generateRepeatDates(
      task.dueDate,
      task.repeatType as RepeatType,
      task.repeatEndDate,
      task.repeatInterval ?? undefined,
      task.repeatIntervalUnit as RepeatIntervalUnit | undefined
    );

    let prepDuration = 0;
    if (task.startDate) {
      prepDuration = task.dueDate.getTime() - task.startDate.getTime();
    }

    for (const repeatDueDate of repeatDates) {
      const childStartDate = prepDuration > 0
        ? new Date(repeatDueDate.getTime() - prepDuration)
        : null;

      const childTask: InsertTask = {
        title: task.title,
        description: task.description,
        type: task.type,
        courseName: task.courseName,
        startDate: childStartDate,
        dueDate: repeatDueDate,
        eventStartTime: task.eventStartTime,
        eventEndTime: task.eventEndTime,
        reminder1: task.reminder1,
        reminder2: task.reminder2,
        reminder3: task.reminder3,
        reminder4: task.reminder4,
        weekNumber: getWeekNumber(repeatDueDate, semesterStart, activeSemester?.readingWeekStart),
        priority: task.priority,
        notes: task.notes,
        referenceLink: task.referenceLink,
        attachments: task.attachments,
        repeatType: "none",
        parentTaskId: task.id,
      };

      try {
        const createdChild = await storage.createTask(childTask);
        const childEvent = await createCalendarEvent({
          id: createdChild.id,
          title: createdChild.title,
          description: createdChild.description,
          dueDate: createdChild.dueDate,

```

```

        courseName: createdChild.courseName,
    });
    await storage.updateTask(createdChild.id, {
        calendarEventId: childEvent.id,
        calendarProvider: "google",
    });
    } catch (childErr) {
        console.error("Failed to create recurring child from PATCH:", childErr);
    }
    } catch (recurErr) {
        console.error("Error generating recurring instances from PATCH:", recurErr);
    }
}

// Handle prep calendar event changes
// If startDate was removed (had prep days, now has 0), delete prep events
if (existingTask?.startDate && !task.startDate) {
    // Delete main prep event
    if (existingTask.prepCalendarEventId) {
        try {
            await deleteCalendarEvent(existingTask.prepCalendarEventId);
            await storage.updateTask(task.id, { prepCalendarEventId: null });
        } catch (prepErr) {
            console.error("Failed to delete prep calendar event:", prepErr);
        }
    }
    // Delete second account prep event
    if (existingTask.secondAccountPrepEventId) {
        try {
            await deleteEventFromSecondAccount(existingTask.secondAccountPrepEventId);
            await storage.updateTask(task.id, { secondAccountPrepEventId: null });
        } catch (secPrepErr) {
            console.error("Failed to delete second account prep event:", secPrepErr);
        }
    }
}
// If startDate was added or changed, update/create prep events
else if (task.startDate && task.prepCalendarEventId) {
    try {
        await updatePrepCalendarEvent(task.prepCalendarEventId, {
            title: task.title,
            description: task.description,
            startDate: task.startDate,
            dueDate: task.dueDate,
            courseName: task.courseName,
        });
    } catch (prepErr) {
        console.error("Failed to update prep calendar event:", prepErr);
    }
}

res.json(task);
} catch (err) {
    if (err instanceof z.ZodError) {
        return res.status(400).json({
            message: err.errors[0].message,
            field: err.errors[0].path.join('.'),
        });
    }
    throw err;
});
});

// DELETE /api/tasks/:id
app.delete(api.tasks.delete.path, async (req, res) => {
    const taskId = Number(req.params.id);
    const task = await storage.getTask(taskId);

    if (!task) {
        return res.status(404).json({ message: 'Task not found' });
    }

    // Delete from Google Calendar if synced
    if (task.calendarEventId) {
        try {
            await deleteCalendarEvent(task.calendarEventId);
        } catch (calErr) {
            console.error("Auto-delete from Google Calendar failed:", calErr);
        }
    }

    // Delete prep calendar event if synced
    if (task.prepCalendarEventId) {
        try {
            await deleteCalendarEvent(task.prepCalendarEventId);
        } catch (prepErr) {
            console.error("Auto-delete prep event from Google Calendar failed:", prepErr);
        }
    }

    // Delete from secondary calendar if synced
    if (task.secondaryCalendarEventId) {
        const activeSemester = await storage.getActiveSemesterSettings();
        if (activeSemester?.secondaryCalendarId) {
            try {
                await deleteEventFromCalendar(activeSemester.secondaryCalendarId, task.secondaryCalendarEventId);
            } catch (secErr) {
                console.error("Auto-delete from secondary calendar failed:", secErr);
            }
        }
    }
}

// Delete from second Google account if synced
if (task.secondAccountCalendarEventId) {
    try {
        await deleteEventFromSecondAccount(task.secondAccountCalendarEventId);
    } catch (secAccErr) {

```

```

        console.error("Auto-delete from second Google account failed:", secAccErr);
    }
}
if (task.secondAccountPrepEventId) {
    try {
        await deleteEventFromSecondAccount(task.secondAccountPrepEventId);
    } catch (secAccErr) {
        console.error("Auto-delete prep event from second account failed:", secAccErr);
    }
}

// If this is a parent task with repeat, also delete all child tasks
if (task.repeatType && task.repeatType !== "none") {
    const childTasks = await storage.getChildTasks(taskId);
    for (const child of childTasks) {
        // Delete child from calendar
        if (child.calendarEventId) {
            try {
                await deleteCalendarEvent(child.calendarEventId);
            } catch (calErr) {
                console.error("Failed to delete child calendar event:", calErr);
            }
        }
        // Delete child from second Google account
        if (child.secondAccountCalendarEventId) {
            try {
                await deleteEventFromSecondAccount(child.secondAccountCalendarEventId);
            } catch (secAccErr) {
                console.error("Failed to delete child from second account:", secAccErr);
            }
        }
    }
    // Delete all child tasks
    await storage.deleteChildTasks(taskId);
}

// Delete all subtasks and related links for this task
await storage.deleteSubtasksByTask(taskId);

await storage.deleteTask(taskId);
res.status(204).end();
});

// DELETE /api/tasks/:id/future - Delete this task and all future recurring instances
app.delete("/api/tasks/:id/future", async (req, res) => {
    const taskId = Number(req.params.id);
    const task = await storage.getTask(taskId);
    if (!task) return res.status(404).json({ message: 'Task not found' });

    const parentId = task.parentTaskId || taskId;
    const allSiblings = await storage.getChildTasks(parentId);
    const parentTask = task.parentTaskId ? await storage.getTask(parentId) : task;

    const tasksToDelete = [task];
    for (const sibling of allSiblings) {
        if (sibling.id !== task.id && new Date(sibling.dueDate) >= new Date(task.dueDate)) {
            tasksToDelete.push(sibling);
        }
    }
    if (parentTask && parentTask.id !== task.id && new Date(parentTask.dueDate) >= new Date(task.dueDate)) {
        tasksToDelete.push(parentTask);
    }

    for (const t of tasksToDelete) {
        if (t.calendarEventId) {
            try { await deleteCalendarEvent(t.calendarEventId); } catch {}
        }
        if (t.secondAccountCalendarEventId) {
            try { await deleteEventFromSecondAccount(t.secondAccountCalendarEventId); } catch {}
        }
        if (t.prepCalendarEventId) {
            try { await deleteCalendarEvent(t.prepCalendarEventId); } catch {}
        }
        if (t.secondAccountPrepEventId) {
            try { await deleteEventFromSecondAccount(t.secondAccountPrepEventId); } catch {}
        }
        await storage.deleteSubtasksByTask(t.id);
        await storage.deleteTask(t.id);
    }

    res.status(204).end();
});

// PATCH /api/tasks/:id/update-recurring - Update all recurring siblings with same fields
app.patch("/api/tasks/:id/update-recurring", async (req, res) => {
    try {
        const taskId = Number(req.params.id);
        const task = await storage.getTask(taskId);
        if (!task) return res.status(404).json({ message: 'Task not found' });

        const fields = { ...req.body };
        const dateFields = ['dueDate', 'startDate', 'completedAt', 'repeatEndDate', 'createdAt', 'deletedAt', 'lastMove...'];
        for (const key of dateFields) {
            if (fields[key] && typeof fields[key] === 'string') fields[key] = new Date(fields[key]);
        }
        const parentId = task.parentTaskId || taskId;
        const allSiblings = await storage.getChildTasks(parentId);
        const parentTask = task.parentTaskId ? await storage.getTask(parentId) : task;

        const tasksToUpdate: number[] = [taskId];
        for (const sibling of allSiblings) {
            if (sibling.id !== taskId) {
                tasksToUpdate.push(sibling.id);
            }
        }
        if (parentTask && parentTask.id !== taskId) {
            tasksToUpdate.push(parentTask.id);
        }
    }
});

```

```

if (tasksToUpdate.length <= 1 && fields.originalTitle && task.courseName) {
  const allTasksList = await storage.getTasks();
  for (const t of allTasksList) {
    if (t.id !== taskId && t.title === fields.originalTitle && t.courseName === task.courseName) {
      tasksToUpdate.push(t.id);
    }
  }
  delete fields.originalTitle;
} else {
  delete fields.originalTitle;
}

// Calculate prep duration from the edited task so we can apply it relatively
// to each sibling's own dueDate
let prepDuration = 0;
if (fields.startDate && fields.dueDate) {
  prepDuration = new Date(fields.dueDate).getTime() - new Date(fields.startDate).getTime();
} else if (fields.startDate && task.dueDate) {
  prepDuration = new Date(task.dueDate).getTime() - new Date(fields.startDate).getTime();
}

const uniqueIds = [...new Set(tasksToUpdate)];
for (const id of uniqueIds) {
  if (id === taskId) {
    await storage.updateTask(id, fields);
  } else {
    const siblingTask = allSiblings.find(s => s.id === id) || (parentTask?.id === id ? parentTask : null);
    const siblingDueDate = siblingTask?.dueDate ? new Date(siblingTask.dueDate) : null;
    const siblingFields = { ...fields };

    if (siblingDueDate && !isNaN(siblingDueDate.getTime())) {
      if (fields.eventStartTime && typeof fields.eventStartTime === 'string' && fields.eventStartTime.includes(...
const [newHour, newMin] = (fields.eventStartTime as string).split(':').map(Number);
const fmt = new Intl.DateTimeFormat('en-US', { timeZone: 'America/Toronto', year: 'numeric', month: '2-...
const parts = fmt.formatToParts(siblingDueDate);
const yy = Number(parts.find(p => p.type === 'year')!.value);
const mm = Number(parts.find(p => p.type === 'month')!.value);
const dd = Number(parts.find(p => p.type === 'day')!.value);
const etDateStr = `${yy}-${String(mm).padStart(2, '0')}-${String(dd).padStart(2, '0')}`;
const probe = new Date(etDateStr + 'Z');
const probeETHour = parseInt(probe.toLocaleString('en-US', { timeZone: 'America/Toronto', hour: 'numeri...
const offsetHours = probeETHour - probe.getUTCHours();
const utcDate = new Date(probe.getTime() - offsetHours * 3600000);
siblingFields.dueDate = utcDate;
}

if (prepDuration > 0 && (fields.startDate !== undefined)) {
  const dueDateForPrep = siblingFields.dueDate ? new Date(siblingFields.dueDate as Date) : siblingDueDate;
  siblingFields.startDate = new Date(dueDateForPrep.getTime() - prepDuration);
} else {
  delete siblingFields.startDate;
}
}

delete siblingFields.weekNumber;
if (!(fields.eventStartTime && typeof fields.eventStartTime === 'string' && fields.eventStartTime.includes(...
delete siblingFields.dueDate;
delete siblingFields.startDate;
}
if (!siblingFields.dueDate) delete siblingFields.dueDate;
await storage.updateTask(id, siblingFields);
}
}

res.json({ updated: uniqueIds.length });
} catch (error) {
  console.error("Error updating recurring tasks:", error);
  res.status(500).json({ message: "Failed to update recurring tasks" });
}
});

// PATCH /api/tasks/:id/complete
app.patch(api.tasks.complete.path, async (req, res) => {
  const { isCompleted } = req.body;
  const task = await storage.updateTask(Number(req.params.id), { isCompleted });
  if (!task) {
    return res.status(404).json({ message: 'Task not found' });
  }
  res.json(task);
});

app.patch("/api/tasks/:id/flag", async (req, res) => {
  const { flagged } = req.body;
  const task = await storage.updateTask(Number(req.params.id), { flagged: !!flagged });
  if (!task) {
    return res.status(404).json({ message: 'Task not found' });
  }
  res.json(task);
});

// ===== SEMESTER SETTINGS ROUTES =====

// GET /api/semesters - Get all semester settings
app.get("/api/semesters", async (_req, res) => {
  try {
    const all = await storage.getAllSemesterSettings();
    res.json(all);
  } catch (err) {
    console.error("Error fetching all semester settings:", err);
    res.status(500).json({ error: "Failed to fetch semester settings" });
  }
});

// GET /api/semester - Get active semester settings
app.get(api.semester.get.path, async (_req, res) => {
  try {
    const settings = await storage.getActiveSemesterSettings();

```

```

    res.json(settings || null);
  } catch (err) {
    console.error("Error fetching semester settings:", err);
    res.status(500).json({ error: "Failed to fetch semester settings" });
  }
});

// POST /api/semester - Create new semester settings
app.post(api.semester.create.path, async (req, res) => {
  try {
    const input = api.semester.create.input.parse(req.body);
    const settings = await storage.createSemesterSettings(input);
    res.status(201).json(settings);
  } catch (err) {
    if (err instanceof z.ZodError) {
      return res.status(400).json({
        message: err.errors[0].message,
        field: err.errors[0].path.join('.'),
      });
    }
    console.error("Error creating semester settings:", err);
    res.status(500).json({ error: "Failed to create semester settings" });
  }
});

// PATCH /api/semester - Update active semester settings (all fields)
app.patch("/api/semester", async (req, res) => {
  try {
    const activeSemester = await storage.getActiveSemesterSettings();
    if (!activeSemester) {
      return res.status(404).json({ error: "No active semester settings found" });
    }
    const dateFields = ['semesterStartDate', 'semesterEndDate', 'course1StartDate', 'course1EndDate', 'course2Start...'];
    const body = { ...req.body };
    for (const field of dateFields) {
      if (body[field] && typeof body[field] === 'string') {
        body[field] = new Date(body[field]);
      }
    }
    const updated = await storage.updateSemesterSettings(activeSemester.id, body);
    res.json(updated);
  } catch (err) {
    console.error("Error updating semester settings:", err);
    res.status(500).json({ error: "Failed to update semester settings" });
  }
});

// PATCH /api/semesters/:id - Update any semester settings row by ID
app.patch("/api/semesters/:id", async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    if (isNaN(id)) return res.status(400).json({ error: "Invalid ID" });
    const dateFields = ['semesterStartDate', 'semesterEndDate', 'course1StartDate', 'course1EndDate', 'course2Start...'];
    const body = { ...req.body };
    for (const field of dateFields) {
      if (body[field] && typeof body[field] === 'string') {
        body[field] = new Date(body[field]);
      }
    }
    const updated = await storage.updateSemesterSettings(id, body);
    res.json(updated);
  } catch (err) {
    console.error("Error updating semester settings by ID:", err);
    res.status(500).json({ error: "Failed to update semester settings" });
  }
});

// PATCH /api/semester-settings/calendar - Update secondary calendar
app.patch("/api/semester-settings/calendar", async (req, res) => {
  try {
    const { secondaryCalendarId } = req.body;
    const activeSemester = await storage.getActiveSemesterSettings();
    if (!activeSemester) {
      return res.status(404).json({ error: "No active semester settings found" });
    }
    const updated = await storage.updateSemesterSettings(activeSemester.id, { secondaryCalendarId });
    res.json(updated);
  } catch (err) {
    console.error("Error updating secondary calendar:", err);
    res.status(500).json({ error: "Failed to update secondary calendar" });
  }
});

// PATCH /api/semester-settings/professor-emails - Update professor emails
app.patch("/api/semester-settings/professor-emails", async (req, res) => {
  try {
    const { course1ProfessorEmail, course2ProfessorEmail, course3ProfessorEmail } = req.body;
    const activeSemester = await storage.getActiveSemesterSettings();
    if (!activeSemester) {
      return res.status(404).json({ error: "No active semester settings found" });
    }
    const updated = await storage.updateSemesterSettings(activeSemester.id, {
      course1ProfessorEmail,
      course2ProfessorEmail,
      course3ProfessorEmail
    });
    res.json(updated);
  } catch (err) {
    console.error("Error updating professor emails:", err);
    res.status(500).json({ error: "Failed to update professor emails" });
  }
});

app.get("/api/semester-checklist", async (_req, res) => {
  try {
    const semester = await storage.getActiveSemesterSettings();
    if (!semester) return res.json({ items: [], allChecked: true, semesterId: null });
    const courseCodes = [semester.course1Code, semester.course2Code, semester.course3Code].filter(Boolean);
  }
});

```



```

    const items = await storage.initSemesterChecklist(semester.id, courseCodes);
    const allChecked = items.every(i => i.isChecked);
    res.json({ items, allChecked, semesterId: semester.id });
  } catch (err) {
    console.error("Error fetching semester checklist:", err);
    res.status(500).json({ error: "Failed to fetch semester checklist" });
  }
});

app.patch("/api/semester-checklist", async (req, res) => {
  try {
    const { semesterSettingsId, courseCode, itemType, isChecked } = req.body;
    const updated = await storage.upsertSemesterCheckListItem({ semesterSettingsId, courseCode, itemType, isChecked });
    const allItems = await storage.getSemesterChecklist(semesterSettingsId);
    const allChecked = allItems.every(i => i.isChecked);
    res.json({ item: updated, allChecked });
  } catch (err) {
    console.error("Error updating semester checklist:", err);
    res.status(500).json({ error: "Failed to update semester checklist" });
  }
});

app.get("/api/course-week-mappings", async (req, res) => {
  try {
    const semester = await storage.getActiveSemesterSettings();
    if (!semester) return res.json([]);
    const mappings = await storage.getAllCourseWeekMappings(semester.id);
    res.json(mappings);
  } catch (err) {
    console.error("Error fetching all course week mappings:", err);
    res.status(500).json({ error: "Failed to fetch course week mappings" });
  }
});

app.get("/api/course-week-mappings/:courseCode", async (req, res) => {
  try {
    const semester = await storage.getActiveSemesterSettings();
    if (!semester) return res.json([]);
    const mappings = await storage.getCourseWeekMappings(req.params.courseCode, semester.id);
    res.json(mappings);
  } catch (err) {
    console.error("Error fetching course week mappings:", err);
    res.status(500).json({ error: "Failed to fetch course week mappings" });
  }
});

app.put("/api/course-week-mappings", async (req, res) => {
  try {
    const semester = await storage.getActiveSemesterSettings();
    if (!semester) return res.status(400).json({ error: "No active semester" });
    const { courseCode, weekNumber, confirmed, courseWeekLabel, notes } = req.body;
    const mapping = await storage.upsertCourseWeekMapping({
      courseCode,
      semesterSettingsId: semester.id,
      weekNumber,
      confirmed,
      courseWeekLabel: courseWeekLabel || null,
      notes: notes || null,
    });
    res.json(mapping);
  } catch (err) {
    console.error("Error updating course week mapping:", err);
    res.status(500).json({ error: "Failed to update course week mapping" });
  }
});

app.put("/api/course-week-mappings/bulk", async (req, res) => {
  try {
    const semester = await storage.getActiveSemesterSettings();
    if (!semester) return res.status(400).json({ error: "No active semester" });
    const { courseCode, mappings } = req.body;
    const results = [];
    for (const m of mappings) {
      const mapping = await storage.upsertCourseWeekMapping({
        courseCode,
        semesterSettingsId: semester.id,
        weekNumber: m.weekNumber,
        confirmed: m.confirmed,
        courseWeekLabel: m.courseWeekLabel || null,
        notes: m.notes || null,
      });
      results.push(mapping);
    }
    res.json(results);
  } catch (err) {
    console.error("Error bulk updating course week mappings:", err);
    res.status(500).json({ error: "Failed to bulk update course week mappings" });
  }
});

const weatherCache: { data: any | null; timestamp: number } = { data: null, timestamp: 0 };

app.get("/api/weather", async (_req, res) => {
  try {
    const now = Date.now();
    if (weatherCache.data && now - weatherCache.timestamp < 15 * 60 * 1000) {
      return res.json(weatherCache.data);
    }

    const baseUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
    const headers = { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' };

    const ecConditionToWmo: Record<string, number> = {
      'sunny': 0, 'clear-night': 0, 'clear': 0,
      'partlycloudy': 2, 'partly-cloudy': 2,
      'cloudy': 3, 'overcast': 3,
      'fog': 45,
      'rainy': 63, 'pouring': 65,
    };
  }
});

```

```

    'snowy': 73, 'snowy-rainy': 77,
    'hail': 99,
    'lightning': 95, 'lightning-rainy': 96,
    'windy': 2, 'windy-variant': 3,
    'exceptional': 0,
  });

const [stateRes, forecastRes, sunRes] = await Promise.all([
  fetch(`${haUrl}/api/states/weather.toronto_forecast`, { headers }),
  fetch(`${haUrl}/api/services/weather/get_forecasts?return_response`, {
    method: 'POST', headers,
    body: JSON.stringify({ entity_id: 'weather.toronto_forecast', type: 'daily' }),
  }),
  fetch('https://api.open-meteo.com/v1/forecast?latitude=43.6275&longitude=-79.3962&daily=sunrise,sunset,weathe...
]);

const stateData = await stateRes.json();
const forecastData = await forecastRes.json();
const sunData = await sunRes.json();

const attrs = stateData.attributes || {};
const currentCondition = stateData.state || 'cloudy';
const currentTemp = attrs.temperature ?? 0;
const currentWind = attrs.wind_speed ?? 0;

const nowToronto = new Date(new Date().toLocaleString('en-US', { timeZone: 'America/Toronto' }));
const hour = nowToronto.getHours();
const isDay = hour >= 6 && hour < 21;

const forecasts = forecastData?.service_response?.['weather.toronto_forecast']?.forecast || [];

const sunTimes = sunData?.daily?.time || [];
const sunrises = sunData?.daily?.sunrise || [];
const sunsets = sunData?.daily?.sunset || [];
const sunMap: Record<string, { sunrise: string; sunset: string }> = {};
sunTimes.forEach((d: string, i: number) => {
  sunMap[d] = { sunrise: sunrises[i], sunset: sunsets[i] };
});

const todayStr = `${nowToronto.getFullYear()}-${String(nowToronto.getMonth() + 1).padStart(2, '0')}-${String(now...
const todaySun = sunMap[todayStr];

const dailyEntries: { date: string; high: number; low: number; weatherCode: number; sunrise?: string; sunset?: ...
for (const fc of forecasts) {
  const fcDate = fc.datetime ? fc.datetime.substring(0, 10) : '';
  if (!fcDate) continue;
  const wmo = ecConditionToWmo[fc.condition] ?? 3;
  const sun = sunMap[fcDate];
  dailyEntries.push({
    date: fcDate,
    high: fc.temperature != null ? Math.round(fc.temperature) : 0,
    low: fc.templow != null ? Math.round(fc.templow) : 0,
    weatherCode: wmo,
    sunrise: sun?.sunrise,
    sunset: sun?.sunset,
  });
}

const haDates = new Set(dailyEntries.map(d => d.date));
const omTimes = sunData?.daily?.time || [];
const omWeatherCodes = sunData?.daily?.weather_code || [];
const omMaxTemps = sunData?.daily?.temperature_2m_max || [];
const omMinTemps = sunData?.daily?.temperature_2m_min || [];
for (let i = 0; i < omTimes.length; i++) {
  const d = omTimes[i];
  if (!haDates.has(d)) {
    dailyEntries.push({
      date: d,
      high: omMaxTemps[i] != null ? Math.round(omMaxTemps[i]) : 0,
      low: omMinTemps[i] != null ? Math.round(omMinTemps[i]) : 0,
      weatherCode: omWeatherCodes[i] ?? 3,
      sunrise: sunrises[i],
      sunset: sunsets[i],
    });
  }
}
dailyEntries.sort((a, b) => a.date.localeCompare(b.date));

const result = {
  current: {
    weather_code: ecConditionToWmo[currentCondition] ?? 3,
    temperature_2m: currentTemp,
    wind_speed_10m: currentWind,
    is_day: isDay ? 1 : 0,
  },
  daily: {
    time: dailyEntries.map(d => d.date),
    weather_code: dailyEntries.map(d => d.weatherCode),
    temperature_2m_max: dailyEntries.map(d => d.high),
    temperature_2m_min: dailyEntries.map(d => d.low),
    sunrise: dailyEntries.map(d => d.sunrise || ''),
    sunset: dailyEntries.map(d => d.sunset || ''),
  },
  _source: 'environment_canada_ha',
};

if (todaySun) {
  (result as any).todaySunrise = todaySun.sunrise;
  (result as any).todaySunset = todaySun.sunset;
}

weatherCache.data = result;
weatherCache.timestamp = now;
res.json(result);
} catch (err) {
  console.error("Error fetching weather from HA/EC:", err);
  try {

```

```

const fallback = await fetch('https://api.open-meteo.com/v1/forecast?latitude=43.6275&longitude=-79.3962&curr...
const data = await fallback.json();
weatherCache.data = data;
weatherCache.timestamp = Date.now();
res.json(data);
} catch (fallbackErr) {
console.error("Fallback weather also failed:", fallbackErr);
res.status(500).json({ error: "Failed to fetch weather" });
}
});

const weatherAlertCache: { data: any | null; timestamp: number } = { data: null, timestamp: 0 };

const WEATHER_ALERT_ZONES = ['on61', 'on20', 'on21', 'on29'];

async function fetchWeatherAlertsFromECCC(): Promise<{ title: string; summary: string; type: string; updated: strin...
const seen = new Set<string>();
const alerts: { title: string; summary: string; type: string; updated: string }[] = [];

const fetches = WEATHER_ALERT_ZONES.map(async (zone) => {
try {
const controller = new AbortController();
const timeout = setTimeout(() => controller.abort(), 8000);
const response = await fetch('https://weather.gc.ca/warnings/report_e.html?${zone}=', {
signal: controller.signal,
headers: { 'User-Agent': 'Mozilla/5.0 (compatible; UniCal/1.0)' },
redirect: 'follow',
});
clearTimeout(timeout);
const html = await response.text();

const stateMatch = html.match(/window\.__INITIAL_STATE__\s*=\s*({[\s\S]*?});\s*\n/);
if (!stateMatch) return;
const stateJson = stateMatch[1].replace(/\\u002F/g, '/');
const state = JSON.parse(stateJson);
const alertObj = state?.alert?.alert || {};
for (const zoneKey of Object.keys(alertObj)) {
const zoneData = alertObj[zoneKey];
if (!zoneData || typeof zoneData !== 'object') continue;
const zoneAlerts = zoneData.alerts || [];
const zoneName = zoneData.displayName || zoneKey;
for (const a of zoneAlerts) {
if (!a || a.status !== 'active') continue;
const rawTitle = a.alertBannerText || a.type || '';
const dedup = `${zoneKey}-${rawTitle}`;
if (seen.has(dedup)) continue;
seen.add(dedup);
const title = `${rawTitle}${zoneName ? ` - ${zoneName}` : ''}`.replace(/&/g, '&');
const isWarning = /warning/i.test(rawTitle);
const isWatch = /watch/i.test(rawTitle);
const isAdvisory = /advisory|statement|special/i.test(rawTitle);
alerts.push({
title,
summary: (a.issueTimeText || '').slice(0, 300),
type: isWarning ? 'warning' : isWatch ? 'watch' : isAdvisory ? 'advisory' : 'info',
updated: a.issueTime || new Date().toISOString(),
});
}
}
} catch (e) {
console.error(`[Weather Alerts] Failed to fetch zone ${zone}:`, e);
}
});

await Promise.all(fetches);
return alerts;
}

app.get("/api/weather-alerts", async (_req, res) => {
try {
const now = Date.now();
if (weatherAlertCache.data && now - weatherAlertCache.timestamp < 5 * 60 * 1000) {
return res.json(weatherAlertCache.data);
}

const alerts = await fetchWeatherAlertsFromECCC();
const result = { alerts, count: alerts.length, hasAlerts: alerts.length > 0, timestamp: new Date().toISOString(...
weatherAlertCache.data = result;
weatherAlertCache.timestamp = now;
res.json(result);
} catch (err) {
console.error("Error fetching weather alerts:", err);
res.status(500).json({ error: "Failed to fetch weather alerts", alerts: [], count: 0, hasAlerts: false });
}
});

app.get("/api/ha/sensor/weather-alerts", async (_req, res) => {
try {
const now = Date.now();
if (!weatherAlertCache.data || now - weatherAlertCache.timestamp > 5 * 60 * 1000) {
const alerts = await fetchWeatherAlertsFromECCC();
weatherAlertCache.data = { alerts, count: alerts.length, hasAlerts: alerts.length > 0, timestamp: new Date()....
weatherAlertCache.timestamp = now;
}
const d = weatherAlertCache.data;
res.json({
state: d.count,
attributes: {
friendly_name: "Weather Alerts Toronto",
has_alerts: d.hasAlerts,
alert_count: d.count,
alert_1_title: d.alerts[0]?.title || '',
alert_1_type: d.alerts[0]?.type || '',
alert_1_summary: d.alerts[0]?.summary || '',
alert_2_title: d.alerts[1]?.title || '',
alert_2_type: d.alerts[1]?.type || '',
alert_2_summary: d.alerts[1]?.summary || '',
}
});

```

```

        alert_3_title: d.alerts[2]?.title || '',
        alert_3_type: d.alerts[2]?.type || '',
        alert_3_summary: d.alerts[2]?.summary || '',
        last_updated: d.timestamp,
    },
    });
} catch (err) {
    console.error("Error fetching weather alerts for HA:", err);
    res.status(500).json({ error: "Failed to fetch weather alerts" });
}
});

const pollenCache: { data: any | null; timestamp: number } = { data: null, timestamp: 0 };

app.get("/api/pollen", async (_req, res) => {
    try {
        const now = Date.now();
        if (pollenCache.data && now - pollenCache.timestamp < 30 * 60 * 1000) {
            return res.json(pollenCache.data);
        }
        const response = await fetch('https://air-quality-api.open-meteo.com/v1/air-quality?latitude=43.6275&longitude=...');
        const raw = await response.json();
        const c = raw.current || {};
        const treePollen = Math.max(c.alder_pollen || 0, c.birch_pollen || 0, c.olive_pollen || 0);
        const grassPollen = c.grass_pollen || 0;
        const ragweedPollen = c.ragweed_pollen || 0;
        const mugwortPollen = c.mugwort_pollen || 0;
        const weedPollen = Math.max(ragweedPollen, mugwortPollen);
        const overallMax = Math.max(treePollen, grassPollen, weedPollen);

        const getLevel = (val: number) => {
            if (val <= 10) return 'Low';
            if (val <= 30) return 'Moderate';
            if (val <= 60) return 'High';
            return 'Very High';
        };

        const result = {
            tree: { value: treePollen, level: getLevel(treePollen), details: { alder: c.alder_pollen || 0, birch: c.birch...
            grass: { value: grassPollen, level: getLevel(grassPollen) },
            weed: { value: weedPollen, level: getLevel(weedPollen), details: { ragweed: ragweedPollen, mugwort: mugwortPo...
            overall: { value: overallMax, level: getLevel(overallMax) },
            aqi: c.european_aqi || 0,
            timestamp: new Date().toISOString(),
        };
        pollenCache.data = result;
        pollenCache.timestamp = now;
        res.json(result);
    } catch (err) {
        console.error("Error fetching pollen:", err);
        res.status(500).json({ error: "Failed to fetch pollen data" });
    }
});

app.get("/api/ha/sensor/pollen", async (_req, res) => {
    try {
        const now = Date.now();
        if (!pollenCache.data || now - pollenCache.timestamp > 30 * 60 * 1000) {
            const response = await fetch('https://air-quality-api.open-meteo.com/v1/air-quality?latitude=43.6275&longitud...');
            const raw = await response.json();
            const c = raw.current || {};
            const treePollen = Math.max(c.alder_pollen || 0, c.birch_pollen || 0, c.olive_pollen || 0);
            const grassPollen = c.grass_pollen || 0;
            const ragweedPollen = c.ragweed_pollen || 0;
            const mugwortPollen = c.mugwort_pollen || 0;
            const weedPollen = Math.max(ragweedPollen, mugwortPollen);
            const overallMax = Math.max(treePollen, grassPollen, weedPollen);
            const getLevel = (val: number) => {
                if (val <= 10) return 'Low';
                if (val <= 30) return 'Moderate';
                if (val <= 60) return 'High';
                return 'Very High';
            };
            pollenCache.data = {
                tree: { value: treePollen, level: getLevel(treePollen), details: { alder: c.alder_pollen || 0, birch: c.bir...
                grass: { value: grassPollen, level: getLevel(grassPollen) },
                weed: { value: weedPollen, level: getLevel(weedPollen), details: { ragweed: ragweedPollen, mugwort: mugwort...
                overall: { value: overallMax, level: getLevel(overallMax) },
                aqi: c.european_aqi || 0,
                timestamp: new Date().toISOString(),
            };
            pollenCache.timestamp = now;
        }
        const p = pollenCache.data;
        res.json({
            state: p.overall.value,
            attributes: {
                unit_of_measurement: "grains/m³",
                friendly_name: "Pollen Index Toronto",
                level: p.overall.level,
                tree_pollen: p.tree.value,
                tree_level: p.tree.level,
                tree_alder: p.tree.details.alder,
                tree_birch: p.tree.details.birch,
                grass_pollen: p.grass.value,
                grass_level: p.grass.level,
                weed_pollen: p.weed.value,
                weed_level: p.weed.level,
                weed_ragweed: p.weed.details.ragweed,
                weed_mugwort: p.weed.details.mugwort,
                aqi: p.aqi,
                last_updated: p.timestamp,
            },
        });
    } catch (err) {
        console.error("Error fetching pollen for HA:", err);
        res.status(500).json({ error: "Failed to fetch pollen sensor data" });
    }
});

```

```

});

const newsCache: { data: any[] | null; timestamp: number } = { data: null, timestamp: 0 };

app.use('/api/ticker-assets', express.static(path.join(process.cwd(), 'attached_assets')));

app.get("/api/ticker", async (req, res) => {
  try {
    const LOGOS: Record<string, { file: string; height: number }> = {
      'CNN': { file: 'CNN_1773536484180.png', height: 28 },
      'CBC': { file: 'cbc-news-logo-black-and-white_1773536865600.png', height: 22 },
      'CTV': { file: 'CTV2_1773545440801.png', height: 20 },
      'Global': { file: 'Global_White_1773536754594.png', height: 24 },
      'MSNBC': { file: 'MSNBC_1773536950584.png', height: 14 },
      'Politico': { file: 'Politico_1773537080711.png', height: 20 },
      'Raw Story': { file: 'Raw_Story_1773607642361.png', height: 18 },
      'ABC News': { file: 'ABC_1773609250051.png', height: 24 },
      'BBC': { file: 'BBC_1773609711103.png', height: 13 },
      'Fox News': { file: 'Fox_News_1773610204651.png', height: 15 },
    };
    const ALERT_LOGO_FILE = 'Weather_Alert_1773608511887.png';
    const WMO: Record<number, string> = {0: 'Clear', 1: 'Mainly Clear', 2: 'Partly Cloudy', 3: 'Overcast', 45: 'Fog', 48: 'Rim...
    const US_SOURCES = ['CNN', 'Politico', 'Raw Story', 'MSNBC', 'ABC News', 'Fox News'];
    const DAYS = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'];

    const now = Date.now();
    const withTimeout = (url: string, ms: number) => {
      const controller = new AbortController();
      const timer = setTimeout(() => controller.abort(), ms);
      return fetch(url, { signal: controller.signal }).then(r => { clearTimeout(timer); return r.json(); }).catch(...
    );
    const port = process.env.PORT || 5000;

    const [alertRes, wxRes, pollenRes, newsRes] = await Promise.all([
      weatherAlertCache.data ? Promise.resolve(weatherAlertCache.data) : withTimeout('http://localhost:${port}/api/...
      weatherCache.data ? Promise.resolve(weatherCache.data) : withTimeout('http://localhost:${port}/api/weather', ...
      pollenCache.data ? Promise.resolve(pollenCache.data) : withTimeout('http://localhost:${port}/api/pollen', 3000),
      newsCache.data ? Promise.resolve(newsCache.data) : withTimeout('http://localhost:${port}/api/news', 4000),
    ]);

    let tickerItems = '';

    if (alertRes && alertRes.alerts) {
      for (const a of alertRes.alerts) {
        tickerItems += `<span class="t-item t-alert"><span class="t-forecast">= 3) {
        const parts = wxRes.daily.time.slice(0, 3).map((t: string, i: number) => {
          const dt = new Date(t + 'T12:00:00');
          return `${DAYS[dt.getDay()]}: ${Math.round(wxRes.daily.temperature_2m_max[i])}/${Math.round(wxRes.daily...
        });
        tickerItems += `<span class="t-item"><span class="t-forecast"> = {0: 'clear', 1: 'mostly clear', 2: 'partly cloudy', 3: 'overcast', 45: 'fo...
        const briefParts: string[] = [];
        briefParts.push('Currently ${temp}° and ${WMO_BRIEF[c.weather_code as number] || 'mixed conditions'}. Today...
        for (let bi = 1; bi <= 2; bi++) {
          if (wxRes.daily.time[bi]) {
            const bdt = new Date(wxRes.daily.time[bi] + 'T12:00:00');
            const bDesc = WMO_BRIEF[wxRes.daily.weather_code?.[bi] as number] || 'mixed conditions';
            briefParts.push(`${DAYS[bdt.getDay()]}: ${bDesc}, ${Math.round(wxRes.daily.temperature_2m_max[bi])}/${${...
          }
        }
        tickerItems += `<span class="t-item"><span class="t-forecast"><span class="t-forecast"><0<B? <b>POLLEN</b></span> <span class="t-data" s...
    }

    const customAnnouncements = await storage.getAnnouncements();
    const tickerNow = new Date();
    for (const a of customAnnouncements) {
      if (!a.courseName) continue;
      const received = new Date(a.receivedAt);
      const isCustom = a.courseName === 'Custom';
      if (!isCustom) {
        const dayOfWeek = received.getDay();
        const daysUntilFriday = dayOfWeek <= 5 ? (5 - dayOfWeek) : (5 + 7 - dayOfWeek);
        const fridayEnd = new Date(received);
        fridayEnd.setDate(received.getDate() + daysUntilFriday);
        fridayEnd.setHours(23, 59, 59, 999);
        if (tickerNow > fridayEnd) continue;
      }
      const escapedBody = (a.body || a.snippet || a.subject || '').replace(/</g, '&lt;');
      const label = isCustom ? '0=0i' : '0=0A <b>${a.courseName}</b>';
      tickerItems += `<span class="t-item"><span class="t-forecast">${label}${isCustom ? '' : ':'}</span> <span cla...
    }

    if (newsRes && Array.isArray(newsRes)) {
      const ca: any[] = [], us: any[] = [], bbc: any[] = [];
      newsRes.forEach((h: any) => {
        if (h.source === 'BBC') bbc.push(h);
        else if (US_SOURCES.includes(h.source)) us.push(h);
        else ca.push(h);
      });
      const max = Math.max(ca.length, us.length, bbc.length);
      const interleaved: any[] = [];
      for (let i = 0; i < max; i++) {
        if (i < ca.length) interleaved.push(ca[i]);
      }
    }
  }
}

```

```

    if (i < us.length) interleaved.push(us[i]);
    if (i < bbc.length) interleaved.push(bbc[i]);
  }
  for (const item of interleaved) {
    const logoInfo = LOGOS[item.source];
    const escapedTitle = (item.title || '').replace(/"/g, '&quot;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
    const escapedLink = (item.link || '#').replace(/"/g, '&quot;');
    const logoHtml = logoInfo
      ? ``
      : `<span style="font-size:11px;font-weight:700;padding:0 4px;border-radius:3px;background:#555;color:#fff...
    let timeAgoHtml = '';
    if (item.publishedAt) {
      const diff = Date.now() - new Date(item.publishedAt).getTime();
      const mins = Math.floor(diff / 60000);
      if (mins >= 0 && mins <= 4320) {
        const ago = mins < 60 ? `${mins}m` : mins < 1440 ? `${Math.floor(mins / 60)}h` : `${Math.floor(mins / 1...
        timeAgoHtml = `<span style="font-size:11px;color:rgba(255,255,255,0.6);margin-left:4px">${ago}</span>`;
      }
    }
    tickerItems += `<span class="t-item"><a href="${escapedLink}" target="_blank">${logoHtml}<span class="t-sep...
  }
}

const authQS = req.query.auth ? `?auth=${req.query.auth}` : '';

if (!tickerItems) {
  const retryHtml = `<!DOCTYPE html><html><head><meta charset="utf-8"><meta http-equiv="refresh" content="5"><s...
  res.setHeader('Content-Type', 'text/html');
  res.setHeader('X-Frame-Options', 'ALLOWALL');
  res.setHeader('Content-Security-Policy', '');
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Cache-Control', 'no-cache, no-store, must-revalidate');
  return res.send(retryHtml);
}

const html = `<!DOCTYPE html>
<html><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<style>
*(margin:0;padding:0;box-sizing:border-box;font-family:-apple-system,BlinkMacSystemFont,'Segoe UI',Roboto,Helvetica,A...
html,body{width:100%;height:100%;overflow:hidden;background:#000}
@keyframes tickerScroll{0%{transform:translate3d(var(--ticker-start),0,0)}100%{transform:translate3d(var(--ticker-end...
@keyframes tickerAlertBlink{0%,100%{opacity:1}50%{opacity:0.3}}
.ticker-wrap{position:fixed;left:0;right:0;bottom:0;width:100vw;height:38px;overflow:hidden;background:#000;border-to...
.ticker-track{display:flex;align-items:center;height:100%;white-space:nowrap;position:relative;will-change:transform;...
.t-item{display:inline-flex;align-items:center;gap:6px;margin:0 16px;opacity:1}
.t-item a{display:inline-flex;align-items:center;gap:6px;text-decoration:none}
.t-item a:hover{text-decoration:underline}
.t-alert{animation:tickerAlertBlink 1s ease-in-out infinite}
.t-alert-text{font-size:13px;font-weight:700;color:#ff4444;text-shadow:0 0 6px rgba(255,68,68,0.5)}
.t-forecast{font-size:13px;font-weight:400;color:rgb(0,255,0);text-shadow:0 0 4px rgba(0,255,0,0.3)}
.t-data{font-size:13px;font-weight:400;color:rgba(255,255,255,0.95)}
.t-headline{font-size:13px;font-weight:400;color:rgba(255,255,255,0.9)}
.t-sep{color:rgba(255,255,255,0.85);margin:0 4px;font-weight:300;font-size:13px;line-height:1;vertical-align:middle}
.t-logo{border-radius:2px;object-fit:contain;vertical-align:middle}
</style></head><body>
<div class="ticker-wrap"><div class="ticker-track" id="track">${tickerItems}</div></div>
<script>
(function(){
  const track=document.getElementById('track');
  if(!track) return;
  function initScroll(){
    const cw=track.scrollWidth;
    const sw=window.innerWidth;
    const total=sw+cw;
    const dur=total/65;
    track.style.setProperty('--ticker-start',sw+'px');
    track.style.setProperty('--ticker-end','-'+cw+'px');
    track.style.animation='none';
    track.offsetHeight;
    track.style.animation='tickerScroll '+dur+'s linear infinite';
  }
  requestAnimationFrame(initScroll);
  const hasNews = track.querySelectorAll('.t-headline').length > 0;
  if (!hasNews) {
    setTimeout(() => location.reload(), 30000);
  } else {
    let lastRefresh = Date.now();
    function refreshContent(){
      if (Date.now() - lastRefresh < 120000) return;
      lastRefresh = Date.now();
      var url = location.href;
      fetch(url).then(function(r){return r.text();}).then(function(html){
        var parser = new DOMParser();
        var doc = parser.parseFromString(html,'text/html');
        var newTrack = doc.getElementById('track');
        if(newTrack && newTrack.innerHTML !== track.innerHTML){
          track.innerHTML = newTrack.innerHTML;
          requestAnimationFrame(initScroll);
        }
      }).catch(function(){});
    }
    track.addEventListener('animationiteration', refreshContent);
  }
})();
</script></body></html>`;
res.setHeader('Content-Type', 'text/html');
res.setHeader('X-Frame-Options', 'ALLOWALL');
res.setHeader('Content-Security-Policy', '');
res.setHeader('Access-Control-Allow-Origin', '*');
res.setHeader('Cache-Control', 'no-cache, no-store, must-revalidate');
res.send(html);
} catch (err) {
  console.error("Error serving ticker:", err);
  res.status(500).send("Ticker error");
}
});
app.get("/api/news", async (_req, res) => {

```

```

try {
  const now = Date.now();
  if (newsCache.data && now - newsCache.timestamp < 10 * 60 * 1000) {
    return res.json(newsCache.data);
  }

  const feeds = [
    { source: 'CNN', url: 'https://news.google.com/rss/search?q=site:cnn.com&hl=en-US&gl=US&ceid=US:en' },
    { source: 'CBC', url: 'https://www.cbc.ca/cmlink/rss-topstories' },
    { source: 'CTV', url: 'https://news.google.com/rss/search?q=site:ctvnews.ca&hl=en-CA&gl=CA&ceid=CA:en...' },
    { source: 'Global', url: 'https://globalnews.ca/politics/feed/' },
    { source: 'MSNBC', url: 'https://msnbc.com/feed' },
    { source: 'Politico', url: 'https://rss.politico.com/politics-news.xml' },
    { source: 'Raw Story', url: 'https://www.rawstory.com/feed', count: 3 },
    { source: 'ABC News', url: 'https://abcnews.go.com/abcnews/topstories', count: 3 },
    { source: 'BBC', url: 'https://feeds.bbc.co.uk/news/politics/rss.xml', count: 2 },
    { source: 'Fox News', url: 'https://moxie.foxnews.com/google-publisher/politics.xml', count: 3 },
  ];

  const results: { title: string; source: string; link: string; publishedAt?: string }[] = [];

  await Promise.allSettled(feeds.map(async (feed) => {
    try {
      const controller = new AbortController();
      const timeout = setTimeout(() => controller.abort(), 8000);
      const response = await fetch(feed.url, {
        signal: controller.signal,
        headers: { 'User-Agent': 'Mozilla/5.0 (compatible; UniCal/1.0)' }
      });
      clearTimeout(timeout);
      const xml = await response.text();

      const maxItems = (feed as any).count || 7;
      const items = xml.split(/<item\s*>/i).slice(1, maxItems + 1);
      for (const item of items) {
        const titleMatch = item.match(/<title[^\>]*>(?!<![CDATA\[.*\]]>?<\/title>/is);
        const linkMatch = item.match(/<link[^\>]*>(?!<![CDATA\[.*\]]>?<\/link>/is);
        const pubDateMatch = item.match(/<pubDate[^\>]*>(?!<![CDATA\[.*\]]>?<\/pubDate>/is);
        if (titleMatch?.[1]) {
          let title = titleMatch[1].replace(/&/g, '&').replace(/</g, '<').replace(/>/g, '>').replace(/...
          title = title.replace(/s*-s*(CTV News|Google News|CNN)/i, '').trim();
          const sportsFilter = /\b(NFL|NBA|NHL|MLB|CFL|MLS|FIFA|UEFA|NASCAR|PGA|ATP|WTA|Premier League|Super Bowl...
          const junkFilter = /^LIVE:|^WATCH:|^BREAKING:|^s$|^Video:|^Photos:|^Gallery:|^News Live$|^Live Stream|^Fu...
          if (title && title !== feed.source && title !== 'Google News' && title !== 'MS NOW' && !sportsFilter.te...
            const publishedAt = pubDateMatch?.[1]??.trim() ? new Date(pubDateMatch[1].trim()).toISOString() : unde...
            results.push({
              title,
              source: feed.source,
              link: linkMatch?.[1]??.trim() || '',
              ...(publishedAt && !isNaN(new Date(publishedAt).getTime()) ? { publishedAt } : {}),
            });
          }
        }
      }
    } catch (e) {
      console.error(`[News] Failed to fetch ${feed.source}:`, e);
    }
  }));

  const bySource: Record<string, typeof results> = {};
  for (const item of results) {
    if (!bySource[item.source]) bySource[item.source] = [];
    if (bySource[item.source].length < 3) {
      bySource[item.source].push(item);
    }
  }
  const sources = Object.keys(bySource).sort(() => Math.random() - 0.5);
  const interleaved: typeof results = [];
  let round = 0;
  let added = true;
  while (added) {
    added = false;
    for (const src of sources) {
      if (round < bySource[src].length) {
        interleaved.push(bySource[src][round]);
        added = true;
      }
    }
    round++;
  }
  newsCache.data = interleaved;
  newsCache.timestamp = now;
  res.json(interleaved);
} catch (err) {
  console.error("Error fetching news:", err);
  res.status(500).json({ error: "Failed to fetch news" });
}
});

app.get("/api/ha/news", async (_req, res) => {
  try {
    const now = Date.now();
    if (newsCache.data && now - newsCache.timestamp < 10 * 60 * 1000) {
      return res.json({ headlines: newsCache.data });
    }
    const controller = new AbortController();
    const timeout = setTimeout(() => controller.abort(), 8000);
    const response = await fetch(`http://localhost:${port}/api/news`, { signal: controller.signal });
    clearTimeout(timeout);
    if (response.ok) {
      const data = await response.json();
      return res.json({ headlines: data });
    }
    res.json({ headlines: [] });
  } catch (err) {
    console.error("Error in /api/ha/news:", err);
    res.json({ headlines: [] });
  }
}

```

```

});

app.get("/api/scholarships", async (_req, res) => {
  try {
    const data = await storage.getScholarships();
    res.json(data);
  } catch (err) {
    console.error("Error fetching scholarships:", err);
    res.status(500).json({ error: "Failed to fetch scholarships" });
  }
});

app.post("/api/scholarships", async (req, res) => {
  try {
    const created = await storage.createScholarship(req.body);
    res.json(created);
  } catch (err) {
    console.error("Error creating scholarship:", err);
    res.status(500).json({ error: "Failed to create scholarship" });
  }
});

app.patch("/api/scholarships/:id", async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const updated = await storage.updateScholarship(id, req.body);
    res.json(updated);
  } catch (err) {
    console.error("Error updating scholarship:", err);
    res.status(500).json({ error: "Failed to update scholarship" });
  }
});

app.delete("/api/scholarships/:id", async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    await storage.deleteScholarship(id);
    res.json({ success: true });
  } catch (err) {
    console.error("Error deleting scholarship:", err);
    res.status(500).json({ error: "Failed to delete scholarship" });
  }
});

app.post("/api/scholarships/calendar-events", async (req, res) => {
  try {
    const { name, applicationsOpen, deadline, description, recurring } = req.body;
    const results: any[] = [];
    if (applicationsOpen) {
      const openEvent = await createYearlyScholarshipEvent({ name, date: applicationsOpen, type: 'open', descriptio...
      if (openEvent) results.push({ type: 'open', eventId: openEvent.id });
    }
    if (deadline) {
      const deadlineEvent = await createYearlyScholarshipEvent({ name, date: deadline, type: 'deadline', descriptio...
      if (deadlineEvent) results.push({ type: 'deadline', eventId: deadlineEvent.id });
    }
    res.json({ success: true, events: results });
  } catch (err) {
    console.error("Error creating scholarship calendar events:", err);
    res.status(500).json({ error: "Failed to create scholarship calendar events" });
  }
});

app.get("/api/key-contacts", async (_req, res) => {
  try {
    const data = await storage.getKeyContacts();
    res.json(data);
  } catch (err) {
    console.error("Error fetching key contacts:", err);
    res.status(500).json({ error: "Failed to fetch key contacts" });
  }
});

app.get("/api/key-contacts/export.vcf", async (_req, res) => {
  try {
    const contacts = await storage.getKeyContacts();
    const vcards = contacts.map(c => {
      const nameParts = c.name.trim().split(/\s+/);
      const lastName = nameParts.length > 1 ? nameParts.pop() : '';
      const firstName = nameParts.join(' ');
      let vcard = `BEGIN:VCARD\r\nVERSION:3.0\r\n`;
      vcard += `N:${lastName};${firstName};;\r\n`;
      vcard += `FN:${c.name}\r\n`;
      if (c.title) vcard += `TITLE:${c.title}\r\n`;
      if (c.organization) vcard += `ORG:${c.organization}${c.department ? ';' + c.department : ''}\r\n`;
      if (c.email) vcard += `EMAIL;TYPE=INTERNET:${c.email}\r\n`;
      if (c.phone) vcard += `TEL;TYPE=CELL:${c.phone}\r\n`;
      if (c.office) vcard += `ADR;TYPE=WORK:;;${c.office};;\r\n`;
      if (c.notes) vcard += `NOTE:${c.notes.replace(/\n/g, '\n')}\r\n`;
      if (c.category) vcard += `CATEGORIES:${c.category}\r\n`;
      vcard += `END:VCARD`;
      return vcard;
    });
    res.setHeader('Content-Type', 'text/vcard; charset=utf-8');
    res.setHeader('Content-Disposition', 'attachment; filename="contacts.vcf"');
    res.send(vcards);
  } catch (err) {
    console.error("Error exporting contacts:", err);
    res.status(500).json({ error: "Failed to export contacts" });
  }
});

app.post("/api/key-contacts", async (req, res) => {
  try {
    const created = await storage.createKeyContact(req.body);
    res.json(created);
  } catch (err) {
    console.error("Error creating key contact:", err);
  }
});

```



```

    res.status(500).json({ error: "Failed to create key contact" });
  }
});

app.patch("/api/key-contacts/:id", async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const updated = await storage.updateKeyContact(id, req.body);
    res.json(updated);
  } catch (err) {
    console.error("Error updating key contact:", err);
    res.status(500).json({ error: "Failed to update key contact" });
  }
});

app.delete("/api/key-contacts/:id", async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    await storage.deleteKeyContact(id);
    res.json({ success: true });
  } catch (err) {
    console.error("Error deleting key contact:", err);
    res.status(500).json({ error: "Failed to delete key contact" });
  }
});

app.post("/api/key-contacts/sync-professor", async (req, res) => {
  try {
    const { professorName, professorEmail, courseCode } = req.body;
    if (!professorName?.trim()) {
      return res.json({ synced: false, reason: 'no name' });
    }
    const existing = await storage.getKeyContacts();
    const match = existing.find(c =>
      c.category === 'professor' && (
        (c.email && professorEmail && c.email.toLowerCase() === professorEmail.toLowerCase()) ||
        c.name.toLowerCase() === professorName.toLowerCase()
      )
    );
    if (match) {
      const updates: Record<string, string> = {};
      if (professorEmail && match.email !== professorEmail) updates.email = professorEmail;
      if (courseCode && (!match.organization || !match.organization.includes(courseCode))) {
        updates.organization = match.organization ? `${match.organization}, ${courseCode}` : courseCode;
      }
      if (Object.keys(updates).length > 0) {
        await storage.updateKeyContact(match.id, updates);
      }
      res.json({ synced: true, action: 'updated', id: match.id });
    } else {
      const created = await storage.createKeyContact({
        name: professorName.trim(),
        email: professorEmail?.trim() || null,
        category: 'professor',
        organization: courseCode || null,
      });
      res.json({ synced: true, action: 'created', id: created.id });
    }
  } catch (err) {
    console.error("Error syncing professor to contacts:", err);
    res.status(500).json({ error: "Failed to sync professor" });
  }
});

app.get("/api/entity-comments/:entityType/:entityId", async (req, res) => {
  try {
    const comments = await storage.getEntityComments(req.params.entityType, req.params.entityId);
    res.json(comments);
  } catch (err) {
    console.error("Error getting entity comments:", err);
    res.status(500).json({ error: "Failed to get comments" });
  }
});

app.put("/api/entity-comments/:entityType/:entityId", async (req, res) => {
  try {
    const { content } = req.body;
    if (content === undefined) return res.status(400).json({ error: "Content required" });
    if (!content.trim()) {
      const existing = await storage.getEntityComments(req.params.entityType, req.params.entityId);
      if (existing.length > 0) {
        await storage.deleteEntityComment(existing[0].id);
      }
      return res.json({ deleted: true });
    }
    const comment = await storage.upsertEntityComment(req.params.entityType, req.params.entityId, content);
    res.json(comment);
  } catch (err) {
    console.error("Error saving entity comment:", err);
    res.status(500).json({ error: "Failed to save comment" });
  }
});

app.delete("/api/entity-comments/:id", async (req, res) => {
  try {
    await storage.deleteEntityComment(parseInt(req.params.id));
    res.json({ success: true });
  } catch (err) {
    console.error("Error deleting entity comment:", err);
    res.status(500).json({ error: "Failed to delete comment" });
  }
});

app.post("/api/semester-reset-files", async (_req, res) => {
  try {
    const allFiles = await storage.getFiles();
    let resetCount = 0;
    for (const file of allFiles) {

```

```

        if (file.listened) {
            await storage.updateFile(file.id, { listened: false, lastChunkIndex: 0, totalChunks: 0, checkedChunks: '' });
            resetCount++;
        }
    }
    console.log(`[Semester Reset] Reset listened status on ${resetCount} files`);
    res.json({ success: true, resetCount });
} catch (err) {
    console.error("Error resetting files:", err);
    res.status(500).json({ error: "Failed to reset files" });
}
});

app.get("/api/feedback-notes", async (_req, res) => {
    try {
        const notes = await db.select().from(feedbackNotes).orderBy(sql`created_at DESC`);
        res.json(notes);
    } catch (err) {
        res.status(500).json({ error: "Failed to fetch feedback notes" });
    }
});

app.post("/api/feedback-notes", async (req, res) => {
    try {
        const parsed = insertFeedbackNoteSchema.parse(req.body);
        const [note] = await db.insert(feedbackNotes).values(parsed).returning();
        res.json(note);
    } catch (err) {
        res.status(500).json({ error: "Failed to save feedback note" });
    }
});

app.delete("/api/feedback-notes/:id", async (req, res) => {
    try {
        await db.delete(feedbackNotes).where(eq(feedbackNotes.id, parseInt(req.params.id)));
        res.json({ ok: true });
    } catch (err) {
        res.status(500).json({ error: "Failed to delete feedback note" });
    }
});

app.get("/api/shift-schedule", async (_req, res) => {
    try {
        res.set('Cache-Control', 'no-cache, no-store, must-revalidate');
        res.set('Pragma', 'no-cache');
        res.set('Expires', '0');
        const schedule = await storage.getShiftSchedule();
        res.json(schedule);
    } catch (err) {
        console.error("Error fetching shift schedule:", err);
        res.status(500).json({ error: "Failed to fetch shift schedule" });
    }
});

app.post("/api/shift-schedule", async (req, res) => {
    try {
        const validShiftTypes = ['day', 'night', 'off'];
        const dateRegex = /^\\d{4}-\\d{2}-\\d{2}$/;
        const { date, shiftType, bulk } = req.body;
        if (bulk && Array.isArray(bulk)) {
            const validatedBulk = bulk.filter((e: any) =>
                typeof e.date === 'string' && dateRegex.test(e.date) &&
                typeof e.shiftType === 'string' && validShiftTypes.includes(e.shiftType)
            );
            await storage.setShiftBulk(validatedBulk);
            const schedule = await storage.getShiftSchedule();
            return res.json(schedule);
        }
        if (!date || !shiftType) {
            return res.status(400).json({ error: "date and shiftType required" });
        }
        if (!dateRegex.test(date) || !validShiftTypes.includes(shiftType)) {
            return res.status(400).json({ error: "Invalid date format (YYYY-MM-DD) or shiftType (day/night/off)" });
        }
        const result = await storage.setShiftDay(date, shiftType);
        res.json(result);
    } catch (err) {
        console.error("Error setting shift schedule:", err);
        res.status(500).json({ error: "Failed to set shift schedule" });
    }
});

app.delete("/api/shift-schedule/:date", async (req, res) => {
    try {
        await storage.deleteShiftDay(req.params.date);
        res.json({ success: true });
    } catch (err) {
        console.error("Error deleting shift day:", err);
        res.status(500).json({ error: "Failed to delete shift day" });
    }
});

// POST /api/tasks/auto-create-file-tasks - Auto-create tasks for module/reading files without corresponding tasks
app.post("/api/tasks/auto-create-file-tasks", async (req, res) => {
    try {
        const { weekNumber } = req.body;
        if (!weekNumber) return res.status(400).json({ error: "weekNumber required" });

        const activeSemester = await storage.getActiveSemesterSettings();
        const semesterStart = activeSemester ? new Date(activeSemester.semesterStartDate) : undefined;
        const readingWeekStart = activeSemester?.readingWeekStart || null;
        const weekDates = getWeekDates(weekNumber, semesterStart, readingWeekStart);

        const friday = new Date(weekDates.start);
        friday.setDate(friday.getDate() + 6);
        friday.setHours(12, 0, 0, 0);

        const allFiles = await storage.getFiles();
    }
});

```

```

const allTasks = await storage.getTasks({ weekNumber });

const created: any[] = [];

for (const course of COURSES) {
  const courseCode = course.code.toLowerCase();
  const courseFullName = `${course.code} - ${course.name}`;
  const moduleFolderKey = `week-${weekNumber}-${courseCode}-module`;
  const readingFolderKey = `week-${weekNumber}-${courseCode}-reading`;

  const hasModuleFiles = allFiles.some(f => f.folder === moduleFolderKey);
  const hasReadingFiles = allFiles.some(f => f.folder === readingFolderKey);

  if (!hasModuleFiles && !hasReadingFiles) continue;

  const courseTasksForWeek = allTasks.filter(t => {
    const taskCourse = t.courseName?.split(' - ')[0]?.toUpperCase() || '';
    return taskCourse === course.code.toUpperCase() && (t.type === 'module' || t.type === 'reading');
  });

  if (courseTasksForWeek.length > 0) continue;

  const title = hasModuleFiles && hasReadingFiles
    ? `${course.code} Module & Reading`
    : hasModuleFiles
    ? `${course.code} Module`
    : `${course.code} Reading`;

  const newTask = await storage.createTask({
    title,
    type: "module",
    courseName: courseFullName,
    dueDate: friday,
    weekNumber,
    priority: "medium",
    description: "",
    isCompleted: false,
  } as InsertTask);

  created.push(newTask);
}

res.json({ created, count: created.length });
} catch (err) {
  console.error("Error auto-creating file tasks:", err);
  res.status(500).json({ error: "Failed to auto-create file tasks" });
}
});

// POST /api/semester/generate-class-tasks - Auto-create class tasks for virtual courses
app.post("/api/semester/generate-class-tasks", async (req, res) => {
  try {
    const activeSemester = await storage.getActiveSemesterSettings();
    if (!activeSemester) {
      return res.status(404).json({ error: "No active semester settings found" });
    }

    const validDeliveryModes = ['virtual', 'online', '', null];
    const validDays = ['monday', 'tuesday', 'wednesday', 'thursday', 'friday', 'saturday', 'sunday', '', null];

    const courseConfigs = [
      { prefix: 'course1' as const, code: activeSemester.course1Code, name: activeSemester.course1Name },
      { prefix: 'course2' as const, code: activeSemester.course2Code, name: activeSemester.course2Name },
      { prefix: 'course3' as const, code: activeSemester.course3Code, name: activeSemester.course3Name },
    ];

    const dayToNumber: Record<string, number> = {
      sunday: 0, monday: 1, tuesday: 2, wednesday: 3,
      thursday: 4, friday: 5, saturday: 6,
    };

    const allTasks = await storage.getTasks();
    const existingClassTasks = allTasks.filter(t => t.type === 'class');

    const createdTasks: any[] = [];
    let skippedCount = 0;

    for (const config of courseConfigs) {
      const deliveryMode = (activeSemester as any)[`${config.prefix}DeliveryMode`];
      if (deliveryMode !== 'virtual') continue;

      const classDay1 = (activeSemester as any)[`${config.prefix}ClassDay1`];
      const classDay2 = (activeSemester as any)[`${config.prefix}ClassDay2`];
      const classTime = (activeSemester as any)[`${config.prefix}ClassTime`] || '09:00';
      const classEndTime = (activeSemester as any)[`${config.prefix}ClassEndTime`] || '12:00';
      const courseStartDate = (activeSemester as any)[`${config.prefix}StartDate`];
      const courseEndDate = (activeSemester as any)[`${config.prefix}EndDate`];

      if (!validDays.includes(classDay1) || !validDays.includes(classDay2)) continue;

      const classDays: number[] = [];
      if (classDay1 && dayToNumber[classDay1] !== undefined) classDays.push(dayToNumber[classDay1]);
      if (classDay2 && dayToNumber[classDay2] !== undefined) classDays.push(dayToNumber[classDay2]);

      if (classDays.length === 0) continue;

      const semesterStart = courseStartDate ? new Date(courseStartDate) : new Date(activeSemester.semesterStartDate);
      const semesterEnd = courseEndDate
        ? new Date(courseEndDate)
        : (activeSemester.semesterEndDate ? new Date(activeSemester.semesterEndDate) : new Date(semesterStart.getTi...

      const courseName = config.name.startsWith(config.code) ? config.name : `${config.code} - ${config.name}`;
      const displayName = config.name.startsWith(config.code) ? config.name.replace(`${config.code} - `, '') : config...
      const dmLabel = deliveryMode === 'virtual' ? 'Online' : deliveryMode === 'online' ? 'Online' : deliveryMode =...
      const classTitle = dmLabel ? `${dmLabel} ${displayName} Class` : `${displayName} Class`;
      const [startHour, startMinute] = classTime.split(':').map(Number);
      const [endHour, endMinute] = classEndTime.split(':').map(Number);

```

```

const current = new Date(semesterStart);
while (current <= semesterEnd) {
  if (classDays.includes(current.getDay())) {
    const taskDate = new Date(current);
    taskDate.setHours(12, 0, 0, 0);

    const weekNum = getWeekNumber(taskDate, undefined, activeSemester?.readingWeekStart);
    if (weekNum >= FIRST_WEEK && weekNum <= LAST_WEEK) {
      const dateStr = formatLocalDate(taskDate);
      const isDuplicate = existingClassTasks.some(t => {
        if (!t.dueDate) return false;
        const existingDateStr = formatLocalDate(new Date(t.dueDate));
        return existingDateStr === dateStr
          && t.courseName === courseName
          && t.eventStartTime === classTime;
      });

      if (isDuplicate) {
        skippedCount++;
      } else {
        const task = await storage.createTask({
          title: classTitle,
          type: "class",
          courseName,
          dueDate: taskDate,
          eventStartTime: classTime,
          eventEndTime: classEndTime,
          weekNumber: weekNum,
          priority: "medium",
          reminder1: DEFAULT_REMINDER_1,
          reminder2: DEFAULT_REMINDER_2,
        });
        createdTasks.push(task);
      }
    }
    current.setDate(current.getDate() + 1);
  }
}

const message = skippedCount > 0
  ? `Created ${createdTasks.length} class tasks (${skippedCount} duplicates skipped)`
  : `Created ${createdTasks.length} class tasks`;

res.json({ message, tasks: createdTasks });
} catch (err) {
  console.error("Error generating class tasks:", err);
  res.status(500).json({ error: "Failed to generate class tasks" });
}
});

// ===== FILE MANAGEMENT ROUTES =====

// POST /api/files/sync-names - Sync file names to include course names and module numbers
app.post("/api/files/sync-names", async (_req, res) => {
  try {
    // First, delete duplicate files (keep only the one with the highest ID for each object_path)
    await db.execute(sql`
      DELETE FROM files
      WHERE id NOT IN (
        SELECT MAX(id) FROM files GROUP BY object_path
      )
    `);

    // Update CPPA122 files that don't have the course name yet
    await db.execute(sql`
      UPDATE files SET display_name = REPLACE(display_name, 'CPPA122', 'CPPA122 - Local Politics and Government, ')
      WHERE display_name LIKE 'CPPA122,%' AND display_name NOT LIKE 'CPPA122 - Local Politics and Government,%'
    `);

    // Update CFNF400 files that don't have the course name yet
    await db.execute(sql`
      UPDATE files SET display_name = REPLACE(display_name, 'CFNF400', 'CFNF400 - Human Sexuality, ')
      WHERE display_name LIKE 'CFNF400,%' AND display_name NOT LIKE 'CFNF400 - Human Sexuality,%'
    `);

    // Update CASL101 files that don't have the course name yet
    await db.execute(sql`
      UPDATE files SET display_name = REPLACE(display_name, 'CASL101', 'CASL101 - Sign Language, ')
      WHERE display_name LIKE 'CASL101,%' AND display_name NOT LIKE 'CASL101 - Sign Language,%'
    `);

    res.json({ success: true, message: "File names synced and duplicates removed" });
  } catch (err) {
    console.error("Error syncing file names:", err);
    res.status(500).json({ error: "Failed to sync file names" });
  }
});

// POST /api/cleanup-duplicates - Remove duplicate tasks and files (with CORS)
app.options("/api/cleanup-duplicates", (_req, res) => {
  res.header("Access-Control-Allow-Origin", "*");
  res.header("Access-Control-Allow-Methods", "POST, OPTIONS");
  res.header("Access-Control-Allow-Headers", "Content-Type");
  res.sendStatus(200);
});

app.post("/api/cleanup-duplicates", async (_req, res) => {
  res.header("Access-Control-Allow-Origin", "*");

  try {
    // Delete duplicate files (keep only the one with the highest ID for each object_path)
    await db.execute(sql`
      DELETE FROM files
      WHERE id NOT IN (
        SELECT MAX(id) FROM files GROUP BY object_path
      )
    `);
  }
});

```

```

// Delete duplicate tasks (keep only the one with highest ID for same title + due_date + course_name)
await db.execute(sql`
  DELETE FROM tasks
  WHERE id NOT IN (
    SELECT MAX(id) FROM tasks GROUP BY title, due_date, course_name
  )
`);

// Update file display names with course names
await db.execute(sql`
  UPDATE files SET display_name = REPLACE(display_name, 'CPPA122, ', 'CPPA122 - Local Politics and Government, ')
  WHERE display_name LIKE 'CPPA122,%' AND display_name NOT LIKE 'CPPA122 - Local Politics and Government,%'
`);
await db.execute(sql`
  UPDATE files SET display_name = REPLACE(display_name, 'CFNF400, ', 'CFNF400 - Human Sexuality, ')
  WHERE display_name LIKE 'CFNF400,%' AND display_name NOT LIKE 'CFNF400 - Human Sexuality,%'
`);
await db.execute(sql`
  UPDATE files SET display_name = REPLACE(display_name, 'CASL101, ', 'CASL101 - Sign Language, ')
  WHERE display_name LIKE 'CASL101,%' AND display_name NOT LIKE 'CASL101 - Sign Language,%'
`);

res.json({ success: true, message: "Duplicates removed and file names synced" });
} catch (err) {
  console.error("Error cleaning up duplicates:", err);
  res.status(500).json({ error: "Failed to clean up duplicates" });
}
});

// GET /api/onedrive/week-counts/:weekNum - Fetch file counts from OneDrive for a given week
app.get("/api/onedrive/week-counts/:weekNum", async (req, res) => {
  try {
    const weekNum = parseInt(req.params.weekNum);
    if (isNaN(weekNum)) return res.status(400).json({ error: "Invalid week number" });

    const semester = await storage.getActiveSemesterSettings();
    const year = semester?.semesterStartDate ? new Date(semester.semesterStartDate).getFullYear() : new Date().getFullYear();
    const semFolder = getSemesterTypeFolder(semester?.semesterType);
    const basePath = `/School/1. TMU/Courses/${year}/${semFolder}`;
    const courses: string[] = [];
    if (semester?.course1Code) courses.push(semester.course1Code);
    if (semester?.course2Code) courses.push(semester.course2Code);
    if (semester?.course3Code) courses.push(semester.course3Code);
    if (courses.length === 0) return res.json({});
    const counts: Record<string, { total: number; listened: number; unlistened: number }> = {};

    // Get list of course folders from OneDrive
    const { getOneDriveClient } = await import("./onedrive");
    const client = await getOneDriveClient();

    let baseFolders: any[] = [];
    try {
      const baseResp = await client.api(`/me/drive/root:${basePath}:children`).get();
      baseFolders = baseResp.value || [];
    } catch (e) {
      return res.json(counts);
    }

    // Check files from database first to get listened status
    const dbFiles = await storage.getFiles();
    const dbListened = new Set<string>();
    for (const f of dbFiles) {
      if (f.listened && f.originalName) {
        dbListened.add(f.originalName.toLowerCase());
      }
    }

    // Process each course in parallel
    await Promise.all(courses.map(async (courseCode) => {
      const courseId = courseCode.toLowerCase();
      try {
        const matchedFolder = baseFolders.find((f: any) =>
          f.folder && f.name.toUpperCase().startsWith(courseCode)
        );
        if (!matchedFolder) return;

        const coursePath = `${basePath}/${matchedFolder.name}`;
        const courseResp = await client.api(`/me/drive/root:${coursePath}:children`).get();
        const courseFolders = courseResp.value || [];

        const weekFolder = courseFolders.find((f: any) =>
          f.folder && f.name.toLowerCase().startsWith(`week ${weekNum}`)
        );
        if (!weekFolder) return;

        const weekPath = `${coursePath}/${weekFolder.name}`;
        const weekResp = await client.api(`/me/drive/root:${weekPath}:children`).get();
        const weekContents = weekResp.value || [];

        // Count files in Module and Reading subfolders
        for (const subfolder of weekContents) {
          if (!subfolder.folder) continue;
          const subName = subfolder.name.toLowerCase();
          let type: string | null = null;
          if (subName.includes('module')) type = 'module';
          else if (subName.includes('reading')) type = 'reading';
          if (!type) continue;

          const subPath = `${weekPath}/${subfolder.name}`;
          const subResp = await client.api(`/me/drive/root:${subPath}:children`).get();
          const files = (subResp.value || []).filter((f: any) => !f.folder);

          const key = `week-${weekNum}-${courseId}-${type}`;
          let listened = 0;
          let unlistened = 0;
          for (const file of files) {
            if (dbListened.has(file.name.toLowerCase())) {

```

```

        listened++;
    } else {
        unlisted++;
    }
}
counts[key] = { total: files.length, listened, unlisted };
}
} catch (e) {
    // Skip course on error
}
});
res.json(counts);
} catch (err) {
    console.error("Error fetching OneDrive week counts:", err);
    res.status(500).json({ error: "Failed to fetch OneDrive week counts" });
}
});

// GET /api/files/counts - Fast file counts by week/course/type with listened breakdown
// Returns: { "week-4-cppai22-module": { total: 3, listened: 1, unlisted: 2 }, ... }
app.get("/api/files/counts", async (_req, res) => {
    try {
        const files = await storage.getFiles();
        const counts: Record<string, { total: number; listened: number; unlisted: number; partialProgress: number }> = ...

        for (const file of files) {
            if (!file.folder) continue;

            const folderMatch = file.folder.match(/^week-(\d+)-([a-z0-9]+)-(module|reading|other)$/i);
            if (!folderMatch) continue;

            const folderKey = file.folder.toLowerCase();
            if (!counts[folderKey]) {
                counts[folderKey] = { total: 0, listened: 0, unlisted: 0, partialProgress: 0 };
            }

            const isPrepared = file.listened || (file.totalChunks && file.totalChunks > 0);
            if (!isPrepared) continue;

            counts[folderKey].total++;
            if (file.listened) {
                counts[folderKey].listened++;
                counts[folderKey].partialProgress += 100;
            } else {
                counts[folderKey].unlisted++;
                if (file.checkedChunks && file.checkedChunks !== 'null' && file.checkedChunks !== '[]' && file.totalChunks ...)
                    try {
                        const checked = JSON.parse(file.checkedChunks);
                        if (Array.isArray(checked) && checked.length > 0) {
                            counts[folderKey].partialProgress += Math.round((checked.length / file.totalChunks) * 100);
                        } else if (file.lastChunkIndex !== null && file.lastChunkIndex > 0) {
                            counts[folderKey].partialProgress += Math.round((file.lastChunkIndex / file.totalChunks) * 100);
                        }
                    } catch {
                        if (file.lastChunkIndex !== null && file.lastChunkIndex > 0) {
                            counts[folderKey].partialProgress += Math.round((file.lastChunkIndex / file.totalChunks) * 100);
                        }
                    }
                } else if (file.totalChunks && file.totalChunks > 0 && file.lastChunkIndex !== null && file.lastChunkIndex > ...)
                    counts[folderKey].partialProgress += Math.round((file.lastChunkIndex / file.totalChunks) * 100);
            }
        }
    }
    res.json(counts);
} catch (err) {
    console.error("Error fetching file counts:", err);
    res.status(500).json({ error: "Failed to fetch file counts" });
}
});

// GET /api/files - List all uploaded files
app.get("/api/files", async (_req, res) => {
    try {
        const files = await storage.getFiles();
        res.json(files);
    } catch (err) {
        console.error("Error fetching files:", err);
        res.status(500).json({ error: "Failed to fetch files" });
    }
});

// GET /api/files/recently-prepared (must be before :id route)
app.get("/api/files/recently-prepared", async (_req, res) => {
    res.json({ files: (globalThis as any).__recentlyPreparedFiles || [] });
});

// GET /api/files/:id - Get single file
app.get("/api/files/:id", async (req, res) => {
    try {
        const id = Number(req.params.id);
        if (isNaN(id)) {
            return res.status(400).json({ error: "Invalid file ID" });
        }
        const file = await storage.getFile(id);
        if (!file) {
            return res.status(404).json({ error: "File not found" });
        }
        res.json(file);
    } catch (err) {
        console.error("Error fetching file:", err);
        res.status(500).json({ error: "Failed to fetch file" });
    }
});

// GET /api/proxy-pdf - Proxy a PDF from an external URL (e.g., OneDrive download URL)
app.get("/api/proxy-pdf", async (req, res) => {

```

```

try {
  const url = req.query.url as string;
  if (!url || !url.startsWith('http')) {
    return res.status(400).json({ error: "Valid URL is required" });
  }

  const pdfResponse = await fetch(url);
  if (!pdfResponse.ok) {
    return res.status(502).json({ error: `Failed to fetch PDF: ${pdfResponse.status}` });
  }

  res.setHeader('Content-Type', 'application/pdf');
  const contentLength = pdfResponse.headers.get('content-length');
  if (contentLength) {
    res.setHeader('Content-Length', contentLength);
  }

  if (pdfResponse.body) {
    const reader = pdfResponse.body.getReader();
    while (true) {
      const { done, value } = await reader.read();
      if (done) break;
      res.write(value);
    }
    res.end();
  } else {
    const buffer = await pdfResponse.arrayBuffer();
    res.send(Buffer.from(buffer));
  }
} catch (err) {
  console.error("Error proxying PDF:", err);
  res.status(500).json({ error: "Failed to proxy PDF" });
}
});

// GET /api/files/:id/download - Download the actual file
app.get("/api/files/:id/download", async (req, res) => {
  try {
    const file = await storage.getFile(Number(req.params.id));
    if (!file) {
      return res.status(404).json({ error: "File not found" });
    }

    const mediaUrl = file.objectPath;

    if (mediaUrl.startsWith("/objects/")) {
      const { ObjectStorageService } = await import("./replit_integrations/object_storage");
      const objectStorageService = new ObjectStorageService();
      const objectFile = await objectStorageService.getObjectEntityFile(mediaUrl);

      await objectStorageService.downloadObject(objectFile, res);
    } else if (mediaUrl.startsWith("onedrive://")) {
      // OneDrive file - look up the actual download URL from OneDrive
      const fileName = mediaUrl.split('/').pop() || '';
      const folderPart = mediaUrl.replace('onedrive://', '').split('/')[0] || '';
      // Parse folder like "week-5-cppa122-module" to find the OneDrive path
      const parts = folderPart.split('-');
      const weekNum = parts[1];
      const courseCode = parts[2]?.toUpperCase();

      if (courseCode && weekNum) {
        try {
          const basePath = `School/1. TMU/Courses/2026/Winter`;
          const baseFolders = await listOneDriveItems(basePath);
          const matchedFolder = baseFolders.find((f: any) =>
            f.type === 'folder' && f.name.toUpperCase().startsWith(courseCode)
          );
          if (matchedFolder) {
            const courseFolders = await listOneDriveItems(matchedFolder.path);
            const weekFolder = courseFolders.find((f: any) =>
              f.type === 'folder' && f.name.toLowerCase().startsWith(`week ${weekNum}`)
            );
            if (weekFolder) {
              const weekContents = await listOneDriveItems(weekFolder.path);
              const typeFolder = weekContents.find((f: any) =>
                f.type === 'folder' && f.name.toLowerCase().includes(folderPart.includes('reading') ? 'reading' : '...')
              );
              if (typeFolder) {
                const files = await listOneDriveItems(typeFolder.path);
                const matchedFile = files.find((f: any) => f.name === fileName);
                if (matchedFile?.downloadUrl) {
                  const pdfResponse = await fetch(matchedFile.downloadUrl);
                  if (pdfResponse.ok && pdfResponse.body) {
                    res.setHeader('Content-Type', 'application/pdf');
                    res.setHeader('Content-Disposition', 'inline; filename="${fileName}"');
                    const reader = pdfResponse.body.getReader();
                    const pump = async () => {
                      while (true) {
                        const { done, value } = await reader.read();
                        if (done) break;
                        res.write(value);
                      }
                      res.end();
                    };
                    await pump();
                    return;
                  }
                }
              }
            }
          }
        } catch (onedriveErr) {
          return res.status(404).json({ error: "OneDrive file not found" });
        }
      }
    } catch (onedriveErr) {
      console.error("Error fetching from OneDrive:", onedriveErr);
      return res.status(500).json({ error: "Failed to fetch OneDrive file" });
    }
  } else {
    return res.status(400).json({ error: "Invalid OneDrive path format" });
  }
}

```

```

    }
  } else if (mediaUrl.startsWith("/School/")) {
    try {
      const { getOneDriveClient } = await import("./onedrive");
      const client = await getOneDriveClient();
      const encodedPath = encodeURIComponent(mediaUrl.replace(/%2F/g, '/'));
      const item = await client.api('/me/drive/root:${encodedPath}`).get();
      const downloadUrl = item['@microsoft.graph.downloadUrl'];
      if (!downloadUrl) {
        return res.status(400).json({ error: "Could not get OneDrive download URL" });
      }
      const pdfResponse = await fetch(downloadUrl);
      if (pdfResponse.ok && pdfResponse.body) {
        res.setHeader('Content-Type', pdfResponse.headers.get('content-type') || 'application/pdf');
        res.setHeader('Content-Disposition', `inline; filename="${mediaUrl.split('/').pop()}"`);
        const reader = pdfResponse.body.getReader();
        while (true) {
          const { done, value } = await reader.read();
          if (done) break;
          res.write(value);
        }
        res.end();
      } else {
        res.status(502).json({ error: "Failed to fetch file from OneDrive" });
      }
    } catch (fetchErr) {
      console.error("Error fetching OneDrive file for download:", fetchErr);
      res.status(500).json({ error: "Failed to fetch OneDrive file" });
    }
  } else if (mediaUrl.startsWith("http")) {
    // External URL - proxy the download to avoid CORS issues
    try {
      const pdfResponse = await fetch(mediaUrl);
      if (pdfResponse.ok && pdfResponse.body) {
        res.setHeader('Content-Type', pdfResponse.headers.get('content-type') || 'application/pdf');
        const reader = pdfResponse.body.getReader();
        while (true) {
          const { done, value } = await reader.read();
          if (done) break;
          res.write(value);
        }
        res.end();
      } else {
        res.status(502).json({ error: "Failed to fetch external file" });
      }
    } catch (fetchErr) {
      console.error("Error proxying file:", fetchErr);
      res.status(502).json({ error: "Failed to proxy file download" });
    }
  } else {
    res.status(400).json({ error: "Unsupported file path format" });
  }
} catch (err) {
  console.error("Error downloading file:", err);
  res.status(500).json({ error: "Failed to download file" });
}
});

// GET /api/files/:id/text - Extract text content from a file (for PDF reading with highlighting)
app.get("/api/files/:id/text", async (req, res) => {
  try {
    const file = await storage.getFile(Number(req.params.id));
    if (!file) {
      return res.status(404).json({ error: "File not found" });
    }

    if (file.extractedText) {
      console.log(`[TextAPI] Using cached extractedText for file ${file.id} (${file.extractedText.length} chars)`);
      return res.json({ text: file.extractedText });
    }

    const mediaUrl = file.objectPath;
    let textContent = "";
    let fileBuffer: Buffer | null = null;

    if (mediaUrl.startsWith("/objects/")) {
      const { ObjectStorageService } = await import("./replit_integrations/object_storage");
      const objectStorage = new ObjectStorageService();

      try {
        const objectFile = await objectStorage.getObjectEntityFile(mediaUrl);
        const [content] = await objectFile.download();
        fileBuffer = content;
      } catch (error) {
        console.error("Error reading from object storage:", error);
        return res.status(400).json({ error: "Failed to read file from storage" });
      }
    } else if (mediaUrl.startsWith("/School/") || mediaUrl.startsWith("onedrive://")) {
      try {
        const { getOneDriveClient } = await import("./onedrive");
        const client = await getOneDriveClient();
        const encodedPath = encodeURIComponent(mediaUrl.replace(/%2F/g, '/'));
        const item = await client.api('/me/drive/root:${encodedPath}`).get();
        const downloadUrl = item['@microsoft.graph.downloadUrl'];
        if (!downloadUrl) {
          return res.status(400).json({ error: "Could not get OneDrive download URL" });
        }
        const fileResponse = await fetch(downloadUrl);
        if (!fileResponse.ok) {
          return res.status(400).json({ error: "Failed to fetch file from OneDrive" });
        }
        const arrayBuffer = await fileResponse.arrayBuffer();
        fileBuffer = Buffer.from(arrayBuffer);
      } catch (error) {
        console.error("Error fetching from OneDrive for text extraction:", error);
        return res.status(400).json({ error: "Failed to fetch file from OneDrive" });
      }
    } else if (mediaUrl.startsWith("http")) {

```



```

const fileResponse = await fetch(mediaUrl);
if (!fileResponse.ok) {
  return res.status(400).json({ error: "Failed to fetch file content" });
}
const arrayBuffer = await fileResponse.arrayBuffer();
fileBuffer = Buffer.from(arrayBuffer);
} else {
  return res.status(400).json({ error: "Unsupported file path format" });
}

if (!fileBuffer) {
  return res.status(400).json({ error: "Failed to read file" });
}

const isPDF = fileBuffer.slice(0, 4).toString() === '%PDF';

if (isPDF) {
  try {
    const PdfParser = await getPdfParser();
    const parser = new PdfParser({ data: new Uint8Array(fileBuffer), verbosity: 0 });
    await parser.load();
    const pdfText = await parser.getText();

    const PAGE_BREAK_MARKER = '\n\n---PAGE---\n\n';

    console.log(`[PDF Parse] getText type: ${typeof pdfText}, isArray: ${Array.isArray(pdfText)}, keys: ${pdfTe...`);
    if (pdfText && typeof pdfText === 'object' && (pdfText as any).pages) {
      console.log(`[PDF Parse] pages count: ${((pdfText as any).pages.length)}`);
    }

    if (pdfText && typeof pdfText === 'object') {
      if (pdfText.pages && Array.isArray(pdfText.pages)) {
        console.log(`[PDF Parse] Extracting from ${pdfText.pages.length} pages`);
        textContent = pdfText.pages.map((page: any) => page.text || '').join(PAGE_BREAK_MARKER);
      } else if (Array.isArray(pdfText)) {
        console.log(`[PDF Parse] Array with ${pdfText.length} items`);
        textContent = pdfText.map((item: any) => typeof item === 'string' ? item : item.text || '').join(PAGE_B...`);
      } else if (pdfText.text) {
        textContent = pdfText.text;
      } else {
        textContent = Object.values(pdfText).filter(v => typeof v === 'string').join(PAGE_BREAK_MARKER);
      }
    } else if (typeof pdfText === 'string') {
      textContent = pdfText;
    } else {
      textContent = String(pdfText || '');
    }
    console.log(`[PDF Parse] Raw extracted text length: ${textContent.length}`);
    await parser.destroy();
  } catch (error) {
    console.error("Error parsing PDF:", error);
    return res.status(400).json({ error: "Failed to parse PDF" });
  }
} else {
  textContent = fileBuffer.toString('utf-8');
}

// Filter out boxed content (sidebars, callout boxes, etc.)
// Heuristics: detect blocks that appear to be in boxes based on formatting patterns
const filterBoxedContent = (text: string): string => {
  return text;
};

// Apply boxed content filter
textContent = filterBoxedContent(textContent);

// Apply the same TTS cleaning to display text (removes French, JSTOR refs, URLs, etc.)
// This ensures what users see matches what they hear
textContent = cleanTextForTTS(textContent);

// Restore some formatting for display (TTS cleaning removes newlines)
// Re-add paragraph breaks where there were multiple spaces
textContent = textContent
  .replace(/\. {2,}/g, '\n\n')
  .replace(/\\s+/g, ' ')
  .trim();

if (textContent && file.id) {
  storage.updateFile(file.id, { extractedText: textContent }).then(() => {
    console.log(`[TextAPI] Cached ${textContent.length} chars for file ${file.id}`);
  }).catch((e: any) => console.error(`[TextAPI] Cache save failed:`, e.message));
}

res.json({ text: textContent, fileName: file.displayName || file.originalName });
} catch (err) {
  console.error("Error extracting file text:", err);
  res.status(500).json({ error: "Failed to extract file text" });
}
});

// POST /api/extract-text-from-url - Extract text from a PDF at a given URL (for OneDrive files)
app.post("/api/extract-text-from-url", async (req, res) => {
  try {
    let { url } = req.body;
    if (!url) {
      return res.status(400).json({ error: "URL is required" });
    }

    // Decode HTML entities in URL (OneDrive returns & instead of &)
    url = url.replace(/&/g, '&');
    console.log("Extracting text from URL:", url.substring(0, 100) + "...");

    // Fetch the PDF from the URL
    const response = await fetch(url);
    if (!response.ok) {
      console.error("Failed to fetch file, status:", response.status, response.statusText);
      return res.status(400).json({ error: `Failed to fetch file from URL: ${response.status}` });
    }
  }

```

```

console.log("Successfully fetched PDF, extracting text...");

const buffer = Buffer.from(await response.arrayBuffer());

// Extract text using pdf-parse (same pattern as file extraction)
const PdfParser = await getPdfParser();
const parser = new PdfParser({ data: new Uint8Array(buffer), verbosity: 0 });
const pdfText = await parser.getText();

// Use PAGE_BREAK_MARKER for page breaks
const PAGE_BREAK_MARKER = '\n\n---PAGE---\n\n';
let textContent = '';

if (pdfText && typeof pdfText === 'object') {
  if (pdfText.pages && Array.isArray(pdfText.pages)) {
    textContent = pdfText.pages.map((page: any) => page.text || '').join(PAGE_BREAK_MARKER);
  } else if (Array.isArray(pdfText)) {
    textContent = pdfText.map((item: any) => typeof item === 'string' ? item : item.text || '').join(PAGE_BREAK...
  } else if (pdfText.text) {
    textContent = pdfText.text;
  } else {
    textContent = Object.values(pdfText).filter(v => typeof v === 'string').join(PAGE_BREAK_MARKER);
  }
} else if (typeof pdfText === 'string') {
  textContent = pdfText;
} else {
  textContent = String(pdfText || '');
}
await parser.destroy();

// Clean up text
textContent = textContent
  .replace(/\r\n/g, '\n')
  .replace(/\s\n/g, '\n')
  .replace(/\n{3,}/g, '\n\n')
  .replace(/\t/g, ' ')
  .replace(/ {2,}/g, ' ')
  .trim();

console.log("Text extraction complete. Length:", textContent.length, "chars, first 100:", textContent.substring...
res.json({ text: textContent });
} catch (err) {
  console.error("Error extracting text from URL:", err);
  res.status(500).json({ error: "Failed to extract text from URL" });
}
});

// PATCH /api/files/:id - Update file (rename, change folder, mark listened, or save progress)
app.patch("/api/files/:id", async (req, res) => {
  try {
    const { displayName, folder, listened, lastChunkIndex, totalChunks, checkedChunks } = req.body;
    if (!displayName && folder === undefined && listened === undefined && lastChunkIndex === undefined && totalChun...
      return res.status(400).json({ error: "displayName, folder, listened, lastChunkIndex, totalChunks, or checkedC...
    }
    const file = await storage.updateFile(Number(req.params.id), { displayName, folder, listened, lastChunkIndex, t...
    if (!file) {
      return res.status(404).json({ error: "File not found" });
    }
    res.json(file);
  } catch (err) {
    console.error("Error updating file:", err);
    res.status(500).json({ error: "Failed to update file" });
  }
});

// POST /api/files/ensure - Create or find a file by objectPath (for OneDrive files that need DB entries)
app.post("/api/files/ensure", async (req, res) => {
  try {
    const { objectPath, originalName, displayName, folder } = req.body;
    if (!objectPath || !originalName) {
      return res.status(400).json({ error: "objectPath and originalName are required" });
    }

    // Check if file already exists by objectPath
    let file = await storage.getFileByPath(objectPath);
    if (file) {
      // Update folder if provided and different
      if (folder && file.folder !== folder) {
        file = await storage.updateFile(file.id, { folder }) || file;
      }
      return res.json(file);
    }

    // Fallback: check by originalName + folder to prevent duplicates
    // (same file may exist under a different objectPath, e.g. uploaded vs OneDrive path)
    if (folder) {
      const allFiles = await storage.getFiles();
      const match = allFiles.find(f => f.originalName === originalName && f.folder === folder);
      if (match) {
        // Update the objectPath to the new stable path so future lookups match
        file = await storage.updateFile(match.id, { objectPath }) || match;
        return res.json(file);
      }
    }

    // Create new file entry
    file = await storage.createFile({
      objectPath,
      originalName,
      displayName: displayName || originalName,
      folder: folder || null,
      contentType: 'application/pdf',
      size: 0,
      listened: false,
      lastChunkIndex: 0,
      totalChunks: 0,
    });
  }
});

```

```

    res.json(file);
  } catch (err) {
    console.error("Error ensuring file:", err);
    res.status(500).json({ error: "Failed to ensure file" });
  }
});

// POST /api/files/:id/assign - Assign file to a task
app.post("/api/files/:id/assign", async (req, res) => {
  try {
    const { taskId } = req.body;
    if (!taskId) {
      return res.status(400).json({ error: "taskId is required" });
    }

    const file = await storage.getFile(Number(req.params.id));
    if (!file) {
      return res.status(404).json({ error: "File not found" });
    }

    const task = await storage.getTask(Number(taskId));
    if (!task) {
      return res.status(404).json({ error: "Task not found" });
    }

    // Add file to task attachments if not already there
    const currentAttachments = task.attachments || [];
    if (!currentAttachments.includes(file.objectPath)) {
      const updatedTask = await storage.updateTask(task.id, {
        attachments: [...currentAttachments, file.objectPath],
      });
      res.json({ success: true, task: updatedTask, file });
    } else {
      res.json({ success: true, message: "File already attached to task", task, file });
    }
  } catch (err) {
    console.error("Error assigning file:", err);
    res.status(500).json({ error: "Failed to assign file to task" });
  }
});

// POST /api/files/assign-by-path - Assign file to a task by object path
app.post("/api/files/assign-by-path", async (req, res) => {
  try {
    const { objectPath, taskId } = req.body;
    if (!objectPath || !taskId) {
      return res.status(400).json({ error: "objectPath and taskId are required" });
    }

    const file = await storage.getFileByPath(objectPath);
    if (!file) {
      return res.status(404).json({ error: "File not found" });
    }

    const task = await storage.getTask(Number(taskId));
    if (!task) {
      return res.status(404).json({ error: "Task not found" });
    }

    // Add file to task attachments if not already there
    const currentAttachments = task.attachments || [];
    if (!currentAttachments.includes(file.objectPath)) {
      const updatedTask = await storage.updateTask(task.id, {
        attachments: [...currentAttachments, file.objectPath],
      });
      res.json({ success: true, task: updatedTask, file });
    } else {
      res.json({ success: true, message: "File already attached to task", task, file });
    }
  } catch (err) {
    console.error("Error assigning file by path:", err);
    res.status(500).json({ error: "Failed to assign file to task" });
  }
});

// DELETE /api/files/:id - Delete file and remove from all task attachments
app.delete("/api/files/:id", async (req, res) => {
  try {
    const file = await storage.getFile(Number(req.params.id));
    if (!file) {
      return res.status(404).json({ error: "File not found" });
    }

    // Remove file reference from all tasks that have it attached
    const allTasks = await storage.getTasks({});
    for (const task of allTasks) {
      if (task.attachments && task.attachments.includes(file.objectPath)) {
        const updatedAttachments = task.attachments.filter(a => a !== file.objectPath);
        await storage.updateTask(task.id, { attachments: updatedAttachments });
      }
    }

    await storage.deleteFile(Number(req.params.id));
    res.status(204).end();
  } catch (err) {
    console.error("Error deleting file:", err);
    res.status(500).json({ error: "Failed to delete file" });
  }
});

// GET /api/files/:id/tasks - Get tasks that have this file attached
app.get("/api/files/:id/tasks", async (req, res) => {
  try {
    const file = await storage.getFile(Number(req.params.id));
    if (!file) {
      return res.status(404).json({ error: "File not found" });
    }
  }
});

```

```

    const allTasks = await storage.getTasks({});
    const tasksWithFile = allTasks.filter(task =>
      task.attachments && task.attachments.includes(file.objectPath)
    );

    res.json(tasksWithFile);
  } catch (err) {
    console.error("Error fetching tasks for file:", err);
    res.status(500).json({ error: "Failed to fetch tasks" });
  }
});

// ===== END FILE MANAGEMENT ROUTES =====

// ===== DELETED FOLDERS ROUTES =====

// GET /api/deleted-folders - Get all deleted folders
app.get("/api/deleted-folders", async (_req, res) => {
  try {
    const folders = await storage.getDeletedFolders();
    res.json(folders);
  } catch (err) {
    console.error("Error fetching deleted folders:", err);
    res.status(500).json({ error: "Failed to fetch deleted folders" });
  }
});

// POST /api/deleted-folders - Add a folder to deleted list
app.post("/api/deleted-folders", async (req, res) => {
  try {
    const { folderId } = req.body;
    if (!folderId) {
      return res.status(400).json({ error: "folderId is required" });
    }
    const folder = await storage.addDeletedFolder(folderId);
    res.json(folder);
  } catch (err) {
    console.error("Error adding deleted folder:", err);
    res.status(500).json({ error: "Failed to add deleted folder" });
  }
});

// DELETE /api/deleted-folders/:folderId - Remove a folder from deleted list (restore it)
app.delete("/api/deleted-folders/:folderId", async (req, res) => {
  try {
    const folderId = decodeURIComponent(req.params.folderId);
    await storage.removeDeletedFolder(folderId);
    res.json({ success: true });
  } catch (err) {
    console.error("Error removing deleted folder:", err);
    res.status(500).json({ error: "Failed to remove deleted folder" });
  }
});

// ===== END DELETED FOLDERS ROUTES =====

// ===== CUSTOM FOLDERS ROUTES =====

// GET /api/custom-folders - Get all custom folders
app.get("/api/custom-folders", async (_req, res) => {
  try {
    const folders = await storage.getCustomFolders();
    res.json(folders);
  } catch (err) {
    console.error("Error fetching custom folders:", err);
    res.status(500).json({ error: "Failed to fetch custom folders" });
  }
});

// POST /api/custom-folders - Create a custom folder
app.post("/api/custom-folders", async (req, res) => {
  try {
    const { parentFolderId, name } = req.body;
    if (!parentFolderId || !name) {
      return res.status(400).json({ error: "parentFolderId and name are required" });
    }
    const folder = await storage.createCustomFolder({ parentFolderId, name });
    res.json(folder);
  } catch (err) {
    console.error("Error creating custom folder:", err);
    res.status(500).json({ error: "Failed to create custom folder" });
  }
});

// PATCH /api/custom-folders/:id - Rename a custom folder
app.patch("/api/custom-folders/:id", async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const { name } = req.body;
    if (!name) {
      return res.status(400).json({ error: "name is required" });
    }
    const folder = await storage.updateCustomFolder(id, name);
    res.json(folder);
  } catch (err) {
    console.error("Error renaming custom folder:", err);
    res.status(500).json({ error: "Failed to rename custom folder" });
  }
});

// DELETE /api/custom-folders/:id - Delete a custom folder
app.delete("/api/custom-folders/:id", async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    await storage.deleteCustomFolder(id);
    res.json({ success: true });
  } catch (err) {
    console.error("Error deleting custom folder:", err);
  }
});

```

```

    res.status(500).json({ error: "Failed to delete custom folder" });
  }
});

// DELETE /api/custom-folders - Delete ALL custom folders
app.delete("/api/custom-folders", async (_req, res) => {
  try {
    const folders = await storage.getCustomFolders();
    for (const folder of folders) {
      await storage.deleteCustomFolder(folder.id);
    }
    res.json({ success: true, deleted: folders.length });
  } catch (err) {
    console.error("Error deleting all custom folders:", err);
    res.status(500).json({ error: "Failed to delete custom folders" });
  }
});

// ===== END CUSTOM FOLDERS ROUTES =====

// ===== ONEDRIVE ROUTES =====

// GET /api/onedrive/files - List files in a OneDrive folder
app.get("/api/onedrive/files", async (req, res) => {
  try {
    const path = (req.query.path as string) || '/';
    const items = await listOneDriveItems(path);
    res.json(items);
  } catch (err: any) {
    console.error("Error listing OneDrive files:", err);
    res.status(500).json({ error: err.message || "Failed to list OneDrive files" });
  }
});

// GET /api/onedrive/file/:id - Get file details and download URL
app.get("/api/onedrive/file/:id", async (req, res) => {
  try {
    const itemId = req.params.id;
    const file = await getOneDriveFile(itemId);
    res.json(file);
  } catch (err: any) {
    console.error("Error getting OneDrive file:", err);
    res.status(500).json({ error: err.message || "Failed to get OneDrive file" });
  }
});

// GET /api/onedrive/search - Search for files
app.get("/api/onedrive/search", async (req, res) => {
  try {
    const query = req.query.q as string;
    if (!query) {
      return res.status(400).json({ error: "Search query is required" });
    }
    const items = await searchOneDriveFiles(query);
    res.json(items);
  } catch (err: any) {
    console.error("Error searching OneDrive:", err);
    res.status(500).json({ error: err.message || "Failed to search OneDrive" });
  }
});

const QUICKNOTES_PATH = '/QuickNotes';
const QUICKNOTES_DEFAULT_FILE = 'notes.txt';

app.get("/api/quicknotes/debug", async (req, res) => {
  try {
    const path = (req.query.path as string) || QUICKNOTES_PATH;
    const items = await listOneDriveItems(path);
    res.json(items);
  } catch (err: any) {
    res.status(500).json({ error: err.message });
  }
});

app.get("/api/quicknotes/files", async (_req, res) => {
  try {
    const items = await listOneDriveItems(QUICKNOTES_PATH);
    const notes = items.filter((f: any) => {
      const name = (f.name || '').toLowerCase();
      return name.endsWith('.txt') || name.endsWith('.md') || name.endsWith('.html');
    });
    res.json(notes);
  } catch (err: any) {
    if (err.statusCode === 404 || err.code === 'itemNotFound' || err.message?.includes('itemNotFound') || err.messa...) {
      try {
        await createOneDriveFolder('/', 'QuickNotes');
        await createOneDriveTextFile(QUICKNOTES_PATH, QUICKNOTES_DEFAULT_FILE, 'Type your notes here from your phon...');
        const items = await listOneDriveItems(QUICKNOTES_PATH);
        const notes = items.filter((f: any) => {
          const name = (f.name || '').toLowerCase();
          return name.endsWith('.txt') || name.endsWith('.md') || name.endsWith('.html');
        });
        res.json(notes);
      } catch (createErr: any) {
        console.error("Error creating QuickNotes folder:", createErr);
        res.status(500).json({ error: createErr.message || "Failed to create QuickNotes folder" });
      }
    } else {
      console.error("Error listing QuickNotes:", err);
      res.status(500).json({ error: err.message || "Failed to list notes" });
    }
  }
});

app.get("/api/quicknotes/file/:id/content", async (req, res) => {
  try {
    const content = await getOneDriveFileContentAsText(req.params.id);
    res.json({ content });
  }
});

```

```

    } catch (err: any) {
      console.error("Error getting QuickNotes content:", err);
      res.status(500).json({ error: err.message || "Failed to get note content" });
    }
  });

app.get("/api/quicknotes/file/:id/meta", async (req, res) => {
  try {
    const meta = await getOneDriveFile(req.params.id);
    res.json(meta);
  } catch (err: any) {
    console.error("Error getting QuickNotes meta:", err);
    res.status(500).json({ error: err.message || "Failed to get note metadata" });
  }
});

app.put("/api/quicknotes/file/:id/content", async (req, res) => {
  try {
    const { content } = req.body;
    if (typeof content !== 'string') {
      return res.status(400).json({ error: "content field required" });
    }
    const result = await updateOneDriveFileContent(req.params.id, content);
    res.json(result);
  } catch (err: any) {
    console.error("Error saving QuickNotes content:", err);
    res.status(500).json({ error: err.message || "Failed to save note" });
  }
});

app.post("/api/quicknotes/files", async (req, res) => {
  try {
    const { name, content } = req.body;
    const fileName = name || `Note ${new Date().toLocaleString('en-US', { month: 'short', day: 'numeric', hour: 'nu...
    const result = await createOneDriveTextFile(QUICKNOTES_PATH, fileName, content || '');
    res.json(result);
  } catch (err: any) {
    console.error("Error creating QuickNote:", err);
    res.status(500).json({ error: err.message || "Failed to create note" });
  }
});

app.patch("/api/quicknotes/file/:id/rename", async (req, res) => {
  try {
    const { name } = req.body;
    if (!name || typeof name !== 'string') {
      return res.status(400).json({ error: "Name is required" });
    }
    const { renameOneDriveItem } = await import("./onedrive");
    await renameOneDriveItem(req.params.id, name);
    res.json({ success: true, name });
  } catch (err: any) {
    console.error("Error renaming QuickNote:", err);
    res.status(500).json({ error: err.message || "Failed to rename note" });
  }
});

app.delete("/api/quicknotes/file/:id", async (req, res) => {
  try {
    await deleteOneDriveItem(req.params.id);
    res.json({ success: true });
  } catch (err: any) {
    console.error("Error deleting QuickNote:", err);
    res.status(500).json({ error: err.message || "Failed to delete note" });
  }
});

app.get("/api/quicknotes/search", async (req, res) => {
  try {
    const q = (req.query.q as string) || '';
    if (!q.trim()) return res.json([]);
    const allFiles = await listOneDriveItems(QUICKNOTES_PATH);
    const textFiles = allFiles.filter((f: any) => {
      const name = (f.name || '').toLowerCase();
      return name.endsWith('.txt') || name.endsWith('.md') || name.endsWith('.html');
    });
    const matching = textFiles.filter((f: any) => f.name.toLowerCase().includes(q.toLowerCase()));
    res.json(matching);
  } catch (err: any) {
    console.error("Error searching QuickNotes:", err);
    res.status(500).json({ error: err.message || "Failed to search notes" });
  }
});

app.get("/api/onenote/notebooks", async (req, res) => {
  try {
    const { listOneNoteNotebooks } = await import("./onedrive");
    const notebooks = await listOneNoteNotebooks();
    res.json(notebooks);
  } catch (err: any) {
    console.error("Error listing OneNote notebooks:", err);
    res.status(500).json({ error: err.message || "Failed to list notebooks" });
  }
});

app.get("/api/onenote/pages", async (req, res) => {
  try {
    const notebookPath = req.query.notebook as string;
    const section = req.query.section as string;
    const notebookName = req.query.notebookName as string;
    if (!section) {
      return res.status(400).json({ error: "section query param required" });
    }
    if (notebookName) {
      const { getOneNotePagesViaApi } = await import("./onedrive");
      const pages = await getOneNotePagesViaApi(notebookName, section);
      return res.json(pages);
    }
  }
});

```

```

    if (!notebookPath) {
      return res.status(400).json({ error: "notebook or notebookName query param required" });
    }
    const { getOneNotePages } = await import("./onedrive");
    const pages = await getOneNotePages(notebookPath, section + '.one');
    res.json(pages);
  } catch (err: any) {
    console.error("Error getting OneNote pages:", err);
    res.status(500).json({ error: err.message || "Failed to get pages" });
  }
});

app.post("/api/onenote/pages", async (req, res) => {
  try {
    const { notebook, section, title, content } = req.body;
    if (!notebook || !section || !title) {
      return res.status(400).json({ error: "notebook, section, and title are required" });
    }
    const { createOneNotePage } = await import("./onedrive");
    const page = await createOneNotePage(notebook, section, title, content || '');
    if (page) {
      res.json({ success: true, page });
    } else {
      res.status(500).json({ error: "Failed to create page" });
    }
  } catch (err: any) {
    console.error("Error creating OneNote page:", err);
    res.status(500).json({ error: err.message || "Failed to create page" });
  }
});

app.delete("/api/onenote/page", async (req, res) => {
  try {
    const { notebook, section, title } = req.body;
    if (!notebook || !section || !title) {
      return res.status(400).json({ error: "notebook, section, and title are required" });
    }
    const { deleteOneNotePage } = await import("./onedrive");
    const deleted = await deleteOneNotePage(notebook, section, title);
    if (deleted) {
      res.json({ success: true });
    } else {
      res.status(404).json({ error: "Page not found in OneNote" });
    }
  } catch (err: any) {
    console.error("Error deleting OneNote page:", err);
    res.status(500).json({ error: err.message || "Failed to delete page" });
  }
});

app.post("/api/onedrive/ensure-semester-folders", async (req, res) => {
  try {
    const { semesterId } = req.body;
    let semester: any;
    if (semesterId) {
      const allSemesters = await storage.getAllSemesterSettings();
      semester = allSemesters.find((s: any) => s.id === semesterId);
    } else {
      semester = await storage.getActiveSemesterSettings();
    }
    if (!semester) return res.status(404).json({ error: "Semester not found" });

    const { createOneDriveFolder } = await import("./onedrive");
    const semType = getSemesterTypeFolder(semester.semesterType);
    const startDate = semester.semesterStartDate ? new Date(semester.semesterStartDate) : new Date();
    const year = startDate.getFullYear();
    const basePath = `/School/1. TMU/Courses/${year}`;
    const semFolder = semType;

    await createOneDriveFolder(basePath, semFolder);
    const semPath = `${basePath}/${semFolder}`;

    const results: string[] = [];

    for (let i = 1; i <= 3; i++) {
      const code = ((semester as any)[`course${i}Code`] || '').replace(/\\s/g, '');
      if (!code) continue;
      const name = (semester as any)[`course${i}Name`] || '';
      const folderName = name ? `${code} - ${name}` : code;
      await createOneDriveFolder(semPath, folderName);
      const coursePath = `${semPath}/${folderName}`;

      const weekNames = generateWeekFolderNames(semester, i);
      for (const weekName of weekNames) {
        await createOneDriveFolder(coursePath, weekName);
        const weekPath = `${coursePath}/${weekName}`;
        await createOneDriveFolder(weekPath, "Module");
        await createOneDriveFolder(weekPath, "Reading");
      }
      results.push(`${folderName} (${weekNames.length} weeks)`);
    }

    console.log(`[OneDrive] Ensured semester folders for ${semester.semesterName}: ${results.join(', ')}`);
    res.json({ success: true, folders: results });
  } catch (err: any) {
    console.error("Error ensuring semester folders:", err);
    res.status(500).json({ error: err.message });
  }
});

app.post("/api/onedrive/rename-course-folder", async (req, res) => {
  try {
    const { semesterId, courseIndex, oldCode, oldName, newCode, newName } = req.body;
    if (!courseIndex || (!newCode && !newName)) {
      return res.status(400).json({ error: "Missing courseIndex, newCode, or newName" });
    }
  }

  let semester: any;

```

```

if (semesterId) {
  const allSemesters = await storage.getAllSemesterSettings();
  semester = allSemesters.find((s: any) => s.id === semesterId);
} else {
  semester = await storage.getActiveSemesterSettings();
}
if (!semester) return res.status(404).json({ error: "Semester not found" });

const { renameOneDriveFolder, createOneDriveFolder, checkOneDriveFolderExists } = await import("./onedrive");
const semType = getSemesterTypeFolder(semester.semesterType);
const startDate = semester.semesterStartDate ? new Date(semester.semesterStartDate) : new Date();
const year = startDate.getFullYear();
const semPath = `/School/1. TMU/Courses/${year}/${semType}`;

const effectiveOldCode = (oldCode || '').replace(/\\s/g, '');
const effectiveNewCode = (newCode || '').replace(/\\s/g, '');
const oldFolderName = oldName ? `${effectiveOldCode} - ${oldName}` : effectiveOldCode;
const newFolderName = newName ? `${effectiveNewCode} - ${newName}` : effectiveNewCode;

if (oldFolderName === newFolderName) {
  return res.json({ success: true, action: 'no_change' });
}

const oldPath = `${semPath}/${oldFolderName}`;
const oldExists = await checkOneDriveFolderExists(oldPath);

if (oldExists) {
  const result = await renameOneDriveFolder(oldPath, newFolderName);
  console.log(`[OneDrive] Renamed folder: ${oldFolderName} !' ${newFolderName}: ${JSON.stringify(result)}`);
  res.json({ success: true, action: 'renamed', from: oldFolderName, to: newFolderName, ...result });
} else {
  await createOneDriveFolder(semPath, newFolderName);
  const coursePath = `${semPath}/${newFolderName}`;
  const weekNames = generateWeekFolderNames(semester, courseIndex);
  for (const weekName of weekNames) {
    await createOneDriveFolder(coursePath, weekName);
    const weekPath = `${coursePath}/${weekName}`;
    await createOneDriveFolder(weekPath, "Module");
    await createOneDriveFolder(weekPath, "Reading");
  }
  console.log(`[OneDrive] Created new folder structure: ${newFolderName} (${weekNames.length} weeks, old '${old...`);
  res.json({ success: true, action: 'created', folder: newFolderName, weeks: weekNames.length });
}
} catch (err: any) {
  console.error("Error renaming course folder:", err);
  res.status(500).json({ error: err.message });
}
});

app.post("/api/onedrive/create-semester-folders", async (req, res) => {
  try {
    const { semesterName, semesterFolder, year, courses, numWeeks } = req.body;
    if (!semesterFolder || !year || !courses || !numWeeks) {
      return res.status(400).json({ error: "Missing required fields: semesterFolder, year, courses, numWeeks" });
    }

    const basePath = `/School/1. TMU/Courses/${year}`;
    const results: any[] = [];

    const semFolderResult = await createOneDriveFolder(basePath, semesterFolder);
    results.push({ path: `${basePath}/${semesterFolder}`, ...semFolderResult });
    const semPath = `${basePath}/${semesterFolder}`;

    const semester = await storage.getActiveSemesterSettings();
    for (let ci = 0; ci < courses.length; ci++) {
      const course = courses[ci];
      const courseFolderName = `${course.code} - ${course.name}`;
      const courseResult = await createOneDriveFolder(semPath, courseFolderName);
      results.push({ path: `${semPath}/${courseFolderName}`, ...courseResult });
      const coursePath = `${semPath}/${courseFolderName}`;

      const weekNames = semester ? generateWeekFolderNames(semester, ci + 1) : Array.from({ length: numWeeks }, (_, ...`);
      for (const weekFolderName of weekNames) {
        const weekResult = await createOneDriveFolder(coursePath, weekFolderName);
        results.push({ path: `${coursePath}/${weekFolderName}`, ...weekResult });
        const weekPath = `${coursePath}/${weekFolderName}`;

        const moduleResult = await createOneDriveFolder(weekPath, "Module");
        results.push({ path: `${weekPath}/Module`, ...moduleResult });

        const readingResult = await createOneDriveFolder(weekPath, "Reading");
        results.push({ path: `${weekPath}/Reading`, ...readingResult });
      }
    }

    const created = results.filter(r => r.created).length;
    const existed = results.filter(r => r.exists).length;
    console.log(`[OneDrive] Created ${created} folders, ${existed} already existed for ${semesterName} || semesterFo...`);
    res.json({ success: true, created, existed, total: results.length, details: results });
  } catch (err: any) {
    console.error("Error creating semester folders:", err);
    res.status(500).json({ error: err.message || "Failed to create semester folders" });
  }
});

app.post("/api/onedrive/ensure-all-semester-folders", async (req, res) => {
  try {
    const allSemesters = await storage.getAllSemesterSettings();

    const { createOneDriveFolder } = await import("./onedrive");
    const allResults: Array<{ semester: string; folders: string[] }> = [];

    if (allSemesters && allSemesters.length > 0) {
      for (const semester of allSemesters) {
        const semType = getSemesterTypeFolder(semester.semesterType);
        const startDate = semester.semesterStartDate ? new Date(semester.semesterStartDate) : new Date();
        const year = startDate.getFullYear();
        const basePath = `/School/1. TMU/Courses/${year}`;

```



```

    await createOneDriveFolder(basePath, semType);
    const semPath = `${basePath}/${semType}`;
    const courseResults: string[] = [];

    for (let i = 1; i <= 3; i++) {
      const code = ((semester as any)['course${i}Code'] || '').replace(/\\s/g, '');
      if (!code) continue;
      const name = (semester as any)['course${i}Name'] || '';
      const folderName = name ? `${code} - ${name}` : code;
      await createOneDriveFolder(semPath, folderName);
      const coursePath = `${semPath}/${folderName}`;

      const weekNames = generateWeekFolderNames(semester, i);
      for (const weekName of weekNames) {
        await createOneDriveFolder(coursePath, weekName);
        const weekPath = `${coursePath}/${weekName}`;
        await createOneDriveFolder(weekPath, "Module");
        await createOneDriveFolder(weekPath, "Reading");
      }
      courseResults.push(`${folderName} (${weekNames.length} weeks)`);
    }

    allResults.push({ semester: semester.semesterName || `${semType} ${year}`, folders: courseResults });
    console.log(`[OneDrive] Ensured folders for ${semester.semesterName}: ${courseResults.join(', ')}`);
  }
}

const now = new Date();
const currentYear = now.getFullYear();
const semesterTypes = ['Winter', 'Spring & Summer', 'Fall'];
const existingKeys = new Set(
  (allSemesters || []).map((s: any) => {
    const semType = getSemesterTypeFolder(s.semesterType);
    const year = s.semesterStartDate ? new Date(s.semesterStartDate).getFullYear() : currentYear;
    return `${year}/${semType}`;
  })
);

const placeholderResults: string[] = [];
for (let year = currentYear; year <= currentYear + 2; year++) {
  const yearPath = `/School/1. TMU/Courses/${year}`;
  try {
    await createOneDriveFolder(`${School}/1. TMU/Courses`, String(year));
  } catch (e: any) {
    console.log(`[OneDrive] Year folder ${year}: ${e.message}`);
  }
  for (const semType of semesterTypes) {
    const key = `${year}/${semType}`;
    if (existingKeys.has(key)) continue;
    try {
      await createOneDriveFolder(yearPath, semType);
      placeholderResults.push(`${year}/${semType}`);
      console.log(`[OneDrive] Created placeholder folder: ${year}/${semType}`);
    } catch (e: any) {
      console.log(`[OneDrive] Placeholder ${year}/${semType}: ${e.message}`);
    }
  }
}

res.json({ success: true, semesters: allResults, placeholders: placeholderResults });
} catch (err: any) {
  console.error("Error ensuring all semester folders:", err);
  res.status(500).json({ error: err.message });
}
});

app.post("/api/onedrive/ensure-placeholder-folders", async (req, res) => {
  try {
    const allSemesters = await storage.getAllSemesterSettings();
    const { createOneDriveFolder } = await import("./onedrive");

    const now = new Date();
    const currentYear = now.getFullYear();
    const semesterTypes = ['Winter', 'Spring & Summer', 'Fall'];
    const existingKeys = new Set(
      (allSemesters || []).map((s: any) => {
        const semType = getSemesterTypeFolder(s.semesterType);
        const year = s.semesterStartDate ? new Date(s.semesterStartDate).getFullYear() : currentYear;
        return `${year}/${semType}`;
      })
    );

    const placeholderResults: string[] = [];
    for (let year = currentYear; year <= currentYear + 2; year++) {
      const yearPath = `/School/1. TMU/Courses/${year}`;
      try {
        await createOneDriveFolder(`${School}/1. TMU/Courses`, String(year));
      } catch (e: any) {
        // ignore if year folder already exists
      }
      for (const semType of semesterTypes) {
        const key = `${year}/${semType}`;
        if (existingKeys.has(key)) continue;
        try {
          await createOneDriveFolder(yearPath, semType);
          placeholderResults.push(`${year}/${semType}`);
          console.log(`[OneDrive] Created placeholder folder: ${year}/${semType}`);
        } catch (e: any) {
          // ignore if folder already exists
        }
      }
    }

    res.json({ success: true, placeholders: placeholderResults });
  } catch (err: any) {
    console.error("Error ensuring placeholder folders:", err);
    res.status(500).json({ error: err.message });
  }
}

```

```

    }
  });

app.post("/api/onedrive/rename-week-folders", async (req, res) => {
  try {
    const { courseCode, courseName, weekStyle } = req.body;
    if (!courseCode || !weekStyle) {
      return res.status(400).json({ error: "courseCode and weekStyle are required" });
    }

    const allSemesters = await storage.getAllSemesterSettings();
    const { listOneDriveFolderChildren, renameOneDriveItem } = await import("./onedrive");

    let targetSemester: any = null;
    let courseIndex = -1;
    for (const sem of allSemesters || []) {
      for (let i = 1; i <= 5; i++) {
        const code = sem[`course${i}Code`] as keyof typeof sem;
        if (code && String(code).toLowerCase() === courseCode.toLowerCase()) {
          targetSemester = sem;
          courseIndex = i;
          break;
        }
      }
    }
    if (targetSemester) break;

    if (!targetSemester || courseIndex < 0) {
      return res.status(404).json({ error: "Course not found in any semester" });
    }

    const semType = getSemesterTypeFolder(targetSemester.semesterType);
    const year = targetSemester.semesterStartDate
      ? new Date(targetSemester.semesterStartDate).getFullYear()
      : new Date().getFullYear();
    const cName = targetSemester[`course${courseIndex}Name`] as keyof typeof targetSemester || courseName || course...
    const cCode = String(targetSemester[`course${courseIndex}Code`] as keyof typeof targetSemester || courseCode);
    const courseFolderName = `${cCode} - ${cName}`;
    const courseFolderPath = `School/1. TMU/Courses/${year}/${semType}/${courseFolderName}`;

    const children = await listOneDriveFolderChildren(courseFolderPath);
    const weekFolders = (children || []).filter((c: any) =>
      c.folder && (c.name.startsWith("Week ") || c.name.startsWith("Reading Week")))
      .sort((a: any, b: any) => a.name.localeCompare(b.name, undefined, { numeric: true }));

    if (weekFolders.length === 0) {
      return res.json({ success: true, message: "No week folders found to rename.", renamed: 0 });
    }

    const newNames = generateWeekFolderNames(targetSemester, courseIndex);
    let renamedCount = 0;

    for (let i = 0; i < weekFolders.length && i < newNames.length; i++) {
      const folder = weekFolders[i];
      const newName = newNames[i];
      if (folder.name !== newName) {
        try {
          await renameOneDriveItem(folder.id, newName);
          renamedCount++;
          console.log(`[OneDrive] Renamed "${folder.name}" to "${newName}"`);
        } catch (e: any) {
          console.error(`[OneDrive] Failed to rename "${folder.name}":`, e.message);
        }
      }
    }

    res.json({ success: true, message: `Renamed ${renamedCount} week folders to "${weekStyle}" style.`, renamed: re...
  } catch (err: any) {
    console.error("Rename week folders error:", err);
    res.status(500).json({ error: err.message });
  }
});

// ===== END ONEDRIVE ROUTES =====

// ===== D2L ANNOUNCEMENTS =====
app.get("/api/announcements", async (_req, res) => {
  try {
    const all = await storage.getAnnouncements();
    const now = new Date();
    const active = [];
    for (const a of all) {
      if (a.courseName === 'Custom' || a.courseName === 'REMINDER' || a.courseName === 'URGENT') {
        active.push(a);
        continue;
      }
      const received = new Date(a.receivedAt);
      const dayOfWeek = received.getDay();
      const daysUntilFriday = dayOfWeek <= 5 ? (5 - dayOfWeek) : (5 + 7 - dayOfWeek);
      const fridayEnd = new Date(received);
      fridayEnd.setDate(received.getDate() + daysUntilFriday);
      fridayEnd.setHours(23, 59, 59, 999);
      if (now <= fridayEnd) {
        active.push(a);
      } else {
        storage.deleteAnnouncement(a.id).catch(() => {});
      }
    }
    res.json(active);
  } catch (err: any) {
    console.error("Error fetching announcements:", err.message);
    res.json([]);
  }
});

app.post("/api/announcements", async (req, res) => {
  try {
    const { body } = req.body;

```

```

    if (!body || typeof body !== 'string' || !body.trim()) {
      return res.status(400).json({ error: "body is required" });
    }
    const tag = req.body.courseName || 'Custom';
    const created = await storage.createAnnouncement({
      emailId: `manual-${Date.now()}`,
      subject: tag === 'Custom' ? 'Custom Ticker' : `${tag} Ticker`,
      body: body.trim(),
      snippet: body.trim().substring(0, 200),
      courseName: tag,
      receivedAt: new Date(),
    });
    console.log(`[Ticker] Manually added ticker item id:${created.id} body:"${body.trim().substring(0, 50)}"`);
    res.json(created);
  } catch (err: any) {
    console.error("Error creating announcement:", err.message);
    res.status(500).json({ error: err.message });
  }
});

app.delete("/api/announcements/:id", async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    if (isNaN(id)) return res.status(400).json({ error: "Invalid id" });
    await storage.deleteAnnouncement(id);
    console.log(`[Ticker] Deleted ticker item id:${id}`);
    res.json({ success: true });
  } catch (err: any) {
    console.error("Error deleting announcement:", err.message);
    res.status(500).json({ error: err.message });
  }
});

app.post("/api/announcements/reorder", async (req, res) => {
  try {
    const { orderedIds } = req.body;
    if (!Array.isArray(orderedIds)) return res.status(400).json({ error: "orderedIds array required" });
    for (let i = 0; i < orderedIds.length; i++) {
      await db.update(announcements).set({ sortOrder: i }).where(eq(announcements.id, orderedIds[i]));
    }
    res.json({ success: true });
  } catch (err: any) {
    console.error("Error reordering announcements:", err.message);
    res.status(500).json({ error: err.message });
  }
});

app.post("/api/announcements/webhook", async (req, res) => {
  try {
    const { emailId, subject, body, snippet, courseName, receivedAt } = req.body;
    if (!subject) {
      return res.status(400).json({ error: "subject is required" });
    }
    const id = emailId || `manual-${Date.now()}`;
    const existing = await storage.getAnnouncementByEmailId(id);
    if (existing) {
      return res.json({ success: true, message: "Already exists", id: existing.id });
    }
    const created = await storage.createAnnouncement({
      emailId: id,
      subject,
      body: body || '',
      snippet: snippet || subject.slice(0, 100),
      courseName: courseName || extractCourseFromSubject(subject),
      receivedAt: receivedAt ? new Date(receivedAt) : new Date(),
    });
    console.log(`[Announcements] New: "${subject}" (${created.courseName})`);
    res.json({ success: true, id: created.id });
  } catch (err: any) {
    console.error("Announcement webhook error:", err.message);
    res.status(500).json({ error: err.message });
  }
});

function extractCourseFromSubject(subject: string): string {
  const match = subject.match(/^[^\\s]+\\s/);
  if (match) return match[1].trim();
  const codeMatch = subject.match(/[A-Z]{3,4}\\s?\\d{3}/i);
  if (codeMatch) return codeMatch[1];
  return 'University';
}

app.post('/api/webhook/reminder', async (req, res) => {
  try {
    const { subject, body, auth } = req.body;
    if (auth !== '5747') {
      return res.status(401).json({ error: 'Unauthorized' });
    }
    const title = (subject || '').replace(/^reminder\\s*/i, '').trim() || (body || '').trim() || 'Reminder';
    const description = body?.trim() || null;

    const dueDate = new Date();
    dueDate.setHours(9, 0, 0, 0);

    const semesterSettings = await storage.getActiveSemesterSettings();
    let weekNumber = 1;
    if (semesterSettings?.semesterStartDate) {
      const { getWeekNumber } = await import('../shared/schema');
      weekNumber = getWeekNumber(
        new Date(),
        new Date(semesterSettings.semesterStartDate),
        semesterSettings.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : null
      );
    }

    const task = await storage.createTask({
      title,
      type: 'reminder',
    });
  }
});

```

```

    courseName: null,
    dueDate,
    eventStartTime: null,
    eventEndTime: null,
    weekNumber,
    priority: 'medium',
    description,
    isCompleted: false,
    isAcknowledged: false,
  });
  console.log(`[Email Reminder] Created reminder #${task.id}: "${title}"`);
  return res.json({ action: 'created', id: task.id, title });
} catch (err: any) {
  console.error('[Email Reminder] Error:', err.message);
  return res.status(500).json({ error: err.message });
}
});

app.post('/api/webhook/delete', async (req, res) => {
  try {
    const { body, auth } = req.body;
    if (auth !== '5747') {
      return res.status(401).json({ error: 'Unauthorized' });
    }
    if (!body || typeof body !== 'string') {
      return res.status(400).json({ error: 'Missing body' });
    }
    const text = body.trim().toLowerCase().replace(/^delete\s+/i, '');

    const tickerMatch = text.match(/^ticker\s+(?:item\s+)?(.+)/i);
    if (tickerMatch) {
      const target = tickerMatch[1].trim();
      const all = await storage.getAnnouncements();
      const matches = all.filter(a => a.body.toLowerCase().includes(target) || a.snippet?.toLowerCase().includes(target));
      for (const m of matches) {
        await storage.deleteAnnouncement(m.id);
      }
      console.log(`[Email Delete] Deleted ${matches.length} ticker items matching "${target}"`);
      return res.json({ action: 'deleted', type: 'ticker', target, count: matches.length });
    }

    const calendarMatch = text.match(/^calendar\s+(?:item\s+|entry\s+)?(.+)/i);
    if (calendarMatch) {
      const target = calendarMatch[1].trim();
      const allTasks = await storage.getTasks();
      const matches = allTasks.filter((t: any) => t.title.toLowerCase().includes(target) || t.description?.toLowerCase().includes(target));
      for (const m of matches) {
        if ((m as any).calendarEventId) {
          try { await deleteCalendarEvent((m as any).calendarEventId); } catch (e: any) { console.log(`[Email Delete] Error: ${e.message}`); }
        }
        if ((m as any).prepCalendarEventId) {
          try { await deleteCalendarEvent((m as any).prepCalendarEventId); } catch (e: any) { console.log(`[Email Delete] Error: ${e.message}`); }
        }
        if ((m as any).secondAccountCalendarEventId) {
          try { await deleteEventFromSecondAccount((m as any).secondAccountCalendarEventId); } catch (e: any) { console.log(`[Email Delete] Error: ${e.message}`); }
        }
        if ((m as any).secondAccountPrepEventId) {
          try { await deleteEventFromSecondAccount((m as any).secondAccountPrepEventId); } catch (e: any) { console.log(`[Email Delete] Error: ${e.message}`); }
        }
        if ((m as any).repeatType && (m as any).repeatType !== 'none') {
          const children = await storage.getChildTasks(m.id);
          for (const child of children) {
            if ((child as any).calendarEventId) { try { await deleteCalendarEvent((child as any).calendarEventId); } catch (e: any) { console.log(`[Email Delete] Error: ${e.message}`); } }
            if ((child as any).secondAccountCalendarEventId) { try { await deleteEventFromSecondAccount((child as any).secondAccountCalendarEventId); } catch (e: any) { console.log(`[Email Delete] Error: ${e.message}`); } }
            await storage.deleteTask(child.id);
          }
        }
        await storage.deleteTask(m.id);
      }
      console.log(`[Email Delete] Deleted ${matches.length} calendar tasks matching "${target}" (with Google Calendar)`);
      return res.json({ action: 'deleted', type: 'calendar', target, count: matches.length });
    }

    const todoMatch = text.match(/^(?:todo|reminder|to-do|to do)\s+(.+)/i);
    if (todoMatch) {
      const target = todoMatch[1].trim();
      const allTasks = await storage.getTasks();
      const matches = allTasks.filter((t: any) => t.type === 'reminder' && (t.title.toLowerCase().includes(target) || t.description?.toLowerCase().includes(target)));
      for (const m of matches) {
        await storage.deleteTask(m.id);
      }
      console.log(`[Email Delete] Deleted ${matches.length} todo/reminder tasks matching "${target}"`);
      return res.json({ action: 'deleted', type: 'todo', target, count: matches.length });
    }

    return res.status(400).json({ error: 'Unrecognized delete target. Use: "ticker item xyz", "calendar item xyz", ...' });
  } catch (err: any) {
    console.error('[Email Delete] Error:', err.message);
    return res.status(500).json({ error: err.message });
  }
});

// Unified email intake - routes emails from homeworkbryn@gmail.com to the correct handler
app.post('/api/webhook/email', async (req, res) => {
  try {
    const { emailId, subject, body, from, auth } = req.body;
    if (auth !== '5747') {
      return res.status(401).json({ error: 'Unauthorized' });
    }
    const subjectLower = (subject || '').trim().toLowerCase();
    const bodyLower = (body || '').trim().toLowerCase();

    // Route based on subject prefix
    if (subjectLower.startsWith('ticker')) {
      // Forward to ticker handler
      const tickerResp = await fetch(`http://localhost:${process.env.PORT || 5000}/api/webhook/ticker`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ emailId, subject, body, from, auth }),
      });
      return res.json(tickerResp);
    }
  }
});

```

```

    body: JSON.stringify({ emailId, subject, body, auth }),
  });
  const tickerData = await tickerResp.json();
  console.log(`[Email Router] Routed to ticker: "${subject}"`);
  return res.json({ routed: 'ticker', ...tickerData });
}

if (subjectLower.startsWith('reminder') || subjectLower.startsWith('remind')) {
  // Forward to reminder handler
  const remResp = await fetch(`http://localhost:${process.env.PORT || 5000}/api/webhook/reminder`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ subject, body, auth }),
  });
  const remData = await remResp.json();
  console.log(`[Email Router] Routed to reminder: "${subject}"`);
  return res.json({ routed: 'reminder', ...remData });
}

if (subjectLower.startsWith('delete')) {
  // Forward to delete handler
  const delResp = await fetch(`http://localhost:${process.env.PORT || 5000}/api/webhook/delete`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ body: subject, auth }),
  });
  const delData = await delResp.json();
  console.log(`[Email Router] Routed to delete: "${subject}"`);
  return res.json({ routed: 'delete', ...delData });
}

// Default: route to email-homework (task creation)
const hwResp = await fetch(`http://localhost:${process.env.PORT || 5000}/api/webhook/email-homework`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', 'X-Webhook-Secret': auth },
  body: JSON.stringify({ subject, body, from }),
});
const hwData = await hwResp.json();
console.log(`[Email Router] Routed to email-homework: "${subject}"`);
return res.json({ routed: 'email-homework', ...hwData });
} catch (err: any) {
  console.error(`[Email Router] Error:`, err.message);
  return res.status(500).json({ error: err.message });
}
});
console.log(`[Email Router] Unified email intake ready at POST /api/webhook/email`);

const tickerExpirations = new Map<number, NodeJS.Timeout>();
app.post('/api/webhook/ticker', async (req, res) => {
  try {
    const { emailId, body, subject, auth } = req.body;
    if (auth !== '5747') {
      return res.status(401).json({ error: 'Unauthorized' });
    }
    let rawBody = (body && typeof body === 'string' && body.trim()) ? body.trim() : null;
    if (rawBody) {
      rawBody = rawBody.replace(/`ticker`s+item`s+/i, '').trim();
    }
    const subjectText = (subject && typeof subject === 'string') ? subject.replace(/`ticker`s*/i, '').trim() : null;
    const messageBody = rawBody || subjectText;
    if (!messageBody) {
      return res.status(400).json({ error: 'Missing body and subject' });
    }
    const { command, target, expireMinutes } = parseTickerCommand(messageBody);

    if (command === 'clear') {
      const all = await storage.getAnnouncements();
      for (const a of all) {
        await storage.deleteAnnouncement(a.id);
        if (tickerExpirations.has(a.id)) { clearTimeout(tickerExpirations.get(a.id)!); tickerExpirations.delete(a.id); }
      }
      console.log(`[Gmail Ticker] Cleared all ${all.length} ticker items`);
      return res.json({ action: 'cleared', count: all.length });
    }

    if (command === 'delete' && target) {
      const all = await storage.getAnnouncements();
      const lowerTarget = target.toLowerCase();
      const matches = all.filter(a => a.body.toLowerCase().includes(lowerTarget) || a.snippet?.toLowerCase().includes(lowerTarget));
      for (const m of matches) {
        await storage.deleteAnnouncement(m.id);
        if (tickerExpirations.has(m.id)) { clearTimeout(tickerExpirations.get(m.id)!); tickerExpirations.delete(m.id); }
      }
      console.log(`[Gmail Ticker] Deleted ${matches.length} ticker items matching "${target}"`);
      return res.json({ action: 'deleted', target, count: matches.length });
    }

    if (command === 'expire' && expireMinutes) {
      const all = await storage.getAnnouncements();
      if (all.length > 0) {
        const latest = all[all.length - 1];
        const ms = expireMinutes * 60 * 1000;
        const timeout = setTimeout(async () => {
          try {
            await storage.deleteAnnouncement(latest.id);
            tickerExpirations.delete(latest.id);
            console.log(`[Gmail Ticker] Auto-expired ticker item: "${latest.body.slice(0, 60)}..." (id: ${latest.id})`);
          } catch (err: any) {
            console.error(`[Gmail Ticker] Expire cleanup error:`, err.message);
          }
        }, ms);
        tickerExpirations.set(latest.id, timeout);
        console.log(`[Gmail Ticker] Set ${expireMinutes}min expiration for: "${latest.body.slice(0, 60)}..." (id: ${latest.id})`);
        return res.json({ action: 'expire_set', minutes: expireMinutes, itemId: latest.id });
      }
      return res.json({ action: 'expire_noop', reason: 'no items' });
    }
  }
});

```

```

const { cleanBody, expireMinutes: inlineExpire } = extractInlineExpiry(messageBody);
const finalBody = cleanBody || messageBody;

const msgId = emailId || `webhook-${Date.now()}`;
const existing = await storage.getAnnouncementByEmailId(msgId);
if (existing) {
  return res.json({ action: 'duplicate', emailId: msgId });
}

const created = await storage.createAnnouncement({
  emailId: msgId,
  subject: 'Ticker Update',
  body: finalBody,
  snippet: finalBody.slice(0, 200),
  courseName: 'Custom',
  receivedAt: new Date(),
});
console.log(`[Gmail Ticker] Added to ticker: "${finalBody.slice(0, 60)}..." (id: ${created.id})`);

if (inlineExpire) {
  const ms = inlineExpire * 60 * 1000;
  const timeout = setTimeout(async () => {
    try {
      await storage.deleteAnnouncement(created.id);
      tickerExpirations.delete(created.id);
      console.log(`[Gmail Ticker] Auto-expired ticker item: "${finalBody.slice(0, 60)}..." (id: ${created.id})`);
    } catch (err: any) {
      console.error(`[Gmail Ticker] Expire cleanup error:`, err.message);
    }
  }, ms);
  tickerExpirations.set(created.id, timeout);
  console.log(`[Gmail Ticker] Set inline ${inlineExpire}min expiration for: "${finalBody.slice(0, 60)}..." (id: ...`);
  return res.json({ action: 'added_with_expiry', id: created.id, expiresInMinutes: inlineExpire });
}

return res.json({ action: 'added', id: created.id });
} catch (err: any) {
  console.error(`[Gmail Ticker] Webhook error:`, err.message);
  return res.status(500).json({ error: err.message });
}
});
console.log(`[Gmail Ticker] Webhook endpoint ready at POST /api/webhook/ticker`);

// GET /api/calendar/list - List all available Google calendars for selection
app.get("/api/calendar/list", async (_req, res) => {
  try {
    const calendars = await listCalendars();
    const formattedCalendars = calendars.map((c: any) => ({
      id: c.id,
      summary: c.summary,
      primary: c.primary || false,
      backgroundColor: c.backgroundColor,
    }));
    res.json(formattedCalendars);
  } catch (err) {
    console.error("List calendars error:", err);
    res.status(500).json({ error: String(err) });
  }
});

// GET /api/calendar/debug - Debug Google Calendar connection
app.get("/api/calendar/debug", async (_req, res) => {
  try {
    const calendars = await listCalendars();
    const primary = calendars.find((c: any) => c.primary);

    // Get events from the past week to now + 2 months
    const now = new Date();
    const twoMonthsLater = new Date(now.getTime() + 60 * 24 * 60 * 1000);
    const events = await listEvents(now, twoMonthsLater);

    res.json({
      connectedEmail: primary?.id || 'unknown',
      calendarSummary: primary?.summary || 'unknown',
      totalCalendars: calendars.length,
      upcomingEventsCount: events.length,
      sampleEvents: events.slice(0, 5).map((e: any) => ({
        id: e.id,
        summary: e.summary,
        start: e.start?.dateTime || e.start?.date,
      })),
    });
  } catch (err) {
    console.error("Calendar debug error:", err);
    res.status(500).json({ error: String(err) });
  }
});

// ===== SECOND GOOGLE ACCOUNT ROUTES =====

// GET /api/google/second-account/status - Check if second account is connected
app.get("/api/google/second-account/status", async (_req, res) => {
  try {
    const status = await isSecondAccountConnected();
    res.json(status);
  } catch (err) {
    console.error("Second account status error:", err);
    res.json({ connected: false, error: String(err) });
  }
});

// GET /api/google/second-account/auth - Start OAuth flow for second account
app.get("/api/google/second-account/auth", async (_req, res) => {
  try {
    const authUrl = getSecondAccountAuthUrl();
    res.json({ authUrl });
  } catch (err) {

```

```

        console.error("Second account auth error:", err);
        res.status(500).json({ error: String(err) });
    }
});

// GET /api/google/second-account/callback - OAuth callback handler
app.get("/api/google/second-account/callback", async (req, res) => {
    try {
        const code = req.query.code as string;
        if (!code) {
            return res.status(400).send('Missing authorization code');
        }

        const account = await exchangeCodeForTokens(code);

        // Redirect back to dashboard with success message
        res.send(`
        <!DOCTYPE html>
        <html>
        <head><title>Success</title></head>
        <body>
        <h1>Second Google Account Connected!</h1>
        <p>Connected email: ${account.email}</p>
        <p>You can close this window and return to the app.</p>
        <script>
            setTimeout(() => {
                window.opener?.postMessage({ type: 'SECOND_ACCOUNT_CONNECTED', email: '${account.email}' }, '*');
                window.close();
            }, 1500);
        </script>
        </body>
        </html>
        `);
    } catch (err) {
        console.error("Second account callback error:", err);
        res.status(500).send(`
        <!DOCTYPE html>
        <html>
        <head><title>Error</title></head>
        <body>
        <h1>Connection Failed</h1>
        <p>Error: ${String(err)}</p>
        <p>Please close this window and try again.</p>
        </body>
        </html>
        `);
    }
});

// DELETE /api/google/second-account - Disconnect second account
app.delete("/api/google/second-account", async (_req, res) => {
    try {
        await disconnectSecondAccount();
        res.json({ success: true });
    } catch (err) {
        console.error("Second account disconnect error:", err);
        res.status(500).json({ error: String(err) });
    }
});

// GET /api/google/second-account/events - Get events from second account
app.get("/api/google/second-account/events", async (req, res) => {
    try {
        const timeMin = new Date(req.query.timeMin as string || Date.now());
        const timeMax = new Date(req.query.timeMax as string || Date.now() + 30 * 24 * 60 * 60 * 1000);

        const events = await getEventsFromSecondAccount(timeMin, timeMax);
        res.json(events);
    } catch (err) {
        console.error("Second account events error:", err);
        res.status(500).json({ error: String(err) });
    }
});

// ===== END SECOND GOOGLE ACCOUNT ROUTES =====

// ===== THIRD GOOGLE ACCOUNT ROUTES (CRCU - Partner Shifts) =====

app.get("/api/google/third-account/status", async (_req, res) => {
    try {
        const status = await isThirdAccountConnected();
        res.json(status);
    } catch (err) {
        console.error("Third account status error:", err);
        res.json({ connected: false, error: String(err) });
    }
});

app.get("/api/google/third-account/auth", async (_req, res) => {
    try {
        const authUrl = getThirdAccountAuthUrl();
        res.json({ authUrl });
    } catch (err) {
        console.error("Third account auth error:", err);
        res.status(500).json({ error: String(err) });
    }
});

app.get("/api/google/third-account/callback", async (req, res) => {
    try {
        const code = req.query.code as string;
        if (!code) {
            return res.status(400).send('Missing authorization code');
        }

        const account = await exchangeCodeForTokensThird(code);

        res.send(`

```

```

<!DOCTYPE html>
<html>
  <head><title>Success</title></head>
  <body>
    <h1>CRCU Account Connected!</h1>
    <p>Connected email: ${account.email}</p>
    <p>You can close this window and return to the app.</p>
    <script>
      setTimeout(() => {
        window.opener?.postMessage({ type: 'THIRD_ACCOUNT_CONNECTED', email: '${account.email}' }, '');
        window.close();
      }, 1500);
    </script>
  </body>
</html>
`);
} catch (err) {
  console.error("Third account callback error:", err);
  res.status(500).send(`
    <!DOCTYPE html>
    <html>
      <head><title>Error</title></head>
      <body>
        <h1>Connection Failed</h1>
        <p>Error: ${String(err)}</p>
        <p>Please close this window and try again.</p>
      </body>
    </html>
  `);
}
});

app.delete("/api/google/third-account", async (_req, res) => {
  try {
    await disconnectThirdAccount();
    res.json({ success: true });
  } catch (err) {
    console.error("Third account disconnect error:", err);
    res.status(500).json({ error: String(err) });
  }
});

app.get("/api/google/third-account/events", async (req, res) => {
  try {
    const calendarId = req.query.calendarId as string || 'primary';
    const timeMin = new Date(req.query.timeMin as string || Date.now());
    const timeMax = new Date(req.query.timeMax as string || Date.now() + 30 * 24 * 60 * 60 * 1000);

    const events = calendarId === 'primary'
      ? await getEventsFromThirdAccount(timeMin, timeMax)
      : await getEventsFromThirdAccountCalendar(calendarId, timeMin, timeMax);
    res.json(events);
  } catch (err) {
    console.error("Third account events error:", err);
    res.status(500).json({ error: String(err) });
  }
});

app.get("/api/google/third-account/calendars", async (_req, res) => {
  try {
    const calendars = await listThirdAccountCalendars();
    res.json(calendars);
  } catch (err) {
    console.error("Third account calendars error:", err);
    res.status(500).json({ error: String(err) });
  }
});

app.post("/api/google/third-account/sync-shifts", async (req, res) => {
  try {
    const calendarId = 'family01331437021788124598@group.calendar.google.com';
    const now = new Date();
    const timeMin = new Date(now.getFullYear(), 0, 1);
    const timeMax = new Date(now.getFullYear() + 1, 0, 31, 23, 59, 59);

    const events = await getEventsFromThirdAccountCalendar(calendarId, timeMin, timeMax);

    const shiftEntries: { date: string; shiftType: string }[] = [];

    for (const event of events) {
      const summary = (event.summary || '').toLowerCase();
      if (!summary.includes('crcu')) continue;

      const startStr = event.start?.dateTime || event.start?.date;
      if (!startStr) continue;

      const startDate = new Date(startStr);
      const dateStr = `${startDate.getFullYear()}-${String(startDate.getMonth() + 1).padStart(2, '0')}-${String(sta...

      const hour = startDate.getHours();
      const minute = startDate.getMinutes();
      const timeVal = hour + minute / 60;
      const isNight = timeVal >= 14 || timeVal < 4 || summary.includes('0<ß ');

      shiftEntries.push({
        date: dateStr,
        shiftType: isNight ? 'night' : 'day',
      });
    }

    await storage.clearAllShifts();

    if (shiftEntries.length > 0) {
      await storage.setShiftBulk(shiftEntries);
    }

    const schedule = await storage.getShiftSchedule();
    res.json({ synced: shiftEntries.length, schedule });
  }
});

```



```

    } catch (err) {
      console.error("CRUCU shift sync error:", err);
      res.status(500).json({ error: String(err) });
    }
  });

// ===== END THIRD GOOGLE ACCOUNT ROUTES =====

// POST /api/tasks/:id/sync-calendar - Manually sync task to Google Calendar
app.post("/api/tasks/:id/sync-calendar", async (req, res) => {
  try {
    const task = await storage.getTask(Number(req.params.id));
    if (!task) {
      return res.status(404).json({ message: 'Task not found' });
    }

    // If already synced, update the event
    if (task.calendarEventId) {
      await updateCalendarEvent(task.calendarEventId, {
        title: task.title,
        description: task.description,
        dueDate: task.dueDate,
        courseName: task.courseName,
      });
      return res.json({ success: true, message: 'Calendar event updated' });
    }

    // Create new calendar event
    const event = await createCalendarEvent({
      id: task.id,
      title: task.title,
      description: task.description,
      dueDate: task.dueDate,
      courseName: task.courseName,
    });

    // Update task with calendar event ID
    const updatedTask = await storage.updateTask(task.id, {
      calendarEventId: event.id,
      calendarProvider: "google",
    });

    res.json({ success: true, task: updatedTask, message: 'Task synced to Google Calendar' });
  } catch (err) {
    console.error("Error syncing task to calendar:", err);
    res.status(500).json({ message: 'Failed to sync task to Google Calendar', error: String(err) });
  }
});

// POST /api/tasks/sync-all-calendar - Sync all tasks to Google Calendar
app.post("/api/tasks/sync-all-calendar", async (req, res) => {
  try {
    const tasks = await storage.getTasks({});
    const results = { dueEvents: { created: 0, updated: 0, failed: 0 }, prepEvents: { created: 0, updated: 0, failed: 0 } };
    const delay = (ms: number) => new Promise(r => setTimeout(r, ms));

    for (let ti = 0; ti < tasks.length; ti++) {
      const task = tasks[ti];
      if (ti > 0 && ti % 5 === 0) await delay(1000);
      try {
        if (task.calendarEventId) {
          const updatedEvent = await updateCalendarEvent(task.calendarEventId, {
            title: task.title,
            description: task.description,
            dueDate: task.dueDate,
            courseName: task.courseName,
          });
          if (updatedEvent?.deleted) {
            const event = await createCalendarEvent({
              id: task.id, title: task.title, description: task.description,
              dueDate: task.dueDate, courseName: task.courseName,
            });
            await storage.updateTask(task.id, { calendarEventId: event.id, calendarProvider: "google" });
            results.dueEvents.created++;
          } else {
            if (updatedEvent.id && updatedEvent.id !== task.calendarEventId) {
              await storage.updateTask(task.id, { calendarEventId: updatedEvent.id });
            }
            results.dueEvents.updated++;
          }
        } else {
          const summary = `${task.courseName} ? ` + `${task.courseName}` + ` : ` + `${task.title}`;
          const dateStr = formatLocalDate(new Date(task.dueDate));
          const existingId = await findExistingEventBySummary(summary, dateStr);
          if (existingId) {
            await storage.updateTask(task.id, { calendarEventId: existingId, calendarProvider: "google" });
            results.dueEvents.updated++;
          } else {
            const event = await createCalendarEvent({
              id: task.id, title: task.title, description: task.description,
              dueDate: task.dueDate, courseName: task.courseName,
            });
            await storage.updateTask(task.id, { calendarEventId: event.id, calendarProvider: "google" });
            results.dueEvents.created++;
          }
        }
      } catch (err: any) {
        const status = err?.status || err?.code;
        if (status === 410) {
          try {
            const event = await createCalendarEvent({
              id: task.id, title: task.title, description: task.description,
              dueDate: task.dueDate, courseName: task.courseName,
            });
            await storage.updateTask(task.id, { calendarEventId: event.id, calendarProvider: "google" });
            results.dueEvents.created++;
          } catch (recreateErr) {
            console.error('Failed to recreate event for task ${task.id}:', recreateErr);
          }
        }
      }
    }
  }
});

```

```

        results.dueEvents.failed++;
    }
    } else {
        console.error('Failed to sync due date for task ${task.id}:', err);
        results.dueEvents.failed++;
    }
}

await delay(300);

if (task.startDate) {
    try {
        if (task.prepCalendarEventId) {
            const updatedEvent = await updatePrepCalendarEvent(task.prepCalendarEventId, {
                title: task.title, description: task.description,
                startDate: task.startDate, dueDate: task.dueDate, courseName: task.courseName,
            });
            if (updatedEvent?.deleted) {
                const event = await createPrepCalendarEvent({
                    id: task.id, title: task.title, description: task.description,
                    startDate: task.startDate, dueDate: task.dueDate, courseName: task.courseName,
                });
                await storage.updateTask(task.id, { prepCalendarEventId: event.id });
                results.prepEvents.created++;
            } else {
                if (updatedEvent.id && updatedEvent.id !== task.prepCalendarEventId) {
                    await storage.updateTask(task.id, { prepCalendarEventId: updatedEvent.id });
                }
                results.prepEvents.updated++;
            }
        } else {
            const event = await createPrepCalendarEvent({
                id: task.id, title: task.title, description: task.description,
                startDate: task.startDate, dueDate: task.dueDate, courseName: task.courseName,
            });
            await storage.updateTask(task.id, { prepCalendarEventId: event.id });
            results.prepEvents.created++;
        }
    } catch (err: any) {
        const status = err?.status || err?.code;
        if (status === 410) {
            try {
                const event = await createPrepCalendarEvent({
                    id: task.id, title: task.title, description: task.description,
                    startDate: task.startDate, dueDate: task.dueDate, courseName: task.courseName,
                });
                await storage.updateTask(task.id, { prepCalendarEventId: event.id });
                results.prepEvents.created++;
            } catch (recreateErr) {
                console.error('Failed to recreate prep event for task ${task.id}:', recreateErr);
                results.prepEvents.failed++;
            }
        } else {
            console.error('Failed to sync prep event for task ${task.id}:', err);
            results.prepEvents.failed++;
        }
    }
    await delay(300);
} else {
    results.prepEvents.skipped++;
}
}

res.json({ success: true, results });
} catch (err) {
    console.error("Error syncing all tasks:", err);
    res.status(500).json({ message: 'Failed to sync tasks', error: String(err) });
}
});

app.post("/api/calendar/deduplicate", async (req, res) => {
    try {
        const { startDate, endDate } = req.body;
        const timeMin = startDate ? new Date(startDate).toISOString() : new Date().toISOString();
        const end = endDate ? new Date(endDate) : new Date(Date.now() + 90 * 24 * 60 * 60 * 1000);
        const timeMax = end.toISOString();
        const result = await findAndDeleteDuplicateEvents(timeMin, timeMax);
        res.json({ success: true, ...result });
    } catch (err) {
        console.error("Error deduplicating calendar:", err);
        res.status(500).json({ message: 'Failed to deduplicate', error: String(err) });
    }
});

// PATCH /api/tasks/:id/reschedule
app.patch(api.tasks.reschedule.path, async (req, res) => {
    const { dueDate, weekNumber } = req.body;
    const parsedDate = new Date(dueDate);
    const etHour = parseInt(parsedDate.toLocaleString('en-US', { hour: 'numeric', hour12: false, timeZone: 'America/T...'
    const etMin = parseInt(parsedDate.toLocaleString('en-US', { minute: 'numeric', timeZone: 'America/Toronto' })), 10);
    const hasTime = etHour !== 0 || etMin !== 0;
    const eventStartTime = hasTime ? `${etHour.toString().padStart(2, '0')}:${etMin.toString().padStart(2, '0')}` : n...
    const updateFields: Record<string, any> = {
        dueDate: parsedDate,
        weekNumber,
        isMissed: false
    };
    if (eventStartTime) {
        updateFields.eventStartTime = eventStartTime;
    }
    const task = await storage.updateTask(Number(req.params.id), updateFields);
    if (!task) {
        return res.status(404).json({ message: 'Task not found' });
    }
}

// Auto-sync to Google Calendar if task has calendar event
if (task.calendarEventId) {
    try {

```

```

        await updateCalendarEvent(task.calendarEventId, {
            title: task.title,
            description: task.description,
            dueDate: task.dueDate,
            courseName: task.courseName,
        });
    } catch (calErr) {
        console.error("Auto-update Google Calendar on reschedule failed:", calErr);
    }
}

res.json(task);
});

// GET /api/tasks/:id/ics - Generate ICS file for calendar sync
app.get(api.tasks.exportCalendar.path, async (req, res) => {
    const task = await storage.getTask(Number(req.params.id));
    if (!task) {
        return res.status(404).json({ message: 'Task not found' });
    }

    const dueDate = new Date(task.dueDate);
    const icsContent = generateICS(task.title, task.description || '', dueDate, task.type);

    res.setHeader('Content-Type', 'text/calendar');
    res.setHeader('Content-Disposition', 'attachment; filename="${task.title.replace(/[^\a-z0-9]/gi, '_')}.ics"');
    res.send(icsContent);
});

// GET /api/weeks/current
app.get(api.weeks.current.path, async (_req, res) => {
    const activeSemester = await storage.getActiveSemesterSettings();
    const semesterStart = activeSemester ? new Date(activeSemester.semesterStartDate) : new Date(2026, 0, 10, 12, 0, 0);
    const readingWeek = activeSemester?.readingWeekStart || null;
    const now = new Date();
    const weekNum = getWeekNumber(now, semesterStart, readingWeek);
    const { start, end } = getWeekDates(weekNum, semesterStart, readingWeek);
    // Format as YYYY-MM-DD using local time
    const formatDateOnly = (d: Date) => {
        return `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, '0')}-${String(d.getDate()).padStart(2, '0')}`;
    };
    res.json({
        weekNumber: weekNum,
        startDate: formatDateOnly(start),
        endDate: formatDateOnly(end),
    });
});

// GET /api/weeks
app.get(api.weeks.list.path, async (_req, res) => {
    const activeSemester = await storage.getActiveSemesterSettings();
    const semesterStart = activeSemester ? new Date(activeSemester.semesterStartDate) : new Date(2026, 0, 10, 12, 0, 0);
    const readingWeek = activeSemester?.readingWeekStart || null;
    const taskCounts = await storage.getTaskCountByWeek();
    const weeks = [];

    // Format as YYYY-MM-DD using local time
    const formatDateOnly = (d: Date) => {
        return `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, '0')}-${String(d.getDate()).padStart(2, '0')}`;
    };
    for (let w = FIRST_WEEK; w <= LAST_WEEK; w++) {
        const { start, end } = getWeekDates(w, semesterStart, readingWeek);
        weeks.push({
            weekNumber: w,
            startDate: formatDateOnly(start),
            endDate: formatDateOnly(end),
            taskCount: taskCounts[w] || 0,
        });
    }

    res.json(weeks);
});

// =====
// SUBTASK ROUTES
// =====

// GET /api/tasks/:taskId/subtasks - Get all subtasks for a task
app.get("/api/tasks/:taskId/subtasks", async (req, res) => {
    try {
        const taskId = Number(req.params.taskId);
        const subtasks = await storage.getSubtasksByTask(taskId);
        res.json(subtasks);
    } catch (err) {
        console.error("Error fetching subtasks:", err);
        res.status(500).json({ message: "Failed to fetch subtasks" });
    }
});

// POST /api/tasks/:taskId/subtasks - Create a subtask
app.post("/api/tasks/:taskId/subtasks", async (req, res) => {
    try {
        const taskId = Number(req.params.taskId);
        const subtaskData = {
            ...req.body,
            parentTaskId: taskId,
            startDate: req.body.startDate ? new Date(req.body.startDate) : null,
            dueDate: req.body.dueDate ? new Date(req.body.dueDate) : null,
        };
        const subtask = await storage.createSubtask(subtaskData);
        res.status(201).json(subtask);
    } catch (err) {
        console.error("Error creating subtask:", err);
        res.status(500).json({ message: "Failed to create subtask" });
    }
});

// PATCH /api/subtasks/:id - Update a subtask

```

```

app.patch("/api/subtasks/:id", async (req, res) => {
  try {
    const subtaskId = Number(req.params.id);
    const updates = {
      ...req.body,
      startDate: req.body.startDate ? new Date(req.body.startDate) : undefined,
      dueDate: req.body.dueDate ? new Date(req.body.dueDate) : undefined,
    };
    // Remove undefined values
    Object.keys(updates).forEach(key => updates[key] === undefined && delete updates[key]);
    const subtask = await storage.updateSubtask(subtaskId, updates);
    res.json(subtask);
  } catch (err) {
    console.error("Error updating subtask:", err);
    res.status(500).json({ message: "Failed to update subtask" });
  }
});

// DELETE /api/subtasks/:id - Delete a subtask
app.delete("/api/subtasks/:id", async (req, res) => {
  try {
    const subtaskId = Number(req.params.id);
    await storage.deleteSubtask(subtaskId);
    res.status(204).send();
  } catch (err) {
    console.error("Error deleting subtask:", err);
    res.status(500).json({ message: "Failed to delete subtask" });
  }
});

// =====
// TASK LINK ROUTES
// =====

// GET /api/tasks/:taskId/links - Get all links for a task
app.get("/api/tasks/:taskId/links", async (req, res) => {
  try {
    const taskId = Number(req.params.taskId);
    const links = await storage.getLinksForTask(taskId);
    res.json(links);
  } catch (err) {
    console.error("Error fetching task links:", err);
    res.status(500).json({ message: "Failed to fetch task links" });
  }
});

// GET /api/subtasks/:subtaskId/links - Get all links for a subtask
app.get("/api/subtasks/:subtaskId/links", async (req, res) => {
  try {
    const subtaskId = Number(req.params.subtaskId);
    const links = await storage.getLinksForSubtask(subtaskId);
    res.json(links);
  } catch (err) {
    console.error("Error fetching subtask links:", err);
    res.status(500).json({ message: "Failed to fetch subtask links" });
  }
});

// GET /api/links - Get all task links
app.get("/api/links", async (req, res) => {
  try {
    const links = await storage.getAllTaskLinks();
    res.json(links);
  } catch (err) {
    console.error("Error fetching all task links:", err);
    res.status(500).json({ message: "Failed to fetch task links" });
  }
});

// POST /api/links - Create a new link between tasks/subtasks
app.post("/api/links", async (req, res) => {
  try {
    const { sourceType, sourceId, targetType, targetId, linkType } = req.body;
    const link = await storage.createTaskLink({
      sourceType,
      sourceId,
      targetType,
      targetId,
      linkType: linkType || "relates_to",
    });
    res.status(201).json(link);
  } catch (err) {
    console.error("Error creating task link:", err);
    res.status(500).json({ message: "Failed to create task link" });
  }
});

// DELETE /api/links/:id - Delete a link
app.delete("/api/links/:id", async (req, res) => {
  try {
    const linkId = Number(req.params.id);
    await storage.deleteTaskLink(linkId);
    res.status(204).send();
  } catch (err) {
    console.error("Error deleting task link:", err);
    res.status(500).json({ message: "Failed to delete task link" });
  }
});

// =====
// VOICE COMMAND ROUTES (Home Assistant / Alexa integration)
// =====

// POST /api/voice/add-task - Add a task via voice command
// Accepts natural language like "Read chapter 5 for CPPA122 due Friday"
app.post("/api/voice/add-task", async (req, res) => {
  try {
    const { text, course, due, type, week } = req.body;

```

```

if (!text || typeof text !== "string") {
  return res.status(400).json({ success: false, message: "Task description is required" });
}

// Parse due date from natural language
let dueDate = new Date();
dueDate.setHours(23, 59, 0, 0); // Default to end of day

const dueLower = (due || "").toLowerCase().trim();
const today = new Date();

if (dueLower === "today") {
  dueDate = new Date(today);
  dueDate.setHours(23, 59, 0, 0);
} else if (dueLower === "tomorrow") {
  dueDate = new Date(today);
  dueDate.setDate(dueDate.getDate() + 1);
  dueDate.setHours(23, 59, 0, 0);
} else if (dueLower.startsWith("next ")) {
  const dayName = dueLower.replace("next ", "");
  const days = ["sunday", "monday", "tuesday", "wednesday", "thursday", "friday", "saturday"];
  const targetDay = days.indexOf(dayName);
  if (targetDay >= 0) {
    dueDate = new Date(today);
    const currentDay = dueDate.getDay();
    let daysToAdd = targetDay - currentDay;
    if (daysToAdd <= 0) daysToAdd += 7;
    daysToAdd += 7; // "next" means the following week
    dueDate.setDate(dueDate.getDate() + daysToAdd);
    dueDate.setHours(23, 59, 0, 0);
  }
} else if (["sunday", "monday", "tuesday", "wednesday", "thursday", "friday", "saturday"].includes(dueLower)) {
  const days = ["sunday", "monday", "tuesday", "wednesday", "thursday", "friday", "saturday"];
  const targetDay = days.indexOf(dueLower);
  dueDate = new Date(today);
  const currentDay = dueDate.getDay();
  let daysToAdd = targetDay - currentDay;
  if (daysToAdd <= 0) daysToAdd += 7;
  dueDate.setDate(dueDate.getDate() + daysToAdd);
  dueDate.setHours(23, 59, 0, 0);
} else if (dueLower.includes("in ") && dueLower.includes(" day")) {
  const match = dueLower.match(/in (\d+) day/);
  if (match) {
    dueDate = new Date(today);
    dueDate.setDate(dueDate.getDate() + parseInt(match[1]));
    dueDate.setHours(23, 59, 0, 0);
  }
} else if (due) {
  // Try to parse as a date string
  const parsed = new Date(due);
  if (!isNaN(parsed.getTime())) {
    dueDate = parsed;
    dueDate.setHours(23, 59, 0, 0);
  }
}

// Match course code
let courseName = "";
const courseUpper = (course || "").toUpperCase().trim();
const courses = ["CPPA122", "CFNF400", "CASL101"];

// Check for exact match or partial match
for (const c of courses) {
  if (courseUpper === c || courseUpper.includes(c) || c.includes(courseUpper)) {
    courseName = c;
    break;
  }
}

// Also check in the text itself for course codes
if (!courseName) {
  const textUpper = text.toUpperCase();
  for (const c of courses) {
    if (textUpper.includes(c)) {
      courseName = c;
      break;
    }
  }
}

// Determine task type
let taskType = type || "other";
const textLower = text.toLowerCase();
if (textLower.includes("read") || textLower.includes("chapter")) taskType = "reading";
else if (textLower.includes("module")) taskType = "module";
else if (textLower.includes("essay") || textLower.includes("paper") || textLower.includes("write")) taskType = ...
else if (textLower.includes("quiz")) taskType = "quiz";
else if (textLower.includes("exam") || textLower.includes("test")) taskType = "exam";
else if (textLower.includes("discuss")) taskType = "discussion";
else if (textLower.includes("project")) taskType = "project";

// Get current week number
const activeSemester = await storage.getActiveSemesterSettings();
const semesterStart = activeSemester?.semesterStartDate
  ? new Date(activeSemester.semesterStartDate)
  : undefined;
const weekNumber = week || getWeekNumber(dueDate, semesterStart, activeSemester?.readingWeekStart);

// Create the task
const task = await storage.createTask({
  title: text,
  description: "",
  weekNumber,
  type: taskType as any,
  dueDate,
  isCompleted: false,
  courseName: courseName || undefined,
});

```

```

    repeatType: "none",
  });
  // Auto-sync to Google Calendar
  try {
    const event = await createCalendarEvent({
      id: task.id,
      title: task.title,
      description: task.description,
      dueDate: task.dueDate,
      courseName: task.courseName,
    });
    await storage.updateTask(task.id, {
      calendarEventId: event.id,
      calendarProvider: "google",
    });
  } catch (calErr) {
    console.error("Calendar sync failed:", calErr);
  }

  // Prepare response message
  const dayNames = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"];
  const dueDateStr = `${dayNames[dueDate.getDay()]}, ${dueDate.toLocaleDateString()}`;
  const courseStr = courseName ? ` for ${courseName}` : "";

  res.json({
    success: true,
    message: `Task added${courseStr}: "${text}" due ${dueDateStr}`,
    task: {
      id: task.id,
      title: task.title,
      dueDate: task.dueDate,
      courseName: task.courseName,
      type: task.type,
      weekNumber: task.weekNumber,
    },
  });
} catch (err) {
  console.error("Voice add-task error:", err);
  res.status(500).json({ success: false, message: "Failed to add task" });
}
});

// GET /api/voice/tasks-today - Get tasks due today (for voice query)
app.get("/api/voice/tasks-today", async (_req, res) => {
  try {
    const tasks = await storage.getTasks();
    const today = new Date();
    today.setHours(0, 0, 0, 0);
    const tomorrow = new Date(today);
    tomorrow.setDate(tomorrow.getDate() + 1);

    const todayTasks = tasks.filter(t => {
      const due = new Date(t.dueDate);
      return due >= today && due < tomorrow && !t.isCompleted;
    });

    if (todayTasks.length === 0) {
      return res.json({ success: true, message: "You have no tasks due today.", tasks: [] });
    }

    const taskList = todayTasks.map(t => t.title).join(", ");
    res.json({
      success: true,
      message: `You have ${todayTasks.length} task${todayTasks.length > 1 ? 's' : ''} due today: ${taskList}`,
      tasks: todayTasks,
    });
  } catch (err) {
    console.error("Voice tasks-today error:", err);
    res.status(500).json({ success: false, message: "Failed to get tasks" });
  }
});

// =====
// TEXT-TO-SPEECH ROUTES (OpenAI TTS for Fire tablets)
// =====

app.post("/api/tts/clean-text", async (req, res) => {
  try {
    const { text } = req.body;
    if (!text || typeof text !== "string") {
      return res.status(400).json({ message: "Text is required" });
    }
    const cleaned = cleanTextForTTS(text);
    res.json({ text: cleaned });
  } catch (err) {
    console.error("Error cleaning text:", err);
    res.status(500).json({ message: "Failed to clean text" });
  }
});

// GET /api/tts-audio/:filename - Proxy TTS audio from object storage
app.get("/api/tts-audio/:filename", async (req, res) => {
  try {
    const audioPath = decodeURIComponent(req.params.filename);
    const publicPath = process.env.PUBLIC_OBJECT_SEARCH_PATHS?.split(',')[0]?.trim();
    if (!publicPath) {
      return res.status(500).json({ error: "Storage not configured" });
    }
    const { bucketName, objectName } = parsePublicObjectPath(`${publicPath}/${audioPath}`);
    const bucket = objectStorageClient.bucket(bucketName);
    const file = bucket.file(objectName);

    const [exists] = await file.exists();
    if (!exists) {
      return res.status(404).json({ error: "Audio file not found" });
    }
  }
});

```

```

    res.set('Content-Type', 'audio/mpeg');
    res.set('Cache-Control', 'public, max-age=3600');
    const stream = file.createReadStream();
    stream.pipe(res);
  } catch (error: any) {
    console.error("Error serving TTS audio:", error.message);
    res.status(500).json({ error: "Failed to serve audio" });
  }
});

// POST /api/tts - Generate speech from text using OpenAI
app.post("/api/tts", async (req, res) => {
  try {
    const { text, voice = "echo" } = req.body;

    if (!text || typeof text !== "string") {
      return res.status(400).json({ message: "Text is required" });
    }

    const validVoices = ["alloy", "echo", "fable", "onyx", "nova", "shimmer"] as const;
    const selectedVoice = validVoices.includes(voice) ? voice : "echo";

    let normalizedText = cleanTextForTTS(text);

    normalizedText = normalizedText
      .replace(/doi:[^\s]+/gi, '')
      .replace(/\[d+(?:\s*d+)*\]/g, '')
      .replace(/\[A-Z\][a-z]+(?:\s+(?:&|and)\s+[A-Z][a-z]+)*,?\s*d{4}[a-z]?/g, '')
      .replace(/pp?\s*\d+(?:\s*[-]\s*\d+)?/gi, '')
      .replace(/\([^\)]{50,}/g, '')
      .replace(/[\_]/g, ' ')
      .replace(/["]/g, "'")
      .replace(/[']/g, '"')
      .replace(/\\s+/g, ' ')
      .trim();

    const trimmedText = normalizedText.slice(0, 4096);
    const startTime = Date.now();
    console.log(`TTS request: ${trimmedText.length} chars, voice: ${selectedVoice}`);

    const audioBuffer = await textToSpeech(
      trimmedText,
      selectedVoice as "alloy" | "echo" | "fable" | "onyx" | "nova" | "shimmer",
      "mp3",
      true
    );

    console.log(`TTS completed in ${Date.now() - startTime}ms, ${audioBuffer.length} bytes`);

    res.set({
      "Content-Type": "audio/mpeg",
      "Content-Length": audioBuffer.length.toString(),
    });
    res.send(audioBuffer);
  } catch (err) {
    console.error("Error generating TTS:", err);
    res.status(500).json({ message: "Failed to generate speech" });
  }
});

app.post("/api/tts/speaker", async (req, res) => {
  try {
    const { text, voice = "echo", entityId } = req.body;
    console.log(`[TTS Speaker] Request received - entity: ${entityId}, voice: ${voice}, text length: ${text?.length...}`);

    if (!text || typeof text !== "string") {
      return res.status(400).json({ error: "Text is required" });
    }

    if (!entityId) {
      return res.status(400).json({ error: "entityId is required" });
    }

    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }

    const baseUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
    const validVoices = ["alloy", "echo", "fable", "onyx", "nova", "shimmer"] as const;
    const selectedVoice = validVoices.includes(voice as any) ? voice : "echo";

    const cleanedText = cleanTextForTTS(text);
    console.log(`[TTS Speaker] Generating OpenAI audio (${cleanedText.length} chars, voice: ${selectedVoice})`);

    const isNonAlexa = NON_ALEXA_ENTITIES.includes(entityId);
    const wordCount = cleanedText.split(/\s+/).length;
    const estimatedDurationMs = Math.max(5000, (wordCount / 145) * 60 * 1000 + 2000);

    let playResp: Response;

    if (isNonAlexa) {
      const audioPath = await generateAndSaveTTSAudio(cleanedText, `speaker-tts-${Date.now()}`);
      const appUrl = DEPLOYED_APP_URL;
      const fullAudioUrl = `${appUrl}${audioPath}`;
      console.log(`[TTS Speaker] Non-Alexa: Generated audio at ${audioPath}, playing on ${entityId} via play_media`);

      playResp = await fetch(`${baseUrl}/api/services/media_player/play_media`, {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
        body: JSON.stringify({
          entity_id: entityId,
          media_content_id: fullAudioUrl,
          media_content_type: "music",
        }),
      });
    } else {
      const ssmlContent = `<speak><prosody rate=90%>${cleanedText}</prosody></speak>`;
      console.log(`[TTS Speaker] Alexa: Sending TTS to ${entityId} via notify/alexa_media (${cleanedText.length} ch...`);
    }
  }
});

```

```

    playResp = await fetch(`${haUrl}/api/services/notify/alexa_media`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({
        message: ssmContent,
        target: entityId,
        data: { type: "tts" },
      })),
    });
  }

  if (!playResp.ok) {
    const errText = await playResp.text();
    console.error(`[TTS Speaker] ${isNonAlexa ? 'play_media' : 'alexa_media'} FAILED: ${playResp.status} ${errText}`);
    return res.status(500).json({ error: "Failed to play audio on speaker" });
  }

  console.log(`[TTS Speaker] SUCCESS - playing on ${entityId} via ${isNonAlexa ? 'play_media' : 'notify/alexa_med...'}`);

  res.json({ success: true, entityId, method: isNonAlexa ? "openai_audio_play_media" : "alexa_media_tts", estimatedTime: 0 });
} catch (err) {
  console.error("[TTS Speaker] Error:", err);
  res.status(500).json({ error: "Failed to play TTS on speaker" });
}
});

// =====
// EMAIL REMINDER ROUTES
// =====

// POST /api/email/test - Send a test email
app.post("/api/email/test", async (req, res) => {
  try {
    const result = await sendTestEmail();
    if (result.success) {
      res.json({ message: "Test email sent successfully" });
    } else {
      res.status(500).json({ message: result.error || "Failed to send test email" });
    }
  } catch (err) {
    console.error("Error sending test email:", err);
    res.status(500).json({ message: "Failed to send test email" });
  }
});

// POST /api/email/reminder - Send a reminder for a specific task
app.post("/api/email/reminder", async (req, res) => {
  try {
    const { taskId } = req.body;
    if (!taskId) {
      return res.status(400).json({ message: "taskId is required" });
    }

    const task = await storage.getTask(Number(taskId));
    if (!task) {
      return res.status(404).json({ message: "Task not found" });
    }

    const result = await sendTaskReminder({
      id: task.id,
      title: task.title,
      dueDate: task.dueDate.toISOString(),
      courseName: task.courseName,
      type: task.type,
    });

    if (result.success) {
      res.json({ message: "Reminder sent successfully" });
    } else {
      res.status(500).json({ message: result.error || "Failed to send reminder" });
    }
  } catch (err) {
    console.error("Error sending reminder:", err);
    res.status(500).json({ message: "Failed to send reminder" });
  }
});

// POST /api/email/digest - Send a digest of upcoming tasks
app.post("/api/email/digest", async (req, res) => {
  try {
    const now = new Date();
    const threeDaysFromNow = new Date(now.getTime() + 3 * 24 * 60 * 60 * 1000);

    // Get all incomplete tasks due in the next 3 days
    const allTasks = await storage.getTasks({ showCompleted: false });
    const upcomingTasks = allTasks.filter(task => {
      const dueDate = new Date(task.dueDate);
      return dueDate >= now && dueDate <= threeDaysFromNow;
    });

    const taskReminders: TaskReminder[] = upcomingTasks.map(task => ({
      id: task.id,
      title: task.title,
      dueDate: task.dueDate.toISOString(),
      courseName: task.courseName,
      type: task.type,
    }));

    const result = await sendDailyDigest(taskReminders);

    if (result.success) {
      res.json({ message: `Digest sent with ${taskReminders.length} tasks` });
    } else {
      res.status(500).json({ message: result.error || "Failed to send digest" });
    }
  } catch (err) {
    console.error("Error sending digest:", err);
    res.status(500).json({ message: "Failed to send digest" });
  }
});

```



```

    }
  });

  // POST /api/sms/test - Send a test SMS
  app.post("/api/sms/test", async (_req, res) => {
    try {
      const result = await sendTestSms();
      if (result.success) {
        res.json({ message: "Test SMS sent successfully" });
      } else {
        res.status(500).json({ message: result.error || "Failed to send test SMS" });
      }
    } catch (err) {
      console.error("Error sending test SMS:", err);
      res.status(500).json({ message: "Failed to send test SMS" });
    }
  });

  // POST /api/sms/reminder - Send an SMS reminder for a specific task
  app.post("/api/sms/reminder", async (req, res) => {
    try {
      const { taskId } = req.body;
      if (!taskId) {
        return res.status(400).json({ message: "taskId is required" });
      }

      const task = await storage.getTask(Number(taskId));
      if (!task) {
        return res.status(404).json({ message: "Task not found" });
      }

      const result = await sendSmsReminder({
        id: task.id,
        title: task.title,
        dueDate: task.dueDate.toISOString(),
        courseName: task.courseName,
        type: task.type,
      });

      if (result.success) {
        res.json({ message: "SMS reminder sent successfully" });
      } else {
        res.status(500).json({ message: result.error || "Failed to send SMS reminder" });
      }
    } catch (err) {
      console.error("Error sending SMS reminder:", err);
      res.status(500).json({ message: "Failed to send SMS reminder" });
    }
  });

  // === Server-side Pomodoro Timer ===
  let pomodoroState: {
    mode: "work" | "shortBreak" | "longBreak";
    duration: number;
    startedAt: number | null;
    pausedRemaining: number | null;
    running: boolean;
    count: number;
  } = {
    mode: "work",
    duration: 25 * 60,
    startedAt: null,
    pausedRemaining: null,
    running: false,
    count: 0,
  };

  let pomodoroTimeout: ReturnType<typeof setTimeout> | null = null;

  function clearPomodoroTimeout() {
    if (pomodoroTimeout) {
      clearTimeout(pomodoroTimeout);
      pomodoroTimeout = null;
    }
  }

  function schedulePomodoroEnd(remainingSeconds: number) {
    clearPomodoroTimeout();
    pomodoroTimeout = setTimeout(async () => {
      pomodoroState.running = false;
      pomodoroState.startedAt = null;
      pomodoroState.pausedRemaining = null;

      if (pomodoroState.mode === "work") {
        pomodoroState.count += 1;
        const breakMsg = pomodoroState.count % 4 === 0 ? "Time for a long break!" : "Time for a short break!";
        try {
          await sendEchoVoiceAnnouncement("Pomodoro complete! " + breakMsg);
        } catch (e) { console.error("Pomodoro announcement error:", e); }
        if (pomodoroState.count % 4 === 0) {
          pomodoroState.mode = "longBreak";
          pomodoroState.duration = 15 * 60;
        } else {
          pomodoroState.mode = "shortBreak";
          pomodoroState.duration = 5 * 60;
        }
      } else {
        try {
          await sendEchoVoiceAnnouncement("Break is over! Time to focus!");
        } catch (e) { console.error("Pomodoro announcement error:", e); }
        pomodoroState.mode = "work";
        pomodoroState.duration = 25 * 60;
      }
      pomodoroState.pausedRemaining = pomodoroState.duration;
      console.log("Pomodoro timer expired, mode now:", pomodoroState.mode);
    }, remainingSeconds * 1000);
  }

  app.get("/api/ha-url", (_req, res) => {

```

```

const url = process.env.HOME_ASSISTANT_URL;
if (url) {
  res.json({ url });
} else {
  res.status(404).json({ error: "Home Assistant URL not configured" });
}
});

// GET /api/pomodoro/status
app.get("/api/pomodoro/status", (_req, res) => {
  let remaining = pomodoroState.pausedRemaining ?? pomodoroState.duration;
  if (pomodoroState.running && pomodoroState.startedAt) {
    const elapsed = Math.floor((Date.now() - pomodoroState.startedAt) / 1000);
    remaining = Math.max(0, pomodoroState.duration - elapsed);
  }
  res.json({
    mode: pomodoroState.mode,
    running: pomodoroState.running,
    remaining,
    count: pomodoroState.count,
  });
});

// POST /api/pomodoro/start
app.post("/api/pomodoro/start", (req, res) => {
  const { mode, duration, count } = req.body;
  if (mode) pomodoroState.mode = mode;
  if (typeof count === "number") pomodoroState.count = count;
  const remaining = pomodoroState.pausedRemaining ?? duration ?? pomodoroState.duration;
  pomodoroState.duration = remaining;
  pomodoroState.startedAt = Date.now();
  pomodoroState.pausedRemaining = null;
  pomodoroState.running = true;
  schedulePomodoroEnd(remaining);
  res.json({ message: "Pomodoro started", remaining });
});

// POST /api/pomodoro/pause
app.post("/api/pomodoro/pause", (_req, res) => {
  if (pomodoroState.running && pomodoroState.startedAt) {
    const elapsed = Math.floor((Date.now() - pomodoroState.startedAt) / 1000);
    pomodoroState.pausedRemaining = Math.max(0, pomodoroState.duration - elapsed);
  }
  pomodoroState.running = false;
  pomodoroState.startedAt = null;
  clearPomodoroTimeout();
  res.json({ message: "Pomodoro paused", remaining: pomodoroState.pausedRemaining });
});

// POST /api/pomodoro/reset
app.post("/api/pomodoro/reset", (req, res) => {
  clearPomodoroTimeout();
  const { mode } = req.body || {};
  pomodoroState.mode = mode || pomodoroState.mode;
  if (pomodoroState.mode === "work") pomodoroState.duration = 25 * 60;
  else if (pomodoroState.mode === "shortBreak") pomodoroState.duration = 5 * 60;
  else pomodoroState.duration = 15 * 60;
  pomodoroState.startedAt = null;
  pomodoroState.pausedRemaining = pomodoroState.duration;
  pomodoroState.running = false;
  res.json({ message: "Pomodoro reset", remaining: pomodoroState.duration });
});

// POST /api/pomodoro/skip
app.post("/api/pomodoro/skip", (_req, res) => {
  clearPomodoroTimeout();
  pomodoroState.running = false;
  pomodoroState.startedAt = null;
  if (pomodoroState.mode === "work") {
    pomodoroState.count += 1;
    if (pomodoroState.count % 4 === 0) {
      pomodoroState.mode = "longBreak";
      pomodoroState.duration = 15 * 60;
    } else {
      pomodoroState.mode = "shortBreak";
      pomodoroState.duration = 5 * 60;
    }
  } else {
    pomodoroState.mode = "work";
    pomodoroState.duration = 25 * 60;
  }
  pomodoroState.pausedRemaining = pomodoroState.duration;
  res.json({ message: "Pomodoro skipped", mode: pomodoroState.mode, remaining: pomodoroState.duration, count: pomodoroState.count });
});

const translationCache = new Map<string, string>();
app.post("/api/translate-ja", async (req, res) => {
  try {
    const { texts } = req.body as { texts: string[] };
    if (!texts || !Array.isArray(texts) || texts.length === 0) {
      return res.status(400).json({ error: "texts array required" });
    }
    const results: Record<string, string> = {};
    const toTranslate: string[] = [];
    for (const t of texts) {
      if (translationCache.has(t)) {
        results[t] = translationCache.get(t)!;
      } else if (/^[\u3000-\u303F\u3040-\u309F\u30A0-\u30FF\u4E00-\u9FFF\uFF00-\uFFEF\s\d\-. ,!{}]+$/i.test(t)) {
        results[t] = t;
        translationCache.set(t, t);
      } else {
        toTranslate.push(t);
      }
    }
    if (toTranslate.length > 0) {
      const OpenAI = (await import("openai")).default;
      const openai = new OpenAI({
        apiKey: process.env.AI_INTEGRATIONS_OPENAI_API_KEY,

```

```

    baseUrl: process.env.AI_INTEGRATIONS_OPENAI_BASE_URL,
  });
  const batch = toTranslate.slice(0, 20);
  const prompt = batch.map((t, i) => `${i + 1}. ${t}`).join("\n");
  const completion = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [
      { role: "system", content: "Translate each line to Japanese. For song/album/artist names, use the commonl..." },
      { role: "user", content: prompt },
    ],
    temperature: 0.3,
    max_tokens: 1000,
  });
  const responseText = completion.choices[0]?.message?.content || "";
  const lines = responseText.split("\n").filter(l => l.trim());
  for (let i = 0; i < batch.length; i++) {
    const line = lines[i] || "";
    const translated = line.replace(/^\d+\.\s*/, "").trim() || batch[i];
    results[batch[i]] = translated;
    translationCache.set(batch[i], translated);
  }
}
res.json({ translations: results });
} catch (err: any) {
  console.error("[Translate] Error:", err.message);
  res.status(500).json({ error: "Translation failed" });
}
});

// POST /api/ha-announce - Send a voice announcement to Echo speakers
app.post("/api/ha-announce", async (req, res) => {
  try {
    const { message } = req.body;
    if (!message) {
      return res.status(400).json({ message: "Message is required" });
    }
    const result = await sendEchoVoiceAnnouncement(message);
    if (result.success) {
      res.json({ message: "Announcement sent" });
    } else {
      res.status(500).json({ message: result.error || "Failed to send announcement" });
    }
  } catch (err) {
    console.error("Error sending announcement:", err);
    res.status(500).json({ message: "Failed to send announcement" });
  }
});

// POST /api/ha-push/test - Send a test push notification via Home Assistant
app.post("/api/ha-push/test", async (_req, res) => {
  try {
    const result = await sendTestHaPush();
    if (result.success) {
      res.json({ message: "Test push notification sent successfully via Home Assistant" });
    } else {
      res.status(500).json({ message: result.error || "Failed to send test push notification" });
    }
  } catch (err) {
    console.error("Error sending test HA push:", err);
    res.status(500).json({ message: "Failed to send test push notification" });
  }
});

app.get("/api/travelling", (_req, res) => {
  res.json({ isTravelling: getIsTravellingMode(), travelStartDate, travelEndDate });
});

app.post("/api/travelling", (req, res) => {
  const { isTravelling, startDate, endDate } = req.body;
  isTravellingMode = !!isTravelling;
  travelStartDate = startDate || null;
  travelEndDate = endDate || null;
  const effectivelyTravelling = getIsTravellingMode();
  console.log(`[Travelling] Mode set to: ${isTravellingMode}, dates: ${travelStartDate} - ${travelEndDate}, effecti...`);
  res.json({ isTravelling: effectivelyTravelling });
});

app.post("/api/echo/test", async (_req, res) => {
  try {
    const result = await sendEchoVoiceAnnouncement("Hey Bryn! This is a test announcement from Uni-Cal.");
    if (result.success) {
      res.json({ message: "Test voice announcement sent to Echo devices" });
    } else {
      res.status(500).json({ message: result.error || "Failed to send Echo announcement" });
    }
  } catch (err) {
    console.error("Error sending test Echo announcement:", err);
    res.status(500).json({ message: "Failed to send Echo announcement" });
  }
});

app.get("/api/reminders/status", (_req, res) => {
  res.json(getSchedulerStatus());
});

// POST /api/ha-push/reminder - Send a push notification reminder for a specific task via Home Assistant
app.post("/api/ha-push/reminder", async (req, res) => {
  try {
    const { taskId } = req.body;
    if (!taskId) {
      return res.status(400).json({ message: "taskId is required" });
    }

    const task = await storage.getTask(Number(taskId));
    if (!task) {
      return res.status(404).json({ message: "Task not found" });
    }
  }
});

```

```

    const result = await sendHaTaskReminder({
      id: task.id,
      title: task.title,
      dueDate: task.dueDate.toISOString(),
      courseName: task.courseName,
      type: task.type,
    });

    if (result.success) {
      res.json({ message: "Push notification reminder sent successfully" });
    } else {
      res.status(500).json({ message: result.error || "Failed to send push notification reminder" });
    }
  } catch (err) {
    console.error("Error sending HA push reminder:", err);
    res.status(500).json({ message: "Failed to send push notification reminder" });
  }
});

// =====
// PROJECT ROUTES
// =====

// GET /api/projects - Get all projects
app.get("/api/projects", async (req, res) => {
  try {
    const projects = await storage.getProjects();
    res.json(projects);
  } catch (err) {
    console.error("Error fetching projects:", err);
    res.status(500).json({ message: "Failed to fetch projects" });
  }
});

// GET /api/projects/:id - Get a single project
app.get("/api/projects/:id", async (req, res) => {
  try {
    const projectId = Number(req.params.id);
    const project = await storage.getProject(projectId);
    if (!project) {
      return res.status(404).json({ message: "Project not found" });
    }
    res.json(project);
  } catch (err) {
    console.error("Error fetching project:", err);
    res.status(500).json({ message: "Failed to fetch project" });
  }
});

// GET /api/projects/:id/tasks - Get all tasks for a project
app.get("/api/projects/:id/tasks", async (req, res) => {
  try {
    const projectId = Number(req.params.id);
    const tasks = await storage.getTasksByProject(projectId);
    res.json(tasks);
  } catch (err) {
    console.error("Error fetching project tasks:", err);
    res.status(500).json({ message: "Failed to fetch project tasks" });
  }
});

// POST /api/projects - Create a new project
app.post("/api/projects", async (req, res) => {
  try {
    const { name, description, color, status, courseName, startDate, targetDate, priority, notes } = req.body;
    const project = await storage.createProject({
      name,
      description,
      color,
      status,
      courseName,
      startDate: startDate ? new Date(startDate) : null,
      targetDate: targetDate ? new Date(targetDate) : null,
      priority,
      notes,
    });
    res.status(201).json(project);
  } catch (err) {
    console.error("Error creating project:", err);
    res.status(500).json({ message: "Failed to create project" });
  }
});

// PATCH /api/projects/:id - Update a project
app.patch("/api/projects/:id", async (req, res) => {
  try {
    const projectId = Number(req.params.id);
    const updates = req.body;
    if (updates.startDate) updates.startDate = new Date(updates.startDate);
    if (updates.targetDate) updates.targetDate = new Date(updates.targetDate);
    const project = await storage.updateProject(projectId, updates);
    res.json(project);
  } catch (err) {
    console.error("Error updating project:", err);
    res.status(500).json({ message: "Failed to update project" });
  }
});

// DELETE /api/projects/:id - Delete a project
app.delete("/api/projects/:id", async (req, res) => {
  try {
    const projectId = Number(req.params.id);
    await storage.deleteProject(projectId);
    res.status(204).send();
  } catch (err) {
    console.error("Error deleting project:", err);
    res.status(500).json({ message: "Failed to delete project" });
  }
});

```

```

});

// POST /api/projects/:id/tasks/:taskId - Add a task to a project
app.post("/api/projects/:id/tasks/:taskId", async (req, res) => {
  try {
    const projectId = Number(req.params.id);
    const taskId = Number(req.params.taskId);
    const task = await storage.updateTask(taskId, { projectId });
    res.json(task);
  } catch (err) {
    console.error("Error adding task to project:", err);
    res.status(500).json({ message: "Failed to add task to project" });
  }
});

// DELETE /api/projects/:id/tasks/:taskId - Remove a task from a project
app.delete("/api/projects/:id/tasks/:taskId", async (req, res) => {
  try {
    const taskId = Number(req.params.taskId);
    const task = await storage.updateTask(taskId, { projectId: null });
    res.json(task);
  } catch (err) {
    console.error("Error removing task from project:", err);
    res.status(500).json({ message: "Failed to remove task from project" });
  }
});

// ===== Sticky Notes Routes =====

// GET /api/sticky-notes - Get all sticky notes
app.get("/api/sticky-notes", async (req, res) => {
  try {
    const notes = await storage.getStickyNotes();
    res.json(notes);
  } catch (err) {
    console.error("Error fetching sticky notes:", err);
    res.status(500).json({ message: "Failed to fetch sticky notes" });
  }
});

// POST /api/sticky-notes - Create a new sticky note
app.post("/api/sticky-notes", async (req, res) => {
  try {
    const note = await storage.createStickyNote(req.body);
    res.json(note);
  } catch (err) {
    console.error("Error creating sticky note:", err);
    res.status(500).json({ message: "Failed to create sticky note" });
  }
});

// PATCH /api/sticky-notes/:id - Update a sticky note
app.patch("/api/sticky-notes/:id", async (req, res) => {
  try {
    const id = Number(req.params.id);
    const updates = { ...req.body };
    // Convert lastMovedAt from string to Date if provided
    if (updates.lastMovedAt && typeof updates.lastMovedAt === 'string') {
      updates.lastMovedAt = new Date(updates.lastMovedAt);
    }
    // Convert reminderTime from string to Date if provided
    if (updates.reminderTime && typeof updates.reminderTime === 'string') {
      updates.reminderTime = new Date(updates.reminderTime);
    }
    const note = await storage.updateStickyNote(id, updates);
    res.json(note);
  } catch (err) {
    console.error("Error updating sticky note:", err);
    res.status(500).json({ message: "Failed to update sticky note" });
  }
});

// DELETE /api/sticky-notes/:id - Delete a sticky note
app.delete("/api/sticky-notes/:id", async (req, res) => {
  try {
    const id = Number(req.params.id);
    await storage.deleteStickyNote(id);
    res.json({ success: true });
  } catch (err) {
    console.error("Error deleting sticky note:", err);
    res.status(500).json({ message: "Failed to delete sticky note" });
  }
});

// Access Token routes for time-limited sharing
app.get("/api/access-tokens", async (req, res) => {
  try {
    const tokens = await storage.getAccessTokens();
    res.json(tokens);
  } catch (err) {
    console.error("Error getting access tokens:", err);
    res.status(500).json({ message: "Failed to get access tokens" });
  }
});

app.post("/api/access-tokens", async (req, res) => {
  try {
    const { name } = req.body;
    const token = crypto.randomUUID();
    const created = await storage.createAccessToken({ token, name: name || null, firstUsedAt: null, expiresAt: null... });
    res.json(created);
  } catch (err) {
    console.error("Error creating access token:", err);
    res.status(500).json({ message: "Failed to create access token" });
  }
});

app.post("/api/access-tokens/validate", async (req, res) => {

```

```

try {
  const { token } = req.body;
  if (!token) {
    return res.status(400).json({ valid: false, message: "Token required" });
  }

  const accessToken = await storage.getAccessToken(token);
  if (!accessToken) {
    return res.json({ valid: false, message: "Invalid token" });
  }

  if (accessToken.isRevoked) {
    return res.json({ valid: false, message: "Token has been revoked" });
  }

  const now = new Date();

  // Check if token has expired
  if (accessToken.expiresAt && new Date(accessToken.expiresAt) < now) {
    return res.json({ valid: false, message: "Token has expired" });
  }

  // If first use, set expiration to 1 hour from now
  if (!accessToken.firstUsedAt) {
    const expiresAt = new Date(now.getTime() + 60 * 60 * 1000); // 1 hour
    await storage.updateAccessToken(accessToken.id, { firstUsedAt: now, expiresAt });
    return res.json({ valid: true, message: "Access granted", expiresAt });
  }

  return res.json({ valid: true, message: "Access granted", expiresAt: accessToken.expiresAt });
} catch (err) {
  console.error("Error validating access token:", err);
  res.status(500).json({ valid: false, message: "Failed to validate token" });
}
});

app.delete("/api/access-tokens/:id", async (req, res) => {
  try {
    const id = Number(req.params.id);
    await storage.deleteAccessToken(id);
    res.json({ success: true });
  } catch (err) {
    console.error("Error deleting access token:", err);
    res.status(500).json({ message: "Failed to delete access token" });
  }
});

app.patch("/api/access-tokens/:id/revoke", async (req, res) => {
  try {
    const id = Number(req.params.id);
    const updated = await storage.updateAccessToken(id, { isRevoked: true });
    res.json(updated);
  } catch (err) {
    console.error("Error revoking access token:", err);
    res.status(500).json({ message: "Failed to revoke access token" });
  }
});

app.post("/api/background-photo/upload", async (req, res) => {
  try {
    const { ObjectStorageService } = await import("./replit_integrations/object_storage");
    const objectStorageService = new ObjectStorageService();
    const uploadURL = await objectStorageService.getObjectEntityUploadURL();
    const objectPath = objectStorageService.normalizeObjectEntityPath(uploadURL);
    res.json({ uploadURL, objectPath });
  } catch (error) {
    console.error("Error generating background photo upload URL:", error);
    res.status(500).json({ error: "Failed to generate upload URL" });
  }
});

// Profile photo upload
app.post("/api/profile-photo/upload", async (req, res) => {
  try {
    const { ObjectStorageService } = await import("./replit_integrations/object_storage");
    const objectStorageService = new ObjectStorageService();
    const uploadURL = await objectStorageService.getObjectEntityUploadURL();
    const objectPath = objectStorageService.normalizeObjectEntityPath(uploadURL);
    res.json({ uploadURL, objectPath });
  } catch (error) {
    console.error("Error generating profile photo upload URL:", error);
    res.status(500).json({ error: "Failed to generate upload URL" });
  }
});

// Register object storage routes for file uploads
registerObjectStorageRoutes(app);

app.post("/api/uploads/direct", async (req, res) => {
  try {
    const chunks: Buffer[] = [];
    req.on('data', (chunk: Buffer) => chunks.push(chunk));
    await new Promise<void>((resolve, reject) => {
      req.on('end', resolve);
      req.on('error', reject);
    });
    const fileBuffer = Buffer.concat(chunks);
    const fileName = (req.headers['x-file-name'] as string) || 'upload.pdf';
    const contentType = (req.headers['content-type'] as string) || 'application/octet-stream';

    const { ObjectStorageService } = await import("./replit_integrations/object_storage");
    const objectStorageService = new ObjectStorageService();
    const privateDir = process.env.PRIVATE_OBJECT_DIR;
    if (!privateDir) throw new Error("PRIVATE_OBJECT_DIR not set");

    const { randomUUID } = await import("crypto");
    const objectId = randomUUID();
    const fullPath = `${privateDir}/uploads/${objectId}`;
  }
});

```

```

const pathParts = fullPath.replace(/^\//, '').split('/');
const bucketName = pathParts[0];
const objectName = pathParts.slice(1).join('/');

const bucket = objectStorageClient.bucket(bucketName);
const file = bucket.file(objectName);
await new Promise<void>((resolve, reject) => {
  const stream = file.createWriteStream({ contentType, resumable: false });
  stream.on('finish', resolve);
  stream.on('error', reject);
  stream.end(fileBuffer);
});

const objectPath = `/objects/uploads/${objectId}`;
const createdFile = await storage.createFile({
  originalName: fileName,
  displayName: fileName,
  objectPath,
  contentType,
  size: fileBuffer.length,
});

console.log(`[Upload] Direct upload success: ${fileName} -> ${objectPath} (${fileBuffer.length} bytes)`);
res.json({ objectPath, fileId: createdFile.id, metadata: { name: fileName, size: fileBuffer.length, contentTyp...
} catch (error: any) {
  console.error("[Upload] Direct upload error:", error);
  res.status(500).json({ error: error.message || "Failed to upload file" });
}
});

// =====
// SHOWER AUTOMATION - Auto-play PDFs on motion
// =====

// POST /api/debug/setup-semester - Set up semester settings on production
app.post("/api/debug/setup-semester", async (req, res) => {
  try {
    // Check if settings already exist
    const existing = await storage.getActiveSemesterSettings();
    if (existing) {
      return res.json({ message: "Semester settings already exist", settings: existing });
    }

    // Create Winter 2026 settings (Jan 12, 2026 start)
    const newSettings = await storage.createSemesterSettings({
      semesterName: "Winter 2026",
      semesterStartDate: new Date("2026-01-12T00:00:00"),
      course1Code: "CPPA122",
      course1Name: "Local Politics and Government",
      course2Code: "CFNF400",
      course2Name: "Human Sexuality",
      course3Code: "CASL101",
      course3Name: "American Sign Language",
      isActive: true
    });

    return res.json({ message: "Semester settings created", settings: newSettings });
  } catch (error: any) {
    return res.status(500).json({ error: error.message });
  }
});

// GET /api/debug/week - Debug endpoint to check current week calculation
app.get("/api/debug/week", async (req, res) => {
  try {
    const semesterSettings = await storage.getActiveSemesterSettings();
    const today = torontoDate();

    if (!semesterSettings?.semesterStartDate) {
      return res.json({
        error: "No semester settings found",
        semesterSettings: null,
        serverTime: today.toISOString()
      });
    }

    const startDate = new Date(semesterSettings.semesterStartDate);
    const diffTime = today.getTime() - startDate.getTime();
    const diffDays = Math.floor(diffTime / (1000 * 60 * 60 * 24));
    const currentWeekNumber = getWeekNumber(today, startDate, semesterSettings.readingWeekStart);

    return res.json({
      semesterStartDate: semesterSettings.semesterStartDate,
      startDateParsed: startDate.toISOString(),
      serverTime: today.toISOString(),
      diffDays,
      currentWeekNumber,
      semesterName: semesterSettings.semesterName
    });
  } catch (error: any) {
    return res.status(500).json({ error: error.message });
  }
});

// In-memory storage for playback progress (persists until server restart)
const playbackProgress: Record<string, { chunkIndex: number; totalChunks: number; lastPlayed: Date; lastCompletedCh...

// Track active kitchen playback session (for stopping)
let kitchenPlaybackActive = false;
let kitchenPlaybackAbortController: AbortController | null = null;

const pendingTabletCommands: Record<string, { action: string; url?: string; goodbyeText?: string; timestamp: number...

let tabletTableReady = false;
async function ensureTabletTable() {
  if (!tabletTableReady) {
    await db.execute(sql`CREATE TABLE IF NOT EXISTS tablet_commands (device TEXT PRIMARY KEY, cmd JSONB NOT NULL)`);
  }
}

```

```

    tabletTableReady = true;
  }
}

async function dbSetTabletCommand(device: string, cmd: { action: string; url?: string; goodbyeText?: string; timest...
  try {
    await ensureTabletTable();
    const cmdJson = JSON.stringify(cmd);
    await db.execute(sql`INSERT INTO tablet_commands (device, cmd) VALUES (${device}, ${cmdJson::jsonb}) ON CONFLIC...
    const verify = await db.execute(sql`SELECT cmd FROM tablet_commands WHERE device = ${device}`);
    console.log(`[Tablet Nav DB] SET ${device}: wrote ${cmdJson.substring(0, 80)}, verify rows=${verify.rows?.length...
  } catch (e: any) {
    console.log(`[Tablet Nav DB] Error saving ${device}: ${e.message}`);
  }
}

async function dbGetTabletCommand(device: string): Promise<{ action: string; url?: string; goodbyeText?: string; ti...
  try {
    await ensureTabletTable();
    const result = await db.execute(sql`SELECT cmd FROM tablet_commands WHERE device = ${device}`);
    if (result.rows && result.rows.length > 0) {
      const row = result.rows[0] as any;
      const cmd = typeof row.cmd === 'string' ? JSON.parse(row.cmd) : row.cmd;
      console.log(`[Tablet Nav DB] GET ${device}: found rows=${result.rows.length} action=${cmd?.action} ts=${cmd?...
      return cmd;
    }
    console.log(`[Tablet Nav DB] GET ${device}: no rows found`);
  } catch (e: any) {
    console.log(`[Tablet Nav DB] Error loading ${device}: ${e.message}`);
  }
  return null;
}

async function dbClearTabletCommand(device: string) {
  try {
    await db.execute(sql`DELETE FROM tablet_commands WHERE device = ${device}`);
  } catch {}
}

async function setTabletCommand(cmd: { action: string; url?: string; goodbyeText?: string; timestamp: number }, pro...
  pendingTabletCommands[device] = cmd;
  await dbSetTabletCommand(device, cmd);
  if (propagate) {
    try {
      await fetch(`${DEPLOYED_APP_URL}/api/tablet-nav/set?auth=${encodeURIComponent(process.env.SITE_PASSWORD || '...
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(cmd),
      });
    } catch (e: any) {
      console.log(`[Tablet Nav] Failed to propagate ${device} to deployed: ${e.message}`);
    }
  }
}

app.get("/api/ha-entity-state", async (req, res) => {
  const entity = req.query.entity as string;
  if (!entity) return res.status(400).json({ error: "entity required" });
  try {
    const resp = await fetch(`http://172.24.0.2:8123/api/states/${encodeURIComponent(entity)}`, {
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}` },
    });
    if (!resp.ok) return res.status(resp.status).json({ error: "HA error" });
    const data = await resp.json() as any;
    res.json({ entity_id: data.entity_id, state: data.state });
  } catch {
    res.status(500).json({ error: "Failed to reach HA" });
  }
});

app.get("/api/tablet-nav", async (req, res) => {
  res.set('Cache-Control', 'no-store, no-cache, must-revalidate, proxy-revalidate');
  res.set('Pragma', 'no-cache');
  res.set('Expires', '0');
  const rawDevice = (req.query.device as string) || 'master';
  const device = rawDevice === 'follower' ? 'master' : rawDevice;
  let cmd = pendingTabletCommands[device];
  let source = 'memory';
  if (!cmd) {
    cmd = (await dbGetTabletCommand(device)) || undefined;
    source = 'db';
    if (cmd) pendingTabletCommands[device] = cmd;
  }
  if (cmd && cmd.action) {
    const age = Date.now() - cmd.timestamp;
    if (age < 120000) {
      console.log(`[Tablet Nav GET] ${device} FOUND from ${source}: action=${cmd.action} age=${age}ms`);
      return res.json(cmd);
    } else {
      console.log(`[Tablet Nav GET] ${device} EXPIRED from ${source}: action=${cmd.action} age=${age}ms`);
      delete pendingTabletCommands[device];
      await dbClearTabletCommand(device);
    }
  }
  res.json({ action: null });
});

app.get("/api/tablet-nav/debug", (_req, res) => {
  const devices = Object.keys(pendingTabletCommands);
  const cmds: Record<string, any> = {};
  for (const d of devices) {
    const c = pendingTabletCommands[d];
    cmds[d] = c ? { action: c.action, age: Date.now() - c.timestamp, url: c.url?.substring(0, 60) } : null;
  }
  res.json({ pendingCommands: cmds, deviceCount: devices.length });
});

```



```

app.get("/api/cat-wash/find-next", async (req, res) => {
  const authParam = (req.query.auth as string) || '';
  if (authParam !== (process.env.SITE_PASSWORD || '')) {
    return res.status(401).json({ error: "Unauthorized" });
  }
  try {
    const semesterSettings = await storage.getActiveSemesterSettings();
    let currentWeekNumber = 1;
    const semStart = semesterSettings?.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : new Date(...
    const rwStart = semesterSettings?.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : new Date("20...
    currentWeekNumber = getWeekNumber(torontoDate(), semStart, rwStart);
    const nextFile = await findNextCatWashFile(storage, currentWeekNumber);
    if (!nextFile) {
      return res.json({ found: false, weekNumber: currentWeekNumber });
    }
    res.json({ found: true, fileId: nextFile.id, fileName: nextFile.displayName || nextFile.originalName, folder: n...
  } catch (e: any) {
    res.status(500).json({ error: e.message });
  }
});

app.post("/api/tablet-nav/ack", async (req, res) => {
  const { timestamp, device } = req.body;
  const deviceKey = device || 'master';
  const cmd = pendingTabletCommands[deviceKey];
  if (cmd && cmd.timestamp === timestamp) {
    delete pendingTabletCommands[deviceKey];
    await dbClearTabletCommand(deviceKey);
    console.log(`[Tablet Nav] Command acknowledged and cleared for ${deviceKey}`);
  }
  res.json({ ok: true });
});

app.post("/api/tablet-nav/set", async (req, res) => {
  const authParam = (req.query.auth as string) || '';
  if (authParam !== (process.env.SITE_PASSWORD || '')) {
    return res.status(401).json({ error: "Unauthorized" });
  }
  const deviceParam = (req.query.device as string) || 'master';
  const { action, url, goodbyeText, timestamp } = req.body;
  if (action) {
    const ts = timestamp || Date.now();
    const cmd = { action, url, goodbyeText, timestamp: ts };
    pendingTabletCommands[deviceParam] = cmd;
    await dbSetTabletCommand(deviceParam, cmd);
    console.log(`[Tablet Nav] Command SET for ${deviceParam}: ${action} url=${url || 'none'} ts=${ts} pending=${JSON...
  }
  res.json({ ok: true });
});

app.post("/api/fix-tablets", async (req, res) => {
  const authParam = (req.query.auth as string) || '';
  if (authParam !== (process.env.SITE_PASSWORD || '')) {
    return res.status(401).json({ error: "Unauthorized" });
  }
  const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
  const haHeaders = { 'Authorization': 'Bearer ${HOME_ASSISTANT_TOKEN}', 'Content-Type': 'application/json' };
  const dashboardUrl = `${DEPLOYED_APP_URL}/tablet`;
  const results: string[] = [];

  const tabletAdbEntities = [
    { entity: "media_player.tablet_hallway_entrance", name: "Hallway Entrance" },
    { entity: "media_player.tablet_hallway", name: "Hallway Main" },
    { entity: "media_player.tablet_11", name: "Living Room" },
    { entity: "media_player.bd24bb29_04a116d8_king", name: "King Bedroom" },
    { entity: "media_player.tablet_queen", name: "Queen Bedroom" },
    { entity: "media_player.tablet_kitchen_island", name: "Kitchen Island" },
    { entity: "media_player.tablet_cat", name: "Cat Washroom" },
  ];

  for (const tablet of tabletAdbEntities) {
    try {
      await fetch(`${haUrl}/api/services/androidtv/adb_command`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: tablet.entity, command: 'input keyevent KEYCODE_WAKEUP' }),
      });
      await new Promise(r => setTimeout(r, 500));

      await fetch(`${haUrl}/api/services/androidtv/adb_command`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: tablet.entity, command: 'am start --activity-clear-task -a android.intent...
      });
      await new Promise(r => setTimeout(r, 500));

      await fetch(`${haUrl}/api/services/androidtv/adb_command`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: tablet.entity, command: 'settings put global policy_control immersive.ful...
      });
      await fetch(`${haUrl}/api/services/androidtv/adb_command`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: tablet.entity, command: 'input keyevent KEYCODE_F11' }),
      });
      results.push(`${tablet.name}: OK`);
      console.log(`[Fix Tablets] ${tablet.name} (${tablet.entity}): navigated to dashboard + fullscreen`);
    } catch (e: any) {
      results.push(`${tablet.name}: ERROR ${e.message}`);
      console.error(`[Fix Tablets] ${tablet.name} error: ${e.message}`);
    }
  }

  pendingTabletCommands['master'] = { action: 'go_home', timestamp: Date.now() };
  pendingTabletCommands['tv'] = { action: 'go_home', timestamp: Date.now() };
  await dbSetTabletCommand('master', { action: 'go_home', timestamp: Date.now() });
  await dbSetTabletCommand('tv', { action: 'go_home', timestamp: Date.now() });

  res.json({ ok: true, results });
});

```

```

let currentTvFollowUrl: string | null = null;
let currentTabletReaderUrl: string | null = null;

app.get("/api/cat-wash/tablet-redirect", (_req, res) => {
  if (currentTabletReaderUrl) {
    console.log(`[Cat Wash Tablet] Redirecting to: ${currentTabletReaderUrl}`);
    return res.redirect(currentTabletReaderUrl);
  }
  res.status(404).send('<html><body style="background:#0a0a1a;color:white;display:flex;align-items:center;justify-c...
});

app.get("/api/cat-wash/tv-follow", (_req, res) => {
  const tvUrl = (_req.query.url ? decodeURIComponent(String(_req.query.url)) : '') || currentTvFollowUrl || '';
  const baseUrl = DEPLOYED_APP_URL;
  console.log(`[Cat Wash TV] Serving fullscreen wrapper, target: ${tvUrl} || 'none'`);
  res.setHeader('Cache-Control', 'no-store, no-cache, must-revalidate');
  res.setHeader('Content-Type', 'text/html');
  res.send('<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,initial-scale=1,maximum-scale=1,user-scalable=no">
<meta name="mobile-web-app-capable" content="yes">
<title>Uni-Cal TV</title>
<style>
*{margin:0;padding:0;box-sizing:border-box}
html,body{width:100%;height:100%;overflow:hidden;background:#0a0a1a}
iframe{width:100%;height:100%;border:none;position:fixed;top:0;left:0;right:0;bottom:0}
#waiting{display:flex;align-items:center;justify-content:center;height:100vh;color:white;font-family:sans-serif;font-...
</style>
</head>
<body>
${tvUrl ? `<iframe id="frame" src="${tvUrl}" allow="fullscreen;autoplay"></iframe>` : `<div id="waiting">No active pl...
<script>
(function(){
  function goFullscreen() {
    var el = document.documentElement;
    try {
      if (el.requestFullscreen) el.requestFullscreen();
      else if (el.webkitRequestFullscreen) el.webkitRequestFullscreen();
      else if (el.mozRequestFullScreen) el.mozRequestFullScreen();
    } catch(e){}
  }
  document.addEventListener('click', goFullscreen);
  document.addEventListener('touchstart', goFullscreen);
  setTimeout(goFullscreen, 500);
  setTimeout(goFullscreen, 2000);

  var lastTs = 0;
  function poll() {
    var url = `${baseUrl}/api/tablet-nav?device=tv&auth=5747&t=' + Date.now();
    fetch(url, {cache:'no-store'}).then(function(r){return r.json();}).then(function(data){
      if (!data || !data.action) return;
      if (data.timestamp && data.timestamp <= lastTs) return;
      if (data.timestamp && (Date.now() - data.timestamp > 120000)) return;
      var frame = document.getElementById('frame');
      var waiting = document.getElementById('waiting');
      if (data.action === 'navigate' && data.url) {
        lastTs = data.timestamp || Date.now();
        fetch(`${baseUrl}/api/tablet-nav/ack`, {method:'POST',headers:{'Content-Type':'application/json'},body:JSON.s...
        if (data.url.indexOf('tv-follow') !== -1) return;
        var targetUrl = data.url;
        if (targetUrl.indexOf('/tablet?') !== -1) {
          try { targetUrl = new URL(targetUrl).searchParams.get('target') || targetUrl; } catch(e){}
        }
        if (!frame) {
          if (waiting) waiting.remove();
          frame = document.createElement('iframe');
          frame.id = 'frame';
          frame.allow = 'fullscreen;autoplay';
          frame.style.cssText = 'width:100vw;height:100vh;border:none;position:fixed;top:0;left:0';
          document.body.appendChild(frame);
        }
        frame.src = targetUrl;
      } else if (data.action === 'stop_playback') {
        lastTs = data.timestamp || Date.now();
        fetch(`${baseUrl}/api/tablet-nav/ack`, {method:'POST',headers:{'Content-Type':'application/json'},body:JSON.s...
      }
    }).catch(function(){});
  }
  setInterval(poll, 3000);
  poll();
})();
</script>
</body>
</html>`);
});

app.get("/api/cat-wash/test-tv-browser", async (_req, res) => {
  const entityId = FIRE_STICK_ADB_ENTITY;
  const testUrl = `${DEPLOYED_APP_URL}/api/cat-wash/tv-follow`;
  const method = String(_req.query.method || 'all');
  const results: { method: string; success: boolean; error?: string }[] = [];

  currentTvFollowUrl = `${DEPLOYED_APP_URL}/pdf-reader?catWashFollow=true`;

  console.log(`[TV Test] ===== TEST START (method=${method}) =====`);
  console.log(`[TV Test] Entity: ${entityId}`);
  console.log(`[TV Test] URL: ${testUrl}`);

  try {
    const stateResp = await fetch(`${HOME_ASSISTANT_URL.replace(/\$/,'')}/api/states/${entityId}`, {
      headers: { 'Authorization': 'Bearer ${HOME_ASSISTANT_TOKEN}', 'Content-Type': 'application/json' }
    });
    if (stateResp.ok) {
      const stateData = await stateResp.json();
      console.log(`[TV Test] Fire Stick state: "${stateData.state}", attrs: ${JSON.stringify(stateData.attributes?...)}`
    }
  }

```

```

} catch (e: any) {
  console.log(`[TV Test] State check failed: ${e.message}`);
}

if (method === 'all' || method === 'wake') {
  try {
    await haServiceCall('media_player/turn_on', { entity_id: CAT_TV_ENTITY }, 'TV Test Samsung TurnOn');
    console.log(`[TV Test] Samsung TV turn_on: OK`);
    results.push({ method: 'samsung_turn_on', success: true });
  } catch (e: any) {
    console.log(`[TV Test] Samsung TV turn_on failed: ${e.message}`);
    results.push({ method: 'samsung_turn_on', success: false, error: e.message });
  }

  try {
    await haServiceCall('media_player/turn_on', { entity_id: entityId }, 'TV Test FireStick TurnOn');
    console.log(`[TV Test] Fire Stick turn_on: OK`);
    results.push({ method: 'firestick_turn_on', success: true });
  } catch (e: any) {
    console.log(`[TV Test] Fire Stick turn_on failed: ${e.message}`);
    results.push({ method: 'firestick_turn_on', success: false, error: e.message });
  }

  try {
    await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: 'input keyevent KEYCODE_WAKEUP' ...
    console.log(`[TV Test] WAKEUP: OK`);
    results.push({ method: 'wakeUp', success: true });
  } catch (e: any) {
    console.log(`[TV Test] WAKEUP failed: ${e.message}`);
    results.push({ method: 'wakeUp', success: false, error: e.message });
  }

  console.log(`[TV Test] Waiting 12s for TV to boot...`);
  await new Promise(r => setTimeout(r, 12000));
}

if (method === 'all' || method === 'silk') {
  try {
    await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: 'am force-stop com.amazon.cloud9...
    console.log(`[TV Test] force-stop Silk: OK`);
  } catch (e: any) {
    console.log(`[TV Test] force-stop Silk failed: ${e.message}`);
  }
  await new Promise(r => setTimeout(r, 1500));

  const adbCmd = `am start --activity-clear-task -a android.intent.action.VIEW -d "${testUrl}" com.amazon.cloud9`;
  try {
    await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: adbCmd }, 'TV Test Silk');
    console.log(`[TV Test] Silk am start: OK`);
    results.push({ method: 'silk_am_start', success: true });
  } catch (e: any) {
    console.log(`[TV Test] Silk am start failed: ${e.message}`);
    results.push({ method: 'silk_am_start', success: false, error: e.message });
  }
}

if (method === 'silk-no-flags') {
  try {
    await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: 'am force-stop com.amazon.cloud9...
  } catch (e: any) {}
  await new Promise(r => setTimeout(r, 1500));

  const adbCmd = `am start -a android.intent.action.VIEW -d "${testUrl}" com.amazon.cloud9`;
  try {
    await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: adbCmd }, 'TV Test Silk NoFlags');
    console.log(`[TV Test] Silk (no flags): OK`);
    results.push({ method: 'silk_no_flags', success: true });
  } catch (e: any) {
    console.log(`[TV Test] Silk (no flags) failed: ${e.message}`);
    results.push({ method: 'silk_no_flags', success: false, error: e.message });
  }
}

if (method === 'generic') {
  const adbCmd = `am start -a android.intent.action.VIEW -d "${testUrl}"`;
  try {
    await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: adbCmd }, 'TV Test Generic');
    console.log(`[TV Test] Generic intent: OK`);
    results.push({ method: 'generic_intent', success: true });
  } catch (e: any) {
    console.log(`[TV Test] Generic intent failed: ${e.message}`);
    results.push({ method: 'generic_intent', success: false, error: e.message });
  }
}

if (method === 'play_media') {
  try {
    await haServiceCall('media_player/play_media', { entity_id: entityId, media_content_id: testUrl, media_conten...
    console.log(`[TV Test] play_media: OK`);
    results.push({ method: 'play_media', success: true });
  } catch (e: any) {
    console.log(`[TV Test] play_media failed: ${e.message}`);
    results.push({ method: 'play_media', success: false, error: e.message });
  }
}

if (method === 'silk-activity') {
  try {
    await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: 'am force-stop com.amazon.cloud9...
  } catch (e: any) {}
  await new Promise(r => setTimeout(r, 1500));

  const adbCmd = `am start -n com.amazon.cloud9/.BrowserActivity -a android.intent.action.VIEW -d "${testUrl}"`;
  try {
    await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: adbCmd }, 'TV Test SilkActivity');
    console.log(`[TV Test] Silk BrowserActivity: OK`);
    results.push({ method: 'silk_browser_activity', success: true });
  } catch (e: any) {}
}

```

```

        console.log(`[TV Test] Silk BrowserActivity failed: ${e.message}`);
        results.push({ method: 'silk_browser_activity', success: false, error: e.message });
    }
}

if (method === 'list-browsers') {
    const cmds = [
        'pm list packages | grep -i brows',
        'pm list packages | grep -i silk',
        'pm list packages | grep -i cloud9',
        'pm list packages | grep -i chrome',
        'pm list packages | grep -i firefox',
        'pm list packages | grep -i web',
    ];
    for (const cmd of cmds) {
        try {
            await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: cmd }, `TV Test List ${cmd.split(' ')}`);
            console.log(`[TV Test] ${cmd}: OK (check HA logs for output)`);
            results.push({ method: cmd, success: true });
        } catch (e: any) {
            console.log(`[TV Test] ${cmd}: ${e.message}`);
            results.push({ method: cmd, success: false, error: e.message });
        }
    }
}

if (method === 'adb-test') {
    const testCmds = [
        { cmd: 'input keyevent KEYCODE_HOME', label: 'HOME button' },
        { cmd: 'input keyevent 26', label: 'POWER button' },
        { cmd: 'dumpsys package com.amazon.cloud9 | head -1', label: 'Check Silk installed' },
        { cmd: 'getprop ro.product.model', label: 'Get device model' },
    ];
    for (const { cmd, label } of testCmds) {
        try {
            await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: cmd }, `TV Test ${label}`);
            console.log(`[TV Test] ${label}: OK`);
            results.push({ method: label, success: true });
        } catch (e: any) {
            console.log(`[TV Test] ${label}: FAILED - ${e.message}`);
            results.push({ method: label, success: false, error: e.message });
        }
        await new Promise(r => setTimeout(r, 2000));
    }
}

if (method === 'remote') {
    const remoteEntities = [
        'remote.fire_stick_cat_wr',
        'remote.cat_wr_fire_stick',
        'remote.fire_tv_cat_wr',
    ];
    for (const remoteEntity of remoteEntities) {
        try {
            const stateResp2 = await fetch(`${HOME_ASSISTANT_URL.replace(/\/$/, '')}/api/states/${remoteEntity}`, {
                headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' }
            });
            if (stateResp2.ok) {
                const stateData2 = await stateResp2.json();
                console.log(`[TV Test] Remote entity ${remoteEntity}: state=${stateData2.state}`);
                results.push({ method: `remote_check_${remoteEntity}`, success: true });
            } else {
                console.log(`[TV Test] Remote entity ${remoteEntity}: HTTP ${stateResp2.status}`);
                results.push({ method: `remote_check_${remoteEntity}`, success: false, error: `HTTP ${stateResp2.status}`... });
            }
        } catch (e: any) {
            console.log(`[TV Test] Remote entity ${remoteEntity}: ${e.message}`);
            results.push({ method: `remote_check_${remoteEntity}`, success: false, error: e.message });
        }
    }
}

if (method === 'samsung-browser') {
    const tvEntity = CAT_TV_ENTITY;
    console.log(`[TV Test] Testing Samsung Smart TV browser methods on ${tvEntity}`);

    const samsungMethods = [
        { service: 'media_player/play_media', data: { entity_id: tvEntity, media_content_id: testUrl, media_content_t... }, label: 'Samsung s...' },
        { service: 'media_player/play_media', data: { entity_id: tvEntity, media_content_id: testUrl, media_content_t... }, label: 'Samsung s...' },
        { service: 'media_player/select_source', data: { entity_id: tvEntity, source: 'Internet' }, label: 'Samsung s...' },
        { service: 'media_player/select_source', data: { entity_id: tvEntity, source: 'Web Browser' }, label: 'Samsung s...' },
        { service: 'media_player/select_source', data: { entity_id: tvEntity, source: 'Browser' }, label: 'Samsung s...' },
        { service: 'samsungtv/select_source', data: { entity_id: tvEntity, source: 'Internet' }, label: 'Samsung sams...' },
    ];
    for (const { service, data, label } of samsungMethods) {
        try {
            await haServiceCall(service, data, `TV Test ${label}`);
            console.log(`[TV Test] ${label}: OK`);
            results.push({ method: label, success: true });
        } catch (e: any) {
            console.log(`[TV Test] ${label}: FAILED - ${e.message}`);
            results.push({ method: label, success: false, error: e.message });
        }
        await new Promise(r => setTimeout(r, 3000));
    }
}

if (method === 'samsung-open-url') {
    const tvEntity = CAT_TV_ENTITY;
    try {
        await haServiceCall('media_player/play_media', { entity_id: tvEntity, media_content_id: testUrl, media_conten... }, `TV Test Samsung open URL: OK`);
        console.log(`[TV Test] Samsung open URL: OK`);
        results.push({ method: 'samsung_open_url', success: true });
    } catch (e: any) {
        console.log(`[TV Test] Samsung open URL: FAILED - ${e.message}`);
        results.push({ method: 'samsung_open_url', success: false, error: e.message });
    }
}

```

```

if (method === 'samsung-sources') {
  const tvEntity = CAT_TV_ENTITY;
  try {
    const stateResp2 = await fetch(`${HOME_ASSISTANT_URL.replace(/\$/g, '')}/api/states/${tvEntity}`, {
      headers: { 'Authorization': 'Bearer ${HOME_ASSISTANT_TOKEN}', 'Content-Type': 'application/json' }
    });
    if (stateResp2.ok) {
      const stateData2 = await stateResp2.json();
      console.log(`[TV Test] Samsung TV state: "${stateData2.state}"`);
      console.log(`[TV Test] Samsung TV source: "${stateData2.attributes?.source}"`);
      console.log(`[TV Test] Samsung TV source_list: ${JSON.stringify(stateData2.attributes?.source_list)}`);
      console.log(`[TV Test] Samsung TV app_name: "${stateData2.attributes?.app_name}"`);
      console.log(`[TV Test] Samsung TV supported_features: ${stateData2.attributes?.supported_features}`);
      results.push({ method: 'samsung_state', success: true, error: `state=${stateData2.state}, source=${stateData2.source}` });
    }
  } catch (e: any) {
    results.push({ method: 'samsung_state', success: false, error: e.message });
  }
}

if (method === 'firestick-internet') {
  try {
    await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: 'am force-stop com.amazon.cloud9...' });
  } catch (e: any) {}
  await new Promise(r => setTimeout(r, 1000));

  const browsers = [
    { pkg: 'com.amazon.cloud9', activity: 'com.amazon.cloud9.BrowserActivity', label: 'Silk BrowserActivity' },
    { pkg: 'com.amazon.cloud9', activity: 'com.amazon.cloud9.MainActivity', label: 'Silk MainActivity' },
    { pkg: 'com.amazon.cloud9', activity: null, label: 'Silk (package only)' },
    { pkg: 'com.amazon.internet', activity: null, label: 'Amazon Internet' },
    { pkg: 'org.mozilla.tv.firefox', activity: null, label: 'Firefox TV' },
    { pkg: 'com.phlox.tvwebbrowser', activity: null, label: 'TV Web Browser' },
  ];
  for (const { pkg, activity, label } of browsers) {
    try {
      const cmd = activity
        ? `am start -n ${pkg}/${activity} -a android.intent.action.VIEW -d "${testUrl}"`
        : `am start -a android.intent.action.VIEW -d "${testUrl}" ${pkg}`;
      await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: cmd }, 'TV Test ${label}');
      console.log(`[TV Test] ${label}: OK`);
      results.push({ method: label, success: true });
    } catch (e: any) {
      console.log(`[TV Test] ${label}: FAILED - ${e.message}`);
      results.push({ method: label, success: false, error: e.message });
    }
    await new Promise(r => setTimeout(r, 3000));
  }
}

console.log(`[TV Test] ===== TEST COMPLETE =====`);
console.log(`[TV Test] Results: ${JSON.stringify(results)}`);
res.json({ results, testUrl });
});

const SERVER_START_TIME = Date.now();
const SERVER_STARTUP_COOLDOWN_MS = 60 * 1000;

// ===== HA Connectivity Health Monitor =====
interface HAHealthState {
  connected: boolean;
  lastSuccessAt: number | null;
  lastFailureAt: number | null;
  lastCheckAt: number | null;
  consecutiveFailures: number;
  consecutiveSuccesses: number;
  totalChecks: number;
  totalFailures: number;
  lastError: string | null;
  wasDownSince: number | null;
}

const haHealth: HAHealthState = {
  connected: true,
  lastSuccessAt: null,
  lastFailureAt: null,
  lastCheckAt: null,
  consecutiveFailures: 0,
  consecutiveSuccesses: 0,
  totalChecks: 0,
  totalFailures: 0,
  lastError: null,
  wasDownSince: null,
};

async function checkHAConnectivity(): Promise<boolean> {
  haHealth.totalChecks++;
  haHealth.lastCheckAt = Date.now();
  try {
    const controller = new AbortController();
    const timer = setTimeout(() => controller.abort(), 8000);
    const resp = await fetch(`${HOME_ASSISTANT_URL.replace(/\$/g, '')}/api/`, {
      headers: { 'Authorization': 'Bearer ${HOME_ASSISTANT_TOKEN}' },
      signal: controller.signal,
    });
    clearTimeout(timer);
    if (resp.ok) {
      const wasDown = !haHealth.connected;
      const downSince = haHealth.wasDownSince;
      haHealth.connected = true;
      haHealth.lastSuccessAt = Date.now();
      haHealth.consecutiveSuccesses++;
      haHealth.consecutiveFailures = 0;
      haHealth.lastError = null;
      haHealth.wasDownSince = null;

      if (wasDown && downSince) {
        const downDuration = Math.round((Date.now() - downSince) / 1000);
      }
    }
  }
}

```

```

        console.log(`[HA Health] ' Connection RESTORED after ${downDuration}s of downtime`);
        try {
            await fetch(`${HOME_ASSISTANT_URL.replace(/\$/ , '')}/api/services/persistent_notification/create`, {
                method: 'POST',
                headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
                body: JSON.stringify({
                    title: "Study Dashboard Reconnected",
                    message: `Dashboard connection restored after ${downDuration}s of downtime. Automations are active ag...`,
                    notification_id: "study_dashboard_health",
                }),
            });
        } catch {}
        if (haCommandQueue.length > 0) {
            console.log(`[HA Health] Draining ${haCommandQueue.length} queued command(s) after reconnection`);
            processHACommandQueue().catch(e => console.warn(`[HA Queue] Drain error: ${e.message}`));
        }
    }
    if (haCommandQueue.length > 0 && !haQueueProcessing) {
        processHACommandQueue().catch(() => {});
    }
    return true;
}
}
throw new Error(`HTTP ${resp.status}`);
} catch (e: any) {
    haHealth.consecutiveFailures++;
    haHealth.consecutiveSuccesses = 0;
    haHealth.totalFailures++;
    haHealth.lastFailureAt = Date.now();
    haHealth.lastError = e?.message || String(e);

    if (haHealth.connected) {
        haHealth.connected = false;
        haHealth.wasDownSince = Date.now();
        console.error(`[HA Health] ' Connection LOST: ${haHealth.lastError}`);
    } else {
        console.warn(`[HA Health] ' Still disconnected (${haHealth.consecutiveFailures} consecutive failures): ${haHe...`);
    }
    return false;
}
}

const HA_HEALTH_CHECK_INTERVAL_MS = 60 * 1000;
const HA_HEARTBEAT_INTERVAL_MS = 30 * 1000;
let haHealthInterval: ReturnType<typeof setInterval> | null = null;
let haHeartbeatInterval: ReturnType<typeof setInterval> | null = null;

async function sendHAHeartbeat() {
    try {
        const uptimeSeconds = Math.round((Date.now() - SERVER_START_TIME) / 1000);
        const haUrl = HOME_ASSISTANT_URL.replace(/\$/ , '');
        await fetch(`${haUrl}/api/states/sensor.study_dashboard_heartbeat`, {
            method: 'POST',
            headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
            body: JSON.stringify({
                state: "online",
                attributes: {
                    friendly_name: "Study Dashboard Heartbeat",
                    device_class: "connectivity",
                    uptime_seconds: uptimeSeconds,
                    last_heartbeat: new Date().toISOString(),
                    ha_connected: haHealth.connected,
                    consecutive_ha_failures: haHealth.consecutiveFailures,
                },
            }),
        });
    } catch (e: any) {
        console.warn(`[Heartbeat] Failed to send heartbeat to HA: ${e.message}`);
    }
}

async function sendStartupNotification() {
    try {
        const haUrl = HOME_ASSISTANT_URL.replace(/\$/ , '');
        const startTime = new Date().toLocaleString("en-US", { timeZone: "America/Toronto", hour: "numeric", minute: "2..." });
        await fetch(`${haUrl}/api/services/persistent_notification/create`, {
            method: 'POST',
            headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
            body: JSON.stringify({
                title: "Study Dashboard Online",
                message: `Dashboard started at ${startTime}. Monitoring and automations are active.`,
                notification_id: "study_dashboard_startup",
            }),
        });
    } catch {}
    console.log(`[Startup] Sent startup notification to HA`);
    await fetch(`${haUrl}/api/services/notify/mobile_app_iphone_10`, {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
        body: JSON.stringify({
            title: "Study Dashboard Online",
            message: `Dashboard started at ${startTime}`,
            data: { push: { sound: "default" } },
        }),
    });
    console.log(`[Startup] Sent startup push notification`);
} catch (e: any) {
    console.warn(`[Startup] Failed to send startup notification: ${e.message}`);
}

}

setTimeout(async () => {
    const ok = await checkHACConnectivity();
    console.log(`[HA Health] Initial check: ${ok ? 'connected' : 'disconnected'}`);
    if (ok) {
        await sendStartupNotification();
        await sendHAHeartbeat();
    }
    haHealthInterval = setInterval(() => {
        checkHACConnectivity();

```

```

    }, HA_HEALTH_CHECK_INTERVAL_MS);
    haHeartbeatInterval = setInterval(() => {
      sendHAHeartbeat();
    }, HA_HEARTBEAT_INTERVAL_MS);
  }, 10000);

app.get("/api/health", async (_req, res) => {
  const uptimeSeconds = Math.round((Date.now() - SERVER_START_TIME) / 1000);
  res.json({
    status: "ok",
    uptime: uptimeSeconds,
    ha: {
      connected: haHealth.connected,
      lastSuccessAt: haHealth.lastSuccessAt ? new Date(haHealth.lastSuccessAt).toISOString() : null,
      lastFailureAt: haHealth.lastFailureAt ? new Date(haHealth.lastFailureAt).toISOString() : null,
      consecutiveFailures: haHealth.consecutiveFailures,
      totalChecks: haHealth.totalChecks,
      totalFailures: haHealth.totalFailures,
      lastError: haHealth.lastError,
      downSince: haHealth.wasDownSince ? new Date(haHealth.wasDownSince).toISOString() : null,
      commandQueue: {
        pending: haCommandQueue.length,
        processing: haQueueProcessing,
        oldest: haCommandQueue.length > 0 ? new Date(haCommandQueue[0].queuedAt).toISOString() : null,
      },
    },
    heartbeat: {
      interval_seconds: HA_HEARTBEAT_INTERVAL_MS / 1000,
      sensor: "sensor.study_dashboard_heartbeat",
    },
    timestamp: new Date().toISOString(),
  });
});

async function checkAndActivateSemester(): Promise<void> {
  try {
    const now = new Date();
    const allSemesters = await storage.getAllSemesterSettings();
    const currentActive = allSemesters.find(s => s.isActive);

    if (currentActive) {
      const endDate = currentActive.semesterEndDate ? new Date(currentActive.semesterEndDate) : null;
      if (endDate && now < endDate) return;
    }

    const sorted = allSemesters
      .filter(s => {
        const start = new Date(s.semesterStartDate);
        const end = s.semesterEndDate ? new Date(s.semesterEndDate) : null;
        return start <= now && (!end || now <= end);
      })
      .sort((a, b) => new Date(b.semesterStartDate).getTime() - new Date(a.semesterStartDate).getTime());

    const shouldBeActive = sorted[0];
    if (!shouldBeActive) {
      const upcoming = allSemesters
        .filter(s => new Date(s.semesterStartDate) > now)
        .sort((a, b) => new Date(a.semesterStartDate).getTime() - new Date(b.semesterStartDate).getTime());
      if (upcoming.length > 0) {
        const nextStart = new Date(upcoming[0].semesterStartDate);
        const daysUntil = Math.ceil((nextStart.getTime() - now.getTime()) / (1000 * 60 * 60 * 24));
        console.log(`[Semester Auto] Between semesters - next is "${upcoming[0].semesterName}" in ${daysUntil} days`);
      }
      return;
    }

    if (currentActive && currentActive.id === shouldBeActive.id) return;

    if (currentActive) {
      await storage.updateSemesterSettings(currentActive.id, { isActive: false });
    }
    await storage.updateSemesterSettings(shouldBeActive.id, { isActive: true });
    console.log(`[Semester Auto] Activated "${shouldBeActive.semesterName}" (ID ${shouldBeActive.id})`);
  } catch (e: any) {
    console.error(`[Semester Auto] Error: ${e.message}`);
  }
}

checkAndActivateSemester();
setInterval(checkAndActivateSemester, 6 * 60 * 60 * 1000);

// Self-ping to keep Replit from sleeping (every 4 minutes)
const SELF_PING_INTERVAL_MS = 4 * 60 * 1000;
const APP_URL = DEPLOYED_APP_URL;
setTimeout(() => {
  setInterval(async () => {
    try {
      const controller = new AbortController();
      const timer = setTimeout(() => controller.abort(), 10000);
      await fetch(`${APP_URL}/api/health`, { signal: controller.signal });
      clearTimeout(timer);
    } catch (e: any) {
      console.warn(`[Self-Ping] Failed: ${e.message}`);
    }
  }, SELF_PING_INTERVAL_MS);
}, 30000);

// Track active cat-wash playback session with unique session ID to prevent concurrent loops
let catWashPlaybackActive = false;
let catWashSessionId = 0;
let catWashPlaybackStartedAt: Date | null = null;
let catWashManuallyStoppedAt: Date | null = null;
let catWashPlaybackTrigger: 'lights' | 'manual' | null = null;
let lastVolumeChange: { volume: number; direction: string; timestamp: number } | null = null;
let catLightsConfirmResolve: ((value: boolean) => void) | null = null;
let catLightsLastPromptAt: number | null = null;
let catLightsPromptPending = false;
let coursePlayPriority: Record<string, number> = {};

```

```

const CAT_LIGHTS_PROMPT_COOLDOWN_MS = 3 * 60 * 1000;
let catLightsBypassCooldown = false;
let lastPlaybackStoppedAt: number = 0;
let toothbrushPollInterval: ReturnType<typeof setInterval> | null = null;

function catWashTrace(source: string, event: string, details?: Record<string, any>): {
  const now = new Date();
  const ts = now.toLocaleString('en-US', { timeZone: 'America/Toronto', hour12: true, hour: 'numeric', minute: '2-d...
  const state: Record<string, any> = {
    active: catWashPlaybackActive,
    sessionId: catWashSessionId,
    file: catWashPlaybackState?.fileName || null,
    chunk: catWashPlaybackState ? `${catWashPlaybackState.chunkIndex}/${catWashPlaybackState.totalChunks}` : null,
    lightsPrompt: catLightsPromptPending,
    trigger: catWashPlaybackTrigger,
    msSinceStop: lastPlaybackStoppedAt ? Date.now() - lastPlaybackStoppedAt : null,
    msSinceManualStop: catWashManuallyStoppedAt ? Date.now() - catWashManuallyStoppedAt.getTime() : null,
    msSinceStart: catWashPlaybackStartedAt ? Date.now() - catWashPlaybackStartedAt.getTime() : null,
  };
  const detailStr = details ? ` | ${JSON.stringify(details)}` : '';
  console.log(`[TRACE ${ts}] [${source}] ${event} | state=${JSON.stringify(state)}${detailStr}`);
}

interface PersistedPlaybackSession {
  fileId: number;
  fileName: string;
  chunkIndex: number;
  totalChunks: number;
  trigger: 'lights' | 'manual';
  startedAt: string;
  updatedAt: string;
  status: 'active' | 'paused';
}

async function savePlaybackSession(data: PersistedPlaybackSession): Promise<void> {
  try {
    const json = JSON.stringify(data);
    const existing = await db.select().from(appState).where(eq(appState.key, 'playback_session')).limit(1);
    if (existing.length > 0) {
      await db.update(appState).set({ value: json, updatedAt: new Date() }).where(eq(appState.key, 'playback_sessio...
    } else {
      await db.insert(appState).values({ key: 'playback_session', value: json });
    }
  } catch (e: any) {
    console.error(`[PlaybackPersist] Save failed: ${e.message}`);
  }
}

async function getPersistedPlaybackSession(): Promise<PersistedPlaybackSession | null> {
  try {
    const rows = await db.select().from(appState).where(eq(appState.key, 'playback_session')).limit(1);
    if (rows.length > 0 && rows[0].value) {
      return JSON.parse(rows[0].value) as PersistedPlaybackSession;
    }
  } catch (e: any) {
    console.error(`[PlaybackPersist] Load failed: ${e.message}`);
  }
  return null;
}

async function clearPlaybackSession(): Promise<void> {
  try {
    await db.delete(appState).where(eq(appState.key, 'playback_session'));
  } catch (e: any) {
    console.error(`[PlaybackPersist] Clear failed: ${e.message}`);
  }
}

const PROF_REQD_COURSES = new Set(['CPPA101', 'CPPA102', 'CPPA120', 'CPPA121', 'CPPA122', 'CPPA124', 'CPPA125']);
const LIBERAL_CODES = new Set(LIBERAL_STUDIES_COURSES.map(c => c.code.replace(/\\s/g, '')));
const OPEN_ELECTIVE_CODES = new Set(OPEN_ELECTIVE_COURSES.map(c => c.code.replace(/\\s/g, '')));

const semesterHalvesStore: Record<string, { springStartDate?: string; springEndDate?: string; summerStartDate?: str...

app.post("/api/semester-halves", (req, res) => {
  const { semKey, springStartDate, springEndDate, summerStartDate, summerEndDate } = req.body;
  if (!semKey) return res.status(400).json({ error: "semKey required" });
  semesterHalvesStore[semKey] = { springStartDate, springEndDate, summerStartDate, summerEndDate };
  res.json({ ok: true });
});

app.get("/api/semester-halves", (_req, res) => {
  res.json(semesterHalvesStore);
});

function isInSpringSummerHalfA(): boolean {
  const { toZonedTime } = require('date-fns-tz');
  const now = toZonedTime(new Date(), 'America/Toronto');

  for (const [key, halves] of Object.entries(semesterHalvesStore)) {
    if (halves.springStartDate && halves.springEndDate) {
      const springStart = new Date(halves.springStartDate);
      const springEnd = new Date(halves.springEndDate + 'T23:59:59');
      if (now >= springStart && now <= springEnd) return true;
    }
    if (halves.summerStartDate && halves.summerEndDate) {
      const summerStart = new Date(halves.summerStartDate);
      const summerEnd = new Date(halves.summerEndDate + 'T23:59:59');
      if (now >= summerStart && now <= summerEnd) return false;
    }
  }
}

const ssDefs = [
  { key: 'ss2025', start: '2025-05-05', mid: '2025-06-20' },
  { key: 'ss2026', start: '2026-05-04', mid: '2026-06-19' },
  { key: 'ss2027', start: '2027-05-03', mid: '2027-06-18' },
  { key: 'ss2028', start: '2028-05-01', mid: '2028-06-16' },
];

```



```

    for (const ss of ssDefs) {
        if (now >= new Date(ss.start) && now < new Date(ss.mid + 'T23:59:59')) return true;
    }
    return false;
}

function getCoursePriorityForFile(f: any): number {
    const folder = (f.folder || '').toLowerCase();
    const name = (f.originalName || '').toLowerCase();
    const codeWithNum = folder.match(/[a-z]{3,5}\s?\d{3}/i)?.[1]?.toUpperCase().replace(/\\s/g, '') ||
        name.match(/[a-z]{3,5}\s?\d{3}/i)?.[1]?.toUpperCase().replace(/\\s/g, '') || '';
    if (!codeWithNum) return 999;

    const abSuffix = isInSpringSummerHalfA() ? 'A' : 'B';
    for (const [key, priority] of Object.entries(coursePlayPriority)) {
        const parts = key.split(':');
        const keyCode = (parts[1] || '').toUpperCase().replace(/\\s/g, '');
        const keySuffix = parts[2] || '';
        if (keyCode === codeWithNum && priority > 0) {
            if (keySuffix) {
                if (keySuffix === abSuffix) return priority;
            } else {
                return priority;
            }
        }
    }

    if (PROF_REQD_COURSES.has(codeWithNum)) return 100;
    if (LIBERAL_CODES.has(codeWithNum)) return 200;
    if (OPEN_ELECTIVE_CODES.has(codeWithNum)) return 300;
    return 999;
}

function getSemesterTypeFolder(semType: string | null | undefined): string {
    const t = (semType || 'winter').toLowerCase();
    if (t.includes('spring') || t.includes('summer')) return 'Spring & Summer';
    if (t.includes('fall')) return 'Fall';
    return 'Winter';
}

function generateWeekFolderNames(semester: any, courseIndex: number): string[] {
    const semType = (semester.semesterType || 'winter').toLowerCase();
    const isSpSu = semType.includes('spring') || semType.includes('summer');
    const months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];

    function fmt(d: Date): string {
        return `${months[d.getMonth()]} ${d.getDate()}`;
    }

    if (!isSpSu) {
        const semStart = semester.semesterStartDate ? new Date(semester.semesterStartDate) : new Date();
        const readingWeekStart = semester.readingWeekStart ? new Date(semester.readingWeekStart) : null;
        const weeks: string[] = [];

        const firstDay = new Date(semStart);
        firstDay.setHours(0, 0, 0, 0);

        const firstDayOfWeek = firstDay.getDay();
        const firstFriday = new Date(firstDay);
        const daysUntilFriday = (5 - firstDayOfWeek + 7) % 7;
        firstFriday.setDate(firstFriday.getDate() + (daysUntilFriday === 0 ? 0 : daysUntilFriday));

        let weekNum = 1;
        let readingWeekInserted = false;

        const week1Start = new Date(firstDay);
        const week1End = new Date(firstFriday);

        if (readingWeekStart && !readingWeekInserted) {
            const rwStart = new Date(readingWeekStart);
            rwStart.setHours(0, 0, 0, 0);
            const rwSat = new Date(rwStart);
            const rwDay = rwSat.getDay();
            if (rwDay !== 6) rwSat.setDate(rwSat.getDate() - ((rwDay + 1) % 7));
            if (week1Start.getTime() >= rwSat.getTime() && week1Start.getTime() <= new Date(rwSat.getTime() + 6 * 86400000...)) {
                weeks.push('Reading Week - STUDY');
                readingWeekInserted = true;
            }
        }

        if (!readingWeekInserted || weeks.length === 0) {
            const startStr = fmt(week1Start);
            const endStr = week1Start.getMonth() === week1End.getMonth() ? `${week1End.getDate()}` : fmt(week1End);
            weeks.push(`Week ${weekNum} - ${startStr}-${endStr}`);
            weekNum++;
        } else {
            weekNum++;
        }

        let satAfterFirstFriday = new Date(firstFriday);
        satAfterFirstFriday.setDate(satAfterFirstFriday.getDate() + 1);

        for (let i = 0; weekNum <= 13; i++) {
            const wStart = new Date(satAfterFirstFriday);
            wStart.setDate(wStart.getDate() + i * 7);
            const wEnd = new Date(wStart);
            wEnd.setDate(wEnd.getDate() + 6);

            if (readingWeekStart && !readingWeekInserted) {
                const rwStart = new Date(readingWeekStart);
                rwStart.setHours(0, 0, 0, 0);
                const rwSat = new Date(rwStart);
                const rwDay = rwSat.getDay();
                if (rwDay !== 6) rwSat.setDate(rwSat.getDate() - ((rwDay + 1) % 7));
                if (wStart.getTime() === rwSat.getTime()) {
                    weeks.push('Reading Week - STUDY');
                    readingWeekInserted = true;
                    continue;
                }
            }
        }
    }
}

```

```

    }
  }

  const startStr = fmt(wStart);
  const endStr = wStart.getMonth() === wEnd.getMonth() ? `${wEnd.getDate()}` : fmt(wEnd);
  weeks.push(`Week ${weekNum} - ${startStr}-${endStr}`);
  weekNum++;
  if (i > 20) break;
}
return weeks;
}

const springSummerTerm = (semester[`course${courseIndex}SpringSummerTerm`] || 'full').toLowerCase();
let courseStart: Date;
let courseEnd: Date;
const semStart = new Date(semester.semesterStartDate || Date.now());

if (springSummerTerm === 'first_half') {
  courseStart = semester[`course${courseIndex}StartDate`] ? new Date(semester[`course${courseIndex}StartDate`]) : ...
  courseEnd = semester[`course${courseIndex}EndDate`] ? new Date(semester[`course${courseIndex}EndDate`]) : new D...
} else if (springSummerTerm === 'second_half') {
  courseStart = semester[`course${courseIndex}StartDate`] ? new Date(semester[`course${courseIndex}StartDate`]) : ...
  courseEnd = semester[`course${courseIndex}EndDate`] ? new Date(semester[`course${courseIndex}EndDate`]) : new D...
} else {
  courseStart = semester[`course${courseIndex}StartDate`] ? new Date(semester[`course${courseIndex}StartDate`]) : ...
  courseEnd = semester[`course${courseIndex}EndDate`] ? new Date(semester[`course${courseIndex}EndDate`]) : new D...
}
courseStart.setHours(0, 0, 0, 0);
courseEnd.setHours(0, 0, 0, 0);

const weeks: string[] = [];
const firstDayOW = courseStart.getDay();
const firstFri = new Date(courseStart);
const dUntilFri = (5 - firstDayOW + 7) % 7;
firstFri.setDate(firstFri.getDate() + (dUntilFri === 0 ? 0 : dUntilFri));

const w1StartStr = fmt(courseStart);
const w1EndStr = courseStart.getMonth() === firstFri.getMonth() ? `${firstFri.getDate()}` : fmt(firstFri);
let weekNum = 1;

if (springSummerTerm === 'full') {
  const midpoint = 8;
  let secondHalfNum = 1;
  weeks.push(`Week 1 - ${w1StartStr}-${w1EndStr}`);
  weekNum = 2;

  let satStart = new Date(firstFri);
  satStart.setDate(satStart.getDate() + 1);
  while (satStart.getTime() < courseEnd.getTime()) {
    const wStart = new Date(satStart);
    const wEnd = new Date(wStart);
    wEnd.setDate(wEnd.getDate() + 6);
    const startStr = fmt(wStart);
    const endStr = wStart.getMonth() === wEnd.getMonth() ? `${wEnd.getDate()}` : fmt(wEnd);
    if (weekNum >= midpoint) {
      weeks.push(`Week ${weekNum} (${secondHalfNum}) - ${startStr}-${endStr}`);
      secondHalfNum++;
    } else {
      weeks.push(`Week ${weekNum} - ${startStr}-${endStr}`);
    }
    weekNum++;
    satStart.setDate(satStart.getDate() + 7);
  }
} else {
  weeks.push(`Week 1 - ${w1StartStr}-${w1EndStr}`);
  weekNum = 2;

  let satStart = new Date(firstFri);
  satStart.setDate(satStart.getDate() + 1);
  while (satStart.getTime() < courseEnd.getTime()) {
    const wStart = new Date(satStart);
    const wEnd = new Date(wStart);
    wEnd.setDate(wEnd.getDate() + 6);
    const startStr = fmt(wStart);
    const endStr = wStart.getMonth() === wEnd.getMonth() ? `${wEnd.getDate()}` : fmt(wEnd);
    weeks.push(`Week ${weekNum} - ${startStr}-${endStr}`);
    weekNum++;
    satStart.setDate(satStart.getDate() + 7);
  }
}
return weeks;
}

async function getSemesterOneDriveCourses(semesterSettings: any): Promise<Array<{ code: string; path: string }>> {
  if (!semesterSettings) return [];
  const semType = getSemesterTypeFolder(semesterSettings.semesterType);
  const startDate = semesterSettings.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : new Date();
  const year = startDate.getFullYear();
  const basePath = `/School/1. TMU/Courses/${year}/${semType}`;
  const courses: Array<{ code: string; path: string }> = [];
  for (let i = 1; i <= 3; i++) {
    const code = (semesterSettings as any)[`course${i}Code`];
    const name = (semesterSettings as any)[`course${i}Name`];
    if (!code || !code.trim()) continue;
    const codeClean = code.replace(/\\s/g, '');
    const folderName = name ? `${codeClean} - ${name}` : codeClean;
    courses.push({ code: codeClean, path: `${basePath}/${folderName}` });
  }
  return courses;
}

async function syncOneDriveFilesForWeek(semesterSettings: any, currentWeekNumber: number, logPrefix: string = '[Syn...
  try {
    const { listOneDriveItems } = await import("./onedrive");
    const { ObjectStorageService } = await import("./replit_integrations/object_storage");
    const objectStorageSync = new ObjectStorageService();
    const courses = await getSemesterOneDriveCourses(semesterSettings);
    console.log(`${logPrefix} Syncing OneDrive for ${courses.length} courses, week ${currentWeekNumber}`);
  }
}

```

```

for (const course of courses) {
  try {
    const weekFolders = await listOneDriveItems(course.path);
    const currentWeekFolder = weekFolders.find((f: any) => f.type === 'folder' && f.name.match(/Week\s+(\d+)/i)...
    if (!currentWeekFolder) continue;
    const weekContents = await listOneDriveItems(currentWeekFolder.path);
    for (const subType of ['module', 'reading']) {
      const subFolder = weekContents.find((f: any) => f.type === 'folder' && f.name.toLowerCase() === subType);
      if (!subFolder) continue;
      const subFiles = await listOneDriveItems(subFolder.path);
      for (const file of subFiles) {
        if (file.type !== 'file' || !file.name.endsWith('.pdf')) continue;
        const existingFiles = await storage.getFiles();
        const folderName = `week-${currentWeekNumber}-${course.code.toLowerCase()}-${subType}`;
        if (existingFiles.some((f: any) => f.originalName === file.name && f.folder === folderName)) continue;
        const downloadResponse = await fetch(file.downloadUrl);
        if (!downloadResponse.ok) continue;
        const fileBuffer = Buffer.from(await downloadResponse.arrayBuffer());
        const uploadUrl = await objectStorageSync.getObjectEntityUploadURL();
        const uploadResponse = await fetch(uploadUrl, { method: 'PUT', body: fileBuffer, headers: { 'Content-Ty...
        if (!uploadResponse.ok) continue;
        const objectPath = objectStorageSync.normalizeObjectEntityPath(uploadUrl);
        const newFile = await storage.createFile({ originalName: file.name, displayName: file.name, objectPath, ...
        console.log(`${logPrefix} Synced new file: ${file.name} !' ${folderName}`);
        if (newFile?.id) {
          queueFileForPreparation(newFile.id);
        }
      }
    }
  } catch (e: any) { console.log(`${logPrefix} OneDrive sync error for ${course.code}: ${e.message}`); }
}
console.log(`${logPrefix} OneDrive sync complete`);
} catch (e: any) { console.log(`${logPrefix} OneDrive sync failed: ${e.message}`); }
}

let audioPreparationActive = false;
let audioPreparationPaused = false;
const audioPreparationQueue: number[] = [];

async function prepareFileAudio(fileId: number): Promise<void> {
  try {
    const file = await storage.getFile(fileId);
    if (!file) {
      console.log(`[AudioPrep] File ${fileId} not found - skipping`);
      return;
    }
    if (file.preparedAudioPaths) {
      console.log(`[AudioPrep] File ${fileId} (${file.originalName}) already prepared - skipping`);
      return;
    }

    console.log(`[AudioPrep] ===== Preparing file ${fileId}: ${file.originalName} =====`);
    const startTime = Date.now();

    let text = await extractFileText(file);
    if (!text || text.length < 20) {
      console.log(`[AudioPrep] No usable text extracted from ${file.originalName} - skipping`);
      return;
    }

    const chunks = chunkTextForNest(text);
    if (chunks.length === 0) {
      console.log(`[AudioPrep] No chunks generated for ${file.originalName} - skipping`);
      return;
    }

    console.log(`[AudioPrep] ${file.originalName}: ${text.length} chars !' ${chunks.length} chunks`);
    const audioPaths: string[] = [];
    const voice = 'echo';

    let consecutiveRateLimits = 0;
    for (let i = 0; i < chunks.length; i++) {
      while (audioPreparationPaused) {
        await new Promise(r => setTimeout(r, 2000));
      }
      try {
        console.log(`[AudioPrep] Generating chunk ${i + 1}/${chunks.length} for ${file.originalName} (${chunks[i].l...
        const audioPath = await generateAndSaveTTSAudio(chunks[i], `prep-${fileId}-chunk-${i}`, voice, true);
        audioPaths.push(audioPath);
        consecutiveRateLimits = 0;
      } catch (e: any) {
        const isRateLimit = e.message?.includes('429') || e.message?.includes('rate limit') || e.message?.includes(...
        console.error(`[AudioPrep] Failed to generate chunk ${i + 1} for ${file.originalName}: ${e.message}`);
        if (isRateLimit) {
          consecutiveRateLimits++;
          const backoffMs = Math.min(consecutiveRateLimits * 15000, 120000);
          console.log(`[AudioPrep] Rate limited (${consecutiveRateLimits}x) - backing off ${backoffMs / 1000}s`);
          await new Promise(r => setTimeout(r, backoffMs));
          i--;
          continue;
        }
        audioPaths.push('');
        await new Promise(r => setTimeout(r, 2000));
      }
      await new Promise(r => setTimeout(r, 800));
    }

    await storage.updateFile(fileId, {
      extractedText: text,
      totalChunks: chunks.length,
      preparedAudioPaths: JSON.stringify(audioPaths),
      preparedAt: new Date(),
    });

    const elapsed = Math.round((Date.now() - startTime) / 1000);
    console.log(`[AudioPrep] ===== Completed ${file.originalName}: ${chunks.length} chunks in ${elapsed}s =====`);
  } catch (e: any) {
    console.error(`[AudioPrep] Error preparing file ${fileId}: ${e.message}`);
  }
}

```

```

    }
  }

  async function processAudioPreparationQueue(): Promise<void> {
    if (audioPreparationActive) return;
    audioPreparationActive = true;
    try {
      while (audioPreparationQueue.length > 0) {
        while (audioPreparationPaused) {
          console.log(`[AudioPrep] Queue paused – waiting for live playback to finish`);
          await new Promise(r => setTimeout(r, 5000));
        }
        const fileId = audioPreparationQueue.shift()!;
        await prepareFileAudio(fileId);
      }
    } finally {
      audioPreparationActive = false;
    }
  }

  function queueFileForPreparation(fileId: number): void {
    if (!audioPreparationQueue.includes(fileId)) {
      audioPreparationQueue.push(fileId);
      console.log(`[AudioPrep] Queued file ${fileId} for preparation (queue size: ${audioPreparationQueue.length})`);
      processAudioPreparationQueue().catch(e => console.error(`[AudioPrep] Queue processing error: ${e.message}`));
    }
  }

  setTimeout(async () => {
    try {
      const allFiles = await storage.getFiles();
      const voiceMigrationDone = globalThis.__voiceMigrationDone;
      if (!voiceMigrationDone) {
        const filesWithPrep = allFiles.filter((f: any) => f.preparedAudioPaths);
        if (filesWithPrep.length > 0) {
          console.log(`[AudioPrep] One-time: clearing ${filesWithPrep.length} pre-generated audio caches (voice migra...`);
          for (const f of filesWithPrep) {
            try { await storage.updateFile(f.id, { preparedAudioPaths: null }); } catch {}
          }
        }
        (globalThis as any).__voiceMigrationDone = true;
      }
      const citationCleanupDone = (globalThis as any).__citationCleanupDone;
      if (!citationCleanupDone) {
        const filesWithText = allFiles.filter((f: any) => f.extractedText);
        if (filesWithText.length > 0) {
          console.log(`[AudioPrep] One-time: clearing ${filesWithText.length} cached extracted texts (citation cleanu...`);
          for (const f of filesWithText) {
            try { await storage.updateFile(f.id, { extractedText: null, preparedAudioPaths: null }); } catch {}
          }
        }
        (globalThis as any).__citationCleanupDone = true;
      }
      const unprepared = allFiles.filter((f: any) => !f.preparedAudioPaths && !f.listened);
      if (unprepared.length > 0) {
        const semesterSettings = await storage.getActiveSemesterSettings();
        const semStart = semesterSettings?.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : new Date...
        const rwStart = semesterSettings?.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : null;
        const currentWeek = getWeekNumber(torontoDate(), semStart, rwStart);

        const getFileWeek = (f: any) => {
          const m = f.folder?.match(/week-(\d+)/i);
          return m ? parseInt(m[1], 10) : 999;
        };
        unprepared.sort((a: any, b: any) => {
          const aw = getFileWeek(a), bw = getFileWeek(b);
          const aDist = Math.abs(aw - currentWeek), bDist = Math.abs(bw - currentWeek);
          return aDist - bDist;
        });

        console.log(`[AudioPrep] Startup: ${unprepared.length} unprepared files found, queuing (current week ${currentWeek})`);
        for (const f of unprepared) {
          queueFileForPreparation(f.id);
        }
      } else {
        console.log(`[AudioPrep] Startup: all files already prepared or listened`);
      }
    } catch (e: any) {
      console.error(`[AudioPrep] Startup scan error: ${e.message}`);
    }
  }, AUDIO_REPAIR_INTERVAL_MS);

  const AUDIO_REPAIR_INTERVAL_MS = 30 * 60 * 1000;
  setInterval(async () => {
    if (audioPreparationActive || audioPreparationPaused || catWashPlaybackActive) return;
    try {
      const allFiles = await storage.getFiles();
      const filesWithGaps = allFiles.filter((f: any) => {
        if (!f.preparedAudioPaths || f.listened) return false;
        try {
          const paths: string[] = JSON.parse(f.preparedAudioPaths);
          return paths.some(p => !p || p.length === 0);
        } catch { return false; }
      });
      if (filesWithGaps.length > 0) {
        console.log(`[AudioPrep Repair] Found ${filesWithGaps.length} file(s) with incomplete audio – re-preparing`);
        for (const f of filesWithGaps) {
          await storage.updateFile(f.id, { preparedAudioPaths: null });
          queueFileForPreparation(f.id);
        }
      }
    } catch (e: any) {
      console.error(`[AudioPrep Repair] Error: ${e.message}`);
    }
  }, AUDIO_REPAIR_INTERVAL_MS);

  try {
    const persisted = await getPersistedPlaybackSession();
    if (persisted) {

```

```

const sessionAge = Date.now() - new Date(persisted.updatedAt).getTime();
const maxAge = 30 * 60 * 1000;
if (sessionAge > maxAge) {
  console.log('[PlaybackRecovery] Found stale session (${Math.round(sessionAge / 60000)}m old, status=${persi...
  await clearPlaybackSession();
} else if (persisted.status === 'paused') {
  console.log('[PlaybackRecovery] Found paused session: "${persisted.fileName}" at chunk ${persisted.chunkInd...
  try {
    await storage.updateFile(persisted.fileId, { lastChunkIndex: persisted.chunkIndex });
  } catch {}
  await clearPlaybackSession();
} else {
  console.log('[PlaybackRecovery] Found active session: "${persisted.fileName}" at chunk ${persisted.chunkInd...
  const file = await storage.getFile(persisted.fileId);
  if (file && !file.listened) {
    const resumeChunk = persisted.chunkIndex;
    try {
      await storage.updateFile(persisted.fileId, { lastChunkIndex: resumeChunk });
    } catch {}
    await clearPlaybackSession();

    catWashPlaybackTrigger = persisted.trigger || 'manual';
    console.log('[PlaybackRecovery] Starting full playback flow from chunk ${resumeChunk}');
    const updatedFile = await storage.getFile(persisted.fileId);
    if (updatedFile) {
      startConfirmedPlaybackFlow(updatedFile, '[PlaybackRecovery]', 'echo', null);
    }
  } else {
    console.log('[PlaybackRecovery] File ${persisted.fileId} not found or already listened – clearing');
    await clearPlaybackSession();
  }
}
} catch (e: any) {
  console.error('[PlaybackRecovery] Startup recovery check error: ${e.message}');
}
}, 15000);

function findNextFileByPriority(allFiles: any[], currentWeekNumber: number, excludeFileId?: number): any | null {
  const weekFiles = allFiles.filter((f: any) => {
    if (f.listened) return false;
    if (excludeFileId && f.id === excludeFileId) return false;
    const weekMatch = f.folder?.match(/week-(\d+)/i);
    return weekMatch && parseInt(weekMatch[1], 10) === currentWeekNumber;
  });
  if (weekFiles.length === 0) {
    console.log('[FileOrder] No unlistened files found in week ${currentWeekNumber}');
    return null;
  }
  const ordered = orderFilesByCoursePriority(weekFiles);
  console.log('[FileOrder] Found unlistened files in week ${currentWeekNumber}: ${ordered.map((f: any) => `${f.orig...
  return ordered[0] || null;
}

function isSpotifyPlayingOnEverywhere(): boolean {
  return spotifyActivePlaybacks.has(EVERYWHERE_GROUP_ENTITY);
}

async function stopAllCatWashroomSpeakers(haUrl: string): Promise<void> {
  const haHeaders = { 'Authorization': 'Bearer ${HOME_ASSISTANT_TOKEN}', 'Content-Type': 'application/json' };
  const everywhereActive = isSpotifyPlayingOnEverywhere();
  if (everywhereActive) {
    console.log('[Speakers] Everywhere group is playing Spotify – only stopping Nest speaker + media group, preserv...
    await Promise.allSettled([
      fetch(`${haUrl}/api/services/media_player/media_stop`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: NEST_SPEAKER_ENTITY }),
      }),
      fetch(`${haUrl}/api/services/media_player/media_stop`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: CAT_WR_MEDIA_GROUP }),
      }),
      fetch(`${haUrl}/api/services/media_player/media_pause`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: CAT_WR_MEDIA_GROUP }),
      }),
    ]);
  } else {
    await Promise.allSettled([
      fetch(`${haUrl}/api/services/media_player/media_stop`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: NEST_SPEAKER_ENTITY }),
      }),
      fetch(`${haUrl}/api/services/media_player/media_stop`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: CAT_ECHO_ENTITIES }),
      }),
      fetch(`${haUrl}/api/services/media_player/media_stop`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: CAT_WR_MEDIA_GROUP }),
      }),
      fetch(`${haUrl}/api/services/media_player/media_pause`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: CAT_ECHO_ENTITIES }),
      }),
      fetch(`${haUrl}/api/services/media_player/media_pause`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: CAT_WR_MEDIA_GROUP }),
      }),
    ]);
    console.log('[Speakers] Stopped Nest + cat washroom Echos + media group (stop + pause)');
  }
}

async function playChumFmRadio(haUrl: string): Promise<void> {
  if (catWashPlaybackActive) {
    console.log('[Radio] BLOCKED: CPPA playback is active – refusing to play CHUM FM');
    return;
  }

```

```

}
try {
  await haServiceCall('media_player/play_media', {
    entity_id: CAT_WR_MEDIA_GROUP, media_content_type: "custom", media_content_id: "play 104.5 chum fm"
  }, 'CHUM FM');
  console.log(`[Radio] Playing CHUM FM 104.5 on Cat Washroom speaker group`);
} catch (e: any) {
  console.error(`[Radio] Failed to play CHUM FM: ${e.message}`);
}
}

async function startConfirmedPlaybackFlow(
  fileToPlay: any,
  logPrefix: string,
  voice: string = "echo",
  confirmationTTS: string | null = null
): Promise<void> {
  const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
  const appUrl = DEPLOYED_APP_URL;
  const authParam = encodeURIComponent(process.env.SITE_PASSWORD || '');
  const haHeaders = { 'Authorization': 'Bearer ${HOME_ASSISTANT_TOKEN}', 'Content-Type': 'application/json' };

  catWashTrace(`PlaybackFlow`, `STARTING file=${fileToPlay.displayName || fileToPlay.originalName} id=${fileToPlay....
  audioPreparationPaused = true;
  console.log(`${logPrefix} Paused audio preparation for live playback`);

  const fileName = fileToPlay.displayName || fileToPlay.originalName || 'Unknown file';
  const savedChunk = fileToPlay.lastChunkIndex || 0;
  const resumeFromChunk = Math.max(0, savedChunk > 0 ? savedChunk - 1 : 0);
  console.log(`${logPrefix} Will resume from chunk ${resumeFromChunk} (saved: ${savedChunk})`);

  if (savedChunk > 0 && resumeFromChunk < savedChunk) {
    try {
      let currentChecked: number[] = [];
      if (fileToPlay.checkedChunks) {
        try { currentChecked = JSON.parse(fileToPlay.checkedChunks); } catch {}
      }
      const contextIdx = resumeFromChunk;
      if (currentChecked.includes(contextIdx)) {
        currentChecked = currentChecked.filter(c => c !== contextIdx);
        await storage.updateFile(fileToPlay.id, { checkedChunks: JSON.stringify(currentChecked) });
        console.log(`${logPrefix} Removed checkmark from context chunk ${contextIdx} for replay`);
      }
    } catch (e: any) {
      console.log(`${logPrefix} Error removing context chunk checkmark: ${e.message}`);
    }
  }
}

const readerUrl = `${appUrl}/pdf-reader/${fileToPlay.id}?catWashFollow=true&autoplay=false&resumeChunk=${resumeFr...
const tvFollowUrl = `${appUrl}/pdf-reader/${fileToPlay.id}?catWashFollow=true&autoplay=false&resumeChunk=${resume...

catWashSessionId++;
const currentSession = catWashSessionId;
if (catWashPlaybackActive) {
  console.log(`${logPrefix} Stopping previous playback session`);
  if (nestPlaybackAbort) nestPlaybackAbort();
}
catWashPlaybackActive = true;
catWashPlaybackStartedAt = new Date();
startToothbrushPolling();

const lightsNavTimestamp = Date.now();
await setTabletCommand({ action: 'navigate', url: readerUrl, timestamp: lightsNavTimestamp }, true, 'master');
console.log(`${logPrefix} tablet-nav set for master (TV nav deferred until Silk launches)`);

const textExtractionPromise = extractFileText(fileToPlay);

try {
  await fetch(`${haUrl}/api/services/media_player/volume_set`, {
    method: 'POST', headers: haHeaders,
    body: JSON.stringify({ entity_id: CAT_WR_HA_VOICE_ENTITY, volume_level: 0.45 }),
  });
  console.log(`${logPrefix} Set HA Voice volume to 0.45`);
} catch (e: any) { console.warn(`${logPrefix} HA Voice volume set error (non-fatal): ${e.message}`); }

const fileText = await textExtractionPromise;
console.log(`${logPrefix} Text extraction ready (${fileText ? fileText.length : 0} chars)`);
const fileChunks = fileText ? chunkTextForNest(fileText) : [];
const totalChunksCalc = fileChunks.length;

if (fileToPlay.totalChunks !== totalChunksCalc && totalChunksCalc > 0) {
  try { await storage.updateFile(fileToPlay.id, { totalChunks: totalChunksCalc }); } catch {}
}

let preGeneratedChunk0Path: string | null = null;
let prePreparedPaths: string[] = [];
if (fileToPlay.preparedAudioPaths) {
  try { prePreparedPaths = JSON.parse(fileToPlay.preparedAudioPaths); } catch {}
}
const chunk0PreGenPromise = (fileChunks.length > resumeFromChunk)
  ? (prePreparedPaths[resumeFromChunk] && prePreparedPaths[resumeFromChunk].length > 0
    ? Promise.resolve().then(() => { preGeneratedChunk0Path = prePreparedPaths[resumeFromChunk]; console.log(`$...
      : generateAndSaveTTSAudio(fileChunks[resumeFromChunk], `nest-chunk-${fileToPlay.id}-${resumeFromChunk}`-${Da...
        .then(p => { preGeneratedChunk0Path = p; console.log(`${logPrefix} Pre-generated chunk ${resumeFromChun...
          .catch(e => { console.warn(`${logPrefix} Chunk 0 pre-gen failed (will retry): ${e.message}`); })
        )
      : Promise.resolve();

const initialChunkText = fileChunks[resumeFromChunk] || '';
const initialCleanedText = cleanTextForTTS(initialChunkText);
const initialWords = initialCleanedText.split(/\s+/).filter((w: string) => w.length > 0);
const initialWordCount = initialWords.length;
const initialEstimatedMs = Math.max(5000, (initialWordCount / 175) * 60 * 1000 + 1000);

catWashPlaybackState = {
  fileId: fileToPlay.id,
  fileName,
  chunkIndex: resumeFromChunk,

```

```

totalChunks: totalChunksCalc,
chunks: fileChunks,
currentWords: initialWords,
wordIndex: 0,
startedAt: new Date(),
chunkStartedAt: new Date(),
estimatedChunkDuration: initialEstimatedMs,
playbackMode: 'server-tts',
};

await savePlaybackSession({
  fileId: fileToPlay.id,
  fileName,
  chunkIndex: resumeFromChunk,
  totalChunks: totalChunksCalc,
  trigger: catWashPlaybackTrigger || 'manual',
  startedAt: new Date().toISOString(),
  updatedAt: new Date().toISOString(),
  status: 'active',
});
console.log(`${logPrefix} Persisted playback session to DB (file=${fileToPlay.id}, chunk=${resumeFromChunk}/${tot...

const tabletSetupPromise = (async () => {
  try {
    console.log(`${logPrefix} ===== TABLET SETUP START (tablet-nav polling) =====`);
    await setTabletCommand({ action: 'navigate', url: readerUrl, timestamp: Date.now(), true, 'master'});
    console.log(`${logPrefix} tablet-nav set for master - tablet will navigate via polling`);
    console.log(`${logPrefix} ===== TABLET SETUP COMPLETE =====`);
  } catch (e: any) {
    console.error(`${logPrefix} Tablet setup error: ${e.message}`);
  }
})();

const tvSetupPromise = (async () => {
  try {
    console.log(`${logPrefix} ===== TV SETUP START =====`);
    catWashTrace('TV', 'SETUP START');
    currentTvFollowUrl = tvFollowUrl;

    const haUrl = HOME_ASSISTANT_URL?.replace(/\/$/, '') || '';
    const haHdrs = { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' };

    let fireStickState = 'unknown';
    try {
      const fsResp = await fetch(`${haUrl}/api/states/${FIRE_STICK_ADB_ENTITY}`, { headers: haHdrs });
      if (fsResp.ok) {
        const fsData = await fsResp.json();
        fireStickState = fsData?.state || 'unknown';
      }
    } catch (e: any) {
      console.warn(`${logPrefix} Could not query Fire Stick state: ${e.message}`);
    }
    console.log(`${logPrefix} Fire Stick current state: ${fireStickState}`);
    catWashTrace('TV', `Fire Stick state: ${fireStickState}`);

    const fireStickIsOff = fireStickState === 'off' || fireStickState === 'standby' || fireStickState === 'unavai...

    try {
      await haServiceCallSafe('media_player/turn_on', { entity_id: CAT_TV_ENTITY }, 'Samsung TV TurnOn (WoL)');
      console.log(`${logPrefix} Samsung TV turn_on sent (WoL)`);
    } catch (e: any) {
      console.warn(`${logPrefix} Samsung TV turn_on failed (non-fatal): ${e.message}`);
    }

    if (fireStickIsOff) {
      catWashTrace('TV', 'Fire Stick is OFF/standby - turning ON directly (CEC will wake Samsung TV). No OFF cycl...
      console.log(`${logPrefix} Fire Stick is off/standby - just turning ON (preserves Silk if backgrounded)`);
      for (let fsOnAttempt = 1; fsOnAttempt <= 3; fsOnAttempt++) {
        try {
          await haServiceCall('media_player/turn_on', { entity_id: FIRE_STICK_ADB_ENTITY }, 'FireStick TurnOn att...
          console.log(`${logPrefix} Fire Stick turn_on: OK (attempt ${fsOnAttempt}) - CEC should turn on Samsung ...
          catWashTrace('TV', `Fire Stick turn_on OK on attempt ${fsOnAttempt}`);
          break;
        } catch (e: any) {
          catWashTrace('TV', `Fire Stick turn_on attempt ${fsOnAttempt}/3 FAILED`, { error: e.message });
          console.error(`${logPrefix} Fire Stick turn_on attempt ${fsOnAttempt}/3 failed: ${e.message}`);
          if (fsOnAttempt < 3) await new Promise(r => setTimeout(r, 3000));
        }
      }
    } else {
      catWashTrace('TV', 'Fire Stick is already ON - doing OFF!ON cycle for CEC wake');
      console.log(`${logPrefix} Fire Stick is already on ${fireStickState} - OFF!ON cycle for CEC`);
      try {
        await haServiceCall('media_player/turn_off', { entity_id: FIRE_STICK_ADB_ENTITY }, 'FireStick TurnOff for...
        console.log(`${logPrefix} Fire Stick turned OFF for CEC reset`);
      } catch (e: any) {
        console.warn(`${logPrefix} Fire Stick turn_off failed (non-fatal): ${e.message}`);
      }
      await new Promise(r => setTimeout(r, 5000));

      for (let fsOnAttempt = 1; fsOnAttempt <= 3; fsOnAttempt++) {
        try {
          await haServiceCall('media_player/turn_on', { entity_id: FIRE_STICK_ADB_ENTITY }, 'FireStick TurnOn att...
          console.log(`${logPrefix} Fire Stick turn_on: OK (attempt ${fsOnAttempt}) - CEC should turn on Samsung ...
          catWashTrace('TV', `Fire Stick turn_on OK on attempt ${fsOnAttempt}`);
          break;
        } catch (e: any) {
          catWashTrace('TV', `Fire Stick turn_on attempt ${fsOnAttempt}/3 FAILED`, { error: e.message });
          console.error(`${logPrefix} Fire Stick turn_on attempt ${fsOnAttempt}/3 failed: ${e.message}`);
          if (fsOnAttempt < 3) await new Promise(r => setTimeout(r, 3000));
        }
      }
    }

    try {
      await haServiceCall('androidtv/adb_command', { entity_id: FIRE_STICK_ADB_ENTITY, command: 'input keyevent K...
      console.log(`${logPrefix} Fire Stick WAKEUP sent`);
    } catch (e: any) {

```

```

    console.warn(`${logPrefix} Fire Stick WAKEUP failed (non-fatal): ${e.message}`);
}

catWashTrace('TV', 'Waiting 8s for Fire Stick + Samsung TV to wake');
console.log(`${logPrefix} Waiting 8s for TV + Fire Stick boot...`);
await new Promise(resolve => setTimeout(resolve, 8000));

for (let tvCheck = 1; tvCheck <= 3; tvCheck++) {
  try {
    const tvStateResp = await fetch(`${HOME_ASSISTANT_URL.replace(/\$/ /, '')}/api/states/${CAT_TV_ENTITY}`, {...
    if (tvStateResp.ok) {
      const tvStateData = await tvStateResp.json();
      const tvState = tvStateData?.state || 'unknown';
      console.log(`${logPrefix} Samsung TV state check ${tvCheck}/3: ${tvState}`);
      catWashTrace('TV', `Samsung TV state check ${tvCheck}: ${tvState}`);
      if (tvState === 'on' || tvState === 'playing' || tvState === 'idle' || tvState === 'paused') {
        console.log(`${logPrefix} Samsung TV is ON - proceeding`);
        break;
      }
      if (tvCheck < 3) {
        console.log(`${logPrefix} Samsung TV still off - sending another WoL + CEC kick`);
        await haServiceCallSafe('media_player/turn_on', { entity_id: CAT_TV_ENTITY }, `Samsung TV WoL retry $...
        await haServiceCall('androidtv/adb_command', { entity_id: FIRE_STICK_ADB_ENTITY, command: 'input keye...
        await new Promise(r => setTimeout(r, 5000));
      }
    }
  } catch (e: any) {
    console.warn(`${logPrefix} Samsung TV state check ${tvCheck} failed: ${e.message}`);
  }
}

const launchSilkOnTV = async (silkAttempt: number) => {
  try {
    await haServiceCall('androidtv/adb_command', { entity_id: FIRE_STICK_ADB_ENTITY, command: 'monkey -p com....
    console.log(`${logPrefix} Silk launched via monkey command (attempt ${silkAttempt})`);
    return true;
  } catch (e: any) {
    console.warn(`${logPrefix} Silk monkey attempt ${silkAttempt} failed: ${e.message}`);
    return false;
  }
};

console.log(`${logPrefix} Waiting 5s before Silk launch to let Fire Stick UI fully boot...`);
await new Promise(r => setTimeout(r, 5000));

try {
  await haServiceCall('androidtv/adb_command', { entity_id: FIRE_STICK_ADB_ENTITY, command: 'input keyevent K...
  console.log(`${logPrefix} Extra WAKEUP sent before Silk launch`);
} catch (e: any) {}

await new Promise(r => setTimeout(r, 2000));

try {
  await haServiceCall('androidtv/adb_command', { entity_id: FIRE_STICK_ADB_ENTITY, command: 'am force-stop co...
  console.log(`${logPrefix} Force-stopped Silk before fresh launch`);
  await new Promise(r => setTimeout(r, 2000));
} catch (e: any) {
  console.warn(`${logPrefix} Force-stop Silk failed (non-fatal): ${e.message}`);
}

let silkLaunched = false;
for (let silkAttempt = 1; silkAttempt <= 3; silkAttempt++) {
  try {
    const silkCmd = tvFollowUrl
      ? 'am start --activity-clear-task -a android.intent.action.VIEW -d "${tvFollowUrl}" com.amazon.cloud9'
      : 'monkey -p com.amazon.cloud9 -c android.intent.category.LAUNCHER 1';
    await haServiceCall('androidtv/adb_command', { entity_id: FIRE_STICK_ADB_ENTITY, command: silkCmd }, `Fir...
    console.log(`${logPrefix} Silk launched with URL (attempt ${silkAttempt})`);
    silkLaunched = true;
    break;
  } catch (e: any) {
    console.warn(`${logPrefix} Silk launch attempt ${silkAttempt}/3 failed: ${e.message}`);
    if (silkAttempt < 3) {
      const waitMs = silkAttempt === 1 ? 5000 : 8000;
      if (silkAttempt === 2) {
        try {
          await haServiceCall('androidtv/adb_command', { entity_id: FIRE_STICK_ADB_ENTITY, command: 'input ke...
          await new Promise(r => setTimeout(r, 1000));
          await haServiceCall('androidtv/adb_command', { entity_id: FIRE_STICK_ADB_ENTITY, command: 'input ke...
        } catch (e2: any) {}
      }
      await new Promise(r => setTimeout(r, waitMs));
    }
  }
}

if (silkLaunched) {
  await new Promise(r => setTimeout(r, 3000));
  console.log(`${logPrefix} Setting TV tablet-nav command now that Silk is launched`);
  await setTabletCommand({ action: 'navigate', url: tvFollowUrl, timestamp: Date.now() }, true, 'tv');
}

try {
  await haServiceCall('androidtv/adb_command', { entity_id: FIRE_STICK_ADB_ENTITY, command: 'settings put glo...
  console.log(`${logPrefix} Fire Stick immersive mode set`);
} catch (e: any) {
  console.log(`${logPrefix} Fire Stick immersive failed: ${e.message}`);
}

catWashTrace('TV', `SETUP COMPLETE`);
console.log(`${logPrefix} ===== TV SETUP COMPLETE =====`);
} catch (e: any) {
  catWashTrace('TV', `SETUP ERROR`, { error: e.message });
  console.error(`${logPrefix} ===== TV SETUP ERROR: ${e.message} =====`);
}
})();

const confirmTTPromise = confirmationTTS ? (async () => {

```



```

try {
  try {
    await haServiceCallSafe('media_player/volume_set', { entity_id: NEST_SPEAKER_ENTITY, volume_level: 0.75 }, ...
  } catch (e: any) { console.warn(`${logPrefix} Pre-confirm volume set error (non-fatal): ${e.message}`); }
  let confirmPlayed = false;
  try {
    const confirmPath = await generateAndSaveTTSAudio(confirmationTTS, `confirm-${Date.now()}`);
    const nestResult = await playOnNestSpeaker(`${appUrl}${confirmPath}`);
    if (nestResult.success) {
      confirmPlayed = true;
      console.log(`${logPrefix} Confirm TTS played on Nest speaker via OpenAI/Edge TTS (actuallyPlaying=${nestR...
    } else {
      console.warn(`${logPrefix} Nest speaker confirm failed – falling back to HA Voice`);
    }
  } catch (e: any) {
    console.warn(`${logPrefix} Nest speaker confirm failed: ${e.message} – falling back to HA Voice`);
  }
  if (!confirmPlayed) {
    try {
      await haServiceCall('tts/speak', {
        entity_id: HA_CLOUD_TTS_ENTITY,
        media_player_entity_id: CAT_WR_HA_VOICE_ENTITY,
        message: confirmationTTS
      }, 'Confirm HA Cloud TTS');
      console.log(`${logPrefix} Confirm TTS played via HA Cloud TTS on HA Voice speaker (fallback)`);
    } catch (e: any) {
      console.warn(`${logPrefix} HA Voice confirm also failed: ${e.message}`);
    }
  }
  const confirmWordCount = confirmationTTS.split(/\s+/).length;
  const confirmWaitMs = Math.max(4000, (confirmWordCount / 140) * 60 * 1000 + 1500);
  console.log(`${logPrefix} Confirm TTS playing, waiting ${Math.round(confirmWaitMs / 1000)}s`);
  await new Promise(r => setTimeout(r, confirmWaitMs));
  console.log(`${logPrefix} Confirm TTS finished`);
} catch (e: any) {
  console.error(`${logPrefix} Confirm TTS error: ${e.message}`);
  await new Promise(r => setTimeout(r, 5000));
}
})(); // Promise.resolve();

await Promise.allSettled([tabletSetupPromise, tvSetupPromise, chunk0PreGenPromise, confirmTTSPromise]);

try {
  await haServiceCallSafe('media_player/volume_set', { entity_id: NEST_SPEAKER_ENTITY, volume_level: 0.75 }, 'Nes...
  console.log(`${logPrefix} Nest volume set to 0.75 for playback`);
} catch (e: any) { console.warn(`${logPrefix} Nest playback volume set error (non-fatal): ${e.message}`); }

currentTabletReaderUrl = readerUrl;
startNestChunkPlayback(fileToPlay.id, fileName, fileChunks, resumeFromChunk, currentSession, voice, preGeneratedC...
}

function describeFileForTTS(file: any, weekNumber: number): string {
  const folder = (file.folder || '').toLowerCase();
  const origName = (file.originalName || '').toLowerCase();
  const dispName = (file.displayName || '').toLowerCase();
  const combinedName = `${origName} ${dispName}`;
  const codeMatch = folder.match(/([a-z]{3,5}\s?\d{3})/i) || combinedName.match(/([a-z]{3,5}\s?\d{3})/i);
  const courseCode = codeMatch ? codeMatch[1].toUpperCase().replace(/\s/g, '') : '';
  const shortCode = courseCode.length >= 4 ? courseCode.substring(0, 4) : courseCode;
  const isModule = folder.includes('module') || origName.includes('module') || dispName.includes('module');
  const fileType = isModule ? 'Module' : 'Reading';
  if (shortCode) {
    return `your ${shortCode} ${fileType} for week ${weekNumber}`;
  }
  return `your ${fileType} for week ${weekNumber}`;
}

function orderFilesByCoursePriority(files: any[]): any[] {
  const isModule = (f: any) =>
    f.folder?.toLowerCase().includes('module') ||
    f.originalName?.toLowerCase().includes('module');

  const getCourseCode = (f: any): string => {
    const folder = (f.folder || '').toLowerCase();
    const name = (f.originalName || '').toLowerCase();
    const match = folder.match(/([a-z]{3,5}\s?\d{3})/i) || name.match(/([a-z]{3,5}\s?\d{3})/i);
    return match ? match[1].toUpperCase().replace(/\s/g, '') : 'UNKNOWN';
  };

  const hasAnyUnlistenedModule = files.some(f => isModule(f) && !f.listened);

  const eligible = files.filter(f => {
    if (isModule(f)) return true;
    return !hasAnyUnlistenedModule;
  });

  const withPriority = eligible.map(f => ({
    file: f,
    coursePriority: getCoursePriorityForFile(f),
    isModule: isModule(f) ? 0 : 1,
  }));

  withPriority.sort((a, b) => {
    if (a.coursePriority !== b.coursePriority) return a.coursePriority - b.coursePriority;
    return a.isModule - b.isModule;
  });

  console.log(`[FileOrder] ${files.length} files, modulesBlocked=${hasAnyUnlistenedModule}, ${eligible.length} elig...

  return withPriority.map(w => w.file);
}

async function isNestSpeakerPlaying(): Promise<boolean> {
  try {
    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
    const resp = await fetch(`${haUrl}/api/states/${NEST_SPEAKER_ENTITY}`, {
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}` },
    });
  }
}

```

```

    if (resp.ok) {
      const data = await resp.json();
      return (data.state || '').toLowerCase() === 'playing';
    }
  } catch {}
  return false;
}

const startToothbrushPolling = () => {
  if (toothbrushPollInterval) clearInterval(toothbrushPollInterval);
  console.log('[Toothbrush] Starting polling for sensor.toothbrush_bryn_toothbrush_state');
  let tbPollCount = 0;
  toothbrushPollInterval = setInterval(async () => {
    tbPollCount++;
    const nestPlaying = await isNestSpeakerPlaying();
    if (!catWashPlaybackActive && !nestPlaying) {
      console.log('[Toothbrush] Playback no longer active and Nest not playing, stopping poll (after ${tbPollCount}...
      if (toothbrushPollInterval) { clearInterval(toothbrushPollInterval); toothbrushPollInterval = null; }
      return;
    }
  }, 3000);

  try {
    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
    const resp = await fetch(`${haUrl}/api/states/sensor.toothbrush_bryn_toothbrush_state`, {
      headers: { 'Authorization': 'Bearer ${HOME_ASSISTANT_TOKEN}' },
    });
    if (resp.ok) {
      const data = await resp.json();
      const state = (data.state || '').toLowerCase();
      if (tbPollCount <= 3 || tbPollCount % 10 === 0) {
        console.log('[Toothbrush] Poll #${tbPollCount}: state="${data.state}" (playbackActive=${catWashPlaybackAc...
      }
      if (state === 'running' || state === 'brushing') {
        console.log('[Toothbrush] State changed to "${data.state}" - stopping cat wash playback (poll #${tbPollCo...
        if (toothbrushPollInterval) { clearInterval(toothbrushPollInterval); toothbrushPollInterval = null; }

        if (catWashPlaybackActive) {
          try {
            const stopUrl = `http://localhost:${process.env.PORT || 5000}/api/webhook/cat-wash-stop`;
            await fetch(stopUrl, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.st...
          } catch (e: any) {
            console.log('[Toothbrush] Error calling stop endpoint: ${e.message}');
          }
        } else if (nestPlaying) {
          console.log('[Toothbrush] Nest still playing (orphaned session after deploy) - stopping Nest directly');
          try {
            await stopNestSpeaker();
            const goodbyeText = "Stopping. Your reading position has been saved. See you next time Bryn.";
            const appUrl = DEPLOYED_APP_URL;
            const goodbyePath = await generateAndSaveTTSAudio(goodbyeText, 'nest-goodbye-tb-${Date.now()}', "echo");
            await new Promise(r => setTimeout(r, 500));
            await playOnNestSpeaker(`${appUrl}${goodbyePath}`);
          } catch (e: any) {
            console.log('[Toothbrush] Error stopping orphaned Nest playback: ${e.message}');
          }
        }
      }
    } else {
      console.log('[Toothbrush] HA returned ${resp.status} for toothbrush state');
    }
  } catch (e: any) {
    console.log('[Toothbrush] Poll error: ${e.message}');
  }
}, 3000);

const stopToothbrushPolling = () => {
  if (toothbrushPollInterval) {
    clearInterval(toothbrushPollInterval);
    toothbrushPollInterval = null;
    console.log('[Toothbrush] Polling stopped');
  }
};

let catWashPlaybackState: {
  fileId: number;
  fileName: string;
  chunkIndex: number;
  totalChunks: number;
  chunks: string[];
  currentWords: string[];
  wordIndex: number;
  startedAt: Date;
  chunkStartedAt: Date;
  estimatedChunkDuration: number;
  playbackMode?: 'tablet-bluetooth' | 'server-tts';
} | null = null;

let nestPlaybackAbort: (() => void) | null = null;
let wordAdvanceInterval: ReturnType<typeof setInterval> | null = null;

function startWordAdvancement() {
  stopWordAdvancement();
  wordAdvanceInterval = setInterval(() => {
    if (!catWashPlaybackState || !catWashPlaybackActive) {
      stopWordAdvancement();
      return;
    }
    const { currentWords, wordIndex, chunkStartedAt, estimatedChunkDuration } = catWashPlaybackState;
    if (currentWords.length === 0) return;
    const elapsed = Date.now() - chunkStartedAt.getTime();
    const progress = Math.min(elapsed / estimatedChunkDuration, 1);
    const newIndex = Math.min(Math.floor(progress * currentWords.length), currentWords.length - 1);
    if (newIndex !== wordIndex) {
      catWashPlaybackState.wordIndex = newIndex;
    }
  }, 200);
}

function stopWordAdvancement() {

```

```

    if (wordAdvanceInterval) {
      clearInterval(wordAdvanceInterval);
      wordAdvanceInterval = null;
    }
  }
}

async function uploadAudioToHA(audioPath: string): Promise<string | null> {
  const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
  try {
    const localPath = audioPath.startsWith('/') ? audioPath : `/${audioPath}`;
    const fullLocalUrl = `http://localhost:${process.env.PORT || 5000}${localPath}`;
    const audioResp = await fetch(fullLocalUrl);
    if (!audioResp.ok) {
      console.error(`[HA Upload] Failed to fetch local audio: ${audioResp.status}`);
      return null;
    }
    const audioBuffer = Buffer.from(await audioResp.arrayBuffer());
    const fileName = `tts_${Date.now()}.mp3`;

    const boundary = `---HAUpload${Date.now()}`;
    const bodyParts = [
      `--${boundary}\r\nContent-Disposition: form-data; name="media_content_id"\r\n\r\nmedia-source://media_source/...`,
      `--${boundary}\r\nContent-Disposition: form-data; name="file"; filename="${fileName}"\r\n\r\nContent-Type: audio/...`
    ];
    const bodyEnd = `\r\n--${boundary}--\r\n`;

    const bodyBuffer = Buffer.concat([
      Buffer.from(bodyParts[0]),
      Buffer.from(bodyParts[1]),
      audioBuffer,
      Buffer.from(bodyEnd),
    ]);

    const uploadResp = await fetch(`${haUrl}/api/media_source/local_source/upload`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
        'Content-Type': `multipart/form-data; boundary=${boundary}`,
      },
      body: bodyBuffer,
    });

    if (!uploadResp.ok) {
      const errText = await uploadResp.text();
      console.error(`[HA Upload] Upload failed: ${uploadResp.status} ${errText}`);
      return null;
    }
    const result = await uploadResp.json() as any;
    const mediaId = result.media_content_id || `media-source://media_source/local/./${fileName}`;
    console.log(`[HA Upload] Uploaded ${fileName} (${Math.round(audioBuffer.length / 1024)}KB) !`);
    return mediaId;
  } catch (e: any) {
    console.error(`[HA Upload] Error: ${e.message}`);
    return null;
  }
}

async function playOnNestSpeaker(audioUrl: string, maxRetries: number = 2): Promise<{ success: boolean; actuallyPla...
  const fullUrl = audioUrl.startsWith('http') ? audioUrl : `${DEPLOYED_APP_URL}${audioUrl}`;
  console.log(`[Nest] Playing audio: ${fullUrl}`);
  try {
    await haServiceCall('media_player/play_media', {
      entity_id: NEST_SPEAKER_ENTITY, media_content_id: fullUrl, media_content_type: "music"
    }, 'Nest Play Direct');
  } catch (e: any) {
    console.error(`[Nest] play_media failed: ${e.message}`);
    return { success: false, actuallyPlaying: false };
  }
}

for (let check = 0; check <= maxRetries; check++) {
  await new Promise(r => setTimeout(r, check === 0 ? 4000 : 3000));
  try {
    const { state: speakerState } = await getNestMediaState();
    console.log(`[Nest] Speaker state check ${check + 1}: ${speakerState}`);
    if (speakerState === 'playing' || speakerState === 'buffering') {
      return { success: true, actuallyPlaying: true };
    }
    if (speakerState === 'unknown') {
      console.log(`[Nest] State is "unknown" - assuming play_media succeeded`);
      return { success: true, actuallyPlaying: false };
    }
    if (check === maxRetries) {
      console.log(`[Nest] State "${speakerState}" after ${maxRetries + 1} checks - play_media was sent, assuming ...`);
      return { success: true, actuallyPlaying: false };
    }
    console.log(`[Nest] State "${speakerState}" - rechecking...`);
  } catch (e: any) {
    console.warn(`[Nest] State check error: ${e.message} - assuming play_media succeeded`);
    return { success: true, actuallyPlaying: false };
  }
}
return { success: true, actuallyPlaying: false };
}

async function playChunkViaHACloudTTS(chunkText: string, sessionId: number, speakerEntity: string = CAT_WR_HA_VOICE...
  const MAX_TTS_CHARS = 3500;
  const textToSpeak = chunkText.length > MAX_TTS_CHARS ? chunkText.substring(0, MAX_TTS_CHARS) : chunkText;
  try {
    await haServiceCall('tts/speak', {
      entity_id: HA_CLOUD_TTS_ENTITY,
      media_player_entity_id: speakerEntity,
      message: textToSpeak
    }, 'Chunk HA Cloud TTS');
    console.log(`[HA Cloud TTS] Playing chunk (${textToSpeak.length} chars) on ${speakerEntity}`);
    const wordCount = textToSpeak.split(/\s+/).length;
    const estimatedMs = Math.max(5000, (wordCount / 155) * 60 * 1000 + 2000);
    console.log(`[HA Cloud TTS] Waiting ~${Math.round(estimatedMs / 1000)}s for chunk to finish`);
    const ABORT_CHECK_MS = 2000;
    for (let waited = 0; waited < estimatedMs; waited += ABORT_CHECK_MS) {

```

```

    if (!catWashPlaybackActive || catWashSessionId !== sessionId) {
      console.log('[HA Cloud TTS] Abort detected during wait');
      await haServiceCallSafe('media_player/media_stop', { entity_id: speakerEntity }, 'HA Voice Abort Stop');
      return false;
    }
    await new Promise(r => setTimeout(r, ABORT_CHECK_MS));
  }
  return true;
} catch (e: any) {
  console.error('[HA Cloud TTS] Failed: ${e.message}');
  return false;
}
}

async function playChunkViaEchoTTS(chunkText: string, sessionId: number): Promise<boolean> {
  const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
  const MAX_ALEXA_CHARS = 3000;
  const textToSpeak = chunkText.length > MAX_ALEXA_CHARS ? chunkText.substring(0, MAX_ALEXA_CHARS) : chunkText;
  const ssmContent = `<speak><prosody rate="90%">${textToSpeak.replace(/&/g, '&amp;').replace(/</g, '&lt;')}.replac...
  try {
    const resp = await fetch(`${haUrl}/api/services/notify/alexa_media`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({
        message: ssmContent,
        target: [
          "media_player.echo_cat_left_am",
          "media_player.echo_cat_right_am",
          "media_player.echo_cat_washroom_middle",
        ],
        data: { type: "tts" },
      })),
    );
    if (!resp.ok) {
      console.error('[Echo TTS] alexa_media failed: ${resp.status}');
      return false;
    }
    console.log('[Echo TTS] Playing chunk (${textToSpeak.length} chars) on Echo washroom speakers');
    const wordCount = textToSpeak.split(/s+/).length;
    const estimatedMs = Math.max(5000, (wordCount / 155) * 60 * 1000 + 2000);
    console.log('[Echo TTS] waiting ~${Math.round(estimatedMs / 1000)}s for chunk to finish');
    const ABORT_CHECK_MS = 2000;
    for (let waited = 0; waited < estimatedMs; waited += ABORT_CHECK_MS) {
      if (!catWashPlaybackActive || catWashSessionId !== sessionId) {
        console.log('[Echo TTS] Abort detected during wait');
        return false;
      }
      await new Promise(r => setTimeout(r, ABORT_CHECK_MS));
    }
    return true;
  } catch (e: any) {
    console.error('[Echo TTS] Failed: ${e.message}');
    return false;
  }
}

async function promptFallbackSwitch(sessionId: number, fallbackName: string, promptMessage: string): Promise<boolean> {
  console.log(`${fallbackName} Fallback] Prompting user`);
  try {
    await haServiceCallSafe('media_player/volume_set', { entity_id: CAT_WR_HA_VOICE_ENTITY, volume_level: 0.35 }, '...
    await haServiceCallSafe('input_boolean/turn_off', { entity_id: MODULE_READING_CONFIRMED }, 'Fallback Bool Reset');
    await haServiceCall('tts/speak', {
      entity_id: HA_CLOUD_TTS_ENTITY,
      media_player_entity_id: CAT_WR_HA_VOICE_ENTITY,
      message: promptMessage
    }, 'Fallback Prompt TTS');
    console.log(`${fallbackName} Fallback] Prompt sent, waiting for confirmation (up to 30s)`);
    const POLL_MS = 1500;
    const MAX_WAIT_MS = 30000;
    for (let waited = 0; waited < MAX_WAIT_MS; waited += POLL_MS) {
      if (!catWashPlaybackActive || catWashSessionId !== sessionId) {
        console.log(`${fallbackName} Fallback] Session aborted during wait`);
        return false;
      }
      try {
        const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
        const resp = await haFetch(`${haUrl}/api/states/${MODULE_READING_CONFIRMED}`, {
          headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}` },
        }, 2, 'Fallback Confirm Check');
        const data = await resp.json();
        if (data.state === 'on') {
          console.log(`${fallbackName} Fallback] User confirmed – switching to ${fallbackName} playback`);
          return true;
        }
      } catch {}
      await new Promise(r => setTimeout(r, POLL_MS));
    }
    console.log(`${fallbackName} Fallback] No confirmation received within ${MAX_WAIT_MS / 1000}s`);
    return false;
  } catch (e: any) {
    console.error(`${fallbackName} Fallback] Error: ${e.message}`);
    return false;
  }
}

async function stopNestSpeaker(): Promise<void> {
  try {
    await haServiceCall('media_player/media_stop', { entity_id: NEST_SPEAKER_ENTITY }, 'Nest Stop');
    console.log('[Nest] Stopped playback');
  } catch (e: any) {
    console.error('[Nest] Error stopping: ${e.message}');
  }
}

async function getNestMediaState(): Promise<{ state: string; position?: number; duration?: number }> {
  const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
  try {
    const resp = await haFetch(`${haUrl}/api/states/${NEST_SPEAKER_ENTITY}`, {

```

```

    headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}` },
  }, 2, 'Nest State');
const data = await resp.json();
const result = {
  state: data.state,
  position: data.attributes?.media_position,
  duration: data.attributes?.media_duration,
};
console.log(`[Nest State] entity=${NEST_SPEAKER_ENTITY} state=${data.state} media_title=${data.attributes?.medi...
return result;
} catch (e: any) {
  console.warn(`[Nest State] Failed to query: ${e.message}`);
}
return { state: 'unknown' };
}

async function waitForNestPlaybackEnd(estimatedMs: number, sessionId: number): Promise<boolean> {
  const startTime = Date.now();
  const ABORT_CHECK_MS = 2000;
  const timerWaitMs = estimatedMs + 1000;

  for (let waited = 0; waited < timerWaitMs; waited += ABORT_CHECK_MS) {
    if (!catWashPlaybackActive || catWashSessionId !== sessionId) {
      console.log(`[Nest] Abort detected during timer wait (${Math.round(waited / 1000)}s in)`);
      return false;
    }
    await new Promise(r => setTimeout(r, ABORT_CHECK_MS));
  }

  if (!haHealth.connected) {
    console.log(`[Nest] Timer done (${Math.round(estimatedMs / 1000)}s). HA offline – trusting timer, moving to nex...
    return true;
  }

  const HA_CHECK_INTERVAL_MS = 2000;
  const maxExtraWait = 15000;

  for (let extra = 0; extra < maxExtraWait; extra += HA_CHECK_INTERVAL_MS) {
    if (!catWashPlaybackActive || catWashSessionId !== sessionId) return false;
    try {
      const state = await getNestMediaState();
      if (state.state === 'idle' || state.state === 'off' || state.state === 'paused') {
        console.log(`[Nest] Chunk confirmed done via HA (${Math.round((Date.now() - startTime) / 1000)}s total)`);
        return true;
      }
    } catch {
      console.log(`[Nest] HA state check failed – trusting timer`);
      return true;
    }
    await new Promise(r => setTimeout(r, HA_CHECK_INTERVAL_MS));
  }
  console.log(`[Nest] Timer + HA check window elapsed (${Math.round((Date.now() - startTime) / 1000)}s) – moving to...
  return true;
}

async function startNestChunkPlayback(
  fileId: number,
  fileName: string,
  chunks: string[],
  startChunk: number,
  sessionId: number,
  voice: string = "echo",
  preGeneratedFirstChunkPath: string | null = null
) {
  const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
  const appUrl = DEPLOYED_APP_URL;
  let aborted = false;
  nestPlaybackAbort = () => { aborted = true; };

  console.log(`[Nest Playback] Starting session ${sessionId}: "${fileName}" from chunk ${startChunk}/${chunks.length...

  try {
    if (chunks.length === 0) {
      console.log(`[Nest Playback] No chunks for "${fileName}" (id=${fileId}) – text extraction failed, marking lis...
      await clearPlaybackSession();
      try {
        await storage.updateFile(fileId, { listened: true });
        console.log(`[Nest Playback] Marked file ${fileId} as listened (extraction failed)`);
      } catch (e: any) {
        console.error(`[Nest Playback] Failed to mark file ${fileId} listened: ${e.message}`);
      }
    }
    try {
      const allFiles = await storage.GetFiles();
      const semesterSettings = await storage.getActiveSemesterSettings();
      const semStart = semesterSettings?.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : new Date...
      const rwStart = semesterSettings?.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : new Date...
      const currentWeekNumber = getWeekNumber(torontoDate(), semStart, rwStart);

      const nextFile = findNextFileByPriority(allFiles, currentWeekNumber, fileId);
      if (nextFile) {
        console.log(`[Nest Playback] Skipping to next file: ${nextFile.displayName || nextFile.originalName} (id=...
        const nextText = await extractFileText(nextFile);
        if (nextText) {
          const nextChunks = chunkTextForNest(nextText);
          if (nextChunks.length > 0) {
            catWashPlaybackState = {
              fileId: nextFile.id,
              fileName: nextFile.displayName || nextFile.originalName,
              chunkIndex: 0,
              totalChunks: nextChunks.length,
              chunks: nextChunks,
              currentWords: [],
              wordIndex: 0,
              startedAt: new Date(),
              chunkStartedAt: new Date(),
            };
            const nextName = nextFile.displayName || nextFile.originalName;
            await savePlaybackSession({

```

```

        fileId: nextFile.id,
        fileName: nextName,
        chunkIndex: 0,
        totalChunks: nextChunks.length,
        trigger: catWashPlaybackTrigger || 'manual',
        startedAt: catWashPlaybackStartedAt?.toISOString() || new Date().toISOString(),
        updatedAt: new Date().toISOString(),
        status: 'active',
    });
    startNestChunkPlayback(nextFile.id, nextName, nextChunks, 0, sessionId, voice);
    return;
}
}
console.log(`[Nest Playback] Next file also has no extractable text, stopping`);
} else {
    console.log(`[Nest Playback] No more unlisted files for weeks 1-${currentWeekNumber}`);
}
} catch (e: any) {
    console.error(`[Nest Playback] Error finding next file: ${e.message}`);
}
catWashPlaybackActive = false;
catWashPlaybackStartedAt = null;
catWashPlaybackState = null;
return;
}

console.log(`[Nest Playback] Starting first chunk immediately`);

let preparedAudioCache: string[] = [];
if (fileId) {
    try {
        const fileForCache = await storage.getFile(fileId);
        if (fileForCache?.preparedAudioPaths) {
            preparedAudioCache = JSON.parse(fileForCache.preparedAudioPaths);
            console.log(`[Nest Playback] Loaded ${preparedAudioCache.filter(p => p && p.length > 0).length} pre-prepa...`);
        }
    } catch {}
}

let checkedChunksSet = new Set<number>();
if (fileId) {
    try {
        const fileForChecked = await storage.getFile(fileId);
        if (fileForChecked?.checkedChunks) {
            const parsed = JSON.parse(fileForChecked.checkedChunks);
            if (Array.isArray(parsed)) checkedChunksSet = new Set(parsed);
        }
    } catch {}
    if (checkedChunksSet.size > 0) {
        console.log(`[Nest Playback] ${checkedChunksSet.size} chunks already checked – will skip them`);
    }
}

let chunksPlayedSinceLastPrompt = 0;
const ATTENTION_INTERVAL = 3;
let lookaheadAudioPath: string | null = null;
let lookaheadChunkIndex: number = -1;
let lookaheadPromise: Promise<string | null> | null = null;
let consecutivePlayFailures = 0;
const MAX_PLAY_FAILURES = 3;

for (let i = startChunk; i < chunks.length; i++) {
    if (aborted || !catWashPlaybackActive || catWashSessionId !== sessionId) {
        console.log(`[Nest Playback] Aborted at chunk ${i}`);
        if (fileId) {
            try { await storage.updateFile(fileId, { lastChunkIndex: i }); } catch {}
            console.log(`[Nest Playback] Saved progress on abort: chunk ${i}`);
        }
        break;
    }

    if (checkedChunksSet.has(i)) {
        console.log(`[Nest Playback] Skipping chunk ${i + 1}/${chunks.length} (already checked/strikethrough)`);
        if (fileId) {
            try { await storage.updateFile(fileId, { lastChunkIndex: i + 1 }); } catch {}
        }
        continue;
    }

    if (catWashPlaybackState) {
        catWashPlaybackState.chunkIndex = i;
        const chunkText = chunks[i] || '';
        const cleanedChunkText = cleanTextForTTS(chunkText);
        catWashPlaybackState.currentWords = cleanedChunkText.split(/\\s+/).filter((w: string) => w.length > 0);
        catWashPlaybackState.wordIndex = 0;
    }

    if (i > startChunk) {
        savePlaybackSession({
            fileId: fileId || 0,
            fileName,
            chunkIndex: i,
            totalChunks: chunks.length,
            trigger: catWashPlaybackTrigger || 'manual',
            startedAt: catWashPlaybackStartedAt?.toISOString() || new Date().toISOString(),
            updatedAt: new Date().toISOString(),
            status: 'active',
        }).catch(() => {});
    }

    if (chunksPlayedSinceLastPrompt >= ATTENTION_INTERVAL) {
        console.log(`[Nest Playback] Attention prompt after ${chunksPlayedSinceLastPrompt} chunks`);
        const promptPath = await generateAndSaveTTSAudio("Bryn, are you paying attention?", "nest-attention-${Date....}");
        await playOnNestSpeaker( `${appUrl}${promptPath}` );
        await new Promise(r => setTimeout(r, 4000));
        chunksPlayedSinceLastPrompt = 0;
    }
}

```

```

const chunkText = chunks[i];

try {
  let audioPath: string;
  if (preparedAudioCache[i] && preparedAudioCache[i].length > 0) {
    audioPath = preparedAudioCache[i];
    console.log(`[Nest Playback] Using pre-prepared audio for chunk ${i + 1}/${chunks.length}`);
  } else if (i === startChunk && preGeneratedFirstChunkPath) {
    audioPath = preGeneratedFirstChunkPath;
    console.log(`[Nest Playback] Using pre-generated audio for chunk ${i + 1}/${chunks.length}`);
  } else if (lookaheadChunkIndex === i && lookaheadAudioPath) {
    audioPath = lookaheadAudioPath;
    console.log(`[Nest Playback] Using look-ahead audio for chunk ${i + 1}/${chunks.length}`);
  } else {
    if (lookaheadChunkIndex === i && lookaheadPromise) {
      console.log(`[Nest Playback] Waiting for in-flight look-ahead for chunk ${i + 1}/${chunks.length}`);
      const resolved = await lookaheadPromise;
      if (resolved) {
        audioPath = resolved;
        console.log(`[Nest Playback] Look-ahead resolved for chunk ${i + 1}/${chunks.length}`);
      } else {
        console.log(`[Nest Playback] Look-ahead failed, regenerating chunk ${i + 1}/${chunks.length} (${chunk...
        audioPath = await generateAndSaveTTSAudio(chunkText, `nest-chunk-${fileId}-${i}-${Date.now()}`, 'voice...
      }
    } else {
      console.log(`[Nest Playback] Generating chunk ${i + 1}/${chunks.length} (${chunkText.length} chars)`);
      audioPath = await generateAndSaveTTSAudio(chunkText, `nest-chunk-${fileId}-${i}-${Date.now()}`, 'voice, ...
    }
  }
  lookaheadAudioPath = null;
  lookaheadChunkIndex = -1;
  lookaheadPromise = null;

  const wordCount = chunkText.split(/\s+/).length;
  const estimatedMs = Math.max(5000, (wordCount / 175) * 60 * 1000 + 1000);

  if (catWashPlaybackState) {
    catWashPlaybackState.estimatedChunkDuration = estimatedMs;
  }

  let chunkPlaying = false;
  const playResult = await playOnNestSpeaker(`${appUrl}${audioPath}`);
  if (playResult.success) {
    chunkPlaying = true;
    consecutivePlayFailures = 0;
    if (!playResult.actuallyPlaying) {
      console.log(`[Nest Playback] Nest state unconfirmed for chunk ${i + 1} – trusting play_media succeeded`);
    }
  } else {
    consecutivePlayFailures++;
    console.error(`[Nest Playback] Chunk ${i + 1} play_media FAILED (${consecutivePlayFailures}/${MAX_PLAY_FA...
  }
  if (consecutivePlayFailures >= MAX_PLAY_FAILURES) {
    if (!haHealth.connected) {
      console.warn(`[Nest Playback] HA offline – pausing playback at chunk ${i}`);
      if (fileId) { try { await storage.updateFile(fileId, { lastChunkIndex: i }); } catch {} }
      await savePlaybackSession({
        fileId: fileId || 0, fileName, chunkIndex: i, totalChunks: chunks.length,
        trigger: catWashPlaybackTrigger || 'manual',
        startedAt: catWashPlaybackStartedAt?.toISOString() || new Date().toISOString(),
        updatedAt: new Date().toISOString(), status: 'paused',
      }).catch(() => {});
      let waitedForReconnect = 0;
      const RECONNECT_TIMEOUT_MS = 5 * 60 * 1000;
      const RECONNECT_CHECK_MS = 10000;
      while (waitedForReconnect < RECONNECT_TIMEOUT_MS && catWashPlaybackActive && catWashSessionId === sessi...
        if (haHealth.connected) {
          console.log(`[Nest Playback] HA reconnected after ${Math.round(waitedForReconnect / 1000)}s – resum...
          consecutivePlayFailures = 0;
          i--;
          break;
        }
        await new Promise(r => setTimeout(r, RECONNECT_CHECK_MS));
        waitedForReconnect += RECONNECT_CHECK_MS;
      }
      if (waitedForReconnect >= RECONNECT_TIMEOUT_MS) {
        console.error(`[Nest Playback] HA still offline after ${RECONNECT_TIMEOUT_MS / 1000}s – stopping`);
        break;
      }
      continue;
    }
    console.error(`[Nest Playback] CIRCUIT BREAKER: ${MAX_PLAY_FAILURES} consecutive Nest failures – trying E...

  const echoConfirmed = await promptFallbackSwitch(sessionId, 'Echo',
    "Bryn, the Nest speaker encountered errors. Do you want to play your file on the Echo speakers for now?");
  if (echoConfirmed && catWashPlaybackActive && catWashSessionId === sessionId) {
    console.log(`[Echo Fallback] User confirmed – playing remaining chunks via Echo TTS`);
    let echoFailures = 0;
    for (let ei = i; ei < chunks.length; ei++) {
      if (!catWashPlaybackActive || catWashSessionId !== sessionId) break;
      if (catWashPlaybackState) {
        catWashPlaybackState.chunkIndex = ei;
        const echoCleanedText = cleanTextForTTS(chunks[ei] || '');
        catWashPlaybackState.currentWords = echoCleanedText.split(/\s+/).filter((w: string) => w.length > 0);
        catWashPlaybackState.wordIndex = 0;
      }
      const ok = await playChunkViaEchoTTS(chunks[ei], sessionId);
      if (!ok) {
        if (!catWashPlaybackActive || catWashSessionId !== sessionId) break;
        echoFailures++;
        if (echoFailures >= 3) {
          console.error(`[Echo Fallback] 3 consecutive Echo failures – trying HA Voice`);
          break;
        }
      }
      continue;
    }
    echoFailures = 0;
    if (fileId) {

```

```

        try { await storage.updateFile(fileId, { lastChunkIndex: ei }); } catch {}
    }
}
if (echoFailures < 3) break;
}

if (!catWashPlaybackActive || catWashSessionId !== sessionId) break;

const haVoiceConfirmed = await promptFallbackSwitch(sessionId, 'HA Voice',
    "Bryn, all other voice software encountered errors. Do you want to play your file here on Home Assistan...
if (haVoiceConfirmed && catWashPlaybackActive && catWashSessionId === sessionId) {
    console.log(`[HA Voice Fallback] User confirmed – playing remaining chunks via HA Cloud TTS`);
    for (let hi = 1; hi < chunks.length; hi++) {
        if (!catWashPlaybackActive || catWashSessionId !== sessionId) break;
        if (catWashPlaybackState) {
            catWashPlaybackState.chunkIndex = hi;
            const haCleanedText = cleanTextForTTS(chunks[hi] || '');
            catWashPlaybackState.currentWords = haCleanedText.split(/\s+/).filter((w: string) => w.length > 0);
            catWashPlaybackState.wordIndex = 0;
        }
        const ok = await playChunkViaHACloudTTS(chunks[hi], sessionId);
        if (!ok) {
            if (!catWashPlaybackActive || catWashSessionId !== sessionId) break;
            continue;
        }
        if (fileId) {
            try { await storage.updateFile(fileId, { lastChunkIndex: hi }); } catch {}
        }
    }
} else {
    console.log(`[Fallback] No fallback confirmed – stopping playback`);
    haServiceCallSafe('tts/speak', {
        entity_id: HA_CLOUD_TTS_ENTITY,
        media_player_entity_id: CAT_WR_HA_VOICE_ENTITY,
        message: "Stopping. Your position has been saved."
    }, 'Error TTS');
}
break;
}
if (!chunkPlaying) {
    await new Promise(r => setTimeout(r, 3000));
    continue;
}
if (catWashPlaybackState) {
    catWashPlaybackState.chunkStartedAt = new Date(Date.now() + 500);
    catWashPlaybackState.wordIndex = 0;
}
startWordAdvancement();
console.log(`[Nest Playback] Playing chunk ${i + 1}, ~${Math.round(estimatedMs / 1000)}s`);

const nextIdx = i + 1;
if (nextIdx < chunks.length && !(preparedAudioCache[nextIdx] && preparedAudioCache[nextIdx].length > 0)) {
    console.log(`[Nest Playback] Look-ahead: pre-generating chunk ${nextIdx + 1}/${chunks.length} in backgrou...
    lookaheadChunkIndex = nextIdx;
    lookaheadPromise = generateAndSaveTTSAudio(chunks[nextIdx], `nest-chunk-${fileId}-${nextIdx}-${Date.now()}...
    .then(path => { lookaheadAudioPath = path; return path; })
    .catch(err => { console.log(`[Nest Playback] Look-ahead generation failed for chunk ${nextIdx + 1}: ${e...
}

const completed = await waitForNestPlaybackEnd(estimatedMs, sessionId);
if (!completed) {
    console.log(`[Nest Playback] Session ended during chunk ${i + 1}`);
    if (fileId) {
        try { await storage.updateFile(fileId, { lastChunkIndex: i }); } catch {}
        console.log(`[Nest Playback] Saved progress on session end: chunk ${i}`);
    }
    break;
}

chunksPlayedSinceLastPrompt++;
console.log(`[Nest Playback] Chunk ${i + 1} done, chunksPlayedSinceLastPrompt=${chunksPlayedSinceLastPrompt}...

if (fileId) {
    const existingFile = await storage.getFile(fileId);
    let currentChecked: number[] = [];
    if (existingFile?.checkedChunks) {
        try { currentChecked = JSON.parse(existingFile.checkedChunks); } catch {}
    }
    if (!currentChecked.includes(i)) {
        currentChecked.push(i);
        currentChecked.sort((a, b) => a - b);
    }
    try {
        await storage.updateFile(fileId, {
            lastChunkIndex: i,
            checkedChunks: JSON.stringify(currentChecked),
        });
        console.log(`[Nest Playback] Saved chunk ${i} as checked (${currentChecked.length}/${chunks.length} tot...
    } catch {}
}
} catch (chunkErr: any) {
    console.error(`[Nest Playback] Error on chunk ${i + 1}: ${chunkErr.message}`);
    await new Promise(r => setTimeout(r, 2000));
}

}

if (!aborted && catWashPlaybackActive && catWashSessionId === sessionId) {
    console.log(`[Nest Playback] All chunks complete for "${fileName}"`);
    const allChecked = Array.from({ length: chunks.length }, (_, i) => i);
    try { await storage.updateFile(fileId, { listened: true, lastChunkIndex: chunks.length, checkedChunks: JSON.s...

const semesterSettings = await storage.getActiveSemesterSettings();
const semStart = semesterSettings?.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : new Dat...
const rwStart = semesterSettings?.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : new Date("...
const currentWeekNumber = getWeekNumber(torontoDate(), semStart, rwStart);
const completedFileDesc = describeFileForTTS({ folder: catWashPlaybackState?.fileName || fileName, originalNa...

const allFilesNow = await storage.getFiles();

```



```

const nextFile = findNextFileByPriority(allFilesNow, currentWeekNumber, fileId);

if (nextFile) {
  const nextFileDesc = describeFileForTTS(nextFile, currentWeekNumber);
  const transitionText = `${completedFileDesc} is complete. Now playing ${nextFileDesc}.`;
  console.log(`[Nest Playback] Transition: "${transitionText}"`);
  const transitionPath = await generateAndSaveTTSAudio(transitionText, `nest-transition-${Date.now()}`, voice);
  await playOnNestSpeaker(`${appUrl}${transitionPath}`);
  await new Promise(r => setTimeout(r, 6000));

  const nextText = await extractFileText(nextFile);
  if (nextText) {
    const nextChunks = chunkTextForNest(nextText);
    if (nextChunks.length > 0) {
      const nextName = nextFile.displayName || nextFile.originalName;
      console.log(`[Nest Playback] Starting next file: "${nextName}" (${nextChunks.length} chunks)`);
      catWashPlaybackState = {
        fileId: nextFile.id,
        fileName: nextName,
        chunkIndex: 0,
        totalChunks: nextChunks.length,
        chunks: nextChunks,
        currentWords: [],
        wordIndex: 0,
        startedAt: new Date(),
        chunkStartedAt: new Date(),
        estimatedChunkDuration: 0,
        playbackMode: 'server-tts',
      };
      await savePlaybackSession({
        fileId: nextFile.id,
        fileName: nextName,
        chunkIndex: 0,
        totalChunks: nextChunks.length,
        trigger: catWashPlaybackTrigger || 'manual',
        startedAt: catWashPlaybackStartedAt?.toISOString() || new Date().toISOString(),
        updatedAt: new Date().toISOString(),
        status: 'active',
      });
      startNestChunkPlayback(nextFile.id, nextName, nextChunks, 0, sessionId, voice);
      return;
    } else {
      console.error(`[Nest Playback] Next file "${nextFile.displayName || nextFile.originalName}" produced 0 ...`);
    }
  } else {
    console.error(`[Nest Playback] Failed to extract text from next file "${nextFile.displayName || nextFile....`);
  }
}

const completionPath = await generateAndSaveTTSAudio("All readings for this week are complete. Great job Bryn...");
await playOnNestSpeaker(`${appUrl}${completionPath}`);
await clearPlaybackSession();
console.log(`[Nest Playback] All readings complete – cleared persisted session`);
}

} catch (err: any) {
  console.error(`[Nest Playback] Fatal error: ${err.message}`, err.stack?.split('\n').slice(0, 3).join(' | '));
  try {
    await haServiceCall('tts/speak', {
      entity_id: HA_CLOUD_TTS_ENTITY,
      media_player_entity_id: CAT_WIR_HA_VOICE_ENTITY,
      message: "Sorry, playback encountered an error and stopped."
    }, 'Fatal Error TTS');
  } catch {}
} finally {
  if (catWashSessionId === sessionId) {
    catWashTrace('NestPlayback', 'FINALLY BLOCK – cleaning up session ${sessionId}');
    catWashPlaybackActive = false;
    catWashPlaybackStartedAt = null;
    catWashPlaybackState = null;
    nestPlaybackAbort = null;
    lastPlaybackStoppedAt = Date.now();
    stopToothbrushPolling();
    stopWordAdvancement();
  } else {
    catWashTrace('NestPlayback', 'FINALLY BLOCK – session mismatch (loop=${sessionId}, current=${catWashSessionId...`);
  }
  audioPreparationPaused = false;
  console.log(`[Nest Playback] Resumed audio preparation`);
}
}

function chunkTextForNest(text: string, maxLength: number = 1500): string[] {
  const sentences = text.split(/(?<=[!?\])\s+/);
  const chunks: string[] = [];
  let current = '';
  for (const sentence of sentences) {
    if (current.length + sentence.length + 1 > maxLength && current.length > 0) {
      chunks.push(current.trim());
      current = '';
    }
    current += (current ? ' ' : '') + sentence;
  }
  if (current.trim()) chunks.push(current.trim());
  return chunks.filter(c => c.length > 10);
}

async function extractFileText(file: any): Promise<string | null> {
  try {
    if (file.extractedText) {
      console.log(`[ExtractText] Using cached text for ${file.originalName} (${file.extractedText.length} chars)`);
      return file.extractedText;
    }

    let buffer: Buffer | null = null;

    if (file.objectPath?.startsWith('/School/')) {
      console.log(`[ExtractText] Downloading from OneDrive path: ${file.objectPath}`);
    }
  }
}

```

```

const { getOneDriveClient } = await import("./onedrive");
const client = await getOneDriveClient();
const encodedPath = encodeURIComponent(file.objectPath).replace(/%2F/g, '/');
const item = await client.api('/me/drive/root:${encodedPath}').get();
const downloadUrl = item['@microsoft.graph.downloadUrl'];
if (!downloadUrl) {
  console.error('[ExtractText] No download URL from OneDrive for: ${file.objectPath}');
  return null;
}
const pdfResponse = await fetch(downloadUrl);
if (!pdfResponse.ok) {
  console.error('[ExtractText] OneDrive download failed: ${pdfResponse.status}');
  return null;
}
buffer = Buffer.from(await pdfResponse.arrayBuffer());
console.log('[ExtractText] Downloaded ${buffer.length} bytes from OneDrive');
} else if (file.objectPath) {
  try {
    const { ObjectStorageService } = await import("./replit_integrations/object_storage");
    const objectStorage = new ObjectStorageService();
    const foundFile = await objectStorage.getObjectEntityFile(file.objectPath);
    if (foundFile) {
      const chunks: Buffer[] = [];
      await new Promise<void>((resolve, reject) => {
        const stream = foundFile.createReadStream();
        stream.on('data', (chunk: Buffer) => chunks.push(chunk));
        stream.on('end', () => resolve());
        stream.on('error', (err: Error) => reject(err));
      });
      buffer = Buffer.concat(chunks);
      console.log('[ExtractText] Downloaded ${buffer.length} bytes from object storage');
    } else {
      console.error('[ExtractText] File not found in object storage: ${file.objectPath}');
    }
  } catch (osErr: any) {
    console.error('[ExtractText] Object storage download error: ${osErr.message}');
  }
}

if (!buffer) return null;
const PdfParser = await getPdfParser();
const parser = new PdfParser({ data: new Uint8Array(buffer), verbosity: 0 });
await parser.load();
const pdfText = await parser.getText();
let textContent = '';
if (pdfText && typeof pdfText === 'object') {
  if (pdfText.pages && Array.isArray(pdfText.pages)) {
    textContent = pdfText.pages.map((p: any) => p.text || '').join('\n\n');
  } else if (pdfText.text) {
    textContent = pdfText.text;
  }
} else if (typeof pdfText === 'string') {
  textContent = pdfText;
}
console.log('[ExtractText] Extracted ${textContent.length} chars from ${file.originalName}');
const cleanedText = cleanTextForTTS(textContent);

if (cleanedText && file.id) {
  try {
    await storage.updateFile(file.id, { extractedText: cleanedText });
    console.log('[ExtractText] Cached ${cleanedText.length} chars for file ${file.id}');
  } catch (cacheErr: any) {
    console.error('[ExtractText] Failed to cache text: ${cacheErr.message}');
  }
}

return cleanedText;
} catch (e: any) {
  console.error('[ExtractText] Error for ${file.originalName}: ${e.message}');
  return null;
}
}

let voiceCommandPauseState_: {
  fileId: number;
  chunkIndex: number;
  fileName: string;
  pausedAt: Date;
  autoStopTimer: ReturnType<typeof setTimeout>;
} | null = null;

function clearVoiceCommandPause_() {
  if (voiceCommandPauseState_) {
    clearTimeout(voiceCommandPauseState_.autoStopTimer);
    voiceCommandPauseState_ = null;
  }
}

async function stopNestPlaybackWithGoodbye(reason: string, keepOpen: boolean = false): Promise<void> {
  const haUrl = HOME_ASSISTANT_URL.replace(/\$/g, '');
  const appUrl = DEPLOYED_APP_URL;

  catWashTrace('StopWithGoodbye', `ENTER reason=${reason} keepOpen=${keepOpen}`);
  lastPlaybackStoppedAt = Date.now();

  clearVoiceCommandPause_();

  if (nestPlaybackAbort) {
    nestPlaybackAbort();
    nestPlaybackAbort = null;
  }

  await Promise.allSettled([
    stopNestSpeaker(),
    haServiceCallSafe('media_player/media_stop', { entity_id: CAT_WR_HA_VOICE_ENTITY }, 'Stop HA Voice'),
  ]);

  let fileName = catWashPlaybackState?.fileName || '';

```

```

const savedFileId = catWashPlaybackState?.fileId;
const savedChunk = catWashPlaybackState?.chunkIndex || 0;

if (savedFileId && savedChunk > 0) {
  try {
    await storage.updateFile(savedFileId, { lastChunkIndex: savedChunk });
    console.log(`[Nest Stop] Saved progress: file ${savedFileId}, chunk ${savedChunk}`);
  } catch {}
}

const cleanName = fileName ? fileName.replace(/\.pdf$/i, '').replace(/[_-]+/g, ' ').replace(/\s+/g, ' ').trim() : ...
const goodbyeText = cleanName
  ? `Stopping. Your position has been saved. See you next time Bryn.`
  : `Stopping. Your position has been saved. See you next time Bryn.`;

console.log(`[Nest Stop] Reason: ${reason}. Goodbye: "${goodbyeText}"`);

try {
  await new Promise(r => setTimeout(r, 500));
  await haServiceCall('tts/speak', {
    entity_id: HA_CLOUD_TTS_ENTITY,
    media_player_entity_id: CAT_WR_HA_VOICE_ENTITY,
    message: goodbyeText
  }, 'Goodbye TTS');
  const wordCount = goodbyeText.split(/\s+/).length;
  await new Promise(r => setTimeout(r, Math.max(3000, (wordCount / 175) * 60 * 1000)));
} catch (e: any) {
  console.error(`[Nest Stop] Error playing goodbye: ${e.message}`);
}

catWashPlaybackActive = false;
catWashPlaybackStartedAt = null;
catWashPlaybackState = null;
nestPlaybackAbort = null;
currentTvFollowUrl = null;
currentTabletReaderUrl = null;
lastPlaybackStoppedAt = Date.now();
stopToothbrushPolling();
stopWordAdvancement();
await clearPlaybackSession();
console.log(`[Nest Stop] Cleared persisted playback session`);

const stopTimestamp = Date.now();
await Promise.all([
  setTabletCommand({ action: 'stop_playback', goodbyeText: '', keepOpen, timestamp: stopTimestamp }, true, 'maste...'),
  setTabletCommand({ action: 'stop_playback', keepOpen, timestamp: stopTimestamp }, true, 'tv'),
]);

catWashTrace('TV', `TURN OFF — Fire Stick + Samsung TV (reason=${reason})`);
await Promise.allSettled([
  haServiceCallSafe('media_player/turn_off', { entity_id: FIRE_STICK_ADB_ENTITY }, 'Nest Stop FireStick'),
  haServiceCallSafe('media_player/turn_off', { entity_id: CAT_TV_ENTITY }, 'Nest Stop Samsung TV'),
]);
console.log(`[Nest Stop] Fire Stick + Samsung TV turn-off sent`);
catWashTrace('StopWithGoodbye', `EXIT reason=${reason}`);
}

// GET /api/shower/next-reading - Get next unlistened module/reading file for current week
app.get("/api/shower/next-reading", async (req, res) => {
  try {
    const allFiles = await storage.GetFiles();

    // Get current week number from semester settings
    const semesterSettings = await storage.getActiveSemesterSettings();
    let currentWeekNumber = 1;
    const semStart = semesterSettings?.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : new Date(...
    const rwStart = semesterSettings?.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : new Date("20...
    currentWeekNumber = getWeekNumber(torontoDate(), semStart, rwStart);

    const nextFile = findNextFileByPriority(allFiles, currentWeekNumber);

    if (!nextFile) {
      return res.json({
        message: `All modules and readings for weeks 1-${currentWeekNumber} have been listened to!`,
        nextFile: null,
        allComplete: true,
        currentWeek: currentWeekNumber
      });
    }

    // Check if we have progress for this file
    const progressKey = `file-${nextFile.id}`;
    const progress = playbackProgress[progressKey];

    const checkModule = (f: any) =>
      f.folder?.toLowerCase().includes('module') ||
      f.originalName?.toLowerCase().includes('module');
    const fileType = checkModule(nextFile) ? 'module' : 'reading';

    res.json({
      file: {
        id: nextFile.id,
        name: nextFile.displayName || nextFile.originalName,
        folder: nextFile.folder,
        objectPath: nextFile.objectPath,
        type: fileType
      },
      readerUrl: `/pdf-reader/${nextFile.id}`,
      currentWeek: currentWeekNumber,
      progress: progress || { chunkIndex: 0, totalChunks: 0 },
      resuming: !!progress
    });
  } catch (error) {
    console.error("Error getting next reading:", error);
    res.status(500).json({ error: "Failed to get next reading" });
  }
});

```

```

async function findNextCatWashFile(storageRef: any, weekNumber: number, excludeFileId?: number) {
  const allFiles = await storageRef.getFiles();
  const isModuleFile = (f: any) => f.folder?.toLowerCase().includes('module');

  const isPartiallyListened = (f: any) => {
    if (f.listened || f.id === excludeFileId) return false;
    const hasCheckedChunks = () => {
      if (!f.checkedChunks) return false;
      try { const arr = JSON.parse(f.checkedChunks); return Array.isArray(arr) && arr.length > 0; } catch { return ...
    }());
    const hasLastChunk = f.lastChunkIndex !== null && f.lastChunkIndex > 0;
    if (!hasCheckedChunks && !hasLastChunk) return false;
    if (f.totalChunks && f.totalChunks > 0) {
      if (hasCheckedChunks) {
        try { const arr = JSON.parse(f.checkedChunks); if (arr.length >= f.totalChunks) return false; } catch {}
      }
      if (hasLastChunk && f.lastChunkIndex >= f.totalChunks) return false;
    }
    return true;
  };

  const getFileWeek = (f: any) => {
    const weekMatch = f.folder?.match(/week-(\d+)/i);
    return weekMatch ? parseInt(weekMatch[1], 10) : -1;
  };

  const getCourseCodeForFile = (f: any): string => {
    const folder = (f.folder || '').toLowerCase();
    const name = (f.originalName || '').toLowerCase();
    const match = folder.match(/([a-z]{3,5}\s?\d{3})/i) || name.match(/([a-z]{3,5}\s?\d{3})/i);
    return match ? match[1].toUpperCase().replace(/\s/g, '') : 'UNKNOWN';
  };

  const w = weekNumber;
  const weekPartials = allFiles.filter((f: any) => isPartiallyListened(f) && getFileWeek(f) === w);
  const allPartialIds = new Set(weekPartials.map((f: any) => f.id));

  const unlistenedFiles = allFiles.filter((f: any) => {
    if (f.listened || f.id === excludeFileId) return false;
    if (allPartialIds.has(f.id)) return false;
    return getFileWeek(f) === w;
  });

  const allWeekUnlistened = allFiles.filter((f: any) => !f.listened && f.id !== excludeFileId && getFileWeek(f) ===...

  if (allWeekUnlistened.length === 0 && weekPartials.length === 0) {
    console.log(`[CatWashFile] No unlistened files found in week ${weekNumber}`);
    return null;
  }

  const hasAnyUnlistenedModule = allWeekUnlistened.some(f => isModuleFile(f));

  const filteredUnlistened = unlistenedFiles.filter(f => {
    if (isModuleFile(f)) return true;
    return !hasAnyUnlistenedModule;
  });

  const filteredPartials = weekPartials.filter(f => {
    if (isModuleFile(f)) return true;
    return !hasAnyUnlistenedModule;
  });

  const allCandidates = [...filteredPartials, ...filteredUnlistened];
  const orderedFiles = allCandidates.sort((a, b) => {
    const aPri = getCoursePriorityForFile(a);
    const bPri = getCoursePriorityForFile(b);
    if (aPri !== bPri) return aPri - bPri;
    const aModule = isModuleFile(a) ? 0 : 1;
    const bModule = isModuleFile(b) ? 0 : 1;
    return aModule - bModule;
  });

  console.log(`[CatWashFile] week=${w}, unlistened=${allWeekUnlistened.length}, modulesBlocked=${hasAnyUnlistenedMo...

  if (orderedFiles.length > 0) return orderedFiles[0];
  console.log(`[CatWashFile] No unlistened files found in week ${weekNumber}`);
  return null;
}

async function extractAndChunkPdf(file: any): Promise<{ textContent: string; chunks: string[] } | null> {
  let textContent = "";
  try {
    let content: Buffer;
    const mediaUrl = file.objectPath || '';

    if (mediaUrl.startsWith("/objects/")) {
      const { ObjectStorageService } = await import("./replit_integrations/object_storage");
      const objectStorage = new ObjectStorageService();
      const objectFile = await objectStorage.getObjectEntityFile(mediaUrl);
      const [downloaded] = await objectFile.download();
      content = downloaded;
    } else if (mediaUrl.startsWith("/School/")) {
      const { getOneDriveClient } = await import("./onedrive");
      const client = await getOneDriveClient();
      const encodedPath = encodeURIComponent(mediaUrl.replace(/%2F/g, '/'));
      const item = await client.api(`/me/drive/root:${encodedPath}`).get();
      const downloadUrl = item["@microsoft.graph.downloadUrl"];
      if (!downloadUrl) throw new Error("Could not get OneDrive download URL");
      const pdfResponse = await fetch(downloadUrl);
      if (!pdfResponse.ok) throw new Error(`OneDrive download failed: ${pdfResponse.status}`);
      content = Buffer.from(await pdfResponse.arrayBuffer());
    } else if (mediaUrl.startsWith("onedrive://")) {
      const folderPart = mediaUrl.replace('onedrive://', '').split('/')[0] || '';
      const fileNamePart = mediaUrl.split('/').pop() || '';
      const parts = folderPart.split('-');
      const weekNum = parts[1];
      const courseCode = parts[2]?.toUpperCase();
      const basePath = `/School/1. TMU/Courses/2026/Winter`;
    }
  } catch {}
}

```

```

const baseFolders = await listOneDriveItems(basePath);
const matchedFolder = baseFolders.find((f: any) => f.type === 'folder' && f.name.toUpperCase().startsWith(cou...
if (!matchedFolder) throw new Error("Course folder not found in OneDrive");
const courseFolders = await listOneDriveItems(matchedFolder.path);
const weekFolder = courseFolders.find((f: any) => f.type === 'folder' && f.name.toLowerCase().startsWith('week...
if (!weekFolder) throw new Error("Week folder not found in OneDrive");
const weekContents = await listOneDriveItems(weekFolder.path);
const typeFolder = weekContents.find((f: any) => f.type === 'folder' && f.name.toLowerCase().includes(folderP...
if (!typeFolder) throw new Error("Type folder not found in OneDrive");
const files = await listOneDriveItems(typeFolder.path);
const matchedFile = files.find((f: any) => f.name === fileNamePart);
if (!matchedFile?.downloadUrl) throw new Error("File not found in OneDrive");
const pdfResponse = await fetch(matchedFile.downloadUrl);
content = Buffer.from(await pdfResponse.arrayBuffer());
} else if (mediaUrl.startsWith("http")) {
const pdfResponse = await fetch(mediaUrl);
content = Buffer.from(await pdfResponse.arrayBuffer());
} else {
throw new Error(`Unsupported path format: ${mediaUrl.substring(0, 50)}`);
}

const isPDF = content.slice(0, 4).toString() === '%PDF';
if (isPDF) {
const PdfParser = await getPdfParser();
const parser = new PdfParser({ data: new Uint8Array(content), verbosity: 0 });
await parser.load();
const pdfText = await parser.getText();
if (pdfText && typeof pdfText === 'object') {
if (pdfText.pages && Array.isArray(pdfText.pages)) {
textContent = pdfText.pages.map((page: any) => page.text || '').join(' ');
} else if (Array.isArray(pdfText)) {
textContent = pdfText.map((item: any) => typeof item === 'string' ? item : item.text || '').join(' ');
} else if (pdfText.text) {
textContent = pdfText.text;
}
} else {
textContent = String(pdfText || '');
}
await parser.destroy();
} else {
textContent = content.toString('utf-8');
}
} catch (error: any) {
console.error(`[Cat Wash] Text extraction error for ${file.id}:`, error.message);
return null;
}

let cleanedContent = textContent.trim().replace(/[\x20-\x7E\n]/g, ' ');
if (cleanedContent.length < 10) return null;

const paragraphs = cleanedContent.split(/\n\s*\n/).map(p => p.trim()).filter(p => p.length > 0);
const maxLen = 4000;
const chunks: string[] = [];

if (paragraphs.length > 1) {
let currentChunk = "";
for (const para of paragraphs) {
if (para.length > maxLen) {
if (currentChunk) { chunks.push(currentChunk.trim()); currentChunk = ""; }
let remaining = para;
while (remaining.length > 0) {
if (remaining.length <= maxLen) { chunks.push(remaining); break; }
let splitAt = remaining.lastIndexOf('.', maxLen);
if (splitAt < 500) splitAt = remaining.lastIndexOf(' ', maxLen);
if (splitAt < 300) splitAt = maxLen;
chunks.push(remaining.slice(0, splitAt + 1).trim());
remaining = remaining.slice(splitAt + 1).trim();
}
} else if ((currentChunk + "\n\n" + para).length > maxLen) {
if (currentChunk) chunks.push(currentChunk.trim());
currentChunk = para;
} else {
currentChunk = currentChunk ? currentChunk + "\n\n" + para : para;
}
}
if (currentChunk) chunks.push(currentChunk.trim());
} else {
let remaining = cleanedContent.replace(/\s+/g, ' ');
while (remaining.length > 0) {
if (remaining.length <= maxLen) { chunks.push(remaining); break; }
let splitAt = remaining.lastIndexOf('.', maxLen);
if (splitAt < 500) splitAt = remaining.lastIndexOf(' ', maxLen);
if (splitAt < 300) splitAt = maxLen;
chunks.push(remaining.slice(0, splitAt + 1).trim());
remaining = remaining.slice(splitAt + 1).trim();
}
}

return { textContent: cleanedContent, chunks };
}

async function openUrlOnFireDevice(haUrl: string, browserIds: string[], url: string, deviceName: string): Promise<b...
const results: string[] = [];

const urlObj = new URL(url);
const navigatePath = `${urlObj.pathname}${urlObj.search}`;
const intentUri = `intent://${urlObj.host}${urlObj.pathname}${urlObj.search}#Intent;scheme=https;package=com.amaz...

const bustUrl = url + (url.includes('?') ? '&' : '?') + '_t=' + Date.now();
const directNavCode = `window.location.replace('${bustUrl}');`;

const intentCode = `
var a = document.createElement('a');
a.href = '${intentUri}';
a.style.display = 'none';
document.body.appendChild(a);
a.click();
document.body.removeChild(a);

```

;

```
for (const browserId of browserIds) {
  try {
    const resp = await fetch(`${haUrl}/api/services/browser_mod/javascript`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ browser_id: browserId, code: directNavCode }),
    });
    const body = await resp.text();
    console.log(`[Device] ${deviceName} !' browser_mod.javascript/direct_nav (${browserId}): ${resp.status} body=$...`);
    const bodyEmpty = body.trim() === '[]' || body.trim() === '';
    if (resp.ok && !bodyEmpty) {
      results.push(`${browserId}:direct_nav:${resp.status}`);
      console.log(`[Device] ${deviceName} results: [${results.join(', ')}] success=true (direct nav)`);
      return true;
    }
    if (bodyEmpty) {
      console.log(`[Device] ${deviceName} !' browser_mod returned empty (browser not connected)`);
    }
    results.push(`${browserId}:direct_nav:${resp.status}:${bodyEmpty ? 'no_browser' : 'ok'}`);
  } catch (e: any) {
    console.log(`[Device] ${deviceName} !' browser_mod.javascript/direct_nav (${browserId}) ERROR: ${e.message}`);
    results.push(`${browserId}:direct_nav:error`);
  }
}

try {
  const resp = await fetch(`${haUrl}/api/services/browser_mod/javascript`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ browser_id: browserId, code: intentCode }),
  });
  const body = await resp.text();
  console.log(`[Device] ${deviceName} !' browser_mod.javascript/silk_intent (${browserId}): ${resp.status} body=$...`);
  const bodyEmpty = body.trim() === '[]' || body.trim() === '';
  if (resp.ok && !bodyEmpty) {
    results.push(`${browserId}:silk_intent:${resp.status}`);
    console.log(`[Device] ${deviceName} results: [${results.join(', ')}] success=true (intent fallback)`);
    return true;
  }
  results.push(`${browserId}:silk_intent:${resp.status}:${bodyEmpty ? 'no_browser' : 'ok'}`);
} catch (e: any) {
  console.log(`[Device] ${deviceName} !' browser_mod.javascript/silk_intent (${browserId}) ERROR: ${e.message}`);
  results.push(`${browserId}:silk_intent:error`);
}
}

console.log(`[Device] ${deviceName} results: [${results.join(', ')}] success=false`);
return false;
}

// Helper to open URL on Fire Stick via androidtv integration
async function openUrlOnFireStick(haUrl: string, entityId: string, url: string): Promise<boolean> {
  console.log(`[Cat Wash] ===== openUrlOnFireStick START (${entityId}) =====`);

  for (let wakeAttempt = 1; wakeAttempt <= 2; wakeAttempt++) {
    try {
      await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: 'input keyevent KEYCODE_WAKEUP' } ...
      console.log(`[Cat Wash] Fire Stick WAKEUP sent (attempt ${wakeAttempt})`);
      break;
    } catch (e: any) {
      console.log(`[Cat Wash] Fire Stick WAKEUP attempt ${wakeAttempt} failed: ${e.message}`);
      if (wakeAttempt < 2) await new Promise(r => setTimeout(r, 2000));
    }
  }
  await new Promise(resolve => setTimeout(resolve, 4000));

  try {
    await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: 'input keyevent KEYCODE_HOME' }, '...
    console.log(`[Cat Wash] Fire Stick HOME sent`);
    await new Promise(resolve => setTimeout(resolve, 2000));
  } catch (e: any) {
    console.log(`[Cat Wash] Fire Stick HOME failed: ${e.message}`);
  }

  try {
    await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: 'am force-stop com.amazon.cloud9' } ...
    console.log(`[Cat Wash] Fire Stick force-stopped Silk`);
    await new Promise(resolve => setTimeout(resolve, 1500));
  } catch (e: any) {
    console.log(`[Cat Wash] Fire Stick force-stop Silk failed: ${e.message}`);
  }

  currentTvFollowUrl = url;
  const appUrl = DEPLOYED_APP_URL;
  const redirectUrl = `${appUrl}/api/cat-wash/tv-follow`;
  console.log(`[Cat Wash] TV redirect URL stored. Opening: ${redirectUrl}`);
  console.log(`[Cat Wash] TV will redirect to: ${url}`);

  for (let attempt = 1; attempt <= 3; attempt++) {
    try {
      const adbCmd = `am start --activity-clear-task -a android.intent.action.VIEW -d "${redirectUrl}" com.amazon.c...
      await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: adbCmd }, `FireStick Open URL ${...
      console.log(`[Cat Wash] Fire Stick URL opened (attempt ${attempt})`);

      const sendFullscreenCmds = async (fAttempt: number) => {
        try {
          await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: 'settings put global policy....
          console.log(`[Cat Wash] Fire Stick immersive mode set (attempt ${fAttempt})`);
        } catch (e: any) {
          console.log(`[Cat Wash] Fire Stick immersive ${fAttempt} failed: ${e.message}`);
        }
        try {
          await haServiceCall('androidtv/adb_command', { entity_id: entityId, command: 'input keyevent KEYCODE_DPAD...
          console.log(`[Cat Wash] Fire Stick DPAD_CENTER sent (attempt ${fAttempt})`);
        } catch (e: any) {
          console.log(`[Cat Wash] Fire Stick DPAD ${fAttempt} failed: ${e.message}`);
        }
      }
    }
  }
}
```

```

    };

    setTimeout(() => sendFullscreenCmds(1), 5000);
    setTimeout(() => sendFullscreenCmds(2), 12000);
    setTimeout(() => sendFullscreenCmds(3), 20000);

    console.log(`[Cat Wash] ===== openUrlOnFireStick SUCCESS =====`);
    return true;
  } catch (e: any) {
    console.log(`[Cat Wash] Fire Stick adb_command attempt ${attempt} failed: ${e.message}`);
    if (attempt < 3) await new Promise(r => setTimeout(r, 4000));
  }
}

console.log(`[Cat Wash] All ADB attempts failed - trying play_media fallback`);
try {
  await haServiceCall('media_player/play_media', { entity_id: entityId, media_content_id: redirectUrl, media_cont...
  console.log(`[Cat Wash] Fire Stick play_media url succeeded`);
  return true;
} catch (e: any) {
  console.log(`[Cat Wash] Fire Stick play_media also failed: ${e.message}`);
}

console.log(`[Cat Wash] ===== openUrlOnFireStick FAILED =====`);
return false;
}

// playCatWashFile is no longer needed for server-side TTS.
// The tablet handles all audio playback via browser <audio> !' Bluetooth !' Echo.
// Auto-continuation is handled by the tablet calling POST /api/cat-wash/update-progress
// with { completed: true }, which returns the next file URL for the tablet to navigate to.

app.get("/api/webhook/status", (_req, res) => {
  res.json({
    ok: true,
    timestamp: new Date().toISOString(),
    catWashPlaybackActive,
    catWashPlaybackStartedAt,
    currentFile: catWashPlaybackState?.fileName || null,
    currentChunk: catWashPlaybackState?.chunkIndex || 0,
    totalChunks: catWashPlaybackState?.totalChunks || 0,
    endpoints: ["/api/webhook/cat-lights", "/api/webhook/cat-lights-confirm", "/api/webhook/cat-shower-button", "/a...
  });
});

app.post("/api/webhook/test-tablet-open", async (req, res) => {
  try {
    const baseUrl = HOME_ASSISTANT_URL?.replace(/\/$/, '');
    if (!baseUrl || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }
    const apiUrl = DEPLOYED_APP_URL;
    const authParam = encodeURIComponent(process.env.SITE_PASSWORD || '');
    const testUrl = `${apiUrl}/pdf-reader/139?catWashFollow=true&autoplay=true&auth=${authParam}`;
    const { device: targetDevice, method: testMethod } = req.body;

    console.log(`[TEST] ===== TABLET OPEN TEST =====`);
    console.log(`[TEST] URL: ${testUrl}`);
    console.log(`[TEST] Target device: ${targetDevice || 'all'}`);
    console.log(`[TEST] Method: ${testMethod || 'all'}`);

    await setTabletCommand({ action: 'navigate', url: testUrl, timestamp: Date.now() });

    const results: Record<string, any> = {};

    const tablets = [
      {
        name: 'tablet_cat_wall',
        browserIds: ['browser_mod_0da8b0a7_fd42ec2e'],
        notifyServices: ['mobile_app_tablet_cat', 'mobile_app_fire_tablet_cat', 'mobile_app_tablet_cat_wall'],
        mediaPlayer: 'media_player.tablet_cat',
      },
    ];

    const filteredTablets = targetDevice ? tablets.filter(t => t.name === targetDevice) : tablets;

    for (const device of filteredTablets) {
      console.log(`[TEST] --- Testing ${device.name} ---`);
      const deviceResults: Record<string, any> = {};

      if (!testMethod || testMethod === 'browser_mod') {
        const opened = await openUrlOnFireDevice(baseUrl, device.browserIds, testUrl, device.name);
        deviceResults.browser_mod = opened;
      }

      if (!testMethod || testMethod === 'notify_command') {
        for (const svc of device.notifyServices) {
          try {
            const resp = await fetch(`${baseUrl}/api/services/notify/${svc}`, {
              method: 'POST',
              headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
              body: JSON.stringify({
                message: "command_webview",
                data: { url: testUrl },
              }),
            });
            const body = await resp.text();
            console.log(`[TEST] ${device.name} notify/${svc} command_webview: ${resp.status} body=${body.substring(...
            deviceResults[`${notify}_${svc}_webview`] = resp.ok;
            if (resp.ok) break;
          } catch (e: any) {
            console.log(`[TEST] ${device.name} notify/${svc} ERROR: ${e.message}`);
            deviceResults[`${notify}_${svc}_webview`] = `error: ${e.message}`;
          }
        }
      }

      if (!testMethod || testMethod === 'notify_url') {

```

```

    for (const svc of device.notifyServices) {
      try {
        const resp = await fetch(`${haUrl}/api/services/notify/${svc}`, {
          method: 'POST',
          headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
          body: JSON.stringify({
            title: "PDF Reader",
            message: "Opening reading...",
            data: { clickAction: testUrl, url: testUrl, tag: "test-tablet", importance: "high", channel: "cat-w..." },
          })),
        const body = await resp.text();
        console.log(`[TEST] ${device.name} notify/${svc} url notification: ${resp.status} body=${body.substring...`
        deviceResults[`${notify}_${svc}_url`] = resp.ok;
        if (resp.ok) break;
      } catch (e: any) {
        deviceResults[`${notify}_${svc}_url`] = `error: ${e.message}`;
      }
    }
  }
}

if (!testMethod || testMethod === 'play_media') {
  try {
    const resp = await fetch(`${haUrl}/api/services/media_player/play_media`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ entity_id: device.mediaPlayer, media_content_id: testUrl, media_content_type: 'u...`
    });
    const body = await resp.text();
    console.log(`[TEST] ${device.name} play_media (${device.mediaPlayer}): ${resp.status} body=${body.substri...`
    deviceResults.play_media = resp.ok;
  } catch (e: any) {
    deviceResults.play_media = `error: ${e.message}`;
  }
}

results[device.name] = deviceResults;
}

console.log(`[TEST] Results: ${JSON.stringify(results)}`);
res.json({ action: "test_complete", url: testUrl, results });
} catch (e: any) {
  console.error(`[TEST] Error: ${e.message}`, e.stack);
  res.status(500).json({ error: e.message });
}
});

app.post("/api/webhook/cat-shower-button", async (req, res) => {
  try {
    console.log(`[Shower Button] ===== WEBHOOK TRIGGERED =====`);
    console.log(`[Shower Button] Timestamp: ${new Date().toLocaleString('en-US', { timeZone: 'America/Toronto' })}`);

    const timeSinceStart = Date.now() - SERVER_START_TIME;
    if (timeSinceStart < SERVER_STARTUP.COOLDOWN_MS) {
      console.log(`[Shower Button] Ignoring - server started ${Math.round(timeSinceStart / 1000)}s ago (cooldown)`);
      return res.json({ action: "ignored", reason: "Server startup cooldown" });
    }

    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }

    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
    const haHeaders = { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' };

    if (catWashPlaybackActive) {
      console.log(`[Shower Button] Playback already active - letting it continue (button is toggling shower/fan off...`
      return res.json({ action: "skipped", reason: "Playback already active - not interrupting" });
    }

    if (catLightsPromptPending) {
      console.log(`[Shower Button] Lights prompt pending - skipping (button is toggling shower/fan off)`);
      return res.json({ action: "skipped", reason: "Prompt already pending - not interrupting" });
    }

    const today = torontoDate();
    const semesterSettings = await storage.getActiveSemesterSettings();

    if (!semesterSettings) {
      console.log(`[Shower Button] No active semester - playing CHUM FM`);
      await playChumFmRadio(haUrl);
      return res.json({ action: "radio", reason: "No active semester" });
    }

    let currentWeekNumber = 1;
    const semStart = semesterSettings?.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : new Date(...`
    const rwStart = semesterSettings?.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : new Date("20...`
    currentWeekNumber = getWeekNumber(today, semStart, rwStart);

    const allFilesBefore = await storage.getFiles();
    let nextFile = findNextFileByPriority(allFilesBefore, currentWeekNumber);

    if (!nextFile) {
      console.log(`[Shower Button] No cached files found - syncing OneDrive for weeks 1-${currentWeekNumber}`);
      for (let w = 1; w <= currentWeekNumber; w++) {
        await syncOneDriveFilesForWeek(semesterSettings, w, '[Shower Button]');
      }
      const allFilesAfter = await storage.getFiles();
      nextFile = findNextFileByPriority(allFilesAfter, currentWeekNumber);
    } else {
      console.log(`[Shower Button] Using cached file - syncing OneDrive in background`);
      syncOneDriveFilesForWeek(semesterSettings, currentWeekNumber, '[Shower Button]').catch(e => console.log(`[Sho...`
    }

    if (!nextFile) {
      console.log(`[Shower Button] No unlisted files for weeks 1-${currentWeekNumber} - playing CHUM FM`);
      await playChumFmRadio(haUrl);
      return res.json({ action: "radio", reason: `All weeks 1-${currentWeekNumber} readings complete - playing CHUM...`
    }
  }
}

```



```

}

const fileName = nextFile.displayName || nextFile.originalName || 'Unknown file';
const fileDesc = describeFileForTTS(nextFile, currentWeekNumber);
console.log('[Shower Button] Found file: ${fileDesc} - ${fileName} (id=${nextFile.id})');

res.json({ action: "playing", file: { id: nextFile.id, name: fileName }, currentWeek: currentWeekNumber });

const confirmTTS = `Okay, I will now play ${fileDesc}.`;

catWashPlaybackTrigger = 'button';
await startConfirmedPlaybackFlow(nextFile, '[Shower Button]', 'echo', confirmTTS);

} catch (error: any) {
  console.error("[Shower Button] Error:", error);
  res.status(500).json({ error: "Failed to handle shower button webhook", details: error.message });
}
});

app.post("/api/webhook/cat-lights-confirm", async (req, res) => {
  console.log('[Cat Lights Confirm] ===== CONFIRMATION RECEIVED =====');
  if (catLightsConfirmResolve) {
    catLightsConfirmResolve(true);
    catLightsConfirmResolve = null;

    if (!isSpotifyPlayingOnEverywhere()) {
      try {
        const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
        const catEchoEntities = CAT_ECHO_ENTITIES;
        await fetch(`${haUrl}/api/services/media_player/media_stop`, {
          method: 'POST',
          headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
          body: JSON.stringify({ entity_id: catEchoEntities }),
        });
        console.log('[Cat Lights Confirm] Stopped media on cat washroom Echos');
      } catch (e: any) {
        console.warn('[Cat Lights Confirm] Failed to stop Echos (non-fatal): ${e.message}');
      }
    } else {
      console.log('[Cat Lights Confirm] Everywhere group playing - preserving cat washroom Echos');
    }

    res.json({ action: "confirmed", message: "Module reading confirmed - starting playback" });
  } else {
    console.log('[Cat Lights Confirm] No pending confirmation to resolve');
    res.json({ action: "ignored", reason: "No pending confirmation" });
  }
});

// POST /api/webhook/cat-lights - Triggered when light.cat_lights turns on/off
// If the current week's CPPA module hasn't been fully listened to,
// turning the light ON starts/resumes playback on Cat Wash speaker group, turning it OFF stops and saves progress.
app.post("/api/webhook/cat-lights", async (req, res) => {
  try {
    catWashTrace('CatLights', 'WEBHOOK RECEIVED', { body: req.body });
    console.log('[Cat Lights] ===== WEBHOOK TRIGGERED =====');
    console.log('[Cat Lights] Timestamp: ${new Date().toLocaleString('en-US', { timeZone: 'America/Toronto' })}');
    console.log('[Cat Lights] Request body: ${JSON.stringify(req.body)}');

    const timeSinceStart = Date.now() - SERVER_START_TIME;
    if (timeSinceStart < SERVER_STARTUP_COOLDOWN_MS) {
      console.log('[Cat Lights] Ignoring - server started ${Math.round(timeSinceStart / 1000)}s ago (cooldown: ${SE...}');
      return res.json({ action: "ignored", reason: "Server startup cooldown" });
    }

    const haUrl0 = HOME_ASSISTANT_URL.replace(/\/$/, '') || '';
    const haHeaders0 = { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' };

    let lightState = 'unknown';
    const rawToState = req.body?.to_state;
    const toStateValue = typeof rawToState === 'object' && rawToState !== null ? rawToState.state : rawToState;
    const bodyState = req.body?.state || req.body?.new_state?.state || toStateValue || req.body?.trigger?.to_state?..
    const bodyAction = req.body?.action || '';
    console.log('[Cat Lights] Body parsing: state="${bodyState}", action="${bodyAction}", rawToState=${JSON.stringi...');
    if (bodyState === 'on' || bodyState === 'off') {
      lightState = bodyState;
      console.log('[Cat Lights] Using body state directly: ${lightState}');
    } else if (bodyAction === 'turn_off' || bodyAction === 'off') {
      lightState = 'off';
      console.log('[Cat Lights] Using body action as OFF: ${bodyAction}');
    } else if (bodyAction === 'turn_on' || bodyAction === 'on') {
      lightState = 'on';
      console.log('[Cat Lights] Using body action as ON: ${bodyAction}');
    } else {
      await new Promise(r => setTimeout(r, 1000));
      for (let attempt = 1; attempt <= 3; attempt++) {
        try {
          const lightResp = await fetch(`${haUrl0}/api/states/${CAT_LIGHTS_ENTITY}`, { headers: haHeaders0 });
          if (lightResp.ok) {
            const lightData = await lightResp.json();
            lightState = lightData?.state || 'unknown';
            if (lightState !== 'unknown') {
              if (attempt === 1 && lightState === 'on') {
                console.log('[Cat Lights] First query returned ON - verifying with second query after 500ms');
                await new Promise(r => setTimeout(r, 500));
                try {
                  const verifyResp2 = await fetch(`${haUrl0}/api/states/${CAT_LIGHTS_ENTITY}`, { headers: haHeaders... });
                  if (verifyResp2.ok) {
                    const verifyData2 = await verifyResp2.json();
                    const confirmedState = verifyData2?.state || 'unknown';
                    if (confirmedState === 'off') {
                      console.log('[Cat Lights] Second query shows OFF - light was briefly on or transitioning. Usi...');
                      lightState = 'off';
                    }
                  }
                } catch {}
              }
            }
          }
        } catch {}
      }
      break;
    }
  }
});

```

```

    }
  } catch (e: any) {
    console.log(`[Cat Lights] Attempt ${attempt}/3 failed to query light state: ${e.message}`);
  }
  if (attempt < 3) {
    console.log(`[Cat Lights] Retrying light state query in 1s (attempt ${attempt + 1}/3)...`);
    await new Promise(r => setTimeout(r, 1000));
  }
}
catWashTrace('CatLights', `STATE RESOLVED: ${lightState}`, { bodyState, bodyAction, method: bodyState === light...
console.log(`[Cat Lights] Light state: ${lightState} (body state: ${bodyState})`);
console.log(`[Cat Lights] Architecture: server-side TTS !' Google Nest speaker (media_player.play_media)`);

if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
  return res.status(500).json({ error: "Home Assistant not configured" });
}

const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
const appUrl = DEPLOYED_APP_URL;
const authParam = encodeURIComponent(process.env.SITE_PASSWORD || '');

// === LIGHT TURNED OFF !' Stop all playback + save progress ===
if (lightState === 'off') {
  catWashTrace('CatLights', 'LIGHT OFF - stopping all playback');
  console.log("[Cat Lights] Light off - stopping all playback and saving progress");
  const stopped: string[] = [];
  if (catWashPlaybackActive) {
    stopped.push(`playback:${catWashPlaybackState?.fileName || ''}`);
    await stopNestPlaybackWithGoodbye('light_off');
  }
  if (currentTTSsession) {
    console.log(`[Cat Lights] Stopping active TTS session (light off)`);
    stopTTSsession("Light turned off - stopping playback");
    stopped.push("ttsSession");
  }
  try {
    await stopAllCatWashroomSpeakers(haUrl);
    stopped.push("echoSpeakers");
  } catch (e: any) {
    console.warn(`[Cat Lights] Failed to stop Echo speakers: ${e.message}`);
  }
  await Promise.allSettled([
    haServiceCallSafe('media_player/turn_off', { entity_id: FIRE_STICK_ADB_ENTITY }, 'Stop TV FireStick'),
    haServiceCallSafe('media_player/turn_off', { entity_id: CAT_TV_ENTITY }, 'Stop TV Samsung'),
  ]);
  stopped.push("tv");
  console.log(`[Cat Lights] Fire Stick + Samsung TV turn-off sent`);
  catLightsPromptPending = false;
  catWashPlaybackTrigger = null;
  await clearPlaybackSession();
  console.log(`[Cat Lights] Stopped: ${stopped.join(', ')} || 'nothing was playing'`);
  return res.json({ action: "stopped", reason: "Light turned off", stoppedItems: stopped });
}

// === LIGHT TURNED ON !' Check if CPPA module needs playing ===
if (lightState !== 'on') {
  return res.json({ action: "ignored", reason: `Unknown state: ${lightState}` });
}

// Cooldown: skip if prompted within last 5 minutes
const bypassRequested = req.body?.bypass === true;
if (bypassRequested) {
  console.log(`[Cat Lights] Cooldown BYPASSED via request`);
  catLightsPromptPending = false;
}

if (catLightsPromptPending) {
  console.log(`[Cat Lights] Prompt already pending - skipping duplicate`);
  return res.json({ action: "skipped", reason: "Prompt already pending" });
}

const msSinceStop = Date.now() - lastPlaybackStoppedAt;
if (msSinceStop < 60000) {
  catWashTrace('CatLights', 'BLOCKED: post-stop cooldown', { msSinceStop });
  console.log(`[Cat Lights] Playback was stopped ${Math.round(msSinceStop / 1000)}s ago - skipping prompt (60s ...`);
  return res.json({ action: "skipped", reason: "Post-stop cooldown" });
}

if (catWashManuallyStoppedAt) {
  const msSinceManualStop = Date.now() - catWashManuallyStoppedAt.getTime();
  if (msSinceManualStop < 120000) {
    catWashTrace('CatLights', 'BLOCKED: manual-stop cooldown', { msSinceManualStop });
    console.log(`[Cat Lights] Manual stop was ${Math.round(msSinceManualStop / 1000)}s ago - skipping prompt (1...`);
    return res.json({ action: "skipped", reason: "Post-manual-stop cooldown" });
  }
}

if (catWashPlaybackActive && catWashPlaybackState) {
  const msSinceStart = catWashPlaybackStartedAt ? Date.now() - catWashPlaybackStartedAt.getTime() : 0;
  const chunkStillAtStart = catWashPlaybackState.chunkIndex === 0;
  const likelyStale = (msSinceStart > 3 * 60 * 1000 && chunkStillAtStart) || msSinceStart > 10 * 60 * 1000;

  if (likelyStale) {
    console.log(`[Cat Lights] Clearing stale playback state (started ${Math.round(msSinceStart / 1000)}s ago, c...`);
    catWashPlaybackActive = false;
    catWashPlaybackStartedAt = null;
    catWashPlaybackState = null;
  } else {
    console.log(`[Cat Lights] Already playing: "${catWashPlaybackState.fileName}" chunk ${catWashPlaybackState....`);
    return res.json({ action: "skipped", reason: "Playback already active", currentFile: catWashPlaybackState.f...
  }
}

try {
  const verifyResp = await fetch(`${haUrl}/api/states/${CAT_LIGHTS_ENTITY}`, { headers: haHeaders0 });
  if (verifyResp.ok) {

```

```

    const verifyData = await verifyResp.json();
    if (verifyData?.state === 'off') {
        console.log('[Cat Lights] Body said ON but HA confirms light is OFF – aborting');
        return res.json({ action: "ignored", reason: "Light actually off (verified)" });
    }
}
} catch {}

catLightsLastPromptAt = Date.now();
catLightsPromptPending = true;

res.json({ action: "processing", reason: "Light on – processing in background" });

const haHeaders = { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' };

const nowToronto = torontoDate();
const currentHour = nowToronto.getHours();
const currentMinute = nowToronto.getMinutes();
const inHARestartWindow = currentHour === 4 && currentMinute < 10;

if (inHARestartWindow) {
    const minutesLeft = 10 - currentMinute;
    console.log(`[Cat Lights] HA restart window detected (4:${String(currentMinute).padStart(2, '0')} AM) – waiti...

    try {
        await haServiceCall('tts/speak', {
            entity_id: HA_CLOUD_TTS_ENTITY,
            media_player_entity_id: CAT_WR_HA_VOICE_ENTITY,
            message: 'Home Assistant is currently restarting. I'll check your readings in about ${minutesLeft} minute...',
        }, 'Cat Lights HA Restart Notice');
        console.log(`[Cat Lights] HA restart notice played`);
    } catch (e: any) {
        console.warn(`[Cat Lights] HA restart notice TTS failed (expected during restart): ${e.message}`);
    }

    const msUntil410 = minutesLeft * 60 * 1000;
    await new Promise(r => setTimeout(r, msUntil410));

    try {
        const lightCheckResp = await fetch(`${haUrl0}/api/states/${CAT_LIGHTS_ENTITY}`, { headers: haHeaders0 });
        if (lightCheckResp.ok) {
            const lightCheckData = await lightCheckResp.json();
            if (lightCheckData?.state === 'off') {
                console.log(`[Cat Lights] Lights turned off during HA restart wait – aborting`);
                catLightsPromptPending = false;
                return;
            }
            console.log(`[Cat Lights] Lights still on after HA restart wait – proceeding`);
        }
    } catch (e: any) {
        console.warn(`[Cat Lights] Post-restart light check failed: ${e.message} – proceeding anyway`);
    }
}

const earlyDeviceWakePromise = (async () => {
    const tabletEntity = 'media_player.tablet_cat';
    try {
        await Promise.allSettled([
            haServiceCallSafe('androidtv/adb_command', { entity_id: tabletEntity, command: 'input keyevent KEYCODE_WA...
            haServiceCallSafe('androidtv/adb_command', { entity_id: tabletEntity, command: 'settings put system scree...
        ]);
        console.log(`[Cat Lights] Early device wake: tablet wake + brightness sent (TV deferred until confirmation)`);
    } catch (e: any) {
        console.warn(`[Cat Lights] Early device wake failed (non-fatal): ${e.message}`);
    }
})();

const immediatePromptPromise = (async () => {
    try {
        await Promise.allSettled([
            haServiceCallSafe('media_player/volume_set', { entity_id: CAT_WR_HA_VOICE_ENTITY, volume_level: 0.35 }, '...
            haServiceCallSafe('input_boolean/turn_off', { entity_id: MODULE_READING_CONFIRMED }, 'Cat Lights Bool'),
            haServiceCallSafe('input_boolean/turn_on', { entity_id: MODULE_READING_PENDING }, 'Cat Lights Bool'),
        ]);
        await haServiceCall('tts/speak', {
            entity_id: HA_CLOUD_TTS_ENTITY,
            media_player_entity_id: CAT_WR_HA_VOICE_ENTITY,
            message: "One moment, checking your readings."
        }, 'Cat Lights Quick TTS');
        console.log(`[Cat Lights] Quick acknowledgment played via HA Cloud TTS`);
    } catch (e: any) {
        console.warn(`[Cat Lights] Quick acknowledgment failed: ${e.message}`);
    }
})();

const today = torontoDate();
const semesterSettings = await storage.getActiveSemesterSettings();

if (!semesterSettings) {
    console.log(`[Cat Lights] No active semester – skipping prompt`);
    catLightsPromptPending = false;
    return;
}

let currentWeekNumber = 1;
const semStart = semesterSettings?.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : new Date(...)
const rwStart = semesterSettings?.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : new Date("20...
currentWeekNumber = getWeekNumber(today, semStart, rwStart);

const allFilesBefore = await storage.GetFiles();
let nextFile = findNextFileByPriority(allFilesBefore, currentWeekNumber);

if (!nextFile) {
    console.log(`[Cat Lights] No cached files found – syncing OneDrive for weeks 1-${currentWeekNumber}`);
    for (let w = 1; w <= currentWeekNumber; w++) {
        await syncOneDriveFilesForWeek(semesterSettings, w, '[Cat Lights]');
    }
    const allFilesAfter = await storage.GetFiles();

```

```

    nextFile = findNextFileByPriority(allFilesAfter, currentWeekNumber);
  } else {
    console.log(`[Cat Lights] Using cached file – syncing OneDrive in background`);
    syncOneDriveFilesForWeek(semesterSettings, currentWeekNumber, '[Cat Lights]').catch(e => console.log(`[Cat Li...
  }

await Promise.allSettled([immediatePromptPromise, earlyDeviceWakePromise]);

try {
  const lightCheckResp = await fetch(`${haUrl0}/api/states/${CAT_LIGHTS_ENTITY}`, { headers: haHeaders0 });
  if (lightCheckResp.ok) {
    const lightCheckData = await lightCheckResp.json();
    if (lightCheckData?.state === 'off') {
      console.log(`[Cat Lights] Light turned off during file lookup – aborting prompt`);
      catLightsPromptPending = false;
      return;
    }
  }
} catch {}

if (!nextFile) {
  console.log(`[Cat Lights] No unlisted files for weeks 1-${currentWeekNumber} – playing CHUM FM on Echo spea...
  catLightsPromptPending = false;
  await playChumFmRadio(haUrl);
  return;
}

const fileName = nextFile.displayName || nextFile.originalName || 'Unknown file';
const fileDesc = describeFileForTTS(nextFile, currentWeekNumber);
console.log(`[Cat Lights] Found next file: ${fileDesc} – ${fileName} (id=${nextFile.id})`);

const ttsMessage = `Would you like to play ${fileDesc}?`;
console.log(`[Cat Lights] Sending TTS prompt: "${ttsMessage}"`);
try {
  try {
    await stopAllCatWashroomSpeakers(haUrl);
    console.log(`[Cat Lights] Stopped any existing media before TTS prompt`);
  } catch (e: any) {
    console.warn(`[Cat Lights] Pre-prompt media stop error (non-fatal): ${e.message}`);
  }

  await haServiceCallSafe('media_player/volume_set', { entity_id: NEST_SPEAKER_ENTITY, volume_level: 0.35 }, 'C...

  let ttsPlayed = false;
  try {
    await haServiceCall('tts/speak', {
      entity_id: HA_CLOUD_TTS_ENTITY,
      media_player_entity_id: CAT_WR_HA_VOICE_ENTITY,
      message: ttsMessage
    }, 'Cat Lights Cloud TTS');
    ttsPlayed = true;
    console.log(`[Cat Lights] TTS prompt played via HA Cloud TTS`);
  } catch (e: any) {
    console.warn(`[Cat Lights] HA Cloud TTS failed: ${e.message} – trying Edge TTS fallback`);
    try {
      const audioPath = await generateAndSaveTTSAudio(ttsMessage, `cat-lights-prompt-${Date.now()}`);
      const appUrl2 = DEPLOYED_APP_URL;
      await haServiceCall('media_player/play_media', { entity_id: CAT_WR_HA_VOICE_ENTITY, media_content_id: `${...
      ttsPlayed = true;
      console.log(`[Cat Lights] TTS prompt played via Edge TTS on HA Voice`);
    } catch (e2: any) {
      console.error(`[Cat Lights] All TTS methods failed: ${e2.message}`);
    }
  }
}
if (!ttsPlayed) {
  console.error(`[Cat Lights] Could not play TTS prompt after all retries – falling back to CHUM FM`);
  catLightsPromptPending = false;
  await playChumFmRadio(haUrl);
  return;
}
} catch (e: any) {
  console.error(`[Cat Lights] Critical failure in prompt setup: ${e.message} – falling back to CHUM FM`);
  catLightsPromptPending = false;
  await playChumFmRadio(haUrl);
  return;
}

await new Promise(r => setTimeout(r, 2000));

try {
  const lightCheck2 = await fetch(`${haUrl0}/api/states/${CAT_LIGHTS_ENTITY}`, { headers: haHeaders0 });
  if (lightCheck2.ok) {
    const ld2 = await lightCheck2.json();
    if (ld2?.state === 'off') {
      console.log(`[Cat Lights] Light turned off during confirmation wait – aborting`);
      catLightsPromptPending = false;
      await Promise.allSettled([
        haServiceCallSafe('input_boolean/turn_off', { entity_id: MODULE_READING_PENDING }, 'Cat Lights Bool'),
        haServiceCallSafe('input_boolean/turn_off', { entity_id: MODULE_READING_CONFIRMED }, 'Cat Lights Bool'),
      ]);
      return;
    }
  }
} catch {}

{
  const maxWaitMs = 23000;
  console.log(`[Cat Lights] Waiting up to ${maxWaitMs / 1000}s for confirmation (webhook-primary, backup poll e...

  const confirmed = await new Promise<boolean>((resolve) => {
    let resolved = false;
    const finish = (val: boolean) => { if (!resolved) { resolved = true; clearTimeout(timeout); clearInterval(b...
    const timeout = setTimeout(() => finish(false), maxWaitMs);
    catLightsConfirmResolve = () => finish(true);
    const boolPoll = setInterval(async () => {
      if (!catLightsPromptPending) {
        finish(false);
        return;
      }
    }, 1000);
  });
}

```

```

    }
    try {
      const resp = await haFetch(`${haUrl}/api/states/${MODULE_READING_CONFIRMED}`, { headers: { 'Authorizati...
      const data = await resp.json();
      if (data.state === 'on') {
        console.log('[Cat Lights] Confirmation received via backup poll');
        finish(true);
      }
    } catch {}
  }, 10000);
});

await Promise.allSettled([
  haServiceCallSafe('input_boolean/turn_off', { entity_id: MODULE_READING_PENDING }, 'Cat Lights Bool'),
  haServiceCallSafe('input_boolean/turn_off', { entity_id: MODULE_READING_CONFIRMED }, 'Cat Lights Bool'),
]);

if (!confirmed) {
  catLightsPromptPending = false;
  if (catWashPlaybackActive) {
    console.log('[Cat Lights] No confirmation received but CPPA playback already active – skipping CHUM FM');
    return;
  }
  console.log('[Cat Lights] No confirmation received – playing CHUM FM on Echo speakers');
  await playChumFmRadio(haUrl);
  return;
}

try {
  const lightCheck3 = await fetch(`${haUrl0}/api/states/${CAT_LIGHTS_ENTITY}`, { headers: haHeaders0 });
  if (lightCheck3.ok) {
    const ld3 = await lightCheck3.json();
    if (ld3?.state === 'off') {
      console.log('[Cat Lights] Light turned off after confirmation – aborting playback start');
      catLightsPromptPending = false;
      return;
    }
  }
} catch {}

catWashTrace('CatLights', 'CONFIRMATION RECEIVED – starting playback');
console.log('[Cat Lights] Confirmation received – starting playback');
catLightsPromptPending = false;

const confirmTTS = `Okay, I will now play ${fileDesc}.`;
catWashPlaybackTrigger = 'lights';
await startConfirmedPlaybackFlow(nextFile, '[Cat Lights]', 'echo', confirmTTS);
return;
}

} catch (error: any) {
  console.error("[Cat Lights] Error:", error);
  res.status(500).json({ error: "Failed to handle cat lights webhook", details: error.message });
}
});

// POST /api/webhook/cat-wash-stop - Triggered when toothbrush starts running (idle/charging != running)
// Stops cat wash playback and saves progress.
app.post("/api/webhook/cat-wash-stop", async (req, res) => {
  try {
    catWashTrace('CatWashStopWebhook', 'WEBHOOK TRIGGERED', { body: req.body });
    console.log('[Cat Wash Stop Webhook] ===== WEBHOOK TRIGGERED =====');
    console.log('[Cat Wash Stop Webhook] Timestamp: ${new Date().toISOString()}');
    console.log('[Cat Wash Stop Webhook] Request body: ${JSON.stringify(req.body)}');

    const stopped: string[] = [];
    const fileName = catWashPlaybackState?.fileName || '';

    if (catWashPlaybackActive) {
      stopped.push(`playback:${fileName}`);
      await stopNestPlaybackWithGoodbye(req.body?.trigger || 'toothbrush', !!req.body?.keepOpen);
    }

    if (currentTTSSession) {
      console.log('[Cat Wash Stop Webhook] Stopping active TTS session');
      stopTTSSession("Toothbrush started running - stopping playback");
      stopped.push("ttsSession");
    }

    try {
      const haUrlStop = HOME_ASSISTANT_URL.replace(/\/$/, '');
      await stopAllCatWashroomSpeakers(haUrlStop);
      stopped.push("echoSpeakers");
    } catch (e: any) {
      console.warn('[Cat Wash Stop Webhook] Failed to stop Echo speakers: ${e.message}');
    }

    await Promise.allSettled([
      haServiceCallSafe('media_player/turn_off', { entity_id: FIRE_STICK_ADB_ENTITY }, 'Stop TV FireStick'),
      haServiceCallSafe('media_player/turn_off', { entity_id: CAT_TV_ENTITY }, 'Stop TV Samsung!'),
    ]);
    stopped.push("tv");
    console.log('[Cat Wash Stop Webhook] Fire Stick + Samsung TV turn-off sent');

    catWashPlaybackTrigger = null;
    console.log('[Cat Wash Stop Webhook] Stopped: ${stopped.join(', ') || 'nothing was playing'}');
    res.json({ action: "stopped", stoppedItems: stopped });

  } catch (error: any) {
    console.error("[Cat Wash Stop Webhook] Error:", error);
    res.status(500).json({ error: "Failed to handle stop webhook", details: error.message });
  }
});

// POST /api/webhook/kitchen-volume - Control Kitchen Echo Studio volume (fire-and-forget for HA automations)
app.post("/api/webhook/kitchen-volume", async (req, res) => {
  try {

```

```

const { direction, speed } = req.body || {};
console.log(`[Kitchen Volume] ===== WEBHOOK TRIGGERED ===== direction=${direction} speed=${speed}`);

res.json({ success: true, direction, speed });

const baseUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
const haHeaders = { 'Authorization': 'Bearer ${HOME_ASSISTANT_TOKEN}', 'Content-Type': 'application/json' };
const entity = KITCHEN_ECHO_ENTITY;

const stateResp = await fetch(`${baseUrl}/api/states/${entity}`, { headers: haHeaders });
const stateData = stateResp.ok ? await stateResp.json() : null;
const currentVolume = stateData?.attributes?.volume_level ?? 0.5;

const step = speed === 'fast' ? 0.15 : 0.05;
let newVolume: number;

if (direction === 'up') {
  newVolume = Math.min(1, currentVolume + step);
} else {
  newVolume = Math.max(0, currentVolume - step);
}

await fetch(`${baseUrl}/api/services/media_player/volume_set`, {
  method: 'POST', headers: haHeaders,
  body: JSON.stringify({ entity_id: entity, volume_level: newVolume }),
});
console.log(`[Kitchen Volume] Set volume: ${currentVolume} !' ${newVolume} (${direction}, ${speed})`);
} catch (err: any) {
  console.error(`[Kitchen Volume] Error: ${err.message}`);
}
});

app.post("/api/webhook/cat-volume", async (req, res) => {
  try {
    const { direction, speed } = req.body || {};
    console.log(`[Cat Volume] ===== WEBHOOK TRIGGERED ===== direction=${direction} speed=${speed}`);

    res.json({ success: true, direction, speed });

    const baseUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
    const haHeaders = { 'Authorization': 'Bearer ${HOME_ASSISTANT_TOKEN}', 'Content-Type': 'application/json' };

    const allCatEntities = [
      NEST_SPEAKER_ENTITY,
      CAT_WR_MEDIA_GROUP,
    ];

    const activeEntities: string[] = [];
    await Promise.all(allCatEntities.map(async (entityId) => {
      try {
        const resp = await fetch(`${baseUrl}/api/states/${entityId}`, { headers: haHeaders });
        if (resp.ok) {
          const data = await resp.json();
          if (data.state === 'playing' || data.state === 'paused' || data.state === 'buffering') {
            activeEntities.push(entityId);
          }
        }
      } catch {}
    }));

    if (activeEntities.length === 0) {
      activeEntities.push(NEST_SPEAKER_ENTITY);
      console.log(`[Cat Volume] No active speakers found - defaulting to Nest`);
    }

    const step = speed === 'fast' ? 0.15 : 0.05;

    for (const entityId of activeEntities) {
      const stateResp = await fetch(`${baseUrl}/api/states/${entityId}`, { headers: haHeaders });
      const stateData = stateResp.ok ? await stateResp.json() : null;
      const currentVolume = stateData?.attributes?.volume_level ?? 0.5;

      let newVolume: number;
      if (direction === 'up') {
        newVolume = Math.min(1, currentVolume + step);
      } else {
        newVolume = Math.max(0, currentVolume - step);
      }

      await fetch(`${baseUrl}/api/services/media_player/volume_set`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: entityId, volume_level: newVolume }),
      });
      console.log(`[Cat Volume] ${entityId}: ${currentVolume} !' ${newVolume} (${direction}, ${speed})`);
      lastVolumeChange = { volume: Math.round(newVolume * 100), direction, timestamp: Date.now() };
    }

    console.log(`[Cat Volume] Adjusted ${activeEntities.length} active speaker(s)`);
  } catch (err: any) {
    console.error(`[Cat Volume] Error: ${err.message}`);
  }
});

// POST /api/webhook/cat-knob-stop - STOP button: stops all playback on cat washroom speakers + Nest
app.post("/api/webhook/cat-knob-stop", async (req, res) => {
  try {
    catWashTrace('KnobPress', 'STOP BUTTON PRESSED', { body: req.body });
    console.log(`[Cat Knob] ===== KNOB PRESS RECEIVED (STOP BUTTON) =====`);
    console.log(`[Cat Knob] Body: ${JSON.stringify(req.body)}`);
    const baseUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

    if (catWashPlaybackActive) {
      console.log(`[Cat Knob] Playback active - stopping via cat-wash-stop`);
      try {
        await fetch(`http://localhost:${process.env.PORT || 5000}/api/webhook/cat-wash-stop`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ trigger: 'knob_press', keepOpen: true }),
        });
      }
    }
  }
});

```

```

    });
  } catch (e: any) {
    console.error(`[Cat Knob] Error calling stop: ${e.message}`);
  }
  res.json({ success: true, action: 'stopped' });
  return;
}

await stopAllCatWashroomSpeakers(haUrl);
console.log(`[Cat Knob] Stopped all cat washroom speakers (Nest + Echos)`);
res.json({ success: true, action: 'stopped' });
} catch (err: any) {
  console.error(`[Cat Knob] Error: ${err.message}`);
  res.status(500).json({ error: err.message });
}
});

// GET /api/cat-wash/progress - Returns current playback state for the active session
app.get("/api/cat-wash/progress", (_req, res) => {
  if (catWashPlaybackActive && catWashPlaybackState && catWashPlaybackStartedAt) {
    const msSinceStart = Date.now() - catWashPlaybackStartedAt.getTime();
    const chunkStillAtStart = catWashPlaybackState.chunkIndex === 0;
    if (msSinceStart > 10 * 60 * 1000 && chunkStillAtStart) {
      console.log(`[Cat Wash Progress] Auto-clearing stale state (started ${Math.round(msSinceStart / 1000)}s ago, ...`);
      catWashPlaybackActive = false;
      catWashPlaybackStartedAt = null;
      catWashPlaybackState = null;
      return res.json({ active: false });
    }
  }

  if (!catWashPlaybackActive || !catWashPlaybackState) {
    return res.json({ active: false });
  }

  const state = catWashPlaybackState;
  const volumeEvent = lastVolumeChange && (Date.now() - lastVolumeChange.timestamp < 5000) ? lastVolumeChange : null;
  res.json({
    active: true,
    fileId: state.fileId,
    fileName: state.fileName,
    chunkIndex: state.chunkIndex,
    totalChunks: state.totalChunks,
    chunkText: state.chunks[state.chunkIndex] || '',
    words: state.currentWords,
    wordIndex: state.wordIndex || 0,
    volumeChange: volumeEvent,
  });
});

// POST /api/cat-wash/update-progress - Tablet reports its playback progress
app.post("/api/cat-wash/update-progress", async (req, res) => {
  const { fileId, chunkIndex, totalChunks, words, wordIndex, completed } = req.body;

  if (catWashPlaybackState && catWashPlaybackState.fileId === fileId) {
    catWashPlaybackState.chunkIndex = chunkIndex ?? catWashPlaybackState.chunkIndex;
    catWashPlaybackState.totalChunks = totalChunks ?? catWashPlaybackState.totalChunks;
    catWashPlaybackState.currentWords = words ?? catWashPlaybackState.currentWords;
    catWashPlaybackState.wordIndex = wordIndex ?? catWashPlaybackState.wordIndex;
    catWashPlaybackState.chunkStartedAt = new Date();
    if (req.body.chunkText && chunkIndex != null) {
      catWashPlaybackState.chunks[chunkIndex] = req.body.chunkText;
    }
  } else if (fileId && !catWashPlaybackActive) {
    console.log(`[Cat Wash] Re-activating playback state from tablet progress report (fileId=${fileId})`);
    catWashPlaybackActive = true;
    catWashPlaybackStartedAt = new Date();
    startToothbrushPolling();
    const file = await storage.getFile(fileId);
    catWashPlaybackState = {
      fileId,
      fileName: file?.displayName || file?.originalName || 'Unknown',
      chunkIndex: chunkIndex ?? 0,
      totalChunks: totalChunks ?? 0,
      chunks: [],
      currentWords: words || [],
      wordIndex: wordIndex ?? 0,
      startedAt: new Date(),
      chunkStartedAt: new Date(),
      estimatedChunkDuration: 0,
    };
  }

  if (fileId && chunkIndex != null) {
    await storage.updateFile(fileId, { lastChunkIndex: chunkIndex, totalChunks: totalChunks ?? undefined });
  }

  // When the tablet signals file completion, find and return the next file
  if (completed && fileId) {
    console.log(`[Cat Wash] Tablet reports file ${fileId} complete`);
    await storage.updateFile(fileId, { listened: true });

    const semesterSettings = await storage.getActiveSemesterSettings();
    let currentWeekNumber = 1;
    const semStart = semesterSettings?.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : new Date(...);
    const rwStart = semesterSettings?.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : new Date("20...");
    currentWeekNumber = getWeekNumber(torontoDate(), semStart, rwStart);

    const nextFile = await findNextCatWashFile(storage, currentWeekNumber, fileId);
    if (nextFile) {
      const appUrl = DEPLOYED_APP_URL;
      const authParam = encodeURIComponent(process.env.SITE_PASSWORD || '');
      const nextReaderUrl = `${appUrl}/pdf-reader/${nextFile.id}?catWashFollow=true&autoplay=true&auth=${authParam}`;
      const nextFileName = nextFile.displayName || nextFile.originalName || 'Unknown file';

      catWashPlaybackState = {
        fileId: nextFile.id,
        fileName: nextFileName,
      };
    }
  }
});

```

```

        chunkIndex: 0,
        totalChunks: 0,
        chunks: [],
        currentWords: [],
        wordIndex: 0,
        startedAt: new Date(),
        chunkStartedAt: new Date(),
        estimatedChunkDuration: 0,
    };

    console.log(`[Cat Wash] Next file: ${nextFileName} (id=${nextFile.id})`);
    return res.json({ nextFile: { id: nextFile.id, name: nextFileName, readerUrl: nextReaderUrl } });
} else {
    catWashPlaybackActive = false;
    catWashPlaybackStartedAt = null;
    catWashPlaybackState = null;
    console.log("[Cat Wash] All files complete");
    return res.json({ allComplete: true });
}
}

res.json({ ok: true });
});

// POST /api/cat-wash/stop - Stop ALL playback (cat wash, cat lights, TTS sessions, all echo devices)
app.post("/api/cat-wash/stop", async (req, res) => {
    const keepOpen = req.body?.keepOpen === true;
    catWashTrace('CatWashStop', 'STOP ALL PLAYBACK keepOpen=${keepOpen}');
    console.log('[Cat Wash Stop] === STOP ALL PLAYBACK === (keepOpen=${keepOpen})');

    clearVoiceCommandPause();
    const stopped: string[] = [];

    if (catWashPlaybackActive && catWashPlaybackState) {
        const { fileId, chunkIndex, fileName } = catWashPlaybackState;
        console.log(`[Cat Wash Stop] Stopping cat wash playback (file: ${fileName}, chunk: ${chunkIndex})`);
        if (fileId && chunkIndex !== null) {
            try {
                await storage.updateFile(fileId, { lastChunkIndex: chunkIndex });
                console.log(`[Cat Wash Stop] Saved progress: file ${fileId}, chunk ${chunkIndex}`);
            } catch (e: any) {
                console.error(`[Cat Wash Stop] Failed to save progress: ${e.message}`);
            }
        }
        stopped.push("catWashPlayback");
    }
    catWashPlaybackActive = false;
    catWashPlaybackStartedAt = null;
    catWashPlaybackState = null;
    catWashManuallyStoppedAt = new Date();
    lastPlaybackStoppedAt = Date.now();
    currentTvFollowUrl = null;
    currentTabletReaderUrl = null;
    stopToothbrushPolling();
    await clearPlaybackSession();

    if (currentTTSSession) {
        console.log(`[Cat Wash Stop] Stopping active TTS session (entity: ${currentTTSSession.targetEntity})`);
        stopTTSSession("Force stopped via cat-wash/stop");
        stopped.push("ttsSession");
    }

    if (nestPlaybackAbort) {
        nestPlaybackAbort();
        nestPlaybackAbort = null;
        stopped.push("nestPlaybackAbort");
    }

    try {
        const baseUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
        await fetch(`${baseUrl}/api/services/media_player/media_stop`, {
            method: 'POST',
            headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
            body: JSON.stringify({ entity_id: NEST_SPEAKER_ENTITY }),
        });
        console.log(`[Cat Wash Stop] Sent media_stop to Nest speaker`);
        stopped.push("nestSpeaker");

        const apiUrl = DEPLOYED_APP_URL;
        try {
            const stopConfirmPath = await generateAndSaveTTSAudio("Stop received.", `cat-wash-stop-confirm-${Date.now()}`);
            await playOnNestSpeaker(`${apiUrl}${stopConfirmPath}`);
            console.log(`[Cat Wash Stop] Played "Stop received" on Nest`);
        } catch (e2: any) {
            console.log(`[Cat Wash Stop] Stop confirm TTS error: ${e2.message}`);
        }
    } catch (e: any) {
        console.error(`[Cat Wash Stop] Failed to stop Nest speaker: ${e.message}`);
    }

    if (!keepOpen) {
        const stopTs = Date.now();
        await Promise.all([
            setTabletCommand({ action: 'stop_playback', timestamp: stopTs }, true, 'master'),
            setTabletCommand({ action: 'stop_playback', timestamp: stopTs }, true, 'tv'),
        ]);
    }

    try {
        const baseUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
        await fetch(`${baseUrl}/api/services/media_player/turn_off`, {
            method: 'POST',
            headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
            body: JSON.stringify({ entity_id: FIRE_STICK_ADB_ENTITY }),
        });
        await fetch(`${baseUrl}/api/services/media_player/turn_off`, {
            method: 'POST',
            headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
            body: JSON.stringify({ entity_id: CAT_TV_ENTITY }),
        });
    }

```



```

    });
    console.log(`[Cat Wash Stop] TV turned off`);
    stopped.push("tv");
  } catch (e: any) {
    console.log(`[Cat Wash Stop] TV turn off error: ${e.message}`);
  }
} else {
  console.log(`[Cat Wash Stop] keepOpen=true - tablet/TV stay on reader`);
}

stopWordAdvancement();
console.log(`[Cat Wash Stop] Stopped: ${stopped.join(', ')}`);
res.json({ stopped: true, stoppedItems: stopped });
});

app.post("/api/webhook/voice-command", async (req, res) => {
  const command = (req.body?.command || '').toLowerCase().trim();
  console.log(`[Voice Command] ===== WEBHOOK TRIGGERED ===== command="${command}"`);
  console.log(`[Voice Command] Timestamp: ${new Date().toLocaleString('en-US', { timeZone: 'America/Toronto' })}`);

  if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
    return res.status(500).json({ error: "Home Assistant not configured" });
  }

  const baseUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
  const apiUrl = DEPLOYED_APP_URL;
  const headers = { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' };

  try {
    if (command === 'pause') {
      if (!catWashPlaybackActive || !catWashPlaybackState) {
        console.log(`[Voice Command] Pause requested but no active playback`);
        try {
          const noPlayPath = await generateAndSaveTTSAudio("Nothing is playing right now.", `vc-no-play-${Date.now(...)}`);
          await playOnNestSpeaker(`${apiUrl}${noPlayPath}`);
        } catch {}
        return res.json({ action: "ignored", reason: "No active playback" });
      }

      const { fileId, chunkIndex, fileName } = catWashPlaybackState;
      console.log(`[Voice Command] Pausing: "${fileName}" at chunk ${chunkIndex}`);

      if (nestPlaybackAbort) {
        nestPlaybackAbort();
        nestPlaybackAbort = null;
      }
      await stopNestSpeaker();
      stopWordAdvancement();

      if (fileId && chunkIndex !== null) {
        try {
          await storage.updateFile(fileId, { lastChunkIndex: chunkIndex });
          console.log(`[Voice Command] Saved progress: file ${fileId}, chunk ${chunkIndex}`);
        } catch (e: any) {
          console.error(`[Voice Command] Failed to save progress: ${e.message}`);
        }
      }
    }

    catWashPlaybackActive = false;
    catWashPlaybackStartedAt = null;
    const savedState = { ...catWashPlaybackState };
    catWashPlaybackState = null;
    stopToothbrushPolling();

    const PAUSE_TIMEOUT_MS = 10 * 60 * 1000;
    clearVoiceCommandPause_();
    const autoStopTimer = setTimeout(async () => {
      console.log(`[Voice Command] 10-minute pause timeout - auto-stopping`);
      voiceCommandPauseState_ = null;
      await clearPlaybackSession();

      const stopTimestamp = Date.now();
      await Promise.all([
        setTabletCommand({ action: 'stop_playback', goodbyeText: '', timestamp: stopTimestamp }, true, 'master'),
        setTabletCommand({ action: 'stop_playback', timestamp: stopTimestamp }, true, 'tv'),
      ]);

      try {
        await fetch(`${baseUrl}/api/services/media_player/turn_off`, {
          method: 'POST', headers: headers,
          body: JSON.stringify({ entity_id: FIRE_STICK_ADB_ENTITY }),
        });
        await fetch(`${baseUrl}/api/services/media_player/turn_off`, {
          method: 'POST', headers: headers,
          body: JSON.stringify({ entity_id: CAT_TV_ENTITY }),
        });
      } catch {}

      try {
        const timeoutPath = await generateAndSaveTTSAudio("Pause timed out. Playback has been stopped. Your progr...");
        await playOnNestSpeaker(`${apiUrl}${timeoutPath}`);
      } catch {}
    }, PAUSE_TIMEOUT_MS);

    voiceCommandPauseState_ = {
      fileId: savedState.fileId,
      chunkIndex: savedState.chunkIndex,
      fileName: savedState.fileName,
      pausedAt: new Date(),
      autoStopTimer,
    };

    savePlaybackSession({
      fileId: savedState.fileId,
      fileName: savedState.fileName,
      chunkIndex: savedState.chunkIndex,
      totalChunks: savedState.totalChunks || 0,
      trigger: catWashPlaybackTrigger || 'manual',
    });
  }
});

```

```

        startedAt: catWashPlaybackStartedAt?.toISOString() || new Date().toISOString(),
        updatedAt: new Date().toISOString(),
        status: 'paused',
    }).catch(() => {});

    try {
        const pausePath = await generateAndSaveTTSAudio("Paused. Say resume to continue, or I'll stop in 10 minutes...");
        await playOnNestSpeaker(`${appUrl}${pausePath}`);
    } catch {}

    return res.json({ action: "paused", file: fileName, chunk: chunkIndex });

} else if (command === 'resume') {
    if (!voiceCommandPauseState_) {
        console.log(`[Voice Command] Resume requested but nothing is paused`);
        try {
            const noPausePath = await generateAndSaveTTSAudio("Nothing is paused right now.", `vc-no-pause-${Date.now()}`);
            await playOnNestSpeaker(`${appUrl}${noPausePath}`);
        } catch {}
        return res.json({ action: "ignored", reason: "Nothing paused" });
    }

    const { fileId, fileName } = voiceCommandPauseState_;
    console.log(`[Voice Command] Resuming: "${fileName}" (fileId=${fileId})`);
    clearVoiceCommandPause_();

    const file = await storage.getFile(fileId);
    if (!file) {
        console.log(`[Voice Command] File ${fileId} not found`);
        try {
            const notFoundPath = await generateAndSaveTTSAudio("The file could not be found.", `vc-notfound-${Date.now()}`);
            await playOnNestSpeaker(`${appUrl}${notFoundPath}`);
        } catch {}
        return res.json({ action: "error", reason: "File not found" });
    }

    try {
        const echoEntities = CAT_ECHO_ENTITIES;
        await fetch(`${haUrl}/api/services/media_player/media_stop`, {
            method: 'POST', headers: haHeaders,
            body: JSON.stringify({ entity_id: echoEntities }),
        });
        console.log(`[Voice Command] Cleared Echo speakers before resume`);
    } catch (e: any) {
        console.warn(`[Voice Command] Echo clear error (non-fatal): ${e.message}`);
    }

    const fileDesc = describeFileForTTS(file, 0).replace(/ for week 0$/, '');
    const confirmTTS = `Resuming ${fileDesc}`;
    catWashPlaybackTrigger = catWashPlaybackTrigger || 'manual';
    catLightsPromptPending = false;

    res.json({ action: "resuming", file: fileName, fileId });

    await startConfirmedPlaybackFlow(file, 'Voice Resume', 'echo', confirmTTS);
    return;

} else if (command === 'stop') {
    const wasPaused = !!voiceCommandPauseState_;
    clearVoiceCommandPause_();
    await clearPlaybackSession();

    if (!catWashPlaybackActive && !wasPaused) {
        console.log(`[Voice Command] Stop requested but nothing is playing or paused`);
        try {
            const noPlayPath = await generateAndSaveTTSAudio("Nothing is playing.", `vc-nostop-${Date.now()}`);
            await playOnNestSpeaker(`${appUrl}${noPlayPath}`);
        } catch {}
        return res.json({ action: "ignored", reason: "Nothing playing or paused" });
    }

    console.log(`[Voice Command] Stopping all playback`);

    if (catWashPlaybackActive) {
        await stopNestPlaybackWithGoodbye('voice_command_stop');
    } else {
        const stopTimestamp = Date.now();
        await Promise.all([
            setTabletCommand({ action: 'stop_playback', goodbyeText: '', timestamp: stopTimestamp }, true, 'master'),
            setTabletCommand({ action: 'stop_playback', timestamp: stopTimestamp }, true, 'tv'),
        ]);

        try {
            await fetch(`${haUrl}/api/services/media_player/turn_off`, {
                method: 'POST', headers: haHeaders,
                body: JSON.stringify({ entity_id: FIRE_STICK_ADB_ENTITY }),
            });
            await fetch(`${haUrl}/api/services/media_player/turn_off`, {
                method: 'POST', headers: haHeaders,
                body: JSON.stringify({ entity_id: CAT_TV_ENTITY }),
            });
        } catch {}

        try {
            const stopPath = await generateAndSaveTTSAudio("Stopped. Your progress has been saved. See you next time ...");
            await playOnNestSpeaker(`${appUrl}${stopPath}`);
        } catch {}
    }

    if (currentTTSSession) {
        stopTTSSession("Voice command stop");
    }

    return res.json({ action: "stopped" });

} else if (command === 'restart' || command === 'go_back') {
    let targetFileId: number | null = null;
    let targetChunk: number = 0;

```

```

let targetFileName: string = '';

if (catWashPlaybackActive && catWashPlaybackState) {
  targetFileId = catWashPlaybackState.fileId;
  targetChunk = Math.max(0, catWashPlaybackState.chunkIndex - 1);
  targetFileName = catWashPlaybackState.fileName;

  if (nestPlaybackAbort) {
    nestPlaybackAbort();
    nestPlaybackAbort = null;
  }
  await stopNestSpeaker();
  stopWordAdvancement();
  catWashPlaybackActive = false;
  catWashPlaybackStartedAt = null;
  catWashPlaybackState = null;
  stopToothbrushPolling();
} else if (voiceCommandPauseState_) {
  targetFileId = voiceCommandPauseState_.fileId;
  targetChunk = Math.max(0, voiceCommandPauseState_.chunkIndex - 1);
  targetFileName = voiceCommandPauseState_.fileName;
  clearVoiceCommandPause_();
}

if (!targetFileId) {
  console.log(`[Voice Command] Restart requested but no active/paused playback`);
  try {
    const noPath = await generateAndSaveTTSAudio("Nothing is playing to restart.", `vc-norestart-${Date.now()}...`);
    await playOnNestSpeaker(`${appUrl}${noPath}`);
  } catch {}
  return res.json({ action: "ignored", reason: "Nothing to restart" });
}

console.log(`[Voice Command] Restart: "${targetFileName}" going back to chunk ${targetChunk}`);

try {
  await storage.updateFile(targetFileId, { lastChunkIndex: targetChunk });
} catch (e: any) {
  console.error(`[Voice Command] Failed to update chunk: ${e.message}`);
}

const file = await storage.getFile(targetFileId);
if (!file) {
  return res.json({ action: "error", reason: "File not found" });
}

try {
  const echoEntities = CAT_ECHO_ENTITIES;
  await fetch(`${haUrl}/api/services/media_player/media_stop`, {
    method: 'POST', headers: haHeaders,
    body: JSON.stringify({ entity_id: echoEntities }),
  });
} catch {}

const confirmTTS = `Going back. Restarting from an earlier section.`;
catWashPlaybackTrigger = catWashPlaybackTrigger || 'manual';
catLightsPromptPending = false;

res.json({ action: "restarting", file: targetFileName, fromChunk: targetChunk });

await startConfirmedPlaybackFlow(file, '[Voice Restart]', 'echo', confirmTTS);
return;

} else if (command === 'reset') {
  let targetFileId: number | null = null;
  let targetFileName: string = '';

  if (catWashPlaybackActive && catWashPlaybackState) {
    targetFileId = catWashPlaybackState.fileId;
    targetFileName = catWashPlaybackState.fileName;

    if (nestPlaybackAbort) {
      nestPlaybackAbort();
      nestPlaybackAbort = null;
    }
    await stopNestSpeaker();
    stopWordAdvancement();
    catWashPlaybackActive = false;
    catWashPlaybackStartedAt = null;
    catWashPlaybackState = null;
    stopToothbrushPolling();
  } else if (voiceCommandPauseState_) {
    targetFileId = voiceCommandPauseState_.fileId;
    targetFileName = voiceCommandPauseState_.fileName;
    clearVoiceCommandPause_();
  }

  if (!targetFileId) {
    console.log(`[Voice Command] Reset requested but no active/paused playback`);
    try {
      const noPath = await generateAndSaveTTSAudio("Nothing is playing to reset.", `vc-noreset-${Date.now()}`);
      await playOnNestSpeaker(`${appUrl}${noPath}`);
    } catch {}
    return res.json({ action: "ignored", reason: "Nothing to reset" });
  }

  console.log(`[Voice Command] Reset: "${targetFileName}" back to chunk 0`);

  try {
    await storage.updateFile(targetFileId, { lastChunkIndex: 0, checkedChunks: '[]' });
  } catch (e: any) {
    console.error(`[Voice Command] Failed to reset file: ${e.message}`);
  }

  const file = await storage.getFile(targetFileId);
  if (!file) {
    return res.json({ action: "error", reason: "File not found" });
  }
}

```

```

try {
  const echoEntities = CAT_ECHO_ENTITIES;
  await fetch(`${haUrl}/api/services/media_player/media_stop`, {
    method: 'POST', headers: haHeaders,
    body: JSON.stringify({ entity_id: echoEntities }),
  });
} catch {}

const fileDesc = describeFileForTTS(file, 0).replace(/ for week 0$/, '');
const confirmTTS = `Resetting ${fileDesc}. Starting from the beginning.`;
catWashPlaybackTrigger = catWashPlaybackTrigger || 'manual';
catLightsPromptPending = false;

res.json({ action: "resetting", file: targetFileName });

await startConfirmedPlaybackFlow(file, ['Voice Reset'], 'echo', confirmTTS);
return;

} else if (command === 'skip') {
  let currentFileId: number | null = null;
  let currentFileName: string = '';

  if (catWashPlaybackActive && catWashPlaybackState) {
    currentFileId = catWashPlaybackState.fileId;
    currentFileName = catWashPlaybackState.fileName;

    if (nestPlaybackAbort) {
      nestPlaybackAbort();
      nestPlaybackAbort = null;
    }
    await stopNestSpeaker();
    stopWordAdvancement();
    catWashPlaybackActive = false;
    catWashPlaybackStartedAt = null;
    catWashPlaybackState = null;
    stopToothbrushPolling();
  } else if (voiceCommandPauseState_) {
    currentFileId = voiceCommandPauseState_.fileId;
    currentFileName = voiceCommandPauseState_.fileName;
    clearVoiceCommandPause_();
  }

  if (!currentFileId) {
    console.log(`[Voice Command] Skip requested but no active/paused playback`);
    try {
      const noPath = await generateAndSaveTTSAudio("Nothing is playing to skip.", `vc-noskip-${Date.now()}`);
      await playOnNestSpeaker(`${appUrl}${noPath}`);
    } catch {}
    return res.json({ action: "ignored", reason: "Nothing to skip" });
  }

  console.log(`[Voice Command] Skip: marking "${currentFileName}" as listened and finding next`);

  try {
    await storage.updateFile(currentFileId, { listened: true });
  } catch (e: any) {
    console.error(`[Voice Command] Failed to mark listened: ${e.message}`);
  }

  const semesterSettings = await storage.getActiveSemesterSettings();
  const semStart = semesterSettings?.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : new Date...
  const rwStart = semesterSettings?.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : new Date("...
  const currentWeekNumber = getWeekNumber(torontoDate(), semStart, rwStart);

  const allFiles = await storage.getFiles();
  const nextFile = findNextFileByPriority(allFiles, currentWeekNumber, currentFileId);

  if (!nextFile) {
    console.log(`[Voice Command] No more files for week ${currentWeekNumber}`);

    const stopTimestamp = Date.now();
    await Promise.all([
      setTabletCommand({ action: 'stop_playback', goodbyeText: '', timestamp: stopTimestamp }, true, 'master'),
      setTabletCommand({ action: 'stop_playback', timestamp: stopTimestamp }, true, 'tv'),
    ]);

    try {
      await fetch(`${haUrl}/api/services/media_player/turn_off`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: FIRE_STICK_ADB_ENTITY }),
      });
      await fetch(`${haUrl}/api/services/media_player/turn_off`, {
        method: 'POST', headers: haHeaders,
        body: JSON.stringify({ entity_id: CAT_TV_ENTITY }),
      });
    } catch {}

    try {
      const donePath = await generateAndSaveTTSAudio("Skipped. No more readings for this week. Great work Bryn!...
      await playOnNestSpeaker(`${appUrl}${donePath}`);
    } catch {}

    return res.json({ action: "skipped_and_done", skippedFile: currentFileName });
  }

  const nextFileName = nextFile.displayName || nextFile.originalName || 'Unknown';
  const fileDesc = describeFileForTTS(nextFile, currentWeekNumber);
  console.log(`[Voice Command] Skipping to next: ${fileDesc} (id=${nextFile.id})`);

  try {
    const echoEntities = CAT_ECHO_ENTITIES;
    await fetch(`${haUrl}/api/services/media_player/media_stop`, {
      method: 'POST', headers: haHeaders,
      body: JSON.stringify({ entity_id: echoEntities }),
    });
  } catch {}

```

```

const confirmTTS = `Skipped. Now playing ${fileDesc}.`;
catWashPlaybackTrigger = catWashPlaybackTrigger || 'manual';
catLightsPromptPending = false;

res.json({ action: "skipping", skippedFile: currentFileName, nextFile: nextFileName });

await startConfirmedPlaybackFlow(nextFile, '[Voice Skip]', 'echo', confirmTTS);
return;

} else if (command === 'clear_player' || command === 'clear player') {
  console.log(`[Voice Command] Clear player requested - resetting all playback state`);

  clearVoiceCommandPause_();

  if (nestPlaybackAbort) {
    nestPlaybackAbort();
    nestPlaybackAbort = null;
  }
  await stopNestSpeaker();
  stopWordAdvancement();
  stopToothbrushPolling();

  if (currentTTSSession) {
    stopTTSSession("Clear player command");
  }

  catWashPlaybackActive = false;
  catWashPlaybackStartedAt = null;
  catWashPlaybackState = null;
  catWashManuallyStoppedAt = null;
  catLightsPromptPending = false;
  currentTvFollowUrl = null;
  currentTabletReaderUrl = null;

  try {
    const echoEntities = CAT_ECHO_ENTITIES;
    await fetch(`${haUrl}/api/services/media_player/media_stop`, {
      method: 'POST', headers: haHeaders,
      body: JSON.stringify({ entity_id: echoEntities }),
    });
  } catch {}

  try {
    const clearPath = await generateAndSaveTTSAudio("Player cleared. Ready for a new session.", `vc-clear-${Dat...}`);
    await playOnNestSpeaker(` ${appUrl}${clearPath}`);
  } catch {}

  return res.json({ action: "cleared" });

} else {
  console.log(`[Voice Command] Unknown command: "${command}"`);
  return res.json({ action: "unknown", command });
}

} catch (error: any) {
  console.error(`[Voice Command] Error: ${error.message}`);
  res.status(500).json({ error: "Voice command failed", details: error.message });
}
});

app.get("/api/voice-command/status", (_req, res) => {
  const status: any = {
    playbackActive: catWashPlaybackActive,
    paused: !!voiceCommandPauseState_,
  };
  if (catWashPlaybackActive && catWashPlaybackState) {
    status.playing = {
      fileId: catWashPlaybackState.fileId,
      fileName: catWashPlaybackState.fileName,
      chunkIndex: catWashPlaybackState.chunkIndex,
      totalChunks: catWashPlaybackState.totalChunks,
    };
  }
  if (voiceCommandPauseState_) {
    status.pauseInfo = {
      fileId: voiceCommandPauseState_.fileId,
      fileName: voiceCommandPauseState_.fileName,
      chunkIndex: voiceCommandPauseState_.chunkIndex,
      pausedAt: voiceCommandPauseState_.pausedAt.toISOString(),
      autoStopIn: Math.max(0, 10 * 60 * 1000 - (Date.now() - voiceCommandPauseState_.pausedAt.getTime())),
    };
  }
  res.json(status);
});

app.get("/api/ha/entities", async (_req, res) => {
  try {
    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
    const response = await fetch(`${haUrl}/api/states`, {
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
    });
    if (!response.ok) return res.status(response.status).json({ error: "HA request failed" });
    const states: any[] = await response.json();
    const mediaPlayers = states
      .filter((s: any) => s.entity_id.startsWith("media_player."))
      .map((s: any) => ({
        entity_id: s.entity_id,
        friendly_name: s.attributes?.friendly_name || s.entity_id,
        state: s.state,
        device_class: s.attributes?.device_class,
        supported_features: s.attributes?.supported_features,
      }));
    res.json(mediaPlayers);
  } catch (error: any) {
    res.status(500).json({ error: error.message });
  }
});

```

```

app.get("/api/flick/rooms", (_req, res) => {
  res.json(Flick_DEVICES);
});

app.post("/api/flick", async (req, res) => {
  try {
    const { deviceId, fileId, currentChunkIndex, totalChunks: totalChunksCount } = req.body;
    if (!deviceId) return res.status(400).json({ error: "deviceId is required" });
    if (!fileId) return res.status(400).json({ error: "fileId is required" });

    let device: FlickDevice | undefined;
    for (const group of Flick_DEVICES) {
      device = group.devices.find(d => d.id === deviceId);
      if (device) break;
    }
    if (!device) return res.status(404).json({ error: "Device not found" });

    const baseUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
    console.log(`[Flick] Sending file ${fileId} to ${device.name} (${device.room}) (chunk ${currentChunkIndex}/${totalChunksCount})`);

    if (typeof currentChunkIndex === 'number' && currentChunkIndex > 0) {
      try {
        await storage.updateFile(fileId, {
          lastChunkIndex: currentChunkIndex,
          totalChunks: totalChunksCount || 0
        });
      } catch (e) {
        console.error("[Flick] Failed to save progress:", e);
      }
    }

    const appUrl = `https://${req.get('host')} || new URL(DEPLOYED_APP_URL).host`;
    const readerUrl = `${appUrl}/pdf-reader/${fileId}?autoplay=true&speaker=${encodeURIComponent(device.entityId)}`;

    const navigateToReader = async (targetDevice: FlickDevice) => {
      if (!targetDevice.canDisplay) return;
      const speakerEntity = device.entityId;
      const deviceReaderUrl = `${appUrl}/pdf-reader/${fileId}?autoplay=true&speaker=${encodeURIComponent(speakerEntity)}`;
      try {
        if (targetDevice.type === "tablet" || targetDevice.type === "echo_show") {
          const navResp = await fetch(`${baseUrl}/api/services/browser_mod/navigate`, {
            method: 'POST',
            headers: {
              'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
              'Content-Type': 'application/json',
            },
            body: JSON.stringify({
              browser_id: targetDevice.entityId,
              path: deviceReaderUrl
            })
          });
          console.log(`[Flick] Navigated ${targetDevice.entityId} via browser_mod: ${navResp.status}`);
        } else if (targetDevice.type === "tv") {
          const castResp = await fetch(`${baseUrl}/api/services/media_player/play_media`, {
            method: 'POST',
            headers: {
              'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
              'Content-Type': 'application/json',
            },
            body: JSON.stringify({
              entity_id: targetDevice.entityId,
              media_content_id: deviceReaderUrl,
              media_content_type: "url"
            })
          });
          console.log(`[Flick] Cast to TV ${targetDevice.entityId}: ${castResp.status}`);
        }
      } catch (navErr) {
        console.error(`[Flick] Display navigation failed for ${targetDevice.name}:`, navErr);
      }
    };

    if (device.canDisplay) {
      await navigateToReader(device);
    } else if (device.type === "group") {
      const roomGroup = Flick_DEVICES.find(g => g.devices.some(d => d.id === device.id));
      if (roomGroup) {
        const screenDevices = roomGroup.devices.filter(d => d.canDisplay && d.id !== device.id);
        for (const screenDevice of screenDevices) {
          await navigateToReader(screenDevice);
        }
      }
    }

    res.json({
      success: true,
      device: device.name,
      room: device.room,
      readerUrl,
      entityId: device.entityId,
      canDisplay: device.canDisplay,
      resumeChunk: currentChunkIndex || 0
    });
  } catch (error: any) {
    console.error("[Flick] Error:", error);
    res.status(500).json({ error: "Failed to flick", details: error.message });
  }
});

app.get("/api/course-play-priority", (_req, res) => {
  res.json(coursePlayPriority);
});

app.post("/api/course-play-priority", (req, res) => {
  coursePlayPriority = req.body || {};
  console.log(`[Course Priority] Updated: ${JSON.stringify(coursePlayPriority)}`);
  res.json({ success: true });
});

```

```

});

app.post("/api/trigger-playback", async (req, res) => {
  try {
    const { fileId, weekNumber } = req.body || {};
    console.log(`[Trigger Playback] Manual trigger - fileId=${fileId}, week=${weekNumber}`);

    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }

    const semesterSettings = await storage.getActiveSemesterSettings();
    if (!semesterSettings) {
      return res.status(400).json({ error: "No active semester" });
    }

    const today = torontoDate();
    const semStart = semesterSettings.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : new Date();
    const rwStart = semesterSettings.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : semStart;
    const currentWeek = weekNumber || getWeekNumber(today, semStart, rwStart);

    await syncOneDriveFilesForWeek(semesterSettings, currentWeek, '[Trigger Playback]');

    let targetFile: any = null;
    const allFiles = await storage.getFiles();

    if (fileId) {
      targetFile = allFiles.find((f: any) => f.id === fileId);
    }
    if (!targetFile) {
      targetFile = findNextFileByPriority(allFiles, currentWeek);
    }

    if (!targetFile) {
      const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
      console.log(`[Trigger Playback] No files - playing CHUM FM`);
      await playChumFmRadio(haUrl);
      return res.json({ action: "radio", reason: `All week ${currentWeek} readings complete - playing CHUM FM` });
    }

    const fileName = targetFile.displayName || targetFile.originalName || 'Unknown';
    console.log(`[Trigger Playback] Starting playback: ${fileName} (id=${targetFile.id})`);

    res.json({ action: "playing", file: { id: targetFile.id, name: fileName }, currentWeek });

    await startConfirmedPlaybackFlow(targetFile, '[Trigger Playback]', 'echo');
  } catch (error: any) {
    console.error("[Trigger Playback] Error:", error);
    res.status(500).json({ error: "Failed to trigger playback", details: error.message });
  }
});

app.post("/api/webhook/email-homework", async (req, res) => {
  try {
    const webhookSecret = process.env.SITE_PASSWORD;
    const authHeader = req.headers['x-webhook-secret'] || req.headers['authorization']?.replace('Bearer ', '');
    if (webhookSecret && authHeader !== webhookSecret) {
      console.warn("[Email Homework] Unauthorized attempt");
      return res.status(401).json({ error: "Unauthorized" });
    }

    const { subject, body, from } = req.body || {};
    console.log(`[Email Homework] ===== RECEIVED =====`);
    console.log(`[Email Homework] From: ${from}, Subject: ${subject}`);
    console.log(`[Email Homework] Body: ${body}`);

    if (!subject) {
      return res.status(400).json({ error: "Missing subject" });
    }

    const coursePattern = /^[A-Z]{3,5}\d{3}\s+/i;
    const courseMatch = subject.match(coursePattern);
    const courseName = courseMatch ? courseMatch[1].toUpperCase() : null;
    const titleText = courseMatch ? subject.slice(courseMatch[0].length).trim() : subject.trim();

    const fullText = [titleText, body || ''].join(' ');

    let dueDate: Date | null = null;
    const dueDatePatterns = [
      /due\s+(\d{4})-(\d{2})-(\d{2})/i,
      /due\s+(\d{1,2})\/(\d{1,2})\/(\d{2,4})/i,
      /due\s+(\w+\s+(\d{1,2})(?:,?\s*\d{4})?)/i,
    ];
    for (const pat of dueDatePatterns) {
      const m = fullText.match(pat);
      if (m) {
        let dateStr = m[1];
        const hasYear = /\d{4}/.test(dateStr);
        if (!hasYear) {
          dateStr = dateStr.replace(/,?\s*$/, '') + ' ' + new Date().getFullYear();
        }
        const parsed = new Date(dateStr);
        if (!isNaN(parsed.getTime())) {
          if (parsed.getTime() < Date.now() - 86400000 && !hasYear) {
            parsed.setFullYear(parsed.getFullYear() + 1);
          }
          dueDate = parsed;
          break;
        }
      }
    }

    if (!dueDate) {
      dueDate = new Date();
      dueDate.setDate(dueDate.getDate() + 1);
    }
  }
});

```

```

let eventStartTime: string | null = null;
let eventEndTime: string | null = null;
const noonMatch = fullText.match(/bat\s+noon\b/i);
const timeMatch = fullText.match(/bat\s+(\d{1,2})(?:\d{2})?\s*(am|pm)?\b/i);
const rangeMatch = fullText.match(/(\d{1,2})(?:\d{2})?\s*(am|pm)?\s*[-to]+\s*(\d{1,2})(?:\d{2})?\s*(am|p...

if (rangeMatch) {
  let startH = parseInt(rangeMatch[1]);
  const startM = rangeMatch[2] ? parseInt(rangeMatch[2]) : 0;
  const startAmPm = (rangeMatch[3] || rangeMatch[6] || '').toLowerCase();
  let endH = parseInt(rangeMatch[4]);
  const endM = rangeMatch[5] ? parseInt(rangeMatch[5]) : 0;
  const endAmPm = rangeMatch[6].toLowerCase();
  if (startAmPm === 'pm' && startH !== 12) startH += 12;
  if (startAmPm === 'am' && startH === 12) startH = 0;
  if (endAmPm === 'pm' && endH !== 12) endH += 12;
  if (endAmPm === 'am' && endH === 12) endH = 0;
  if (!startAmPm && startH < endH - 5) { if (endH >= 12) startH += 12; }
  eventStartTime = `${String(startH).padStart(2, '0')}:${String(startM).padStart(2, '0')}`;
  eventEndTime = `${String(endH).padStart(2, '0')}:${String(endM).padStart(2, '0')}`;
  dueDate.setHours(startH, startM, 0, 0);
} else if (noonMatch) {
  eventStartTime = '12:00';
  dueDate.setHours(12, 0, 0, 0);
} else if (timeMatch) {
  let h = parseInt(timeMatch[1]);
  const mins = timeMatch[2] ? parseInt(timeMatch[2]) : 0;
  const ampm = (timeMatch[3] || '').toLowerCase();
  if (ampm === 'pm' && h !== 12) h += 12;
  if (ampm === 'am' && h === 12) h = 0;
  if (!ampm && h >= 1 && h <= 7) h += 12;
  eventStartTime = `${String(h).padStart(2, '0')}:${String(mins).padStart(2, '0')}`;
  dueDate.setHours(h, mins, 0, 0);
} else {
  dueDate.setHours(9, 0, 0, 0);
}

let taskType = courseName ? 'reading' : 'reminder';
const typeLower = fullText.toLowerCase();
if (/bessay\b/.test(typeLower)) taskType = 'essay';
else if (/bquiz\b/.test(typeLower)) taskType = 'quiz';
else if (/bexam\b|btest\b|bfinal\b|bmidterm\b/.test(typeLower)) taskType = 'exam';
else if (/bdiscussion\b|bforum\b|bpost\b/.test(typeLower)) taskType = 'discussion';
else if (/bproject\b|bpresentation\b/.test(typeLower)) taskType = 'project';
else if (/bpoll\b|bsurvey\b/.test(typeLower)) taskType = 'poll';
else if (/bmodule\b/.test(typeLower)) taskType = 'module';

let weekNumber = 1;
const weekMatch = fullText.match(/week\s*(\d+)/i);
if (weekMatch) {
  weekNumber = parseInt(weekMatch[1], 10);
} else {
  const semesterSettings = await storage.getActiveSemesterSettings();
  if (semesterSettings?.semesterStartDate) {
    const { getWeekNumber } = await import('../shared/schema');
    weekNumber = getWeekNumber(
      new Date(),
      new Date(semesterSettings.semesterStartDate),
      semesterSettings.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : null
    );
  }
}

let priority = 'medium';
if (/burgent\b|bhigh\b|basap\b|bimportant\b/i.test(fullText)) priority = 'high';
else if (/blow\b|boptional\b/i.test(fullText)) priority = 'low';

const description = body?.trim() || null;

const emailMessageId = req.body.messageId || `gmail-${Date.now()}`;
const existing = await storage.getPendingReviewItemByExternalId(emailMessageId, 'gmail_task');
if (existing) {
  console.log(`[Email Homework] Already queued: ${emailMessageId}`);
  return res.json({ success: true, queued: true, alreadyExists: true, id: existing.id });
}

const reviewItem = await storage.createPendingReviewItem({
  source: 'gmail_task',
  sourceEmail: from || null,
  externalId: emailMessageId,
  title: titleText,
  description: description,
  startDate: dueDate,
  endDate: null,
  eventStartTime: eventStartTime,
  eventEndTime: eventEndTime,
  location: null,
  rawData: JSON.stringify({ subject, body, from, parsedType: taskType, courseName, weekNumber, priority }),
  status: 'pending',
  courseName: courseName,
  taskType: taskType,
});

console.log(`[Email Homework] Queued for review #${reviewItem.id}: "${titleText}" (${taskType}, ${courseName})`);
res.json({ success: true, queued: true, reviewItemId: reviewItem.id, title: titleText, type: taskType, courseNa...

} catch (error: any) {
  console.error("[Email Homework] Error:", error);
  res.status(500).json({ error: "Failed to create task from email", details: error.message });
}
});

// POST /api/webhook/play-urgent-pdf - Home Assistant webhook to play most urgent unlistened PDF
// Priority: 1) CPPA modules, 2) Other course modules, 3) CPPA readings
// Resumes from last position if partially listened
app.post("/api/webhook/play-urgent-pdf", async (req, res) => {
  try {
    console.log("[Webhook] play-urgent-pdf triggered");
  }

```



```

// Authenticate webhook using SITE_PASSWORD as shared secret
const webhookSecret = process.env.SITE_PASSWORD;
const authHeader = req.headers['x-webhook-secret'] || req.headers['authorization']?.replace('Bearer ', '');
if (webhookSecret && authHeader !== webhookSecret) {
  console.warn("[Webhook] Unauthorized play-urgent-pdf attempt");
  return res.status(401).json({ error: "Unauthorized" });
}

if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
  return res.status(500).json({ error: "Home Assistant not configured" });
}

const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

// Validate entity_id against known speakers
const allowedEntities = [
  BATHROOM_ECHO_ENTITY, KITCHEN_ECHO_ENTITY, NEST_SPEAKER_ENTITY, CAT_WR_HA_VOICE_ENTITY,
  "media_player.cat_wash_2",
  ...CAT_ECHO_ENTITIES, "media_player.echo_closet_am",
  "media_player.echo_lr_couch_r_am", "media_player.echo_hallway_entrance_am",
  "media_player.echo_king_l_am", "media_player.echo_king_r_am",
  "media_player.echo_king_tv_am", "media_player.echo_kitchen_cupboards_left_am",
  "media_player.echo_kitchen_cupboards_r_am", "media_player.echo_kitchen_fridge_am",
  "media_player.echo_kitchen_hutch_am", "media_player.echo_kitchen_island_corner_am",
  "media_player.echo_kitchen_studio_black_am", "media_player.echo_lr_hub_am"
];
const requestedEntity = req.body?.entity_id || NEST_SPEAKER_ENTITY;
const targetEntity = allowedEntities.includes(requestedEntity) ? requestedEntity : NEST_SPEAKER_ENTITY;

// Get current week number
const semesterSettings = await storage.getActiveSemesterSettings();
let currentWeekNumber = 1;
if (semesterSettings?.semesterStartDate) {
  currentWeekNumber = getWeekNumber(torontoDate(), new Date(semesterSettings.semesterStartDate));
}

// Get all files
const allFiles = await storage.GetFiles();

// Filter for unlistened files in current week
const unlistenedFiles = allFiles.filter((f: any) => {
  if (f.listened) return false;
  const weekMatch = f.folder?.match(/week-(\d+)/i);
  return weekMatch && parseInt(weekMatch[1], 10) === currentWeekNumber;
});

// Priority buckets
const isCPPA = (f: any) => f.folder?.toLowerCase().includes('cpa');
const isModule = (f: any) => f.folder?.toLowerCase().includes('module');

const cppaModules = unlistenedFiles.filter((f: any) => isCPPA(f) && isModule(f));
const otherModules = unlistenedFiles.filter((f: any) => !isCPPA(f) && isModule(f));
const cppaReadings = unlistenedFiles.filter((f: any) => isCPPA(f) && !isModule(f));
const otherReadings = unlistenedFiles.filter((f: any) => !isCPPA(f) && !isModule(f));

const orderedFiles = [...cppaModules, ...otherModules, ...cppaReadings, ...otherReadings];

if (orderedFiles.length === 0) {
  // Announce that all readings are complete
  const isNonAlexaTarget = NON_ALEXA_ENTITIES.includes(targetEntity);
  console.log("[Webhook] No urgent PDFs found, announcing completion");
  const completionMsg = `All week ${currentWeekNumber} readings are complete. Great job!`;
  if (isNonAlexaTarget) {
    const audioPath = await generateAndSaveTTSAudio(completionMsg, `tts-done-${Date.now()}`, "echo");
    const appUrl = DEPLOYED_APP_URL;
    await fetch(`${haUrl}/api/services/media_player/play_media`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ entity_id: targetEntity, media_content_id: `${appUrl}${audioPath}`, media_content...
    });
  } else {
    await fetch(`${haUrl}/api/services/notify/alexa_media`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ message: completionMsg, target: targetEntity, data: { type: "tts" } });
    });
  }
  return res.json({ action: "complete", message: `All week ${currentWeekNumber} readings done` });
}

const nextFile = orderedFiles[0];
const fileName = nextFile.displayName || nextFile.originalName;
const courseMatch = nextFile.folder?.match(/(cpa|cfn|csoc|cphl|casl)\d*/i);
const courseName = courseMatch ? courseMatch[0].toUpperCase() : '';
const fileType = isModule(nextFile) ? 'module' : 'reading';

console.log(`[Webhook] Selected: ${courseName} ${fileType} - ${fileName}`);

// Stop any existing TTS session
if (currentTTSSession) {
  stopTTSSession("New webhook playback requested");
}

// Extract text from the PDF
const { ObjectStorageService } = await import("./replit_integrations/object_storage");
const objectStorage = new ObjectStorageService();

let textContent = "";
try {
  const objectFile = await objectStorage.getObjectEntityFile(nextFile.objectPath);
  const [content] = await objectFile.download();

  const isPDF = content.slice(0, 4).toString() === '%PDF';
  if (isPDF) {
    const PdfParser = await getPdfParser();
    const parser = new PdfParser({ data: new Uint8Array(content), verbosity: 0 });
    await parser.load();
  }
}

```

```

const pdfText = await parser.getText();

if (pdfText && typeof pdfText === 'object') {
  if (pdfText.pages && Array.isArray(pdfText.pages)) {
    textContent = pdfText.pages.map((page: any) => page.text || '').join(' ');
  } else if (Array.isArray(pdfText)) {
    textContent = pdfText.map((item: any) => typeof item === 'string' ? item : item.text || '').join(' ');
  } else if (pdfText.text) {
    textContent = pdfText.text;
  } else {
    textContent = Object.values(pdfText).filter(v => typeof v === 'string').join(' ');
  }
} else {
  textContent = String(pdfText || '');
}
await parser.destroy();
} else {
  textContent = content.toString('utf-8');
}
} catch (error) {
  console.error("[Webhook] Error extracting text:", error);
  return res.status(500).json({ error: "Failed to extract text from file" });
}

if (!textContent.trim()) {
  return res.status(400).json({ error: "File is empty or not readable" });
}

// Clean text for TTS
const fullCleanedText = cleanTextForTTS(textContent);

// Determine resume position from database progress or in-memory progress
let resumePosition = 0;
const progressKey = `file-${nextFile.id}`;
const memProgress = playbackProgress[progressKey];

if (nextFile.lastChunkIndex && nextFile.lastChunkIndex > 0 && nextFile.totalChunks && nextFile.totalChunks > 0) {
  // Estimate character position from chunk progress
  const chunkRatio = nextFile.lastChunkIndex / nextFile.totalChunks;
  resumePosition = Math.floor(chunkRatio * fullCleanedText.length);
  console.log(`[Webhook] Resuming from DB progress: chunk ${nextFile.lastChunkIndex}/${nextFile.totalChunks} (~...`
} else if (memProgress) {
  const chunkRatio = memProgress.chunkIndex / memProgress.totalChunks;
  resumePosition = Math.floor(chunkRatio * fullCleanedText.length);
  console.log(`[Webhook] Resuming from memory progress: chunk ${memProgress.chunkIndex}/${memProgress.totalChun...`
}

// Build announcement
const isResuming = resumePosition > 0;
const progressPct = resumePosition > 0 ? Math.round((resumePosition / fullCleanedText.length) * 100) : 0;
const announcement = isResuming
  ? `Resuming ${courseName} ${fileType}, ${fileName.replace('.pdf', '')}, at ${progressPct} percent.`
  : `Now reading ${courseName} ${fileType}, ${fileName.replace('.pdf', '')}. ${orderedFiles.length} file${order...`

// Get the text from resume position
const remainingText = fullCleanedText.substring(resumePosition);

// Create TTS session
currentTTSsession = {
  fullText: remainingText.length > 100000 ? remainingText.substring(0, 100000) : remainingText,
  currentPosition: 0,
  startTime: Date.now(),
  isPlaying: true,
  autoTimer: null,
  consecutiveErrors: 0,
  sessionCreatedAt: Date.now(),
  targetEntity
};

// Restore volume
await fetch(`${baseUrl}/api/services/media_player/volume_set`, {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    entity_id: targetEntity,
    volume_level: 0.5
  }),
});

// Build first chunk with announcement
let firstChunk = cleanTextForTTS(remainingText.substring(0, 3000));
firstChunk = getChunkWithSentenceBoundary(firstChunk, CHUNK_SIZE);
const fullFirstMessage = `${announcement} ... ${firstChunk}`;
const isNonAlexaTarget = NON_ALEXA_ENTITIES.includes(targetEntity);

console.log(`[Webhook] Sending first chunk (${firstChunk.length} chars) to ${targetEntity} ${isNonAlexaTarget ? ...`

let response: Response;
if (isNonAlexaTarget) {
  const audioPath = await generateAndSaveTTSaudio(fullFirstMessage, `tts-first-${Date.now()}`, "echo");
  const appUrl = DEPLOYED_APP_URL;
  const fullAudioUrl = `${appUrl}${audioPath}`;
  console.log(`[Webhook] Non-Alexa: Generated audio at ${audioPath}`);

  response = await fetch(`${baseUrl}/api/services/media_player/play_media`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ entity_id: targetEntity, media_content_id: fullAudioUrl, media_content_type: "music"...
  });
} else {
  const ssmContent = `<speak><prosody rate="90%">${fullFirstMessage}</prosody></speak>`;
  response = await fetch(`${baseUrl}/api/services/notify/alexa_media`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ message: ssmContent, target: targetEntity, data: { type: "tts" } }),
  });
}

```

```

    });
  }

  if (!response.ok) {
    const errorText = await response.text();
    console.error("[Webhook] HA TTS error:", errorText);
    currentTTSSession = null;
    return res.status(response.status).json({ error: "Failed to start playback" });
  }

  // Update position after first chunk
  currentTTSSession.currentPosition = firstChunk.length;

  // Save progress with correct resume chunk index
  const totalChunksForFile = Math.ceil(fullCleanedText.length / CHUNK_SIZE);
  const resumeChunkIndex = resumePosition > 0 ? Math.floor(resumePosition / CHUNK_SIZE) : 0;
  playbackProgress[progressKey] = {
    chunkIndex: resumeChunkIndex,
    totalChunks: totalChunksForFile,
    lastPlayed: new Date(),
    fileId: nextFile.id
  };

  // Schedule auto-continuation
  scheduleNextChunk();

  console.log(`[Webhook] Playback started: ${courseName} ${fileType} - ${fileName}`);

  res.json({
    action: "playing",
    file: { id: nextFile.id, name: fileName, folder: nextFile.folder },
    course: courseName,
    type: fileType,
    resuming: isResuming,
    progressPercent: progressPct,
    remainingFiles: orderedFiles.length,
    currentWeek: currentWeekNumber
  });

} catch (error: any) {
  console.error("[Webhook] play-urgent-pdf error:", error);
  res.status(500).json({ error: "Failed to play urgent PDF", details: error.message });
}
});

// POST /api/ha/register-play-urgent-script - Register a Home Assistant script for Alexa routines
app.post("/api/ha/register-play-urgent-script", async (req, res) => {
  try {
    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }

    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
    const appUrl = req.body?.appUrl || `https://${req.headers.host}`;

    // Create a HA script that calls our webhook endpoint
    const scriptConfig = {
      alias: "Play Urgent School Reading",
      description: "Plays the most urgent unlistened PDF from school courses on Echo speakers",
      icon: "mdi:book-open-page-variant",
      mode: "single",
      sequence: [
        {
          service: "rest_command.play_urgent_pdf",
          data: {}
        }
      ]
    };

    // First register the REST command
    // We need to create a rest_command via HA config - notify user of manual step
    const webhookUrl = `${appUrl}/api/webhook/play-urgent-pdf`;

    res.json({
      success: true,
      webhookUrl,
      haSetupInstructions: {
        step1: "Add this to your Home Assistant configuration.yaml under rest_command:",
        restCommand: {
          play_urgent_pdf: {
            url: webhookUrl,
            method: "POST",
            content_type: "application/json",
            headers: {
              "x-webhook-secret": "YOUR_SITE_PASSWORD_HERE"
            },
            payload: '{"entity_id": "media_player.bathroom_speaker"}'
          }
        },
        step2: "Add this script to your configuration.yaml under script:",
        script: {
          play_urgent_school_reading: scriptConfig
        },
        step3: "Restart Home Assistant, then expose 'Play Urgent School Reading' to Alexa via Nabu Casa",
        step4: "The script will appear as an action in Alexa routines"
      }
    });

  } catch (error: any) {
    console.error("Register HA script error:", error);
    res.status(500).json({ error: "Failed to generate HA config", details: error.message });
  }
});

// POST /api/files/generate-all-tts - Pre-generate TTS audio for all module files
app.post("/api/files/generate-all-tts", async (req, res) => {
  try {
    const { weekNumber } = req.body;

```

```

// Get all module files for the specified week (or all weeks if not specified)
const allFiles = await storage.getFiles();
const moduleFiles = allFiles.filter((f: FileRecord) => {
  const isModule = f.folder?.includes('-module');
  const matchesWeek = weekNumber ? f.folder?.includes(`week-${weekNumber}-`) : true;
  const needsGeneration = !f.ttsAudioUrl;
  return isModule && matchesWeek && needsGeneration;
});

console.log(`Pre-generating TTS for ${moduleFiles.length} module files...`);

const results: { fileId: number; name: string; status: string; audioUrl?: string; error?: string }[] = [];

for (const file of moduleFiles) {
  try {
    console.log(`Generating TTS for: ${file.displayName}`);

    // Download the PDF from OneDrive
    const objectPath = file.objectPath;
    const content = await getOneDriveFile(objectPath);

    if (!content) {
      results.push({ fileId: file.id, name: file.displayName, status: 'error', error: 'Empty file' });
      continue;
    }

    // Extract text from PDF
    let textContent = '';
    if (file.contentType?.includes('pdf') || file.originalName.endsWith('.pdf')) {
      const PdfParser = await getPdfParser();
      const parsed = await PdfParser(content);
      textContent = parsed.text;
    } else if (Buffer.isBuffer(content)) {
      textContent = content.toString('utf-8');
    }

    if (!textContent.trim()) {
      results.push({ fileId: file.id, name: file.displayName, status: 'error', error: 'No text content' });
      continue;
    }

    // Generate OpenAI TTS audio and save to object storage
    const audioPath = await generateAndSaveTTSAudio(textContent, `module-${file.id}`);
    const audioUrl = `${DEPLOYED_APP_URL}${audioPath}`;

    // Update file record with audio URL
    await storage.updateFile(file.id, {
      ttsAudioUrl: audioUrl,
      ttsGeneratedAt: new Date()
    });

    results.push({ fileId: file.id, name: file.displayName, status: 'success', audioUrl });
    console.log(`TTS generated for: ${file.displayName}`);
  } catch (error: any) {
    console.error(`Error generating TTS for ${file.displayName}:`, error);
    results.push({ fileId: file.id, name: file.displayName, status: 'error', error: error.message });
  }
}

res.json({
  success: true,
  totalFiles: moduleFiles.length,
  results
});

} catch (error: any) {
  console.error("Error generating TTS files:", error);
  res.status(500).json({ error: "Failed to generate TTS files", details: error.message });
}
});

// POST /api/shower/mark-listened - Mark a file as listened
app.post("/api/shower/mark-listened", async (req, res) => {
  try {
    const { fileId } = req.body;

    if (!fileId) {
      return res.status(400).json({ error: "File ID required" });
    }

    // Update the file to mark it as listened
    await storage.updateFile(fileId, { listened: true });

    // Clear progress for this file
    delete playbackProgress[ `file-${fileId}` ];

    res.json({ success: true, fileId, listened: true });
  } catch (error) {
    console.error("Error marking file as listened:", error);
    res.status(500).json({ error: "Failed to mark file as listened" });
  }
});

// POST /api/shower/next-chunk - Continue to next chunk of current file
app.post("/api/shower/next-chunk", async (req, res) => {
  try {
    const { fileId, entityId } = req.body;
    const targetEntity = entityId || NEST_SPEAKER_ENTITY;

    if (!fileId) {
      return res.status(400).json({ error: "File ID required" });
    }

    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }
  }
});

```

```

const haUrl = HOME_ASSISTANT_URL.replace(/\$/ , '');

// Get the file
const allFiles = await storage.GetFiles();
const file = allFiles.find((f: any) => f.id === fileId);

if (!file) {
    return res.status(404).json({ error: "File not found" });
}

// Get current progress
const progressKey = `file-${fileId}`;
const progress = playbackProgress[progressKey];

if (!progress) {
    return res.status(400).json({ error: "No progress found for this file. Use /trigger to start." });
}

const nextChunkIndex = progress.chunkIndex + 1;

// If we've finished all chunks, mark as listened
if (nextChunkIndex >= progress.totalChunks) {
    await storage.updateFile(fileId, { listened: true });
    delete playbackProgress[progressKey];

    return res.json({
        action: "completed",
        message: "File reading complete!",
        fileId,
        listened: true
    });
}

// Extract text again to get the chunk
const { ObjectStorageService } = await import("./replit_integrations/object_storage");
const objectStorage = new ObjectStorageService();

let textContent = "";
try {
    const objectFile = await objectStorage.getObjectEntityFile(file.objectPath);
    const [content] = await objectFile.download();

    const isPDF = content.slice(0, 4).toString() === '%PDF';
    if (isPDF) {
        const PdfParser = await getPdfParser();
        const parser = new PdfParser({ data: new Uint8Array(content), verbosity: 0 });
        await parser.load();
        const pdfText = await parser.getText();

        if (pdfText && typeof pdfText === 'object') {
            if (pdfText.pages && Array.isArray(pdfText.pages)) {
                textContent = pdfText.pages.map((page: any) => page.text || '').join(' ');
            } else if (Array.isArray(pdfText)) {
                textContent = pdfText.map((item: any) => typeof item === 'string' ? item : item.text || '').join(' ');
            } else if (pdfText.text) {
                textContent = pdfText.text;
            }
        } else {
            textContent = String(pdfText || '');
        }
        await parser.destroy();
    } else {
        textContent = content.toString('utf-8');
    }
} catch (error) {
    console.error("Error extracting text:", error);
    return res.status(500).json({ error: "Failed to extract text from file" });
}

// Chunk the text
let cleanedContent = textContent.trim().replace(/\s+/g, ' ').replace(/[^\x20-\x7E]/g, ' ');
const chunks = cleanedContent.match(/.{1,450}[.!?]\s*/g) || [cleanedContent];
const chunk = chunks[nextChunkIndex];

// Send TTS
const chunkMessage = `Section ${nextChunkIndex + 1} of ${chunks.length}. ${chunk}`;
const isNonAlexa = NON_ALEXA_ENTITIES.includes(targetEntity);
if (isNonAlexa) {
    const audioPath = await generateAndSaveTTSAudio(chunkMessage, `next-chunk-${fileId}-${nextChunkIndex}`, "echo");
    const appUrl = DEPLOYED_APP_URL;
    await fetch(`${haUrl}/api/services/media_player/play_media`, {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
        body: JSON.stringify({ entity_id: targetEntity, media_content_id: `${appUrl}${audioPath}`, media_content_ty...
    });
} else {
    await fetch(`${haUrl}/api/services/notify/alexa_media`, {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
        body: JSON.stringify({ message: chunkMessage, target: targetEntity, data: { type: "tts" } }),
    });
}

// Update progress
playbackProgress[progressKey] = {
    chunkIndex: nextChunkIndex,
    totalChunks: chunks.length,
    lastPlayed: new Date()
};

res.json({
    action: "reading",
    fileId,
    chunkIndex: nextChunkIndex,
    totalChunks: chunks.length,
    hasMore: nextChunkIndex < chunks.length - 1
});

```

```

    } catch (error) {
      console.error("Error playing next chunk:", error);
      res.status(500).json({ error: "Failed to play next chunk" });
    }
  });

// POST /api/shower/update-progress - Update playback progress
app.post("/api/shower/update-progress", async (req, res) => {
  try {
    const { taskId, chunkIndex, totalChunks } = req.body;

    if (!taskId) {
      return res.status(400).json({ error: "Task ID required" });
    }

    const progressKey = `task-${taskId}`;
    playbackProgress[progressKey] = {
      chunkIndex: chunkIndex || 0,
      totalChunks: totalChunks || 0,
      lastPlayed: new Date()
    };

    res.json({ success: true, progress: playbackProgress[progressKey] });
  } catch (error) {
    console.error("Error updating progress:", error);
    res.status(500).json({ error: "Failed to update progress" });
  }
});

// GET /api/partner-status - Check if partner's phone is away from home
app.post("/api/ha/service", async (req, res) => {
  try {
    const { domain, service, data } = req.body;
    if (!domain || !service) return res.status(400).json({ error: "domain and service required" });
    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
    const resp = await fetch(`${haUrl}/api/services/${domain}/${service}`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify(data || {}),
    });
    console.log(`[HA Service] ${domain}/${service}: ${resp.status}`);
    res.json({ status: resp.status });
  } catch (e: any) {
    res.status(500).json({ error: e.message });
  }
});

app.get("/api/partner-status", async (req, res) => {
  try {
    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }

    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

    const response = await fetch(`${haUrl}/api/states/${PARTNER_PHONE_ENTITY}`, {
      headers: {
        'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
        'Content-Type': 'application/json',
      },
    });

    if (!response.ok) {
      throw new Error(`Failed to get partner status: ${response.status}`);
    }

    const data = await response.json();
    const state = data.state?.toLowerCase() || '';

    // Partner is "at work" if state is "work" (zone.work)
    // Partner is "away" if state is not "home" (could be "not_home", "work", etc.)
    const isAtWork = state === 'work';
    const isAway = state !== 'home';

    res.json({
      isAway,
      isAtWork,
      state: data.state,
      friendlyName: data.attributes?.friendly_name || PARTNER_PHONE_ENTITY
    });
  } catch (error: any) {
    console.error("Error checking partner status:", error);
    res.status(500).json({ error: "Failed to check partner status", details: error.message });
  }
});

// POST /api/ha/automation/partner-leaves-work - Create HA automation to notify when partner leaves work zone
app.post("/api/ha/automation/partner-leaves-work", async (req, res) => {
  try {
    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }

    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

    const automationConfig = {
      alias: "Notify when Yasu leaves work",
      description: "Send push notification to iPhone when Yasu's phone leaves the work zone",
      mode: "single",
      trigger: [
        {
          platform: "zone",
          entity_id: PARTNER_PHONE_ENTITY,
          zone: "zone.work",
          event: "leave"
        }
      ],
    };
  },

```

```

    condition: [],
    action: [
      {
        service: "notify.mobile_app_iphone_10",
        data: {
          title: "Yasu Left Work",
          message: "Yasu just left the work zone.",
          data: {
            push: {
              sound: "default",
              interruption_level: "time-sensitive"
            }
          }
        }
      }
    ]
  }
];

const response = await fetch(`${haUrl}/api/services/automation/create`, {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify(automationConfig),
});

if (!response.ok) {
  const errorText = await response.text();
  console.error('HA automation create error, trying config endpoint:', errorText);

  const configResponse = await fetch(`${haUrl}/api/config/automation/config/uni_cal_partner_leaves_work`, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(automationConfig),
  });

  if (!configResponse.ok) {
    const configError = await configResponse.text();
    console.error('HA automation config error:', configError);
    return res.status(500).json({ error: "Failed to create automation", details: configError });
  }
}

console.log('HA automation created: Notify when Yasu leaves work');
res.json({ success: true, message: "Automation created: You'll get a notification when Yasu leaves the work zon..." });
} catch (error: any) {
  console.error("Error creating partner leaves work automation:", error);
  res.status(500).json({ error: "Failed to create automation", details: error.message });
}
});

// GET /api/ha/device-trackers - List all device trackers to find correct entity IDs
app.get("/api/ha/device-trackers", async (req, res) => {
  try {
    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }

    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

    const response = await fetch(`${haUrl}/api/states`, {
      headers: {
        'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
        'Content-Type': 'application/json',
      },
    });

    if (!response.ok) {
      throw new Error(`Failed to get states: ${response.status}`);
    }

    const data = await response.json();
    // Filter to device_tracker and person entities
    const trackers = data.filter((entity: any) =>
      entity.entity_id.startsWith('device_tracker.') ||
      entity.entity_id.startsWith('person.')
    ).map((entity: any) => ({
      entity_id: entity.entity_id,
      state: entity.state,
      friendly_name: entity.attributes?.friendly_name
    }));

    res.json(trackers);
  } catch (error: any) {
    console.error("Error listing device trackers:", error);
    res.status(500).json({ error: "Failed to list device trackers", details: error.message });
  }
});

// POST /api/kitchen/trigger - Trigger reading on kitchen Echo (same as shower but different speaker)
app.post("/api/kitchen/trigger", async (req, res) => {
  try {
    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }

    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

    // First, sync OneDrive to get latest files
    console.log("Kitchen trigger: Syncing OneDrive...");
    try {
      const syncResp = await fetch('http://localhost:5000/api/onedrive/sync', { method: 'POST' });
      if (!syncResp.ok) {
        console.log("OneDrive sync warning:", await syncResp.text());
      }
    }
  }
});

```

```

    }
  } catch (syncErr) {
    console.log("OneDrive sync skipped:", syncErr);
  }

  // Get current week number from semester settings
  const semesterSettings = await storage.getActiveSemesterSettings();
  const today = torontoDate();
  let currentWeekNumber = 1;

  if (semesterSettings?.semesterStartDate) {
    currentWeekNumber = getWeekNumber(today, new Date(semesterSettings.semesterStartDate));
  }

  console.log(`Kitchen trigger: Current week is ${currentWeekNumber}`);

  // Get all files and filter to current week's unlisted files
  const allFiles = await storage.getFiles();
  const weekFiles = allFiles.filter((f: any) => {
    if (f.listed) return false;
    const folderLower = (f.folder || '').toLowerCase();
    const weekMatch = folderLower.match(/week[- ]?(\d+)/);
    if (!weekMatch) return false;
    const fileWeek = parseInt(weekMatch[1], 10);
    return fileWeek === currentWeekNumber;
  });

  const orderedFiles = orderFilesByCoursePriority(
    weekFiles.filter((f: any) => {
      const folder = (f.folder || '').toLowerCase();
      return !folder.includes('casl') && !folder.includes('asl');
    })
  );

  // If no unlisted files, play radio
  if (orderedFiles.length === 0) {
    console.log("Kitchen trigger: All readings complete, playing radio");

    await fetch(`${baseUrl}/api/services/media_player/play_media`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        entity_id: KITCHEN_ECHO_ENTITY,
        media_content_id: "play 104.5 chumfm",
        media_content_type: "custom"
      })
    });

    return res.json({
      action: "radio",
      message: "All readings complete for this week! Playing CHUM FM 104.5",
      currentWeek: currentWeekNumber
    });
  }

  // Get the next file to read
  const nextFile = orderedFiles[0];
  const fileName = nextFile.displayName || nextFile.originalName || 'Unknown file';
  const folder = nextFile.folder || '';

  // Determine course and type for announcement
  const isCPPA = folder.toLowerCase().includes('cppa');
  const isModule = folder.toLowerCase().includes('module');
  const courseName = isCPPA ? 'CPPA 122 Local Politics and Government' : 'CFNF 400 Human Sexuality';
  const contentType = isModule ? 'Module' : 'Reading';

  console.log(`Kitchen trigger: Playing ${fileName}`);

  // Check for existing progress - resume from the same chunk (replay it)
  const progressKey = `file-${nextFile.id}`;
  const existingProgress = playbackProgress[progressKey];
  // Resume from the same chunk that was last playing (replay it)
  const resumeFromChunk = existingProgress?.lastCompletedChunk ?? 0;

  // Extract text from the file
  const { ObjectStorageService } = await import("./replit_integrations/object_storage");
  const objectStorage = new ObjectStorageService();

  let textContent = "";
  try {
    const objectFile = await objectStorage.getObjectEntityFile(nextFile.objectPath);
    const [content] = await objectFile.download();

    const isPDF = content.slice(0, 4).toString() === '%PDF';
    if (isPDF) {
      const PdfParser = await getPdfParser();
      const parser = new PdfParser({ data: new Uint8Array(content), verbosity: 0 });
      await parser.load();
      const pdfText = await parser.getText();

      if (pdfText && typeof pdfText === 'object') {
        if (pdfText.pages && Array.isArray(pdfText.pages)) {
          textContent = pdfText.pages.map((page: any) => page.text || '').join(' ');
        } else if (Array.isArray(pdfText)) {
          textContent = pdfText.map((item: any) => typeof item === 'string' ? item : item.text || '').join(' ');
        } else if (pdfText.text) {
          textContent = pdfText.text;
        }
      } else {
        textContent = String(pdfText || '');
      }
      await parser.destroy();
    } else {
      textContent = content.toString('utf-8');
    }
  }

```



```

} catch (error) {
  console.error("Error extracting text from file:", error);
  return res.status(500).json({ error: "Failed to extract text from file" });
}

// Chunk the text (~450 chars per chunk for TTS)
let cleanedContent = textContent.trim().replace(/\s+/g, ' ').replace(/[^\x20-\x7E]/g, ' ');
const chunks = cleanedContent.match(/.{1,450}[.!?]?[^\s]*/g) || [cleanedContent];

// Announce what we're about to read
const announcement = resumeFromChunk > 0
  ? `Resuming ${courseName}, ${contentType}: ${fileName.replace('.pdf', '')}. Starting from section ${resumeFromChunk} of ${sections.length} sections`
  : `Now reading ${courseName}, ${contentType}: ${fileName.replace('.pdf', '')}. ${chunks.length} sections total`;

// Send announcement
await fetch(`${baseUrl}/api/services/notify/alexa_media`, {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    message: announcement,
    target: KITCHEN_ECHO_ENTITY,
    data: { type: "tts" }
  })
});

// Return immediately - play chunks in background
res.json({
  action: "reading",
  file: {
    id: nextFile.id,
    name: fileName,
    folder: nextFile.folder
  },
  currentWeek: currentWeekNumber,
  totalChunks: chunks.length,
  startingFromChunk: resumeFromChunk + 1,
  remainingFiles: orderedFiles.length - 1
});

// Set up abort controller for stopping playback
kitchenPlaybackAbortController = new AbortController();
kitchenPlaybackActive = true;

// Play chunks in background (don't await)
(async () => {
  try {
    // Wait for announcement to finish (~5 seconds should be enough)
    await new Promise(resolve => setTimeout(resolve, 6000));

    for (let i = resumeFromChunk; i < chunks.length; i++) {
      // Check if playback was stopped
      if (!kitchenPlaybackActive) {
        console.log('Kitchen trigger: Playback stopped at chunk ${i + 1}');
        break;
      }

      const chunk = chunks[i];
      console.log('Kitchen trigger: Playing chunk ${i + 1} of ${chunks.length}');

      await fetch(`${baseUrl}/api/services/notify/alexa_media`, {
        method: 'POST',
        headers: {
          'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({
          message: chunk,
          target: KITCHEN_ECHO_ENTITY,
          data: { type: "tts" }
        })
      });

      // Save progress after each chunk starts
      playbackProgress[progressKey] = {
        chunkIndex: i,
        lastCompletedChunk: i,
        totalChunks: chunks.length,
        lastPlayed: new Date(),
        fileId: nextFile.id
      };

      // IMPORTANT: Alexa TTS takes time to process before speaking
      // Use ~50 chars per second (slower estimate) + 5 second buffer for processing
      const estimatedDuration = Math.max(8000, (chunk.length / 50) * 1000 + 5000);
      console.log('Kitchen trigger: Waiting ${estimatedDuration}ms for chunk ${i + 1} (${chunk.length} chars)');
      await new Promise(resolve => setTimeout(resolve, estimatedDuration));
    }

    // All chunks complete - mark file as listened (only if not stopped)
    if (kitchenPlaybackActive) {
      console.log('Kitchen trigger: All ${chunks.length} chunks complete, marking file ${nextFile.id} as listened...');
      await storage.updateFile(nextFile.id, { listened: true });
      delete playbackProgress[progressKey];
    }

    kitchenPlaybackActive = false;
    kitchenPlaybackAbortController = null;
  } catch (error) {
    console.error("Kitchen background playback error:", error);
    kitchenPlaybackActive = false;
    kitchenPlaybackAbortController = null;
  }
})();
} catch (error: any) {

```

```

        console.error("Kitchen trigger error:", error);
        res.status(500).json({ error: "Failed to trigger kitchen reading", details: error.message });
    }
});

// POST /api/kitchen/stop - Stop playback on kitchen Echo
app.post("/api/kitchen/stop", async (req, res) => {
    try {
        if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
            return res.status(500).json({ error: "Home Assistant not configured" });
        }

        const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

        // Stop background playback loop
        kitchenPlaybackActive = false;
        if (kitchenPlaybackAbortController) {
            kitchenPlaybackAbortController.abort();
            kitchenPlaybackAbortController = null;
        }

        // Stop media playback on Echo
        await fetch(`${haUrl}/api/services/media_player/media_stop`, {
            method: 'POST',
            headers: {
                'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
                'Content-Type': 'application/json',
            },
            body: JSON.stringify({
                entity_id: KITCHEN_ECHO_ENTITY
            }),
        });

        console.log("Kitchen playback stopped (both background loop and Echo)");

        res.json({ success: true, message: "Playback stopped on Kitchen Echo" });
    } catch (error: any) {
        console.error("Kitchen stop error:", error);
        res.status(500).json({ error: "Failed to stop kitchen playback", details: error.message });
    }
});

// GET /api/kitchen/status - Get current playback status
app.get("/api/kitchen/status", async (req, res) => {
    res.json({
        isPlaying: kitchenPlaybackActive,
        progress: Object.values(playbackProgress).find(p => (p as any).fileId) || null
    });
});

// GET /api/tts/status - Get TTS session status
app.get("/api/tts/status", (req, res) => {
    if (!currentTTSSession) {
        return res.json({ active: false });
    }
    res.json({
        active: true,
        isPlaying: currentTTSSession.isPlaying,
        position: currentTTSSession.currentPosition,
        totalLength: currentTTSSession.fullText.length,
        progressPercent: Math.round((currentTTSSession.currentPosition / currentTTSSession.fullText.length) * 100),
        consecutiveErrors: currentTTSSession.consecutiveErrors,
        sessionAgeMinutes: Math.round((Date.now() - currentTTSSession.sessionCreatedAt) / 60000),
        targetEntity: currentTTSSession.targetEntity
    });
});

// POST /api/tts/force-stop - Force stop any running TTS session
app.post("/api/tts/force-stop", (req, res) => {
    if (currentTTSSession) {
        stopTTSSession("Force stopped via API");
        currentTTSSession = null;
    }
    res.json({ success: true, message: "TTS session force stopped" });
});

// POST /api/shower/start-reading - Start TTS playback on Echo
app.post("/api/shower/start-reading", async (req, res) => {
    try {
        const { text, taskId, chunkIndex = 0 } = req.body;

        if (!text) {
            return res.status(400).json({ error: "Text content required" });
        }

        if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
            return res.status(500).json({ error: "Home Assistant not configured" });
        }

        // Chunk text for TTS (max ~500 chars for Echo TTS)
        const chunks = text.match(/.{1,450}[.!?]?[^\s]*(\s|$)/g) || [text];
        const chunk = chunks[chunkIndex] || chunks[0];

        // Update progress
        if (taskId) {
            playbackProgress[`task-${taskId}`] = {
                chunkIndex,
                totalChunks: chunks.length,
                lastPlayed: new Date()
            };
        }

        const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

        // Send TTS to Nest speaker
        const isNonAlexa = NON_ALEXA_ENTITIES.includes(NEST_SPEAKER_ENTITY);
        let response;
        if (isNonAlexa) {

```

```

const audioPath = await generateAndSaveTTSAudio(chunk, `start-reading-${Date.now()}`, "echo");
const appUrl = DEPLOYED_APP_URL;
response = await fetch(`${haUrl}/api/services/media_player/play_media`, {
  method: 'POST',
  headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
  body: JSON.stringify({ entity_id: NEST_SPEAKER_ENTITY, media_content_id: `${appUrl}${audioPath}`, media_con...
});
} else {
  response = await fetch(`${haUrl}/api/services/notify/alexa_media`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ message: chunk, target: NEST_SPEAKER_ENTITY, data: { type: "tts" } });
  });
}

if (!response.ok) {
  const errorText = await response.text();
  console.error("Home Assistant TTS error:", errorText);
  return res.status(response.status).json({ error: "Failed to send TTS" });
}

res.json({
  success: true,
  chunkIndex,
  totalChunks: chunks.length,
  hasMore: chunkIndex < chunks.length - 1
});
} catch (error) {
  console.error("Error starting reading:", error);
  res.status(500).json({ error: "Failed to start reading" });
}
});

// POST /api/shower/sync-onedrive - Auto-sync module/reading files from OneDrive for all weeks
app.post("/api/shower/sync-onedrive", async (req, res) => {
  try {
    const { listOneDriveItems } = await import("./onedrive");
    const { ObjectStorageService } = await import("./replit_integrations/object_storage");
    const objectStorage = new ObjectStorageService();

    const activeSemester = await storage.getActiveSemesterSettings();
    const allSemesters = await storage.getAllSemesterSettings();
    const semester = activeSemester || allSemesters[0];
    if (!semester) {
      return res.json({ success: true, message: 'No semester configured', synced: [] });
    }
    const semesterTypeMap: Record<string, string> = { winter: 'Winter', fall: 'Fall', spring_summer: 'Spring_Summer...' };
    const semesterFolder = semesterTypeMap[semester.semesterType] || semester.semesterType;
    const year = semester.semesterName?.match(/d{4}/)?.[0] || '2026';
    const basePath = `/School/1. TMU/Courses/${year}/${semesterFolder}`;
    const courseCodes = [semester.course1Code, semester.course2Code, semester.course3Code].filter(Boolean) as string[];
    console.log(`[Sync] Using semester: ${semester.semesterName}, path: ${basePath}, courses: ${courseCodes.join('...', ...

    // Get all existing files once to avoid repeated DB queries
    const existingFiles = await storage.getFiles();
    const existingFileKeys = new Set(
      existingFiles.map((f: any) => `${f.originalName}||${f.folder}`)
    );

    // List base folder to find course folders dynamically
    const baseFolders = await listOneDriveItems(basePath);

    const syncedFiles: any[] = [];
    const errors: any[] = [];

    for (const courseCode of courseCodes) {
      try {
        const matchedFolder = baseFolders.find((f: any) =>
          f.type === 'folder' && f.name.toUpperCase().startsWith(courseCode)
        );
        if (!matchedFolder) {
          console.log(`No OneDrive folder found for ${courseCode}`);
          continue;
        }

        const coursePath = matchedFolder.path;
        const weekFolders = await listOneDriveItems(coursePath);

        // Process ALL week folders (not just current week)
        for (const weekFolder of weekFolders) {
          if (weekFolder.type !== 'folder') continue;
          const weekMatch = weekFolder.name.match(/Week\s+(\d+)/i);
          if (!weekMatch) continue;
          const weekNum = parseInt(weekMatch[1], 10);

          const weekContents = await listOneDriveItems(weekFolder.path);

          // Process Module and Reading subfolders
          for (const subFolder of weekContents) {
            if (subFolder.type !== 'folder') continue;
            const subName = subFolder.name.toLowerCase();
            let type: string | null = null;
            if (subName.includes('module')) type = 'module';
            else if (subName.includes('reading')) type = 'reading';
            if (!type) continue;

            const folderName = `week-${weekNum}-${courseCode.toLowerCase()}-${type}`;
            const subFiles = await listOneDriveItems(subFolder.path);

            for (const file of subFiles) {
              if (file.type !== 'file' || !file.name.toLowerCase().endsWith('.pdf')) continue;

              const fileKey = `${file.name}||${folderName}`;
              if (existingFileKeys.has(fileKey)) continue;

              try {
                const downloadResponse = await fetch(file.downloadUrl);
                if (!downloadResponse.ok) {

```

```

        errors.push({ file: file.name, week: weekNum, error: 'Download failed' });
        continue;
    }

    const fileBuffer = Buffer.from(await downloadResponse.arrayBuffer());
    const uploadUrl = await objectStorage.getObjectEntityUploadURL();
    const uploadResponse = await fetch(uploadUrl, {
        method: 'PUT',
        body: fileBuffer,
        headers: { 'Content-Type': 'application/pdf' }
    });

    if (!uploadResponse.ok) {
        errors.push({ file: file.name, week: weekNum, error: 'Upload to storage failed' });
        continue;
    }

    const objectPath = objectStorage.normalizeObjectEntityPath(uploadUrl);

    const newFile = await storage.createFile({
        originalName: file.name,
        displayName: file.name,
        objectPath: objectPath,
        contentType: 'application/pdf',
        size: file.size,
        folder: folderName,
        listened: false
    });

    existingFileKeys.add(fileKey);

    let textLength = 0;
    let totalChunks = 0;
    try {
        const extractedText = await extractFileText({ ...newFile, objectPath });
        if (extractedText) {
            textLength = extractedText.length;
            totalChunks = Math.ceil(extractedText.length / CHUNK_SIZE);
            await storage.updateFile(newFile.id, { totalChunks, extractedText });
        }
    } catch (parseErr: any) {
        console.error(`[Sync] Text extraction failed for ${file.name}:`, parseErr.message);
    }

    const preparedEntry = {
        id: newFile.id,
        name: file.name,
        folder: folderName,
        totalChunks,
        textLength,
        preparedAt: new Date().toISOString(),
    };
    recentlyPreparedFiles.push(preparedEntry);

    syncedFiles.push({ name: file.name, folder: folderName, course: courseCode, week: weekNum });
    console.log(`[Sync] New file: ${file.name} -> ${folderName} (${totalChunks} chunks, ${textLength} c...`);
    } catch (fileErr: any) {
        errors.push({ file: file.name, week: weekNum, error: fileErr.message });
    }
}
}
} catch (courseError: any) {
    errors.push({ course: courseCode, error: courseError.message });
}
}

res.json({
    success: true,
    totalSynced: syncedFiles.length,
    synced: syncedFiles,
    errors: errors.length > 0 ? errors : undefined
});
} catch (error: any) {
    console.error("Error syncing OneDrive files:", error);
    res.status(500).json({ error: "Failed to sync OneDrive files", details: error.message });
}
});

app.post("/api/course-week-upload", async (req, res) => {
    try {
        const courseCode = (req.headers['x-course-code'] as string || '').trim();
        const courseName = (req.headers['x-course-name'] as string || '').trim();
        const weekNum = parseInt(req.headers['x-week-num'] as string || '0', 10);
        const uploadType = (req.headers['x-upload-type'] as string || 'module').toLowerCase();
        const weekDateRange = (req.headers['x-week-date-range'] as string || '').trim();
        const fileName = (req.headers['x-file-name'] as string || 'upload.pdf').trim();
        const contentType = (req.headers['content-type'] as string || 'application/pdf');

        if (!courseCode || !weekNum || !weekDateRange) {
            return res.status(400).json({ error: 'Missing required headers: x-course-code, x-week-num, x-week-date-range...' });
        }

        if (uploadType !== 'module' && uploadType !== 'reading') {
            return res.status(400).json({ error: 'x-upload-type must be "module" or "reading"' });
        }

        const chunks: Buffer[] = [];
        req.on('data', (chunk: Buffer) => chunks.push(chunk));
        await new Promise<void>((resolve, reject) => {
            req.on('end', resolve);
            req.on('error', reject);
        });
        const fileBuffer = Buffer.concat(chunks);

        if (fileBuffer.length === 0) {
            return res.status(400).json({ error: 'Empty file' });
        }
    }
}

```

```

const clientYear = (req.headers['x-semester-year'] as string || '').trim();
const clientSemType = (req.headers['x-semester-type'] as string || '').trim();

let year: number;
let semType: string;
if (clientYear && clientSemType) {
  year = parseInt(clientYear, 10);
  semType = getSemesterTypeFolder(clientSemType);
} else {
  const semesterSettings = await storage.getSemesterSchedule();
  semType = getSemesterTypeFolder(semesterSettings?.semesterType);
  const startDate = semesterSettings?.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : new Da...
  year = startDate.getFullYear();
}

const codeClean = courseCode.replace(/s/g, '');
const courseFolderName = courseName ? `${codeClean} - ${courseName}` : codeClean;
const weekFolderName = `Week ${weekNum} - ${weekDateRange}`;
const subFolder = uploadType === 'module' ? 'Module' : 'Reading';
const oneDrivePath = `/School/1. TMU/Courses/${year}/${semType}/${courseFolderName}/${weekFolderName}/${subFold...

console.log(`[CourseUpload] Uploading ${fileName} to OneDrive: ${oneDrivePath}`);

const { uploadOneDriveFile, createOneDriveFolder } = await import("./onedrive");

const basePath = `/School/1. TMU/Courses/${year}/${semType}`;
await createOneDriveFolder(basePath, courseFolderName);
await createOneDriveFolder(`${basePath}/${courseFolderName}`, weekFolderName);
await createOneDriveFolder(`${basePath}/${courseFolderName}/${weekFolderName}`, subFolder);

const oneDriveResult = await uploadOneDriveFile(oneDrivePath, fileName, fileBuffer, contentType);
console.log(`[CourseUpload] OneDrive upload success: ${fileName} !' ${oneDrivePath} (${fileBuffer.length} bytes)`);

const { ObjectStorageService } = await import("./replit_integrations/object_storage");
const objectStorage = new ObjectStorageService();
const uploadUrl = await objectStorage.getObjectEntityUploadURL();
const uploadResponse = await fetch(uploadUrl, {
  method: 'PUT',
  body: fileBuffer,
  headers: { 'Content-Type': contentType }
});
if (!uploadResponse.ok) {
  console.error(`[CourseUpload] Object storage upload failed: ${uploadResponse.status}`);
  return res.status(500).json({ error: 'Object storage upload failed' });
}
const objectPath = objectStorage.normalizeObjectEntityPath(uploadUrl);

const folderName = `week-${weekNum}-${codeClean.toLowerCase()}-${uploadType}`;
const newFile = await storage.createFile({
  originalName: fileName,
  displayName: fileName,
  objectPath,
  contentType,
  size: fileBuffer.length,
  folder: folderName,
  listened: false,
});
console.log(`[CourseUpload] File record created: ${fileName} !' ${folderName} (id: ${newFile?.id})`);

if (newFile?.id) {
  queueFileForPreparation(newFile.id);
  console.log(`[CourseUpload] Queued ${fileName} for TTS preparation`);
}

res.json({
  success: true,
  fileId: newFile?.id,
  folder: folderName,
  oneDrivePath: `${oneDrivePath}/${fileName}`,
  oneDriveWebUrl: oneDriveResult?.webUrl,
  objectPath,
});
} catch (error: any) {
  console.error("[CourseUpload] Error:", error);
  res.status(500).json({ error: error.message || "Failed to upload course file" });
}
});

const recentlyPreparedFiles: { id: number; name: string; folder: string; totalChunks: number; textLength: number; p...
(globalThis as any).__recentlyPreparedFiles = recentlyPreparedFiles;

app.post("/api/files/prepare-audio", async (req, res) => {
  try {
    const { fileId } = req.body || {};
    if (fileId) {
      queueFileForPreparation(fileId);
      return res.json({ action: "queued", fileId });
    }
    const allFiles = await storage.GetFiles();
    const unprepared = allFiles.filter((f: any) => !f.preparedAudioPaths && f.folder && (f.folder.includes('-module...
    for (const file of unprepared) {
      queueFileForPreparation(file.id);
    }
    res.json({ action: "queued", count: unprepared.length, files: unprepared.map((f: any) => ({ id: f.id, name: f.o...
  } catch (e: any) {
    res.status(500).json({ error: e.message });
  }
});

const TTS_FILTER_VERSION = 4;
setTimeout(async () => {
  try {
    const rows = await db.select().from(appState).where(eq(appState.key, 'tts_filter_version'));
    const currentVersion = rows.length > 0 ? parseInt(rows[0].value) : 0;
    if (currentVersion < TTS_FILTER_VERSION) {
      await db.execute(sql`UPDATE files SET extracted_text = NULL, prepared_audio_paths = NULL, prepared_at = NULL ...
      await db.insert(appState).values({ key: 'tts_filter_version', value: String(TTS_FILTER_VERSION) })
      .onConflictDoUpdate({ target: appState.key, set: { value: String(TTS_FILTER_VERSION) } });
    }
  }
});

```

```

    console.log(`[Startup] TTS filter version ${currentVersion} != ${TTS_FILTER_VERSION}: cleared cached text and ...
  }
} catch (e: any) {
  console.error(`[Startup] Filter version check failed: ${e.message}`);
}

try {
  const allFiles = await storage.GetFiles();
  const unprepared = allFiles.filter((f: any) => !f.preparedAudioPaths && f.folder && (f.folder.includes('-module...
  if (unprepared.length > 0) {
    console.log(`[AudioPrep] Startup scan: ${unprepared.length} unprepared files found - queueing`);
    for (const file of unprepared) {
      queueFileForPreparation(file.id);
    }
  } else {
    console.log(`[AudioPrep] Startup scan: all files already prepared`);
  }
} catch (e: any) {
  console.error(`[AudioPrep] Startup scan error: ${e.message}`);
}
}, 30000);

app.post("/api/files/monitor-sync", async (req, res) => {
  try {
    const { listOneDriveItems, getOneDriveFile } = await import("./onedrive");
    const { ObjectStorageService } = await import("./replit_integrations/object_storage");
    const objectStorage = new ObjectStorageService();

    const allSemesters = await storage.getAllSemesterSettings();
    const springSummer = allSemesters.find(s => s.semesterType === 'spring_summer' && s.semesterName?.includes('202...
    if (!springSummer) {
      return res.json({ success: true, message: 'No Spring/Summer 2026 semester found', synced: [] });
    }

    const courseCodes = [springSummer.course1Code, springSummer.course2Code, springSummer.course3Code].filter(Boole...
    const basePath = `/School/1. TMU/Courses/2026/Spring_Summer`;

    const existingFiles = await storage.GetFiles();
    const existingFileKeys = new Set(
      existingFiles.map((f: any) => `${f.originalName}||${f.folder}`)
    );

    let baseFolders: any[] = [];
    try {
      baseFolders = await listOneDriveItems(basePath);
    } catch (e: any) {
      return res.json({ success: true, message: 'OneDrive Spring_Summer folder not found yet', synced: [] });
    }

    const syncedFiles: any[] = [];
    const errors: any[] = [];

    for (const courseCode of courseCodes) {
      try {
        const matchedFolder = baseFolders.find((f: any) =>
          f.type === 'folder' && f.name.toUpperCase().startsWith(courseCode.toUpperCase())
        );
        if (!matchedFolder) continue;

        const coursePath = matchedFolder.path;
        const weekFolders = await listOneDriveItems(coursePath);

        for (const weekFolder of weekFolders) {
          if (weekFolder.type !== 'folder') continue;
          const weekMatch = weekFolder.name.match(/Week\s+(\d+)/i);
          if (!weekMatch) continue;
          const weekNum = parseInt(weekMatch[1], 10);

          const weekContents = await listOneDriveItems(weekFolder.path);

          for (const subfolder of weekContents) {
            if (subfolder.type !== 'folder') continue;
            const subName = subfolder.name.toLowerCase();
            let type: string | null = null;
            if (subName.includes('module')) type = 'module';
            else if (subName.includes('reading')) type = 'reading';
            if (!type) continue;

            const folderName = `week-${weekNum}-${courseCode.toLowerCase()}-${type}`;
            const subFiles = await listOneDriveItems(subfolder.path);

            for (const file of subFiles) {
              if (file.type !== 'file' || !file.name.toLowerCase().endsWith('.pdf')) continue;

              const fileKey = `${file.name}||${folderName}`;
              if (existingFileKeys.has(fileKey)) continue;

              try {
                const downloadResponse = await fetch(file.downloadUrl);
                if (!downloadResponse.ok) {
                  errors.push({ file: file.name, week: weekNum, error: 'Download failed' });
                  continue;
                }

                const fileBuffer = Buffer.from(await downloadResponse.arrayBuffer());

                const uploadUrl = await objectStorage.getObjectEntityUploadURL();
                const uploadResponse = await fetch(uploadUrl, {
                  method: 'PUT',
                  body: fileBuffer,
                  headers: { 'Content-Type': 'application/pdf' },
                });

                if (!uploadResponse.ok) {
                  errors.push({ file: file.name, week: weekNum, error: 'Upload to storage failed' });
                  continue;
                }
              }
            }
          }
        }
      }
    }

    return res.json({ success: true, message: 'Synced files', synced: syncedFiles, errors });
  }
});

```

```

const objectPath = objectStorage.normalizeObjectEntityPath(uploadUrl);

const newFile = await storage.createFile({
  originalName: file.name,
  displayName: file.name,
  objectPath,
  contentType: 'application/pdf',
  size: file.size,
  folder: folderName,
  listened: false,
});

existingFileKeys.add(fileKey);

let textLength = 0;
let totalChunks = 0;
try {
  const PdfParser = await getPdfParser();
  const parsed = await PdfParser(fileBuffer);
  let textContent = '';
  if (parsed && typeof parsed === 'object') {
    if (parsed.text) textContent = parsed.text;
    else if ((parsed as any).pages && Array.isArray((parsed as any).pages)) {
      textContent = (parsed as any).pages.map((p: any) => p.text || '').join('\n\n');
    }
  } else if (typeof parsed === 'string') {
    textContent = parsed;
  }
  const cleaned = cleanTextForTTS(textContent);
  textLength = cleaned.length;
  totalChunks = Math.ceil(cleaned.length / CHUNK_SIZE);

  await storage.updateFile(newFile.id, { totalChunks, extractedText: cleaned });
} catch (parseErr: any) {
  console.error('[Monitor] Failed to extract text from ${file.name}:', parseErr.message);
}

const preparedEntry = {
  id: newFile.id,
  name: file.name,
  folder: folderName,
  totalChunks,
  textLength,
  preparedAt: new Date().toISOString(),
};
recentlyPreparedFiles.push(preparedEntry);
syncdFiles.push(preparedEntry);

console.log('[Monitor] Synced & prepared: ${file.name} -> ${folderName} (${totalChunks} chunks, ${t...
} catch (fileErr: any) {
  errors.push({ file: file.name, week: weekNum, error: fileErr.message });
}
}
}
} catch (courseError: any) {
  errors.push({ course: courseCode, error: courseError.message });
}
}

res.json({
  success: true,
  totalSynced: syncdFiles.length,
  syncd: syncdFiles,
  errors: errors.length > 0 ? errors : undefined,
});
} catch (error: any) {
  console.error("[Monitor] Error syncing Spring/Summer files:", error);
  res.status(500).json({ error: "Failed to monitor Spring/Summer files", details: error.message });
}
});

app.post("/api/files/pre-extract", async (_req, res) => {
  try {
    const allFiles = await storage.getFiles();
    const needExtraction = allFiles.filter((f: any) => !f.extractedText && f.objectPath);
    console.log('[Pre-Extract] Found ${needExtraction.length} files needing text extraction');

    if (needExtraction.length === 0) {
      return res.json({ success: true, message: 'All files already have cached text', extracted: 0 });
    }

    res.json({ success: true, message: `Extracting ${needExtraction.length} files in background`, queued: needExtra...

    (async () => {
      let extracted = 0;
      let failed = 0;
      for (const file of needExtraction) {
        try {
          const text = await extractFileText(file);
          if (text) {
            extracted++;
            console.log('[Pre-Extract] ${extracted}/${needExtraction.length}: ${file.originalName} (${text.length} ...
          } else {
            failed++;
            console.log('[Pre-Extract] Failed: ${file.originalName} (no text)`);
          }
        } catch (e: any) {
          failed++;
          console.error('[Pre-Extract] Error: ${file.originalName}: ${e.message}`);
        }
      }
      console.log('[Pre-Extract] Complete: ${extracted} extracted, ${failed} failed`);
    })();
  } catch (error: any) {
    console.error("[Pre-Extract] Error:", error);
    res.status(500).json({ error: error.message });
  }
}

```

```

});

app.get("/api/files/recently-prepared", async (_req, res) => {
  res.json({ files: recentlyPreparedFiles });
});

app.post("/api/files/acknowledge-prepared", async (req, res) => {
  const { fileIds } = req.body;
  if (fileIds && Array.isArray(fileIds)) {
    for (let i = recentlyPreparedFiles.length - 1; i >= 0; i--) {
      if (fileIds.includes(recentlyPreparedFiles[i].id)) {
        recentlyPreparedFiles.splice(i, 1);
      }
    }
  } else {
    recentlyPreparedFiles.length = 0;
  }
  res.json({ success: true });
});

// POST /api/echo/tts - Send text-to-speech to Home Assistant Echo device
app.post("/api/echo/tts", async (req, res) => {
  try {
    const { message } = req.body;

    if (!message) {
      return res.status(400).json({ error: "Message is required" });
    }

    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }

    const baseUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

    // Use Nest speaker (non-Alexa) for cat washroom TTS
    const audioPath = await generateAndSaveTTSAudio(message, `echo-tts-${Date.now()}`, "echo");
    const apiUrl = DEPLOYED_APP_URL;
    const response = await fetch(`${baseUrl}/api/services/media_player/play_media`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ entity_id: NEST_SPEAKER_ENTITY, media_content_id: `${apiUrl}${audioPath}`, media_conte...
    });

    if (!response.ok) {
      const errorText = await response.text();
      console.error("Home Assistant TTS error:", errorText);
      return res.status(response.status).json({ error: "Failed to send TTS to Home Assistant" });
    }

    res.json({ success: true, message: "Text-to-speech sent to Nest speaker" });
  } catch (error) {
    console.error("TTS error:", error);
    res.status(500).json({ error: "Failed to send text-to-speech" });
  }
});

// POST /api/media/play - Extract text from PDF and read it aloud via TTS
app.post("/api/media/play", async (req, res) => {
  try {
    const { mediaUrl, entityId } = req.body;
    const targetEntity = entityId || NEST_SPEAKER_ENTITY;

    if (!mediaUrl) {
      return res.status(400).json({ error: "Media URL is required" });
    }

    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }

    // Read file content from object storage
    let textContent = "";
    let fileBuffer: Buffer | null = null;

    if (mediaUrl.startsWith("/objects/")) {
      // It's an object storage path - read from object storage
      const { ObjectStorageService } = await import("../replit_integrations/object_storage");
      const objectStorage = new ObjectStorageService();

      try {
        const objectFile = await objectStorage.getObjectEntityFile(mediaUrl);

        // Download the file content as buffer
        const [content] = await objectFile.download();
        fileBuffer = content;
      } catch (error) {
        console.error("Error reading from object storage:", error);
        return res.status(400).json({ error: "Failed to read file from storage" });
      }
    } else {
      // It's a regular URL - fetch it
      const fileResponse = await fetch(mediaUrl);
      if (!fileResponse.ok) {
        return res.status(400).json({ error: "Failed to fetch file content" });
      }
      const arrayBuffer = await fileResponse.arrayBuffer();
      fileBuffer = Buffer.from(arrayBuffer);
    }

    if (!fileBuffer) {
      return res.status(400).json({ error: "Failed to read file" });
    }

    // Check if it's a PDF by looking at the magic bytes
    const isPDF = fileBuffer.slice(0, 4).toString() === '%PDF';

    if (isPDF) {

```



```

// Parse PDF and extract text using PDFParse class
try {
  const PdfParser = await getPdfParser();
  const parser = new PdfParser({ data: new Uint8Array(fileBuffer), verbosity: 0 });
  const pdfText = await parser.getText();
  // getText() returns an object with pages array containing text
  if (pdfText && typeof pdfText === 'object') {
    if (pdfText.pages && Array.isArray(pdfText.pages)) {
      // Extract text from each page
      textContent = pdfText.pages.map((page: any) => page.text || '').join(' ');
    } else if (Array.isArray(pdfText)) {
      textContent = pdfText.map((item: any) => typeof item === 'string' ? item : item.text || '').join(' ');
    } else if (pdfText.text) {
      textContent = pdfText.text;
    } else {
      // Fallback - try to extract any text properties
      textContent = Object.values(pdfText).filter(v => typeof v === 'string').join(' ');
    }
  } else if (typeof pdfText === 'string') {
    textContent = pdfText;
  } else {
    textContent = String(pdfText || '');
  }
  await parser.destroy();
} catch (error) {
  console.error("Error parsing PDF:", error);
  return res.status(400).json({ error: "Failed to parse PDF" });
}
} else {
  // Treat as plain text
  textContent = fileBuffer.toString('utf-8');
}

if (!textContent || textContent.trim().length === 0) {
  return res.status(400).json({ error: "File is empty or not readable" });
}

// Skip first page if it's a title/cover page (< 300 words with academic keywords)
const titlePageKeywords = /jstor|published|publisher|author[s]?|doi:|copyright|issn|isbn|volume\s+\d|issue\s+...
const firstPageBreak = textContent.indexOf('---PAGE---');
if (firstPageBreak > 0) {
  const firstPageContent = textContent.substring(0, firstPageBreak);
  const firstPageWordCount = firstPageContent.split(/\s+/).length;
  if (firstPageWordCount < 300 && titlePageKeywords.test(firstPageContent.toLowerCase())) {
    textContent = textContent.substring(firstPageBreak + 10);
    console.log("Skipped first page (title page detected)");
  }
}

// Also skip learning objectives section at the beginning
const learningObjMatch = textContent.match(/^[\s\S]*?(?:Learning Objectives?|By the end of this (?:module|chapt...
if (learningObjMatch && learningObjMatch[0].length < 2000) {
  const afterObjectives = textContent.substring(learningObjMatch[0].length);
  if (afterObjectives.trim().length > 500) {
    textContent = afterObjectives;
    console.log("Skipped learning objectives section");
  }
}

const baseUrl = HOME_ASSISTANT_URL.replace(/\/$/, ' ');

// Clean up the text for TTS
let cleanedContent = textContent.trim();
// Remove excessive whitespace and newLines
cleanedContent = cleanedContent.replace(/\s+/g, ' ');
// Remove special Unicode characters that cause Simon Says error
cleanedContent = cleanedContent
  .replace(/[\u2018\u2019]/g, '"')
  .replace(/[\u201C\u201D]/g, '"')
  .replace(/[\u2013\u2014]/g, '-')
  .replace(/[\x20-\x7E]/g, ' ');
cleanedContent = cleanedContent.replace(/\s+/g, ' ').trim();
// Limit length for TTS - Alexa can handle up to ~3000 chars per message
if (cleanedContent.length > 3000) {
  cleanedContent = cleanedContent.substring(0, 3000);
}
// Save session for resume functionality - store full cleaned text (up to 100,000 chars)
// Apply the same French/JSTOR filtering to the full text so all chunks are clean
const fullCleanedText = cleanTextForTTS(textContent);

currentTTSSession = {
  fullText: fullCleanedText.length > 100000 ? fullCleanedText.substring(0, 100000) : fullCleanedText,
  currentPosition: 0,
  startTime: Date.now(),
  isPlaying: true,
  autoTimer: null,
  consecutiveErrors: 0,
  sessionCreatedAt: Date.now()
};

// Use simple message without prefix for Simon Says compatibility
// cleanedContent = "Now reading your document. " + cleanedContent;

// Use TTS type like the working /api/tts endpoint
console.log("TTS content preview:", cleanedContent.substring(0, 200));
console.log("Total document length:", fullCleanedText.length, "characters");

// Store the target entity in session for resume
currentTTSSession.targetEntity = targetEntity;

// Restore volume in case it was muted by previous stop
await fetch(`${baseUrl}/api/services/media_player/volume_set`, {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    entity_id: targetEntity,

```

```

        volume_level: 0.5
    }},
  });

// Clean and chunk the content
cleanedContent = cleanTextForTTS(cleanedContent);
cleanedContent = getChunkWithSentenceBoundary(cleanedContent, CHUNK_SIZE);

console.log("=== TTS PLAY REQUEST ===");
console.log("Target entity:", targetEntity);
console.log("Cleaned message length:", cleanedContent.length);
console.log("HA URL:", haUrl);
console.log("Message preview:", cleanedContent.substring(0, 100));

const isNonAlexa = NON_ALEXA_ENTITIES.includes(targetEntity);
let response: Response;
if (isNonAlexa) {
  const audioPath = await generateAndSaveTTSAudio(cleanedContent, `tts-play-${Date.now()}`, "echo");
  const appUrl = DEPLOYED_APP_URL;
  console.log(`[TTS Play] Non-Alexa: Generated audio at ${audioPath}`);
  response = await fetch(`${haUrl}/api/services/media_player/play_media`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ entity_id: targetEntity, media_content_id: `${appUrl}${audioPath}`, media_content_ty...
  });
} else {
  const ssmContent = `<prosody rate="90%">${cleanedContent}</prosody></speak>`;
  response = await fetch(`${haUrl}/api/services/notify/alexa_media`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ message: ssmContent, target: targetEntity, data: { type: "tts" } })
  });
}

const responseText = await response.text();
console.log("HA Response status:", response.status);
console.log("HA Response:", responseText);

if (!response.ok) {
  console.error("Home Assistant TTS error:", responseText);
  currentTTSSession = null;
  return res.status(response.status).json({ error: "Failed to read file content" });
}

// Update position AFTER sending first chunk - advance by the chunk length we just sent
currentTTSSession.currentPosition = cleanedContent.length;
console.log("First chunk sent, updated position to:", currentTTSSession.currentPosition);

// Schedule automatic continuation for the rest of the document
scheduleNextChunk();

res.json({ success: true, message: "Reading PDF content aloud", targetEntity });
} catch (error) {
  console.error("Play media error:", error);
  res.status(500).json({ error: "Failed to play media" });
}
});

// POST /api/media/stop - Stop media playback on Echo device
app.post("/api/media/stop", async (req, res) => {
  try {
    const { entityId } = req.body || {};
    const targetEntity = entityId || currentTTSSession?.targetEntity || NEST_SPEAKER_ENTITY;

    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }

    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

    // Cancel auto-continuation timer and clear session completely
    if (currentTTSSession) {
      // Clear the auto timer
      if (currentTTSSession.autoTimer) {
        clearTimeout(currentTTSSession.autoTimer);
        currentTTSSession.autoTimer = null;
      }

      // Calculate position before clearing
      if (currentTTSSession.isPlaying) {
        const elapsedSeconds = (Date.now() - currentTTSSession.startTime) / 1000;
        const charsRead = Math.floor(elapsedSeconds * CHARS_PER_SECOND);
        currentTTSSession.currentPosition = Math.min(
          currentTTSSession.currentPosition + charsRead,
          currentTTSSession.fullText.length
        );
        console.log(`TTS stopped at position ${currentTTSSession.currentPosition} of ${currentTTSSession.fullText.l...
      }

      // Mark as stopped
      currentTTSSession.isPlaying = false;
      // Clear the session entirely to prevent further chunks
      currentTTSSession = null;
    }

    // Send stop command to the media player
    await fetch(`${haUrl}/api/services/media_player/media_stop`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        entity_id: targetEntity,
      }),
    });
  } catch (error) {
    console.log("Stopped media player");
  }
});

```

```

// Also try the Alexa-specific stop command (only for Alexa devices)
if (!NON_ALEXA_ENTITIES.includes(targetEntity)) {
  try {
    await fetch(`${haUrl}/api/services/notify/alexa_media`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ message: "stop", target: targetEntity, data: { type: "tts" } }),
    });
  } catch (e) {
    console.log("Alexa media stop command not available");
  }
}

res.json({ success: true, canResume: false });
} catch (error) {
  console.error("Stop error:", error);
  res.status(500).json({ error: "Failed to stop media" });
}
});

// POST /api/media/resume - Resume TTS from where it was stopped
app.post("/api/media/resume", async (_req, res) => {
  try {
    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(500).json({ error: "Home Assistant not configured" });
    }

    if (!currentTTSSession || currentTTSSession.currentPosition >= currentTTSSession.fullText.length) {
      return res.status(400).json({ error: "Nothing to resume" });
    }

    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

    // Restore volume in case it was muted by stop
    await fetch(`${haUrl}/api/services/media_player/volume_set`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        entity_id: currentTTSSession.targetEntity || NEST_SPEAKER_ENTITY,
        volume_level: 0.5
      }),
    });

    // Get remaining text from current position
    let remainingText = currentTTSSession.fullText.substring(currentTTSSession.currentPosition);

    if (remainingText.trim().length === 0) {
      return res.status(400).json({ error: "Already finished reading" });
    }

    // Clean the text the same way as Play
    remainingText = remainingText
      .replace(/[\u2018\u2019]/g, '')
      .replace(/[\u201C\u201D]/g, '')
      .replace(/[\u2013\u2014]/g, '-')
      .replace(/[\u2020-\u207E]/g, '');
    remainingText = remainingText.replace(/s+/g, ' ').trim();

    // Limit length for TTS (about 4 minutes of reading)
    if (remainingText.length > 3000) {
      remainingText = remainingText.substring(0, 3000);
    }

    // Add prefix
    remainingText = "Continuing. " + remainingText;

    // Update session
    currentTTSSession.startTime = Date.now();
    currentTTSSession.isPlaying = true;
    currentTTSSession.consecutiveErrors = 0;

    console.log("Resuming TTS from position", currentTTSSession.currentPosition, "preview:", remainingText.substrin...);

    const targetEntity = currentTTSSession.targetEntity || NEST_SPEAKER_ENTITY;
    const isNonAlexa = NON_ALEXA_ENTITIES.includes(targetEntity);
    let response: Response;
    if (isNonAlexa) {
      const audioPath = await generateAndSaveTTSAudio(remainingText, `tts-resume-${Date.now()}`, "echo");
      const appUrl = DEPLOYED_APP_URL;
      response = await fetch(`${haUrl}/api/services/media_player/play_media`, {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
        body: JSON.stringify({ entity_id: targetEntity, media_content_id: `${appUrl}${audioPath}`, media_content_ty...
      });
    } else {
      const ssmlContent = `<speak><prosody rate="90%">${remainingText}</prosody></speak>`;
      response = await fetch(`${haUrl}/api/services/notify/alexa_media`, {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
        body: JSON.stringify({ message: ssmlContent, target: targetEntity, data: { type: "tts" } }),
      });
    }

    if (!response.ok) {
      const errorText = await response.text();
      console.error("Home Assistant resume error:", errorText);
      return res.status(response.status).json({ error: "Failed to resume" });
    }

    // Schedule automatic continuation for the rest of the document
    scheduleNextChunk();

    res.json({ success: true, message: "Resumed reading" });
  } catch (error) {

```

```

        console.error("Resume error:", error);
        res.status(500).json({ error: "Failed to resume" });
    }
});

// POST /api/media/restart - Restart TTS from beginning
app.post("/api/media/restart", async (req, res) => {
    try {
        const { mediaUrl, entityId } = req.body;
        const targetEntity = entityId || NEST_SPEAKER_ENTITY;

        if (!mediaUrl) {
            return res.status(400).json({ error: "Media URL is required" });
        }

        if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
            return res.status(500).json({ error: "Home Assistant not configured" });
        }

        const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

        // Clear any existing session
        if (currentTTSSession) {
            if (currentTTSSession.autoTimer) {
                clearTimeout(currentTTSSession.autoTimer);
            }
            currentTTSSession = null;
        }

        // Restore volume in case it was muted
        await fetch(`${haUrl}/api/services/media_player/volume_set`, {
            method: 'POST',
            headers: {
                'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
                'Content-Type': 'application/json',
            },
            body: JSON.stringify({
                entity_id: targetEntity,
                volume_level: 0.5
            }),
        });

        // Re-trigger play from the beginning by calling the play endpoint logic
        // This is a simplified restart - just call the play endpoint
        res.json({ success: true, message: "Use play endpoint to restart" });
    } catch (error) {
        console.error("Restart error:", error);
        res.status(500).json({ error: "Failed to restart" });
    }
});

// POST /api/media/skip-chunk - Skip to next or previous chunk in TTS
app.post("/api/media/attention-prompt", async (req, res) => {
    try {
        const appUrl = DEPLOYED_APP_URL;
        const promptPath = await generateAndSaveTTSAudio("Bryn, are you paying attention?", `attention-prompt-${Date.now()}`);
        await playOnNestSpeaker(`${appUrl}${promptPath}`);
        console.log(`[Attention Prompt] Played on Nest speaker`);
        res.json({ ok: true });
    } catch (error: any) {
        console.error(`[Attention Prompt] Error: ${error.message}`);
        res.status(500).json({ error: error.message });
    }
});

app.post("/api/media/skip-chunk", async (req, res) => {
    try {
        const { direction, entityId } = req.body; // "forward" or "backward"
        const targetEntity = entityId || currentTTSSession?.targetEntity || NEST_SPEAKER_ENTITY;

        if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
            return res.status(500).json({ error: "Home Assistant not configured" });
        }

        if (!currentTTSSession) {
            return res.status(400).json({ error: "No active TTS session" });
        }

        const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

        // Clear the auto timer
        if (currentTTSSession.autoTimer) {
            clearTimeout(currentTTSSession.autoTimer);
            currentTTSSession.autoTimer = null;
        }

        // Calculate new position based on direction
        if (direction === "forward") {
            // Skip forward by one chunk
            currentTTSSession.currentPosition = Math.min(
                currentTTSSession.currentPosition + CHUNK_SIZE,
                currentTTSSession.fullText.length
            );
        } else {
            // Skip backward by one chunk
            currentTTSSession.currentPosition = Math.max(
                currentTTSSession.currentPosition - CHUNK_SIZE,
                0
            );
        }

        console.log(`Skipping ${direction} to position ${currentTTSSession.currentPosition}`);

        // If we have more content, send the next chunk immediately
        if (currentTTSSession.currentPosition < currentTTSSession.fullText.length) {
            currentTTSSession.isPlaying = true;
            currentTTSSession.consecutiveErrors = 0;
            currentTTSSession.startTime = Date.now();
        }
    }
});

```

```

        sendNextChunk();
        res.json({ success: true, position: currentTTSession.currentPosition });
    } else {
        currentTTSession.isPlaying = false;
        res.json({ success: true, message: "Reached end of document" });
    }
} catch (error) {
    console.error("Skip chunk error:", error);
    res.status(500).json({ error: "Failed to skip chunk" });
}
});

// POST /api/media/seek - Skip forward or backward on Echo device
app.post("/api/media/seek", async (req, res) => {
    try {
        const { direction } = req.body; // "forward" or "backward"

        if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
            return res.status(500).json({ error: "Home Assistant not configured" });
        }

        const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

        // Use Alexa Media Player's skip commands
        const service = direction === "forward" ? "media_next_track" : "media_previous_track";

        const response = await fetch(`${haUrl}/api/services/media_player/${service}`, {
            method: 'POST',
            headers: {
                'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
                'Content-Type': 'application/json',
            },
            body: JSON.stringify({
                entity_id: NEST_SPEAKER_ENTITY,
            }),
        });

        if (!response.ok) {
            const errorText = await response.text();
            console.error("Home Assistant seek error:", errorText);
            return res.status(response.status).json({ error: "Failed to seek" });
        }

        res.json({ success: true });
    } catch (error) {
        console.error("Seek error:", error);
        res.status(500).json({ error: "Failed to seek" });
    }
});

// POST /api/media/play-radio - Play TuneIn radio station on Alexa
app.post("/api/media/play-radio", async (req, res) => {
    try {
        const { stationId, entityId } = req.body;

        if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
            return res.status(400).json({ error: "Home Assistant not configured" });
        }

        const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

        // Use the entity passed from the frontend
        const device = entityId || "media_player.echo_lr_studio_white_am";

        console.log(`Playing CHUM FM on ${device}`);

        const response = await fetch(`${haUrl}/api/services/media_player/play_media`, {
            method: 'POST',
            headers: {
                'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
                'Content-Type': 'application/json',
            },
            body: JSON.stringify({
                entity_id: device,
                media_content_id: "play 104.5 chumfm",
                media_content_type: "custom"
            }),
        });

        const responseText = await response.text();
        console.log(`Response (${response.status}):`, responseText);

        res.json({ success: true });
    } catch (error) {
        console.error("Play radio error:", error);
        res.status(500).json({ error: "Failed to play radio" });
    }
});

// POST /api/media/play-radio-all - Play radio on ALL Echo devices
app.post("/api/media/play-radio-all", async (req, res) => {
    try {
        if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
            return res.status(400).json({ error: "Home Assistant not configured" });
        }

        const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

        // All Echo devices (excluding lr studio and lr couch left)
        const devices = [
            NEST_SPEAKER_ENTITY,
            ...CAT_ECHO_ENTITIES,
            "media_player.echo_closet_am",
            "media_player.echo_lr_couch_r_am",
            "media_player.echo_hallway_entrance_am",
            "media_player.echo_king_l_am",
            "media_player.echo_king_r_am",
            "media_player.echo_king_tv_am",
        ];
    }
});

```

```

    "media_player.echo_kitchen_cupboards_left_am",
    "media_player.echo_kitchen_cupboards_r_am",
    "media_player.echo_kitchen_fridge_am",
    "media_player.echo_kitchen_hutch_am",
    "media_player.echo_kitchen_island_corner_am",
    "media_player.echo_kitchen_studio_black_am",
    "media_player.echo_lr_hub_am",
    "media_player.echo_lr_tv_shelf_am",
    "media_player.echo_queen_balcony_am",
    "media_player.echo_queen_bed_l_am",
    "media_player.echo_queen_bed_r_am",
    "media_player.echo_show_pug_am"
  ];

  console.log(`Playing CHUM FM on ALL devices: ${devices.join(', ')}`);

  for (const device of devices) {
    await fetch(`${haUrl}/api/services/media_player/play_media`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        entity_id: device,
        media_content_id: "play 104.5 chumfm",
        media_content_type: "custom"
      }),
    });
  }

  console.log(`All devices started`);
  res.json({ success: true });
} catch (error) {
  console.error("Play radio all error:", error);
  res.status(500).json({ error: "Failed to play radio on all devices" });
}
});

// POST /api/media/stop-radio - Stop radio on Echo devices
app.post("/api/media/stop-radio", async (req, res) => {
  try {
    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(400).json({ error: "Home Assistant not configured" });
    }

    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

    // Stop ALL devices that might be playing
    const devices = [
      NEST_SPEAKER_ENTITY,
      ...CAT_ECHO_ENTITIES,
      "media_player.echo_closet_am",
      "media_player.echo_lr_couch_l_am",
      "media_player.echo_lr_couch_r_am",
      "media_player.echo_hallway_entrance_am",
      "media_player.echo_king_l_am",
      "media_player.echo_king_r_am",
      "media_player.echo_king_tv_am",
      "media_player.echo_kitchen_cupboards_left_am",
      "media_player.echo_kitchen_cupboards_r_am",
      "media_player.echo_kitchen_fridge_am",
      "media_player.echo_kitchen_hutch_am",
      "media_player.echo_kitchen_island_corner_am",
      "media_player.echo_kitchen_studio_black_am",
      "media_player.echo_lr_hub_am",
      "media_player.echo_lr_studio_white_am",
      "media_player.echo_lr_tv_shelf_am",
      "media_player.echo_queen_balcony_am",
      "media_player.echo_queen_bed_l_am",
      "media_player.echo_queen_bed_r_am",
      "media_player.echo_show_pug_am",
      EVERYWHERE_GROUP_ENTITY,
      "media_player.everywhere_2"
    ];

    console.log(`Stopping media on all devices`);

    // Send all stop commands in parallel for faster execution
    await Promise.all(devices.map(device =>
      fetch(`${haUrl}/api/services/media_player/media_stop`, {
        method: 'POST',
        headers: {
          'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({
          entity_id: device
        }),
      }).catch(err => console.error(`Failed to stop ${device}:`, err))
    ));

    console.log(`All devices stopped`);
    res.json({ success: true });
  } catch (error) {
    console.error("Stop radio error:", error);
    res.status(500).json({ error: "Failed to stop radio" });
  }
});

// POST /api/media/volume - Adjust volume on Echo devices
app.post("/api/media/volume", async (req, res) => {
  try {
    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(400).json({ error: "Home Assistant not configured" });
    }

    const { direction, level, entityId } = req.body; // direction: 'up' or 'down', or level: 0-100, entityId: speci...

```

```

const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

// Use the specified entityId or default to a single device
const targetDevice = entityId || "media_player.echo_lr_studio_white_am";

console.log(`Setting volume on device: ${targetDevice}`);

// First get current volume from the target device
const statesResponse = await fetch(`${haUrl}/api/states/${targetDevice}`, {
  method: 'GET',
  headers: {
    'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
    'Content-Type': 'application/json',
  },
});

let currentVolume = 0.5; // Default 50%
if (statesResponse.ok) {
  const stateData = await statesResponse.json();
  currentVolume = stateData.attributes?.volume_level || 0.5;
  console.log(`Current volume on ${targetDevice}: ${currentVolume}`);
}

// Calculate new volume
let newVolume: number;
if (level !== undefined) {
  newVolume = level / 100; // Convert 0-100 to 0-1
} else {
  const step = 0.1; // 10% steps
  newVolume = direction === 'up'
    ? Math.min(1, currentVolume + step)
    : Math.max(0, currentVolume - step);
}

console.log(`Setting volume to ${newVolume} on ${targetDevice}`);

if (targetDevice === EVERYWHERE_GROUP_ENTITY) {
  const excludeSet = new Set<string>(Array.isArray(req.body.excludeEntityIds) ? req.body.excludeEntityIds : []);
  const allEchoDevices = FLICK_DEVICES.flatMap(g => g.devices).filter(d => (d.type === "echo" || d.type === "ec...
  console.log(`[Volume] BYhome group !` setting volume on ${allEchoDevices.length} individual Echo devices (excl...
  await Promise.allSettled(allEchoDevices.map(d =>
    fetch(`${haUrl}/api/services/media_player/volume_set`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ entity_id: d.entityId, volume_level: newVolume }),
    })
  ));
  console.log(`[Volume] Set volume to ${newVolume} on all Echo devices`);
  res.json({ success: true, direction, newVolume: Math.round(newVolume * 100) });
} else {
  const volumeResponse = await fetch(`${haUrl}/api/services/media_player/volume_set`, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      entity_id: targetDevice,
      volume_level: newVolume
    }),
  });

  if (!volumeResponse.ok) {
    const errorText = await volumeResponse.text();
    console.error(`Home Assistant volume_set failed: ${volumeResponse.status} - ${errorText}`);
    return res.status(500).json({ error: `Failed to set volume: ${volumeResponse.status}` });
  }

  console.log(`Volume set successfully on ${targetDevice}`);
  res.json({ success: true, direction, newVolume: Math.round(newVolume * 100) });
}

} catch (error) {
  console.error("Volume control error:", error);
  res.status(500).json({ error: "Failed to adjust volume" });
}
});

// POST /api/media/volume-all - Adjust volume on ALL Echo devices
app.post("/api/media/volume-all", async (req, res) => {
  try {
    if (!HOME_ASSISTANT_URL || !HOME_ASSISTANT_TOKEN) {
      return res.status(400).json({ error: "Home Assistant not configured" });
    }
  }

  const { direction, level } = req.body;
  const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

  // All Echo devices
  const devices = [
    NEST_SPEAKER_ENTITY,
    ...CAT_ECHO_ENTITIES,
    "media_player.echo_closet_am",
    "media_player.echo_lr_couch_r_am",
    "media_player.echo_hallway_entrance_am",
    "media_player.echo_king_l_am",
    "media_player.echo_king_r_am",
    "media_player.echo_king_tv_am",
    "media_player.echo_kitchen_cupboards_left_am",
    "media_player.echo_kitchen_cupboards_r_am",
    "media_player.echo_kitchen_fridge_am",
    "media_player.echo_kitchen_hutch_am",
    "media_player.echo_kitchen_island_corner_am",
    "media_player.echo_kitchen_studio_black_am",
    "media_player.echo_lr_hub_am",
    "media_player.echo_lr_studio_white_am",
    "media_player.echo_lr_tv_shelf_am",
    "media_player.echo_queen_balcony_am",
    "media_player.echo_queen_bed_l_am",
  ]

```

```

    "media_player.echo_queen_bed_r_am",
    "media_player.echo_show_pug_am"
  ];

  console.log(`Setting volume on ALL ${devices.length} devices`);

  // Get current volume from first device
  const statesResponse = await fetch(`${baseUrl}/api/states/${devices[0]}`, {
    method: 'GET',
    headers: {
      'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
      'Content-Type': 'application/json',
    },
  });

  let currentVolume = 0.5;
  if (statesResponse.ok) {
    const stateData = await statesResponse.json();
    currentVolume = stateData.attributes?.volume_level || 0.5;
  }

  // Calculate new volume
  let newVolume: number;
  if (level !== undefined) {
    newVolume = level / 100;
  } else {
    const step = 0.1;
    newVolume = direction === 'up'
      ? Math.min(1, currentVolume + step)
      : Math.max(0, currentVolume - step);
  }

  console.log(`Setting volume to ${newVolume} on all devices`);

  // Set volume on all devices in parallel
  await Promise.all(devices.map(device =>
    fetch(`${baseUrl}/api/services/media_player/volume_set`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        entity_id: device,
        volume_level: newVolume
      }),
    })
  ));

  console.log(`Volume set successfully on all devices`);
  res.json({ success: true, direction, newVolume: Math.round(newVolume * 100) });
} catch (error) {
  console.error("Volume all control error:", error);
  res.status(500).json({ error: "Failed to adjust volume on all devices" });
}
});

// POST /api/tasks/:id/calendar - Add task to Google Calendar
app.post("/api/tasks/:id/calendar", async (req, res) => {
  try {
    const taskId = Number(req.params.id);
    const task = await storage.getTask(taskId);

    if (!task) {
      return res.status(404).json({ error: "Task not found" });
    }

    const event = await createCalendarEvent({
      id: task.id,
      title: task.title,
      description: task.description,
      dueDate: task.dueDate,
      courseName: task.courseName,
    });

    // Update task with calendar event ID
    await storage.updateTask(taskId, {
      calendarEventId: event.id,
      calendarProvider: "google",
    });

    res.json({ success: true, eventId: event.id, eventLink: event.htmlLink });
  } catch (error) {
    console.error("Google Calendar error:", error);
    res.status(500).json({ error: "Failed to add to Google Calendar" });
  }
});

// DELETE /api/tasks/:id/calendar - Remove task from Google Calendar
app.delete("/api/tasks/:id/calendar", async (req, res) => {
  try {
    const taskId = Number(req.params.id);
    const task = await storage.getTask(taskId);

    if (!task) {
      return res.status(404).json({ error: "Task not found" });
    }

    if (!task.calendarEventId) {
      return res.status(400).json({ error: "Task is not on calendar" });
    }

    await deleteCalendarEvent(task.calendarEventId);

    // Clear calendar info from task
    await storage.updateTask(taskId, {
      calendarEventId: null,
      calendarProvider: null,
    });
  }
});

```



```

    });
    res.json({ success: true });
  } catch (error) {
    console.error("Google Calendar delete error:", error);
    res.status(500).json({ error: "Failed to remove from Google Calendar" });
  }
});

// DELETE /api/calendar/event/:eventId - Delete a calendar event by ID
app.delete("/api/calendar/event/:eventId", async (req, res) => {
  try {
    await deleteCalendarEvent(req.params.eventId);
    res.json({ success: true });
  } catch (err) {
    console.error("Calendar event delete error:", err);
    res.status(500).json({ error: "Failed to delete calendar event" });
  }
});

// POST /api/calendar/class - Create a recurring class event
app.post("/api/calendar/class", async (req, res) => {
  try {
    const { courseName, courseCode, startDate, endDate, startTime, endTime, daysOfWeek, location } = req.body;

    const event = await createRecurringClassEvent({
      courseName,
      courseCode,
      startDate,
      endDate,
      startTime,
      endTime,
      daysOfWeek,
      location
    });

    res.json({ success: true, event });
  } catch (err) {
    console.error("Error creating class event:", err);
    res.status(500).json({ message: "Failed to create class event", error: String(err) });
  }
});

// GET /api/calendar/events - Fetch events from both Google accounts
app.get("/api/calendar/upcoming-events", async (req, res) => {
  try {
    const now = new Date();
    const start = new Date(now.getFullYear(), now.getMonth(), now.getDate());
    const end = new Date(start.getTime() + 120 * 24 * 60 * 60 * 1000);

    let primaryEvents: any[] = [];
    try {
      primaryEvents = await listEvents(start, end);
    } catch (err) {
      console.error("Failed to fetch primary upcoming events:", err);
    }

    let secondAccountEvents: any[] = [];
    try {
      const secondStatus = await isSecondAccountConnected();
      if (secondStatus.connected) {
        secondAccountEvents = await getEventsFromSecondAccount(start, end);
      }
    } catch (err) {
      console.error("Failed to fetch second account upcoming events:", err);
    }

    let tmuUpcomingEvents: any[] = [];
    try {
      const tmuFormatted = await fetchTMUCalendarEvents(start, end);
      tmuUpcomingEvents = tmuFormatted.map(e => ({
        id: e.id,
        start: { dateTime: e.startDate },
        end: { dateTime: e.endDate },
        summary: e.title,
        description: e.description,
        location: e.location,
        htmlLink: '',
        source: 'tmu',
      }));
    } catch (err) {
      console.error("Failed to fetch TMU upcoming events:", err);
    }

    const allEvents = [...primaryEvents, ...secondAccountEvents, ...tmuUpcomingEvents];

    const tasks = await storage.getTasks({});
    const syncedEventIds = new Set([
      ...tasks.map(t => t.calendarEventId).filter(Boolean),
      ...tasks.map(t => t.secondAccountCalendarEventId).filter(Boolean),
      ...tasks.map(t => t.secondAccountPrepEventId).filter(Boolean),
      ...tasks.map(t => t.prepCalendarEventId).filter(Boolean),
      ...tasks.map(t => t.secondaryCalendarEventId).filter(Boolean),
    ]);

    const externalEvents = allEvents.filter(event => event.id && !syncedEventIds.has(event.id));

    const formattedEvents = externalEvents.map(event => {
      const isAllDay = !event.start?.dateTime;
      let startDate = event.start?.dateTime || event.start?.date;
      let endDate = event.end?.dateTime || event.end?.date;
      if (isAllDay && startDate && !startDate.includes('T')) {
        startDate = `${startDate}T12:00:00`;
      }
      if (isAllDay && endDate && !endDate.includes('T')) {
        endDate = `${endDate}T12:00:00`;
      }
      return {

```

```

        id: event.id,
        title: event.summary || 'Untitled Event',
        description: event.description || '',
        startDate,
        endDate,
        isAllDay,
        htmlLink: event.htmlLink,
        source: event._source === 'tmu' ? 'tmu' : 'google',
    });
});

const seen = new Set<string>();
const dedupedEvents = formattedEvents.filter(e => {
    const key = `${e.title}||${e.startDate}`;
    if (seen.has(key)) return false;
    seen.add(key);
    return true;
});

// Further deduplicate: collapse events that match the same course + similar task type at the same time
const courseTaskSeen2 = new Set<string>();
const finalEvents2 = dedupedEvents.filter(e => {
    const titleMatch = e.title.match(/^[A-Z]{2,5}\d{3}[A-Z]?[s*-\s*[\^]]+)\s*(.*)/i);
    if (titleMatch) {
        const courseId = titleMatch[1].split('-')[0].trim().toUpperCase().replace(/\s/g, '');
        const taskPart = titleMatch[2].trim().toLowerCase();
        const normalizedType = taskPart.includes('module') ? 'module' : taskPart.includes('reading') ? 'reading' : ...
        const startKey = e.startDate ? e.startDate.substring(0, 16) : '';
        const dedupeKey = `${courseId}||${normalizedType}||${startKey}`;
        if (courseTaskSeen2.has(dedupeKey)) return false;
        courseTaskSeen2.add(dedupeKey);
    }
    return true;
});

res.json(finalEvents2);
} catch (error) {
    console.error("Fetch upcoming calendar events error:", error);
    res.status(500).json({ error: "Failed to fetch upcoming calendar events" });
}
});

app.get("/api/calendar/events", async (req, res) => {
    try {
        const activeSemester = await storage.getActiveSemesterSettings();
        const semesterStart = activeSemester ? new Date(activeSemester.semesterStartDate) : undefined;
        const weekNumber = Number(req.query.weekNumber) || getWeekNumber(torontoDate(), semesterStart, activeSemester?...
        const { start, end } = getWeekDates(weekNumber, semesterStart, activeSemester?.readingWeekStart);

        const endOfWeek = new Date(end.getTime() + 24 * 60 * 60 * 1000);

        // Fetch events from primary account
        let primaryEvents: any[] = [];
        try {
            primaryEvents = await listEvents(start, endOfWeek);
        } catch (err) {
            console.error("Failed to fetch primary account events:", err);
        }

        // Fetch events from second Google account
        let secondAccountEvents: any[] = [];
        try {
            const secondStatus = await isSecondAccountConnected();
            if (secondStatus.connected) {
                secondAccountEvents = await getEventsFromSecondAccount(start, end);
            }
        } catch (err) {
            console.error("Failed to fetch second account events:", err);
        }

        let tmuEvents: any[] = [];
        try {
            const tmuFormatted = await fetchTMUCalendarEvents(start, endOfWeek);
            tmuEvents = tmuFormatted.map(e => ({
                id: e.id,
                start: { dateTime: e.startDate },
                end: { dateTime: e.endDate },
                summary: e.title,
                description: e.description,
                location: e.location,
                htmlLink: '',
                source: 'tmu',
            }));
        } catch (err) {
            console.error("Failed to fetch TMU calendar events:", err);
        }

        // Combine events from all accounts
        const allEvents = [...primaryEvents, ...secondAccountEvents, ...tmuEvents];

        // Get all tasks to find which events are already synced from this app
        const tasks = await storage.getTasks({});
        const syncedEventIds = new Set([
            ...tasks.map(t => t.calendarEventId).filter(Boolean),
            ...tasks.map(t => t.secondAccountCalendarEventId).filter(Boolean),
            ...tasks.map(t => t.secondAccountPrepEventId).filter(Boolean),
            ...tasks.map(t => t.prepCalendarEventId).filter(Boolean),
            ...tasks.map(t => t.secondaryCalendarEventId).filter(Boolean),
        ]);

        const allGoogleEventIds = new Set(allEvents.map(e => e.id).filter(Boolean));
        const protectedTypes = new Set(['class', 'assignment', 'quiz', 'exam', 'essay', 'module', 'reading', 'discussio...
        const tasksToDelete = tasks.filter(t => {
            const hasCalId = t.calendarEventId || t.secondAccountCalendarEventId;
            if (!hasCalId) return false;
            const calIdExists = t.calendarEventId && allGoogleEventIds.has(t.calendarEventId);
            const secIdExists = t.secondAccountCalendarEventId && allGoogleEventIds.has(t.secondAccountCalendarEventId);
            if (calIdExists || secIdExists) return false;
        });
    }
});

```

```

    if (protectedTypes.has(t.type)) return false;
    if (t.gradeWeight != null || t.gradeValue != null) return false;
    if (t.courseName) return false;
    const taskDate = new Date(t.dueDate);
    if (taskDate < start || taskDate > endOfWeek) return false;
    return true;
  });
  if (tasksToDelete.length > 0) {
    console.log('[CalSync] Detected ${tasksToDelete.length} task(s) deleted from calendar, removing from app (onl...
    for (const t of tasksToDelete) {
      try {
        await storage.deleteTask(t.id);
        console.log('[CalSync] Deleted task ${t.id} ("${t.title}") - calendar event ${t.calendarEventId} no longe...
      } catch (err) {
        console.error('[CalSync] Failed to delete task ${t.id}:`, err);
      }
    }
  }

  // Filter out events that are already synced from this app
  const externalEvents = allEvents.filter(event => event.id && !syncedEventIds.has(event.id));

  // Transform to a simpler format
  const formattedEvents = externalEvents.map(event => {
    const isAllDay = !event.start?.dateTime;
    let startDate = event.start?.dateTime || event.start?.date;
    let endDate = event.end?.dateTime || event.end?.date;

    if (isAllDay && startDate && !startDate.includes('T')) {
      startDate = `${startDate}T12:00:00`;
    }
    if (isAllDay && endDate && !endDate.includes('T')) {
      endDate = `${endDate}T12:00:00`;
    }

    return {
      id: event.id,
      title: event.summary || 'Untitled Event',
      description: event.description || '',
      startDate,
      endDate,
      isAllDay,
      htmlLink: event.htmlLink,
      source: event._source === 'tmu' ? 'tmu' : 'google',
    };
  });

  // Deduplicate events by title + startDate to prevent duplicates from multiple accounts
  const seen = new Set<string>();
  const dedupedEvents = formattedEvents.filter(e => {
    const key = `${e.title}||${e.startDate}`;
    if (seen.has(key)) return false;
    seen.add(key);
    return true;
  });

  // Further deduplicate: collapse events that match the same course + similar task type at the same time
  const courseTaskSeen = new Set<string>();
  const finalEvents = dedupedEvents.filter(e => {
    const titleMatch = e.title.match(/^[A-Z]{2,5}\d{3}[A-Z]?[s*~\s*[\^]]+)\s*(.*)/i);
    if (titleMatch) {
      const courseId = titleMatch[1].split('-')[0].trim().toUpperCase().replace(/\s/g, '');
      const taskPart = titleMatch[2].trim().toLowerCase();
      const normalizedType = taskPart.includes('module') ? 'module' : taskPart.includes('reading') ? 'reading' : ...
      const startKey = e.startDate ? e.startDate.substring(0, 16) : '';
      const dedupeKey = `${courseId}||${normalizedType}||${startKey}`;
      if (courseTaskSeen.has(dedupeKey)) return false;
      courseTaskSeen.add(dedupeKey);
    }
    return true;
  });

  if (tasksToDelete.length > 0) {
    res.json({ events: finalEvents, _deletedTasks: tasksToDelete.length });
  } else {
    res.json(finalEvents);
  }
} catch (error) {
  console.error("Fetch Google Calendar events error:", error);
  res.status(500).json({ error: "Failed to fetch Google Calendar events" });
}
});

// Export all data for sync (with CORS for cross-origin sync)
app.options("/api/export", (_req, res) => {
  res.header("Access-Control-Allow-Origin", "");
  res.header("Access-Control-Allow-Methods", "GET, OPTIONS");
  res.header("Access-Control-Allow-Headers", "Content-Type");
  res.sendStatus(200);
});

app.get("/api/export", async (_req, res) => {
  res.header("Access-Control-Allow-Origin", "");

  try {
    const tasks = await storage.getTasks();
    const files = await storage.GetFiles();
    const semester = await storage.getActiveSemesterSettings();
    const deletedFolders = await storage.getDeletedFolders();
    const customFolders = await storage.getCustomFolders();

    const exportData = {
      version: 1,
      exportedAt: new Date().toISOString(),
      tasks,
      files,
      semester,
      deletedFolders,
    };
  }

```

```

        customFolders,
    };

    res.json(exportData);
} catch (err) {
    console.error("Export error:", err);
    res.status(500).json({ error: "Failed to export data" });
}
});

// Import data for sync (with CORS for cross-origin sync)
app.options("/api/import", (_req, res) => {
    res.header("Access-Control-Allow-Origin", "*");
    res.header("Access-Control-Allow-Methods", "POST, OPTIONS");
    res.header("Access-Control-Allow-Headers", "Content-Type");
    res.sendStatus(200);
});

app.post("/api/import", async (req, res) => {
    res.header("Access-Control-Allow-Origin", "*");

    try {
        const { tasks, files, semester, deletedFolders, customFolders } = req.body;

        let imported = { tasks: 0, files: 0, semester: false, deletedFolders: 0, customFolders: 0 };

        // Import semester settings
        if (semester) {
            const existing = await storage.getActiveSemesterSettings();
            const { id, createdAt, ...semesterData } = semester;
            if (semesterData.semesterStartDate) semesterData.semesterStartDate = new Date(semesterData.semesterStartDate);
            if (semesterData.semesterEndDate) semesterData.semesterEndDate = new Date(semesterData.semesterEndDate);
            if (semesterData.readingWeekStart) semesterData.readingWeekStart = new Date(semesterData.readingWeekStart);
            if (semesterData.course1StartDate) semesterData.course1StartDate = new Date(semesterData.course1StartDate);
            if (semesterData.course1EndDate) semesterData.course1EndDate = new Date(semesterData.course1EndDate);
            if (semesterData.course2StartDate) semesterData.course2StartDate = new Date(semesterData.course2StartDate);
            if (semesterData.course2EndDate) semesterData.course2EndDate = new Date(semesterData.course2EndDate);
            if (semesterData.course3StartDate) semesterData.course3StartDate = new Date(semesterData.course3StartDate);
            if (semesterData.course3EndDate) semesterData.course3EndDate = new Date(semesterData.course3EndDate);
            if (existing) {
                await storage.updateSemesterSettings(existing.id, semesterData);
            } else {
                await storage.createSemesterSettings(semesterData);
            }
            imported.semester = true;
        }

        // Import tasks - create or update all
        if (tasks && Array.isArray(tasks)) {
            for (const task of tasks) {
                try {
                    const existing = await storage.getTask(task.id);
                    const taskData = {
                        ...task,
                        dueDate: new Date(task.dueDate),
                        startDate: task.startDate ? new Date(task.startDate) : null,
                        repeatEndDate: task.repeatEndDate ? new Date(task.repeatEndDate) : null,
                        completedAt: task.completedAt ? new Date(task.completedAt) : null,
                    };

                    if (existing) {
                        const { id, ...updates } = taskData;
                        await storage.updateTask(task.id, updates);
                    } else {
                        const { id, ...newTask } = taskData;
                        await storage.createTask(newTask);
                    }
                    imported.tasks++;
                } catch (err) {
                    console.error("Error importing task:", err);
                }
            }
        }

        // Import files - create or update all
        if (files && Array.isArray(files)) {
            for (const file of files) {
                try {
                    const existing = await storage.getFileByPath(file.objectPath);
                    if (existing) {
                        await storage.updateFile(existing.id, {
                            displayName: file.displayName,
                            folder: file.folder,
                            listened: file.listened,
                        });
                    } else {
                        const { id, createdAt, ...fileData } = file;
                        await storage.createFile(fileData);
                    }
                    imported.files++;
                } catch (err) {
                    console.error("Error importing file:", err);
                }
            }
        }

        // Import deleted folders
        if (deletedFolders && Array.isArray(deletedFolders)) {
            for (const folder of deletedFolders) {
                try {
                    await storage.addDeletedFolder(folder.folderId);
                    imported.deletedFolders++;
                } catch (err) {
                    // Folder might already be marked as deleted
                }
            }
        }
    }
}
});

```

```

// Import custom folders
if (customFolders && Array.isArray(customFolders)) {
  // Get existing folders to check for duplicates
  const existingFolders = await storage.getCustomFolders();
  const existingByParentAndName = new Map(existingFolders.map(f => [`${f.parentFolderId}:${f.name}`, f]));

  for (const folder of customFolders) {
    try {
      const key = `${folder.parentFolderId}:${folder.name}`;
      const existing = existingByParentAndName.get(key);
      if (existing) {
        // Already exists, skip or update name if different
        await storage.updateCustomFolder(existing.id, folder.name);
      } else {
        await storage.createCustomFolder({ name: folder.name, parentFolderId: folder.parentFolderId });
      }
      imported.customFolders++;
    } catch (err) {
      console.error("Error importing custom folder:", err);
    }
  }
}

res.json({ success: true, imported });
} catch (err) {
  console.error("Import error:", err);
  res.status(500).json({ error: "Failed to import data" });
}
});

app.post("/api/dev-dismiss", async (req, res) => {
  try {
    const { taskId, text } = req.body;
    const filePath = path.join(process.cwd(), "client", "public", "dev-tasks.json");
    const raw = await fs.promises.readFile(filePath, "utf-8");
    const tasks: { id: string; text: string; complete?: boolean }[] = JSON.parse(raw);
    const filtered = tasks.filter(t => t.id !== taskId && t.text !== text);
    await fs.promises.writeFile(filePath, JSON.stringify(filtered, null, 2) + "\n");
    res.json({ ok: true, remaining: filtered.length });
  } catch (err) {
    console.error("dev-dismiss error:", err);
    res.status(500).json({ error: "Failed to update dev-tasks.json" });
  }
});

app.post("/api/tasks/parse-ics", async (req, res) => {
  try {
    const { icsContent } = req.body;
    if (!icsContent || typeof icsContent !== 'string') {
      return res.status(400).json({ error: "icsContent is required" });
    }

    const events: Array<{
      title: string;
      description: string;
      startDate: string | null;
      endDate: string | null;
      location: string;
      allDay: boolean;
    }> = [];

    const lines = icsContent.replace(/\r\n /g, '').replace(/\r\n\t/g, '').split(/\r?\n/);
    let inEvent = false;
    let currentEvent: Record<string, string> = {};

    for (const line of lines) {
      if (line === 'BEGIN:VEVENT') {
        inEvent = true;
        currentEvent = {};
      } else if (line === 'END:VEVENT') {
        inEvent = false;
        const summary = currentEvent['SUMMARY'] || '';
        const description = (currentEvent['DESCRIPTION'] || '').replace(/\n/g, '\n').replace(/\./g, ',').replace(...
        const dtstart = currentEvent['DTSTART'] || currentEvent['DTSTART;VALUE=DATE'] || '';
        const dtend = currentEvent['DTEND'] || currentEvent['DTEND;VALUE=DATE'] || '';
        const location = (currentEvent['LOCATION'] || '').replace(/\./g, ',').replace(/\\\\/g, '\\');
        const allDay = !!(currentEvent['DTSTART;VALUE=DATE'] || (dtstart && dtstart.length === 8));

        const parseIcsDate = (val: string): string | null => {
          if (!val) return null;
          const clean = val.replace(/^[^0-9TZ]/g, '');
          if (clean.length === 8) {
            return `${clean.slice(0,4)}-${clean.slice(4,6)}-${clean.slice(6,8)}T00:00:00.000Z`;
          }
          if (clean.length >= 15) {
            const d = `${clean.slice(0,4)}-${clean.slice(4,6)}-${clean.slice(6,8)}T${clean.slice(9,11)}:${clean.sli...
            return d;
          }
          return null;
        };

        if (summary) {
          events.push({
            title: summary.replace(/\./g, ',').replace(/\\\\/g, '\\'),
            description,
            startDate: parseIcsDate(dtstart),
            endDate: parseIcsDate(dtend),
            location,
            allDay,
          });
        }
      } else if (inEvent) {
        const colonIdx = line.indexOf(':');
        if (colonIdx > 0) {
          let key = line.slice(0, colonIdx);
          const value = line.slice(colonIdx + 1);
          if (key.includes(';') && !key.startsWith('DTSTART;VALUE') && !key.startsWith('DTEND;VALUE')) {
            key = key.split(';')[0];

```

```

        }
        currentEvent[key] = value;
    }
}
}

res.json({ events, count: events.length });
} catch (err) {
  console.error("ICS parse error:", err);
  res.status(500).json({ error: "Failed to parse ICS file" });
}
});

app.get("/api/ui-settings/:key", async (req, res) => {
  try {
    const row = await db.select().from(appState).where(eq(appState.key, `ui_${req.params.key}`)).limit(1);
    res.json({ value: row.length > 0 ? row[0].value : null });
  } catch (err: any) {
    res.json({ value: null });
  }
});

app.post("/api/ui-settings/:key", async (req, res) => {
  try {
    const key = `ui_${req.params.key}`;
    const { value } = req.body;
    const row = await db.select().from(appState).where(eq(appState.key, key)).limit(1);
    if (row.length > 0) {
      await db.update(appState).set({ value: String(value), updatedAt: new Date() }).where(eq(appState.key, key));
    } else {
      await db.insert(appState).values({ key, value: String(value) });
    }
    res.json({ success: true });
  } catch (err: any) {
    res.status(500).json({ error: err.message });
  }
});

app.get("/api/syllabus/paths", async (_req, res) => {
  try {
    const row = await db.select().from(appState).where(eq(appState.key, 'courseSyllabusPaths')).limit(1);
    const paths = row.length > 0 ? JSON.parse(row[0].value) : {};
    res.json(paths);
  } catch (err: any) {
    console.error("Error getting syllabus paths:", err);
    res.json({});
  }
});

app.post("/api/syllabus/paths", async (req, res) => {
  try {
    const { courseCode, objectPath } = req.body;
    if (!courseCode) return res.status(400).json({ error: "courseCode required" });
    const row = await db.select().from(appState).where(eq(appState.key, 'courseSyllabusPaths')).limit(1);
    const paths = row.length > 0 ? JSON.parse(row[0].value) : {};
    if (objectPath) {
      paths[courseCode] = objectPath;
    } else {
      delete paths[courseCode];
    }
    const value = JSON.stringify(paths);
    if (row.length > 0) {
      await db.update(appState).set({ value, updatedAt: new Date() }).where(eq(appState.key, 'courseSyllabusPaths'));
    } else {
      await db.insert(appState).values({ key: 'courseSyllabusPaths', value });
    }
    res.json({ success: true });
  } catch (err: any) {
    console.error("Error saving syllabus path:", err);
    res.status(500).json({ error: err.message });
  }
});

app.get("/api/syllabus/view", async (req, res) => {
  try {
    const objectPath = req.query.path as string;
    if (!objectPath) return res.status(400).json({ error: "path required" });

    const privateDir = process.env.PRIVATE_OBJECT_DIR || '';
    const pathParts = privateDir.replace(/\^\/, '').split('/');
    const bucketName = pathParts[0];
    const filePath = objectPath.replace('/objects/', '');
    const fullObjectName = `private/${filePath}`;
    console.log(`[Syllabus View] bucket=${bucketName}, object=${fullObjectName}`);
    const [buffer] = await objectStorageClient.bucket(bucketName).file(fullObjectName).download();
    res.setHeader('Content-Type', 'application/pdf');
    res.setHeader('Content-Disposition', 'inline');
    res.send(buffer);
  } catch (err: any) {
    console.error("Error serving syllabus:", err);
    res.status(500).json({ error: "Failed to load syllabus" });
  }
});

app.post("/api/syllabus/parse", async (req, res) => {
  try {
    const { objectPath, courseName, courseCode, fileName } = req.body;
    if (!objectPath || !courseName) {
      return res.status(400).json({ error: "objectPath and courseName are required" });
    }

    const privateDir = process.env.PRIVATE_OBJECT_DIR || '';
    const pathParts = privateDir.replace(/\^\/, '').split('/');
    const bucketName = pathParts[0];
    const filePath = objectPath.replace('/objects/', '');
    const fullObjectName = `private/${filePath}`;
    let fileBuffer: Buffer;
    try {

```

```

    console.log(`[Syllabus Parse] Downloading from bucket=${bucketName}, object=${fullObjectName}`);
    const [buffer] = await objectStorageClient.bucket(bucketName).file(fullObjectName).download();
    fileBuffer = buffer;
  } catch (d1Err) {
    console.error("Failed to download syllabus from object storage:", d1Err);
    return res.status(500).json({ error: "Failed to read uploaded file" });
  }

  let pdfText = '';
  try {
    const { PDFParse } = await import('pdf-parse');
    const parser = new PDFParse({ data: new Uint8Array(fileBuffer), verbosity: 0 });
    const result = await parser.getText();
    pdfText = result.text || '';
  } catch (pdfErr) {
    console.error("Failed to parse syllabus PDF:", pdfErr);
    return res.status(500).json({ error: "Failed to parse PDF content" });
  }

  if (!pdfText.trim()) {
    return res.status(400).json({ error: "Could not extract text from syllabus PDF" });
  }

  const OpenAI = (await import('openai')).default;
  const openai = new OpenAI({
    apiKey: process.env.AI_INTEGRATIONS_OPENAI_API_KEY,
    baseURL: process.env.AI_INTEGRATIONS_OPENAI_BASE_URL,
  });

  const prompt = `You are analyzing a university course syllabus for "${courseName}" (${courseCode} || '').

```

Extract ALL of the following information from this syllabus. Be thorough – do not skip anything.

Return a JSON object with these fields:

```

1. "courseInfo": {
  "professor": string or null (professor/instructor name),
  "professorEmail": string or null (professor email),
  "officeHours": string or null (office hours description),
  "textbook": string or null (required textbook/materials),
  "description": string or null (course description summary)
}

2. "gradingBreakdown": array of objects, each with:
  - "component": string (e.g. "Midterm Exam", "Final Paper", "Participation")
  - "weight": number (percentage, e.g. 30 for 30%)
  - "description": string or null (any details about this component)

3. "items": array of ALL deadlines, assignments, exams, quizzes, discussions, projects, and important dates found. Ea...
  - "title": string (e.g. "Assignment 1 - Essay on Policy")
  - "type": string (one of: reading, essay, exam, quiz, discussion, poll, project, module, assignment, other)
  - "date": string or null (ISO date if found, e.g. "2026-01-30", or null if no specific date)
  - "dateDescription": string or null (date as written in syllabus, e.g. "Week 5, Friday 11:59pm")
  - "time": string or null (time if specified, e.g. "23:59")
  - "weight": number or null (grade percentage weight if mentioned)
  - "description": string (details, requirements, word count, format, etc.)
  - "category": string (one of: assignment, exam, deadline, event, policy)

4. "policies": array of notable course policies, each with:
  - "title": string (e.g. "Late Submission Policy")
  - "description": string (summary of the policy)

5. "weekNumbering": {
  - "style": string (one of: "skip_break" if the course skips the break/reading week number so Week 6 is followed by ...
  - "breakWeekLabel": string or null (how the syllabus labels the break/reading week, e.g. "Reading Week", "Study Wee...
  - "totalWeeks": number or null (total number of instructional weeks mentioned)
  - "evidence": string (quote or describe the part of the syllabus that shows how weeks are numbered around the break)
}

```

Be extremely thorough. Include EVERY assignment, deadline, exam, quiz, discussion post, and important date mentioned ...

Syllabus text:
 \${pdfText.substring(0, 12000)}

Return ONLY the JSON object, no markdown formatting.`;

```

    const completion = await openai.chat.completions.create({
      model: "gpt-4o-mini",
      messages: [{ role: "user", content: prompt }],
      temperature: 0.1,
    });

    const responseText = completion.choices[0]?.message?.content || '';
    let parsed: any;
    try {
      const jsonStr = responseText.replace(/```\n?/g, '').replace(/```\n?/g, '').trim();
      parsed = JSON.parse(jsonStr);
    } catch {
      return res.status(500).json({ error: "Failed to parse AI response", raw: responseText });
    }

    console.log(`[Syllabus] Parsed ${courseName}: ${parsed.items?.length || 0} items, ${parsed.gradingBreakdown?.le...

    res.json({
      ...parsed,
      objectPath,
      fileName: fileName || 'Syllabus.pdf',
    });
  } catch (err: any) {
    console.error("Error parsing syllabus:", err);
    res.status(500).json({ error: err.message || "Internal server error" });
  }
});

```

```

// Parse uploaded assignment PDF and extract task details
app.post("/api/tasks/parse-assignment-pdf", async (req, res) => {
  try {
    const { objectPath, courseName, fileName } = req.body;

```

```

    if (!objectPath || !courseName) {
      return res.status(400).json({ error: "objectPath and courseName are required" });
    }

    const bucketId = process.env.PUBLIC_OBJECT_SEARCH_PATHS?.split('/')[1] || '';
    const filePath = objectPath.replace('/objects/', '');
    let fileBuffer: Buffer;
    try {
      const [buffer] = await objectStorageClient.bucket(bucketId).file(filePath).download();
      fileBuffer = buffer;
    } catch (dlErr) {
      console.error("Failed to download file from object storage:", dlErr);
      return res.status(500).json({ error: "Failed to read uploaded file" });
    }

    let pdfText = '';
    try {
      const { PDFParse } = await import('pdf-parse');
      const parser = new PDFParse({ data: new Uint8Array(fileBuffer), verbosity: 0 });
      const result = await parser.getText();
      pdfText = result.text || '';
    } catch (pdfErr) {
      console.error("Failed to parse PDF:", pdfErr);
      return res.status(500).json({ error: "Failed to parse PDF content" });
    }

    if (!pdfText.trim()) {
      return res.status(400).json({ error: "Could not extract text from PDF" });
    }

    const OpenAI = (await import('openai')).default;
    const openai = new OpenAI({
      apiKey: process.env.AI_INTEGRATIONS_OPENAI_API_KEY,
      baseURL: process.env.AI_INTEGRATIONS_OPENAI_BASE_URL,
    });

    const prompt = `Extract assignment details from this academic document. Return a JSON object with these fields:
    - title: string (the assignment name, e.g. "Assignment 2 - Municipal Issues Policy Paper")
    - type: string (one of: reading, essay, exam, quiz, discussion, poll, project, module, class, assignment, other)
    - description: string (comprehensive summary including requirements, word count, format, grading breakdown, topics/op...
    - gradeWeight: number or null (percentage of final grade if mentioned, e.g. 30 for 30%)
    - dueDescription: string (the due date description as written, e.g. "End of Week 10, Friday 11:59 pm")
    - wordCount: string or null (required word count if mentioned)
    - format: string or null (format requirements like "APA", "double spaced", etc.)

    Document text:
    ${pdfText.substring(0, 4000)}`;

    Return ONLY the JSON object, no markdown formatting.`;

    const completion = await openai.chat.completions.create({
      model: "gpt-4o-mini",
      messages: [{ role: "user", content: prompt }],
      temperature: 0.1,
    });

    const responseText = completion.choices[0]?.message?.content || '';
    let parsed: any;
    try {
      const jsonStr = responseText.replace(/```\n?/g, '').replace(/```\n?/g, '').trim();
      parsed = JSON.parse(jsonStr);
    } catch {
      return res.status(500).json({ error: "Failed to parse AI response", raw: responseText });
    }

    res.json({
      ...parsed,
      objectPath,
      fileName: fileName || 'Assignment.pdf',
    });
  } catch (err: any) {
    console.error("Error parsing assignment PDF:", err);
    res.status(500).json({ error: err.message || "Internal server error" });
  }
});

// Seed database with sample tasks
await seedDatabase();

// NOTE: Alexa reminder announcements are handled ONLY by server/reminderScheduler.ts
// which persists sent keys to the database. Do NOT add a duplicate here.

// One-time fix: CFNF Module 8 (file id=28) incorrectly marked as listened on production
try {
  const file28 = await storage.getFile(28);
  if (file28 && file28.listened === true && file28.displayName?.includes('Module 8')) {
    await storage.updateFile(28, { listened: false });
    console.log('[Startup Fix] Corrected CFNF Module 8 (file 28) listened=false');
  }
} catch (e) {
  // Ignore if file doesn't exist on this environment
}

// One-time fix: Essay Assignment 1 (task id=23) lost eventStartTime during reschedule
try {
  const task23 = await storage.getTask(23);
  if (task23 && !task23.eventStartTime && task23.title === 'Essay Assignment 1') {
    await storage.updateTask(23, { eventStartTime: '18:00' });
    console.log('[Startup Fix] Restored Essay Assignment 1 (task 23) eventStartTime=18:00');
  }
} catch (e) {
  // Ignore if task doesn't exist on this environment
}

// Auto-sync CRCU shifts on startup
(async () => {
  try {
    const thirdAccStatus = await isThirdAccountConnected();

```



```

if (thirdAccStatus.connected) {
  const calendarId = 'family01331437021788124598@group.calendar.google.com';
  const now = new Date();
  const timeMin = new Date(now.getFullYear(), 0, 1);
  const timeMax = new Date(now.getFullYear() + 1, 0, 31, 23, 59, 59);
  const events = await getEventsFromThirdAccountCalendar(calendarId, timeMin, timeMax);
  const shiftEntries: { date: string; shiftType: string }[] = [];
  for (const event of events) {
    const summary = (event.summary || '').toLowerCase();
    if (!summary.includes('crcu')) continue;
    const startStr = event.start?.dateTime || event.start?.date;
    if (!startStr) continue;
    const startDate = new Date(startStr);
    const dateStr = `${startDate.getFullYear()}-${String(startDate.getMonth() + 1).padStart(2, '0')}-${String(s...
    const hour = startDate.getHours();
    const isNight = hour >= 18 || hour < 6 || summary.includes('<0<0 ');
    shiftEntries.push({ date: dateStr, shiftType: isNight ? 'night' : 'day' });
  }
  await storage.clearAllShifts();
  if (shiftEntries.length > 0) {
    await storage.setShiftBulk(shiftEntries);
  }
  console.log(`[Startup] Auto-synced ${shiftEntries.length} CRCU shifts`);
} else {
  console.log(`[Startup] Third Google account not connected, skipping CRCU shift sync`);
}
} catch (err) {
  console.error(`[Startup] CRCU shift auto-sync failed:`, err);
}
})();

(async () => {
  try {
    const sem = await storage.getActiveSemesterSettings();
    if (sem && !sem.course1BorderColor && !sem.course1CourseRowColor && !sem.course1TaskBgColor) {
      const rows = await db.select().from(degreeTrackingData).where(eq(degreeTrackingData.key, 'coursesData'));
      if (rows.length > 0 && rows[0].value) {
        const parsed = typeOf rows[0].value === 'string' ? JSON.parse(rows[0].value) : rows[0].value;
        const courses = parsed?.courses || [];
        const colorPayload: Record<string, any> = {};
        for (let i = 0; i < Math.min(courses.length, 3); i++) {
          const prefix = `course${i + 1}`;
          const c = courses[i];
          if (c.borderColor) colorPayload[`${prefix}BorderColor`] = c.borderColor;
          if (c.courseRowColor) colorPayload[`${prefix}CourseRowColor`] = c.courseRowColor;
          if (c.taskBgColor) colorPayload[`${prefix}TaskBgColor`] = c.taskBgColor;
        }
        if (Object.keys(colorPayload).length > 0) {
          await storage.updateSemesterSettings(sem.id, colorPayload);
          console.log(`[Startup] Seeded semester color columns from degree_tracking_data`);
        }
      }
    }
  } catch (err) {
    console.error(`[Startup] Color migration failed:`, err);
  }
})();

app.post("/api/tasks/compare-course-list", async (req, res) => {
  try {
    const { courseListText, courseName } = req.body;
    if (!courseListText || typeOf courseListText !== 'string') {
      return res.status(400).json({ error: "courseListText is required" });
    }

    const existingTasks = await storage.getTasks();
    const courseCode = courseName?.split('-')[0]?.trim()?.toUpperCase() || '';
    const courseTasks = existingTasks.filter(t => {
      const tc = t.courseName?.split('-')[0]?.trim()?.toUpperCase() || '';
      return tc === courseCode;
    });

    const lines = courseListText.split(/\r?\n/).map(l => l.trim()).filter(Boolean);

    const parsedItems: Array<{
      title: string;
      type: string;
      weekNumber: number | null;
      dueDate: string | null;
      startDate: string | null;
      gradeWeight: number | null;
      referenceLink: string | null;
      eventStartTime: string | null;
      eventEndTime: string | null;
    }> = [];

    const typeKeywords: Record<string, string[]> = {
      'reading': ['reading', 'read ch', 'textbook', 'chapter'],
      'module': ['module', 'lesson', 'lecture', 'watch'],
      'discussion': ['discussion', 'forum', 'post', 'respond', 'reply'],
      'quiz': ['quiz', 'test'],
      'exam': ['exam', 'midterm', 'final exam'],
      'essay': ['essay', 'paper', 'write', 'report', 'assignment', 'submit'],
      'poll': ['poll', 'survey', 'vote'],
      'project': ['project', 'presentation', 'group'],
    };

    const detectType = (text: string): string => {
      const lower = text.toLowerCase();
      for (const [type, keywords] of Object.entries(typeKeywords)) {
        if (keywords.some(kw => lower.includes(kw))) return type;
      }
      return 'other';
    };

    const parseDate = (text: string): string | null => {
      const datePatterns = [
        /(\d{4}-\d{2}-\d{2})/,

```

```

/(\d{1,2}\d{1,2}\d{2,4})/,
/((?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)[a-z]*\s+\d{1,2}(?:,\s+\d{4})?)/i,
/((?:Monday|Tuesday|Wednesday|Thursday|Friday|Saturday|Sunday),?\s+(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|O...
});
for (const pattern of datePatterns) {
  const match = text.match(pattern);
  if (match) {
    const d = new Date(match[1]);
    if (!isNaN(d.getTime())) return d.toISOString();
  }
}
return null;
};

const parseTime = (text: string): { start: string | null; end: string | null } => {
  const timeRange = text.match(/(\d{1,2}:\d{2}\s*(?:am|pm|AM|PM))\s*[-]\s*(\d{1,2}:\d{2}\s*(?:am|pm|AM|PM)?)/);
  if (timeRange) {
    return { start: timeRange[1].trim(), end: timeRange[2].trim() };
  }
  const ampmRange = text.match(/(\d{1,2}\s*(?:am|pm|AM|PM))\s*[-]\s*(\d{1,2}\s*(?:am|pm|AM|PM)?)/);
  if (ampmRange) {
    return { start: ampmRange[1].trim(), end: ampmRange[2].trim() };
  }
  return { start: null, end: null };
};

const parseWeek = (text: string): number | null => {
  const weekMatch = text.match(/week\s*(\d+)/i);
  if (weekMatch) return parseInt(weekMatch[1]);
  return null;
};

const parseWeight = (text: string): number | null => {
  const weightMatch = text.match(/(\d+(?:\.\d+)?)\s*/);
  if (weightMatch) return parseFloat(weightMatch[1]);
  return null;
};

let currentWeek: number | null = null;
let currentDate: string | null = null;

for (const line of lines) {
  const weekHeader = line.match(/^week\s*(\d+)/i);
  if (weekHeader) {
    currentWeek = parseInt(weekHeader[1]);
    const headerDate = parseDate(line);
    if (headerDate) currentDate = headerDate;
    continue;
  }

  const dateHeader = parseDate(line);
  if (dateHeader && line.match(/^(Mon|Tue|Wed|Thu|Fri|Sat|Sun|Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec|\...
    currentDate = dateHeader;
    continue;
  }

  if (line.match(/^[^=]+$/)) || line.match(/^[1,3]\s/) || line.length < 3) continue;

  const title = line
    .replace(/(\d{4}-\d{2}-\d{2})/g, '')
    .replace(/(\d{1,2}\d{1,2}\d{2,4})/g, '')
    .replace(/((?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)[a-z]*\s+\d{1,2}(?:,\s+\d{4})?)/gi, '')
    .replace(/(\d+(?:\.\d+)?)\s*/g, '')
    .replace(/week\s*\d+/gi, '')
    .replace(/(\d{1,2}:\d{2}\s*(?:am|pm|AM|PM))\s*[-]\s*(\d{1,2}:\d{2}\s*(?:am|pm|AM|PM)?)/g, '')
    .replace(/(\d{1,2}\s*(?:am|pm|AM|PM))\s*[-]\s*(\d{1,2}\s*(?:am|pm|AM|PM)?)/g, '')
    .replace(/\s*[-]*\s*/g, ' ')
    .replace(/\s*[-]\s*[-]\s*/g, ' - ')
    .replace(/\s*[-]\s*$\s*/g, ' ')
    .replace(/\/\s*[-]\s*/g, ' ')
    .replace(/\/\s*[-]\s*/g, ' ')
    .trim();

  if (!title || title.length < 3) continue;

  const type = detectType(line);
  const week = parseWeek(line) || currentWeek;
  const date = parseDate(line) || currentDate;
  const weight = parseWeight(line);
  const times = parseTime(line);

  parsedItems.push({
    title,
    type,
    weekNumber: week,
    dueDate: date,
    startDate: null,
    gradeWeight: weight,
    referenceLink: null,
    eventStartTime: times.start,
    eventEndTime: times.end,
  });
}

const changes: Array<{
  id: string;
  changeType: 'new' | 'modified' | 'removed';
  category: string;
  description: string;
  details: Record<string, { old: string; new: string }>;
  parsed: typeof parsedItems[0] | null;
  existingTaskId: number | null;
}> = [];

let changeIdx = 0;
const matchedExistingIds = new Set<number>();

for (const item of parsedItems) {

```

```

const normalizeTitle = (t: string) => t.toLowerCase().replace(/^[a-z0-9]/g, '');
const matchingTask = courseTasks.find(t => {
  const titleMatch = normalizeTitle(t.title) === normalizeTitle(item.title);
  const fuzzyMatch = normalizeTitle(t.title).includes(normalizeTitle(item.title)) ||
    normalizeTitle(item.title).includes(normalizeTitle(t.title));
  const weekMatch = item.weekNumber === null || t.weekNumber === item.weekNumber;
  return (titleMatch || (fuzzyMatch && weekMatch));
});

if (matchingTask) {
  matchedExistingIds.add(matchingTask.id);
  const diffs: Record<string, { old: string; new: string }> = {};
  if (item.dueDate && matchingTask.dueDate) {
    const existingDate = formatLocalDate(new Date(matchingTask.dueDate));
    const newDate = formatLocalDate(new Date(item.dueDate));
    if (existingDate !== newDate) diffs['dueDate'] = { old: existingDate, new: newDate };
  }
  if (item.type !== 'other' && item.type !== matchingTask.type) {
    diffs['type'] = { old: matchingTask.type, new: item.type };
  }
  if (item.gradeWeight !== null && item.gradeWeight !== matchingTask.gradeWeight) {
    diffs['gradeWeight'] = { old: String(matchingTask.gradeWeight || ''), new: String(item.gradeWeight) };
  }
  if (item.eventStartTime && item.eventStartTime !== matchingTask.eventStartTime) {
    diffs['eventStartTime'] = { old: matchingTask.eventStartTime || '', new: item.eventStartTime };
  }
  if (item.eventEndTime && item.eventEndTime !== matchingTask.eventEndTime) {
    diffs['eventEndTime'] = { old: matchingTask.eventEndTime || '', new: item.eventEndTime };
  }
  if (Object.keys(diffs).length > 0) {
    changes.push({
      id: `change-${changeIdx++}`,
      changeType: 'modified',
      category: item.type,
      description: `Update "${matchingTask.title}" (Week ${matchingTask.weekNumber})`,
      details: diffs,
      parsed: item,
      existingTaskId: matchingTask.id,
    });
  }
} else {
  changes.push({
    id: `change-${changeIdx++}`,
    changeType: 'new',
    category: item.type,
    description: `Add "${item.title}"${item.weekNumber ? ` (Week ${item.weekNumber})` : ''}`,
    details: {},
    parsed: item,
    existingTaskId: null,
  });
}

for (const task of courseTasks) {
  if (!matchedExistingIds.has(task.id) && !task.isCompleted) {
    changes.push({
      id: `change-${changeIdx++}`,
      changeType: 'removed',
      category: task.type,
      description: `Remove "${task.title}" (Week ${task.weekNumber})`,
      details: {},
      parsed: null,
      existingTaskId: task.id,
    });
  }
}

res.json({ changes, parsedCount: parsedItems.length, existingCount: courseTasks.length });
} catch (error) {
  console.error("Error comparing course list:", error);
  res.status(500).json({ error: "Failed to compare course list" });
}
});

app.post("/api/tasks/apply-course-changes", async (req, res) => {
  try {
    const { changes, courseName } = req.body;
    if (!changes || !Array.isArray(changes)) {
      return res.status(400).json({ error: "changes array is required" });
    }

    const results = { created: 0, updated: 0, deleted: 0, errors: 0 };
    const courseCode = courseName?.split(' - ')[0]?.trim()?.toUpperCase() || '';

    for (const change of changes) {
      try {
        if (change.existingTaskId) {
          const existingTask = await storage.getTask(change.existingTaskId);
          if (existingTask) {
            const taskCode = existingTask.courseName?.split(' - ')[0]?.trim()?.toUpperCase() || '';
            if (courseCode && taskCode !== courseCode) {
              results.errors++;
              continue;
            }
          }
        }
        if (change.changeType === 'new' && change.parsed) {
          await storage.createTask({
            title: change.parsed.title,
            type: change.parsed.type || 'other',
            courseName: courseName || '',
            weekNumber: change.parsed.weekNumber || 1,
            dueDate: change.parsed.dueDate ? new Date(change.parsed.dueDate) : new Date(),
            startDate: change.parsed.startDate ? new Date(change.parsed.startDate) : null,
            gradeWeight: change.parsed.gradeWeight,
            eventStartTime: change.parsed.eventStartTime,
            eventEndTime: change.parsed.eventEndTime,
          });
        }
      }
    }
  }
});

```

```

        results.created++;
    } else if (change.changeType === 'modified' && change.existingTaskId) {
        const updates: Record<string, unknown> = {};
        for (const [field, diff] of Object.entries(change.details)) {
            const d = diff as { old: string; new: string };
            if (field === 'dueDate') updates.dueDate = new Date(d.new);
            else if (field === 'gradeWeight') updates.gradeWeight = Math.round(parseFloat(d.new));
            else updates[field] = d.new;
        }
        await storage.updateTask(change.existingTaskId, updates);
        results.updated++;
    } else if (change.changeType === 'removed' && change.existingTaskId) {
        await storage.deleteTask(change.existingTaskId);
        results.deleted++;
    }
} catch (err) {
    console.error("Error applying change:", change, err);
    results.errors++;
}
}

res.json(results);
} catch (error) {
    console.error("Error applying course changes:", error);
    res.status(500).json({ error: "Failed to apply changes" });
}
});

app.get("/api/spotify/status", async (_req, res) => {
    res.json({ connected: spotifyApi.isConnected() });
});

app.get("/api/spotify/login", async (req, res) => {
    try {
        const authUrl = spotifyApi.getAuthUrl();
        res.redirect(authUrl);
    } catch (error: any) {
        res.status(500).send("Failed to start Spotify login: " + error.message);
    }
});

app.get("/api/spotify/callback", async (req, res) => {
    try {
        const code = req.query.code as string;
        if (!code) {
            return res.status(400).send("No authorization code received");
        }
        await spotifyApi.handleCallback(code);
        res.redirect("/?auth=5747&spotify=connected");
    } catch (error: any) {
        console.error("Spotify callback error:", error?.message || error);
        res.status(500).send("Spotify connection failed: " + error.message);
    }
});

app.get("/api/spotify/now-playing", async (_req, res) => {
    try {
        const playback = await spotifyApi.getNowPlaying();
        if (!playback || !playback.item) {
            return res.json({ playing: false });
        }
        const track = playback.item as any;
        res.json({
            playing: playback.is_playing,
            name: track.name,
            artist: track.artists?.map((a: any) => a.name).join(", ") || "Unknown",
            album: track.album?.name || "",
            albumArt: track.album?.images?.[0]?.url || "",
            albumArtSmall: track.album?.images?.[track.album.images.length - 1]?.url || "",
            progress: playback.progress_ms,
            duration: track.duration_ms,
            trackUrl: track.external_urls?.spotify || "",
        });
    } catch (error: any) {
        console.error("Spotify now-playing error:", error?.message || error);
        res.status(500).json({ error: "Failed to get Spotify status" });
    }
});

app.get("/api/spotify/recent", async (_req, res) => {
    try {
        const recent = await spotifyApi.getRecentTracks(5);
        const tracks = (recent?.items || []).map((item: any) => ({
            name: item.track.name,
            artist: item.track.artists?.map((a: any) => a.name).join(", ") || "Unknown",
            album: item.track.album?.name || "",
            albumArt: item.track.album?.images?.[0]?.url || "",
            albumArtSmall: item.track.album?.images?.[item.track.album.images.length - 1]?.url || "",
            playedAt: item.played_at,
            trackUrl: item.track.external_urls?.spotify || "",
        }));
        res.json(tracks);
    } catch (error: any) {
        console.error("Spotify recent error:", error?.message || error);
        res.status(500).json({ error: "Failed to get recent tracks" });
    }
});

app.put("/api/spotify/play", async (_req, res) => {
    try {
        await spotifyApi.play();
        res.json({ ok: true });
    } catch (error: any) {
        console.error("Spotify play error:", error?.message || error);
        res.status(500).json({ error: "Failed to play" });
    }
});

```

```

app.put("/api/spotify/pause", async (_req, res) => {
  try {
    await spotifyApi.pause();
    res.json({ ok: true });
  } catch (error: any) {
    console.error("Spotify pause error:", error?.message || error);
    res.status(500).json({ error: "Failed to pause" });
  }
});

app.post("/api/spotify/next", async (_req, res) => {
  try {
    await spotifyApi.next();
    res.json({ ok: true });
  } catch (error: any) {
    console.error("Spotify next error:", error?.message || error);
    res.status(500).json({ error: "Failed to skip" });
  }
});

app.post("/api/spotify/previous", async (_req, res) => {
  try {
    await spotifyApi.previous();
    res.json({ ok: true });
  } catch (error: any) {
    console.error("Spotify previous error:", error?.message || error);
    res.status(500).json({ error: "Failed to go back" });
  }
});

app.get("/api/spotify/playlists", async (_req, res) => {
  try {
    const data = await spotifyApi.getPlaylists();
    const items = (data?.items || []).map((p: any) => ({
      id: p.id,
      name: p.name,
      image: p.images?.[0]?.url || "",
      imageSmall: p.images?.[p.images.length - 1]?.url || p.images?.[0]?.url || "",
      trackCount: p.tracks?.total || 0,
      uri: p.uri,
      owner: p.owner?.display_name || "",
    }));
    res.json(items);
  } catch (error: any) {
    res.status(500).json({ error: "Failed to fetch playlists" });
  }
});

app.get("/api/spotify/albums", async (_req, res) => {
  try {
    const data = await spotifyApi.getSavedAlbums();
    const items = (data?.items || []).map((item: any) => ({
      id: item.album.id,
      name: item.album.name,
      artist: item.album.artists?.map((a: any) => a.name).join(", ") || "",
      image: item.album.images?.[0]?.url || "",
      imageSmall: item.album.images?.[item.album.images.length - 1]?.url || item.album.images?.[0]?.url || "",
      trackCount: item.album.total_tracks || 0,
      uri: item.album.uri,
      year: item.album.release_date?.substring(0, 4) || "",
    }));
    res.json(items);
  } catch (error: any) {
    res.status(500).json({ error: "Failed to fetch albums" });
  }
});

app.get("/api/spotify/artists", async (_req, res) => {
  try {
    const data = await spotifyApi.getTopArtists();
    const items = (data?.items || []).map((a: any) => ({
      id: a.id,
      name: a.name,
      image: a.images?.[0]?.url || "",
      imageSmall: a.images?.[a.images.length - 1]?.url || a.images?.[0]?.url || "",
      genres: a.genres?.slice(0, 3) || [],
      uri: a.uri,
    }));
    res.json(items);
  } catch (error: any) {
    res.status(500).json({ error: "Failed to fetch artists" });
  }
});

app.get("/api/spotify/tracks", async (_req, res) => {
  try {
    const data = await spotifyApi.getSavedTracks();
    const items = (data?.items || []).map((item: any) => ({
      id: item.track.id,
      name: item.track.name,
      artist: item.track.artists?.map((a: any) => a.name).join(", ") || "",
      album: item.track.album?.name || "",
      image: item.track.album?.images?.[0]?.url || "",
      imageSmall: item.track.album?.images?.[item.track.album.images.length - 1]?.url || "",
      duration: item.track.duration_ms || 0,
      uri: item.track.uri,
    }));
    res.json(items);
  } catch (error: any) {
    res.status(500).json({ error: "Failed to fetch tracks" });
  }
});

app.put("/api/spotify/volume", async (req, res) => {
  try {
    const { volume } = req.body;
    await spotifyApi.setVolume(volume);
    res.json({ ok: true });
  }
});

```

```

    } catch (error: any) {
      res.status(500).json({ error: "Failed to set volume" });
    }
  });

app.put("/api/spotify/play-context", async (req, res) => {
  try {
    const { uri, offset } = req.body;
    await spotifyApi.playContext(uri, offset);
    res.json({ ok: true });
  } catch (error: any) {
    res.status(500).json({ error: "Failed to play" });
  }
});

app.put("/api/spotify/play-tracks", async (req, res) => {
  try {
    const { uris } = req.body;
    await spotifyApi.playTracks(uris);
    res.json({ ok: true });
  } catch (error: any) {
    res.status(500).json({ error: "Failed to play tracks" });
  }
});

app.put("/api/spotify/shuffle", async (req, res) => {
  try {
    const { state } = req.body;
    await spotifyApi.setShuffle(state);
    res.json({ ok: true });
  } catch (error: any) {
    res.status(500).json({ error: "Failed to set shuffle" });
  }
});

app.put("/api/spotify/repeat", async (req, res) => {
  try {
    const { state } = req.body;
    await spotifyApi.setRepeat(state);
    res.json({ ok: true });
  } catch (error: any) {
    res.status(500).json({ error: "Failed to set repeat" });
  }
});

app.get("/api/spotify/playback-state", async (_req, res) => {
  try {
    const data = await spotifyApi.getPlaybackState();
    if (!data) {
      res.json({ active: false });
      return;
    }
    res.json({
      active: true,
      volume: data.device?.volume_percent ?? 50,
      shuffle: data.shuffle_state ?? false,
      repeat: data.repeat_state ?? "off",
      deviceName: data.device?.name || "Unknown",
      deviceType: data.device?.type || "Unknown",
    });
  } catch (error: any) {
    res.status(500).json({ error: "Failed to get playback state" });
  }
});

app.get("/api/spotify/search", async (req, res) => {
  try {
    const q = req.query.q as string;
    if (!q) { res.json({ tracks: [], artists: [], albums: [], playlists: [] }); return; }
    const data = await spotifyApi.search(q);
    res.json({
      tracks: (data?.tracks?.items || []).map((t: any) => ({
        id: t.id, name: t.name,
        artist: t.artists?.map((a: any) => a.name).join(", ") || "",
        album: t.album?.name || "",
        image: t.album?.images?.[0]?.url || "",
        imageSmall: t.album?.images?.[t.album.images.length - 1]?.url || "",
        duration: t.duration_ms || 0, uri: t.uri,
      })),
      artists: (data?.artists?.items || []).map((a: any) => ({
        id: a.id, name: a.name,
        image: a.images?.[0]?.url || "",
        imageSmall: a.images?.[a.images.length - 1]?.url || "",
        genres: a.genres?.slice(0, 3) || [], uri: a.uri,
      })),
      albums: (data?.albums?.items || []).map((a: any) => ({
        id: a.id, name: a.name,
        artist: a.artists?.map((ar: any) => ar.name).join(", ") || "",
        image: a.images?.[0]?.url || "",
        imageSmall: a.images?.[a.images.length - 1]?.url || "",
        year: a.release_date?.substring(0, 4) || "", uri: a.uri,
      })),
      playlists: (data?.playlists?.items || []).map((p: any) => ({
        id: p.id, name: p.name,
        image: p.images?.[0]?.url || "",
        imageSmall: p.images?.[p.images.length - 1]?.url || p.images?.[0]?.url || "",
        owner: p.owner?.display_name || "", uri: p.uri,
      })),
    });
  } catch (error: any) {
    res.status(500).json({ error: "Failed to search" });
  }
});

app.get("/api/spotify/related-artists", async (req, res) => {
  try {
    const artistId = req.query.artistId as string;
    if (!artistId) return res.json({ artists: [] });
  }
});

```

```

const data = await spotifyApi.getRelatedArtists(artistId);
const artists = (data?.artists || []).slice(0, 8).map((a: any) => ({
  id: a.id, name: a.name,
  image: a.images?.[0]?.url || "",
  imageSmall: a.images?.[a.images.length - 1]?.url || "",
  genres: a.genres?.slice(0, 3) || [],
  uri: a.uri,
})));
res.json({ artists });
} catch (error: any) {
  res.status(500).json({ error: "Failed to get related artists" });
}
});

app.post("/api/spotify/bulk-images", async (req, res) => {
  try {
    const { items } = req.body as { items: { name: string; uri: string; searchQuery?: string }[] };
    if (!items || !Array.isArray(items)) return res.json({ images: {} });
    const images: Record<string, string> = {};
    const ids: Record<string, string> = {};
    for (const item of items) {
      try {
        const parts = item.uri.split(":");
        const type = parts[1];
        const id = parts[2];
        if (type === "artist") {
          try {
            const data = await spotifyApi.getArtistById(id);
            if (data?.images?.[0]?.url) images[item.name] = data.images[0].url;
            if (data?.id) ids[item.name] = data.id;
          } catch {}
          if (!images[item.name]) {
            const q = item.searchQuery || item.name;
            const searchData = await spotifyApi.search(q, 'artist', 3);
            const artists = searchData?.artists?.items || [];
            for (const a of artists) {
              if (a?.images?.[0]?.url) {
                images[item.name] = a.images[0].url;
                if (a.id) ids[item.name] = a.id;
                break;
              }
            }
          }
        } else if (type === "playlist") {
          const data = await spotifyApi.getPlaylistById(id);
          if (data?.images?.[0]?.url) images[item.name] = data.images[0].url;
        } else if (type === "track") {
          try {
            const trackData = await spotifyApi.getTrackById(id);
            if (trackData?.album?.images?.[0]?.url) images[item.name] = trackData.album.images[0].url;
            if (trackData?.artists?.[0]?.id) ids[item.name] = trackData.artists[0].id;
          } catch {}
          const q = item.searchQuery || item.name;
          const searchData = await spotifyApi.search(q, 'track', 1);
          const track = searchData?.tracks?.items?.[0];
          if (track?.album?.images?.[0]?.url) images[item.name] = track.album.images[0].url;
        }
      } catch (e: any) {
        console.error(`[Spotify] Bulk image fetch failed for ${item.name}:`, e.message);
      }
    }
    const found = Object.keys(images);
    const missing = items.map(i => i.name).filter(n => !images[n]);
    console.log(`[Spotify] Bulk images: ${found.length} found, ${missing.length} missing${missing.length ? ': ' + m...`);
    res.json({ images, ids });
  } catch (error: any) {
    console.error("[Spotify] Bulk images error:", error.message);
    res.status(500).json({ error: "Failed to fetch images" });
  }
});

app.get("/api/spotify/devices", async (_req, res) => {
  try {
    const data = await spotifyApi.getDevices();
    const devices = (data?.devices || []).map((d: any) => ({
      id: d.id,
      name: d.name,
      type: d.type,
      isActive: d.is_active,
      volume: d.volume_percent,
    })));
    res.json(devices);
  } catch (error: any) {
    res.status(500).json({ error: "Failed to get devices" });
  }
});

app.put("/api/spotify/transfer", async (req, res) => {
  try {
    const { deviceId } = req.body;
    await spotifyApi.transferPlayback(deviceId);
    res.json({ ok: true });
  } catch (error: any) {
    res.status(500).json({ error: "Failed to transfer playback" });
  }
});

app.get("/api/spotify/rooms", async (_req, res) => {
  try {
    const rooms = FLICK_DEVICES.map(g => ({
      room: g.room,
      icon: g.icon,
      speakers: g.devices.filter(d => d.type === "echo" || d.type === "echo_show" || d.type === "speaker" || d.type...
      id: d.id,
      name: d.name,
      entityId: d.entityId,
      type: d.type,
    })));
  }
});

```

```

        room: d.room,
      })),
    })).filter(g => g.speakers.length > 0);
  res.json(rooms);
} catch (error: any) {
  res.status(500).json({ error: "Failed to get rooms" });
}
});

app.post("/api/spotify/play-on-speaker", async (req, res) => {
  try {
    const { entityId, spotifyUri, artistName, searchQuery, deviceType, announceMessage, command } = req.body;
    if (!entityId) return res.status(400).json({ error: "entityId required" });
    const baseUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

    if (command === "pause" || command === "stop") {
      console.log(`[Spotify] ${command} command for entity: ${entityId}`);
      try {
        await fetch(`${baseUrl}/api/services/media_player/media_${command === "stop" ? "stop" : "pause"}`, {
          method: 'POST',
          headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
          body: JSON.stringify({ entity_id: SPOTIFYPLUS_ENTITY }),
        });
        console.log(`[Spotify] SpotifyPlus ${command} sent`);
      } catch (e: any) {
        console.log(`[Spotify] SpotifyPlus ${command} failed: ${e.message}`);
      }
      try {
        await fetch(`${baseUrl}/api/services/media_player/media_${command === "stop" ? "stop" : "pause"}`, {
          method: 'POST',
          headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
          body: JSON.stringify({ entity_id: entityId }),
        });
        console.log(`[Spotify] ${command} sent to ${entityId}`);
      } catch (e: any) {
        console.log(`[Spotify] ${command} on ${entityId} failed: ${e.message}`);
      }
      if (command === "stop") {
        spotifyActivePlaybacks.delete(entityId);
        if (spotifyActivePlaybacks.size === 0) clearSpotifyStateTimer();
      } else {
        startSpotifyStateTimer();
      }
      return res.json({ ok: true, action: command });
    }

    const isEcho = entityId.includes("echo") || entityId.includes("_am") || deviceType === "echo" || deviceType === ...

    if (isEcho) {
      let targetEntity = entityId;
      if (entityId === EVERYWHERE_GROUP_ENTITY) {
        const anyEcho = FLICK_DEVICES.flatMap(g => g.devices).find(d => d.type === "echo" && d.entityId.includes("_...
        if (anyEcho) {
          console.log(`[Spotify] BYhome group !' using Echo for voice command: ${anyEcho.entityId}`);
          targetEntity = anyEcho.entityId;
        }
      } else if (entityId.includes("_group") || entityId.includes("_media_group")) {
        const roomGroup = FLICK_DEVICES.find(g =>
          g.devices.some(d => d.entityId === entityId) ||
          g.speakers?.some(s => s.entityId === entityId)
        );
        if (roomGroup) {
          const echoDevice = roomGroup.devices.find(d => d.type === "echo" && d.entityId.includes("_am"));
          if (echoDevice) {
            console.log(`[Spotify] Resolved group ${entityId} !' individual Echo: ${echoDevice.entityId}`);
            targetEntity = echoDevice.entityId;
          }
        }
        if (targetEntity === entityId) {
          for (const group of FLICK_DEVICES) {
            for (const dev of group.devices) {
              if (dev.type === "group" && dev.entityId === entityId) {
                const echoInRoom = group.devices.find(d => d.type === "echo" && d.entityId.includes("_am"));
                if (echoInRoom) {
                  console.log(`[Spotify] Resolved group ${entityId} !' Echo in ${group.room}: ${echoInRoom.entityId}`);
                  targetEntity = echoInRoom.entityId;
                }
                break;
              }
            }
          }
          if (targetEntity !== entityId) break;
        }
      }
    }

    if (announceMessage) {
      const isEverywhereGroup = entityId === EVERYWHERE_GROUP_ENTITY;
      if (isEverywhereGroup) {
        const allEchoTargets = FLICK_DEVICES.flatMap(g => g.devices).filter(d => (d.type === "echo" || d.type === ...
        console.log(`[Spotify] TTS announce on ALL ${allEchoTargets.length} Echo speakers: "${announceMessage}"`);
        try {
          await fetch(`${baseUrl}/api/services/notify/alexa_media`, {
            method: 'POST',
            headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
            body: JSON.stringify({
              message: announceMessage,
              data: { type: "tts" },
              target: allEchoTargets,
            }),
          });
          await new Promise(resolve => setTimeout(resolve, 2500));
        } catch (ttsErr: any) {
          console.log(`[Spotify] TTS announce on all speakers failed (continuing): ${ttsErr.message}`);
        }
      } else {
        console.log(`[Spotify] TTS announce on ${targetEntity}: "${announceMessage}"`);
        try {

```



```

        await fetch(`${haUrl}/api/services/notify/alexa_media`, {
            method: 'POST',
            headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
            body: JSON.stringify({
                message: announceMessage,
                data: { type: "tts" },
                target: [targetEntity],
            })),
        });
        await new Promise(resolve => setTimeout(resolve, 2500));
    } catch (ttsErr: any) {
        console.log(`[Spotify] TTS announce failed (continuing): ${ttsErr.message}`);
    }
}

const spotifyPlusSourceMap: Record<string, string> = {
    [EVERYWHERE_GROUP_ENTITY]: "Byhome",
    "media_player.king_bedroom_media_group": "King Bedroom",
    "media_player.queen_bedroom_media_group": "Queen Bedroom",
    "media_player.living_room_media_group": "Echo - LR Studio White AM",
    "media_player.kitchen_media_group": "Echo - Kitchen Studio Black AM",
    "media_player.hallway_media_group": "Echo - Hallway Corner",
    "media_player.closet_media_group": "Echo - Closet AM",
    "media_player.pug_media_group": "Echo Show - Pug AM",
    "media_player.echo_closet_am": "Echo - Closet AM",
    "media_player.echo_show_pug_am": "Echo Show - Pug AM",
    "media_player.echo_lr_couch_l_am": "Echo - LR Couch (L) AM",
    "media_player.echo_lr_studio_white_am": "Echo - LR Studio White AM",
    "media_player.echo_king_l_am": "Echo - King (L) AM",
    "media_player.echo_king_r_am": "Echo - King (R) AM",
    "media_player.echo_king_tv_am": "Echo - King TV AM",
    "media_player.echo_queen_bed_l_am": "Echo - Queen Bed (L) AM",
    "media_player.echo_queen_bed_r_am": "Echo - Queen Bed (R) AM",
    "media_player.echo_queen_balcony_am": "Echo - Queen Balcony AM",
    "media_player.echo_kitchen_island_corner_am": "Echo - Kitchen Island Corner AM",
    "media_player.echo_kitchen_studio_black_am": "Echo - Kitchen Studio Black AM",
    "media_player.echo_kitchen_cupboards_left_am": "Echo - Kitchen Cupboards (Left) AM",
    "media_player.echo_kitchen_cupboards_r_am": "Echo - Kitchen Cupboards (R) AM",
    "media_player.echo_kitchen_hutch_am": "Echo - Kitchen Hutch AM",
    "media_player.echo_kitchen_fridge_am": "Echo - Kitchen Fridge AM",
    "media_player.echo_hallway_entrance_am": "Echo - Hallway Corner",
    "media_player.echo_lr_hub_am": "Echo - LR Hub AM",
    "media_player.echo_lr_tv_shelf_am": "Echo - LR TV Shelf AM",
};

const groupId = entityId !== targetEntity ? entityId : null;
const spSource = spotifyPlusSourceMap[entityId] || spotifyPlusSourceMap[targetEntity];

const isRadioCommand = !spotifyUri && searchQuery && (searchQuery.toLowerCase().includes("fm") || searchQuery...
if (isRadioCommand) {
    const voiceCommand = `play ${searchQuery}`;
    console.log(`[Spotify] Radio/TuneIn content detected, sending voice command to ${targetEntity}: "${voiceCom...`);
    try {
        await fetch(`${haUrl}/api/services/media_player/turn_on`, {
            method: 'POST',
            headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
            body: JSON.stringify({ entity_id: targetEntity }),
        });
        await new Promise(resolve => setTimeout(resolve, 1500));
    } catch (wakeErr: any) {
        console.log(`[Spotify] Wake-up failed (continuing): ${wakeErr.message}`);
    }
    await fetch(`${haUrl}/api/services/media_player/play_media`, {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
        body: JSON.stringify({ entity_id: targetEntity, media_content_id: voiceCommand, media_content_type: "cust...`);
    });
    trackSpotifyPlayback(entityId, artistName);
    clearSpotifyStaleTimer();
} else if (spSource && spotifyUri) {
    console.log(`[Spotify] Using SpotifyPlus: source="${spSource}", uri=${spotifyUri}`);

    try {
        await fetch(`${haUrl}/api/services/media_player/turn_on`, {
            method: 'POST',
            headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
            body: JSON.stringify({ entity_id: SPOTIFYPLUS_ENTITY }),
        });
        await new Promise(resolve => setTimeout(resolve, 500));
    } catch (e: any) {
        console.log(`[Spotify] SpotifyPlus turn_on failed (continuing): ${e.message}`);
    }
}

const selectResp = await fetch(`${haUrl}/api/services/media_player/select_source`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({
        entity_id: SPOTIFYPLUS_ENTITY,
        source: spSource,
    })),
    });
    console.log(`[Spotify] SpotifyPlus select_source "${spSource}": ${selectResp.status}`);
    await new Promise(resolve => setTimeout(resolve, 2000));

const isArtistUri = spotifyUri.startsWith("spotify:artist:");
if (isArtistUri) {
    console.log(`[Spotify] Artist URI detected, using player_media_play_context for shuffle play`);
    const playResp = await fetch(`${haUrl}/api/services/spotifyplus/player_media_play_context`, {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
        body: JSON.stringify({
            entity_id: SPOTIFYPLUS_ENTITY,
            context_uri: spotifyUri,
            position_ms: 0,
            delay: 0.50,
        })),
        });

```

```

    }},
  });
  const playText = await playResp.text();
  console.log(`[Spotify] SpotifyPlus player_media_play_context response: ${playResp.status} body=${playText...}`);
  if (!playResp.ok) {
    console.log(`[Spotify] player_media_play_context failed, falling back to voice command`);
    const searchTerm = searchQuery || artistName || "music";
    const voiceCommand = `play ${searchTerm} on Spotify`;
    console.log(`[Spotify] Sending voice command to ${targetEntity}: "${voiceCommand}"`);
    await fetch(`${haUrl}/api/services/media_player/play_media`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ entity_id: targetEntity, media_content_id: voiceCommand, media_content_type: "... " });
    });
  }
} else {
  const playResp = await fetch(`${haUrl}/api/services/media_player/play_media`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({
      entity_id: SPOTIFYPLUS_ENTITY,
      media_content_id: spotifyUri,
      media_content_type: "spotify",
    }),
  });
  const playText = await playResp.text();
  console.log(`[Spotify] SpotifyPlus play response: ${playResp.status} body=${playText.substring(0, 300)}`);
}
trackSpotifyPlayback(entityId, artistName);
clearSpotifyStaleTimer();
} else {
  console.log(`[Spotify] No SpotifyPlus source for ${entityId}, using voice command fallback`);
  const searchTerm = searchQuery || artistName || "music";
  const voiceCommand = `play ${searchTerm} on Spotify`;

  console.log(`[Spotify] Waking up Echo: ${targetEntity}`);
  try {
    await fetch(`${haUrl}/api/services/media_player/turn_on`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ entity_id: targetEntity }),
    });
    await new Promise(resolve => setTimeout(resolve, 1500));
  } catch (wakeErr: any) {
    console.log(`[Spotify] Wake-up failed (continuing): ${wakeErr.message}`);
  }

  console.log(`[Spotify] Sending voice command to ${targetEntity}: "${voiceCommand}"`);
  await fetch(`${haUrl}/api/services/media_player/play_media`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({
      entity_id: targetEntity,
      media_content_id: voiceCommand,
      media_content_type: "custom",
    }),
  });
}
} else {
  const searchTerm = searchQuery || artistName || "music";
  const voiceCmd = `play ${searchTerm} on Spotify`;
  console.log(`[Spotify] Non-echo device, sending voice command to ${entityId}: "${voiceCmd}"`);
  await fetch(`${haUrl}/api/services/media_player/play_media`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ entity_id: entityId, media_content_id: voiceCmd, media_content_type: "custom" }),
  });
}
console.log(`[Spotify] Play command complete for ${entityId}: "${searchQuery || artistName}"`);
trackSpotifyPlayback(entityId, artistName);
clearSpotifyStaleTimer();

const volumeTarget = entityId;
try {
  await fetch(`${haUrl}/api/services/media_player/volume_set`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ entity_id: volumeTarget, volume_level: 0.35 }),
  });
  console.log(`[Spotify] Set volume to 35% on ${volumeTarget}`);
} catch (volErr: any) {
  console.log(`[Spotify] Volume set failed (continuing): ${volErr.message}`);
}

res.json({ ok: true });
} catch (error: any) {
  console.error(`[Spotify] Play on speaker error:`, error);
  res.status(500).json({ error: "Failed to play on speaker" });
}
});

app.post("/api/spotify/group-speakers", async (req, res) => {
  try {
    const { sourceEntityId, targetEntityId } = req.body;
    if (!sourceEntityId || !targetEntityId) return res.status(400).json({ error: "Both entity IDs required" });
    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

    await fetch(`${haUrl}/api/services/media_player/join`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ entity_id: sourceEntityId, group_members: [targetEntityId] }),
    });
    console.log(`[Spotify] Grouped ${targetEntityId} into ${sourceEntityId}`);
    res.json({ ok: true });
  } catch (error: any) {
    console.error(`[Spotify] Group speakers error:`, error);
    res.status(500).json({ error: "Failed to group speakers" });
  }
}

```

```

});

let spotifyActivePlaybacks: Map<string, { entityId: string; startedAt: number; artistName?: string }> = new Map();
let spotifyStaleTimer: NodeJS.Timeout | null = null;
const SPOTIFY_STALE_TIMEOUT_MS = 10 * 60 * 1000;

function trackSpotifyPlayback(entityId: string, artistName?: string) {
    spotifyActivePlaybacks.set(entityId, { entityId, startedAt: Date.now(), artistName });
    console.log(`[Spotify] Tracking playback on ${entityId} (${artistName || 'unknown'}). Active: ${spotifyActivePlay...}`);
}

function clearSpotifyStaleTimer() {
    if (spotifyStaleTimer) {
        clearTimeout(spotifyStaleTimer);
        spotifyStaleTimer = null;
    }
}

function startSpotifyStaleTimer() {
    clearSpotifyStaleTimer();
    spotifyStaleTimer = setTimeout(async () => {
        if (spotifyActivePlaybacks.size === 0) return;
        console.log(`[Spotify] Stale timeout (${SPOTIFY_STALE_TIMEOUT_MS / 1000}s) reached. Clearing ${spotifyActivePla...}`);
        const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

        try {
            await fetch(`${haUrl}/api/services/media_player/media_stop`, {
                method: 'POST',
                headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
                body: JSON.stringify({ entity_id: SPOTIFYPLUS_ENTITY }),
            });
            console.log(`[Spotify] Stale cleanup: SpotifyPlus stopped`);
        } catch (e: any) {
            console.log(`[Spotify] Stale cleanup: SpotifyPlus stop failed: ${e.message}`);
        }
    }, SPOTIFY_STALE_TIMEOUT_MS);
    console.log(`[Spotify] Stale timer started (${SPOTIFY_STALE_TIMEOUT_MS / 1000}s)`);
}

app.post("/api/spotify/stop-all", async (_req, res) => {
    try {
        const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

        const results: string[] = [];
        clearSpotifyStaleTimer();

        try {
            await fetch(`${haUrl}/api/services/media_player/media_stop`, {
                method: 'POST',
                headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
                body: JSON.stringify({ entity_id: SPOTIFYPLUS_ENTITY }),
            });
            results.push("SpotifyPlus stopped");
        } catch (e: any) {
            results.push(`SpotifyPlus stop failed: ${e.message}`);
        }

        for (const [, pb] of spotifyActivePlaybacks) {
            try {
                await fetch(`${haUrl}/api/services/media_player/media_stop`, {
                    method: 'POST',
                    headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
                    body: JSON.stringify({ entity_id: pb.entityId }),
                });
                results.push(`Stopped ${pb.entityId}`);
            } catch (e: any) {
                results.push(`${pb.entityId} stop failed: ${e.message}`);
            }
        }

        try {
            await spotifyApi.pause();
            results.push("Spotify API paused");
        } catch (e: any) {
            results.push(`Spotify API pause failed: ${e.message}`);
        }

        const count = spotifyActivePlaybacks.size;
        spotifyActivePlaybacks.clear();
        console.log(`[Spotify] Stop-all: cleared ${count} tracked playbacks. Results: ${results.join(', ')}`);
        res.json({ ok: true, cleared: count, results });
    } catch (error: any) {
        console.error("[Spotify] Stop-all error:", error);
        res.status(500).json({ error: "Failed to stop all" });
    }
});

app.post("/api/spotify/flick", async (req, res) => {
    try {
        const { deviceId } = req.body;
        if (!deviceId) return res.status(400).json({ error: "deviceId is required" });

        let device: FlickDevice | undefined;
        let deviceRoom = "";
    }

```

```

for (const group of FLICK_DEVICES) {
  const found = group.devices.find(d => d.id === deviceId);
  if (found) { device = found; deviceRoom = group.room; break; }
}
if (!device) return res.status(404).json({ error: "Device not found" });

const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
const appUrl = `https://${req.get('host')} | new URL(DEPLOYED_APP_URL).host`;
const authParam = req.query.auth || "bryn";
const spotifyUrl = `${appUrl}/spotify?auth=${authParam}`;
console.log(`[Spotify Flick] Sending to ${device.name} (${deviceRoom}): ${spotifyUrl}`);

const navigateDevice = async (targetDevice: FlickDevice) => {
  if (!targetDevice.canDisplay) return;
  try {
    if (targetDevice.type === "tablet" || targetDevice.type === "echo_show") {
      const navResp = await fetch(`${haUrl}/api/services/browser_mod/navigate`, {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
        body: JSON.stringify({ browser_id: targetDevice.entityId, path: spotifyUrl }),
      });
      console.log(`[Spotify Flick] Navigated ${targetDevice.entityId} via browser_mod: ${navResp.status}`);
    } else if (targetDevice.type === "tv") {
      const castResp = await fetch(`${haUrl}/api/services/media_player/play_media`, {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
        body: JSON.stringify({ entity_id: targetDevice.entityId, media_content_id: spotifyUrl, media_content_ty...
      });
      console.log(`[Spotify Flick] Cast to TV ${targetDevice.entityId}: ${castResp.status}`);
    }
  } catch (navErr: any) {
    console.error(`[Spotify Flick] Navigation failed for ${targetDevice.name}: ${navErr.message}`);
  }
};

if (device.canDisplay) {
  await navigateDevice(device);
} else if (device.type === "group") {
  const roomGroup = FLICK_DEVICES.find(g => g.devices.some(d => d.id === device!.id));
  if (roomGroup) {
    const screenDevices = roomGroup.devices.filter(d => d.canDisplay && d.id !== device!.id);
    for (const screenDevice of screenDevices) {
      await navigateDevice(screenDevice);
    }
  }
}

res.json({ success: true, device: device.name, room: deviceRoom });
} catch (error: any) {
  console.error("[Spotify Flick] Error:", error);
  res.status(500).json({ error: "Failed to flick", details: error.message });
}
});

app.post("/api/spotify/ungroup-speaker", async (req, res) => {
  try {
    const { entityId } = req.body;
    if (!entityId) return res.status(400).json({ error: "entityId required" });
    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');

    await fetch(`${haUrl}/api/services/media_player/unjoin`, {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${HOME_ASSISTANT_TOKEN}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ entity_id: entityId }),
    });
    console.log(`[Spotify] Ungrouped ${entityId}`);
    res.json({ ok: true });
  } catch (error: any) {
    console.error("[Spotify] Ungroup speaker error:", error);
    res.status(500).json({ error: "Failed to ungroup speaker" });
  }
});

app.post("/api/spotify/go-home", async (req, res) => {
  try {
    const haUrl = HOME_ASSISTANT_URL.replace(/\/$/, '');
    const homeUrl = "http://172.24.0.2:8123/lovelace/test-home";
    const tabletWrapperUrl = `${DEPLOYED_APP_URL}/tablet`;
    const tabletAdbEntities = [
      { entity: "media_player.tablet_hallway_entrance", name: "Hallway Entrance" },
      { entity: "media_player.tablet_hallway", name: "Hallway Main" },
      { entity: "media_player.tablet_11", name: "Living Room" },
      { entity: "media_player.bd24bb29_04a116d8_king", name: "King Bedroom" },
      { entity: "media_player.tablet_queen", name: "Queen Bedroom" },
      { entity: "media_player.tablet_kitchen_island", name: "Kitchen Island" },
      { entity: "media_player.tablet_cat", name: "Cat Washroom" },
    ];
    console.log(`[Spotify Home] Navigating ${tabletAdbEntities.length} tablets to ${homeUrl} via ADB`);
    res.json({ ok: true, navigating: tabletAdbEntities.length });

    await Promise.allSettled(
      tabletAdbEntities.map(async (tablet) => {
        try {
          await haServiceCall('androidtv/adb_command', {
            entity_id: tablet.entity,
            command: `am start --activity-clear-task -a android.intent.action.VIEW -d "${homeUrl}" com.amazon.cloud9`,
            `Spotify Home ADB ${tablet.name}`
          });
          console.log(`[Spotify Home] ${tablet.name} !' ADB navigate sent`);
        } catch (e: any) {
          console.log(`[Spotify Home] ${tablet.name} !' ADB failed: ${e.message}`);
        }
      })
    );
  } catch (error: any) {
    console.error("[Spotify Home] Error:", error);
    if (!res.headersSent) res.status(500).json({ error: error.message });
  }
});

```

```

app.get("/api/pending-review", async (_req, res) => {
  try {
    const status = (_req.query.status as string) || undefined;
    const items = await storage.getPendingReviewItems(status);
    const todayStart = new Date(torontoDate().toDateString());
    const normalizeTitle = (t: string) => t.toLowerCase().replace(/s+/g, ' ').replace(/^[a-z0-9 ]/g, '').trim();
    const allTasks = await storage.getTasks();
    const taskKeys = new Set(allTasks.map(t => {
      const normT = normalizeTitle(t.title || '');
      const dateK = t.dueDate ? new Date(t.dueDate).toISOString().split('T')[0] : 'nodate';
      return `${normT}||${dateK}`;
    }));
    const filtered = items.filter(item => {
      if (item.startDate) {
        const itemDate = new Date(new Date(item.startDate).toDateString());
        if (itemDate < todayStart) return false;
      }
      const normT = normalizeTitle(item.title || '');
      const dateK = item.startDate ? new Date(item.startDate).toISOString().split('T')[0] : 'nodate';
      if (taskKeys.has(`${normT}||${dateK}`)) return false;
      return true;
    });
    res.json(filtered);
  } catch (error: any) {
    console.error("[Review] Error fetching pending items:", error);
    res.status(500).json({ error: error.message });
  }
});

app.post("/api/pending-review/:id/accept", async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const item = (await storage.getPendingReviewItems()).find(i => i.id === id);
    if (!item) return res.status(404).json({ error: "Not found" });

    const rawData = item.rawData ? JSON.parse(item.rawData) : {};

    let weekNumber = 1;
    if (rawData.weekNumber) {
      weekNumber = rawData.weekNumber;
    } else {
      const semesterSettings = await storage.getActiveSemesterSettings();
      if (semesterSettings?.semesterStartDate) {
        const { getWeekNumber } = await import('../shared/schema');
        weekNumber = getWeekNumber(
          new Date(),
          new Date(semesterSettings.semesterStartDate),
          semesterSettings.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : null
        );
      }
    }

    const taskData: any = {
      title: item.title,
      type: item.taskType || 'meeting',
      courseName: item.courseName || null,
      dueDate: item.startDate || new Date(),
      eventStartTime: item.eventStartTime || null,
      eventEndTime: item.eventEndTime || null,
      weekNumber,
      priority: rawData.priority || 'medium',
      description: item.description || null,
      isCompleted: false,
      isAcknowledged: true,
      ...(req.body.overrides || {}),
    };

    const task = await storage.createTask(taskData);

    try {
      const event = await createCalendarEvent({
        id: task.id,
        title: task.title,
        description: task.description,
        dueDate: task.dueDate,
        courseName: task.courseName,
      });
      if (event?.id) {
        await storage.updateTask(task.id, { calendarEventId: event.id, calendarProvider: 'google' });
      }
    } catch (calErr: any) {
      console.log(`[Review] Calendar sync failed (non-fatal): ${calErr.message}`);
    }

    await storage.updatePendingReviewItem(id, { status: 'accepted' });

    console.log(`[Review] Accepted item #${id} -> task #${task.id}: "${task.title}"`);
    res.json({ success: true, task });
  } catch (error: any) {
    console.error("[Review] Error accepting item:", error);
    res.status(500).json({ error: error.message });
  }
});

app.post("/api/pending-review/:id/reject", async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    await storage.updatePendingReviewItem(id, { status: 'rejected' });
    console.log(`[Review] Rejected item #${id}`);
    res.json({ success: true });
  } catch (error: any) {
    console.error("[Review] Error rejecting item:", error);
    res.status(500).json({ error: error.message });
  }
});

app.delete("/api/pending-review/:id", async (req, res) => {

```

```

    try {
      const id = parseInt(req.params.id);
      await storage.deletePendingReviewItem(id);
      res.json({ success: true });
    } catch (error: any) {
      console.error("[Review] Error deleting item:", error);
      res.status(500).json({ error: error.message });
    }
  });

app.post("/api/pending-review/dedup", async (_req, res) => {
  try {
    const items = await storage.getPendingReviewItems('pending');
    const seen = new Map<string, number>();
    let removed = 0;
    const normalizeTitle = (t: string) => t.toLowerCase().replace(/\\s+/g, ' ').replace(/^[a-z0-9 ]/g, '').trim();
    for (const item of items) {
      const normTitle = normalizeTitle(item.title || '');
      const dateKey = item.startDate ? new Date(item.startDate).toISOString().split('T')[0] : 'nodate';
      const key = `${normTitle}||${dateKey}`;
      if (seen.has(key)) {
        await storage.deletePendingReviewItem(item.id);
        removed++;
      } else {
        seen.set(key, item.id);
      }
    }
    console.log(`[Review] Dedup: removed ${removed} duplicates from ${items.length} pending items`);
    res.json({ removed, remaining: items.length - removed });
  } catch (error: any) {
    console.error("[Review] Dedup error:", error);
    res.status(500).json({ error: error.message });
  }
});

app.post("/api/outlook/sync", async (_req, res) => {
  try {
    const result = await syncOutlookEventsToReview();
    res.json(result);
  } catch (error: any) {
    console.error("[Outlook] Sync error:", error);
    res.status(500).json({ error: error.message });
  }
});

app.get("/api/outlook/events-debug", async (_req, res) => {
  try {
    const events = await fetchOutlookCalendarEvents(30);
    res.json({ count: events.length, events: events.map(e => ({ id: e.id, subject: e.subject, start: e.start, end: ...
  } catch (error: any) {
    console.error("[Outlook] Events debug error:", error);
    res.status(500).json({ error: error.message });
  }
});

app.post("/api/outlook/migrate-folder", async (req, res) => {
  try {
    const { sourceFolder, destFolder, deleteRulesContaining, keepDomains } = req.body;

    if (deleteRulesContaining) {
      const deleted = await deleteMailRulesByName(deleteRulesContaining);
      console.log(`[Outlook Mail] Deleted ${deleted} rules containing "${deleteRulesContaining}"`);
    }

    if (sourceFolder && destFolder) {
      const srcId = await getMailFolderId(sourceFolder);
      const dstId = await findOrCreateMailFolder(destFolder);
      if (!srcId) return res.status(404).json({ error: `Folder "${sourceFolder}" not found` });

      if (keepDomains && Array.isArray(keepDomains)) {
        const result = await moveEmailsNotFromDomains(srcId, dstId, keepDomains);
        res.json({ success: true, ...result, from: sourceFolder, to: destFolder });
      } else {
        const moved = await moveAllEmailsFromFolder(srcId, dstId);
        res.json({ success: true, moved, from: sourceFolder, to: destFolder });
      }
    } else {
      res.json({ success: true, message: "Rules cleaned up" });
    }
  } catch (error: any) {
    console.error("[Outlook Mail] Migrate error:", error);
    res.status(500).json({ error: error.message });
  }
});

app.post("/api/outlook/file-emails", async (req, res) => {
  try {
    const { folderName, senderDomain, moveExisting } = req.body;
    if (!folderName || !senderDomain) {
      return res.status(400).json({ error: "folderName and senderDomain are required" });
    }

    const folderId = await findOrCreateMailFolder(folderName);

    const ruleName = `Auto-file ${senderDomain} !' ${folderName}`;
    const rule = await createMailRule(ruleName, senderDomain, folderId);

    let movedCount = 0;
    if (moveExisting !== false) {
      movedCount = await moveExistingEmailsToFolder(senderDomain, folderId);
    }

    res.json({
      success: true,
      folderId,
      rule: { id: rule.id, displayName: rule.displayName },
      movedExisting: movedCount
    });
  }
});

```

```

    } catch (error: any) {
      console.error("[Outlook Mail] File emails error:", error);
      res.status(500).json({ error: error.message });
    }
  });

```

```

app.post("/api/tasks/:id/invite", async (req, res) => {
  try {
    const taskId = parseInt(req.params.id);
    const task = await storage.getTask(taskId);
    if (!task) return res.status(404).json({ error: "Task not found" });

    const { emails } = req.body;
    if (!emails || !Array.isArray(emails) || emails.length === 0) {
      return res.status(400).json({ error: "emails array required" });
    }

    await storage.updateTask(taskId, { inviteEmails: emails } as any);

    const startDate = task.dueDate ? new Date(task.dueDate) : new Date();
    if (task.eventStartTime) {
      const [h, m] = task.eventStartTime.split(':').map(Number);
      startDate.setHours(h, m, 0, 0);
    }

    let endDate: Date | undefined;
    if (task.eventEndTime) {
      endDate = new Date(startDate);
      const [h, m] = task.eventEndTime.split(':').map(Number);
      endDate.setHours(h, m, 0, 0);
    }

    const result = await sendCalendarInvite({
      to: emails,
      title: task.title,
      description: task.description || undefined,
      startDate,
      endDate,
      organizerName: 'Bryn',
      organizerEmail: 'bryn.kai-hendricks@outlook.com',
    });

    if (result.success) {
      console.log(`[Invite] Sent .ics to ${emails.join(', ')} for task #${taskId}`);
      res.json({ success: true, sentTo: emails });
    } else {
      res.status(500).json({ error: result.error });
    }
  } catch (error: any) {
    console.error("[Invite] Error:", error);
    res.status(500).json({ error: error.message });
  }
});

```

```

app.post("/api/code-checker/analyze", async (req, res) => {
  try {
    const { code, language } = req.body;
    if (!code || !language) return res.status(400).json({ error: "code and language required" });

    const OpenAI = (await import("openai")).default;
    const openai = new OpenAI({
      apiKey: process.env.AI_INTEGRATIONS_OPENAI_API_KEY,
      baseURL: process.env.AI_INTEGRATIONS_OPENAI_BASE_URL,
    });

    const completion = await openai.chat.completions.create({
      model: "gpt-4o-mini",
      messages: [
        {
          role: "system",
          content: `You are a helpful code reviewer for a beginner programmer. Analyze the provided ${language} code...
1. List any syntax errors, bugs, or logical issues you find
2. For each issue, explain WHAT is wrong and HOW to fix it in simple terms
3. If the code is correct, say so and suggest any improvements
4. Show the corrected version of the code if there are errors
Keep your tone friendly and educational. Format your response clearly with numbered points.`
        },
        { role: "user", content: code }
      ],
      max_tokens: 2000,
    });

    const analysis = completion.choices[0]?.message?.content || "No analysis generated";
    res.json({ analysis });
  } catch (error: any) {
    console.error("[CodeChecker] Analysis error:", error);
    res.status(500).json({ error: error.message || "Analysis failed" });
  }
});

```

```

app.post("/api/code-checker/email", async (req, res) => {
  try {
    const { code, language, analysis } = req.body;
    if (!analysis) return res.status(400).json({ error: "analysis required" });

    const { getGmailAccessToken } = await import("./gmail");
    const accessToken = await getGmailAccessToken();

    const hostname = process.env.REPLIT_CONNECTORS_HOSTNAME;
    const xReplitToken = process.env.REPL_IDENTITY
      ? 'repl ' + process.env.REPL_IDENTITY
      : process.env.WEB_REPL_RENEWAL
      ? 'depl ' + process.env.WEB_REPL_RENEWAL
      : null;

    let outlookEmail = "";
    if (xReplitToken && hostname) {
      const outlookRes = await fetch(

```

```

    'https://' + hostname + '/api/v2/connection?include_secrets=true&connector_names=outlook',
    { headers: { 'Accept': 'application/json', 'X-Replit-Token': xReplitToken } }
  );
  const outlookData = await outlookRes.json();
  const outlookConn = outlookData.items?.[0];
  const outlookToken = outlookConn?.settings?.access_token;
  if (outlookToken) {
    const profileRes = await fetch('https://graph.microsoft.com/v1.0/me', {
      headers: { 'Authorization': 'Bearer ' + outlookToken }
    });
    const profile = await profileRes.json();
    outlookEmail = profile.mail || profile.userPrincipalName || "";
  }
}

if (!outlookEmail) return res.status(500).json({ error: "Could not get Outlook email" });

const subject = `Code Review: ${language} - ${new Date().toLocaleDateString()}`;
const emailBody = `CODE REVIEW RESULTS\n${"=".repeat(40)}\nLanguage: ${language}\nDate: ${new Date().toLocaleSt...

const boundary = 'boundary_' + Date.now();
const rawEmail = [
  'Content-Type: text/plain; charset="UTF-8"',
  'MIME-Version: 1.0',
  'To: ${outlookEmail}',
  'From: homeworkbryn@gmail.com',
  'Subject: ${subject}',
  '',
  emailBody,
].join('\r\n');

const encodedMessage = Buffer.from(rawEmail)
  .toString('base64')
  .replace(/\+/g, '-')
  .replace(/\//g, '_')
  .replace(/=+$/, '');

const sendRes = await fetch('https://gmail.googleapis.com/gmail/v1/users/me/messages/send', {
  method: 'POST',
  headers: { 'Authorization': 'Bearer ' + accessToken, 'Content-Type': 'application/json' },
  body: JSON.stringify({ raw: encodedMessage })
});

const sendResult = await sendRes.json();
if (sendResult.error) throw new Error(JSON.stringify(sendResult.error));

console.log(`[CodeChecker] Email sent to ${outlookEmail}, ID: ${sendResult.id}`);
res.json({ success: true, messageId: sendResult.id, sentTo: outlookEmail });
} catch (error: any) {
  console.error("[CodeChecker] Email error:", error);
  res.status(500).json({ error: error.message || "Email failed" });
}
});

return httpServer;
}

function generateICS(title: string, description: string, dueDate: Date, type: string, reminderMinutes?: number[]): string {
  const formatDate = (d: Date) => d.toISOString().replace(/[-:]/g, '').split('.')[0] + 'Z';
  const uid = `task-${Date.now()}@schoolplanner`;

  const activeReminders = reminderMinutes?.filter(m => m > 0) || [DEFAULT_REMINDER_1, DEFAULT_REMINDER_2];

  const reminders = activeReminders.map(minutes =>
    `VALARM\r\nACTION:DISPLAY\r\nDESCRIPTION:Reminder\r\nTRIGGER:-PT${minutes}M\r\nEND:VALARM`
  ).join('\r\nBEGIN:');

  const endDate = new Date(dueDate.getTime() + 60 * 60 * 1000);

  return `BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//School Task Planner//EN
BEGIN:VEVENT
UID:${uid}
DTSTAMP:${formatDate(new Date())}
DTSTART:${formatDate(dueDate)}
DTEND:${formatDate(endDate)}
SUMMARY:${title} [${type.toUpperCase()}]
DESCRIPTION:${description.replace(/\n/g, '\\n')}
BEGIN:${reminders}
END:VEVENT
END:VCALENDAR`;
}

async function seedDatabase() {
  const existingTasks = await storage.getTasks();
  if (existingTasks.length === 0) {
    console.log("Seeding database with sample tasks...");

    const sampleTasks = [
      {
        title: "Read Chapter 5: Data Structures",
        description: "Complete reading and take notes on arrays and linked lists",
        type: "reading",
        courseName: "CS 201",
        dueDate: new Date("2026-01-19T23:59:00"),
        weekNumber: 3,
        priority: "high",
        repeatType: "none" as const,
      },
      {
        title: "Module 2: Introduction to Algorithms",
        description: "Complete all video lectures and quizzes",
        type: "module",
        courseName: "CS 201",
        dueDate: new Date("2026-01-20T23:59:00"),
        weekNumber: 3,
        priority: "medium",
      },
    ];
  }
}

```



```

    repeatType: "none" as const,
  },
  {
    title: "Essay: Impact of AI on Society",
    description: "2000 words minimum, APA format",
    type: "essay",
    courseName: "ENG 101",
    dueDate: new Date("2026-01-24T17:00:00"),
    weekNumber: 3,
    priority: "high",
    repeatType: "none" as const,
  },
  {
    title: "Group Project: Database Design",
    description: "Submit ER diagram and schema documentation",
    type: "project",
    courseName: "CS 301",
    dueDate: new Date("2026-01-31T23:59:00"),
    weekNumber: 4,
    priority: "high",
    repeatType: "none" as const,
  },
  {
    title: "Discussion: Ethics in Technology",
    description: "Post initial response and reply to 2 classmates",
    type: "discussion",
    courseName: "PHIL 200",
    dueDate: new Date("2026-01-22T23:59:00"),
    weekNumber: 3,
    priority: "medium",
    repeatType: "none" as const,
  },
  {
    title: "Weekly Poll: Study Habits",
    description: "Complete the class survey",
    type: "poll",
    courseName: "PSY 101",
    dueDate: new Date("2026-01-17T18:00:00"),
    weekNumber: 2,
    priority: "low",
    repeatType: "none" as const,
  },
  {
    title: "Midterm Exam: Computer Networks",
    description: "Covers chapters 1-6, bring calculator",
    type: "exam",
    courseName: "CS 401",
    dueDate: new Date("2026-02-14T10:00:00"),
    weekNumber: 6,
    priority: "high",
    repeatType: "none" as const,
  },
  {
    title: "Quiz: SQL Basics",
    description: "Online quiz, 30 minutes, open book",
    type: "quiz",
    courseName: "CS 301",
    dueDate: new Date("2026-01-18T14:00:00"),
    weekNumber: 2,
    priority: "medium",
    repeatType: "none" as const,
  },
];

for (const task of sampleTasks) {
  await storage.createTask(task);
}
console.log("Seeding complete.");
}

// Seed file records if they don't exist (for production sync)
const existingFiles = await storage.getFiles();
if (existingFiles.length === 0) {
  console.log("Seeding database with file records...");

  const fileRecords = [
    {
      originalName: "CPPA122, Module 1 - Welcome.pdf",
      displayName: "CPPA122, Module 1 - Welcome.pdf",
      objectPath: "/objects/uploads/8dbf73fc-9cc2-45d3-bfa9-166b64d9b598",
      contentType: "application/pdf",
      size: 380949,
      folder: "week-1-cppa122-module",
      listened: false,
    },
    {
      originalName: "A Citizen's Guide to Government.pdf",
      displayName: "A Citizen's Guide to Government.pdf",
      objectPath: "/objects/uploads/92c68aff-ae80-438e-8ece-c33dd1647630",
      contentType: "application/pdf",
      size: 617534,
      folder: "week-1-cppa122-reading",
      listened: false,
    },
    {
      originalName: "CPPA122, Module 2 - Introduction.pdf",
      displayName: "CPPA122, Module 2 - Introduction.pdf",
      objectPath: "/objects/uploads/d2596e74-c454-4d29-9b77-e0a14e060957",
      contentType: "application/pdf",
      size: 292550,
      folder: "week-2-cppa122-module",
      listened: false,
    },
    {
      originalName: "156CBA1A.pdf",
      displayName: "156CBA1A.pdf",
      objectPath: "/objects/uploads/b1e539de-a397-454e-8912-e97df6f4feaf",
      contentType: "application/pdf",
    }
  ];
}

```

```

        size: 1016007,
        folder: "week-2-cppa122-reading",
        listened: false,
      },
      {
        originalName: "156C823F.pdf",
        displayName: "156C823F.pdf",
        objectPath: "/objects/uploads/35d6e9ef-f8fc-46dc-88b7-0d441307862d",
        contentType: "application/pdf",
        size: 1348962,
        folder: "week-2-cppa122-reading",
        listened: false,
      },
      {
        originalName: "44320906-Supplementary.pdf",
        displayName: "44320906-Supplementary.pdf",
        objectPath: "/objects/uploads/7fbb3260-4e04-4578-8a79-207c29f64fa8",
        contentType: "application/pdf",
        size: 630516,
        folder: "week-3-cppa122-reading",
        listened: false,
      },
      {
        originalName: "AM.OrgCdnLocGovt.Spicer.pdf",
        displayName: "AM.OrgCdnLocGovt.Spicer.pdf",
        objectPath: "/objects/uploads/b12aa8b5-4ca0-4b01-9b07-7b7b919157e1",
        contentType: "application/pdf",
        size: 414847,
        folder: "week-3-cppa122-reading",
        listened: false,
      },
      {
        originalName: "IMFG_Paper_No47_Power_and_Purpose_Taylor_Dobson.pdf",
        displayName: "IMFG_Paper_No47_Power_and_Purpose_Taylor_Dobson.pdf",
        objectPath: "/objects/uploads/eb371306-ee4d-4920-b911-160bd535d8e1",
        contentType: "application/pdf",
        size: 1177012,
        folder: "week-3-cppa122-reading",
        listened: false,
      },
      {
        originalName: "CPPA122, Module 3 - Introduction.pdf",
        displayName: "CPPA122, Module 3 - Introduction.pdf",
        objectPath: "/objects/uploads/fb73649f-402b-4dbe-834e-8b165e2c0535",
        contentType: "application/pdf",
        size: 385275,
        folder: "week-3-cppa122-module",
        listened: false,
      },
    ],
  };

  for (const file of fileRecords) {
    try {
      await storage.createFile(file);
    } catch (err) {
      // Ignore duplicate key errors (file already exists)
      console.log(`File ${file.displayName} already exists or error:`, err);
    }
  }
  console.log("File seeding complete.");
}
}

```

`server/secondGoogleAccount.ts`

```

// Second Google Account OAuth Integration
import { google } from 'googleapis';
import { storage } from './storage';
import type { SecondGoogleAccount } from '@shared/schema';

const SCOPES = [
  'https://www.googleapis.com/auth/calendar',
  'https://www.googleapis.com/auth/calendar.events',
  'https://www.googleapis.com/auth/userinfo.email',
];

function getOAuth2Client() {
  const clientId = process.env.GOOGLE_SECOND_ACCOUNT_CLIENT_ID;
  const clientSecret = process.env.GOOGLE_SECOND_ACCOUNT_CLIENT_SECRET;

  if (!clientId || !clientSecret) {
    throw new Error('Google OAuth credentials not configured for second account');
  }

  // Determine the correct redirect URI based on environment
  const domain = process.env.REPLIT_DEV_DOMAIN || process.env.REPLIT_DOMAINS?.split(',')[0];
  const redirectUri = domain
    ? `https://${domain}/api/google/second-account/callback`
    : 'http://localhost:5000/api/google/second-account/callback';

  return new google.auth.OAuth2(clientId, clientSecret, redirectUri);
}

// Generate OAuth URL for second account login
export function getSecondAccountAuthUrl(): string {
  const oauth2Client = getOAuth2Client();

  return oauth2Client.generateAuthUrl({
    access_type: 'offline',
    scope: SCOPES,
    prompt: 'consent', // Force consent to get refresh token
  });
}

```

```

// Exchange authorization code for tokens
export async function exchangeCodeForTokens(code: string): Promise<SecondGoogleAccount> {
  const oauth2Client = getOAuth2Client();

  const { tokens } = await oauth2Client.getToken(code);

  if (!tokens.access_token || !tokens.refresh_token) {
    throw new Error('Failed to get required tokens from Google');
  }

  oauth2Client.setCredentials(tokens);

  // Get user email
  const oauth2 = google.oauth2({ version: 'v2', auth: oauth2Client });
  const userInfo = await oauth2.userinfo.get();
  const email = userInfo.data.email || 'unknown';

  // Calculate expiry time
  const expiresAt = tokens.expiry_date
    ? new Date(tokens.expiry_date)
    : new Date(Date.now() + 3600 * 1000); // Default 1 hour

  // Save to database
  const account = await storage.saveSecondGoogleAccount({
    email,
    accessToken: tokens.access_token,
    refreshToken: tokens.refresh_token,
    expiresAt,
  });

  return account;
}

// Get a valid access token, refreshing if needed
async function getValidAccessToken(account: SecondGoogleAccount): Promise<string> {
  // Check if token is still valid (with 5 min buffer)
  const now = new Date();
  const expiresAt = new Date(account.expiresAt);
  const bufferMs = 5 * 60 * 1000; // 5 minutes

  if (expiresAt.getTime() - now.getTime() > bufferMs) {
    return account.accessToken;
  }

  // Token expired or expiring soon, refresh it
  const oauth2Client = getOAuth2Client();
  oauth2Client.setCredentials({
    refresh_token: account.refreshToken,
  });

  const { credentials } = await oauth2Client.refreshAccessToken();

  if (!credentials.access_token) {
    throw new Error('Failed to refresh access token');
  }

  // Update stored token
  const newExpiresAt = credentials.expiry_date
    ? new Date(credentials.expiry_date)
    : new Date(Date.now() + 3600 * 1000);

  await storage.updateSecondGoogleAccount(account.id, {
    accessToken: credentials.access_token,
    expiresAt: newExpiresAt,
  });

  return credentials.access_token;
}

// Get Google Calendar client for second account
export async function getSecondAccountCalendarClient() {
  const account = await storage.getSecondGoogleAccount();

  if (!account) {
    throw new Error('Second Google account not connected');
  }

  const accessToken = await getValidAccessToken(account);

  const oauth2Client = new google.auth.OAuth2();
  oauth2Client.setCredentials({ access_token: accessToken });

  return google.calendar({ version: 'v3', auth: oauth2Client });
}

// Check if second account is connected
export async function isSecondAccountConnected(): Promise<{ connected: boolean; email?: string }> {
  try {
    const account = await storage.getSecondGoogleAccount();
    if (account) {
      return { connected: true, email: account.email };
    }
    return { connected: false };
  } catch {
    return { connected: false };
  }
}

// Disconnect second account
export async function disconnectSecondAccount(): Promise<void> {
  await storage.deleteSecondGoogleAccount();
}

// Create event in second account's primary calendar
export async function createEventInSecondAccount(task: {
  id: number;
  title: string;
  description?: string | null;
}) {

```

```

    dueDate: Date | string;
    courseName?: string | null;
  }) {
    const calendar = await getSecondAccountCalendarClient();

    const dueDate = new Date(task.dueDate);
    const hour = dueDate.getHours();
    const isAllDay = hour === 0 || hour === 23;

    const summary = `${task.courseName ? `${task.courseName}` : ''}${task.title}`;

    let event: any;

    if (isAllDay) {
      const dateStr = dueDate.toISOString().split('T')[0];
      const nextDay = new Date(dueDate);
      nextDay.setDate(nextDay.getDate() + 1);
      const endDateStr = nextDay.toISOString().split('T')[0];

      event = {
        summary,
        description: task.description || '',
        start: { date: dateStr },
        end: { date: endDateStr },
        reminders: { useDefault: false, overrides: [] },
      };
    } else {
      const startTime = dueDate.toISOString();
      const endTime = new Date(dueDate.getTime() + 60 * 60 * 1000).toISOString();

      event = {
        summary,
        description: task.description || '',
        start: { dateTime: startTime, timeZone: 'America/Toronto' },
        end: { dateTime: endTime, timeZone: 'America/Toronto' },
        reminders: {
          useDefault: false,
          overrides: [
            { method: 'popup', minutes: 60 * 24 * 2 },
            { method: 'popup', minutes: 60 * 24 },
            { method: 'popup', minutes: 120 },
            { method: 'popup', minutes: 30 },
          ],
        },
      };
    }

    const response = await calendar.events.insert({
      calendarId: 'primary',
      requestBody: event,
    });

    return response.data;
  }

  // Create prep event in second account
  export async function createPrepEventInSecondAccount(task: {
    id: number;
    title: string;
    description?: string | null;
    startDate: Date | string;
    dueDate: Date | string;
    courseName?: string | null;
  }) {
    const calendar = await getSecondAccountCalendarClient();

    const startDate = new Date(task.startDate);
    const dueDate = new Date(task.dueDate);
    const hour = startDate.getHours();
    const isAllDay = hour === 0 || hour === 23;

    const summary = `[PREP] ${task.courseName ? `${task.courseName}` : ''}${task.title}`;
    const description = `Start preparing for: ${task.title}\nDue: ${dueDate.toLocaleDateString()}\n\n${task.description...}`;

    let event: any;

    if (isAllDay) {
      const dateStr = startDate.toISOString().split('T')[0];
      const nextDay = new Date(startDate);
      nextDay.setDate(nextDay.getDate() + 1);
      const endDateStr = nextDay.toISOString().split('T')[0];

      event = {
        summary,
        description,
        start: { date: dateStr },
        end: { date: endDateStr },
        reminders: { useDefault: false, overrides: [] },
      };
    } else {
      const startTime = startDate.toISOString();
      const endTime = new Date(startDate.getTime() + 60 * 60 * 1000).toISOString();

      event = {
        summary,
        description,
        start: { dateTime: startTime, timeZone: 'America/Toronto' },
        end: { dateTime: endTime, timeZone: 'America/Toronto' },
        reminders: {
          useDefault: false,
          overrides: [
            { method: 'popup', minutes: 60 * 24 },
            { method: 'popup', minutes: 120 },
            { method: 'popup', minutes: 30 },
          ],
        },
      };
    }
  }
}

```

```

const response = await calendar.events.insert({
  calendarId: 'primary',
  requestBody: event,
});

return response.data;
}

// Delete event from second account
export async function deleteEventFromSecondAccount(eventId: string): Promise<void> {
  const calendar = await getSecondAccountCalendarClient();

  await calendar.events.delete({
    calendarId: 'primary',
    eventId: eventId,
  });
}

// Update event in second account
export async function updateEventInSecondAccount(eventId: string, task: {
  title: string;
  description?: string | null;
  dueDate: Date | string;
  courseName?: string | null;
}) {
  const calendar = await getSecondAccountCalendarClient();

  const dueDate = new Date(task.dueDate);
  const hour = dueDate.getHours();
  const isAllDay = hour === 0 || hour === 23;

  const summary = `${task.courseName ? `[${task.courseName}]` : ''} ${task.title}`;

  let event: any;

  if (isAllDay) {
    const dateStr = dueDate.toISOString().split('T')[0];
    const nextDay = new Date(dueDate);
    nextDay.setDate(nextDay.getDate() + 1);
    const endDateStr = nextDay.toISOString().split('T')[0];

    event = {
      summary,
      description: task.description || '',
      start: { date: dateStr },
      end: { date: endDateStr },
    };
  } else {
    const startTime = dueDate.toISOString();
    const endTime = new Date(dueDate.getTime() + 60 * 60 * 1000).toISOString();

    event = {
      summary,
      description: task.description || '',
      start: { dateTime: startTime, timeZone: 'America/Toronto' },
      end: { dateTime: endTime, timeZone: 'America/Toronto' },
    };
  }

  const response = await calendar.events.update({
    calendarId: 'primary',
    eventId: eventId,
    requestBody: event,
  });

  return response.data;
}

// Get events from second account (for conflict detection)
export async function getEventsFromSecondAccount(timeMin: Date, timeMax: Date) {
  try {
    const calendar = await getSecondAccountCalendarClient();

    const response = await calendar.events.list({
      calendarId: 'primary',
      timeMin: timeMin.toISOString(),
      timeMax: timeMax.toISOString(),
      singleEvents: true,
      orderBy: 'startTime',
    });

    return response.data.items || [];
  } catch (error) {
    console.error('Error fetching events from second account:', error);
    return [];
  }
}

```

`server/spotify.ts`

```

// Spotify integration using user's own Spotify app with full OAuth flow
import fs from 'fs';
import path from 'path';

const TOKEN_FILE = path.join(process.cwd(), '.spotify-token.json');

interface SpotifyTokenData {
  accessToken: string;
  refreshToken: string;
  expiresAt: number;
}

function loadTokenFromDisk(): SpotifyTokenData | null {

```

```

try {
  if (fs.existsSync(TOKEN_FILE)) {
    const data = JSON.parse(fs.readFileSync(TOKEN_FILE, 'utf-8'));
    return data;
  } catch {}
  return null;
}

function saveTokenToDisk(token: SpotifyTokenData) {
  fs.writeFileSync(TOKEN_FILE, JSON.stringify(token, null, 2));
}

let cachedToken: SpotifyTokenData | null = loadTokenFromDisk();

export function getAuthUrl(): string {
  const clientId = process.env.SPOTIFY_CLIENT_ID;
  if (!clientId) throw new Error('SPOTIFY_CLIENT_ID not set');

  const host = process.env.REPLIT_DOMAINS?.split(',')[0] || process.env.REPL_SLUG + '.' + process.env.REPL_OWNER + '....';
  const redirectUri = `https://${host}/api/spotify/callback`;

  const scopes = [
    'user-read-playback-state',
    'user-modify-playback-state',
    'user-read-currently-playing',
    'user-read-recently-played',
    'user-read-email',
    'user-read-private',
    'user-library-read',
    'user-top-read',
    'streaming',
    'playlist-read-private',
    'playlist-read-collaborative',
  ].join(' ');

  const params = new URLSearchParams({
    response_type: 'code',
    client_id: clientId,
    scope: scopes,
    redirect_uri: redirectUri,
    show_dialog: 'true',
  });

  return `https://accounts.spotify.com/authorize?${params.toString()}`;
}

export async function handleCallback(code: string): Promise<void> {
  const clientId = process.env.SPOTIFY_CLIENT_ID;
  const clientSecret = process.env.SPOTIFY_CLIENT_SECRET;
  if (!clientId || !clientSecret) throw new Error('Spotify credentials not configured');

  const host = process.env.REPLIT_DOMAINS?.split(',')[0] || process.env.REPL_SLUG + '.' + process.env.REPL_OWNER + '....';
  const redirectUri = `https://${host}/api/spotify/callback`;

  const basicAuth = Buffer.from(`${clientId}:${clientSecret}`).toString('base64');
  const res = await fetch(`https://accounts.spotify.com/api/token`, {
    method: 'POST',
    headers: {
      'Authorization': `Basic ${basicAuth}`,
      'Content-Type': 'application/x-www-form-urlencoded',
    },
    body: new URLSearchParams({
      grant_type: 'authorization_code',
      code,
      redirect_uri: redirectUri,
    }).toString(),
  });

  if (!res.ok) {
    const errText = await res.text();
    throw new Error(`Token exchange failed: ${res.status} ${errText}`);
  }

  const data = await res.json();
  cachedToken = {
    accessToken: data.access_token,
    refreshToken: data.refresh_token,
    expiresAt: Date.now() + (data.expires_in - 60) * 1000,
  };
  saveTokenToDisk(cachedToken);
}

async function refreshAccessToken(): Promise<string> {
  if (!cachedToken?.refreshToken) throw new Error('No refresh token - connect Spotify first');

  const clientId = process.env.SPOTIFY_CLIENT_ID;
  const clientSecret = process.env.SPOTIFY_CLIENT_SECRET;
  if (!clientId || !clientSecret) throw new Error('Spotify credentials not configured');

  const basicAuth = Buffer.from(`${clientId}:${clientSecret}`).toString('base64');
  const res = await fetch(`https://accounts.spotify.com/api/token`, {
    method: 'POST',
    headers: {
      'Authorization': `Basic ${basicAuth}`,
      'Content-Type': 'application/x-www-form-urlencoded',
    },
    body: new URLSearchParams({
      grant_type: 'refresh_token',
      refresh_token: cachedToken.refreshToken,
    }).toString(),
  });

  if (!res.ok) {
    const errText = await res.text();
    cachedToken = null;
    try { fs.unlinkSync(TOKEN_FILE); } catch {}
    throw new Error(`Token refresh failed: ${res.status} ${errText}`);
  }

```

```

}

const data = await res.json();
cachedToken = {
  accessToken: data.access_token,
  refreshToken: data.refresh_token || cachedToken.refreshToken,
  expiresAt: Date.now() + (data.expires_in - 60) * 1000,
};
saveTokenToDisk(cachedToken);
return cachedToken.accessToken;
}

async function getSpotifyAccessToken(): Promise<string> {
  if (cachedToken && Date.now() < cachedToken.expiresAt) {
    return cachedToken.accessToken;
  }

  if (cachedToken?.refreshToken) {
    return refreshAccessToken();
  }

  throw new Error('Spotify not connected - visit /api/spotify/login to connect');
}

export function isConnected(): boolean {
  return !!cachedToken?.refreshToken;
}

async function spotifyFetch(endpoint: string, method: string = 'GET', body?: any) {
  const token = await getSpotifyAccessToken();
  const res = await fetch(`https://api.spotify.com/v1${endpoint}`, {
    method,
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json',
    },
    body: body ? JSON.stringify(body) : undefined,
  });
  if (res.status === 204) return null;
  if (res.status === 401) {
    cachedToken = null;
    throw new Error('Spotify token expired - reconnect at /api/spotify/login');
  }
  if (!res.ok) {
    const errText = await res.text();
    throw new Error(`Spotify API ${res.status}: ${errText}`);
  }
  return res.json();
}

export async function getNowPlaying() {
  return spotifyFetch('/me/player/currently-playing');
}

export async function getRecentTracks(limit: number = 5) {
  return spotifyFetch(`/me/player/recently-played?limit=${limit}`);
}

export async function play() {
  return spotifyFetch('/me/player/play', 'PUT');
}

export async function pause() {
  return spotifyFetch('/me/player/pause', 'PUT');
}

export async function next() {
  return spotifyFetch('/me/player/next', 'POST');
}

export async function previous() {
  return spotifyFetch('/me/player/previous', 'POST');
}

export async function getPlaylists(limit: number = 50) {
  return spotifyFetch(`/me/playlists?limit=${limit}`);
}

export async function getSavedAlbums(limit: number = 50) {
  return spotifyFetch(`/me/albums?limit=${limit}`);
}

export async function getTopArtists(limit: number = 50) {
  return spotifyFetch(`/me/top/artists?limit=${limit}&time_range=medium_term`);
}

export async function getSavedTracks(limit: number = 50) {
  return spotifyFetch(`/me/tracks?limit=${limit}`);
}

export async function setVolume(volumePercent: number) {
  return spotifyFetch(`/me/player/volume?volume_percent=${Math.round(volumePercent)}`, 'PUT');
}

export async function getDevices() {
  return spotifyFetch('/me/player/devices');
}

export async function playContext(contextUri: string, offset?: number) {
  const body: any = { context_uri: contextUri };
  if (offset !== undefined) body.offset = { position: offset };
  return spotifyFetch('/me/player/play', 'PUT', body);
}

export async function playTracks(uris: string[]) {
  return spotifyFetch('/me/player/play', 'PUT', { uris });
}

```

```

export async function setShuffle(state: boolean) {
  return spotifyFetch(`/me/player/shuffle?state=${state}`, 'PUT');
}

export async function setRepeat(state: string) {
  return spotifyFetch(`/me/player/repeat?state=${state}`, 'PUT');
}

export async function getPlaybackState() {
  return spotifyFetch(`/me/player`);
}

export async function search(query: string, types: string = 'track,artist,album,playlist', limit: number = 20) {
  return spotifyFetch(`/search?q=${encodeURIComponent(query)}&type=${types}&limit=${limit}`);
}

export async function transferPlayback(deviceId: string, play: boolean = true) {
  return spotifyFetch(`/me/player`, 'PUT', { device_ids: [deviceId], play });
}

export async function getRelatedArtists(artistId: string) {
  return spotifyFetch(`/artists/${artistId}/related-artists`);
}

export async function getArtistTopTracks(artistId: string, market: string = 'US') {
  return spotifyFetch(`/artists/${artistId}/top-tracks?market=${market}`);
}

export async function getArtistById(artistId: string) {
  return spotifyFetch(`/artists/${artistId}`);
}

export async function getTrackById(trackId: string) {
  return spotifyFetch(`/tracks/${trackId}`);
}

export async function getPlaylistById(playlistId: string) {
  return spotifyFetch(`/playlists/${playlistId}?fields=id,name,images`);
}

```

`server/static.ts`

```

import express, { type Express } from "express";
import fs from "fs";
import path from "path";

export function serveStatic(app: Express) {
  const distPath = path.resolve(__dirname, "public");
  if (!fs.existsSync(distPath)) {
    throw new Error(
      `Could not find the build directory: ${distPath}, make sure to build the client first`,
    );
  }

  // Serve static assets with cache-busting headers
  app.use(express.static(distPath, {
    etag: false,
    maxAge: 0,
    setHeaders: (res, filePath) => {
      // No caching for HTML files
      if (filePath.endsWith('.html')) {
        res.setHeader('Cache-Control', 'no-cache, no-store, must-revalidate');
        res.setHeader('Pragma', 'no-cache');
        res.setHeader('Expires', '0');
      } else if (filePath.match(/\.js|css$/)) {
        // Short cache for JS/CSS (they have hashes in filenames)
        res.setHeader('Cache-Control', 'public, max-age=60');
      }
    }
  }));

  // fall through to index.html if the file doesn't exist
  app.use("/*path", (_req, res) => {
    res.setHeader('Cache-Control', 'no-cache, no-store, must-revalidate');
    res.setHeader('Pragma', 'no-cache');
    res.setHeader('Expires', '0');
    res.sendFile(path.resolve(distPath, "index.html"));
  });
}

```

`server/storage.ts`

```

import { db } from "../db";
import { tasks, files, semesterSettings, secondGoogleAccount, thirdGoogleAccount, deletedFolders, customFolders, subtasks } from "../data";
import { eq, and, gte, lte, desc, or, isNull } from "drizzle-orm";

export interface IStorage {
  getTasks(filters?: { weekNumber?: number; type?: string; showCompleted?: boolean }): Promise<Task[]>;
  getTask(id: number): Promise<Task | undefined>;
  createTask(task: InsertTask): Promise<Task>;
  updateTask(id: number, updates: UpdateTaskRequest): Promise<Task>;
  deleteTask(id: number): Promise<void>;
  deleteChildTasks(parentTaskId: number): Promise<void>;
  getChildTasks(parentTaskId: number): Promise<Task[]>;
  getTaskCountByWeek(): Promise<Record<number, number>>;
  getFiles(): Promise<FileRecord[]>;
  getFile(id: number): Promise<FileRecord | undefined>;
  getFileByPath(objectPath: string): Promise<FileRecord | undefined>;
  createFile(file: InsertFile): Promise<FileRecord>;
  updateFile(id: number, updates: { displayName?: string; folder?: string | null; listened?: boolean; lastChunkIndex?..

```



```

deleteFile(id: number): Promise<void>;
getActiveSemesterSettings(): Promise<SemesterSettings | undefined>;
getAllSemesterSettings(): Promise<SemesterSettings[]>;
createSemesterSettings(settings: InsertSemesterSettings): Promise<SemesterSettings>;
updateSemesterSettings(id: number, updates: Partial<SemesterSettings>): Promise<SemesterSettings>;
getSecondGoogleAccount(): Promise<SecondGoogleAccount | undefined>;
saveSecondGoogleAccount(account: InsertSecondGoogleAccount): Promise<SecondGoogleAccount>;
updateSecondGoogleAccount(id: number, updates: Partial<SecondGoogleAccount>): Promise<SecondGoogleAccount>;
deleteSecondGoogleAccount(): Promise<void>;
getThirdGoogleAccount(): Promise<ThirdGoogleAccount | undefined>;
saveThirdGoogleAccount(account: InsertThirdGoogleAccount): Promise<ThirdGoogleAccount>;
updateThirdGoogleAccount(id: number, updates: Partial<ThirdGoogleAccount>): Promise<ThirdGoogleAccount>;
deleteThirdGoogleAccount(): Promise<void>;
getDeletedFolders(): Promise<DeletedFolder[]>;
addDeletedFolder(folderId: string): Promise<DeletedFolder>;
removeDeletedFolder(folderId: string): Promise<void>;
getCustomFolders(): Promise<CustomFolder[]>;
createCustomFolder(folder: InsertCustomFolder): Promise<CustomFolder>;
updateCustomFolder(id: number, name: string): Promise<CustomFolder>;
deleteCustomFolder(id: number): Promise<void>;
// Subtasks
getSubtasksByTask(taskId: number): Promise<Subtask[]>;
getSubtask(id: number): Promise<Subtask | undefined>;
createSubtask(subtask: InsertSubtask): Promise<Subtask>;
updateSubtask(id: number, updates: Partial<InsertSubtask>): Promise<Subtask>;
deleteSubtask(id: number): Promise<void>;
deleteSubtasksByTask(taskId: number): Promise<void>;
// Task Links
getAllTaskLinks(): Promise<TaskLink[]>;
getLinksForTask(taskId: number): Promise<TaskLink[]>;
getLinksForSubtask(subtaskId: number): Promise<TaskLink[]>;
createTaskLink(link: InsertTaskLink): Promise<TaskLink>;
deleteTaskLink(id: number): Promise<void>;
// Projects
getProjects(): Promise<Project[]>;
getProject(id: number): Promise<Project | undefined>;
createProject(project: InsertProject): Promise<Project>;
updateProject(id: number, updates: Partial<InsertProject>): Promise<Project>;
deleteProject(id: number): Promise<void>;
getTasksByProject(projectId: number): Promise<Task[]>;
// Sticky Notes
getStickyNotes(): Promise<StickyNote[]>;
getDeletedStickyNotes(): Promise<StickyNote[]>;
getStickyNote(id: number): Promise<StickyNote | undefined>;
createStickyNote(note: InsertStickyNote): Promise<StickyNote>;
updateStickyNote(id: number, updates: Partial<InsertStickyNote>): Promise<StickyNote>;
deleteStickyNote(id: number): Promise<void>;
restoreStickyNote(id: number): Promise<StickyNote>;
// Access Tokens
getAccessTokens(): Promise<AccessToken[]>;
getAccessToken(token: string): Promise<AccessToken | undefined>;
createAccessToken(token: InsertAccessToken): Promise<AccessToken>;
updateAccessToken(id: number, updates: Partial<InsertAccessToken>): Promise<AccessToken>;
deleteAccessToken(id: number): Promise<void>;
getShiftSchedule(): Promise<ShiftScheduleEntry[]>;
getShiftForDate(date: string): Promise<ShiftScheduleEntry | undefined>;
setShiftDay(date: string, shiftType: string): Promise<ShiftScheduleEntry>;
setShiftBulk(entries: InsertShiftScheduleEntry[]): Promise<void>;
deleteShiftDay(date: string): Promise<void>;
clearAllShifts(): Promise<void>;
getSemesterChecklist(semesterSettingsId: number): Promise<SemesterCheckListItem[]>;
upsertSemesterCheckListItem(item: InsertSemesterCheckListItem): Promise<SemesterCheckListItem>;
initSemesterChecklist(semesterSettingsId: number, courseCodes: string[]): Promise<SemesterCheckListItem[]>;
getCourseWeekMappings(courseCode: string, semesterSettingsId: number): Promise<CourseWeekMapping[]>;
getAllCourseWeekMappings(semesterSettingsId: number): Promise<CourseWeekMapping[]>;
upsertCourseWeekMapping(data: InsertCourseWeekMapping): Promise<CourseWeekMapping>;
getScholarships(): Promise<Scholarship[]>;
getScholarship(id: number): Promise<Scholarship | undefined>;
createScholarship(data: InsertScholarship): Promise<Scholarship>;
updateScholarship(id: number, updates: Partial<InsertScholarship>): Promise<Scholarship>;
deleteScholarship(id: number): Promise<void>;
getKeyContacts(): Promise<KeyContact[]>;
getKeyContact(id: number): Promise<KeyContact | undefined>;
createKeyContact(data: InsertKeyContact): Promise<KeyContact>;
updateKeyContact(id: number, updates: Partial<InsertKeyContact>): Promise<KeyContact>;
deleteKeyContact(id: number): Promise<void>;
getAnnouncements(): Promise<Announcement[]>;
createAnnouncement(data: InsertAnnouncement): Promise<Announcement>;
getAnnouncementByEmailId(emailId: string): Promise<Announcement | undefined>;
deleteAnnouncement(id: number): Promise<void>;
getEntityComments(entityType: string, entityId: string): Promise<EntityComment[]>;
upsertEntityComment(entityType: string, entityId: string, content: string): Promise<EntityComment>;
deleteEntityComment(id: number): Promise<void>;
getPendingReviewItems(status?: string): Promise<PendingReviewItem[]>;
getPendingReviewItemByExternalId(externalId: string, source: string): Promise<PendingReviewItem | undefined>;
createPendingReviewItem(item: InsertPendingReviewItem): Promise<PendingReviewItem>;
updatePendingReviewItem(id: number, updates: Partial<InsertPendingReviewItem>): Promise<PendingReviewItem>;
deletePendingReviewItem(id: number): Promise<void>;
}

export class DatabaseStorage implements IStorage {
  async getTasks(filters?: { weekNumber?: number; type?: string; showCompleted?: boolean }): Promise<Task[]> {
    let query = db.select().from(tasks);

    const conditions = [];
    if (filters?.weekNumber) {
      // Include tasks from previous, selected, and following week (for calendar visibility)
      conditions.push(
        eq(tasks.weekNumber, filters.weekNumber - 1),
        eq(tasks.weekNumber, filters.weekNumber),
        eq(tasks.weekNumber, filters.weekNumber + 1)
      );
    }
  }
}

```

```

    if (filters?.type) {
      conditions.push(eq(tasks.type, filters.type));
    }
    if (filters?.showCompleted === false) {
      conditions.push(eq(tasks.isCompleted, false));
    }

    if (conditions.length > 0) {
      return await db.select().from(tasks).where(and(...conditions)).orderBy(tasks.dueDate);
    }

    return await db.select().from(tasks).orderBy(tasks.dueDate);
  }

  async getTask(id: number): Promise<Task | undefined> {
    const [task] = await db.select().from(tasks).where(eq(tasks.id, id));
    return task;
  }

  async createTask(insertTask: InsertTask): Promise<Task> {
    const [task] = await db.insert(tasks).values(insertTask).returning();
    return task;
  }

  async updateTask(id: number, updates: UpdateTaskRequest): Promise<Task> {
    // If marking as completed, set completedAt timestamp
    // If marking as not completed, clear completedAt
    const modifiedUpdates = { ...updates } as typeof updates & { completedAt?: Date | null };
    if ('isCompleted' in updates) {
      if (updates.isCompleted === true) {
        modifiedUpdates.completedAt = new Date();
      } else if (updates.isCompleted === false) {
        modifiedUpdates.completedAt = null;
      }
    }

    const [updated] = await db
      .update(tasks)
      .set(modifiedUpdates)
      .where(eq(tasks.id, id))
      .returning();
    return updated;
  }

  async deleteTask(id: number): Promise<void> {
    await db.delete(tasks).where(eq(tasks.id, id));
  }

  async deleteChildTasks(parentTaskId: number): Promise<void> {
    await db.delete(tasks).where(eq(tasks.parentTaskId, parentTaskId));
  }

  async getChildTasks(parentTaskId: number): Promise<Task[]> {
    return await db.select().from(tasks).where(eq(tasks.parentTaskId, parentTaskId)).orderBy(tasks.dueDate);
  }

  async getTaskCountByWeek(): Promise<Record<number, number>> {
    const allTasks = await db.select().from(tasks);
    const counts: Record<number, number> = {};
    for (const task of allTasks) {
      counts[task.weekNumber] = (counts[task.weekNumber] || 0) + 1;
    }
    return counts;
  }

  async getFiles(): Promise<FileRecord[]> {
    return await db.select().from(files).orderBy(files.createdAt);
  }

  async getFile(id: number): Promise<FileRecord | undefined> {
    const [file] = await db.select().from(files).where(eq(files.id, id));
    return file;
  }

  async getFileByPath(objectPath: string): Promise<FileRecord | undefined> {
    const [file] = await db.select().from(files).where(eq(files.objectPath, objectPath));
    return file;
  }

  async createFile(insertFile: InsertFile): Promise<FileRecord> {
    const [file] = await db.insert(files).values(insertFile).returning();
    return file;
  }

  async updateFile(id: number, updates: { displayName?: string; folder?: string | null; listened?: boolean; lastChunk...
    const setData: Record<string, unknown> = {};
    if (updates.displayName !== undefined) {
      setData.displayName = updates.displayName;
    }
    if (updates.folder !== undefined) {
      setData.folder = updates.folder;
    }
    if (updates.listened !== undefined) {
      setData.listened = updates.listened;
    }
    if (updates.lastChunkIndex !== undefined) {
      setData.lastChunkIndex = updates.lastChunkIndex;
    }
    if (updates.totalChunks !== undefined) {
      setData.totalChunks = updates.totalChunks;
    }
    if (updates.checkedChunks !== undefined) {
      setData.checkedChunks = updates.checkedChunks;
    }
    if (updates.ttsAudioUrl !== undefined) {
      setData.ttsAudioUrl = updates.ttsAudioUrl;
    }
  }

```

```

    }
    if (updates.ttsGeneratedAt !== undefined) {
      setData.ttsGeneratedAt = updates.ttsGeneratedAt;
    }
    if (updates.objectPath !== undefined) {
      setData.objectPath = updates.objectPath;
    }
    if (updates.extractedText !== undefined) {
      setData.extractedText = updates.extractedText;
    }
    if (updates.preparedAudioPaths !== undefined) {
      setData.preparedAudioPaths = updates.preparedAudioPaths;
    }
    if (updates.preparedAt !== undefined) {
      setData.preparedAt = updates.preparedAt;
    }
    const [updated] = await db
      .update(files)
      .set(setData)
      .where(eq(files.id, id))
      .returning();
    return updated;
  }

  async deleteFile(id: number): Promise<void> {
    await db.delete(files).where(eq(files.id, id));
  }

  async getActiveSemesterSettings(): Promise<SemesterSettings | undefined> {
    const [settings] = await db
      .select()
      .from(semesterSettings)
      .where(eq(semesterSettings.isActive, true))
      .orderBy(desc(semesterSettings.createdAt))
      .limit(1);
    return settings;
  }

  async getAllSemesterSettings(): Promise<SemesterSettings[]> {
    return db
      .select()
      .from(semesterSettings)
      .orderBy(semesterSettings.semesterStartDate);
  }

  async createSemesterSettings(settings: InsertSemesterSettings): Promise<SemesterSettings> {
    await db.update(semesterSettings).set({ isActive: false });
    const [newSettings] = await db.insert(semesterSettings).values({
      ...settings,
      isActive: true,
    }).returning();
    return newSettings;
  }

  async updateSemesterSettings(id: number, updates: Partial<SemesterSettings>): Promise<SemesterSettings> {
    const [updated] = await db
      .update(semesterSettings)
      .set(updates)
      .where(eq(semesterSettings.id, id))
      .returning();
    return updated;
  }

  async getSecondGoogleAccount(): Promise<SecondGoogleAccount | undefined> {
    const [account] = await db
      .select()
      .from(secondGoogleAccount)
      .orderBy(desc(secondGoogleAccount.createdAt))
      .limit(1);
    return account;
  }

  async saveSecondGoogleAccount(account: InsertSecondGoogleAccount): Promise<SecondGoogleAccount> {
    // Delete any existing accounts first (only one second account allowed)
    await db.delete(secondGoogleAccount);
    const [newAccount] = await db.insert(secondGoogleAccount).values(account).returning();
    return newAccount;
  }

  async updateSecondGoogleAccount(id: number, updates: Partial<SecondGoogleAccount>): Promise<SecondGoogleAccount> {
    const [updated] = await db
      .update(secondGoogleAccount)
      .set({ ...updates, updatedAt: new Date() })
      .where(eq(secondGoogleAccount.id, id))
      .returning();
    return updated;
  }

  async deleteSecondGoogleAccount(): Promise<void> {
    await db.delete(secondGoogleAccount);
  }

  async getThirdGoogleAccount(): Promise<ThirdGoogleAccount | undefined> {
    const [account] = await db
      .select()
      .from(thirdGoogleAccount)
      .orderBy(desc(thirdGoogleAccount.createdAt))
      .limit(1);
    return account;
  }

  async saveThirdGoogleAccount(account: InsertThirdGoogleAccount): Promise<ThirdGoogleAccount> {
    await db.delete(thirdGoogleAccount);
    const [newAccount] = await db.insert(thirdGoogleAccount).values(account).returning();
    return newAccount;
  }

  async updateThirdGoogleAccount(id: number, updates: Partial<ThirdGoogleAccount>): Promise<ThirdGoogleAccount> {

```

```

    const [updated] = await db
      .update(thirdGoogleAccount)
      .set({ ...updates, updatedAt: new Date() })
      .where(eq(thirdGoogleAccount.id, id))
      .returning();
    return updated;
  }

  async deleteThirdGoogleAccount(): Promise<void> {
    await db.delete(thirdGoogleAccount);
  }

  async getDeletedFolders(): Promise<DeletedFolder[]> {
    return await db.select().from(deletedFolders);
  }

  async addDeletedFolder(folderId: string): Promise<DeletedFolder> {
    const [folder] = await db.insert(deletedFolders).values({ folderId }).returning();
    return folder;
  }

  async removeDeletedFolder(folderId: string): Promise<void> {
    await db.delete(deletedFolders).where(eq(deletedFolders.folderId, folderId));
  }

  async getCustomFolders(): Promise<CustomFolder[]> {
    return await db.select().from(customFolders);
  }

  async createCustomFolder(folder: InsertCustomFolder): Promise<CustomFolder> {
    const [created] = await db.insert(customFolders).values(folder).returning();
    return created;
  }

  async updateCustomFolder(id: number, name: string): Promise<CustomFolder> {
    const [updated] = await db
      .update(customFolders)
      .set({ name })
      .where(eq(customFolders.id, id))
      .returning();
    return updated;
  }

  async deleteCustomFolder(id: number): Promise<void> {
    await db.delete(customFolders).where(eq(customFolders.id, id));
  }

  // Subtask methods
  async getSubtasksByTask(taskId: number): Promise<Subtask[]> {
    return await db.select().from(subtasks)
      .where(eq(subtasks.parentTaskId, taskId))
      .orderBy(subtasks.position, subtasks.createdAt);
  }

  async getSubtask(id: number): Promise<Subtask | undefined> {
    const [subtask] = await db.select().from(subtasks).where(eq(subtasks.id, id));
    return subtask;
  }

  async createSubtask(subtask: InsertSubtask): Promise<Subtask> {
    const [created] = await db.insert(subtasks).values(subtask).returning();
    return created;
  }

  async updateSubtask(id: number, updates: Partial<InsertSubtask>): Promise<Subtask> {
    const modifiedUpdates = { ...updates } as typeof updates & { completedAt?: Date | null };
    if ('isCompleted' in updates) {
      if (updates.isCompleted === true) {
        modifiedUpdates.completedAt = new Date();
      } else if (updates.isCompleted === false) {
        modifiedUpdates.completedAt = null;
      }
    }
    const [updated] = await db
      .update(subtasks)
      .set(modifiedUpdates)
      .where(eq(subtasks.id, id))
      .returning();
    return updated;
  }

  async deleteSubtask(id: number): Promise<void> {
    // Also delete any links associated with this subtask
    await db.delete(taskLinks).where(
      or(
        and(eq(taskLinks.sourceType, 'subtask'), eq(taskLinks.sourceId, id)),
        and(eq(taskLinks.targetType, 'subtask'), eq(taskLinks.targetId, id))
      )
    );
    await db.delete(subtasks).where(eq(subtasks.id, id));
  }

  async deleteSubtasksByTask(taskId: number): Promise<void> {
    // Get all subtask IDs for this task first
    const taskSubtasks = await db.select({ id: subtasks.id }).from(subtasks)
      .where(eq(subtasks.parentTaskId, taskId));

    // Delete links for each subtask
    for (const sub of taskSubtasks) {
      await db.delete(taskLinks).where(
        or(
          and(eq(taskLinks.sourceType, 'subtask'), eq(taskLinks.sourceId, sub.id)),
          and(eq(taskLinks.targetType, 'subtask'), eq(taskLinks.targetId, sub.id))
        )
      );
    }

    await db.delete(subtasks).where(eq(subtasks.parentTaskId, taskId));
  }

```

```

}

// Task Link methods
async getAllTaskLinks(): Promise<TaskLink[]> {
  return await db.select().from(taskLinks);
}

async getLinksForTask(taskId: number): Promise<TaskLink[]> {
  return await db.select().from(taskLinks).where(
    or(
      and(eq(taskLinks.sourceType, 'task'), eq(taskLinks.sourceId, taskId)),
      and(eq(taskLinks.targetType, 'task'), eq(taskLinks.targetId, taskId))
    )
  );
}

async getLinksForSubtask(subtaskId: number): Promise<TaskLink[]> {
  return await db.select().from(taskLinks).where(
    or(
      and(eq(taskLinks.sourceType, 'subtask'), eq(taskLinks.sourceId, subtaskId)),
      and(eq(taskLinks.targetType, 'subtask'), eq(taskLinks.targetId, subtaskId))
    )
  );
}

async createTaskLink(link: InsertTaskLink): Promise<TaskLink> {
  const [created] = await db.insert(taskLinks).values(link).returning();
  return created;
}

async deleteTaskLink(id: number): Promise<void> {
  await db.delete(taskLinks).where(eq(taskLinks.id, id));
}

// Project methods
async getProjects(): Promise<Project[]> {
  return await db.select().from(projects).orderBy(desc(projects.createdAt));
}

async getProject(id: number): Promise<Project | undefined> {
  const [project] = await db.select().from(projects).where(eq(projects.id, id));
  return project;
}

async createProject(project: InsertProject): Promise<Project> {
  const [created] = await db.insert(projects).values(project).returning();
  return created;
}

async updateProject(id: number, updates: Partial<InsertProject>): Promise<Project> {
  const modifiedUpdates = { ...updates, updatedAt: new Date() } as typeof updates & { updatedAt: Date; completedAt?: ...
  if ('status' in updates) {
    if (updates.status === 'completed') {
      modifiedUpdates.completedAt = new Date();
    } else {
      modifiedUpdates.completedAt = null;
    }
  }
  const [updated] = await db.update(projects).set(modifiedUpdates).where(eq(projects.id, id)).returning();
  return updated;
}

async deleteProject(id: number): Promise<void> {
  // Unlink tasks from this project before deleting
  await db.update(tasks).set({ projectId: null }).where(eq(tasks.projectId, id));
  await db.delete(projects).where(eq(projects.id, id));
}

async getTasksByProject(projectId: number): Promise<Task[]> {
  return await db.select().from(tasks).where(eq(tasks.projectId, projectId)).orderBy(tasks.dueDate);
}

// Sticky Notes implementation
async getStickyNotes(): Promise<StickyNote[]> {
  return await db.select().from(stickyNotes).where(eq(stickyNotes.isDeleted, false)).orderBy(desc(stickyNotes.zInde...
}

async getDeletedStickyNotes(): Promise<StickyNote[]> {
  return await db.select().from(stickyNotes).where(eq(stickyNotes.isDeleted, true)).orderBy(desc(stickyNotes.delete...
}

async getStickyNote(id: number): Promise<StickyNote | undefined> {
  const [note] = await db.select().from(stickyNotes).where(eq(stickyNotes.id, id));
  return note;
}

async createStickyNote(note: InsertStickyNote): Promise<StickyNote> {
  const [created] = await db.insert(stickyNotes).values(note).returning();
  return created;
}

async updateStickyNote(id: number, updates: Partial<InsertStickyNote>): Promise<StickyNote> {
  const [updated] = await db.update(stickyNotes).set(updates).where(eq(stickyNotes.id, id)).returning();
  return updated;
}

async deleteStickyNote(id: number): Promise<void> {
  await db.update(stickyNotes).set({ isDeleted: true, deletedAt: new Date() }).where(eq(stickyNotes.id, id));
}

async restoreStickyNote(id: number): Promise<StickyNote> {
  const [restored] = await db.update(stickyNotes).set({ isDeleted: false, deletedAt: null }).where(eq(stickyNotes.i...
  return restored;
}

// Access Tokens implementation
async getAccessTokens(): Promise<AccessToken[]> {
  return await db.select().from(accessTokens).orderBy(desc(accessTokens.createdAt));
}

```

```

}

async getAccessToken(token: string): Promise<AccessToken | undefined> {
  const [found] = await db.select().from(accessTokens).where(eq(accessTokens.token, token));
  return found;
}

async createAccessToken(token: InsertAccessToken): Promise<AccessToken> {
  const [created] = await db.insert(accessTokens).values(token).returning();
  return created;
}

async updateAccessToken(id: number, updates: Partial<InsertAccessToken>): Promise<AccessToken> {
  const [updated] = await db.update(accessTokens).set(updates).where(eq(accessTokens.id, id)).returning();
  return updated;
}

async deleteAccessToken(id: number): Promise<void> {
  await db.delete(accessTokens).where(eq(accessTokens.id, id));
}

async getShiftSchedule(): Promise<ShiftScheduleEntry[]> {
  return await db.select().from(shiftSchedule);
}

async getShiftForDate(date: string): Promise<ShiftScheduleEntry | undefined> {
  const [found] = await db.select().from(shiftSchedule).where(eq(shiftSchedule.date, date));
  return found;
}

async setShiftDay(date: string, shiftType: string): Promise<ShiftScheduleEntry> {
  const existing = await this.getShiftForDate(date);
  if (existing) {
    if (shiftType === "off") {
      await db.delete(shiftSchedule).where(eq(shiftSchedule.date, date));
      return { ...existing, shiftType: "off" };
    }
    const [updated] = await db.update(shiftSchedule).set({ shiftType }).where(eq(shiftSchedule.date, date)).returning();
    return updated;
  }
  if (shiftType === "off") {
    return { id: 0, date, shiftType: "off" };
  }
  const [created] = await db.insert(shiftSchedule).values({ date, shiftType }).returning();
  return created;
}

async setShiftBulk(entries: InsertShiftScheduleEntry[]): Promise<void> {
  if (entries.length === 0) return;
  const toInsert = entries.filter(e => e.shiftType !== "off");
  const toDelete = entries.filter(e => e.shiftType === "off").map(e => e.date);

  if (toDelete.length > 0) {
    for (const date of toDelete) {
      await db.delete(shiftSchedule).where(eq(shiftSchedule.date, date));
    }
  }
  if (toInsert.length > 0) {
    for (const entry of toInsert) {
      const existing = await this.getShiftForDate(entry.date);
      if (existing) {
        await db.update(shiftSchedule).set({ shiftType: entry.shiftType }).where(eq(shiftSchedule.date, entry.date));
      } else {
        await db.insert(shiftSchedule).values(entry);
      }
    }
  }
}

async deleteShiftDay(date: string): Promise<void> {
  await db.delete(shiftSchedule).where(eq(shiftSchedule.date, date));
}

async clearAllShifts(): Promise<void> {
  await db.delete(shiftSchedule);
}

async getSemesterChecklist(semesterSettingsId: number): Promise<SemesterCheckListItem[]> {
  return await db.select().from(semesterChecklist).where(eq(semesterChecklist.semesterSettingsId, semesterSettingsId));
}

async upsertSemesterCheckListItem(item: InsertSemesterCheckListItem): Promise<SemesterCheckListItem> {
  const existing = await db.select().from(semesterChecklist).where(
    and(
      eq(semesterChecklist.semesterSettingsId, item.semesterSettingsId),
      eq(semesterChecklist.courseCode, item.courseCode),
      eq(semesterChecklist.itemType, item.itemType)
    )
  );
  if (existing.length > 0) {
    const [updated] = await db.update(semesterChecklist)
      .set({ isChecked: item.isChecked, checkedAt: item.isChecked ? new Date() : null })
      .where(eq(semesterChecklist.id, existing[0].id))
      .returning();
    return updated;
  }
  const [created] = await db.insert(semesterChecklist).values({
    ...item,
    checkedAt: item.isChecked ? new Date() : null,
  }).returning();
  return created;
}

async initSemesterChecklist(semesterSettingsId: number, courseCodes: string[]): Promise<SemesterCheckListItem[]> {
  const existing = await this.getSemesterChecklist(semesterSettingsId);
  if (existing.length > 0) return existing;

  const itemTypes = ['tasks', 'modules', 'readings'];

```

```

const items: SemesterCheckListItem[] = [];
for (const courseCode of courseCodes) {
  for (const itemType of itemTypes) {
    const [created] = await db.insert(semesterCheckList).values({
      semesterSettingsId,
      courseCode,
      itemType,
      isChecked: false,
    }).returning();
    items.push(created);
  }
}
return items;
}

async getCourseWeekMappings(courseCode: string, semesterSettingsId: number): Promise<CourseWeekMapping[]> {
  return await db.select().from(courseWeekMappings).where(
    and(eq(courseWeekMappings.courseCode, courseCode), eq(courseWeekMappings.semesterSettingsId, semesterSettingsId))
  );
}

async getAllCourseWeekMappings(semesterSettingsId: number): Promise<CourseWeekMapping[]> {
  return await db.select().from(courseWeekMappings).where(
    eq(courseWeekMappings.semesterSettingsId, semesterSettingsId)
  );
}

async upsertCourseWeekMapping(data: InsertCourseWeekMapping): Promise<CourseWeekMapping> {
  const existing = await db.select().from(courseWeekMappings).where(
    and(
      eq(courseWeekMappings.courseCode, data.courseCode),
      eq(courseWeekMappings.semesterSettingsId, data.semesterSettingsId),
      eq(courseWeekMappings.weekNumber, data.weekNumber)
    )
  );
  if (existing.length > 0) {
    const [updated] = await db.update(courseWeekMappings)
      .set({ confirmed: data.confirmed, courseWeekLabel: data.courseWeekLabel, notes: data.notes })
      .where(eq(courseWeekMappings.id, existing[0].id))
      .returning();
    return updated;
  }
  const [created] = await db.insert(courseWeekMappings).values(data).returning();
  return created;
}

async getScholarships(): Promise<Scholarship[]> {
  return await db.select().from(scholarships);
}

async getScholarship(id: number): Promise<Scholarship | undefined> {
  const [s] = await db.select().from(scholarships).where(eq(scholarships.id, id));
  return s;
}

async createScholarship(data: InsertScholarship): Promise<Scholarship> {
  const [created] = await db.insert(scholarships).values(data).returning();
  return created;
}

async updateScholarship(id: number, updates: Partial<InsertScholarship>): Promise<Scholarship> {
  const [updated] = await db.update(scholarships).set(updates).where(eq(scholarships.id, id)).returning();
  return updated;
}

async deleteScholarship(id: number): Promise<void> {
  await db.delete(scholarships).where(eq(scholarships.id, id));
}

async getKeyContacts(): Promise<KeyContact[]> {
  return await db.select().from(keyContacts);
}

async getKeyContact(id: number): Promise<KeyContact | undefined> {
  const [c] = await db.select().from(keyContacts).where(eq(keyContacts.id, id));
  return c;
}

async createKeyContact(data: InsertKeyContact): Promise<KeyContact> {
  const [created] = await db.insert(keyContacts).values(data).returning();
  return created;
}

async updateKeyContact(id: number, updates: Partial<InsertKeyContact>): Promise<KeyContact> {
  const [updated] = await db.update(keyContacts).set(updates).where(eq(keyContacts.id, id)).returning();
  return updated;
}

async deleteKeyContact(id: number): Promise<void> {
  await db.delete(keyContacts).where(eq(keyContacts.id, id));
}

async getAnnouncements(): Promise<Announcement[]> {
  return db.select().from(announcements).orderBy(announcements.sortOrder, announcements.receivedAt);
}

async createAnnouncement(data: InsertAnnouncement): Promise<Announcement> {
  const [created] = await db.insert(announcements).values(data).returning();
  return created;
}

async getAnnouncementByEmailId(emailId: string): Promise<Announcement | undefined> {
  const [a] = await db.select().from(announcements).where(eq(announcements.emailId, emailId));
  return a;
}

async deleteAnnouncement(id: number): Promise<void> {
  await db.delete(announcements).where(eq(announcements.id, id));
}

```

```

async getEntityComments(entityType: string, entityId: string): Promise<EntityComment[]> {
  return db.select().from(entityComments).where(and(eq(entityComments.entityType, entityType), eq(entityComments.en...
})

async upsertEntityComment(entityType: string, entityId: string, content: string): Promise<EntityComment> {
  const existing = await db.select().from(entityComments).where(and(eq(entityComments.entityType, entityType), eq(e...
  if (existing.length > 0) {
    const [updated] = await db.update(entityComments).set({ content, updatedAt: new Date() }).where(eq(entityCommen...
    return updated;
  }
  const [created] = await db.insert(entityComments).values({ entityType, entityId, content }).returning();
  return created;
}

async deleteEntityComment(id: number): Promise<void> {
  await db.delete(entityComments).where(eq(entityComments.id, id));
}

async getPendingReviewItems(status?: string): Promise<PendingReviewItem[]> {
  if (status) {
    return db.select().from(pendingReviewItems).where(eq(pendingReviewItems.status, status)).orderBy(desc(pendingRe...
  }
  return db.select().from(pendingReviewItems).orderBy(desc(pendingReviewItems.createdAt));
}

async getPendingReviewItemByExternalId(externalId: string, source: string): Promise<PendingReviewItem | undefined> {
  const [item] = await db.select().from(pendingReviewItems).where(and(eq(pendingReviewItems.externalId, externalId)...
  return item;
}

async createPendingReviewItem(item: InsertPendingReviewItem): Promise<PendingReviewItem> {
  const [created] = await db.insert(pendingReviewItems).values(item).returning();
  return created;
}

async updatePendingReviewItem(id: number, updates: Partial<InsertPendingReviewItem>): Promise<PendingReviewItem> {
  const [updated] = await db.update(pendingReviewItems).set(updates).where(eq(pendingReviewItems.id, id)).returning();
  return updated;
}

async deletePendingReviewItem(id: number): Promise<void> {
  await db.delete(pendingReviewItems).where(eq(pendingReviewItems.id, id));
}
}

export const storage = new DatabaseStorage();

```

`server/thirdGoogleAccount.ts`

```

import { google } from 'googleapis';
import { storage } from './storage';
import type { ThirdGoogleAccount } from '@shared/schema';

const SCOPES = [
  'https://www.googleapis.com/auth/calendar.readonly',
  'https://www.googleapis.com/auth/userinfo.email',
];

function getOAuth2Client() {
  const clientId = process.env.GOOGLE_SECOND_ACCOUNT_CLIENT_ID;
  const clientSecret = process.env.GOOGLE_SECOND_ACCOUNT_CLIENT_SECRET;

  if (!clientId || !clientSecret) {
    throw new Error('Google OAuth credentials not configured for third account');
  }

  const domain = process.env.REPLIT_DEV_DOMAIN || process.env.REPLIT_DOMAINS?.split(',')[0];
  const redirectUri = domain
    ? `https://${domain}/api/google/third-account/callback`
    : 'http://localhost:5000/api/google/third-account/callback';

  return new google.auth.OAuth2(clientId, clientSecret, redirectUri);
}

export function getThirdAccountAuthUrl(): string {
  const oauth2Client = getOAuth2Client();

  return oauth2Client.generateAuthUrl({
    access_type: 'offline',
    scope: SCOPES,
    prompt: 'consent',
  });
}

export async function exchangeCodeForTokensThird(code: string): Promise<ThirdGoogleAccount> {
  const oauth2Client = getOAuth2Client();

  const { tokens } = await oauth2Client.getToken(code);

  if (!tokens.access_token || !tokens.refresh_token) {
    throw new Error('Failed to get required tokens from Google');
  }

  oauth2Client.setCredentials(tokens);

  const oauth2 = google.oauth2({ version: 'v2', auth: oauth2Client });
  const userInfo = await oauth2.userinfo.get();
  const email = userInfo.data.email || 'unknown';

  const expiresAt = tokens.expiry_date
    ? new Date(tokens.expiry_date)
    : new Date(Date.now() + 3600 * 1000);

  const account = await storage.saveThirdGoogleAccount({

```



```

        email,
        accessToken: tokens.access_token,
        refreshToken: tokens.refresh_token,
        expiresAt,
    });
}

return account;
}

async function getValidAccessToken(account: ThirdGoogleAccount): Promise<string> {
    const now = new Date();
    const expiresAt = new Date(account.expiresAt);
    const bufferMs = 5 * 60 * 1000;

    if (expiresAt.getTime() - now.getTime() > bufferMs) {
        return account.accessToken;
    }

    const oauth2Client = getOAuth2Client();
    oauth2Client.setCredentials({
        refresh_token: account.refreshToken,
    });

    const { credentials } = await oauth2Client.refreshAccessToken();

    if (!credentials.access_token) {
        throw new Error('Failed to refresh access token');
    }

    const newExpiresAt = credentials.expiry_date
        ? new Date(credentials.expiry_date)
        : new Date(Date.now() + 3600 * 1000);

    await storage.updateThirdGoogleAccount(account.id, {
        accessToken: credentials.access_token,
        expiresAt: newExpiresAt,
    });

    return credentials.access_token;
}

export async function getThirdAccountCalendarClient() {
    const account = await storage.getThirdGoogleAccount();

    if (!account) {
        throw new Error('Third Google account (CRCU) not connected');
    }

    const accessToken = await getValidAccessToken(account);

    const oauth2Client = new google.auth.OAuth2();
    oauth2Client.setCredentials({ access_token: accessToken });

    return google.calendar({ version: 'v3', auth: oauth2Client });
}

export async function isThirdAccountConnected(): Promise<{ connected: boolean; email?: string }> {
    try {
        const account = await storage.getThirdGoogleAccount();
        if (account) {
            return { connected: true, email: account.email };
        }
        return { connected: false };
    } catch {
        return { connected: false };
    }
}

export async function disconnectThirdAccount(): Promise<void> {
    await storage.deleteThirdGoogleAccount();
}

export async function getEventsFromThirdAccount(timeMin: Date, timeMax: Date) {
    try {
        const calendar = await getThirdAccountCalendarClient();

        const response = await calendar.events.list({
            calendarId: 'primary',
            timeMin: timeMin.toISOString(),
            timeMax: timeMax.toISOString(),
            singleEvents: true,
            orderBy: 'startTime',
        });

        return response.data.items || [];
    } catch (error) {
        console.error('Error fetching events from third account (CRCU):', error);
        return [];
    }
}

export async function listThirdAccountCalendars() {
    try {
        const calendar = await getThirdAccountCalendarClient();
        const response = await calendar.calendarList.list();
        return response.data.items || [];
    } catch (error) {
        console.error('Error listing third account calendars:', error);
        return [];
    }
}

export async function getEventsFromThirdAccountCalendar(calendarId: string, timeMin: Date, timeMax: Date) {
    try {
        const calendar = await getThirdAccountCalendarClient();

        const response = await calendar.events.list({
            calendarId,

```

```

        timeMin: timeMin.toISOString(),
        timeMax: timeMax.toISOString(),
        singleEvents: true,
        orderBy: 'startTime',
    });

    return response.data.items || [];
} catch (error) {
    console.error('Error fetching events from third account calendar:', error);
    return [];
}
}

```

`server/timezone.ts`

```

/**
 * =====
 * LOCKED TIMEZONE: America/Toronto (Eastern Time)
 * =====
 * Password-protected. The timezone CANNOT be changed without
 * providing auth code 5747. This is enforced at runtime.
 *
 * ALL server-side date logic MUST use these functions.
 * NEVER use raw new Date() for comparisons or day boundaries.
 * =====
 */

let LOCKED_TZ = 'America/Toronto';
let tzLocked = true;
const TZ_PASSWORD = '5747';

export function getTimezone(): string {
    return LOCKED_TZ;
}

export function isTimezoneLocked(): boolean {
    return tzLocked;
}

export function changeTimezone(newTz: string, password: string): { success: boolean; error?: string } {
    if (password !== TZ_PASSWORD) {
        console.warn('[Timezone] BLOCKED: unauthorized attempt to change timezone to "${newTz}"');
        return { success: false, error: 'Incorrect password. Timezone change denied.' };
    }
    const old = LOCKED_TZ;
    LOCKED_TZ = newTz;
    process.env.TZ = newTz;
    console.log('[Timezone] Changed from "${old}" to "${newTz}" (authorized with password)');
    return { success: true };
}

export function easternNow(): Date {
    return new Date(new Date().toLocaleString('en-US', { timeZone: LOCKED_TZ }));
}

export function easternDateStr(date: Date = new Date()): string {
    return new Date(date).toLocaleDateString('en-CA', { timeZone: LOCKED_TZ });
}

export function easternHour(date: Date = new Date()): number {
    return parseInt(
        date.toLocaleString('en-US', { hour: 'numeric', hour12: false, timeZone: LOCKED_TZ }),
        10,
    ) % 24;
}

export function easternMidnight(date: Date = new Date()): Date {
    const d = easternNow();
    if (date !== undefined) {
        const str = new Date(date).toLocaleString('en-US', { timeZone: LOCKED_TZ });
        const parsed = new Date(str);
        d.setFullYear(parsed.getFullYear(), parsed.getMonth(), parsed.getDate());
    }
    d.setHours(0, 0, 0, 0);
    return d;
}

export function easternTodayStr(): string {
    return easternDateStr(new Date());
}

export function easternTomorrowMidnight(): Date {
    const m = easternMidnight();
    m.setDate(m.getDate() + 1);
    return m;
}

export function taskDateStr(dueDate: string | Date): string {
    return easternDateStr(new Date(dueDate));
}

export function addDays(date: Date, days: number): Date {
    const d = new Date(date);
    d.setDate(d.getDate() + days);
    return d;
}

export function easternTimestamp(): string {
    return new Date().toLocaleString('en-US', { timeZone: LOCKED_TZ });
}

```

`server/tmuCalendar.ts`

```

process.env.TZ = 'America/Toronto';

interface ICSEvent {
  uid: string;
  summary: string;
  description: string;
  location: string;
  dtstart: string;
  dtend: string;
  isAllDay: boolean;
  rrule?: string;
  recurrenceId?: string;
}

interface FormattedTMUEvent {
  id: string;
  title: string;
  description: string;
  location: string;
  startDate: string;
  endDate: string;
  isAllDay: boolean;
  source: 'tmu';
}

let cachedRawEvents: ICSEvent[] = [];
let lastFetchTime = 0;
const CACHE_DURATION_MS = 5 * 60 * 1000;

function unfoldICS(raw: string): string {
  return raw.replace(/\r\n[ \t]/g, '').replace(/\\r/g, '');
}

function unescapeICS(val: string): string {
  return val
    .replace(/\\n/g, '\n')
    .replace(/\\,/g, ',')
    .replace(/\\;/g, ';')
    .replace(/\\\\/g, '\\');
}

function parseICSDate(val: string, propline: string): { date: Date; isAllDay: boolean } {
  const tzMatch = propline.match(/TZID=([^;]+)/);
  const cleanVal = val.replace(/^[^dTZ]/g, '');

  if (cleanVal.length === 8) {
    const y = parseInt(cleanVal.substring(0, 4));
    const m = parseInt(cleanVal.substring(4, 6)) - 1;
    const d = parseInt(cleanVal.substring(6, 8));
    return { date: new Date(y, m, d), isAllDay: true };
  }

  const y = parseInt(cleanVal.substring(0, 4));
  const mo = parseInt(cleanVal.substring(4, 6)) - 1;
  const d = parseInt(cleanVal.substring(6, 8));
  const h = parseInt(cleanVal.substring(9, 11)) || 0;
  const mi = parseInt(cleanVal.substring(11, 13)) || 0;
  const s = parseInt(cleanVal.substring(13, 15)) || 0;

  if (cleanVal.endsWith('Z')) {
    return { date: new Date(Date.UTC(y, mo, d, h, mi, s)), isAllDay: false };
  }

  return { date: new Date(y, mo, d, h, mi, s), isAllDay: false };
}

function parseICSFeed(icsText: string): ICSEvent[] {
  const unfolded = unfoldICS(icsText);
  const lines = unfolded.split('\n');
  const events: ICSEvent[] = [];
  let currentEvent: Partial<ICSEvent> | null = null;

  for (const line of lines) {
    const trimmed = line.trim();

    if (trimmed === 'BEGIN:VEVENT') {
      currentEvent = {};
      continue;
    }

    if (trimmed === 'END:VEVENT') {
      if (currentEvent && currentEvent.dtstart) {
        events.push({
          uid: currentEvent.uid || '',
          summary: currentEvent.summary || 'Untitled',
          description: currentEvent.description || '',
          location: currentEvent.location || '',
          dtstart: currentEvent.dtstart || '',
          dtend: currentEvent.dtend || currentEvent.dtstart || '',
          isAllDay: currentEvent.isAllDay || false,
          rrule: currentEvent.rrule,
          recurrenceId: currentEvent.recurrenceId,
        });
      }
      currentEvent = null;
      continue;
    }

    if (!currentEvent) continue;

    const colonIdx = trimmed.indexOf(':');
    if (colonIdx < 0) continue;

    const propPart = trimmed.substring(0, colonIdx);
    const valPart = trimmed.substring(colonIdx + 1);
    const propName = propPart.split(';')[0].toUpperCase();
  }
}

```

```

switch (propName) {
  case 'UID':
    currentEvent.uid = valPart;
    break;
  case 'SUMMARY':
    currentEvent.summary = unescapeICS(valPart);
    break;
  case 'DESCRIPTION':
    currentEvent.description = unescapeICS(valPart);
    break;
  case 'LOCATION':
    currentEvent.location = unescapeICS(valPart);
    break;
  case 'DTSTART': {
    const parsed = parseICSDate(valPart, propPart);
    currentEvent.dtstart = parsed.date.toISOString();
    currentEvent.isAllDay = parsed.isAllDay;
    break;
  }
  case 'DTEND': {
    const parsed = parseICSDate(valPart, propPart);
    currentEvent.dtend = parsed.date.toISOString();
    break;
  }
  case 'RRULE':
    currentEvent.rrule = valPart;
    break;
  case 'RECURRENCE-ID': {
    const parsed = parseICSDate(valPart, propPart);
    currentEvent.recurrenceId = parsed.date.toISOString();
    break;
  }
}
}
return events;
}

function expandRecurringEvents(events: ICSEvent[], rangeStart: Date, rangeEnd: Date): ICSEvent[] {
  const overrides = new Map<string, Set<string>>();
  for (const ev of events) {
    if (ev.recurrenceId) {
      if (!overrides.has(ev.uid)) overrides.set(ev.uid, new Set());
      overrides.get(ev.uid)?.add(ev.recurrenceId);
    }
  }

  const result: ICSEvent[] = [];

  for (const ev of events) {
    if (ev.recurrenceId) {
      const evStart = new Date(ev.dtstart);
      if (evStart >= rangeStart && evStart <= rangeEnd) {
        result.push(ev);
      }
      continue;
    }

    if (!ev.rrule) {
      const evStart = new Date(ev.dtstart);
      if (evStart >= rangeStart && evStart <= rangeEnd) {
        result.push(ev);
      }
      continue;
    }

    const ruleMap: Record<string, string> = {};
    for (const part of ev.rrule.split(';')) {
      const [k, v] = part.split('=');
      if (k && v) ruleMap[k.toUpperCase()] = v;
    }

    if (ruleMap['FREQ'] !== 'WEEKLY') {
      const evStart = new Date(ev.dtstart);
      if (evStart >= rangeStart && evStart <= rangeEnd) {
        result.push(ev);
      }
      continue;
    }

    const baseStart = new Date(ev.dtstart);
    const baseEnd = new Date(ev.dtend);
    const durationMs = baseEnd.getTime() - baseStart.getTime();

    const untilStr = ruleMap['UNTIL'];
    let until = new Date(rangeEnd);
    if (untilStr) {
      const parsed = parseICSDate(untilStr, 'DTSTART');
      until = parsed.date;
    }
    const count = ruleMap['COUNT'] ? parseInt(ruleMap['COUNT']) : undefined;
    const interval = ruleMap['INTERVAL'] ? parseInt(ruleMap['INTERVAL']) : 1;
    const uidOverrides = overrides.get(ev.uid);

    let occurrences = 0;
    const maxDate = until < rangeEnd ? until : rangeEnd;
    const cursor = new Date(baseStart);

    while (cursor <= maxDate) {
      if (count && occurrences >= count) break;

      if (cursor >= rangeStart) {
        const isoStart = cursor.toISOString();
        if (!uidOverrides?.has(isoStart)) {
          result.push({
            ...ev,
            dtstart: isoStart,
            dtend: new Date(cursor.getTime() + durationMs).toISOString(),
          });
        }
      }
      cursor.add(interval * 60 * 60 * 1000);
      occurrences++;
    }
  }
}

```

```

        uid: `${ev.uid}_${isoStart}`,
      });
    }
  }

  cursor.setDate(cursor.getDate() + 7 * interval);
  occurrences++;

  if (occurrences > 200) break;
}

return result;
}

function formatEvents(events: ICSEvent[]): FormattedTMUEvent[] {
  return events.map(ev => ({
    id: `tmu-${ev.uid}`,
    title: ev.summary,
    description: ev.description,
    location: ev.location,
    startDate: ev.dtstart,
    endDate: ev.dtend,
    isAllDay: ev.isAllDay,
    source: 'tmu' as const,
  }));
}

export async function fetchTMUCalendarEvents(rangeStart: Date, rangeEnd: Date): Promise<FormattedTMUEvent[]> {
  const icalUrl = process.env.TMU_ICAL_URL;
  if (!icalUrl) {
    return [];
  }

  const now = Date.now();
  const needsFetch = now - lastFetchTime >= CACHE_DURATION_MS || cachedRawEvents.length === 0;

  if (needsFetch) {
    try {
      const response = await fetch(icalUrl);
      if (!response.ok) {
        console.error(`[TMU Calendar] Failed to fetch ICS feed: ${response.status}`);
      } else {
        const icsText = await response.text();
        cachedRawEvents = parseICSFeed(icsText);
        lastFetchTime = now;
        console.log(`[TMU Calendar] Parsed ${cachedRawEvents.length} events from ICS feed`);
      }
    } catch (err: any) {
      console.error(`[TMU Calendar] Error fetching ICS feed: ${err.message}`);
    }
  }

  const expanded = expandRecurringEvents(cachedRawEvents, rangeStart, rangeEnd);
  return formatEvents(expanded);
}

export function clearTMUCalendarCache(): void {
  cachedRawEvents = [];
  lastFetchTime = 0;
}

```

`server/vite.ts`

```

import { type Express } from "express";
import { createServer as createViteServer, createLogger } from "vite";
import { type Server } from "http";
import viteConfig from "../vite.config";
import fs from "fs";
import path from "path";
import { nanoid } from "nanoid";

const viteLogger = createLogger();

export async function setupVite(server: Server, app: Express) {
  const serverOptions = {
    middlewareMode: true,
    hmr: { server, path: "/vite-hmr" },
    allowedHosts: true as const,
  };

  const vite = await createViteServer({
    ...viteConfig,
    configFile: false,
    customLogger: {
      ...viteLogger,
      error: (msg, options) => {
        viteLogger.error(msg, options);
        process.exit(1);
      },
    },
    server: serverOptions,
    appType: "custom",
  });

  app.use(vite.middlewares);

  app.use("/*path", async (req, res, next) => {
    const url = req.originalUrl;

    try {
      const clientTemplate = path.resolve(
        import.meta.dirname,
        "..",

```

```

    "client",
    "index.html",
  );

  // always reload the index.html file from disk incase it changes
  let template = await fs.promises.readFile(clientTemplate, "utf-8");
  template = template.replace(
    'src="/src/main.tsx"',
    'src="/src/main.tsx?v=${nanoid()}"',
  );
  const page = await vite.transformIndexHtml(url, template);
  res.status(200).set({ "Content-Type": "text/html" }).end(page);
} catch (e) {
  vite.ssrFixStacktrace(e as Error);
  next(e);
}
});
}

```

`client/index.html`

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=1024" />
    <title>Uni-Cal</title>
    <link rel="icon" type="image/png" href="/favicon.png?v=4" />
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Architects+Daughter&family=DM+Sans:ital,opsz,wght@0,9..40,10...
  </script>
  // Prevent HA iframe scroll-down on load
  window.scrollTo(0, 0);
  document.addEventListener('DOMContentLoaded', function() { window.scrollTo(0, 0); });
  window.addEventListener('load', function() { window.scrollTo(0, 0); });
  var _scrollLock = setInterval(function() { if (window.scrolly !== 0) window.scrollTo(0, 0); }, 100);
  setTimeout(function() { clearInterval(_scrollLock); }, 5000);
</script>
<script>
  // Early error detection for older browsers
  window.onerror = function(msg, url, line, col, error) {
    document.body.innerHTML = '<div style="padding:20px;background:red;color:white;font-size:16px;">' +
      '<h2>Error Loading App</h2>' +
      '<p>' + msg + '</p>' +
      '<p>File: ' + url + '</p>' +
      '<p>Line: ' + line + '</p>' +
      '</div>';
    return true;
  };
</script>
</head>
<body>
  <div id="root">
    <noscript>
      <div style="padding:20px;text-align:center;">
        <h2>JavaScript Required</h2>
        <p>Please enable JavaScript to use this app.</p>
      </div>
    </noscript>
    <div id="loading-fallback" style="padding:40px;text-align:center;font-family:sans-serif;">
      <p>Loading University Planner...</p>
      <p style="font-size:12px;color:#666;margin-top:20px;">If this message stays for more than 10 seconds, your br...
    </div>
  </div>
  <script type="module" src="/src/main.tsx"></script>
  <script nomodule>
    // Fallback for browsers that don't support ES modules
    document.getElementById('loading-fallback').innerHTML =
      <div style="padding:20px;background:#ff6600;color:white;">' +
      '<h2>Browser Not Supported</h2>' +
      '<p>Your browser does not support modern JavaScript modules.</p>' +
      '<p>Please update your device or try a different browser.</p>' +
      '</div>';
  </script>
</body>
</html>

```

`client/public/silk-test.html`

```

<!DOCTYPE html>
<html>
  <head><title>Silk Test</title></head>
  <body>
    <div id="root"></div>
    <script src="https://unpkg.com/react@18.2.0/umd/react.production.min.js"></script>
    <script src="https://unpkg.com/react-dom@18.2.0/umd/react-dom.production.min.js"></script>
    <script>
      try {
        var root = ReactDOM.createRoot(document.getElementById('root'));
        root.render(React.createElement('h1', null, 'React works on Silk!'));
      } catch(e) {
        document.body.innerHTML = '<h1 style="color:red">Error: ' + e.message + '</h1><pre>' + e.stack + '</pre>';
      }
    </script>
  </body>
</html>

```

`client/replit\_integrations/audio/audio-playback-worklet.js`

```

/**
 * Reusable AudioWorklet for streaming PCM16 audio playback.
 * Place in public/ folder and load via audioContext.audioWorklet.addModule()
 */
class RingBuffer {
  constructor(initialCapacity) {
    this.capacity = initialCapacity;
    this.buffer = new Float32Array(initialCapacity);
    this.readIndex = 0;
    this.writeIndex = 0;
    this.availableData = 0;
  }

  push(data) {
    const len = data.length;
    // Auto-grow if needed
    while (this.availableData + len > this.capacity) {
      this.grow();
    }
    for (let i = 0; i < len; i++) {
      this.buffer[this.writeIndex] = data[i];
      this.writeIndex = (this.writeIndex + 1) % this.capacity;
      this.availableData++;
    }
  }

  grow() {
    const newCapacity = this.capacity * 2;
    const newBuffer = new Float32Array(newCapacity);
    // Copy existing data maintaining order
    for (let i = 0; i < this.availableData; i++) {
      const srcIndex = (this.readIndex + i) % this.capacity;
      newBuffer[i] = this.buffer[srcIndex];
    }
    this.buffer = newBuffer;
    this.readIndex = 0;
    this.writeIndex = this.availableData;
    this.capacity = newCapacity;
  }

  pull(outputBuffer) {
    const len = outputBuffer.length;
    const available = Math.min(len, this.availableData);
    for (let i = 0; i < available; i++) {
      outputBuffer[i] = this.buffer[this.readIndex];
      this.readIndex = (this.readIndex + 1) % this.capacity;
    }
    // Pad remaining with silence
    for (let i = available; i < len; i++) {
      outputBuffer[i] = 0;
    }
    this.availableData -= available;
    return available > 0;
  }

  available() {
    return this.availableData;
  }

  clear() {
    this.readIndex = 0;
    this.writeIndex = 0;
    this.availableData = 0;
  }
}

class AudioPlaybackProcessor extends AudioWorkletProcessor {
  constructor() {
    super();
    this.ringBuffer = new RingBuffer(24000 * 30); // 30s initial capacity
    this.isPlaying = false;
    this.streamComplete = false;

    this.port.onmessage = (event) => {
      const { type, samples } = event.data;
      if (type === "audio") {
        this.ringBuffer.push(samples);
        this.isPlaying = true;
      } else if (type === "clear") {
        this.ringBuffer.clear();
        this.isPlaying = false;
        this.streamComplete = false;
      } else if (type === "streamComplete") {
        this.streamComplete = true;
      } else if (type === "stop") {
        this.isPlaying = false;
        this.streamComplete = false;
      }
    };
  }

  process(inputs, outputs) {
    const output = outputs[0];
    if (!output || output.length === 0) return true;

    const channel = output[0];
    if (this.isPlaying) {
      this.ringBuffer.pull(channel);
      if (this.streamComplete && this.ringBuffer.available() === 0) {
        this.isPlaying = false;
        this.streamComplete = false;
        this.port.postMessage({ type: "ended" });
      }
    } else {
      channel.fill(0);
    }
    return true;
  }
}

```

```

    }
  }

  registerProcessor("audio-playback-processor", AudioPlaybackProcessor);

```

`client/replit\_integrations/audio/audio-utils.ts`

```

/**
 * Audio utility functions for voice chat.
 * Handles PCM16 decoding and AudioContext initialization.
 */

/**
 * Decode base64 PCM16 audio to Float32Array for Web Audio API
 */
export function decodePCM16ToFloat32(base64Audio: string): Float32Array {
  const raw = atob(base64Audio);
  const bytes = new Uint8Array(raw.length);
  for (let i = 0; i < raw.length; i++) {
    bytes[i] = raw.charCodeAt(i);
  }
  const pcm16 = new Int16Array(bytes.buffer);
  const float32 = new Float32Array(pcm16.length);
  for (let i = 0; i < pcm16.length; i++) {
    float32[i] = pcm16[i] / 32768;
  }
  return float32;
}

/**
 * Create and initialize AudioContext with worklet
 */
export async function createAudioPlaybackContext(
  workletPath = "/audio-playback-worklet.js",
  sampleRate = 24000
): Promise<{ ctx: AudioContext; worklet: AudioWorkletNode }> {
  const ctx = new AudioContext({ sampleRate });
  await ctx.audioWorklet.addModule(workletPath);
  const worklet = new AudioWorkletNode(ctx, "audio-playback-processor");
  worklet.connect(ctx.destination);
  return { ctx, worklet };
}

```

`client/replit\_integrations/audio/index.ts`

```

/**
 * Voice chat client utilities for Replit AI Integrations.
 *
 * Usage:
 * 1. Copy audio-playback-worklet.js to your public/ folder
 * 2. Import and use the React hooks in your components
 *
 * Example:
 * ```tsx
 * import { useVoiceRecorder, useVoiceStream } from "./audio";
 *
 * function VoiceChat() {
 *   const [transcript, setTranscript] = useState("");
 *   const recorder = useVoiceRecorder();
 *   const stream = useVoiceStream({
 *     onTranscript: (_, full) => setTranscript(full),
 *     onComplete: (text) => console.log("Done:", text),
 *   });
 *
 *   const handleClick = async () => {
 *     if (recorder.state === "recording") {
 *       const blob = await recorder.stopRecording();
 *       await stream.streamVoiceResponse("/api/voice-conversations/1/messages", blob);
 *     } else {
 *       await recorder.startRecording();
 *     }
 *   };
 *
 *   return (
 *     <div>
 *       <button onClick={handleClick}>
 *         {recorder.state === "recording" ? "Stop" : "Record"}
 *       </button>
 *       <p>{transcript}</p>
 *     </div>
 *   );
 * }
 * ``
 */

export { decodePCM16ToFloat32, createAudioPlaybackContext } from "./audio-utils";
export { useVoiceRecorder, type RecordingState } from "./useVoiceRecorder";
export { useAudioPlayback, type PlaybackState } from "./useAudioPlayback";
export { useVoiceStream } from "./useVoiceStream";

```

`client/replit\_integrations/audio/useAudioPlayback.ts`


```

/**
 * React hook for streaming audio playback using AudioWorklet.
 * Supports real-time PCM16 audio streaming from SSE responses.
 * Includes sequence buffer for reordering out-of-order chunks.
 */
import { useRef, useCallbck, useState } from "react";
import { decodePCM16ToFloat32 } from "../audio-utils";

export type PlaybackState = "idle" | "playing" | "ended";

/**
 * Reorders audio chunks that may arrive out of sequence.
 * Buffers chunks until they can be played in correct order.
 *
 * Example: If chunks arrive as seq 2, seq 0, seq 1:
 * - seq 2 arrives !' buffered (waiting for seq 0)
 * - seq 0 arrives !' played immediately, then check buffer
 * - seq 1 arrives !' played immediately (seq 0 done), seq 2 now plays
 */
class SequenceBuffer {
  private pending = new Map<number, string[]>();
  private nextSeq = 0;

  /** Add chunk with sequence number, returns chunks ready to play in order */
  push(seq: number, data: string): string[] {
    // Store the chunk under its sequence number
    if (!this.pending.has(seq)) {
      this.pending.set(seq, []);
    }
    this.pending.get(seq)!.push(data);

    // Drain consecutive ready sequences
    const ready: string[] = [];
    while (this.pending.has(this.nextSeq)) {
      ready.push(...this.pending.get(this.nextSeq)!);
      this.pending.delete(this.nextSeq);
      this.nextSeq++;
    }
    return ready;
  }

  reset() {
    this.pending.clear();
    this.nextSeq = 0;
  }
}

export function useAudioPlayback(workletPath = "/audio-playback-worklet.js") {
  const [state, setState] = useState<PlaybackState>("idle");
  const ctxRef = useRef<AudioContext | null>(null);
  const workletRef = useRef<AudioWorkletNode | null>(null);
  const readyRef = useRef(false);
  const seqBufferRef = useRef(new SequenceBuffer());

  const init = useCallback(async () => {
    if (readyRef.current) return;

    const ctx = new AudioContext({ sampleRate: 24000 });
    await ctx.audioWorklet.addModule(workletPath);
    const worklet = new AudioWorkletNode(ctx, "audio-playback-processor");
    worklet.connect(ctx.destination);

    worklet.port.onmessage = (e) => {
      if (e.data.type === "ended") setState("idle");
    };

    ctxRef.current = ctx;
    workletRef.current = worklet;
    readyRef.current = true;
  }, [workletPath]);

  /** Push audio directly (no sequencing) - for simple streaming */
  const pushAudio = useCallback((base64Audio: string) => {
    if (!workletRef.current) return;
    const samples = decodePCM16ToFloat32(base64Audio);
    workletRef.current.port.postMessage({ type: "audio", samples });
    setState("playing");
  }, []);

  /** Push audio with sequence number - reorders before playback */
  const pushSequencedAudio = useCallback((seq: number, base64Audio: string) => {
    if (!workletRef.current) return;

    const readyChunks = seqBufferRef.current.push(seq, base64Audio);
    for (const chunk of readyChunks) {
      const samples = decodePCM16ToFloat32(chunk);
      workletRef.current.port.postMessage({ type: "audio", samples });
    }
    if (readyChunks.length > 0) {
      setState("playing");
    }
  }, []);

  const signalComplete = useCallback(() => {
    workletRef.current?.port.postMessage({ type: "streamComplete" });
  }, []);

  const clear = useCallback(() => {
    workletRef.current?.port.postMessage({ type: "clear" });
    seqBufferRef.current.reset();
    setState("idle");
  }, []);

  return { state, init, pushAudio, pushSequencedAudio, signalComplete, clear };
}

```

client/replit_integrations/audio/useVoiceRecorder.ts`

```

/**
 * React hook for voice recording using MediaRecorder API.
 * Records audio in WebM/Opus format for efficient streaming.
 */
import { useRef, useCallback, useState } from "react";

export type RecordingState = "idle" | "recording" | "stopped";

export function useVoiceRecorder() {
  const [state, setState] = useState<RecordingState>("idle");
  const mediaRecorderRef = useRef<MediaRecorder | null>(null);
  const chunksRef = useRef<Blob[]>([]);

  const startRecording = useCallback(async (): Promise<void> => {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    const recorder = new MediaRecorder(stream, {
      mimeType: "audio/webm;codecs=opus",
    });

    mediaRecorderRef.current = recorder;
    chunksRef.current = [];

    recorder.ondataavailable = (e) => {
      if (e.data.size > 0) chunksRef.current.push(e.data);
    };

    recorder.start(100); // Collect chunks every 100ms
    setState("recording");
  }, []);

  const stopRecording = useCallback(() : Promise<Blob> => {
    return new Promise((resolve) => {
      const recorder = mediaRecorderRef.current;
      if (!recorder || recorder.state !== "recording") {
        resolve(new Blob());
        return;
      }

      recorder.onstop = () => {
        const blob = new Blob(chunksRef.current, { type: "audio/webm" });
        recorder.stream.getTracks().forEach((t) => t.stop());
        setState("stopped");
        resolve(blob);
      };

      recorder.stop();
    });
  }, []);

  return { state, startRecording, stopRecording };
}

```

`client/replit\_integrations/audio/useVoiceStream.ts`

```

/**
 * React hook for handling SSE voice streaming responses.
 * Converts audio blob to base64 and sends as JSON to match server expectations.
 */
import { useCallback } from "react";
import { useAudioPlayback } from "../useAudioPlayback";

interface StreamCallbacks {
  onUserTranscript?: (text: string) => void;
  onTranscript?: (text: string, full: string) => void;
  onComplete?: (transcript: string) => void;
  onError?: (error: Error) => void;
}

export function useVoiceStream(callbacks: StreamCallbacks = {}) {
  const playback = useAudioPlayback();

  const streamVoiceResponse = useCallback(
    async (url: string, audioBlob: Blob) => {
      await playback.init();
      playback.clear();

      // Convert blob to base64 for JSON body (server expects express.json())
      const base64Audio = await new Promise<string>((resolve) => {
        const fileReader = new FileReader();
        fileReader.onload = () => {
          const result = fileReader.result as string;
          resolve(result.split(",")[1]); // Remove data URL prefix
        };
        fileReader.readAsDataURL(audioBlob);
      });

      const response = await fetch(url, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ audio: base64Audio }),
      });
      if (!response.ok) throw new Error("Voice request failed");

      const streamReader = response.body?.getReader();
      if (!streamReader) throw new Error("No response body");

      const decoder = new TextDecoder();
      let buffer = "";
      let fullTranscript = "";

      while (true) {

```

```

const { done, value } = await streamReader.read();
if (done) break;

buffer += decoder.decode(value, { stream: true });
const lines = buffer.split("\n");
buffer = lines.pop() || "";

for (const line of lines) {
  if (!line.startsWith("data: ")) continue;

  try {
    const event = JSON.parse(line.slice(6));

    switch (event.type) {
      case "user_transcript":
        callbacks.onUserTranscript?.(event.data);
        break;
      case "transcript":
        fullTranscript += event.data;
        callbacks.onTranscript?.(event.data, fullTranscript);
        break;
      case "audio":
        playback.pushAudio(event.data);
        break;
      case "done":
        playback.signalComplete();
        callbacks.onComplete?.(fullTranscript);
        break;
      case "error":
        throw new Error(event.error);
    }
  } catch (e) {
    if (!(e instanceof SyntaxError)) {
      callbacks.onError?.(e as Error);
    }
  }
}
},
[playback, callbacks]
);

return { streamVoiceResponse, playbackState: playback.state };
}

```

`client/src/App.tsx`

```

import { useState, useEffect, useCallback, useRef } from "react";
import { Switch, Route, useLocation } from "wouter";
import { queryClient } from "../lib/queryClient";
import { QueryClientProvider } from "@tanstack/react-query";
import { Toaster } from "@components/ui/toaster";
import { TooltipProvider } from "@components/ui/tooltip";
import { AccessGate } from "@components/access-gate";
import { WifiOff } from "lucide-react";

import NotFound from "@pages/not-found";
import Dashboard from "@pages/dashboard";
import FilesPage from "@pages/files";
import ProjectsPage from "@pages/projects";
import PDFReaderPage from "@pages/pdf-reader";
import PDFViewerPage from "@pages/pdf-viewer";
import OneDrivePage from "@pages/onedrive";
import SpotifyPlayerPage from "@pages/spotify-player";
import OneNotePage from "@pages/onenote";
import MobileNotesPage from "@pages/mobile-notes";
import CodeCheckerPage from "@pages/code-checker";
import TickerPage from "@pages/ticker";

function useAutoFullscreen() {
  const [requested, setRequested] = useState(false);
  const isSilk = typeof navigator !== 'undefined' &&
    (/bSilk\b/i.test(navigator.userAgent) || /\bKF[A-Z]{2,4}\b/i.test(navigator.userAgent) || /\bFireTV\b/i.test(navi...
  const urlWantsFullscreen = typeof window !== 'undefined' && new URLSearchParams(window.location.search).get('fullsc...
  const shouldFullscreen = isSilk || urlWantsFullscreen;
  useEffect(() => {
    if (!shouldFullscreen) return;
    const isInIframe = window.self !== window.top;
    const heightVal = isInIframe ? '100%' : '100vh';
    const widthVal = isInIframe ? '100%' : '100vw';
    document.documentElement.style.cssText += `;position:fixed;top:0;left:0;width:${widthVal};height:${heightVal};ove...
    document.body.style.cssText += `;margin:0;padding:0;min-height:${heightVal};width:${widthVal};height:${heightVal}...
    const meta = document.querySelector('meta[name="viewport"]');
    if (meta) {
      meta.setAttribute('content', 'width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no, viewp...
    }
  }, [shouldFullscreen]);

  const requestFullscreen = useCallback(() => {
    if (requested) return;
    if (document.fullscreenElement) { setRequested(true); return; }
    const el = document.documentElement as any;
    const fn = el.requestFullscreen || el.webkitRequestFullscreen || el.mozRequestFullScreen || el.msRequestFullscreen;
    if (fn) {
      fn.call(el).then(() => setRequested(true)).catch(() => {});
    }
  }, [requested]);

  useEffect(() => {
    if (!shouldFullscreen || requested) return;
    requestFullscreen();
    const interval = setInterval(() => {
      if (document.fullscreenElement) { setRequested(true); clearInterval(interval); return; }
      requestFullscreen();
    }, 1000);
  }, [shouldFullscreen, requested]);
}

```

```

    }, 2000);
    const handler = () => requestFullscreen();
    document.addEventListener('click', handler);
    document.addEventListener('touchstart', handler);
    document.addEventListener('keydown', handler);
    window.addEventListener('focus', handler);
    return () => {
        clearInterval(interval);
        document.removeEventListener('click', handler);
        document.removeEventListener('touchstart', handler);
        document.removeEventListener('keydown', handler);
        window.removeEventListener('focus', handler);
    };
    }, [shouldFullscreen, requested, requestFullscreen]);
}

function ConnectionBanner() {
    const [offline, setOffline] = useState(false);
    const [downSeconds, setDownSeconds] = useState(0);
    const failedAt = useRef<number | null>(null);
    const checkInterval = useRef<ReturnType<typeof setInterval> | null>(null);
    const tickInterval = useRef<ReturnType<typeof setInterval> | null>(null);

    useEffect(() => {
        const check = async () => {
            try {
                const controller = new AbortController();
                const timer = setTimeout(() => controller.abort(), 5000);
                const resp = await fetch("/api/health", { signal: controller.signal });
                clearTimeout(timer);
                if (resp.ok) {
                    if (failedAt.current) {
                        failedAt.current = null;
                        setOffline(false);
                        setDownSeconds(0);
                    }
                    return;
                }
                throw new Error("not ok");
            } catch {
                if (!failedAt.current) failedAt.current = Date.now();
                setOffline(true);
            }
        };
        check();
        checkInterval.current = setInterval(check, 10000);
        tickInterval.current = setInterval(() => {
            if (failedAt.current) setDownSeconds(Math.round((Date.now() - failedAt.current) / 1000));
        }, 1000);
        return () => {
            if (checkInterval.current) clearInterval(checkInterval.current);
            if (tickInterval.current) clearInterval(tickInterval.current);
        };
    }, []);

    if (!offline) return null;

    const mins = Math.floor(downSeconds / 60);
    const secs = downSeconds % 60;
    const timeStr = mins > 0 ? `${mins}m ${secs}s` : `${secs}s`;

    return (
        <div data-testid="connection-lost-banner" style={{
            position: "fixed", top: 0, left: 0, right: 0, zIndex: 99999,
            background: "linear-gradient(90deg, #dc2626, #b91c1c)",
            color: "#fff", display: "flex", alignItems: "center", justifyContent: "center",
            gap: 8, padding: "6px 16px", fontSize: 13, fontWeight: 600,
            boxShadow: "0 2px 12px rgba(220,38,38,0.5)",
            animation: "pulse 2s ease-in-out infinite",
        }}>
            <WifiOff style={{ width: 16, height: 16 }} />
            <span>Dashboard server unreachable – down for {timeStr}</span>
        </div>
    );
}

function Router() {
    return (
        <Switch>
            <Route path="/" component={Dashboard} />
            <Route path="/files" component={FilesPage} />
            <Route path="/onedrive" component={OneDrivePage} />
            <Route path="/projects" component={ProjectsPage} />
            <Route path="/pdf-reader/onedrive" component={PDFReaderPage} />
            <Route path="/pdf-reader/:fileId" component={PDFReaderPage} />
            <Route path="/pdf-reader" component={PDFReaderPage} />
            <Route path="/pdf-viewer/*" component={PDFViewerPage} />
            <Route path="/spotify" component={SpotifyPlayerPage} />
            <Route path="/onenote" component={OneNotePage} />
            <Route path="/mobile/notes" component={MobileNotesPage} />
            <Route path="/code-checker" component={CodeCheckerPage} />
            <Route component={NotFound} />
        </Switch>
    );
}

function App() {
    useAutoFullscreen();
    const [location] = useLocation();

    if (location === '/ticker') {
        return (
            <QueryClientProvider client={queryClient}>
                <TickerPage />
            </QueryClientProvider>
        );
    }
}

```

```

    return (
      <QueryClientProvider client={queryClient}>
        <TooltipProvider>
          <ConnectionBanner />
          <AccessGate>
            <Toaster />
            <Router />
          </AccessGate>
        </TooltipProvider>
      </QueryClientProvider>
    );
  }
}

export default App;

```

`client/src/components/AddDeviceDialog.tsx`

```

import { useState } from "react";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { insertDeviceSchema, ROOMS, DEVICE_TYPES, type InsertDevice } from "@shared/schema";
import { useCreateDevice } from "@hooks/use-devices";
import {
  Dialog,
  DialogContent,
  DialogHeader,
  DialogTitle,
  DialogTrigger,
} from "@components/ui/dialog";
import {
  Form,
  FormControl,
  FormField,
  FormItem,
  FormLabel,
  FormMessage,
} from "@components/ui/form";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@components/ui/select";
import { Input } from "@components/ui/input";
import { Button } from "@components/ui/button";
import { Plus, Loader2 } from "lucide-react";
import { z } from "zod";

const formSchema = insertDeviceSchema.extend({
  value: z.coerce.number().optional(),
});

export function AddDeviceDialog() {
  const [open, setOpen] = useState(false);
  const { mutate, isPending } = useCreateDevice();

  const form = useForm<InsertDevice>({
    resolver: zodResolver(formSchema),
    defaultValues: {
      name: "",
      type: "switch",
      room: "Living Room",
      icon: "power",
      isOn: false,
      value: 0,
      unit: "",
    },
  });

  function onSubmit(data: InsertDevice) {
    mutate(data, {
      onSuccess: () => {
        setOpen(false);
        form.reset();
      },
    });
  }

  return (
    <Dialog open={open} onOpenChange={setOpen}>
      <DialogTrigger asChild>
        <Button className="gap-2 bg-primary hover:bg-primary/90 text-white shadow-lg shadow-primary/25">
          <Plus className="w-4 h-4" /> Add Device
        </Button>
      </DialogTrigger>
      <DialogContent className="sm:max-w-[425px] bg-card border-white/10 text-foreground">
        <DialogHeader>
          <DialogTitle className="font-display text-xl">Add New Device</DialogTitle>
        </DialogHeader>
        <Form {...form}>
          <form onSubmit={form.handleSubmit(onSubmit)} className="space-y-4 mt-4">
            <FormField>
              <control={form.control}>
                <name="name">
                  <render={({ field }) => (
                    <FormItem>
                      <FormLabel>Device Name</FormLabel>
                      <FormControl>
                        <input placeholder="e.g., Ceiling Fan" className="bg-secondary border-transparent focus:border-pr...>
                      </FormControl>
                      <FormMessage />
                    </FormItem>
                  )}
                </name>
              </control>
            </FormField>

```

```

/>

<div className="grid grid-cols-2 gap-4">
  <FormField
    control={form.control}
    name="type"
    render={({ field }) => (
      <FormItem>
        <FormLabel>Type</FormLabel>
        <Select onValueChange={field.onChange} defaultValue={field.value}>
          <FormControl>
            <SelectTrigger className="bg-secondary border-transparent">
              <SelectValue placeholder="Select type" />
            </SelectTrigger>
          </FormControl>
          <SelectContent className="bg-card border-white/10">
            {DEVICE_TYPES.map((type) => (
              <SelectItem key={type} value={type} className="capitalize cursor-pointer">
                {type}
              </SelectItem>
            ))}
          </SelectContent>
        </Select>
      </FormMessage />
    </FormItem>
  )}
/>

  <FormField
    control={form.control}
    name="room"
    render={({ field }) => (
      <FormItem>
        <FormLabel>Room</FormLabel>
        <Select onValueChange={field.onChange} defaultValue={field.value}>
          <FormControl>
            <SelectTrigger className="bg-secondary border-transparent">
              <SelectValue placeholder="Select room" />
            </SelectTrigger>
          </FormControl>
          <SelectContent className="bg-card border-white/10">
            {ROOMS.map((room) => (
              <SelectItem key={room} value={room} className="cursor-pointer">
                {room}
              </SelectItem>
            ))}
          </SelectContent>
        </Select>
      </FormMessage />
    </FormItem>
  )}
/>
</div>

<FormField
  control={form.control}
  name="icon"
  render={({ field }) => (
    <FormItem>
      <FormLabel>Icon (Lucide Name)</FormLabel>
      <Select onValueChange={field.onChange} defaultValue={field.value}>
        <FormControl>
          <SelectTrigger className="bg-secondary border-transparent">
            <SelectValue placeholder="Select icon" />
          </SelectTrigger>
        </FormControl>
        <SelectContent className="bg-card border-white/10 h-48">
          {[
            "lightbulb", "power", "thermometer", "activity", "fan", "tv", "wifi", "lock"
          ].map((icon) => (
            <SelectItem key={icon} value={icon} className="capitalize cursor-pointer">
              {icon}
            </SelectItem>
          ))}
        </SelectContent>
      </Select>
    </FormMessage />
  </FormItem>
)}
/>

{form.watch("type") === "climate" && (
  <div className="grid grid-cols-2 gap-4">
    <FormField
      control={form.control}
      name="value"
      render={({ field }) => (
        <FormItem>
          <FormLabel>Initial Value</FormLabel>
          <FormControl>
            <Input type="number" placeholder="20" className="bg-secondary border-transparent" {...field} />
          </FormControl>
          <FormMessage />
        </FormItem>
      )}
    </div>
    <FormField
      control={form.control}
      name="unit"
      render={({ field }) => (
        <FormItem>
          <FormLabel>Unit</FormLabel>
          <FormControl>
            <Input placeholder="°C" className="bg-secondary border-transparent" {...field} />
          </FormControl>
          <FormMessage />
        </FormItem>
      )}
    </div>
  )}
</div>

```

```

    ))
    <div className="flex justify-end pt-4">
      <Button type="submit" disabled={isPending} className="bg-primary hover:bg-primary/90">
        {isPending ? (
          <>
            <Loader2 className="mr-2 h-4 w-4 animate-spin" />
            Adding...
          </>
        ) : (
          "Add Device"
        )}
      </Button>
    </div>
  </form>
</Form>
</DialogContent>
</Dialog>
);
}

```

`client/src/components/BookAnimation.tsx`

```

import { useState, useEffect, useCallback } from 'react';
import { createPortal } from 'react-dom';

interface BookAnimationProps {
  isOpen: boolean;
  onComplete: () => void;
  bookColor?: string;
  title?: string;
  courseCode?: string;
}

export default function BookAnimation({ isOpen, onComplete, bookColor = '#8B4513', title = '', courseCode = '' }: Boo...
  const [phase, setPhase] = useState<'idle' | 'pull' | 'flip' | 'move' | 'pages' | 'open' | 'done'>('idle');

  useEffect(() => {
    if (!isOpen) {
      setPhase('idle');
      return;
    }
    setPhase('pull');
    const t1 = setTimeout(() => setPhase('flip'), 600);
    const t2 = setTimeout(() => setPhase('move'), 1200);
    const t3 = setTimeout(() => setPhase('pages'), 1800);
    const t4 = setTimeout(() => setPhase('open'), 3000);
    const t5 = setTimeout(() => {
      setPhase('done');
      onComplete();
    }, 3800);
    return () => { clearTimeout(t1); clearTimeout(t2); clearTimeout(t3); clearTimeout(t4); clearTimeout(t5); };
  }, [isOpen, onComplete]);

  if (!isOpen && phase === 'idle') return null;

  const pageColors = ['#fef3c7', '#fef9c3', '#ffffeb', '#fefe8', '#fef3c7', '#fef9c3'];

  return createPortal(
    <div style={{
      position: 'fixed',
      inset: 0,
      zIndex: 999999,
      backgroundColor: phase === 'open' || phase === 'done' ? 'rgba(0,0,0,0.85)' : 'rgba(0,0,0,0.6)',
      transition: 'background-color 0.5s ease',
      perspective: '1500px',
      overflow: 'hidden',
    }}>
      <style>{`
        @keyframes bookPullOut {
          0% { transform: translateX(-100%) rotateY(0deg) scale(0.8); opacity: 0; }
          30% { transform: translateX(-60%) rotateY(-5deg) scale(0.85); opacity: 1; }
          100% { transform: translateX(0%) rotateY(0deg) scale(1); opacity: 1; }
        }
        @keyframes bookFlip {
          0% { transform: rotateY(0deg) rotateZ(0deg) scale(1); }
          50% { transform: rotateY(90deg) rotateZ(5deg) scale(1.05); }
          100% { transform: rotateY(180deg) rotateZ(0deg) scale(1); }
        }
        @keyframes bookMoveRight {
          0% { transform: translate(-50%, -50%) rotateY(180deg); }
          100% { transform: translate(30%, -50%) rotateY(180deg); }
        }
        @keyframes pageFlip {
          0% { transform: rotateY(0deg); }
          100% { transform: rotateY(-180deg); }
        }
        @keyframes bookOpen {
          0% { transform: translate(30%, -50%) rotateY(0deg) scale(1); }
          100% { transform: translate(0%, -50%) rotateY(0deg) scale(1.3); }
        }
        @keyframes pageRuffle {
          0%, 100% { transform: rotateY(0deg); }
          50% { transform: rotateY(-12deg); }
        }
        @keyframes shelfSlide {
          0% { transform: translateX(0); }
          100% { transform: translateX(-40px); }
        }
      `}</style>

      {(phase === 'pull' || phase === 'flip') && (
        <div style={{
          position: 'absolute',

```

```

    left: '5%',
    top: '50%',
    transform: 'translateY(-50%)',
    display: 'flex',
    gap: '3px',
    alignItems: 'flex-end',
  })>
  {[0,1,2,3,4].map(i => (
    <div key={i} style={{
      width: '20px',
      height: `${100 + i * 15}px`,
      backgroundColor: ['#654321', '#8B6914', '#556B2F', '#4A0E0E', '#2F4F4F'][i],
      borderRadius: '2px 3px 3px 2px',
      boxShadow: 'inset -2px 0 4px rgba(0,0,0,0.3)',
      animation: phase === 'pull' && i === 2 ? 'shelfSlide 0.5s ease-out forwards' : 'none',
      opacity: phase === 'pull' && i === 2 ? 0 : 1,
      transition: 'opacity 0.3s ease 0.2s',
    }} />
  ))}
</div>
))
<div style={{
  position: 'absolute',
  left: '50%',
  top: '50%',
  width: '180px',
  height: '240px',
  transformStyle: 'preserve-3d',
  ...(phase === 'pull' ? {
    animation: 'bookPullOut 0.6s ease-out forwards',
  } : phase === 'flip' ? {
    animation: 'bookFlip 0.6s ease-in-out forwards',
  } : phase === 'move' ? {
    animation: 'bookMoveRight 0.6s ease-in-out forwards',
  } : phase === 'pages' || phase === 'open' || phase === 'done' ? {
    transform: phase === 'open' || phase === 'done' ? 'translate(0%, -50%) scale(1.3)' : 'translate(30%, -50%) ...',
    transition: 'transform 0.8s ease',
  } : {
    opacity: 0,
  })},
  >
  <div style={{
    position: 'absolute',
    inset: 0,
    backgroundColor: bookColor,
    borderRadius: '4px 8px 8px 4px',
    boxShadow: '0 10px 40px rgba(0,0,0,0.5), inset 0 0 20px rgba(0,0,0,0.2)',
    display: 'flex',
    flexDirection: 'column',
    alignItems: 'center',
    justifyContent: 'center',
    padding: '20px',
    backfaceVisibility: phase === 'flip' ? 'visible' : 'visible',
  }}>
    <div style={{
      position: 'absolute',
      left: 0,
      top: '10px',
      bottom: '10px',
      width: '12px',
      background: `linear-gradient(90deg, ${bookColor} 0%, rgba(0,0,0,0.3) 50%, ${bookColor} 100%)`,
      borderRadius: '4px 0 0 4px',
    }} />

    <div style={{
      color: '#D4AF37',
      fontSize: '14px',
      fontWeight: 700,
      textAlign: 'center',
      textTransform: 'uppercase',
      letterSpacing: '2px',
      textShadow: '0 1px 3px rgba(0,0,0,0.5)',
      lineHeight: 1.3,
      maxWidth: '140px',
      wordBreak: 'break-word',
    }}>
      {courseCode || 'Course'}
    </div>
    {title && (
      <div style={{
        color: '#D4AF37',
        fontSize: '9px',
        fontWeight: 400,
        textAlign: 'center',
        marginTop: '8px',
        opacity: 0.8,
        maxWidth: '130px',
        overflow: 'hidden',
        textOverflow: 'ellipsis',
        whiteSpace: 'nowrap',
      }}>
        {title}
      </div>
    )}
  </div>
  <div style={{
    position: 'absolute',
    bottom: '15px',
    left: '20px',
    right: '20px',
    height: '2px',
    backgroundColor: '#D4AF37',
    opacity: 0.5,
  }} />
  <div style={{
    position: 'absolute',
    top: '15px',

```



```

        left: '20px',
        right: '20px',
        height: '2px',
        backgroundColor: '#D4AF37',
        opacity: 0.5,
      }} />
</div>

{
  (phase === 'pages' || phase === 'open') && (
    <div style={{
      position: 'absolute',
      inset: 0,
      transformStyle: 'preserve-3d',
    }}>
      {pageColors.map((color, i) => (
        <div key={i} style={{
          position: 'absolute',
          right: '4px',
          top: '4px',
          bottom: '4px',
          width: '170px',
          backgroundColor: color,
          borderRadius: '0 6px 6px 0',
          transformOrigin: 'left center',
          animation: 'pageFlip 0.2s ease-in-out ${i * 0.15}s forwards',
          boxShadow: '2px 0 4px rgba(0,0,0,0.1)',
          display: 'flex',
          flexDirection: 'column',
          padding: '12px 10px',
          gap: '4px',
        }}>
          {[...Array(12)].map((_, li) => (
            <div key={li} style={{
              height: '3px',
              backgroundColor: 'rgba(0,0,0,0.08)',
              borderRadius: '1px',
              width: `${60 + Math.random() * 35}%`,
            }} />
          ))}
        </div>
      ))}
    </div>
  )}
</div>

{
  (phase === 'open' || phase === 'done') && (
    <div style={{
      position: 'absolute',
      right: '5%',
      top: '50%',
      transform: 'translateY(-50%)',
      width: '400px',
      height: '500px',
      display: 'flex',
      opacity: phase === 'done' ? 1 : 0,
      transition: 'opacity 0.5s ease 0.3s',
    }}>
      <div style={{
        width: '30px',
        background: `linear-gradient(90deg, ${bookColor} 0%, rgba(0,0,0,0.4) 40%, ${bookColor} 80%, rgba(0,0,0,0.4) 100%)`,
        borderRadius: '4px 0 0 4px',
        boxShadow: 'inset -3px 0 8px rgba(0,0,0,0.3)',
      }} />
      <div style={{
        flex: 1,
        backgroundColor: '#fef3c7',
        borderRadius: '0 8px 8px 0',
        boxShadow: '4px 4px 20px rgba(0,0,0,0.3)',
        padding: '30px 25px',
        display: 'flex',
        flexDirection: 'column',
        gap: '6px',
        overflow: 'hidden',
      }}>
        <div style={{ fontSize: '16px', fontWeight: 700, color: '#1a1a1a', marginBottom: '10px', borderBottom: '1px solid #1a1a1a' }}>
          {courseCode} – {title || 'Reading'}
        </div>
        {[...Array(20)].map((_, i) => (
          <div key={i} style={{
            height: '8px',
            backgroundColor: 'rgba(0,0,0,0.06)',
            borderRadius: '2px',
            width: `${50 + Math.random() * 45}%`,
          }} />
        ))}
      </div>
    </div>
  )}
</div>,
document.body
);
}

```

`client/src/components/CourseDetailDialog.tsx`

```

import React, { useState, useMemo, useEffect, useCallback, useRef } from "react";
import { createPortal } from "react-dom";
import { useQuery, useMutation } from "@tanstack/react-query";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";
import {
  Plus,
  Trash2,

```

```

BookOpen,
Video,
Globe,
Mail,
User,
Clock,
Calendar,
CheckCircle2,
ExternalLink,
GraduationCap,
FileText,
MessageSquare,
ClipboardCheck,
AlertCircle,
Pencil,
Check,
Upload,
Loader2,
Paperclip,
X,
GripVertical,
FolderPlus,
Copy,
ChevronDown,
ChevronLeft,
ChevronRight,
ArrowUp,
ArrowDown,
Flag,
ClipboardList,
} from "lucide-react";
import zoomLogoPath from "@assets/Zoom2_1773776262533.png";
import wifiLogoPath from "@assets/wifi_1773656687145.png";
import { apiRequest, queryClient } from "@lib/queryClient";
import { useToast } from "@hooks/use-toast";
import { useUpload } from "@hooks/use-upload";
import { TASK_TYPES, getWeekNumber, getWeekDates, FIRST_WEEK, LAST_WEEK, REMINDER_OPTIONS, REPEAT_TYPES, REPEAT_INTER...
import type { Task, CourseWeekMapping } from "@shared/schema";

const TASK_TYPE_OPTIONS = [
  { value: "reading", label: "Reading", icon: BookOpen },
  { value: "essay", label: "Essay", icon: FileText },
  { value: "exam", label: "Exam", icon: AlertCircle },
  { value: "quiz", label: "Quiz", icon: ClipboardCheck },
  { value: "discussion", label: "Discussion Post", icon: MessageSquare },
  { value: "poll", label: "Review Poll", icon: ClipboardCheck },
  { value: "project", label: "Project", icon: FileText },
  { value: "module", label: "Module", icon: BookOpen },
  { value: "class", label: "Class", icon: Video },
  { value: "other", label: "Other", icon: FileText },
];

const REMINDER_PRESETS = [
  { value: 0, label: "None" },
  { value: 15, label: "15 min" },
  { value: 30, label: "30 min" },
  { value: 60, label: "1 hour" },
  { value: 120, label: "2 hours" },
  { value: 1440, label: "1 day" },
  { value: 2880, label: "2 days" },
  { value: 10080, label: "1 week" },
];

interface CourseInfo {
  courseCode: string;
  courseName: string;
  fullName: string;
  professor: string;
  professorEmail: string;
  color: string;
  colorEnd?: string;
  colorStops?: string;
  borderColor?: string;
  courseRowColor?: string;
  taskBgColor?: string;
  deliveryMode: string;
  classDay: string;
  classDay2?: string;
  classTime: string;
  classEndTime?: string;
  classTime2?: string;
  classEndTime2?: string;
  zoomLink?: string;
  courseType: string;
  startDate?: string;
  endDate?: string;
  semesterTerm?: string;
  year?: string;
}

interface CourseDetailDialogProps {
  courseInfo: CourseInfo;
  onClose: () => void;
  onSaveCourseInfo?: (updates: { professor?: string; professorEmail?: string; deliveryMode?: string; classDay?: strin...
  onLiveColorChange?: (updates: { color?: string; colorEnd?: string; colorStops?: string; borderColor?: string; cours...
  onGradeCalculated?: (grade: string, percent: string) => void;
  onDeleteCourse?: () => void;
  onOpenEditTask?: (task: Task) => void;
  semesterStart: Date;
  readingWeekStart: Date | null;
  certificateName?: string;
  onPushUndo?: (action: { type: string; description: string; data: any }) => void;
  initialEditMode?: boolean;
  courseRank?: number;
  usedRanks?: number[];
  isSpringSummer?: boolean;
  courseSpSuTerm?: string;
  courseRankA?: number;
}

```

```

courseRankB?: number;
usedRanksA?: number[];
usedRanksB?: number[];
onRankChange?: (suffix: string, val: number) => void;
semesterSettings?: Record<string, { week1StartDate?: string; springStartDate?: string; springEndDate?: string; summ...
}

interface NewTaskForm {
  title: string;
  type: string;
  dueDate: string;
  dueTime: string;
  description: string;
  reminder1: number;
  reminder2: number;
  reminder3: number;
  reminder4: number;
  gradeWeight: string;
  gradeTotal: string;
  gradeValue: string;
}

function DebouncedGradeInput({ value, onSave, placeholder, testId, disabled }: { value: number | null | undefined; on...
const fmt = (v: number | null | undefined) => v != null ? v.toFixed(2) : '';
const [local, setLocal] = useState(fmt(value));
const [editing, setEditing] = useState(false);
const lastAcceptedRef = useRef(value);

useEffect(() => {
  if (!editing && value !== lastAcceptedRef.current) {
    lastAcceptedRef.current = value;
    setLocal(fmt(value));
  }
}, [value, editing]);

return (
  <input
    type="text"
    inputMode="decimal"
    pattern="[0-9]*\\.?[0-9]*"
    className="w-[33px] h-[23px] text-[9px] text-center bg-transparent border border-amber-400/60 rounded text-whit...
    placeholder={placeholder}
    value={local}
    disabled={disabled}
    onFocus={() => setEditing(true)}
    onChange={(e) => setLocal(e.target.value)}
    onBlur={() => {
      const parsed = local !== '' ? parseFloat(String(local)) : null;
      const finalVal = parsed !== null && !isNaN(parsed) ? Math.round(parsed * 100) / 100 : null;
      const current = value ?? null;
      const changed = finalVal !== null ? current !== null : Math.abs((finalVal) - (current || 0)) > 0.001;
      if (changed) {
        lastAcceptedRef.current = finalVal;
        setLocal(fmt(finalVal));
        onSave(finalVal);
      }
      setEditing(false);
    }}
    onKeyDown={(e) => {
      if (e.key === 'Enter') (e.target as HTMLInputElement).blur();
    }}
    data-testid={testId}
  />
);
}

function createEmptyTaskForm(): NewTaskForm {
  return {
    title: "",
    type: "reading",
    dueDate: "",
    dueTime: "18:00",
    description: "",
    reminder1: 30,
    reminder2: 120,
    reminder3: 0,
    reminder4: 0,
    gradeWeight: "",
    gradeTotal: "",
    gradeValue: "",
  };
}

function percentToLetterGrade(pct: number): string {
  if (pct >= 90) return 'A+';
  if (pct >= 85) return 'A';
  if (pct >= 80) return 'A-';
  if (pct >= 77) return 'B+';
  if (pct >= 73) return 'B';
  if (pct >= 70) return 'B-';
  if (pct >= 67) return 'C+';
  if (pct >= 63) return 'C';
  if (pct >= 60) return 'C-';
  if (pct >= 50) return 'D';
  return 'F';
}

const SEMESTER_OPTIONS: { key: string; label: string; term: string; year: string; start: string; end: string }[] = [
  { key: 'ss2025', label: 'Spring/Summer 2025', term: 'spring_summer_full', year: '2025', start: '2025-05-05', end: '...',
  { key: 'f2025', label: 'Fall 2025', term: 'fall', year: '2025', start: '2025-09-08', end: '2025-12-07' },
  { key: 'w2026', label: 'Winter 2026', term: 'winter', year: '2026', start: '2026-01-13', end: '2026-04-17' },
  { key: 'ss2026', label: 'Spring/Summer 2026', term: 'spring_summer_full', year: '2026', start: '2026-05-04', end: '...',
  { key: 'f2026', label: 'Fall 2026', term: 'fall', year: '2026', start: '2026-09-14', end: '2026-12-07' },
  { key: 'w2027', label: 'Winter 2027', term: 'winter', year: '2027', start: '2027-01-11', end: '2027-04-16' },
  { key: 'ss2027', label: 'Spring/Summer 2027', term: 'spring_summer_full', year: '2027', start: '2027-05-03', end: '...',
  { key: 'f2027', label: 'Fall 2027', term: 'fall', year: '2027', start: '2027-09-13', end: '2027-12-06' },
  { key: 'w2028', label: 'Winter 2028', term: 'winter', year: '2028', start: '2028-01-10', end: '2028-04-14' },
  { key: 'ss2028', label: 'Spring/Summer 2028', term: 'spring_summer_full', year: '2028', start: '2028-05-01', end: '...'

```

```

    { key: 'f2028', label: 'Fall 2028', term: 'fall', year: '2028', start: '2028-09-11', end: '2028-12-04' },
    { key: 'w2029', label: 'Winter 2029', term: 'winter', year: '2029', start: '2029-01-08', end: '2029-04-13' },
  ];

function semesterKeyFromTermYear(term?: string, year?: string): string {
  if (!term || !year) return '';
  if (term.startsWith('spring_summer')) return `ss${year}`;
  if (term === 'fall') return `f${year}`;
  if (term === 'winter') return `w${year}`;
  return '';
}

export function CourseDetailDialog({ courseInfo, onClose, onSaveCourseInfo, onLiveColorChange, onGradeCalculated, onD...
  const { toast } = useToast();
  const [showAddForm, setShowAddForm] = useState(false);
  const [newTask, setNewTask] = useState<NewTaskForm>(createEmptyTaskForm());
  const [expandedTaskId, setExpandedTaskId] = useState<number | null>(null);
  const [editTaskFields, setEditTaskFields] = useState<any>(null);
  const [commentTarget, setCommentTarget] = useState<{ type: string; id: string; label: string } | null>(null);
  const [commentText, setCommentText] = useState('');
  const [commentLoading, setCommentLoading] = useState(false);
  const [commentSaving, setCommentSaving] = useState(false);
  const [notesTaskId, setNotesTaskId] = useState<number | null>(null);
  const [notesText, setNotesText] = useState('');
  const [notesSaving, setNotesSaving] = useState(false);
  const [dialogPos, setDialogPos] = useState<{ x: number; y: number }>({ x: 0, y: 0 });
  const dialogDragRef = useRef<{ dragging: boolean; startX: number; startY: number; origX: number; origY: number }>({...

  useEffect(() => {
    if (!commentTarget) return;
    setCommentLoading(true);
    fetch(`/api/entity-comments/${commentTarget.type}/${commentTarget.id}`)
      .then(r => r.json())
      .then(data => { setCommentText(Array.isArray(data) && data.length > 0 ? data[0].content : ''); })
      .catch(() => setCommentText(''))
      .finally(() => setCommentLoading(false));
  }, [commentTarget?.type, commentTarget?.id]);

  const saveComment = useCallback(() => {
    if (!commentTarget) return;
    setCommentSaving(true);
    fetch(`/api/entity-comments/${commentTarget.type}/${commentTarget.id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ content: commentText }),
    })
      .then(() => toast({ title: 'Comment saved' }))
      .catch(() => toast({ title: 'Failed to save', variant: 'destructive' }))
      .finally(() => setCommentSaving(false));
  }, [commentTarget, commentText, toast]);

  const [isEditingInfo, setIsEditingInfo] = useState(!initialEditMode);
  const [activeGradientStop, setActiveGradientStop] = useState<'start' | 'end' | number | null>(null);
  const [showDeleteConfirm, setShowDeleteConfirm] = useState(false);
  const [isParsingPdf, setIsParsingPdf] = useState(false);
  const [isParsingSyllabus, setIsParsingSyllabus] = useState(false);
  const [syllabusData, setSyllabusData] = useState<any>(null);
  const [syllabusItemStates, setSyllabusItemStates] = useState<Record<number, { accepted: boolean | null; editing: bo...
  const [syllabusObjectPath, setSyllabusObjectPath] = useState<string>('');

  useEffect(() => {
    fetch('/api/syllabus/paths')
      .then(r => r.json())
      .then(paths => {
        if (paths[courseInfo.courseCode]) {
          setSyllabusObjectPath(paths[courseInfo.courseCode]);
        } else {
          try {
            const saved = localStorage.getItem('courseSyllabusPaths');
            const local = saved ? JSON.parse(saved) : {};
            if (local[courseInfo.courseCode]) {
              setSyllabusObjectPath(local[courseInfo.courseCode]);
              fetch('/api/syllabus/paths', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ courseCode: courseInfo.courseCode, objectPath: local[courseInfo.courseCode] }),
              }).catch(() => {});
            }
          } catch {}
        }
      })
      .catch(() => {
        try {
          const saved = localStorage.getItem('courseSyllabusPaths');
          const local = saved ? JSON.parse(saved) : {};
          if (local[courseInfo.courseCode]) setSyllabusObjectPath(local[courseInfo.courseCode]);
        } catch {}
      });
  }, [courseInfo.courseCode]);

  const [showWeekMappings, setShowWeekMappings] = useState(false);
  const [showAssignments, setShowAssignments] = useState(!initialEditMode);
  const weekMappingsRef = useRef<HTMLDivElement>(null);
  const assignmentsRef = useRef<HTMLDivElement>(null);
  const [weekMappingEdits, setWeekMappingEdits] = useState<Record<number, { confirmed: boolean; courseWeekLabel: stri...
  const [weekStyleChoice, setWeekStyleChoice] = useState<string | null>(null);
  const [showWeekCalendar, setShowWeekCalendar] = useState(false);
  const [weekCalendarMonth, setWeekCalendarMonth] = useState(() => {
    const s = semesterStart ? new Date(semesterStart) : new Date();
    return new Date(s.getFullYear(), s.getMonth(), 1);
  });

  const [courseWeek1Start, setCourseWeek1Start] = useState<Date | null>(null);
  const [courseWeekLength, setCourseWeekLength] = useState(7);
  const [readingWeekVariable, setReadingWeekVariable] = useState(false);
  const [readingWeekExclusions, setReadingWeekExclusions] = useState<Set<number>>(new Set());
  const [showReadingWeekCalendar, setShowReadingWeekCalendar] = useState(false);
  const [readingWeekCalMonth, setReadingWeekCalMonth] = useState(new Date());
  const [selectedReadingWeekStart, setSelectedReadingWeekStart] = useState<Date | null>(null);
  const [weekUploadingState, setWeekUploadingState] = useState<Record<string, boolean>>({});
  const [courseWeekCalendarOpen, setCourseWeekCalendarOpen] = useState<number | null>(null);

```

```

const [courseWeekCalMonth, setCourseWeekCalMonth] = useState(new Date());

const handleWeekFileUpload = useCallback(async (weekNum: number, uploadType: 'module' | 'reading') => {
  const input = document.createElement('input');
  input.type = 'file';
  input.accept = '.pdf,.doc,.docx,.ppt,.pptx,.txt';
  input.onchange = async (e) => {
    const file = (e.target as HTMLInputElement).files?.[0];
    if (!file) return;
    const stateKey = `${weekNum}-${uploadType}`;
    setWeekUploadingState(prev => ({ ...prev, [stateKey]: true }));
    try {
      const weekDates = getWeekDates(weekNum, semesterStart, readingWeekStart);
      const ws = new Date(weekDates.start);
      const we = new Date(weekDates.end);
      const months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
      const startStr = `${months[ws.getMonth()]} ${ws.getDate()}`;
      const endStr = ws.getMonth() === we.getMonth() ? `${we.getDate()}` : `${months[we.getMonth()]} ${we.getDate()}`;
      const dateRange = `${startStr}-${endStr}`;

      const codeClean = courseInfo.courseCode.replace(/\\s/g, '');
      const namePart = courseInfo.fullName || courseInfo.courseName || '';

      const semYear = semesterStart ? new Date(semesterStart).getFullYear().toString() : '';
      const semMonth = semesterStart ? new Date(semesterStart).getMonth() : 0;
      const semTypeHeader = semMonth >= 8 ? 'fall' : semMonth >= 4 ? 'spring_summer' : 'winter';

      const resp = await fetch('/api/course-week-upload', {
        method: 'POST',
        headers: {
          'Content-Type': file.type || 'application/pdf',
          'x-course-code': codeClean,
          'x-course-name': namePart,
          'x-week-num': String(weekNum),
          'x-upload-type': uploadType,
          'x-week-date-range': dateRange,
          'x-file-name': file.name,
          'x-semester-year': semYear,
          'x-semester-type': semTypeHeader,
        },
        body: file,
      });
      const result = await resp.json();
      if (resp.ok && result.success) {
        toast({ title: `${uploadType} === 'module' ? 'Module' : 'Reading' uploaded`, description: `${file.name} sav...` });
      } else {
        toast({ title: 'Upload failed', description: result.error || 'Unknown error', variant: 'destructive' });
      }
    } catch (err: any) {
      toast({ title: 'Upload error', description: err.message, variant: 'destructive' });
    } finally {
      setWeekUploadingState(prev => ({ ...prev, [stateKey]: false }));
    }
  };
  input.click();
}, [courseInfo.courseCode, courseInfo.courseName, courseInfo.fullName, semesterStart, readingWeekStart, toast]);
const { uploadFile, isUploading } = useUpload();
const [editInfo, setEditInfo] = useState({
  professor: courseInfo.professor || '',
  professorEmail: courseInfo.professorEmail || '',
  deliveryMode: courseInfo.deliveryMode || '',
  classDay: courseInfo.classDay || '',
  classDay2: courseInfo.classDay2 || '',
  classTime: courseInfo.classTime || '',
  classEndTime: courseInfo.classEndTime || '',
  classTime2: courseInfo.classTime2 || '',
  classEndTime2: courseInfo.classEndTime2 || '',
  zoomLink: courseInfo.zoomLink || '',
  semesterTerm: courseInfo.semesterTerm || '',
  year: courseInfo.year || '',
  startDate: () => {
    if (courseInfo.startDate) return courseInfo.startDate;
    const semKey = semesterKeyFromTermYear(courseInfo.semesterTerm, courseInfo.year);
    const ss = semesterSettings?.[semKey];
    if (ss && semKey.startsWith('ss')) {
      const ssTerm = courseSpSuTerm || 'full';
      if (ssTerm === 'first_half' && ss.springStartDate) return ss.springStartDate;
      if (ssTerm === 'second_half' && ss.summerStartDate) return ss.summerStartDate;
      if (ssTerm === 'full' && ss.springStartDate) return ss.springStartDate;
      if (ss.week1StartDate) return ss.week1StartDate;
    }
    if (ss?.week1StartDate) return ss.week1StartDate;
    const opt = SEMESTER_OPTIONS.find(o => o.key === semKey);
    return opt?.start || '';
  },
  endDate: () => {
    if (courseInfo.endDate) return courseInfo.endDate;
    const semKey = semesterKeyFromTermYear(courseInfo.semesterTerm, courseInfo.year);
    const ss = semesterSettings?.[semKey];
    if (ss && semKey.startsWith('ss')) {
      const ssTerm = courseSpSuTerm || 'full';
      if (ssTerm === 'first_half' && ss.springEndDate) return ss.springEndDate;
      if (ssTerm === 'second_half' && ss.summerEndDate) return ss.summerEndDate;
      if (ssTerm === 'full' && ss.summerEndDate) return ss.summerEndDate;
    }
    const opt = SEMESTER_OPTIONS.find(o => o.key === semKey);
    return opt?.end || '';
  },
  color: courseInfo.color || '#3b82f6',
  colorEnd: courseInfo.colorEnd || courseInfo.color || '#3b82f6',
  colorStops: courseInfo.colorStops || '',
  borderColor: courseInfo.borderColor || '',
  courseRowColor: courseInfo.courseRowColor || '',
  taskBgColor: courseInfo.taskBgColor || '',
  courseRank: courseInfo.courseRank ?? 0,
  springSummerTerm: courseSpSuTerm || 'full',
});
const ssTermLocked = isSpringSummer && isEditingInfo && !editInfo.springSummerTerm;

```

```

const editFieldsLocked = isEditingInfo && (!editInfo.semesterTerm || (isSpringSummer && !editInfo.springSummerTerm));

useEffect(() => {
  if (onLiveColorChange && isEditingInfo) {
    onLiveColorChange({
      color: editInfo.color,
      colorEnd: editInfo.colorEnd,
      colorStops: editInfo.colorStops,
      borderColor: editInfo.borderColor,
      courseRowColor: editInfo.courseRowColor,
      taskBgColor: editInfo.taskBgColor,
    });
  }
}, [editInfo.color, editInfo.colorEnd, editInfo.colorStops, editInfo.borderColor, editInfo.courseRowColor, editInfo...

const CERTIFICATE_TYPE_OPTIONS = [
  { group: 'Certificate 1', options: [
    'C1 - Mandatory Required Professional',
    'C1 - Selected Required Professional',
    'C1 - Liberal Studies Elective - Lower Level Table A',
    'C1 - Professionally Related (Open) Elective Table 1',
  ]},
  { group: 'Certificate 2', options: [
    'C2 - Mandatory Required Professional',
    'C2 - Selected Required Professional',
    'C2 - Liberal Studies Elective - Lower Level Table A',
    'C2 - Professionally Related Elective - Upper Level Table B',
    'C2 - Open Elective',
  ]},
  { group: 'Certificate 3', options: [
    'C3 - Required Group 1',
    'C3 - Required Group 2',
    'C3 - Liberal Studies Elective Lower Level Table A',
    'C3 - Liberal Studies Elective Upper Level Table B',
    'C3 - Open Elective',
  ]},
];
const [certificateType, setCertificateType] = useState(() => {
  try {
    const saved = localStorage.getItem('courseCertificateTypes');
    const parsed = saved ? JSON.parse(saved) : {};
    return parsed[courseInfo.courseCode] || '';
  } catch { return ''; }
});
const updateCertificateType = (val: string) => {
  setCertificateType(val);
  try {
    const saved = localStorage.getItem('courseCertificateTypes');
    const parsed = saved ? JSON.parse(saved) : {};
    parsed[courseInfo.courseCode] = val;
    localStorage.setItem('courseCertificateTypes', JSON.stringify(parsed));
  } catch {}
};

const { data: allTasks = [] } = useQuery<Task[]>({
  queryKey: ["/api/tasks"],
});

const courseTasks = useMemo(() => {
  const codeUpper = courseInfo.courseCode.toUpperCase().replace(/s/g, '');
  const codeNoC = codeUpper.replace(/^(?=[A-Z]{2,})/, '');
  return allTasks
    .filter((t) => {
      if (t.type === 'class' || t.type === 'module') return false;
      if (!t.courseName) return false;
      const tCode = t.courseName.split(' - ')[0]?.trim().toUpperCase().replace(/s/g, '');
      return tCode === codeUpper || tCode === 'C' + codeUpper || tCode === codeNoC;
    })
    .sort((a, b) => (a.sortOrder || 0) - (b.sortOrder || 0) || new Date(a.dueDate).getTime() - new Date(b.dueDate)...
}, [allTasks, courseInfo.courseCode]);

const { data: weekMappingsData } = useQuery<CourseWeekMapping[]>({
  queryKey: ["/api/course-week-mappings", courseInfo.courseCode],
  queryFn: () => fetch(`/api/course-week-mappings/${courseInfo.courseCode}`).then(r => r.json()),
});

useEffect(() => {
  if (weekMappingsData && weekMappingsData.length > 0) {
    const edits: Record<number, { confirmed: boolean; courseWeekLabel: string; notes: string }> = {};
    for (const m of weekMappingsData) {
      edits[m.weekNumber] = { confirmed: m.confirmed ?? false, courseWeekLabel: m.courseWeekLabel || '', notes: m.n...
    }
    setWeekMappingEdits(prev => {
      const merged = { ...prev };
      for (const [k, v] of Object.entries(edits)) {
        if (!(Number(k) in merged)) merged[Number(k)] = v;
      }
      return merged;
    });
  }
}, [weekMappingsData]);

const saveWeekMapping = useCallback(async (weekNumber: number, data: { confirmed: boolean; courseWeekLabel: string;...
  try {
    await fetch('/api/course-week-mappings', {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ courseCode: courseInfo.courseCode, weekNumber, ...data }),
    });
    queryClient.invalidateQueries({ queryKey: ["/api/course-week-mappings", courseInfo.courseCode] });
  } catch {}
}, [courseInfo.courseCode]);

const confirmAllWeeks = useCallback(async () => {
  const mappings = [];
  for (let w = FIRST_WEEK; w <= LAST_WEEK; w++) {
    const existing = weekMappingEdits[w];
    mappings.push({ weekNumber: w, confirmed: true, courseWeekLabel: existing?.courseWeekLabel || '', notes: existi...

```

```

}
try {
  await fetch('/api/course-week-mappings/bulk', {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ courseCode: courseInfo.courseCode, mappings }),
  });
  const edits: Record<number, { confirmed: boolean; courseWeekLabel: string; notes: string }> = {};
  for (const m of mappings) edits[m.weekNumber] = { confirmed: true, courseWeekLabel: m.courseWeekLabel, notes: m...
  setWeekMappingEdits(edits);
  queryClient.invalidateQueries({ queryKey: ['/api/course-week-mappings', courseInfo.courseCode] });
  toast({ title: 'All weeks confirmed' });
} catch {}
}, [courseInfo.courseCode, weekMappingEdits, toast]);

type SortField = 'manual' | 'title' | 'dueDate' | 'score' | 'total' | 'weight' | 'percent';
const [sortField, setSortField] = useState<SortField>('manual');
const [sortDir, setSortDir] = useState<'asc' | 'desc'>('asc');

const toggleSort = (field: SortField) => {
  if (sortField === field) {
    if (sortDir === 'asc') setSortDir('desc');
    else { setSortField('manual'); setSortDir('asc'); }
  } else {
    setSortField(field);
    setSortDir('asc');
  }
};

const sortedTasks = useMemo(() => {
  const pushCompleted = (list: typeof courseTasks) => {
    const incomplete = list.filter(t => !t.isCompleted);
    const completed = list.filter(t => t.isCompleted);
    return [...incomplete, ...completed];
  };
  if (sortField === 'manual') return pushCompleted(courseTasks);
  const sorted = [...courseTasks].sort((a, b) => {
    let cmp = 0;
    switch (sortField) {
      case 'title': cmp = (a.title || '').localeCompare(b.title || ''); break;
      case 'dueDate': cmp = new Date(a.dueDate).getTime() - new Date(b.dueDate).getTime(); break;
      case 'score': cmp = (a.gradeValue || 0) - (b.gradeValue || 0); break;
      case 'total': cmp = (a.gradeTotal || 0) - (b.gradeTotal || 0); break;
      case 'weight': cmp = (a.gradeWeight || 0) - (b.gradeWeight || 0); break;
      case 'percent': {
        const pA = a.gradeValue != null && a.gradeTotal ? (a.gradeValue / a.gradeTotal) : -1;
        const pB = b.gradeValue != null && b.gradeTotal ? (b.gradeValue / b.gradeTotal) : -1;
        cmp = pA - pB;
        break;
      }
    }
    return sortDir === 'desc' ? -cmp : cmp;
  });
  return pushCompleted(sorted);
}, [courseTasks, sortField, sortDir]);

const [dragId, setDragId] = useState<number | null>(null);
const [dragOverId, setDragOverId] = useState<number | null>(null);
const [collapsedGroups, setCollapsedGroups] = useState<Set<string>>(() => new Set());
const [showGroupInput, setShowGroupInput] = useState(false);
const [newGroupName, setNewGroupName] = useState('');
const [assignToGroup, setAssignToGroup] = useState<number | null>(null);

const groups = useMemo(() => {
  const g = new Set<string>();
  courseTasks.forEach(t => { if (t.assignmentGroup) g.add(t.assignmentGroup); });
  return Array.from(g).sort();
}, [courseTasks]);

const ungroupedTasks = useMemo(() => sortedTasks.filter(t => !t.assignmentGroup), [sortedTasks]);

const groupedTasks = useMemo(() => {
  const map: Record<string, Task[]> = {};
  groups.forEach(g => { map[g] = sortedTasks.filter(t => t.assignmentGroup === g); });
  return map;
}, [sortedTasks, groups]);

const reorderMutation = useMutation({
  mutationFn: async (updates: { id: number; sortOrder: number; assignmentGroup?: string | null }[]) => {
    await apiRequest('POST', '/api/tasks/reorder', { updates });
  },
  onSuccess: () => { queryClient.invalidateQueries({ queryKey: ['/api/tasks'] }); },
});

const handleDragStart = (taskId: number) => { setDragId(taskId); };
const handleDragOver = (e: React.DragEvent, taskId: number) => { e.preventDefault(); setDragOverId(taskId); };
const handleDragEnd = () => { setDragId(null); setDragOverId(null); };

const handleDrop = (e: React.DragEvent, targetId: number, targetGroup?: string | null) => {
  e.preventDefault();
  if (dragId === null || dragId === targetId) { handleDragEnd(); return; }
  const taskList = targetGroup ? (groupedTasks[targetGroup] || []) : ungroupedTasks;
  const allList = [...taskList];
  const dragIdx = allList.findIndex(t => t.id === dragId);
  const targetIdx = allList.findIndex(t => t.id === targetId);
  if (dragIdx < 0) {
    const dragTask = courseTasks.find(t => t.id === dragId);
    if (dragTask) allList.splice(targetIdx, 0, dragTask);
  } else {
    const [moved] = allList.splice(dragIdx, 1);
    allList.splice(targetIdx, 0, moved);
  }
  const updates = allList.map((t, i) => ({ id: t.id, sortOrder: i, assignmentGroup: targetGroup ?? null }));
  reorderMutation.mutate(updates);
  handleDragEnd();
};

const handleDropOnGroup = (e: React.DragEvent, groupName: string) => {

```

```

    e.preventDefault();
    if (dragId === null) return;
    const existing = groupedTasks[groupName] || [];
    const updates = [{ id: dragId, sortOrder: existing.length, assignmentGroup: groupName }];
    reorderMutation.mutate(updates);
    handleDragEnd();
  };

  const [pendingGroups, setPendingGroups] = useState<string[]>([]);
  const allGroups = useMemo(() => {
    const merged = new Set([...groups, ...pendingGroups]);
    return Array.from(merged).sort();
  }, [groups, pendingGroups]);

  const createGroup = () => {
    if (!newGroupName.trim()) return;
    setPendingGroups(prev => [...prev, newGroupName.trim()]);
    setNewGroupName('');
    setShowGroupInput(false);
  };

  const assignTaskToGroup = (taskId: number, groupName: string | null) => {
    const updates = [{ id: taskId, sortOrder: 0, assignmentGroup: groupName }];
    reorderMutation.mutate(updates);
    setAssignToGroup(null);
  };

  const toggleGroupCollapse = (g: string) => {
    setCollapsedGroups(prev => { const n = new Set(prev); if (n.has(g)) n.delete(g); else n.add(g); return n; });
  };

  const completedCount = courseTasks.filter((t) => t.isCompleted).length;
  const totalWeight = courseTasks.reduce((s, t) => s + (t.gradeWeight || 0), 0);

  const gradeCalc = useMemo(() => {
    const gradedTasks = courseTasks.filter(t => !t.excludeFromGpa && t.gradeTotal && t.gradeValue !== null && t.grade...
    if (gradedTasks.length === 0) return null;
    const sumScore = gradedTasks.reduce((s, t) => s + (t.gradeValue || 0), 0);
    const sumTotal = gradedTasks.reduce((s, t) => s + (t.gradeTotal || 0), 0);
    const gradedWeight = gradedTasks.reduce((s, t) => s + (t.gradeWeight || 0), 0);
    const currentPercent = sumTotal > 0 ? (sumScore / sumTotal) * 100 : 0;
    return {
      currentPercent: Math.round(currentPercent * 100) / 100,
      currentGrade: percentToLetterGrade(currentPercent),
      gradedWeight,
      gradedCount: gradedTasks.length,
    };
  }, [courseTasks, totalWeight]);

  const onGradeCalculatedRef = useRef(onGradeCalculated);
  onGradeCalculatedRef.current = onGradeCalculated;

  useEffect(() => {
    if (onGradeCalculatedRef.current) {
      if (gradeCalc) {
        onGradeCalculatedRef.current(gradeCalc.currentGrade, String(gradeCalc.currentPercent));
      } else {
        onGradeCalculatedRef.current('', '');
      }
    }
  }, [gradeCalc?.currentPercent, gradeCalc?.currentGrade]);

  const createTaskMutation = useMutation({
    mutationFn: async (taskData: Record<string, any>) => {
      return apiRequest("POST", "/api/tasks", taskData);
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
      setNewTask(createEmptyTaskForm());
      setShowAddForm(false);
      toast({ title: "Assignment added", description: "Task created and added to your calendar." });
    },
    onError: () => {
      toast({ title: "Error", description: "Failed to create task.", variant: "destructive" });
    },
  });

  const toggleTaskMutation = useMutation({
    mutationFn: async ({ id, isCompleted, _task }: { id: number; isCompleted: boolean; _task?: any }) => {
      if (onPushUndo && _task) {
        onPushUndo({
          type: isCompleted ? 'complete' : 'uncomplete',
          description: `${isCompleted ? 'Completed' : 'Uncompleted'} "${_task.title}"`,
          data: { taskId: id, taskTitle: _task.title }
        });
      }
      const patch: Record<string, any> = { isCompleted };
      if (isCompleted) patch.excludeFromGpa = false;
      return apiRequest("PATCH", `/api/tasks/${id}`, patch);
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    },
  });

  const deleteTaskMutation = useMutation({
    mutationFn: async ({ id, _task }: { id: number; _task?: any }) => {
      if (onPushUndo && _task) {
        onPushUndo({
          type: 'delete',
          description: `Deleted "${_task.title}"`,
          data: { taskId: _task.id, taskTitle: _task.title, title: _task.title, description: _task.description, type: ...
        });
      }
      return apiRequest("DELETE", `/api/tasks/${id}`);
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    },
  });

```



```

    toast({ title: "Deleted", description: "Assignment removed." });
  },
});

const updateGradeValueMutation = useMutation({
  mutationFn: async ({ id, gradeValue, _task }: { id: number; gradeValue: number | null; _task?: any }) => {
    if (onPushUndo && _task) {
      onPushUndo({
        type: 'edit',
        description: `Changed grade for "${_task.title}"`,
        data: { taskId: id, taskTitle: _task.title, oldFields: { gradeValue: _task.gradeValue }, newFields: { grade...
      });
    }
    return apiRequest("PATCH", `/api/tasks/${id}`, { gradeValue });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
  },
});

const updateTaskMutation = useMutation({
  mutationFn: async ({ id, data, _task }: { id: number; data: Record<string, any>; _task?: any }) => {
    if (onPushUndo && _task) {
      const oldFields: Record<string, any> = {};
      const newFields: Record<string, any> = {};
      for (const key of Object.keys(data)) {
        const oldVal = (_task as any)[key];
        if (String(oldVal ?? '') !== String(data[key] ?? '')) {
          oldFields[key] = oldVal;
          newFields[key] = data[key];
        }
      }
      if (Object.keys(oldFields).length > 0) {
        onPushUndo({
          type: 'edit',
          description: `Updated "${_task.title}"`,
          data: { taskId: id, taskTitle: _task.title, oldFields, newFields }
        });
      }
    }
    return apiRequest("PATCH", `/api/tasks/${id}`, data);
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
  },
  onError: () => {
    toast({ title: "Error", description: "Failed to update task.", variant: "destructive" });
  },
});

const toggleFlagMutation = useMutation({
  mutationFn: async ({ id, flagged }: { id: number; flagged: boolean }) => {
    return apiRequest("PATCH", `/api/tasks/${id}/flag`, { flagged });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
  },
});

const handleUploadAssignment = async (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (!file) return;
  e.target.value = '';

  if (!file.name.toLowerCase().endsWith('.pdf')) {
    toast({ title: "Invalid file", description: "Please upload a PDF file.", variant: "destructive" });
    return;
  }

  setIsParsingPdf(true);
  toast({ title: "Uploading...", description: `Uploading ${file.name}` });

  try {
    const uploadResult = await uploadFile(file);
    if (!uploadResult) throw new Error("Upload failed");

    toast({ title: "Analyzing...", description: "AI is reading the assignment document..." });

    const parseResp = await fetch("/api/tasks/parse-assignment-pdf", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        objectPath: uploadResult.objectPath,
        courseName: courseInfo.fullName,
        fileName: file.name,
      }),
    });

    if (!parseResp.ok) {
      const err = await parseResp.json().catch(() => ({}));
      throw new Error(err.error || "Failed to parse assignment");
    }

    const parsed = await parseResp.json();

    const existingTask = courseTasks.find(t =>
      t.title.toLowerCase().includes(parsed.title?.toLowerCase()?.split(' - ')[0]?.trim() || '___') ||
      parsed.title?.toLowerCase().includes(t.title.toLowerCase().split(' - ')[0]?.trim() || '___')
    );

    if (existingTask) {
      await apiRequest("PATCH", `/api/tasks/${existingTask.id}`, {
        title: parsed.title || existingTask.title,
        description: parsed.description || existingTask.description,
        type: parsed.type || existingTask.type,
        gradeWeight: parsed.gradeWeight || existingTask.gradeWeight,
        attachments: [...(existingTask.attachments || []), uploadResult.objectPath],
      });
    }
  }

```

```

    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    toast({ title: "Assignment updated", description: `${parsed.title} updated with PDF attached.` });
  } else {
    await apiRequest("POST", "/api/tasks", {
      title: parsed.title || file.name.replace('.pdf', ''),
      description: parsed.description || "",
      type: parsed.type || "assignment",
      courseName: courseInfo.fullName,
      dueDate: new Date().toISOString(),
      priority: "high",
      weekNumber: 1,
      gradeWeight: parsed.gradeWeight || null,
      attachments: [uploadResult.objectPath],
      excludeFromGpa: true,
    });
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    toast({ title: "Assignment created", description: `${parsed.title} created with PDF attached. Please set th...` });
  }
} catch (err: any) {
  console.error("Upload assignment error:", err);
  toast({ title: "Error", description: err.message || "Failed to process assignment.", variant: "destructive" });
} finally {
  setIsParsingPdf(false);
}
};

const handleUploadSyllabus = async (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (!file) return;
  e.target.value = '';

  if (!file.name.toLowerCase().endsWith('.pdf')) {
    toast({ title: "Invalid file", description: "Please upload a PDF file.", variant: "destructive" });
    return;
  }

  setIsParsingSyllabus(true);
  toast({ title: "Uploading syllabus...", description: `Uploading ${file.name}` });

  try {
    console.log('[Syllabus] Starting direct upload for:', file.name, file.size, 'bytes');
    const uploadResp = await fetch('/api/uploads/direct', {
      method: 'POST',
      headers: {
        'Content-Type': file.type || 'application/pdf',
        'X-File-Name': file.name,
      },
      body: file,
    });
    if (!uploadResp.ok) {
      const errData = await uploadResp.json().catch(() => ({}));
      throw new Error(errData.error || 'Upload failed!');
    }
    const uploadResult = await uploadResp.json();
    console.log('[Syllabus] Upload success:', uploadResult.objectPath);
    if (!uploadResult?.objectPath) throw new Error("Upload failed - no object path returned");

    setSyllabusObjectPath(uploadResult.objectPath);
    try {
      await fetch('/api/syllabus/paths', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ courseCode: courseInfo.courseCode, objectPath: uploadResult.objectPath }),
      });
      console.log('[Syllabus] Path saved to server');
    } catch (e) { console.error('[Syllabus] Failed to save path to server:', e); }
    try {
      const saved = localStorage.getItem('courseSyllabusPaths');
      const local = saved ? JSON.parse(saved) : {};
      local[courseInfo.courseCode] = uploadResult.objectPath;
      localStorage.setItem('courseSyllabusPaths', JSON.stringify(local));
      console.log('[Syllabus] Path saved to local storage');
    } catch {}

    toast({ title: "Syllabus uploaded!", description: "Now analyzing with AI..." });

    const parseResp = await fetch("/api/syllabus/parse", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        objectPath: uploadResult.objectPath,
        courseName: courseInfo.fullName,
        courseCode: courseInfo.courseCode,
        fileName: file.name,
      }),
    });
    if (!parseResp.ok) {
      const err = await parseResp.json().catch(() => ({}));
      console.error('[Syllabus] Parse failed:', err);
      toast({ title: "Syllabus saved", description: "AI parsing failed but your syllabus PDF is attached. You can v..." });
      setIsParsingSyllabus(false);
      return;
    }

    const parsed = await parseResp.json();
    setSyllabusData(parsed);

    const allItems = [
      ...(parsed.items || []).map((item: any, i: number) => ({ ...item, _idx: i, _source: 'item' })),
      ...(parsed.gradingBreakdown || []).map((g: any, i: number) => ({ title: g.component, weight: g.weight, descri... })),
    ];

    const initialStates: Record<number, { accepted: boolean | null; editing: boolean; edits: any }> = {};
    allItems.forEach((item: any) => {
      initialStates[item._idx] = { accepted: null, editing: false, edits: { ...item } };
    });
  }
};

```

```

    if (parsed.weekNumbering) {
      initialStates[-1] = { accepted: null, editing: false, edits: { ...parsed.weekNumbering } };
    }

    setSyllabusItemStates(initialStates);

    if (parsed.courseInfo) {
      const ci = parsed.courseInfo;
      if (ci.professor || ci.professorEmail) {
        const updates: any = {};
        if (ci.professor && !courseInfo.professor) updates.professor = ci.professor;
        if (ci.professorEmail && !courseInfo.professorEmail) updates.professorEmail = ci.professorEmail;
        if (Object.keys(updates).length > 0 && onSaveCourseInfo) {
          onSaveCourseInfo(updates);
        }
      }
    }
  }
}

toast({ title: "Syllabus parsed!", description: `Found ${parsed.items?.length || 0} items. Review them below.` });
} catch (err: any) {
  console.error("Syllabus parse error:", err);
  toast({ title: "Error", description: err.message || "Failed to process syllabus.", variant: "destructive" });
} finally {
  setIsParsingSyllabus(false);
}
};

const handleAcceptSyllabusItem = async (idx: number) => {
  const state = syllabusItemStates[idx];
  if (!state) return;
  const item = state.edits;

  if (item._source === 'grading') {
    setSyllabusItemStates(prev => ({ ...prev, [idx]: { ...prev[idx], accepted: true } }));
    toast({ title: "Accepted", description: `${item.title} - ${item.weight}% noted.` });
    return;
  }

  try {
    const existingDupe = courseTasks.find(t =>
      t.title.trim().toLowerCase() === (item.title || '').trim().toLowerCase()
    );
    if (existingDupe) {
      setSyllabusItemStates(prev => ({ ...prev, [idx]: { ...prev[idx], accepted: true } }));
      toast({ title: "Already exists", description: `${item.title}` is already in your assignments.` });
      return;
    }

    let dueDate: Date;
    if (item.date) {
      dueDate = new Date(item.date);
      if (item.time) {
        const [h, m] = item.time.split(':').map(Number);
        dueDate.setHours(h, m, 0, 0);
      } else {
        dueDate.setHours(23, 59, 0, 0);
      }
    } else {
      dueDate = new Date();
      dueDate.setHours(23, 59, 0, 0);
    }

    await apiRequest("POST", "/api/tasks", {
      title: item.title,
      description: item.description || "",
      type: item.type || "other",
      courseName: courseInfo.fullName,
      dueDate: dueDate.toISOString(),
      priority: item.type === "exam" || item.type === "quiz" ? "high" : "medium",
      weekNumber: getWeekNumber(dueDate, semesterStart, readingWeekStart),
      reminder1: 30,
      reminder2: 120,
      gradeWeight: item.weight || null,
      excludeFromGpa: true,
    });

    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    setSyllabusItemStates(prev => ({ ...prev, [idx]: { ...prev[idx], accepted: true } }));
    toast({ title: "Added", description: `${item.title}` added to assignments and calendar.` });
  } catch (err: any) {
    toast({ title: "Error", description: err.message || "Failed to create task.", variant: "destructive" });
  }
}

const handleDeclineSyllabusItem = (idx: number) => {
  setSyllabusItemStates(prev => ({ ...prev, [idx]: { ...prev[idx], accepted: false } }));
};

const handleAcceptWeekNumbering = async (style: string) => {
  try {
    const resp = await fetch("/api/onedrive/rename-week-folders", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        courseCode: courseInfo.courseCode,
        courseName: courseInfo.courseName,
        weekStyle: style,
      }),
    });
    if (!resp.ok) throw new Error("Failed to update folders");
    const data = await resp.json();
    setSyllabusItemStates(prev => ({ ...prev, [-1]: { ...prev[-1], accepted: true } }));
    toast({ title: "Folders updated", description: data.message || `Week numbering set to "${style}"` });
  } catch (err: any) {
    toast({ title: "Error", description: err.message || "Failed to update week folders.", variant: "destructive" });
  }
}
};

```

```

const handleAddTask = () => {
  if (!newTask.title.trim() || !newTask.dueDate) {
    toast({ title: "Missing fields", description: "Title and due date are required.", variant: "destructive" });
    return;
  }

  const dueDate = new Date(newTask.dueDate);
  if (newTask.dueTime) {
    const [h, m] = newTask.dueTime.split(":").map(Number);
    dueDate.setHours(h, m, 0, 0);
  } else {
    dueDate.setHours(23, 59, 0, 0);
  }

  createTaskMutation.mutate({
    title: newTask.title,
    description: newTask.description || "",
    type: newTask.type || "other",
    courseName: courseInfo.fullName,
    dueDate: dueDate.toISOString(),
    priority: newTask.type === "exam" || newTask.type === "quiz" ? "high" : "medium",
    weekNumber: getWeekNumber(dueDate, semesterStart, readingWeekStart),
    reminder1: newTask.reminder1 || 30,
    reminder2: newTask.reminder2 || 120,
    reminder3: newTask.reminder3 || null,
    reminder4: newTask.reminder4 || null,
    gradeWeight: newTask.gradeWeight ? parseInt(newTask.gradeWeight) : null,
    gradeTotal: newTask.gradeTotal ? parseInt(newTask.gradeTotal) : null,
    gradeValue: newTask.gradeValue ? parseInt(newTask.gradeValue) : null,
    excludeFromGpa: true,
  });
};

const formatDate = (d: string | Date) => {
  const date = new Date(d);
  return date.toLocaleDateString("en-US", { weekday: "short", month: "short", day: "numeric" });
};

const formatTime = (d: string | Date) => {
  const date = new Date(d);
  return date.toLocaleTimeString("en-US", { hour: "numeric", minute: "2-digit" });
};

const isOverdue = (d: string | Date) => new Date(d) < new Date();

const deliveryLabel = courseInfo.deliveryMode === "virtual" ? "Virtual (Live Zoom)" : courseInfo.deliveryMode === "...

const renderAssignmentRow = (task: Task, currentGroup: string | null) => {
  const typeIcon = TASK_TYPE_OPTIONS.find((t) => t.value === task.type)?.icon || FileText;
  const overdue = !task.isCompleted && isOverdue(task.dueDate);
  const isDragging = dragId === task.id;
  const isDragOver = dragOverId === task.id;
  return (
    <React.Fragment key={task.id}>
      <div
        draggable
        onDragStart={() => handleDragStart(task.id)}
        onDragOver={(e) => handleDragOver(e, task.id)}
        onDrop={(e) => handleDrop(e, task.id, currentGroup)}
        onDragEnd={handleDragEnd}
        className={`flex items-center px-1.5 py-1.5 rounded-md border transition-all ${
          isDragging ? "opacity-40 border-blue-400/50" :
          isDragOver ? "border-blue-400 bg-blue-400/10" :
          task.isCompleted ? "bg-white/5 border-white/5" :
          overdue ? "bg-red-500/10 border-red-500/20" :
          "bg-white/5 border-white/10 hover:bg-white/8"
        }`}
      >
        <div className="flex-shrink-0 cursor-grab active:cursor-grabbing text-white/30 hover:text-white/60" style={{ ...
          <GripVertical className="h-3.5 w-3.5" />
        </div>
        <button
          onClick={(e) => { e.stopPropagation(); toggleTaskMutation.mutate({ id: task.id, isCompleted: !task.isComple...
          className={`flex-shrink-0 w-4 h-4 rounded-sm border-2 flex items-center justify-center transition-colors ${
            task.isCompleted ? "bg-green-500 border-green-500" : "border-white/30 hover:border-white/50"
          }`}
          style={{ marginRight: '10px' }}
          data-testid={`button-toggle-task-${task.id}`}
        >
          {task.isCompleted && <Check className="h-3 w-3 text-white" />}
        </button>
        {assignToGroup === task.id ? (
          <select
            className="h-5 text-[8px] bg-white/10 border border-white/20 rounded text-white px-0.5 flex-shrink-0"
            style={{ marginLeft: '3px', marginRight: '10px' }}
            value={task.assignmentGroup || ''}
            onChange={(e) => assignTaskToGroup(task.id, e.target.value || null)}
            autoFocus
            onBlur={() => setAssignToGroup(null)}
            onClick={(e) => e.stopPropagation()}
            data-testid={`select-group-${task.id}`}
          >
            <option value="">No Group</option>
            {allGroups.map(g => <option key={g} value={g}>{g}</option>)}
          </select>
        ) : (
          <button
            onClick={(e) => { e.stopPropagation(); setAssignToGroup(task.id); }}
            className="flex-shrink-0 text-white hover:text-white/60 transition-colors p-0.5"
            style={{ marginLeft: '3px', marginRight: '10px' }}
            title="Assign to group"
            data-testid={`button-assign-group-${task.id}`}
          >
            <FolderPlus className="h-[15px] w-[15px]" />
          </button>
        )}
      </div>
    </React.Fragment>
  );
};

```

```

        className={`h-[14px] w-[14px] flex-shrink-0 cursor-pointer transition-colors ${task.flagged ? 'text-red-400...' : ''}
        style={{ marginLeft: '8px', marginRight: '4px' }}
        onClick={(e) => { e.stopPropagation(); toggleFlagMutation.mutate({ id: task.id, flagged: !task.flagged }); }}
        data-testid={`flag-toggle-${task.id}`}
    />
    <MessageSquare className={`h-[19px] w-[19px] flex-shrink-0 cursor-pointer hover:opacity-70 transition-opacity...
    <Plus className={`h-[14px] w-[14px] flex-shrink-0 cursor-pointer hover:opacity-70 transition-opacity ${task.i...
    <div className="flex-1 min-w-0" style={{ marginLeft: '36px' }}>
        <div
            className={`text-[10px] font-medium truncate flex items-center gap-1 cursor-pointer hover:underline ${tas...
            onClick={(e) => {
                e.stopPropagation();
                e.preventDefault();
                if (expandedTaskId === task.id) {
                    setExpandedTaskId(null);
                    setEditTaskFields(null);
                } else {
                    setExpandedTaskId(task.id);
                    const d = task.dueDate ? new Date(task.dueDate) : null;
                    setEditTaskFields({
                        title: task.title || '',
                        type: task.type || 'other',
                        dueDate: d ? `${d.getFullYear()}-${String(d.getMonth()+1).padStart(2,'0')}-${String(d.getDate()).pa...
                        dueTime: d ? `${String(d.getHours()).padStart(2,'0')}:${String(d.getMinutes()).padStart(2,'0')}` : '',
                        description: task.description || '',
                        gradeWeight: task.gradeWeight?.toString() || '',
                        gradeTotal: task.gradeTotal?.toString() || '',
                        gradeValue: task.gradeValue?.toString() || '',
                        reminder1: task.reminder1 ?? 30,
                        reminder2: task.reminder2 ?? 120,
                        reminder3: task.reminder3 ?? null,
                        reminder4: task.reminder4 ?? null,
                    });
                }
            }}
            data-testid={`link-edit-task-${task.id}`}
        >
            {task.title}
            {task.attachments && task.attachments.length > 0 && (
                <Paperclip className="h-2.5 w-2.5 text-blue-400 flex-shrink-0 inline" />
            )}
        </div>
        <div className="flex items-center gap-2 text-[8px] text-white">
            <span className={overdue ? "text-red-400" : ""}>
                {formatDate(task.dueDate)} {formatTime(task.dueDate)}
            </span>
            <span className="capitalize">{task.type}</span>
        </div>
        <div
            className="flex items-center flex-shrink-0" style={{ gap: '10px', position: 'relative', left: '-8px' }} ...
            <DebounceGradeInput
                value={task.gradeValue}
                onSave={(val) => updateGradeValueMutation.mutate({ id: task.id, gradeValue: val, _task: task })}
                placeholder="Scr"
                testId={`input-grade-value-${task.id}`}
            />
            <DebounceGradeInput
                value={task.gradeTotal}
                onSave={(val) => updateTaskMutation.mutate({ id: task.id, data: { gradeTotal: val }, _task: task })}
                placeholder="Tot"
                testId={`input-grade-total-${task.id}`}
            />
            <DebounceGradeInput
                value={task.gradeWeight}
                onSave={(val) => updateTaskMutation.mutate({ id: task.id, data: { gradeWeight: val }, _task: task })}
                placeholder="Wt"
                testId={`input-grade-weight-${task.id}`}
            />
            <span className="text-[9px] text-white w-[33px] text-center" data-testid={`text-grade-percent-${task.id}`}>
                {task.gradeValue !== null && task.gradeValue !== undefined && task.gradeTotal ? `${((task.gradeValue / ta...
            </span>
        </div>
        <div className="flex items-center flex-shrink-0" style={{ gap: '10px', marginLeft: '8px' }} onClick={(e) => e...
            <label className="flex items-center cursor-pointer" title={task.excludeFromGpa ? "Excluded from grade" : "I...
                <div className="relative" onClick={() => updateTaskMutation.mutate({ id: task.id, data: { excludeFromGpa:...
                    <div className={`w-6 h-3.5 rounded-full transition-colors ${task.excludeFromGpa ? 'bg-red-500/60' : 'bg...
                        <div className={`absolute top-0.5 w-2.5 h-2.5 rounded-full bg-white transition-transform ${task.exclude...
                    </div>
                </label>
                <button
                    onClick={(e) => {
                        e.stopPropagation();
                        const { id, gradeValue, isCompleted, ...rest } = task;
                        createTaskMutation.mutate({
                            ...rest,
                            title: `${task.title} (copy)`,
                            isCompleted: false,
                            gradeValue: null,
                        });
                    }}
                    className="flex-shrink-0 text-white hover:text-blue-400 transition-colors p-0.5"
                    style={{ marginLeft: '6px' }}
                    title="Duplicate task"
                    data-testid={`button-duplicate-task-${task.id}`}
                >
                    <Copy className="h-[15px] w-[15px]" />
                </button>
                <button
                    onClick={(e) => { e.stopPropagation(); deleteTaskMutation.mutate({ id: task.id, _task: task }); }}
                    className="flex-shrink-0 text-white hover:text-red-400 transition-colors p-0.5"
                    data-testid={`button-delete-task-${task.id}`}
                >
                    <Trash2 className="h-[15px] w-[15px]" />
                </button>
            </div>
        </div>
        {expandedTaskId === task.id && editTaskFields && (
            <div className="bg-white/5 border border-white/15 rounded-lg p-3 mb-1 space-y-2 ml-6" data-testid={`inline-ed...

```

```

<div className="grid grid-cols-2 gap-2">
  <div>
    <Label className="text-[9px] text-white mb-0.5 block">Title</Label>
    <Input value={editTaskFields.title} onChange={(e) => setEditTaskFields({ ...editTaskFields, title: e.ta...
  </div>
  <div>
    <Label className="text-[9px] text-white mb-0.5 block">Type</Label>
    <select value={editTaskFields.type} onChange={(e) => setEditTaskFields({ ...editTaskFields, type: e.tar...
      {TASK_TYPE_OPTIONS.map((t) => (<option key={t.value} value={t.value} className="bg-gray-800">{t.label}...
    </select>
  </div>
</div>
<div className="grid grid-cols-2 gap-2">
  <div>
    <Label className="text-[9px] text-white mb-0.5 block">Due Date</Label>
    <Input type="date" value={editTaskFields.dueDate} onChange={(e) => setEditTaskFields({ ...editTaskField...
  </div>
  <div>
    <Label className="text-[9px] text-white mb-0.5 block">Due Time</Label>
    <Input type="time" value={editTaskFields.dueTime} onChange={(e) => setEditTaskFields({ ...editTaskField...
  </div>
</div>
<div>
  <Label className="text-[9px] text-white mb-0.5 block">Description</Label>
  <Input value={editTaskFields.description} onChange={(e) => setEditTaskFields({ ...editTaskFields, descrip...
</div>
<div className="grid grid-cols-3 gap-2">
  <div>
    <Label className="text-[9px] text-white mb-0.5 block">Grade Weight (%)</Label>
    <Input type="number" value={editTaskFields.gradeWeight} onChange={(e) => setEditTaskFields({ ...editTas...
  </div>
  <div>
    <Label className="text-[9px] text-white mb-0.5 block">Total Points</Label>
    <Input type="number" value={editTaskFields.gradeTotal} onChange={(e) => setEditTaskFields({ ...editTask...
  </div>
  <div>
    <Label className="text-[9px] text-white mb-0.5 block">Score Earned</Label>
    <Input type="number" value={editTaskFields.gradeValue} onChange={(e) => setEditTaskFields({ ...editTask...
  </div>
</div>
<div className="grid grid-cols-2 gap-2">
  <div>
    <Label className="text-[9px] text-white mb-0.5 block">Reminder 1</Label>
    <select value={editTaskFields.reminder1 ?? ''} onChange={(e) => setEditTaskFields({ ...editTaskFields, ...
      [REMINDER_PRESETS.map((r) => (<option key={r.value} value={r.value} className="bg-gray-800">{r.label}...
    </select>
  </div>
  <div>
    <Label className="text-[9px] text-white mb-0.5 block">Reminder 2</Label>
    <select value={editTaskFields.reminder2 ?? ''} onChange={(e) => setEditTaskFields({ ...editTaskFields, ...
      [REMINDER_PRESETS.map((r) => (<option key={r.value} value={r.value} className="bg-gray-800">{r.label}...
    </select>
  </div>
</div>
<div className="grid grid-cols-2 gap-2">
  <div>
    <Label className="text-[9px] text-white mb-0.5 block">Reminder 3</Label>
    <select value={editTaskFields.reminder3 ?? ''} onChange={(e) => setEditTaskFields({ ...editTaskFields, ...
      [REMINDER_PRESETS.map((r) => (<option key={r.value} value={r.value} className="bg-gray-800">{r.label}...
    </select>
  </div>
  <div>
    <Label className="text-[9px] text-white mb-0.5 block">Reminder 4</Label>
    <select value={editTaskFields.reminder4 ?? ''} onChange={(e) => setEditTaskFields({ ...editTaskFields, ...
      [REMINDER_PRESETS.map((r) => (<option key={r.value} value={r.value} className="bg-gray-800">{r.label}...
    </select>
  </div>
</div>
<div className="flex items-center gap-2 pt-1">
  <Label className="flex items-center gap-2 cursor-pointer" data-testid={`toggle-hide-summary-${task.id}`}>
    <input
      type="checkbox"
      checked={editTaskFields.hideFromSummary}
      onChange={(e) => setEditTaskFields({ ...editTaskFields, hideFromSummary: e.target.checked })}
      className="w-3.5 h-3.5 rounded accent-red-500"
    </>
    <span className="text-[10px] text-white/70">Hide from summary rows & homework box</span>
  </Label>
</div>
<div className="flex items-center justify-end gap-3 pt-2">
  <Button size="sm" variant="ghost" onClick={() => { setExpandedTaskId(null); setEditTaskFields(null); }} c...
  <Button
    size="sm"
    onClick={() => {
      const updates: Record<string, any> = {};
      if (editTaskFields.title !== task.title) updates.title = editTaskFields.title;
      if (editTaskFields.type !== task.type) updates.type = editTaskFields.type;
      if (editTaskFields.description !== (task.description || '')) updates.description = editTaskFields.des...
      if (editTaskFields.dueDate) {
        const dd = new Date(editTaskFields.dueDate);
        if (editTaskFields.dueTime) {
          const [h, m] = editTaskFields.dueTime.split(':').map(Number);
          dd.setHours(h, m, 0, 0);
        } else {
          dd.setHours(23, 59, 0, 0);
        }
      }
      updates.dueDate = dd.toISOString();
    }
    const gw = editTaskFields.gradeWeight ? parseInt(editTaskFields.gradeWeight) : null;
    const gt = editTaskFields.gradeTotal ? parseInt(editTaskFields.gradeTotal) : null;
    const gv = editTaskFields.gradeValue ? parseInt(editTaskFields.gradeValue) : null;
    if (gw !== (task.gradeWeight ?? null)) updates.gradeWeight = gw;
    if (gt !== (task.gradeTotal ?? null)) updates.gradeTotal = gt;
    if (gv !== (task.gradeValue ?? null)) updates.gradeValue = gv;
    if (editTaskFields.reminder1 !== (task.reminder1 ?? null)) updates.reminder1 = editTaskFields.reminder1;
    if (editTaskFields.reminder2 !== (task.reminder2 ?? null)) updates.reminder2 = editTaskFields.reminder2;
    if (editTaskFields.reminder3 !== (task.reminder3 ?? null)) updates.reminder3 = editTaskFields.reminder3;
    if (editTaskFields.reminder4 !== (task.reminder4 ?? null)) updates.reminder4 = editTaskFields.reminder4;
  </Button>
</div>

```

```

        if (editTaskFields.hideFromSummary !== (task.hideFromSummary ?? false)) updates.hideFromSummary = edi...
        if (Object.keys(updates).length > 0) {
            updateTaskMutation.mutate({ id: task.id, data: updates, _task: task });
        }
        setExpandedTaskId(null);
        setEditTaskFields(null);
        toast({ title: "Task updated" });
    }}
    className="h-9 px-5 text-[13px] font-semibold bg-blue-600 hover:bg-blue-500 text-white"
    data-testid={`button-inline-save-${task.id}`}
  >
    Save Changes
  </Button>
</div>
</div>
)}
</React.Fragment>
);
};

const handleDialogDragStart = (e: React.MouseEvent | React.TouchEvent) => {
  const clientX = 'touches' in e ? e.touches[0].clientX : e.clientX;
  const clientY = 'touches' in e ? e.touches[0].clientY : e.clientY;
  dialogDragRef.current = { dragging: true, startX: clientX, startY: clientY, origX: dialogPos.x, origY: dialogPos.y };

  const handleMove = (ev: MouseEvent | TouchEvent) => {
    if (!dialogDragRef.current.dragging) return;
    const cx = 'touches' in ev ? ev.touches[0].clientX : ev.clientX;
    const cy = 'touches' in ev ? ev.touches[0].clientY : ev.clientY;
    const dx = cx - dialogDragRef.current.startX;
    const dy = cy - dialogDragRef.current.startY;
    setDialogPos({ x: dialogDragRef.current.origX + dx, y: dialogDragRef.current.origY + dy });
  };
  const handleUp = () => {
    dialogDragRef.current.dragging = false;
    window.removeEventListener('mousemove', handleMove);
    window.removeEventListener('mouseup', handleUp);
    window.removeEventListener('touchmove', handleMove);
    window.removeEventListener('touchend', handleUp);
  };
  window.addEventListener('mousemove', handleMove);
  window.addEventListener('mouseup', handleUp);
  window.addEventListener('touchmove', handleMove, { passive: false });
  window.addEventListener('touchend', handleUp);
};

return createPortal(
  <div
    className="fixed inset-0 z-[10003] flex items-center justify-center"
    style={{ background: "rgba(0,0,0,0.5)", pointerEvents: 'none' }}
    data-testid="course-detail-overlay"
  >
    <div
      className="flex flex-col text-white rounded-lg"
      style={{
        pointerEvents: 'auto',
        width: "960px",
        maxWidth: "95vw",
        height: "88vh",
        background: 'linear-gradient(180deg, #3a8bbf 0%, color-mix(in srgb, #16a772 70%, black) 100%)',
        border: '1.5px solid rgba(255,255,255,0.35)',
        boxShadow: '0 8px 32px rgba(0,0,0,0.2), inset 0 1px 0 rgba(255,255,255,0.3), inset 0 -1px 0 rgba(0,0,0,0.05)',
        transform: `translate(${dialogPos.x}px, ${dialogPos.y}px)`,
      }}
      data-testid="course-detail-dialog"
    >
      <div
        className="flex items-center justify-between px-4 py-3 border-b border-white/40 flex-shrink-0 rounded-t-lg"
        onMouseDown={handleDialogDragStart}
        onTouchStart={handleDialogDragStart}
        style={{
          cursor: 'grab',
          backdropFilter: 'blur(30px)',
          WebkitBackdropFilter: 'blur(30px)',
          background: `linear-gradient(180deg, rgba(255,255,255,0.28) 0%, ${editInfo.color || courseInfo.color}cc 4...`,
          boxShadow: 'inset 0 1px 0 rgba(255,255,255,0.45), inset 0 2px 4px rgba(255,255,255,0.15), inset 0 -1px 0 ...',
        }}
      >
        <div className="flex items-center gap-2 min-w-0">
          {courseInfo.courseType && (
            <span className="text-[9px] text-white px-1.5 rounded-md border border-white/30 bg-white/10 flex-shrink-0">
              {courseInfo.courseType === "core" ? "Core" : courseInfo.courseType === "open_elective" ? "Elective" : ...}
            </span>
          )}
          <GraduationCap className="text-white flex-shrink-0" style={{ width: '15px', height: '15px' }} />
        </div>
        <div className="min-w-0 flex items-center gap-2">
          <h2
            className="font-normal text-white truncate"
            style={{ fontFamily: "Avenir, 'Avenir Next', -apple-system, BlinkMacSystemFont, sans-serif", textShad...
            data-testid="text-course-title"
          >
            {courseInfo.courseCode} – {courseInfo.courseName}
          </h2>
          <div>
            {gradeCalc && (
              <div className="flex items-center gap-1.5 px-2 py-0.5 rounded-md flex-shrink-0" style={{ background: ...
                <span className="text-[11px] font-bold text-white" data-testid="text-current-grade">{gradeCalc.curr...
                <span className="text-[10px] text-white">{gradeCalc.currentPercent.toFixed(1)}%</span>
                <span className="text-[9px] text-white/70" style={{ marginLeft: '2px' }}>{gradeCalc.gradedCount} gr...
              </div>
            )}
          </div>
        </div>
      </div>
    </div>
  );
};

```

```

    </div>
  </div>
  <div className="flex items-center justify-between px-3 py-2 border-b border-white/10 flex-shrink-0">
    <div className="flex items-center gap-2">
      <span className="text-[11px] text-white uppercase font-medium">Course Info</span>
      {syllabusObjectPath && (
        <Button
          variant="outline"
          size="sm"
          onClick={() => {
            window.open(`/api/syllabus/view?path=${encodeURIComponent(syllabusObjectPath)}`, '_blank');
          }}
          className="px-2 py-0 text-[10px] text-white border-white/40 hover:bg-white/15 hover:text-white bg-whi...
          style={{ height: '22px', minHeight: '22px', maxHeight: '22px' }}
          data-testid="button-view-syllabus"
        >
          <Paperclip className="w-3 h-3 mr-1" />
          View Syllabus
        </Button>
      )}
    </div>
    {isSpringSummer ? (
      <div>
        {(courseSpSuTerm === 'first_half' || courseSpSuTerm === 'full') && (
          <div>
            <span className="text-[9px] text-white">Rank A</span>
            <div className="flex items-center gap-0.5 px-1.5 py-0.5 rounded-md" style={{ background: 'rgba(25...
              <select className="h-4 text-[10px] bg-transparent border-none text-white rounded px-0.5 outline...
                const val = parseInt(e.target.value);
                if (onRankChange) onRankChange('A', val);
              </select>
              <div data-testid="select-course-rank-header-A">
                <option value={0} className="bg-gray-800"></option>
                {[1, 2, 3].map(n => {
                  const taken = (usedRanksA || []).includes(n) && (courseRankA ?? 0) !== n;
                  return <option key={n} value={n} className="bg-gray-800" disabled={taken} style={taken ? { ...
                })}
              </div>
            </div>
          )}
        )}
        {(courseSpSuTerm === 'second_half' || courseSpSuTerm === 'full') && (
          <div>
            <span className="text-[9px] text-white">Rank B</span>
            <div className="flex items-center gap-0.5 px-1.5 py-0.5 rounded-md" style={{ background: 'rgba(25...
              <select className="h-4 text-[10px] bg-transparent border-none text-white rounded px-0.5 outline...
                const val = parseInt(e.target.value);
                if (onRankChange) onRankChange('B', val);
              </select>
              <div data-testid="select-course-rank-header-B">
                <option value={0} className="bg-gray-800"></option>
                {[1, 2, 3].map(n => {
                  const taken = (usedRanksB || []).includes(n) && (courseRankB ?? 0) !== n;
                  return <option key={n} value={n} className="bg-gray-800" disabled={taken} style={taken ? { ...
                })}
              </div>
            </div>
          )}
        )}
      </div>
    ) : (
      <div>
        <span className="text-[9px] text-white">Rank</span>
        <div className="flex items-center gap-0.5 px-1.5 py-0.5 rounded-md" style={{ background: 'rgba(255,25...
          <select className="h-4 text-[10px] bg-transparent border-none text-white rounded px-0.5 outline-non...
            const val = parseInt(e.target.value);
            onSaveCourseInfo?.({ courseRank: val });
          </select>
          <div data-testid="select-course-rank-header">
            <option value={0} className="bg-gray-800"></option>
            {[1, 2, 3].map(n => {
              const taken = (usedRanks || []).includes(n) && courseRank !== n;
              return <option key={n} value={n} className="bg-gray-800" disabled={taken} style={taken ? { colo...
            })}
          </div>
        </div>
      </div>
    )}
  </div>
  {!isEditingInfo ? (
    <div className="flex items-center gap-2">
      <button
        onClick={() => setIsEditingInfo(true)}
        className="flex items-center gap-1.5 text-[11px] text-white hover:text-white transition-colors fo...
        data-testid="button-edit-course-info"
      >
        Edit
      </button>
      <Pencil className="w-4 h-4" style={{ color: courseInfo.colorEnd || courseInfo.color || '#3b82f6' ...
      </button>
    </div>
  ) : (
    <div className="flex items-center gap-1.5">
      <button
        onClick={() => {
          setEditInfo({
            professor: courseInfo.professor || '',
            professorEmail: courseInfo.professorEmail || '',
            deliveryMode: courseInfo.deliveryMode || '',
            classDay: courseInfo.classDay || '',
            classDay2: courseInfo.classDay2 || '',
            classTime: courseInfo.classTime || '',
            classEndTime: courseInfo.classEndTime || '',
            classTime2: courseInfo.classTime2 || '',
            classEndTime2: courseInfo.classEndTime2 || '',
            zoomLink: courseInfo.zoomLink || '',
            startDate: courseInfo.startDate || '',
            endDate: courseInfo.endDate || '',
            color: courseInfo.color || '#3b82f6',

```



```

        colorEnd: courseInfo.colorEnd || courseInfo.color || '#3b82f6',
        colorStops: courseInfo.colorStops || '',
        borderColor: courseInfo.borderColor || '',
        courseRowColor: courseInfo.courseRowColor || '',
        taskBgColor: courseInfo.taskBgColor || '',
    });
    if (onLiveColorChange) {
        onLiveColorChange({
            color: courseInfo.color || '#3b82f6',
            colorEnd: courseInfo.colorEnd || courseInfo.color || '#3b82f6',
            colorStops: courseInfo.colorStops || '',
            borderColor: courseInfo.borderColor || '',
            courseRowColor: courseInfo.courseRowColor || '',
            taskBgColor: courseInfo.taskBgColor || '',
        });
    }
    setIsEditingInfo(false);
}
className="text-[11px] text-white hover:text-white transition-colors px-3 py-1 rounded border bor...
data-testid="button-cancel-edit-info"
>
</div>
</button>
<button
    onClick={() => {
        if (editInfo.deliveryMode === 'virtual' && !editInfo.zoomLink?.trim()) {
            toast({ title: "URL is required for virtual courses", variant: "destructive" });
            return;
        }
        if (isSpringSummer && !editInfo.springSummerTerm) {
            toast({ title: "Please select a Spring/Summer term first", variant: "destructive" });
            return;
        }
        if (onSaveCourseInfo) {
            const semKey = semesterKeyFromTermYear(editInfo.semesterTerm, editInfo.year);
            onSaveCourseInfo({ ...editInfo, semesterKey: semKey || undefined, courseRank: editInfo.course...
        }
        if (editInfo.professor?.trim()) {
            fetch('/api/key-contacts/sync-professor', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ professorName: editInfo.professor, professorEmail: editInfo.professo...
            }).catch(() => {});
        }
        setIsEditingInfo(false);
        toast({ title: "Course info updated" });
    }}
    className="flex items-center gap-1 text-[11px] text-emerald-600 font-medium transition-colors px-...
    data-testid="button-save-edit-info"
>
<Check className="w-3 h-3" />
Save
</button>
</div>
))
</div>
<div className="flex-1 overflow-y-auto" style={{ scrollbarWidth: "thin", scrollbarColor: "rgba(255,255,255,0....
    <div className="p-3 space-y-2">
        {isEditingInfo ? (
            <div className="space-y-2 text-[10px]">
                <div className="flex items-center gap-1.5 justify-start">
                    <label className="cursor-pointer" data-testid="button-upload-assignment">
                        <input
                            type="file"
                            accept=".pdf"
                            className="hidden"
                            onChange={handleUploadAssignment}
                            disabled={isParsingPdf || isUploading}
                        />
                        <div className={`h-6 px-2 text-[9px] bg-blue-600/30 hover:bg-blue-600/50 text-white border bord...
                            {isParsingPdf || isUploading ? <Loader2 className="h-3 w-3 animate-spin" /> : <Upload classNa...
                            {isParsingPdf ? 'Parsing...' : isUploading ? 'Uploading...' : 'Upload PDF'}
                        </div>
                    </label>
                    <label className={`cursor-pointer ${syllabusObjectPath ? 'opacity-40 pointer-events-none' : ''}`}>...
                        <input
                            type="file"
                            accept=".pdf"
                            className="hidden"
                            onChange={handleUploadSyllabus}
                            disabled={isParsingSyllabus || isUploading || !syllabusObjectPath}
                        />
                        <div className={`h-6 px-2 text-[9px] ${syllabusObjectPath ? 'bg-gray-500/40 text-white/50 borde...
                            {isParsingSyllabus ? <Loader2 className="h-3 w-3 animate-spin" /> : <FileText className="h-3 ...
                            {isParsingSyllabus ? 'Parsing...' : 'Add Syllabus'}
                        </div>
                    </label>
                    <Paperclip
                        className={`h-3.5 w-3.5 ${syllabusObjectPath ? 'text-white/40' : 'text-white'} ${syllabusObject...
                        onClick={() => {
                            if (syllabusObjectPath) window.open(`/api/syllabus/view?path=${encodeURIComponent(syllabusObj...
                        )}
                        data-testid="button-view-syllabus-edit"
                    </>
                </div>
            </div>
        ) : (
            <Trash2
                className={`h-3.5 w-3.5 ${syllabusObjectPath ? 'text-white cursor-pointer hover:opacity-70' : '...'
                onClick={async () => {
                    if (!syllabusObjectPath) return;
                    setSyllabusObjectPath('');
                    try {
                        await fetch('/api/syllabus/paths', {
                            method: 'POST',
                            headers: { 'Content-Type': 'application/json' },
                            body: JSON.stringify({ courseCode: courseInfo.courseCode, objectPath: '' }),
                        });
                        const saved = localStorage.getItem('courseSyllabusPaths');
                        const local = saved ? JSON.parse(saved) : {};
                        delete local[courseInfo.courseCode];
                    } catch {

```

```

        localStorage.setItem('courseSyllabusPaths', JSON.stringify(local));
      } catch {}
      toast({ title: "Syllabus removed" });
    }
    data-testid="button-delete-syllabus"
  }
</div>
<div className="grid grid-cols-3 gap-2">
  <div>
    <label className="text-white text-[9px] mb-0.5 block font-semibold">Semester <span className="tex...
    <select className="w-full h-6 text-[10px] bg-white/10 border border-white/15 text-white rounded p...
      <option value="" className="bg-gray-800"></option>
      {SEMESTER_OPTIONS.map(o => <option key={o.key} value={o.key} className="bg-gray-800">{o.label}<...
    </select>
    {!editInfo.semesterTerm && <p className="text-[8px] text-yellow-400 mt-0.5">Select a semester fir...
  </div>
  <div className={!editInfo.semesterTerm ? 'opacity-30 pointer-events-none' : ''}>
    <label className="text-white text-[9px] mb-0.5 block">Start</label>
    <input type="date" className="w-full h-6 text-[10px] bg-white/10 border border-white/15 rounded p...
  </div>
  <div className={!editInfo.semesterTerm ? 'opacity-30 pointer-events-none' : ''}>
    <label className="text-white text-[9px] mb-0.5 block">End</label>
    <input type="date" className="w-full h-6 text-[10px] bg-white/10 border border-white/15 rounded p...
  </div>
</div>
<div>
  {isSpringSummer && (
    <div className="mb-2">
      <label className="text-white text-[9px] mb-0.5 block font-semibold">Spring/Summer Term <span clas...
      <select className="w-full h-6 text-[10px] bg-white/10 border border-white/15 text-white rounded p...
        const term = e.target.value;
        const semKey = semesterKeyFromTermYear(editInfo.semesterTerm, editInfo.year);
        const ss = semesterSettings?.[semKey];
        let newStart = editInfo.startDate;
        let newEnd = editInfo.endDate;
        if (ss) {
          if (term === 'first_half') {
            if (ss.springStartDate) newStart = ss.springStartDate;
            if (ss.springEndDate) newEnd = ss.springEndDate;
          } else if (term === 'second_half') {
            if (ss.summerStartDate) newStart = ss.summerStartDate;
            if (ss.summerEndDate) newEnd = ss.summerEndDate;
          } else if (term === 'full') {
            if (ss.springStartDate) newStart = ss.springStartDate;
            else if (ss.week1StartDate) newStart = ss.week1StartDate;
            if (ss.summerEndDate) newEnd = ss.summerEndDate;
          }
        }
        setEditInfo({...editInfo, springSummerTerm: term, startDate: newStart, endDate: newEnd });
      }} data-testid="select-edit-spsu-term">
      <option value="" className="bg-gray-800">-- Select Term First --</option>
      <option value="first_half" className="bg-gray-800">Spring (First Half) – May-Jun</option>
      <option value="second_half" className="bg-gray-800">Summer (Second Half) – Jun-Aug</option>
      <option value="full" className="bg-gray-800">Full Semester – May-Aug</option>
    </select>
    {!editInfo.springSummerTerm && <p className="text-[8px] text-yellow-400 mt-0.5">You must select a...
  </div>
)}
<div className={`grid grid-cols-2 gap-2 ${editFieldsLocked ? 'opacity-30 pointer-events-none' : ''}`>
  <div>
    <label className="text-white text-[9px] mb-0.5 block">Professor</label>
    <input className="w-full h-6 text-[10px] bg-white/10 border border-white/15 text-white rounded px...
  </div>
  <div>
    <label className="text-white text-[9px] mb-0.5 block">Email</label>
    <input className="w-full h-6 text-[10px] bg-white/10 border border-white/15 text-white rounded px...
  </div>
</div>
<div className={`grid grid-cols-[auto_1fr] gap-2 ${editFieldsLocked ? 'opacity-30 pointer-events-none' : ''}`>
  <div>
    <label className="text-white text-[9px] mb-0.5 block">Delivery Mode</label>
    <select className="w-full h-6 text-[10px] bg-white/10 border border-white/15 text-white rounded p...
      <option value="" className="bg-gray-800">Not set</option>
      <option value="virtual" className="bg-gray-800">Virtual (Live Zoom)</option>
      <option value="online" className="bg-gray-800">Online (Async)</option>
      <option value="in-person" className="bg-gray-800">In-Person</option>
    </select>
  </div>
  <div>
    <label className="text-white text-[9px] mb-0.5 block">Zoom Link{editInfo.deliveryMode === 'virtua...
    <input className={`w-full h-6 text-[10px] bg-white/10 text-white rounded px-1.5 placeholder:text-...
  </div>
</div>
{editInfo.deliveryMode !== 'online' && !editFieldsLocked && () => {
  const isSpSu = editInfo.semesterTerm?.startsWith('spring_summer');
  return (
    <div className="space-y-2">
      <div className="grid gap-2 grid-cols-3">
        <div>
          <label className="text-white text-[9px] mb-0.5 block">{isSpSu ? 'Day of Week 1' : 'Day of W...
          <select className="w-full h-6 text-[10px] bg-white/10 border border-white/15 text-white rou...
            <option value="" className="bg-gray-800"></option>
            {[ 'monday', 'tuesday', 'wednesday', 'thursday', 'friday', 'saturday', 'sunday' ].map(d => <optio...
          </select>
        </div>
        <div>
          <label className="text-white text-[9px] mb-0.5 block">Start</label>
          <input type="time" className="w-full h-6 text-[10px] bg-white/10 border border-white/15 rou...
        </div>
        <div>
          <label className="text-white text-[9px] mb-0.5 block">End</label>
          <input type="time" className="w-full h-6 text-[10px] bg-white/10 border border-white/15 rou...
        </div>
      </div>
    </div>
    {isSpSu && (
      <div className="grid gap-2 grid-cols-3">
        <div>
          <label className="text-white text-[9px] mb-0.5 block">Day of Week 2</label>
          <select className="w-full h-6 text-[10px] bg-white/10 border border-white/15 text-white r...

```

```

        <option value="" className="bg-gray-800"></option>
        [['monday', 'tuesday', 'wednesday', 'thursday', 'friday', 'saturday', 'sunday'].map(d => <opt...
    </select>
</div>
<div>
    <label className="text-white text-[9px] mb-0.5 block">Start</label>
    <input type="time" className="w-full h-6 text-[10px] bg-white/10 border border-white/15 r...
</div>
<div>
    <label className="text-white text-[9px] mb-0.5 block">End</label>
    <input type="time" className="w-full h-6 text-[10px] bg-white/10 border border-white/15 r...
</div>
</div>
    )}
</div>
});
})();
<div className={editFieldsLocked ? 'opacity-30 pointer-events-none' : ''}>
<label className="text-white text-[9px] mb-0.5 block">Certificate Type</label>
<select className="w-full h-6 text-[10px] bg-white/10 border border-white/15 text-white rounded px-...
    <option value="" className="bg-gray-800">-- Select --</option>
    {CERTIFICATE_TYPE_OPTIONS.map(g => {
        <optgroup key={g.group} label={g.group}>
        {g.options.map(o => <option key={o} value={o} className="bg-gray-800">{o}</option>)}
    </optgroup>
    )}
</select>
</div>
<div className={`flex items-end justify-between ${editFieldsLocked ? 'opacity-30 pointer-events-none'...
    (() => {
        const midStops: Array<{ position: number; color: string }> = editInfo.colorStops ? () => { try {...
        const allStops = [
            { position: 0, color: editInfo.color, key: 'start' as const },
            ...midStops.map(s: any, i: number) => ({ position: s.position, color: s.color, key: i as numbe...
            { position: 100, color: editInfo.colorEnd, key: 'end' as const },
        ].sort((a, b) => a.position - b.position);
        const gradientCss = `linear-gradient(to right, ${allStops.map(s => `${s.color} ${s.position}%`).j...
        const getActiveColor = (): string => {
            if (activeGradientStop === 'start') return editInfo.color;
            if (activeGradientStop === 'end') return editInfo.colorEnd;
            if (typeof activeGradientStop === 'number' && midStops[activeGradientStop]) return midStops[act...
            return '#000000';
        };
        const setActiveColor = (hex: string) => {
            if (activeGradientStop === 'start') setEditInfo({...editInfo, color: hex});
            else if (activeGradientStop === 'end') setEditInfo({...editInfo, colorEnd: hex});
            else if (typeof activeGradientStop === 'number') {
                const updated = [...midStops];
                updated[activeGradientStop] = { ...updated[activeGradientStop], color: hex };
                setEditInfo({...editInfo, colorStops: JSON.stringify(updated)});
            }
        };
        const hexToHue = (c: string) => { const r = parseInt(c.slice(1,3),16)/255, g = parseInt(c.slice(3...
        const hueToHex = (hue: number) => `#${[0,8,4].map(n => { const k = (n + hue/30) % 12; const c2 = ...
        const hexToSvPos = (hex: string) => {
            const r = parseInt(hex.slice(1,3),16)/255, g = parseInt(hex.slice(3,5),16)/255, b = parseInt(he...
            const max = Math.max(r,g,b), min = Math.min(r,g,b);
            const v = max;
            const s = max === 0 ? 0 : (max - min) / max;
            return { x: s, y: 1 - v };
        };
        const svToHex = (hue: number, sx: number, sy: number) => {
            const s = sx, v = 1 - sy;
            const c = v * s, x2 = c * (1 - Math.abs((hue / 60) % 2 - 1)), m = v - c;
            let r1 = 0, g1 = 0, b1 = 0;
            if (hue < 60) { r1 = c; g1 = x2; } else if (hue < 120) { r1 = x2; g1 = c; } else if (hue < 180)...
            const f = (ch: number) => Math.round(255 * Math.max(0, Math.min(1, ch + m))).toString(16).padSt...
            return `#${f(r1)}${f(g1)}${f(b1)}`;
        };
    });
    const gradBarRef = useRef<HTMLDivElement>(null);
    return (
        <div className="flex items-start gap-3">
        <div style={{ width: '200px' }}>
            <div className="flex items-center justify-between mb-1">
                <label className="text-white text-[9px]">Course Colour</label>
                <button className="text-white hover:text-white/80 text-[8px] flex items-center gap-0.5" onClick=...
                    const revMid = midStops.map(s => ({ position: 100 - s.position, color: s.color })).reverse();
                    setEditInfo({...editInfo, color: editInfo.colorEnd, colorEnd: editInfo.color, colorStops: r...
                }) data-testid="button-reverse-gradient"><svg width="10" height="10" viewBox="0 0 16 16" fill=...
            </div>
            <div ref={gradBarRef} className="rounded" style={{ border: '1px solid rgba(255,255,255,0.15)', ...
                onDoubleClick={(e) => {
                    const bar = gradBarRef.current;
                    if (!bar) return;
                    const rect = bar.getBoundingClientRect();
                    const pct = Math.round(Math.max(5, Math.min(95, ((e.clientX - rect.left - 3) / (rect.width ...
                    const leftStop = allStops.filter(s => s.position <= pct).pop()!;
                    const rightStop = allStops.find(s => s.position >= pct)!;
                    const t = rightStop.position === leftStop.position ? 0 : (pct - leftStop.position) / (right...
                    const lR = parseInt(leftStop.color.slice(1,3),16), lG = parseInt(leftStop.color.slice(3,5),...
                    const rR = parseInt(rightStop.color.slice(1,3),16), rG = parseInt(rightStop.color.slice(3,5)...
                    const mR = Math.round((lR + (rR-lR)*t), mG = Math.round((lG + (rG-lG)*t), mB = Math.round((lB ...
                    const newColor = `#${mR.toString(16).padStart(2,'0')}${mG.toString(16).padStart(2,'0')}${mB...
                    const newMid = [...midStops, { position: pct, color: newColor }].sort((a, b) => a.position ...
                    const newIndex = newMid.findIndex(s => s.position === pct && s.color === newColor);
                    setEditInfo({...editInfo, colorStops: JSON.stringify(newMid)});
                    setActiveGradientStop(newIndex);
                }
            </div>
            <div style={{ height: '18px', borderRadius: '3px', background: gradientCss }} data-testid="gr...
        </div>
        <div className="relative" style={{ height: '16px', marginTop: '1px' }}>
            <div style={{ position: 'absolute', left: '0px', top: 0, cursor: 'pointer', zIndex: 10 }} onCli...
                <svg width="12" height="12" viewBox="0 0 12 12"><polygon points="6,0 12,0 10,10" fill={edit...
            </div>
            {midStops.map((stop, idx) => (
                <div key={idx} style={{ position: 'absolute', left: `calc(${stop.position}% - 6px)`, top: 0...
                    onClick={() => setActiveGradientStop(activeGradientStop === idx ? null : idx)}
                    onPointerDown={(e) => {

```

```

    e.preventDefault();
    e.stopPropagation();
    const el = e.currentTarget as HTMLElement;
    el.setPointerCapture(e.pointerId);
    const bar = gradBarRef.current;
    if (!bar) return;
    const barRect = bar.getBoundingClientRect();
    const barW = barRect.width;
    const onMove = (ev: PointerEvent) => {
      const pct = Math.round(Math.max(1, Math.min(99, ((ev.clientX - barRect.left) / barW) ...
      const updated = [...midStops];
      updated[idx] = { ...updated[idx], position: pct };
      setEditInfo(prev => ({...prev, colorStops: JSON.stringify(updated.sort((a, b) => a.po...
    });
    const onUp = () => { el.releasePointerCapture(e.pointerId); window.removeEventListener(...
    window.addEventListener('pointermove', onMove);
    window.addEventListener('pointerup', onUp);
  })
  data-testid={`gradient-stop-mid-${idx}`}>
  <svg width="12" height="12" viewBox="0 0 12 12"><polygon points="6,0 11,6 6,12 1,6" fill=...
</div>
))
<div style={{ position: 'absolute', right: '0px', top: 0, cursor: 'pointer', zIndex: 10 }} on...
<svg width="12" height="12" viewBox="0 0 12 12"><polygon points="6,0 12,10 0,10" fill={edit...
</div>
{midStops.length > 0 && <span className="text-white text-[9px] absolute" style={{ bottom: '-2...
</div>
{midStops.length === 0 && <div className="text-white text-[11px] text-left" style={{ marginT...
{midStops.length > 0 && <div style={{ height: '24px' }} />
{activeGradientStop !== null && (
  <div className="mt-1 rounded" style={{ border: '1px solid rgba(255,255,255,0.15)', background...
    <div className="flex items-center gap-2 mb-1.5">
      <div className="w-5 h-5 rounded border border-white/30 shrink-0" style={{ backgroundColo...
      <span className="text-white/60 text-[8px] uppercase tracking-wider">{activeGradientStop =...
      {typeof activeGradientStop === 'number' && (
        <button className="text-red-400/70 hover:text-red-400 text-[8px] ml-auto mr-1" onClick=...
          const updated = midStops.filter((_, i) => i !== activeGradientStop);
          setEditInfo({...editInfo, colorStops: updated.length ? JSON.stringify(updated) : ''});
          setActiveGradientStop(null);
        <div data-testid="button-delete-stop"><Trash2 className="w-3 h-3" /></button>
      )
    <button className={` ${typeof activeGradientStop === 'number' ? '' : 'ml-auto' } text-white...
  </div>
  {typeof activeGradientStop === 'number' && (
    <div className="flex items-center gap-2 mb-1.5">
      <span className="text-white/50 text-[8px]">Position</span>
      <input type="range" min={1} max={99} value={midStops[activeGradientStop]?.position || 5...
      const updated = [...midStops];
      updated[activeGradientStop as number] = { ...updated[activeGradientStop as number], p...
      setEditInfo({...editInfo, colorStops: JSON.stringify(updated.sort((a, b) => a.positio...
    <div className="flex-1" style={{ height: '6px', accentColor: getActiveColor() }} data-tes...
    <span className="text-white/50 text-[8px] w-6 text-right">{midStops[activeGradientStop]...
  </div>
)
<div className="relative rounded cursor-crosshair" style={{ height: '92px', touchAction: 'n...
  onPointerDown={(e) => {
    e.preventDefault();
    const el = e.currentTarget;
    el.setPointerCapture(e.pointerId);
    const rect = el.getBoundingClientRect();
    const hue = hexToHue(getActiveColor());
    const update = (ev: PointerEvent) => {
      const x = Math.max(0, Math.min(1, (ev.clientX - rect.left) / rect.width));
      const y = Math.max(0, Math.min(1, (ev.clientY - rect.top) / rect.height));
      setActiveColor(svToHex(hue, x, y));
    };
    update(e.nativeEvent);
    const onMove = (ev: PointerEvent) => update(ev);
    const onUp = () => { el.releasePointerCapture(e.pointerId); window.removeEventListener(...
    window.addEventListener('pointermove', onMove);
    window.addEventListener('pointerup', onUp);
  })
  <div style={{ position: 'absolute', inset: 0, borderRadius: '3px', background: 'linear-gr...
  <div style={{ position: 'absolute', inset: 0, borderRadius: '3px', background: 'linear-gr...
  (() => {
    const pos = hexToSvPos(getActiveColor());
    return (
      <div style={{ position: 'absolute', left: `${pos.x * 100}%`, top: `${pos.y * 100}%`, ...
    );
  })()
</div>
<div className="relative mt-1.5 rounded cursor-pointer" style={{ height: '14px', touchActio...
  onClick={(e) => {
    const rect = e.currentTarget.getBoundingClientRect();
    const x = Math.max(0, Math.min(1, (e.clientX - rect.left) / rect.width));
    setActiveColor(hueToHex(Math.round(x * 360)));
  })
  onPointerDown={(e) => {
    e.preventDefault();
    const el = e.currentTarget;
    el.setPointerCapture(e.pointerId);
    const rect = el.getBoundingClientRect();
    const update = (ev: PointerEvent) => {
      const x = Math.max(0, Math.min(1, (ev.clientX - rect.left) / rect.width));
      setActiveColor(hueToHex(Math.round(x * 360)));
    };
    update(e.nativeEvent);
    const onMove = (ev: PointerEvent) => update(ev);
    const onUp = () => { el.releasePointerCapture(e.pointerId); window.removeEventListener(...
    window.addEventListener('pointermove', onMove);
    window.addEventListener('pointerup', onUp);
  })
  <div style={{ position: 'absolute', inset: 0, background: 'linear-gradient(to right, #ff0...
  <div style={{ position: 'absolute', top: '-1px', left: `${(hexToHue(getActiveColor()) / 3...
</div>
<div className="flex items-center gap-2 mt-1.5">
  <input type="color" value={getActiveColor()} onChange={(e) => setActiveColor(e.target.val...
  <input type="text" value={getActiveColor().toUpperCase()} onChange={(e) => { let v = e.ta...

```



```

        <span className="text-white">
            {courseInfo.startDate ? new Date(courseInfo.startDate + 'T00:00:00').toLocaleDateString('en-U...
            { ' - ' }
            {courseInfo.endDate ? new Date(courseInfo.endDate + 'T00:00:00').toLocaleDateString('en-US', ...
        </span>
    </div>
    ))
    {courseInfo.deliveryMode !== "online" && (() => {
        const isSpSu = courseInfo.semesterTerm?.startsWith('spring_summer');
        if (!isSpSu) return null;
        const fmt = (t: string) => { const [h,m] = t.split(':').map(Number); const p = h >= 12 ? 'PM' : '...'
        return (
            <div />
            <div style={{ display: 'grid', gridTemplateColumns: '12px auto 1fr', gap: '6px', alignItems: ...
                <Calendar className="h-3 w-3 text-white flex-shrink-0" style={{ justifySelf: 'end' }} />
                <span className="text-white" style={{ justifySelf: 'end' }}>Day 2:</span>
                <span className="text-white capitalize" style={{ justifySelf: 'end' }}>
                    {courseInfo.classDay2
                    ? `${courseInfo.classDay2}${courseInfo.classTime2 ? ` ${fmt(courseInfo.classTime2)}` : ...
                    : "Not set"}
                </span>
            </div>
        </div>
    );
    })()
    </div>
    {courseInfo.zoomLink && (
        <div className="flex items-center gap-1.5" style={{ maxWidth: '55%', marginTop: '10px' }}>
            <a
                href={courseInfo.zoomLink}
                target="_blank"
                rel="noopener noreferrer"
                className="flex items-center gap-1.5 text-[10px] text-white hover:text-white/80 bg-white/10 bor...
                data-testid="link-zoom"
            >
                <img src={zoomLogoPath} alt="Zoom" style={{ width: '38px', height: 'auto', filter: 'brightness(...
                <span className="truncate">{courseInfo.zoomLink}</span>
            <ExternalLink className="h-2.5 w-2.5 ml-auto flex-shrink-0" />
            </a>
            <button
                className="flex items-center justify-center h-7 w-7 rounded bg-white/10 border border-white/20 ...
                onClick={() => { navigator.clipboard.writeText(courseInfo.zoomLink || ''); toast({ title: 'Copi...
                data-testid="button-copy-zoom-link"
            >
                <Copy className="h-3 w-3 text-white" />
            </button>
        </div>
    ))
    </div>
    <div style={{ height: '10px' }} />
    {syllabusData && Object.keys(syllabusItemStates).length > 0 && (
        <div className="mx-3 mb-2 border border-emerald-400/30 rounded-lg overflow-hidden" data-testid="syllabus-...
        <div className="flex items-center justify-between px-3 py-1.5 bg-emerald-900/30 border-b border-emerald...
        <div className="flex items-center gap-1.5">
            <FileText className="h-3 w-3 text-emerald-400" />
            <span className="text-[10px] text-white font-medium">Syllabus Review</span>
            <span className="text-[8px] text-white/60">
                {Object.values(syllabusItemStates).filter(s => s.accepted === true).length} accepted ·{' '}
                {Object.values(syllabusItemStates).filter(s => s.accepted === null).length} pending
            </span>
        </div>
        <button onClick={() => { setSyllabusData(null); setSyllabusItemStates({}); }} className="text-white/6...
        Dismiss
    </button>
    </div>
    <div className="max-h-[300px] overflow-y-auto" style={{ scrollbarWidth: 'thin', scrollbarColor: 'rgba(2...
    {syllabusData.weekNumbering && syllabusItemStates[-1] && syllabusItemStates[-1].accepted === null && (
        <div className="px-3 py-2 border-b border-white/10 bg-amber-900/15" data-testid="syllabus-week-numb...
        <div className="flex items-center gap-1.5 mb-1">
            <Calendar className="h-3 w-3 text-amber-400" />
            <span className="text-[10px] text-white font-medium">Week Numbering Style</span>
        </div>
        <p className="text-[9px] text-white/70 mb-1.5">
            Detected: <span className="text-white font-medium">{syllabusData.weekNumbering.style === 'skip_...
            {syllabusData.weekNumbering.breakWeekLabel && <> · Break label: "{syllabusData.weekNumbering.br...
        </p>
        {syllabusData.weekNumbering.evidence && (
            <p className="text-[8px] text-white/50 mb-2 italic">{syllabusData.weekNumbering.evidence}</p>
        )}
        <div className="flex items-center gap-1.5">
            <button
                onClick={() => handleAcceptWeekNumbering(syllabusData.weekNumbering.style)}
                className="h-5 px-2 text-[8px] bg-green-600/30 hover:bg-green-600/50 text-white border border...
                data-testid="button-accept-week-style"
            >
                <Check className="h-2.5 w-2.5" /> Accept
            </button>
            <button
                onClick={() => {
                    setWeekStyleChoice(syllabusData.weekNumbering.style);
                    handleDeclineSyllabusItem(-1);
                }}
                className="h-5 px-2 text-[8px] bg-red-600/20 hover:bg-red-600/30 text-white border border-red...
                data-testid="button-decline-week-style"
            >
                <X className="h-2.5 w-2.5" /> Decline
            </button>
        </div>
    </div>
    ))
    {syllabusData.weekNumbering && syllabusItemStates[-1] && syllabusItemStates[-1].accepted === false &&...
    <div className="px-3 py-2 border-b border-white/10 bg-amber-900/15" data-testid="syllabus-week-numb...

```

```

<div className="flex items-center gap-1.5 mb-1.5">
  <Calendar className="h-3 w-3 text-amber-400" />
  <span className="text-[10px] text-white font-medium">Choose Week Numbering</span>
</div>
<div className="flex flex-col gap-1">
  {[ 'skip_break', 'include_break', 'continuous' ].filter(s => s !== weekStyleChoice).map(style => (
    <button
      key={style}
      onClick={() => { handleAcceptWeekNumbering(style); setWeekStyleChoice(null); }}
      className="h-6 px-2 text-[9px] bg-white/5 hover:bg-white/10 text-white border border-white/...
      data-testid={`button-week-style-${style}`}
    >
      <span className="font-medium">{style === 'skip_break' ? 'Skip break number' : style === 'in...
    </button>
  )]}
</div>
</div>
))

{syllabusData.courseInfo?.description && (
  <div className="px-3 py-1.5 border-b border-white/10 bg-white/3">
    <p className="text-[9px] text-white/60 italic">{syllabusData.courseInfo.description}</p>
  </div>
)}

{() => {
  const items = [
    ...(syllabusData.items || []).map((item: any, i: number) => ({ ...item, _idx: i, _source: 'item' ...
    ...(syllabusData.gradingBreakdown || []).map((g: any, i: number) => ({ title: g.component, weight...
  ];
  return items.map((item: any) => {
    const state = syllabusItemStates[item._idx];
    if (!state) return null;
    const isAccepted = state.accepted === true;
    const isDeclined = state.accepted === false;
    const isEditing = state.editing;
    const edits = state.edits;

    return (
      <div
        key={item._idx}
        className={`px-3 py-1.5 border-b border-white/8 flex items-start gap-2 transition-colors ${is...
        data-testid={`syllabus-item-${item._idx}`}
      >
        <div className="flex flex-col gap-0.5 mt-0.5 flex-shrink-0">
          {!isAccepted && !isDeclined ? (
            <button
              onClick={() => handleAcceptSyllabusItem(item._idx)}
              className="w-4 h-4 rounded border border-green-400/40 bg-green-600/20 hover:bg-green-...
              title="Accept"
              data-testid={`button-accept-item-${item._idx}`}
            >
              <Check className="h-2.5 w-2.5 text-green-400" />
            </button>
            <button
              onClick={() => handleDeclineSyllabusItem(item._idx)}
              className="w-4 h-4 rounded border border-red-400/30 bg-red-600/15 hover:bg-red-600/30...
              title="Decline"
              data-testid={`button-decline-item-${item._idx}`}
            >
              <X className="h-2.5 w-2.5 text-red-400" />
            </button>
          </>
          <div className="w-4 h-4 rounded bg-green-600/30 flex items-center justify-center">
            <Check className="h-2.5 w-2.5 text-green-400" />
          </div>
          <div className="w-4 h-4 rounded bg-red-600/20 flex items-center justify-center">
            <X className="h-2.5 w-2.5 text-red-400" />
          </div>
        </div>
      )
    )
  )
}
</div>

<div className="flex-1 min-w-0">
  {isEditing ? (
    <div className="space-y-1">
      <input
        className="w-full h-5 text-[9px] bg-white/10 border border-white/20 rounded px-1.5 te...
        value={edits.title || ''}
        onChange={(e) => setSyllabusItemStates(prev => ({ ...prev, [item._idx]: { ...prev[ite...
        placeholder="Title"
        data-testid={`input-edit-syllabus-title-${item._idx}`}
      >
    </div>
    <div className="flex gap-1">
      <input
        type="date"
        className="flex-1 h-5 text-[9px] bg-white/10 border border-white/20 rounded px-1.5 ...
        style={{ colorScheme: 'dark' }}
        value={edits.date || ''}
        onChange={(e) => setSyllabusItemStates(prev => ({ ...prev, [item._idx]: { ...prev[i...
        data-testid={`input-edit-syllabus-date-${item._idx}`}
      >
      <input
        type="time"
        className="w-20 h-5 text-[9px] bg-white/10 border border-white/20 rounded px-1.5 te...
        style={{ colorScheme: 'dark' }}
        value={edits.time || ''}
        onChange={(e) => setSyllabusItemStates(prev => ({ ...prev, [item._idx]: { ...prev[i...
        data-testid={`input-edit-syllabus-time-${item._idx}`}
      >
      <input
        className="w-14 h-5 text-[9px] bg-white/10 border border-white/20 rounded px-1.5 te...
        value={edits.weight ?? ''}
        onChange={(e) => setSyllabusItemStates(prev => ({ ...prev, [item._idx]: { ...prev[i...
        placeholder="Wt%"
        data-testid={`input-edit-syllabus-weight-${item._idx}`}
      >
    </div>
  )
}

```

```

    />
  </div>
  <textarea
    className="w-full h-10 text-[8px] bg-white/10 border border-white/20 rounded px-1.5 p...
    value={edits.description || ''}
    onChange={(e) => setSyllabusItemStates(prev => ({ ...prev, [item._idx]: { ...prev[item...
    placeholder="Description"
    data-testid={`input-edit-syllabus-desc-${item._idx}`}}
  />
  <button
    onClick={() => setSyllabusItemStates(prev => ({ ...prev, [item._idx]: { ...prev[item...
    className="h-4 px-2 text-[8px] bg-white/10 hover:bg-white/20 text-white rounded"
    data-testid={`button-done-edit-${item._idx}`}}
  >
    Done
  </button>
</div>
) : (
  <>
    <div className="flex items-center gap-1.5">
      <span className="text-[10px] text-white font-medium truncate">{edits.title || item.ti...
      {item._source === 'grading' && (
        <span className="text-[7px] px-1 py-0.5 bg-purple-500/20 text-purple-300 rounded">G...
      )}
      {edits.weight !== null && (
        <span className="text-[8px] text-amber-400 flex-shrink-0">{edits.weight}%</span>
      )}
    </div>
    <div className="flex items-center gap-2 text-[8px] text-white/60">
      {(edits.date || edits.dateDescription) && (
        <span>{edits.date || edits.dateDescription}</span>
      )}
      {edits.type && edits.type !== 'other' && (
        <span className="capitalize">{edits.type}</span>
      )}
    </div>
    {edits.description && (
      <p className="text-[8px] text-white/50 mt-0.5 line-clamp-2">{edits.description}</p>
    )}
  </>
</div>

  {!isAccepted && !isDeclined && !isEditing && (
    <button
      onClick={() => setSyllabusItemStates(prev => ({ ...prev, [item._idx]: { ...prev[item._idx...
      className="flex-shrink-0 w-5 h-5 rounded hover:bg-white/10 flex items-center justify-cent...
      title="Edit"
      data-testid={`button-edit-item-${item._idx}`}}
    >
      <Pencil className="h-2.5 w-2.5 text-white/50" />
    </button>
  )}
</div>
);
});
})();

{syllabusData.policies && syllabusData.policies.length > 0 && (
  <div className="px-3 py-2 border-t border-white/15">
    <div className="text-[9px] text-white/70 font-medium mb-1">Policies</div>
    {syllabusData.policies.map((p: any, i: number) => (
      <div key={i} className="text-[8px] text-white/50 mb-1">
        <span className="text-white/70 font-medium">{p.title}</span> {p.description}
      </div>
    ))}
  </div>
)}
</div>
)}
</div>
</div>
))
<div style={{ padding: '12px', marginTop: '-12px', display: 'flex', flexDirection: 'column', gap: '0px' }}>
  <div ref={weekMappingsRef}>
    <div style={{ border: '2px solid rgba(255,255,255,0.2)', borderRadius: '8px', padding: '12px' }}>
      <div
        className="flex items-center justify-between cursor-pointer group"
        onClick={() => { const next = !showWeekMappings; setShowWeekMappings(next); if (next) setTimeout(() => ...
        data-testid="button-toggle-week-mappings"
      >
        <div className="flex items-center gap-2">
          <Calendar className="h-3.5 w-3.5 text-white/70" />
          <h3 className="text-[11px] font-medium text-white uppercase">Weeks and Modules</h3>
          {(() => {
            const confirmed = Object.values(weekMappingEdits).filter(v => v.confirmed).length;
            const total = LAST_WEEK - FIRST_WEEK + 1;
            return (
              <span className={`text-[9px] ${confirmed === total ? 'text-green-400' : 'text-white/50'}`}>
                {confirmed}/{total} confirmed
              </span>
            );
          })()}
        </div>
        <div className="flex items-center gap-1">
          {showWeekMappings ? <ChevronDown className="h-3 w-3 text-white/50" /> : <ChevronRight className="h-3 ...
        </div>
      </div>
    </div>
  </div>
  {showWeekMappings && (
    <div className="mt-2 bg-white/5 border border-white/15 rounded-lg p-3 space-y-1" data-testid="week-mapp...
    <div className="flex items-center justify-between mb-2">
      <span className="text-[9px] text-white">Confirm each week follows the standard TMU academic calenda...
      <button
        size="sm"
        onClick={confirmAllWeeks}
        className="h-5 px-2 text-[8px] bg-green-500/20 hover:bg-green-500/30 text-green-300 border border...
        data-testid="button-confirm-all-weeks"
      >

```



```

        <Check className="h-2.5 w-2.5 mr-1" />
        Confirm All
    </Button>
</div>

<div className="mb-3 border border-white/15 rounded-lg p-2" style={{ background: 'rgba(255,255,255,0....
    <button
        className="flex items-center gap-1.5 text-[9px] font-medium text-white hover:text-white/80 transi...
        onClick={() => setShowWeekCalendar(!showWeekCalendar)}
        data-testid="button-toggle-week-calendar"
    >
        <Calendar className="h-3 w-3" />
        <span>Set Week 1 Start Date</span>
        {courseWeek1Start && <span className="text-[8px] text-green-300 ml-1">({courseWeek1Start.toLocale...
        {showWeekCalendar ? <ChevronDown className="h-2.5 w-2.5 ml-auto" /> : <ChevronRight className="h-...
    </button>

    {showWeekCalendar && (() => {
        const month = weekCalendarMonth;
        const year = month.getFullYear();
        const mo = month.getMonth();
        const firstDay = new Date(year, mo, 1).getDay();
        const daysInMonth = new Date(year, mo + 1, 0).getDate();
        const monthName = month.toLocaleDateString('en-US', { month: 'long', year: 'numeric' });
        const dayLabels = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'];

        const handleDayClick = (day: number) => {
            const selected = new Date(year, mo, day);
            const dayOfWeek = selected.getDay();
            const saturdayOffset = dayOfWeek === 6 ? 0 : -(dayOfWeek + 1);
            const weekStart = new Date(year, mo, day + saturdayOffset);
            setCourseWeek1Start(weekStart);

            const newEdits = { ...weekMappingEdits };
            let currentStart = new Date(weekStart);
            let courseWeekNum = 1;
            for (let w = FIRST_WEEK; w <= LAST_WEEK; w++) {
                const isExcluded = readingWeekVariable && readingWeekExclusions.has(w);
                const weekEnd = new Date(currentStart);
                weekEnd.setDate(weekEnd.getDate() + 6);
                if (!isExcluded) {
                    newEdits[w] = {
                        ...(newEdits[w] || { confirmed: false, notes: '' }),
                        confirmed: true,
                        courseWeekLabel: `Week ${courseWeekNum}`,
                    };
                    courseWeekNum++;
                } else {
                    newEdits[w] = {
                        ...(newEdits[w] || { confirmed: false, notes: '' }),
                        confirmed: true,
                        courseWeekLabel: 'Reading Week',
                    };
                }
                currentStart = new Date(weekEnd);
                currentStart.setDate(currentStart.getDate() + 1);
            }
            setWeekMappingEdits(newEdits);
            for (let w = FIRST_WEEK; w <= LAST_WEEK; w++) {
                saveWeekMapping(w, newEdits[w]);
            }
        };

        const getWeekOfDay = (day: number) => {
            if (!courseWeek1Start) return null;
            const d = new Date(year, mo, day);
            const diff = Math.floor((d.getTime() - courseWeek1Start.getTime()) / (7 * 24 * 60 * 60 * 1000));
            if (diff < 0) return null;
            return diff + FIRST_WEEK;
        };

        return (
            <div className="mt-2" data-testid="week-calendar-picker">
                <div className="flex items-center justify-between mb-2">
                    <button onClick={() => setWeekCalendarMonth(new Date(year, mo - 1, 1))} className="text-whi...
                        <ChevronLeft className="h-3.5 w-3.5" />
                    </button>
                    <span className="text-[13px] font-medium text-white">{monthName}</span>
                    <button onClick={() => setWeekCalendarMonth(new Date(year, mo + 1, 1))} className="text-whi...
                        <ChevronRight className="h-3.5 w-3.5" />
                    </button>
                </div>
                <div className="grid grid-cols-7 gap-0.5 mb-1">
                    {dayLabels.map(d => <div key={d} className="text-[8px] text-white text-center font-medium">...
                </div>
                <div className="grid grid-cols-7 gap-0.5">
                    {Array.from({ length: firstDay }, (_, i) => <div key={`e${i}`} />)}
                    {Array.from({ length: daysInMonth }, (_, i) => {
                        const day = i + 1;
                        const d = new Date(year, mo, day);
                        const dayNorm = new Date(year, mo, day);
                        dayNorm.setHours(0,0,0,0);
                        const w1 = courseWeek1Start ? new Date(courseWeek1Start) : null;
                        if (w1) w1.setHours(0,0,0,0);
                        const isWeek1Start = w1 && dayNorm.getTime() === w1.getTime();
                        const isInWeek1 = w1 && dayNorm >= w1 && dayNorm < new Date(w1.getTime() + 7 * 24 * 60 * ...
                        const weekIdx = getWeekOfDay(day);
                        const isExcludedWeek = weekIdx !== null && readingWeekExclusions.has(weekIdx);
                        return (
                            <button
                                key={day}
                                onClick={() => handleDayClick(day)}
                                className={`h-6 text-[11px] rounded transition-colors ${
                                    isWeek1Start ? 'bg-green-500 text-white font-bold' :
                                    isInWeek1 ? 'bg-green-500/30 text-white' :
                                    isExcludedWeek ? 'bg-amber-500/20 text-white' :
                                    'text-white hover:bg-white/15'
                                }`
                            >
                                {day}
                            </button>
                        );
                    });
                </div>
            </div>
        );
    }
);

```

```

        data-testid={`week-cal-day-${day}`}
      >
        {day}
      </button>
    );
  });
</div>
<div className="mt-2 text-[8px] text-white">
  Click any day to set the Saturday of that week as Week 1 start. All subsequent weeks will b...
</div>
</div>
);
})();

<div className="mt-2 flex items-center gap-2">
  <label className="flex items-center gap-1.5 cursor-pointer" data-testid="checkbox-reading-week-va...
    <div
      className="flex items-center justify-center border border-white/40 rounded-sm cursor-pointer"
      style={{ width: '13px', height: '13px', background: readingWeekVariable ? 'white' : 'transpar...
      onClick={() => {
        const next = !readingWeekVariable;
        setReadingWeekVariable(next);
        if (!next) {
          setReadingWeekExclusions(new Set());
          setShowReadingWeekCalendar(false);
        }
      }}
    >
      {readingWeekVariable && <span style={{ color: 'black', fontSize: '9px', fontWeight: 'bold', l...
    </div>
    <span className="text-[10px] text-white">Reading week variable</span>
  </label>
</div>

{readingWeekVariable && (
  <div className="mt-2">
    <button
      className="flex items-center gap-1 text-[8px] text-white/70 hover:text-white transition-colors"
      onClick={() => setShowReadingWeekCalendar(!showReadingWeekCalendar)}
      data-testid="button-toggle-reading-week-cal"
    >
      <Calendar className="h-2.5 w-2.5" />
      <span>Select reading week</span>
      {selectedReadingWeekStart && (
        <span className="text-[8px] text-amber-300 ml-1">
          ({selectedReadingWeekStart.toLocaleDateString('en-US', { month: 'short', day: 'numeric' })...
        </span>
      )
    </button>
    {showReadingWeekCalendar ? <ChevronDown className="h-2.5 w-2.5 ml-auto" /> : <ChevronRight cl...
  </div>

  {showReadingWeekCalendar && (() => {
    const rwMonth = readingWeekCalMonth;
    const rwYear = rwMonth.getFullYear();
    const rwMo = rwMonth.getMonth();
    const rwFirstDay = new Date(rwYear, rwMo, 1).getDay();
    const rwDaysInMonth = new Date(rwYear, rwMo + 1, 0).getDate();
    const rwMonthName = rwMonth.toLocaleDateString('en-US', { month: 'long', year: 'numeric' });
    const rwDayLabels = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'];

    const handleReadingWeekDayClick = (day: number) => {
      const selected = new Date(rwYear, rwMo, day);
      const dayOfWeek = selected.getDay();
      const satOffset = dayOfWeek === 6 ? 0 : -(dayOfWeek + 1);
      const rwStart = new Date(rwYear, rwMo, day + satOffset);
      setSelectedReadingWeekStart(rwStart);

      if (courseWeek1Start) {
        const w1 = new Date(courseWeek1Start);
        w1.setHours(12, 0, 0, 0);
        rwStart.setHours(12, 0, 0, 0);
        const diffMs = rwStart.getTime() - w1.getTime();
        const diffWeeks = Math.round(diffMs / (7 * 24 * 60 * 60 * 1000));
        const rwWeekNum = diffWeeks + FIRST_WEEK;

        const next = new Set<number>();
        if (rwWeekNum >= FIRST_WEEK && rwWeekNum <= LAST_WEEK) {
          next.add(rwWeekNum);
        }
        setReadingWeekExclusions(next);

        const newEdits = { ...weekMappingEdits };
        let courseWeekNum = 1;
        for (let w = FIRST_WEEK; w <= LAST_WEEK; w++) {
          const excl = next.has(w);
          if (!excl) {
            newEdits[w] = { ...(newEdits[w] || { confirmed: false, notes: '' }), confirmed: true, ...
            courseWeekNum++;
          } else {
            newEdits[w] = { ...(newEdits[w] || { confirmed: false, notes: '' }), confirmed: true, ...
          }
        }
        setWeekMappingEdits(newEdits);
        for (let w = FIRST_WEEK; w <= LAST_WEEK; w++) saveWeekMapping(w, newEdits[w]);
      }
    }
  )
);

return (
  <div className="mt-2" data-testid="reading-week-calendar-picker">
    <div className="flex items-center justify-between mb-2">
      <button onClick={() => setReadingWeekCalMonth(new Date(rwYear, rwMo - 1, 1))} className...
      <ChevronLeft className="h-3.5 w-3.5" />
    </button>
    <span className="text-[13px] font-medium text-white">{rwMonthName}</span>
    <button onClick={() => setReadingWeekCalMonth(new Date(rwYear, rwMo + 1, 1))} className...
    <ChevronRight className="h-3.5 w-3.5" />
  </button>
</div>

```

```

<div className="grid grid-cols-7 gap-0.5 mb-1">
  {rwDayLabels.map(d => <div key={d} className="text-[8px] text-white text-center font-me...
</div>
<div className="grid grid-cols-7 gap-0.5">
  {Array.from({ length: rwFirstDay }, (_, i) => <div key={`re${i}`} />)}
  {Array.from({ length: rwDaysInMonth }, (_, i) => {
    const day = i + 1;
    const dayDate = new Date(rwYear, rwMo, day);
    dayDate.setHours(0, 0, 0, 0);
    const rwSel = selectedReadingWeekStart ? new Date(selectedReadingWeekStart) : null;
    if (rwSel) rwSel.setHours(0, 0, 0, 0);
    const isRWStart = rwSel && dayDate.getTime() === rwSel.getTime();
    const isInRW = rwSel && dayDate >= rwSel && dayDate < new Date(rwSel.getTime() + 7 * ...
    return (
      <button
        key={day}
        onClick={() => handleReadingWeekDayClick(day)}
        className={`h-6 text-[11px] rounded transition-colors ${
          isRWStart ? 'bg-amber-500 text-white font-bold' :
          isInRW ? 'bg-amber-500/30 text-white' :
          'text-white hover:bg-white/15'
        }`}
        data-testid={`rw-cal-day-${day}`}
      >
        {day}
      </button>
    );
  })}
</div>
<div className="mt-2 text-[8px] text-white">
  Click any day to select the reading week. Weeks after the reading week will shift up by...
</div>
</div>
);
})();
</div>
)}
</div>

{Array.from({ length: LAST_WEEK - FIRST_WEEK + 1 }, (_, i) => i + FIRST_WEEK).map((weekNum) => {
  const weekDates = getWeekDates(weekNum, semesterStart, readingWeekStart);
  const weekStart = new Date(weekDates.start);
  const weekEnd = new Date(weekDates.end);
  const formatDate = (d: Date) => d.toLocaleDateString('en-US', { month: 'short', day: 'numeric' });
  const dateRange = `${formatDate(weekStart)} - ${formatDate(weekEnd)}`;
  const edit = weekMappingEdits[weekNum] || { confirmed: false, courseWeekLabel: '', notes: '' };
  const isConfirmed = edit.confirmed;
  const hasCustomLabel = edit.courseWeekLabel && edit.courseWeekLabel !== '';
  const effectiveRWStart = selectedReadingWeekStart || readingWeekStart;
  const currentWeek = getWeekNumber(new Date(), semesterStart, effectiveRWStart);
  const isCurrent = weekNum === currentWeek;

  return (
    <div
      key={weekNum}
      className={`flex items-center gap-2 px-2 py-1.5 rounded ${isCurrent ? 'bg-blue-500/10 border bo...
      data-testid={`week-mapping-row-${weekNum}`}
    >
      <button
        onClick={() => {
          const newState = { ...edit, confirmed: !isConfirmed };
          setWeekMappingEdits(prev => ({ ...prev, [weekNum]: newState }));
          saveWeekMapping(weekNum, newState);
        }}
        className={`flex-shrink-0 w-5 h-5 rounded border flex items-center justify-center transition-...
          isConfirmed
            ? 'bg-green-500/30 border-green-500/50 text-green-300'
            : 'bg-white/5 border-white/20 text-white/30 hover:border-white/40'
        }`}
        data-testid={`button-confirm-week-${weekNum}`}
      >
        {isConfirmed && <Check className="h-3 w-3" />}
      </button>

      <div className="flex-1 min-w-0">
        <div className="flex items-center gap-1.5">
          <span className={`text-[10px] font-medium ${isCurrent ? 'text-blue-300' : 'text-white'}`}>
            Week {weekNum}
          </span>
          <span className="text-[8px] text-white">{dateRange}</span>
        </div>
        {isCurrent && <span className="text-[7px] px-1 py-0.5 bg-blue-500/20 text-blue-300 rounded"...
          {hasCustomLabel && (
            <span className="text-[8px] px-1 py-0.5 bg-amber-500/15 text-amber-300 rounded">
              Course: {edit.courseWeekLabel}
            </span>
          )}
        </div>
      </div>
    </div>

    <button
      onClick={() => handleWeekFileUpload(weekNum, 'reading')}
      disabled={weekUploadingState[`${weekNum}-reading`]}
      className="flex-shrink-0 h-5 px-2.5 text-[9px] font-medium bg-white hover:bg-white/90 text-bl...
      data-testid={`button-upload-reading-${weekNum}`}
    >
      {weekUploadingState[`${weekNum}-reading`] ? <Loader2 className="h-3 w-3 animate-spin" /> : <U...
      Reading
    </button>

    <button
      onClick={() => handleWeekFileUpload(weekNum, 'module')}
      disabled={weekUploadingState[`${weekNum}-module`]}
      className="flex-shrink-0 h-5 px-2.5 text-[9px] font-medium bg-white hover:bg-white/90 text-bl...
      data-testid={`button-upload-module-${weekNum}`}
    >
      {weekUploadingState[`${weekNum}-module`] ? <Loader2 className="h-3 w-3 animate-spin" /> : <Up...
      Module
    </button>
  )}

```

```

<div className="relative flex-shrink-0">
  <button
    onClick={() => { setCourseWeekCalendarOpen(courseWeekCalendarOpen === weekNum ? null : week...
    className="w-20 h-5 text-[8px] bg-white/10 border border-white/25 rounded px-1.5 text-white...
    data-testid={ `button-course-week-label-${weekNum}` }
  >
    <Calendar className="h-2.5 w-2.5 flex-shrink-0 text-white/50" />
    <span className="truncate">{edit.courseWeekLabel || 'Set week'}</span>
  </button>
  {courseWeekCalendarOpen === weekNum && (
    <div className="absolute right-0 top-6 z-50 bg-gray-900 border border-white/25 rounded-lg p...
    <div className="flex items-center justify-between mb-1">
      <button onClick={() => setCourseWeekCalMonth(new Date(courseWeekCalMonth.getFullYear(),...
      <span className="text-[9px] text-white font-medium">{courseWeekCalMonth.toLocaleDateStr...
      <button onClick={() => setCourseWeekCalMonth(new Date(courseWeekCalMonth.getFullYear(),...
    </div>
    <div className="grid grid-cols-7 gap-0.5 text-center">
      {[ 'S', 'M', 'T', 'W', 'T', 'F', 'S' ].map((d, i) => <div key={i} className="text-[7px] text-wh...
      {(() => {
        const yr = courseWeekCalMonth.getFullYear();
        const mo = courseWeekCalMonth.getMonth();
        const firstDow = new Date(yr, mo, 1).getDay();
        const dim = new Date(yr, mo + 1, 0).getDate();
        const cells = [];
        for (let i = 0; i < firstDow; i++) cells.push(<div key={`e-${i}`} />);
        for (let d = 1; d <= dim; d++) {
          const dt = new Date(yr, mo, d);
          const wn = getWeekNumber(dt, semesterStart, readingWeekStart);
          const isThisWeek = wn === weekNum;
          cells.push(
            <button
              key={d}
              className={`text-[8px] py-0.5 rounded transition-colors ${isThisWeek ? 'bg-blue...
              onClick={() => {
                const label = `Week ${weekNum}`;
                const newState = { ...edit, courseWeekLabel: label };
                setWeekMappingEdits(prev => ({ ...prev, [weekNum]: newState }));
                saveWeekMapping(weekNum, newState);
                setCourseWeekCalendarOpen(null);
              }}
            >{d}</button>
          );
        }
        return cells;
      })())
    </div>
    <div className="mt-1 flex gap-1">
      <input
        type="text"
        placeholder="Custom label"
        value={edit.courseWeekLabel}
        onChange={(e) => setWeekMappingEdits(prev => ({ ...prev, [weekNum]: { ...edit, course...
        onBlur={() => saveWeekMapping(weekNum, edit)}
        className="flex-1 h-5 text-[8px] bg-white/10 border border-white/25 rounded px-1.5 te...
        data-testid={`input-course-week-label-${weekNum}`}
      />
      <button onClick={() => setCourseWeekCalendarOpen(null)} className="text-[7px] text-whit...
    </div>
  )}
</div>

<input
  type="text"
  placeholder="Notes"
  value={edit.notes}
  onChange={(e) => {
    setWeekMappingEdits(prev => ({ ...prev, [weekNum]: { ...edit, notes: e.target.value } }));
  }}
  onBlur={() => saveWeekMapping(weekNum, edit)}
  className="w-20 h-5 text-[8px] bg-white/10 border border-white/25 rounded px-1.5 text-white p...
  data-testid={`input-week-notes-${weekNum}`}
/>
</div>
);
}}
</div>
)}
</div>

<div ref={assignmentsRef} style={{ marginTop: '10px' }}>
  <div style={{ border: '2px solid rgba(255,255,255,0.2)', borderRadius: '8px', padding: '12px' }}>
    <div
      className="flex items-center justify-between cursor-pointer group"
      onClick={() => { const next = !showAssignments; setShowAssignments(next); if (next) setTimeout(() => { ...
      data-testid="button-toggle-assignments"
    >
      <div className="flex items-center gap-2">
        <ClipboardList className="h-3.5 w-3.5 text-white/70" />
        <h3 className="text-[11px] font-medium text-white uppercase">Assignments</h3>
        <span className="text-[9px] text-white">
          {completedCount}/{courseTasks.length} done
          {totalWeight > 0 && <span className="text-[11px] font-medium" style={{ color: totalWeight > 100 ? '...
        </span>
      </div>
    </div>
    <div className="flex items-center gap-1.5">
      {showAssignments && (
        <Button
          size="sm"
          onClick={(e) => { e.stopPropagation(); setShowAddForm(!showAddForm); }}
          className="px-2 text-[9px] bg-white/10 hover:bg-white/20 text-white border border-white/20"
          style={{ height: '19px', marginBottom: '5px' }}
          data-testid="button-add-assignment"
        >
          <Plus className="h-3 w-3 mr-1" />
          Add
        </Button>
      )
    </div>
  </div>

```

```

    ))
    {showAssignments ? <ChevronDown className="h-3 w-3 text-white/50" /> : <ChevronRight className="h-3 w-3 text-white/50" />}
  </div>
</div>

{showAssignments && (<

{showAddForm && (
  <div className="bg-white/5 border border-white/15 rounded-lg p-3 mb-3 space-y-2" data-testid="add-assig...
    <div className="grid grid-cols-2 gap-2">
      <div>
        <Label className="text-[9px] text-white mb-0.5 block">Title *</Label>
        <Input
          value={newTask.title}
          onChange={(e) => setNewTask({ ...newTask, title: e.target.value })}
          placeholder="e.g. Midterm Exam"
          className="h-7 text-[10px] bg-white/10 border-white/15 text-white placeholder:text-white/25"
          data-testid="input-task-title"
        />
      </div>
      <div>
        <Label className="text-[9px] text-white mb-0.5 block">Type</Label>
        <select
          value={newTask.type}
          onChange={(e) => setNewTask({ ...newTask, type: e.target.value })}
          className="w-full h-7 rounded bg-white/10 border border-white/15 text-white text-[10px] px-1.5"
          data-testid="select-task-type"
        >
          {TASK_TYPE_OPTIONS.map((t) => (
            <option key={t.value} value={t.value} className="bg-gray-800">{t.label}</option>
          ))}
        </select>
      </div>
    </div>
    <div className="grid grid-cols-2 gap-2">
      <div>
        <Label className="text-[9px] text-white mb-0.5 block">Due Date *</Label>
        <Input
          type="date"
          value={newTask.dueDate}
          onChange={(e) => setNewTask({ ...newTask, dueDate: e.target.value })}
          className="h-7 !text-[10px] text-white bg-white/10 border-white/15"
          style={{ colorScheme: 'dark' }}
          data-testid="input-task-due-date"
        />
      </div>
      <div>
        <Label className="text-[9px] text-white mb-0.5 block">Due Time</Label>
        <Input
          type="time"
          value={newTask.dueTime}
          onChange={(e) => setNewTask({ ...newTask, dueTime: e.target.value })}
          className="h-7 !text-[10px] text-white bg-white/10 border-white/15"
          style={{ colorScheme: 'dark' }}
          data-testid="input-task-due-time"
        />
      </div>
    </div>
    <div>
      <Label className="text-[9px] text-white mb-0.5 block">Description</Label>
      <Input
        value={newTask.description}
        onChange={(e) => setNewTask({ ...newTask, description: e.target.value })}
        placeholder="Optional description"
        className="h-7 text-[10px] bg-white/10 border-white/15 text-white placeholder:text-white/25"
        data-testid="input-task-description"
      />
    </div>
    <div className="grid grid-cols-3 gap-2">
      <div>
        <Label className="text-[9px] text-white mb-0.5 block">Grade Weight (%)</Label>
        <Input
          type="number"
          value={newTask.gradeWeight}
          onChange={(e) => setNewTask({ ...newTask, gradeWeight: e.target.value })}
          placeholder="e.g. 20"
          className="h-7 text-[10px] bg-white/10 border-white/15 text-white placeholder:text-white/25"
          data-testid="input-task-weight"
        />
      </div>
      <div>
        <Label className="text-[9px] text-white mb-0.5 block">Total Points</Label>
        <Input
          type="number"
          value={newTask.gradeTotal}
          onChange={(e) => setNewTask({ ...newTask, gradeTotal: e.target.value })}
          placeholder="e.g. 100"
          className="h-7 text-[10px] bg-white/10 border-white/15 text-white placeholder:text-white/25"
          data-testid="input-task-total"
        />
      </div>
      <div>
        <Label className="text-[9px] text-white mb-0.5 block">Score Earned</Label>
        <Input
          type="number"
          value={newTask.gradeValue}
          onChange={(e) => setNewTask({ ...newTask, gradeValue: e.target.value })}
          placeholder="e.g. 85"
          className="h-7 text-[10px] bg-white/10 border-white/15 text-white placeholder:text-white/25"
          data-testid="input-task-value"
        />
      </div>
    </div>
    <div className="grid grid-cols-2 gap-2">

```

```

<div>
  <Label className="text-[9px] text-white mb-0.5 block">Reminder 1</Label>
  <select
    value={newTask.reminder1}
    onChange={(e) => setNewTask({ ...newTask, reminder1: parseInt(e.target.value) })}
    className="w-full h-7 rounded bg-white/10 border border-white/15 text-white text-[10px] px-1.5"
    data-testid="select-reminder-1"
  >
    {REMINDER_PRESETS.map((r) => (
      <option key={r.value} value={r.value} className="bg-gray-800">{r.label}</option>
    ))}
  </select>
</div>
<div>
  <Label className="text-[9px] text-white mb-0.5 block">Reminder 2</Label>
  <select
    value={newTask.reminder2}
    onChange={(e) => setNewTask({ ...newTask, reminder2: parseInt(e.target.value) })}
    className="w-full h-7 rounded bg-white/10 border border-white/15 text-white text-[10px] px-1.5"
    data-testid="select-reminder-2"
  >
    {REMINDER_PRESETS.map((r) => (
      <option key={r.value} value={r.value} className="bg-gray-800">{r.label}</option>
    ))}
  </select>
</div>
</div>

<div className="grid grid-cols-2 gap-2">
  <div>
    <Label className="text-[9px] text-white mb-0.5 block">Reminder 3</Label>
    <select
      value={newTask.reminder3}
      onChange={(e) => setNewTask({ ...newTask, reminder3: parseInt(e.target.value) })}
      className="w-full h-7 rounded bg-white/10 border border-white/15 text-white text-[10px] px-1.5"
      data-testid="select-reminder-3"
    >
      {REMINDER_PRESETS.map((r) => (
        <option key={r.value} value={r.value} className="bg-gray-800">{r.label}</option>
      ))}
    </select>
  </div>
  <div>
    <Label className="text-[9px] text-white mb-0.5 block">Reminder 4</Label>
    <select
      value={newTask.reminder4}
      onChange={(e) => setNewTask({ ...newTask, reminder4: parseInt(e.target.value) })}
      className="w-full h-7 rounded bg-white/10 border border-white/15 text-white text-[10px] px-1.5"
      data-testid="select-reminder-4"
    >
      {REMINDER_PRESETS.map((r) => (
        <option key={r.value} value={r.value} className="bg-gray-800">{r.label}</option>
      ))}
    </select>
  </div>
</div>

<div className="flex items-center justify-end gap-2 pt-1">
  <Button
    size="sm"
    variant="ghost"
    onClick={() => { setShowAddForm(false); setNewTask(createEmptyTaskForm()); }}
    className="h-7 text-[10px] text-white hover:text-white hover:bg-white/10"
    data-testid="button-cancel-add"
  >
    Cancel
  </Button>
  <Button
    size="sm"
    onClick={handleAddTask}
    disabled={createTaskMutation.isPending}
    className="h-7 text-[10px] bg-white/20 hover:bg-white/30 text-white"
    data-testid="button-save-assignment"
  >
    {createTaskMutation.isPending ? "Adding..." : "Add to Calendar"}
  </Button>
</div>
</div>
))

{courseTasks.length === 0 && !showAddForm && (
  <div className="text-center py-8 text-white/30 text-[10px]" data-testid="text-no-assignments">
    <ClipboardCheck className="h-8 w-8 mx-auto mb-2 opacity-30" />
    <p>No assignments yet</p>
    <p className="mt-1">Click "Add" to create your first assignment</p>
  </div>
)}

{courseTasks.length > 0 && (() => {
  const SortIcon = ({ field }: { field: SortField }) => {
    if (sortField !== field) return null;
    return sortDir === 'asc' ? <ArrowUp className="h-2 w-2 inline ml-0.5" /> : <ArrowDown className="h-2 w-2 inline ml-0.5" />
  };
  const hdrCls = (field: SortField) =>
    'cursor-pointer select-none hover:text-white/80 transition-colors ${sortField === field ? 'text-white...'
  return (
    <div className="flex items-end px-1.5 py-1 text-[8px] font-bold text-white" style={{ margin: '0 4px', ...
    <div className="flex-shrink-0" style={{ width: '14px', marginRight: '10px' }} />
    <div className="flex-shrink-0 flex justify-center" style={{ width: '16px', marginRight: '10px', ove...
    <span className="text-[8px] font-bold text-white" style={{ whiteSpace: 'nowrap' }}>Done</span>
    </div>
    <div className="flex-shrink-0 flex justify-center" style={{ width: '15px', marginLeft: '8px', margi...
    <span className="text-[8px] font-bold text-white" style={{ whiteSpace: 'nowrap' }}>Assign</span>
    </div>
    <div className="flex-shrink-0 flex justify-center" style={{ width: '14px', marginLeft: '8px', margi...
    </div>
    <div className="flex-shrink-0 flex justify-center" style={{ width: '19px', marginLeft: '15px', marg...
    <span className="text-[8px] font-bold text-white" style={{ whiteSpace: 'nowrap' }}>Comments</span>
  )
})

```



```

        <div style={{ width: '24px', height: '14px' }} />
        <div style={{ padding: '2px', marginLeft: '6px' }}><div style={{ width: '15px', height: '15px' }} /...
        <div style={{ padding: '2px' }}><div style={{ width: '15px', height: '15px' }} /></div>
      </div>
    </div>
  )}
</>}}
</div>
</div>
</div>
</div>
</div>
</div>

<div className="px-4 py-3 border-t border-white/20 flex items-center justify-between flex-shrink-0" style={{ ...
  <div>
    {onDeleteCourse && (
      <button
        onClick={() => setShowDeleteConfirm(true)}
        className="p-1.5 rounded hover:bg-white/15 transition-colors"
        data-testid="button-delete-course"
      >
        <Trash2 className="w-3.5 h-3.5 text-white/40 hover:text-red-400" />
      </button>
    )}
  </div>
  <div className="flex gap-2">
    <Button
      type="button"
      variant="outline"
      onClick={(e) => {
        e.preventDefault(); e.stopPropagation();
        if (onLiveColorChange) {
          onLiveColorChange({
            color: courseInfo.color || '#3b82f6',
            colorEnd: courseInfo.colorEnd || courseInfo.color || '#3b82f6',
            colorStops: courseInfo.colorStops || '',
            borderColor: courseInfo.borderColor || '',
            courseRowColor: courseInfo.courseRowColor || '',
            taskBgColor: courseInfo.taskBgColor || '',
          });
        }
        onClose();
      }}
      onPointerDown={(e) => { e.stopPropagation(); }}
      disabled={expandedTaskId !== null || isEditingInfo}
      className={`border transition-all duration-200 h-6 w-[110px] disabled:cursor-not-allowed ${!isEditingIn...
      style={{ fontSize: '12px', pointerEvents: expandedTaskId !== null || isEditingInfo ? 'none' : 'auto', p...
      data-testid="button-cancel-course-detail"
    >
      Cancel
    </Button>
    <Button
      variant="outline"
      onClick={() => {
        if (onSaveCourseInfo) {
          const semKey = semesterKeyFromTermYear(editInfo.semesterTerm, editInfo.year);
          onSaveCourseInfo({ ...editInfo, semesterKey: semKey || undefined, courseRank: editInfo.courseRank |...
        }
        if (editInfo.professor?.trim()) {
          fetch('/api/key-contacts/sync-professor', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ professorName: editInfo.professor, professorEmail: editInfo.professorEmail...
          }).catch(() => {});
        }
        onClose();
      }}
      disabled={expandedTaskId !== null || isEditingInfo}
      className={`border transition-all duration-200 h-6 w-[110px] disabled:cursor-not-allowed ${!isEditingIn...
      style={{
        boxShadow: !isEditingInfo && expandedTaskId === null ? '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(...
        fontSize: '12px'
      }}
      data-testid="button-save-course-detail"
    >
      Save
    </Button>
  </div>
</div>
</div>
</div>

{commentTarget && (
  <div
    className="fixed z-[10005] flex flex-col rounded-lg overflow-hidden"
    style={{
      top: '50%', left: '50%', transform: 'translate(-50%, -50%)',
      width: '320px', minHeight: '240px',
      background: 'linear-gradient(135deg, #fef08a 0%, #fde047 100%)',
      boxShadow: '0 8px 32px rgba(0,0,0,0.3), 4px 4px 0 rgba(0,0,0,0.1)',
      border: '1px solid rgba(0,0,0,0.1)',
    }}
    onClick={(e) => e.stopPropagation()}
    data-testid="comment-postit"
  >
    <div className="flex items-center justify-between px-3 py-2 border-b border-amber-400/50">
      <span className="text-[11px] font-bold text-amber-900 truncate flex-1">{commentTarget.label}</span>
      <button onClick={() => { saveComment(); setCommentTarget(null); }} className="text-amber-800 hover:text-a...
      <X className="h-4 w-4" />
    </button>
  </div>
  <div className="flex-1 p-2">
    {commentLoading ? (
      <div className="flex items-center justify-center h-full"><Loader2 className="h-5 w-5 animate-spin text-...
    ) : (
      <textarea
        className="w-full h-[160px] bg-transparent text-[12px] text-amber-950 placeholder:text-amber-700/50 r...
        placeholder="Write your comments here..."
      >

```



```

        value={commentText}
        onChange={(e) => setCommentText(e.target.value)}
        autoFocus
        data-testid="input-comment-text"
      />
    ))
  </div>
  <div className="flex items-center justify-end px-3 py-2 border-t border-amber-400/50">
    <button
      onClick={saveComment}
      disabled={commentSaving}
      className="text-[10px] font-bold px-3 py-1 rounded bg-amber-800 text-white hover:bg-amber-900 disabled:..."
      data-testid="button-save-comment"
    >
      {commentSaving ? 'Saving...' : 'Save'}
    </button>
  </div>
</div>
))

{showDeleteConfirm && (
  <div
    className="fixed inset-0 z-[10004] flex items-center justify-center"
    style={{ background: 'rgba(0,0,0,0.6)' }}
    onClick={() => setShowDeleteConfirm(false)}
    data-testid="delete-course-confirm-overlay"
  >
    <div
      className="rounded-lg overflow-hidden text-white w-[340px]"
      style={{
        background: 'linear-gradient(180deg, #1e1e2e 0%, #0d0d1a 100%)',
        border: '1px solid rgba(255,255,255,0.2)',
        boxShadow: '0 8px 32px rgba(0,0,0,0.4)',
      }}
      onClick={(e) => e.stopPropagation()}
      data-testid="delete-course-confirm-dialog"
    >
      <div className="px-5 py-4 border-b border-white/15 flex items-center gap-2">
        <Trash2 className="w-4 h-4 text-red-400" />
        <h3 className="text-sm font-semibold">Delete Course</h3>
      </div>
      <div className="px-5 py-4">
        <p className="text-sm text-white/80 leading-relaxed">
          Are you sure you want to delete <strong>{courseInfo.courseCode}</strong>? This will remove the course...
        </p>
      </div>
      <div className="flex items-center justify-end gap-3 px-5 py-3 border-t border-white/15">
        <button
          className="px-4 py-1.5 text-[11px] bg-white/10 hover:bg-white/20 rounded text-white transition-colors"
          onClick={() => setShowDeleteConfirm(false)}
          data-testid="button-cancel-delete-course"
        >
          Cancel
        </button>
        <button
          className="px-4 py-1.5 text-[11px] bg-red-500/80 hover:bg-red-500 rounded text-white transition-colors"
          onClick={() => {
            onDeleteCourse?();
            setShowDeleteConfirm(false);
          }}
          data-testid="button-confirm-delete-course"
        >
          Delete
        </button>
      </div>
    </div>
  </div>
))

{notesTaskId !== null && (
  <div className="fixed inset-0 flex items-center justify-center" style={{ zIndex: 10020, background: 'rgba(0,0,0,0.6)' }}>
    <div className="rounded-xl border border-white/20 p-5 flex flex-col shadow-2xl" style={{ width: '420px', maxHeight: '420px' }}>
      <div className="flex items-center justify-between mb-4">
        <span className="text-[13px] font-semibold text-white tracking-wide">Notes</span>
        <button className="text-white/50 hover:text-white p-1 rounded hover:bg-white/10 transition-colors" onPo...>
          Cancel
        </button>
      </div>
      <textarea
        className="flex-1 w-full rounded-lg border border-white/20 bg-white/5 text-white text-[12px] p-3 resize-none"
        style={{ minHeight: '200px' }}
        value={notesText}
        onChange={(e) => setNotesText(e.target.value)}
        placeholder="Type your notes here..."
        autoFocus
        data-testid="input-notes-text"
      />
      <div className="flex justify-end gap-2 mt-4">
        <button
          className="px-5 py-2 text-[11px] font-medium text-white/70 rounded-lg border border-white/20 hover:bg-white/10"
          onPointerDown={(e) => { e.preventDefault(); e.stopPropagation(); setNotesTaskId(null); setNotesText('') }}
          data-testid="button-notes-cancel"
        >
          Cancel
        </button>
        <button
          className="px-5 py-2 text-[11px] font-medium text-white rounded-lg bg-blue-600 hover:bg-blue-500 disabled:..."
          disabled={notesSaving}
          onPointerDown={async (e) => {
            e.preventDefault();
            e.stopPropagation();
            setNotesSaving(true);
            try {
              await apiRequest(`/api/tasks/${notesTaskId}`, 'PATCH', { notes: notesText || null });
              queryClient.invalidateQueries({ queryKey: ['/api/tasks'] });
              toast({ title: "Notes saved" });
              setNotesTaskId(null);
              setNotesText('');
            } catch (err) { console.error('[Notes] save error:', err); toast({ title: "Error saving notes", var... }); }
            setNotesSaving(false);
          }}
        >
          Save
        </button>
      </div>
    </div>
  </div>
)

```

```

        }}
        data-testid="button-notes-save"
      >
        {notesSaving ? <Loader2 className="h-3 w-3 animate-spin inline mr-1" /> : null}
        Save
      </button>
    </div>
  </div>
</div>
)}
</div>,
document.body
);
}

```

`client/src/components/DeviceCard.tsx`

```

import { Device, UpdateDeviceRequest } from "@shared/schema";
import {
  Lightbulb,
  Power,
  Thermometer,
  Activity,
  Fan,
  Tv,
  Wifi,
  Lock,
  MoreVertical,
  Trash2
} from "lucide-react";
import { motion } from "framer-motion";
import { cn } from "@lib/utils";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from "@components/ui/dropdown-menu";

interface DeviceCardProps {
  device: Device;
  onUpdate: (updates: UpdateDeviceRequest) => void;
  onDelete: (id: number) => void;
}

const iconMap: Record<string, any> = {
  lightbulb: Lightbulb,
  power: Power,
  thermometer: Thermometer,
  activity: Activity,
  fan: Fan,
  tv: Tv,
  wifi: Wifi,
  lock: Lock,
};

export function DeviceCard({ device, onUpdate, onDelete }: DeviceCardProps) {
  const Icon = iconMap[device.icon] || Activity;
  const isClimate = device.type === 'climate';
  const isSensor = device.type === 'sensor';
  const isActive = device.isOn;

  return (
    <motion.div
      initial={{ opacity: 0, scale: 0.95 }}
      animate={{ opacity: 1, scale: 1 }}
      className={cn(
        "relative p-5 rounded-2xl border transition-all duration-300 group",
        "flex flex-col gap-4 overflow-hidden",
        isActive
          ? "bg-primary/10 border-primary/40 shadow-lg shadow-primary/10"
          : "bg-card border-white/5 hover:border-white/10"
      )}
    >
      { /* Header */ }
      <div className="flex justify-between items-start z-10">
        <div className={cn(
          "p-3 rounded-xl transition-colors duration-300",
          isActive ? "bg-primary text-white" : "bg-secondary text-muted-foreground"
        )}>
          <Icon className="w-6 h-6" />
        </div>

        <div className="flex gap-2">
          { /* Main Toggle Switch (visible if not a sensor) */ }
          { !isSensor && (
            <button
              onClick={() => onUpdate({ isOn: !device.isOn })}
              className={cn(
                "w-10 h-6 rounded-full relative transition-colors duration-300 flex items-center px-0.5",
                isActive ? "bg-primary" : "bg-muted"
              )}
            >
              <motion.div
                animate={{ x: isActive ? 16 : 0 }}
                className="w-5 h-5 bg-white rounded-full shadow-sm"
              />
            </button>
          ) }

          { /* Context Menu */ }
          <DropdownMenu>
            <DropdownMenuTrigger asChild>
              <button className="text-muted-foreground hover:text-white p-1 rounded-md hover:bg-white/10 transition-c...

```

```

        <MoreVertical className="w-5 h-5" />
      </button>
    </DropdownMenuTrigger>
    <DropdownMenuContent align="end" className="bg-card border-white/10 text-foreground">
      <DropdownMenuItem
        onClick={() => onDelete(device.id)}
        className="text-destructive focus:text-destructive focus:bg-destructive/10 cursor-pointer"
      >
        <Trash2 className="w-4 h-4 mr-2" />
        Remove Device
      </DropdownMenuItem>
    </DropdownMenuContent>
  </DropdownMenu>
</div>

{/* Info */}
<div className="z-10">
  <h3 className="text-lg font-bold font-display truncate">{device.name}</h3>
  <p className="text-sm text-muted-foreground">{device.room}</p>
</div>

{/* Controls / Status */}
<div className="mt-auto z-10 min-h-[2rem] flex items-end">
  {device.type === 'light' && isActive && {
    <div className="w-full">
      <div className="flex justify-between text-xs mb-2 text-muted-foreground">
        <span>Brightness</span>
        <span>{device.value}%</span>
      </div>
      <input
        type="range"
        min="0"
        max="100"
        value={device.value || 0}
        onChange={(e) => onUpdate({ value: parseInt(e.target.value) })}
        className="w-full h-1.5 bg-secondary rounded-lg appearance-none cursor-pointer accent-primary"
      />
    </div>
  )}

  {isClimate && {
    <div className="flex items-center justify-between w-full">
      <div className="text-2xl font-bold font-display">
        {device.value}{device.unit || '°C'}
      </div>
      {isActive && {
        <div className="flex gap-2">
          <button
            onClick={() => onUpdate({ value: (device.value || 20) - 1 })}
            className="p-1 rounded bg-secondary hover:bg-secondary/80 text-foreground"
          >
            -
          </button>
          <button
            onClick={() => onUpdate({ value: (device.value || 20) + 1 })}
            className="p-1 rounded bg-secondary hover:bg-secondary/80 text-foreground"
          >
            +
          </button>
        </div>
      }
    </div>
  )}

  {isSensor && {
    <div className="text-2xl font-bold text-accent font-display">
      {device.value}{device.unit}
    </div>
  }
}
</div>

{/* Glow Effect Background */}
{isActive && {
  <div className="absolute -right-10 -bottom-10 w-40 h-40 bg-primary/20 blur-[60px] rounded-full pointer-events-none" />
}}
</motion.div>
);
}

```

`client/src/components/FastInput.tsx`

```

import { useState, useEffect, useRef, startTransition, useCallback, forwardRef } from "react";

interface FastInputProps extends Omit<React.InputHTMLAttributes<HTMLInputElement>, 'onChange'> {
  value: string;
  onValueChange: (value: string) => void;
  onChange?: never;
}

export const FastInput = forwardRef<HTMLInputElement, FastInputProps>((
  ({ value, onValueChange, ...props }, ref) => {
    const [localValue, setLocalValue] = useState(value);
    const isUserTyping = useRef(false);

    useEffect(() => {
      if (!isUserTyping.current) {
        setLocalValue(value);
      }
    }, [value]);

    const handleChange = useCallback((e: React.ChangeEvent<HTMLInputElement>) => {
      const newVal = e.target.value;
      isUserTyping.current = true;
    }, []);
  })

```

```

        setLocalValue(newVal);
        startTransition(() => {
            onValueChange(newVal);
            isUserTyping.current = false;
        });
    }, [onValueChange]);

    return (
        <input
            {...props}
            ref={ref}
            value={localValue}
            onChange={handleChange}
        />
    );
}
);
FastInput.displayName = "FastInput";

interface FastTextareaProps extends Omit<React.TextareaHTMLAttributes<HTMLTextAreaElement>, 'onChange'> {
    value: string;
    onValueChange: (value: string) => void;
    onChange?: never;
}

export const FastTextarea = forwardRef<HTMLTextAreaElement, FastTextareaProps>(
    ({ value, onValueChange, ...props }, ref) => {
        const [localValue, setLocalValue] = useState(value);
        const isUserTyping = useRef(false);

        useEffect(() => {
            if (!isUserTyping.current) {
                setLocalValue(value);
            }
        }, [value]);

        const handleChange = useCallback((e: React.ChangeEvent<HTMLTextAreaElement>) => {
            const newVal = e.target.value;
            isUserTyping.current = true;
            setLocalValue(newVal);
            startTransition(() => {
                onValueChange(newVal);
                isUserTyping.current = false;
            });
        }, [onValueChange]);

        return (
            <textarea
                {...props}
                ref={ref}
                value={localValue}
                onChange={handleChange}
            />
        );
    }
);
FastTextarea.displayName = "FastTextarea";

```

`client/src/components/NewCourseWizard.tsx`

```

import { useState, useRef } from "react";
import { createPortal } from "react-dom";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
import {
    GraduationCap,
    ChevronLeft,
    ChevronRight,
    X,
    Plus,
    Trash2,
    Bell,
    BookOpen,
    ClipboardCheck,
    FileText,
    MessageSquare,
    Vote,
    AlertCircle,
} from "lucide-react";
import { REMINDER_OPTIONS, TASK_TYPES } from "@shared/schema";

const WIZARD_STEPS = [
    { id: 1, label: "Course Info" },
    { id: 2, label: "Professor" },
    { id: 3, label: "Course Type" },
    { id: 4, label: "Semester" },
    { id: 5, label: "Assignments & Grades" },
    { id: 6, label: "Review" },
];

const COURSE_TYPES = [
    { value: "core", label: "Core Course", description: "Required for your program" },
    { value: "open_elective", label: "Open Elective", description: "Elective of your choice" },
    { value: "liberal_studies", label: "Liberal Studies", description: "General education requirement" },
];

const TASK_TYPE_OPTIONS = [
    { value: "reading", label: "Reading" },
    { value: "essay", label: "Essay" },
    { value: "exam", label: "Exam" },
    { value: "quiz", label: "Quiz" },
    { value: "discussion", label: "Discussion Post" },
];

```

```

    { value: "poll", label: "Review Poll" },
    { value: "project", label: "Project" },
    { value: "module", label: "Module" },
    { value: "class", label: "Class" },
    { value: "other", label: "Other" },
  ];

interface WizardTask {
  title: string;
  type: string;
  dueDate: string;
  dueTime: string;
  description: string;
  reminder1: number;
  reminder2: number;
  reminder3: number;
  reminder4: number;
  gradeWeight: number | null;
  gradeTotal: number | null;
}

interface WizardData {
  courseCode: string;
  courseName: string;
  professorName: string;
  professorEmail: string;
  courseType: string;
  color: string;
  colorEnd: string;
  colorStops: string;
  borderColor: string;
  courseRowColor: string;
  taskBgColor: string;
  semesterType: string;
  deliveryMode: string;
  classDay: string;
  classDay2: string;
  classTime: string;
  classEndTime: string;
  startDate: string;
  endDate: string;
  springSummerTerm: string;
  zoomLink: string;
  tasks: WizardTask[];
}

interface NewCourseWizardProps {
  onSave: (data: WizardData) => void;
  onClose: () => void;
  existingSemesterType?: string;
  colorSettings?: { mainBackground: string; mainBackgroundGradientEnd: string; headerBar: string };
}

const COLORS = [
  "#6366F1", "#EC4899", "#F59E0B", "#10B981", "#3B82F6",
  "#8B5CF6", "#EF4444", "#14B8A6", "#F97316", "#06B6D4",
];

const DAY_OPTIONS = [
  { value: "", label: "None" },
  { value: "monday", label: "Monday" },
  { value: "tuesday", label: "Tuesday" },
  { value: "wednesday", label: "Wednesday" },
  { value: "thursday", label: "Thursday" },
  { value: "friday", label: "Friday" },
  { value: "saturday", label: "Saturday" },
  { value: "sunday", label: "Sunday" },
];

function createEmptyTask(): WizardTask {
  return {
    title: "",
    type: "reading",
    dueDate: "",
    dueTime: "",
    description: "",
    reminder1: 30,
    reminder2: 120,
    reminder3: 0,
    reminder4: 0,
    gradeWeight: null,
    gradeTotal: null,
  };
}

export function NewCourseWizard({ onSave, onClose, existingSemesterType, colorSettings }: NewCourseWizardProps) {
  const [step, setStep] = useState(1);
  const [data, setData] = useState<WizardData>({
    courseCode: "",
    courseName: "",
    professorName: "",
    professorEmail: "",
    courseType: "core",
    color: "#6366F1",
    colorEnd: "#EC4899",
    colorStops: "",
    borderColor: "",
    courseRowColor: "",
    taskBgColor: "",
    semesterType: existingSemesterType || "winter",
    deliveryMode: "",
    classDay: "",
    classDay2: "",
    classTime: "",
    classEndTime: "",
    startDate: "",
    endDate: "",
    springSummerTerm: "",
  });

```

```

    zoomLink: "",
    tasks: [],
  });

const [wizardActiveGradientStop, setWizardActiveGradientStop] = useState<'start' | 'end' | number | null>(null);
const wizardGradBarRef = useRef<HTMLDivElement>(null);

const updateField = <K extends keyof WizardData>(field: K, value: WizardData[K]) => {
  setData(prev => ({ ...prev, [field]: value }));
};

const addTask = () => {
  setData(prev => ({ ...prev, tasks: [...prev.tasks, createEmptyTask()] }));
};

const updateTask = (index: number, field: keyof WizardTask, value: any) => {
  setData(prev => {
    const tasks = [...prev.tasks];
    tasks[index] = { ...tasks[index], [field]: value };
    return { ...prev, tasks };
  });
};

const removeTask = (index: number) => {
  setData(prev => ({ ...prev, tasks: prev.tasks.filter((_, i) => i !== index) }));
};

const canProceed = () => {
  switch (step) {
    case 1: return data.courseCode.trim() !== "" && data.courseName.trim() !== "";
    case 2: return true;
    case 3: return data.courseType !== "";
    case 4: return !(data.semesterType === "spring_summer" && !data.springSummerTerm);
    case 5: return true;
    case 6: return true;
    default: return true;
  }
};

const handleNext = () => {
  if (step < 6 && canProceed()) setStep(step + 1);
};

const handleBack = () => {
  if (step > 1) setStep(step - 1);
};

const handleSave = () => {
  onSave(data);
};

const totalGradeWeight = data.tasks.reduce((sum, t) => sum + (t.gradeWeight || 0), 0);

const renderStepIndicator = () => (
  <div className="flex items-center gap-1 px-4 py-2 bg-white/10 border-b border-white/20">
    {WIZARD_STEPS.map((s, i) => (
      <div key={s.id} className="flex items-center">
        <button
          onClick={() => { if (s.id < step) setStep(s.id); }}
          className={`flex items-center gap-1 px-2 py-1 rounded-full text-[9px] transition-all ${
            s.id === step
              ? "bg-white/20 text-white font-medium"
              : s.id < step
                ? "text-white/60 cursor-pointer hover:text-white/80"
                : "text-white/30 cursor-default"
            }`}
        />
        data-testid={`wizard-step-${s.id}`}
      >
        <span className={`w-4 h-4 rounded-full flex items-center justify-center text-[8px] font-bold ${
          s.id === step ? "bg-white text-black" : s.id < step ? "bg-white/40 text-white" : "bg-white/10 text-whit...
        }`} />
        {s.id}
      </span>
      <span className="hidden sm:inline">{s.label}</span>
    </button>
    {i < WIZARD_STEPS.length - 1 && (
      <div className={`w-3 h-px mx-0.5 ${s.id < step ? "bg-white/40" : "bg-white/10"}`} />
    )}
  </div>
  </div>
);

const renderStep1 = () => (
  <div className="space-y-4">
    <div className="text-center mb-4">
      <BookOpen className="h-8 w-8 text-indigo-400 mx-auto mb-2" />
      <h3 className="text-sm font-medium text-white">Course Information</h3>
      <p className="text-[9px] text-white/50 mt-1">Enter the course code and name</p>
    </div>
    <div>
      <Label className="text-[10px] text-white/70 mb-1.5 block">Course Code *</Label>
      <Input
        value={data.courseCode}
        onChange={(e) => updateField("courseCode", e.target.value.toUpperCase())}
        placeholder="e.g. CSOC103"
        className="h-9 text-[11px] bg-white/10 border-white/20 text-white placeholder:text-white/30"
        data-testid="wizard-input-course-code"
      />
    </div>
    <div>
      <Label className="text-[10px] text-white/70 mb-1.5 block">Course Name *</Label>
      <Input
        value={data.courseName}
        onChange={(e) => updateField("courseName", e.target.value)}
        placeholder="e.g. How Society Works"
        className="h-9 text-[11px] bg-white/10 border-white/20 text-white placeholder:text-white/30"
        data-testid="wizard-input-course-name"
      />
    </div>
  </div>
);

```

```

    />
  </div>
  (() => {
    const hexToHue = (c: string) => { const r = parseInt(c.slice(1,3),16)/255, g = parseInt(c.slice(3,5),16)/255,...
    const hueToHex = (hue: number) => `#${[0,8,4].map(n => { const k = (n + hue/30) % 12; const c2 = 0.5 - 0.5 * ...
    const hexToSvPos = (hex: string) => { const r = parseInt(hex.slice(1,3),16)/255, g = parseInt(hex.slice(3,5),...
    const svToHex = (hue: number, sx: number, sy: number) => { const s = sx, v = 1 - sy; const c = v * s, x2 = c ...
    const midStops: Array<{position: number; color: string}> = (() => { try { return data.colorStops ? JSON.parse...
    const allStops = [{ position: 0, color: data.color }, ...midStops, { position: 100, color: data.colorEnd }]}.s...
    const gradientCss = `linear-gradient(to right, ${allStops.map(s => `${s.color} ${s.position}%`).join(', ')}`;
    const getActiveColor = (): string => {
      if (wizardActiveGradientStop === 'start') return data.color;
      if (wizardActiveGradientStop === 'end') return data.colorEnd;
      if (typeof wizardActiveGradientStop === 'number' && midStops[wizardActiveGradientStop]) return midStops[wiz...
      return '#000000';
    };
    const setActiveColor = (hex: string) => {
      if (wizardActiveGradientStop === 'start') updateField('color', hex);
      else if (wizardActiveGradientStop === 'end') updateField('colorEnd', hex);
      else if (typeof wizardActiveGradientStop === 'number') {
        const updated = [...midStops];
        updated[wizardActiveGradientStop] = { ...updated[wizardActiveGradientStop], color: hex };
        updateField('colorStops', JSON.stringify(updated));
      }
    };
    return (
    <div>
      <div className="flex items-start gap-3">
        <div style={{ flex: 1, minWidth: 0 }}>
          <div className="flex items-center justify-between mb-1">
            <Label className="text-[10px] text-white/70">Label Gradient</Label>
            <button className="text-white hover:text-white/80 text-[8px] flex items-center gap-0.5" onClick={() => {
              const revMid = midStops.map(s => ({ position: 100 - s.position, color: s.color })).reverse();
              setData(prev => ({...prev, color: prev.colorEnd, colorEnd: prev.color, colorStops: revMid.length ? ...
              }) data-testid="wizard-button-reverse-gradient"><svg width="10" height="10" viewBox="0 0 16 16" fill=...
            </div>
            <div ref={wizardGradBarRef} className="rounded" style={{ border: '1px solid rgba(255,255,255,0.15)', pa...
              onDoubleClick={() => {
                const bar = wizardGradBarRef.current;
                if (!bar) return;
                const rect = bar.getBoundingClientRect();
                const pct = Math.round(Math.max(5, Math.min(95, ((e.clientX - rect.left - 3) / (rect.width - 6)) * ...
                const leftStop = allStops.filter(s => s.position <= pct).pop();
                const rightStop = allStops.find(s => s.position >= pct);
                const t = rightStop.position === leftStop.position ? 0 : (pct - leftStop.position) / (rightStop.pos...
                const lR = parseInt(leftStop.color.slice(1,3),16), lG = parseInt(leftStop.color.slice(3,5),16), lB ...
                const rR = parseInt(rightStop.color.slice(1,3),16), rG = parseInt(rightStop.color.slice(3,5),16), r...
                const mR = Math.round(lR + (rR-lR)*t), mG = Math.round(lG + (rG-lG)*t), mB = Math.round(lB + (rB-lB...
                const newColor = `#${mR.toString(16).padStart(2,'0')}${mG.toString(16).padStart(2,'0')}${mB.toStrin...
                const newMid = [...midStops, { position: pct, color: newColor }].sort((a, b) => a.position - b.posi...
                const newIdx = newMid.findIndex(s => s.position === pct && s.color === newColor);
                updateField('colorStops', JSON.stringify(newMid));
                setWizardActiveGradientStop(newIdx);
              }>
            <div style={{ height: '18px', borderRadius: '3px', background: gradientCss }} data-testid="wizard-gra...
          </div>
          <div className="relative" style={{ height: '16px', marginTop: '1px' }}>
            <div style={{ position: 'absolute', left: '0px', top: 0, cursor: 'pointer', zIndex: 10 }} onClick={()...
              <svg width="12" height="12" viewBox="0 0 12 12"><polygon points="6,0 12,0 10,10" fill={data.color} ...
            </div>
            {midStops.map((stop, idx) => {
              <div key={idx} style={{ position: 'absolute', left: `calc(${stop.position}% - 6px)`, top: 0, cursor...
                onClick={() => setWizardActiveGradientStop(wizardActiveGradientStop === idx ? null : idx)}
                onPointerDown={() => {
                  e.preventDefault(); e.stopPropagation();
                  const el = e.currentTarget as HTMLElement;
                  el.setPointerCapture(e.pointerId);
                  const bar = wizardGradBarRef.current;
                  if (!bar) return;
                  const barRect = bar.getBoundingClientRect();
                  const barW = barRect.width;
                  const onMove = (ev: PointerEvent) => {
                    const pct = Math.round(Math.max(1, Math.min(99, ((ev.clientX - barRect.left) / barW) * 100)));
                    const updated = [...midStops];
                    updated[idx] = { ...updated[idx], position: pct };
                    setData(prev => ({...prev, colorStops: JSON.stringify(updated.sort((a, b) => a.position - b.p...
                    });
                    const onUp = () => { el.releasePointerCapture(e.pointerId); window.removeEventListener('pointer...
                    window.addEventListener('pointermove', onMove);
                    window.addEventListener('pointerup', onUp);
                  }
                }
                data-testid={`wizard-gradient-stop-mid-${idx}`}>
                <svg width="12" height="12" viewBox="0 0 12 12"><polygon points="6,0 11,6 12 1,6" fill={stop.co...
              </div>
            )}
            <div style={{ position: 'absolute', right: '0px', top: 0, cursor: 'pointer', zIndex: 10 }} onClick={()...
              <svg width="12" height="12" viewBox="0 0 12 12"><polygon points="6,0 12,10 0,10" fill={data.colorEn...
            </div>
          </div>
          {midStops.length === 0 && <div className="text-white/50 text-[9px] mt-1">Double-click gradient bar to a...
          {midStops.length > 0 && <div className="text-white/40 text-[8px]" style={{ marginTop: '4px' }}>Double-c...
          {wizardActiveGradientStop !== null && (
            <div className="mt-1.5 rounded" style={{ border: '1px solid rgba(255,255,255,0.15)', background: 'rgb...
              <div className="flex items-center gap-2 mb-1.5">
                <div className="w-5 h-5 rounded border border-white/30 shrink-0" style={{ backgroundColor: getAct...
                  <span className="text-white/60 text-[8px] uppercase tracking-wider">{wizardActiveGradientStop ===...
                  {typeof wizardActiveGradientStop === 'number' && (
                    <button className="text-red-400/70 hover:text-red-400 text-[8px] ml-auto mr-1" onClick={() => {
                      const updated = midStops.filter((_, i) => i !== wizardActiveGradientStop);
                      updateField('colorStops', updated.length ? JSON.stringify(updated) : '');
                      setWizardActiveGradientStop(null);
                    }>
                  <div data-testid="wizard-button-delete-stop"><Trash2 className="w-3 h-3" /></button>
                )}
                <button className={` ${typeof wizardActiveGradientStop === 'number' ? '' : 'ml-auto '}text-white/4...
              </div>
              {typeof wizardActiveGradientStop === 'number' && (
                <div className="flex items-center gap-2 mb-1.5">
                  <span className="text-white/50 text-[8px]">Position</span>

```



```

        placeholder="e.g. smith@university.ca"
        className="h-9 text-[11px] bg-white/10 border-white/20 text-white placeholder:text-white/30"
        data-testid="wizard-input-professor-email"
      />
    </div>
  </div>
);

const renderStep3 = () => (
  <div className="space-y-4">
    <div className="text-center mb-4">
      <ClipboardCheck className="h-8 w-8 text-amber-400 mx-auto mb-2" />
      <h3 className="text-sm font-medium text-white">Course Type</h3>
      <p className="text-[9px] text-white/50 mt-1">How does this course count toward your degree?</p>
    </div>
    <div className="space-y-2">
      {COURSE_TYPES.map(ct => (
        <button
          key={ct.value}
          onClick={() => updateField("courseType", ct.value)}
          className={`w-full text-left p-3 rounded-lg border transition-all ${
            data.courseType === ct.value
              ? "bg-white/15 border-white/40"
              : "bg-white/5 border-white/10 hover:bg-white/10"
          }`}
          data-testid={`wizard-course-type-${ct.value}`}
        >
          <div className="text-[11px] font-medium text-white">{ct.label}</div>
          <div className="text-[9px] text-white/50 mt-0.5">{ct.description}</div>
        </button>
      ))}
    </div>
  </div>
);

const renderStep4 = () => (
  <div className="space-y-3">
    <div className="text-center mb-3">
      <FileText className="h-8 w-8 text-blue-400 mx-auto mb-2" />
      <h3 className="text-sm font-medium text-white">Semester & Schedule</h3>
      <p className="text-[9px] text-white/50 mt-1">Configure how this course fits in your semester</p>
    </div>

    <div className="grid grid-cols-2 gap-3">
      <div>
        <Label className="text-[9px] text-white/60 mb-1 block">Semester Type</Label>
        <select
          value={data.semesterType}
          onChange={(e) => updateField("semesterType", e.target.value)}
          className="w-full h-8 rounded bg-white/10 border border-white/20 text-white text-[10px] px-2"
          data-testid="wizard-select-semester-type"
        >
          <option value="fall" className="bg-gray-800">Fall</option>
          <option value="winter" className="bg-gray-800">Winter</option>
          <option value="spring_summer" className="bg-gray-800">Spring/Summer</option>
        </select>
      </div>
      <div>
        <Label className="text-[9px] text-white/60 mb-1 block">Delivery Mode</Label>
        <select
          value={data.deliveryMode}
          onChange={(e) => updateField("deliveryMode", e.target.value)}
          className="w-full h-8 rounded bg-white/10 border border-white/20 text-white text-[10px] px-2"
          data-testid="wizard-select-delivery"
        >
          <option value="" className="bg-gray-800">Select...</option>
          <option value="virtual" className="bg-gray-800">Virtual (live class)</option>
          <option value="online" className="bg-gray-800">Online (async)</option>
        </select>
      </div>
    </div>

    {data.semesterType === "spring_summer" && (
      <div>
        <Label className="text-[9px] text-white/60 mb-1 block font-semibold">Spring/Summer Term <span className="te...
        <select
          value={data.springSummerTerm}
          onChange={(e) => updateField("springSummerTerm", e.target.value)}
          className={`w-full h-8 rounded bg-white/10 text-white text-[10px] px-2 ${!data.springSummerTerm ? 'border...
          data-testid="wizard-select-term"
        >
          <option value="" className="bg-gray-800">-- Select Term First --</option>
          <option value="first_half" className="bg-gray-800">Spring (First Half) – May-Jun</option>
          <option value="second_half" className="bg-gray-800">Summer (Second Half) – Jun-Aug</option>
          <option value="full" className="bg-gray-800">Full Semester – May-Aug</option>
        </select>
        {!data.springSummerTerm && <p className="text-[8px] text-yellow-400 mt-0.5">You must select a term before f...
      </div>
    )}

    <div className={data.semesterType === "spring_summer" && !data.springSummerTerm ? 'opacity-30 pointer-events-no...
    {data.deliveryMode === "virtual" && (
      <div>
        <div className="grid grid-cols-4 gap-2">
          <div>
            <Label className="text-[9px] text-white/60 mb-1 block">Day 1</Label>
            <select
              value={data.classDay}
              onChange={(e) => updateField("classDay", e.target.value)}
              className="w-full h-8 rounded bg-white/10 border border-white/20 text-white text-[10px] px-2"
              data-testid="wizard-select-day1"
            >
              {DAY_OPTIONS.map(d => (
                <option key={d.value} value={d.value} className="bg-gray-800">{d.label}</option>
              ))}
            </select>
          </div>
          <div>

```

```

<Label className="text-[9px] text-white/60 mb-1 block">Day 2</Label>
<select
  value={data.classDay2}
  onChange={(e) => updateField("classDay2", e.target.value)}
  className="w-full h-8 rounded bg-white/10 border border-white/20 text-white text-[10px] px-2"
  data-testid="wizard-select-day2"
>
  {DAY_OPTIONS.map(d => (
    <option key={d.value} value={d.value} className="bg-gray-800">{d.label}</option>
  ))}
</select>
</div>
<div>
<Label className="text-[9px] text-white/60 mb-1 block">Start Time</Label>
<Input
  type="time"
  value={data.classTime}
  onChange={(e) => updateField("classTime", e.target.value)}
  className="h-8 !text-[10px] !text-black"
  data-testid="wizard-input-start-time"
  />
</div>
<div>
<Label className="text-[9px] text-white/60 mb-1 block">End Time</Label>
<Input
  type="time"
  value={data.classEndTime}
  onChange={(e) => updateField("classEndTime", e.target.value)}
  className="h-8 !text-[10px] !text-black"
  data-testid="wizard-input-end-time"
  />
</div>
</div>
<div>
<Label className="text-[9px] text-white/60 mb-1 block">Zoom Link</Label>
<Input
  value={data.zoomLink}
  onChange={(e) => updateField("zoomLink", e.target.value)}
  placeholder="https://tmuni.zoom.us/j/..."
  className="h-8 text-[10px] bg-white/10 border-white/20 text-white placeholder:text-white/30"
  data-testid="wizard-input-zoom-link"
  />
</div>
</>
))

{data.deliveryMode === "online" && (
  <div className="bg-blue-500/10 border border-blue-500/20 rounded-lg p-2.5 text-[9px] text-blue-300">
    <p className="font-medium mb-1">Online (Asynchronous)</p>
    <p className="text-blue-300/70">Modules follow the weekly school calendar and change every Saturday. Module...</p>
  </div>
)}

<div className="grid grid-cols-2 gap-3">
  <div>
    <Label className="text-[9px] text-white/60 mb-1 block">Course Start Date</Label>
    <Input
      type="date"
      value={data.startDate}
      onChange={(e) => updateField("startDate", e.target.value)}
      className="h-8 !text-[10px] !text-black"
      data-testid="wizard-input-start-date"
    />
  </div>
  <div>
    <Label className="text-[9px] text-white/60 mb-1 block">Course End Date</Label>
    <Input
      type="date"
      value={data.endDate}
      onChange={(e) => updateField("endDate", e.target.value)}
      className="h-8 !text-[10px] !text-black"
      data-testid="wizard-input-end-date"
    />
  </div>
</div>
</div>
);

const renderStep5 = () => (
  <div className="space-y-3">
    <div className="text-center mb-2">
      <ClipboardCheck className="h-7 w-7 text-purple-400 mx-auto mb-1" />
      <h3 className="text-sm font-medium text-white">Assignments & Grades</h3>
      <p className="text-[9px] text-white/50 mt-0.5">Add all known assignments, exams, and other graded items</p>
    </div>

    {totalGradeWeight > 0 && (
      <div className={`flex items-center gap-2 px-3 py-1.5 rounded-lg text-[10px] ${
        totalGradeWeight === 100 ? "bg-green-500/20 text-green-300" :
        totalGradeWeight > 100 ? "bg-red-500/20 text-red-300" :
        "bg-amber-500/20 text-amber-300"
      }`} >
        <AlertCircle className="h-3 w-3" />
        Total grade weight: {totalGradeWeight}%{totalGradeWeight === 100 ? " (Complete)" : totalGradeWeight > 100 ? "..."}
      </div>
    )}

    <div className="space-y-2 max-h-[320px] overflow-y-auto pr-1" style={{ scrollbarWidth: "thin" }}>
      {data.tasks.map((task, index) => (
        <div key={index} className="bg-white/5 border border-white/10 rounded-lg p-2.5 space-y-2">
          <div className="flex items-center justify-between">
            <span className="text-[9px] text-white/40 font-medium">TASK {index + 1}</span>
            <button
              onClick={() => removeTask(index)}
              className="text-red-400/60 hover:text-red-400 p-0.5"
              data-testid={`wizard-remove-task-${index}`}
            >

```

```

        <Trash2 className="h-3 w-3" />
      </button>
    </div>

    <div className="grid grid-cols-2 gap-2">
      <div>
        <Label className="text-[9px] text-white/50 mb-0.5 block">Title </Label>
        <Input
          value={task.title}
          onChange={(e) => updateTask(index, "title", e.target.value)}
          placeholder="e.g. Midterm Exam"
          className="h-7 text-[10px] bg-white/10 border-white/15 text-white placeholder:text-white/25"
          data-testid={`wizard-task-title-${index}`}
        />
      </div>
      <div>
        <Label className="text-[9px] text-white/50 mb-0.5 block">Type</Label>
        <select
          value={task.type}
          onChange={(e) => updateTask(index, "type", e.target.value)}
          className="w-full h-7 rounded bg-white/10 border border-white/15 text-white text-[10px] px-1.5"
          data-testid={`wizard-task-type-${index}`}
        >
          {TASK_TYPE_OPTIONS.map(t => (
            <option key={t.value} value={t.value} className="bg-gray-800">{t.label}</option>
          ))}
        </select>
      </div>
    </div>

    <div className="grid grid-cols-2 gap-2">
      <div>
        <Label className="text-[9px] text-white/50 mb-0.5 block">Due Date</Label>
        <Input
          type="date"
          value={task.dueDate}
          onChange={(e) => updateTask(index, "dueDate", e.target.value)}
          className="h-7 !text-[10px] !text-black"
          data-testid={`wizard-task-due-${index}`}
        />
      </div>
      <div>
        <Label className="text-[9px] text-white/50 mb-0.5 block">Due Time</Label>
        <Input
          type="time"
          value={task.dueTime}
          onChange={(e) => updateTask(index, "dueTime", e.target.value)}
          className="h-7 !text-[10px] !text-black"
          data-testid={`wizard-task-time-${index}`}
        />
      </div>
    </div>

    <div>
      <Label className="text-[9px] text-white/50 mb-0.5 block">Description</Label>
      <Input
        value={task.description}
        onChange={(e) => updateTask(index, "description", e.target.value)}
        placeholder="Optional description"
        className="h-7 text-[10px] bg-white/10 border-white/15 text-white placeholder:text-white/25"
        data-testid={`wizard-task-desc-${index}`}
      />
    </div>

    <div className="grid grid-cols-2 gap-2 pt-1 border-t border-white/5">
      <div>
        <Label className="text-[9px] text-white/50 mb-0.5 block flex items-center gap-1">
          <span>Task Value</span>
          <span className="text-white/30">(total points)</span>
        </Label>
        <Input
          type="number"
          min="0"
          value={task.gradeTotal ?? ""}
          onChange={(e) => updateTask(index, "gradeTotal", e.target.value ? Number(e.target.value) : null)}
          placeholder="e.g. 40"
          className="h-7 text-[10px] bg-white/10 border-white/15 text-white placeholder:text-white/25"
          data-testid={`wizard-task-grade-total-${index}`}
        />
      </div>
      <div>
        <Label className="text-[9px] text-white/50 mb-0.5 block flex items-center gap-1">
          <span>% of Grade</span>
        </Label>
        <Input
          type="number"
          min="0"
          max="100"
          value={task.gradeWeight ?? ""}
          onChange={(e) => updateTask(index, "gradeWeight", e.target.value ? Number(e.target.value) : null)}
          placeholder="e.g. 25"
          className="h-7 text-[10px] bg-white/10 border-white/15 text-white placeholder:text-white/25"
          data-testid={`wizard-task-grade-weight-${index}`}
        />
      </div>
    </div>

    <div className="pt-1 border-t border-white/5">
      <Label className="text-[9px] text-white/50 mb-1 block flex items-center gap-1">
        <Bell className="h-2.5 w-2.5" />
        Reminders
      </Label>
      <div className="grid grid-cols-2 gap-1.5">
        {[[["reminder1", "Reminder 1"], ["reminder2", "Reminder 2"], ["reminder3", "Reminder 3"], ["reminder4...
          <div key={field}>
            <span className="text-[8px] text-white/40">{label}</span>
            <select
              value={task[field]}

```



```

        <GraduationCap className="text-white" style={{ width: '15px', height: '15px' }} />
        <h2 className="font-normal text-white" style={{ fontFamily: 'Avenir', 'Avenir Next', '-apple-system, BlinkM...
          NEW COURSE WIZARD
        </h2>
      </div>
    </div>

    {renderStepIndicator()}

    <div className="overflow-y-auto p-4" style={{ scrollbarWidth: "thin", height: "400px" }}>
      {step === 1 && renderStep1()}
      {step === 2 && renderStep2()}
      {step === 3 && renderStep3()}
      {step === 4 && renderStep4()}
      {step === 5 && renderStep5()}
      {step === 6 && renderStep6()}
    </div>

    <div className="flex items-center justify-between px-4 py-3 border-t border-white/20 bg-white/10 flex-shrink-0">
      <Button
        variant="outline"
        onClick={step === 1 ? onClose : handleBack}
        className="border !border-white/30 text-white/60 transition-all duration-200"
        style={{ fontSize: "11px" }}
        data-testid="wizard-back"
      >
      {step === 1 ? (
        "Cancel"
      ) : (
        <<<ChevronLeft className="h-3 w-3 mr-1" /> Back</>
      )}
    </Button>

    {step < 6 ? (
      <Button
        variant="outline"
        onClick={handleNext}
        disabled={!canProceed()}
        className="border !border-white/50 text-white transition-all duration-200 disabled:opacity-30"
        style={{
          boxShadow: canProceed() ? "0 0 6px rgba(255,255,255,0.4), 0 0 12px rgba(255,255,255,0.2)" : "none",
          fontSize: "11px",
        }}
        data-testid="wizard-next"
      >
        Next <ChevronRight className="h-3 w-3 ml-1" />
      </Button>
    ) : (
      <Button
        variant="outline"
        onClick={handleSave}
        className="border !border-white/50 text-white transition-all duration-200"
        style={{
          boxShadow: "0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255,255,255,...
          fontSize: "11px",
        }}
        data-testid="wizard-save"
      >
        Save Course
      </Button>
    )}
  </div>
</div>
</div>,
document.body
);
}

```

`client/src/components/ObjectUploader.tsx`

```

import { useState } from "react";
import type { ReactNode } from "react";
import Uppy from "@uppy/core";
import type { UppyFile, UploadResult } from "@uppy/core";
import DashboardModal from "@uppy/react/dashboard-modal";
import "@uppy/core/css/style.min.css";
import "@uppy/dashboard/css/style.min.css";
import AwsS3 from "@uppy/aws-s3";
import { Button } from "@components/ui/button";

interface ObjectUploaderProps {
  maxNumberOfFiles?: number;
  maxFileSize?: number;
  /**
   * Function to get upload parameters for each file.
   * IMPORTANT: This receives the file object - use file.name, file.size, file.type
   * to request per-file presigned URLs from your backend.
   */
  onGetUploadParameters: (
    file: UppyFile<Record<string, unknown>, Record<string, unknown>>
  ) => Promise<{
    method: "PUT";
    url: string;
    headers?: Record<string, string>;
  }>;
  onComplete?: (
    result: UploadResult<Record<string, unknown>, Record<string, unknown>>
  ) => void;
  buttonClassName?: string;
  children: ReactNode;
}

/**
 * A file upload component that renders as a button and provides a modal interface for

```

```

* file management.
*
* Features:
* - Renders as a customizable button that opens a file upload modal
* - Provides a modal interface for:
*   - File selection
*   - File preview
*   - Upload progress tracking
*   - Upload status display
*
* The component uses Uppy v5 under the hood to handle all file upload functionality.
* All file management features are automatically handled by the Uppy dashboard modal.
*
* @param props - Component props
* @param props.maxNumberOfFiles - Maximum number of files allowed to be uploaded
* (default: 1)
* @param props.maxFileSize - Maximum file size in bytes (default: 10MB)
* @param props.onGetUploadParameters - Function to get upload parameters for each file.
*   Receives the UppyFile object with file.name, file.size, file.type properties.
*   Use these to request per-file presigned URLs from your backend. Returns method,
*   url, and optional headers for the upload request.
* @param props.onComplete - Callback function called when upload is complete. Typically
*   used to make post-upload API calls to update server state and set object ACL
*   policies.
* @param props.buttonClassName - Optional CSS class name for the button
* @param props.children - Content to be rendered inside the button
*/
export function ObjectUploader({
  maxNumberOfFiles = 1,
  maxFileSize = 10485760, // 10MB default
  onGetUploadParameters,
  onComplete,
  buttonClassName,
  children,
}: ObjectUploaderProps) {
  const [showModal, setShowModal] = useState(false);
  const [uppy] = useState(() =>
    new Uppy({
      restrictions: {
        maxNumberOfFiles,
        maxFileSize,
      },
      autoProceed: false,
    })
    .use(AwsS3, {
      shouldUseMultipart: false,
      getUploadParameters: onGetUploadParameters,
    })
    .on("complete", (result) => {
      onComplete?.(result);
    })
  );

  return (
    <div>
      <Button onClick={() => setShowModal(true)} className={buttonClassName}>
        {children}
      </Button>

      <DashboardModal
        uppy={uppy}
        open={showModal}
        onRequestClose={() => setShowModal(false)}
        proudlyDisplayPoweredByUppy={false}
      />
    </div>
  );
}

```

`client/src/components/OtherRowEditDialog.tsx`

```

import { useState, useRef, useCallback, useEffect } from 'react';
import { X, Trash2, RotateCcw, Layers } from 'lucide-react';

```

```

interface OtherRowColors {
  labelStart: string;
  labelEnd: string;
  labelStops: string;
  cellBg: string;
  borderColor: string;
  taskBgColor: string;
  courseRowColor: string;
}

```

```

const DEFAULTS: OtherRowColors = {
  labelStart: '#374151',
  labelEnd: '#9ca3af',
  labelStops: '',
  cellBg: '#3a3f4a',
  borderColor: '#5c6370',
  taskBgColor: '#363b44',
  courseRowColor: '#3a3f4a',
};

```

```

interface Props {
  open: boolean;
  onClose: () => void;
  colors: OtherRowColors;
  onSave: (colors: OtherRowColors) => void;
  onPreview?: (colors: OtherRowColors) => void;
  headerBarColor?: string;
}

```

```

export default function OtherRowEditDialog({ open, onClose, colors, onSave, onPreview, headerBarColor = '#051729' }: ...) {
  const [edit, setEdit] = useState<OtherRowColors>(colors);
  const [activeGradientStop, setActiveGradientStop] = useState<'start' | 'end' | number | null>(null);
  const [activeSwatchPicker, setActiveSwatchPicker] = useState<'border' | 'rowBg' | 'taskBg' | null>(null);
  const [dialogPos, setDialogPos] = useState({ x: 0, y: 0 });
  const dialogRef = useRef<HTMLDivElement>(null);
  const gradBarRef = useRef<HTMLDivElement>(null);
  const dragStart = useRef<{ x: number; y: number; mx: number; my: number } | null>(null);
  const originalColorsRef = useRef<OtherRowColors>(colors);

  useEffect(() => {
    if (open) {
      setEdit(colors);
      originalColorsRef.current = colors;
    }
  }, [open, colors]);

  useEffect(() => {
    if (open && onPreview) onPreview(edit);
  }, [edit, open]);

  const handleDragStart = useCallback((e: React.MouseEvent | React.TouchEvent) => {
    const pt = 'touches' in e ? e.touches[0] : e;
    dragStart.current = { x: dialogPos.x, y: dialogPos.y, mx: pt.clientX, my: pt.clientY };
    const onMove = (ev: MouseEvent | TouchEvent) => {
      if (!dragStart.current) return;
      const p = 'touches' in ev ? (ev as TouchEvent).touches[0] : (ev as MouseEvent);
      setDialogPos({
        x: dragStart.current.x + p.clientX - dragStart.current.mx,
        y: dragStart.current.y + p.clientY - dragStart.current.my,
      });
    };
    const onUp = () => { dragStart.current = null; window.removeEventListener('mousemove', onMove); window.removeEven...
    window.addEventListener('mousemove', onMove);
    window.addEventListener('mouseup', onUp);
    window.addEventListener('touchmove', onMove);
    window.addEventListener('touchend', onUp);
  }, [dialogPos]);

  if (!open) return null;

  const midStops: Array<{ position: number; color: string }> = edit.labelStops ? () => { try { return JSON.parse(edi...
  const allStops = [
    { position: 0, color: edit.labelStart, key: 'start' as const },
    ...midStops.map((s, i) => ({ position: s.position, color: s.color, key: i as number })),
    { position: 100, color: edit.labelEnd, key: 'end' as const },
  ].sort((a, b) => a.position - b.position);
  const gradientCss = `linear-gradient(to right, ${allStops.map(s => ` ${s.color} ${s.position}%`).join(', ')} `);

  const getActiveColor = (): string => {
    if (activeGradientStop === 'start') return edit.labelStart;
    if (activeGradientStop === 'end') return edit.labelEnd;
    if (typeof activeGradientStop === 'number' && midStops[activeGradientStop]) return midStops[activeGradientStop].c...
    return '#000000';
  };

  const setActiveColor = (hex: string) => {
    if (activeGradientStop === 'start') setEdit({ ...edit, labelStart: hex });
    else if (activeGradientStop === 'end') setEdit({ ...edit, labelEnd: hex });
    else if (typeof activeGradientStop === 'number') {
      const updated = [...midStops];
      updated[activeGradientStop] = { ...updated[activeGradientStop], color: hex };
      setEdit({ ...edit, labelStops: JSON.stringify(updated) });
    }
  };

  const hexToHue = (c: string) => {
    const r = parseInt(c.slice(1, 3), 16) / 255, g = parseInt(c.slice(3, 5), 16) / 255, b = parseInt(c.slice(5, 7), 1...
    const max = Math.max(r, g, b), min = Math.min(r, g, b);
    if (max === min) return 0;
    let h = 0;
    if (max === r) h = ((g - b) / (max - min)) % 6;
    else if (max === g) h = (b - r) / (max - min) + 2;
    else h = (r - g) / (max - min) + 4;
    h = Math.round(h * 60);
    return h < 0 ? h + 360 : h;
  };

  const hueToHex = (hue: number) => `#${[0, 8, 4].map(n => {
    const k = (n + hue / 30) % 12;
    const c2 = 0.5 - 0.5 * Math.max(Math.min(k - 3, 9 - k, 1), -1);
    return Math.round(255 * Math.max(0, Math.min(1, c2))).toString(16).padStart(2, '0');
  }).join('')}`;

  const hexToSvPos = (hex: string) => {
    const r = parseInt(hex.slice(1, 3), 16) / 255, g = parseInt(hex.slice(3, 5), 16) / 255, b = parseInt(hex.slice(5, ...
    const max = Math.max(r, g, b), min = Math.min(r, g, b);
    const v = max;
    const s = max === 0 ? 0 : (max - min) / max;
    return { x: s, y: 1 - v };
  };

  const svToHex = (hue: number, sx: number, sy: number) => {
    const s = sx, v = 1 - sy;
    const c = v * s, x2 = c * (1 - Math.abs((hue / 60) % 2 - 1)), m = v - c;
    let r1 = 0, g1 = 0, b1 = 0;
    if (hue < 60) { r1 = c; g1 = x2; } else if (hue < 120) { r1 = x2; g1 = c; } else if (hue < 180) { g1 = c; b1 = x2...
    const f = (ch: number) => Math.round(255 * Math.max(0, Math.min(1, ch + m))).toString(16).padStart(2, '0');
    return `#${f(r1)}${f(g1)}${f(b1)}`;
  };

  const headerGradient = `linear-gradient(180deg, rgba(255,255,255,0.28) 0%, ${headerBarColor}cc 40%, ${headerBarColo...

  return (
    <div className="fixed inset-0 z-[10002] flex items-center justify-center" onClick={() => { if (onPreview) onPrevi...
    <div className="absolute inset-0 bg-black/40" />
    <div
      ref={dialogRef}

```

```

className="relative rounded-lg"
style={{
  width: '420px',
  maxHeight: '520px',
  overflowY: 'auto',
  scrollbarWidth: 'thin',
  background: 'linear-gradient(180deg, #3a8bbf 0%, color-mix(in srgb, #164a72 70%, black) 100%)',
  border: '1.5px solid rgba(255,255,255,0.35)',
  boxShadow: '0 8px 32px rgba(0,0,0,0.2)', inset 0 1px 0 rgba(255,255,255,0.3), inset 0 -1px 0 rgba(0,0,0,0.05)',
  transform: `translate(${dialogPos.x}px, ${dialogPos.y}px)`,
}}
onClick={(e) => e.stopPropagation()}
data-testid="other-row-edit-dialog"
>
<div
  className="flex items-center justify-between px-4 py-3 border-b border-white/40 flex-shrink-0 rounded-t-lg"
  onMouseDown={handleDragStart}
  onTouchStart={handleDragStart}
  style={{
    cursor: 'grab',
    backdropFilter: 'blur(30px)',
    WebkitBackdropFilter: 'blur(30px)',
    background: headerGradient,
    boxShadow: 'inset 0 1px 0 rgba(255,255,255,0.45), inset 0 2px 4px rgba(255,255,255,0.15), inset 0 -1px 0 ...
  }}
>
<div className="flex items-center gap-2 min-w-0">
  <Layers className="text-white flex-shrink-0" style={{ width: '15px', height: '15px' }} />
  <h2
    className="font-normal text-white truncate"
    style={{ fontFamily: 'Avenir, 'Avenir Next', -apple-system, BlinkMacSystemFont, sans-serif', textShadow...
    data-testid="text-other-row-title"
  >
    OTHER Row Appearance
  </h2>
</div>
<button onClick={() => { if (onPreview) onPreview(originalColorsRef.current); onClose(); }} className="text...
</div>

<div className="p-4 space-y-4">
  <div>
    <div className="flex items-start gap-3">
      <div style={{ width: '200px' }}>
        <div className="flex items-center justify-between mb-1">
          <label className="text-white text-[9px]">Label Gradient</label>
          <button className="text-white hover:text-white/80 text-[8px] flex items-center gap-0.5" onClick={() => {
            setEdit({ ...edit, labelStart: edit.labelEnd, labelEnd: edit.labelStart, labelStops: midStops.length...
          }} data-testid="button-reverse-gradient"><svg width="10" height="10" viewBox="0 0 16 16" fill="none" ...
          </div>
        <div ref={gradBarRef} className="rounded" style={{ border: '1px solid rgba(255,255,255,0.15)', padding:...
          onDoubleClick={(e) => {
            const bar = gradBarRef.current;
            if (!bar) return;
            const rect = bar.getBoundingClientRect();
            const pct = Math.round(Math.max(5, Math.min(95, ((e.clientX - rect.left - 3) / (rect.width - 6)) * ...
            const leftStop = allStops.filter(s => s.position <= pct).pop()!;
            const rightStop = allStops.find(s => s.position >= pct)!;
            const t = rightStop.position - leftStop.position ? 0 : (pct - leftStop.position) / (rightStop.pos...
            const lR = parseInt(leftStop.color.slice(1,3),16), lG = parseInt(leftStop.color.slice(3,5),16), lB ...
            const rR = parseInt(rightStop.color.slice(1,3),16), rG = parseInt(rightStop.color.slice(3,5),16), r...
            const mR = Math.round(lR + (rR-lR)*t), mG = Math.round(lG + (rG-lG)*t), mB = Math.round(lB + (rB-lB...
            const newColor = `#${mR.toString(16).padStart(2,'0')}${mG.toString(16).padStart(2,'0')}${mB.toStrin...
            const newMid = [...midStops, { position: pct, color: newColor }].sort((a, b) => a.position - b.posi...
            const newIdx = newMid.findIndex(s => s.position === pct && s.color === newColor);
            setEdit({ ...edit, labelStops: JSON.stringify(newMid) });
            setActiveGradientStop(newIdx);
          }}
          data-testid="gradient-bar">
          <div style={{ height: '18px', borderRadius: '3px', background: gradientCss }} data-testid="gradient-p...
        </div>
      <div className="relative" style={{ height: '16px', marginTop: '1px' }}>
        <div style={{ position: 'absolute', left: '0px', top: 0, cursor: 'pointer', zIndex: 10 }} onClick={()...
          <svg width="12" height="12" viewBox="0 0 12 12"><polygon points="6,0 12,10 0,10" fill={edit.labelSt...
          </div>
        {midStops.map((stop, idx) => {
          <div key={idx} style={{ position: 'absolute', left: `calc(${stop.position}% - 6px)`, top: 0, cursor...
            onClick={() => { setActiveGradientStop(activeGradientStop === idx ? null : idx); setActiveSwatchP...
            onPointerDown={(e) => {
              e.preventDefault();
              e.stopPropagation();
              const el = e.currentTarget as HTMLElement;
              el.setPointerCapture(e.pointerId);
              const bar = gradBarRef.current;
              if (!bar) return;
              const barRect = bar.getBoundingClientRect();
              const barW = barRect.width;
              const onMove = (ev: PointerEvent) => {
                const pct = Math.round(Math.max(1, Math.min(99, ((ev.clientX - barRect.left) / barW) * 100));
                const updated = [...midStops];
                updated[idx] = { ...updated[idx], position: pct };
                setEdit(prev => ({ ...prev, labelStops: JSON.stringify(updated.sort((a, b) => a.position - b....
                });
                const onUp = () => { el.releasePointerCapture(e.pointerId); window.removeEventListener('pointer...
                window.addEventListener('pointermove', onMove);
                window.addEventListener('pointerup', onUp);
              }}
              data-testid={`gradient-stop-mid-${idx}`}>
              <svg width="12" height="12" viewBox="0 0 12 12"><polygon points="6,0 11,6 6,12 1,6" fill={stop.co...
            </div>
          })}
        <div style={{ position: 'absolute', right: '0px', top: 0, cursor: 'pointer', zIndex: 10 }} onClick={()...
          <svg width="12" height="12" viewBox="0 0 12 12"><polygon points="6,0 12,10 0,10" fill={edit.labelEn...
          </div>
        {midStops.length > 0 && <span className="text-white text-[9px] absolute" style={{ bottom: '-23px', le...
        </div>
        {midStops.length === 0 && <div className="text-white text-[9px] text-left" style={{ marginTop: '7px', m...
        {midStops.length > 0 && <div style={{ height: '24px' }} />
      <activeGradientStop != null && (

```



```

<div className="mt-1 rounded" style={{ border: '1px solid rgba(255,255,255,0.15)', background: 'rgba(...
  <div className="flex items-center gap-2 mb-1.5">
    <div className="w-5 h-5 rounded border border-white/30 shrink-0" style={{ backgroundColor: getAct...
      <span className="text-white/60 text-[8px] uppercase tracking-wider">{activeGradientStop === 'star...
        {typeof activeGradientStop === 'number' && (
          <button className="text-red-400/70 hover:text-red-400 text-[8px] ml-auto mr-1" onClick={() => {
            const updated = midStops.filter((_, i) => i !== activeGradientStop);
            setEdit({ ...edit, labelStops: updated.length ? JSON.stringify(updated) : '' });
            setActiveGradientStop(null);
          }} data-testid="button-delete-stop"><Trash2 className="w-3 h-3" /></button>
        )}
      <button className={` ${typeof activeGradientStop === 'number' ? '' : 'ml-auto '} text-white/40 hove...
    </div>
    {typeof activeGradientStop === 'number' && (
      <div className="flex items-center gap-2 mb-1.5">
        <span className="text-white/50 text-[8px]">Position</span>
        <input type="range" min={1} max={99} value={midStops[activeGradientStop]?.position || 50} onCha...
          const updated = [...midStops];
          updated[activeGradientStop as number] = { ...updated[activeGradientStop as number], position:...
          setEdit({ ...edit, labelStops: JSON.stringify(updated.sort((a, b) => a.position - b.position)...
        )} className="flex-1" style={{ height: '6px', accentColor: getActiveColor() }} data-testid="sli...
        <span className="text-white/50 text-[8px] w-6 text-right">{midStops[activeGradientStop]?.positi...
      </div>
    )}
  </div>
  <div className="relative rounded cursor-crosshair" style={{ height: '92px', touchAction: 'none', ov...
    onPointerDown={(e) => {
      e.preventDefault();
      const el = e.currentTarget;
      el.setPointerCapture(e.pointerId);
      const rect = el.getBoundingClientRect();
      const hue = hexToHue(getActiveColor());
      const update = (ev: PointerEvent) => {
        const x = Math.max(0, Math.min(1, (ev.clientX - rect.left) / rect.width));
        const y = Math.max(0, Math.min(1, (ev.clientY - rect.top) / rect.height));
        setActiveColor(svToHex(hue, x, y));
      };
      update(e.nativeEvent);
      const onMove = (ev: PointerEvent) => update(ev);
      const onUp = () => { el.releasePointerCapture(e.pointerId); window.removeEventListener('pointer...
      window.addEventListener('pointermove', onMove);
      window.addEventListener('pointerup', onUp);
    }
  >
  <div style={{ position: 'absolute', inset: 0, borderRadius: '3px', background: 'linear-gradient(t...
  <div style={{ position: 'absolute', inset: 0, borderRadius: '3px', background: 'linear-gradient(t...
    (() => {
      const pos = hexToSvPos(getActiveColor());
      return (
        <div style={{ position: 'absolute', left: `${pos.x * 100}%`, top: `${pos.y * 100}%`, transfor...
      );
    })()
  </div>
  <div className="relative mt-1.5 rounded cursor-pointer" style={{ height: '14px', touchAction: 'none...
    onClick={(e) => {
      const rect = e.currentTarget.getBoundingClientRect();
      const x = Math.max(0, Math.min(1, (e.clientX - rect.left) / rect.width));
      setActiveColor(hueToHex(Math.round(x * 360)));
    }
    onPointerDown={(e) => {
      e.preventDefault();
      const el = e.currentTarget;
      el.setPointerCapture(e.pointerId);
      const rect = el.getBoundingClientRect();
      const update = (ev: PointerEvent) => {
        const x = Math.max(0, Math.min(1, (ev.clientX - rect.left) / rect.width));
        setActiveColor(hueToHex(Math.round(x * 360)));
      };
      update(e.nativeEvent);
      const onMove = (ev: PointerEvent) => update(ev);
      const onUp = () => { el.releasePointerCapture(e.pointerId); window.removeEventListener('pointer...
      window.addEventListener('pointermove', onMove);
      window.addEventListener('pointerup', onUp);
    }
  >
  <div style={{ position: 'absolute', inset: 0, background: 'linear-gradient(to right, #ff0000, #ff...
  <div style={{ position: 'absolute', top: '-1px', left: `${(hexToHue(getActiveColor()) / 360) * 10...
</div>
<div className="flex items-center gap-2 mt-1.5">
  <input type="color" value={getActiveColor()} onChange={(e) => setActiveColor(e.target.value)} cla...
  <input type="text" value={getActiveColor().toUpperCase()} onChange={(e) => { let v = e.target.val...
</div>
</div>
)}
</div>
<div style={{ marginLeft: '12px', display: 'flex', gap: '6px', alignItems: 'flex-start', marginTop: '2px'...
  {[
    { key: 'border' as const, label: 'Border', field: 'borderColor' as const },
    { key: 'rowBg' as const, label: 'Row BG', field: 'courseRowColor' as const },
    { key: 'taskBg' as const, label: 'Task BG', field: 'taskBgColor' as const },
  ]).map(({ key, label, field }) => {
    const val = edit[field] || edit.labelStart;
    const hexVal = (() => { const m = val.match(/rgba?(\d+),\s*(\d+),\s*(\d+)/); if (m) return `#${pars...
    return (
      <div key={key} style={{ display: 'flex', flexDirection: 'column', alignItems: 'center' }}>
        <label className="text-white text-[9px] mb-1">{label}</label>
        <div
          className="rounded-sm border cursor-pointer"
          style={{ width: '20px', height: '20px', backgroundColor: hexVal, borderColor: activeSwatchPicke...
          onClick={() => { setActiveSwatchPicker(activeSwatchPicker === key ? null : key); setActiveGradi...
          data-testid={`input-${field}-swatch`}
        />
        <input type="text" value={edit[field]} onChange={e => setEdit({ ...edit, [field]: e.target.value ...
      </div>
    );
  })
</div>
</div>
</div>
{activeSwatchPicker && (() => {
  const fieldMap = { border: 'borderColor' as const, rowBg: 'courseRowColor' as const, taskBg: 'taskBgColor...

```

```

const labelMap = { border: 'Border', rowBg: 'Row BG', taskBg: 'Task BG' };
const field = fieldMap[activeSwatchPicker];
const val = edit[field] || edit.labelStart;
const hexVal = (() => { const m = val.match(/rgba?(\d+),\s*(\d+),\s*(\d+)/); if (m) return `#${parseInt...
const setSwatchColor = (hex: string) => setEdit({ ...edit, [field]: hex });
return (
  <div className="rounded" style={{ border: '1px solid rgba(255,255,255,0.15)', background: 'rgba(0,0,0,0...
    <div className="flex items-center gap-2 mb-1.5">
      <div className="w-5 h-5 rounded border border-white/30 shrink-0" style={{ backgroundColor: hexVal }}...
      <span className="text-white/60 text-[8px] uppercase tracking-wider">{labelMap[activeSwatchPicker]} ...
      <button className="ml-auto text-white/40 hover:text-white" onClick={() => setActiveSwatchPicker(nul...
    </div>
    <div className="relative rounded cursor-crosshair" style={{ height: '92px', touchAction: 'none' }} da...
      onPointerDown={(e) => {
        e.preventDefault();
        const el = e.currentTarget;
        el.setPointerCapture(e.pointerId);
        const rect = el.getBoundingClientRect();
        const hue = hexToHue(hexVal);
        const update = (ev: PointerEvent) => {
          const x = Math.max(0, Math.min(1, (ev.clientX - rect.left) / rect.width));
          const y = Math.max(0, Math.min(1, (ev.clientY - rect.top) / rect.height));
          setSwatchColor(svToHex(hue, x, y));
        };
        update(e.nativeEvent);
        const onMove = (ev: PointerEvent) => update(ev);
        const onUp = () => { el.releasePointerCapture(e.pointerId); window.removeEventListener('pointermove...
        window.addEventListener('pointermove', onMove);
        window.addEventListener('pointerup', onUp);
      }>
      <div style={{ position: 'absolute', inset: 0, borderRadius: '3px', background: 'linear-gradient(to ...
      <div style={{ position: 'absolute', inset: 0, borderRadius: '3px', background: 'linear-gradient(to ...
      {(() => {
        const pos = hexToSvPos(hexVal);
        return (
          <div style={{ position: 'absolute', left: `${pos.x * 100}%`, top: `${pos.y * 100}%`, transform:...
        );
      })()}
    </div>
    <div className="relative mt-1.5 rounded cursor-pointer" style={{ height: '14px', touchAction: 'none' ...
      onClick={(e) => {
        const rect = e.currentTarget.getBoundingClientRect();
        const x = Math.max(0, Math.min(1, (e.clientX - rect.left) / rect.width));
        setSwatchColor(hueToHex(Math.round(x * 360)));
      }}
      onPointerDown={(e) => {
        e.preventDefault();
        const el = e.currentTarget;
        el.setPointerCapture(e.pointerId);
        const rect = el.getBoundingClientRect();
        const onMove = (ev: PointerEvent) => {
          const x = Math.max(0, Math.min(1, (ev.clientX - rect.left) / rect.width));
          setSwatchColor(hueToHex(Math.round(x * 360)));
        };
        onMove(e.nativeEvent);
        const onUp = () => { el.releasePointerCapture(e.pointerId); window.removeEventListener('pointermove...
        window.addEventListener('pointermove', onMove);
        window.addEventListener('pointerup', onUp);
      }}
      <div style={{ position: 'absolute', inset: 0, background: 'linear-gradient(to right, #ff0000, #ffff...
      <div style={{ position: 'absolute', top: '-1px', left: `${(hexToHue(hexVal) / 360) * 100}%`, transf...
    </div>
    <div className="flex items-center gap-2 mt-1.5">
      <input type="text" value={hexVal.toUpperCase()} onChange={(e) => { let v = e.target.value; if (!v.s...
    </div>
  </div>
);
})();
<div>
  <div className="text-white/60 text-[9px] uppercase tracking-wider mb-2">Cell Background</div>
  <input type="text" value={edit.cellBg} onChange={e => setEdit({ ...edit, cellBg: e.target.value })} class...
</div>

<div className="flex items-center justify-between gap-2 pt-2">
  <button
    className="text-white/50 hover:text-white text-[10px] flex items-center gap-1"
    onClick={() => { setEdit(DEFAULTS); setActiveGradientStop(null); }}
    data-testid="button-reset-defaults"
  >
    <RotateCcw className="w-3 h-3" /> Reset to defaults
  </button>
  <div className="flex gap-2">
    <button className="px-3 py-1.5 text-[10px] text-white/70 hover:text-white rounded border border-white/2...
    <button className="px-3 py-1.5 text-[10px] text-white bg-white/20 hover:bg-white/30 rounded border bord...
  </div>
</div>

</div>
</div>
</div>
);
}

```

`client/src/components/SemesterChecklistDialog.tsx`

```

import { useState, useEffect, useRef, memo } from "react";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogDescription, DialogFooter } from "@components/ui/di...
import { useToast } from "@hooks/use-toast";
import { Check } from "lucide-react";

```

```

type ChecklistItem = {
  id: number;

```

```

semesterSettingsId: number;
courseCode: string;
itemType: string;
isChecked: boolean | null;
checkedAt: string | null;
};

interface SemesterChecklistDialogProps {
  open: boolean;
  onOpenChange: (open: boolean) => void;
  items: ChecklistItem[];
  onItemsChange: (items: ChecklistItem[]) => void;
  semesterSettingsId?: number;
}

export const SemesterChecklistDialog = memo(function SemesterChecklistDialog({
  open,
  onOpenChange,
  items,
  onItemsChange,
  semesterSettingsId,
}: SemesterChecklistDialogProps) {
  const { toast } = useToast();
  const [snoozeValue, setSnoozeValue] = useState(30);
  const [snoozeUnit, setSnoozeUnit] = useState<'minutes' | 'hours'>('minutes');
  const [localChecked, setLocalChecked] = useState<Record<number, boolean>>({});
  const itemsRef = useRef(items);
  itemsRef.current = items;
  const onItemsChangeRef = useRef(onItemsChange);
  onItemsChangeRef.current = onItemsChange;
  const onOpenChangeRef = useRef(onOpenChange);
  onOpenChangeRef.current = onOpenChange;
  const pendingRef = useRef<Set<number>>(new Set());

  useEffect(() => {
    const map: Record<number, boolean> = {};
    items.forEach(i => { map[i.id] = !!i.isChecked; });
    setLocalChecked(map);
  }, [items]);

  const handleCheck = (item: ChecklistItem) => {
    if (pendingRef.current.has(item.id)) return;
    const newChecked = !localChecked[item.id];
    setLocalChecked(prev => ({ ...prev, [item.id]: newChecked }));
    pendingRef.current.add(item.id);

    fetch('/api/semester-checklist', {
      method: 'PATCH',
      headers: { 'Content-Type': 'application/json' },
      credentials: 'include',
      body: JSON.stringify({ semesterSettingsId: item.semesterSettingsId, courseCode: item.courseCode, itemType: item...
    })
    .then(r => r.json())
    .then(data => {
      pendingRef.current.delete(item.id);
      const updated = itemsRef.current.map(i => i.id === item.id ? { ...i, isChecked: newChecked } : i);
      onItemsChangeRef.current(updated);
      if (data.allChecked) {
        toast({ title: "All done!", description: "Semester setup checklist complete." });
        if (semesterSettingsId) {
          localStorage.setItem(`semChecklist_allDone_${semesterSettingsId}`, 'true');
        }
        setTimeout(() => onOpenChangeRef.current(false), 600);
      }
    })
    .catch(err => {
      pendingRef.current.delete(item.id);
      console.error('Error updating checklist:', err);
    });
  });

  const grouped: Record<string, ChecklistItem[]> = {};
  items.forEach(item => {
    if (!grouped[item.courseCode]) grouped[item.courseCode] = [];
    grouped[item.courseCode].push(item);
  });

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="max-w-md text-white" data-testid="semester-checklist-dialog">
        <DialogHeader>
          <DialogTitle className="text-white text-lg font-semibold">Hey Bryn, have you...</DialogTitle>
          <DialogDescription className="text-white/60 text-sm">Complete these items to get your semester started righ...
        </DialogHeader>
        <div className="flex flex-col gap-4 mt-2">
          {Object.entries(grouped).map(([courseCode, courseItems]) => (
            <div key={courseCode} className="flex flex-col gap-1">
              <span className="text-xs font-bold text-white/80 uppercase tracking-wider">{courseCode}</span>
              {courseItems.map(item => {
                const label = item.itemType === 'tasks' ? 'Input all assignments, essays, projects and exams' : item...
                const checked = !!localChecked[item.id];
                return (
                  <div
                    key={item.id}
                    className="flex items-center gap-3 rounded px-2 py-2.5 active:bg-white/10 select-none"
                    style={{ WebkitTapHighlightColor: 'transparent', touchAction: 'manipulation', cursor: 'pointer', ...
                    onClick={() => handleCheck(item)}
                    data-testid={`checklist-${courseCode}-${item.itemType}`}
                  >
                    <div
                      className="shrink-0 flex items-center justify-center rounded border-2 transition-colors"
                      style={{
                        width: '28px',
                        height: '28px',
                        backgroundColor: checked ? '#22c55e' : 'transparent',
                        borderColor: checked ? '#22c55e' : 'rgba(255,255,255,0.5)',
                      }}
                      data-testid={`checkbox-${courseCode}-${item.itemType}`}
                    </div>

```

```

        >
        {checked && <Check className="h-5 w-5 text-white" strokeWidth={3} />}
      </div>
      <span className={ `text-sm ${checked ? 'text-white/50 line-through' : 'text-white/90'}` }>{label}</...
    </div>
  );
  });
  </div>
  )))
</div>
<DialogFooter className="mt-4 flex flex-col gap-3 sm:flex-col">
  <div className="flex items-center gap-2 w-full" data-testid="checklist-snooze-section">
    <button
      className="px-4 py-2 text-sm font-medium bg-amber-600 hover:bg-amber-500 text-white rounded transition-...
      onClick={() => {
        const ms = snoozeUnit === 'hours' ? snoozeValue * 60 * 60 * 1000 : snoozeValue * 60 * 1000;
        const until = Date.now() + ms;
        if (semesterSettingsId) {
          localStorage.setItem(`semChecklist_snoozeUntil_${semesterSettingsId}`, String(until));
        }
        onOpenChange(false);
        toast({ title: "Snoozed", description: `Checklist snoozed for ${snoozeValue} ${snoozeUnit}.` });
      }}
      data-testid="button-snooze-checklist"
    >
      Snooze
    </button>
    <select
      value={snoozeValue}
      onChange={e => setSnoozeValue(parseInt(e.target.value, 10))}
      className="border border-white/30 rounded px-2 py-2 text-sm w-16 text-center"
      style={{ backgroundColor: '#2a2a2a', color: 'ffffff', WebkitTextFillColor: 'ffffff', opacity: 1, Webk...
      data-testid="select-snooze-value"
      size={1}
    >
      {snoozeUnit === 'minutes'
        ? Array.from({ length: 60 }, (_, i) => i + 1).map(v => <option key={v} value={v} style={{ backgroundC...
          : [1, 2, 3, 4, 6, 8, 12, 24].map(v => <option key={v} value={v} style={{ backgroundColor: '#2a2a2a', ...
        )
      </select>
      <select
        value={snoozeUnit}
        onChange={e => {
          const unit = e.target.value as 'minutes' | 'hours';
          setSnoozeUnit(unit);
          setSnoozeValue(unit === 'minutes' ? 30 : 1);
        }}
        className="border border-white/30 rounded px-2 py-2 text-sm"
        style={{ backgroundColor: '#2a2a2a', color: 'ffffff', WebkitTextFillColor: 'ffffff', opacity: 1, Webk...
        data-testid="select-snooze-unit"
      >
        <option value="minutes" style={{ backgroundColor: '#2a2a2a', color: 'ffffff', WebkitTextFillColor: '#f...
        <option value="hours" style={{ backgroundColor: '#2a2a2a', color: 'ffffff', WebkitTextFillColor: '#fff...
      </select>
    </div>
    <button
      className="px-4 py-2 text-sm font-medium text-white/50 hover:text-white/80 transition-colors self-start"
      onClick={() => onOpenChange(false)}
      data-testid="button-dismiss-checklist"
    >
      Dismiss for now
    </button>
  </DialogFooter>
</DialogContent>
</Dialog>
);
});

```

`client/src/components/Sidebar.tsx`

```

import { cn } from "@lib/utls";
import { LayoutDashboard, Armchair, Utensils, BedDouble, Briefcase, DoorOpen } from "lucide-react";

interface SidebarProps {
  selectedRoom: string | null;
  onSelectRoom: (room: string | null) => void;
  className?: string;
}

const roomIcons: Record<string, any> = {
  "Living Room": Armchair,
  "Kitchen": Utensils,
  "Bedroom": BedDouble,
  "Office": Briefcase,
  "Entrance": DoorOpen,
};

export function Sidebar({ selectedRoom, onSelectRoom, className }: SidebarProps) {
  const rooms = ["Living Room", "Kitchen", "Bedroom", "Office", "Entrance"];

  return (
    <div className={cn("bg-card/50 backdrop-blur-xl border-r border-white/5 h-screen p-6 flex flex-col gap-8", classN...
      <div className="flex items-center gap-3 px-2">
        <div className="w-8 h-8 rounded-lg bg-gradient-to-br from-primary to-accent flex items-center justify-center">
          <LayoutDashboard className="text-white w-5 h-5" />
        </div>
        <span className="font-display font-bold text-xl tracking-tight">HomeOS</span>
      </div>

      <nav className="flex flex-col gap-2">
        <p className="text-xs font-semibold text-muted-foreground uppercase tracking-wider px-2 mb-2">Rooms</p>

        <button
          onClick={() => onSelectRoom(null)}
        >

```

```

      className={cn(
        "flex items-center gap-3 px-3 py-2.5 rounded-xl text-sm font-medium transition-all duration-200",
        selectedRoom === null
          ? "bg-primary text-white shadow-lg shadow-primary/20"
          : "text-muted-foreground hover:bg-white/5 hover:text-white"
      )}
    >
    <LayoutDashboard className="w-4 h-4" />
    All Devices
  </button>

  {rooms.map((room) => {
    const Icon = roomIcons[room] || LayoutDashboard;
    return (
      <button
        key={room}
        onClick={() => onSelectRoom(room)}
        className={cn(
          "flex items-center gap-3 px-3 py-2.5 rounded-xl text-sm font-medium transition-all duration-200",
          selectedRoom === room
            ? "bg-primary text-white shadow-lg shadow-primary/20"
            : "text-muted-foreground hover:bg-white/5 hover:text-white"
        )}
      >
        <Icon className="w-4 h-4" />
        {room}
      </button>
    );
  })}
</nav>

<div className="mt-auto">
  <div className="p-4 rounded-2xl bg-gradient-to-br from-card to-secondary border border-white/5">
    <p className="text-xs text-muted-foreground mb-2">System Status</p>
    <div className="flex items-center gap-2 mb-1">
      <div className="w-2 h-2 rounded-full bg-green-500 animate-pulse" />
      <span className="text-sm font-medium text-white">Online</span>
    </div>
    <p className="text-xs text-muted-foreground">Firmware v2.4.0</p>
  </div>
</div>
);
}

```

`client/src/components/StickyNoteItem.tsx`

```

import { useState, useRef, useCallback, memo } from "react";
import { useMutation, useQueryClient } from "@tanstack/react-query";
import { apiRequest } from "@lib/queryClient";
import type { StickyNote as StickyNoteType } from "@shared/schema";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from "@components/ui/dropdown-menu";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";
import { Checkbox } from "@components/ui/checkbox";
import {
  Palette,
  Bell,
  Paperclip,
  Link2,
  Mail,
  Smartphone,
  Volume2,
  List,
  Plus,
  Minus,
  Trash2,
} from "lucide-react";
import { format } from "date-fns";

const NOTE_COLORS: Record<string, { bg: string; border: string; header: string }> = {
  yellow: { bg: '#FFFACD', border: '#E6D200', header: '#FFFF00' },
  pink: { bg: '#FFE4EC', border: '#FF69B4', header: '#FFB6C1' },
  blue: { bg: '#E0F0FF', border: '#4DA6FF', header: '#87CEEB' },
  green: { bg: '#E0FFE0', border: '#32CD32', header: '#98FB98' },
  orange: { bg: '#FFE4CC', border: '#FF8C00', header: '#FFCC99' },
  purple: { bg: '#F0E0FF', border: '#9370DB', header: '#DDA0DD' },
};

function hexToRgba(hex: string, alpha: number) {
  const r = parseInt(hex.slice(1, 3), 16);
  const g = parseInt(hex.slice(3, 5), 16);
  const b = parseInt(hex.slice(5, 7), 16);
  return `rgba(${r}, ${g}, ${b}, ${alpha})`;
}

interface StickyNoteItemProps {
  note: StickyNoteType;
  isDragging: boolean;
  dragPosition: { x: number; y: number } | null;
  maxStickyZIndex: number;
  tasks: Array<{ id: number; title: string }>;
  allProjects: Array<{ id: number; name: string }>;
  onPointerDown: (e: React.MouseEvent | React.TouchEvent, noteId: number, note: StickyNoteType) => void;
  onResizeStart: (e: React.MouseEvent | React.TouchEvent, noteId: number, note: StickyNoteType) => void;
  onDelete: (note: StickyNoteType) => void;
}

```

```

    onBringToFront: (noteId: number) => void;
  }

const StickyNoteItem = memo(function StickyNoteItem({
  note,
  isDragging,
  dragPosition,
  maxStickyZIndex,
  tasks,
  allProjects,
  onPointerDown,
  onResizeStart,
  onDelete,
  onBringToFront,
}: StickyNoteItemProps) {
  const queryClient = useQueryClient();
  const [localContent, setLocalContent] = useState(note.content);
  const [localTitle, setLocalTitle] = useState(note.title || "Note Name");
  const contentTimeout = useRef<NodeJS.Timeout | null>(null);
  const titleTimeout = useRef<NodeJS.Timeout | null>(null);
  const prevNoteIdRef = useRef(note.id);
  const prevContentRef = useRef(note.content);
  const prevTitleRef = useRef(note.title);

  if (prevNoteIdRef.current !== note.id) {
    prevNoteIdRef.current = note.id;
    setLocalContent(note.content);
    setLocalTitle(note.title || "Note Name");
    prevContentRef.current = note.content;
    prevTitleRef.current = note.title;
  } else {
    if (prevContentRef.current !== note.content && !contentTimeout.current) {
      setLocalContent(note.content);
    }
    prevContentRef.current = note.content;
    if (prevTitleRef.current !== note.title && !titleTimeout.current) {
      setLocalTitle(note.title || "Note Name");
    }
    prevTitleRef.current = note.title;
  }

  const updateMutation = useMutation({
    mutationFn: ({ id, updates }: { id: number; updates: Partial<StickyNoteType> }) =>
      apiRequest("PATCH", "/api/sticky-notes/${id}", updates),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ["/api/sticky-notes"] }),
  });

  const handleContentChange = useCallback((newContent: string) => {
    setLocalContent(newContent);
    if (contentTimeout.current) clearTimeout(contentTimeout.current);
    contentTimeout.current = setTimeout(() => {
      updateMutation.mutate({ id: note.id, updates: { content: newContent } });
      contentTimeout.current = null;
    }, 500);
  }, [note.id, updateMutation]);

  const handleTitleChange = useCallback((newTitle: string) => {
    setLocalTitle(newTitle);
    if (titleTimeout.current) clearTimeout(titleTimeout.current);
    titleTimeout.current = setTimeout(() => {
      updateMutation.mutate({ id: note.id, updates: { title: newTitle } });
      titleTimeout.current = null;
    }, 500);
  }, [note.id, updateMutation]);

  const toggleBullets = useCallback(() => {
    const lines = localContent.split('\n');
    const hasBullets = lines.some(line => line.trimStart().startsWith('\u25CF ') || line.trimStart().startsWith('\u2022...'));
    let newContent: string;
    if (hasBullets) {
      newContent = lines.map(line => {
        const trimmed = line.trimStart();
        if (trimmed.startsWith('\u25CF ')) return line.replace(/\u25CF /, '');
        if (trimmed.startsWith('\u2022 ')) return line.replace(/\u2022 /, '');
        return line;
      }).join('\n');
    } else {
      newContent = lines.map(line => {
        if (line.trim() === '') return line;
        return '\u25CF ' + line;
      }).join('\n');
    }
    handleContentChange(newContent);
  }, [localContent, handleContentChange]);

  const handleKeyDown = useCallback((e: React.KeyboardEvent<HTMLTextAreaElement>) => {
    if (e.key === 'Enter') {
      const textarea = e.currentTarget;
      const { selectionStart } = textarea;
      const content = textarea.value;
      const lineStart = content.lastIndexOf('\n', selectionStart - 1) + 1;
      const currentLine = content.substring(lineStart, selectionStart);

      if (currentLine.trimStart().startsWith('\u25CF ') || currentLine.trimStart().startsWith('\u2022 ')) {
        const bulletChar = currentLine.trimStart().startsWith('\u25CF ') ? '\u25CF' : '\u2022';
        if (currentLine.trim() === bulletChar) {
          e.preventDefault();
          const before = content.substring(0, lineStart);
          const after = content.substring(selectionStart);
          handleContentChange(before + after);
          setTimeout(() => { textarea.selectionStart = textarea.selectionEnd = lineStart; }, 0);
        } else {
          e.preventDefault();
          const before = content.substring(0, selectionStart);
          const after = content.substring(selectionStart);
          const newContent = before + '\n' + bulletChar + ' ' + after;
          handleContentChange(newContent);
          setTimeout(() => { textarea.selectionStart = textarea.selectionEnd = selectionStart + 3; }, 0);
        }
      }
    }
  });

```

```

    }
  }
}, [handleContentChange]);

const colors = note.customColor
? { bg: hexToRgba(note.customColor, 0.3), border: note.customColor, header: note.customColor }
: (NOTE_COLORS[note.color] || NOTE_COLORS.yellow);

const displayX = Math.max(0, Math.min(note.positionX, window.innerWidth - 50));
const displayY = Math.max(0, Math.min(note.positionY, window.innerHeight - 30));

return (
  <div
    data-sticky-note
    data-sticky-note-id={note.id}
    className="fixed shadow-lg rounded-md overflow-hidden"
    style={{
      left: `${displayX}px`,
      top: `${displayY}px`,
      width: `${note.width}px`,
      height: note.isMinimized ? '28px' : `${note.height}px`,
      zIndex: isDragging ? 10000 : (note.zIndex || 100),
      backgroundColor: colors.bg,
      border: '1px solid ${colors.border}',
      pointerEvents: 'auto',
    }}
    data-testid={`sticky-note-${note.id}`}
  >
    <div
      className="flex items-center justify-between px-1 py-1 select-none cursor-move"
      style={{ backgroundColor: colors.header, borderBottom: '1px solid ${colors.border}', touchAction: 'none' }}
      onMouseDown={(e) => onPointerDown(e, note.id, note)}
      onTouchStart={(e) => onPointerDown(e, note.id, note)}
    >
      <div className="flex items-center gap-1 flex-1">
        <DropdownMenu>
          <DropdownMenuTrigger asChild>
            <button
              className="flex items-center justify-center text-gray-600 hover:text-gray-800"
              onMouseDown={(e) => e.stopPropagation()}
            >
              <Palette className="h-3 w-3" />
            </button>
          </DropdownMenuTrigger>
          <DropdownMenuContent className="min-w-0 p-2" onClick={(e) => e.stopPropagation()}>
            <div className="flex gap-2">
              <button
                className="h-5 w-5 rounded-full border border-gray-300 hover:scale-110 transition-transform"
                style={{ backgroundColor: '#4ade80' }}
                title="CPA122"
                onClick={() => updateMutation.mutate({ id: note.id, updates: { customColor: '#4ade80', color: 'cust...' })}
              />
              <button
                className="h-5 w-5 rounded-full border border-gray-300 hover:scale-110 transition-transform"
                style={{ backgroundColor: '#f472b6' }}
                title="CFN400"
                onClick={() => updateMutation.mutate({ id: note.id, updates: { customColor: '#f472b6', color: 'cust...' })}
              />
              <button
                className="h-5 w-5 rounded-full border border-gray-300 hover:scale-110 transition-transform"
                style={{ backgroundColor: '#818cf8' }}
                title="CAS101"
                onClick={() => updateMutation.mutate({ id: note.id, updates: { customColor: '#818cf8', color: 'cust...' })}
              />
            </div>
          </DropdownMenuContent>
        </DropdownMenu>
        <input
          type="text"
          value={localTitle}
          onChange={(e) => handleTitleChange(e.target.value)}
          onClick={(e) => e.stopPropagation()}
          onMouseDown={(e) => e.stopPropagation()}
          className="text-[10px] text-gray-700 font-medium border-none outline-none w-20 cursor-text rounded px-0.5"
          style={{ backgroundColor: 'white', marginRight: '-8px' }}
          placeholder="Note Name"
          data-testid={`sticky-note-title-${note.id}`}
        />
      </div>
      <div className="flex items-center gap-2">
        {(note.taskId || note.projectId) && (
          <span className="text-[8px] text-gray-600 truncate max-w-[60px]" title={
            note.taskId
              ? tasks.find(t => t.id === note.taskId)?.title || 'Task'
              : allProjects?.find(p => p.id === note.projectId)?.name || 'Project'
          }>
            <Link2 className="h-2 w-2 inline" />
          </span>
        )}
        <DropdownMenu>
          <DropdownMenuTrigger asChild>
            <button
              className={`flex items-center justify-center ${note.reminderTime ? 'text-amber-600' : 'text-gray-600'}...`
              title="Set reminder"
            >
              <Bell className="h-3 w-3" />
            </button>
          </DropdownMenuTrigger>
          <DropdownMenuContent className="w-56 p-2" onClick={(e) => e.stopPropagation()}>
            <DropdownMenuLabel className="text-[9px] py-1" style={{ marginLeft: '0px' }}>Reminder Settings</DropdownMenuLabel>
            <div className="space-y-2 p-1">
              <div className="space-y-1">
                <Label className="text-[9px]">Reminder Time</Label>
                <Input
                  type="datetime-local"
                  className="h-6 text-[8px] px-1"
                  value={note.reminderTime ? format(new Date(note.reminderTime), "yyyy-MM-dd'T'HH:mm") : ''}
                />
              </div>
            </div>
          </DropdownMenuContent>
        </DropdownMenu>
      </div>
    </div>
  </div>
)

```

```

      onChange={e} => {
        const value = e.target.value ? new Date(e.target.value) : null;
        updateMutation.mutate({ id: note.id, updates: { reminderTime: value } });
      }
    }
  }
</div>
<DropdownMenuSeparator />
<div className="flex items-center justify-between">
  <div className="flex items-center gap-1">
    <Volume2 className="h-3 w-3" />
    <span className="text-[10px]">Alarm</span>
  </div>
  <Checkbox
    checked={note.reminderAlarm}
    onChange={checked => updateMutation.mutate({ id: note.id, updates: { reminderAlarm: !!checked })}
  />
</div>
<div className="flex items-center justify-between">
  <div className="flex items-center gap-1">
    <Mail className="h-3 w-3" />
    <span className="text-[10px]">Email</span>
  </div>
  <Checkbox
    checked={note.reminderEmail}
    onChange={checked => updateMutation.mutate({ id: note.id, updates: { reminderEmail: !!checked })}
  />
</div>
<div className="flex items-center justify-between">
  <div className="flex items-center gap-1">
    <Smartphone className="h-3 w-3" />
    <span className="text-[10px]">Push</span>
  </div>
  <Checkbox
    checked={note.reminderPush}
    onChange={checked => updateMutation.mutate({ id: note.id, updates: { reminderPush: !!checked })}
  />
</div>
{note.reminderTime && (
  <Button
    variant="ghost"
    size="sm"
    className="w-full h-6 text-[10px] text-red-600 hover:text-red-700"
    onClick={() => updateMutation.mutate({ id: note.id, updates: { reminderTime: null, reminderAlarm: ... })}
  >
    Clear Reminder
  </Button>
)}
</div>
</DropdownMenuContent>
</DropdownMenu>
<DropdownMenu>
  <DropdownMenuTrigger asChild>
    <button
      className="flex items-center justify-center text-gray-600 hover:text-gray-800"
      title="Attach to task or project"
    >
      <Paperclip className="h-3 w-3" />
    </button>
  </DropdownMenuTrigger>
  <DropdownMenuContent className="max-h-[300px] overflow-y-auto w-48">
    <DropdownMenuLabel className="text-[10px] py-1">Attach to Task</DropdownMenuLabel>
    <DropdownMenuItem
      className="text-[10px] py-1"
      onClick={() => updateMutation.mutate({ id: note.id, updates: { taskId: null } })}
    >
      <span className="text-gray-500">None</span>
    </DropdownMenuItem>
    {tasks.slice(0, 20).map(task => (
      <DropdownMenuItem
        key={task.id}
        className="text-[10px] py-1 truncate"
        onClick={() => updateMutation.mutate({ id: note.id, updates: { taskId: task.id, projectId: null } })}
      >
        <span className={note.taskId === task.id ? "font-semibold" : ""}>
          {task.title}
        </span>
      </DropdownMenuItem>
    ))}
    <DropdownMenuSeparator />
    <DropdownMenuLabel className="text-[10px] py-1">Attach to Project</DropdownMenuLabel>
    <DropdownMenuItem
      className="text-[10px] py-1"
      onClick={() => updateMutation.mutate({ id: note.id, updates: { projectId: null } })}
    >
      <span className="text-gray-500">None</span>
    </DropdownMenuItem>
    {allProjects?.map(project => (
      <DropdownMenuItem
        key={project.id}
        className="text-[10px] py-1 truncate"
        onClick={() => updateMutation.mutate({ id: note.id, updates: { projectId: project.id, taskId: null } })}
      >
        <span className={note.projectId === project.id ? "font-semibold" : ""}>
          {project.name}
        </span>
      </DropdownMenuItem>
    ))}
  </DropdownMenuContent>
</DropdownMenu>
<div className="relative">
  <input
    type="color"
    value={note.customColor || NOTE_COLORS[note.color]?.header || '#FFFACD'}
    onChange={e => updateMutation.mutate({ id: note.id, updates: { customColor: e.target.value, color: 'c...' })}
    className="absolute opacity-0 w-0 h-0"
    id={`color-picker-${note.id}`}
  />
  <label

```



```

        htmlFor={`color-picker-${note.id}`}
        className="h-3 w-3 rounded-full border border-gray-400 hover:opacity-80 cursor-pointer block"
        style={{ backgroundColor: note.customColor || colors.header }}
      />
    </div>
    <button
      className="h-4 w-4 flex items-center justify-center text-gray-600 hover:text-gray-800"
      onClick={toggleBullets}
      onMouseDown={(e) => e.stopPropagation()}
      onTouchStart={(e) => e.stopPropagation()}
      onPointerDown={(e) => e.stopPropagation()}
      title="Toggle bullet list"
      data-testid={`sticky-note-bullets-${note.id}`}
    >
      <List className="h-3 w-3" />
    </button>
    <button
      className="h-4 w-4 flex items-center justify-center text-gray-600 hover:text-gray-800"
      onClick={() => updateMutation.mutate({ id: note.id, updates: { isMinimized: !note.isMinimized } })}
      onMouseDown={(e) => e.stopPropagation()}
      onTouchStart={(e) => e.stopPropagation()}
      onPointerDown={(e) => e.stopPropagation()}
    >
      {note.isMinimized ? <Plus className="h-3 w-3" /> : <Minus className="h-3 w-3" />}
    </button>
    <button
      className="h-4 w-4 flex items-center justify-center text-gray-600 hover:text-red-600"
      onClick={() => onDelete(note)}
      onMouseDown={(e) => e.stopPropagation()}
      onTouchStart={(e) => e.stopPropagation()}
      onPointerDown={(e) => e.stopPropagation()}
      title="Delete note"
      data-testid={`sticky-note-delete-${note.id}`}
    >
      <Trash2 className="h-3 w-3" />
    </button>
  </div>
</div>
{!note.isMinimized && (
  <>
    <textarea
      className="w-full h-[calc(100%-28px)] p-2 text-[11px] resize-none border-0 outline-none !font-normal"
      style={{ backgroundColor: 'transparent', fontFamily: 'inherit' }}
      value={localContent}
      onChange={(e) => handleContentChange(e.target.value)}
      onKeyDown={handleKeyDown}
      placeholder="Write your note here..."
      data-testid={`sticky-note-content-${note.id}`}
    />
    <div
      className="absolute bottom-0 right-0 w-4 h-4 cursor-se-resize"
      style={{
        background: `linear-gradient(135deg, transparent 50%, ${colors.header} 50%)`,
      }}
      onMouseDown={(e) => onResizeStart(e, note.id, note)}
      onTouchStart={(e) => onResizeStart(e, note.id, note)}
      data-testid={`sticky-note-resize-${note.id}`}
    />
  </>
)}
</div>
);
});

export default StickyNoteItem;

```

`client/src/components/access-gate.tsx`

```

import { useState, useEffect, createContext, useContext } from "react";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Lock, AlertCircle } from "lucide-react";

interface AccessGateProps {
  children: React.ReactNode;
}

interface AccessContextType {
  isReadOnly: boolean;
  isAdmin: boolean;
}

const AccessContext = createContext<AccessContextType>({ isReadOnly: false, isAdmin: true });

export function useAccessMode() {
  return useContext(AccessContext);
}

export function AccessGate({ children }: AccessGateProps) {
  const [isAuthorized, setIsAuthorized] = useState<boolean | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [passwordInput, setPasswordInput] = useState("");
  const [isValidating, setIsValidating] = useState(false);

  useEffect(() => {
    checkAccess();
  }, []);

  const checkAccess = async () => {
    try {
      const urlParams = new URLSearchParams(window.location.search);
      const authParam = urlParams.get('auth');
      const checkUrl = authParam ? `/api/auth/check?auth=${encodeURIComponent(authParam)}` : "/api/auth/check";
      const res = await fetch(checkUrl);
    }
  };
}

```

```

const data = await res.json();
if (data.authenticated) {
  if (data.token) {
    localStorage.setItem('uni_cal_token', data.token);
  }
  if (authParam) {
    const profileNames = ['bryn', 'yasu', 'guest'];
    if (!profileNames.includes(authParam.toLowerCase())) {
      urlParams.delete('auth');
      const newSearch = urlParams.toString();
      const newUrl = window.location.pathname + (newSearch ? `?${newSearch}` : '');
      window.history.replaceState({}, '', newUrl);
    }
  }
  setIsAuthorized(true);
  return;
} catch {}

if (import.meta.env.DEV) {
  setIsAuthorized(true);
  return;
}

setIsAuthorized(false);
};

const handlePasswordSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!passwordInput.trim()) return;
  setIsValidating(true);
  setError(null);

  try {
    const res = await fetch("/api/auth/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ password: passwordInput.trim() }),
    });
    const data = await res.json();
    if (data.success) {
      if (data.token) {
        localStorage.setItem('uni_cal_token', data.token);
      }
      setIsAuthorized(true);
    } else {
      setError(data.message || "Incorrect password");
    }
  } catch {
    setError("Connection error. Please try again.");
  } finally {
    setIsValidating(false);
  }
};

if (isAuthorized === null) {
  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-900">
      <div className="animate-pulse text-white">Loading...</div>
    </div>
  );
}

if (!isAuthorized) {
  return (
    <div className="min-h-screen flex items-center justify-center bg-gradient-to-br from-gray-900 via-gray-800 to-gray-700">
      <div className="max-w-sm w-full bg-gray-800/50 backdrop-blur-sm border border-gray-700 rounded-lg p-6 sm:p-8 ...">
        <div className="w-14 h-14 sm:w-16 sm:h-16 bg-blue-500/20 rounded-full flex items-center justify-center mx-a...">
          <Lock className="w-7 h-7 sm:w-8 sm:h-8 text-blue-400" />
        </div>

        <h1 className="text-xl sm:text-2xl font-bold text-white mb-1 sm:mb-2">Uni-Cal</h1>
        <p className="text-gray-400 text-sm sm:text-base mb-4 sm:mb-6">
          Enter password to continue
        </p>

        {error && (
          <div className="bg-red-500/10 border border-red-500/30 rounded-lg p-3 mb-4 flex items-center gap-2 text-r...">
            <AlertCircle className="w-4 h-4 flex-shrink-0" />
            <span className="text-sm">{error}</span>
          </div>
        )}

        <form onSubmit={handlePasswordSubmit} className="space-y-3 sm:space-y-4">
          <Input
            type="password"
            placeholder="Password"
            value={passwordInput}
            onChange={(e) => setPasswordInput(e.target.value)}
            className="bg-gray-900/50 border-gray-600 text-white text-base h-11"
            autoFocus
            data-testid="input-password"
          />
          <Button
            type="submit"
            className="w-full h-11"
            disabled={isValidating || !passwordInput.trim()}
            data-testid="button-login"
          >
            {isValidating ? "Checking..." : "Enter"}
          </Button>
        </form>
      </div>
    </div>
  );
}

return (

```

```
    <AccessContext.Provider value={{ isReadOnly: false, isAdmin: true }}>
      {children}
    </AccessContext.Provider>
  );
}
```

`client/src/components/ui/accordion.tsx`

```
import * as React from "react"
import * as AccordionPrimitive from "@radix-ui/react-accordion"
import { ChevronDown } from "lucide-react"

import { cn } from "@lib/utils"

const Accordion = AccordionPrimitive.Root

const AccordionItem = React.forwardRef<
  React.ElementRef<typeof AccordionPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof AccordionPrimitive.Item>
>(({ className, ...props }, ref) => {
  <AccordionPrimitive.Item
    ref={ref}
    className={cn("border-b", className)}
    {...props}
  />
))
AccordionItem.displayName = "AccordionItem"

const AccordionTrigger = React.forwardRef<
  React.ElementRef<typeof AccordionPrimitive.Trigger>,
  React.ComponentPropsWithoutRef<typeof AccordionPrimitive.Trigger>
>(({ className, children, ...props }, ref) => {
  <AccordionPrimitive.Header className="flex">
    <AccordionPrimitive.Trigger
      ref={ref}
      className={cn(
        "flex flex-1 items-center justify-between py-4 font-medium transition-all hover:underline [&[data-state=open]...
      )}
      {...props}
    >
      {children}
      <ChevronDown className="h-4 w-4 shrink-0 transition-transform duration-200" />
    </AccordionPrimitive.Trigger>
  </AccordionPrimitive.Header>
))
AccordionTrigger.displayName = AccordionPrimitive.Trigger.displayName

const AccordionContent = React.forwardRef<
  React.ElementRef<typeof AccordionPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof AccordionPrimitive.Content>
>(({ className, children, ...props }, ref) => {
  <AccordionPrimitive.Content
    ref={ref}
    className="overflow-hidden text-sm transition-all data-[state=closed]:animate-accordion-up data-[state=open]:anim...
    {...props}
  >
    <div className={cn("pb-4 pt-0", className)}>{children}</div>
  </AccordionPrimitive.Content>
))
AccordionContent.displayName = AccordionPrimitive.Content.displayName

export { Accordion, AccordionItem, AccordionTrigger, AccordionContent }
```

`client/src/components/ui/alert-dialog.tsx`

```
import * as React from "react"
import * as AlertDialogPrimitive from "@radix-ui/react-alert-dialog"

import { cn } from "@lib/utils"
import { buttonVariants } from "@components/ui/button"

const AlertDialog = AlertDialogPrimitive.Root

const AlertDialogTrigger = AlertDialogPrimitive.Trigger

const AlertDialogPortal = AlertDialogPrimitive.Portal

const AlertDialogOverlay = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Overlay>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Overlay>
>(({ className, ...props }, ref) => {
  <AlertDialogPrimitive.Overlay
    className={cn(
      "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=close...
    )}
    {...props}
    ref={ref}
  />
))
AlertDialogOverlay.displayName = AlertDialogPrimitive.Overlay.displayName

const AlertDialogContent = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Content>
>(({ className, ...props }, ref) => {
  <AlertDialogPortal>
    <AlertDialogOverlay />
  </AlertDialogPortal>
  <AlertDialogPrimitive.Content
    ref={ref}
    className={cn(
      "relative z-50 w-full md:w-sm md:left-auto md:right-0 p-6 text-left transition-all",
      className
    )}
    {...props}
  />
))
AlertDialogContent.displayName = AlertDialogPrimitive.Content.displayName

export { AlertDialog, AlertDialogTrigger, AlertDialogContent, AlertDialogOverlay, AlertDialogPortal }
```

```

        <AlertDialogPrimitive.Content
          ref={ref}
          className={cn(
            "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4 border bg-b...
            className
          )}
          {...props}
        />
      </AlertDialogPortal>
    )
    AlertDialogContent.displayName = AlertDialogPrimitive.Content.displayName

    const AlertDialogHeader = ({
      className,
      ...props
    }: React.HTMLAttributes<HTMLDivElement>) => (
      <div
        className={cn(
          "flex flex-col space-y-2 text-center sm:text-left",
          className
        )}
        {...props}
      />
    )
    AlertDialogHeader.displayName = "AlertDialogHeader"

    const AlertDialogFooter = ({
      className,
      ...props
    }: React.HTMLAttributes<HTMLDivElement>) => (
      <div
        className={cn(
          "flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2",
          className
        )}
        {...props}
      />
    )
    AlertDialogFooter.displayName = "AlertDialogFooter"

    const AlertDialogTitle = React.forwardRef<
      React.ElementRef<typeof AlertDialogPrimitive.Title>,
      React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Title>
    >(({ className, ...props }, ref) => (
      <AlertDialogPrimitive.Title
        ref={ref}
        className={cn("text-lg font-semibold", className)}
        {...props}
      />
    ))
    AlertDialogTitle.displayName = AlertDialogPrimitive.Title.displayName

    const AlertDialogDescription = React.forwardRef<
      React.ElementRef<typeof AlertDialogPrimitive.Description>,
      React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Description>
    >(({ className, ...props }, ref) => (
      <AlertDialogPrimitive.Description
        ref={ref}
        className={cn("text-sm text-muted-foreground", className)}
        {...props}
      />
    ))
    AlertDialogDescription.displayName =
      AlertDialogPrimitive.Description.displayName

    const AlertDialogAction = React.forwardRef<
      React.ElementRef<typeof AlertDialogPrimitive.Action>,
      React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Action>
    >(({ className, ...props }, ref) => (
      <AlertDialogPrimitive.Action
        ref={ref}
        className={cn(buttonVariants(), className)}
        {...props}
      />
    ))
    AlertDialogAction.displayName = AlertDialogPrimitive.Action.displayName

    const AlertDialogCancel = React.forwardRef<
      React.ElementRef<typeof AlertDialogPrimitive.Cancel>,
      React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Cancel>
    >(({ className, ...props }, ref) => (
      <AlertDialogPrimitive.Cancel
        ref={ref}
        className={cn(
          buttonVariants({ variant: "outline" }),
          "mt-2 sm:mt-0",
          className
        )}
        {...props}
      />
    ))
    AlertDialogCancel.displayName = AlertDialogPrimitive.Cancel.displayName

    export {
      AlertDialog,
      AlertDialogPortal,
      AlertDialogOverlay,
      AlertDialogTrigger,
      AlertDialogContent,
      AlertDialogHeader,
      AlertDialogFooter,
      AlertDialogTitle,
      AlertDialogDescription,
      AlertDialogAction,
      AlertDialogCancel,
    }

```

`client/src/components/ui/alert.tsx`

```

import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@lib/utils"

const alertVariants = cva(
  "relative w-full rounded-lg border p-4 [&svg~*]:pl-7 [&svg+div]:translate-y-[-3px] [&svg]:absolute [&svg]:left-...",
  {
    variants: {
      variant: {
        default: "bg-background text-foreground",
        destructive:
          "border-destructive/50 text-destructive dark:border-destructive [&svg]:text-destructive",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  },
)

const Alert = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement> & VariantProps<typeof alertVariants>
>(({ className, variant, ...props }, ref) => {
  <div
    ref={ref}
    role="alert"
    className={cn(alertVariants({ variant }), className)}
    {...props}
  />
))
Alert.displayName = "Alert"

const AlertTitle = React.forwardRef<
  HTMLParagraphElement,
  React.HTMLAttributes<HTMLHeadingElement>
>(({ className, ...props }, ref) => {
  <h5
    ref={ref}
    className={cn("mb-1 font-medium leading-none tracking-tight", className)}
    {...props}
  />
))
AlertTitle.displayName = "AlertTitle"

const AlertDescription = React.forwardRef<
  HTMLParagraphElement,
  React.HTMLAttributes<HTMLParagraphElement>
>(({ className, ...props }, ref) => {
  <div
    ref={ref}
    className={cn("text-sm [&p]:leading-relaxed", className)}
    {...props}
  />
))
AlertDescription.displayName = "AlertDescription"

export { Alert, AlertTitle, AlertDescription }

```

`client/src/components/ui/aspect-ratio.tsx`

```

import * as AspectRatioPrimitive from "@radix-ui/react-aspect-ratio"

const AspectRatio = AspectRatioPrimitive.Root

export { AspectRatio }

```

`client/src/components/ui/avatar.tsx`

```

"use client"

import * as React from "react"
import * as AvatarPrimitive from "@radix-ui/react-avatar"

import { cn } from "@lib/utils"

const Avatar = React.forwardRef<
  React.ElementRef<typeof AvatarPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof AvatarPrimitive.Root>
>(({ className, ...props }, ref) => {
  <AvatarPrimitive.Root
    ref={ref}
    className={cn(
      "after:content-[''] after:block after:absolute after:inset-0 after:rounded-full after:pointer-events-none after:...
      relative flex h-10 w-10 shrink-0 overflow-hidden rounded-full",
      className
    )}
    {...props}
  />
))
Avatar.displayName = AvatarPrimitive.Root.displayName

const AvatarImage = React.forwardRef<
  React.ElementRef<typeof AvatarPrimitive.Image>,
  React.ComponentPropsWithoutRef<typeof AvatarPrimitive.Image>
>(({ className, ...props }, ref) => {
  <AvatarPrimitive.Image

```

```

    ref={ref}
    className={cn("aspect-square h-full w-full", className)}
    {...props}
  />
))
AvatarImage.displayName = AvatarPrimitive.Image.displayName

const AvatarFallback = React.forwardRef<
  React.ElementRef<typeof AvatarPrimitive.Fallback>,
  React.ComponentPropsWithoutRef<typeof AvatarPrimitive.Fallback>
>(({ className, ...props }, ref) => (
  <AvatarPrimitive.Fallback
    ref={ref}
    className={cn(
      "flex h-full w-full items-center justify-center rounded-full bg-muted",
      className
    )}
    {...props}
  />
))
AvatarFallback.displayName = AvatarPrimitive.Fallback.displayName

export { Avatar, AvatarImage, AvatarFallback }

```

`client/src/components/ui/badge.tsx`

```

import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@lib/utils"

const badgeVariants = cva(
  // whitespace-nowrap: Badges should never wrap.
  "whitespace-nowrap inline-flex items-center rounded-md border px-2.5 py-0.5 text-xs font-semibold transition-colors...",
  " hover-elevate " ,
  {
    variants: {
      variant: {
        default:
          "border-transparent bg-primary text-primary-foreground shadow-xs",
        secondary: "border-transparent bg-secondary text-secondary-foreground",
        destructive:
          "border-transparent bg-destructive text-destructive-foreground shadow-xs",

        outline: " border [border-color:var(--badge-outline)] shadow-xs",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  },
)

export interface BadgeProps
  extends React.HTMLAttributes<HTMLDivElement>,
    VariantProps<typeof badgeVariants> {}

function Badge({ className, variant, ...props }: BadgeProps) {
  return (
    <div className={cn(badgeVariants({ variant }), className)} {...props} />
  );
}

export { Badge, badgeVariants }

```

`client/src/components/ui/breadcrumb.tsx`

```

import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { ChevronRight, MoreHorizontal } from "lucide-react"

import { cn } from "@lib/utils"

const Breadcrumb = React.forwardRef<
  HTMLElement,
  React.ComponentPropsWithoutRef<"nav"> & {
    separator?: React.ReactNode
  }
>(({ ...props }, ref) => <nav ref={ref} aria-label="breadcrumb" {...props} />)
Breadcrumb.displayName = "Breadcrumb"

const BreadcrumbList = React.forwardRef<
  HTMLListElement,
  React.ComponentPropsWithoutRef<"ol">
>(({ className, ...props }, ref) => (
  <ol
    ref={ref}
    className={cn(
      "flex flex-wrap items-center gap-1.5 break-words text-sm text-muted-foreground sm:gap-2.5",
      className
    )}
    {...props}
  />
))
BreadcrumbList.displayName = "BreadcrumbList"

const BreadcrumbItem = React.forwardRef<
  HTMLLIElement,
  React.ComponentPropsWithoutRef<"li">
>(({ className, ...props }, ref) => (

```

```

    <li
      ref={ref}
      className={cn("inline-flex items-center gap-1.5", className)}
      {...props}
    />
  ))
  BreadcrumbItem.displayName = "BreadcrumbItem"

  const BreadcrumbLink = React.forwardRef<
    HTMLAnchorElement,
    React.ComponentPropsWithoutRef<"a"> & {
      asChild?: boolean
    }
  >(({ asChild, className, ...props }, ref) => {
    const Comp = asChild ? Slot : "a"

    return (
      <Comp
        ref={ref}
        className={cn("transition-colors hover:text-foreground", className)}
        {...props}
      />
    )
  })
  BreadcrumbLink.displayName = "BreadcrumbLink"

  const BreadcrumbPage = React.forwardRef<
    HTMLSpanElement,
    React.ComponentPropsWithoutRef<"span">
  >(({ className, ...props }, ref) => (
    <span
      ref={ref}
      role="link"
      aria-disabled="true"
      aria-current="page"
      className={cn("font-normal text-foreground", className)}
      {...props}
    />
  ))
  BreadcrumbPage.displayName = "BreadcrumbPage"

  const BreadcrumbSeparator = ({
    children,
    className,
    ...props
  }): React.ComponentProps<"li"> => (
    <li
      role="presentation"
      aria-hidden="true"
      className={cn("[&svg]:w-3.5 [&svg]:h-3.5", className)}
      {...props}
    >
    {children ?? <ChevronRight />}
    </li>
  )
  BreadcrumbSeparator.displayName = "BreadcrumbSeparator"

  const BreadcrumbEllipsis = ({
    className,
    ...props
  }): React.ComponentProps<"span"> => (
    <span
      role="presentation"
      aria-hidden="true"
      className={cn("flex h-9 w-9 items-center justify-center", className)}
      {...props}
    >
    <MoreHorizontal className="h-4 w-4" />
    <span className="sr-only">More</span>
    </span>
  )
  BreadcrumbEllipsis.displayName = "BreadcrumbElipssis"

  export {
    Breadcrumb,
    BreadcrumbList,
    BreadcrumbItem,
    BreadcrumbLink,
    BreadcrumbPage,
    BreadcrumbSeparator,
    BreadcrumbEllipsis,
  }
}

```

`client/src/components/ui/button.tsx`

```

import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@/lib/utls"

const buttonVariants = cva(
  "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-medium focus-visible:outli...
  " hover-elevate active-elevate-2",
  {
    variants: {
      variant: {
        default:
          "bg-primary text-primary-foreground border border-primary-border",
        destructive:
          "bg-destructive text-destructive-foreground border border-destructive-border",
        outline:
          // Shows the background color of whatever card / sidebar / accent background it is inside of.
          // Inherits the current text color.

```

```

        " border [border-color:var(--button-outline)] shadow-xs active:shadow-none ",
        secondary: "border bg-secondary text-secondary-foreground border border-secondary-border ",
        // Add a transparent border so that when someone toggles a border on later, it doesn't shift layout/size.
        ghost: "border border-transparent",
    },
    // Heights are set as "min" heights, because sometimes Ai will place large amount of content
    // inside buttons. With a min-height they will look appropriate with small amounts of content,
    // but will expand to fit large amounts of content.
    size: {
        default: "min-h-9 px-4 py-2",
        sm: "min-h-8 rounded-md px-3 text-xs",
        lg: "min-h-10 rounded-md px-8",
        icon: "h-9 w-9",
    },
},
defaultVariants: {
    variant: "default",
    size: "default",
},
},
),
)

export interface ButtonProps
    extends React.ButtonHTMLAttributes<HTMLButtonElement>,
        VariantProps<typeof buttonVariants> {
    asChild?: boolean
}

const Button = React.forwardRef<HTMLButtonElement, ButtonProps>(
    ({ className, variant, size, asChild = false, ...props }, ref) => {
        const Comp = asChild ? Slot : "button"
        return (
            <Comp
                className={cn(buttonVariants({ variant, size, className })))
                ref={ref}
                {...props}
            />
        ),
    },
)
Button.displayName = "Button"

export { Button, buttonVariants }

```

`client/src/components/ui/calendar.tsx`

```

import * as React from "react"
import { ChevronLeft, ChevronRight } from "lucide-react"
import { DayPicker } from "react-day-picker"

import { cn } from "@lib/utlis"
import { buttonVariants } from "@components/ui/button"

export type CalendarProps = React.ComponentProps<typeof DayPicker>

function Calendar({
    className,
    classNames,
    showOutsideDays = true,
    ...props
}: CalendarProps) {
    return (
        <DayPicker
            showOutsideDays={showOutsideDays}
            className={cn("p-3", className)}
            classNames={{
                months: "flex flex-col sm:flex-row space-y-4 sm:space-x-4 sm:space-y-0",
                month: "space-y-4",
                caption: "flex justify-center pt-1 relative items-center",
                caption_label: "text-sm font-medium",
                nav: "space-x-1 flex items-center",
                nav_button: cn(
                    buttonVariants({ variant: "outline" }),
                    "h-7 w-7 bg-transparent p-0 opacity-50 hover:opacity-100"
                ),
                nav_button_previous: "absolute left-1",
                nav_button_next: "absolute right-1",
                table: "w-full border-collapse space-y-1",
                head_row: "flex",
                head_cell:
                    "text-muted-foreground rounded-md w-9 font-normal text-[0.8rem]",
                row: "flex w-full mt-2",
                cell: "h-9 w-9 text-center text-sm p-0 relative [&:has([aria-selected].day-range-end)]:rounded-r-md [&:has([a...
                day: cn(
                    buttonVariants({ variant: "ghost" }),
                    "h-9 w-9 p-0 font-normal aria-selected:opacity-100"
                ),
                day_range_end: "day-range-end",
                day_selected:
                    "bg-primary text-primary-foreground hover:bg-primary hover:text-primary-foreground focus:bg-primary focus:t...
                day_today: "bg-accent text-accent-foreground",
                day_outside:
                    "day-outside text-muted-foreground aria-selected:bg-accent/50 aria-selected:text-muted-foreground",
                day_disabled: "text-muted-foreground opacity-50",
                day_range_middle:
                    "aria-selected:bg-accent aria-selected:text-accent-foreground",
                day_hidden: "invisible",
                ...classNames,
            }}
            components={{
                IconLeft: ({ className, ...props }) => (
                    <ChevronLeft className={cn("h-4 w-4", className)} {...props} />
                ),
                IconRight: ({ className, ...props }) => (

```



```

        <ChevronRight className={cn("h-4 w-4", className)} {...props} />
      ),
    }},
    {...props}
  />
)
}
Calendar.displayName = "Calendar"

export { Calendar }

```

`client/src/components/ui/card.tsx`

```

import * as React from "react"

import { cn } from "@/lib/utils"

const Card = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn(
      "shadcn-card rounded-xl border bg-card border-card-border text-card-foreground shadow-sm",
      className
    )}
    {...props}
  />
));
Card.displayName = "Card"

const CardHeader = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("flex flex-col space-y-1.5 p-6", className)}
    {...props}
  />
));
CardHeader.displayName = "CardHeader"

const CardTitle = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn(
      "text-2xl font-semibold leading-none tracking-tight",
      className
    )}
    {...props}
  />
));
CardTitle.displayName = "CardTitle"

const CardDescription = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props}
  />
));
CardDescription.displayName = "CardDescription"

const CardContent = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div ref={ref} className={cn("p-6 pt-0", className)} {...props} />
))
CardContent.displayName = "CardContent"

const CardFooter = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("flex items-center p-6 pt-0", className)}
    {...props}
  />
))
CardFooter.displayName = "CardFooter"
export {
  Card,
  CardHeader,
  CardFooter,
  CardTitle,
  CardDescription,
  CardContent,
}

```

`client/src/components/ui/carousel.tsx`

```

import * as React from "react"
import useEmblaCarousel, {
  type UseEmblaCarouselType,
} from "embla-carousel-react"
import { ArrowLeft, ArrowRight } from "lucide-react"

import { cn } from "@lib/utills"
import { Button } from "@components/ui/button"

type CarouselApi = UseEmblaCarouselType[1]
type UseCarouselParameters = Parameters<typeof useEmblaCarousel>
type CarouselOptions = UseCarouselParameters[0]
type CarouselPlugin = UseCarouselParameters[1]

type CarouselProps = {
  opts?: CarouselOptions
  plugins?: CarouselPlugin
  orientation?: "horizontal" | "vertical"
  setApi?: (api: CarouselApi) => void
}

type CarouselContextProps = {
  carouselRef: ReturnType<typeof useEmblaCarousel>[0]
  api: ReturnType<typeof useEmblaCarousel>[1]
  scrollPrev: () => void
  scrollNext: () => void
  canScrollPrev: boolean
  canScrollNext: boolean
} & CarouselProps

const CarouselContext = React.createContext<CarouselContextProps | null>(null)

function useCarousel() {
  const context = React.useContext(CarouselContext)

  if (!context) {
    throw new Error("useCarousel must be used within a <Carousel />")
  }

  return context
}

const Carousel = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement> & CarouselProps
>((
  {
    orientation = "horizontal",
    opts,
    setApi,
    plugins,
    className,
    children,
    ...props
  },
  ref
) => {
  const [carouselRef, api] = useEmblaCarousel(
    {
      ...opts,
      axis: orientation === "horizontal" ? "x" : "y",
    },
    plugins
  )
  const [canScrollPrev, setCanScrollPrev] = React.useState(false)
  const [canScrollNext, setCanScrollNext] = React.useState(false)

  const onSelect = React.useCallback((api: CarouselApi) => {
    if (!api) {
      return
    }

    setCanScrollPrev(api.canScrollPrev())
    setCanScrollNext(api.canScrollNext())
  }, [])

  const scrollPrev = React.useCallback(() => {
    api?.scrollPrev()
  }, [api])

  const scrollNext = React.useCallback(() => {
    api?.scrollNext()
  }, [api])

  const handleKeyDown = React.useCallback(
    (event: React.KeyboardEvent<HTMLDivElement>) => {
      if (event.key === "ArrowLeft") {
        event.preventDefault()
        scrollPrev()
      } else if (event.key === "ArrowRight") {
        event.preventDefault()
        scrollNext()
      }
    },
    [scrollPrev, scrollNext]
  )

  React.useEffect(() => {
    if (!api || !setApi) {
      return
    }

    setApi(api)
  }, [api, setApi])

  React.useEffect(() => {

```

```

    if (!api) {
      return
    }

    onSelect(api)
    api.on("reInit", onSelect)
    api.on("select", onSelect)

    return () => {
      api?.off("select", onSelect)
    }
  }, [api, onSelect])

  return (
    <CarouselContext.Provider
      value={{
        carouselRef,
        api: api,
        opts,
        orientation:
          orientation || (opts?.axis === "y" ? "vertical" : "horizontal"),
        scrollPrev,
        scrollNext,
        canScrollPrev,
        canScrollNext,
      }}
    >
      <div
        ref={ref}
        onKeyDownCapture={handleKeyDown}
        className={cn("relative", className)}
        role="region"
        aria-roledescription="carousel"
        {...props}
      >
        {children}
      </div>
    </CarouselContext.Provider>
  )
}
)
Carousel.displayName = "Carousel"

const CarouselContent = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => {
  const { carouselRef, orientation } = useCarousel()

  return (
    <div ref={carouselRef} className="overflow-hidden">
      <div
        ref={ref}
        className={cn(
          "flex",
          orientation === "horizontal" ? "-ml-4" : "-mt-4 flex-col",
          className
        )}
        {...props}
      />
    </div>
  )
})
CarouselContent.displayName = "CarouselContent"

const CarouselItem = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => {
  const { orientation } = useCarousel()

  return (
    <div
      ref={ref}
      role="group"
      aria-roledescription="slide"
      className={cn(
        "min-w-0 shrink-0 grow-0 basis-full",
        orientation === "horizontal" ? "pl-4" : "pt-4",
        className
      )}
      {...props}
    />
  )
})
CarouselItem.displayName = "CarouselItem"

const CarouselPrevious = React.forwardRef<
  HTMLButtonElement,
  React.ComponentProps<typeof Button>
>(({ className, variant = "outline", size = "icon", ...props }, ref) => {
  const { orientation, scrollPrev, canScrollPrev } = useCarousel()

  return (
    <Button
      ref={ref}
      variant={variant}
      size={size}
      className={cn(
        "absolute h-8 w-8 rounded-full",
        orientation === "horizontal"
          ? "-left-12 top-1/2 -translate-y-1/2"
          : "-top-12 left-1/2 -translate-x-1/2 rotate-90",
        className
      )}
      disabled={!canScrollPrev}
      onClick={scrollPrev}
      {...props}
    />
  )
})

```

```

    <ArrowLeft className="h-4 w-4" />
    <span className="sr-only">Previous slide</span>
  </Button>
)
})
CarouselPrevious.displayName = "CarouselPrevious"

const CarouselNext = React.forwardRef<
  HTMLButtonElement,
  React.ComponentProps<typeof Button>
>(({ className, variant = "outline", size = "icon", ...props }, ref) => {
  const { orientation, scrollNext, canScrollNext } = useCarousel()

  return (
    <Button
      ref={ref}
      variant={variant}
      size={size}
      className={cn(
        "absolute h-8 w-8 rounded-full",
        orientation === "horizontal"
          ? "right-12 top-1/2 -translate-y-1/2"
          : "bottom-12 left-1/2 -translate-x-1/2 rotate-90",
        className
      )}
      disabled={!canScrollNext}
      onClick={scrollNext}
      {...props}
    >
      <ArrowRight className="h-4 w-4" />
      <span className="sr-only">Next slide</span>
    </Button>
  )
})
CarouselNext.displayName = "CarouselNext"

export {
  type CarouselApi,
  Carousel,
  CarouselContent,
  CarouselItem,
  CarouselPrevious,
  CarouselNext,
}

```

`client/src/components/ui/chart.tsx`

```

"use client"

import * as React from "react"
import * as RechartsPrimitive from "recharts"

import { cn } from "@lib/utlis"

// Format: { THEME_NAME: CSS_SELECTOR }
const THEMES = { light: "", dark: ".dark" } as const

export type ChartConfig = {
  [k in string]: {
    label?: React.ReactNode
    icon?: React.ComponentType
  } & (
    | { color?: string; theme?: never }
    | { color?: never; theme: Record<keyof typeof THEMES, string> }
  )
}

type ChartContextProps = {
  config: ChartConfig
}

const ChartContext = React.createContext<ChartContextProps | null>(null)

function useChart() {
  const context = React.useContext(ChartContext)

  if (!context) {
    throw new Error("useChart must be used within a <ChartContainer />")
  }

  return context
}

const ChartContainer = React.forwardRef<
  HTMLDivElement,
  React.ComponentProps<"div"> & {
    config: ChartConfig
    children: React.ComponentProps<
      typeof RechartsPrimitive.ResponsiveContainer
    >["children"]
  }
>(({ id, className, children, config, ...props }, ref) => {
  const uniqueId = React.useId()
  const chartId = `chart-${id || uniqueId.replace(/:/g, "")}`

  return (
    <ChartContext.Provider value={{ config }}>
      <div
        data-chart={chartId}
        ref={ref}
        className={cn(
          "flex aspect-video justify-center text-xs [&_.recharts-cartesian-axis-tick_text]:fill-muted-foreground [&....
        className
      >
    </ChartContext.Provider>
  )
}

```

```

    })
    {...props}
  >
  <ChartStyle id={chartId} config={config} />
  <RechartsPrimitive.ResponsiveContainer>
    {children}
  </RechartsPrimitive.ResponsiveContainer>
</div>
</ChartContext.Provider>
)
})
ChartContainer.displayName = "Chart"

const ChartStyle = ({ id, config }: { id: string; config: ChartConfig }) => {
  const colorConfig = Object.entries(config).filter(
    ([, config]) => config.theme || config.color
  )

  if (!colorConfig.length) {
    return null
  }

  return (
    <style
      dangerouslySetInnerHTML={{
        __html: Object.entries(THEMES)
          .map(
            ([theme, prefix]) => `
${prefix} [data-chart=${id}] {
${colorConfig
  .map(([key, itemConfig]) => {
    const color =
      itemConfig.theme?.[theme as keyof typeof itemConfig.theme] ||
      itemConfig.color
    return color ? `--color-${key}: ${color};` : null
  })
  .join("\n")}
}
`
          .join("\n"),
      }}
    />
  )
}

const ChartTooltip = RechartsPrimitive.Tooltip

const ChartTooltipContent = React.forwardRef<
  HTMLDivElement,
  React.ComponentProps<typeof RechartsPrimitive.Tooltip> &
  React.ComponentProps<"div"> & {
    hideLabel?: boolean
    hideIndicator?: boolean
    indicator?: "line" | "dot" | "dashed"
    nameKey?: string
    labelKey?: string
  }
>({
  {
    active,
    payload,
    className,
    indicator = "dot",
    hideLabel = false,
    hideIndicator = false,
    label,
    labelFormatter,
    labelClassName,
    formatter,
    color,
    nameKey,
    labelKey,
  },
  ref
) => {
  const { config } = useChart()

  const tooltipLabel = React.useMemo(() => {
    if (hideLabel || !payload?.length) {
      return null
    }
  }, [hideLabel, payload])

  const [item] = payload
  const key = `${labelKey || item?.dataKey || item?.name || "value"}`
  const itemConfig = getPayloadConfigFromPayload(config, item, key)
  const value =
    !labelKey && typeof label === "string"
      ? config[label as keyof typeof config]?.label || label
      : itemConfig?.label

  if (labelFormatter) {
    return (
      <div className={cn("font-medium", labelClassName)}>
        {labelFormatter(value, payload)}
      </div>
    )
  }

  if (!value) {
    return null
  }

  return <div className={cn("font-medium", labelClassName)}>{value}</div>
}, [
  label,
  labelFormatter,

```

```

    payload,
    hideLabel,
    labelClassName,
    config,
    labelKey,
  ])

  if (!active || !payload?.length) {
    return null
  }

  const nestLabel = payload.length === 1 && indicator !== "dot"

  return (
    <div
      ref={ref}
      className={cn(
        "grid min-w-[8rem] items-start gap-1.5 rounded-lg border border-border/50 bg-background px-2.5 py-1.5 text-...
        className
      )}
    >
      {!nestLabel ? tooltipLabel : null}
      <div className="grid gap-1.5">
        {payload.map((item, index) => {
          const key = `${nameKey} || item.name || item.dataKey || "value"}`
          const itemConfig = getPayloadConfigFromPayload(config, item, key)
          const indicatorColor = color || item.payload.fill || item.color

          return (
            <div
              key={item.dataKey}
              className={cn(
                "flex w-full flex-wrap items-stretch gap-2 [&svg]:h-2.5 [&svg]:w-2.5 [&svg]:text-muted-foreground",
                indicator === "dot" && "items-center"
              )}
            >
              {formatter && item?.value !== undefined && item.name ? (
                formatter(item.value, item.name, item, index, item.payload)
              ) : (
                <>
                  {itemConfig?.icon ? (
                    <itemConfig.icon />
                  ) : (
                    !hideIndicator && (
                      <div
                        className={cn(
                          "shrink-0 rounded-[2px] border-[--color-border] bg-[--color-bg]",
                          {
                            "h-2.5 w-2.5": indicator === "dot",
                            "w-1": indicator === "line",
                            "w-0 border-[1.5px] border-dashed bg-transparent":
                              indicator === "dashed",
                            "my-0.5": nestLabel && indicator === "dashed",
                          }
                        )}
                        style={
                          {
                            "--color-bg": indicatorColor,
                            "--color-border": indicatorColor,
                          } as React.CSSProperties
                        }
                      />
                    )
                  )}
                </div>
              )}
            </div>
          )}
        )}
      <div
        className={cn(
          "flex flex-1 justify-between leading-none",
          nestLabel ? "items-end" : "items-center"
        )}
      >
        <div className="grid gap-1.5">
          {nestLabel ? tooltipLabel : null}
          <span className="text-muted-foreground">
            {itemConfig?.label || item.name}
          </span>
        </div>
        {item.value && (
          <span className="font-mono font-medium tabular-nums text-foreground">
            {item.value.toLocaleString()}
          </span>
        )}
      </div>
    </div>
  )
}

ChartTooltipContent.displayName = "ChartTooltip"

const ChartLegend = RechartsPrimitive.Legend

const ChartLegendContent = React.forwardRef<
  HTMLDivElement,
  React.ComponentProps<"div"> &
  Pick<RechartsPrimitive.LegendProps, "payload" | "verticalAlign"> & {
    hideIcon?: boolean
    nameKey?: string
  }
>({
  ref,
  { className, hideIcon = false, payload, verticalAlign = "bottom", nameKey },
  => {

```

```

const { config } = useChart()

if (!payload?.length) {
  return null
}

return (
  <div
    ref={ref}
    className={cn(
      "flex items-center justify-center gap-4",
      verticalAlign === "top" ? "pb-3" : "pt-3",
      className
    )}
  >
    {payload.map((item) => {
      const key = `${nameKey} || item.dataKey || "value"}`
      const itemConfig = getPayloadConfigFromPayload(config, item, key)

      return (
        <div
          key={item.value}
          className={cn(
            "flex items-center gap-1.5 [&gt;svg]:h-3 [&gt;svg]:w-3 [&gt;svg]:text-muted-foreground"
          )}
        >
          {itemConfig?.icon && !hideIcon ? (
            <itemConfig.icon />
          ) : (
            <div
              className="h-2 w-2 shrink-0 rounded-[2px]"
              style={{
                backgroundColor: item.color,
              }}
            />
          )}
          {itemConfig?.label}
        </div>
      )
    })}
  </div>
)
)
ChartLegendContent.displayName = "ChartLegend"

// Helper to extract item config from a payload.
function getPayloadConfigFromPayload(
  config: ChartConfig,
  payload: unknown,
  key: string
) {
  if (typeof payload !== "object" || payload === null) {
    return undefined
  }

  const payloadPayload =
    "payload" in payload &&
    typeof payload.payload === "object" &&
    payload.payload !== null
      ? payload.payload
      : undefined

  let configLabelKey: string = key

  if (
    key in payload &&
    typeof payload[key as keyof typeof payload] === "string"
  ) {
    configLabelKey = payload[key as keyof typeof payload] as string
  } else if (
    payloadPayload &&
    key in payloadPayload &&
    typeof payloadPayload[key as keyof typeof payloadPayload] === "string"
  ) {
    configLabelKey = payloadPayload[
      key as keyof typeof payloadPayload
    ] as string
  }

  return configLabelKey in config
    ? config[configLabelKey]
    : config[key as keyof typeof config]
}

export {
  ChartContainer,
  ChartTooltip,
  ChartTooltipContent,
  ChartLegend,
  ChartLegendContent,
  ChartStyle,
}

```

`client/src/components/ui/checkbox.tsx`

```

import * as React from "react"
import * as CheckboxPrimitive from "@radix-ui/react-checkbox"
import { Check } from "lucide-react"

import { cn } from "@lib/utls"

const Checkbox = React.forwardRef<
  React.ElementRef<typeof CheckboxPrimitive.Root>,

```

```

    React.ComponentPropsWithoutRef<typeof CheckboxPrimitive.Root>
  >(({ className, ...props }, ref) => (
    <CheckboxPrimitive.Root
      ref={ref}
      className={cn(
        "peer h-2 w-2 shrink-0 rounded-sm border border-white ring-offset-background focus-visible:outline-none focus-v...
        className
      )}
      {...props}
    >
      <CheckboxPrimitive.Indicator
        className={cn("flex items-center justify-center text-current")}
      >
        <Check className="h-3 w-3" />
      </CheckboxPrimitive.Indicator>
    </CheckboxPrimitive.Root>
  ))
Checkbox.displayName = CheckboxPrimitive.Root.displayName

export { Checkbox }

```

`client/src/components/ui/collapsible.tsx`

```

"use client"

import * as CollapsiblePrimitive from "@radix-ui/react-collapsible"

const Collapsible = CollapsiblePrimitive.Root

const CollapsibleTrigger = CollapsiblePrimitive.CollapsibleTrigger

const CollapsibleContent = CollapsiblePrimitive.CollapsibleContent

export { Collapsible, CollapsibleTrigger, CollapsibleContent }

```

`client/src/components/ui/command.tsx`

```

import * as React from "react"
import { type DialogProps } from "@radix-ui/react-dialog"
import { Command as CommandPrimitive } from "cmdk"
import { Search } from "lucide-react"

import { cn } from "@lib/utlis"
import { Dialog, DialogContent } from "@components/ui/dialog"

const Command = React.forwardRef<
  React.ElementRef<typeof CommandPrimitive>,
  React.ComponentPropsWithoutRef<typeof CommandPrimitive>
>(({ className, ...props }, ref) => (
  <CommandPrimitive
    ref={ref}
    className={cn(
      "flex h-full w-full flex-col overflow-hidden rounded-md bg-popover text-popover-foreground",
      className
    )}
    {...props}
  />
))
Command.displayName = CommandPrimitive.displayName

const CommandDialog = ({ children, ...props }: DialogProps) => {
  return (
    <Dialog {...props}>
      <DialogContent className="overflow-hidden p-0 shadow-lg">
        <Command className={`${cn[cmdk-group-heading]]:px-2 [amp[cmdk-group-heading]]:font-medium [amp[cmdk-group-heading...
          {children}
        </Command>
      </DialogContent>
    </Dialog>
  )
}

const CommandInput = React.forwardRef<
  React.ElementRef<typeof CommandPrimitive.Input>,
  React.ComponentPropsWithoutRef<typeof CommandPrimitive.Input>
>(({ className, ...props }, ref) => (
  <div className="flex items-center border-b px-3 cmdk-input-wrapper="">
    <Search className="mr-2 h-4 w-4 shrink-0 opacity-50" />
    <CommandPrimitive.Input
      ref={ref}
      className={cn(
        "flex h-11 w-full rounded-md bg-transparent py-3 text-sm outline-none placeholder:text-muted-foreground disab...
      )}
      {...props}
    />
  </div>
))

CommandInput.displayName = CommandPrimitive.Input.displayName

const CommandList = React.forwardRef<
  React.ElementRef<typeof CommandPrimitive.List>,
  React.ComponentPropsWithoutRef<typeof CommandPrimitive.List>
>(({ className, ...props }, ref) => (
  <CommandPrimitive.List
    ref={ref}
    className={cn("max-h-[300px] overflow-y-auto overflow-x-hidden", className)}
    {...props}
  />
))

```



```

))

CommandList.displayName = CommandPrimitive.List.displayName

const CommandEmpty = React.forwardRef<
  React.ElementRef<typeof CommandPrimitive.Empty>,
  React.ComponentPropsWithoutRef<typeof CommandPrimitive.Empty>
>((props, ref) => (
  <CommandPrimitive.Empty
    ref={ref}
    className="py-6 text-center text-sm"
    {...props}
  />
))

CommandEmpty.displayName = CommandPrimitive.Empty.displayName

const CommandGroup = React.forwardRef<
  React.ElementRef<typeof CommandPrimitive.Group>,
  React.ComponentPropsWithoutRef<typeof CommandPrimitive.Group>
>(({ className, ...props }, ref) => (
  <CommandPrimitive.Group
    ref={ref}
    className={cn(
      "overflow-hidden p-1 text-foreground [&_cmdk-group-heading]:px-2 [&_cmdk-group-heading]:py-1.5 [&_cmdk-gro...
    className
  )}
  {...props}
  />
))

CommandGroup.displayName = CommandPrimitive.Group.displayName

const CommandSeparator = React.forwardRef<
  React.ElementRef<typeof CommandPrimitive.Separator>,
  React.ComponentPropsWithoutRef<typeof CommandPrimitive.Separator>
>(({ className, ...props }, ref) => (
  <CommandPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 h-px bg-border", className)}
    {...props}
  />
))

CommandSeparator.displayName = CommandPrimitive.Separator.displayName

const CommandItem = React.forwardRef<
  React.ElementRef<typeof CommandPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof CommandPrimitive.Item>
>(({ className, ...props }, ref) => (
  <CommandPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex cursor-default gap-2 select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none data-[...
    className
  )}
  {...props}
  />
))

CommandItem.displayName = CommandPrimitive.Item.displayName

const CommandShortcut = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLSpanElement>) => {
  return (
    <span
      className={cn(
        "ml-auto text-xs tracking-widest text-muted-foreground",
        className
      )}
      {...props}
    />
  )
}

CommandShortcut.displayName = "CommandShortcut"

export {
  Command,
  CommandDialog,
  CommandInput,
  CommandList,
  CommandEmpty,
  CommandGroup,
  CommandItem,
  CommandShortcut,
  CommandSeparator,
}

```

`client/src/components/ui/context-menu.tsx`

```

import * as React from "react"
import * as ContextMenuPrimitive from "@radix-ui/react-context-menu"
import { Check, ChevronRight, Circle } from "lucide-react"

import { cn } from "@/lib/utls"

const ContextMenu = ContextMenuPrimitive.Root

const ContextMenuTrigger = ContextMenuPrimitive.Trigger

const ContextMenuGroup = ContextMenuPrimitive.Group

const ContextMenuPortal = ContextMenuPrimitive.Portal

```

```

const ContextMenuSub = ContextMenuPrimitive.Sub

const ContextMenuRadioGroup = ContextMenuPrimitive.RadioGroup

const ContextMenuSubTrigger = React.forwardRef<
  React.ElementRef<typeof ContextMenuPrimitive.SubTrigger>,
  React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.SubTrigger> & {
    inset?: boolean
  }
>(({ className, inset, children, ...props }, ref) => (
  <ContextMenuPrimitive.SubTrigger
    ref={ref}
    className={cn(
      "flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-accent focus:...",
      inset && "pl-8",
      className
    )}
    {...props}
  >
    {children}
    <ChevronRight className="ml-auto h-4 w-4" />
  </ContextMenuPrimitive.SubTrigger>
))
ContextMenuSubTrigger.displayName = ContextMenuPrimitive.SubTrigger.displayName

const ContextMenuSubContent = React.forwardRef<
  React.ElementRef<typeof ContextMenuPrimitive.SubContent>,
  React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.SubContent>
>(({ className, ...props }, ref) => (
  <ContextMenuPrimitive.SubContent
    ref={ref}
    className={cn(
      "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-md data-[sta...",
      className
    )}
    {...props}
  />
))
ContextMenuSubContent.displayName = ContextMenuPrimitive.SubContent.displayName

const ContextMenuContent = React.forwardRef<
  React.ElementRef<typeof ContextMenuPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.Content>
>(({ className, ...props }, ref) => (
  <ContextMenuPrimitive.Portal>
    <ContextMenuPrimitive.Content
      ref={ref}
      className={cn(
        "z-50 max-h-[--radix-context-menu-content-available-height] min-w-[8rem] overflow-y-auto overflow-x-hidden ro...",
        className
      )}
      {...props}
    />
  </ContextMenuPrimitive.Portal>
))
ContextMenuContent.displayName = ContextMenuPrimitive.Content.displayName

const ContextMenuItem = React.forwardRef<
  React.ElementRef<typeof ContextMenuPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.Item> & {
    inset?: boolean
  }
>(({ className, inset, ...props }, ref) => (
  <ContextMenuPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-ac...",
      inset && "pl-8",
      className
    )}
    {...props}
  />
))
ContextMenuItem.displayName = ContextMenuPrimitive.Item.displayName

const ContextMenuCheckboxItem = React.forwardRef<
  React.ElementRef<typeof ContextMenuPrimitive.CheckboxItem>,
  React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.CheckboxItem>
>(({ className, children, checked, ...props }, ref) => (
  <ContextMenuPrimitive.CheckboxItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none focus:b...",
      className
    )}
    checked={checked}
    {...props}
  >
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <ContextMenuPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </ContextMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </ContextMenuPrimitive.CheckboxItem>
))
ContextMenuCheckboxItem.displayName =
  ContextMenuPrimitive.CheckboxItem.displayName

const ContextMenuRadioItem = React.forwardRef<
  React.ElementRef<typeof ContextMenuPrimitive.RadioItem>,
  React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.RadioItem>
>(({ className, children, ...props }, ref) => (
  <ContextMenuPrimitive.RadioItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none focus:b...",

```

```

        className
      })
      {...props}
    >
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <ContextMenuPrimitive.ItemIndicator>
        <Circle className="h-2 w-2 fill-current" />
      </ContextMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </ContextMenuPrimitive.RadioItem>
))
ContextMenuRadioItem.displayName = ContextMenuPrimitive.RadioItem.displayName

const ContextMenuLabel = React.forwardRef<
  React.ElementRef<typeof ContextMenuPrimitive.Label>,
  React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.Label> & {
    inset?: boolean
  }
>(({ className, inset, ...props }, ref) => (
  <ContextMenuPrimitive.Label
    ref={ref}
    className={cn(
      "px-2 py-1.5 text-sm font-semibold text-foreground",
      inset && "pl-8",
      className
    )}
    {...props}
  />
))
ContextMenuLabel.displayName = ContextMenuPrimitive.Label.displayName

const ContextMenuSeparator = React.forwardRef<
  React.ElementRef<typeof ContextMenuPrimitive.Separator>,
  React.ComponentPropsWithoutRef<typeof ContextMenuPrimitive.Separator>
>(({ className, ...props }, ref) => (
  <ContextMenuPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-border", className)}
    {...props}
  />
))
ContextMenuSeparator.displayName = ContextMenuPrimitive.Separator.displayName

const ContextMenuShortcut = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLSpanElement>) => {
  return (
    <span
      className={cn(
        "ml-auto text-xs tracking-widest text-muted-foreground",
        className
      )}
      {...props}
    />
  )
}
ContextMenuShortcut.displayName = "ContextMenuShortcut"

export {
  ContextMenu,
  ContextMenuTrigger,
  ContextMenuContent,
  ContextMenuItem,
  ContextMenuCheckboxItem,
  ContextMenuRadioItem,
  ContextMenuLabel,
  ContextMenuSeparator,
  ContextMenuShortcut,
  ContextMenuGroup,
  ContextMenuPortal,
  ContextMenuSub,
  ContextMenuSubContent,
  ContextMenuSubTrigger,
  ContextMenuRadioGroup,
}

```

`client/src/components/ui/dialog.tsx`

```

"use client"

import * as React from "react"
import * as DialogPrimitive from "@radix-ui/react-dialog"
import { X } from "lucide-react"

import { cn } from "@/lib/utls"

const Dialog = DialogPrimitive.Root

const DialogTrigger = DialogPrimitive.Trigger

const DialogPortal = DialogPrimitive.Portal

const DialogClose = DialogPrimitive.Close

const DialogOverlay = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Overlay>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Overlay>
>(({ className, ...props }, ref) => (
  <DialogPrimitive.Overlay
    ref={ref}
    className={cn(
      "fixed inset-0 z-[10001] bg-black/50 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=c...

```

```

        className
      })
    {...props}
  />
))
DialogOverlay.displayName = DialogPrimitive.Overlay.displayName

const DialogContent = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Content>
>(({ className, children, ...props }, ref) => {
  <DialogPortal>
    <DialogOverlay />
    <DialogPrimitive.Content
      ref={ref}
      className={cn(
        "fixed left-[50%] top-[50%] z-[10001] grid w-full max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4 border...
        className
      )}
      style={{
        background: 'var(--dialog-bg, linear-gradient(180deg, #3a8bbf 0%, color-mix(in srgb, #164a72 70%, black) 100%...)
        ...(props.style || {}),
      }}
      {...props}
    >
      {children}
      <DialogPrimitive.Close className="absolute right-4 top-4 rounded-sm opacity-70 ring-offset-background transitio...
        <X className="h-4 w-4" />
      </DialogPrimitive.Close>
    </DialogPrimitive.Content>
  </DialogPortal>
))
DialogContent.displayName = DialogPrimitive.Content.displayName

const DialogHeader = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLDivElement>) => {
  <div
    className={cn(
      "flex flex-col space-y-1.5 text-center sm:text-left",
      className
    )}
    {...props}
  />
)
DialogHeader.displayName = "DialogHeader"

const DialogFooter = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLDivElement>) => {
  <div
    className={cn(
      "flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2",
      className
    )}
    {...props}
  />
)
DialogFooter.displayName = "DialogFooter"

const DialogTitle = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Title>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Title>
>(({ className, ...props }, ref) => {
  <DialogPrimitive.Title
    ref={ref}
    className={cn(
      "text-lg font-semibold leading-none tracking-tight",
      className
    )}
    {...props}
  />
)
DialogTitle.displayName = DialogPrimitive.Title.displayName

const DialogDescription = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Description>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Description>
>(({ className, ...props }, ref) => {
  <DialogPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props}
  />
)
DialogDescription.displayName = DialogPrimitive.Description.displayName

export {
  Dialog,
  DialogPortal,
  DialogOverlay,
  DialogClose,
  DialogTrigger,
  DialogContent,
  DialogHeader,
  DialogFooter,
  DialogTitle,
  DialogDescription,
}
```

`client/src/components/ui/drawer.tsx`

```

"use client"

import * as React from "react"
import { Drawer as DrawerPrimitive } from "vaul"

import { cn } from "@lib/utils"

const Drawer = ({
  shouldScaleBackground = true,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Root>) => (
  <DrawerPrimitive.Root
    shouldScaleBackground={shouldScaleBackground}
    {...props}
  />
)
Drawer.displayName = "Drawer"

const DrawerTrigger = DrawerPrimitive.Trigger

const DrawerPortal = DrawerPrimitive.Portal

const DrawerClose = DrawerPrimitive.Close

const DrawerOverlay = React.forwardRef<
  React.ElementRef<typeof DrawerPrimitive.Overlay>,
  React.ComponentPropsWithoutRef<typeof DrawerPrimitive.Overlay>
>(({ className, ...props }, ref) => (
  <DrawerPrimitive.Overlay
    ref={ref}
    className={cn("fixed inset-0 z-50 bg-black/80", className)}
    {...props}
  />
))
DrawerOverlay.displayName = DrawerPrimitive.Overlay.displayName

const DrawerContent = React.forwardRef<
  React.ElementRef<typeof DrawerPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof DrawerPrimitive.Content>
>(({ className, children, ...props }, ref) => (
  <DrawerPortal>
    <DrawerOverlay />
    <DrawerPrimitive.Content
      ref={ref}
      className={cn(
        "fixed inset-x-0 bottom-0 z-50 mt-24 flex h-auto flex-col rounded-t-[10px] border bg-background",
        className
      )}
      {...props}
    >
    <div className="mx-auto mt-4 h-2 w-[100px] rounded-full bg-muted" />
    {children}
  </DrawerPrimitive.Content>
</DrawerPortal>
))
DrawerContent.displayName = "DrawerContent"

const DrawerHeader = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLDivElement>) => (
  <div
    className={cn("grid gap-1.5 p-4 text-center sm:text-left", className)}
    {...props}
  />
)
DrawerHeader.displayName = "DrawerHeader"

const DrawerFooter = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLDivElement>) => (
  <div
    className={cn("mt-auto flex flex-col gap-2 p-4", className)}
    {...props}
  />
)
DrawerFooter.displayName = "DrawerFooter"

const DrawerTitle = React.forwardRef<
  React.ElementRef<typeof DrawerPrimitive.Title>,
  React.ComponentPropsWithoutRef<typeof DrawerPrimitive.Title>
>(({ className, ...props }, ref) => (
  <DrawerPrimitive.Title
    ref={ref}
    className={cn(
      "text-lg font-semibold leading-none tracking-tight",
      className
    )}
    {...props}
  />
))
DrawerTitle.displayName = DrawerPrimitive.Title.displayName

const DrawerDescription = React.forwardRef<
  React.ElementRef<typeof DrawerPrimitive.Description>,
  React.ComponentPropsWithoutRef<typeof DrawerPrimitive.Description>
>(({ className, ...props }, ref) => (
  <DrawerPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props}
  />
))
DrawerDescription.displayName = DrawerPrimitive.Description.displayName

export {
  Drawer,

```

```

DrawerPortal,
DrawerOverlay,
DrawerTrigger,
DrawerClose,
DrawerContent,
DrawerHeader,
DrawerFooter,
DrawerTitle,
DrawerDescription,
}

```

`client/src/components/ui/dropdown-menu.tsx`

```

import * as React from "react"
import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu"
import { Check, ChevronRight, Circle } from "lucide-react"

import { cn } from "@/lib/utils"

const DropdownMenu = DropdownMenuPrimitive.Root

const DropdownMenuTrigger = DropdownMenuPrimitive.Trigger

const DropdownMenuGroup = DropdownMenuPrimitive.Group

const DropdownMenuPortal = DropdownMenuPrimitive.Portal

const DropdownMenuSub = DropdownMenuPrimitive.Sub

const DropdownMenuRadioGroup = DropdownMenuPrimitive.RadioGroup

const DropdownMenuSubTrigger = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.SubTrigger>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.SubTrigger> & {
    inset?: boolean
  }
>(({ className, inset, children, ...props }, ref) => (
  <DropdownMenuPrimitive.SubTrigger
    ref={ref}
    className={cn(
      "flex cursor-pointer select-none items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-accent...
      inset && "pl-8",
      className
    )}
    {...props}
  >
    {children}
    <ChevronRight className="ml-auto" />
  </DropdownMenuPrimitive.SubTrigger>
))
DropdownMenuSubTrigger.displayName =
  DropdownMenuPrimitive.SubTrigger.displayName

const DropdownMenuSubContent = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.SubContent>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.SubContent>
>(({ className, ...props }, ref) => (
  <DropdownMenuPrimitive.SubContent
    ref={ref}
    className={cn(
      "z-50 min-w-[8rem] overflow-hidden rounded-md border p-1 shadow-lg data-[state=open]:animate-in data-[state=clo...
      className
    )}
    style={{ backgroundColor: 'white', color: 'black' }}
    {...props}
  />
))
DropdownMenuSubContent.displayName =
  DropdownMenuPrimitive.SubContent.displayName

const DropdownMenuContent = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Content> & {
    variant?: "default" | "transparent"
    noAnimation?: boolean
  }
>(({ className, sideOffset = 4, style, variant = "default", noAnimation = false, ...props }, ref) => {
  const transparentStyles = variant === "transparent" ? {
    background: 'transparent',
    backgroundColor: 'transparent',
    border: 'none',
    boxShadow: 'none',
    color: 'white'
  } : {};

  const hasAnimNone = className?.includes('animate-none');
  const skipAnim = noAnimation || hasAnimNone;

  return (
    <DropdownMenuPrimitive.Portal>
      <DropdownMenuPrimitive.Content
        ref={ref}
        sideOffset={sideOffset}
        {...props}
        className={cn(
          "z-50 max-h-[var(--radix-dropdown-menu-content-available-height)] min-w-[8rem] overflow-y-auto overflow-x-h...
          !skipAnim && "data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 d...
          variant === "default" && "border bg-popover text-popover-foreground shadow-md",
          className
        )}
        style={{ ...style, ...transparentStyles }}
      />
    </DropdownMenuPrimitive.Portal>
  );

```

```

})
DropdownMenuContent.displayName = DropdownMenuPrimitive.Content.displayName

const DropdownMenuItem = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Item> & {
    inset?: boolean
    variant?: "default" | "transparent"
  }
>(({ className, inset, style, variant = "default", ...props }, ref) => {
  const transparentStyles = variant === "transparent" ? {
    color: 'white',
    backgroundColor: 'transparent'
  } : {};

  return (
    <DropdownMenuPrimitive.Item
      ref={ref}
      {...props}
      className={cn(
        "relative flex cursor-default select-none items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-none tran...
        variant === "default" && "focus:bg-accent hover:bg-accent data-[highlighted]:bg-accent data-[highlighted]:tex...
        inset && "pl-8",
        className
      )}
      style={{ ...style, ...transparentStyles }}
      data-variant={variant}
    />
  );
})
DropdownMenuItem.displayName = DropdownMenuPrimitive.Item.displayName

const DropdownMenuCheckboxItem = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.CheckboxItem>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.CheckboxItem>
>(({ className, children, checked, ...props }, ref) => (
  <DropdownMenuPrimitive.CheckboxItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none transit...
      className
    )}
    checked={checked}
    {...props}
  >
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </DropdownMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </DropdownMenuPrimitive.CheckboxItem>
))
DropdownMenuCheckboxItem.displayName =
  DropdownMenuPrimitive.CheckboxItem.displayName

const DropdownMenuRadioItem = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.RadioItem>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.RadioItem>
>(({ className, children, ..props }, ref) => (
  <DropdownMenuPrimitive.RadioItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none transit...
      className
    )}
    {...props}
  >
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
        <Circle className="h-2 w-2 fill-current" />
      </DropdownMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </DropdownMenuPrimitive.RadioItem>
))
DropdownMenuRadioItem.displayName = DropdownMenuPrimitive.RadioItem.displayName

const DropdownMenuLabel = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Label>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Label> & {
    inset?: boolean
  }
>(({ className, inset, ...props }, ref) => (
  <DropdownMenuPrimitive.Label
    ref={ref}
    className={cn(
      "px-2 py-1.5 text-sm font-semibold",
      inset && "pl-8",
      className
    )}
    {...props}
  />
))
DropdownMenuLabel.displayName = DropdownMenuPrimitive.Label.displayName

const DropdownMenuSeparator = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Separator>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Separator>
>(({ className, ...props }, ref) => (
  <DropdownMenuPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-muted", className)}
    {...props}
  />
))
DropdownMenuSeparator.displayName = DropdownMenuPrimitive.Separator.displayName

```

```

const DropdownMenuShortcut = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLSpanElement>) => {
  return (
    <span
      className={cn("ml-auto text-xs tracking-widest opacity-60", className)}
      {...props}
    />
  )
}
DropdownMenuShortcut.displayName = "DropdownMenuShortcut"

export {
  DropdownMenu,
  DropdownMenuTrigger,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuCheckboxItem,
  DropdownMenuRadioItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuShortcut,
  DropdownMenuGroup,
  DropdownMenuPortal,
  DropdownMenuSub,
  DropdownMenuSubContent,
  DropdownMenuSubTrigger,
  DropdownMenuRadioGroup,
}

```

`client/src/components/ui/form.tsx`

```

"use client"

import * as React from "react"
import * as LabelPrimitive from "@radix-ui/react-label"
import { Slot } from "@radix-ui/react-slot"
import {
  Controller,
  FormProvider,
  useFormContext,
  type ControllerProps,
  type FieldPath,
  type FieldValues,
} from "react-hook-form"

import { cn } from "@lib/utils"
import { Label } from "@components/ui/label"

const Form = FormProvider

type FormFieldContextValue<
  TFieldValues extends FieldValues = FieldValues,
  TName extends FieldPath<TFieldValues> = FieldPath<TFieldValues>
> = {
  name: TName
}

const FormFieldContext = React.createContext<FormFieldContextValue>(
  {} as FormFieldContextValue
)

const FormField = <
  TFieldValues extends FieldValues = FieldValues,
  TName extends FieldPath<TFieldValues> = FieldPath<TFieldValues>
>({
  ...props
}: ControllerProps<TFieldValues, TName>) => {
  return (
    <FormFieldContext.Provider value={{ name: props.name }}>
      <Controller {...props} />
    </FormFieldContext.Provider>
  )
}

const useFormField = () => {
  const fieldContext = React.useContext(FormFieldContext)
  const itemContext = React.useContext(FormItemContext)
  const { getFieldState, formState } = useFormContext()

  const fieldState = getFieldState(fieldContext.name, formState)

  if (!fieldContext) {
    throw new Error("useFormField should be used within <FormField>")
  }

  const { id } = itemContext

  return {
    id,
    name: fieldContext.name,
    formItemId: `${id}-form-item`,
    formDescriptionId: `${id}-form-item-description`,
    formMessageId: `${id}-form-item-message`,
    ...fieldState,
  }
}

type FormItemContextValue = {
  id: string
}

const FormItemContext = React.createContext<FormItemContextValue>(

```



```

    {} as FormItemContextValue
  )

const FormItem = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => {
  const id = React.useId()

  return (
    <FormItemContext.Provider value={{ id }}>
      <div ref={ref} className={cn("space-y-2", className)} {...props} />
    </FormItemContext.Provider>
  )
})
FormItem.displayName = "FormItem"

const FormLabel = React.forwardRef<
  React.ElementRef<typeof LabelPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof LabelPrimitive.Root>
>(({ className, ...props }, ref) => {
  const { error, formItemId } = useFormField()

  return (
    <Label
      ref={ref}
      className={cn(error && "text-destructive", className)}
      htmlFor={formItemId}
      {...props}
    />
  )
})
FormLabel.displayName = "FormLabel"

const FormControl = React.forwardRef<
  React.ElementRef<typeof Slot>,
  React.ComponentPropsWithoutRef<typeof Slot>
>(({ ...props }, ref) => {
  const { error, formItemId, formDescriptionId, formMessageId } = useFormField()

  return (
    <Slot
      ref={ref}
      id={formItemId}
      aria-describedby={
        !error
          ? `${formDescriptionId}`
          : `${formDescriptionId} ${formMessageId}`
      }
      aria-invalid={!error}
      {...props}
    />
  )
})
FormControl.displayName = "FormControl"

const FormDescription = React.forwardRef<
  HTMLParagraphElement,
  React.HTMLAttributes<HTMLParagraphElement>
>(({ className, ...props }, ref) => {
  const { formDescriptionId } = useFormField()

  return (
    <p
      ref={ref}
      id={formDescriptionId}
      className={cn("text-sm text-muted-foreground", className)}
      {...props}
    />
  )
})
FormDescription.displayName = "FormDescription"

const FormMessage = React.forwardRef<
  HTMLParagraphElement,
  React.HTMLAttributes<HTMLParagraphElement>
>(({ className, children, ...props }, ref) => {
  const { error, formMessageId } = useFormField()
  const body = error ? String(error?.message ?? "") : children

  if (!body) {
    return null
  }

  return (
    <p
      ref={ref}
      id={formMessageId}
      className={cn("text-sm font-medium text-destructive", className)}
      {...props}
    >
      {body}
    </p>
  )
})
FormMessage.displayName = "FormMessage"

export {
  useFormField,
  Form,
  FormItem,
  FormLabel,
  FormControl,
  FormDescription,
  FormMessage,
  FormField,
}
```

`client/src/components/ui/hover-card.tsx`

```
"use client"

import * as React from "react"
import * as HoverCardPrimitive from "@radix-ui/react-hover-card"

import { cn } from "@/lib/utils"

const HoverCard = HoverCardPrimitive.Root

const HoverCardTrigger = HoverCardPrimitive.Trigger

const HoverCardContent = React.forwardRef<
  React.ElementRef<typeof HoverCardPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof HoverCardPrimitive.Content>
>(({ className, align = "center", sideOffset = 4, ...props }, ref) => {
  <HoverCardPrimitive.Content
    ref={ref}
    align={align}
    sideOffset={sideOffset}
    className={cn(
      "z-50 w-64 rounded-md border bg-popover p-4 text-popover-foreground shadow-md outline-none data-[state=open]:an...
      className
    )}
    {...props}
  />
))
HoverCardContent.displayName = HoverCardPrimitive.Content.displayName

export { HoverCard, HoverCardTrigger, HoverCardContent }
```

`client/src/components/ui/input-otp.tsx`

```
import * as React from "react"
import { OTPInput, OTPInputContext } from "input-otp"
import { Dot } from "lucide-react"

import { cn } from "@/lib/utils"

const InputOTP = React.forwardRef<
  React.ElementRef<typeof OTPInput>,
  React.ComponentPropsWithoutRef<typeof OTPInput>
>(({ className, containerClassName, ...props }, ref) => {
  <OTPInput
    ref={ref}
    containerClassName={cn(
      "flex items-center gap-2 has-[:disabled]:opacity-50",
      containerClassName
    )}
    className={cn("disabled:cursor-not-allowed", className)}
    {...props}
  />
))
InputOTP.displayName = "InputOTP"

const InputOTPGroup = React.forwardRef<
  React.ElementRef<"div">,
  React.ComponentPropsWithoutRef<"div">
>(({ className, ...props }, ref) => {
  <div ref={ref} className={cn("flex items-center", className)} {...props} />
))
InputOTPGroup.displayName = "InputOTPGroup"

const InputOTPSlot = React.forwardRef<
  React.ElementRef<"div">,
  React.ComponentPropsWithoutRef<"div"> & { index: number }
>(({ index, className, ...props }, ref) => {
  const inputOTPContext = React.useContext(OTPInputContext)
  const { char, hasFakeCaret, isActive } = inputOTPContext.slots[index]

  return (
    <div
      ref={ref}
      className={cn(
        "relative flex h-10 w-10 items-center justify-center border-y border-r border-input text-sm transition-all fi...
        isActive && "z-10 ring-2 ring-ring ring-offset-background",
        className
      )}
      {...props}
    >
      {char}
      {hasFakeCaret && (
        <div className="pointer-events-none absolute inset-0 flex items-center justify-center">
          <div className="h-4 w-px animate-caret-blink bg-foreground duration-1000" />
        </div>
      )}
    </div>
  )
)}
InputOTPSlot.displayName = "InputOTPSlot"

const InputOTPSeparator = React.forwardRef<
  React.ElementRef<"div">,
  React.ComponentPropsWithoutRef<"div">
>(({ ...props }, ref) => {
  <div ref={ref} role="separator" {...props}>
    <Dot />
  </div>
)}
InputOTPSeparator.displayName = "InputOTPSeparator"
```

```
InputOTPSeparator.displayName = "InputOTPSeparator"

export { InputOTP, InputOTPGroup, InputOTPSlot, InputOTPSeparator }
```

`client/src/components/ui/input.tsx`

```
import * as React from "react"

import { cn } from "@lib/utils"

const Input = React.forwardRef<HTMLInputElement, React.ComponentProps<"input">>(>({
  className, type, ...props }, ref) => {
  // h-9 to match icon buttons and default buttons.
  return (
    <input
      type={type}
      className={cn(
        "flex h-9 w-full rounded-md border border-input bg-background px-3 py-2 text-base !font-normal ring-offset-...",
        className
      )}
      ref={ref}
      {...props}
    />
  )
})

Input.displayName = "Input"

export { Input }
```

`client/src/components/ui/label.tsx`

```
import * as React from "react"
import * as LabelPrimitive from "@radix-ui/react-label"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@lib/utils"

const labelVariants = cva(
  "text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70"
)

const Label = React.forwardRef<
  React.ElementRef<typeof LabelPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof LabelPrimitive.Root> &
    VariantProps<typeof labelVariants>
>(({ className, ...props }, ref) => {
  <LabelPrimitive.Root
    ref={ref}
    className={cn(labelVariants(), className)}
    {...props}
  />
))
Label.displayName = LabelPrimitive.Root.displayName

export { Label }
```

`client/src/components/ui/menubar.tsx`

```
"use client"

import * as React from "react"
import * as MenubarPrimitive from "@radix-ui/react-menubar"
import { Check, ChevronRight, Circle } from "lucide-react"

import { cn } from "@lib/utils"

function MenubarMenu({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Menu>) {
  return <MenubarPrimitive.Menu {...props} />
}

function MenubarGroup({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Group>) {
  return <MenubarPrimitive.Group {...props} />
}

function MenubarPortal({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Portal>) {
  return <MenubarPrimitive.Portal {...props} />
}

function MenubarRadioGroup({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.RadioGroup>) {
  return <MenubarPrimitive.RadioGroup {...props} />
}

function MenubarSub({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Sub>) {
  return <MenubarPrimitive.Sub data-slot="menubar-sub" {...props} />
}
```

```

}

const Menubar = React.forwardRef<
  React.ElementRef<typeof MenubarPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof MenubarPrimitive.Root>
>(({ className, ...props }, ref) => (
  <MenubarPrimitive.Root
    ref={ref}
    className={cn(
      "flex h-10 items-center space-x-1 rounded-md border bg-background p-1",
      className
    )}
    {...props}
  />
))
Menubar.displayName = MenubarPrimitive.Root.displayName

const MenubarTrigger = React.forwardRef<
  React.ElementRef<typeof MenubarPrimitive.Trigger>,
  React.ComponentPropsWithoutRef<typeof MenubarPrimitive.Trigger>
>(({ className, ...props }, ref) => (
  <MenubarPrimitive.Trigger
    ref={ref}
    className={cn(
      "flex cursor-default select-none items-center rounded-sm px-3 py-1.5 text-sm font-medium outline-none focus:bg-...",
      className
    )}
    {...props}
  />
))
MenubarTrigger.displayName = MenubarPrimitive.Trigger.displayName

const MenubarSubTrigger = React.forwardRef<
  React.ElementRef<typeof MenubarPrimitive.SubTrigger>,
  React.ComponentPropsWithoutRef<typeof MenubarPrimitive.SubTrigger> & {
    inset?: boolean
  }
>(({ className, inset, children, ...props }, ref) => (
  <MenubarPrimitive.SubTrigger
    ref={ref}
    className={cn(
      "flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-accent focus-...",
      inset && "pl-8",
      className
    )}
    {...props}
  >
    {children}
  <ChevronRight className="ml-auto h-4 w-4" />
  </MenubarPrimitive.SubTrigger>
))
MenubarSubTrigger.displayName = MenubarPrimitive.SubTrigger.displayName

const MenubarSubContent = React.forwardRef<
  React.ElementRef<typeof MenubarPrimitive.SubContent>,
  React.ComponentPropsWithoutRef<typeof MenubarPrimitive.SubContent>
>(({ className, ...props }, ref) => (
  <MenubarPrimitive.SubContent
    ref={ref}
    className={cn(
      "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground data-[state=open]:a...",
      className
    )}
    {...props}
  />
))
MenubarSubContent.displayName = MenubarPrimitive.SubContent.displayName

const MenubarContent = React.forwardRef<
  React.ElementRef<typeof MenubarPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof MenubarPrimitive.Content>
>((
  {
    className, align = "start", alignOffset = -4, sideOffset = 8, ...props
  },
  ref
) => (
  <MenubarPrimitive.Portal>
    <MenubarPrimitive.Content
      ref={ref}
      align={align}
      alignOffset={alignOffset}
      sideOffset={sideOffset}
      className={cn(
        "z-50 min-w-[12rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-md data-...",
        className
      )}
      {...props}
    />
  </MenubarPrimitive.Portal>
))
MenubarContent.displayName = MenubarPrimitive.Content.displayName

const MenubarItem = React.forwardRef<
  React.ElementRef<typeof MenubarPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof MenubarPrimitive.Item> & {
    inset?: boolean
  }
>(({ className, inset, ...props }, ref) => (
  <MenubarPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-none focus:bg-acc...",
      inset && "pl-8",
      className
    )}
    {...props}
  />
))

```

```

))
MenubarItem.displayName = MenubarPrimitive.Item.displayName

const MenubarCheckboxItem = React.forwardRef<
  React.ElementRef<typeof MenubarPrimitive.CheckboxItem>,
  React.ComponentPropsWithoutRef<typeof MenubarPrimitive.CheckboxItem>
>(({ className, children, checked, ...props }, ref) => (
  <MenubarPrimitive.CheckboxItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none focus:b...
      className
    )}
    checked={checked}
    {...props}
  >
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <MenubarPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </MenubarPrimitive.ItemIndicator>
    </span>
    {children}
  </MenubarPrimitive.CheckboxItem>
))
MenubarCheckboxItem.displayName = MenubarPrimitive.CheckboxItem.displayName

const MenubarRadioItem = React.forwardRef<
  React.ElementRef<typeof MenubarPrimitive.RadioItem>,
  React.ComponentPropsWithoutRef<typeof MenubarPrimitive.RadioItem>
>(({ className, children, ...props }, ref) => (
  <MenubarPrimitive.RadioItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none focus:b...
      className
    )}
    {...props}
  >
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <MenubarPrimitive.ItemIndicator>
        <Circle className="h-2 w-2 fill-current" />
      </MenubarPrimitive.ItemIndicator>
    </span>
    {children}
  </MenubarPrimitive.RadioItem>
))
MenubarRadioItem.displayName = MenubarPrimitive.RadioItem.displayName

const MenubarLabel = React.forwardRef<
  React.ElementRef<typeof MenubarPrimitive.Label>,
  React.ComponentPropsWithoutRef<typeof MenubarPrimitive.Label> & {
    inset?: boolean
  }
>(({ className, inset, ...props }, ref) => (
  <MenubarPrimitive.Label
    ref={ref}
    className={cn(
      "px-2 py-1.5 text-sm font-semibold",
      inset && "pl-8",
      className
    )}
    {...props}
  />
))
MenubarLabel.displayName = MenubarPrimitive.Label.displayName

const MenubarSeparator = React.forwardRef<
  React.ElementRef<typeof MenubarPrimitive.Separator>,
  React.ComponentPropsWithoutRef<typeof MenubarPrimitive.Separator>
>(({ className, ...props }, ref) => (
  <MenubarPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-muted", className)}
    {...props}
  />
))
MenubarSeparator.displayName = MenubarPrimitive.Separator.displayName

const MenubarShortcut = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLSpanElement>) => {
  return (
    <span
      className={cn(
        "ml-auto text-xs tracking-widest text-muted-foreground",
        className
      )}
      {...props}
    />
  )
}
MenubarShortcut.displayName = "MenubarShortcut"

export {
  Menubar,
  MenubarMenu,
  MenubarTrigger,
  MenubarContent,
  MenubarItem,
  MenubarSeparator,
  MenubarLabel,
  MenubarCheckboxItem,
  MenubarRadioGroup,
  MenubarRadioItem,
  MenubarPortal,
  MenubarSubContent,
  MenubarSubTrigger,

```

```
  MenubarGroup,  
  MenubarSub,  
  MenubarShortcut,  
}
```

`client/src/components/ui/navigation-menu.tsx`

```
import * as React from "react"  
import * as NavigationMenuPrimitive from "@radix-ui/react-navigation-menu"  
import { cva } from "class-variance-authority"  
import { ChevronDown } from "lucide-react"  
  
import { cn } from "@lib/utils"  
  
const NavigationMenu = React.forwardRef(  
  React.ElementRef<typeof NavigationMenuPrimitive.Root>,  
  React.ComponentPropsWithoutRef<typeof NavigationMenuPrimitive.Root>  
>(({ className, children, ...props }, ref) => (  
  <NavigationMenuPrimitive.Root  
    ref={ref}  
    className={cn(  
      "relative z-10 flex max-w-max flex-1 items-center justify-center",  
      className  
    )}  
    {...props}  
  >  
    {children}  
    <NavigationMenuViewport />  
  </NavigationMenuPrimitive.Root>  
)  
)  
NavigationMenu.displayName = NavigationMenuPrimitive.Root.displayName  
  
const NavigationMenuList = React.forwardRef(  
  React.ElementRef<typeof NavigationMenuPrimitive.List>,  
  React.ComponentPropsWithoutRef<typeof NavigationMenuPrimitive.List>  
>(({ className, ...props }, ref) => (  
  <NavigationMenuPrimitive.List  
    ref={ref}  
    className={cn(  
      "group flex flex-1 list-none items-center justify-center space-x-1",  
      className  
    )}  
    {...props}  
  />  
)  
)  
NavigationMenuList.displayName = NavigationMenuPrimitive.List.displayName  
  
const NavigationMenuItem = NavigationMenuPrimitive.Item  
  
const navigationMenuTriggerStyle = cva(  
  "group inline-flex h-10 w-max items-center justify-center rounded-md bg-background px-4 py-2 text-sm font-medium tr..."  
)  
  
const NavigationMenuTrigger = React.forwardRef(  
  React.ElementRef<typeof NavigationMenuPrimitive.Trigger>,  
  React.ComponentPropsWithoutRef<typeof NavigationMenuPrimitive.Trigger>  
>(({ className, children, ...props }, ref) => (  
  <NavigationMenuPrimitive.Trigger  
    ref={ref}  
    className={cn(navigationMenuTriggerStyle(), "group", className)}  
    {...props}  
  >  
    {children}{" "  
    <ChevronDown  
      className="relative top-[1px] ml-1 h-3 w-3 transition duration-200 group-data-[state=open]:rotate-180"  
      aria-hidden="true"  
    />  
  </NavigationMenuPrimitive.Trigger>  
)  
)  
NavigationMenuTrigger.displayName = NavigationMenuPrimitive.Trigger.displayName  
  
const NavigationMenuContent = React.forwardRef(  
  React.ElementRef<typeof NavigationMenuPrimitive.Content>,  
  React.ComponentPropsWithoutRef<typeof NavigationMenuPrimitive.Content>  
>(({ className, ...props }, ref) => (  
  <NavigationMenuPrimitive.Content  
    ref={ref}  
    className={cn(  
      "left-0 top-0 w-full data-[motion^=from-]:animate-in data-[motion^=to-]:animate-out data-[motion^=from-]:fade-i..."  
      className  
    )}  
    {...props}  
  />  
)  
)  
NavigationMenuContent.displayName = NavigationMenuPrimitive.Content.displayName  
  
const NavigationMenuLink = NavigationMenuPrimitive.Link  
  
const NavigationMenuViewport = React.forwardRef(  
  React.ElementRef<typeof NavigationMenuPrimitive.Viewport>,  
  React.ComponentPropsWithoutRef<typeof NavigationMenuPrimitive.Viewport>  
>(({ className, ...props }, ref) => (  
  <div className={cn("absolute left-0 top-full flex justify-center")}>  
    <NavigationMenuPrimitive.Viewport  
      className={cn(  
        "origin-top-center relative mt-1.5 h-[var(--radix-navigation-menu-viewport-height)] w-full overflow-hidden ro..."  
        className  
      )}  
      ref={ref}  
      {...props}  
    />  
  </div>  
)  
)  
NavigationMenuViewport.displayName =
```

```

    NavigationMenuPrimitive.Viewport.displayName

const NavigationMenuIndicator = React.forwardRef<
  React.ElementRef<typeof NavigationMenuPrimitive.Indicator>,
  React.ComponentPropsWithoutRef<typeof NavigationMenuPrimitive.Indicator>
>(({ className, ...props }, ref) => (
  <NavigationMenuPrimitive.Indicator
    ref={ref}
    className={cn(
      "top-full z-[1] flex h-1.5 items-end justify-center overflow-hidden data-[state=visible]:animate-in data-[state...
      className
    )}
    {...props}
  >
    <div className="relative top-[60%] h-2 w-2 rotate-45 rounded-tl-sm bg-border shadow-md" />
  </NavigationMenuPrimitive.Indicator>
))
NavigationMenuIndicator.displayName =
  NavigationMenuPrimitive.Indicator.displayName

export {
  navigationMenuTriggerStyle,
  NavigationMenu,
  NavigationMenuList,
  NavigationMenuItem,
  NavigationMenuContent,
  NavigationMenuTrigger,
  NavigationMenuLink,
  NavigationMenuIndicator,
  NavigationMenuViewport,
}

```

`client/src/components/ui/pagination.tsx`

```

import * as React from "react"
import { ChevronLeft, ChevronRight, MoreHorizontal } from "lucide-react"

import { cn } from "@lib/utills"
import { ButtonProps, buttonVariants } from "@components/ui/button"

const Pagination = ({ className, ...props }: React.ComponentProps<"nav">) => (
  <nav
    role="navigation"
    aria-label="pagination"
    className={cn("mx-auto flex w-full justify-center", className)}
    {...props}
  />
)
Pagination.displayName = "Pagination"

const PaginationContent = React.forwardRef<
  HTMLUListElement,
  React.ComponentProps<"ul">
>(({ className, ...props }, ref) => (
  <ul
    ref={ref}
    className={cn("flex flex-row items-center gap-1", className)}
    {...props}
  />
))
PaginationContent.displayName = "PaginationContent"

const PaginationItem = React.forwardRef<
  HTMLLIElement,
  React.ComponentProps<"li">
>(({ className, ...props }, ref) => (
  <li ref={ref} className={cn("", className)} {...props} />
))
PaginationItem.displayName = "PaginationItem"

type PaginationLinkProps = {
  isActive?: boolean
} & Pick<ButtonProps, "size"> &
  React.ComponentProps<"a">

const PaginationLink = ({
  className,
  isActive,
  size = "icon",
  ...props
}: PaginationLinkProps) => (
  <a
    aria-current={isActive ? "page" : undefined}
    className={cn(
      buttonVariants({
        variant: isActive ? "outline" : "ghost",
        size,
      }),
      className
    )}
    {...props}
  />
)
PaginationLink.displayName = "PaginationLink"

const PaginationPrevious = ({
  className,
  ...props
}: React.ComponentProps<typeof PaginationLink>) => (
  <PaginationLink
    aria-label="Go to previous page"
    size="default"
    className={cn("gap-1 pl-2.5", className)}
    {...props}
  />
)

```

```

    >
    <ChevronLeft className="h-4 w-4" />
    <span>Previous</span>
  </PaginationLink>
)
PaginationPrevious.displayName = "PaginationPrevious"

const PaginationNext = ({
  className,
  ...props
}): React.ComponentProps<typeof PaginationLink> => (
  <PaginationLink
    aria-label="Go to next page"
    size="default"
    className={cn("gap-1 pr-2.5", className)}
    {...props}
  >
    <span>Next</span>
    <ChevronRight className="h-4 w-4" />
  </PaginationLink>
)
PaginationNext.displayName = "PaginationNext"

const PaginationEllipsis = ({
  className,
  ...props
}): React.ComponentProps<"span"> => (
  <span
    aria-hidden
    className={cn("flex h-9 w-9 items-center justify-center", className)}
    {...props}
  >
    <MoreHorizontal className="h-4 w-4" />
    <span className="sr-only">More pages</span>
  </span>
)
PaginationEllipsis.displayName = "PaginationEllipsis"

export {
  Pagination,
  PaginationContent,
  PaginationEllipsis,
  paginationItem,
  PaginationLink,
  PaginationNext,
  PaginationPrevious,
}

```

`client/src/components/ui/popover.tsx`

```

import * as React from "react"
import * as PopoverPrimitive from "@radix-ui/react-popover"

import { cn } from "@lib/utlis"

const Popover = PopoverPrimitive.Root

const PopoverTrigger = PopoverPrimitive.Trigger

const PopoverContent = React.forwardRef<
  React.ElementRef<typeof PopoverPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof PopoverPrimitive.Content>
>(({ className, align = "center", sideOffset = 4, ...props }, ref) => (
  <PopoverPrimitive.Portal>
    <PopoverPrimitive.Content
      ref={ref}
      align={align}
      sideOffset={sideOffset}
      className={cn(
        "z-50 w-72 rounded-md border bg-popover p-4 text-popover-foreground shadow-md outline-none data-[state=open]:...
        className
      )}
      {...props}
    >
  </PopoverPrimitive.Portal>
))
PopoverContent.displayName = PopoverPrimitive.Content.displayName

export { Popover, PopoverTrigger, PopoverContent }

```

`client/src/components/ui/progress.tsx`

```

"use client"

import * as React from "react"
import * as ProgressPrimitive from "@radix-ui/react-progress"

import { cn } from "@lib/utlis"

const Progress = React.forwardRef<
  React.ElementRef<typeof ProgressPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof ProgressPrimitive.Root>
>(({ className, value, ...props }, ref) => (
  <ProgressPrimitive.Root
    ref={ref}
    className={cn(
      "relative h-4 w-full overflow-hidden rounded-full bg-secondary",
      className
    )}
    {...props}
  >

```



```

>
  <ProgressPrimitive.Indicator
    className="h-full w-full flex-1 bg-primary transition-all"
    style={{ transform: `translateX(-${100 - (value || 0)}%)` }}
  />
</ProgressPrimitive.Root>
))
Progress.displayName = ProgressPrimitive.Root.displayName

export { Progress }

```

`client/src/components/ui/radio-group.tsx`

```

import * as React from "react"
import * as RadioGroupPrimitive from "@radix-ui/react-radio-group"
import { Circle } from "lucide-react"

import { cn } from "@lib/utils"

const RadioGroup = React.forwardRef<
  React.ElementRef<typeof RadioGroupPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof RadioGroupPrimitive.Root>
>(({ className, ...props }, ref) => {
  return (
    <RadioGroupPrimitive.Root
      className={cn("grid gap-2", className)}
      {...props}
      ref={ref}
    />
  )
})
RadioGroup.displayName = RadioGroupPrimitive.Root.displayName

const RadioGroupItem = React.forwardRef<
  React.ElementRef<typeof RadioGroupPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof RadioGroupPrimitive.Item>
>(({ className, ...props }, ref) => {
  return (
    <RadioGroupPrimitive.Item
      ref={ref}
      className={cn(
        "aspect-square h-4 w-4 rounded-full border border-primary text-primary ring-offset-background focus:outline-n...
        className
      )}
      {...props}
    >
    <RadioGroupPrimitive.Indicator className="flex items-center justify-center">
      <Circle className="h-2.5 w-2.5 fill-current text-current" />
    </RadioGroupPrimitive.Indicator>
    </RadioGroupPrimitive.Item>
  )
})
RadioGroupItem.displayName = RadioGroupPrimitive.Item.displayName

export { RadioGroup, RadioGroupItem }

```

`client/src/components/ui/resizable.tsx`

```

"use client"

import { GripVertical } from "lucide-react"
import * as ResizablePrimitive from "react-resizable-panels"

import { cn } from "@lib/utils"

const ResizablePanelGroup = ({
  className,
  ...props
}: React.ComponentProps<typeof ResizablePrimitive.PanelGroup>) => (
  <ResizablePrimitive.PanelGroup
    className={cn(
      "flex h-full w-full data-[panel-group-direction=vertical]:flex-col",
      className
    )}
    {...props}
  />
)

const ResizablePanel = ResizablePrimitive.Panel

const ResizableHandle = ({
  withHandle,
  className,
  ...props
}: React.ComponentProps<typeof ResizablePrimitive.PanelResizeHandle> & {
  withHandle?: boolean
}) => (
  <ResizablePrimitive.PanelResizeHandle
    className={cn(
      "relative flex w-px items-center justify-center bg-border after:absolute after:inset-y-0 after:left-1/2 after:w...
      className
    )}
    {...props}
  >
    {withHandle && (
      <div className="z-10 flex h-4 w-3 items-center justify-center rounded-sm border bg-border">
        <GripVertical className="h-2.5 w-2.5" />
      </div>
    )}
  </ResizablePrimitive.PanelResizeHandle>
)

```

```
)  
  
export { ResizablePanelGroup, ResizablePanel, ResizableHandle }
```

`client/src/components/ui/scroll-area.tsx`

```
import * as React from "react"  
import * as ScrollAreaPrimitive from "@radix-ui/react-scroll-area"  
  
import { cn } from "@lib/utils"  
  
const ScrollArea = React.forwardRef(  
  React.ElementRef<typeof ScrollAreaPrimitive.Root>,  
  React.ComponentPropsWithoutRef<typeof ScrollAreaPrimitive.Root>  
>(({ className, children, ...props }, ref) => {  
  <ScrollAreaPrimitive.Root  
    ref={ref}  
    className={cn("relative overflow-hidden", className)}  
    {...props}  
  >  
    <ScrollAreaPrimitive.Viewport className="h-full w-full rounded-[inherit]">  
      {children}  
    </ScrollAreaPrimitive.Viewport>  
    <ScrollBar />  
    <ScrollAreaPrimitive.Corner />  
  </ScrollAreaPrimitive.Root>  
)})  
ScrollArea.displayName = ScrollAreaPrimitive.Root.displayName  
  
const ScrollBar = React.forwardRef(  
  React.ElementRef<typeof ScrollAreaPrimitive.ScrollAreaScrollbar>,  
  React.ComponentPropsWithoutRef<typeof ScrollAreaPrimitive.ScrollAreaScrollbar>  
>(({ className, orientation = "vertical", ...props }, ref) => {  
  <ScrollAreaPrimitive.ScrollAreaScrollbar  
    ref={ref}  
    orientation={orientation}  
    className={cn(  
      "flex touch-none select-none transition-colors",  
      orientation === "vertical" &&  
        "h-full w-2.5 border-l border-l-transparent p-[1px]",  
      orientation === "horizontal" &&  
        "h-2.5 flex-col border-t border-t-transparent p-[1px]",  
      className  
    )}  
    {...props}  
  >  
    <ScrollAreaPrimitive.ScrollAreaThumb className="relative flex-1 rounded-full bg-border" />  
  </ScrollAreaPrimitive.ScrollAreaScrollbar>  
)})  
ScrollBar.displayName = ScrollAreaPrimitive.ScrollAreaScrollbar.displayName  
  
export { ScrollArea, ScrollBar }
```

`client/src/components/ui/select.tsx`

```
"use client"  
  
import * as React from "react"  
import * as SelectPrimitive from "@radix-ui/react-select"  
import { Check, ChevronDown, ChevronUp } from "lucide-react"  
  
import { cn } from "@lib/utils"  
  
const Select = SelectPrimitive.Root  
  
const SelectGroup = SelectPrimitive.Group  
  
const SelectValue = SelectPrimitive.Value  
  
const SelectTrigger = React.forwardRef(  
  React.ElementRef<typeof SelectPrimitive.Trigger>,  
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Trigger>  
>(({ className, children, ...props }, ref) => {  
  <SelectPrimitive.Trigger  
    ref={ref}  
    className={cn(  
      "flex h-9 w-full items-center justify-between rounded-md border border-input bg-background px-3 py-2 text-sm !f...  
      className  
    )}  
    {...props}  
  >  
    {children}  
    <SelectPrimitive.Icon asChild>  
      <ChevronDown className="h-4 w-4 opacity-50" />  
    </SelectPrimitive.Icon>  
  </SelectPrimitive.Trigger>  
)})  
SelectTrigger.displayName = SelectPrimitive.Trigger.displayName  
  
const SelectScrollUpButton = React.forwardRef(  
  React.ElementRef<typeof SelectPrimitive.ScrollUpButton>,  
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.ScrollUpButton>  
>(({ className, ...props }, ref) => {  
  <SelectPrimitive.ScrollUpButton  
    ref={ref}  
    className={cn(  
      "flex cursor-default items-center justify-center py-1",  
      className  
    )}  
    {...props}  
  >  
    </SelectPrimitive.ScrollUpButton>  
  </SelectPrimitive.ScrollUpButton>  
)})  
SelectScrollUpButton.displayName = SelectPrimitive.ScrollUpButton.displayName
```

```

    <ChevronUp className="h-4 w-4" />
  </SelectPrimitive.ScrollUpButton>
))
SelectScrollUpButton.displayName = SelectPrimitive.ScrollUpButton.displayName

const SelectScrollDownButton = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.ScrollDownButton>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.ScrollDownButton>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.ScrollDownButton
    ref={ref}
    className={cn(
      "flex cursor-default items-center justify-center py-1",
      className
    )}
    {...props}
  >
    <ChevronDown className="h-4 w-4" />
  </SelectPrimitive.ScrollDownButton>
))
SelectScrollDownButton.displayName =
  SelectPrimitive.ScrollDownButton.displayName

const SelectContent = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Content>
>(({ className, children, position = "popper", ...props }, ref) => (
  <SelectPrimitive.Portal>
    <SelectPrimitive.Content
      ref={ref}
      className={cn(
        "relative z-[10002] max-h-[--radix-select-content-available-height] min-w-[8rem] overflow-y-auto overflow-x-h...
        position === "popper" &&
        "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1 data-[side=right]:translate-x-1 data-[sid...
        className
      )}
      style={{ backgroundColor: 'white', color: 'black' }}
      position={position}
      {...props}
    >
      <SelectScrollUpButton />
      <SelectPrimitive.Viewport
        className={cn(
          "p-1",
          position === "popper" &&
          "h-[var(--radix-select-trigger-height)] w-full min-w-[var(--radix-select-trigger-width)]"
        )}
      >
        {children}
      </SelectPrimitive.Viewport>
      <SelectScrollDownButton />
    </SelectPrimitive.Content>
  </SelectPrimitive.Portal>
))
SelectContent.displayName = SelectPrimitive.Content.displayName

const SelectLabel = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Label>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Label>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.Label
    ref={ref}
    className={cn("py-1.5 pl-8 pr-2 text-sm !font-normal", className)}
    {...props}
  />
))
SelectLabel.displayName = SelectPrimitive.Label.displayName

const SelectItem = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Item>
>(({ className, children, style, ...props }, ref) => (
  <SelectPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex w-full cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm !font-normal ...
      !style?.backgroundColor && "focus:bg-accent",
      className
    )}
    style={{ color: style?.color || 'black', ...style }}
    {...props}
  >
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center" style={{ color: style?.color || 'b...
      <SelectPrimitive.ItemIndicator>
      <Check className="h-4 w-4" />
    </SelectPrimitive.ItemIndicator>
  </span>

  <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>
</SelectPrimitive.Item>
))
SelectItem.displayName = SelectPrimitive.Item.displayName

const SelectSeparator = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Separator>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Separator>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-muted", className)}
    {...props}
  />
))
SelectSeparator.displayName = SelectPrimitive.Separator.displayName

export {
  Select,

```

```
SelectGroup,  
SelectValue,  
SelectTrigger,  
SelectContent,  
SelectLabel,  
SelectItem,  
SelectSeparator,  
SelectScrollUpButton,  
SelectScrollDownButton,  
}
```

`client/src/components/ui/separator.tsx`

```
import * as React from "react"  
import * as SeparatorPrimitive from "@radix-ui/react-separator"  
  
import { cn } from "@lib/utils"  
  
const Separator = React.forwardRef(  
  React.ElementRef<typeof SeparatorPrimitive.Root>,  
  React.ComponentPropsWithoutRef<typeof SeparatorPrimitive.Root>  
>(  
  (  
    { className, orientation = "horizontal", decorative = true, ...props },  
    ref  
  ) => (  
    <SeparatorPrimitive.Root  
      ref={ref}  
      decorative={decorative}  
      orientation={orientation}  
      className={cn(  
        "shrink-0 bg-border",  
        orientation === "horizontal" ? "h-[1px] w-full" : "h-full w-[1px]",  
        className  
      )}  
      {...props}  
    />  
  )  
)  
Separator.displayName = SeparatorPrimitive.Root.displayName  
  
export { Separator }
```

`client/src/components/ui/sheet.tsx`

```
"use client"  
  
import * as React from "react"  
import * as SheetPrimitive from "@radix-ui/react-dialog"  
import { cva, type VariantProps } from "class-variance-authority"  
import { X } from "lucide-react"  
  
import { cn } from "@lib/utils"  
  
const Sheet = SheetPrimitive.Root  
  
const SheetTrigger = SheetPrimitive.Trigger  
  
const SheetClose = SheetPrimitive.Close  
  
const SheetPortal = SheetPrimitive.Portal  
  
const SheetOverlay = React.forwardRef(  
  React.ElementRef<typeof SheetPrimitive.Overlay>,  
  React.ComponentPropsWithoutRef<typeof SheetPrimitive.Overlay>  
>(({ className, ...props }, ref) => (  
  <SheetPrimitive.Overlay  
    className={cn(  
      "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",  
      className  
    )}  
    {...props}  
    ref={ref}  
  />  
)  
)  
SheetOverlay.displayName = SheetPrimitive.Overlay.displayName  
  
const sheetVariants = cva(  
  "fixed z-50 gap-4 bg-background p-6 shadow-lg transition ease-in-out data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",  
  {  
    variants: {  
      side: {  
        top: "inset-x-0 top-0 border-b data-[state=closed]:slide-out-to-top data-[state=open]:slide-in-from-top",  
        bottom: "inset-x-0 bottom-0 border-t data-[state=closed]:slide-out-to-bottom data-[state=open]:slide-in-from-bottom",  
        left: "inset-y-0 left-0 h-full w-3/4 border-r data-[state=closed]:slide-out-to-left data-[state=open]:slide-in-from-left data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",  
        right: "inset-y-0 right-0 h-full w-3/4 border-l data-[state=closed]:slide-out-to-right data-[state=open]:slide-in-from-right data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",  
      },  
    },  
    defaultVariants: {  
      side: "right",  
    },  
  }  
)  
  
interface SheetContentProps  
  extends React.ComponentPropsWithoutRef<typeof SheetPrimitive.Content>,  
    VariantProps<typeof sheetVariants> {}
```

```

const SheetContent = React.forwardRef<
  React.ElementRef<typeof SheetPrimitive.Content>,
  SheetContentProps
>(({ side = "right", className, children, ...props }, ref) => (
  <SheetPortal>
    <SheetOverlay />
    <SheetPrimitive.Content
      ref={ref}
      className={cn(sheetVariants({ side }), className)}
      {...props}
    >
      {children}
    <SheetPrimitive.Close className="absolute right-4 top-4 rounded-sm opacity-70 ring-offset-background transition...
      <X className="h-4 w-4" />
    </SheetPrimitive.Close>
  </SheetPrimitive.Content>
</SheetPortal>
))
SheetContent.displayName = SheetPrimitive.Content.displayName

const SheetHeader = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLDivElement>) => (
  <div
    className={cn(
      "flex flex-col space-y-2 text-center sm:text-left",
      className
    )}
    {...props}
  />
)
SheetHeader.displayName = "SheetHeader"

const SheetFooter = ({
  className,
  ...props
}: React.HTMLAttributes<HTMLDivElement>) => (
  <div
    className={cn(
      "flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2",
      className
    )}
    {...props}
  />
)
SheetFooter.displayName = "SheetFooter"

const SheetTitle = React.forwardRef<
  React.ElementRef<typeof SheetPrimitive.Title>,
  React.ComponentPropsWithoutRef<typeof SheetPrimitive.Title>
>(({ className, ...props }, ref) => (
  <SheetPrimitive.Title
    ref={ref}
    className={cn("text-lg font-semibold text-foreground", className)}
    {...props}
  />
))
SheetTitle.displayName = SheetPrimitive.Title.displayName

const SheetDescription = React.forwardRef<
  React.ElementRef<typeof SheetPrimitive.Description>,
  React.ComponentPropsWithoutRef<typeof SheetPrimitive.Description>
>(({ className, ...props }, ref) => (
  <SheetPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props}
  />
))
SheetDescription.displayName = SheetPrimitive.Description.displayName

export {
  Sheet,
  SheetPortal,
  SheetOverlay,
  SheetTrigger,
  SheetClose,
  SheetContent,
  SheetHeader,
  SheetFooter,
  SheetTitle,
  SheetDescription,
}

```

`client/src/components/ui/sidebar.tsx`

```

"use client"

import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { cva, VariantProps } from "class-variance-authority"
import { PanelLeftIcon } from "lucide-react"

import { useIsMobile } from "@hooks/use-mobile"
import { cn } from "@lib/utils"
import { Button } from "@components/ui/button"
import { Input } from "@components/ui/input"
import { Separator } from "@components/ui/separator"
import {
  Sheet,
  SheetContent,
  SheetDescription,
  SheetHeader,

```

```

    SheetTitle,
  } from "@components/ui/sheet"
import { Skeleton } from "@components/ui/skeleton"
import {
  Tooltip,
  TooltipContent,
  TooltipProvider,
  TooltipTrigger,
} from "@components/ui/tooltip"

const SIDEBAR_COOKIE_NAME = "sidebar_state"
const SIDEBAR_COOKIE_MAX_AGE = 60 * 60 * 24 * 7
const SIDEBAR_WIDTH = "16rem"
const SIDEBAR_WIDTH_MOBILE = "18rem"
const SIDEBAR_WIDTH_ICON = "3rem"
const SIDEBAR_KEYBOARD_SHORTCUT = "b"

type SidebarContextProps = {
  state: "expanded" | "collapsed"
  open: boolean
  setOpen: (open: boolean) => void
  openMobile: boolean
  setOpenMobile: (open: boolean) => void
  isMobile: boolean
  toggleSidebar: () => void
}

const SidebarContext = React.createContext<SidebarContextProps | null>(null)

function useSidebar() {
  const context = React.useContext(SidebarContext)
  if (!context) {
    throw new Error("useSidebar must be used within a SidebarProvider.")
  }

  return context
}

function SidebarProvider({
  defaultOpen = true,
  open: openProp,
  onOpenChange: setOpenProp,
  className,
  style,
  children,
  ...props
}: React.ComponentProps<"div"> & {
  defaultOpen?: boolean
  open?: boolean
  onOpenChange?: (open: boolean) => void
}) {
  const isMobile = useIsMobile()
  const [openMobile, setOpenMobile] = React.useState(false)

  // This is the internal state of the sidebar.
  // We use openProp and setOpenProp for control from outside the component.
  const [_open, _setOpen] = React.useState(defaultOpen)
  const open = openProp ?? _open
  const setOpen = React.useCallback(
    (value: boolean | ((value: boolean) => boolean)) => {
      const openState = typeof value === "function" ? value(open) : value
      if (setOpenProp) {
        setOpenProp(openState)
      } else {
        _setOpen(openState)
      }
    },
    [setOpenProp, open]
  )

  // This sets the cookie to keep the sidebar state.
  document.cookie = `${SIDEBAR_COOKIE_NAME}=${openState}; path=/; max-age=${SIDEBAR_COOKIE_MAX_AGE}`

  // Helper to toggle the sidebar.
  const toggleSidebar = React.useCallback(() => {
    return isMobile ? setOpenMobile((open) => !open) : setOpen((open) => !open)
  }, [isMobile, setOpen, setOpenMobile])

  // Adds a keyboard shortcut to toggle the sidebar.
  React.useEffect(() => {
    const handleKeyDown = (event: KeyboardEvent) => {
      if (
        event.key === SIDEBAR_KEYBOARD_SHORTCUT &&
        (event.metaKey || event.ctrlKey)
      ) {
        event.preventDefault()
        toggleSidebar()
      }
    }

    window.addEventListener("keydown", handleKeyDown)
    return () => window.removeEventListener("keydown", handleKeyDown)
  }, [toggleSidebar])

  // We add a state so that we can do data-state="expanded" or "collapsed".
  // This makes it easier to style the sidebar with Tailwind classes.
  const state = open ? "expanded" : "collapsed"

  const contextValue = React.useMemo<SidebarContextProps>(
    () => ({
      state,
      open,
      setOpen,
      isMobile,
      openMobile,
      setOpenMobile,
      toggleSidebar,
    }),
  ),

```

```

    [state, open, setOpen, isMobile, openMobile, setOpenMobile, toggleSidebar]
  )
}

return (
  <SidebarContext.Provider value={contextValue}>
    <TooltipProvider delayDuration={0}>
      <div
        data-slot="sidebar-wrapper"
        style={
          {
            "--sidebar-width": SIDEBAR_WIDTH,
            "--sidebar-width-icon": SIDEBAR_WIDTH_ICON,
            ...style,
          } as React.CSSProperties
        }
        className={cn(
          "group/sidebar-wrapper has-data-[variant=inset]:bg-sidebar flex min-h-svh w-full",
          className
        )}
        {...props}
      >
        {children}
      </div>
    </TooltipProvider>
  </SidebarContext.Provider>
)
}

function Sidebar({
  side = "left",
  variant = "sidebar",
  collapsible = "offcanvas",
  className,
  children,
  ...props
}: React.ComponentProps<"div"> & {
  side?: "left" | "right"
  variant?: "sidebar" | "floating" | "inset"
  collapsible?: "offcanvas" | "icon" | "none"
}) {
  const { isMobile, state, openMobile, setOpenMobile } = useSidebar()

  if (collapsible === "none") {
    return (
      <div
        data-slot="sidebar"
        className={cn(
          "bg-sidebar text-sidebar-foreground flex h-full w-[var(--sidebar-width)] flex-col",
          className
        )}
        {...props}
      >
        {children}
      </div>
    )
  }

  if (isMobile) {
    return (
      <Sheet open={openMobile} onOpenChange={setOpenMobile} {...props}>
        <SheetContent
          data-sidebar="sidebar"
          data-slot="sidebar"
          data-mobile="true"
          className="bg-sidebar text-sidebar-foreground w-[var(--sidebar-width)] p-0 [&button]:hidden"
          style={
            {
              "--sidebar-width": SIDEBAR_WIDTH_MOBILE,
            } as React.CSSProperties
          }
          side={side}
        >
          <SheetHeader className="sr-only">
            <SheetTitle>Sidebar</SheetTitle>
            <SheetDescription>Displays the mobile sidebar.</SheetDescription>
          </SheetHeader>
          <div className="flex h-full w-full flex-col">{children}</div>
        </SheetContent>
      </Sheet>
    )
  }

  return (
    <div
      className="group peer text-sidebar-foreground hidden md:block"
      data-state={state}
      data-collapsible={state === "collapsed" ? collapsible : ""}
      data-variant={variant}
      data-side={side}
      data-slot="sidebar"
    >
      { /* This is what handles the sidebar gap on desktop */ }
      <div
        data-slot="sidebar-gap"
        className={cn(
          "relative w-[var(--sidebar-width)] bg-transparent transition-[width] duration-200 ease-linear",
          "group-data-[collapsible=offcanvas]:w-0",
          "group-data-[side=right]:rotate-180",
          variant === "floating" || variant === "inset"
            ? "group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)+var(--spacing-4))]"
            : "group-data-[collapsible=icon]:w-[var(--sidebar-width-icon)]"
        )}
      >
        <div
          data-slot="sidebar-container"
          className={cn(
            "fixed inset-y-0 z-10 hidden h-svh w-[var(--sidebar-width)] transition-[left,right,width] duration-200 ease..."
            side === "left"
          )}
        >

```

```

        ? "left-0 group-data-[collapsible=offcanvas]:left-[calc(var(--sidebar-width)*-1)]"
        : "right-0 group-data-[collapsible=offcanvas]:right-[calc(var(--sidebar-width)*-1)]",
    // Adjust the padding for floating and inset variants.
    variant === "floating" || variant === "inset"
      ? "p-2 group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)+var(--spacing-4)+2px)]"
      : "group-data-[collapsible=icon]:w-[var(--sidebar-width-icon)] group-data-[side=left]:border-r group-data-...
    className
  )}
  {...props}
>
  <div
    data-sidebar="sidebar"
    data-slot="sidebar-inner"
    className="bg-sidebar group-data-[variant=floating]:border-sidebar-border flex h-full w-full flex-col group-...
  >
    {children}
  </div>
</div>
</div>
}

function SidebarTrigger({
  className,
  onClick,
  ...props
}: React.ComponentProps<typeof Button>) {
  const { toggleSidebar } = useSidebar()

  return (
    <Button
      data-sidebar="trigger"
      data-slot="sidebar-trigger"
      variant="ghost"
      size="icon"
      className={cn("h-7 w-7", className)}
      onClick={(event) => {
        onClick?.(event)
        toggleSidebar()
      }}
      {...props}
    >
      <PanelLeftIcon />
      <span className="sr-only">Toggle Sidebar</span>
    </Button>
  )
}

function SidebarRail({ className, ...props }: React.ComponentProps<"button">) {
  const { toggleSidebar } = useSidebar()

  // Note: Tailwind v3.4 doesn't support "in-" selectors. So the rail won't work perfectly.
  return (
    <button
      data-sidebar="rail"
      data-slot="sidebar-rail"
      aria-label="Toggle Sidebar"
      tabIndex={-1}
      onClick={toggleSidebar}
      title="Toggle Sidebar"
      className={cn(
        "hover:after:bg-sidebar-border absolute inset-y-0 z-20 hidden w-4 -translate-x-1/2 transition-all ease-linear...",
        "in-data-[side=left]:cursor-w-resize in-data-[side=right]:cursor-e-resize",
        "[data-side=left][data-state=collapsed]&]:cursor-e-resize [[data-side=right][data-state=collapsed]&]:curso...",
        "hover:group-data-[collapsible=offcanvas]:bg-sidebar group-data-[collapsible=offcanvas]:translate-x-0 group-d...",
        "[data-side=left][data-collapsible=offcanvas]&]:-right-2",
        "[data-side=right][data-collapsible=offcanvas]&]:-left-2",
        className
      )}
      {...props}
    >
    </button>
  )
}

function SidebarInset({ className, ...props }: React.ComponentProps<"main">) {
  return (
    <main
      data-slot="sidebar-inset"
      className={cn(
        "bg-background relative flex w-full flex-1 flex-col",
        "md:peer-data-[variant=inset]:m-2 md:peer-data-[variant=inset]:ml-0 md:peer-data-[variant=inset]:rounded-xl m...",
        className
      )}
      {...props}
    >
    </main>
  )
}

function SidebarInput({
  className,
  ...props
}: React.ComponentProps<typeof Input>) {
  return (
    <Input
      data-slot="sidebar-input"
      data-sidebar="input"
      className={cn("bg-background h-8 w-full shadow-none", className)}
      {...props}
    >
    </Input>
  )
}

function SidebarHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-header"
      data-sidebar="header"
      className={cn("flex flex-col gap-2 p-2", className)}
    >
    </div>
  )
}

```



```

    {...props}
  />
}
}

function SidebarFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-footer"
      data-sidebar="footer"
      className={cn("flex flex-col gap-2 p-2", className)}
      {...props}
    />
  )
}

function SidebarSeparator({
  className,
  ...props
}: React.ComponentProps<typeof Separator>) {
  return (
    <Separator
      data-slot="sidebar-separator"
      data-sidebar="separator"
      className={cn("bg-sidebar-border mx-2 w-auto", className)}
      {...props}
    />
  )
}

function SidebarContent({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-content"
      data-sidebar="content"
      className={cn(
        "flex min-h-0 flex-1 flex-col gap-2 overflow-auto group-data-[collapsible=icon]:overflow-hidden",
        className
      )}
      {...props}
    />
  )
}

function SidebarGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-group"
      data-sidebar="group"
      className={cn("relative flex w-full min-w-0 flex-col p-2", className)}
      {...props}
    />
  )
}

function SidebarGroupLabel({
  className,
  asChild = false,
  ...props
}: React.ComponentProps<"div"> & { asChild?: boolean }) {
  const Comp = asChild ? Slot : "div"

  return (
    <Comp
      data-slot="sidebar-group-label"
      data-sidebar="group-label"
      className={cn(
        "text-sidebar-foreground/70 ring-sidebar-ring flex h-8 shrink-0 items-center rounded-md px-2 text-xs font-med...",
        "group-data-[collapsible=icon]:-mt-8 group-data-[collapsible=icon]:opacity-0",
        className
      )}
      {...props}
    />
  )
}

function SidebarGroupAction({
  className,
  asChild = false,
  ...props
}: React.ComponentProps<"button"> & { asChild?: boolean }) {
  const Comp = asChild ? Slot : "button"

  return (
    <Comp
      data-slot="sidebar-group-action"
      data-sidebar="group-action"
      className={cn(
        "text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent hover:text-sidebar-accent-foreground absol...",
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 md:after:hidden",
        "group-data-[collapsible=icon]:hidden",
        className
      )}
      {...props}
    />
  )
}

function SidebarGroupContent({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-group-content"
      data-sidebar="group-content"
      className={cn("w-full text-sm", className)}
    />
  )
}

```

```

    {...props}
  />
}
}

function SidebarMenu({ className, ...props }: React.ComponentProps<"ul">) {
  return (
    <ul
      data-slot="sidebar-menu"
      data-sidebar="menu"
      className={cn("flex w-full min-w-0 flex-col gap-1", className)}
      {...props}
    />
  )
}

function SidebarMenuItem({ className, ...props }: React.ComponentProps<"li">) {
  return (
    <li
      data-slot="sidebar-menu-item"
      data-sidebar="menu-item"
      className={cn("group/menu-item relative", className)}
      {...props}
    />
  )
}

const sidebarMenuButtonVariants = cva(
  "peer/menu-button flex w-full items-center gap-2 overflow-hidden rounded-md p-2 text-left text-sm outline-hidden ri...",
  {
    variants: {
      variant: {
        default: "hover:bg-sidebar-accent hover:text-sidebar-accent-foreground",
        outline:
          "bg-background shadow-[0_0_0px_hsl(var(--sidebar-border))] hover:bg-sidebar-accent hover:text-sidebar-acc...",
      },
      size: {
        default: "h-8 text-sm",
        sm: "h-7 text-xs",
        lg: "h-12 text-sm group-data-[collapsible=icon]:p-0!",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

function SidebarMenuButton({
  asChild = false,
  isActive = false,
  variant = "default",
  size = "default",
  tooltip,
  className,
  ...props
}: React.ComponentProps<"button"> & {
  asChild?: boolean
  isActive?: boolean
  tooltip?: string | React.ComponentProps<typeof TooltipContent>
} & VariantProps<typeof sidebarMenuButtonVariants>) {
  const Comp = asChild ? Slot : "button"
  const { isMobile, state } = useSidebar()

  const button = (
    <Comp
      data-slot="sidebar-menu-button"
      data-sidebar="menu-button"
      data-size={size}
      data-active={isActive}
      className={cn(sidebarMenuButtonVariants({ variant, size }), className)}
      {...props}
    />
  )

  if (!tooltip) {
    return button
  }

  if (typeof tooltip === "string") {
    tooltip = {
      children: tooltip,
    }
  }

  return (
    <Tooltip>
      <TooltipTrigger asChild>{button}</TooltipTrigger>
      <TooltipContent
        side="right"
        align="center"
        hidden={state !== "collapsed" || isMobile}
        {...tooltip}
      />
    </Tooltip>
  )
}

function SidebarMenuAction({
  className,
  asChild = false,
  showOnHover = false,
  ...props
}: React.ComponentProps<"button"> & {
  asChild?: boolean
  showOnHover?: boolean
}) {

```

```

const Comp = asChild ? Slot : "button"

return (
  <Comp
    data-slot="sidebar-menu-action"
    data-sidebar="menu-action"
    className={cn(
      "text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent hover:text-sidebar-accent-foreground peer-...
      // Increases the hit area of the button on mobile.
      "after:absolute after:-inset-2 md:after:hidden",
      "peer-data-[size=sm]/menu-button:top-1",
      "peer-data-[size=default]/menu-button:top-1.5",
      "peer-data-[size=lg]/menu-button:top-2.5",
      "group-data-[collapsible=icon]:hidden",
      showOnHover &&
      "peer-data-[active=true]/menu-button:text-sidebar-accent-foreground group-focus-within/menu-item:opacity-10..."
    )}
    {...props}
  />
)
}

function SidebarMenuBadge({
  className,
  ...props
}): React.ComponentProps<"div"> {
  return (
    <div
      data-slot="sidebar-menu-badge"
      data-sidebar="menu-badge"
      className={cn(
        "text-sidebar-foreground pointer-events-none absolute right-1 flex h-5 min-w-5 items-center justify-center ro...
        "peer-hover/menu-button:text-sidebar-accent-foreground peer-data-[active=true]/menu-button:text-sidebar-accen...
        "peer-data-[size=sm]/menu-button:top-1",
        "peer-data-[size=default]/menu-button:top-1.5",
        "peer-data-[size=lg]/menu-button:top-2.5",
        "group-data-[collapsible=icon]:hidden",
      )}
      className
    >
      <div
        data-slot="sidebar-menu-skeleton"
        data-sidebar="menu-skeleton"
        className={cn("flex h-8 items-center gap-2 rounded-md px-2", className)}
        {...props}
      >
        {showIcon && (
          <Skeleton
            className="size-4 rounded-md"
            data-sidebar="menu-skeleton-icon"
          />
        )}
        <Skeleton
          className="h-4 max-w-[var(--skeleton-width)] flex-1"
          data-sidebar="menu-skeleton-text"
          style={
            {
              "--skeleton-width": width,
            } as React.CSSProperties
          }
        />
      </div>
    >
  )
}

function SidebarMenuSubItem({ className, ...props }): React.ComponentProps<"li"> {
  return (
    <li
      data-slot="sidebar-menu-sub-item"
      data-sidebar="menu-sub-item"
      className={cn("group/menu-sub-item relative", className)}
      {...props}
    >

```

```

    )
  }
}

function SidebarMenuSubButton({
  asChild = false,
  size = "md",
  isActive = false,
  className,
  ...props
}: React.ComponentProps<"a"> & {
  asChild?: boolean
  size?: "sm" | "md"
  isActive?: boolean
}) {
  const Comp = asChild ? Slot : "a"

  return (
    <Comp
      data-slot="sidebar-menu-sub-button"
      data-sidebar="menu-sub-button"
      data-size={size}
      data-active={isActive}
      className={cn(
        "text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent hover:text-sidebar-accent-foreground activ...",
        "data-[active=true]:bg-sidebar-accent data-[active=true]:text-sidebar-accent-foreground",
        size === "sm" && "text-xs",
        size === "md" && "text-sm",
        "group-data-[collapsible=icon]:hidden",
        className
      )}
      {...props}
    />
  )
}

export {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarGroup,
  SidebarGroupAction,
  SidebarGroupContent,
  SidebarGroupLabel,
  SidebarHeader,
  SidebarInput,
  SidebarInset,
  SidebarMenu,
  SidebarMenuAction,
  SidebarMenuBadge,
  SidebarMenuButton,
  SidebarMenuItem,
  SidebarMenuSkeleton,
  SidebarMenuSub,
  SidebarMenuSubButton,
  SidebarMenuSubItem,
  SidebarProvider,
  SidebarRail,
  SidebarSeparator,
  SidebarTrigger,
  useSidebar,
}

```

`client/src/components/ui/skeleton.tsx`

```

import { cn } from "@/lib/utls"

function Skeleton({
  className,
  ...props
}: React.HTMLAttributes<HTMLDivElement>) {
  return (
    <div
      className={cn("animate-pulse rounded-md bg-muted", className)}
      {...props}
    />
  )
}

export { Skeleton }

```

`client/src/components/ui/slider.tsx`

```

import * as React from "react"
import * as SliderPrimitive from "@radix-ui/react-slider"

import { cn } from "@/lib/utls"

const Slider = React.forwardRef<
  React.ElementRef<typeof SliderPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof SliderPrimitive.Root>
>(({ className, ...props }, ref) => {
  <SliderPrimitive.Root
    ref={ref}
    className={cn(
      "relative flex w-full touch-none select-none items-center",
      className
    )}
    {...props}
  />

```

```
    <SliderPrimitive.Track className="relative h-2 w-full grow overflow-hidden rounded-full bg-secondary">
      <SliderPrimitive.Range className="absolute h-full bg-primary" />
    </SliderPrimitive.Track>
    <SliderPrimitive.Thumb className="block h-5 w-5 rounded-full border-2 border-primary bg-background ring-offset-ba...
  </SliderPrimitive.Root>
})
Slider.displayName = SliderPrimitive.Root.displayName

export { Slider }
```

`client/src/components/ui/switch.tsx`

```
import * as React from "react"
import * as SwitchPrimitives from "@radix-ui/react-switch"

import { cn } from "@lib/utils"

const Switch = React.forwardRef<
  React.ElementRef<typeof SwitchPrimitives.Root>,
  React.ComponentPropsWithoutRef<typeof SwitchPrimitives.Root>
>(({ className, ...props }, ref) => (
  <SwitchPrimitives.Root
    className={cn(
      "peer inline-flex h-3 w-6 shrink-0 cursor-pointer items-center rounded-full border border-transparent transitio...
      className
    )}
    {...props}
    ref={ref}
  >
    <SwitchPrimitives.Thumb
      className={cn(
        "pointer-events-none block h-2 w-2 rounded-full bg-background shadow-lg ring-0 transition-transform data-[sta...
      )}
    </SwitchPrimitives.Thumb>
  </SwitchPrimitives.Root>
))
Switch.displayName = SwitchPrimitives.Root.displayName

export { Switch }
```

`client/src/components/ui/table.tsx`

```
import * as React from "react"

import { cn } from "@lib/utils"

const Table = React.forwardRef<
  HTMLTableElement,
  React.HTMLAttributes<HTMLTableElement>
>(({ className, ...props }, ref) => (
  <div className="relative w-full overflow-auto">
    <table
      ref={ref}
      className={cn("w-full caption-bottom text-sm", className)}
      {...props}
    >
      </div>
  </div>
))
Table.displayName = "Table"

const TableHeader = React.forwardRef<
  HTMLTableSectionElement,
  React.HTMLAttributes<HTMLTableSectionElement>
>(({ className, ...props }, ref) => (
  <thead ref={ref} className={cn("[&_tr]:border-b", className)} {...props} />
))
TableHeader.displayName = "TableHeader"

const TableBody = React.forwardRef<
  HTMLTableSectionElement,
  React.HTMLAttributes<HTMLTableSectionElement>
>(({ className, ...props }, ref) => (
  <tbody
    ref={ref}
    className={cn("[&_tr:last-child]:border-0", className)}
    {...props}
  >
    </div>
  </div>
))
TableBody.displayName = "TableBody"

const TableFooter = React.forwardRef<
  HTMLTableSectionElement,
  React.HTMLAttributes<HTMLTableSectionElement>
>(({ className, ...props }, ref) => (
  <tfoot
    ref={ref}
    className={cn(
      "border-t bg-muted/50 font-medium [&_tr]:last:border-b-0",
      className
    )}
    {...props}
  >
    </div>
  </div>
))
TableFooter.displayName = "TableFooter"

const TableRow = React.forwardRef<
  HTMLTableRowElement,
  React.HTMLAttributes<HTMLTableRowElement>
>(({ className, ...props }, ref) => (
  <tr
```

```

      ref={ref}
      className={cn(
        "border-b transition-colors hover:bg-muted/50 data-[state=selected]:bg-muted",
        className
      )}
    )}
    {...props}
  )}
  />
))
TableRow.displayName = "TableRow"

const TableHead = React.forwardRef<
  HTMLTableCaptionElement,
  React.ThHTMLAttributes<HTMLTableCaptionElement>
>(({ className, ...props }, ref) => (
  <th
    ref={ref}
    className={cn(
      "h-12 px-4 text-left align-middle font-medium text-muted-foreground [&:has([role=checkbox])]:pr-0",
      className
    )}
    {...props}
  )}
  />
))
TableHead.displayName = "TableHead"

const TableCell = React.forwardRef<
  HTMLTableCaptionElement,
  React.TdHTMLAttributes<HTMLTableCaptionElement>
>(({ className, ...props }, ref) => (
  <td
    ref={ref}
    className={cn("p-4 align-middle [&:has([role=checkbox])]:pr-0", className)}
    {...props}
  )}
  />
))
TableCell.displayName = "TableCell"

const TableCaption = React.forwardRef<
  HTMLTableCaptionElement,
  React.HTMLAttributes<HTMLTableCaptionElement>
>(({ className, ...props }, ref) => (
  <caption
    ref={ref}
    className={cn("mt-4 text-sm text-muted-foreground", className)}
    {...props}
  )}
  />
))
TableCaption.displayName = "TableCaption"

export {
  Table,
  TableHeader,
  TableBody,
  TableFooter,
  TableHead,
  TableRow,
  TableCell,
  TableCaption,
}

```

`client/src/components/ui/tabs.tsx`

```

import * as React from "react"
import * as TabsPrimitive from "@radix-ui/react-tabs"

import { cn } from "@lib/utlis"

const Tabs = TabsPrimitive.Root

const TabsList = React.forwardRef<
  React.ElementRef<typeof TabsPrimitive.List>,
  React.ComponentPropsWithoutRef<typeof TabsPrimitive.List>
>(({ className, ...props }, ref) => (
  <TabsPrimitive.List
    ref={ref}
    className={cn(
      "inline-flex h-10 items-center justify-center rounded-md bg-muted p-1 text-muted-foreground",
      className
    )}
    {...props}
  )}
  />
))
TabsList.displayName = TabsPrimitive.List.displayName

const TabsTrigger = React.forwardRef<
  React.ElementRef<typeof TabsPrimitive.Trigger>,
  React.ComponentPropsWithoutRef<typeof TabsPrimitive.Trigger>
>(({ className, ...props }, ref) => (
  <TabsPrimitive.Trigger
    ref={ref}
    className={cn(
      "inline-flex items-center justify-center whitespace-nowrap rounded-sm px-3 py-1.5 text-sm font-medium ring-offs...
    )}
    {...props}
  )}
  />
))
TabsTrigger.displayName = TabsPrimitive.Trigger.displayName

const TabsContent = React.forwardRef<
  React.ElementRef<typeof TabsPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof TabsPrimitive.Content>
>(({ className, ...props }, ref) => (
  <TabsPrimitive.Content

```

```

      ref={ref}
      className={cn(
        "mt-2 ring-offset-background focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visi...
      )}
    )}
    {...props}
  )}
  />
)
TabsContent.displayName = TabsPrimitive.Content.displayName

export { Tabs, TabsList, TabsTrigger, TabsContent }

```

`client/src/components/ui/textarea.tsx`

```

import * as React from "react"

import { cn } from "@lib/utlis"

const Textarea = React.forwardRef<
  HTMLTextAreaElement,
  React.ComponentProps<"textarea">
>(({ className, ...props }, ref) => {
  return (
    <textarea
      className={cn(
        "flex min-h-[80px] w-full rounded-md border border-input bg-background px-3 py-2 text-base ring-offset-backgr...
      className
      )}
      ref={ref}
      {...props}
    />
  )
})
Textarea.displayName = "Textarea"

export { Textarea }

```

`client/src/components/ui/toast.tsx`

```

import * as React from "react"
import * as ToastPrimitives from "@radix-ui/react-toast"
import { cva, type VariantProps } from "class-variance-authority"
import { X } from "lucide-react"

import { cn } from "@lib/utlis"

const ToastProvider = ToastPrimitives.Provider

const ToastViewport = React.forwardRef<
  React.ElementRef<typeof ToastPrimitives.Viewport>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitives.Viewport>
>(({ className, ...props }, ref) => {
  <ToastPrimitives.Viewport
    ref={ref}
    className={cn(
      "fixed top-0 z-[100] flex max-h-screen w-full flex-col-reverse p-4 sm:bottom-0 sm:right-0 sm:top-auto sm:flex-c...
    className
    )}
    {...props}
  />
})
ToastViewport.displayName = ToastPrimitives.Viewport.displayName

const toastVariants = cva(
  "group pointer-events-auto relative flex w-full items-center justify-between space-x-4 overflow-hidden rounded-md b...
  {
    variants: {
      variant: {
        default: "border bg-background text-foreground",
        destructive:
          "destructive group border-destructive bg-destructive text-destructive-foreground",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
)

const Toast = React.forwardRef<
  React.ElementRef<typeof ToastPrimitives.Root>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitives.Root> &
  VariantProps<typeof toastVariants>
>(({ className, variant, ...props }, ref) => {
  return (
    <ToastPrimitives.Root
      ref={ref}
      className={cn(toastVariants({ variant }), className)}
      {...props}
    />
  )
})
Toast.displayName = ToastPrimitives.Root.displayName

const ToastAction = React.forwardRef<
  React.ElementRef<typeof ToastPrimitives.Action>,
  React.ComponentPropsWithoutRef<typeof ToastPrimitives.Action>
>(({ className, ...props }, ref) => {
  <ToastPrimitives.Action
    ref={ref}

```

```

        className={cn(
          "inline-flex h-8 shrink-0 items-center justify-center rounded-md border bg-transparent px-3 text-sm font-medium...",
          className
        )}
      {...props}
    ) />
  ))
  ToastAction.displayName = ToastPrimitives.Action.displayName

  const ToastClose = React.forwardRef<
    React.ElementRef<typeof ToastPrimitives.Close>,
    React.ComponentPropsWithoutRef<typeof ToastPrimitives.Close>
  >(({ className, ...props }, ref) => (
    <ToastPrimitives.Close
      ref={ref}
      className={cn(
        "absolute right-2 top-2 rounded-md p-1 text-foreground/50 opacity-0 transition-opacity hover:text-foreground fo...",
        className
      )}
      toast-close=""
      {...props}
    >
    <X className="h-4 w-4" />
    </ToastPrimitives.Close>
  ))
  ToastClose.displayName = ToastPrimitives.Close.displayName

  const ToastTitle = React.forwardRef<
    React.ElementRef<typeof ToastPrimitives.Title>,
    React.ComponentPropsWithoutRef<typeof ToastPrimitives.Title>
  >(({ className, ...props }, ref) => (
    <ToastPrimitives.Title
      ref={ref}
      className={cn("text-sm font-semibold", className)}
      {...props}
    />
  ))
  ToastTitle.displayName = ToastPrimitives.Title.displayName

  const ToastDescription = React.forwardRef<
    React.ElementRef<typeof ToastPrimitives.Description>,
    React.ComponentPropsWithoutRef<typeof ToastPrimitives.Description>
  >(({ className, ...props }, ref) => (
    <ToastPrimitives.Description
      ref={ref}
      className={cn("text-sm opacity-90", className)}
      {...props}
    />
  ))
  ToastDescription.displayName = ToastPrimitives.Description.displayName

  type ToastProps = React.ComponentPropsWithoutRef<typeof Toast>

  type ToastActionElement = React.ReactElement<typeof ToastAction>

  export {
    type ToastProps,
    type ToastActionElement,
    ToastProvider,
    ToastViewport,
    Toast,
    ToastTitle,
    ToastDescription,
    ToastClose,
    ToastAction,
  }
}

```

`client/src/components/ui/toaster.tsx`

```

import { useToast } from "@hooks/use-toast"
import {
  Toast,
  ToastClose,
  ToastDescription,
  ToastProvider,
  ToastTitle,
  ToastViewport,
} from "@components/ui/toast"

export function Toaster() {
  const { toasts } = useToast()

  return (
    <ToastProvider duration={3000}>
      {toasts.map(function ({ id, title, description, action, ...props }) {
        return (
          <Toast key={id} {...props}>
            <div className="grid gap-1">
              {title && <ToastTitle>{title}</ToastTitle>}
              {description && (
                <ToastDescription>{description}</ToastDescription>
              )}
            </div>
            {action}
            <ToastClose />
          </Toast>
        )
      })}
    <ToastViewport />
  </ToastProvider>
)
}

```

`client/src/components/ui/toggle-group.tsx`


```

"use client"

import * as React from "react"
import * as ToggleGroupPrimitive from "@radix-ui/react-toggle-group"
import { type VariantProps } from "class-variance-authority"

import { cn } from "@lib/utils"
import { toggleVariants } from "@components/ui/toggle"

const ToggleGroupContext = React.createContext<
  VariantProps<typeof toggleVariants>
>({
  size: "default",
  variant: "default",
})

const ToggleGroup = React.forwardRef<
  React.ElementRef<typeof ToggleGroupPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof ToggleGroupPrimitive.Root> &
  VariantProps<typeof toggleVariants>
>(({ className, variant, size, children, ...props }, ref) => {
  <ToggleGroupPrimitive.Root
    ref={ref}
    className={cn("flex items-center justify-center gap-1", className)}
    {...props}>
    <ToggleGroupContext.Provider value={{ variant, size }}>
      {children}
    </ToggleGroupContext.Provider>
  </ToggleGroupPrimitive.Root>
))

ToggleGroup.displayName = ToggleGroupPrimitive.Root.displayName

const ToggleGroupItem = React.forwardRef<
  React.ElementRef<typeof ToggleGroupPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof ToggleGroupPrimitive.Item> &
  VariantProps<typeof toggleVariants>
>(({ className, children, variant, size, ...props }, ref) => {
  const context = React.useContext(ToggleGroupContext)

  return (
    <ToggleGroupPrimitive.Item
      ref={ref}
      className={cn(
        toggleVariants({
          variant: context.variant || variant,
          size: context.size || size,
        }),
        className
      )}
      {...props}>
      {children}
    </ToggleGroupPrimitive.Item>
  )
})

ToggleGroupItem.displayName = ToggleGroupPrimitive.Item.displayName

export { ToggleGroup, ToggleGroupItem }

```

`client/src/components/ui/toggle.tsx`

```

import * as React from "react"
import * as TogglePrimitive from "@radix-ui/react-toggle"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@lib/utils"

const toggleVariants = cva(
  "inline-flex items-center justify-center rounded-md text-sm font-medium ring-offset-background transition-colors ho...
  {
    variants: {
      variant: {
        default: "bg-transparent",
        outline:
          "border border-input bg-transparent hover:bg-accent hover:text-accent-foreground",
      },
      size: {
        default: "h-10 px-3 min-w-10",
        sm: "h-9 px-2.5 min-w-9",
        lg: "h-11 px-5 min-w-11",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

const Toggle = React.forwardRef<
  React.ElementRef<typeof TogglePrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof TogglePrimitive.Root> &
  VariantProps<typeof toggleVariants>
>(({ className, variant, size, ...props }, ref) => {
  <TogglePrimitive.Root
    ref={ref}
    className={cn(toggleVariants({ variant, size, className })))}
    {...props}>
  />

```

```
))

Toggle.displayName = TogglePrimitive.Root.displayName

export { Toggle, toggleVariants }
```

`client/src/components/ui/tooltip.tsx`

```
"use client"

import * as React from "react"
import * as TooltipPrimitive from "@radix-ui/react-tooltip"

import { cn } from "@lib/utils"

const TooltipProvider = TooltipPrimitive.Provider

const Tooltip = TooltipPrimitive.Root

const TooltipTrigger = TooltipPrimitive.Trigger

const TooltipContent = React.forwardRef<
  React.ElementRef<typeof TooltipPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof TooltipPrimitive.Content>
>(({ className, sideOffset = 4, ...props }, ref) => {
  <TooltipPrimitive.Content
    ref={ref}
    sideOffset={sideOffset}
    className={cn(
      "z-50 overflow-hidden rounded-md border bg-popover px-3 py-1.5 text-sm text-popover-foreground shadow-md animat...
      className
    )}
    {...props}
  />
))
TooltipContent.displayName = TooltipPrimitive.Content.displayName

export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider }
```

`client/src/hooks/use-devices.ts`

```
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import { api, buildUrl, type InsertDevice, type UpdateDeviceRequest } from "@shared/routes";
import { useToast } from "@hooks/use-toast";

export function useDevices() {
  return useQuery({
    queryKey: [api.devices.list.path],
    queryFn: async () => {
      const res = await fetch(api.devices.list.path);
      if (!res.ok) throw new Error("Failed to fetch devices");
      return api.devices.list.responses[200].parse(await res.json());
    },
  });
}

export function useDevice(id: number) {
  return useQuery({
    queryKey: [api.devices.get.path, id],
    queryFn: async () => {
      const url = buildUrl(api.devices.get.path, { id });
      const res = await fetch(url);
      if (res.status === 404) return null;
      if (!res.ok) throw new Error("Failed to fetch device");
      return api.devices.get.responses[200].parse(await res.json());
    },
  });
}

export function useCreateDevice() {
  const queryClient = useQueryClient();
  const { toast } = useToast();

  return useMutation({
    mutationFn: async (data: InsertDevice) => {
      const validated = api.devices.create.input.parse(data);
      const res = await fetch(api.devices.create.path, {
        method: api.devices.create.method,
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(validated),
      });

      if (!res.ok) {
        if (res.status === 400) {
          const error = api.devices.create.responses[400].parse(await res.json());
          throw new Error(error.message);
        }
        throw new Error("Failed to create device");
      }
      return api.devices.create.responses[201].parse(await res.json());
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: [api.devices.list.path] });
      toast({ title: "Success", description: "Device added successfully" });
    },
    onError: (error) => {
      toast({
        title: "Error",
        description: error instanceof Error ? error.message : "Failed to add device",
        variant: "destructive"
      });
    }
  });
}
```

```

    });
  }
});

export function useUpdateDevice() {
  const queryClient = useQueryClient();
  const { toast } = useToast();

  return useMutation({
    mutationFn: async ({ id, ...updates }: { id: number } & UpdateDeviceRequest) => {
      const validated = api.devices.update.input.parse(updates);
      const url = buildUrl(api.devices.update.path, { id });

      const res = await fetch(url, {
        method: api.devices.update.method,
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(validated),
      });

      if (!res.ok) {
        if (res.status === 400) {
          const error = api.devices.update.responses[400].parse(await res.json());
          throw new Error(error.message);
        }
        if (res.status === 404) throw new Error("Device not found");
        throw new Error("Failed to update device");
      }
      return api.devices.update.responses[200].parse(await res.json());
    },
    onMutate: async ({ id, ...updates }) => {
      // Optimistic update
      await queryClient.cancelQueries({ queryKey: [api.devices.list.path] });
      const previousDevices = queryClient.getQueryData([api.devices.list.path]);

      queryClient.setQueryData([api.devices.list.path], (old: any[]) => {
        return old?.map((device) => {
          device.id === id ? { ...device, ...updates } : device
        });
      });

      return { previousDevices };
    },
    onError: (err, newTodo, context) => {
      queryClient.setQueryData([api.devices.list.path], context?.previousDevices);
      toast({
        title: "Error",
        description: "Failed to update device",
        variant: "destructive"
      });
    },
    onSettled: () => {
      queryClient.invalidateQueries({ queryKey: [api.devices.list.path] });
    },
  });
}

export function useDeleteDevice() {
  const queryClient = useQueryClient();
  const { toast } = useToast();

  return useMutation({
    mutationFn: async (id: number) => {
      const url = buildUrl(api.devices.delete.path, { id });
      const res = await fetch(url, { method: api.devices.delete.method });
      if (res.status === 404) throw new Error("Device not found");
      if (!res.ok) throw new Error("Failed to delete device");
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: [api.devices.list.path] });
      toast({ title: "Success", description: "Device removed" });
    },
    onError: () => {
      toast({
        title: "Error",
        description: "Failed to remove device",
        variant: "destructive"
      });
    },
  });
}

```

`client/src/hooks/use-mobile.tsx`

```

import * as React from "react"

const MOBILE_BREAKPOINT = 768

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState<boolean | undefined>(undefined)

  React.useEffect(() => {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`)
    const onChange = () => {
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
    }
    mql.addEventListener("change", onChange)
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
    return () => mql.removeEventListener("change", onChange)
  }, [])

  return !isMobile
}

```

`client/src/hooks/use-toast.ts`

```
import * as React from "react"

import type {
  ToastActionElement,
  ToastProps,
} from "@components/ui/toast"

const TOAST_LIMIT = 1
const TOAST_REMOVE_DELAY = 3000

type ToasterToast = ToastProps & {
  id: string
  title?: React.ReactNode
  description?: React.ReactNode
  action?: ToastActionElement
}

const actionTypes = {
  ADD_TOAST: "ADD_TOAST",
  UPDATE_TOAST: "UPDATE_TOAST",
  DISMISS_TOAST: "DISMISS_TOAST",
  REMOVE_TOAST: "REMOVE_TOAST",
} as const

let count = 0

function genId() {
  count = (count + 1) % Number.MAX_SAFE_INTEGER
  return count.toString()
}

type ActionType = typeof actionTypes

type Action =
  | {
      type: ActionType["ADD_TOAST"]
      toast: ToasterToast
    }
  | {
      type: ActionType["UPDATE_TOAST"]
      toast: Partial<ToasterToast>
    }
  | {
      type: ActionType["DISMISS_TOAST"]
      toastId?: ToasterToast["id"]
    }
  | {
      type: ActionType["REMOVE_TOAST"]
      toastId?: ToasterToast["id"]
    }

interface State {
  toasts: ToasterToast[]
}

const toastTimeouts = new Map<string, ReturnType<typeof setTimeout>>()

const addToRemoveQueue = (toastId: string) => {
  if (toastTimeouts.has(toastId)) {
    return
  }

  const timeout = setTimeout(() => {
    toastTimeouts.delete(toastId)
    dispatch({
      type: "REMOVE_TOAST",
      toastId: toastId,
    })
  }, TOAST_REMOVE_DELAY)

  toastTimeouts.set(toastId, timeout)
}

export const reducer = (state: State, action: Action): State => {
  switch (action.type) {
    case "ADD_TOAST":
      return {
        ...state,
        toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
      }

    case "UPDATE_TOAST":
      return {
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === action.toast.id ? { ...t, ...action.toast } : t
        ),
      }

    case "DISMISS_TOAST": {
      const { toastId } = action

      // ! Side effects ! - This could be extracted into a dismissToast() action,
      // but I'll keep it here for simplicity
      if (toastId) {
        addToRemoveQueue(toastId)
      } else {
        state.toasts.forEach((toast) => {
          addToRemoveQueue(toast.id)
        })
      }
    }
  }
}
```

```

    return {
      ...state,
      toasts: state.toasts.map((t) =>
        t.id === toastId || toastId === undefined
          ? {
              ...t,
              open: false,
            }
          : t
        ),
    }
  }
}
case "REMOVE_TOAST":
  if (action.toastId === undefined) {
    return {
      ...state,
      toasts: [],
    }
  }
  return {
    ...state,
    toasts: state.toasts.filter((t) => t.id !== action.toastId),
  }
}
}

const listeners: Array<(state: State) => void> = []

let memoryState: State = { toasts: [] }

function dispatch(action: Action) {
  memoryState = reducer(memoryState, action)
  listeners.forEach((listener) => {
    listener(memoryState)
  })
}

type Toast = Omit<ToasterToast, "id">

function toast({ ...props }: Toast) {
  const id = genId()

  const update = (props: ToasterToast) =>
    dispatch({
      type: "UPDATE_TOAST",
      toast: { ...props, id },
    })
  const dismiss = () => dispatch({ type: "DISMISS_TOAST", toastId: id })

  dispatch({
    type: "ADD_TOAST",
    toast: {
      ...props,
      id,
      open: true,
      onOpenChange: (open) => {
        if (!open) dismiss()
      },
    },
  })

  setTimeout(() => {
    dismiss()
  }, 3000)

  return {
    id: id,
    dismiss,
    update,
  }
}

function useToast() {
  const [state, setState] = React.useState<State>(memoryState)

  React.useEffect(() => {
    listeners.push(setState)
    return () => {
      const index = listeners.indexOf(setState)
      if (index > -1) {
        listeners.splice(index, 1)
      }
    }
  }, [state])

  return {
    ...state,
    toast,
    dismiss: (toastId?: string) => dispatch({ type: "DISMISS_TOAST", toastId }),
  }
}

export { useToast, toast }

```

`client/src/hooks/use-upload.ts`

```

import { useState, useCallback } from "react";
import type { UppyFile } from "@uppy/core";

interface UploadMetadata {
  name: string;
  size: number;
  contentType: string;
}

```

```

interface UploadResponse {
  uploadURL: string;
  objectPath: string;
  metadata: UploadMetadata;
}

interface UseUploadOptions {
  onSuccess?: (response: UploadResponse) => void;
  onError?: (error: Error) => void;
}

/**
 * React hook for handling file uploads with presigned URLs.
 *
 * This hook implements the two-step presigned URL upload flow:
 * 1. Request a presigned URL from your backend (sends JSON metadata, NOT the file)
 * 2. Upload the file directly to the presigned URL
 *
 * @example
 * ```tsx
 * function FileUploader() {
 *   const { uploadFile, isUploading, error } = useUpload({
 *     onSuccess: (response) => {
 *       console.log("Uploaded to:", response.objectPath);
 *     },
 *   });
 *
 *   const handleFileChange = async (e: React.ChangeEvent<HTMLInputElement>) => {
 *     const file = e.target.files?.[0];
 *     if (file) {
 *       await uploadFile(file);
 *     }
 *   };
 *
 *   return (
 *     <div>
 *       <input type="file" onChange={handleFileChange} disabled={isUploading} />
 *       {isUploading && <p>Uploading...</p>}
 *       {error && <p>Error: {error.message}</p>}
 *     </div>
 *   );
 * }
 * ```
 */
export function useUpload(options: UseUploadOptions = {}) {
  const [isUploading, setIsUploading] = useState(false);
  const [error, setError] = useState<Error | null>(null);
  const [progress, setProgress] = useState(0);

  /**
   * Request a presigned URL from the backend.
   * IMPORTANT: Send JSON metadata, NOT the file itself.
   */
  const requestUploadUrl = useCallback(
    async (file: File): Promise<UploadResponse> => {
      const response = await fetch("/api/uploads/request-url", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          name: file.name,
          size: file.size,
          contentType: file.type || "application/octet-stream",
        }),
      });

      if (!response.ok) {
        const errorData = await response.json().catch(() => ({}));
        throw new Error(errorData.error || "Failed to get upload URL");
      }

      return response.json();
    },
    []
  );

  /**
   * Upload a file directly to the presigned URL.
   */
  const uploadToPresignedUrl = useCallback(
    async (file: File, uploadURL: string): Promise<void> => {
      const response = await fetch(uploadURL, {
        method: "PUT",
        body: file,
        headers: {
          "Content-Type": file.type || "application/octet-stream",
        },
      });

      if (!response.ok) {
        throw new Error("Failed to upload file to storage");
      }
    },
    []
  );

  /**
   * Upload a file using the presigned URL flow.
   *
   * @param file - The file to upload
   * @returns The upload response containing the object path
   */
  const uploadFile = useCallback(
    async (file: File): Promise<UploadResponse | null> => {
      setIsUploading(true);
      setError(null);
    },
    []
  );

```

```

    setProgress(0);

    try {
      // Step 1: Request presigned URL (send metadata as JSON)
      setProgress(10);
      const uploadResponse = await requestUploadUrl(file);

      // Step 2: Upload file directly to presigned URL
      setProgress(30);
      await uploadToPresignedUrl(file, uploadResponse.uploadURL);

      setProgress(100);
      options.onSuccess?.(uploadResponse);
      return uploadResponse;
    } catch (err) {
      const error = err instanceof Error ? err : new Error("Upload failed");
      setError(error);
      options.onError?.(error);
      return null;
    } finally {
      setIsUploading(false);
    }
  },
  [requestUploadUrl, uploadToPresignedUrl, options]
);

/**
 * Get upload parameters for Uppy's AWS S3 plugin.
 *
 * IMPORTANT: This function receives the UppyFile object from Uppy.
 * Use file.name, file.size, file.type to request per-file presigned URLs.
 *
 * Use this with the ObjectUploader component:
 * ```tsx
 * <ObjectUploader onGetUploadParameters={getUploadParameters}>
 *   Upload
 * </ObjectUploader>
 * ```
 */
const getUploadParameters = useCallback(
  async (
    file: UppyFile<Record<string, unknown>, Record<string, unknown>>
  ): Promise<{
    method: "PUT";
    url: string;
    headers?: Record<string, string>;
  }> => {
    // Use the actual file properties to request a per-file presigned URL
    const response = await fetch("/api/uploads/request-url", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        name: file.name,
        size: file.size,
        contentType: file.type || "application/octet-stream",
      }),
    });

    if (!response.ok) {
      throw new Error("Failed to get upload URL");
    }

    const data = await response.json();
    return {
      method: "PUT",
      url: data.uploadURL,
      headers: { "Content-Type": file.type || "application/octet-stream" },
    };
  },
  []
);

return {
  uploadFile,
  getUploadParameters,
  isUploading,
  error,
  progress,
};
};
}

```

`client/src/index.css`

```

/* Avenir font family with fallbacks */

@tailwind base;
@tailwind components;
@tailwind utilities;

html, body {
  overflow: hidden !important;
  height: 100% !important;
  max-height: 100% !important;
  min-height: 100% !important;
  margin: 0 !important;
  padding: 0 !important;
  width: 100% !important;
  overscroll-behavior: none !important;
}

#root {

```

```

height: 100% !important;
max-height: 100% !important;
overflow: hidden !important;
overscroll-behavior: none !important;
}

:root {
  --background: 15 50% 93%;
  --foreground: 222 47% 20%;

  --primary: 221 83% 53%;
  --primary-foreground: 0 0% 100%;

  --secondary: 15 30% 88%;
  --secondary-foreground: 222 47% 20%;

  --card: 0 0% 100%;
  --card-foreground: 222 47% 20%;
  --card-border: 15 20% 85%;

  --muted: 15 25% 90%;
  --muted-foreground: 220 10% 45%;

  --accent: 15 30% 88%;
  --accent-foreground: 222 47% 20%;

  --destructive: 0 84% 60%;
  --destructive-foreground: 0 0% 100%;

  --border: 15 20% 85%;
  --input: 15 20% 85%;
  --ring: 221 83% 53%;

  --sidebar: 0 0% 100%;
  --sidebar-foreground: 222 47% 20%;
  --sidebar-border: 15 20% 88%;
  --sidebar-primary: 221 83% 53%;
  --sidebar-primary-foreground: 0 0% 100%;
  --sidebar-accent: 15 30% 92%;
  --sidebar-accent-foreground: 222 47% 20%;
  --sidebar-ring: 221 83% 53%;

  --popover: 0 0% 100%;
  --popover-foreground: 222 47% 20%;
  --popover-border: 15 20% 85%;

  --chart-1: 221 83% 53%;
  --chart-2: 142 71% 45%;
  --chart-3: 38 92% 50%;
  --chart-4: 280 65% 60%;
  --chart-5: 340 75% 55%;

  --radius: 0.75rem;
}

.dark {
  --background: 203 45% 35%;
  --foreground: 210 40% 96%;

  --primary: 217 91% 60%;
  --primary-foreground: 0 0% 100%;

  --secondary: 250 20% 20%;
  --secondary-foreground: 210 40% 96%;

  --card: 250 25% 16%;
  --card-foreground: 210 40% 96%;
  --card-border: 250 20% 25%;

  --muted: 250 20% 20%;
  --muted-foreground: 215 20% 65%;

  --accent: 250 20% 22%;
  --accent-foreground: 210 40% 96%;

  --destructive: 0 62% 50%;
  --destructive-foreground: 0 0% 100%;

  --border: 250 20% 25%;
  --input: 250 20% 25%;
  --ring: 217 91% 60%;

  --sidebar: 250 25% 14%;
  --sidebar-foreground: 210 40% 96%;
  --sidebar-border: 250 20% 25%;
  --sidebar-primary: 217 91% 60%;
  --sidebar-primary-foreground: 0 0% 100%;
  --sidebar-accent: 250 20% 22%;
  --sidebar-accent-foreground: 210 40% 96%;
  --sidebar-ring: 217 91% 60%;

  --popover: 250 25% 16%;
  --popover-foreground: 210 40% 96%;
  --popover-border: 250 20% 25%;

  --chart-1: 217 91% 60%;
  --chart-2: 160 60% 45%;
  --chart-3: 30 80% 55%;
  --chart-4: 280 65% 60%;
  --chart-5: 340 75% 55%;
}

/* Force normal font weight on all form elements */
input,
select,
option,
textarea,

```



```

[role="combobox"],
[role="listbox"],
[role="option"] {
  font-weight: 400 !important;
}

/* Force normal font weight on dialog form elements */
[role="dialog"] input,
[role="dialog"] select,
[role="dialog"] option,
[role="dialog"] textarea,
[role="dialog"] [data-radix-collection-item] {
  font-weight: 400 !important;
}

/* Force normal font weight for ALL dialog/flyout content */
[data-task-form] *,
[role="dialog"] *,
[data-radix-popover-content-wrapper] *,
[data-radix-select-content] *,
[data-radix-select-trigger] *,
[data-radix-select-viewport] * {
  font-weight: 400 !important;
}

/* Specifically target select value display text */
[data-radix-select-trigger] > span,
[data-radix-select-trigger] [data-placeholder],
button[role="combobox"] > span {
  font-weight: 400 !important;
}

@keyframes burst-from-top {
  0% {
    opacity: 0;
    transform: translate(-50%, -50%) scale(0);
  }
  100% {
    opacity: 1;
    transform: translate(-50%, -50%) scale(1);
  }
}

@keyframes burst-to-top {
  0% {
    opacity: 1;
    transform: translate(-50%, -50%) scale(1);
  }
  100% {
    opacity: 0;
    transform: translate(-50%, -50%) scale(0);
  }
}

@keyframes dialog-burst-from-top {
  0% {
    opacity: 0;
    scale: 0;
  }
  100% {
    opacity: 1;
    scale: 1;
  }
}

@keyframes dialog-burst-to-top {
  0% {
    opacity: 1;
    scale: 1;
  }
  100% {
    opacity: 0;
    scale: 0;
  }
}

.animate-burst-from-top {
  animation: burst-from-top 0.4s ease-out forwards;
  transform-origin: 50% calc(-50vh);
}

.animate-burst-to-top {
  animation: burst-to-top 0.25s ease-in forwards;
  transform-origin: 50% calc(-50vh);
}

.animate-dialog-burst-from-top {
  animation: dialog-burst-from-top 0.4s ease-out forwards;
  transform-origin: top center;
}

.animate-dialog-burst-to-top {
  animation: dialog-burst-to-top 0.25s ease-in forwards;
  transform-origin: top center;
}

@keyframes urgent-blink {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.4; }
}

.animate-urgent-blink {
  animation: urgent-blink 1s ease-in-out infinite;
}

.animate-urgent-blink.no-blink {
  animation: none !important;
  opacity: 1 !important;
}

```

```

}

@keyframes today-blink {
  0%, 100% { box-shadow: 0 0 0 3px hsl(217 91% 60%); }
  50% { box-shadow: 0 0 0 3px hsl(217 91% 60% / 0.3); }
}

.animate-today-blink {
  animation: today-blink 1.5s ease-in-out infinite;
}

@keyframes due-box-blink {
  0%, 70% { opacity: 1; }
  85% { opacity: 0.3; }
  100% { opacity: 1; }
}

.animate-due-box-blink {
  animation: due-box-blink 2s ease-in-out infinite;
}

@keyframes gentle-jiggle {
  0%, 100% { transform: rotate(0deg); }
  25% { transform: rotate(-2deg); }
  75% { transform: rotate(2deg); }
}

.animate-gentle-bounce {
  animation: gentle-jiggle 0.3s ease-in-out 3;
}

@keyframes pill-nudge {
  0%, 90% { transform: translateX(0px); }
  93% { transform: translateX(-5px); }
  96% { transform: translateX(0px); }
  98% { transform: translateX(-3px); }
  100% { transform: translateX(0px); }
}

.pill-arrow-idle {
  animation: pill-nudge 18s ease-in-out infinite !important;
}

@keyframes bottom-pill-container-nudge {
  0%, 70% { transform: translateX(-50%) translateY(56px); }
  74% { transform: translateX(-50%) translateY(50px); }
  78% { transform: translateX(-50%) translateY(56px); }
  82% { transform: translateX(-50%) translateY(51px); }
  86% { transform: translateX(-50%) translateY(56px); }
  90% { transform: translateX(-50%) translateY(52px); }
  94% { transform: translateX(-50%) translateY(56px); }
  100% { transform: translateX(-50%) translateY(56px); }
}

.bottom-pill-container-idle {
  animation: bottom-pill-container-nudge 6s ease-in-out infinite !important;
}

@keyframes bottom-tab-bounce {
  0%, 70% { transform: translateX(-50%) translateY(0); }
  74% { transform: translateX(-50%) translateY(-5px); }
  78% { transform: translateX(-50%) translateY(0); }
  82% { transform: translateX(-50%) translateY(-4px); }
  86% { transform: translateX(-50%) translateY(0); }
  90% { transform: translateX(-50%) translateY(-3px); }
  94% { transform: translateX(-50%) translateY(0); }
  100% { transform: translateX(-50%) translateY(0); }
}

.bottom-tab-bounce {
  animation: bottom-tab-bounce 6s ease-in-out 1s infinite;
}

@keyframes top-pill-container-nudge {
  0%, 70% { transform: translateY(-68px); }
  74% { transform: translateY(-62px); }
  78% { transform: translateY(-68px); }
  82% { transform: translateY(-63px); }
  86% { transform: translateY(-68px); }
  90% { transform: translateY(-64px); }
  94% { transform: translateY(-68px); }
  100% { transform: translateY(-68px); }
}

@keyframes left-tab-nudge {
  0%, 40% { transform: translateY(-50%) translateX(0px); }
  44% { transform: translateY(-50%) translateX(5px); }
  48% { transform: translateY(-50%) translateX(0px); }
  52% { transform: translateY(-50%) translateX(3px); }
  56% { transform: translateY(-50%) translateX(0px); }
  60% { transform: translateY(-50%) translateX(2px); }
  64% { transform: translateY(-50%) translateX(0px); }
  70%, 100% { transform: translateY(-50%) translateX(0px); }
}

@keyframes right-tab-nudge {
  0%, 40% { transform: translateY(-50%) translateX(0px); }
  44% { transform: translateY(-50%) translateX(-5px); }
  48% { transform: translateY(-50%) translateX(0px); }
  52% { transform: translateY(-50%) translateX(-3px); }
  56% { transform: translateY(-50%) translateX(0px); }
  60% { transform: translateY(-50%) translateX(-2px); }
  64% { transform: translateY(-50%) translateX(0px); }
  70%, 100% { transform: translateY(-50%) translateX(0px); }
}

.left-tab-bounce {

```

```

    animation: left-tab-nudge 6s 3s ease-in-out infinite;
}

.right-tab-bounce {
    animation: right-tab-nudge 6s 3s ease-in-out infinite;
}

.semester-tab-bounce {
    animation: semester-tab-nudge 6s 6s ease-in-out infinite;
}

@keyframes semester-tab-nudge {
    0%, 40% { transform: translateX(0px); }
    44% { transform: translateX(4px); }
    48% { transform: translateX(0px); }
    52% { transform: translateX(2px); }
    56% { transform: translateX(0px); }
    60%, 100% { transform: translateX(0px); }
}

@keyframes burst-in {
    0% { transform: scale(0); opacity: 0; }
    60% { transform: scale(1.05); opacity: 1; }
    100% { transform: scale(1); opacity: 1; }
}

@keyframes ctrl-marquee {
    0% { transform: translateX(0); }
    100% { transform: translateX(-50%); }
}

.top-pill-container-idle {
    animation: top-pill-container-nudge 6s ease-in-out infinite;
}

@keyframes blink {
    0%, 100% {
        outline: 3px solid #c0c0c0;
        outline-offset: 0px;
        box-shadow: 0 0 8px 3px rgba(192, 192, 192, 0.7), inset 0 0 20px rgba(192, 192, 192, 0.3);
        filter: brightness(1.1) saturate(0.9);
    }
    50% {
        outline: 3px solid #e8e8e8;
        outline-offset: 2px;
        box-shadow: 0 0 16px 6px rgba(220, 220, 220, 0.9), inset 0 0 30px rgba(255, 255, 255, 0.5);
        filter: brightness(1.25) saturate(0.8);
    }
}

.animate-blink:not(.absolute) {
    position: relative;
}

.animate-blink {
    animation: blink 2s ease-in-out infinite;
    z-index: 10;
    isolation: isolate;
}

.animate-blink-no-sweep {
    animation: blink 2s ease-in-out infinite;
    z-index: 10;
}

.animate-blink::before {
    content: '';
    position: absolute;
    inset: 0;
    background: linear-gradient(
        90deg,
        transparent 0%,
        rgba(255, 255, 255, 0.6) 40%,
        rgba(255, 255, 255, 0.9) 50%,
        rgba(255, 255, 255, 0.6) 60%,
        transparent 100%
    );
    border-radius: inherit;
    pointer-events: none;
    animation: shimmer-sweep 10s ease-in-out infinite;
    transform: translateX(-100%);
}

@keyframes shimmer-sweep {
    0% {
        transform: translateX(-100%);
        opacity: 0;
    }
    5% {
        opacity: 1;
    }
    15% {
        transform: translateX(100%);
        opacity: 1;
    }
    20% {
        opacity: 0;
    }
    100% {
        transform: translateX(100%);
        opacity: 0;
    }
}

@keyframes balloon-pulse {
    0%, 100% {
        transform: scale(1);
        box-shadow: 0 1px 3px rgba(0,0,0,0.12);
    }
}

```

```

}
50% {
  transform: scale(1.18);
  box-shadow: 0 4px 16px rgba(0,0,0,0.25);
}
}

.animate-balloon-pulse {
  animation: balloon-pulse 2s ease-in-out infinite;
  z-index: 20;
  transform-origin: center center;
}

@keyframes zero-day-blink {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.3; }
}

.animate-zero-day-blink {
  animation: zero-day-blink 1.4s ease-in-out infinite;
}

.animate-zero-day-blink:not(.countdown-due-today) {
  color: #dc2626 !important;
}

.animate-zero-day-blink:not(.countdown-due-today) *:not([data-countdown-badge]) {
  color: #dc2626 !important;
}

@keyframes reminder-red-pulse {
  0%, 80%, 100% {
    transform: scale(1);
    background-color: rgba(220, 38, 38, 0.25);
    border-color: rgba(220, 38, 38, 0.6);
    box-shadow: 0 0 0 rgba(220, 38, 38, 0);
  }
  90% {
    transform: scale(1.15);
    background-color: rgba(220, 38, 38, 0.55);
    border-color: rgba(220, 38, 38, 1);
    box-shadow: 0 0 12px rgba(220, 38, 38, 0.6);
  }
}

.course-cell-scroll {
  overflow-y: auto !important;
  overflow-x: hidden !important;
}
.course-cell-scroll::-webkit-scrollbar {
  width: 4px;
}
.course-cell-scroll::-webkit-scrollbar-track {
  background: rgba(0, 0, 0, 0.05);
  border-radius: 2px;
}
.course-cell-scroll::-webkit-scrollbar-thumb {
  background: rgba(0, 0, 0, 0.35);
  border-radius: 2px;
}
.course-cell-scroll::-webkit-scrollbar-thumb:hover {
  background: rgba(0, 0, 0, 0.5);
}

.animate-reminder-pulse {
  animation: reminder-red-pulse 5s ease-in-out infinite;
  z-index: 20;
  transform-origin: center center;
}

.animate-reminder-pulse * {
  color: #dc2626 !important;
}

@keyframes pill-reminder-balloon {
  0%, 80%, 100% {
    transform: scale(1);
    box-shadow: 0 0 0 rgba(220, 38, 38, 0);
  }
  85% {
    transform: scale(1.25);
    box-shadow: 0 0 16px rgba(220, 38, 38, 0.7);
  }
  90% {
    transform: scale(1.15);
    box-shadow: 0 0 12px rgba(220, 38, 38, 0.5);
  }
}

.animate-pill-reminder {
  animation: pill-reminder-balloon 5s ease-in-out infinite !important;
  background: linear-gradient(180deg, rgba(220, 38, 38, 0.5) 0%, rgba(220, 38, 38, 0.3) 100%) !important;
  border-color: rgba(220, 38, 38, 0.8) !important;
}

.pill-button-hover {
  transition: transform 0.15s ease, box-shadow 0.15s ease;
  will-change: transform;
}
.pill-button-hover:hover {
  transform: scale(1.12);
  box-shadow: 0 0 12px 3px rgba(255, 255, 255, 0.3) !important;
}
.pill-button-hover:active {
  transform: scale(1.05);
}
.pill-button-hover[data-state="open"] {
  transform: scale(1);
}

```

```

}

[data-radix-menu-content] [role="menuitem"]:hover,
[data-radix-menu-content] [role="menuitem"][data-highlighted] {
  background-color: hsl(var(--accent)) !important;
  color: hsl(var(--accent-foreground)) !important;
}

.calendar-icon-shimmer {
  position: relative;
  overflow: hidden;
}

.calendar-icon-shimmer::after {
  content: '';
  position: absolute;
  inset: 0;
  background: linear-gradient(
    120deg,
    transparent 0%,
    transparent 30%,
    rgba(255, 255, 255, 0.6) 50%,
    transparent 70%,
    transparent 100%
  );
  animation: calendar-shimmer 2.5s ease-in-out infinite;
}

@keyframes calendar-shimmer {
  0% { transform: translateX(-100%); }
  100% { transform: translateX(100%); }
}

/* Prep bar shimmer - opposite timing to main shimmer */
.animate-prep-shimmer {
  position: relative;
  overflow: hidden;
}

.animate-prep-shimmer::after {
  content: '';
  position: absolute;
  inset: 0;
  background: linear-gradient(
    90deg,
    transparent 0%,
    rgba(255, 255, 255, 0.4) 40%,
    rgba(255, 255, 255, 0.7) 50%,
    rgba(255, 255, 255, 0.4) 60%,
    transparent 100%
  );
  border-radius: inherit;
  pointer-events: none;
  animation: prep-shimmer-sweep 16s ease-in-out infinite;
  animation-delay: 8s; /* Offset by half to be opposite */
  transform: translateX(-100%);
}

@keyframes prep-shimmer-sweep {
  0% {
    transform: translateX(-100%);
    opacity: 0;
  }
  1% {
    opacity: 1;
  }
  5% {
    transform: translateX(100%);
    opacity: 1;
  }
  6% {
    opacity: 0;
  }
  100% {
    transform: translateX(100%);
    opacity: 0;
  }
}

@keyframes slow-blink {
  0%, 2%, 100% { opacity: 1; }
  1% { opacity: 0; }
}

.animate-slow-blink {
  animation: slow-blink 60s ease-in-out infinite;
}

/* Week folder blinking animation - more noticeable pulse */
@keyframes week-folder-blink {
  0%, 100% {
    opacity: 1;
    background-color: transparent;
  }
  50% {
    opacity: 0.6;
    background-color: rgba(239, 68, 68, 0.3);
  }
}

.animate-week-blink {
  animation: week-folder-blink 3s ease-in-out infinite;
}

/* File name blinking animation - prominent dark green to light with white text throughout */
@keyframes file-blink {
  0%, 100% {
    background-color: rgba(34, 80, 34, 0.9);
  }
  50% {

```

```

        background-color: rgba(134, 239, 172, 0.9);
    }
}

.animate-file-blink {
    animation: file-blink 1.5s ease-in-out infinite;
    color: white !important;
}

.animate-file-blink svg {
    color: white !important;
}

.animate-file-blink button {
    border-color: white !important;
}

.animate-file-blink button[data-state="checked"] {
    background-color: white !important;
    border-color: white !important;
}

.animate-file-blink button[data-state="checked"] svg {
    color: black !important;
}

/* Light teal box with black text blinking animation for files in task boxes */
@keyframes file-box-blink {
    0%, 100% {
        background-color: rgba(127, 219, 225, 0.5);
        color: rgba(0, 0, 0, 0.6);
    }
    50% {
        background-color: rgba(127, 219, 225, 1);
        color: rgba(0, 0, 0, 1);
    }
}

.animate-file-box-blink {
    animation: file-box-blink 1.5s ease-in-out infinite;
    border-radius: 4px;
    padding: 2px 6px;
}

.animate-file-box-blink-fast {
    animation: file-box-blink 0.6s ease-in-out infinite;
    border-radius: 4px;
    padding-top: 2px;
    padding-bottom: 2px;
    padding-right: 6px;
}

/* Prominent pulse for today date header - dark to light */
@keyframes today-date-pulse {
    0%, 100% {
        background-color: #164a72;
    }
    50% {
        background-color: #3a8bbf;
    }
}

.animate-today-date {
    animation: today-date-pulse 7s cubic-bezier(0.4, 0, 0.6, 1) infinite;
}

.animate-next-task-line {
    animation: next-task-line-pulse 4s ease-in-out infinite;
}

@keyframes next-task-line-pulse {
    0%, 100% {
        opacity: 0.4;
    }
    50% {
        opacity: 0.85;
    }
}

@keyframes today-day-text-pulse {
    0%, 100% {
        color: rgba(255, 255, 255, 0.8);
    }
    50% {
        color: #000;
    }
}

.animate-today-day-text {
    animation: today-day-text-pulse 7s cubic-bezier(0.4, 0, 0.6, 1) infinite;
}

@keyframes today-text-pulse {
    0%, 100% {
        color: #d5d5d5;
    }
    50% {
        color: #ffff00;
    }
}

.animate-today-text {
    animation: today-text-pulse 7s cubic-bezier(0.4, 0, 0.6, 1) infinite;
}

@keyframes today-yellow-pulse {
    0%, 100% {
        background-color: #E8E656;
    }
}

```

```

    }
    50% {
      background-color: #d4d24e;
    }
  }

  .animate-today-yellow {
    animation: today-yellow-pulse 1.6s cubic-bezier(0.4, 0, 0.6, 1) infinite;
  }

  @keyframes today-time-cell-pulse {
    0%, 100% {
      background-color: #164a72;
    }
    50% {
      background-color: #3a8bbf;
    }
  }

  .animate-today-time-cell {
    animation: today-date-pulse 7s cubic-bezier(0.4, 0, 0.6, 1) infinite;
  }

  /* Reversed pulse for time cells to sync with header */
  @keyframes today-pulse-sync {
    0%, 100% {
      box-shadow: inset 0 0 0 100px rgba(255, 255, 255, 0.15);
    }
    50% {
      box-shadow: inset 0 0 0 100px rgba(0, 0, 0, 0.1);
    }
  }

  .animate-today-column-sync {
    animation: today-pulse-sync 2s cubic-bezier(0.4, 0, 0.6, 1) infinite;
  }

  /* Red shimmer swipe for due-today task boxes on calendar */
  .silver-shimmer-header {
    position: relative;
    overflow: hidden;
  }

  .silver-shimmer-header::after {
    content: '';
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: linear-gradient(
      90deg,
      transparent 0%,
      rgba(220, 38, 38, 0.2) 30%,
      rgba(220, 38, 38, 0.6) 50%,
      rgba(220, 38, 38, 0.2) 70%,
      transparent 100%
    );
    animation: red-shimmer-sweep 20s ease-in-out infinite;
    pointer-events: none;
    transform: translateX(-100%);
    border-radius: inherit;
  }

  @keyframes red-shimmer-sweep {
    0% {
      transform: translateX(-100%);
      opacity: 0;
    }
    5% {
      opacity: 1;
    }
    15% {
      transform: translateX(100%);
      opacity: 1;
    }
    20% {
      opacity: 0;
    }
    100% {
      transform: translateX(100%);
      opacity: 0;
    }
  }

  /* Red background + white shimmer for countdown box blinking tasks */
  .countdown-due-today {
    background: #dc2626;
    border-radius: 6px;
    padding: 2px 8px;
    position: relative;
    overflow: hidden;
  }

  .countdown-due-today *,
  .countdown-due-today .animate-zero-day-blink,
  .countdown-due-today .animate-zero-day-blink * {
    color: #ffffff !important;
  }

  .countdown-due-today [data-countdown-badge] {
    background-color: #ffffff !important;
    color: #dc2626 !important;
  }

  /* Silver shimmer overlay for task rows and prep boxes - every 5 seconds */
  .silver-shimmer-task {

```

```

position: relative;
overflow: hidden;
background: linear-gradient(90deg, #a8a8a8 0%, #c0c0c0 50%, #a8a8a8 100%);
}

.silver-shimmer-task::after {
content: '';
position: absolute;
top: 0;
left: 0;
width: 100%;
height: 100%;
background: linear-gradient(
90deg,
transparent 0%,
rgba(192, 192, 192, 0.4) 35%,
rgba(255, 255, 255, 0.7) 50%,
rgba(192, 192, 192, 0.4) 65%,
transparent 100%
);
animation: silver-shimmer-sweep 5s ease-in-out infinite;
pointer-events: none;
transform: translateX(-100%);
border-radius: inherit;
}

@keyframes silver-shimmer-sweep {
0% {
transform: translateX(-100%);
opacity: 0;
}
5% {
opacity: 1;
}
15% {
transform: translateX(100%);
opacity: 1;
}
20% {
opacity: 0;
}
100% {
transform: translateX(100%);
opacity: 0;
}
}

/* Prep bar shimmer - bright silver shimmer over course gradient background */
.silver-shimmer-prep {
position: relative;
overflow: visible;
}

.silver-shimmer-prep::after {
content: '';
position: absolute;
top: 0;
left: -100%;
width: 300%;
height: 100%;
background: linear-gradient(
90deg,
transparent 0%,
transparent 30%,
rgba(255, 255, 255, 0.15) 40%,
rgba(255, 255, 255, 0.5) 50%,
rgba(255, 255, 255, 0.15) 60%,
transparent 70%,
transparent 100%
);
animation: silver-prep-shimmer-sweep 16s ease-in-out infinite;
pointer-events: none;
}

@keyframes silver-prep-shimmer-sweep {
0% {
transform: translateX(0%);
opacity: 0;
}
1% {
opacity: 1;
}
5% {
transform: translateX(100%);
opacity: 1;
}
6% {
opacity: 0;
}
100% {
transform: translateX(100%);
opacity: 0;
}
}

/* Today column and current hour row shimmer effect - runs every 16 seconds, silver */
.today-hour-shimmer {
position: relative;
overflow: hidden;
}

.today-hour-shimmer::after {
content: '';
position: absolute;
top: 0;
left: 0;
right: 0;
bottom: 0;
}

```



```

background: linear-gradient(
  90deg,
  transparent 0%,
  rgba(192, 192, 192, 0.3) 35%,
  rgba(220, 220, 220, 0.5) 50%,
  rgba(192, 192, 192, 0.3) 65%,
  transparent 100%
);
transform: translateX(-100%);
animation: today-hour-shimmer-sweep 16s ease-in-out infinite;
pointer-events: none;
}

@keyframes today-hour-shimmer-sweep {
  0% {
    transform: translateX(-100%);
    opacity: 0;
  }
  1% {
    opacity: 1;
  }
  5% {
    transform: translateX(100%);
    opacity: 1;
  }
  6% {
    opacity: 0;
  }
  100% {
    transform: translateX(100%);
    opacity: 0;
  }
}

/* Current hour row shimmer - one continuous shimmer across entire row */
.current-hour-row-shimmer {
  position: relative;
}

.current-hour-row-shimmer::after {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: linear-gradient(
    90deg,
    transparent 0%,
    rgba(192, 192, 192, 0.3) 35%,
    rgba(220, 220, 220, 0.5) 50%,
    rgba(192, 192, 192, 0.3) 65%,
    transparent 100%
  );
  transform: translateX(-100%);
  animation: current-hour-row-shimmer-sweep 16s ease-in-out infinite;
  pointer-events: none;
  z-index: 50;
}

@keyframes current-hour-row-shimmer-sweep {
  0% {
    transform: translateX(-100%);
    opacity: 0;
  }
  1% {
    opacity: 1;
  }
  5% {
    transform: translateX(100%);
    opacity: 1;
  }
  6% {
    opacity: 0;
  }
  100% {
    transform: translateX(100%);
    opacity: 0;
  }
}

/* Red separator vertical shimmer */
.red-separator-shimmer-sweep {
  background: linear-gradient(
    180deg,
    transparent 0%,
    rgba(255, 200, 200, 0.4) 20%,
    rgba(255, 255, 255, 0.9) 50%,
    rgba(255, 200, 200, 0.4) 80%,
    transparent 100%
  );
  transform: translateY(-100%);
  animation: red-separator-shimmer-sweep 40s ease-in-out infinite;
  pointer-events: none;
}

@keyframes red-separator-shimmer-sweep {
  0% {
    transform: translateY(-120%);
    opacity: 0;
  }
  2% {
    opacity: 1;
  }
  8% {
    transform: translateY(120%);
    opacity: 1;
  }
}

```

```

10% {
  opacity: 0;
}
100% {
  transform: translateY(120%);
  opacity: 0;
}
}

/* Task box border blink for due today */
.task-blink-border {
  animation: task-border-blink 1.5s ease-in-out infinite;
  border: 1px solid black !important;
}

@keyframes task-border-blink {
  0%, 100% { box-shadow: 0 0 8px 2px rgba(150, 180, 255, 0.5); }
  50% { box-shadow: 0 0 2px 1px rgba(150, 180, 255, 0.2); }
}

/* Clapping animation for celebration animoji */
@keyframes clap {
  0%, 100% {
    transform: scale(1) rotate(0deg);
  }
  10% {
    transform: scale(1.05) rotate(-3deg);
  }
  20% {
    transform: scale(0.95) rotate(3deg);
  }
  30% {
    transform: scale(1.05) rotate(-3deg);
  }
  40% {
    transform: scale(0.95) rotate(3deg);
  }
  50% {
    transform: scale(1.08) rotate(-2deg);
  }
  60% {
    transform: scale(0.95) rotate(2deg);
  }
  70% {
    transform: scale(1.05) rotate(-3deg);
  }
  80% {
    transform: scale(0.95) rotate(3deg);
  }
  90% {
    transform: scale(1.02) rotate(-1deg);
  }
}

.animate-clap {
  animation: clap 0.8s ease-in-out infinite;
}

/* Left hand clapping animation */
@keyframes clap-left {
  0%, 100% {
    transform: rotate(15deg) translateX(0);
  }
  50% {
    transform: rotate(-5deg) translateX(20px);
  }
}

.animate-clap-left {
  animation: clap-left 0.4s ease-in-out infinite;
}

/* Working indicator animation - 9 circles that pulse in teal */
@keyframes working-pulse {
  0%, 100% {
    background-color: #018a96;
    transform: scale(0.8);
  }
  50% {
    background-color: #4dd9e5;
    transform: scale(1);
  }
}

.working-indicator {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 2px;
  padding: 3px;
  border-radius: 4px;
  position: relative;
  background-color: #2d4a6f !important;
}

.working-indicator .circle {
  width: 5px;
  height: 5px;
  border-radius: 50%;
  background-color: rgba(255, 255, 255, 0.8);
  animation: working-pulse 1.2s ease-in-out infinite;
}

/* Slow pulse animation for prep day today column */
@keyframes pulse-slow {
  0%, 100% {
    background-color: rgba(255, 255, 255, 0.15);
  }
  50% {

```

```

    background-color: rgba(255, 255, 255, 0.6);
  }
}

.animate-pulse-slow {
  animation: pulse-slow 1.5s ease-in-out infinite;
}

@keyframes moon-glow {
  0%, 100% {
    filter: drop-shadow(0 0 2px rgba(167, 139, 250, 0.4)) drop-shadow(0 0 4px rgba(139, 92, 246, 0.2));
    opacity: 0.8;
    transform: scale(1) rotate(-8deg);
  }
  25% {
    filter: drop-shadow(0 0 5px rgba(167, 139, 250, 0.7)) drop-shadow(0 0 10px rgba(139, 92, 246, 0.3));
    opacity: 0.95;
    transform: scale(1.08) rotate(-2deg);
  }
  50% {
    filter: drop-shadow(0 0 8px rgba(167, 139, 250, 1)) drop-shadow(0 0 16px rgba(139, 92, 246, 0.5));
    opacity: 1;
    transform: scale(1.15) rotate(4deg);
  }
  75% {
    filter: drop-shadow(0 0 5px rgba(167, 139, 250, 0.7)) drop-shadow(0 0 10px rgba(139, 92, 246, 0.3));
    opacity: 0.95;
    transform: scale(1.08) rotate(0deg);
  }
}

.animate-moon-glow {
  animation: moon-glow 4s ease-in-out infinite;
  transform-origin: center center;
}

@keyframes sun-spin-glow {
  0%, 100% {
    filter: drop-shadow(0 0 2px rgba(250, 204, 21, 0.5)) drop-shadow(0 0 4px rgba(250, 180, 0, 0.2));
    transform: rotate(0deg) scale(1);
  }
  25% {
    filter: drop-shadow(0 0 5px rgba(250, 204, 21, 0.7)) drop-shadow(0 0 10px rgba(250, 180, 0, 0.3));
    transform: rotate(90deg) scale(1.1);
  }
  50% {
    filter: drop-shadow(0 0 8px rgba(250, 204, 21, 1)) drop-shadow(0 0 16px rgba(250, 180, 0, 0.5));
    transform: rotate(180deg) scale(1.18);
  }
  75% {
    filter: drop-shadow(0 0 5px rgba(250, 204, 21, 0.7)) drop-shadow(0 0 10px rgba(250, 180, 0, 0.3));
    transform: rotate(270deg) scale(1.1);
  }
}

.animate-sun-glow {
  animation: sun-spin-glow 6s ease-in-out infinite;
  transform-origin: center center;
}

/* Task color pulse animation - pulses opacity on the task's background color */
@keyframes pulse-task {
  0%, 100% {
    opacity: 0;
  }
  50% {
    opacity: 1;
  }
}

.animate-pulse-task {
  animation: pulse-task 1s ease-in-out infinite;
}

@keyframes pulse-urgent {
  0%, 100% {
    opacity: 1;
  }
  50% {
    opacity: 0.5;
  }
}

.animate-pulse-urgent {
  animation: pulse-urgent 2s ease-in-out infinite;
}

.working-indicator .circle:nth-child(1) { animation-delay: 0s; }
.working-indicator .circle:nth-child(2) { animation-delay: 0.1s; }
.working-indicator .circle:nth-child(3) { animation-delay: 0.2s; }
.working-indicator .circle:nth-child(4) { animation-delay: 0.3s; }
.working-indicator .circle:nth-child(5) { animation-delay: 0.4s; }
.working-indicator .circle:nth-child(6) { animation-delay: 0.5s; }
.working-indicator .circle:nth-child(7) { animation-delay: 0.6s; }
.working-indicator .circle:nth-child(8) { animation-delay: 0.7s; }
.working-indicator .circle:nth-child(9) { animation-delay: 0.8s; }

.working-indicator .days-overlay {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  color: white;
  font-size: 13px;
  font-weight: 700;
  text-shadow: 0 1px 3px rgba(0, 0, 0, 0.9);
  z-index: 10;
}

```

```

/* Right hand clapping animation */
@keyframes clap-right {
  0%, 100% {
    transform: rotate(-15deg) scaleX(-1) translateX(0);
  }
  50% {
    transform: rotate(5deg) scaleX(-1) translateX(20px);
  }
}

.animate-clap-right {
  animation: clap-right 0.4s ease-in-out infinite;
  transform-origin: center;
}

@keyframes shimmer {
  0% {
    opacity: 0;
    transform: translateX(-100%);
  }
  2% {
    opacity: 1;
  }
  10% {
    opacity: 1;
    transform: translateX(100%);
  }
  12% {
    opacity: 0;
    transform: translateX(100%);
  }
  100% {
    opacity: 0;
    transform: translateX(100%);
  }
}

.animate-shimmer:not(.absolute) {
  position: relative;
}

.animate-shimmer {
  overflow: hidden;
}

.animate-shimmer::after {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background: linear-gradient(
    90deg,
    transparent 0%,
    rgba(255, 255, 255, 0.7) 40%,
    rgba(255, 255, 255, 1) 50%,
    rgba(255, 255, 255, 0.7) 60%,
    transparent 100%
  );
  animation: shimmer 5s ease-in-out infinite;
  pointer-events: none;
}

@keyframes working-dot {
  0%, 20% {
    opacity: 0.3;
  }
  40% {
    opacity: 1;
  }
  60%, 100% {
    opacity: 0.3;
  }
}

.working-dots {
  display: flex;
  gap: 2px;
  align-items: center;
  justify-content: center;
}

.working-dot {
  width: 3px;
  height: 3px;
  background-color: white;
  border-radius: 50%;
  animation: working-dot 1.4s ease-in-out infinite;
}

.working-dot:nth-child(1) {
  animation-delay: 0s;
}

.working-dot:nth-child(2) {
  animation-delay: 0.2s;
}

.working-dot:nth-child(3) {
  animation-delay: 0.4s;
}

.today-popup {
  cursor: pointer;
}

```

```

.today-popup:hover {
  animation: none !important;
}

.today-popup-expanded {
  position: absolute !important;
  top: 50% !important;
  left: 50% !important;
  transform: translate(-50%, -50%) scale(2) !important;
  z-index: 100 !important;
  width: 200px !important;
  height: 160px !important;
  animation: none !important;
}

.calendar-container {
  position: relative;
}

.today-backdrop {
  position: fixed;
  inset: 0;
  background: rgba(0, 0, 0, 0.5);
  z-index: 99;
}

@layer base {
  body {
    @apply bg-background text-foreground antialiased;
    font-family: 'Avenir', 'Avenir Next', -apple-system, BlinkMacSystemFont, sans-serif;
  }

  h1, h2, h3, h4, h5, h6 {
    font-family: 'Avenir', 'Avenir Next', -apple-system, BlinkMacSystemFont, sans-serif;
  }
}

.font-raleway,
.font-raleway * {
  font-family: 'Raleway', sans-serif !important;
}

@layer utilities {
  .media-btn {
    position: relative;
    border: none !important;
    border-radius: 50% !important;
    background: transparent !important;
    box-shadow: none !important;
    color: white !important;
    transition: all 0.15s ease !important;
    overflow: visible !important;
    padding: 0 !important;
    display: inline-flex !important;
    align-items: center !important;
    justify-content: center !important;
    cursor: pointer;
    outline: none;
    -webkit-tap-highlight-color: transparent;
  }
  .media-btn::before {
    display: none;
  }
  .media-btn:hover {
    background: rgba(255, 255, 255, 0.15) !important;
  }
  .media-btn:active {
    background: rgba(255, 255, 255, 0.1) !important;
    transform: scale(0.96);
  }
  .media-btn:disabled {
    opacity: 0.35 !important;
    cursor: not-allowed;
  }
  .media-btn-lg {
    width: 44px !important;
    height: 44px !important;
    min-width: 44px !important;
    min-height: 44px !important;
  }
  .media-btn-sm {
    width: 34px !important;
    height: 34px !important;
    min-width: 34px !important;
    min-height: 34px !important;
  }
  .media-btn-xs {
    width: 24px !important;
    height: 24px !important;
    min-width: 24px !important;
    min-height: 24px !important;
  }
  .media-btn-play {
    width: 48px !important;
    height: 48px !important;
    min-width: 48px !important;
    min-height: 48px !important;
  }
  .media-btn-stop {}
  .media-btn-active {
    background: rgba(59, 130, 246, 0.25) !important;
    box-shadow: 0 0 8px rgba(59, 130, 246, 0.3) !important;
  }
  .media-btn-resume {}
  .task-missed {
    @apply border-destructive/50 bg-destructive/5;
  }
}

```

```

.task-completed {
  @apply opacity-60;
}

.checkbox-black {
  all: unset;
  display: inline-block;
  box-sizing: border-box;
  width: 13px;
  height: 13px;
  min-width: 13px;
  min-height: 13px;
  max-width: 13px;
  max-height: 13px;
  border: 1.5px solid #333;
  border-radius: 2px;
  background: #fff;
  cursor: pointer;
  position: relative;
  flex-shrink: 0;
  vertical-align: middle;
  -webkit-print-color-adjust: exact;
  print-color-adjust: exact;
}

.checkbox-black:checked {
  background: #059669;
  border-color: #059669;
}

.checkbox-black:checked::before {
  content: '';
  display: block;
  position: absolute;
  left: 3px;
  top: 0px;
  width: 5px;
  height: 8px;
  border-right: 2px solid #fff;
  border-bottom: 2px solid #fff;
  transform: rotate(45deg);
}

.checkbox-black:disabled {
  opacity: 0.35;
  cursor: not-allowed;
  background: #e5e7eb;
  border-color: #9ca3af;
}

.checkbox-black:disabled:checked {
  background: #059669;
  border-color: #059669;
  opacity: 0.35;
}

/* Color input styling */
input[type="color"] {
  -webkit-appearance: none;
  -moz-appearance: none;
  appearance: none;
  border: 1px solid rgba(255, 255, 255, 0.3);
  border-radius: 50%;
  cursor: pointer;
  padding: 0;
}

input[type="color"]::-webkit-color-swatch-wrapper {
  padding: 2px;
  border-radius: 50%;
}

input[type="color"]::-webkit-color-swatch {
  border: none;
  border-radius: 50%;
}

input[type="color"]::-moz-color-swatch {
  border: none;
  border-radius: 50%;
}

.flip-card {
  background: linear-gradient(180deg, #1a1a1a 0%, #1a1a1a 48%, #0d0d0d 50%, #1a1a1a 52%, #1a1a1a 100%);
  border-radius: 4px;
  box-shadow:
    inset 0 1px 0 rgba(255,255,255,0.1),
    inset 0 -1px 0 rgba(0,0,0,0.3),
    0 2px 4px rgba(0,0,0,0.4);
  position: relative;
  overflow: hidden;
}

.flip-card::before {
  content: '';
  position: absolute;
  left: 0;
  right: 0;
  top: 50%;
  height: 1px;
  background: rgba(0,0,0,0.6);
  z-index: 2;
}

.flip-card::after {
  content: '';
  position: absolute;

```

```

    left: -2px;
    right: -2px;
    top: calc(50% - 2px);
    height: 4px;
    background: #0a0a0a;
    border-radius: 2px;
    z-index: 1;
}

.flip-number {
    font-family: 'Segment7', 'Digital-7', 'DSEG7 Classic', 'Courier New', monospace;
    font-weight: bold;
    text-shadow: 0 0 10px currentColor;
}

/* Custom scrollbar styles for visibility */
.custom-scrollbar {
    scrollbar-width: none; /* Firefox */
    -ms-overflow-style: none; /* IE/Edge */
}

.custom-scrollbar::-webkit-scrollbar {
    display: none; /* Chrome/Safari/Opera */
}

/* Main content scrollbar */
.main-scrollbar::-webkit-scrollbar {
    width: 12px;
}

.main-scrollbar::-webkit-scrollbar-track {
    background: rgba(0, 0, 0, 0.3);
    border-radius: 6px;
}

.main-scrollbar::-webkit-scrollbar-thumb {
    background: rgba(255, 255, 255, 0.7);
    border-radius: 6px;
    border: 2px solid rgba(0, 0, 0, 0.2);
}

.main-scrollbar::-webkit-scrollbar-thumb:hover {
    background: rgba(255, 255, 255, 0.9);
}

/* Calendar overlay scrollbar - doesn't take up space */
.calendar-scrollbar {
    overflow-y: auto;
    scrollbar-width: thin;
    scrollbar-color: rgba(255, 255, 255, 0.4) transparent;
}

.calendar-scrollbar::-webkit-scrollbar {
    width: 8px;
    background: transparent;
}

.calendar-scrollbar::-webkit-scrollbar-track {
    background: transparent;
}

.calendar-scrollbar::-webkit-scrollbar-thumb {
    background: rgba(255, 255, 255, 0.4);
    border-radius: 4px;
}

.calendar-scrollbar::-webkit-scrollbar-thumb:hover {
    background: rgba(255, 255, 255, 0.6);
}

@supports (overflow-y: overlay) {
    .calendar-scrollbar {
        overflow-y: overlay;
    }
}

/* PDF reader white checkboxes */
.pdf-reader-scrollbar input[type="checkbox"].appearance-none:checked::after {
    content: ' ';
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    color: #000;
    font-size: 14px;
    font-weight: 700;
    line-height: 1;
}

/* PDF reader chunk list scrollbar */
.pdf-reader-scrollbar {
    scrollbar-width: thin;
    scrollbar-color: rgba(255, 255, 255, 0.4) rgba(255, 255, 255, 0.05);
}

.pdf-reader-scrollbar::-webkit-scrollbar {
    width: 10px;
}

.pdf-reader-scrollbar::-webkit-scrollbar-track {
    background: rgba(255, 255, 255, 0.05);
    border-radius: 5px;
}

.pdf-reader-scrollbar::-webkit-scrollbar-thumb {
    background: rgba(255, 255, 255, 0.35);
    border-radius: 5px;
}

```

```

border: 2px solid transparent;
background-clip: padding-box;
}

.pdf-reader-scrollbar::-webkit-scrollbar-thumb:hover {
background: rgba(255, 255, 255, 0.55);
background-clip: padding-box;
}

.pdf-reader-scrollbar::-webkit-scrollbar-thumb:active {
background: rgba(255, 255, 255, 0.7);
background-clip: padding-box;
}

/* Hidden scrollbar - scrollable but no visible scrollbar */
.scrollbar-hidden {
scrollbar-width: none;
-ms-overflow-style: none;
}

.scrollbar-hidden::-webkit-scrollbar {
display: none;
}

/* Celebration popup animations */
@keyframes shimmer {
0% {
background-position: -200% center;
text-shadow: 0 0 10px rgba(255, 215, 0, 0.8), 0 0 20px rgba(255, 215, 0, 0.6);
}
50% {
background-position: 200% center;
text-shadow: 0 0 20px rgba(255, 215, 0, 1), 0 0 40px rgba(255, 215, 0, 0.8), 0 0 60px rgba(255, 100, 100, 0.6);
}
100% {
background-position: -200% center;
text-shadow: 0 0 10px rgba(255, 215, 0, 0.8), 0 0 20px rgba(255, 215, 0, 0.6);
}
}

@keyframes bounce-in {
0% {
transform: scale(0) translateY(50px);
opacity: 0;
}
50% {
transform: scale(1.2) translateY(-10px);
}
70% {
transform: scale(0.9) translateY(5px);
}
100% {
transform: scale(1) translateY(0);
opacity: 1;
}
}

@keyframes fade-out {
0% {
opacity: 1;
}
80% {
opacity: 1;
}
100% {
opacity: 0;
}
}

.celebration-text {
font-family: "Avenir", "Avenir Next", -apple-system, BlinkMacSystemFont, sans-serif;
font-weight: 800;
font-size: 2.5rem;
background: linear-gradient(90deg, #ffd700, #ff6b6b, #ffd700, #ff6b6b, #ffd700);
background-size: 400% 100%;
-webkit-background-clip: text;
-webkit-text-fill-color: transparent;
background-clip: text;
animation: shimmer 1s ease-in-out infinite;
text-transform: uppercase;
letter-spacing: 0.1em;
}

.celebration-container {
animation: bounce-in 0.6s ease-out forwards, fade-out 5s ease-in-out forwards;
}

/* Force all dropdown/select content to have black text */
[data-radix-select-content],
[data-radix-select-viewport],
[data-radix-select-item],
[data-radix-dropdown-menu-content],
[data-radix-dropdown-menu-item],
[role="listbox"],
[role="option"],
[data-radix-popover-content-wrapper] div,
[data-radix-collection-item] {
background-color: white !important;
color: black !important;
}

[data-radix-select-item] *,
[data-radix-dropdown-menu-item] *,
[role="option"] *,
[role="option"] span,
[data-radix-collection-item] *,
[data-radix-popover-content-wrapper] span {
color: black !important;
}

```



```

}

[data-settings-dialog] .border.rounded-lg {
border-color: rgba(255, 255, 255, 0.25) !important;
background: rgba(255, 255, 255, 0.08);
}

[data-settings-dialog] .border-b.border-primary {
border-color: rgba(255, 255, 255, 0.35) !important;
}

[data-settings-dialog] span[style*="color: rgb(59, 130, 246)"],
[data-settings-dialog] span[style*="#3b82f6"] {
color: white !important;
}

/* Blue checkboxes matching hamburger menu - only in settings dialog */
[data-settings-dialog] input[type="checkbox"] {
-webkit-appearance: none;
appearance: none;
width: 16px;
height: 16px;
border-radius: 3px;
background: white;
border: 1px solid #4578B0;
cursor: pointer;
position: relative;
}

[data-settings-dialog] input[type="checkbox"]:checked {
background: linear-gradient(180deg, #4578B0 0%, #042550 100%);
}

[data-settings-dialog] input[type="checkbox"]:checked::after {
content: ' ';
position: absolute;
top: 50%;
left: 50%;
transform: translate(-50%, -50%);
color: white;
font-size: 8px;
font-weight: 300;
}

/* Orange gradient slider thumbs */
input[type="range"]::-webkit-slider-thumb {
-webkit-appearance: none;
appearance: none;
width: 12px;
height: 12px;
border-radius: 50%;
background: linear-gradient(180deg, #FF6E3D 0%, #FFDD63 100%);
cursor: pointer;
border: none;
box-shadow: 0 1px 3px rgba(0,0,0,0.3);
}

input[type="range"]::-moz-range-thumb {
width: 12px;
height: 12px;
border-radius: 50%;
background: linear-gradient(180deg, #FF6E3D 0%, #FFDD63 100%);
cursor: pointer;
border: none;
box-shadow: 0 1px 3px rgba(0,0,0,0.3);
}

/* Smaller font for datetime-local inputs */
input[type="datetime-local"] {
font-size: 8px !important;
}

input[type="datetime-local"]::-webkit-calendar-picker-indicator {
width: 10px;
height: 10px;
}

input[type="datetime-local"]::-webkit-datetime-edit {
font-size: 8px;
}

input[type="datetime-local"]::-webkit-datetime-edit-fields-wrapper {
font-size: 8px;
}

input[type="datetime-local"]::-webkit-datetime-edit-text,
input[type="datetime-local"]::-webkit-datetime-edit-month-field,
input[type="datetime-local"]::-webkit-datetime-edit-day-field,
input[type="datetime-local"]::-webkit-datetime-edit-year-field,
input[type="datetime-local"]::-webkit-datetime-edit-hour-field,
input[type="datetime-local"]::-webkit-datetime-edit-minute-field,
input[type="datetime-local"]::-webkit-datetime-edit-ampm-field {
font-size: 8px;
}

/* Settings dialog slider styling */
input[type="range"]::-webkit-slider-thumb {
-webkit-appearance: none;
appearance: none;
width: 8px;
height: 8px;
border-radius: 50%;
background: #3b82f6;
cursor: pointer;
}

input[type="range"]::-moz-range-thumb {
width: 8px;
}

```

```

height: 8px;
border-radius: 50%;
background: #3b82f6;
border: none;
cursor: pointer;
}

/* Settings dialog checkbox styling */
input[type="checkbox"] {
  -webkit-appearance: none;
  appearance: none;
  width: 12px !important;
  height: 12px !important;
  border: 1px solid #9ca3af;
  border-radius: 2px;
  cursor: pointer;
  position: relative;
  background: transparent;
}

input[type="checkbox"]:checked {
  background: #3b82f6;
  border-color: #3b82f6;
}

input[type="checkbox"]:checked::after {
  content: ' ';
  position: absolute;
  left: 1px;
  top: -2px;
  font-size: 8px;
  font-weight: 300;
  color: white;
  line-height: 1;
}

/* Transparent dropdown menu item hover */
[data-variant="transparent"]:hover,
[data-variant="transparent"]:focus {
  background-color: rgba(255, 255, 255, 0.25) !important;
}

[data-variant="transparent"] svg {
  color: white !important;
}

/* Diffuse open animation - grainy speckled effect (no movement, same size) */
@keyframes diffuse-up {
  0% {
    opacity: 0.3;
    filter: brightness(1.6) contrast(1.8);
    mask-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 400 400' xmlns='http://www.w3.org/2000/svg'%3E%3Cfilter i...
    -webkit-mask-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 400 400' xmlns='http://www.w3.org/2000/svg'%3E%3C...
  }
  35% {
    opacity: 0.6;
    filter: brightness(1.4) contrast(1.4);
    mask-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 400 400' xmlns='http://www.w3.org/2000/svg'%3E%3Cfilter i...
    -webkit-mask-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 400 400' xmlns='http://www.w3.org/2000/svg'%3E%3C...
  }
  65% {
    opacity: 0.85;
    filter: brightness(1.2) contrast(1.2);
    mask-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 400 400' xmlns='http://www.w3.org/2000/svg'%3E%3Cfilter i...
    -webkit-mask-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 400 400' xmlns='http://www.w3.org/2000/svg'%3E%3C...
  }
  100% {
    opacity: 1;
    filter: brightness(1) contrast(1);
    mask-image: none;
    -webkit-mask-image: none;
  }
}

/* Diffuse open animation - grainy speckled effect (no movement, same size) */
@keyframes diffuse-down {
  0% {
    opacity: 0.3;
    filter: brightness(1.6) contrast(1.8);
    mask-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 400 400' xmlns='http://www.w3.org/2000/svg'%3E%3Cfilter i...
    -webkit-mask-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 400 400' xmlns='http://www.w3.org/2000/svg'%3E%3C...
  }
  35% {
    opacity: 0.6;
    filter: brightness(1.4) contrast(1.4);
    mask-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 400 400' xmlns='http://www.w3.org/2000/svg'%3E%3Cfilter i...
    -webkit-mask-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 400 400' xmlns='http://www.w3.org/2000/svg'%3E%3C...
  }
  65% {
    opacity: 0.85;
    filter: brightness(1.2) contrast(1.2);
    mask-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 400 400' xmlns='http://www.w3.org/2000/svg'%3E%3Cfilter i...
    -webkit-mask-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 400 400' xmlns='http://www.w3.org/2000/svg'%3E%3C...
  }
  100% {
    opacity: 1;
    filter: brightness(1) contrast(1);
    mask-image: none;
    -webkit-mask-image: none;
  }
}

.beam-up {
  animation: diffuse-up 420ms cubic-bezier(0.22, 1, 0.36, 1) forwards;
  position: relative;
  overflow: visible;
}

```

```

.beam-down {
  animation: diffuse-down 420ms cubic-bezier(0.22, 1, 0.36, 1) forwards;
  position: relative;
  overflow: visible;
}

@keyframes fade-in {
  from {
    opacity: 0;
  }
  to {
    opacity: 1;
  }
}

.animate-fade-in {
  animation: fade-in 550ms ease-out forwards;
}

select option {
  color: black !important;
  background-color: white !important;
}

input[type="date"],
input[type="time"] {
  color: white !important;
}

input[type="date"]::-webkit-calendar-picker-indicator,
input[type="time"]::-webkit-calendar-picker-indicator {
  filter: invert(1);
}

input[type="time"]::-webkit-datetime-edit,
input[type="date"]::-webkit-datetime-edit {
  color: white;
}

input[type="time"]::-webkit-datetime-edit-fields-wrapper,
input[type="date"]::-webkit-datetime-edit-fields-wrapper {
  color: white;
}

input[type="time"]::-webkit-datetime-edit-text,
input[type="time"]::-webkit-datetime-edit-hour-field,
input[type="time"]::-webkit-datetime-edit-minute-field,
input[type="time"]::-webkit-datetime-edit-ampm-field,
input[type="date"]::-webkit-datetime-edit-text,
input[type="date"]::-webkit-datetime-edit-month-field,
input[type="date"]::-webkit-datetime-edit-day-field,
input[type="date"]::-webkit-datetime-edit-year-field {
  color: white;
}

@keyframes soundWave {
  0%, 100% {
    height: 4px;
  }
  10% {
    height: 28px;
  }
  20% {
    height: 10px;
  }
  35% {
    height: 36px;
  }
  45% {
    height: 6px;
  }
  55% {
    height: 32px;
  }
  65% {
    height: 12px;
  }
  80% {
    height: 40px;
  }
  90% {
    height: 8px;
  }
}

[data-testid^="time-slot-"]:hover,
[data-testid^="course-row-"]:hover,
[data-testid^="other-row-"]:hover {
  filter: brightness(0.92);
  transition: filter 0.1s ease;
}

[data-testid^="all-day-slot-"]:hover {
  filter: brightness(0.92);
  transition: filter 0.1s ease;
}

.hide-scrollbar::-webkit-scrollbar,
.scrollbar-hide::-webkit-scrollbar {
  width: 6px;
  height: 6px;
}

.hide-scrollbar::-webkit-scrollbar-track,
.scrollbar-hide::-webkit-scrollbar-track {
  background: transparent;
}

.hide-scrollbar::-webkit-scrollbar-thumb,
.scrollbar-hide::-webkit-scrollbar-thumb {

```

```

background: transparent;
border-radius: 3px;
transition: background 0.3s;
}
.hide-scrollbar:hover::-webkit-scrollbar-thumb,
.scrollbar-hide:hover::-webkit-scrollbar-thumb,
.hide-scrollbar.is-scrolling::-webkit-scrollbar-thumb,
.scrollbar-hide.is-scrolling::-webkit-scrollbar-thumb {
background: rgba(0, 0, 0, 0.3);
}
.scrollbar-hide {
scrollbar-width: thin;
scrollbar-color: transparent transparent;
}
.scrollbar-hide:hover,
.scrollbar-hide.is-scrolling {
scrollbar-color: rgba(0, 0, 0, 0.3) transparent;
}

.course-code-mono {
font-family: ui-monospace, SFMono-Regular, 'SF Mono', Menlo, Consolas, 'Liberation Mono', monospace;
}

.l3-no-scrollbar {
scrollbar-width: none;
-ms-overflow-style: none;
}
.l3-no-scrollbar::-webkit-scrollbar {
display: none;
}

[data-testid="section-coming-up"] input[type="checkbox"] {
-webkit-appearance: none;
appearance: none;
background: transparent;
border: 1.5px solid white;
border-radius: 2px;
position: relative;
}
[data-testid="section-coming-up"] input[type="checkbox"]:checked {
background: white;
}
[data-testid="section-coming-up"] input[type="checkbox"]:checked:after {
content: '';
position: absolute;
left: 2.5px;
top: 0.5px;
width: 4px;
height: 7px;
border: solid #000;
border-width: 0 1.5px 1.5px 0;
transform: rotate(45deg);
}

[data-testid="select-snooze-value"],
[data-testid="select-snooze-unit"] {
color: #ffffff !important;
-webkit-text-fill-color: #ffffff !important;
opacity: 1 !important;
}
[data-testid="select-snooze-value"] option,
[data-testid="select-snooze-unit"] option {
color: #ffffff !important;
-webkit-text-fill-color: #ffffff !important;
background-color: #2a2a2a !important;
}

@keyframes weatherFall {
0% { transform: translateY(-8px); opacity: 0; }
10% { opacity: 1; }
100% { transform: translateY(calc(100vh)); opacity: 0.3; }
}

@keyframes weatherCloudDrift {
0% { transform: translateX(0); }
50% { transform: translateX(12px); }
100% { transform: translateX(0); }
}

@keyframes weatherLightning {
0%, 94%, 100% { background: transparent; }
95% { background: rgba(255,255,255,0.15); }
96% { background: transparent; }
97% { background: rgba(200,200,255,0.1); }
}

.weather-cloud {
position: absolute;
border-radius: 999px;
background: radial-gradient(ellipse at 50% 90%, rgba(210,218,233,0.65) 0%, rgba(190,200,220,0.4) 50%, transparent 7...);
animation: weatherCloudDrift 12s ease-in-out infinite;
pointer-events: none;
filter: blur(0.5px);
}
.weather-cloud::before,
.weather-cloud::after {
content: '';
position: absolute;
border-radius: 999px;
pointer-events: none;
}
.weather-cloud::before {
width: 55%;
height: 130%;
top: -45%;
left: 18%;
background: radial-gradient(ellipse at 50% 60%, rgba(215,222,238,0.7) 0%, rgba(200,210,228,0.45) 45%, transparent 7...);
}

```

```

.weather-cloud::after {
  width: 45%;
  height: 110%;
  top: -30%;
  left: 48%;
  background: radial-gradient(ellipse at 50% 60%, rgba(205,215,232,0.6) 0%, rgba(195,205,225,0.4) 45%, transparent 70%);
}

.weather-lightning {
  animation: weatherLightning 6s ease-in-out infinite;
  pointer-events: none;
}

@keyframes tickerScroll {
  0% { transform: translate3d(var(--ticker-start), 0, 0); }
  100% { transform: translate3d(var(--ticker-end), 0, 0); }
}
@keyframes ticker {
  0% { transform: translateX(0); }
  100% { transform: translateX(-50%); }
}
@keyframes tickerAlertBlink {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.3; }
}
.news-ticker-scroll {
  will-change: transform;
  backface-visibility: hidden;
  -webkit-backface-visibility: hidden;
}

.news-ticker-scroll:hover {
  animation-play-state: paused !important;
}

[data-testid="announcement-ticker"]:hover .animate-ticker,
[data-testid="announcement-ticker"]:hover > div:nth-child(3) {
  animation-play-state: paused !important;
}

@keyframes aas-shimmer-pulse {
  0%, 92% { transform: scale(1); box-shadow: none; }
  94% { transform: scale(1.25); box-shadow: 0 0 6px rgba(245,158,11,0.6), 0 0 12px rgba(245,158,11,0.3); }
  96% { transform: scale(1); box-shadow: 0 0 4px rgba(245,158,11,0.4); }
  98% { transform: scale(1.15); box-shadow: 0 0 8px rgba(245,158,11,0.5), 0 0 14px rgba(245,158,11,0.25); }
  100% { transform: scale(1); box-shadow: none; }
}

.aas-unchecked-pulse {
  animation: aas-shimmer-pulse 10s ease-in-out infinite;
}

@keyframes countdown-due-today-pulse {
  0%, 100% {
    box-shadow: 0 8px 32px rgba(0,0,0,0.08), 0 2px 8px rgba(0,0,0,0.04), inset 0 1px 0 rgba(255,255,255,0.6), inset 0 ...
    border-color: rgba(255,255,255,0.5);
    transform: scale(1);
  }
  50% {
    box-shadow: 0 0 16px rgba(255,60,60,0.5), 0 0 32px rgba(255,60,60,0.25), inset 0 1px 0 rgba(255,255,255,0.6), ins...
    border-color: rgba(255,90,90,0.75);
    transform: scale(1.03);
  }
}

.countdown-due-today-pulse {
  animation: countdown-due-today-pulse 3s ease-in-out infinite;
  transform-origin: center center;
}

@keyframes weather-alert-border-pulse {
  0%, 100% {
    box-shadow: inset 0 0 0 1.5px rgba(255,50,50,0.3);
  }
  50% {
    box-shadow: inset 0 0 0 1.5px rgba(255,50,50,0.9), 0 0 8px rgba(255,50,50,0.3);
  }
}

.weather-alert-border-pulse {
  animation: weather-alert-border-pulse 3s ease-in-out infinite;
}

.section-dim > *:not(.no-dim) {
  opacity: 0.45;
}
.section-dim-tr > td:not(.no-dim) {
  opacity: 0.45;
}

```

`client/src/lib/queryClient.ts`

```

import { QueryClient, QueryFunction } from "@tanstack/react-query";

async function throwIfResNotOk(res: Response) {
  if (!res.ok) {
    const text = (await res.text()) || res.statusText;
    throw new Error(`${res.status}: ${text}`);
  }
}

export async function apiRequest(
  method: string,
  url: string,

```

```

    data?: unknown | undefined,
  ): Promise<Response> {
    const res = await fetch(url, {
      method,
      headers: data ? { "Content-Type": "application/json" } : {},
      body: data ? JSON.stringify(data) : undefined,
      credentials: "include",
    });

    await throwIfResNotOk(res);
    return res;
  }

  type UnauthorizedBehavior = "returnNull" | "throw";
  export const getQueryFn: <T>(options: {
    on401: UnauthorizedBehavior;
  }) => QueryFunction<T> =
    ({ on401: unauthorizedBehavior }) =>
      async ({ queryKey }) => {
        const res = await fetch(queryKey.join("/") as string, {
          credentials: "include",
        });

        if (unauthorizedBehavior === "returnNull" && res.status === 401) {
          return null;
        }

        await throwIfResNotOk(res);
        return await res.json();
      };

  export const queryClient = new QueryClient({
    defaultOptions: {
      queries: {
        queryFn: getQueryFn({ on401: "throw" }),
        refetchInterval: false,
        refetchOnWindowFocus: false,
        staleTime: Infinity,
        retry: false,
      },
      mutations: {
        retry: false,
      },
    },
  });

```

`client/src/lib/utils.ts`

```

import { clsx, type ClassValue } from "clsx"
import { twMerge } from "tailwind-merge"

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}

```

`client/src/main.tsx`

```

import { createRoot } from "react-dom/client";
import App from "../App";
import "../index.css";

const originalFetch = window.fetch.bind(window);
window.fetch = (input: RequestInfo | URL, init?: RequestInit) => {
  const token = localStorage.getItem('uni_cal_token');
  const headers: Record<string, string> = {};
  if (token) {
    headers['Authorization'] = `Bearer ${token}`;
  }
  return originalFetch(input, {
    credentials: 'include',
    ...init,
    headers: {
      ...headers,
      ...(init?.headers || {}),
    },
  });
};

function reportError(message: string, stack?: string) {
  try {
    originalFetch('/api/client-error', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        message,
        stack: stack || '',
        userAgent: navigator.userAgent,
        url: window.location.href,
        timestamp: new Date().toISOString()
      }),
      credentials: 'include'
    }).catch(() => {});
  } catch {}
}

window.addEventListener('error', (event) => {
  if (event.message && typeof event.message === 'string' && event.message.includes('ResizeObserver')) {
    event.stopImmediatePropagation();
    event.preventDefault();
    return;
  }
});

```

```

    }
    if (event.error) {
      console.error('[RUNTIME ERROR CAUGHT]', event.error, event.error?.message, event.error?.stack);
    } else {
      console.error('[RUNTIME ERROR NO-ERROR-OBJ]', event.message, event.filename, event.lineno);
    }
  });

window.addEventListener('unhandledrejection', (event) => {
  console.error('[UNHANDLED REJECTION]', event.reason, typeof event.reason, event.reason?.message, event.reason?.stack);
});

window.onerror = function(message, source, lineno, colno, error) {
  if (typeof message === 'string' && message.includes('ResizeObserver')) return true;
  const msg = `${message} at ${source}:${lineno}:${colno}`;
  reportError(msg, error?.stack);
  return false;
};

window.onunhandledrejection = function(event) {
  const msg = `Unhandled Promise: ${event.reason}`;
  reportError(msg, event.reason?.stack);
};

let knownVersion: string | null = null;
async function checkVersion() {
  try {
    const params = new URLSearchParams(window.location.search);
    if (params.get('followOnly') === 'true') return;
    const resp = await originalFetch('/api/version');
    if (resp.ok) {
      const data = await resp.json();
      if (knownVersion && data.version !== knownVersion) {
        window.location.reload();
        return;
      }
      knownVersion = data.version;
    }
  } catch {}
}
checkVersion();
setInterval(checkVersion, 30000);

createRoot(document.getElementById("root")!).render(<App />);

```

`client/src/pages/code-checker.tsx`

```

import { useState } from "react";
import { useLocation } from "wouter";

const LANGUAGES = [
  "JavaScript", "TypeScript", "Python", "HTML", "CSS", "Java", "C", "C++", "C#", "Go", "Rust", "PHP", "Ruby", "SQL", ...
];

export default function CodeCheckerPage() {
  const [code, setCode] = useState("");
  const [language, setLanguage] = useState("JavaScript");
  const [result, setResult] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);
  const [emailing, setEmailing] = useState(false);
  const [emailSent, setEmailSent] = useState(false);
  const [, setLocation] = useLocation();

  const checkCode = async () => {
    if (!code.trim()) return;
    setLoading(true);
    setResult(null);
    setEmailSent(false);
    try {
      const res = await fetch("/api/code-checker/analyze", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ code, language }),
      });
      const data = await res.json();
      if (data.error) throw new Error(data.error);
      setResult(data.analysis);
    } catch (err: any) {
      setResult("Error: " + (err.message || "Failed to analyze code"));
    } finally {
      setLoading(false);
    }
  };

  const emailResults = async () => {
    if (!result) return;
    setEmailing(true);
    try {
      const res = await fetch("/api/code-checker/email", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ code, language, analysis: result }),
      });
      const data = await res.json();
      if (data.error) throw new Error(data.error);
      setEmailSent(true);
    } catch (err: any) {
      alert("Failed to send email: " + (err.message || "Unknown error"));
    } finally {
      setEmailing(false);
    }
  };

  return (

```

```

<div style={{ minHeight: "100vh", background: "linear-gradient(135deg, #0f172a 0%, #1e293b 50%, #0f172a 100%)", c...
<div style={{ maxWidth: "900px", margin: "0 auto", padding: "30px 20px" }}>
  <div style={{ display: "flex", alignItems: "center", justifyContent: "space-between", marginBottom: "24px" }}>
    <h1 style={{ fontSize: "24px", fontWeight: 700, color: "#f8fafc" }} data-testid="text-page-title">
      Code Checker
    </h1>
    <button
      onClick={() => setLocation("/") }
      style={{ background: "rgba(255,255,255,0.1)", border: "1px solid rgba(255,255,255,0.2)", borderRadius: "8...
      data-testid="button-back-dashboard"
    >
      Back to Dashboard
    </button>
  </div>

  <div style={{ display: "flex", gap: "12px", marginBottom: "16px", alignItems: "center" }}>
    <label style={{ fontSize: "13px", color: "#94a3b8" }}>Language:</label>
    <select
      value={language}
      onChange={(e) => setLanguage(e.target.value)}
      style={{ background: "#1e293b", border: "1px solid rgba(255,255,255,0.2)", borderRadius: "6px", padding: ...
      data-testid="select-language"
    >
      {LANGUAGES.map((lang) => (
        <option key={lang} value={lang}>{lang}</option>
      ))}
    </select>
  </div>

  <textarea
    value={code}
    onChange={(e) => setCode(e.target.value)}
    placeholder="Paste your code here..."
    style={{
      width: "100%",
      minHeight: "280px",
      background: "#0f172a",
      border: "1px solid rgba(255,255,255,0.15)",
      borderRadius: "10px",
      padding: "16px",
      color: "#e2e8f0",
      fontFamily: "'Fira Code', 'Cascadia Code', 'JetBrains Mono', Consolas, monospace",
      fontSize: "13px",
      lineHeight: "1.6",
      resize: "vertical",
      outline: "none",
    }}
    data-testid="input-code"
  />

  <div style={{ display: "flex", gap: "12px", marginTop: "16px" }}>
    <button
      onClick={checkCode}
      disabled={loading || !code.trim()}
      style={{
        background: loading ? "#334155" : "linear-gradient(135deg, #3b82f6, #2563eb)",
        border: "none",
        borderRadius: "8px",
        padding: "10px 24px",
        color: "fff",
        fontSize: "14px",
        fontWeight: 600,
        cursor: loading || !code.trim() ? "not-allowed" : "pointer",
        opacity: loading || !code.trim() ? 0.6 : 1,
      }}
      data-testid="button-check-code"
    >
      {loading ? "Analyzing..." : "Check for Errors"}
    </button>

    {result && (
      <button
        onClick={emailResults}
        disabled={emailing}
        style={{
          background: emailSent ? "#16a34a" : emailing ? "#334155" : "linear-gradient(135deg, #8b5cf6, #7c3aed)",
          border: "none",
          borderRadius: "8px",
          padding: "10px 24px",
          color: "fff",
          fontSize: "14px",
          fontWeight: 600,
          cursor: emailing ? "not-allowed" : "pointer",
          opacity: emailing ? 0.6 : 1,
        }}
        data-testid="button-email-results"
      >
        {emailSent ? "Sent!" : emailing ? "Sending..." : "Email Results to Me"}
      </button>
    )}
  </div>

  {result && (
    <div
      style={{
        marginTop: "24px",
        background: "rgba(255,255,255,0.05)",
        border: "1px solid rgba(255,255,255,0.1)",
        borderRadius: "10px",
        padding: "20px",
      }}
      data-testid="text-analysis-result"
    >
      <h2 style={{ fontSize: "16px", fontWeight: 600, color: "#f8fafc", marginBottom: "12px" }}>Analysis</h2>
      <pre style={{
        whiteSpace: "pre-wrap",
        wordWrap: "break-word",
        fontFamily: "ui-sans-serif, system-ui, sans-serif",

```



```
`client/src/pages/dashboard.tsx`
```

```

Contact,
ClipboardCheck,
Calendar,
CalendarClock,
Clock,
Plus,
Download,
RefreshCw,
Bell,
BellOff,
CalendarDays,
ChevronLeft,
ChevronRight,
ChevronDown,
ChevronUp,
ChevrnsUp,
X,
Link,
Paperclip,
Upload,
Camera,
Loader2,
Play,
Square,
MinusCircle,
PlusCircle,
Folder,
FolderOpen,
FolderPlus,
Trash2,
Sun,
Home,
Cloud,
ArrowLeft,
Repeat2,
Settings,
Timer,
Pause,
SkipForward,
SkipBack,
RotateCcw,
RotateCw,
Gauge,
Menu,
LayoutGrid,
User,
Palette,
Externallink,
Volume2,
VolumeX,
CheckSquare,
ClipboardList,
Undo2,
Redo2,
Radio,
Minus,
ArrowDownToLine,
Music2,
ListChecks,
ZoomIn,
ZoomOut,
MoveUpRight,
TrendingUp,
TrendingDown,
Pencil,
StickyNote,
Grip,
GripVertical,
GripHorizontal,
CheckCircle2,
Check,
ListTodo,
Link2,
Mail,
Smartphone,
Share2,
Share,
Copy,
Eye,
EyeOff,
Lock,
AlertCircle,
Plane,
List,
Search,
Replace,
Maximize,
Maximize2,
Minimize2,
Cast,
Monitor,
Speaker,
Headphones,
Sun as SunIcon,
Moon as MoonIcon,
Mic,
MicOff,
Flag,
} from "lucide-react";
import { Link as RouterLink, useLocation } from "wouter";
import { useAccessMode } from "@components/access-gate";
import type { Task, SemesterSettings, Subtask, Project, TaskLink } from "@shared/schema";
import { TASK_TYPES, COURSES, getWeekNumber, getWeekDates, REMINDER_OPTIONS, DEFAULT_REMINDER_1, DEFAULT_REMINDER_2, ...
import type { CourseWeekMapping } from "@shared/schema";
import { getUpcomingSemesterToConfirm, getNextSemesterByStartDate, FUTURE_SEMESTER_SCHEDULE, type FutureSemesterDates...
import { LIBERAL_STUDIES_COURSES, OPEN_ELECTIVE_COURSES, POG_COURSES, getCoursesForLevel, type ElectiveCourse } from ...
import { format, addDays, subDays, addWeeks, startOfMonth, endOfMonth, eachDayOfInterval, isSameDay, isSameMonth, sta...

```

```

const iconMap: Record<string, React.ComponentType<{ className?: string }>> = {
  reading: BookOpen,
  module: Layers,
  essay: FileText,
  project: FolderKanban,
  discussion: MessageSquare,
  poll: Vote,
  exam: GraduationCap,
  quiz: ClipboardCheck,
  reminder: Bell,
};

const typeColors: Record<string, string> = {
  reading: "bg-blue-500/20 text-blue-600 dark:text-blue-400",
  module: "bg-purple-500/20 text-purple-600 dark:text-purple-400",
  essay: "bg-orange-500/20 text-orange-600 dark:text-orange-400",
  project: "bg-green-500/20 text-green-600 dark:text-green-400",
  discussion: "bg-pink-500/20 text-pink-600 dark:text-pink-400",
  poll: "bg-cyan-500/20 text-cyan-600 dark:text-cyan-400",
  exam: "bg-red-500/20 text-red-600 dark:text-red-400",
  quiz: "bg-amber-500/20 text-amber-600 dark:text-amber-400",
  reminder: "bg-teal-500/20 text-teal-600 dark:text-teal-400",
};

const courseColors: Record<string, { bg: string; border: string; text: string; dot: string; prepBg: string; prepBorde...
  "CPPA101": { bg: "bg-blue-50 dark:bg-blue-900/30", border: "border-blue-400", text: "text-blue-600 dark:text-blue-3...",
  "CPPA102": { bg: "bg-blue-50 dark:bg-blue-900/30", border: "border-blue-400", text: "text-blue-600 dark:text-blue-3...",
  "CPPA120": { bg: "bg-blue-50 dark:bg-blue-900/30", border: "border-blue-400", text: "text-blue-600 dark:text-blue-3...",
  "CPPA121": { bg: "bg-blue-50 dark:bg-blue-900/30", border: "border-blue-400", text: "text-blue-600 dark:text-blue-3...",
  "CPPA122": { bg: "bg-green-50 dark:bg-green-900/30", border: "border-green-400", text: "text-green-600 dark:text-gr...",
  "CPPA125": { bg: "bg-blue-50 dark:bg-blue-900/30", border: "border-blue-400", text: "text-blue-600 dark:text-blue-3...",
  "CPPA235": { bg: "bg-amber-50 dark:bg-amber-900/30", border: "border-amber-400", text: "text-amber-700 dark:text-am...",
  "CFNF400": { bg: "bg-pink-50 dark:bg-pink-900/30", border: "border-pink-400", text: "text-pink-600 dark:text-pink-3...",
  "CASL101": { bg: "bg-indigo-50 dark:bg-indigo-900/30", border: "border-indigo-400", text: "text-indigo-600 dark:tex...",
  "CGCM738": { bg: "bg-orange-50 dark:bg-orange-900/30", border: "border-orange-400", text: "text-orange-600 dark:tex...",
  "CECN210": { bg: "bg-emerald-50 dark:bg-emerald-900/30", border: "border-emerald-400", text: "text-emerald-600 dark...",
  "CPHL110": { bg: "bg-blue-50 dark:bg-blue-900/30", border: "border-blue-500", text: "text-blue-600 dark:text-blue-3...",
  "CHIS105": { bg: "bg-red-50 dark:bg-red-900/30", border: "border-red-400", text: "text-red-600 dark:text-red-300", ...
};

// Display name mapping for course row labels (defaults, overridden by localStorage)
const defaultCourseDisplayNames: Record<string, string> = {
  "CPPA122": "CPPA122-LP",
  "CFNF400": "CFNF400-HS",
  "CASL101": "CASL101 ASL",
};

// Helper function to get display name for course row labels (uses dynamic state)
let _courseDisplayNames: Record<string, string> = { ...defaultCourseDisplayNames };
const getCourseRowDisplayName = (courseName: string): string => {
  const courseCode = courseName.split('-')[0];
  if (_courseDisplayNames[courseCode]) {
    return _courseDisplayNames[courseCode];
  }
  return courseName;
};

// Course folder configuration for sidebar hamburger menus
const SIDEBAR_COURSES = [
  { id: "cpa122", name: "CPPA122", color: "text-green-400", hoverBg: "hover:bg-green-400/20" },
  { id: "cfnf400", name: "CFNF400", color: "text-pink-400", hoverBg: "hover:bg-pink-400/20" },
  { id: "casl101", name: "CASL101", color: "text-indigo-400", hoverBg: "hover:bg-indigo-400/20" },
];

const FOLDER_TYPES = [
  { id: "module", name: "Module" },
  { id: "reading", name: "Reading" },
];

// Helper function to convert 24-hour time to 12-hour format
const formatTimeTo12Hour = (time24: string): string => {
  const [hours, minutes] = time24.split(':').map(Number);
  const period = hours >= 12 ? 'PM' : 'AM';
  const hours12 = hours === 0 ? 12 : hours > 12 ? hours - 12 : hours;
  return `${hours12}:${minutes.toString().padStart(2, '0')} ${period}`;
};

// Speakers list for media controls
const SPEAKERS = [
  { id: "browser_tts", name: "Bluetooth" },
  { id: "media_player.byhome", name: "Apartment" },
  { id: "media_player.cat_wash", name: "Cat Wash" },
  { id: "media_player.cat_wr", name: "Cat Washroom Speakers" },
  { id: "media_player.bathroom_speaker", name: "Nest (Cat Washroom)" },
  { id: "media_player.echo_cat_left_am", name: "Cat Washroom Left" },
  { id: "media_player.echo_cat_right_am", name: "Cat Washroom Right" },
  { id: "media_player.echo_cat_washroom_middle", name: "Cat Washroom Middle" },
  { id: "media_player.echo_closet_am", name: "Closet" },
  { id: "media_player.echo_lr_couch_r_am", name: "Echo Corner" },
  { id: "media_player.echo_hallway_entrance_am", name: "Hallway Entrance" },
  { id: "media_player.echo_king_l_am", name: "King Left" },
  { id: "media_player.echo_king_r_am", name: "King Right" },
  { id: "media_player.echo_king_tv_am", name: "King TV" },
  { id: "media_player.echo_kitchen_cupboards_left_am", name: "Kitchen Cupboards Left" },
  { id: "media_player.echo_kitchen_cupboards_r_am", name: "Kitchen Cupboards Right" },
  { id: "media_player.echo_kitchen_fridge_am", name: "Kitchen Fridge" },
  { id: "media_player.echo_kitchen_hutch_am", name: "Kitchen Hutch" },
  { id: "media_player.echo_kitchen_island_corner_am", name: "Kitchen Island Corner" },
  { id: "media_player.echo_kitchen_studio_black_am", name: "Kitchen Studio Black" },
  { id: "media_player.echo_lr_couch_l_am", name: "Living Room Couch Left" },
  { id: "media_player.echo_lr_hub_am", name: "Living Room Hub" },
  { id: "media_player.echo_lr_studio_white_am", name: "Living Room Studio White" },
  { id: "media_player.echo_lr_tv_shelf_am", name: "Living Room TV Shelf" },
  { id: "media_player.echo_queen_balcony_am", name: "Queen Balcony" },
  { id: "media_player.echo_queen_bed_l_am", name: "Queen Bed Left" },
  { id: "media_player.echo_queen_bed_r_am", name: "Queen Bed Right" },
  { id: "media_player.echo_show_pug_am", name: "Echo Show Pug" },
  { id: "media_player.everywhere_2", name: "Everywhere" },
];

```

```

    { id: "media_player.hallway", name: "Hallway" },
    { id: "media_player.king_bedroom", name: "King Bedroom" },
    { id: "media_player.queen_bedroom", name: "Queen Bedroom" },
  ];

  interface WeekInfo {
    weekNumber: number;
    startDate: string;
    endDate: string;
    taskCount: number;
  }

  function getPointerXY(e: React.MouseEvent | React.TouchEvent | MouseEvent | TouchEvent): { clientX: number; clientY: ...
    if ('touches' in e) {
      const t = e.touches[0] || (e as TouchEvent).changedTouches?.[0];
      return t ? { clientX: t.clientX, clientY: t.clientY } : { clientX: 0, clientY: 0 };
    }
    return { clientX: (e as MouseEvent).clientX, clientY: (e as MouseEvent).clientY };
  }

  const _etDateKeyCache = new Map<number, string>();
  function _etDateKey(date: Date): string {
    const ms = date.getTime();
    const cached = _etDateKeyCache.get(ms);
    if (cached) return cached;
    const key = date.toLocaleDateString('en-CA', { timeZone: 'America/Toronto' });
    if (_etDateKeyCache.size > 2000) _etDateKeyCache.clear();
    _etDateKeyCache.set(ms, key);
    return key;
  }

  const _toETCache = new Map<number, Date>();
  function toET(date: Date): Date {
    const ms = date.getTime();
    const cached = _toETCache.get(ms);
    if (cached) return new Date(cached.getTime());
    const s = date.toLocaleString('en-US', { timeZone: 'America/Toronto' });
    const result = new Date(s);
    if (_toETCache.size > 2000) _toETCache.clear();
    _toETCache.set(ms, result);
    return new Date(result.getTime());
  }

  function getETHours(date: Date): number {
    return toET(date).getHours();
  }

  function getETMinutes(date: Date): number {
    return toET(date).getMinutes();
  }

  function startOfDayET(date: Date): Date {
    const et = toET(date);
    et.setHours(0, 0, 0, 0);
    return et;
  }

  function isSameDayET(a: Date, b: Date): boolean {
    return _etDateKey(a) === _etDateKey(b);
  }

  function getETDateString(date: Date): string {
    return _etDateKey(date);
  }

  const TICKER_LOGO_MAP: Record<string, { src: string; height: number }> = {
    CNN: { src: cnnLogoPath, height: 33 },
    CBC: { src: cbcLogoPath, height: 78 },
    CTV: { src: ctvLogoPath, height: 42 },
    Global: { src: globalLogoPath, height: 42 },
    MSNBC: { src: msnbcLogoPath, height: 68 },
    Politico: { src: politicoLogoPath, height: 57 },
    'Raw Story': { src: rawStoryLogoPath, height: 64 },
    'ABC News': { src: abcNewsLogoPath, height: 56 },
    'BBC': { src: bbcNewsLogoPath, height: 64 },
    'Fox News': { src: foxNewsLogoPath, height: 72 },
  };

  function NewsTickerPortal({ headlines }: { headlines: Array<{ title: string; link: string; source: string; publishedA...
    const containerRef = useRef<HTMLDivElement | null>(null);
    const rootRef = useRef<any>(null);
    const prevHeadlinesKeyRef = useRef<string>('');
    const renderedRef = useRef(false);

    useEffect(() => {
      if (!containerRef.current) {
        const el = document.createElement('div');
        el.id = 'news-ticker-root';
        el.style.position = 'fixed';
        el.style.bottom = '0';
        el.style.left = '0';
        el.style.right = '0';
        el.style.zIndex = '9998';
        el.style.height = '0';
        el.style.overflow = 'visible';
        document.body.appendChild(el);
        containerRef.current = el;
      }
      return () => {
        if (containerRef.current) {
          containerRef.current.remove();
          containerRef.current = null;
          rootRef.current = null;
          renderedRef.current = false;
        }
      };
    }, []);
  }

```

```

useEffect(() => {
  if (!containerRef.current || headlines.length === 0) return;
  const headlinesKey = headlines.map(h => `${h.source}::${h.title}`).join('|');
  if (renderedRef.current && headlinesKey === prevHeadlinesKeyRef.current) return;
  prevHeadlinesKeyRef.current = headlinesKey;
  renderedRef.current = true;
  const html = `<div class="fixed left-0 right-0 z-[9998] overflow-hidden flex" style="bottom:0;height:38px;backgro...
    if (item.source === '_ALERT_') {
      const safeTitle = item.title.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;').replace(/"/g, '&q...
      return `<span class="inline-flex items-center gap-1.5 mx-4" style="animation:tickerAlertBlink 1s ease-in-out ...
    }
    if (item.source === '_FORECAST_' || item.source === '_FORECAST_NOSEP_') {
      const forecastHtml = item.title.replace(/(<b>[^<*</b>:?(?:Toronto Forecast|3-Day Forecast|Forecast Brief:...
      const sep = item.source === '_FORECAST_' ? `<span class="text-white/20 mx-2">|</span>` : '';
      return `<span class="inline-flex items-center gap-1.5 mx-4" data-testid="weather-forecast-${i}"><span class="...
    }
    const logoInfo = TICKER_LOGO_MAP[item.source];
    const logoHtml = logoInfo
      ? `${item.source}</span>`;
    const safeTitle = item.title.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;').replace(/"/g, '&quo...
    let timeAgoHtml = '';
    if (item.publishedAt) {
      const diff = Date.now() - new Date(item.publishedAt).getTime();
      const mins = Math.floor(diff / 60000);
      if (mins >= 0 && mins <= 4320) {
        const ago = mins < 60 ? `${mins}m` : mins < 1440 ? `${Math.floor(mins / 60)}h` : `${Math.floor(mins / 1440)}...
        timeAgoHtml = `<span class="text-[14px]" style="color:rgba(255,255,255,0.6);margin-left:4px">${ago}</span>`;
      }
    }
    return `<a href="${item.link}" target="_blank" rel="noopener noreferrer" class="inline-flex items-center gap-1....
  }).join('')</div></div></div>`;
  containerRef.current.innerHTML = html;
  const applyTickerAnimation = () => {
    const scrollEl = containerRef.current?.querySelector('.news-ticker-scroll') as HTMLElement | null;
    if (!scrollEl) return;
    const parentWidth = scrollEl.parentElement?.clientWidth || window.innerWidth;
    const startContinuousScroll = () => {
      scrollEl.style.transform = '';
      const contentWidth = scrollEl.scrollWidth;
      const totalTravel = parentWidth + contentWidth;
      const speed = 65;
      const dur = totalTravel / speed;
      scrollEl.style.setProperty('--ticker-start', `${parentWidth}px`);
      scrollEl.style.setProperty('--ticker-end', `-${contentWidth}px`);
      scrollEl.style.animation = `tickerScroll ${dur}s linear infinite`;
    };
    startContinuousScroll();
  };
  const imgs = containerRef.current.querySelectorAll('img');
  if (imgs.length > 0) {
    let loaded = 0;
    const onLoad = () => { loaded++; if (loaded >= imgs.length) requestAnimationFrame(applyTickerAnimation); };
    imgs.forEach(img => { if (img.complete) { loaded++; } else { img.addEventListener('load', onLoad); img.addEvent...
    if (loaded >= imgs.length) requestAnimationFrame(applyTickerAnimation);
    setTimeout(applyTickerAnimation, 500);
  } else {
    requestAnimationFrame(applyTickerAnimation);
  }
}, [headlines]);

return null;
}

const PrioritySelect = memo(function PrioritySelect({ priorityKey, initialValue, totalInSem, draftRef, courseCode, us...
const [val, setVal] = useState(initialValue);
useEffect(() => { setVal(initialValue); }, [initialValue]);
const hasSuffix = suffix === 'A' || suffix === 'B';
return (
  <select
    className="text-[11px] font-semibold text-white bg-white/10 rounded px-1 py-0.5 border border-white/20 focus:ou...
    style={{ width: hasSuffix ? '30px' : '22px', minWidth: hasSuffix ? '30px' : '22px', WebkitAppearance: 'none', M...
    value={val}
    onClick={(e) => e.stopPropagation()}
    onChange={(e) => {
      e.stopPropagation();
      const v = parseInt(e.target.value, 10);
      setVal(v);
      draftRef.current = { ...draftRef.current, [priorityKey]: v };
      onPriorityChange(priorityKey, v);
    }}
    data-testid={`select-priority-${courseCode}${hasSuffix ? `-${suffix}` : ''}`}
  >
    <option value={0}></option>
    {Array.from({ length: totalInSem }, (_, i) => {
      const n = i + 1;
      const taken = usedValues.includes(n) && val !== n;
      return <option key={n} value={n} disabled={taken} style={taken ? { color: '#555' } : {}}>{hasSuffix ? `${n}${...
    })}
  </select>
);
});

export default function Dashboard() {
  const { toast } = useToast();

  useEffect(() => {
    const PERF_LOG_DURATION = 5 * 60 * 1000;
    const perfStart = Date.now();
    const pendingClicks = new Map<string, number>();

    const clickHandler = (e: MouseEvent) => {
      if (Date.now() - perfStart > PERF_LOG_DURATION) return;
      const target = e.target as HTMLElement;
      const cn = typeof target.className === 'string' ? target.className : (target.className as any)?.baseVal || '';
      const id = target.getAttribute('data-testid') || target.id || target.tagName + '.' + cn.split(' ')[0];
      const clickTime = performance.now();

```

```

    pendingClicks.set(id, clickTime);
    console.log(`[PERF] CLICK on "${id}" at ${clickTime.toFixed(0)}ms`);
    requestAnimationFrame(() => {
      requestAnimationFrame(() => {
        const responseTime = performance.now() - clickTime;
        console.log(`[PERF] RESPONSE for "${id}": ${responseTime.toFixed(0)}ms`);
        pendingClicks.delete(id);
      });
    });
  });
};

const longTaskObserver = new PerformanceObserver((list) => {
  if (Date.now() - perfStart > PERF_LOG_DURATION) return;
  for (const entry of list.getEntries()) {
    console.log(`[PERF] LONG TASK: ${entry.duration.toFixed(0)}ms (start: ${entry.startTime.toFixed(0)}ms)`);
  }
});

let lastTouchStart: { id: string; time: number; x: number; y: number } | null = null;

const touchStartHandler = (e: TouchEvent) => {
  if (Date.now() - perfStart > PERF_LOG_DURATION) return;
  const touch = e.touches[0];
  if (!touch) return;
  const target = e.target as HTMLElement;
  const cn = typeof target.className === 'string' ? target.className : (target.className as any)?.baseVal || '';
  const id = target.getAttribute('data-testid') || target.id || target.tagName + '.' + cn.split(' ')[0];
  const now = performance.now();
  lastTouchStart = { id, time: now, x: touch.clientX, y: touch.clientY };
  console.log(`[PERF] TOUCH START on "${id}" at ${now.toFixed(0)}ms (${touch.clientX.toFixed(0)}, ${touch.clientY}...`);
};

const touchEndHandler = (e: TouchEvent) => {
  if (Date.now() - perfStart > PERF_LOG_DURATION) return;
  const now = performance.now();
  const target = e.target as HTMLElement;
  const cn = typeof target.className === 'string' ? target.className : (target.className as any)?.baseVal || '';
  const id = target.getAttribute('data-testid') || target.id || target.tagName + '.' + cn.split(' ')[0];
  const duration = lastTouchStart ? (now - lastTouchStart.time).toFixed(0) : '?';
  console.log(`[PERF] TOUCH END on "${id}" at ${now.toFixed(0)}ms (held ${duration}ms)`);
  if (lastTouchStart) {
    requestAnimationFrame(() => {
      requestAnimationFrame(() => {
        const responseTime = performance.now() - now;
        console.log(`[PERF] TOUCH RESPONSE for "${id}": ${responseTime.toFixed(0)}ms after touchend`);
      });
    });
  }
};

const touchMoveHandler = (e: TouchEvent) => {
  if (Date.now() - perfStart > PERF_LOG_DURATION) return;
  if (!lastTouchStart) return;
  const touch = e.touches[0];
  if (!touch) return;
  const dx = Math.abs(touch.clientX - lastTouchStart.x);
  const dy = Math.abs(touch.clientY - lastTouchStart.y);
  if (dx > 10 || dy > 10) {
    console.log(`[PERF] TOUCH MOVE delta (${dx.toFixed(0)}, ${dy.toFixed(0)}) from "${lastTouchStart.id}"`);
    lastTouchStart = null;
  }
};

document.addEventListener('click', clickHandler, true);
document.addEventListener('touchstart', touchStartHandler, { capture: true, passive: true });
document.addEventListener('touchend', touchEndHandler, { capture: true, passive: true });
document.addEventListener('touchmove', touchMoveHandler, { capture: true, passive: true });
try {
  longTaskObserver.observe({ entryTypes: ['longtask'] });
} catch (e) {}

const cleanupTimer = setTimeout(() => {
  document.removeEventListener('click', clickHandler, true);
  document.removeEventListener('touchstart', touchStartHandler, true);
  document.removeEventListener('touchend', touchEndHandler, true);
  document.removeEventListener('touchmove', touchMoveHandler, true);
  try { longTaskObserver.disconnect(); } catch (e) {}
  console.log(`[PERF] Performance logging ended after 5 minutes.`);
}, PERF_LOG_DURATION);

console.log(`[PERF] Performance logging STARTED. Will run for 5 minutes.`);

return () => {
  document.removeEventListener('click', clickHandler, true);
  document.removeEventListener('touchstart', touchStartHandler, true);
  document.removeEventListener('touchend', touchEndHandler, true);
  document.removeEventListener('touchmove', touchMoveHandler, true);
  try { longTaskObserver.disconnect(); } catch (e) {}
  clearTimeout(cleanupTimer);
};
}, []);

// Mobile detection
const [isMobile, setIsMobile] = useState(() => window.innerWidth < 640);

useEffect(() => {
  const handleResize = () => setIsMobile(window.innerWidth < 640);
  window.addEventListener('resize', handleResize);
  return () => window.removeEventListener('resize', handleResize);
}, []);

const [selectedWeek, setSelectedWeek] = useState<number>(2);
const [openCourseDropdown, setOpenCourseDropdown] = useState<string | null>(null);
const [selectedDate, setSelectedDate] = useState<Date | null>(null);
const [currentMonth, setCurrentMonth] = useState(new Date());
const [calendarView, setCalendarView] = useState<"week" | "month">("week");
const [isAddDialogOpen, setIsAddDialogOpen] = useState(false);
const [pendingSleepTask, setPendingSleepTask] = useState<{ day: Date; hour: number; minutes: number } | null>(null);

```

```

const [newTaskType, setNewTaskType] = useState<string>("module");
const [initialStartTime, setInitialStartTime] = useState<string>("");
const [isQuickAddOpen, setIsQuickAddOpen] = useState(false);
const [isAddChooserOpen, setIsAddChooserOpen] = useState(false);
const [showQuickAddCloseConfirm, setShowQuickAddCloseConfirm] = useState(false);
const [quickAddStep, setQuickAddStep] = useState(0);
const [quickAddData, setQuickAddData] = useState({
  type: "",
  title: "",
  courseName: "",
  dueDate: "",
  dueDateHour: "18",
  dueDateMinute: "00",
  prepDays: 0,
  priority: "medium",
  description: "",
  eventStartTime: "",
  eventEndTime: "",
  reminder1: DEFAULT_REMINDER_1,
  reminder2: DEFAULT_REMINDER_2,
  reminder3: null as number | null,
  reminder4: null as number | null,
  attachments: [] as string[],
  pasteUrl: "",
  notes: "",
  referenceLink: "",
  subtasks: [] as { title: string; completed: boolean }[],
  subtaskInput: "",
  projectId: null as number | null,
  repeatType: "none" as string,
  repeatInterval: null as number | null,
  repeatIntervalUnit: null as string | null,
  repeatEndDate: "",
});
const quickAddHasData = quickAddData.type !== "" || quickAddData.title.trim() !== "" || quickAddData.courseName !== ...
const handleQuickAddClose = () => {
  if (quickAddHasData) {
    setShowQuickAddCloseConfirm(true);
  } else {
    setIsQuickAddOpen(false);
  }
};
const icsFileInputRef = useRef<HTMLInputElement>(null);
const [icsImportEvents, setIcsImportEvents] = useState<Array<{
  title: string; description: string; startDate: string | null; endDate: string | null;
  location: string; allDay: boolean; selected: boolean; type: string; courseName: string;
}>>([]);
const [icsImportOpen, setIcsImportOpen] = useState(false);
const [icsImporting, setIcsImporting] = useState(false);

const [showMorningReview, setShowMorningReview] = useState(false);
const [morningReviewItems, setMorningReviewItems] = useState<any[]>([]);
const [morningReviewLoading, setMorningReviewLoading] = useState(false);
const [processingReviewIds, setProcessingReviewIds] = useState<Set<number>>(new Set());
const [reviewMode, setReviewMode] = useState<'all' | 'individual'>('all');
const [reviewCheckedIds, setReviewCheckedIds] = useState<Set<number>>(new Set());

const fetchPendingReview = useCallback(async () => {
  try {
    const res = await fetch('/api/pending-review?status=pending');
    if (res.ok) {
      const items = await res.json();
      setMorningReviewItems(items);
      return items;
    }
  } catch (e) { console.error('[Review] fetch error:', e); }
  return [];
}, []);

useEffect(() => {
  // DISABLED: Morning review auto-popup disconnected for now
  // const checkMorningReview = async () => {
  //   const eastern = new Date(new Date().toLocaleString('en-US', { timeZone: 'America/Toronto' }));
  //   const hour = eastern.getHours();
  //   if (hour < 9) return;
  //   const dismissUntil = localStorage.getItem('morning_review_dismiss_until');
  //   if (dismissUntil && Date.now() < Number(dismissUntil)) return;
  //   try {
  //     await fetch('/api/outlook/sync', { method: 'POST' }).catch(() => {});
  //     await fetch('/api/pending-review/dedup', { method: 'POST' }).catch(() => {});
  //   } catch {}
  //   const items = await fetchPendingReview();
  //   if (items.length > 0) {
  //     setShowMorningReview(true);
  //   }
  // };
  // checkMorningReview();
}, [fetchPendingReview]);

const triggerOutlookSyncAndReview = useCallback(async () => {
  try {
    await fetch('/api/outlook/sync', { method: 'POST' });
    await fetch('/api/pending-review/dedup', { method: 'POST' }).catch(() => {});
    const items = await fetchPendingReview();
    if (items.length > 0) {
      setShowMorningReview(true);
    }
  } catch (e) { console.error('[Review] Outlook sync + review error:', e); }
}, [fetchPendingReview]);

const handleAcceptReview = async (id: number, overrides?: Record<string, any>) => {
  setProcessingReviewIds(prev => new Set(prev).add(id));
  try {
    await fetch(`/api/pending-review/${id}/accept`, {
      method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ overrides }),
    });
    setMorningReviewItems(prev => {

```

```

    const next = prev.filter(i => i.id !== id);
    if (next.length === 0) setTimeout(() => setShowMorningReview(false), 400);
    return next;
  });
  queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
} catch (e) { console.error('[Review] accept error:', e); }
setProcessingReviewIds(prev => { const s = new Set(prev); s.delete(id); return s; });
};

const handleRejectReview = async (id: number) => {
  setProcessingReviewIds(prev => new Set(prev).add(id));
  try {
    await fetch(`/api/pending-review/${id}/reject`, { method: 'POST' });
    setMorningReviewItems(prev => {
      const next = prev.filter(i => i.id !== id);
      if (next.length === 0) setTimeout(() => setShowMorningReview(false), 400);
      return next;
    });
  } catch (e) { console.error('[Review] reject error:', e); }
  setProcessingReviewIds(prev => { const s = new Set(prev); s.delete(id); return s; });
};

const handleAcceptAll = async () => {
  setMorningReviewLoading(true);
  for (const item of morningReviewItems) {
    await handleAcceptReview(item.id);
  }
  setMorningReviewLoading(false);
  setShowMorningReview(false);
};

const handleSkipAllForToday = () => {
  const eastern = new Date(new Date().toLocaleString('en-US', { timeZone: 'America/Toronto' }));
  const tomorrow9am = new Date(eastern);
  tomorrow9am.setDate(tomorrow9am.getDate() + 1);
  tomorrow9am.setHours(9, 0, 0, 0);
  const offsetMs = eastern.getTime() - Date.now();
  localStorage.setItem('morning_review_dismiss_until', String(tomorrow9am.getTime() - offsetMs));
  setShowMorningReview(false);
  setReviewMode('all');
  setReviewCheckedIds(new Set());
};

const handleSkipAllForever = async () => {
  setMorningReviewLoading(true);
  for (const item of morningReviewItems) {
    await handleRejectReview(item.id);
  }
  setMorningReviewLoading(false);
  setShowMorningReview(false);
  setReviewMode('all');
  setReviewCheckedIds(new Set());
};

const handleIndividualConfirm = async () => {
  setMorningReviewLoading(true);
  for (const item of morningReviewItems) {
    if (reviewCheckedIds.has(item.id)) {
      await handleAcceptReview(item.id);
    } else {
      await handleRejectReview(item.id);
    }
  }
  setMorningReviewLoading(false);
  setShowMorningReview(false);
  setReviewMode('all');
  setReviewCheckedIds(new Set());
};

const handleIcsFileUpload = async (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (!file) return;
  e.target.value = '';
  try {
    const text = await file.text();
    const resp = await apiRequest('/api/tasks/parse-ics', 'POST', { icsContent: text });
    const data = await resp.json();
    if (!data.events?.length) {
      toast({ title: "No events found", description: "The ICS file didn't contain any calendar events." });
      return;
    }
    const mapped = data.events.map((ev: any) => ({ ...ev, selected: true, type: 'other', courseName: '' }));
    setIcsImportEvents(mapped);
    setIcsImportOpen(true);
  } catch (err) {
    toast({ title: "Import failed", description: "Could not parse the ICS file." });
  }
};

const handleIcsImportConfirm = async () => {
  const selected = icsImportEvents.filter(ev => ev.selected);
  if (selected.length === 0) { toast({ title: "No events selected" }); return; }
  setIcsImporting(true);
  let imported = 0;
  for (const ev of selected) {
    try {
      const dueDate = ev.endDate || ev.startDate;
      if (!dueDate) continue;
      const dueDateObj = new Date(dueDate);
      const weekNum = getWeekNumber(dueDateObj);
      if (weekNum < 1 || weekNum > 15) continue;
      const startTime = ev.startDate && !ev.allDay ? new Date(ev.startDate).toISOString().slice(0, 5) : '';
      const endTime = ev.endDate && !ev.allDay ? new Date(ev.endDate).toISOString().slice(0, 5) : '';
      const desc = [ev.description, ev.location ? `Location: ${ev.location}` : ''].filter(Boolean).join('\n');
      await apiRequest('/api/tasks', 'POST', {
        title: ev.title,
        description: desc || '',
        type: ev.type,
      });
    }
  }
};

```



```

        courseName: ev.courseName || null,
        dueDate: dueDateObj.toISOString(),
        startDate: ev.startDate && ev.startDate !== ev.endDate ? new Date(ev.startDate).toISOString() : null,
        eventStartTime: startTime || null,
        eventEndTime: endTime || null,
        weekNumber: Math.max(1, Math.min(weekNum, 13)),
        reminder1: DEFAULT_REMINDER_1,
        reminder2: DEFAULT_REMINDER_2,
        priority: 'medium',
    });
    imported++;
} catch (err) {
    console.error('Failed to import ICS event:', ev.title, err);
}
}
setIcsImporting(false);
setIcsImportOpen(false);
setIcsImportEvents([]);
queryClient.invalidateQueries({ queryKey: ['/api/tasks'] });
toast({ title: 'Imported ${imported} event${imported !== 1 ? 's' : ''}', description: 'Successfully created ${imp...
});

const [initialEndTime, setInitialEndTime] = useState<string>('');
const [editingTask, setEditingTaskRaw] = useState<Task | null>(null);
const setEditingTask = useCallback((task: Task | null) => {
    startTransition(() => setEditingTaskRaw(task));
}, []);
const [maxTaskNameWidth, setMaxTaskNameWidth] = useState<number>(0);
const [rescheduleTask, setRescheduleTask] = useState<Task | null>(null);
const [isTodayExpanded, setIsTodayExpanded] = useState(false);
const [bookAnimState, setBookAnimState] = useState<{ isOpen: boolean; courseCode: string; title: string; color: str...
const [bookReaderOverlay, setBookReaderOverlay] = useState<{ url: string; courseCode: string; title: string; color:...
const [hwUploadingState, setHwUploadingState] = useState<Record<string, boolean>>({});
const [hwDragOverTarget, setHwDragOverTarget] = useState<string | null>(null);
const semesterSettingsRef = useRef<any>(null);
const doHwUpload = useCallback(async (file: File, uploadType: 'module' | 'reading', courseCode: string, courseName:...
    const stateKey = `${courseCode}-${week}-${uploadType}`;
    setHwUploadingState(prev => ({ ...prev, [stateKey]: true }));
    try {
        const ss = semesterSettingsRef.current;
        const wd = getWeekDates(week, new Date(ss?.semesterStartDate || Date.now()), ss?.readingWeekStart);
        const ws = new Date(wd.start);
        const we = new Date(wd.end);
        const mos = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
        const s1 = `${mos[ws.getMonth()]} ${ws.getDate()}`;
        const s2 = ws.getMonth() === we.getMonth() ? `${we.getDate()}` : `${mos[we.getMonth()]} ${we.getDate()}`;
        const dr = `${s1}-${s2}`;
        const codeClean = courseCode.replace(/\\/s/g, '');
        const resp = await fetch('/api/course-week-upload', {
            method: 'POST',
            headers: {
                'Content-Type': file.type || 'application/pdf',
                'x-course-code': codeClean,
                'x-course-name': courseName.split(' - ').slice(1).join(' - ') || '',
                'x-week-num': String(week),
                'x-upload-type': uploadType,
                'x-week-date-range': dr,
                'x-file-name': file.name,
            },
            body: file,
        });
        const result = await resp.json();
        if (resp.ok && result.success) {
            toast({ title: `${uploadType} === 'module' ? 'Module' : 'Reading' } uploaded`, description: `${file.name} saved...
            queryClient.invalidateQueries({ queryKey: ['/api/files'] });
            setBookAnimState({
                isOpen: true,
                courseCode: codeClean,
                title: file.name,
                color: courseHexColor,
                onDone: () => {
                    setBookAnimState(null);
                    if (result.fileId) {
                        setBookReaderOverlay({ url: `/pdf-reader/${result.fileId}?autoplay=1`, courseCode: codeClean, title: fi...
                    }
                },
            });
        } else {
            toast({ title: 'Upload failed', description: result.error || 'Unknown error', variant: 'destructive' });
        }
    } catch (err: any) {
        toast({ title: 'Upload error', description: err.message, variant: 'destructive' });
    } finally {
        setHwUploadingState(prev => ({ ...prev, [stateKey]: false }));
    }
}, []);
const [calendarHeight, setCalendarHeight] = useState(() => {
    const defaultHeight = window.innerHeight - 45;
    const minHeight = 200;
    const maxHeight = window.innerHeight - 60;
    const screenWidth = window.screen.width;
    const screenHeight = window.screen.height;
    const pixelRatio = window.devicePixelRatio || 1;
    const deviceId = `device_${screenWidth}x${screenHeight}@${pixelRatio}`;
    const resetKey = 'calendarHeight_reset_v13';
    if (!localStorage.getItem(resetKey)) {
        localStorage.removeItem('calendarHeight');
        localStorage.removeItem(`calendarHeight_${deviceId}`);
        localStorage.removeItem('calendarReduction');
        localStorage.setItem(resetKey, '1');
    }
    const deviceSaved = localStorage.getItem(`calendarHeight_${deviceId}`);
    if (deviceSaved) {
        const val = parseInt(deviceSaved, 10);
        if (!isNaN(val) && val >= minHeight) return Math.min(val, maxHeight);
    }
    const saved = localStorage.getItem('calendarHeight');
    if (saved) {

```

```

    const val = parseInt(saved, 10);
    if (!isNaN(val) && val >= minHeight) return Math.min(val, maxHeight);
  }
  return defaultHeight;
});
const [isResizing, setIsResizing] = useState(false);
const resizeRef = useRef<{ startY: number; startHeight: number } | null>(null);
const [doTodayBounce, setDoTodayBounce] = useState(false);
const todayTaskCountRef = useRef(0);
const calendarWrapperRef = useRef<HTMLDivElement>(null);
const calendarBorderRef = useRef<HTMLDivElement>(null);
const homeworkSpacerRef = useRef<HTMLDivElement>(null);
const homeworkSectionRef = useRef<HTMLDivElement>(null);
const homeworkScrollRef = useRef<HTMLDivElement>(null);
const [hwIsScrolling, setHwIsScrolling] = useState(false);
const hwScrollTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
const [hwVisibleSemLabel, setHwVisibleSemLabel] = useState<string | null>(null);
const [hwScrolledDown, setHwScrolledDown] = useState(false);
const [hwFloating, setHwFloatingRaw] = useState(() => {
  const saved = localStorage.getItem('hwFloating');
  return saved ? { showControls: false, isPlaying: false, ...JSON.parse(saved) } : { detached: false, minimized: fa...
});
const setHwFloating = (val: any) => {
  setHwFloatingRaw((prev: any) => {
    const next = typeof val === 'function' ? val(prev) : val;
    localStorage.setItem('hwFloating', JSON.stringify(next));
    return next;
  });
};
const hwDragRef = useRef<{ startX: number; startY: number; origX: number; origY: number; moved: boolean } | null>(null);
const hwFloatingHandlers = useMemo(() => ({
  onDetach: () => {
    const el = homeworkSectionRef.current;
    const rect = el?.getBoundingClientRect();
    setHwFloating((p: any) => ({ ...p, detached: true, minimized: false, x: rect ? rect.left : 200, y: rect ? rect....
  },
  onDock: () => {
    setHwFloating((p: any) => ({ ...p, detached: false, minimized: false }));
  },
  onMinToggle: () => {
    setHwFloating((p: any) => ({ ...p, minimized: !p.minimized }));
  },
  onDragStart: (e: React.MouseEvent | React.TouchEvent) => {
    e.preventDefault();
    const clientX = 'touches' in e ? e.touches[0].clientX : e.clientX;
    const clientY = 'touches' in e ? e.touches[0].clientY : e.clientY;
    hwDragRef.current = { startX: clientX, startY: clientY, origX: hwFloating.x, origY: hwFloating.y, moved: false };
    const onMove = (ev: MouseEvent | TouchEvent) => {
      if (!hwDragRef.current) return;
      const cx = 'touches' in ev ? ev.touches[0].clientX : ev.clientX;
      const cy = 'touches' in ev ? ev.touches[0].clientY : ev.clientY;
      hwDragRef.current.moved = true;
      const nx = hwDragRef.current.origX + (cx - hwDragRef.current.startX);
      const ny = hwDragRef.current.origY + (cy - hwDragRef.current.startY);
      setHwFloating((p: any) => ({ ...p, x: Math.max(0, nx), y: Math.max(0, ny) }));
    };
    const onUp = () => {
      if (hwDragRef.current) {
        const snapMargin = 20;
        const vw = window.innerWidth;
        const vh = window.innerHeight;
        setHwFloating((p: any) => {
          let fx = p.x, fy = p.y;
          if (fx < snapMargin) fx = 4;
          if (fy < snapMargin) fy = 4;
          if (fx + 340 > vw - snapMargin) fx = vw - 344;
          if (fy + 50 > vh - snapMargin) fy = vh - 54;
          return { ...p, x: fx, y: fy };
        });
      }
      hwDragRef.current = null;
      window.removeEventListener('mousemove', onMove);
      window.removeEventListener('mouseup', onUp);
      window.removeEventListener('touchmove', onMove);
      window.removeEventListener('touchend', onUp);
    };
    window.addEventListener('mousemove', onMove);
    window.addEventListener('mouseup', onUp);
    window.addEventListener('touchmove', onMove);
    window.addEventListener('touchend', onUp);
  },
}), [hwFloating.x, hwFloating.y]);
const [hwDividerPercent, setHwDividerPercent] = useState(() => {
  const saved = localStorage.getItem('hwDividerPercent');
  return saved ? parseFloat(saved) : 33;
});
const effectiveDividerPct = hwFloating.detached ? 0 : hwDividerPercent;
const hwDividerDragRef = useRef<{ startX: number; startPercent: number; containerWidth: number } | null>(null);
const centerCircleOnGroupBar = (el: HTMLDivElement | null) => {
  if (!el) return;
  let groupRow = el.parentElement;
  while (groupRow && !groupRow.hasAttribute('data-hw-group-row')) groupRow = groupRow.parentElement;
  if (!groupRow) return;
  const bar = groupRow.querySelector('[data-hw-group-bar]') as HTMLElement;
  if (!bar) return;
  const barRect = bar.getBoundingClientRect();
  const barCenterX = barRect.left + barRect.width / 2;
  const op = el.offsetParent as HTMLElement;
  if (!op) return;
  const opRect = op.getBoundingClientRect();
  el.style.left = `${barCenterX - opRect.left - 9 + 79}px`;
};
const [hwGroupBarWidth, setHwGroupBarWidth] = useState(() => {
  const saved = localStorage.getItem('hwGroupBarWidth');
  return saved ? parseFloat(saved) : 111;
});
useEffect(() => {
  fetch('/api/ui-settings/hwGroupBarWidth').then(r => r.json()).then(d => {

```

```

    if (d.value !== null) {
      const v = parseFloat(d.value);
      if (!isNaN(v) && v > 0) { setHwGroupBarWidth(v); localStorage.setItem('hwGroupBarWidth', String(v)); }
    }
  }).catch(() => {});
}, []);
const getTodayCellGradient = () => {
  const now = new Date();
  const h = now.getHours() + now.getMinutes() / 60;
  if (h < 5) return 'linear-gradient(180deg, #00182B 0%, #00182B 100%)';
  if (h < 6) return 'linear-gradient(180deg, #00182B 0%, #C97355 100%)';
  if (h < 7) return 'linear-gradient(180deg, #C97355 0%, #D6A276 100%)';
  if (h < 8) return 'linear-gradient(180deg, #D6A276 0%, #EDC261 100%)';
  if (h < 9) return 'linear-gradient(180deg, #EDC261 0%, #8AC3DF 100%)';
  if (h < 16) return 'linear-gradient(180deg, #8AC3DF 0%, #8AC3DF 100%)';
  if (h < 17) return 'linear-gradient(180deg, #8AC3DF 0%, #C0B9BC 100%)';
  if (h < 18) return 'linear-gradient(180deg, #C0B9BC 0%, #ECC47E 100%)';
  if (h < 19) return 'linear-gradient(180deg, #ECC47E 0%, #F9A523 100%)';
  if (h < 20) return 'linear-gradient(180deg, #F9A523 0%, #00182B 100%)';
  return 'linear-gradient(180deg, #00182B 0%, #00182B 100%)';
};
const todayCellBg = getTodayCellGradient();
const hwGroupBarDragRef = useRef<{ startX: number; startWidth: number } | null>(null);
const hwTimelineDividerRef = useRef<HTMLDivElement>(null);
const [hwTimelinePos, setHwTimelinePos] = useState<{ top: number; left: number } | null>(null);
useEffect(() => {
  const update = () => {
    if (hwTimelineDividerRef.current) {
      const scrollParent = hwTimelineDividerRef.current.parentElement;
      const rect = scrollParent ? scrollParent.getBoundingClientRect() : hwTimelineDividerRef.current.getBoundingClientRect();
      setHwTimelinePos(prev => {
        const newTop = Math.round(rect.top - 15);
        const newLeft = Math.round(rect.left + rect.width / 2 - 25);
        if (prev && prev.top === newTop && prev.left === newLeft) return prev;
        return { top: newTop, left: newLeft };
      });
    }
  };
  update();
  const interval = setInterval(update, 2000);
  window.addEventListener('resize', update);
  return () => { clearInterval(interval); window.removeEventListener('resize', update); };
}, [hwGroupBarWidth]);
const [calendarBorderTop, setCalendarBorderTop] = useState(0);
const [clockContainerRef, setClockContainerRef] = useRef<HTMLDivElement>(null);
const [clockWidth, setClockWidth] = useState(0);
const [courseRowsRef, setCourseRowsRef] = useRef<HTMLDivElement>(null);
const [allDayRowRef, setAllDayRowRef] = useRef<HTMLDivElement>(null);
const [calendarTop, setCalendarTop] = useState(252); // Default offset
const [calendarBottom, setCalendarBottom] = useState(0); // Bottom edge of calendar wrapper
const [calendarRight, setCalendarRight] = useState(0); // Right edge of calendar wrapper relative to viewport
const [calendarLeft, setCalendarLeft] = useState(27); // Left edge of calendar wrapper
const [calendarReductionRef, setCalendarReductionRef] = useRef(0);
const [calendarReduction, setCalendarReductionRaw] = useState(() => {
  const sw = window.screen.width;
  const sh = window.screen.height;
  const pr = window.devicePixelRatio || 1;
  const did = `device_${sw}x${sh}@${pr}`;
  const deviceSaved = localStorage.getItem(`calendarReduction_${did}`);
  if (deviceSaved) {
    const dv = parseInt(deviceSaved, 10);
    if (!isNaN(dv)) { setCalendarReductionRef.current = dv; return dv; }
  }
  const saved = localStorage.getItem('calendarReduction');
  const v = saved ? parseInt(saved, 10) : 0;
  setCalendarReductionRef.current = v;
  return v;
});
const setCalendarReduction: typeof setCalendarReductionRaw = (val) => {
  setCalendarReductionRaw(prev => {
    const next = typeof val === 'function' ? (val as (p: number) => number)(prev) : val;
    setCalendarReductionRef.current = next;
    return next;
  });
};
const [calendarReductionUserSet, setCalendarReductionUserSetRaw] = useState(() => {
  const resetKey = 'calendarHeight_reset_v13';
  if (!localStorage.getItem(resetKey)) return false;
  return !!localStorage.getItem('calendarReduction');
});
const [calendarReductionUserSetRef, setCalendarReductionUserSetRef] = useRef(calendarReductionUserSet);
const setCalendarReductionUserSet = (val: boolean) => {
  setCalendarReductionUserSetRef.current = val;
  setCalendarReductionUserSetRaw(val);
};
const [isResizingHomework, setIsResizingHomework] = useState(false);
const [resizingHomeworkRef, setResizingHomeworkRef] = useRef<{ startX: number; startReduction: number } | null>(null);
const [originalCalendarLeft, setOriginalCalendarLeft] = useState(27); // Original left before reduction
const [courseRowsTop, setCourseRowsTop] = useState(0); // Position of course rows container
const [courseRowRefs, setCourseRowRefs] = useRef<(HTMLDivElement | null)[]>([]);
const [courseRowRects, setCourseRowRects] = useState<{top: number; height: number}[]>([]);
const [courseProgressDataRef, setCourseProgressDataRef] = useRef<{
  courseCode: string;
  progressBg: string;
  progressStartColor: string;
  progressEndColor: string;
  courseRowColor?: string;
  taskBgColor?: string;
  moduleP: {percent: number; hasFiles: boolean};
  readingP: {percent: number; hasFiles: boolean};
  moduleUnread: number;
  readingUnread: number;
  hasNoData: boolean;
  handlePlayModule: () => void;
  handlePlayReading: () => void;
  handleUpload: (type: 'module' | 'reading') => void;
  handleFileDrop: (type: 'module' | 'reading', file: File) => void;
}>({});

```

```

const [courseProgressTick, setCourseProgressTick] = useState(0);
const [completedFiles, setCompletedFiles] = useState<Set<string>>(() => {
  const saved = localStorage.getItem('completedFiles');
  return saved ? new Set(JSON.parse(saved)) : new Set();
});

const [isMuted, setIsMuted] = useState(() => {
  const saved = localStorage.getItem('alarmMuteUntil');
  if (saved) {
    const muteTime = parseInt(saved, 10);
    return Date.now() < muteTime;
  }
  return false;
});
const [muteUntil, setMuteUntil] = useState<number | null>(() => {
  const saved = localStorage.getItem('alarmMuteUntil');
  if (saved) {
    const muteTime = parseInt(saved, 10);
    return Date.now() < muteTime ? muteTime : null;
  }
  return null;
});
const [currentTime, setCurrentTime] = useState(new Date());
const [isSettingsPanelOpen, setIsSettingsPanelOpen] = useState(false);
const [isScholarshipsOpen, setIsScholarshipsOpen] = useState(false);
const [isKeyContactsOpen, setIsKeyContactsOpen] = useState(false);
const [isFeedbackOpen, setIsFeedbackOpen] = useState(false);
const [feedbackText, setFeedbackText] = useState('');
const [feedbackSending, setFeedbackSending] = useState(false);
const [showRemainingList, setShowRemainingList] = useState(false);

const [showSemesterChecklist, setShowSemesterChecklist] = useState(false);
const [semesterChecklistItems, setSemesterChecklistItems] = useState<Array<{id: number; semesterSettingsId: number;...}>>();
const [semesterChecklistId, setSemesterChecklistId] = useState<number | null>(null);
const [semesterChecklistShownRef, useRef] = useState(false);

const [showConfirmSemesterDialog, setShowConfirmSemesterDialog] = useState(false);
const [upcomingSemester, setUpcomingSemester] = useState<FutureSemesterDates | null>(null);
const [confirmSemesterForm, setConfirmSemesterForm] = useState({ startDate: '', endDate: '', breakStart: '', breakEnd: '' });
const [showPdfUploadDialog, setShowPdfUploadDialog] = useState(false);
const [pdfUploadShownRef, useRef] = useState(false);
const [confirmSemesterShownRef, useRef] = useState(false);

// Return-from-break reading prompt (dev mode only - when returning to Replit after 2+ hours)
const [showReturnReadingPrompt, setShowReturnReadingPrompt] = useState(false);
const [returnReadingFiles, setReturnReadingFiles] = useState<Array<{id: number; name: string; folder: string; originalName: string}>>();
const [selectedReturnFile, setSelectedReturnFile] = useState<number | null>(null);
const [returnPromptShown, useRef] = useState(false);

useEffect(() => {
  if (returnPromptShown.current) return;
  returnPromptShown.current = true;

  if (window.location.hostname.endsWith('.replit.app')) {
    localStorage.setItem('replitLastActiveTime', String(Date.now()));
    return;
  }

  const lastActive = parseInt(localStorage.getItem('replitLastActiveTime') || '0', 10);
  const now = Date.now();
  const twoHoursMs = 2 * 60 * 60 * 1000;
  const forceWelcome = new URLSearchParams(window.location.search).has('forceWelcome');
  if (!forceWelcome && lastActive > 0 && (now - lastActive) < twoHoursMs) {
    localStorage.setItem('replitLastActiveTime', String(now));
    return;
  }

  Promise.all([
    fetch("/api/files").then(r => r.json()),
    fetch("/api/semester", { credentials: 'include' }).then(r => r.ok ? r.json() : null),
  ])
  .then(([files, semSettings]: [any[], any]) => {
    let currentWeekNum = 1;
    if (semSettings?.semesterStartDate) {
      currentWeekNum = getWeekNumber(new Date(), new Date(semSettings.semesterStartDate), semSettings.readingWeekStart);
    }

    const isPartial = (f: any) => {
      if (f.listened) return false;
      const hasCheckedChunks = () => {
        if (!f.checkedChunks) return false;
        try { const arr = JSON.parse(f.checkedChunks); return Array.isArray(arr) && arr.length > 0; } catch { return false; }
      };
      const hasLastChunk = f.lastChunkIndex != null && f.lastChunkIndex > 0;
      if (!hasCheckedChunks && !hasLastChunk) return false;
      if (f.totalChunks && f.totalChunks > 0) {
        if (hasCheckedChunks) {
          try { const arr = JSON.parse(f.checkedChunks); if (arr.length >= f.totalChunks) return false; } catch {}
        }
        if (hasLastChunk && f.lastChunkIndex >= f.totalChunks) return false;
      }
      return true;
    };

    const currentWeekFiles = files.filter((f: any) => {
      const weekMatch = f.folder?.match(/week-(\d+)/i);
      if (!weekMatch || parseInt(weekMatch[1], 10) !== currentWeekNum) return false;
      if (f.listened) return false;
      return true;
    });

    const partialCurrentWeek = currentWeekFiles.filter(isPartial);
    const unlistenedCurrentWeek = currentWeekFiles.filter((f: any) => !isPartial(f));

    const orderedFiles = [...partialCurrentWeek, ...unlistenedCurrentWeek];

    if (orderedFiles.length > 0) {
      const mapped = orderedFiles.map((f: any) => ({

```

```

        id: f.id,
        name: f.displayName || f.originalName || 'Unknown',
        folder: f.folder || '',
        originalName: f.originalName || '',
        displayName: f.displayName || f.originalName || '',
        objectPath: f.objectPath || '',
        listened: f.listened,
        lastChunkIndex: f.lastChunkIndex,
        totalChunks: f.totalChunks,
        checkedChunks: f.checkedChunks,
    ));
    setReturnReadingFiles(mapped);
    setSelectedReturnFile(mapped[0].id);
  }
  if (orderedFiles.length > 0 && window.location.hostname === 'localhost') {
    setShowReturnReadingPrompt(true);
  }
}
.catch(() => {});

const interval = setInterval(() => {
  localStorage.setItem('replitLastActiveTime', String(Date.now()));
}, 60000);
return () => clearInterval(interval);
}, []);

// Share link state
const [isShareDialogOpen, setIsShareDialogOpen] = useState(false);
const [shareLink, setShareLink] = useState("");
const [isGeneratingLink, setIsGeneratingLink] = useState(false);
const { isReadOnly, isAdmin } = useAccessMode();
const [isCompletedTasksOpen, setIsCompletedTasksOpen] = useState(false);
const [dismissedCalendarEvents, setDismissedCalendarEvents] = useState<Set<string>>(() => {
  const saved = localStorage.getItem('dismissedCalendarEvents');
  return saved ? new Set(JSON.parse(saved)) : new Set();
}));
const [isRadioDialogOpen, setIsRadioDialogOpen] = useState(false);
const [selectedSpeaker, setSelectedSpeaker] = useState("media_player.echo_lr_studio_white_am");
const [radioVolume, setRadioVolume] = useState(50);
const [isFilesFlyoutOpen, setIsFilesFlyoutOpen] = useState(true);
const [isFiles2FlyoutOpen, setIsFiles2FlyoutOpen] = useState(true);
const [lastOpenedFlyout, setLastOpenedFlyout] = useState<'files1' | 'files2'>('files1'); // Track which flyout was ...
const [readingsPopupCourse, setReadingsPopupCourse] = useState<string | null>(null);
const [oneDriveReadingFiles, setOneDriveReadingFiles] = useState<any[]>([]);
const [listenedOneDriveFiles, setListenedOneDriveFiles] = useState<Set<string>>(() => {
  const saved = localStorage.getItem('listenedOneDriveFiles');
  return saved ? new Set(JSON.parse(saved)) : new Set();
}));
useEffect(() => {
  if (!readingsPopupCourse) { setOneDriveReadingFiles([]); return; }
  const courseCode = readingsPopupCourse.toUpperCase();
  const basePath = '/School/1. TMU/Courses/2026/Winter';
  (async () => {
    try {
      const baseRes = await fetch(`/api/onedrive/files?path=${encodeURIComponent(basePath)}`);
      const baseFolders = await baseRes.json();
      if (!Array.isArray(baseFolders)) return;
      const matched = baseFolders.find((f: any) => f.type === 'folder' && f.name.toUpperCase().startsWith(courseCod...
      if (!matched) return;
      const courseRes = await fetch(`/api/onedrive/files?path=${encodeURIComponent(matched.path)}`);
      const courseFolders = await courseRes.json();
      if (!Array.isArray(courseFolders)) return;
      const weekFolder = courseFolders.find((f: any) => f.type === 'folder' && f.name.toLowerCase().startsWith('wee...
      if (!weekFolder) return;
      const weekRes = await fetch(`/api/onedrive/files?path=${encodeURIComponent(weekFolder.path)}`);
      const weekContents = await weekRes.json();
      if (!Array.isArray(weekContents)) return;
      const readingFolder = weekContents.find((f: any) => f.type === 'folder' && f.name.toLowerCase().includes('rea...
      if (!readingFolder) return;
      const readingRes = await fetch(`/api/onedrive/files?path=${encodeURIComponent(readingFolder.path)}`);
      const readingFiles = await readingRes.json();
      if (Array.isArray(readingFiles)) {
        setOneDriveReadingFiles(readingFiles.filter((f: any) => f.type === 'file'));
      }
    } catch (err) {
      console.error('Error loading reading files:', err);
    }
  })();
}, [readingsPopupCourse, selectedWeek]);
const [isWeekReadingsOpen, setIsWeekReadingsOpen] = useState(false);
const [weekReadingSelectedFile, setWeekReadingSelectedFile] = useState<any | null>(null);
const [isWeeksFlyoutOpen, setIsWeeksFlyoutOpen] = useState(false);
const [isUploadDialogOpen, setIsUploadDialogOpen] = useState(false);
const [uploadTargetFolder, setUploadTargetFolder] = useState<string>('week-3-cppa122-reading');
const [renameFileId, setRenameFileId] = useState<number | null>(null);
const [renameFileName, setRenameFileName] = useState<string>('');

const [preparedFilesDialog, setPreparedFilesDialog] = useState<{ id: number; name: string; folder: string; totalChu...
const [showPreparedDialog, setShowPreparedDialog] = useState(false);

useEffect(() => {
  const checkPreparedFiles = async () => {
    try {
      const res = await fetch('/api/files/recently-prepared');
      const data = await res.json();
      if (data.files && data.files.length > 0) {
        setPreparedFilesDialog(data.files);
        setShowPreparedDialog(true);
      }
    } catch {}
  };
  checkPreparedFiles();
  const interval = setInterval(checkPreparedFiles, 30000);
  return () => clearInterval(interval);
}, []);

const acknowledgePreparedFiles = async () => {
  const fileIds = preparedFilesDialog.map(f => f.id);

```

```

try {
  await fetch('/api/files/acknowledge-prepared', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ fileIds }),
  });
} catch {}
setShowPreparedDialog(false);
setPreparedFilesDialog([]);
});

// Partner away popup state - show when partner is at work
const [showPartnerAwayPopup, setShowPartnerAwayPopup] = useState(false);
const [isPartnerAway, setIsPartnerAway] = useState(false);
const [partnerAwayDismissedUntil, setPartnerAwayDismissedUntil] = useState<number | null>(() => {
  const saved = localStorage.getItem('partnerAwayDismissedUntil');
  return saved ? parseInt(saved, 10) : null;
});
const [isInitialized, setIsInitialized] = useState(false);

// Initialize from localStorage on mount
useEffect(() => {
  const saved = localStorage.getItem('partnerAwayDismissedUntil');
  if (saved) {
    setPartnerAwayDismissedUntil(parseInt(saved, 10));
  }
  setIsInitialized(true);
}, []);
const [isKitchenReadingLoading, setIsKitchenReadingLoading] = useState(false);
const [isKitchenPlaying, setIsKitchenPlaying] = useState(false);
const [isPillMenuOpen, setIsPillMenuOpen] = useState(false);
const [sidePillIdle, setSidePillIdle] = useState(false);
const pillMenuTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
const sidePillRef = useRef<HTMLDivElement>(null);
const [sidePillMounted, setSidePillMounted] = useState(false);
const sidePillSlideOffset = useRef(60);
const openSidePill = useCallback(() => {
  const el = sidePillRef.current;
  if (el && sidePillIdle) {
    const computed = getComputedStyle(el);
    const matrix = new DOMMatrixReadOnly(computed.transform);
    const currentY = matrix.m42;
    el.style.transition = 'none';
    el.style.transform = `translateX(-50%) translateY(${currentY}px)`;
    setSidePillIdle(false);
    requestAnimationFrame(() => {
      requestAnimationFrame(() => {
        el.style.transition = 'transform 0.3s ease-in-out';
        el.style.transform = 'translateX(-50%) translateY(0px)';
        startTransition(() => setIsPillMenuOpen(true));
      });
    });
  }
} else {
  setSidePillIdle(false);
  startTransition(() => setIsPillMenuOpen(true));
}
}, [sidePillIdle]);
const closeSidePill = useCallback(() => {
  startTransition(() => setIsPillMenuOpen(false));
}, []);
useEffect(() => {
  if (!isPillMenuOpen && sidePillMounted) {
    const el = sidePillRef.current;
    if (el) {
      const onEnd = () => {
        setSidePillIdle(true);
        el.removeEventListener('transitionend', onEnd);
      };
      el.addEventListener('transitionend', onEnd);
      return () => el.removeEventListener('transitionend', onEnd);
    }
  }
}, [isPillMenuOpen, sidePillMounted]);
useEffect(() => {
  requestAnimationFrame(() => {
    requestAnimationFrame(() => {
      setSidePillMounted(true);
      startTransition(() => setIsPillMenuOpen(true));
    });
  });
  const sideTimeout = setTimeout(() => {
    closeSidePill();
  }, 2200);
  return () => clearTimeout(sideTimeout);
}, []);
const [isTopPillOpen, setIsTopPillOpenState] = useState(false);
const isTopPillOpenRef = useRef(false);
const setTopPillOpen = useCallback((val: boolean) => {
  if (isTopPillOpenRef.current === val) return;
  isTopPillOpenRef.current = val;
  startTransition(() => setIsTopPillOpenState(val));
  document.querySelectorAll<HTMLElement>('[data-tpo]').forEach(el => {
    el.style.opacity = val ? '0' : (el.dataset.tpoOpacity || '1');
    el.style.pointerEvents = val ? 'none' : 'auto';
    el.style.transition = val ? 'opacity 0.3s ease-in-out' : 'opacity 0.1s ease-in-out';
  });
}, []);
const [newsHeadlines, setNewsHeadlines] = useState<{ title: string; source: string; link: string; publishedAt?: str...

useEffect(() => {
  const fetchNews = async () => {
    try {
      const res = await fetch('/api/news');
      if (res.ok) {
        const data = await res.json();
        setNewsHeadlines(data);
      }
    } catch (e) {
  }
}

```

```

        console.error('[News] Failed to fetch:', e);
    }
};
fetchNews();
const interval = setInterval(fetchNews, 10 * 60 * 1000);
return () => clearInterval(interval);
}, []);

const [weatherData, setWeatherData] = useState<{ code: number; temp: number; windSpeed: number; isDay: boolean; sun...
const [pollenData, setPollenData] = useState<{ tree: { value: number; level: string }; grass: { value: number; leve...
const [weatherAlerts, setWeatherAlerts] = useState<{ title: string; summary: string; type: string }[]>([]);
const weatherParticles = useMemo(() => Array.from({ length: 70 }, () => ({
    left: Math.random() * 110 - 5,
    delay: Math.random() * 4,
    sizeFactor: Math.random(),
    durationFactor: Math.random(),
    driftFactor: Math.random(),
    opacityFactor: Math.random(),
})), []);

useEffect(() => {
    const fetchWeather = async () => {
        try {
            const res = await fetch('/api/weather');
            const data = await res.json();
            if (data.current) {
                const daily = data.daily ? data.daily.time.map((d: string, i: number) => ({
                    date: d,
                    high: Math.round(data.daily.temperature_2m_max[i]),
                    low: Math.round(data.daily.temperature_2m_min[i]),
                    weatherCode: data.daily.weather_code?.[i] ?? undefined,
                    sunrise: data.daily.sunrise?.[i] ?? undefined,
                    sunset: data.daily.sunset?.[i] ?? undefined,
                })) : [];
                const todayStr = new Date().toISOString().split('T')[0];
                const todayDaily = daily.find((d: any) => d.date === todayStr);
                setWeatherData({
                    code: data.current.weather_code,
                    temp: data.current.temperature_2m,
                    windSpeed: data.current.wind_speed_10m,
                    isDay: data.current.is_day === 1,
                    sunrise: todayDaily?.sunrise,
                    sunset: todayDaily?.sunset,
                    daily,
                });
            }
        } catch (e) {
            console.error('[Weather] Failed to fetch:', e);
        }
    };
    const fetchPollen = async () => {
        try {
            const res = await fetch('/api/pollen');
            const data = await res.json();
            if (data.overall) setPollenData(data);
        } catch (e) {
            console.error('[Pollen] Failed to fetch:', e);
        }
    };
    const fetchWeatherAlerts = async () => {
        try {
            const res = await fetch('/api/weather-alerts');
            const data = await res.json();
            if (data.alerts) setWeatherAlerts(data.alerts);
        } catch (e) {
            console.error('[Weather Alerts] Failed to fetch:', e);
        }
    };
    fetchWeather();
    fetchPollen();
    fetchWeatherAlerts();
    const weatherInterval = setInterval(fetchWeather, 15 * 60 * 1000);
    const pollenInterval = setInterval(fetchPollen, 30 * 60 * 1000);
    const alertsInterval = setInterval(fetchWeatherAlerts, 5 * 60 * 1000);
    return () => { clearInterval(weatherInterval); clearInterval(pollenInterval); clearInterval(alertsInterval); };
}, []);

const [isVoiceListening, setIsVoiceListening] = useState(false);
const [voiceTranscript, setVoiceTranscript] = useState('');
const voiceRecognitionRef = useRef<any>(null);
const voiceTargetRef = useRef<HTMLInputElement | HTMLTextAreaElement | null>(null);

const startVoiceInput = useCallback(() => {
    const SpeechRecognition = (window as any).SpeechRecognition || (window as any).webkitSpeechRecognition;
    if (!SpeechRecognition) {
        toast({ title: "Not supported", description: "Speech recognition is not supported in this browser." });
        return;
    }
    const activeEl = document.activeElement as HTMLInputElement | HTMLTextAreaElement | null;
    if (activeEl && (activeEl.tagName === 'INPUT' || activeEl.tagName === 'TEXTAREA')) {
        voiceTargetRef.current = activeEl;
    } else {
        voiceTargetRef.current = null;
    }
    const recognition = new SpeechRecognition();
    recognition.continuous = true;
    recognition.interimResults = true;
    recognition.lang = 'en-US';
    let finalText = '';
    recognition.onresult = (event: any) => {
        let interim = '';
        for (let i = event.resultIndex; i < event.results.length; i++) {
            const t = event.results[i][0].transcript;
            if (event.results[i].isFinal) {
                finalText += t;
            } else {
                interim += t;
            }
        }
    }
}, []);

```

```

    }
    setVoiceTranscript(finalText + interim);
  };
  recognition.onerror = (event: any) => {
    console.error('[Voice] Recognition error:', event.error);
    setIsVoiceListening(false);
  };
  recognition.onend = () => {
    setIsVoiceListening(false);
    if (finalText && voiceTargetRef.current) {
      const el = voiceTargetRef.current;
      el.focus();
      document.execCommand('insertText', false, finalText);
    }
    setVoiceTranscript('');
    voiceTargetRef.current = null;
  };
  voiceRecognitionRef.current = recognition;
  recognition.start();
  setIsVoiceListening(true);
  setVoiceTranscript('');
}, [toast]);

const stopVoiceInput = useCallback(() => {
  if (voiceRecognitionRef.current) {
    voiceRecognitionRef.current.stop();
    voiceRecognitionRef.current = null;
  }
}, []);

const [topPillMounted, setTopPillMounted] = useState(false);
const topPillRef = useRef<HTMLDivElement>(null);
const topPillTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
const topPillInitDoneRef = useRef(false);
const openTopPill = useCallback(() => {
  if (!topPillInitDoneRef.current) return;
  setIsTopPillOpen(true);
}, []);
const closeTopPill = useCallback(() => {
  setIsTopPillOpen(false);
}, []);
useEffect(() => {
  const mountDelay = setTimeout(() => {
    setTopPillMounted(true);
    setIsTopPillOpen(true);
  }, 800);
  topPillTimeoutRef.current = setTimeout(() => {
    topPillInitDoneRef.current = true;
    closeTopPill();
  }, 4000);
  return () => { clearTimeout(mountDelay); if (topPillTimeoutRef.current) clearTimeout(topPillTimeoutRef.current); };
}, []);

const [draggedFileForMove, setDraggedFileForMove] = useState<{id: number; folder: string} | null>(null);
const [moveFileId, setMoveFileId] = useState<number | null>(null);
const [moveFileCurrentFolder, setMoveFileCurrentFolder] = useState<string>('');
const [newFolderName, setNewFolderName] = useState('');
const [isCreatingFolder, setIsCreatingFolder] = useState(false);
const [editingFolderId, setEditingFolderId] = useState<string | null>(null);
const [editingFolderName, setEditingFolderName] = useState('');
useEffect(() => {
  const observer = new MutationObserver(() => {
    if (document.body.style.pointerEvents === 'none') {
      const hasOpenDialog = document.querySelector('[data-state="open"][role="dialog"]');
      if (!hasOpenDialog) {
        document.body.style.pointerEvents = '';
      }
    }
  });
  observer.observe(document.body, { attributes: true, attributeFilter: ['style'] });
  return () => observer.disconnect();
}, []);

// Honeycomb navigation state
const [modulesHoneycombOpen, setModulesHoneycombOpen] = useState<string | null>('modules');
const [decorativeHoneycombHover, setDecorativeHoneycombHover] = useState<'left' | 'middle' | 'right' | null>(null);
const [readingsHoneycombOpen, setReadingsHoneycombOpen] = useState(false);
const [moduleMediaControlCourse, setModuleMediaControlCourse] = useState<string | null>(null);
const [flyoutWidth, setFlyoutWidth] = useState(183); // Default flyout width for files (half width)
const [flyout2Width, setFlyout2Width] = useState(183); // Default flyout width for files2 (half width)
const [weeksFlyoutWidth, setWeeksFlyoutWidth] = useState(295); // Default flyout width for week folders
const [isResizingFlyout, setIsResizingFlyout] = useState(false);
const [isResizingFlyout2, setIsResizingFlyout2] = useState(false);
const [isResizingWeeksFlyout, setIsResizingWeeksFlyout] = useState(false);
const [isTodoFlyoutOpen, setIsTodoFlyoutOpen] = useState(false);
const [isProjectsFlyoutOpen, setIsProjectsFlyoutOpen] = useState(false);
const [flyoutZOrder, setFlyoutZOrder] = useState<string[]>(['files', 'projects', 'todo', 'addTask']);

const bringFlyoutToFront = (flyoutId: string) => {
  setFlyoutZOrder(prev => {
    const filtered = prev.filter(id => id !== flyoutId);
    return [...filtered, flyoutId];
  });
};

const getFlyoutZIndex = (flyoutId: string) => {
  const index = flyoutZOrder.indexOf(flyoutId);
  return 200 + (index >= 0 ? index : 0);
};

const [projectStatusFilter, setProjectStatusFilter] = useState<string>("all");
const [expandedProjects, setExpandedProjects] = useState<Set<number>>(new Set());
const [projectDialogOpen, setProjectDialogOpen] = useState(false);
const [editingProject, setEditingProject] = useState<Project | null>(null);
const [projectWizardStep, setProjectWizardStep] = useState(0);
const [projectWizardData, setProjectWizardData] = useState({ name: '', description: '', color: '#6366F1', status: '...' });
const [flyoutExpandedFolders, setFlyoutExpandedFolders] = useState<Set<string>>(new Set());

// Folder context menu state
const [folderContextMenu, setFolderContextMenu] = useState<{
  x: number;

```



```

    y: number;
    parentFolder: string;
  } | null>(null));
const [newFolderDialogOpen, setNewFolderDialogOpen] = useState(false);
const [newFolderParent, setNewFolderParent] = useState('');
const [customFolders, setCustomFolders] = useState<{ id: string; name: string; parent: string }[]>(() => {
  const saved = localStorage.getItem('customFolders');
  return saved ? JSON.parse(saved) : [];
});

// Save custom folders to localStorage
useEffect(() => {
  localStorage.setItem('customFolders', JSON.stringify(customFolders));
}, [customFolders]);

// Load custom folders from database on mount
useEffect(() => {
  const loadFoldersFromDB = async () => {
    try {
      const response = await fetch('/api/custom-folders');
      if (response.ok) {
        const dbFolders = await response.json();
        if (dbFolders.length > 0) {
          // Merge database folders with local folders (database takes precedence)
          const dbFolderMap = new Map(dbFolders.map((f: any) => [f.name + '-' + f.parentFolderId, f]));
          setCustomFolders(prev => {
            const merged = [...prev];
            dbFolders.forEach((dbFolder: any) => {
              const folderId = `${dbFolder.parentFolderId}-subfolder-${dbFolder.name.toLowerCase()}-${dbFolder.id}`;
              const exists = merged.some(f =>
                f.parent === dbFolder.parentFolderId && f.name === dbFolder.name
              );
              if (!exists) {
                merged.push({
                  id: folderId,
                  name: dbFolder.name,
                  parent: dbFolder.parentFolderId
                });
              }
            });
            return merged;
          });
        }
      } catch (err) {
        console.error('Failed to load folders from database:', err);
      }
    }
  };
  loadFoldersFromDB();
}, []);

// Check partner status every 60 seconds to show kitchen reading popup
// Use ref to track if popup is already showing (prevents re-triggering during same session)
const partnerPopupShownRef = useRef(false);

useEffect(() => {
  const checkPartnerStatus = async () => {
    try {
      const response = await fetch('/api/partner-status');
      if (response.ok) {
        const data = await response.json();
        setIsPartnerAway(data.isAway);

        // Show popup if partner is away and we haven't dismissed it recently
        // Also check ref to prevent showing if already shown this session
        if (false && data.isAway && !partnerPopupShownRef.current) {
          const now = Date.now();
          const savedDismiss = localStorage.getItem('partnerAwayDismissedUntil');
          const dismissedUntil = savedDismiss ? parseInt(savedDismiss, 10) : 0;
          if (!dismissedUntil || now > dismissedUntil) {
            setShowPartnerAwayPopup(true);
            partnerPopupShownRef.current = true; // Mark as shown
          }
        } else if (!data.isAway) {
          // Reset ref when partner comes home so popup can show again when they leave
          partnerPopupShownRef.current = false;
        }
      }
    } catch (err) {
      // Silently fail - Home Assistant might not be configured
    }
  };

  // Only start checking after initialization completes
  if (!isInitialized) return;

  // Check immediately, then every 60 seconds
  checkPartnerStatus();
  const interval = setInterval(checkPartnerStatus, 60000);
  return () => clearInterval(interval);
}, [isInitialized]);

// Poll kitchen playback status
useEffect(() => {
  const checkKitchenStatus = async () => {
    try {
      const response = await fetch('/api/kitchen/status');
      if (response.ok) {
        const data = await response.json();
        setIsKitchenPlaying(data.isPlaying);
      }
    } catch (err) {
      // Ignore errors
    }
  };

  checkKitchenStatus();
  const interval = setInterval(checkKitchenStatus, 60000);

```

```

    return () => clearInterval(interval);
  }, []);

const lastTabletNavTimestamp = useRef(() => {
  try { return Number(localStorage.getItem('lastNavTimestamp')) || '0'; } catch { return 0; }
});
useEffect(() => {
  const isFireDevice = /\bSilk\b/i.test(navigator.userAgent) || /\bKF[A-Z]{2,4}\b/.test(navigator.userAgent) || /\b...
  if (isFireDevice) return;
  const urlParams = new URLSearchParams(window.location.search);
  const roleParam = urlParams.get('deviceRole');
  if (roleParam) {
    try { localStorage.setItem('tabletDeviceRole', roleParam); } catch {}
  }
  const isFireTV = /\bFireTV\b/i.test(navigator.userAgent) || /\bAFT[A-Z]\b/.test(navigator.userAgent);
  const deviceRole = roleParam || (() => { try { return localStorage.getItem('tabletDeviceRole'); } catch { return ...
  const saved = lastTabletNavTimestamp.current;
  lastTabletNavTimestamp.current = typeof saved === 'function' ? saved() : saved;
  const beacon = (step: string, data: any) => {
    fetch('/api/debug-beacon', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.string...
  };
  beacon('init', { deviceRole, isFireTV, lastTs: lastTabletNavTimestamp.current, ua: navigator.userAgent.substring(...
  let pollCount = 0;
  const checkTabletNav = async () => {
    try {
      const resp = await fetch('/api/tablet-nav?device=${deviceRole}&t=${Date.now()}');
      const data = await resp.json();
      pollCount++;
      if (pollCount % 20 === 0) {
        beacon('heartbeat', { deviceRole, pollCount, lastTs: lastTabletNavTimestamp.current, action: data.action ||...
      }
      if (!data.action || data.action === null) return;
      beacon('cmd-received', { action: data.action, cmdTs: data.timestamp, lastTs: lastTabletNavTimestamp.current, ...
      if (data.timestamp === lastTabletNavTimestamp.current) {
        return;
      }
      if (Date.now() - data.timestamp > 120000) {
        beacon('cmd-skipped-expired', { cmdTs: data.timestamp, age: Date.now() - data.timestamp });
        return;
      }
      if (data.action === 'navigate' && data.url) {
        beacon('cmd-navigate', { url: data.url.substring(0, 80) });
        lastTabletNavTimestamp.current = data.timestamp;
        try { localStorage.setItem('lastNavTimestamp', String(data.timestamp)); } catch {}
        fetch('/api/tablet-nav/ack', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON....
        window.location.href = data.url;
      } else if (data.action === 'stop_playback') {
        beacon('cmd-stop', {});
        lastTabletNavTimestamp.current = data.timestamp;
        try { localStorage.setItem('lastNavTimestamp', String(data.timestamp)); } catch {}
        fetch('/api/tablet-nav/ack', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON....
      } else if (data.action === 'go_home' && window.location.pathname !== '/') {
        beacon('cmd-go-home', {});
        lastTabletNavTimestamp.current = data.timestamp;
        try { localStorage.setItem('lastNavTimestamp', String(data.timestamp)); } catch {}
        fetch('/api/tablet-nav/ack', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON....
        window.location.href = '/';
      }
    } catch (e: any) {
      beacon('poll-error', { error: e.message });
    }
  };
  const interval = setInterval(checkTabletNav, 10000);
  return () => clearInterval(interval);
}, []);

// Handle kitchen reading trigger
const handleKitchenReadingTrigger = async () => {
  setIsKitchenReadingLoading(true);
  try {
    const response = await fetch('/api/kitchen/trigger', { method: 'POST' });
    const data = await response.json();
    if (response.ok) {
      setIsKitchenPlaying(true); // Start showing as playing immediately
      toast({
        title: data.action === 'radio' ? 'Playing Radio' : 'Playing Reading',
        description: data.action === 'radio'
          ? data.message
          : 'Now playing: ${data.file?.name || 'Unknown file'}'
      });
    } else {
      toast({ title: 'Error', description: data.error || 'Failed to trigger reading', variant: 'destructive' });
    }
  } catch (err) {
    toast({ title: 'Error', description: 'Failed to trigger kitchen reading', variant: 'destructive' });
  } finally {
    setIsKitchenReadingLoading(false);
    // Dismiss for 4 hours when playing readings too (same as clicking "No, not now")
    handleDismissPartnerPopup();
  }
};

// Handle stopping kitchen playback
const handleKitchenStop = async () => {
  try {
    const response = await fetch('/api/kitchen/stop', { method: 'POST' });
    if (response.ok) {
      setIsKitchenPlaying(false);
      toast({ title: 'Playback Stopped', description: 'Kitchen reading stopped' });
    }
  } catch (err) {
    toast({ title: 'Error', description: 'Failed to stop playback', variant: 'destructive' });
  }
};

// Dismiss partner popup for 4 hours
const toggleDismissCalendarEvent = (eventId: string) => {
  setDismissedCalendarEvents(prev => {

```

```

    const next = new Set(prev);
    if (next.has(eventId)) next.delete(eventId);
    else next.add(eventId);
    localStorage.setItem('dismissedCalendarEvents', JSON.stringify([...next]));
    return next;
  });
};

const handleDismissPartnerPopup = () => {
  const dismissUntil = Date.now() + 4 * 60 * 60 * 1000; // 4 hours
  setPartnerAwayDismissedUntil(dismissUntil);
  localStorage.setItem('partnerAwayDismissedUntil', dismissUntil.toString());
  setShowPartnerAwayPopup(false);
};

// Handle folder right-click context menu
const handleFolderContextMenu = (e: React.MouseEvent, parentFolder: string) => {
  e.preventDefault();
  e.stopPropagation();
  setFolderContextMenu({
    x: e.clientX,
    y: e.clientY,
    parentFolder
  });
};

// Create new folder - saves to database first, then updates local state
const handleCreateFolder = async () => {
  if (newFolderName.trim() && newFolderParent) {
    try {
      // Save to database first
      const response = await fetch('/api/custom-folders', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ name: newFolderName.trim(), parentFolderId: newFolderParent })
      });

      if (!response.ok) {
        const errorText = await response.text();
        console.error('Failed to save folder to database:', response.status, errorText);
        toast({ title: "Error", description: "Failed to save folder", variant: "destructive" });
        return;
      }

      const savedFolder = await response.json();
      console.log('Folder saved to database:', savedFolder);

      // Update local state with the database ID
      const folderId = `${newFolderParent}-subfolder-${newFolderName.toLowerCase()}-${savedFolder.id}`;
      setCustomFolders(prev => [...prev, {
        id: folderId,
        name: newFolderName.trim(),
        parent: newFolderParent
      }]);

      setNewFolderName('');
      setNewFolderDialogOpen(false);
      toast({ title: 'Folder "${newFolderName}" created' });
    } catch (err) {
      console.error('Failed to save folder to database:', err);
      toast({ title: "Error", description: "Failed to create folder", variant: "destructive" });
    }
  }
};

// Close context menu when clicking outside
useEffect(() => {
  const handleClick = () => setFolderContextMenu(null);
  if (folderContextMenu) {
    document.addEventListener('click', handleClick);
    return () => document.removeEventListener('click', handleClick);
  }
}, [folderContextMenu]);

const toggleFlyoutFolder = (folderId: string) => {
  setFlyoutExpandedFolders(prev => {
    const newSet = new Set(prev);
    if (newSet.has(folderId)) {
      newSet.delete(folderId);
    } else {
      newSet.add(folderId);
    }
    return newSet;
  });
};

// Week folders for files flyout
const FLYOUT_WEEKS = [
  { id: "week-1", name: "Week 1" },
  { id: "week-2", name: "Week 2" },
  { id: "week-3", name: "Week 3" },
  { id: "week-4", name: "Week 4" },
  { id: "week-5", name: "Week 5" },
  { id: "week-6", name: "Week 6" },
  { id: "week-7", name: "Week 7" },
  { id: "week-8", name: "Week 8" },
  { id: "week-9", name: "Week 9" },
  { id: "week-10", name: "Week 10" },
  { id: "week-11", name: "Week 11" },
  { id: "week-12", name: "Week 12" },
  { id: "week-13", name: "Week 13" },
];

const FLYOUT_COURSES = [
  { id: "cpa122", name: "CPA122", color: "text-green-400" },
  { id: "cfnf400", name: "CFNF400", color: "text-pink-400" },
  { id: "casl101", name: "CASL101", color: "text-indigo-400" },
];

```

```

const FLYOUT_CONTENT = [
  { id: "module", name: "Module" },
  { id: "reading", name: "Reading" },
];

// Get files for a specific folder path
const getFilesInFlyoutFolder = (folderId: string) => {
  return weeklyFiles.filter(f => f.folder === folderId);
};

const getFilesInFlyoutWeek = (weekId: string) => {
  return weeklyFiles.filter(f => f.folder?.startsWith(weekId + "-"));
};

const getFilesInFlyoutCourse = (weekId: string, courseId: string) => {
  return weeklyFiles.filter(f => f.folder?.startsWith(`${weekId}-${courseId}`));
};
const [draggedFile, setDraggedFile] = useState<{ url: string; name: string } | null>(null);
const [showCelebration, setShowCelebration] = useState(false);
const [completedTaskHistory, setCompletedTaskHistory] = useState<number[]>(() => {
  const saved = localStorage.getItem('completedTaskHistory');
  return saved ? JSON.parse(saved) : [];
});

type UndoAction = {
  type: 'complete' | 'uncomplete' | 'delete' | 'edit' | 'move' | 'settings' | 'resize';
  description: string;
  data: any;
};

const [undoStack, setUndoStack] = useState<UndoAction[]>([]);
const [redoStack, setRedoStack] = useState<UndoAction[]>([]);
const celebrationAudioRef = useRef<HTMLAudioElement | null>(null);
// Shared AudioContext ref - only created by user interaction, reused for alarms
const sharedAudioContextRef = useRef<AudioContext | null>(null);

const playBoingSound = useCallback(() => {
  try {
    if (!sharedAudioContextRef.current) {
      sharedAudioContextRef.current = new (window.AudioContext || (window as any).webkitAudioContext)();
    }
    const ctx = sharedAudioContextRef.current;
    const now = ctx.currentTime;
    const osc = ctx.createOscillator();
    const gain = ctx.createGain();
    osc.type = 'sine';
    osc.frequency.setValueAtTime(300, now);
    osc.frequency.exponentialRampToValueAtTime(600, now + 0.08);
    osc.frequency.exponentialRampToValueAtTime(200, now + 0.25);
    osc.frequency.exponentialRampToValueAtTime(400, now + 0.35);
    osc.frequency.exponentialRampToValueAtTime(150, now + 0.5);
    gain.gain.setValueAtTime(0.12, now);
    gain.gain.linearRampToValueAtTime(0.15, now + 0.08);
    gain.gain.linearRampToValueAtTime(0.08, now + 0.25);
    gain.gain.linearRampToValueAtTime(0.1, now + 0.35);
    gain.gain.exponentialRampToValueAtTime(0.001, now + 0.55);
    osc.connect(gain);
    gain.connect(ctx.destination);
    osc.start(now);
    osc.stop(now + 0.55);
  } catch {}
}, []);

const mainContentRef = useRef<HTMLElement | null>(null);

// Celebration popup auto-dismiss and audio
useEffect(() => {
  if (showCelebration) {
    // Play victory fanfare sound
    if (!celebrationAudioRef.current) {
      celebrationAudioRef.current = new Audio(victoryFanfare);
    }
    celebrationAudioRef.current.currentTime = 0;
    celebrationAudioRef.current.volume = 0.7;
    celebrationAudioRef.current.play().catch(() => {});

    // Send "Hooray!" to Home Assistant TTS (Echo speaker)
    fetch("/api/tts", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ text: "Hooray! Great job!" })
    }).catch(() => {});

    // Play clapping sound pattern using Web Audio API
    const playClapping = () => {
      try {
        // Initialize or reuse shared AudioContext for future alarm sounds
        if (!sharedAudioContextRef.current) {
          sharedAudioContextRef.current = new (window.AudioContext || (window as any).webkitAudioContext)();
        }
        const audioContext = sharedAudioContextRef.current;
        const now = audioContext.currentTime;

        // Create a rhythmic clapping pattern (8 claps over 2 seconds)
        const clapTimes = [0, 0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75];

        clapTimes.forEach((time) => {
          // Create noise buffer for clap sound
          const bufferSize = audioContext.sampleRate * 0.08; // 80ms clap
          const buffer = audioContext.createBuffer(1, bufferSize, audioContext.sampleRate);
          const data = buffer.getChannelData(0);

          // Generate noise with envelope for clap-like sound
          for (let i = 0; i < bufferSize; i++) {
            const envelope = Math.exp(-i / (bufferSize * 0.15)); // Quick decay

```

```

    data[i] = (Math.random() * 2 - 1) * envelope;
  }

  const source = audioContext.createBufferSource();
  source.buffer = buffer;

  // Add high-pass filter for sharper clap sound
  const highpass = audioContext.createBiquadFilter();
  highpass.type = 'highpass';
  highpass.frequency.value = 1500;

  // Add bandpass for clap character
  const bandpass = audioContext.createBiquadFilter();
  bandpass.type = 'bandpass';
  bandpass.frequency.value = 2500;
  bandpass.Q.value = 0.5;

  const gainNode = audioContext.createGain();
  gainNode.gain.value = 0.3;

  source.connect(highpass);
  highpass.connect(bandpass);
  bandpass.connect(gainNode);
  gainNode.connect(audioContext.destination);

  source.start(now + time);
});
} catch (e) {
  // Silently fail if Web Audio API not available
}
};

playClapping();

// Play crowd cheer audio
const playCrowdCheer = () => {
  const audio = new Audio(crowdCheer);
  audio.volume = 0.7;
  audio.play().catch(err => console.log("Crowd cheer playback error:", err));
};

setTimeout(() => {
  playCrowdCheer();
}, 500);

const timer = setTimeout(() => setShowCelebration(false), 5000);
return () => clearTimeout(timer);
}
}, [showCelebration]);

// Calculate if it's nighttime in Toronto based on approximate sunrise/sunset
const computeIsNighttime = useCallback(() => {
  const torontoTime = new Date(new Date().toLocaleString('en-US', { timeZone: 'America/Toronto' }));
  const hours = torontoTime.getHours();
  const minutes = torontoTime.getMinutes();
  const currentMinutes = hours * 60 + minutes;
  const dayOfYear = Math.floor((torontoTime.getTime() - new Date(torontoTime.getFullYear(), 0, 0).getTime()) / (100...
  const seasonalFactor = Math.sin((dayOfYear - 80) * 2 * Math.PI / 365);
  const sunriseMinutes = Math.round(397 - seasonalFactor * 67);
  const sunsetMinutes = Math.round(1140 + seasonalFactor * 120);
  return currentMinutes < sunriseMinutes || currentMinutes > sunsetMinutes;
}, []);

const [isNighttime, setIsNighttime] = useState(computeIsNighttime);
useEffect(() => {
  const interval = setInterval(() => {
    setIsNighttime(prev => {
      const next = computeIsNighttime();
      return prev === next ? prev : next;
    });
  }, 300000);
  return () => clearInterval(interval);
}, [computeIsNighttime]);

const blinkSyncDelay = useMemo(() => {
  const ms = Date.now() % 1000;
  return `-${ms / 1000}s`;
}, []);

const [checkedCourses, setCheckedCourses] = useState<Record<string, boolean>>(() => {
  const completedDefaults: Record<string, boolean> = {
    'PPA101': true,
    'PPA102': true,
    'PPA125': true,
    'L1_PPA120': true,
    'L1_PPA121': true,
    'LIBERAL': true,
  };
  const saved = localStorage.getItem('checkedCourses');
  const existing = saved ? JSON.parse(saved) : {};
  delete existing['OPEN1'];
  delete existing['OPEN2'];
  delete existing['L2_OPEN1'];
  const merged = { ...existing, ...completedDefaults };
  localStorage.setItem('checkedCourses', JSON.stringify(merged));
  return merged;
});

const [inProgressCourses, setInProgressCourses] = useState<Record<string, boolean>>(() => {
  const ipDefaults: Record<string, boolean> = {
    'L1_PPA122': true,
    'OPEN1': true,
    'OPEN2': true,
    'L2_LIBERAL': true,
    'L2_OPEN1': true,
    'L1_PPA120': false,
    'L1_PPA121': false,
  };
  const saved = localStorage.getItem('inProgressCourses');
  const existing = saved ? JSON.parse(saved) : {};

```

```

delete existing['ELECTIVE1'];
delete existing['ELECTIVE2'];
const merged = { ...existing, ...ipDefaults };
localStorage.setItem('InProgressCourses', JSON.stringify(merged));
return merged;
});
const toggleInProgress = (courseId: string) => {
  setInProgressCourses(prev => {
    const updated = { ...prev, [courseId]: !prev[courseId] };
    localStorage.setItem('InProgressCourses', JSON.stringify(updated));
    saveDegreeToServer('InProgressCourses', updated);
    return updated;
  });
};

// Profile state
const [isProfileDialogOpen, setIsProfileDialogOpen] = useState(false);
const [isHamburgerOpen, setIsHamburgerOpen] = useState(false);
const isHamburgerOpenRef = useRef(false);
isHamburgerOpenRef.current = isHamburgerOpen;
const hamburgerCloseTimer = useRef<number | null>(null);
const [profilePhotoUrl, setProfilePhotoUrl] = useState<string | null>(() => {
  return localStorage.getItem('profilePhotoUrl');
});
const [isSchoolDialogOpen, setIsSchoolDialogOpen] = useState(false);
const [isSchoolCoursesDialogOpen, setIsSchoolCoursesDialogOpen] = useState(false);
const [otherRowEditOpen, setOtherRowEditOpen] = useState(false);
const [otherRowColors, setOtherRowColors] = useState<{
  labelStart: string;
  labelEnd: string;
  labelStops: string;
  cellBg: string;
  borderColor: string;
  taskBgColor: string;
  courseRowColor: string;
}>(() => {
  try {
    const saved = localStorage.getItem('otherRowColors');
    if (saved) { const p = JSON.parse(saved); return { ...p, courseRowColor: p.courseRowColor || '#3a3f4a' }; }
  } catch {}
  return { labelStart: '#374151', labelEnd: '#9ca3af', labelStops: '', cellBg: '#3a3f4a', borderColor: '#5c6370', t...
});
const saveOtherRowColors = useCallback((colors: typeof otherRowColors) => {
  setOtherRowColors(colors);
  localStorage.setItem('otherRowColors', JSON.stringify(colors));
  fetch('/api/degree-tracking', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ key: 'otherRowColors', value: colors }),
  }).catch(() => {});
}, []);
const [schoolCoursesOpenSource, setSchoolCoursesOpenSource] = useState<'pill' | 'degree'>('degree');
const [semSettingsDialogKey, setSemSettingsDialogKey] = useState<string | null>(null);
const [perSemesterSettings, setPerSemesterSettings] = useState<Record<string, { week1StartDate: string; semesterTyp...
  try { const s = localStorage.getItem('perSemesterSettings'); return s ? JSON.parse(s) : {}; } catch { return {}; }
});
useEffect(() => {
  for (const [semKey, data] of Object.entries(perSemesterSettings)) {
    if (semKey.startsWith('ss')) {
      const d = data as any;
      if (d.springStartDate || d.springEndDate || d.summerStartDate || d.summerEndDate) {
        apiRequest("POST", "/api/semester-halves", { semKey, springStartDate: d.springStartDate || '', springEndDat...
      }
    }
  }
}, []);
const coursesScrollRef = useRef<HTMLDivElement>(null);
const [coursesScrolled, setCoursesScrolled] = useState(false);

type SemCourse = { code: string; name: string; fullName?: string; period: string };
const defaultSemesterCourses: Record<string, SemCourse[]> = {
  'ss2025': [
    { code: 'CPPA101', name: 'CPPA 101', fullName: 'Cdn Public Admin I: Institutions', period: 'May 5 - Jun 20' },
    { code: 'CPPA120', name: 'CPPA 120', fullName: 'Canadian Politics and Government', period: 'May 5 - Jun 20' },
    { code: 'CPPA102', name: 'CPPA 102', fullName: 'Cdn Public Admin II: Processes', period: 'Jun 23 - Aug 8' },
  ],
  'f2025': [
    { code: 'CPPA125', name: 'CPPA 125', fullName: 'Rights, Equity and the State', period: '' },
    { code: 'CGCM738', name: 'CGCM 738', fullName: 'Photoshopped!', period: '' },
    { code: 'CPPA121', name: 'CPPA 121', fullName: 'Ontario Politics and Government', period: '' },
  ],
  'w2026': [
    { code: 'CPPA122', name: 'CPPA 122', fullName: 'Local Politics and Government', period: '' },
    { code: 'CFNF400', name: 'CFNF 400', fullName: 'Human Sexuality', period: '' },
    { code: 'CASL101', name: 'CASL 101', fullName: 'ASL', period: '' },
  ],
  'ss2026': [
    { code: 'CECN210', name: 'CECN 210', fullName: 'Understanding Economics', period: 'May 4 - Jul 31' },
    { code: 'CPHL110', name: 'CPHL 110', fullName: 'Philosophy of Religion', period: 'May 5 - Jun 16' },
    { code: 'CHIS105', name: 'CHIS 105', fullName: 'Inventing Popular Culture', period: 'Jun 23 - Aug 4' },
  ],
  'f2026': [
    { code: 'CPPA235', name: 'CPPA 235', fullName: 'Theories of the State', period: '' },
  ],
};
const [semesterCourseAssignments, setSemesterCourseAssignments] = useState<Record<string, SemCourse[]>>(() => {
  const saved = localStorage.getItem('semesterCourseAssignments');
  if (saved) {
    try { return JSON.parse(saved); } catch {}
  }
  return defaultSemesterCourses;
});
const saveSemesterAssignments = (assignments: Record<string, SemCourse[]>) => {
  setSemesterCourseAssignments(assignments);
  localStorage.setItem('semesterCourseAssignments', JSON.stringify(assignments));
  saveDegreeToServer('semesterCourseAssignments', assignments);
};
const semesterKeyOrder = ['ss2025', 'f2025', 'w2026', 'ss2026', 'f2026', 'w2027', 'ss2027', 'f2027', 'w2028', 'ss20...

```

```

const addCourseToNextAvailableSemester = (courseCode: string, courseName: string, fullName: string) => {
  const codeNorm = courseCode.toUpperCase().replace(/s/g, '');
  const allAssigned = Object.values(semesterCourseAssignments).flat();
  if (allAssigned.some(c => c.code.toUpperCase().replace(/s/g, '') === codeNorm)) return;
  const now = new Date();
  const semStartDatesMap: Record<string, string> = {
    'ss2025': '2025-05-05', 'f2025': '2025-09-08', 'w2026': '2026-01-12',
    'ss2026': '2026-05-04', 'f2026': '2026-09-07', 'w2027': '2027-01-11',
    'ss2027': '2027-05-03', 'f2027': '2027-09-10', 'w2028': '2028-01-10', 'ss2028': '2028-05-01', 'f2028': '2028-09-04',
  };
  const futureSems = semesterKeyOrder.filter(k => {
    const start = semStartDatesMap[k];
    return start && new Date(start) >= new Date(now.getFullYear(), now.getMonth(), 1);
  });
  const targetSem = futureSems.find(k => { semesterCourseAssignments[k] || [].length < 3 } || futureSems[0] || semes...
  if (!targetSem) return;
  const updated = { ...semesterCourseAssignments };
  const courses = [...(updated[targetSem] || [])];
  courses.push({ code: courseCode, name: courseName, fullName, period: '' });
  updated[targetSem] = courses;
  saveSemesterAssignments(updated);
};
const dragCourseRef = useRef<{ code: string; fromSemKey: string } | null>(null);
const [draftCoursePlayPriority, setDraftCoursePlayPriority] = useState<Record<string, number>>({});
const draftCoursePlayPriorityRef = useRef<Record<string, number>>({});
const setDraftPriorityBoth = useCallback((val: Record<string, number> | ((prev: Record<string, number>) => Record<string, number>))) => {
  if (typeof val === 'function') {
    setDraftCoursePlayPriority(prev => {
      const next = val(prev);
      draftCoursePlayPriorityRef.current = next;
      return next;
    });
  } else {
    draftCoursePlayPriorityRef.current = val;
    setDraftCoursePlayPriority(val);
  }
}, []);
const handlePriorityChange = useCallback((priorityKey: string, val: number) => {
  setDraftPriorityBoth(prev => ({ ...prev, [priorityKey]: val }));
  setCoursePlayPriority(prev => {
    const updated = { ...prev, [priorityKey]: val };
    localStorage.setItem('coursePlayPriority', JSON.stringify(updated));
    fetch('/api/course-play-priority', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSO...
    return updated;
  });
}, [setDraftPriorityBoth]);
const [isRemainingCoursesDialogOpen, setIsRemainingCoursesDialogOpen] = useState(false);
const [pastCourseInfo, setPastCourseInfo] = useState<Record<string, { professor: string; email: string; grade: stri...
const courseInfoDefaults: Record<string, { professor: string; email: string; grade: string; semester: string; cre...
  'PPA101': { professor: 'Tom McDowell', email: 'thomas.mcdowell@torontomu.ca', grade: 'A-', semester: 'Spring/Su...
  'CPA101': { professor: 'Tom McDowell', email: 'thomas.mcdowell@torontomu.ca', grade: 'A-', semester: 'Spring/S...
  'PPA102': { professor: 'Tom McDowell', email: 'thomas.mcdowell@torontomu.ca', grade: 'A', semester: 'Spring/Sum...
  'CPA102': { professor: 'Tom McDowell', email: 'thomas.mcdowell@torontomu.ca', grade: 'A', semester: 'Spring/Su...
  'PPA125': { professor: 'Jennifer Zubick', email: 'jzubick@torontomu.ca', grade: 'A+', semester: 'Fall 2025', cr...
  'PPA120': { professor: 'Ryan Phillips', email: 'ryan.j.phillips@torontomu.ca', grade: 'A-', semester: 'Spring/S...
  'CPA120': { professor: 'Ryan Phillips', email: 'ryan.j.phillips@torontomu.ca', grade: 'A-', semester: 'Spring/...
  'PPA121': { professor: 'Justin Rain', email: 'jrain@torontomu.ca', grade: 'B+', semester: 'Fall 2025', credits:...
  'CPA121': { professor: 'Justin Rain', email: 'jrain@torontomu.ca', grade: 'B+', semester: 'Fall 2025', credits:...
  'CPA125': { professor: 'Jennifer Zubick', email: 'jzubick@torontomu.ca', grade: 'A+', semester: 'Fall 2025', c...
  'CCGM738': { professor: 'Shelley Haines', email: 'shelley.haines@torontomu.ca', grade: 'A-', semester: 'Fall 20...
  'CAS101': { professor: '', email: '', grade: '', semester: 'Winter 2026', credits: '1.00' },
  'CFNF400': { professor: '', email: '', grade: '', semester: 'Winter 2026', credits: '1.00' },
  'CPA122': { professor: '', email: '', grade: '', semester: 'Winter 2026', credits: '1.00' },
  'CECN210': { professor: '', email: '', grade: '', semester: 'Spring/Summer 2026', credits: '1.00' },
  'CPH110': { professor: '', email: '', grade: '', semester: 'Spring/Summer 2026', credits: '1.00' },
  'CHIS105': { professor: '', email: '', grade: '', semester: 'Spring/Summer 2026', credits: '1.00' },
};
try {
  const saved = localStorage.getItem('pastCourseInfo');
  const existing = saved ? JSON.parse(saved) : {};
  const merged: Record<string, { professor: string; email: string; grade: string; semester: string; credits: stri...
  for (const [key, val] of Object.entries(existing) as [string, any][]) {
    if (merged[key]) {
      merged[key] = {
        ...merged[key],
        ...val,
        professor: val.professor || merged[key].professor,
        email: val.email || merged[key].email,
        grade: val.grade || merged[key].grade,
      };
    } else {
      merged[key] = val;
    }
  }
  localStorage.setItem('pastCourseInfo', JSON.stringify(merged));
  return merged;
} catch { return courseInfoDefaults; }
});
const [editingSchoolCourseKey, setEditingSchoolCourseKey] = useState<string | null>(null);
const [editingSchoolCourseData, setEditingSchoolCourseData] = useState<{ professor: '', email: '', grade: '', semes...
const [courseCertificateTypes, setCourseCertificateTypes] = useState<Record<string, string>>({}) => {
  try {
    const saved = localStorage.getItem('courseCertificateTypes');
    return saved ? JSON.parse(saved) : {};
  } catch { return {}; }
});
const [coursePlayPriority, setCoursePlayPriority] = useState<Record<string, number>>({}) => {
  try {
    const saved = localStorage.getItem('coursePlayPriority');
    const parsed = saved ? JSON.parse(saved) : {};
    if (Object.keys(parsed).length > 0) {
      fetch('/api/course-play-priority', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: s...
    }
    return parsed;
  } catch { return {}; }
});
const SEMESTER_COURSE_DEFS = [
  { key: 'ss2025', start: '2025-05-05', end: '2025-08-08', codes: ['CPA101', 'CPA120', 'CPA102'] },

```

```

    { key: 'f2025', start: '2025-09-08', end: '2025-12-12', codes: ['CPPA125', 'CGCM738', 'CPPA121'] },
    { key: 'w2026', start: '2026-01-12', end: '2026-04-17', codes: ['CPPA122', 'CFNF400', 'CASL101'] },
    { key: 'ss2026', start: '2026-05-04', end: '2026-08-07', codes: ['CECN210', 'CPHL110', 'CHIS105'] },
  ];

  useEffect(() => {
    const now = new Date();
    const activeSem = SEMESTER_COURSE_DEFS.find(s => now >= new Date(s.start) && now <= new Date(s.end));
    if (!activeSem) return;
    const isSS = activeSem.key.startsWith('ss');
    const allRanked = activeSem.codes.every(code => {
      if (isSS) {
        const keyA = `${activeSem.key}:${code}:A`;
        const keyB = `${activeSem.key}:${code}:B`;
        return (coursePlayPriority[keyA] && coursePlayPriority[keyA] > 0) || (coursePlayPriority[keyB] && coursePlayP...
      }
      const key = `${activeSem.key}:${code}`;
      return coursePlayPriority[key] && coursePlayPriority[key] > 0;
    });
    if (allRanked) {
      setShowRankCoursesReminder(false);
      return;
    }
    const dismissKey = `rankCoursesReminderDismissed_${activeSem.key}`;
    const dismissed = localStorage.getItem(dismissKey);
    if (!dismissed) {
      const timer = setTimeout(() => setShowRankCoursesReminder(true), 3000);
      return () => clearTimeout(timer);
    }
  }, [coursePlayPriority]);

  const CERTIFICATE_TYPE_OPTIONS = [
    { group: 'Certificate 1', options: [
      'C1 - Mandatory Required Professional',
      'C1 - Selected Required Professional',
      'C1 - Liberal Studies Elective - Lower Level Table A',
      'C1 - Professionally Related (Open) Elective Table 1',
    ]},
    { group: 'Certificate 2', options: [
      'C2 - Mandatory Required Professional',
      'C2 - Selected Required Professional',
      'C2 - Liberal Studies Elective - Lower Level Table A',
      'C2 - Professionally Related Elective - Upper Level Table B',
      'C2 - Open Elective',
    ]},
    { group: 'Certificate 3', options: [
      'C3 - Required Group 1',
      'C3 - Required Group 2',
      'C3 - Liberal Studies Elective Lower Level Table A',
      'C3 - Liberal Studies Elective Upper Level Table B',
      'C3 - Open Elective',
    ]},
  ];

  const [courseDisplayNames, setCourseDisplayNames] = useState<Record<string, string>>(() => {
    const saved = localStorage.getItem('courseDisplayNames');
    if (saved) {
      const parsed = JSON.parse(saved);
      _courseDisplayNames = { ...defaultCourseDisplayNames, ...parsed };
      return _courseDisplayNames;
    }
    return { ...defaultCourseDisplayNames };
  });

  const [isSettingsDialogOpen, setIsSettingsDialogOpen] = useState(false);
  // Discussion post checkbox states (persisted per week in localStorage)
  const [startDiscussionComplete, setStartDiscussionComplete] = useState<boolean>(() => {
    const saved = localStorage.getItem('discussionStart_week${selectedWeek}');
    return saved === 'true';
  });

  const [discussionDueComplete, setDiscussionDueComplete] = useState<boolean>(() => {
    const saved = localStorage.getItem('discussionDue_week${selectedWeek}');
    return saved === 'true';
  });

  // Check if today is Friday (day 5) for blinking the Discussion Due column
  const isFriday = new Date().getDay() === 5;

  // Blinking and spacing settings
  const [blinkSettings, setBlinkSettings] = useState<{
    todayColumnBlink: boolean;
    allDayFilesBlink: boolean;
    taskBoxFilesBlink: boolean;
    todayColumnBlinkSpeed: number;
    allDayFilesBlinkSpeed: number;
    taskBoxFilesBlinkSpeed: number;
    buttonSpacing: number;
    mediaControlSpacing: number;
    tallPillButtonSpacing: number;
    tallPillHeight: number;
  }>(() => {
    const saved = localStorage.getItem('blinkSettings');
    const parsed = saved ? JSON.parse(saved) : {};
    return {
      todayColumnBlink: parsed.todayColumnBlink ?? false,
      allDayFilesBlink: parsed.allDayFilesBlink ?? true,
      taskBoxFilesBlink: parsed.taskBoxFilesBlink ?? true,
      todayColumnBlinkSpeed: parsed.todayColumnBlinkSpeed ?? parsed.blinkSpeed ?? 0.6,
      allDayFilesBlinkSpeed: parsed.allDayFilesBlinkSpeed ?? parsed.blinkSpeed ?? 0.6,
      taskBoxFilesBlinkSpeed: parsed.taskBoxFilesBlinkSpeed ?? parsed.blinkSpeed ?? 0.6,
      buttonSpacing: parsed.buttonSpacing ?? 32,
      mediaControlSpacing: parsed.mediaControlSpacing ?? 32,
      tallPillButtonSpacing: parsed.tallPillButtonSpacing ?? 0,
      tallPillHeight: parsed.tallPillHeight ?? 0
    };
  });

  // Save blink settings to localStorage (debounced to avoid lag)
  const blinkSettingsRef = useRef(blinkSettings);
  blinkSettingsRef.current = blinkSettings;

```



```

useEffect(() => {
  const timer = setTimeout(() => {
    localStorage.setItem('blinkSettings', JSON.stringify(blinkSettingsRef.current));
    fetch('/api/degree-tracking', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ key: 'blinkSettings', value: blinkSettingsRef.current }),
    }).catch(() => {});
  }, 1000);
  return () => clearTimeout(timer);
}, [blinkSettings]);

// TEST: Isolated progress bar position (completely separate from task columns)
// Load from localStorage on init
const testProgressBarLeft = 0;
const testTextLeft = 8;
const testCourseLeft = 12;
const testCourseNameLeft = 8;
const testDueDateLeft = 8;

// Task column widths - resizable
const [taskColumnWidths, setTaskColumnWidths] = useState<{
  taskGap: number;
  taskName: number;
  courseCode: number;
  courseName: number;
  dueDate: number;
}>(() => {
  const saved = localStorage.getItem('taskColumnWidths');
  const parsed = saved ? JSON.parse(saved) : {};
  return {
    taskGap: parsed.taskGap ?? 0,
    taskName: parsed.taskName ?? 48,
    courseCode: parsed.courseCode ?? 100,
    courseName: parsed.courseName ?? 145,
    dueDate: parsed.dueDate ?? 55
  };
});

// Save column widths to localStorage
useEffect(() => {
  localStorage.setItem('taskColumnWidths', JSON.stringify(taskColumnWidths));
}, [taskColumnWidths]);

// Column resize state
const [resizingColumn, setResizingColumn] = useState<string | null>(null);
const resizeStartX = useRef(0);
const resizeStartWidth = useRef(0);

// Handle task column resize (inverted = true for left-side handles)
const resizeInvertedRef = useRef(false);
const handleTaskColumnResizeStart = (e: React.MouseEvent | React.TouchEvent, column: string, inverted = false) => {
  e.preventDefault();
  e.stopPropagation();
  const { clientX } = getPointerXY(e);
  setResizingColumn(column);
  resizeStartX.current = clientX;
  resizeStartWidth.current = taskColumnWidths[column as keyof typeof taskColumnWidths];
  resizeInvertedRef.current = inverted;
};

useEffect(() => {
  if (!resizingColumn) return;

  const handleMove = (e: MouseEvent | TouchEvent) => {
    const { clientX } = getPointerXY(e);
    const diff = clientX - resizeStartX.current;
    const adjustedDiff = resizeInvertedRef.current ? -diff : diff;
    const newWidth = Math.max(30, Math.min(300, resizeStartWidth.current + adjustedDiff));
    setTaskColumnWidths(prev => ({
      ...prev,
      [resizingColumn]: newWidth
    }));
  };

  const handleEnd = () => {
    setResizingColumn(null);
  };

  document.addEventListener('mousemove', handleMove);
  document.addEventListener('mouseup', handleEnd);
  document.addEventListener('touchmove', handleMove, { passive: false });
  document.addEventListener('touchend', handleEnd);
  return () => {
    document.removeEventListener('mousemove', handleMove);
    document.removeEventListener('mouseup', handleEnd);
    document.removeEventListener('touchmove', handleMove);
    document.removeEventListener('touchend', handleEnd);
  };
}, [resizingColumn]);

// Save and reload discussion post checkboxes when week changes
useEffect(() => {
  localStorage.setItem('discussionStart_week${selectedWeek}', String(startDiscussionComplete));
}, [startDiscussionComplete, selectedWeek]);

useEffect(() => {
  localStorage.setItem('discussionDue_week${selectedWeek}', String(discussionDueComplete));
}, [discussionDueComplete, selectedWeek]);

// Reset discussion checkboxes when week changes
useEffect(() => {
  const savedStart = localStorage.getItem('discussionStart_week${selectedWeek}');
  const savedDue = localStorage.getItem('discussionDue_week${selectedWeek}');
  setStartDiscussionComplete(savedStart === 'true');
  setDiscussionDueComplete(savedDue === 'true');
}, [selectedWeek]);

```

```

const refreshFileCounts = useCallback(async (retryCount = 0) => {
  try {
    const response = await fetch('/api/files/counts');
    if (response.ok) {
      const counts = await response.json();

      setFileCounts(counts);
      const legacyCounts: Record<string, number> = {};
      for (const [key, value] of Object.entries(counts)) {
        legacyCounts[key] = (value as { total: number }).total;
      }
      setOneDriveFileCounts(legacyCounts);
    }

    try {
      const odController = new AbortController();
      const odTimeout = setTimeout(() => odController.abort(), 10000);
      const odResponse = await fetch(`/api/onedrive/week-counts/${selectedWeek}`, { signal: odController.signal });
      clearTimeout(odTimeout);
      if (odResponse.ok) {
        const odCounts = await odResponse.json();
        const merged = { ...counts };
        for (const [key, value] of Object.entries(odCounts)) {
          const existing = merged[key] as any;
          const odVal = value as any;
          if (!existing || existing.total === 0) {
            merged[key] = value;
          } else if (odVal && odVal.total > existing.total) {
            merged[key] = { ...existing, total: odVal.total, unlisted: odVal.total - existing.listened };
          }
        }
        setFileCounts(merged);
        const mergedLegacy: Record<string, number> = {};
        for (const [key, value] of Object.entries(merged)) {
          mergedLegacy[key] = (value as { total: number }).total;
        }
        setOneDriveFileCounts(mergedLegacy);
      }
    } catch (odError) {
      if ((odError as any)?.name !== 'AbortError') {
        console.error('Error fetching OneDrive week counts:', odError);
      }
    }
  } else if (retryCount < 2) {
    setTimeout(() => refreshFileCounts(retryCount + 1), 1500);
  }
} catch (error) {
  console.error('Error fetching file counts:', error);
  try { fetch('/api/client-error', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON....
  if (retryCount < 2) {
    setTimeout(() => refreshFileCounts(retryCount + 1), 1500);
  }
}
}, [selectedWeek]);

useEffect(() => {
  refreshFileCounts();
}, [refreshFileCounts]);

// Close modules/readings honeycomb when clicking outside
useEffect(() => {
  const handleClickOutside = (e: MouseEvent) => {
    const target = e.target as HTMLElement;
    // Don't close if clicking date navigation arrows
    const isDateNav = target.closest('[data-date-nav]');
    if (isDateNav) return;

    // Don't close if clicking resize handle
    const isResizeHandle = target.closest('[data-resize-handle]');
    if (isResizeHandle) return;

    // Check if click is on modules button or course buttons
    if (modulesHoneycombOpen === 'modules') {
      const isModulesButton = target.closest('[data-modules-button]');
      const isCourseButton = target.closest('[data-course-button]');
      if (!isModulesButton && !isCourseButton) {
        setModulesHoneycombOpen(null);
      }
    }

    // Check if click is on readings button or readings course buttons
    if (modulesHoneycombOpen === 'readings') {
      const isReadingsButton = target.closest('[data-readings-button]');
      const isReadingsCourseButton = target.closest('[data-readings-course-button]');
      if (!isReadingsButton && !isReadingsCourseButton) {
        setModulesHoneycombOpen(null);
      }
    }
  };
  document.addEventListener('click', handleClickOutside);
  return () => document.removeEventListener('click', handleClickOutside);
}, [modulesHoneycombOpen]);

// Default box order: Today, Tomorrow, This Week (left to right)
const getDefaultBoxOrder = (): string[] => {
  return ['today', 'tomorrow', 'this-week'];
};

// Box order state - initialized from default, but can be dragged
const [boxOrder, setBoxOrder] = useState<string[]>(() => {
  // Always use the fixed default order: Today, Tomorrow, This Week
  return getDefaultBoxOrder();
});

// Reset box order at midnight (when date changes)
useEffect(() => {
  const checkMidnight = () => {
    const savedDate = localStorage.getItem('boxOrderDate');
    const today = new Date().toDateString();
  };

```

```

    if (savedDate !== today) {
      const newOrder = getDefaultBoxOrder();
      setBoxOrder(newOrder);
      localStorage.setItem('boxOrder', JSON.stringify(newOrder));
      localStorage.setItem('boxOrderDate', today);
    }
  };

  // Check every minute for midnight
  const interval = setInterval(checkMidnight, 60000);

  // Also save current date on mount
  localStorage.setItem('boxOrderDate', new Date().toDateString());

  return () => clearInterval(interval);
}, []);

// Color settings
const isValidHex = (v: string) => /^[0-9a-fA-F]{3}([0-9a-fA-F]{3})?$/i.test(v);
const safeHex = (v: string, fallback: string) => isValidHex(v) ? v : fallback;

const [colorSettings, setColorSettings] = useState<{
  boxBackground: string;
  headerBar: string;
  mainBackground: string;
  boxGlassEffect: boolean;
  boxTransparency: number;
  mainBackgroundOverlay: boolean;
  mainBackgroundGradient: boolean;
  mainBackgroundGradientEnd: string;
  todayCellBackground: string;
  currentHourRowBackground: string;
  todayCurrentHourCellBackground: string;
  backgroundPhoto: string | null;
}>(() => {
  const saved = localStorage.getItem('colorSettings');
  const defaults = {
    boxBackground: 'ffffff',
    headerBar: '#051729',
    mainBackground: '#3a8bbf',
    boxGlassEffect: true,
    boxTransparency: 20,
    mainBackgroundOverlay: true,
    mainBackgroundGradient: true,
    mainBackgroundGradientEnd: '#164a72',
    todayCellBackground: '#d4d4d4',
    currentHourRowBackground: '#d4d4d4',
    todayCurrentHourCellBackground: '#051729',
    backgroundPhoto: null as string | null
  };
  // Force migration V16: slightly less frosted than original 35
  const migrationV25 = localStorage.getItem('colorSettingsMigrationV25');
  if (!migrationV25) {
    localStorage.removeItem('colorSettings');
    localStorage.removeItem('colorSettingsMigrationV17');
    localStorage.removeItem('colorSettingsMigrationV18');
    localStorage.removeItem('colorSettingsMigrationV18b');
    localStorage.removeItem('colorSettingsMigrationV19');
    localStorage.removeItem('colorSettingsMigrationV20');
    localStorage.removeItem('colorSettingsMigrationV21');
    localStorage.removeItem('colorSettingsMigrationV22');
    localStorage.removeItem('colorSettingsMigrationV23');
    localStorage.removeItem('colorSettingsMigrationV24');
    localStorage.setItem('colorSettingsMigrationV25', 'done');
    return defaults;
  }
  const migrationDone = localStorage.getItem('colorSettingsMigrationV17');
  if (!migrationDone) {
    const existing = saved ? JSON.parse(saved) : {};
    const migrated = { ...defaults, ...existing, mainBackground: existing.mainBackground || '#3a8bbf', mainBackgrou...
    localStorage.setItem('colorSettings', JSON.stringify(migrated));
    localStorage.setItem('colorSettingsMigrationV17', 'done');
    return migrated;
  }
  if (saved) {
    const parsed = JSON.parse(saved);
    return { ...defaults, ...parsed };
  }
  return defaults;
});

useEffect(() => {
  const bg = `linear-gradient(180deg, ${colorSettings.mainBackground} 0%, color-mix(in srgb, ${colorSettings.mainBa...
  document.documentElement.style.setProperty('--dialog-bg', bg);
}, [colorSettings.mainBackground, colorSettings.mainBackgroundGradientEnd]);

const saveDegreeKeyDebounce = useCallback((key: string, value: any) => {
  fetch('/api/degree-tracking', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ key, value }),
  }).catch(() => {});
}, []);

useEffect(() => {
  const timer = setTimeout(() => {
    localStorage.setItem('colorSettings', JSON.stringify(colorSettings));
    saveDegreeKeyDebounce('colorSettings', colorSettings);
  }, 1000);
  return () => clearTimeout(timer);
}, [colorSettings]);

// Store original settings when dialog opens (for cancel functionality)
const [originalColorSettings, setOriginalColorSettings] = useState(colorSettings);
const [originalBlinkSettings, setOriginalBlinkSettings] = useState(blinkSettings);

// Generate a device-specific identifier based on screen dimensions

```

```

// Grid size settings for resizable calendar columns and rows
const [gridSizes, setGridSizes] = useState<{
  timeColumnWidth: number;
  moduleColumnWidth: number;
  dayColumnWidths: number[];
  progressColumnWidth: number;
  allDayRowHeight: number;
  courseRowHeight: number;
  otherRowHeight: number;
  timeSlotHeight: number;
  timeSlotHeights: number[]; // Individual heights for each hour (0-23)
}>(() => {
  const defaultHeights = Array(24).fill(36);
  const defaultSizes = {
    timeColumnWidth: 59,
    moduleColumnWidth: 0,
    dayColumnWidths: [1, 1, 1, 1, 1, 1, 1], // flex proportions for 7 days (Sun-Sat)
    progressColumnWidth: 0.87, // homework progress column
    allDayRowHeight: 36,
    courseRowHeight: 36,
    otherRowHeight: 57,
    timeSlotHeight: 36,
    timeSlotHeights: defaultHeights
  };

  const migrateOldWidths = (parsed: any) => {
    if (parsed.dayColumnWidths && parsed.dayColumnWidths.length === 8) {
      const oldWidths = parsed.dayColumnWidths;
      parsed.progressColumnWidth = oldWidths[6];
      parsed.dayColumnWidths = [...oldWidths.slice(0, 6), oldWidths[7]];
    }
    parsed.dayColumnWidths = [1, 1, 1, 1, 1, 1, 1];
    parsed.progressColumnWidth = 0.87;
  };

  // Check for device-specific saved settings first
  const screenWidth = window.screen.width;
  const screenHeight = window.screen.height;
  const pixelRatio = window.devicePixelRatio || 1;
  const deviceId = `device_${screenWidth}x${screenHeight}@${pixelRatio}`;
  const deviceSaved = localStorage.getItem(`gridSizes_${deviceId}`);

  if (deviceSaved) {
    const parsed = JSON.parse(deviceSaved);
    if (!parsed.timeSlotHeights || parsed.timeSlotHeights.length !== 24) {
      parsed.timeSlotHeights = defaultHeights;
    }
    [0,1,2,3,4,5,6,21,22,23].forEach(i => { if (parsed.timeSlotHeights[i] < 36) parsed.timeSlotHeights[i] = 36; });
    if (!parsed.timeSlotHeight) parsed.timeSlotHeight = 36;
    if (!parsed.courseRowHeight) parsed.courseRowHeight = 48;
    if (!parsed.otherRowHeight || parsed.otherRowHeight < 57) parsed.otherRowHeight = 57;
    if (parsed.moduleColumnWidth === undefined) parsed.moduleColumnWidth = 0;
    if (!parsed.timeColumnWidth) parsed.timeColumnWidth = 59;
    migrateOldWidths(parsed);
    return parsed;
  }

  // Fall back to general saved settings
  const saved = localStorage.getItem('gridSizes');
  if (saved) {
    const parsed = JSON.parse(saved);
    if (!parsed.timeSlotHeights || parsed.timeSlotHeights.length !== 24) {
      parsed.timeSlotHeights = defaultHeights;
    }
    [0,1,2,3,4,5,6,21,22,23].forEach(i => { if (parsed.timeSlotHeights[i] < 36) parsed.timeSlotHeights[i] = 36; });
    if (!parsed.timeSlotHeight) parsed.timeSlotHeight = 36;
    if (!parsed.courseRowHeight) parsed.courseRowHeight = 48;
    if (!parsed.otherRowHeight || parsed.otherRowHeight < 57) parsed.otherRowHeight = 57;
    if (parsed.moduleColumnWidth === undefined) parsed.moduleColumnWidth = 0;
    if (!parsed.timeColumnWidth) parsed.timeColumnWidth = 59;
    migrateOldWidths(parsed);
    return parsed;
  }
  return defaultSizes;
});

useEffect(() => {
  localStorage.setItem('gridSizes', JSON.stringify(gridSizes));
  const sw = window.screen.width, sh = window.screen.height, pr = window.devicePixelRatio || 1;
  localStorage.setItem(`gridSizes_device_${sw}x${sh}@${pr}`, JSON.stringify(gridSizes));
  const timer = setTimeout(() => {
    saveDegreeKeyDebounce('gridSizes', gridSizes);
  }, 1500);
  return () => clearTimeout(timer);
}, [gridSizes]);

// Save calendar height to localStorage
useEffect(() => {
  localStorage.setItem('calendarHeight', calendarHeight.toString());
  const sw = window.screen.width, sh = window.screen.height, pr = window.devicePixelRatio || 1;
  localStorage.setItem(`calendarHeight_device_${sw}x${sh}@${pr}`, calendarHeight.toString());
  const timer = setTimeout(() => {
    saveDegreeKeyDebounce('calendarHeight', calendarHeight);
  }, 1500);
  return () => clearTimeout(timer);
}, [calendarHeight]);

// Column resize state
const [columnResizing, setColumnResizing] = useState<{
  isResizing: boolean;
  columnIndex: number; // -1 for time column, 0-6 for day columns
  startX: number;
  startWidth: number;
} | null>(null);

// Row resize state
const [rowResizing, setRowResizing] = useState<{

```

```

    isResizing: boolean;
    rowType: 'allDay' | 'course' | 'timeSlot';
    hourIndex?: number; // For individual time slot row resizing
    startY: number;
    startHeight: number;
  } | null>(null);

// Context menu state for right-click delete
const [recurringDeleteTask, setRecurringDeleteTask] = useState<{ id: number; title: string; callback?: () => void }...
const [recurringEditPending, setRecurringEditPending] = useState<{ taskId: number; title: string; payload: Record<s...
const [contextMenu, setContextMenu] = useState<{
  x: number;
  y: number;
  taskId: number;
  taskTitle: string;
} | null>(null);

// Long-press timer for touch devices
const longPressTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
const longPressTouchRef = useRef<{ x: number; y: number; taskId: number; taskTitle: string } | null>(null);

const handleTouchStart = (e: React.TouchEvent, taskId: number, taskTitle: string) => {
  const touch = e.touches[0];
  longPressTouchRef.current = { x: touch.clientX, y: touch.clientY, taskId, taskTitle };
  longPressTimerRef.current = setTimeout(() => {
    if (longPressTouchRef.current) {
      setContextMenu({
        x: longPressTouchRef.current.x,
        y: longPressTouchRef.current.y,
        taskId: longPressTouchRef.current.taskId,
        taskTitle: longPressTouchRef.current.taskTitle
      });
      longPressTouchRef.current = null;
    }
  }, 500); // 500ms long press
};

const handleTouchEnd = () => {
  if (longPressTimerRef.current) {
    clearTimeout(longPressTimerRef.current);
    longPressTimerRef.current = null;
  }
  longPressTouchRef.current = null;
};

const handleTouchMove = () => {
  // Cancel long press if user moves finger
  if (longPressTimerRef.current) {
    clearTimeout(longPressTimerRef.current);
    longPressTimerRef.current = null;
  }
  longPressTouchRef.current = null;
};

// Close context menu when clicking elsewhere
useEffect(() => {
  const handleClickOutside = () => setContextMenu(null);
  if (contextMenu) {
    document.addEventListener('click', handleClickOutside);
    document.addEventListener('contextmenu', handleClickOutside);
    return () => {
      document.removeEventListener('click', handleClickOutside);
      document.removeEventListener('contextmenu', handleClickOutside);
    };
  }
}, [contextMenu]);

// Handle column resize
const handleColumnResizeStart = (e: React.MouseEvent | React.TouchEvent, columnIndex: number) => {
  e.preventDefault();
  e.stopPropagation();
  const { clientX } = getPointerXY(e);
  const startWidth = columnIndex === -1
    ? gridSizes.timeColumnWidth
    : columnIndex === -2
    ? gridSizes.moduleColumnWidth
    : gridSizes.dayColumnWidths[columnIndex];
  setColumnResizing({
    isResizing: true,
    columnIndex,
    startX: clientX,
    startWidth
  });
};

// Handle row resize
const handleHwGroupBarDragStart = (e: React.MouseEvent | React.TouchEvent) => {
  e.preventDefault();
  e.stopPropagation();
  const startX = 'touches' in e ? e.touches[0].clientX : e.clientX;
  hwGroupBarDragRef.current = { startX, startWidth: hwGroupBarWidth };
  const onMove = (ev: MouseEvent | TouchEvent) => {
    if (!hwGroupBarDragRef.current) return;
    const clientX = 'touches' in ev ? ev.touches[0].clientX : (ev as MouseEvent).clientX;
    const dx = clientX - hwGroupBarDragRef.current.startX;
    const newW = Math.max(90, Math.min(141, hwGroupBarDragRef.current.startWidth + dx));
    setHwGroupBarWidth(newW);
  };
  const onUp = () => {
    document.removeEventListener('mousemove', onMove);
    document.removeEventListener('mouseup', onUp);
    document.removeEventListener('touchmove', onMove);
    document.removeEventListener('touchend', onUp);
    setHwGroupBarWidth(w => { localStorage.setItem('hwGroupBarWidth', String(w)); fetch('/api/ui-settings/hwGroupBa...
    hwGroupBarDragRef.current = null;
  };
  document.addEventListener('mousemove', onMove);
  document.addEventListener('mouseup', onUp);

```

```

    document.addEventListener('touchmove', onMove);
    document.addEventListener('touchend', onUp);
  });

const handleHwDividerDragStart = (e: React.MouseEvent | React.TouchEvent) => {
  e.preventDefault();
  e.stopPropagation();
  const container = homeworkSectionRef.current;
  if (!container) return;
  const containerWidth = container.getBoundingClientRect().width;
  const startX = 'touches' in e ? e.touches[0].clientX : e.clientX;
  hwDividerDragRef.current = { startX, startPercent: hwDividerPercent, containerWidth };
  const onMove = (ev: MouseEvent | TouchEvent) => {
    if (!hwDividerDragRef.current) return;
    const clientX = 'touches' in ev ? ev.touches[0].clientX : (ev as MouseEvent).clientX;
    const dx = clientX - hwDividerDragRef.current.startX;
    const dpct = (dx / hwDividerDragRef.current.containerWidth) * 100;
    const newPct = Math.max(20, Math.min(63, hwDividerDragRef.current.startPercent + dpct));
    setHwDividerPercent(newPct);
  };
  const onUp = () => {
    document.removeEventListener('mousemove', onMove);
    document.removeEventListener('mouseup', onUp);
    document.removeEventListener('touchmove', onMove);
    document.removeEventListener('touchend', onUp);
    setHwDividerPercent(pct => { localStorage.setItem('hwDividerPercent', String(pct)); return pct; });
    hwDividerDragRef.current = null;
  };
  document.addEventListener('mousemove', onMove);
  document.addEventListener('mouseup', onUp);
  document.addEventListener('touchmove', onMove);
  document.addEventListener('touchend', onUp);
};

const handleRowResizeStart = (e: React.MouseEvent | React.TouchEvent, rowType: 'allDay' | 'course' | 'other' | 'tim...
  e.preventDefault();
  e.stopPropagation();
  let startHeight: number;
  if (rowType === 'allDay') {
    startHeight = gridSizes.allDayRowHeight;
  } else if (rowType === 'course') {
    startHeight = gridSizes.courseRowHeight;
  } else if (rowType === 'other') {
    startHeight = gridSizes.otherRowHeight;
  } else if (hourIndex !== undefined) {
    startHeight = gridSizes.timeSlotHeights[hourIndex];
  } else {
    startHeight = gridSizes.timeSlotHeight;
  }
  const { clientY } = getPointerXY(e);
  setRowResizing({
    isResizing: true,
    rowType,
    hourIndex,
    startY: clientY,
    startHeight
  });
};

// Global move/end handlers for resizing (mouse + touch)
useEffect(() => {
  const handleMove = (e: MouseEvent | TouchEvent) => {
    const { clientX, clientY } = getPointerXY(e);
    if (columnResizing?.isResizing) {
      const delta = clientX - columnResizing.startX;
      if (columnResizing.columnIndex === -1) {
        const newWidth = Math.max(115, Math.min(200, columnResizing.startWidth + delta));
        setGridSizes(prev => ({ ...prev, timeColumnWidth: newWidth }));
      } else if (columnResizing.columnIndex === -2) {
        const newWidth = Math.max(50, Math.min(150, columnResizing.startWidth + delta));
        setGridSizes(prev => ({ ...prev, moduleColumnWidth: newWidth }));
      } else {
        const newWidths = [...gridSizes.dayColumnWidths];
        const newProportion = Math.max(0.5, columnResizing.startWidth + delta / 100);
        newWidths[columnResizing.columnIndex] = newProportion;
        setGridSizes(prev => ({ ...prev, dayColumnWidths: newWidths }));
      }
    }
    if (rowResizing?.isResizing) {
      const delta = clientY - rowResizing.startY;
      const newHeight = Math.max(20, rowResizing.startHeight + delta);
      if (rowResizing.rowType === 'allDay') {
        setGridSizes(prev => ({ ...prev, allDayRowHeight: Math.min(100, newHeight) }));
      } else if (rowResizing.rowType === 'course') {
        setGridSizes(prev => ({ ...prev, courseRowHeight: Math.min(82, newHeight) }));
      } else if (rowResizing.rowType === 'other') {
        setGridSizes(prev => ({ ...prev, otherRowHeight: Math.min(150, newHeight) }));
      } else if (rowResizing.rowType === 'timeSlot' && rowResizing.hourIndex !== undefined) {
        setGridSizes(prev => {
          const newHeights = [...prev.timeSlotHeights];
          newHeights[rowResizing.hourIndex!] = Math.max(24, Math.min(150, newHeight));
          return { ...prev, timeSlotHeights: newHeights };
        });
      } else if (rowResizing.rowType === 'timeSlot') {
        setGridSizes(prev => ({ ...prev, timeSlotHeight: Math.min(100, newHeight) }));
      }
    }
    if (isResizingHomework && resizingHomeworkRef.current) {
      const delta = resizingHomeworkRef.current.startX - clientX;
      const newReduction = Math.max(120, Math.min(window.innerWidth * 0.45, resizingHomeworkRef.current.startReduct...
      setCalendarReduction(newReduction);
      setCalendarReductionUserSet(true);
    }
  };
};

const handleEnd = () => {
  if (isResizingHomework && resizingHomeworkRef.current) {
    const oldReduction = resizingHomeworkRef.current.startReduction;

```

```

    if (oldReduction !== calendarReduction) {
      setUndoStack(prev => [{ type: 'resize', description: 'Resize calendar/homework width', data: { resizeType: ...
        setRedoStack([]);
      }
      setIsResizingHomework(false);
      resizingHomeworkRef.current = null;
      localStorage.setItem('calendarReduction', String(calendarReduction));
      const sw = window.screen.width, sh = window.screen.height, pr = window.devicePixelRatio || 1;
      localStorage.setItem(`calendarReduction_device_${sw}x${sh}@${pr}`, String(calendarReduction));
    } else if (isResizingHomework) {
      setIsResizingHomework(false);
      resizingHomeworkRef.current = null;
      localStorage.setItem('calendarReduction', String(calendarReduction));
      const sw = window.screen.width, sh = window.screen.height, pr = window.devicePixelRatio || 1;
      localStorage.setItem(`calendarReduction_device_${sw}x${sh}@${pr}`, String(calendarReduction));
    }
  }
  if (columnResizing?.isResizing || rowResizing?.isResizing) {
    setTimeout(() => {
      const cur = JSON.stringify(gridSizes);
      localStorage.setItem('gridSizes', cur);
      const sw = window.screen.width, sh = window.screen.height, pr = window.devicePixelRatio || 1;
      localStorage.setItem(`gridSizes_device_${sw}x${sh}@${pr}`, cur);
    }, 0);
  }
  setColumnResizing(null);
  setRowResizing(null);
};

if (columnResizing?.isResizing || rowResizing?.isResizing || isResizingHomework) {
  document.addEventListener('mousemove', handleMove);
  document.addEventListener('mouseup', handleEnd);
  document.addEventListener('touchmove', handleMove, { passive: false });
  document.addEventListener('touchend', handleEnd);
  document.body.style.cursor = (columnResizing?.isResizing || isResizingHomework) ? 'col-resize' : 'row-resize';
  document.body.style.userSelect = 'none';
}

return () => {
  document.removeEventListener('mousemove', handleMove);
  document.removeEventListener('mouseup', handleEnd);
  document.removeEventListener('touchmove', handleMove);
  document.removeEventListener('touchend', handleEnd);
  document.body.style.cursor = '';
  document.body.style.userSelect = '';
};
}, [columnResizing, rowResizing, gridSizes.dayColumnWidths, isResizingHomework, calendarReduction]);

const [draggedBox, setDraggedBox] = useState<string | null>(null);
const [thisWeekBoxHeight, setThisWeekBoxHeight] = useState<number | null>(null);
const [isResizingThisWeek, setIsResizingThisWeek] = useState(false);
const thisWeekResizeStartY = useRef<number>(0);
const thisWeekResizeStartHeight = useRef<number>(0);

// Refs to measure first row column positions for alignment
const row1TaskRef = useRef<HTMLDivElement>(null);
const row1CodeRef = useRef<HTMLDivElement>(null);
const row1CourseRef = useRef<HTMLDivElement>(null);
const row1DueRef = useRef<HTMLDivElement>(null);
const row1DaysRef = useRef<HTMLDivElement>(null);
const row1ProgressBarRef = useRef<HTMLDivElement>(null);
const row1ContainerRef = useRef<HTMLDivElement>(null);
// HARDCODED header positions - NEVER change without explicit user permission
// remaining=25, task=75, code=177, course=214, due=365, days=right-aligned to right edge
const HEADER_POS = { remaining: 25, task: 77, code: 187, course: 230, due: 367 };
const [row1Positions, setRow1Positions] = useState({ task: 70, code: 166, course: 234, due: 331, days: 400, progres...

// Save box order to localStorage
useEffect(() => {
  localStorage.setItem('boxOrder', JSON.stringify(boxOrder));
}, [boxOrder]);

const handleBoxDragStart = (boxId: string) => {
  setDraggedBox(boxId);
};

const handleBoxDragOver = (e: React.DragEvent, targetBoxId: string) => {
  e.preventDefault();
  if (draggedBox && draggedBox !== targetBoxId) {
    const newOrder = [...boxOrder];
    const draggedIdx = newOrder.indexOf(draggedBox);
    const targetIdx = newOrder.indexOf(targetBoxId);
    newOrder.splice(draggedIdx, 1);
    newOrder.splice(targetIdx, 0, draggedBox);
    setBoxOrder(newOrder);
  }
};

const handleBoxDragEnd = () => {
  setDraggedBox(null);
};

// This Week box resize handlers
const handleThisWeekResizeStart = (e: React.MouseEvent | React.TouchEvent) => {
  e.preventDefault();
  const { clientY } = getPointerXY(e);
  setIsResizingThisWeek(true);
  thisWeekResizeStartY.current = clientY;
  const section = (e.target as HTMLElement).closest('section');
  thisWeekResizeStartHeight.current = section?.offsetHeight || 125;
};

useEffect(() => {
  if (!isResizingThisWeek) return;

  const handleMove = (e: MouseEvent | TouchEvent) => {
    const { clientY } = getPointerXY(e);
    const deltaY = clientY - thisWeekResizeStartY.current;
    const newHeight = Math.max(85, thisWeekResizeStartHeight.current + deltaY);

```

```

    setThisWeekBoxHeight(newHeight);
  };

  const handleEnd = () => {
    setIsResizingThisWeek(false);
  };

  document.addEventListener('mousemove', handleMove);
  document.addEventListener('mouseup', handleEnd);
  document.addEventListener('touchmove', handleMove, { passive: false });
  document.addEventListener('touchend', handleEnd);
  return () => {
    document.removeEventListener('mousemove', handleMove);
    document.removeEventListener('mouseup', handleEnd);
    document.removeEventListener('touchmove', handleMove);
    document.removeEventListener('touchend', handleEnd);
  };
}, [isResizingThisWeek]);

const [profileData, setProfileData] = useState<{ firstName: string; lastName: string; birthdate: string; timezone: ...
const saved = localStorage.getItem('profileData');
return saved ? { postalCode: '', location: '', phoneNumber: '', email: '', address: '', country: '', provinceStat...
});
const [schoolData, setSchoolData] = useState<{ schoolLogo: string | null; schoolName: string; numberOfWeeks: number...
const saved = localStorage.getItem('schoolData');
if (saved) {
  const parsed = JSON.parse(saved);
  if (!parsed.lastDayOfSchoolWeek) parsed.lastDayOfSchoolWeek = 'friday';
  return { schoolName: 'Toronto Metropolitan University', ...parsed };
}
return { schoolLogo: null, schoolName: 'Toronto Metropolitan University', numberOfWeeks: 13, week1StartDate: '202...
});

useEffect(() => {
  if (schoolData.isTravelling) {
    if (schoolData.travelEndDate) {
      const rawEnd = schoolData.travelEndDate;
      const endDate = rawEnd.includes('T') ? new Date(rawEnd) : new Date(rawEnd + 'T23:59:59');
      if (new Date() > endDate) {
        const updated = { ...schoolData, isTravelling: false, travelTimezone: undefined, travelStartDate: undefined...
        setSchoolData(updated);
        localStorage.setItem('schoolData', JSON.stringify(updated));
        saveDegreeToServer('schoolData', updated);
        setProfileData(prev => {
          const updatedProfile = { ...prev, travelTimezone: null };
          localStorage.setItem('profileData', JSON.stringify(updatedProfile));
          return updatedProfile;
        });
        fetch('/api/travelling', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ isTravelling: false, startDate: null, endDate: null }),
        }).catch(err => console.error('Failed to sync travel expiry:', err));
        console.log('Travel mode auto-ended: travel end date has passed');
        return;
      }
    }
    fetch('/api/travelling', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        isTravelling: true,
        startDate: schoolData.travelStartDate || null,
        endDate: schoolData.travelEndDate || null,
      }),
    }).catch(err => console.error('Failed to sync initial travelling state:', err));
  }
}, []);

// Derive last school day from firstDayOfWeek (separator goes between last day and first day)
const lastSchoolDayIndex = (() => {
  const dayMap: Record<string, number> = { sunday: 0, monday: 1, tuesday: 2, wednesday: 3, thursday: 4, friday: 5, ...
  const firstDayIdx = dayMap[schoolData.firstDayOfWeek] ?? 6;
  return firstDayIdx === 0 ? 6 : firstDayIdx - 1;
})();

// Progress column grid position (1-based, accounting for time column and optional module column)
const saturdayGridCol = (gridSizes.moduleColumnWidth > 0 ? 2 : 1) + lastSchoolDayIndex + 1 + 1;

const DAY_COL_LEFT_REDUCTION = 10;
const getGridTemplateColumns = () => {
  const allDays = gridSizes.dayColumnWidths.map(w => `${w}fr`).join(' ');
  if (gridSizes.moduleColumnWidth > 0) {
    return `${gridSizes.timeColumnWidth}px ${gridSizes.moduleColumnWidth}px ${allDays}`;
  }
  return `${gridSizes.timeColumnWidth}px ${allDays}`;
};

const [coursesData, setCoursesData] = useState<{ courses: Array<{ name: string; color: string; colorEnd?: string; c...
const defaultCourses = [
  { name: 'CPPA122 - Local Politics and Government', color: '#0F5004', colorEnd: '#47B045', professor: 'Caryl Aru...
  { name: 'CFNF400 - Human Sexuality', color: '#DE1864', colorEnd: '#FA67B3', professor: 'Alex McKay', professorE...
  { name: 'CASL101 - ASL', color: '#974B8A', colorEnd: '#B045A2', professor: 'Christina Moreau', professorEmail: ...
  { name: 'CECN210 - Understanding Economics', color: '#059669', colorEnd: '#34D399', professor: '', professorEma...
  { name: 'CPHL110 - Philosophy of Religion', color: '#2563EB', colorEnd: '#60A5FA', professor: '', professorEmai...
  { name: 'CHIS105 - Inventing Popular Culture', color: '#DC2626', colorEnd: '#F87171', professor: '', professorE...
  { name: 'CPPA235 - Issues in Canadian Politics', color: '#8B6914', colorEnd: '#CD853F', professor: '', professo...
  { name: '', color: '#6b7280', colorEnd: '#9ca3af', professor: '', professorEmail: '' },
  { name: '', color: '#6b7280', colorEnd: '#9ca3af', professor: '', professorEmail: '' },
  { name: '', color: '#6b7280', colorEnd: '#9ca3af', professor: '', professorEmail: '' },
  { name: '', color: '#6b7280', colorEnd: '#9ca3af', professor: '', professorEmail: '' },
  { name: '', color: '#6b7280', colorEnd: '#9ca3af', professor: '', professorEmail: '' },
];
// Always use defaults for professor emails to ensure they stay current
const saved = localStorage.getItem('coursesData');
if (saved) {
  const parsed = JSON.parse(saved);
  const hasNamedCourses = parsed.courses?.some((c: { name: string }) => c.name.trim());
  if (hasNamedCourses) {

```



```

const coursesWithProfessor = parsed.courses.map((c: { name: string; color: string; colorEnd?: string; profess...
  ...c,
  name: c.name?.trim() ? c.name : defaultCourses[i]?.name ?? '',
  color: c.color || defaultCourses[i]?.color || '#6b7280',
  colorEnd: c.colorEnd || defaultCourses[i]?.colorEnd || '',
  professor: c.professor ?? defaultCourses[i]?.professor ?? '',
  professorEmail: defaultCourses[i]?.professorEmail ?? ''
}));
const savedNames = coursesWithProfessor.map((c: { name: string }) => c.name?.split(' - ')[0]?.trim().toUpperCase...
for (const dc of defaultCourses) {
  if (dc.name && !savedNames.includes(dc.name.split(' - ')[0]?.trim().toUpperCase())) {
    const emptyIdx = coursesWithProfessor.findIndex((c: { name: string }) => !c.name?.trim());
    if (emptyIdx >= 0) {
      coursesWithProfessor[emptyIdx] = { ...dc };
    } else {
      coursesWithProfessor.push({ ...dc });
    }
  }
}
localStorage.setItem('coursesData', JSON.stringify({ courses: coursesWithProfessor }));
return { courses: coursesWithProfessor };
}
return { courses: defaultCourses };
}
return { courses: defaultCourses };
});
const [isCoursesDialogOpen, setIsCoursesDialogOpen] = useState(false);
const [isNewCourseDialogOpen, setIsNewCourseDialogOpen] = useState(false);
const [isCourseListUploadOpen, setIsCourseListUploadOpen] = useState(false);
const [courseListChanges, setCourseListChanges] = useState<Array<{
  id: string;
  changeType: 'new' | 'modified' | 'removed';
  category: string;
  description: string;
  details: Record<string, { old: string; new: string }>;
  parsed: Record<string, unknown> | null;
  existingTaskId: number | null;
  accepted: boolean;
}>>([]);
const [courseListUploadCourse, setCourseListUploadCourse] = useState('');
const [courseListParsing, setCourseListParsing] = useState(false);
const [courseListApplying, setCourseListApplying] = useState(false);
const [courseListPendingFile, setCourseListPendingFile] = useState<File | null>(null);
const [courseListFileRef, useRef<HTMLInputElement>(null);
const [isNewCourseWizardOpen, setIsNewCourseWizardOpen] = useState(false);
const [newCourseDialogClosingRef, useRef(false);

const [aasSentStatus, setAasSentStatus] = useState<Record<string, boolean>>(() => {
  const saved = localStorage.getItem('aasSentStatus');
  const parsed = saved ? JSON.parse(saved) : {};
  const pastCourses = ['CPPA101', 'CPPA 101', 'CPPA120', 'CPPA 120', 'CPPA102', 'CPPA 102', 'CPPA125', 'CPPA 125', ...
  let changed = false;
  for (const c of pastCourses) {
    if (parsed[c] === undefined) { parsed[c] = true; changed = true; }
  }
  if (changed) localStorage.setItem('aasSentStatus', JSON.stringify(parsed));
  return parsed;
});
const [showAasReminder, setShowAasReminder] = useState(false);
const [showRankCoursesReminder, setShowRankCoursesReminder] = useState(false);

const toggleAasSent = (courseCode: string) => {
  const norm = courseCode.replace(/s/g, '');
  const withSpace = courseCode.includes(' ') ? courseCode : courseCode.replace(/^[A-Z]+(\d)/, '$1 $2');
  const updated = { ...aasSentStatus };
  const newVal = !updated[norm] || updated[withSpace] || updated[courseCode];
  updated[norm] = newVal;
  updated[withSpace] = newVal;
  updated[courseCode] = newVal;
  setAasSentStatus(updated);
  localStorage.setItem('aasSentStatus', JSON.stringify(updated));
};

useEffect(() => {
  if (!isSchoolCoursesDialogOpen) return;
  const semStartDates: Record<string, string> = {
    'ss2025': '2025-05-05', 'f2025': '2025-09-08', 'w2026': '2026-01-12',
    'ss2026': '2026-05-04', 'f2026': '2026-09-07', 'w2027': '2027-01-11',
    'ss2027': '2027-05-03', 'f2027': '2027-09-13', 'w2028': '2028-01-10', 'ss2028': '2028-05-01', 'f2028': '2028-09-...'
  };
  const semesterDefs = [
    { key: 'ss2025', courses: ['CPPA101', 'CPPA120', 'CPPA102'] },
    { key: 'f2025', courses: ['CPPA125', 'CGCM738', 'CPPA121'] },
    { key: 'w2026', courses: ['CPPA122', 'CFNF400', 'CASL101'] },
    { key: 'ss2026', courses: ['CECN210', 'CPHL110', 'CHIS105'] },
  ];
  const hasUncheckedStartedAas = semesterDefs.some(sem => {
    const start = semStartDates[sem.key];
    if (!start || new Date() < new Date(start)) return false;
    return sem.courses.some(code => !aasSentStatus[code] && !aasSentStatus[code.replace(/^[A-Z]+(\d)/, '$1 $2')]);
  });
  if (!hasUncheckedStartedAas) return;
  const initialDelay = setTimeout(() => playBoingSound(), 500);
  const interval = setInterval(() => playBoingSound(), 10000);
  return () => { clearTimeout(initialDelay); clearInterval(interval); };
}, [isSchoolCoursesDialogOpen, aasSentStatus, playBoingSound]);

// Get the display timezone (travel if set, otherwise home)
const displayTimezone = profileData.travelTimezone || profileData.timezone;
const displayTimezoneRef = useRef(displayTimezone);
displayTimezoneRef.current = displayTimezone;

const toggleCourse = (courseId: string) => {
  setCheckedCourses(prev => {
    const updated = { ...prev, [courseId]: !prev[courseId] };
    localStorage.setItem('checkedCourses', JSON.stringify(updated));
    saveDegreeToServer('checkedCourses', updated);
    return updated;
  });
}

```

```

    });
  });

const saveProfile = (data: { firstName: string; lastName: string; birthdate: string; timezone: string; travelTimezo...
  setProfileData(data);
  localStorage.setItem('profileData', JSON.stringify(data));
  setIsProfileDialogOpen(false);
  toast({ title: "Profile saved", description: "Your profile has been updated." });
});

const saveSchool = (data: { schoolLogo: string | null; schoolName: string; numberOfWeeks: number; week1StartDate: s...
  const { semesterType: semType, ...schoolOnly } = data;
  setSchoolData(schoolOnly);
  localStorage.setItem('schoolData', JSON.stringify(schoolOnly));
  saveDegreeToServer('schoolData', schoolOnly);
  if (semType && semesterSettings) {
    saveSemesterScheduleMutation.mutate({ semesterType: semType });
  }
  fetch('/api/travelling', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      isTravelling: !!data.isTravelling,
      startDate: data.travelStartDate || null,
      endDate: data.travelEndDate || null,
    }),
  }).catch(err => console.error('Failed to sync travelling state:', err));
  setIsSchoolDialogOpen(false);
  toast({ title: "School settings saved", description: "Your school settings have been updated." });
});

const saveCourses = (data: { courses: Array<{ name: string; color: string; colorEnd?: string; colorStops?: string; ...
  const prevCourses = coursesData?.courses || [];
  setCoursesData(data);
  localStorage.setItem('coursesData', JSON.stringify(data));
  saveDegreeToServer('coursesData', data);
  setIsCoursesDialogOpen(false);
  toast({ title: "Courses saved", description: "Your courses have been updated." });
  const semPayload: Record<string, string | null> = {};
  for (let i = 0; i < 3; i++) {
    const course = data.courses[i] as any;
    const prefix = `course${i + 1}`;
    if (course) {
      semPayload[`${prefix}Color`] = course.color || null;
      semPayload[`${prefix}ColorEnd`] = course.colorEnd || null;
      semPayload[`${prefix}ColorStops`] = course.colorStops || null;
      semPayload[`${prefix}BorderColor`] = course.borderColor || null;
      semPayload[`${prefix}CourseRowColor`] = course.courseRowColor || null;
      semPayload[`${prefix}TaskBgColor`] = course.taskBgColor || null;
      const parts = course.name.split(' - ');
      const code = (parts[0] || '').trim();
      const name = parts.slice(1).join(' - ').trim();
      semPayload[`${prefix}Code`] = code || null;
      semPayload[`${prefix}Name`] = course.name || null;
      semPayload[`${prefix}Professor`] = course.professor || null;
      semPayload[`${prefix}ProfessorEmail`] = course.professorEmail || null;
    } else {
      semPayload[`${prefix}Color`] = null;
      semPayload[`${prefix}ColorEnd`] = null;
      semPayload[`${prefix}Code`] = null;
      semPayload[`${prefix}Name`] = null;
    }
  }
  fetch('/api/semester', { method: 'PATCH', headers: { 'Content-Type': 'application/json' }, credentials: 'include'...
    .then(() => {
      queryClient.invalidateQueries({ queryKey: ["/api/semester"] });
      queryClient.invalidateQueries({ queryKey: ["/api/semesters"] });
    })
    .catch(() => {});

  for (let i = 0; i < 3; i++) {
    const newCourse = data.courses[i];
    const oldCourse = prevCourses[i];
    if (!newCourse?.name?.trim()) continue;
    const newParts = newCourse.name.split(' - ');
    const newCode = (newParts[0] || '').trim();
    const newName = newParts.slice(1).join(' - ').trim();
    const oldParts = (oldCourse?.name || '').split(' - ');
    const oldCode = (oldParts[0] || '').trim();
    const oldName = oldParts.slice(1).join(' - ').trim();
    if (newCourse && (newCode !== oldCode || newName !== oldName)) {
      fetch('/api/onedrive/rename-course-folder', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        credentials: 'include',
        body: JSON.stringify({
          courseIndex: i + 1,
          oldCode: oldCode || newCode,
          oldName: oldName,
          newCode,
          newName,
        }),
      }).catch(() => {});
    }
  }
});

const saveSemesterScheduleMutation = useMutation({
  mutationFn: async (data: Record<string, any>) => {
    return apiRequest("PATCH", "/api/semester", data);
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/semester"] });
    queryClient.invalidateQueries({ queryKey: ["/api/semesters"] });
    setIsCoursesDialogOpen(false);
    toast({ title: "Schedule saved", description: "Course schedule has been updated." });
  },
});

```

```

const generateClassTasksMutation = useMutation({
  mutationFn: async () => {
    return apiRequest("POST", "/api/semester/generate-class-tasks", {});
  },
  onSuccess: async (response: any) => {
    const data = await response.json();
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    toast({ title: "Class Tasks Created", description: data.message || "Class tasks have been generated." });
  },
  onError: () => {
    toast({ title: "Error", description: "Failed to generate class tasks.", variant: "destructive" });
  },
});

// TTS settings for word highlighting synchronization
const [ttsSettings, setTtsSettings] = useState<{
  startDelay: number; // seconds before TTS starts speaking
  wordsPerMinute: number; // speech rate in words per minute
  useSmartTiming: boolean; // adjust timing based on word length
}>(() => {
  const saved = localStorage.getItem('ttsSettings');
  if (saved) {
    try {
      return JSON.parse(saved);
    } catch {}
  }
  return {
    startDelay: 27, // 27 seconds default delay
    wordsPerMinute: 120, // 120 WPM default (2 words per second)
    useSmartTiming: true, // enabled by default
  };
});

const saveTtsSettings = (settings: typeof ttsSettings) => {
  setTtsSettings(settings);
  localStorage.setItem('ttsSettings', JSON.stringify(settings));
  toast({ title: "TTS settings saved", description: "Your text-to-speech highlighting settings have been updated." });
};

const generateShareLink = async () => {
  setIsGeneratingLink(true);
  try {
    const response = await fetch('/api/access-tokens', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name: `Share link ${new Date().toLocaleDateString()}` }),
    });
    const token = await response.json();
    const link = `${window.location.origin}?access=${token.token}`;
    setShareLink(link);
    setIsShareDialogOpen(true);
  } catch (err) {
    toast({ title: "Error", description: "Failed to generate share link", variant: "destructive" });
  } finally {
    setIsGeneratingLink(false);
  }
};

const copyShareLink = () => {
  navigator.clipboard.writeText(shareLink);
  toast({ title: "Copied!", description: "Share link copied to clipboard" });
};

const getCourseColor = (courseName: string): string => {
  const course = coursesData.courses.find(c => c.name && courseName.includes(c.name.split(' - ')[0]));
  return course?.color || '#6b7280';
};

// Helper function to convert hex to RGB
const hexToRgb = (hex: string): { r: number; g: number; b: number } => {
  const result = /^#?([a-f\d]{2})([a-f\d]{2})([a-f\d]{2})$/i.exec(hex);
  return result ? {
    r: parseInt(result[1], 16),
    g: parseInt(result[2], 16),
    b: parseInt(result[3], 16)
  } : { r: 107, g: 114, b: 128 }; // gray fallback
};

const dimColor = (color: string, factor = 0.5): string => {
  const rgbaMatch = color.match(/rgba?\((\d+),\s*(\d+),\s*(\d+)\)/);
  if (rgbaMatch) {
    const r = Math.round(parseInt(rgbaMatch[1]) + (255 - parseInt(rgbaMatch[1])) * factor);
    const g = Math.round(parseInt(rgbaMatch[2]) + (255 - parseInt(rgbaMatch[2])) * factor);
    const b = Math.round(parseInt(rgbaMatch[3]) + (255 - parseInt(rgbaMatch[3])) * factor);
    return `rgb(${r}, ${g}, ${b})`;
  }
  const { r, g, b } = hexToRgb(color);
  const dr = Math.round(r + (255 - r) * factor);
  const dg = Math.round(g + (255 - g) * factor);
  const db = Math.round(b + (255 - b) * factor);
  return `rgb(${dr}, ${dg}, ${db})`;
};

const getCourseGradientColors = (courseCode: string): { start: string; end: string } => {
  const course = coursesData.courses.find(c => c.name?.split(' - ')[0]??.toUpperCase() === courseCode.toUpperCase());
  if (course?.colorEnd) return { start: course.color, end: course.colorEnd };
  if (course) {
    const rgb = hexToRgb(course.color);
    return { start: course.color, end: `rgb(${Math.min(255, rgb.r + 100)}, ${Math.min(255, rgb.g + 100)}, ${Math.min(255, rgb.b + 100)})` };
  }
  return { start: '#6b7280', end: '#9ca3af' };
};

const getButtonGradient = (hex: string, hexEnd?: string): string => {
  if (hexEnd) return `linear-gradient(180deg, ${hex} 0%, ${hexEnd} 100%)`;
  const rgb = hexToRgb(hex);
  const darkerR = Math.max(0, rgb.r - 40);

```

```

const darkerG = Math.max(0, rgb.g - 40);
const darkerB = Math.max(0, rgb.b - 40);
const lighterR = Math.min(255, rgb.r + 100);
const lighterG = Math.min(255, rgb.g + 100);
const lighterB = Math.min(255, rgb.b + 100);
return `linear-gradient(180deg, rgb(${darkerR}, ${darkerG}, ${darkerB}) 0%, rgb(${lighterR}, ${lighterG}, ${lighterB}) 100%)`;
};

const getBorderGradient = (hex: string, hexEnd?: string): string => {
  if (hexEnd) return `linear-gradient(0deg, ${hex} 0%, ${hexEnd} 100%)`;
  const rgb = hexToRgb(hex);
  const darkerR = Math.max(0, rgb.r - 40);
  const darkerG = Math.max(0, rgb.g - 40);
  const darkerB = Math.max(0, rgb.b - 40);
  const lighterR = Math.min(255, rgb.r + 100);
  const lighterG = Math.min(255, rgb.g + 100);
  const lighterB = Math.min(255, rgb.b + 100);
  return `linear-gradient(0deg, rgb(${darkerR}, ${darkerG}, ${darkerB}) 0%, rgb(${lighterR}, ${lighterG}, ${lighterB}) 100%)`;
};

// Dynamic course colors based on coursesData
const dynamicCourseColors = useMemo(() => {
  const colors: Record<string, { bg: string; mid: string; border: string; text: string; dot: string; prepBg: string...
  coursesData.courses.forEach(course => {
    if (!course.name) return;
    const courseCode = course.name.split(' - ')[0].toUpperCase();
    const hex = course.color;
    const hexEnd = course.colorEnd;
    const displayHex = hexEnd || hex;
    const rgb = hexToRgb(displayHex);
    const endRgb = hexEnd ? hexToRgb(hexEnd) : rgb;

    const midR = Math.round((rgb.r + endRgb.r) / 2);
    const midG = Math.round((rgb.g + endRgb.g) / 2);
    const midB = Math.round((rgb.b + endRgb.b) / 2);
    colors[courseCode] = {
      hex,
      hexEnd,
      bg: `rgb(${Math.round(endRgb.r + (255 - endRgb.r) * 0.55)}, ${Math.round(endRgb.g + (255 - endRgb.g) * 0.55)})...`,
      mid: `rgb(${midR}, ${midG}, ${midB})`,
      border: displayHex,
      text: displayHex,
      dot: displayHex,
      prepBg: `rgba(${rgb.r}, ${rgb.g}, ${rgb.b}, 0.1)`,
      prepBorder: `rgba(${rgb.r}, ${rgb.g}, ${rgb.b}, 0.4)`,
      prepText: `rgba(${rgb.r}, ${rgb.g}, ${rgb.b}, 0.8)`
    };
  });

  return colors;
}, [coursesData]);

// Common timezones
const timezones = [
  // North America
  { value: 'America/Toronto', label: 'Eastern Time (Toronto)' },
  { value: 'America/New_York', label: 'Eastern Time (New York)' },
  { value: 'America/Chicago', label: 'Central Time (Chicago)' },
  { value: 'America/Denver', label: 'Mountain Time (Denver)' },
  { value: 'America/Phoenix', label: 'Arizona (Phoenix)' },
  { value: 'America/Los_Angeles', label: 'Pacific Time (Los Angeles)' },
  { value: 'America/Vancouver', label: 'Pacific Time (Vancouver)' },
  { value: 'America/Anchorage', label: 'Alaska (Anchorage)' },
  { value: 'Pacific/Honolulu', label: 'Hawaii (Honolulu)' },
  // Central/South America
  { value: 'America/Mexico_City', label: 'Mexico City' },
  { value: 'America/Bogota', label: 'Bogota' },
  { value: 'America/Lima', label: 'Lima' },
  { value: 'America/Sao_Paulo', label: 'São Paulo' },
  { value: 'America/Buenos_Aires', label: 'Buenos Aires' },
  // Europe
  { value: 'Europe/London', label: 'London (GMT/BST)' },
  { value: 'Europe/Dublin', label: 'Dublin' },
  { value: 'Europe/Paris', label: 'Paris (CET/CEST)' },
  { value: 'Europe/Berlin', label: 'Berlin (CET/CEST)' },
  { value: 'Europe/Amsterdam', label: 'Amsterdam' },
  { value: 'Europe/Rome', label: 'Rome' },
  { value: 'Europe/Madrid', label: 'Madrid' },
  { value: 'Europe/Zurich', label: 'Zurich' },
  { value: 'Europe/Stockholm', label: 'Stockholm' },
  { value: 'Europe/Vienna', label: 'Vienna' },
  { value: 'Europe/Prague', label: 'Prague' },
  { value: 'Europe/Warsaw', label: 'Warsaw' },
  { value: 'Europe/Athens', label: 'Athens' },
  { value: 'Europe/Istanbul', label: 'Istanbul' },
  { value: 'Europe/Moscow', label: 'Moscow' },
  // Middle East & Africa
  { value: 'Asia/Dubai', label: 'Dubai (GST)' },
  { value: 'Asia/Jerusalem', label: 'Jerusalem' },
  { value: 'Africa/Cairo', label: 'Cairo' },
  { value: 'Africa/Johannesburg', label: 'Johannesburg' },
  { value: 'Africa/Lagos', label: 'Lagos' },
  // Asia
  { value: 'Asia/Kolkata', label: 'India (Mumbai/Delhi)' },
  { value: 'Asia/Bangkok', label: 'Bangkok' },
  { value: 'Asia/Singapore', label: 'Singapore' },
  { value: 'Asia/Hong_Kong', label: 'Hong Kong' },
  { value: 'Asia/Shanghai', label: 'Shanghai (CST)' },
  { value: 'Asia/Taipei', label: 'Taipei' },
  { value: 'Asia/Seoul', label: 'Seoul' },
  { value: 'Asia/Tokyo', label: 'Tokyo (JST)' },
  // Oceania
  { value: 'Australia/Perth', label: 'Perth (AWST)' },
  { value: 'Australia/Adelaide', label: 'Adelaide (ACST/ACDT)' },
  { value: 'Australia/Brisbane', label: 'Brisbane (AEST)' },
  { value: 'Australia/Sydney', label: 'Sydney (AEST/AEDT)' },
  { value: 'Australia/Melbourne', label: 'Melbourne (AEST/AEDT)' },

```

```

    { value: 'Pacific/Auckland', label: 'Auckland (NZST/NZDT)' },
    { value: 'Pacific/Fiji', label: 'Fiji' },
  ];

const [courseGrades, setCourseGrades] = useState<Record<string, { grade: string; percent: string }>>(() => {
  const completedGrades: Record<string, { grade: string; percent: string }> = {
    'PPA101': { grade: 'A-', percent: '82' },
    'PPA102': { grade: 'A', percent: '87' },
    'PPA125': { grade: 'A+', percent: '93' },
    'L1_PPA120': { grade: 'A-', percent: '82' },
    'L1_PPA121': { grade: 'B+', percent: '78' },
    'LIBERAL': { grade: 'A-', percent: '82' },
  };
  const saved = localStorage.getItem('courseGrades');
  const existing = saved ? JSON.parse(saved) : {};
  const merged = { ...existing, ...completedGrades };
  delete merged['L1_PPA122'];
  localStorage.setItem('courseGrades', JSON.stringify(merged));
  return merged;
});

const [openElectives, setOpenElectives] = useState<Record<string, string>>(() => {
  const electiveDefaults: Record<string, string> = {
    'LIBERAL': 'CGCM 738 Photoshopped! The Art of Image Retouching',
    'OPEN1': 'CASL 101 Introductory American Sign Language I',
    'OPEN2': 'CFNF 400 The Social Content of Human Sexuality',
    'L2_LIBERAL': 'CPHL 110 Philosophy of Religion',
    'L2_OPEN1': 'CSOC 103 How Society Works',
  };
  const saved = localStorage.getItem('openElectives');
  const existing = saved ? JSON.parse(saved) : {};
  const merged = { ...electiveDefaults, ...existing };
  localStorage.setItem('openElectives', JSON.stringify(merged));
  return merged;
});

const [currentPagLevel, setCurrentPagLevel] = useState(1);
const [dashboardCommentTarget, setDashboardCommentTarget] = useState<{ type: string; id: string; label: string } | ...
const [dashboardCommentText, setDashboardCommentText] = useState('');
const [dashboardCommentLoading, setDashboardCommentLoading] = useState(false);
const [dashboardCommentSaving, setDashboardCommentSaving] = useState(false);
useEffect(() => {
  if (!dashboardCommentTarget) return;
  setDashboardCommentLoading(true);
  fetch(`/api/entity-comments/${dashboardCommentTarget.type}/${dashboardCommentTarget.id}`)
    .then(r => r.json())
    .then(data => { setDashboardCommentText(Array.isArray(data) && data.length > 0 ? data[0].content : ''); })
    .catch(() => setDashboardCommentText(''))
    .finally(() => setDashboardCommentLoading(false));
}, [dashboardCommentTarget?.type, dashboardCommentTarget?.id]);
const saveDashboardComment = useCallback(() => {
  if (!dashboardCommentTarget) return;
  setDashboardCommentSaving(true);
  fetch(`/api/entity-comments/${dashboardCommentTarget.type}/${dashboardCommentTarget.id}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ content: dashboardCommentText }),
  }).finally(() => setDashboardCommentSaving(false));
}, [dashboardCommentTarget, dashboardCommentText]);
const [selectedCertCourse, setSelectedCertCourse] = useState<{
  courseCode: string;
  courseName: string;
  certKey: string;
  openInEdit?: boolean;
} | null>(null);
const [draggedTask, setDraggedTask] = useState<Task | null>(null);
const [todoItems, setTodoItems] = useState<string[]>([]);
const [newReminderTitle, setNewReminderTitle] = useState('');
const [newReminderDueDate, setNewReminderDueDate] = useState('');
const [dragOverSlot, setDragOverSlot] = useState<{ day: Date; hour: number } | null>(null);
const [selectedTaskId, setSelectedTaskId] = useState<number | null>(null);
const hoveredCountdownTaskIdRef = useRef<number | null>(null);
const [hoveredCountdownTaskId, setHoveredCountdownTaskId] = useState<number | null>(null);
const setHoveredCountdownTaskIdDebounce = useCallback((id: number | null) => {
  if (hoveredCountdownTaskIdRef.current === id) return;
  hoveredCountdownTaskIdRef.current = id;
  setHoveredCountdownTaskId(id);
}, []);
const [hoveredCalTaskId, setHoveredCalTaskId] = useState<number | null>(null);

// Pomodoro Timer State
const [isFullscreen, setIsFullscreen] = useState(false);

useEffect(() => {
  const onChange = () => setIsFullscreen(!document.fullscreenElement);
  document.addEventListener('fullscreenchange', onChange);
  document.addEventListener('webkitfullscreenchange', onChange);
  return () => {
    document.removeEventListener('fullscreenchange', onChange);
    document.removeEventListener('webkitfullscreenchange', onChange);
  };
}, []);

const toggleFullscreen = () => {
  if (!document.fullscreenElement) {
    const el = document.documentElement as any;
    (el.requestFullscreen || el.webkitRequestFullscreen || el.mozRequestFullscreen || el.msRequestFullscreen)?.call...
  } else {
    const doc = document as any;
    (doc.exitFullscreen || doc.webkitExitFullscreen || doc.mozCancelFullscreen || doc.msExitFullscreen)?.call(doc);
  }
};

useEffect(() => {
  const isFireDevice = /\bSilk\b/i.test(navigator.userAgent) || /\bKF[A-Z]{2,4}\b/.test(navigator.userAgent);
  if (!isFireDevice) return;
  const enterFullscreen = () => {
    if (!document.fullscreenElement) {

```

```

    const el = document.documentElement as any;
    const req = el.requestFullscreen || el.webkitRequestFullscreen || el.mozRequestFullScreen || el.msRequestFull...
    if (req) {
      req.call(el).catch(() => {});
    }
  }
};
enterFullscreen();
const onInteraction = () => {
  enterFullscreen();
  document.removeEventListener('touchstart', onInteraction);
  document.removeEventListener('click', onInteraction);
};
document.addEventListener('touchstart', onInteraction, { once: true });
document.addEventListener('click', onInteraction, { once: true });
const recheckInterval = setInterval(() => {
  if (!document.fullscreenElement) {
    enterFullscreen();
  }
}, 5000);
return () => {
  clearInterval(recheckInterval);
  document.removeEventListener('touchstart', onInteraction);
  document.removeEventListener('click', onInteraction);
};
}, []);

const [pomodoroTime, setPomodoroTime] = useState(25 * 60); // 25 minutes in seconds
const [pomodoroRunning, setPomodoroRunning] = useState(false);
const [pomodoroStarted, setPomodoroStarted] = useState(false);
const [pomodoroMode, setPomodoroMode] = useState<"work" | "shortBreak" | "longBreak">("work");
const [pomodoroCount, setPomodoroCount] = useState(0);
const pomodoroIntervalRef = useRef<ReturnType<typeof setInterval> | null>(null);

// Keyboard delete handler
useEffect(() => {
  const handleKeyDown = (e: KeyboardEvent) => {
    if ((e.key === 'Delete' || e.key === 'Backspace') && selectedTaskId !== null) {
      // Don't delete if user is typing in an input
      if (document.activeElement?.tagName === 'INPUT' || document.activeElement?.tagName === 'TEXTAREA') {
        return;
      }
      e.preventDefault();
      deleteTaskWithUndo(selectedTaskId);
      setSelectedTaskId(null);
    }
    // Escape to deselect
    if (e.key === 'Escape') {
      setSelectedTaskId(null);
    }
  };
  window.addEventListener('keydown', handleKeyDown);
  return () => window.removeEventListener('keydown', handleKeyDown);
}, [selectedTaskId]);

// Restore pomodoro state from server on mount
useEffect(() => {
  fetch("/api/pomodoro/status").then(r => r.json()).then((data: { mode: string; running: boolean; remaining: number...
    setPomodoroMode(data.mode as "work" | "shortBreak" | "longBreak");
    setPomodoroTime(data.remaining);
    setPomodoroCount(data.count);
    if (data.running) {
      setPomodoroRunning(true);
      setPomodoroStarted(true);
    }
  }).catch(() => {});
}, []);

// Pomodoro Timer Effect - local countdown display + server sync
useEffect(() => {
  if (pomodoroRunning && pomodoroTime > 0) {
    pomodoroIntervalRef.current = setInterval(() => {
      setPomodoroTime(prev => prev - 1);
    }, 1000);
  } else if (pomodoroTime === 0 && pomodoroRunning) {
    setPomodoroRunning(false);
    // Play notification sound locally
    const audio = new Audio("data:audio/wav;base64,UklGRnoGAABXQVZFZm10IBAAAAAABAAEQB8AAEAfAAABAAgAZGF0eQoGAACBhYQF...
    audio.play().catch(() => {});
    // Fetch latest state from server (server handles announcement + mode transition)
    fetch("/api/pomodoro/status").then(r => r.json()).then((data: { mode: string; running: boolean; remaining: numb...
      setPomodoroMode(data.mode as "work" | "shortBreak" | "longBreak");
      setPomodoroTime(data.remaining);
      setPomodoroCount(data.count);
      setPomodoroRunning(data.running);
      if (data.mode === "work") {
        toast({ title: "Break Over!", description: "Time to focus!" });
      } else {
        toast({ title: "Pomodoro Complete!", description: data.mode === "longBreak" ? "Time for a long break!" : "T...
      }
    }).catch(() => {});
  }
  return () => {
    if (pomodoroIntervalRef.current) clearInterval(pomodoroIntervalRef.current);
  };
}, [pomodoroRunning, pomodoroTime, toast]);

// Periodically sync with server to recover from app sleep/background
useEffect(() => {
  if (!pomodoroRunning) return;
  const syncInterval = setInterval(() => {
    fetch("/api/pomodoro/status").then(r => r.json()).then((data: { mode: string; running: boolean; remaining: numb...
      if (!data.running && pomodoroRunning) {
        // Timer finished while app was backgrounded
        setPomodoroRunning(false);
        setPomodoroMode(data.mode as "work" | "shortBreak" | "longBreak");
        setPomodoroTime(data.remaining);
      }
    });
  }, 60000);
}, [pomodoroRunning]);

```

```

        setPomodoroCount(data.count);
        const audio = new Audio('data:audio/wav;base64,UklGRnoGAABXQVZFZm10IBAAAAABAAEAB8AAEAfAAABAAgAZGF0eQoGAACB...');
        audio.play().catch(() => {});
        if (data.mode === "work") {
            toast({ title: "Break Over!", description: "Time to focus!" });
        } else {
            toast({ title: "Pomodoro Complete!", description: data.mode === "longBreak" ? "Time for a long break!" : ...
        }
        } else if (data.running) {
            setPomodoroTime(data.remaining);
        }
    }).catch(() => {});
}, 5000);
return () => clearInterval(syncInterval);
}, [pomodoroRunning, toast]);

const formatPomodoroTime = (seconds: number) => {
    const mins = Math.floor(seconds / 60);
    const secs = seconds % 60;
    return `${mins.toString().padStart(2, '0')}:${secs.toString().padStart(2, '0')}`;
};

const togglePomodoro = () => {
    if (!pomodoroRunning) {
        setPomodoroStarted(true);
        setPomodoroRunning(true);
        fetch("/api/pomodoro/start", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ mode: pomodoroMode, duration: pomodoroTime, count: pomodoroCount })
        }).catch(() => {});
    } else {
        setPomodoroRunning(false);
        fetch("/api/pomodoro/pause", { method: "POST" }).catch(() => {});
    }
};

const toggleFileComplete = (fileKey: string) => {
    setCompletedFiles(prev => {
        const newSet = new Set(prev);
        if (newSet.has(fileKey)) {
            newSet.delete(fileKey);
        } else {
            newSet.add(fileKey);
        }
        localStorage.setItem('completedFiles', JSON.stringify(Array.from(newSet)));
        return newSet;
    });
};

const resetPomodoro = () => {
    setPomodoroStarted(false);
    setPomodoroRunning(false);
    if (pomodoroMode === "work") setPomodoroTime(25 * 60);
    else if (pomodoroMode === "shortBreak") setPomodoroTime(5 * 60);
    else setPomodoroTime(15 * 60);
    fetch("/api/pomodoro/reset", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ mode: pomodoroMode })
    }).catch(() => {});
};

const skipPomodoro = () => {
    setPomodoroRunning(false);
    fetch("/api/pomodoro/skip", { method: "POST" }).then(r => r.json()).then((data: { mode: string; remaining: number...
        setPomodoroMode(data.mode as "work" | "shortBreak" | "longBreak");
        setPomodoroTime(data.remaining);
        setPomodoroCount(data.count);
    }).catch(() => {});
    if (pomodoroMode === "work") {
        const newCount = pomodoroCount + 1;
        setPomodoroCount(newCount);
        if (newCount % 4 === 0) {
            setPomodoroMode("longBreak");
            setPomodoroTime(15 * 60);
        } else {
            setPomodoroMode("shortBreak");
            setPomodoroTime(5 * 60);
        }
    } else {
        setPomodoroMode("work");
        setPomodoroTime(25 * 60);
    }
};

const updateOpenElective = (id: string, value: string) => {
    setOpenElectives(prev => {
        const updated = { ...prev, [id]: value };
        localStorage.setItem('openElectives', JSON.stringify(updated));
        saveDegreeToServer('openElectives', updated);
        if (!value.trim() && checkedCourses[id]) {
            setCheckedCourses(prevChecked => {
                const updatedChecked = { ...prevChecked, [id]: false };
                localStorage.setItem('checkedCourses', JSON.stringify(updatedChecked));
                saveDegreeToServer('checkedCourses', updatedChecked);
                return updatedChecked;
            });
        }
    });
    if (value.trim() && !inProgressCourses[id] && !checkedCourses[id]) {
        setInProgressCourses(prevIP => {
            const updatedIP = { ...prevIP, [id]: true };
            localStorage.setItem('inProgressCourses', JSON.stringify(updatedIP));
            saveDegreeToServer('inProgressCourses', updatedIP);
            return updatedIP;
        });
    }
    if (!value.trim() && inProgressCourses[id]) {

```

```

    setInProgressCourses(prevIP => {
      const updatedIP = { ...prevIP, [id]: false };
      localStorage.setItem('inProgressCourses', JSON.stringify(updatedIP));
      saveDegreeToServer('inProgressCourses', updatedIP);
      return updatedIP;
    });
  }
  return updated;
});
});

const saveDegreeToServer = useCallback((key: string, value: any) => {
  fetch('/api/degree-tracking', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ key, value }),
  }).catch(() => {});
}, []);

useEffect(() => {
  fetch('/api/degree-tracking')
    .then(r => r.json())
    .then((data: Record<string, any>) => {
      if (data.checkedCourses) {
        setCheckedCourses(prev => {
          const merged = { ...prev, ...data.checkedCourses };
          localStorage.setItem('checkedCourses', JSON.stringify(merged));
          return merged;
        });
      }
      if (data.inProgressCourses) {
        setInProgressCourses(prev => {
          const merged = { ...prev, ...data.inProgressCourses };
          localStorage.setItem('inProgressCourses', JSON.stringify(merged));
          return merged;
        });
      }
      if (data.courseGrades) {
        setCourseGrades(prev => {
          const merged = { ...prev, ...data.courseGrades };
          localStorage.setItem('courseGrades', JSON.stringify(merged));
          return merged;
        });
      }
      if (data.openElectives) {
        setOpenElectives(prev => {
          const merged = { ...prev, ...data.openElectives };
          localStorage.setItem('openElectives', JSON.stringify(merged));
          return merged;
        });
      }
      if (data.coursesData) {
        setCoursesData(prev => {
          const merged = { ...prev, courses: data.coursesData.courses || prev.courses };
          localStorage.setItem('coursesData', JSON.stringify(merged));
          return merged;
        });
      }
      if (data.schoolData) {
        setSchoolData(prev => {
          const merged = { ...prev, ...data.schoolData };
          localStorage.setItem('schoolData', JSON.stringify(merged));
          return merged;
        });
      }
      if (data.semesterCourseAssignments) {
        setSemesterCourseAssignments(prev => {
          const merged = { ...prev, ...data.semesterCourseAssignments };
          localStorage.setItem('semesterCourseAssignments', JSON.stringify(merged));
          return merged;
        });
      }
      if (data.otherRowColors) {
        setOtherRowColors(prev => {
          const merged = { ...prev, ...data.otherRowColors };
          localStorage.setItem('otherRowColors', JSON.stringify(merged));
          return merged;
        });
      }
      if (data.colorSettings) {
        setColorSettings(prev => {
          const merged = { ...prev, ...data.colorSettings };
          if ([ '#160502', '#1a3a5c', '#0f2540', '#164a72', '#0e3356', '#0a2744', '#071e38' ].includes(merged.headerBar)) m...
          if ([ '#160502', '#1a3a5c', '#0f2540', '#164a72', '#0e3356', '#0a2744', '#071e38' ].includes(merged.todayCurrentH...
          localStorage.setItem('colorSettings', JSON.stringify(merged));
          return merged;
        });
      }
      if (data.blinkSettings) {
        setBlinkSettings(prev => {
          const merged = { ...prev, ...data.blinkSettings };
          localStorage.setItem('blinkSettings', JSON.stringify(merged));
          return merged;
        });
      }
      if (data.certCourseData) {
        localStorage.setItem('certCourseData', JSON.stringify(data.certCourseData));
      }
      if (data.pastCourseInfo) {
        setPastCourseInfo(prev => {
          const merged = { ...prev, ...data.pastCourseInfo };
          localStorage.setItem('pastCourseInfo', JSON.stringify(merged));
          return merged;
        });
      }
      if (data.coursePlayPriority) {
        setCoursePlayPriority(data.coursePlayPriority);
        localStorage.setItem('coursePlayPriority', JSON.stringify(data.coursePlayPriority));
      }
    });

```



```

    }
    if (data.profileData) {
      setProfileData(prev => {
        const merged = { ...prev, ...data.profileData };
        localStorage.setItem('profileData', JSON.stringify(merged));
        return merged;
      });
    }
    if (data.courseDisplayNames) {
      localStorage.setItem('courseDisplayNames', JSON.stringify(data.courseDisplayNames));
    }
    if (data.gridSizes && !localStorage.getItem('gridSizes')) {
      localStorage.setItem('gridSizes', JSON.stringify(data.gridSizes));
      setGridSizes(prev => ({ ...prev, ...data.gridSizes }));
    }
    if (data.calendarHeight && !localStorage.getItem('calendarHeight')) {
      const h = typeof data.calendarHeight === 'number' ? data.calendarHeight : parseInt(data.calendarHeight);
      if (!isNaN(h)) {
        localStorage.setItem('calendarHeight', String(h));
        setCalendarHeight(h);
      }
    }
    if (data.calendarReduction && !localStorage.getItem('calendarReduction')) {
      const r = typeof data.calendarReduction === 'number' ? data.calendarReduction : parseInt(data.calendarReduction);
      if (!isNaN(r)) {
        localStorage.setItem('calendarReduction', String(r));
        setCalendarReduction(r);
      }
    }
    if (data.showAllDayRow !== undefined && !localStorage.getItem('showAllDayRow')) {
      const val = typeof data.showAllDayRow === 'boolean' ? data.showAllDayRow : data.showAllDayRow === 'true';
      localStorage.setItem('showAllDayRow', JSON.stringify(val));
      setShowAllDayRow(val);
    }
  })
  .then(() => {
    const syncKeys = [
      'coursesData', 'schoolData', 'colorSettings', 'blinkSettings',
      'semesterCourseAssignments', 'pastCourseInfo', 'certCourseData',
      'coursePlayPriority', 'courseDisplayNames', 'profileData',
      'checkedCourses', 'inProgressCourses', 'courseGrades', 'openElectives',
      'gridSizes', 'calendarHeight', 'calendarReduction', 'showAllDayRow',
      'otherRowColors',
    ];
    const payload: Record<string, any> = {};
    for (const key of syncKeys) {
      const val = localStorage.getItem(key);
      if (val) {
        try { payload[key] = JSON.parse(val); } catch { payload[key] = val; }
      }
    }
    if (Object.keys(payload).length > 0) {
      fetch('/api/degree-tracking/bulk', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload),
      }).then(r => r.json()).then(d => {
        if (d.ok) console.log(`[Sync] Pushed ${d.saved} localStorage keys to server`);
      }).catch(() => {});
    }
  })
  .catch(() => {});
}, []);

const getSelectedCodes = (siblingIds: string[], currentId: string): Set<string> => {
  const codes = new Set<string>();
  for (const sid of siblingIds) {
    if (sid === currentId) continue;
    const val = openElectives[sid];
    if (val?.trim()) {
      const code = val.split(' ').slice(0, 2).join(' ');
      codes.add(code);
    }
  }
  return codes;
};

const buildElectiveOptions = (courseList: ElectiveCourse[], slotId: string, siblingIds?: string[]) => {
  const sorted = [...courseList].sort((a, b) => {
    const numA = parseInt(a.code.replace(/^[^0-9]/g, '')) || 0;
    const numB = parseInt(b.code.replace(/^[^0-9]/g, '')) || 0;
    return numA - numB || a.code.localeCompare(b.code);
  });
  const used = siblingIds ? getSelectedCodes(siblingIds, slotId) : new Set<string>();
  const filtered = sorted.filter(c => !used.has(c.code));
  const currentVal = openElectives[slotId];
  const options = filtered.map(c => ({ key: c.code, value: `${c.code} ${c.name}`, label: `${c.code} - ${c.name}` }));
  if (currentVal?.trim()) {
    const alreadyInList = options.some(o => o.value === currentVal);
    if (!alreadyInList) {
      const parts = currentVal.split(' ');
      const code = parts.slice(0, 2).join(' ');
      const name = parts.slice(2).join(' ');
      options.unshift({ key: `saved-${code}`, value: currentVal, label: `${code} - ${name}` });
    }
  }
  return options;
};

const noGradeIds = new Set(['L1_PPA122']);

const updateGrade = (courseId: string, grade: string) => {
  if (noGradeIds.has(courseId)) return;
  setCourseGrades(prev => {
    const updated = { ...prev, [courseId]: { ...prev[courseId], grade } };
    localStorage.setItem('courseGrades', JSON.stringify(updated));
    saveDegreeToServer('courseGrades', updated);
    return updated;
  });
};

```

```

    });
  });
};

const updatePercent = (courseId: string, percent: string) => {
  if (noGradeIds.has(courseId)) return;
  setCourseGrades(prev => {
    const updated = { ...prev, [courseId]: { ...prev[courseId], percent } };
    localStorage.setItem('courseGrades', JSON.stringify(updated));
    saveDegreeToServer('courseGrades', updated);
    return updated;
  });
};

const gradeOptions = ['', 'A+', 'A', 'A-', 'B+', 'B', 'B-', 'C+', 'C', 'C-', 'D', 'F'];

const percentToGrade = (percent: string): string => {
  const p = parseFloat(percent);
  if (isNaN(p) || percent.trim() === '') return '';
  if (p >= 90) return 'A+';
  if (p >= 85) return 'A';
  if (p >= 80) return 'A-';
  if (p >= 77) return 'B+';
  if (p >= 73) return 'B';
  if (p >= 70) return 'B-';
  if (p >= 67) return 'C+';
  if (p >= 63) return 'C';
  if (p >= 60) return 'C-';
  if (p >= 50) return 'D';
  return 'F';
};

const allCoursesChecked = ['PPA101', 'PPA102', 'PPA125', 'L1_PPA120', 'L1_PPA121', 'LIBERAL', 'OPEN1', 'OPEN2']
  .every(id => checkedCourses[id]);

const certSections = useMemo(() => ({
  L1: [
    { required: 3, members: ['PPA101', 'PPA102', 'PPA125'] },
    { required: 2, members: ['L1_PPA120', 'L1_PPA121', 'L1_PPA122', 'L1_PPA124'] },
    { required: 1, members: ['LIBERAL'] },
    { required: 2, members: ['OPEN1', 'OPEN2'] },
  ],
  L2: [
    { required: 1, members: ['L2_PPA211'] },
    { required: 3, members: ['L2_PPA120', 'L2_PPA121', 'L2_PPA122', 'L2_PPA124', 'L2_PPA235', 'L2_PPA303', 'L2_PPA3...'],
    { required: 1, members: ['L2_LIBERAL'] },
    { required: 1, members: ['L2_ECN1', 'L2_ECN2', 'L2_ECN3', 'L2_ECN4', 'L2_ECN5', 'L2_ECN6', 'L2_ECN7', 'L2_ECN8']... },
    { required: 2, members: ['L2_OPEN1', 'L2_OPEN2'] },
  ],
  L3: [
    { required: 2, members: ['L3_PPA333', 'L3_PRACTICUM1'] },
    { required: 8, members: ['L3_PPA235', 'L3_PPA301', 'L3_PPA303', 'L3_PPA319', 'L3_PPA335', 'L3_PPA401', 'L3_PPA4...'],
    { required: 2, members: ['L3_P061', 'L3_P062'] },
    { required: 4, members: ['L3_LIBERAL1', 'L3_LIBERAL2', 'L3_LIBERAL3', 'L3_LIBERAL4'] },
    { required: 7, members: ['L3_OPEN1', 'L3_OPEN2', 'L3_OPEN3', 'L3_OPEN4', 'L3_OPEN5', 'L3_OPEN6', 'L3_OPEN7'] },
  ],
}), []);

const sectionCheckedCount = (members: string[]) => members.filter(m => checkedCourses[m] || inProgressCourses[m]).length;
const sectionCompletedCount = (members: string[]) => members.filter(m => checkedCourses[m]).length;
const sectionRemaining = (sectionIndex: number, level: 'L1' | 'L2' | 'L3') => {
  const section = certSections[level][sectionIndex];
  if (!section) return 0;
  const active = section.members.filter(m => !isActiveInOtherLevel(m) && (checkedCourses[m] || inProgressCourses[m]...)).length;
  return Math.max(0, section.required - active);
};

const isSectionAllGreen = (sectionIndex: number, level: 'L1' | 'L2' | 'L3') => {
  const section = certSections[level][sectionIndex];
  if (!section) return false;
  if (sectionRemaining(sectionIndex, level) !== 0) return false;
  const checkedCount = section.members.filter(m => checkedCourses[m]).length;
  if (checkedCount < section.required) return false;
  const allSections = certSections[level];
  const allComplete = allSections.every((_, i) => {
    const s = allSections[i];
    if (!s) return false;
    if (sectionRemaining(i, level) !== 0) return false;
    return s.members.filter(m => checkedCourses[m]).length >= s.required;
  });
  if (allComplete) return false;
  return true;
};

const getSectionForCourse = (courseId: string) => {
  for (const sections of Object.values(certSections)) {
    for (const section of sections) {
      if (section.members.includes(courseId)) return section;
    }
  }
  return null;
};

const l1ToL2Map: Record<string, string> = {
  'L1_PPA120': 'L2_PPA120', 'L1_PPA121': 'L2_PPA121',
  'L1_PPA122': 'L2_PPA122', 'L1_PPA124': 'L2_PPA124',
};

const previousLevelMap: Record<string, string[]> = {
  'L2_PPA120': ['L1_PPA120'], 'L2_PPA121': ['L1_PPA121'],
  'L2_PPA122': ['L1_PPA122'], 'L2_PPA124': ['L1_PPA124'],
  'L3_PPA235': ['L2_PPA235'], 'L3_PPA303': ['L2_PPA303'], 'L3_PPA319': ['L2_PPA319'],
};

const laterLevelMap: Record<string, string[]> = {};
Object.entries(previousLevelMap).forEach(([later, earlierIds]) => {
  earlierIds.forEach(eid => {
    if (!laterLevelMap[eid]) laterLevelMap[eid] = [];
    laterLevelMap[eid].push(later);
  });
});

```

```

});

const isActiveInLaterLevel = (courseId: string): boolean => {
  const laterIds = laterLevelMap[courseId];
  if (!laterIds) return false;
  return laterIds.some(lid => inProgressCourses[lid] || checkedCourses[lid] || isL2InProgressFromL1(lid));
};

const isActiveInEarlierLevel = (courseId: string): boolean => {
  const prevIds = previousLevelMap[courseId];
  if (!prevIds) return false;
  return prevIds.some(pid => {
    if (checkedCourses[pid]) return true;
    if (inProgressCourses[pid] && isSectionFulfilledForCourse(pid)) return false;
    if (inProgressCourses[pid]) return true;
    if (courseGrades[pid]?.percent && courseGrades[pid]?.percent?.trim() !== '') return true;
    return false;
  });
};

const isActiveInOtherLevel = (courseId: string): boolean => {
  return isActiveInLaterLevel(courseId) || isActiveInEarlierLevel(courseId);
};

const isPreviouslyCompleted = (courseId: string): boolean => {
  const prevIds = previousLevelMap[courseId];
  if (!prevIds) return false;
  return prevIds.some(pid => {
    if (checkedCourses[pid]) {
      if (isActiveInLaterLevel(pid)) return false;
      return true;
    }
    if (inProgressCourses[pid] && isSectionFulfilledForCourse(pid)) return false;
    const pidCompleted = inProgressCourses[pid] || !(courseGrades[pid]?.percent && courseGrades[pid]?.percent?.tri...
    if (!pidCompleted) return false;
    if (isActiveInLaterLevel(pid)) return false;
    return true;
  });
};

const isSectionFulfilledByCompleted = (courseId: string): boolean => {
  const section = getSectionForCourse(courseId);
  if (!section) return false;
  return sectionCompletedCount(section.members) >= section.required;
};

const isCourseGreyedOut = (courseId: string): boolean => {
  if (checkedCourses[courseId]) return false;
  return isSectionFulfilledByCompleted(courseId);
};

const isL2InProgressFromL1 = (l2Id: string): boolean => {
  const l1Entry = Object.entries(l1ToL2Map).find(([, v]) => v === l2Id);
  if (!l1Entry) return false;
  const l1Id = l1Entry[0];
  return !(inProgressCourses[l1Id] && isSectionFulfilledByCompleted(l1Id));
};

const isSectionFulfilledForCourse = (id: string): boolean => {
  const level: 'L1' | 'L2' | 'L3' = id.startsWith('L3_') ? 'L3' : id.startsWith('L2_') ? 'L2' : 'L1';
  const sections = certSections[level];
  for (let i = 0; i < sections.length; i++) {
    if (sections[i].members.includes(id)) {
      const otherActive = sections[i].members.filter(m => m !== id && !isActiveInOtherLevel(m) && (checkedCourses[m...
      return otherActive >= sections[i].required;
    }
  }
  return false;
};

const courseRowBg = (id: string): string | undefined => {
  if (isSectionFulfilledForCourse(id) || isActiveInOtherLevel(id) || isCourseGreyedOut(id)) return `rgba(0,0,0,0.04)`;
  const codeNorm = id.replace(/^\d_/, '');
  const gradient = getCourseGradientColors(codeNorm);
  const lightHex = gradient.end.startsWith('#') ? gradient.end : '#9ca3af';
  const rgb = hexToRgb(lightHex);
  const darken = 0.80;
  const r = Math.round(rgb.r * darken);
  const g = Math.round(rgb.g * darken);
  const b = Math.round(rgb.b * darken);
  if (checkedCourses[id]) return `rgba(${r},${g},${b},0.18)`;
  if (inProgressCourses[id] || isL2InProgressFromL1(id)) return `rgba(${r},${g},${b},0.15)`;
  return undefined;
};

const courseRowClass = (id: string) => {
  if (checkedCourses[id]) return 'text-emerald-700';
  if (isSectionFulfilledForCourse(id)) return 'text-gray-400';
  if (isActiveInOtherLevel(id)) return 'text-gray-400';
  if (inProgressCourses[id] || isL2InProgressFromL1(id)) return 'text-amber-800';
  if (isCourseGreyedOut(id)) return 'text-gray-400';
  return '';
};

const shouldStrikethrough = (id: string) => {
  if (checkedCourses[id]) return false;
  if (isSectionFulfilledForCourse(id)) return true;
  if (isActiveInOtherLevel(id)) return true;
  if (inProgressCourses[id] || isL2InProgressFromL1(id)) return false;
  if (isCourseGreyedOut(id)) return true;
  return false;
};

const isCheckDisabled = (id: string) => isCourseGreyedOut(id) || isActiveInOtherLevel(id);
const getElectiveCode = (val: string) => {
  if (!val?.trim()) return '';
  const parts = val.split(' ');
  const prefix = parts[0] || '';
  const num = parts[1] || '';
  if (/^\d+$/.test(num)) return `${prefix} ${num}`;
  return prefix.replace(/([A-Z]+)(\d+)/, '$1 $2');
};

```

```

});

const isDropDownRow = (id: string) =>
  /^(LIBERAL|OPEN\d|L2_LIBERAL|L2_OPEN\d|L3_POG\d|L3_LIBERAL\d|L3_OPEN\d)$/.test(id);

const StrikethroughLabel = ({ id }: { id: string }) => {
  if (isDropDownRow(id)) return null;
  if (checkedCourses[id]) return null;
  if (isSectionFulfilledForCourse(id) || isCourseGreyedOut(id)) return <span className="text-[9px]" style={{ textDe...
  if (isActiveInOtherLevel(id)) return <span className="text-[9px]" style={{ textDecoration: 'none', color: '#000000...
  if (inProgressCourses[id] || isL2InProgressFromL1(id)) return null;
  return null;
};

const CourseName = ({ id, children }: { id: string; children: React.ReactNode }) => {
  const strike = shouldStrikethrough(id);
  return strike ? <span style={{ textDecoration: 'line-through' }}>{children}</span> : <>{children}</>;
};

const renderGradeInput = (id: string) => {
  const struck = shouldStrikethrough(id);
  return (
    <div className="flex items-center justify-center gap-1" style={{ width: '100%' }}>
      <input type="text" className="text-[9px] px-0 border border-gray-400 rounded-sm text-center" style={{ backgro...
      <span className="text-[9px] text-left leading-none" style={{ color: struck ? '#999' : '#333', width: '16px', ...
    </div>
  );
};

const renderTriStateToggle = (id: string) => {
  const activeInOther = isActiveInOtherLevel(id);
  const sectionDone = isSectionFulfilledForCourse(id);
  const greyed = isCourseGreyedOut(id);
  const hasPercent = !!(courseGrades[id]??.percent && courseGrades[id]??.percent?.trim() !== '');
  const isCompleted = checkedCourses[id];
  const isInProgress = inProgressCourses[id] || isL2InProgressFromL1(id);
  const isDisabled = activeInOther || sectionDone || (!isCompleted && !isInProgress && greyed);
  const state: 'red' | 'yellow' | 'green' = isCompleted ? 'green' : isDisabled ? 'red' : isInProgress ? 'yellow' : ...
  const cycle = () => {
    if (isDisabled) return;
    if (state === 'red') {
      if (!inProgressCourses[id]) toggleInProgress(id);
      const info = certCourseMap[id];
      if (info && /^[A-Z]{2,}\d{3}/.test(info.code.replace(/\s/g, '')) && !isDropDownRow(id)) {
        const rawCode = info.code.replace(/\s/g, '');
        const semCode = /^[A-Z]{3,}\d/.test(rawCode) && !rawCode.startsWith('C') ? 'C' + rawCode : rawCode;
        const displayCode = semCode.replace(/([A-Z]+)(\d+)/, '$1 $2');
        addCourseToNextAvailableSemester(semCode, displayCode, info.name);
      }
    } else if (state === 'yellow') {
      if (inProgressCourses[id]) toggleInProgress(id);
      if (!checkedCourses[id]) toggleCourse(id);
    } else {
      if (checkedCourses[id]) toggleCourse(id);
    }
  };
  const bg = state === 'green' ? '#8cb44c' : state === 'yellow' ? '#f0b429' : '#e8526e';
  const knobLeft = state === 'red' ? '1px' : state === 'yellow' ? '50%' : 'calc(100% - 13px)';
  const knobTransform = state === 'yellow' ? 'translateX(-50%)' : 'none';
  return (
    <div
      className={`flex items-center justify-center ${isDisabled ? 'cursor-not-allowed opacity-50' : 'cursor-pointer...
      style={{ fontSize: '7px', lineHeight: 1 }}
      onClick={(e) => { e.stopPropagation(); cycle(); }}
      title={state === 'red' ? 'Not started - click for In Progress' : state === 'yellow' ? 'In Progress - click fo...
      data-testid={`toggle-tri-${id}`}
    >
      <div style={{ width: '32px', height: '14px', borderRadius: '3px', background: bg, position: 'relative', trans...
        <div style={{
          width: '12px', height: '12px', borderRadius: '2px', background: 'eeee',
          position: 'absolute', top: '1px', left: knobLeft, transform: knobTransform,
          transition: 'left 0.2s, transform 0.2s',
          boxShadow: '0 0.5px 1px rgba(0,0,0,0.2)',
          display: 'flex', alignItems: 'center', justifyContent: 'center'
        }}>
          {state === 'red' && <span style={{ fontSize: '7px', fontWeight: 'bold', color: '#e8526e', lineHeight: 1 }}...
          {state === 'yellow' && <span style={{ fontSize: '8px', fontWeight: 'bold', color: '#f0b429', lineHeight: 1...
          {state === 'green' && <span style={{ fontSize: '7px', fontWeight: 'bold', color: '#8cb44c', lineHeight: 1...
        </div>
      </div>
    </div>
  );
};

const cycleTriState = (id: string) => {
  const isCompleted = checkedCourses[id];
  const activeInOther = isActiveInOtherLevel(id);
  const sectionDone = isSectionFulfilledForCourse(id);
  const greyed = isCourseGreyedOut(id);
  const isInProgress = inProgressCourses[id] || isL2InProgressFromL1(id);
  const isDisabled = activeInOther || sectionDone || (!isCompleted && !isInProgress && greyed);
  if (isDisabled) return;
  const state = isCompleted ? 'green' : isInProgress ? 'yellow' : 'red';
  if (state === 'red') {
    if (!inProgressCourses[id]) toggleInProgress(id);
    const info = certCourseMap[id];
    if (info && /^[A-Z]{2,}\d{3}/.test(info.code.replace(/\s/g, '')) && !isDropDownRow(id)) {
      const rawCode = info.code.replace(/\s/g, '');
      const semCode = /^[A-Z]{3,}\d/.test(rawCode) && !rawCode.startsWith('C') ? 'C' + rawCode : rawCode;
      const displayCode = semCode.replace(/([A-Z]+)(\d+)/, '$1 $2');
      addCourseToNextAvailableSemester(semCode, displayCode, info.name);
    }
  } else if (state === 'yellow') {
    if (inProgressCourses[id]) toggleInProgress(id);
    if (!checkedCourses[id]) toggleCourse(id);
  } else {
    if (checkedCourses[id]) toggleCourse(id);
  }
};

```

```

const InProgressToggle = ({ id, inline }: { id: string; inline?: boolean }) => {
  const activeInLater = isActiveInLaterLevel(id);
  const isAutoFromL1 = isL2InProgressFromL1(id) && !inProgressCourses[id];
  const isActive = activeInLater ? false : (inProgressCourses[id] || isL2InProgressFromL1(id));
  return (
    <div
      className={inline ? "ml-auto shrink-0 cursor-pointer flex items-center" : "absolute top-0 right-0 cursor-pointe...
      style={inline ? { fontSize: '7px', lineHeight: 1 } : { padding: '1px 3px', fontSize: '7px', lineHeight: 1 }}
      onClick={(e) => { e.stopPropagation(); if (!isAutoFromL1) toggleInProgress(id); }}
      title={isAutoFromL1 ? 'In progress (from Level 1)' : isActive ? 'Mark not in progress' : 'Mark in progress'}
      data-testid={ `toggle-in-progress-${id}` }
    >
      <div style={{ width: '20px', height: '11px', borderRadius: '6px', background: isActive ? '#f59e0b' : 'rgba(0,0,...
        <div style={{ width: '9px', height: '9px', borderRadius: '50%', background: '#fff', position: 'absolute', top...
      </div>
    </div>
  );
};

const getLevelProgress = (level: 'L1' | 'L2' | 'L3') => {
  let completed = 0;
  let total = 0;
  for (const section of certSections[level]) {
    total += section.required;
    completed += Math.min(section.checkedCount(section.members), section.required);
  }
  return { completed, total };
};

const l1Progress = getLevelProgress('L1');
const l2Progress = getLevelProgress('L2');
const l3Progress = getLevelProgress('L3');
const overallCertProgress = {
  completed: l1Progress.completed + l2Progress.completed + l3Progress.completed,
  total: l1Progress.total + l2Progress.total + l3Progress.total,
};

const getHeaderColor = (progress: { completed: number; total: number }) => {
  if (progress.completed === 0) return '#ef4444';
  if (progress.completed >= progress.total) return '#22c55e';
  return '#eab398';
};

const certCourseMap: Record<string, { code: string; name: string }> = {
  PPA101: { code: 'PPA101', name: 'Cdn Public Admin: Institutions' },
  PPA102: { code: 'PPA102', name: 'Cdn Public Admin: Processes' },
  PPA125: { code: 'PPA125', name: 'Rights, Equity and the State' },
  L1_PPA120: { code: 'PPA120', name: 'Canadian Politics & Gov\'t' },
  L1_PPA121: { code: 'PPA121', name: 'Ontario Politics & Gov\'t' },
  L1_PPA122: { code: 'PPA122', name: 'Local Politics & Gov\'t' },
  L1_PPA124: { code: 'PPA124', name: 'Indigenous Pol. & Governance' },
  LIBERAL: { code: 'LIBERAL', name: 'Liberal Studies Elective (Lower)' },
  OPEN1: { code: 'OPEN1', name: 'Open Elective 1' },
  OPEN2: { code: 'OPEN2', name: 'Open Elective 2' },
  L2_PPA211: { code: 'PPA211', name: 'Public Policy' },
  L2_PPA120: { code: 'PPA120', name: 'Canadian Politics & Gov\'t' },
  L2_PPA121: { code: 'PPA121', name: 'Ontario Politics & Gov\'t' },
  L2_PPA122: { code: 'PPA122', name: 'Local Politics & Gov\'t' },
  L2_PPA124: { code: 'PPA124', name: 'Indigenous Pol. & Governance' },
  L2_PPA235: { code: 'PPA235', name: 'Theories of the State' },
  L2_PPA303: { code: 'PPA303', name: 'Financial Management' },
  L2_PPA319: { code: 'PPA319', name: 'Politics of work and Labour' },
  L2_LIBERAL: { code: 'LIBERAL', name: 'Liberal Studies Elective (Lower)' },
  L2_ECN1: { code: 'ECN101', name: 'Principles of Microeconomics' },
  L2_ECN2: { code: 'ECN104', name: 'Introductory Microeconomics' },
  L2_ECN3: { code: 'ECN110', name: 'The Economy and Society' },
  L2_ECN4: { code: 'ECN201', name: 'Principles of Macroeconomics' },
  L2_ECN5: { code: 'ECN204', name: 'Introductory Macroeconomics' },
  L2_ECN6: { code: 'ECN210', name: 'Understanding Economics' },
  L2_ECN7: { code: 'ECN220', name: 'Evolution of the Global Economy' },
  L2_ECN8: { code: 'ECN320', name: 'Introduction to Financial Economics' },
  L2_OPEN1: { code: 'L2OPEN1', name: 'Open Elective 1' },
  L2_OPEN2: { code: 'L2OPEN2', name: 'Open Elective 2' },
  L3_PPA333: { code: 'PPA333', name: 'Research Methods in Public Admin' },
  L3_PPA235: { code: 'PPA235', name: 'Theories of the State' },
  L3_PPA301: { code: 'PPA301', name: 'Administrative Law' },
  L3_PPA303: { code: 'PPA303', name: 'Financial Management' },
  L3_PPA319: { code: 'PPA319', name: 'Politics of Work and Labour' },
  L3_PPA335: { code: 'PPA335', name: 'Theories of Bureaucracy' },
  L3_PPA401: { code: 'PPA401', name: 'Collaborative Governance' },
  L3_PPA402: { code: 'PPA402', name: 'Program Planning & Evaluation' },
  L3_PPA403: { code: 'PPA403', name: 'e-Government' },
  L3_PPA404: { code: 'PPA404', name: 'Issues in Public Admin' },
  L3_PPA411: { code: 'PPA411', name: 'Advanced Public Policy' },
  L3_PPA414: { code: 'PPA414', name: 'Comparative Public Policy' },
  L3_PPA425: { code: 'PPA425', name: 'Intergovernmental Relations' },
  L3_PPA490: { code: 'PPA490', name: 'Public Administration Themes' },
  L3_PPA501: { code: 'PPA501', name: 'Public Sector Leadership' },
  L3_PRACTICUM1: { code: 'PPA51', name: 'Public Policy Research Paper' },
  L3_POG1: { code: 'POG1', name: 'Required Professional POG 1' },
  L3_POG2: { code: 'POG2', name: 'Required Professional POG 2' },
  L3_LIBERAL1: { code: 'LIB1', name: 'Liberal Studies (Lower) 1' },
  L3_LIBERAL2: { code: 'LIB2', name: 'Liberal Studies (Upper) 2' },
  L3_LIBERAL3: { code: 'LIB3', name: 'Liberal Studies (Upper) 3' },
  L3_LIBERAL4: { code: 'LIB4', name: 'Liberal Studies (Upper) 4' },
  L3_OPEN1: { code: 'L3OPEN1', name: 'PR Elective 1' },
  L3_OPEN2: { code: 'L3OPEN2', name: 'PR Elective 2' },
  L3_OPEN3: { code: 'L3OPEN3', name: 'PR Elective 3' },
  L3_OPEN4: { code: 'L3OPEN4', name: 'PR Elective 4' },
  L3_OPEN5: { code: 'L3OPEN5', name: 'PR Elective 5' },
  L3_OPEN6: { code: 'L3OPEN6', name: 'PR Elective 6' },
  L3_OPEN7: { code: 'L3OPEN7', name: 'PR Elective 7' },
};

const [openedCourseFromDegreeTracking, setOpenedCourseFromDegreeTracking] = useState(false);
const handleCertCourseClick = (certKey: string) => {
  const info = certCourseMap[certKey];

```

```

if (!info) return;
const elective = openElectives[certKey];
let code = info.code;
let name = info.name;
if (elective?.trim()) {
  const parts = elective.trim().split(/\s+/);
  if (parts.length >= 2 && /\^d/.test(parts[1])) {
    code = parts[0] + parts[1];
    name = parts.slice(2).join(' ') || elective;
  } else {
    code = parts[0];
    name = elective;
  }
}
const matchedCourse = coursesData.courses.find(c => {
  const cCode = c.name.split(' - ')[0]?.trim().replace(/\s/g, '');
  const codeNorm = code.replace(/\s/g, '');
  return cCode === codeNorm || cCode === 'C' + codeNorm || cCode.replace(/\^C(?=[A-Z]{2})/, '') === codeNorm;
});
if (matchedCourse) {
  const mCode = matchedCourse.name.split(' - ')[0]?.trim();
  const mName = matchedCourse.name.split(' - ').slice(1).join(' - ').trim();
  code = mCode;
  name = mName;
}
setIsSettingsPanelOpen(false);
setOpenedCourseFromDegreeTracking(true);
startTransition(() => setSelectedCertCourse({ courseCode: code, courseName: name, certKey }));
});

const buildCourseInfoForCert = () => {
  if (!selectedCertCourse) return null;
  const { courseCode, courseName, certKey } = selectedCertCourse;
  const fullName = `${courseCode} - ${courseName}`;
  const codeNorm = courseCode.replace(/\s/g, '');
  const matchedCourse = coursesData.courses.find(c => {
    const cCode = c.name.split(' - ')[0]?.trim().replace(/\s/g, '');
    return cCode === codeNorm || cCode === 'C' + codeNorm || cCode.replace(/\^C(?=[A-Z]{2})/, '') === codeNorm;
  });
  let deliveryMode = '';
  let classDay = '';
  let classDay2 = '';
  let classTime = '';
  let classEndTime = '';
  let classTime2 = '';
  let classEndTime2 = '';
  let startDate = '';
  let endDate = '';
  let zoomLink = '';
  let professor = matchedCourse?.professor || '';
  let professorEmail = matchedCourse?.professorEmail || '';
  let color = matchedCourse?.color || '#6366F1';
  let colorEnd = (matchedCourse as any)?.colorEnd || '';
  let colorStops = (matchedCourse as any)?.colorStops || '';
  let borderColor = (matchedCourse as any)?.borderColor || '';
  let courseRowColor = (matchedCourse as any)?.courseRowColor || '';
  let taskBgColor = (matchedCourse as any)?.taskBgColor || '';
  let courseType = '';
  let semesterTerm = '';
  let year = '';

  const semSearchList = allSemesterSettingsRef.current || (semesterSettings ? [semesterSettings] : []);
  for (const sem of semSearchList) {
    let foundInSem = false;
    for (let i = 1; i <= 3; i++) {
      const prefix = `course${i}` as const;
      const codeField = ((sem as any)[`${prefix}Code`] || '');
      if (codeField.replace(/\s/g, '') === courseCode.replace(/\s/g, '')) {
        deliveryMode = (sem as any)[`${prefix}DeliveryMode`] || '';
        classDay = (sem as any)[`${prefix}ClassDay`] || '';
        classDay2 = (sem as any)[`${prefix}ClassDay2`] || '';
        classTime = (sem as any)[`${prefix}ClassTime`] || '';
        classEndTime = (sem as any)[`${prefix}ClassEndTime`] || '';
        classTime2 = (sem as any)[`${prefix}ClassTime2`] || '';
        classEndTime2 = (sem as any)[`${prefix}ClassEndTime2`] || '';
        zoomLink = (sem as any)[`${prefix}ZoomLink`] || '';
        courseType = (sem as any)[`${prefix}CourseType`] || '';
        if (!professor) professor = (sem as any)[`${prefix}Professor`] || '';
        if (!professorEmail) professorEmail = (sem as any)[`${prefix}ProfessorEmail`] || '';
        const semType = (sem as any).semesterType || '';
        const ssTerm = (sem as any)[`${prefix}SpringSummerTerm`] || '';
        if (semType === 'spring_summer') {
          semesterTerm = ssTerm === 'first_half' ? 'spring_summer_first' : ssTerm === 'second_half' ? 'spring_summe...' : '';
        } else if (semType === 'fall' || semType === 'winter') {
          semesterTerm = semType;
        }
      }
      const courseStartDate = (sem as any)[`${prefix}StartDate`];
      const courseEndDate = (sem as any)[`${prefix}EndDate`];
      const semStart = courseStartDate || (sem as any).semesterStartDate;
      if (courseStartDate) {
        const sdStr = typeof courseStartDate === 'string' && courseStartDate.includes('T') ? courseStartDate.spli...
        if (sdStr) {
          startDate = sdStr;
        } else {
          const sd = new Date(courseStartDate);
          startDate = `${sd.getFullYear()}-${(sd.getMonth()+1).toString().padStart(2,'0')}-${sd.getDate().toStrin...
        }
      }
    }
    if (courseEndDate) {
      const edStr = typeof courseEndDate === 'string' && courseEndDate.includes('T') ? courseEndDate.split('T')...
      if (edStr) {
        endDate = edStr;
      } else {
        const ed = new Date(courseEndDate);
        endDate = `${ed.getFullYear()}-${(ed.getMonth()+1).toString().padStart(2,'0')}-${ed.getDate().toString(...
      }
    }
  }
}

```

```

        if (semStart) {
            year = new Date(semStart).getFullYear().toString();
        }
        foundInSem = true;
        break;
    }
}
if (foundInSem) break;
}

const certData = localStorage.getItem('certCourseData');
const savedData = certData ? JSON.parse(certData) : {};
const saved = savedData[certKey] || {};
if (saved.deliveryMode) deliveryMode = saved.deliveryMode;
if (saved.zoomLink) zoomLink = saved.zoomLink;
if (saved.professor) professor = saved.professor;
if (saved.professorEmail) professorEmail = saved.professorEmail;
if (!matchedCourse && saved.color) color = saved.color;
if (!matchedCourse && saved.colorEnd) colorEnd = saved.colorEnd;

if (!courseType && certKey) {
    const ckUpper = certKey.toUpperCase();
    if (ckUpper.includes('OPEN') || ckUpper.includes('ECN')) {
        courseType = 'open_elective';
    } else if (ckUpper.includes('LIBERAL') || ckUpper.includes('LIB')) {
        courseType = 'liberal_studies';
    } else {
        courseType = 'core';
    }
}
}

return {
    courseCode,
    courseName,
    fullName,
    professor,
    professorEmail,
    color,
    colorEnd,
    colorStops,
    borderColor,
    courseRowColor,
    taskBgColor,
    deliveryMode,
    classDay,
    classDay2,
    classTime,
    classEndTime,
    classTime2,
    classEndTime2,
    startDate,
    endDate,
    zoomLink,
    courseType,
    semesterTerm,
    year,
};
};

// Create jiggle sound using Web Audio API - only if Bluetooth/audio already connected
const playJiggleSound = useCallback(() => {
    try {
        // Don't create new AudioContext - only play if one already exists and is running
        // This prevents initiating Bluetooth connections for today task alarms
        if (!sharedAudioContextRef.current || sharedAudioContextRef.current.state !== 'running') {
            console.log('Audio context not running - skipping alarm sound to avoid Bluetooth initiation');
            return;
        }

        const audioContext = sharedAudioContextRef.current;
        const gainNode = audioContext.createGain();
        gainNode.connect(audioContext.destination);

        const now = audioContext.currentTime;

        // Westminster chime style - two notes
        const playChime = (freq: number, startTime: number, duration: number) => {
            const osc = audioContext.createOscillator();
            const oscGain = audioContext.createGain();
            osc.type = 'sine';
            osc.frequency.setValueAtTime(freq, startTime);
            osc.connect(oscGain);
            oscGain.connect(gainNode);

            // Bell-like decay
            oscGain.gain.setValueAtTime(0.3, startTime);
            oscGain.gain.exponentialRampToValueAtTime(0.01, startTime + duration);

            osc.start(startTime);
            osc.stop(startTime + duration);
        };

        gainNode.gain.setValueAtTime(1, now);

        // Classic "ding-dong" two-note chime
        playChime(659, now, 0.4); // E5 - ding
        playChime(523, now + 0.4, 0.5); // C5 - dong
    } catch (e) {
        console.log('Audio not available');
    }
}, []);

// Speak "New Week" when the week changes (Sunday midnight) with a female voice
const speakNewWeek = useCallback(() => {
    if (!window.speechSynthesis) return;
    try {
        const utterance = new SpeechSynthesisUtterance("New Week");
    }

```

```

utterance.rate = 0.9;
utterance.pitch = 1.1;
const voices = window.speechSynthesis.getVoices() || [];
const femaleVoice = voices.find(v =>
  v.name.toLowerCase().includes('female') ||
  v.name.toLowerCase().includes('samantha') ||
  v.name.toLowerCase().includes('victoria') ||
  v.name.toLowerCase().includes('karen') ||
  v.name.toLowerCase().includes('moira') ||
  v.name.toLowerCase().includes('fiona') ||
  v.name.toLowerCase().includes('zira') ||
  v.name.includes('Google UK English Female') ||
  v.name.includes('Google US English')
);
if (femaleVoice) {
  utterance.voice = femaleVoice;
}
window.speechSynthesis.speak(utterance);
} catch (e) {
  console.log('Speech synthesis not available');
}
}, []);

const speakNewDay = useCallback(() => {
  if (!window.speechSynthesis) return;
  try {
    const utterance = new SpeechSynthesisUtterance("New Day");
    utterance.rate = 0.9;
    utterance.pitch = 1.1;
    const voices = window.speechSynthesis.getVoices() || [];
    const femaleVoice = voices.find(v =>
      v.name.toLowerCase().includes('female') ||
      v.name.toLowerCase().includes('samantha') ||
      v.name.toLowerCase().includes('victoria') ||
      v.name.toLowerCase().includes('karen') ||
      v.name.toLowerCase().includes('moira') ||
      v.name.toLowerCase().includes('fiona') ||
      v.name.toLowerCase().includes('zira') ||
      v.name.includes('Google UK English Female') ||
      v.name.includes('Google US English')
    );
    if (femaleVoice) {
      utterance.voice = femaleVoice;
    }
    window.speechSynthesis.speak(utterance);
  } catch (e) {
    console.log('Speech synthesis not available');
  }
}, []);

// Update clock every second and detect week change (Sunday midnight)
const lastWeekRef = useRef(() => {
  const now = new Date();
  const sun = new Date(now.getFullYear(), now.getMonth(), now.getDate() - now.getDay());
  return `${sun.getFullYear()}-${sun.getMonth()}-${sun.getDate()}`;
})();
const lastDateRef = useRef(new Date().getDate());
useEffect(() => {
  const getWeekNumber = (date: Date) => {
    const sun = new Date(date.getFullYear(), date.getMonth(), date.getDate() - date.getDay());
    return `${sun.getFullYear()}-${sun.getMonth()}-${sun.getDate()}`;
  };
  const currentWeekNum = getWeekNumber(new Date());
  lastWeekRef.current = currentWeekNum;

  let lastMinute = new Date().getMinutes();
  let lastDate = new Date().getDate();
  const timer = setInterval(() => {
    const now = new Date();
    const currentDate = now.getDate();

    const weekNum = getWeekNumber(now);
    const isNewWeek = weekNum !== lastWeekRef.current;
    if (isNewWeek) {
      lastWeekRef.current = weekNum;
      speakNewWeek();
    }

    if (currentDate !== lastDateRef.current) {
      lastDateRef.current = currentDate;
      if (!isNewWeek) {
        const isTravelling = !(JSON.parse(localStorage.getItem('schoolData') || '{}').isTravelling);
        if (!isTravelling) {
          speakNewDay();
        }
      }
    }
  }, 1000);
  const secEl = document.getElementById('clock-seconds');
  if (secEl) secEl.textContent = ':' + String(now.getSeconds()).padStart(2, '0');
  if (now.getMinutes() !== lastMinute) {
    lastMinute = now.getMinutes();
    const hmEl = document.getElementById('clock-hm');
    const ampEl = document.getElementById('clock-ampm');
    const tz = displayTimezoneRef.current;
    if (hmEl) {
      hmEl.textContent = new Intl.DateTimeFormat('en-US', { hour: 'numeric', minute: '2-digit', hour12: true, timeZ...
    }
    if (ampEl) {
      ampEl.textContent = new Intl.DateTimeFormat('en-US', { hour: 'numeric', hour12: true, timeZone: tz }).form...
    }
  }
  if (currentDate !== lastDate) {
    lastDate = currentDate;
    setCurrentTime(now);
  }
}, 10000);
return () => clearInterval(timer);
}, [speakNewWeek, speakNewDay]);

```



```

// Check if mute period has expired
useEffect(() => {
  if (muteUntil && Date.now() >= muteUntil) {
    setIsMuted(false);
    setMuteUntil(null);
    localStorage.removeItem('alarmMuteUntil');
  }
  const checkInterval = setInterval(() => {
    if (muteUntil && Date.now() >= muteUntil) {
      setIsMuted(false);
      setMuteUntil(null);
      localStorage.removeItem('alarmMuteUntil');
    }
  }, 10000); // Check every 10 seconds
  return () => clearInterval(checkInterval);
}, [muteUntil]);

// Toggle mute for 30 minutes
const toggleMute = useCallback(() => {
  if (isMuted) {
    setIsMuted(false);
    setMuteUntil(null);
    localStorage.removeItem('alarmMuteUntil');
  } else {
    const muteTime = Date.now() + 30 * 60 * 1000; // 30 minutes
    setIsMuted(true);
    setMuteUntil(muteTime);
    localStorage.setItem('alarmMuteUntil', muteTime.toString());
  }
}, [isMuted]);

// Jiggle the Do Today box every 1 minute (only if there are tasks and not muted)
useEffect(() => {
  const interval = setInterval(() => {
    if (todayTaskCountRef.current > 0 && !isMuted) {
      setDoTodayBounce(true);
      setTimeout(() => setDoTodayBounce(false), 1000);
    }
  }, 60000); // 1 minute
  return () => clearInterval(interval);
}, [isMuted]);

// Play jiggle sound every 30 minutes (only if there are tasks and not muted)
useEffect(() => {
  const interval = setInterval(() => {
    if (todayTaskCountRef.current > 0 && !isMuted) {
      playJiggleSound();
    }
  }, 1800000); // 30 minutes
  return () => clearInterval(interval);
}, [playJiggleSound, isMuted]);

// Calendar resize handlers
const handleResizeStart = useCallback((e: React.MouseEvent | React.TouchEvent) => {
  e.preventDefault();
  const { clientY } = getPointerXY(e);
  setIsResizing(true);
  resizeRef.current = { startY: clientY, startHeight: calendarHeight, _undoHeight: calendarHeight };
}, [calendarHeight]);

const handleTopResizeStart = useCallback((e: React.MouseEvent | React.TouchEvent) => {
  e.preventDefault();
  const { clientY } = getPointerXY(e);
  setIsResizing(true);
  resizeRef.current = { startY: clientY, startHeight: calendarHeight, _undoHeight: calendarHeight, invertDirection: true };
}, [calendarHeight]);

const handleWidthResizeStart = useCallback((e: React.MouseEvent | React.TouchEvent) => {
  e.preventDefault();
  const { clientX } = getPointerXY(e);
  setIsResizingHomework(true);
  resizingHomeworkRef.current = { startX: clientX, startReduction: calendarReduction };
}, [calendarReduction]);

const handleCombinedResizeStart = useCallback((e: React.MouseEvent | React.TouchEvent) => {
  e.preventDefault();
  const { clientX, clientY } = getPointerXY(e);
  setIsResizing(true);
  setIsResizingHomework(true);
  resizeRef.current = { startY: clientY, startHeight: calendarHeight, _undoHeight: calendarHeight };
  resizingHomeworkRef.current = { startX: clientX, startReduction: calendarReduction };
}, [calendarHeight, calendarReduction]);

const calendarHeightRef = useRef(calendarHeight);
calendarHeightRef.current = calendarHeight;

useEffect(() => {
  if (!isResizing) return;

  let lastHeight = calendarHeightRef.current;

  const handleMove = (e: MouseEvent | TouchEvent) => {
    if (!resizeRef.current) return;
    e.preventDefault();
    const { clientY } = getPointerXY(e);
    const rawDelta = clientY - resizeRef.current.startY;
    const delta = (resizeRef.current as any).invertDirection ? -rawDelta : rawDelta;
    const newHeight = Math.max(200, Math.min(window.innerHeight - 60, resizeRef.current.startHeight + delta));
    lastHeight = newHeight;
    setCalendarHeight(newHeight);
  };

  const handleEnd = () => {
    if (resizeRef.current && resizeRef.current._undoHeight !== undefined) {
      const oldHeight = resizeRef.current._undoHeight;
      if (oldHeight !== lastHeight) {
        setUndoStack(prev => [{ type: 'resize', description: 'Resize calendar height', data: { resizeType: 'height'...

```

```

        setRedoStack([]);
    }
}
localStorage.setItem('calendarHeight', String(lastHeight));
const sw = window.screen.width, sh = window.screen.height, pr = window.devicePixelRatio || 1;
localStorage.setItem('calendarHeight_device_${sw}x${sh}@${pr}', String(lastHeight));
setIsResizing(false);
resizeRef.current = null;
};

document.addEventListener('mousemove', handleMove, true);
document.addEventListener('mouseup', handleEnd, true);
document.addEventListener('touchmove', handleMove, { passive: false, capture: true });
document.addEventListener('touchend', handleEnd, true);

return () => {
    document.removeEventListener('mousemove', handleMove, true);
    document.removeEventListener('mouseup', handleEnd, true);
    document.removeEventListener('touchmove', handleMove, true);
    document.removeEventListener('touchend', handleEnd, true);
};
}, [isResizing]);

const { data: weeks = [] } = useQuery<WeekInfo[]>({
    queryKey: ["/api/weeks"],
});

const { data: allCourseWeekMappings = [] } = useQuery<CourseWeekMapping[]>({
    queryKey: ["/api/course-week-mappings"],
});

const courseWeekVariants = useMemo(() => {
    const variants: Record<number, { courseCode: string; label: string }[]> = {};
    for (const m of allCourseWeekMappings) {
        if (m.courseWeekLabel && m.courseWeekLabel.trim()) {
            if (!variants[m.weekNumber]) variants[m.weekNumber] = [];
            variants[m.weekNumber].push({ courseCode: m.courseCode, label: m.courseWeekLabel });
        }
    }
    return variants;
}, [allCourseWeekMappings]);

// Automatically set selectedWeek based on today's date (re-checks when date changes)
const lastAutoWeekDateRef = useRef(new Date().getDate());
useEffect(() => {
    if (weeks.length > 0) {
        const today = new Date();
        const todayDateOnly = new Date(today.getFullYear(), today.getMonth(), today.getDate());
        const currentWeek = weeks.find(w => {
            const start = parseISO(w.startDate);
            const end = parseISO(w.endDate);
            const startDateOnly = new Date(start.getFullYear(), start.getMonth(), start.getDate());
            const endDateOnly = new Date(end.getFullYear(), end.getMonth(), end.getDate());
            return todayDateOnly >= startDateOnly && todayDateOnly <= endDateOnly;
        });
        if (currentWeek) {
            setSelectedWeek(currentWeek.weekNumber);
        } else {
            const cachedSem = queryClient.getQueryData<SemesterSettings | null>(["/api/semester"]);
            const semStart = cachedSem?.semesterStartDate ? new Date(cachedSem.semesterStartDate) : undefined;
            const readingWeek = cachedSem?.readingWeekStart || null;
            const computedWeek = getWeekNumber(today, semStart, readingWeek);
            setSelectedWeek(computedWeek);
        }
        lastAutoWeekDateRef.current = today.getDate();
    }
}, [weeks]);

useEffect(() => {
    if (weeks.length === 0) return;
    const currentDate = currentTime.getDate();
    if (currentDate !== lastAutoWeekDateRef.current) {
        lastAutoWeekDateRef.current = currentDate;
        const todayDateOnly = new Date(currentTime.getFullYear(), currentTime.getMonth(), currentTime.getDate());
        const currentWeek = weeks.find(w => {
            const start = parseISO(w.startDate);
            const end = parseISO(w.endDate);
            const startDateOnly = new Date(start.getFullYear(), start.getMonth(), start.getDate());
            const endDateOnly = new Date(end.getFullYear(), end.getMonth(), end.getDate());
            return todayDateOnly >= startDateOnly && todayDateOnly <= endDateOnly;
        });
        if (currentWeek) {
            setSelectedWeek(currentWeek.weekNumber);
        } else {
            const cachedSem = queryClient.getQueryData<SemesterSettings | null>(["/api/semester"]);
            const semStart = cachedSem?.semesterStartDate ? new Date(cachedSem.semesterStartDate) : undefined;
            const readingWeek = cachedSem?.readingWeekStart || null;
            const computedWeek = getWeekNumber(currentTime, semStart, readingWeek);
            setSelectedWeek(computedWeek);
        }
    }
}, [currentTime, weeks]);

const { data: allTasks = [] } = useQuery<Task[]>({
    queryKey: ["/api/tasks"],
    queryFn: () => fetch("/api/tasks", { credentials: 'include' }).then(r => { if (!r.ok) throw new Error(r.statusText);
    retry: 2,
    retryDelay: 1000,
});

const hasUnackedReminders = allTasks.some((t: any) => t.type === 'reminder' && t.isAcknowledged === false && !t.isC...

useEffect(() => {
    if (!allTasks || allTasks.length === 0) return;
    const courseTasksMap: Record<string, Task[]> = {};
    for (const t of allTasks) {
        if (!t.courseName) continue;
        const cn = t.courseName.split(' - ')[0]?.trim().replace(/\s/g, '').toUpperCase();

```

```

    if (!cn) continue;
    if (!courseTasksMap[cn]) courseTasksMap[cn] = [];
    courseTasksMap[cn].push(t);
}
const updates: Record<string, { grade: string; percent: string }> = {};
for (const [cn, tasks] of Object.entries(courseTasksMap)) {
    const graded = tasks.filter(t => t.gradeWeight && t.gradeTotal && t.gradeValue !== null && t.gradeValue !== und...
    if (graded.length === 0) continue;
    let wSum = 0, wTotal = 0;
    for (const t of graded) {
        const pct = (t.gradeValue! / t.gradeTotal!) * 100;
        wSum += pct * (t.gradeWeight! / 100);
        wTotal += t.gradeWeight!;
    }
    if (wTotal <= 0) continue;
    const currentPercent = Math.round((wSum / wTotal) * 1000) / 10;
    const grade = percentToGrade(String(currentPercent));
    for (const [ck, info] of Object.entries(certCourseMap)) {
        const mapCode = info.code.replace(/s/g, '').toUpperCase();
        if (mapCode === cn || cn.endsWith(mapCode) || mapCode.endsWith(cn.replace(/C/, ''))) {
            if (!shouldStrikethrough(ck)) {
                updates[ck] = { grade, percent: String(currentPercent) };
            }
        }
    }
}
const strikeClearKeys: string[] = [];
for (const ck of Object.keys(certCourseMap)) {
    if (shouldStrikethrough(ck)) strikeClearKeys.push(ck);
}
if (Object.keys(updates).length > 0 || strikeClearKeys.length > 0) {
    setCourseGrades(prev => {
        const merged = { ...prev };
        let changed = false;
        for (const [k, v] of Object.entries(updates)) {
            if (!merged[k]?.percent || merged[k].percent.trim() === '') {
                merged[k] = v;
                changed = true;
            }
        }
        for (const sk of strikeClearKeys) {
            if (merged[sk]?.percent && merged[sk].percent.trim() !== '') {
                const isDefault = ['PPA101', 'PPA102', 'PPA125', 'L1_PPA120', 'L1_PPA121', 'LIBERAL'].includes(sk);
                if (!isDefault) {
                    delete merged[sk];
                    changed = true;
                }
            }
        }
        if (!changed) return prev;
        localStorage.setItem('courseGrades', JSON.stringify(merged));
        return merged;
    });
}
}, [allTasks]);

useEffect(() => {
    const defaultKeys = new Set(['PPA101', 'PPA102', 'PPA125', 'L1_PPA120', 'L1_PPA121', 'LIBERAL']);
    setCourseGrades(prev => {
        const cleaned = { ...prev };
        let changed = false;
        for (const ck of Object.keys(cleaned)) {
            if (defaultKeys.has(ck)) continue;
            if (shouldStrikethrough(ck) && cleaned[ck]?.percent && cleaned[ck].percent.trim() !== '') {
                delete cleaned[ck];
                changed = true;
            }
        }
        if (!changed) return prev;
        localStorage.setItem('courseGrades', JSON.stringify(cleaned));
        return cleaned;
    });
}, []);

useEffect(() => {
    const measure = () => {
        const els = document.querySelectorAll('[data-upcoming-task-name]');
        if (!els.length) return;
        let max = 0;
        els.forEach(el => {
            const range = document.createRange();
            range.selectNodeContents(el);
            const w = range.getBoundingClientRect().width;
            if (w > max) max = w;
        });
        if (max > 0) setMaxTaskNameWidth(Math.ceil(max));
    };
    requestAnimationFrame(() => setTimeout(measure, 50));
    const t2 = setTimeout(measure, 300);
    const t3 = setTimeout(measure, 800);
    return () => { clearTimeout(t2); clearTimeout(t3); };
}, [allTasks, selectedWeek]);

const { data: tasks = [], isLoading } = useQuery<Task[]>({
    queryKey: ["/api/tasks", { weekNumber: selectedWeek }],
    queryFn: () => fetch(`/api/tasks?weekNumber=${selectedWeek}`, { credentials: 'include' }).then(r => { if (!r.ok) ...
    retry: 2,
    retryDelay: 1000,
});

// Fetch all projects for calendar display
const { data: allProjects = [] } = useQuery<Project[]>({
    queryKey: ["/api/projects"],
    queryFn: () => fetch("/api/projects", { credentials: 'include' }).then(r => { if (!r.ok) throw new Error(r.status...
    retry: 2,
    retryDelay: 1000,
});

```

```

// Sticky notes removed – functionality moved to Quick Notes (/onenote)

// Sticky note handlers removed – moved to Quick Notes

// Sticky note drag/resize handlers removed – moved to Quick Notes

// Files for weekly files flyout (moved up for allTaskFiles dependency)
interface WeeklyFile {
  id: number;
  originalName: string;
  displayName: string;
  objectPath: string;
  contentType: string;
  size: number;
  folder: string | null;
  listened?: boolean;
  lastChunkIndex?: number;
  totalChunks?: number;
  checkedChunks?: string;
}

const { data: weeklyFiles = [] } = useQuery<WeeklyFile[]>({
  queryKey: ["/api/files"],
  retry: 2,
  retryDelay: 1000,
  staleTime: 5 * 1000,
  refetchInterval: 30 * 1000,
  refetchOnWindowFocus: 'always',
});

// OneDrive files for files flyout
interface OneDriveItem {
  id: string;
  name: string;
  type: "folder" | "file";
  mimeType?: string;
  size?: number;
  lastModified?: string;
  downloadUrl?: string;
  path: string;
}

const [oneDrivePath, setOneDrivePath] = useState("/School/1. TMU/Courses/2026/Winter");
const [oneDrivePathHistory, setOneDrivePathHistory] = useState<string[]>([]);

const { data: oneDriveItems = [], isLoading: oneDriveLoading } = useQuery<OneDriveItem[]>({
  queryKey: ["/api/onedrive/files", oneDrivePath],
  queryFn: async () => {
    const response = await fetch(`/api/onedrive/files?path=${encodeURIComponent(oneDrivePath)}`);
    if (!response.ok) {
      const err = await response.json();
      throw new Error(err.error || "Failed to load files");
    }
    return response.json();
  },
});

const oneDriveFolders = oneDriveItems
  .filter(item => item.type === "folder")
  .sort((a, b) => {
    // Course order to match calendar
    const courseOrder = ['CPPA122', 'CFNF400', 'CASL101', 'CECN210', 'CPHL110', 'CHIS105', 'CPPA235'];

    // Check if folders are course folders
    const getCourseIndex = (name: string) => {
      for (let i = 0; i < courseOrder.length; i++) {
        if (name.includes(courseOrder[i])) return i;
      }
      return -1;
    };

    const courseIdxA = getCourseIndex(a.name);
    const courseIdxB = getCourseIndex(b.name);

    // If both are course folders, sort by course order
    if (courseIdxA !== -1 && courseIdxB !== -1) {
      return courseIdxA - courseIdxB;
    }

    // Course folders come first
    if (courseIdxA !== -1) return -1;
    if (courseIdxB !== -1) return 1;

    // Extract week numbers for chronological sorting
    const weekMatch = (name: string) => {
      const match = name.match(/week\s*(\d+)/i);
      return match ? parseInt(match[1], 10) : null;
    };
    const weekA = weekMatch(a.name);
    const weekB = weekMatch(b.name);

    // Current week (week 4 based on semester schedule)
    const currentWeek = 4;

    // If both have week numbers, sort with past weeks at bottom
    if (weekA !== null && weekB !== null) {
      const isPastA = weekA < currentWeek;
      const isPastB = weekB < currentWeek;

      // If one is past and one is current/future, past goes last
      if (isPastA && !isPastB) return 1;
      if (!isPastA && isPastB) return -1;

      // If both are past or both are current/future, sort numerically
      return weekA - weekB;
    }

    // If only one has a week number, put it first
    if (weekA !== null) return -1;
  });

```

```

    if (weekB !== null) return 1;
    // Otherwise sort alphabetically
    return a.name.localeCompare(b.name);
  });
const oneDriveFiles = oneDriveItems.filter(item => item.type === "file");
const oneDrivePdfFiles = oneDriveFiles.filter(item => item.mimeType?.includes("pdf"));

// Extract all unique files from tasks for the FILES row
const allTaskFiles = useMemo(() => {
  const filesMap = new Map<string, { name: string; url: string; taskId: number; courseName: string }>();
  tasks?.forEach(task => {
    if (task.attachments && Array.isArray(task.attachments)) {
      task.attachments.forEach((att: any) => {
        let fileData: { name: string; url: string };
        if (typeof att === 'string') {
          try {
            const parsed = JSON.parse(att);
            fileData = { name: parsed.name || parsed.url?.split('/').pop() || 'File', url: parsed.url || att };
          } catch {
            fileData = { name: att.split('/').pop() || 'File', url: att };
          }
        } else {
          fileData = { name: att.name || att.url?.split('/').pop() || 'File', url: att.url || '' };
        }
        // Try to find matching file in weeklyFiles for displayName
        const matchingWeeklyFile = weeklyFiles.find(wf => wf.objectPath === fileData.url || fileData.url.includes(w...
        if (matchingWeeklyFile) {
          fileData.name = matchingWeeklyFile.displayName || matchingWeeklyFile.originalName || fileData.name;
        }
        const key = fileData.url;
        if (!filesMap.has(key)) {
          filesMap.set(key, { ...fileData, taskId: task.id, courseName: task.courseName || '' });
        }
      });
    }
  });
  return Array.from(filesMap.values());
}, [tasks, weeklyFiles]);

// Google Calendar events query
interface CalendarEvent {
  id: string;
  title: string;
  description: string;
  startDate: string;
  endDate: string;
  isAllDay: boolean;
  htmlLink: string;
  source: string;
}

const { data: calendarEvents = [] } = useQuery<CalendarEvent[]>({
  queryKey: ["/api/calendar/events", { weekNumber: selectedWeek }],
  queryFn: async () => {
    try {
      const r = await fetch(`/api/calendar/events?weekNumber=${selectedWeek}`, { credentials: 'include' });
      if (!r.ok) throw new Error(r.statusText);
      const data = await r.json();
      if (data && data._deletedTasks) {
        queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
        return data.events;
      }
      return Array.isArray(data) ? data : (data.events || []);
    } catch { return []; }
  },
  refetchInterval: 5 * 60 * 1000,
  retry: 2,
  retryDelay: 1000,
});

const { data: upcomingCalendarEvents = [] } = useQuery<CalendarEvent[]>({
  queryKey: ["/api/calendar/upcoming-events"],
  queryFn: () => fetch('/api/calendar/upcoming-events', { credentials: 'include' }).then(r => { if (!r.ok) throw ne...
  refetchInterval: 10 * 60 * 1000,
  retry: 2,
  retryDelay: 1000,
});

const { data: d2lAnnouncements = [] } = useQuery<any[]>({
  queryKey: ["/api/announcements"],
  queryFn: () => fetch('/api/announcements?limit=15', { credentials: 'include' }).then(r => { if (!r.ok) throw new ...
  refetchInterval: 5 * 60 * 1000,
  retry: 1,
});

const [tickerDialogOpen, setTickerDialogOpen] = useState(false);
const [newTickerText, setNewTickerText] = useState('');
const [newTickerTag, setNewTickerTag] = useState('Custom');
const addTickerMutation = useMutation({
  mutationFn: async ({ body, tag }: { body: string; tag: string }) => {
    const res = await apiRequest('POST', '/api/announcements', { body, courseName: tag });
    return res.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['/api/announcements'] });
    setNewTickerText('');
  },
});
const deleteTickerMutation = useMutation({
  mutationFn: async (id: number) => {
    await apiRequest('DELETE', `/api/announcements/${id}`);
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['/api/announcements'] });
  },
});
const reorderTickerMutation = useMutation({
  mutationFn: async (orderedIds: number[]) => {

```

```

    await apiRequest('POST', '/api/announcements/reorder', { orderedIds });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['/api/announcements'] });
  },
});
const [dismissedTodayTaskIds, setDismissedTodayTaskIds] = useState<Set<string>>(() => {
  try { const s = localStorage.getItem('dismissedTodayTaskIds'); if (s) { const parsed = JSON.parse(s); return new ...
    return new Set();
  });
const [tickerDragIdx, setTickerDragIdx] = useState<number | null>(null);
const [tickerDragOverIdx, setTickerDragOverIdx] = useState<number | null>(null);
const tickerTouchDragRef = useRef<{ startY: number; idx: number; el: HTMLElement | null } | null>(null);

// Filter files for the current week (exclude completed/listened files)
const currentWeekFiles = weeklyFiles.filter(f =>
  (f.folder?.startsWith(`week-${selectedWeek}`) || f.folder === `week-${selectedWeek}`) && !f.listened
);

// Semester settings query
const { data: semesterSettings } = useQuery<SemesterSettings | null>({
  queryKey: ["/api/semester"],
  queryFn: () => fetch("/api/semester", { credentials: 'include' }).then(r => { if (!r.ok) throw new Error(r.status...
  retry: 2,
  retryDelay: 1000,
});
useEffect(() => { semesterSettingsRef.current = semesterSettings ?? null; }, [semesterSettings]);

const { data: allSemesterSettings } = useQuery<any[]>({
  queryKey: ["/api/semesters"],
  queryFn: () => fetch("/api/semesters", { credentials: 'include' }).then(r => { if (!r.ok) throw new Error(r.statu...
});
const allSemesterSettingsRef = useRef(allSemesterSettings);
allSemesterSettingsRef.current = allSemesterSettings;

useEffect(() => {
  if (!semesterSettings) return;
  const sem = semesterSettings as any;
  setCoursesData(prev => {
    const updated = [...prev.courses];
    let changed = false;
    for (let i = 0; i < 3; i++) {
      const prefix = `course${i + 1}`;
      const semColor = sem[`${prefix}Color`];
      const semColorEnd = sem[`${prefix}ColorEnd`];
      const semColorStops = sem[`${prefix}ColorStops`];
      const semBorderColor = sem[`${prefix}BorderColor`];
      const semCourseRowColor = sem[`${prefix}CourseRowColor`];
      const semTaskBgColor = sem[`${prefix}TaskBgColor`];
      if (updated[i]) {
        const c = updated[i] as any;
        if (semColor && semColor !== c.color) { c.color = semColor; changed = true; }
        if (semColorEnd && semColorEnd !== c.colorEnd) { c.colorEnd = semColorEnd; changed = true; }
        if (semColorStops !== undefined && semColorStops !== null && semColorStops !== c.colorStops) { c.colorStops...
        if (semBorderColor !== undefined && semBorderColor !== null && semBorderColor !== c.borderColor) { c.border...
        if (semCourseRowColor !== undefined && semCourseRowColor !== null && semCourseRowColor !== c.courseRowColor...
        if (semTaskBgColor !== undefined && semTaskBgColor !== null && semTaskBgColor !== c.taskBgColor) { c.taskBg...
      }
    }
    if (changed) {
      const newData = { courses: updated };
      localStorage.setItem('coursesData', JSON.stringify(newData));
      return newData;
    }
    return prev;
  });
}, [semesterSettings]);

const [deliveryModeVersion, setDeliveryModeVersion] = useState(0);
const courseDeliveryModes = useMemo(() => {
  const map: Record<string, string> = {};
  try {
    const cd = localStorage.getItem('certCourseData');
    if (cd) {
      const sd = JSON.parse(cd);
      for (const [key, val] of Object.entries(sd)) {
        if ((val as any)?.deliveryMode) {
          map[key.replace(/\\s/g, '')] = (val as any).deliveryMode;
        }
      }
    }
  } catch {}
  if (allSemesterSettings) {
    for (const sem of allSemesterSettings) {
      for (let i = 1; i <= 3; i++) {
        const code = (sem[ `course${i}Code` ] || '').replace(/\\s/g, '');
        const mode = sem[ `course${i}DeliveryMode` ] || '';
        if (code && mode) map[code] = mode;
      }
    }
  }
  return map;
}, [allSemesterSettings, deliveryModeVersion]);

useEffect(() => {
  if (!semesterSettings) return;
  const dbColors: Record<number, { color?: string; colorEnd?: string; colorStops?: string }> = {};
  for (let i = 1; i <= 3; i++) {
    const c = (semesterSettings as any)[`course${i}Color`];
    const ce = (semesterSettings as any)[`course${i}ColorEnd`];
    const cs = (semesterSettings as any)[`course${i}ColorStops`];
    if (c) dbColors[i - 1] = { color: c, colorEnd: ce || undefined, colorStops: cs || undefined };
  }
  if (Object.keys(dbColors).length > 0) {
    setCoursesData(prev => {
      const updated = { ...prev, courses: prev.courses.map((course, idx) => {
        const db = dbColors[idx];
        if (db) return { ...course, color: db.color || course.color, colorEnd: db.colorEnd || course.colorEnd, colo...
      }
    }
  }

```

```

        return course;
    });
    localStorage.setItem('coursesData', JSON.stringify(updated));
    saveDegreeToServer('coursesData', updated);
    return updated;
  });
}, [semesterSettings]);

useEffect(() => {
  try {
    const cd = localStorage.getItem('certCourseData');
    if (!cd) return;
    const savedData = JSON.parse(cd);
    let changed = false;
    for (const course of coursesData.courses) {
      if (!course.name?.trim()) continue;
      const code = course.name.split(' - ')[0]?.trim();
      if (savedData[code] && course.color) {
        if (savedData[code].color !== course.color || savedData[code].colorEnd !== (course.colorEnd || '')) {
          savedData[code].color = course.color;
          savedData[code].colorEnd = course.colorEnd || '';
          changed = true;
        }
      }
    }
    if (changed) {
      localStorage.setItem('certCourseData', JSON.stringify(savedData));
      saveDegreeToServer('certCourseData', savedData);
    }
  } catch {}
}, [coursesData]);

useEffect(() => {
  const synced = sessionStorage.getItem('professorContactsSynced');
  if (synced) return;
  sessionStorage.setItem('professorContactsSynced', '1');
  for (const course of coursesData.courses) {
    if (course.professor?.trim()) {
      const code = course.name.split(' - ')[0]?.trim();
      fetch('/api/key-contacts/sync-professor', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ professorName: course.professor, professorEmail: course.professorEmail, courseCode: ...
      }).catch(() => {});
    }
  }
  const syncedNames = new Set(coursesData.courses.filter(c => c.professor?.trim()).map(c => c.professor!.toLowerCase()));
  try {
    const certData = localStorage.getItem('certCourseData');
    if (certData) {
      const savedData = JSON.parse(certData);
      const currentCodes = new Set(coursesData.courses.map(c => c.name.split(' - ')[0]?.trim().toUpperCase().replace(/ /g, '')));
      for (const [certKey, info] of Object.entries(savedData)) {
        const ci = info as any;
        if (ci.professor?.trim() && !currentCodes.has(certKey.toUpperCase().replace(/ /g, ''))) {
          syncedNames.add(ci.professor.toLowerCase());
          fetch('/api/key-contacts/sync-professor', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ professorName: ci.professor, professorEmail: ci.professorEmail || '', courseCode: ...
          }).catch(() => {});
        }
      }
    }
  } catch {}
}, []);

useEffect(() => {
  if (!pastCourseInfo || Object.keys(pastCourseInfo).length === 0) return;
  const synced = sessionStorage.getItem('pastProfContactsSynced');
  if (synced) return;
  sessionStorage.setItem('pastProfContactsSynced', '1');
  const existingNames = new Set(coursesData.courses.filter(c => c.professor?.trim()).map(c => c.professor!.toLowerCase()));
  try {
    const certData = localStorage.getItem('certCourseData');
    if (certData) {
      for (const info of Object.values(JSON.parse(certData))) {
        const ci = info as any;
        if (ci.professor?.trim()) existingNames.add(ci.professor.toLowerCase());
      }
    }
  } catch {}
  for (const [courseKey, info] of Object.entries(pastCourseInfo)) {
    if (info.professor?.trim() && !existingNames.has(info.professor.toLowerCase())) {
      existingNames.add(info.professor.toLowerCase());
      fetch('/api/key-contacts/sync-professor', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ professorName: info.professor, professorEmail: info.email || '', courseCode: courseK...
      }).catch(() => {});
    }
  }
}, [pastCourseInfo]);

useEffect(() => {
  if (!semesterSettings || semesterChecklistShownRef.current) return;

  const now = new Date(new Date().toLocaleString('en-US', { timeZone: 'America/Toronto' }));
  const lastShownKey = `semChecklist_lastShown_${semesterSettings.id}`;
  const snoozeUntilKey = `semChecklist_snoozeUntil_${semesterSettings.id}`;
  const allDoneKey = `semChecklist_allDone_${semesterSettings.id}`;
  if (localStorage.getItem(allDoneKey) === 'true') return;
  const snoozeUntil = parseInt(localStorage.getItem(snoozeUntilKey) || '0', 10);
  if (snoozeUntil && now.getTime() < snoozeUntil) return;
  const lastShown = parseInt(localStorage.getItem(lastShownKey) || '0', 10);
  const hoursSinceShown = (now.getTime() - lastShown) / (1000 * 60 * 60);

```

```

if (hoursSinceShown < 8) return;

fetch('/api/semester-checklist', { credentials: 'include' })
  .then(r => r.json())
  .then(data => {
    if (data.allChecked) {
      localStorage.setItem(allDoneKey, 'true');
      return;
    }
    setSemesterChecklistItems(data.items);
    setSemesterChecklistId(data.semesterId);
    setShowSemesterChecklist(true);
    localStorage.setItem(lastShownKey, String(now.getTime()));
    semesterChecklistShownRef.current = true;
  })
  .catch(err => console.error('Error fetching semester checklist:', err));
}, [semesterSettings]);

useEffect(() => {
  if (confirmSemesterShownRef.current) return;
  const upcoming = getUpcomingSemesterToConfirm();
  if (!upcoming) return;

  const dismissedKey = `semConfirm_dismissed_${upcoming.label}`;
  const dismissed = localStorage.getItem(dismissedKey);
  if (dismissed === 'confirmed') return;

  const lastShownKey = `semConfirm_lastShown_${upcoming.label}`;
  const lastShown = parseInt(localStorage.getItem(lastShownKey) || '0', 10);
  const now = Date.now();
  const hoursSinceShown = (now - lastShown) / (1000 * 60 * 60);
  if (hoursSinceShown < 12) return;

  setUpcomingSemester(upcoming);
  setConfirmSemesterForm({ startDate: upcoming.startDate, endDate: upcoming.endDate, breakStart: upcoming.breakStar...
  setShowConfirmSemesterDialog(true);
  localStorage.setItem(lastShownKey, String(now));
  confirmSemesterShownRef.current = true;
}, []);

useEffect(() => {
  const todaySemester = getNextSemesterByStartDate();
  if (!todaySemester) return;
  const resetKey = `semReset_done_${todaySemester.label}`;
  if (localStorage.getItem(resetKey)) return;

  fetch('/api/semester-reset-files', { method: 'POST', credentials: 'include' })
    .then(r => r.json())
    .then(data => {
      if (data.success) {
        console.log(`[Semester Start] Reset ${data.resetCount} files for ${todaySemester.label}`);
        localStorage.setItem(resetKey, 'true');
      }
    })
    .catch(err => console.error('Error resetting files for new semester:', err));
}, []);

useEffect(() => {
  if (!semesterSettings || pdfUploadShownRef.current) return;
  const now = new Date(new Date().toLocaleString('en-US', { timeZone: 'America/Toronto' }));
  const semEnd = semesterSettings.semesterEndDate ? new Date(semesterSettings.semesterEndDate) : null;
  if (!semEnd) return;
  const daysUntilEnd = Math.floor((semEnd.getTime() - now.getTime()) / (1000 * 60 * 60 * 24));
  if (daysUntilEnd > 14 || daysUntilEnd < 0) return;
  const lastShownKey = `pdfUpload_lastShown_${semesterSettings.id}`;
  const lastShown = parseInt(localStorage.getItem(lastShownKey) || '0', 10);
  const hoursSinceShown = (now.getTime() - lastShown) / (1000 * 60 * 60);
  if (hoursSinceShown < 336) return;
  setShowPdfUploadDialog(true);
  localStorage.setItem(lastShownKey, String(now.getTime()));
  pdfUploadShownRef.current = true;
}, [semesterSettings]);

// Deleted folders query for hamburger menu filtering
const { data: deletedFoldersData = [] } = useQuery<{ id: number; folderId: string }[]>({
  queryKey: ["/api/deleted-folders"],
});
const deletedFolderIds = new Set(deletedFoldersData.map(f => f.folderId));

// Files query for hamburger menu
interface FileItem {
  id: number;
  originalName: string;
  displayName: string;
  objectPath: string;
  folder: string | null;
  listened?: boolean;
  lastChunkIndex?: number;
  totalChunks?: number;
  checkedChunks?: string;
}
const { data: allFiles = [] } = useQuery<FileItem[]>({
  queryKey: ["/api/files"],
});

// File preview dialog state
const [previewFile, setPreviewFile] = useState<FileItem | null>(null);
const previewFileRef = useRef<FileItem | null>(null);
const [oneDrivePreviewFiles, setOneDrivePreviewFiles] = useState<FileItem[]>([]);
const [dialogPos, setDialogPos] = useState<{ x: number; y: number } | null>(null);
const dialogDragRef = useRef<{ startX: number; startY: number; origX: number; origY: number } | null>(null);
const [isLoadingOneDriveFiles, setIsLoadingOneDriveFiles] = useState(false);
// Cache for file counts by folder with listened breakdown (e.g., "week-4-cppa122-module": { total: 3, listened: 1,...
const [fileCounts, setFileCounts] = useState<Record<string, { total: number; listened: number; unlistened: number; ...
// Legacy: for backward compatibility with OneDrive-only counts
const [oneDriveFileCounts, setOneDriveFileCounts] = useState<Record<string, number>>({});
const [fileSelectorGlow, setFileSelectorGlow] = useState(false);
const fileSelectorGlowTimeoutRef = useRef<NodeJS.Timeout | null>(null);

```



```

const clickedButton = null as string | null;
const triggerButtonGlow = (_buttonId: string) => {};
const [previewSpeaker, setPreviewSpeaker] = useState<string>("browser_tts");
const previewSpeakerRef = useRef<string>("browser_tts");
const [volumeOverlay, setVolumeOverlay] = useState<{ volume: number; direction: string } | null>(null);
const volumeOverlayTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
const lastVolumeTimestampRef = useRef<number>(0);
const [previewText, setPreviewText] = useState<string>("");
const [isLoadingText, setIsLoadingText] = useState(false);
const [isPlaying, setIsPlaying] = useState(false);
const [currentWordIndex, setCurrentWordIndex] = useState(0);
const [syncHighlight, setSyncHighlight] = useState(true); // Sync text highlighting with TTS
const [ttsSearchOpen, setTtsSearchOpen] = useState(false);
const [ttsSearchQuery, setTtsSearchQuery] = useState("");
const [ttsSearchMatchIndex, setTtsSearchMatchIndex] = useState(0);
const ttsSearchInputRef = useRef<HTMLInputElement>(null);
const [ttsReplaceText, setTtsReplaceText] = useState("");
const [ttsShowReplace, setTtsShowReplace] = useState(false);
const [deletedWordIndices, setDeletedWordIndices] = useState<Set<number>>(new Set());
const [deletedWordHistory, setDeletedWordHistory] = useState<Set<number>[]>([]);
const [showRemoveButton, setShowRemoveButton] = useState(false);
const [removeButtonPos, setRemoveButtonPos] = useState({ x: 0, y: 0 });
const [isEditingTtsText, setIsEditingTtsText] = useState(false);
const [editableTtsText, setEditableTtsText] = useState("");
const [showFlickMenu, setShowFlickMenu] = useState(false);
const [flickDeviceGroups, setFlickDeviceGroups] = useState<Array<{room: string; icon: string; devices: Array<{id: s...
const [isFlicking, setIsFlicking] = useState(false);
const ttsTextContainerRef = useRef<HTMLDivElement>(null);
const editTextareaRef = useRef<HTMLTextAreaElement>(null);
const [availableVoices, setAvailableVoices] = useState<SpeechSynthesisVoice[]>([]);
const [selectedVoice, setSelectedVoice] = useState<string>(""); // Voice name
const [playStartTime, setPlayStartTime] = useState<number | null>(null);

// Chunked TTS state for reliable playback
const [ttsChunks, setTtsChunks] = useState<string[]>([]);
const ttsSearchMatchCount = useMemo(() => {
  if (ttsSearchQuery.trim().length < 2) return 0;
  const sLower = ttsSearchQuery.toLowerCase().trim();
  if (isEditingTtsText && editableTtsText) {
    const textLower = editableTtsText.toLowerCase();
    let count = 0;
    let pos = 0;
    while ((pos = textLower.indexOf(sLower, pos)) !== -1) {
      count++;
      pos += 1;
    }
    return count;
  }
  if (ttsChunks.length === 0) return 0;
  const sWords = sLower.split(/\s+/).filter(w => w.length > 0);
  const allWords: string[] = [];
  for (const chunk of ttsChunks) {
    const paragraphs = chunk.split(/\n\n+/);
    for (const paragraph of paragraphs) {
      const lines = paragraph.split(/\n/).filter(l => l.trim().length > 0);
      for (const line of lines) {
        const words = line.trim().split(/\s+/).filter(w => w.length > 0);
        allWords.push(...words);
      }
    }
  }
  let count = 0;
  for (let i = 0; i <= allWords.length - sWords.length; i++) {
    let match = true;
    for (let j = 0; j < sWords.length; j++) {
      if (!allWords[i + j].toLowerCase().replace(/[\.,;!?'"()]/g, '').includes(sWords[j])) {
        match = false;
        break;
      }
    }
    if (match) count++;
  }
  return count;
}, [ttsSearchQuery, ttsChunks, isEditingTtsText, editableTtsText]);
useEffect(() => {
  if (ttsSearchQuery.trim().length < 2 || ttsSearchMatchCount === 0) return;
  const timer = setTimeout(() => {
    if (isEditingTtsText && editTextareaRef.current) {
      const textarea = editTextareaRef.current;
      const text = textarea.value.toLowerCase();
      const searchLower = ttsSearchQuery.toLowerCase().trim();
      let pos = -1;
      let matchIdx = 0;
      let startFrom = 0;
      while (matchIdx <= ttsSearchMatchIndex) {
        pos = text.indexOf(searchLower, startFrom);
        if (pos === -1) break;
        if (matchIdx === ttsSearchMatchIndex) break;
        startFrom = pos + 1;
        matchIdx++;
      }
      if (pos >= 0) {
        textarea.focus();
        textarea.setSelectionRange(pos, pos + searchLower.length);
        const linesBefore = textarea.value.substring(0, pos).split('\n').length - 1;
        const lineHeight = parseFloat(getComputedStyle(textarea).lineHeight) || 19;
        const scrollTarget = linesBefore * lineHeight - textarea.clientHeight / 2;
        textarea.scrollTop = Math.max(0, scrollTarget);
      }
    } else {
      const el = document.querySelector('[data-active-search-match="true"]');
      if (el) el.scrollIntoView({ behavior: 'smooth', block: 'center' });
    }
  }, 50);
  return () => clearTimeout(timer);
}, [ttsSearchMatchIndex, ttsSearchQuery, ttsSearchMatchCount, isEditingTtsText]);
const [currentChunkIndex, setCurrentChunkIndex] = useState(0);
const [totalChunks, setTotalChunks] = useState(0);

```

```

const [checkedChunks, setCheckedChunks] = useState<Set<number>>(new Set());
const checkedChunksRef = useRef<Set<number>>(new Set());
const ttsChunksRef = useRef<string[]>([]);
const currentChunkIndexRef = useRef(0);
const shouldContinueRef = useRef(false);
const ttsKeepAliveRef = useRef<ReturnType<typeof setInterval> | null>(null);

// OpenAI TTS for Fire tablets (no browser speechSynthesis)
const openaiAudioRef = useRef<HTMLAudioElement | null>(null);
const activeWordRef = useRef<HTMLSpanElement | null>(null);
const openaiNextChunkRef = useRef<{ blob: Blob; index: number } | null>(null);
const [isOpenAITtsAvailable] = useState(() => !window.speechSynthesis);
const [openaiVoice, setOpenaiVoice] = useState<"alloy" | "ash" | "coral" | "echo" | "fable" | "nova" | "onyx" | "sa...
  const saved = localStorage.getItem('pdf-reader-voice');
  return (saved && ["alloy", "ash", "coral", "echo", "fable", "nova", "onyx", "sage", "shimmer"].includes(saved) ? saved : ...
]);

// Pre-fetch the next chunk for seamless playback
const prefetchNextChunk = async (nextIndex: number, voice: typeof openaiVoice) => {
  if (nextIndex >= ttsChunksRef.current.length) return;
  const nextChunk = ttsChunksRef.current[nextIndex];
  if (!nextChunk) return;

  try {
    const response = await fetch('/api/tts', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ text: nextChunk, voice }),
    });
    if (response.ok) {
      const blob = await response.blob();
      openaiNextChunkRef.current = { blob, index: nextIndex };
    }
  } catch (err) {
    console.log('Prefetch failed, will fetch on demand');
  }
};

// Play text using OpenAI TTS (for Fire tablets and other devices without browser TTS)
const playWithOpenAITts = async (text: string, voice: typeof openaiVoice = openaiVoice, chunkIndex?: number) => {
  try {
    if (openaiAudioRef.current) {
      openaiAudioRef.current.onended = null;
      openaiAudioRef.current.onerror = null;
      openaiAudioRef.current.ontimeupdate = null;
      openaiAudioRef.current.pause();
      openaiAudioRef.current = null;
    }

    let audioBlob: Blob;
    const currentIndex = chunkIndex ?? currentChunkIndexRef.current;

    // Check if we have this chunk pre-fetched
    if (openaiNextChunkRef.current && openaiNextChunkRef.current.index === currentIndex) {
      audioBlob = openaiNextChunkRef.current.blob;
      openaiNextChunkRef.current = null;
    } else {
      toast({ title: "Generating speech..." });

      const response = await fetch('/api/tts', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ text, voice }),
      });

      if (!response.ok) {
        throw new Error('TTS generation failed');
      }

      audioBlob = await response.blob();
    }

    const audioUrl = URL.createObjectURL(audioBlob);
    const audio = new Audio(audioUrl);
    openaiAudioRef.current = audio;

    // Start pre-fetching the next chunk immediately
    if (currentIndex + 1 < ttsChunksRef.current.length) {
      prefetchNextChunk(currentIndex + 1, voice);
    }

    // Calculate word offset for highlighting: count words in all previous chunks
    let wordOffset = 0;
    for (let i = 0; i < currentIndex; i++) {
      wordOffset += ttsChunksRef.current[i].split(/\s+/).filter((w: string) => w.length > 0).length;
    }
    const chunkWords = text.split(/\s+/).filter(w => w.length > 0);
    const chunkWordCount = chunkWords.length;

    let lastSaveTime = 0;
    let highlightRafId: number | null = null;
    const pollHighlight = () => {
      if (!audio || audio.paused || audio.ended) return;
      const now = Date.now();
      if (now - lastSaveTime > 5000) {
        lastSaveTime = now;
        const currentFile = previewFileRef.current;
        if (currentFile) {
          saveTtsProgress(currentFile.id, currentChunkIndexRef.current, 0);
        }
      }
    };
    if (audio.duration && audio.duration > 0 && chunkWordCount > 0) {
      const progress = audio.currentTime / audio.duration;
      const estimatedWordIdx = Math.min(
        Math.floor(progress * chunkWordCount),
        chunkWordCount - 1
      );
      setCurrentWordIndex(wordOffset + estimatedWordIdx);
    }
  }
};

```

```

    }
    highlightRafId = requestAnimationFrame(pollHighlight);
  };
  audio.onplay = () => { highlightRafId = requestAnimationFrame(pollHighlight); };
  audio.onpause = () => { if (highlightRafId) { cancelAnimationFrame(highlightRafId); highlightRafId = null; } };
  audio.ontimeupdate = null;

  audio.onended = () => {
    if (highlightRafId) { cancelAnimationFrame(highlightRafId); highlightRafId = null; }
    URL.revokeObjectURL(audioUrl);
    setCurrentWordIndex(wordOffset + chunkWordCount - 1);
    const currentFile = previewFileRef.current;
    const chunksLen = ttsChunksRef.current.length;
    const curIdx = currentChunkIndexRef.current;
    console.log(`[TTS onended] Chunk ${curIdx + 1}/${chunksLen} ended. shouldContinue=${shouldContinueRef.current...}`);
    autoCheckChunk(curIdx);
    if (shouldContinueRef.current && curIdx < chunksLen - 1) {
      currentChunkIndexRef.current = curIdx + 1;
      setCurrentChunkIndex(curIdx + 1);
      if (currentFile) {
        saveTtsProgress(currentFile.id, curIdx + 1, 0);
      }
      const nextChunk = ttsChunksRef.current[curIdx + 1];
      if (nextChunk) {
        console.log(`[TTS onended] Starting chunk ${curIdx + 2}/${chunksLen}`);
        const blinkHighlight = () => {
          if (!isPlayingRef.current || openaiAudioRef.current) return;
          setCurrentWordIndex(prev => prev >= 0 ? prev : wordOffset + chunkWordCount - 1);
          setTimeout(() => {
            if (isPlayingRef.current && !openaiAudioRef.current) blinkHighlight();
          }, 500);
        };
        blinkHighlight();
        playWithOpenAiTts(nextChunk, voice, curIdx + 1);
      } else {
        console.warn(`[TTS onended] Next chunk ${curIdx + 2} is empty/undefined`);
        setIsPlaying(false);
        isPlayingRef.current = false;
      }
    } else {
      console.log(`[TTS onended] Stopping. shouldContinue=${shouldContinueRef.current}, atEnd=${curIdx >= chunksL...}`);
      setIsPlaying(false);
      isPlayingRef.current = false;
      if (currentFile) {
        saveTtsProgress(currentFile.id, curIdx, 0);
      }
    }
  };
};

audio.onerror = (e) => {
  if (highlightRafId) { cancelAnimationFrame(highlightRafId); highlightRafId = null; }
  console.error(`[TTS audio.onerror]', e);
  URL.revokeObjectURL(audioUrl);
  setIsPlaying(false);
  isPlayingRef.current = false;
  toast({ title: "Audio playback failed", variant: "destructive" });
};

isPlayingRef.current = true;
setIsPlaying(true);
setCurrentWordIndex(wordOffset);
try {
  await audio.play();
} catch (playError) {
  console.warn(`[TTS] audio.play() rejected, retrying with user gesture context...`, playError);
  await new Promise(r => setTimeout(r, 200));
  try {
    await audio.play();
  } catch (retryError) {
    console.error(`[TTS] audio.play() retry also failed', retryError);
    toast({ title: "Autoplay blocked. Tap play to continue.", variant: "destructive" });
    setIsPlaying(false);
    isPlayingRef.current = false;
    return;
  }
}

} catch (error) {
  console.error('OpenAI TTS error:', error);
  toast({ title: "Failed to generate speech", variant: "destructive" });
  setIsPlaying(false);
  isPlayingRef.current = false;
}
};

// Stop OpenAI TTS playback
const stopOpenAiTts = () => {
  if (openaiAudioRef.current) {
    openaiAudioRef.current.onended = null;
    openaiAudioRef.current.onerror = null;
    openaiAudioRef.current.ontimeupdate = null;
    openaiAudioRef.current.pause();
    openaiAudioRef.current = null;
  }
  shouldContinueRef.current = false;
  setIsPlaying(false);
  isPlayingRef.current = false;
};

// Save/load TTS progress for each file
const getTtsProgress = (fileId: number): { chunkIndex: number; wordIndex: number; charPosition?: number } | null => {
  try {
    const saved = localStorage.getItem(`tts-progress-${fileId}`);
    return saved ? JSON.parse(saved) : null;
  } catch { return null; }
};

const saveTtsProgress = (fileId: number, chunkIndex: number, wordIndex: number, charPosition?: number, overrideTotal?: boolean) => {

```

```

try {
  localStorage.setItem(`tts-progress-${fileId}`, JSON.stringify({ chunkIndex, wordIndex, charPosition: charPositi...
  const totalChunksVal = overrideTotalChunks || ttsChunksRef.current.length;
  console.log(`[saveTtsProgress] fileId=${fileId}, chunkIndex=${chunkIndex}, totalChunks=${totalChunksVal}`);
  if (totalChunksVal > 0) {
    const isFinished = chunkIndex + 1 >= totalChunksVal;
    apiRequest("PATCH", "/api/files/${fileId}", {
      lastChunkIndex: chunkIndex + 1,
      totalChunks: totalChunksVal,
      ...(isFinished ? { listened: true } : {}),
    }).then(() => {
      console.log(`[saveTtsProgress] PATCH success for fileId=${fileId}: lastChunkIndex=${chunkIndex + 1}, totalC...
      queryClient.invalidateQueries({ queryKey: ["/api/files"] });
      refreshFileCounts();
    }).catch((err) => {
      console.error(`[saveTtsProgress] PATCH failed for fileId=${fileId}:`, err);
    });
  } else {
    console.warn(`[saveTtsProgress] Skipped PATCH - totalChunks is 0 for fileId=${fileId}`);
  }
} catch (e) {
  console.error(`[saveTtsProgress] Error:`, e);
}
};

const clearTtsProgress = (fileId: number) => {
  try {
    localStorage.removeItem(`tts-progress-${fileId}`);
  } catch {}
};

// Load available TTS voices
useEffect(() => {
  if (typeof window === 'undefined' || !window.speechSynthesis) return;

  const loadVoices = () => {
    const voices = window.speechSynthesis.getVoices();
    if (voices.length > 0) {
      // Filter to English voices only, exclude French and other languages
      const englishVoices = voices.filter(v =>
        v.lang.startsWith('en') &&
        !v.name.toLowerCase().includes('french') &&
        !v.name.toLowerCase().includes('français') &&
        !v.lang.startsWith('fr')
      ).sort((a, b) => a.name.localeCompare(b.name));
      setAvailableVoices(englishVoices);
      // Set default voice - prefer Guy
      if (!selectedVoice) {
        const defaultVoice = englishVoices.find(v => v.name.includes('Guy'))
        || englishVoices.find(v => v.name.includes('Microsoft')) && v.name.includes('Natural'))
        || englishVoices[0];
        if (defaultVoice) setSelectedVoice(defaultVoice.name);
      }
    }
  };

  loadVoices();
  window.speechSynthesis.onvoiceschanged = loadVoices;

  return () => {
    if (window.speechSynthesis) {
      window.speechSynthesis.onvoiceschanged = null;
    }
  };
}, []);

const highlightIntervalRef = useRef<ReturnType<typeof setInterval> | null>(null);
const highlightTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
const isPlayingRef = useRef(false);

// PDF viewer state
const [numPages, setNumPages] = useState<number | null>(null);
const [currentPdfPage, setCurrentPdfPage] = useState(1);
const [pdfUrl, setPdfUrl] = useState<string | null>(null);
const [pageWordBoundaries, setPageWordBoundaries] = useState<number[][]>([]); // Word index where each page starts
const [pdfZoom, setPdfZoom] = useState(1.05); // Default 105% zoom
const [ttsWidth, setTtsWidth] = useState(450);
const ttsDragRef = useRef<{ startX: number; startWidth: number } | null>(null);
const pdfContainerRef = useRef<HTMLDivElement>(null);

// Calculate which PDF page a word index belongs to
const getPageForWordIndex = (wordIndex: number): number => {
  if (pageWordBoundaries.length === 0) return 1;
  for (let i = pageWordBoundaries.length - 1; i >= 0; i--) {
    if (wordIndex >= pageWordBoundaries[i]) {
      return i + 1;
    }
  }
  return 1;
};

// Calculate page boundaries when text is loaded
useEffect(() => {
  if (previewText) {
    // Split by the special page marker inserted by server during PDF extraction
    const PAGE_MARKER = '---PAGE---';
    const pages = previewText.split(PAGE_MARKER);
    const boundaries: number[] = [];
    let wordCount = 0;

    for (const page of pages) {
      boundaries.push(wordCount);
      // Count words excluding the page marker
      const words = page.split(/\s+/).filter(w => w.length > 0 && w !== PAGE_MARKER);
      wordCount += words.length;
    }

    setPageWordBoundaries(boundaries);
    console.log("Page boundaries calculated:", boundaries);
  }

```

```

    }
  }, [previewText]);

// Sync PDF page with current word during playback
useEffect(() => {
  if (isPlaying && syncHighlight && pageWordBoundaries.length > 0) {
    const newPage = getPageForWordIndex(currentWordIndex);
    if (newPage !== currentPdfPage && newPage >= 1 && newPage <= (numPages || 1)) {
      setCurrentPdfPage(newPage);
    }
  }
}, [currentWordIndex, isPlaying, syncHighlight, pageWordBoundaries, numPages]);

// Keep previewFileRef in sync with state (avoids stale closures in TTS callbacks)
// IMPORTANT: Only update when file is set (not null) – the cleanup effect needs the old ref
useEffect(() => {
  if (previewFile) {
    previewFileRef.current = previewFile;
  }
}, [previewFile]);

// Load PDF when file is selected
useEffect(() => {
  if (previewFile && previewFile.id) {
    setPdfUrl(null);

    // If the objectPath is a direct HTTP URL (OneDrive download URL), proxy it
    const objectPath = previewFile.objectPath || '';
    let fetchUrl: string;
    if (objectPath.startsWith('http')) {
      fetchUrl = `/api/proxy-pdf?url=${encodeURIComponent(objectPath)}`;
    } else {
      fetchUrl = `/api/files/${previewFile.id}/download`;
    }

    fetch(fetchUrl, { credentials: 'include' })
      .then(res => {
        if (!res.ok) throw new Error(`Download failed: ${res.status}`);
        return res.blob();
      })
      .then(blob => {
        const url = URL.createObjectURL(new Blob([blob], { type: 'application/pdf' }));
        setPdfUrl(url);
      })
      .catch(err => console.error('Error loading PDF:', err));
  }
  return () => {
    if (pdfUrl) {
      URL.revokeObjectURL(pdfUrl);
      setPdfUrl(null);
    }
  };
}, [previewFile?.id]);

// Function to filter out French text, links, and box content from content
const removeFrenchText = (text: string): string => {
  // First, remove URLs and links
  let cleanedText = text
    .replace(/https?:\/\/[^\s]+/g, '')
    .replace(/www\.[^\s]+/g, '')
    .replace(/[a-zA-Z0-9._%+~]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g, '') // Remove email addresses
    // Remove video references and box content
    .replace(/^\s+Video/gm, '') // Lines starting with "Video"
    .replace(/^\s+Audio/gm, '') // Lines starting with "Audio"
    .replace(/^\s+Link/gm, '') // Lines starting with "Link"
    .replace(/^\s+Watch/gm, '') // Lines starting with "Watch"
    .replace(/^\s+Listen/gm, '') // Lines starting with "Listen"
    .replace(/^\s+Click/gm, '') // Lines starting with "Click"
    .replace(/^\s+See/gm, '') // Lines starting with "See"
    .replace(/^\s+View/gm, '') // Lines starting with "View"
    .replace(/^\s+Author[s]?/gm, '') // Lines starting with "Author" or "Authors"
    .replace(/\(.*\)/g, '') // Remove bracketed content like [Video], [Link], etc.
    .replace(/\(.*?video.*?\)/gi, '') // Remove parenthetical video references
    .replace(/\(.*?link.*?\)/gi, '') // Remove parenthetical link references
    .replace(/\d+:\d+:\d+/g, '') // Remove timestamps like 1:23:45
    .replace(/\d+:\d+/g, '') // Remove timestamps like 1:23
    .replace(/\n\s*\n\s*\n/g, '\n\n') // Clean up extra blank lines
    .replace(/\btabs/gi, '');

  cleanedText = cleanedText.replace(/Copyright\s+\d{4}\s+.*?All Rights Reserved\.?.*?(?=\.\s|$)/gi, '');

  const paragraphs = cleanedText.split(/\n\n+/);
  const filteredParagraphs = paragraphs.filter(para => {
    const words = para.split(/\s+/).length;
    if (words > 60) return true;
    const lowerPara = para.toLowerCase();
    if (lowerPara.includes('jstor') ||
        lowerPara.includes('stable url') ||
        lowerPara.includes('accessed:') ||
        lowerPara.includes('published by:')) {
      //author[s]?/i.test(para.trim()) {
      return false;
    }
  });
  cleanedText = filteredParagraphs.join('\n\n');

  const frenchStructure = /\b(l'|d'|n'|s'|j'|c'|qu'|m'|t')w+/gi;
  const frenchVerbs = /\b(est|sont|était|étaient|être|avoir|été|fait|peut|peuvent|doit|doivent|avait|avaient|sera|s.../gi;
  const frenchPronouns = /\b(je|tu|il|elle|nous|vous|ils|elles|ce|cette|ces|cet)\b/gi;
  const frenchPreps = /\b(dans|avec|pour|sans|sur|sous|chez|vers|entre|depuis|après|avant|donc|mais|dont|où)\b/gi;

  const isFullSentenceFrench = (sentence: string): boolean => {
    const words = sentence.split(/\s+/).filter(w => w.length > 0);
    if (words.length < 5) return false;
    const structMatches = sentence.match(frenchStructure);
    if (!structMatches || structMatches.length < 2) return false;
    const verbMatches = sentence.match(frenchVerbs);

```

```

const pronounMatches = sentence.match(frenchPronouns);
const prepMatches = sentence.match(frenchPreps);
const totalFrench = (structMatches?.length || 0) + (verbMatches?.length || 0) + (pronounMatches?.length || 0) + ...
return (totalFrench / words.length) > 0.4;
};

const allSentences = cleanedText.split(/(?<[!?])\s+/);
const hasFrench = allSentences.some(s => isFullSentenceFrench(s));
if (!hasFrench) return cleanedText;

const paragraphs2 = cleanedText.split(/\n\n+/);
const result = paragraphs2.map(para => {
  const sentences = para.split(/(?<[!?])\s+/);
  return sentences.filter(s => !isFullSentenceFrench(s)).join(' ');
}).filter(p => p.trim().length > 0);

return result.join('\n\n');
};

const joinHyphenatedWords = (text: string): string => {
  const knownJoins: Record<string, string> = {
    "to-day": "today", "to-morrow": "tomorrow", "to-night": "tonight",
    "to-gether": "together", "to-ward": "toward", "to-wards": "towards",
    "al-ready": "already", "al-though": "although", "al-most": "almost",
    "al-ways": "always", "al-so": "also", "al-together": "altogether",
    "any-thing": "anything", "any-one": "anyone", "any-body": "anybody",
    "any-where": "anywhere", "any-way": "anyway",
    "every-thing": "everything", "every-one": "everyone", "every-body": "everybody",
    "every-where": "everywhere", "every-day": "everyday",
    "some-thing": "something", "some-one": "someone", "some-body": "somebody",
    "some-where": "somewhere", "some-how": "somehow", "some-what": "somewhat",
    "some-times": "sometimes", "some-time": "sometime",
    "no-thing": "nothing", "no-body": "nobody", "no-where": "nowhere",
    "mean-while": "meanwhile", "mean-ing": "meaning",
    "how-ever": "however", "what-ever": "whatever", "when-ever": "whenever",
    "where-ever": "wherever", "who-ever": "whoever",
    "my-self": "myself", "him-self": "himself", "her-self": "herself",
    "it-self": "itself", "them-selves": "themselves", "your-self": "yourself",
    "our-selves": "ourselves", "your-selves": "yourselves",
    "with-out": "without", "with-in": "within", "with-stand": "withstand",
    "there-fore": "therefore", "there-by": "thereby", "there-in": "therein",
    "there-after": "thereafter", "there-of": "thereof",
    "be-cause": "because", "be-come": "become", "be-fore": "before",
    "be-hind": "behind", "be-long": "belong", "be-tween": "between",
    "be-yond": "beyond", "be-lieve": "believe", "be-gin": "begin",
    "be-neath": "beneath", "be-side": "beside", "be-sides": "besides",
    "over-come": "overcome", "over-look": "overlook", "over-whelm": "overwhelm",
    "under-stand": "understand", "under-stood": "understood",
    "under-neath": "underneath", "under-go": "undergo",
    "up-on": "upon", "up-stairs": "upstairs", "up-ward": "upward",
    "down-stairs": "downstairs", "down-ward": "downward",
    "out-side": "outside", "out-ward": "outward",
    "in-to": "into", "in-deed": "indeed", "in-stead": "instead",
    "in-side": "inside",
    "for-ever": "forever", "for-ward": "forward",
    "per-haps": "perhaps",
    "break-fast": "breakfast", "rail-way": "railway", "news-paper": "newspaper",
    "bed-room": "bedroom", "class-room": "classroom", "bath-room": "bathroom",
    "play-ground": "playground", "land-lord": "landlord", "land-lady": "landlady",
    "gentle-man": "gentleman", "gentle-men": "gentlemen",
    "church-yard": "churchyard", "court-yard": "courtyard",
    "fire-place": "fireplace", "door-way": "doorway",
    "sun-light": "sunlight", "moon-light": "moonlight", "day-light": "daylight",
    "mid-night": "midnight", "after-noon": "afternoon",
    "birth-day": "birthday",
  };
  let result = text;
  result = result.replace(/(\w+)\-\\s*\n\\s*(\w+)/g, '$1$2');
  result = result.replace(/(\w+)\-\\s*(\w+)/g, (match, p1, p2) => {
    const joined = p1 + p2;
    if (/^[a-z]+$/.test(joined) && joined.length > 3) {
      return joined;
    }
    return match;
  });
  for (const [hyphenated, joined] of Object.entries(knownJoins)) {
    const regex = new RegExp(hyphenated.replace('-', '\\s-'), 'gi');
    result = result.replace(regex, (match) => {
      const isUpper = match[0] === match[0].toUpperCase();
      return isUpper ? joined[0].toUpperCase() + joined.slice(1) : joined;
    });
  }
  return result;
};

useEffect(() => {
  if (!isPlaying) return;
  const pollVolume = async () => {
    try {
      const resp = await fetch("/api/cat-wash/progress");
      const data = await resp.json();
      if (data.volumeChange && data.volumeChange.timestamp > lastVolumeTimestampRef.current) {
        lastVolumeTimestampRef.current = data.volumeChange.timestamp;
        setVolumeOverlay({ volume: data.volumeChange.volume, direction: data.volumeChange.direction });
        if (volumeOverlayTimerRef.current) clearTimeout(volumeOverlayTimerRef.current);
        volumeOverlayTimerRef.current = setTimeout(() => setVolumeOverlay(null), 2500);
      }
    } catch {}
  };
  const interval = setInterval(pollVolume, 800);
  return () => clearInterval(interval);
}, [isPlaying]);

// Fetch text when file is selected for preview
useEffect(() => {
  let cancelled = false;
  if (previewFile) {
    console.log(`[TextFetch] previewFile changed: id=${previewFile.id}, name=${previewFile.originalName}, objectPat...`);
  }
});

```

```

setIsLoadingText(true);
setPreviewText("");
setCurrentWordIndex(0);
setIsPlaying(false);
isPlayingRef.current = false;
setPreviewSpeaker("browser_tts");
previewSpeakerRef.current = "browser_tts";
setDeletedWordIndices(new Set());
setDeletedWordHistory([]);
setShowRemoveButton(false);
if (window.speechSynthesis) {
  window.speechSynthesis.cancel();
}
if (openaiAudioRef.current) {
  openaiAudioRef.current.pause();
  openaiAudioRef.current = null;
}
ttsChunksRef.current = [];
setTtsChunks([]);
setTotalChunks(0);
setCurrentChunkIndex(0);
currentChunkIndexRef.current = 0;
shouldContinueRef.current = false;

const isDirectUrl = previewFile.objectPath?.startsWith('http');
const fileId = previewFile.id;
console.log(`[TextFetch] isDirectUrl=${isDirectUrl}, fetching text for file ${fileId}`);

if (isDirectUrl) {
  fetch('/api/extract-text-from-url', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ url: previewFile.objectPath })
  })
  .then(res => res.json())
  .then(data => {
    if (cancelled) return;
    console.log(`[TextFetch] URL response for ${fileId}: hasText=${!!data.text}, length=${data.text?.length} ...`);
    if (data.text) {
      const filteredText = joinHyphenatedWords(removeFrenchText(data.text));
      setPreviewText(filteredText);
    }
  })
  .catch(err => console.error("Error fetching text from URL:", err))
  .finally(() => { if (!cancelled) setIsLoadingText(false); });
} else {
  fetch(`/api/files/${fileId}/text`)
  .then(res => {
    console.log(`[TextFetch] API response status for file ${fileId}: ${res.status}`);
    return res.json();
  })
  .then(data => {
    if (cancelled) return;
    console.log(`[TextFetch] API response for file ${fileId}: hasText=${!!data.text}, length=${data.text?.len...}`);
    if (data.text) {
      const filteredText = joinHyphenatedWords(removeFrenchText(data.text));
      console.log(`[TextFetch] Setting previewText for file ${fileId}: filteredLength=${filteredText.length}`);
      setPreviewText(filteredText);
    }
  })
  .catch(err => console.error(`[TextFetch] Error fetching text for file ${fileId}:`, err))
  .finally(() => { if (!cancelled) setIsLoadingText(false); });
}
} else {
  // Save progress BEFORE clearing anything – capture current state
  const closingFile = previewFileRef.current;
  const closingChunkIndex = currentChunkIndexRef.current;
  const closingTotalChunks = ttsChunksRef.current.length;

  // Stop any playing TTS immediately
  shouldContinueRef.current = false;
  if (window.speechSynthesis) {
    window.speechSynthesis.cancel();
  }
  if (openaiAudioRef.current) {
    openaiAudioRef.current.pause();
    openaiAudioRef.current = null;
  }

  // Save progress to DB for the file that was playing
  // Pass closingTotalChunks directly since ttsChunksRef may get cleared
  if (closingFile && closingTotalChunks > 0 && closingChunkIndex > 0) {
    let charPosition = 0;
    const closingChunks = ttsChunksRef.current;
    for (let i = 0; i < closingChunkIndex; i++) {
      charPosition += closingChunks[i]?.length || 0;
    }
    saveTtsProgress(closingFile.id, closingChunkIndex, currentWordIndex, charPosition, closingTotalChunks);
  }

  // Now clear everything AFTER saving
  previewFileRef.current = null;
  setPreviewText("");
  setCurrentWordIndex(0);
  setIsPlaying(false);
  isPlayingRef.current = false;
  ttsChunksRef.current = [];
  setTtsChunks([]);
  setTotalChunks(0);
  setCurrentChunkIndex(0);
  currentChunkIndexRef.current = 0;
  if (highlightIntervalRef.current) {
    clearInterval(highlightIntervalRef.current);
    highlightIntervalRef.current = null;
  }
}
return () => { cancelled = true; };
}, [previewFile]);

```

```

// Initialize TTS chunks when previewText changes - ensures display and playback use same chunks
useEffect(() => {
  if (!previewText) {
    ttsChunksRef.current = [];
    setTtsChunks([]);
    setTotalChunks(0);
    return;
  }

  // Apply the same processing as TTS playback to ensure consistency
  let textForTts = previewText;
  const titlePageKeywords = /jstor|published|publisher|author[s]?|doi:|copyright|©|issn|isbn|volume\s+\d|issue\s+\.\.\.

  // Skip title pages
  const firstPageEnd = textForTts.indexOf('---PAGE---');
  if (firstPageEnd !== -1) {
    const firstPageContent = textForTts.substring(0, firstPageEnd).toLowerCase();
    const wordCount = firstPageContent.split(/\s+/).filter(w => w.length > 0).length;
    if (wordCount < 300 && titlePageKeywords.test(firstPageContent)) {
      textForTts = textForTts.substring(firstPageEnd + 10);
    }
  } else {
    const first500Words = textForTts.split(/\s+/).slice(0, 500).join(' ').toLowerCase();
    if (titlePageKeywords.test(first500Words)) {
      const skipTo = textForTts.search(/\n\n[A-Z]/);
      if (skipTo > 100 && skipTo < 2000) {
        textForTts = textForTts.substring(skipTo + 2);
      }
    }
  }
}

// Remove page separators and apply filters
let cleanTextForTts = textForTts.replace(/---PAGE---/g, '');
cleanTextForTts = removeFrenchText(cleanTextForTts);
cleanTextForTts = cleanTextForTts.replace(/[\t]+/g, ' ');
cleanTextForTts = cleanTextForTts.replace(/^[^\s]*%[!'.]\s*/gm, '');
cleanTextForTts = cleanTextForTts.replace(/^(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Foreword|A...
cleanTextForTts = cleanTextForTts.replace(/^(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Foreword|A...
cleanTextForTts = cleanTextForTts.replace(/([a-z,;])\s*\n\s*([a-z])/gi, '$1 $2');
cleanTextForTts = cleanTextForTts.replace(/\n{3,}/g, '\n\n');
cleanTextForTts = cleanTextForTts.replace(/([^\s])*(?:(?:\d{4}[a-z]?|pp?\.\s*\d|[A-Z][a-z]+,\s*\d{4})|\s*\d{4})[ivx]ld...
cleanTextForTts = cleanTextForTts.replace(/([0-9a-zA-Z.,;\s]+)\s+/g, (match, inner) => {
  if (/^\d.,;\s]+$/.test(inner.trim())) return '';
  if (/^[a-zA-Z]([.,;\s]+[a-zA-Z])*[.,;\s]+$/.test(inner.trim())) return '';
  if (/^\d.,;\sa-zA-Z]+$/.test(inner.trim()) && inner.trim().length < 20) return '';
  return match;
});
cleanTextForTts = cleanTextForTts.replace(/\s{2,}/g, ' ');
cleanTextForTts = cleanTextForTts.replace(/([^\s!?\n])$/gm, '$1. ');
cleanTextForTts = cleanTextForTts.replace(/\n\n+/g, '\n\n');
cleanTextForTts = cleanTextForTts.replace(/.\{2,}/g, '. ');
cleanTextForTts = cleanTextForTts.replace(/(\.\s+)(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Fore...
cleanTextForTts = cleanTextForTts.replace(/^(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Foreword|A...

// Use 2000 char chunks for display consistency
const chunks = splitTextIntoChunks(cleanTextForTts, 2000);
ttsChunksRef.current = chunks;
setTtsChunks(chunks);
setTotalChunks(chunks.length);
console.log("[TTS-CHUNKS] Created", chunks.length, "chunks. Chunk 0 first 300 chars:", chunks[0]?.substring(0, 30...
console.log("[TTS-CHUNKS] cleanTextForTts first 500 chars:", cleanTextForTts.substring(0, 500));
console.log("[TTS-CHUNKS] textForTts first 500 chars:", textForTts.substring(0, 500));

// Pre-fetch the first chunk's audio for OpenAI TTS so playback starts instantly
if (chunks.length > 0 && (previewSpeaker === "openai_tts" || !window.speechSynthesis)) {
  // Determine which chunk we'll actually start from (first unchecked)
  let startIdx = 0;
  if (previewFile?.checkedChunks) {
    try {
      const checkedSet = new Set<number>(JSON.parse(previewFile.checkedChunks));
      for (let i = 0; i < chunks.length; i++) {
        if (!checkedSet.has(i)) { startIdx = i; break; }
        if (i === chunks.length - 1) startIdx = 0;
      }
    } catch { /* ignore */ }
  }
  prefetchNextChunk(startIdx, openaiVoice);
}

// Pre-warm browser voices cache
if (window.speechSynthesis && !cachedVoicesRef.current) {
  waitForVoices();
}
}, [previewText]);

const getDashFileKey = () => {
  if (!previewFile) return '';
  return `file_${previewFile.id}`;
};

const loadDashCheckedChunks = (): Set<number> => {
  if (previewFile?.checkedChunks) {
    try {
      return new Set<number>(JSON.parse(previewFile.checkedChunks));
    } catch { /* fall through */ }
  }
  return new Set();
};

const saveDashCheckedChunks = (checked: Set<number>, total: number) => {
  if (!previewFile) return;
  const checkedArr = Array.from(checked);
  const checkedJson = JSON.stringify(checkedArr);
  previewFile.checkedChunks = checkedJson;
  previewFile.totalChunks = total;
  fetch(`/api/files/${previewFile.id}`, {
    method: 'PATCH',

```



```

    headers: { 'Content-Type': 'application/json' },
    credentials: 'include',
    body: JSON.stringify({ checkedChunks: checkedJson, totalChunks: total }),
  }).then(() => {
    queryClient.invalidateQueries({ queryKey: ['/api/files'] });
  }).catch(err => console.error('Failed to save checked chunks:', err));
};

const toggleDashChunkChecked = (idx: number) => {
  if (!previewFile) return;
  const newChecked = new Set(checkedChunks);
  if (newChecked.has(idx)) {
    newChecked.delete(idx);
  } else {
    for (let i = 0; i <= idx; i++) {
      newChecked.add(i);
    }
  }
  setCheckedChunks(newChecked);
  checkedChunksRef.current = newChecked;
  saveDashCheckedChunks(newChecked, totalChunks);
};

const autoCheckChunk = (chunkIdx: number) => {
  const file = previewFileRef.current;
  if (!file) return;
  const current = new Set(checkedChunksRef.current);
  if (current.has(chunkIdx)) return;
  for (let i = 0; i <= chunkIdx; i++) {
    current.add(i);
  }
  const total = ttsChunksRef.current.length || totalChunks;
  setCheckedChunks(new Set(current));
  checkedChunksRef.current = current;
  const checkedJson = JSON.stringify(Array.from(current));
  file.checkedChunks = checkedJson;
  fetch(`/api/files/${file.id}`, {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include',
    body: JSON.stringify({ checkedChunks: checkedJson, totalChunks: total }),
  }).then(() => {
    queryClient.invalidateQueries({ queryKey: ['/api/files'] });
    refreshFileCounts();
  }).catch(err => console.error('Failed to save auto-checked chunks:', err));
};

const prevPreviewFileIdRef = useRef<number | null>(null);
useEffect(() => {
  const currentFileId = previewFile?.id ?? null;
  if (currentFileId !== prevPreviewFileIdRef.current) {
    setCheckedChunks(new Set());
    checkedChunksRef.current = new Set();
    prevPreviewFileIdRef.current = currentFileId;
  }
  if (previewFile && totalChunks > 0) {
    const scrollToFirstUnlisted = (checked: Set<number>) => {
      const attemptScroll = (attempt: number) => {
        if (attempt > 10) return;
        setTimeout(() => {
          const allChunkEls = document.querySelectorAll('[data-testid="checkbox-chunk-"]');
          if (allChunkEls.length === 0) {
            attemptScroll(attempt + 1);
            return;
          }
          for (let i = 0; i < allChunkEls.length; i++) {
            const testId = allChunkEls[i].getAttribute('data-testid') || '';
            const idx = parseInt(testId.replace('checkbox-chunk-', ''), 10);
            if (!isNaN(idx) && !checked.has(idx)) {
              allChunkEls[i]?.scrollIntoView({ behavior: 'smooth', block: 'center' });
              return;
            }
          }
        }, attempt === 0 ? 800 : 400);
      };
      attemptScroll(0);
    };
    fetch(`/api/files/${previewFile.id}`, { credentials: 'include' })
      .then(res => res.json())
      .then((freshFile: any) => {
        if (freshFile && freshFile.checkedChunks) {
          try {
            const loaded = new Set<number>(JSON.parse(freshFile.checkedChunks));
            setCheckedChunks(loaded);
            checkedChunksRef.current = loaded;
            previewFile.checkedChunks = freshFile.checkedChunks;
            previewFile.totalChunks = freshFile.totalChunks;
            scrollToFirstUnlisted(loaded);
            return;
          } catch { /* fall through */ }
          setCheckedChunks(new Set());
          checkedChunksRef.current = new Set();
          scrollToFirstUnlisted(new Set());
        }
      })
      .catch(() => {
        setCheckedChunks(new Set());
        checkedChunksRef.current = new Set();
        scrollToFirstUnlisted(new Set());
      });
  } else if (!previewFile) {
    setCheckedChunks(new Set());
    checkedChunksRef.current = new Set();
  }
}, [previewFile?.id, totalChunks]);

// Cleanup interval and timeout on unmount
useEffect(() => {

```

```

    return () => {
      if (highlightIntervalRef.current) {
        clearInterval(highlightIntervalRef.current);
      }
      if (highlightTimeoutRef.current) {
        clearTimeout(highlightTimeoutRef.current);
      }
    };
  }, []);

// Word highlighting animation with smart timing based on word length
// Uses ttsSettings for configurable delay and speech rate

// Calculate word duration based on length (longer words take longer to say)
const getWordDuration = (word: string, baseMs: number): number => {
  if (!ttsSettings.useSmartTiming) return baseMs;

  // Count syllables approximately (vowel groups)
  const syllables = word.toLowerCase().replace(/^[^aeiouy]/g, '').replace(/[aeiouy]+/g, 'x').length || 1;
  // Longer words with more syllables take longer
  // Average English word has ~1.5 syllables, so adjust based on that
  const syllableFactor = Math.max(0.6, Math.min(2.0, syllables / 1.5));

  // Also consider word length (numbers, punctuation affect timing)
  const lengthFactor = Math.max(0.7, Math.min(1.5, word.length / 5));

  // Combine factors with base timing
  return Math.round(baseMs * (syllableFactor * 0.7 + lengthFactor * 0.3));
};

const startHighlighting = () => {
  const chunkText = ttsChunksRef.current.length > 0 ? ttsChunksRef.current.join(' ') : previewText;
  const words = chunkText.split(/\s+/).filter(w => w.length > 0 && w !== '---PAGE---');
  if (words.length === 0) return;

  setPlayStartTime(Date.now());
  setIsPlaying(true);
  isPlayingRef.current = true;
  setCurrentWordIndex(-1); // Start at -1 to wait for delay

  if (highlightIntervalRef.current) {
    clearInterval(highlightIntervalRef.current);
  }
  if (highlightTimeoutRef.current) {
    clearTimeout(highlightTimeoutRef.current);
  }

  // Calculate base milliseconds per word from WPM setting
  const baseMs = 60000 / ttsSettings.wordsPerMinute;
  const startDelayMs = ttsSettings.startDelay * 1000;

  // Wait for TTS to actually start speaking before highlighting
  highlightTimeoutRef.current = setTimeout(() => {
    if (!isPlayingRef.current) return; // Stopped during delay
    setCurrentWordIndex(0);

    // Use recursive setTimeout instead of setInterval for variable timing
    const scheduleNextWord = (currentIdx: number) => {
      if (!isPlayingRef.current) return;
      if (currentIdx >= words.length - 1) {
        setIsPlaying(false);
        isPlayingRef.current = false;
        return;
      }
    }

    const currentWord = words[currentIdx];
    const duration = getWordDuration(currentWord, baseMs);

    highlightTimeoutRef.current = setTimeout(() => {
      if (!isPlayingRef.current) return;
      const nextIdx = currentIdx + 1;
      setCurrentWordIndex(nextIdx);
      scheduleNextWord(nextIdx);
    }, duration);
  }, startDelayMs);

  scheduleNextWord(0);
}, startDelayMs);
};

const stopHighlighting = () => {
  setIsPlaying(false);
  isPlayingRef.current = false;
  if (highlightIntervalRef.current) {
    clearInterval(highlightIntervalRef.current);
    highlightIntervalRef.current = null;
  }
  if (highlightTimeoutRef.current) {
    clearTimeout(highlightTimeoutRef.current);
    highlightTimeoutRef.current = null;
  }
};

// Media control functions for file preview
// Browser TTS ref
const speechUtteranceRef = useRef<SpeechSynthesisUtterance | null>(null);
const [browserTtsRate, setBrowserTtsRate] = useState(0.9); // 90% speed
const [browserTtsVolume, setBrowserTtsVolume] = useState(1.0); // 100% volume
const browserTtsRateRef = useRef(0.9);
const browserTtsVolumeRef = useRef(1.0);
const ttsCharIndexRef = useRef(0);
useEffect(() => { browserTtsRateRef.current = browserTtsRate; }, [browserTtsRate]);
useEffect(() => { browserTtsVolumeRef.current = browserTtsVolume; }, [browserTtsVolume]);

// Helper to wait for voices to be loaded (Chrome/Android fix)
const cachedVoicesRef = useRef<SpeechSynthesisVoice[] | null>(null);
const waitForVoices = (): Promise<SpeechSynthesisVoice[]> => {
  return new Promise((resolve) => {

```

```

    if (!window.speechSynthesis) {
      resolve([]);
      return;
    }
    if (cachedVoicesRef.current && cachedVoicesRef.current.length > 0) {
      resolve(cachedVoicesRef.current);
      return;
    }
    const voices = window.speechSynthesis.getVoices();
    if (voices && voices.length > 0) {
      cachedVoicesRef.current = voices;
      resolve(voices);
      return;
    }
    const handleVoicesChanged = () => {
      const loadedVoices = window.speechSynthesis.getVoices();
      cachedVoicesRef.current = loadedVoices;
      resolve(loadedVoices || []);
    };
    window.speechSynthesis.addEventListener('voiceschanged', handleVoicesChanged, { once: true });
    let attempts = 0;
    const tryGetVoices = () => {
      attempts++;
      const fallbackVoices = window.speechSynthesis.getVoices() || [];
      if (fallbackVoices.length > 0 || attempts >= 5) {
        cachedVoicesRef.current = fallbackVoices;
        resolve(fallbackVoices);
      } else {
        setTimeout(tryGetVoices, 300);
      }
    };
    setTimeout(tryGetVoices, 300);
  });
};

useEffect(() => {
  fetch("/api/flick/rooms").then(r => r.json()).then(setFlickDeviceGroups).catch(() => {});
}, []);

const handleFlick = async (deviceId: string) => {
  if (!previewFile) return;
  setIsFlicking(true);
  try {
    if (isPlaying && currentChunkRef.current > 0) {
      await fetch(`/api/files/${previewFile.id}`, {
        method: 'PATCH',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ lastChunkIndex: currentChunkRef.current, totalChunks: ttsChunksRef.current.length })
      });
    }
    handleStopMedia();

    const resp = await fetch("/api/flick", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        deviceId,
        fileId: previewFile.id,
        currentChunkIndex: currentChunkRef.current,
        totalChunks: ttsChunksRef.current.length
      })
    });
    const data = await resp.json();
    if (data.success) {
      toast({
        title: `Flicked to ${data.device}`,
        description: data.canDisplay
          ? `Opening on ${data.device} in ${data.room}`
          : `Playing on ${data.device} in ${data.room}`
      });
    } else {
      toast({ title: "Flick failed", description: data.error || "Unknown error" });
    }
  } catch (e: any) {
    toast({ title: "Flick failed", description: e.message });
  } finally {
    setIsFlicking(false);
    setShowFlickMenu(false);
  }
};

// Split text into chunks at sentence boundaries for reliable TTS
const splitTextIntoChunks = (text: string, maxChunkSize: number = 2000): string[] => {
  const chunks: string[] = [];
  let remaining = text;

  while (remaining.length > 0) {
    if (remaining.length <= maxChunkSize) {
      chunks.push(remaining);
      break;
    }

    // Find a good break point (end of sentence)
    let breakPoint = remaining.lastIndexOf('.', maxChunkSize);
    if (breakPoint === -1 || breakPoint < maxChunkSize / 2) {
      breakPoint = remaining.lastIndexOf('?', maxChunkSize);
    }
    if (breakPoint === -1 || breakPoint < maxChunkSize / 2) {
      breakPoint = remaining.lastIndexOf('!', maxChunkSize);
    }
    if (breakPoint === -1 || breakPoint < maxChunkSize / 2) {
      breakPoint = remaining.lastIndexOf('\n', maxChunkSize);
    }
    if (breakPoint === -1 || breakPoint < maxChunkSize / 2) {
      breakPoint = remaining.lastIndexOf(' ', maxChunkSize);
    }
    if (breakPoint === -1) {
      breakPoint = maxChunkSize;
    }
  }

```

```

    }

    chunks.push(remaining.substring(0, breakPoint + 1).trim());
    remaining = remaining.substring(breakPoint + 1).trim();
}

return chunks.filter(c => c.length > 0);
};

// Speak a single chunk and continue to next
const speakChunk = async (chunkIndex: number, chunks: string[], voices: SpeechSynthesisVoice[], wordOffset: number ...)
const currentFile = previewFileRef.current;
if (chunkIndex >= chunks.length || !shouldContinueRef.current) {
    setIsPlaying(false);
    isPlayingRef.current = false;
    if (currentFile) {
        clearTtsProgress(currentFile.id);
        toast({ title: "Finished reading file" });
    }
    return;
}

let chunk = chunks[chunkIndex];
chunk = chunk.replace(/^(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Foreword|Acknowledgements?|Ref...)
chunk = chunk.replace(/(\\.\s+)(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Foreword|Acknowledgement...)
const utterance = new SpeechSynthesisUtterance(chunk);
utterance.rate = browserTtsRateRef.current;
utterance.volume = browserTtsVolumeRef.current;
utterance.pitch = 1;

const voice = selectedVoice
    ? voices.find(v => v.name === selectedVoice)
    : voices.find(v => v.name.includes('Google') && v.lang.startsWith('en'))
    || voices.find(v => v.name.includes('Microsoft') && v.name.includes('Natural'))
    || voices.find(v => v.lang.startsWith('en'))
    || voices[0];
if (voice) {
    utterance.voice = voice;
}

let localWordIndex = 0;
const chunkWordCount = chunk.split(/\\s+/).length;

utterance.onboundary = (event) => {
    if (event.name === 'word') {
        ttsCharIndexRef.current = event.charIndex;
        setCurrentWordIndex(wordOffset + localWordIndex);
        localWordIndex++;
        const fileForSave = previewFileRef.current;
        if (fileForSave && localWordIndex % 10 === 0) {
            let charPosition = 0;
            for (let i = 0; i < chunkIndex; i++) {
                charPosition += chunks[i].length;
            }
            saveTtsProgress(fileForSave.id, chunkIndex, localWordIndex, charPosition);
        }
    }
};

utterance.onstart = () => {
    console.log(`Chunk ${chunkIndex + 1}/${chunks.length} started`);
    ttsCharIndexRef.current = 0;
    setCurrentChunkIndex(chunkIndex);
    currentChunkIndexRef.current = chunkIndex;
};

utterance.onend = () => {
    console.log(`Chunk ${chunkIndex + 1}/${chunks.length} ended`);
    autoCheckChunk(chunkIndex);
    if (shouldContinueRef.current) {
        setTimeout(() => {
            speakChunk(chunkIndex + 1, chunks, voices, wordOffset + chunkWordCount);
        }, 100);
    }
};

utterance.onerror = (event) => {
    console.error("Speech error:", event.error);
    if (event.error !== 'interrupted') {
        toast({ title: "Speech paused at chunk ${chunkIndex + 1}. Tap play to resume.", variant: "default" });
        const fileForSave = previewFileRef.current;
        if (fileForSave) {
            let charPosition = 0;
            for (let i = 0; i < chunkIndex; i++) {
                charPosition += chunks[i].length;
            }
            saveTtsProgress(fileForSave.id, chunkIndex, localWordIndex, charPosition);
        }
    }
    setIsPlaying(false);
    isPlayingRef.current = false;
};

speechUtteranceRef.current = utterance;
window.speechSynthesis.speak(utterance);
};

// Play from a specific chunk index
const handleTtsTextMouseDown = () => {
    setShowRemoveButton(false);
};

const handleTtsTextSelection = (e: React.MouseEvent) => {
    setTimeout(() => {
        const selection = window.getSelection();
        if (!selection || selection.isCollapsed || !selection.toString().trim()) {
            setShowRemoveButton(false);
            return;
        }
    });
};

```

```

    }
    const container = ttsTextContainerRef.current;
    if (!container || !container.contains(selection.anchorNode) || !container.contains(selection.focusNode)) {
      setShowRemoveButton(false);
      return;
    }
    const range = selection.getRangeAt(0);
    const rect = range.getBoundingClientRect();
    const containerRect = container.getBoundingClientRect();
    setShowRemoveButtonPos({
      x: rect.left + rect.width / 2 - containerRect.left,
      y: rect.top - containerRect.top - 32,
    });
    setShowRemoveButton(true);
  }, 10);
};

const handleRemoveSelectedText = () => {
  const selection = window.getSelection();
  if (!selection || selection.isCollapsed) return;
  const container = ttsTextContainerRef.current;
  if (!container) return;

  const wordSpans = container.querySelectorAll('span[data-word-idx]');
  const indicesToDelete = new Set<number>();

  for (const span of wordSpans) {
    if (selection.containsNode(span, true)) {
      const idx = parseInt(span.getAttribute('data-word-idx') || '-1');
      if (idx >= 0) indicesToDelete.add(idx);
    }
  }

  if (indicesToDelete.size > 0) {
    setDeletedWordHistory(prev => [...prev, new Set(deletedWordIndices)]);
    setDeletedWordIndices(prev => {
      const next = new Set(prev);
      indicesToDelete.forEach(i => next.add(i));
      return next;
    });
  }

  selection.removeAllRanges();
  setShowRemoveButton(false);
};

const handleUndoDelete = () => {
  if (deletedWordHistory.length > 0) {
    const prev = deletedWordHistory[deletedWordHistory.length - 1];
    setDeletedWordIndices(prev);
    setDeletedWordHistory(h => h.slice(0, -1));
  }
};

const filterDeletedWordsFromChunks = (chunks: string[], deleted: Set<number>): string[] => {
  if (deleted.size === 0) return chunks;
  let globalIdx = 0;
  return chunks.map(chunk => {
    const words = chunk.split(/\s+/);
    const filtered = words.filter((_, wIdx) => {
      const keep = !deleted.has(globalIdx);
      globalIdx++;
      return keep;
    });
    return filtered.join(' ');
  }).filter(c => c.trim().length > 0);
};

const playFromChunk = async (chunkIndex: number) => {
  const useBrowserTts = !!window.speechSynthesis;

  // Auto-switch to appropriate TTS mode
  if (useBrowserTts && previewSpeaker !== "browser_tts") {
    setPreviewSpeaker("browser_tts");
    previewSpeakerRef.current = "browser_tts";
  }

  // Make sure we have chunks
  if (ttsChunksRef.current.length === 0) {
    // Need to split the text first
    if (!previewText) {
      toast({ title: "No text content available", variant: "destructive" });
      return;
    }
    // Detect and skip title pages (short pages with publication info like JSTOR, author, published, etc.)
    let textForTts = previewText;
    const titlePageKeywords = /jstor|published|publisher|author[s]?|doi:|copyright|issn|isbn|volume\s+\d|issue\s...

    const firstPageEnd = textForTts.indexOf('---PAGE---');
    if (firstPageEnd !== -1) {
      const firstPageContent = textForTts.substring(0, firstPageEnd).toLowerCase();
      const wordCount = firstPageContent.split(/\s+/).filter(w => w.length > 0).length;
      // Check if first page looks like a title page (short + contains publication keywords)
      if (wordCount < 300 && titlePageKeywords.test(firstPageContent)) {
        textForTts = textForTts.substring(firstPageEnd + 10); // Skip past first ---PAGE---
        console.log("Skipped title page (via PAGE marker)");
      }
    } else {
      // No page markers - check beginning of text for title page patterns
      const first500Words = textForTts.split(/\s+/).slice(0, 500).join(' ').toLowerCase();
      if (titlePageKeywords.test(first500Words)) {
        // Find first paragraph break after initial content
        const skipTo = textForTts.search(/\n\n[A-Z]/);
        if (skipTo > 100 && skipTo < 2000) {
          textForTts = textForTts.substring(skipTo + 2);
          console.log("Skipped title content (no page markers)");
        }
      }
    }
  }
}

```

```

}
// First remove page separators
let cleanTextForTts = textForTts.replace(/---PAGE---/g, '');
// Apply the same filters used for display - removes French, URLs, timestamps, video/audio refs
cleanTextForTts = removeFrenchText(cleanTextForTts);
// Normalize whitespace and line breaks
cleanTextForTts = cleanTextForTts.replace(/[ \t]+/g, ' ');
cleanTextForTts = cleanTextForTts.replace(/^[^\s%`!'\.]\s*/gm, '');
cleanTextForTts = cleanTextForTts.replace(/^(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Foreword...
cleanTextForTts = cleanTextForTts.replace(/^(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Foreword...
cleanTextForTts = cleanTextForTts.replace(/([a-z,;])\s*\n\s*([a-z])/gi, '$1 $2');
cleanTextForTts = cleanTextForTts.replace(/\n{3,}/g, '\n\n');
cleanTextForTts = cleanTextForTts.replace(/([^\s?!\n])$/gm, '$1. ');
cleanTextForTts = cleanTextForTts.replace(/\n\n+/g, '\n\n');
cleanTextForTts = cleanTextForTts.replace(/\s{2,}/g, ' ');
cleanTextForTts = cleanTextForTts.replace(/(\.\s+)(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Fo...
cleanTextForTts = cleanTextForTts.replace(/^(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Foreword...
const chunkSize = useBrowserTts ? 2000 : 4000;
const chunks = splitTextIntoChunks(cleanTextForTts, chunkSize);
ttsChunksRef.current = chunks;
setTtsChunks(chunks);
setTotalChunks(chunks.length);
}

const chunks = ttsChunksRef.current;
if (chunks.length === 0) {
  toast({ title: "No chunks available", variant: "destructive" });
  return;
}
// Clamp chunk index to valid range instead of erroring
let validChunkIndex = chunkIndex;
if (chunkIndex >= chunks.length) {
  validChunkIndex = Math.max(0, chunks.length - 1);
  toast({ title: `Adjusted to section ${validChunkIndex + 1} of ${chunks.length}` });
}

// CRITICAL: Cancel ALL current speech to prevent double voices
if (window.speechSynthesis) {
  window.speechSynthesis.cancel();
}
stopOpenAiTts();
shouldContinueRef.current = false;
setIsPlaying(false);
isPlayingRef.current = false;
await new Promise(r => setTimeout(r, 150)); // Slightly longer wait to ensure complete stop

// Start playing from this chunk
setCurrentChunkIndex(validChunkIndex);
currentChunkIndexRef.current = validChunkIndex;
shouldContinueRef.current = true;
setIsPlaying(true);
isPlayingRef.current = true;

toast({ title: `Playing from section ${validChunkIndex + 1} of ${chunks.length}` });

if (useBrowserTts) {
  // Wait for voices
  const voices = await waitForVoices();
  if (voices.length === 0) {
    toast({ title: "No TTS voices found", variant: "destructive" });
    return;
  }

  // Calculate word offset for highlighting
  let wordOffset = 0;
  for (let i = 0; i < validChunkIndex; i++) {
    wordOffset += chunks[i].split(/\s+/).length;
  }

  speakChunk(validChunkIndex, chunks, voices, wordOffset);
} else {
  // Use OpenAI TTS for Fire tablets and devices without browser TTS
  const chunk = chunks[validChunkIndex];
  await playWithOpenAiTts(chunk, openaiVoice);
}
};

const handlePlayFile = async (fileUrl: string, fileName: string, resumeFromProgress: boolean = false) => {
  try {
    // Check TTS availability
    const useBrowserTts = !!window.speechSynthesis;

    // Check if using browser TTS only
    if (previewSpeaker === "browser_tts" || previewSpeaker === "openai_tts") {
      if (!previewText) {
        toast({ title: "No text content available", variant: "destructive" });
        return;
      }
    }

    // CRITICAL: Cancel ALL existing speech to prevent double voices
    if (window.speechSynthesis) {
      window.speechSynthesis.cancel();
    }
    stopOpenAiTts();
    shouldContinueRef.current = false;
    setIsPlaying(false);
    isPlayingRef.current = false;
    await new Promise(r => setTimeout(r, 50));

    // Use pre-initialized chunks if available (from useEffect), otherwise create them
    let chunks = ttsChunksRef.current;
    if (chunks.length === 0) {
      // Fallback: create chunks if not yet initialized
      // Use identical logic to the display useEffect to ensure TTS reads exactly what's shown
      let textForTts = previewText;
      const titlePageKeywords = /jstor|published|publisher|author[s]??:|doi:|copyright|@|issn|isbn|volume\s+\d|iss...

      const firstPageEnd = textForTts.indexOf('---PAGE---');

```

```

if (firstPageEnd !== -1) {
  const firstPageContent = textForTts.substring(0, firstPageEnd).toLowerCase();
  const wordCount = firstPageContent.split(/\s+/).filter(w => w.length > 0).length;
  if (wordCount < 300 && titlePageKeywords.test(firstPageContent)) {
    textForTts = textForTts.substring(firstPageEnd + 10);
  }
} else {
  const first500Words = textForTts.split(/\s+/).slice(0, 500).join(' ').toLowerCase();
  if (titlePageKeywords.test(first500Words)) {
    const skipTo = textForTts.search(/\n\n[A-Z]/);
    if (skipTo > 100 && skipTo < 2000) {
      textForTts = textForTts.substring(skipTo + 2);
    }
  }
}
let cleanTextForTts = textForTts.replace(/---PAGE---/g, '');
cleanTextForTts = removeFrenchText(cleanTextForTts);
cleanTextForTts = cleanTextForTts.replace(/[ \t]+/g, ' ');
cleanTextForTts = cleanTextForTts.replace(/^[^\s%*!'\.]*$/gm, '');
cleanTextForTts = cleanTextForTts.replace(/^(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Fore...
cleanTextForTts = cleanTextForTts.replace(/^(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Fore...
cleanTextForTts = cleanTextForTts.replace(/([a-z,;])\s*\n\s*([a-z])/gi, '$1 $2');
cleanTextForTts = cleanTextForTts.replace(/\n{3,}/g, '\n\n');
cleanTextForTts = cleanTextForTts.replace(/([A-Z])?(?:\d{4}[a-z]?|pp?\s*\d|[A-Z][a-z]+,\s*\d{4})|\s*\d|[\.\.\.
cleanTextForTts = cleanTextForTts.replace(/([0-9a-zA-Z,;\s]+)\s/g, (match, inner) => {
  if (/^\[d,;\s]*$/.test(inner.trim())) return '';
  if (/^[a-zA-Z]([,;\s]+[a-zA-Z])*[,;\s]*$/.test(inner.trim())) return '';
  if (/^\[d,;\s]*a-zA-Z]*$/.test(inner.trim()) && inner.trim().length < 20) return '';
  return match;
});
cleanTextForTts = cleanTextForTts.replace(/s{2,}/g, ' ');
cleanTextForTts = cleanTextForTts.replace(/([^\? \n])$/gm, '$1. ');
cleanTextForTts = cleanTextForTts.replace(/\n\n+/g, '\n\n');
cleanTextForTts = cleanTextForTts.replace(/s{2,}/g, ' ');
cleanTextForTts = cleanTextForTts.replace(/(\s+)(Introduction|Conclusion|Summary|Overview|Abstract|Prefac...
cleanTextForTts = cleanTextForTts.replace(/^(Introduction|Conclusion|Summary|Overview|Abstract|Preface|Fore...
const chunkSize = 2000;
chunks = splitTextIntoChunks(cleanTextForTts, chunkSize);

ttsChunksRef.current = chunks;
setTtsChunks(chunks);
setTotalChunks(chunks.length);
}

// Check for saved progress (localStorage first, then DB fallback)
let startChunk = 0;
let startWordOffset = 0;

if (!resumeFromProgress && previewFile) {
  const checkedSet = checkedChunksRef.current;
  if (checkedSet.size > 0) {
    for (let i = 0; i < chunks.length; i++) {
      if (!checkedSet.has(i)) {
        startChunk = i;
        break;
      }
      if (i === chunks.length - 1) {
        startChunk = 0;
      }
    }
    if (startChunk > 0) {
      for (let i = 0; i < startChunk; i++) {
        startWordOffset += chunks[i].split(/\s+/).length;
      }
      toast({ title: `Starting from section ${startChunk + 1} of ${chunks.length}` });
    }
  }
}

if (resumeFromProgress && previewFile) {
  const progress = getTtsProgress(previewFile.id);
  if (progress) {
    if (progress.charPosition && progress.charPosition > 0) {
      let charCount = 0;
      for (let i = 0; i < chunks.length; i++) {
        charCount += chunks[i].length;
        if (charCount >= progress.charPosition) {
          startChunk = i;
          break;
        }
      }
      startChunk = Math.min(startChunk, chunks.length - 1);
    } else {
      startChunk = Math.min(progress.chunkIndex, chunks.length - 1);
    }
    for (let i = 0; i < startChunk; i++) {
      startWordOffset += chunks[i].split(/\s+/).length;
    }
    toast({ title: `Resuming from chunk ${startChunk + 1} of ${chunks.length}` });
  } else if (previewFile.lastChunkIndex && previewFile.lastChunkIndex > 0 && previewFile.totalChunks && previ...
  // Fallback: use DB-stored progress (lastChunkIndex is 1-indexed, convert to 0-indexed)
  startChunk = Math.min(previewFile.lastChunkIndex, chunks.length - 1);
  for (let i = 0; i < startChunk; i++) {
    startWordOffset += chunks[i].split(/\s+/).length;
  }
  toast({ title: `Resuming from chunk ${startChunk + 1} of ${chunks.length}` });
}
}

setCurrentChunkIndex(startChunk);
currentChunkIndexRef.current = startChunk;
shouldContinueRef.current = true;
setIsPlaying(true);
isPlayingRef.current = true;

toast({ title: `Reading: ${fileName} (${chunks.length} sections)` });

// Start speaking from the appropriate chunk

```

```

    if (useBrowserTts) {
      // Wait for voices to load
      const voices = await waitForVoices();
      console.log("Available voices:", voices.length);

      if (voices.length === 0) {
        toast({ title: "No TTS voices found. Make sure Chrome has TTS enabled.", variant: "destructive" });
        setIsPlaying(false);
        return;
      }
      speakChunk(startChunk, chunks, voices, startWordOffset);
    } else {
      // Use OpenAI TTS for Fire tablets and devices without browser TTS
      const chunk = chunks[startChunk];
      await playWithOpenAiTts(chunk, openaiVoice);
    }
    return;
  }

  // Use Home Assistant for Echo speaker audio (no word-level sync available)
  const response = await fetch("/api/media/play", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ mediaUrl: fileUrl, entityId: previewSpeaker }),
  });
  if (response.ok) {
    const speakerName = SPEAKERS.find(s => s.id === previewSpeaker)?.name || previewSpeaker;
    toast({ title: `Playing on ${speakerName}: ${fileName}` });
    // Use estimated highlighting for Echo speakers
    startHighlighting();
  } else {
    toast({ title: "Failed to play file", variant: "destructive" });
  }
} catch (error) {
  console.error("Play error:", error);
  toast({ title: "Failed to play file", variant: "destructive" });
}
};

const handleStopMedia = async () => {
  try {
    // IMMEDIATE: Stop any OpenAI TTS audio first (for Fire tablets)
    // This check comes first for faster response on Fire tablets
    if (openaiAudioRef.current) {
      shouldContinueRef.current = false; // Stop chunk chain immediately
      openaiAudioRef.current.pause();
      openaiAudioRef.current.currentTime = 0;
      openaiAudioRef.current = null;

      // Save progress so user can resume later
      const stopFile = previewFileRef.current;
      if (stopFile) {
        let charPosition = 0;
        const chunks = ttsChunksRef.current;
        for (let i = 0; i < currentChunkIndexRef.current; i++) {
          charPosition += chunks[i]?.length || 0;
        }
        saveTtsProgress(stopFile.id, currentChunkIndexRef.current, currentWordIndex, charPosition);
      }

      setIsPlaying(false);
      isPlayingRef.current = false;
      toast({ title: `Stopped at section ${currentChunkIndexRef.current + 1} of ${totalChunks}. Progress saved.` });
      return;
    }

    // Stop browser TTS if active
    if (previewSpeaker === "browser_tts" && window.speechSynthesis) {
      shouldContinueRef.current = false; // Stop chunk chain
      window.speechSynthesis.cancel();

      // Save progress so user can resume later
      const stopFile = previewFileRef.current;
      if (stopFile) {
        let charPosition = 0;
        const chunks = ttsChunksRef.current;
        for (let i = 0; i < currentChunkIndexRef.current; i++) {
          charPosition += chunks[i]?.length || 0;
        }
        saveTtsProgress(stopFile.id, currentChunkIndexRef.current, currentWordIndex, charPosition);
        toast({ title: `Paused at section ${currentChunkIndexRef.current + 1} of ${totalChunks}. Progress saved.` });
      }

      setIsPlaying(false);
      isPlayingRef.current = false;
      return;
    }

    // For Echo speakers - call API (non-blocking for responsiveness)
    fetch("/api/media/stop", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ entityId: previewSpeaker }),
    }).catch(err => console.error("Stop API error:", err));

    // Stop word highlighting when audio stops
    stopHighlighting();
    setIsPlaying(false);
    isPlayingRef.current = false;
  } catch (error) {
    console.error("Stop error:", error);
  }
};

// Stop all playback when the app/tab is closed
useEffect(() => {
  const handleBeforeUnload = () => {
    // Stop browser TTS

```



```

    if (window.speechSynthesis) {
      window.speechSynthesis.cancel();
    }
    // Stop OpenAI TTS audio
    if (openaiAudioRef.current) {
      openaiAudioRef.current.pause();
      openaiAudioRef.current = null;
    }
    // Stop Echo/HA speakers via sendBeacon (works during page unload)
    const speaker = previewSpeakerRef.current;
    if (speaker && speaker !== "browser_tts" && speaker !== "openai_tts" && isPlayingRef.current) {
      const blob = new Blob(
        [JSON.stringify({ entityId: speaker })],
        { type: "application/json" }
      );
      navigator.sendBeacon("/api/media/stop", blob);
    }
  };
  window.addEventListener("beforeunload", handleBeforeUnload);
  return () => window.removeEventListener("beforeunload", handleBeforeUnload);
}, []);

// Skip forward/back functions for browser TTS, OpenAI TTS, and Echo
const handleSkipForward = async () => {
  // For OpenAI TTS on Fire tablets - skip to next chunk
  if (previewSpeaker === "openai_tts" || (!window.speechSynthesis && openaiAudioRef.current)) {
    const chunks = ttsChunksRef.current;
    if (chunks.length === 0) return;

    const nextChunk = Math.min(currentChunkIndexRef.current + 1, chunks.length - 1);
    if (nextChunk === currentChunkIndexRef.current) {
      toast({ title: "Already at last section" });
      return;
    }

    // Stop current audio and play next chunk
    if (openaiAudioRef.current) {
      openaiAudioRef.current.pause();
      openaiAudioRef.current = null;
    }

    currentChunkIndexRef.current = nextChunk;
    setCurrentChunkIndex(nextChunk);
    toast({ title: `Skipped to section ${nextChunk + 1} of ${chunks.length}` });

    if (isPlayingRef.current) {
      shouldContinueRef.current = true;
      await playWithOpenAiTts(chunks[nextChunk], openaiVoice, nextChunk);
    }
    return;
  }

  if (previewSpeaker !== "browser_tts") {
    // For Echo speakers - call seek API
    try {
      await fetch('/api/media/seek', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ direction: 'forward', entityId: 'media_player.echo_lr_studio_white_am' }),
      });
    } catch (error) {
      console.error('Seek forward error:', error);
    }
    return;
  }
}

if (!previewText || !window.speechSynthesis) return;

const chunkText = ttsChunksRef.current.length > 0 ? ttsChunksRef.current.join(' ') : previewText;
const words = chunkText.split(/\s+/).filter(w => w.length > 0 && w !== '---PAGE---');
const skipAmount = 20; // Skip 20 words forward
const newIndex = Math.min(currentWordIndex + skipAmount, words.length - 1);

// Cancel current speech and restart from new position
window.speechSynthesis.cancel();
setCurrentWordIndex(newIndex);

// Sync PDF page to the new position
const newPage = getPageForWordIndex(newIndex);
if (newPage !== currentPdfPage && newPage <= (numPages || 1)) {
  setCurrentPdfPage(newPage);
}

if (isPlaying) {
  // Resume from new position
  const remainingText = words.slice(newIndex).join(' ');
  const utterance = new SpeechSynthesisUtterance(remainingText);
  utterance.rate = browserTtsRateRef.current;
  utterance.volume = browserTtsVolumeRef.current;
  utterance.pitch = 1;

  // Use selected voice
  const voices = window.speechSynthesis.getVoices() || [];
  const voice = selectedVoice ? voices.find(v => v.name === selectedVoice) : voices[0];
  if (voice) utterance.voice = voice;

  let localWordIdx = 0;
  utterance.onboundary = (event) => {
    if (event.name === 'word') {
      setCurrentWordIndex(newIndex + localWordIdx);
      localWordIdx++;
    }
  };
}

utterance.onend = () => {
  setIsPlaying(false);
  isPlayingRef.current = false;
};

```

```

        speechUtteranceRef.current = utterance;
        window.speechSynthesis.speak(utterance);
    }
};

// Restart from very beginning (first chunk)
const handleRestartFromBeginning = async () => {
    if (previewFile) {
        const isOneDrive = previewFile.objectPath?.startsWith('http');
        const url = isOneDrive
            ? `/pdf-reader/onedrive?oneDriveUrl=${encodeURIComponent(previewFile.objectPath || '')}&name=${encodeURIComponent...}`
            : `/pdf-reader/${previewFile.id}?autoplay=1`;
        window.location.href = url;
        return;
    }
    if (!previewText || !window.speechSynthesis) return;

    window.speechSynthesis.cancel();
    setCurrentWordIndex(0);
    setCurrentPdfPage(1);

    // Start playing from beginning
    const chunkText = ttsChunksRef.current.length > 0 ? ttsChunksRef.current.join(' ') : previewText;
    const words = chunkText.split(/\s+/).filter(w => w.length > 0 && w !== '---PAGE---');
    const utterance = new SpeechSynthesisUtterance(words.join(' '));
    utterance.rate = browserTtsRateRef.current;
    utterance.volume = browserTtsVolumeRef.current;
    utterance.pitch = 1;

    const voices = window.speechSynthesis.getVoices() || [];
    const voice = selectedVoice ? voices.find(v => v.name === selectedVoice) : voices[0];
    if (voice) utterance.voice = voice;

    let localWordIdx = 0;
    utterance.onboundary = (event) => {
        if (event.name === 'word') {
            setCurrentWordIndex(localWordIdx);
            localWordIdx++;
        }
    };

    utterance.onend = () => {
        setIsPlaying(false);
        isPlayingRef.current = false;
    };

    setIsPlaying(true);
    isPlayingRef.current = true;
    speechUtteranceRef.current = utterance;
    window.speechSynthesis.speak(utterance);
    toast({ title: "Restarted from beginning" });
};

// Restart current chunk (replay from current position)
const handleRestartCurrentChunk = async () => {
    if (previewSpeaker !== "browser_tts") {
        // For Echo speakers - just replay
        toast({ title: "Replaying current section" });
        return;
    }
    if (!previewText || !window.speechSynthesis) return;

    window.speechSynthesis.cancel();

    // Start playing from current word index
    const chunkText = ttsChunksRef.current.length > 0 ? ttsChunksRef.current.join(' ') : previewText;
    const words = chunkText.split(/\s+/).filter(w => w.length > 0 && w !== '---PAGE---');
    const remainingText = words.slice(currentWordIndex).join(' ');
    const utterance = new SpeechSynthesisUtterance(remainingText);
    utterance.rate = browserTtsRateRef.current;
    utterance.volume = browserTtsVolumeRef.current;
    utterance.pitch = 1;

    const voices = window.speechSynthesis.getVoices() || [];
    const voice = selectedVoice ? voices.find(v => v.name === selectedVoice) : voices[0];
    if (voice) utterance.voice = voice;

    let localWordIdx = 0;
    const startIndex = currentWordIndex;
    utterance.onboundary = (event) => {
        if (event.name === 'word') {
            setCurrentWordIndex(startIndex + localWordIdx);
            localWordIdx++;
        }
    };

    utterance.onend = () => {
        setIsPlaying(false);
        isPlayingRef.current = false;
    };

    setIsPlaying(true);
    isPlayingRef.current = true;
    speechUtteranceRef.current = utterance;
    window.speechSynthesis.speak(utterance);
    toast({ title: "Restarted current section" });
};

const handleSkipBack = async () => {
    // For OpenAI TTS on Fire tablets - skip to previous chunk
    if (previewSpeaker === "openai_tts" || (!window.speechSynthesis && openaiAudioRef.current)) {
        const chunks = ttsChunksRef.current;
        if (chunks.length === 0) return;

        const prevChunk = Math.max(currentChunkIndexRef.current - 1, 0);
        if (prevChunk === currentChunkIndexRef.current) {
            toast({ title: "Already at first section" });
            return;
        }
    }

```

```

}

// Stop current audio and play previous chunk
if (openaiAudioRef.current) {
  openaiAudioRef.current.pause();
  openaiAudioRef.current = null;
}

currentChunkIndexRef.current = prevChunk;
setCurrentChunkIndex(prevChunk);
toast({ title: 'Skipped to section ${prevChunk + 1} of ${chunks.length}' });

if (isPlayingRef.current) {
  shouldContinueRef.current = true;
  await playWithOpenAiTts(chunks[prevChunk], openaiVoice, prevChunk);
}
return;
}

if (previewSpeaker !== "browser_tts") {
  // For Echo speakers - call seek API
  try {
    await fetch('/api/media/seek', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ direction: 'backward', entityId: 'media_player.echo_lr_studio_white_am' }),
    });
  } catch (error) {
    console.error('Seek backward error:', error);
  }
  return;
}

if (!previewText || !window.speechSynthesis) return;

const chunkText = ttsChunksRef.current.length > 0 ? ttsChunksRef.current.join(' ') : previewText;
const words = chunkText.split(/\s+/).filter(w => w.length > 0 && w !== '---PAGE---');
const skipAmount = 20; // Skip 20 words back
const newIndex = Math.max(currentWordIndex - skipAmount, 0);

// Cancel current speech and restart from new position
window.speechSynthesis.cancel();
setCurrentWordIndex(newIndex);

// Sync PDF page to the new position
const newPage = getPageForWordIndex(newIndex);
if (newPage !== currentPdfPage && newPage >= 1) {
  setCurrentPdfPage(newPage);
}

if (isPlaying) {
  // Resume from new position
  const remainingText = words.slice(newIndex).join(' ');
  const utterance = new SpeechSynthesisUtterance(remainingText);
  utterance.rate = browserTtsRateRef.current;
  utterance.volume = browserTtsVolumeRef.current;
  utterance.pitch = 1;

  // Use selected voice
  const voices = window.speechSynthesis.getVoices() || [];
  const voice = selectedVoice ? voices.find(v => v.name === selectedVoice) : voices[0];
  if (voice) utterance.voice = voice;

  let localWordIdx = 0;
  utterance.onboundary = (event) => {
    if (event.name === 'word') {
      setCurrentWordIndex(newIndex + localWordIdx);
      localWordIdx++;
    }
  };

  utterance.onend = () => {
    setIsPlaying(false);
    isPlayingRef.current = false;
  };

  speechUtteranceRef.current = utterance;
  window.speechSynthesis.speak(utterance);
}
};

const handleVolumeChange = async (action: "up" | "down") => {
  try {
    // Check what's ACTUALLY playing, not just the speaker setting
    // OpenAI audio takes priority - if it exists, adjust it regardless of speaker setting
    if (openaiAudioRef.current) {
      const currentVolume = openaiAudioRef.current.volume;
      const newVolume = action === "up"
        ? Math.min(currentVolume + 0.1, 1)
        : Math.max(currentVolume - 0.1, 0);
      openaiAudioRef.current.volume = newVolume;
      setRadioVolume(Math.round(newVolume * 100));
      toast({ title: `Volume: ${Math.round(newVolume * 100)}%` });
      return;
    }

    // Browser TTS - adjust volume from current position
    if (previewSpeaker === "browser_tts" || (window.speechSynthesis && window.speechSynthesis.speaking)) {
      setBrowserTtsVolume(prev => {
        const newVol = action === "up" ? Math.min(prev + 0.1, 1) : Math.max(prev - 0.1, 0);
        browserTtsVolumeRef.current = newVol;
        setRadioVolume(Math.round(newVol * 100));
        if (window.speechSynthesis.speaking) {
          const ci = currentChunkIndexRef.current;
          const charPos = ttsCharIndexRef.current;
          const chunks = ttsChunksRef.current;
          const remaining = chunks[ci].substring(charPos);
          window.speechSynthesis.cancel();
          if (remaining.trim()) {

```

```

        const u = new SpeechSynthesisUtterance(remaining);
        u.rate = browserTtsRateRef.current;
        u.volume = newVol;
        const voices = window.speechSynthesis.getVoices();
        const sv = selectedVoice ? voices.find(v => v.name === selectedVoice) : voices.find(v => v.name.include...
        if (sv) u.voice = sv;
        u.onboundary = (ev) => { if (ev.name === 'word') ttsCharIndexRef.current = charPos + ev.charIndex; };
        u.onend = () => { if (shouldContinueRef.current) { let wo = 0; for (let i = 0; i <= ci; i++) wo += chun...
        u.onerror = (ev) => { if (ev.error !== 'interrupted') { setIsPlaying(false); isPlayingRef.current = fal...
        speechUtteranceRef.current = u;
        window.speechSynthesis.speak(u);
    }
}
toast({ title: `Volume: ${Math.round(newVol * 100)}%` });
return newVol;
});
return;
}

// Echo/HA speakers - call API
if (previewSpeaker && previewSpeaker !== "browser_tts" && previewSpeaker !== "openai_tts") {
    await fetch("/api/media/volume", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ action, entityId: previewSpeaker }),
    });
    const newVol = action === "up" ? Math.min(radioVolume + 5, 100) : Math.max(radioVolume - 5, 0);
    setRadioVolume(newVol);
    toast({ title: `Volume: ${newVol}%` });
    return;
}

// Fallback - just update the display
const newVol = action === "up" ? Math.min(radioVolume + 5, 100) : Math.max(radioVolume - 5, 0);
setRadioVolume(newVol);
toast({ title: `Volume: ${newVol}%` });
} catch (error) {
    console.error("Volume error:", error);
}
};

// State for new semester dialog
const [isNewSemesterDialogOpen, setIsNewSemesterDialogOpen] = useState(false);
const [newSemesterForm, setNewSemesterForm] = useState({
    semesterName: "Spring/Summer 2026 Semester",
    semesterType: "spring_summer" as string,
    semesterStartDate: "2026-05-04",
    semesterEndDate: "2026-08-07",
    course1Code: "CECN210",
    course1Name: "CECN210 - Understanding Economics",
    course1Professor: "",
    course1ProfessorEmail: "",
    course1DeliveryMode: "online" as string,
    course1ClassDay: "" as string,
    course1ClassDay2: "" as string,
    course1ClassTime: "",
    course1ClassEndTime: "",
    course1StartDate: "2026-05-04",
    course1EndDate: "2026-07-31",
    course1SpringSummerTerm: "full" as string,
    course2Code: "CPHL110",
    course2Name: "CPHL110 - Philosophy of Religion",
    course2Professor: "",
    course2ProfessorEmail: "",
    course2DeliveryMode: "in-person" as string,
    course2ClassDay: "tuesday" as string,
    course2ClassDay2: "thursday" as string,
    course2ClassTime: "13:00",
    course2ClassEndTime: "16:00",
    course2StartDate: "2026-05-05",
    course2EndDate: "2026-06-16",
    course2SpringSummerTerm: "first_half" as string,
    course3Code: "CHIS105",
    course3Name: "CHIS105 - Inventing Popular Culture",
    course3Professor: "",
    course3ProfessorEmail: "",
    course3DeliveryMode: "in-person" as string,
    course3ClassDay: "tuesday" as string,
    course3ClassDay2: "thursday" as string,
    course3ClassTime: "09:30",
    course3ClassEndTime: "12:30",
    course3StartDate: "2026-06-23",
    course3EndDate: "2026-08-07",
    course3SpringSummerTerm: "second_half" as string,
    secondaryCalendarId: "",
});

const [isCalendarSettingsOpen, setIsCalendarSettingsOpen] = useState(false);
const [showAllDayRow, setShowAllDayRow] = useState<boolean>(() => {
    const saved = localStorage.getItem('showAllDayRow');
    return saved !== null ? JSON.parse(saved) : false;
});
const [tabBounceEnabled, setTabBounceEnabled] = useState<boolean>(() => {
    const saved = localStorage.getItem('tabBounceEnabled');
    return saved !== null ? JSON.parse(saved) : true;
});
const [selectedSecondaryCalendar, setSelectedSecondaryCalendar] = useState<string>("");
const [shiftScheduleOpen, setShiftScheduleOpen] = useState(false);
const [shiftScheduleYear, setShiftScheduleYear] = useState(new Date().getFullYear());
const [localShiftMap, setLocalShiftMap] = useState<Record<string, string>>({});
const [sleepDisabledDays, setSleepDisabledDays] = useState<Set<string>>(() => {
    try {
        const saved = localStorage.getItem('sleepDisabledDays');
        if (saved) {
            const parsed = JSON.parse(saved);
            if (Array.isArray(parsed)) return new Set(parsed);
        }
    } catch {}
    return new Set();
});

```

```

    } catch { return new Set(); }
  });
  const updateSleepDisabledDays = useCallback((dateStr: string) => {
    setSleepDisabledDays(prev => {
      const next = new Set(prev);
      if (next.has(dateStr)) next.delete(dateStr); else next.add(dateStr);
      try { localStorage.setItem('sleepDisabledDays', JSON.stringify(Array.from(next))); } catch {}
      return next;
    });
  }, []);
  const [shiftDirty, setShiftDirty] = useState(false);

  const { data: shiftScheduleData } = useQuery<{ id: number; date: string; shiftType: string }[]>({
    queryKey: ['/api/shift-schedule'],
  });

  const [crcuSynced, setCrcuSynced] = useState(false);
  useEffect(() => {
    if (!crcuSynced) {
      setCrcuSynced(true);
      fetch('/api/google/third-account/status')
        .then(r => r.json())
        .then(status => {
          if (status.connected) {
            fetch('/api/google/third-account/sync-shifts', { method: 'POST' })
              .then(r => r.json())
              .then(result => {
                if (result.synced > 0) {
                  queryClient.invalidateQueries({ queryKey: ['/api/shift-schedule'] });
                }
              })
            .catch(() => {});
          }
        })
      .catch(() => {});
    }
  }, [crcuSynced]);

  useEffect(() => {
    if (shiftScheduleData) {
      const map: Record<string, string> = {};
      for (const entry of shiftScheduleData) {
        map[entry.date] = entry.shiftType;
      }
      setLocalShiftMap(map);
      setShiftDirty(false);
    }
  }, [shiftScheduleData]);

  const saveShiftScheduleMutation = useMutation({
    mutationFn: async ({ shiftMap, previousData }: { shiftMap: Record<string, string>; previousData?: { date: string; ... } }) => {
      const bulk = Object.entries(shiftMap).map(([date, shiftType]) => ({ date, shiftType }));
      if (previousData) {
        for (const entry of previousData) {
          if (!(entry.date in shiftMap)) {
            bulk.push({ date: entry.date, shiftType: 'off' });
          }
        }
      }
      const res = await fetch('/api/shift-schedule', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ bulk }),
      });
      if (!res.ok) throw new Error('Failed to save');
      return res.json();
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['/api/shift-schedule'] });
      setShiftDirty(false);
      toast({ title: "Shift schedule saved" });
    },
  });

  // Fetch available Google calendars
  const { data: availableCalendars } = useQuery<{ id: string; summary: string; primary: boolean }[]>({
    queryKey: ['/api/calendar/list'],
  });

  // Get the current semester name from settings or use default
  const currentSemesterName = semesterSettings?.semesterName || "Winter 2026 Semester";
  const semStart = semesterSettings?.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : undefined;
  const readingWeekStart = semesterSettings?.readingWeekStart ? new Date(semesterSettings.readingWeekStart) : null;

  // Initialize secondary calendar from semester settings
  useEffect(() => {
    if (semesterSettings?.secondaryCalendarId) {
      setSelectedSecondaryCalendar(semesterSettings.secondaryCalendarId);
    }
  }, [semesterSettings?.secondaryCalendarId]);

  // Mutation to update secondary calendar
  const updateSecondaryCalendarMutation = useMutation({
    mutationFn: async (calendarId: string) => {
      return apiRequest("PATCH", "/api/semester-settings/calendar", { secondaryCalendarId: calendarId });
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["/api/semester-settings"] });
      toast({
        title: "Calendar Updated",
        description: "Secondary calendar has been configured.",
      });
    },
  });

  // Second Google Account status and mutation
  const { data: secondAccountStatus, refetch: refetchSecondAccount } = useQuery<{ connected: boolean; email?: string ... }>({
    queryKey: ['/api/google/second-account/status'],
  });

```

```

});

// Listen for OAuth callback message
useEffect(() => {
  const handleMessage = (event: MessageEvent) => {
    if (event.data?.type === 'SECOND_ACCOUNT_CONNECTED') {
      refetchSecondAccount();
      queryClient.invalidateQueries({ queryKey: ['/api/calendar/events'] });
      toast({
        title: "Second Account Connected",
        description: `Connected to ${event.data.email}`,
      });
    }
  };
  window.addEventListener('message', handleMessage);
  const handleTtsState = (e: MessageEvent) => {
    if (e.data?.type === 'tts-state-update') {
      setHwFloating((p: any) => ({ ...p, isPlaying: e.data.isPlaying }));
    }
  };
  window.addEventListener('message', handleTtsState);
  return () => { window.removeEventListener('message', handleMessage); window.removeEventListener('message', handle...
}, [refetchSecondAccount, queryClient, toast]);

const disconnectSecondAccountMutation = useMutation({
  mutationFn: async () => {
    return apiRequest("DELETE", "/api/google/second-account");
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['/api/google/second-account/status'] });
    queryClient.invalidateQueries({ queryKey: ['/api/calendar/events'] });
    toast({
      title: "Account Disconnected",
      description: "Second Google account has been disconnected.",
    });
  },
  onError: (err) => {
    toast({
      title: "Error",
      description: "Failed to disconnect second account",
      variant: "destructive",
    });
  },
});

// Check if we're past Week 13 end date - show new semester prompt
const week13EndDate = weeks.find(w => w.weekNumber === LAST_WEEK)?.endDate;
const isPastSemester = week13EndDate ? new Date() > new Date(week13EndDate) : false;

const completeMutation = useMutation({
  mutationFn: async ({ id, isCompleted, _skipUndo }: { id: number; isCompleted: boolean; _skipUndo?: boolean }) => {
    return apiRequest("PATCH", `/api/tasks/${id}/complete`, { isCompleted });
  },
  onSuccess: (_, variables) => {
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    queryClient.invalidateQueries({ queryKey: ["/api/weeks"] });
    if (variables.isCompleted) {
      setCompletedTaskHistory(prev => {
        const newHistory = [variables.id, ...prev].slice(0, 10);
        localStorage.setItem('completedTaskHistory', JSON.stringify(newHistory));
        return newHistory;
      });
      setShowCelebration(true);
    }
  },
  if (!variables._skipUndo) {
    const task = allTasks.find(t => t.id === variables.id);
    const actionType = variables.isCompleted ? 'complete' : 'uncomplete';
    pushUndo({
      type: actionType,
      description: `${variables.isCompleted ? 'Completed' : 'Uncompleted'} "${task?.title || 'task'}"`,
      data: { taskId: variables.id, taskTitle: task?.title || 'task' }
    });
  }
});

const flagMutation = useMutation({
  mutationFn: async ({ id, flagged }: { id: number; flagged: boolean }) => {
    return apiRequest("PATCH", `/api/tasks/${id}/flag`, { flagged });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
  },
});

const hideFromSummaryMutation = useMutation({
  mutationFn: async ({ id, hideFromSummary }: { id: number; hideFromSummary: boolean }) => {
    return apiRequest("PATCH", `/api/tasks/${id}`, { hideFromSummary });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
  },
});

const pushUndo = (action: UndoAction) => {
  setUndoStack(prev => [action, ...prev].slice(0, 20));
  setRedoStack([]);
};

const handleUndo = () => {
  if (undoStack.length === 0) return;
  const [action, ...rest] = undoStack;
  setUndoStack(rest);

  switch (action.type) {
    case 'complete': {
      const reverseAction: UndoAction = { type: 'uncomplete', description: `Re-complete "${action.data.taskTitle} || ...`
      setRedoStack(prev => [reverseAction, ...prev]);
    }
  }
}

```

```

    completeMutation.mutate({ id: action.data.taskId, isCompleted: false, _skipUndo: true });
    setCompletedTaskHistory(prev => {
      const newHistory = prev.filter(id => id !== action.data.taskId);
      localStorage.setItem('completedTaskHistory', JSON.stringify(newHistory));
      return newHistory;
    });
    toast({ title: "Undone", description: `Unmarked "${action.data.taskTitle || 'task'}" as complete` });
    break;
  }
  case 'uncomplete': {
    const reverseAction: UndoAction = { type: 'complete', description: `Uncomplete "${action.data.taskTitle || 't...'`
    setRedoStack(prev => [reverseAction, ...prev]);
    completeMutation.mutate({ id: action.data.taskId, isCompleted: true, _skipUndo: true });
    setCompletedTaskHistory(prev => {
      const newHistory = [action.data.taskId, ...prev];
      localStorage.setItem('completedTaskHistory', JSON.stringify(newHistory));
      return newHistory;
    });
    break;
  }
  case 'delete': {
    const reverseAction: UndoAction = { type: 'delete', description: `Delete "${action.data.taskTitle || 'task'}"...`
    setRedoStack(prev => [reverseAction, ...prev]);
    const { taskId, ...taskData } = action.data;
    const createPayload = { ...taskData };
    delete createPayload.taskTitle;
    delete createPayload.id;
    fetch('/api/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      credentials: 'include',
      body: JSON.stringify(createPayload),
    }).then(() => {
      queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
      queryClient.invalidateQueries({ queryKey: ["/api/weeks"] });
    });
    toast({ title: "Undone", description: `Restored "${action.data.taskTitle || 'task'}" });
    break;
  }
  case 'edit':
  case 'move': {
    const reverseAction: UndoAction = { type: action.type, description: `Redo edit on "${action.data.taskTitle || '...'`
    setRedoStack(prev => [reverseAction, ...prev]);
    updateTaskFieldsMutation.mutate({ id: action.data.taskId, ...action.data.oldFields });
    toast({ title: "Undone", description: `Reverted changes to "${action.data.taskTitle || 'task'}" });
    break;
  }
  case 'settings': {
    const reverseAction: UndoAction = { type: 'settings', description: 'Redo settings change', data: { oldColor: ...`
    setRedoStack(prev => [reverseAction, ...prev]);
    setColorSettings(action.data.oldColor);
    if (action.data.oldBlink) setBlinkSettings(action.data.oldBlink);
    localStorage.setItem('colorSettings', JSON.stringify(action.data.oldColor));
    if (action.data.oldBlink) localStorage.setItem('blinkSettings', JSON.stringify(action.data.oldBlink));
    setOriginalColorSettings({...action.data.oldColor});
    if (action.data.oldBlink) setOriginalBlinkSettings({...action.data.oldBlink});
    toast({ title: "Settings reverted", description: "Your previous settings have been restored." });
    break;
  }
  case 'resize': {
    const reverseAction: UndoAction = { type: 'resize', description: `Redo ${action.data.resizeType} resize`, dat...`
    ? { resizeType: 'both', oldHeight: action.data.newHeight, oldReduction: action.data.newReduction, newHeight...`
    : { resizeType: action.data.resizeType, oldValue: action.data.newValue, newValue: action.data.oldValue } };
    setRedoStack(prev => [reverseAction, ...prev]);
    if (action.data.resizeType === 'width') {
      setCalendarReduction(action.data.oldValue);
      setCalendarReductionUserSet(true);
      localStorage.setItem('calendarReduction', String(action.data.oldValue));
    } else if (action.data.resizeType === 'height') {
      setCalendarHeight(action.data.oldValue);
      localStorage.setItem('calendarHeight', String(action.data.oldValue));
    } else if (action.data.resizeType === 'both') {
      setCalendarHeight(action.data.oldHeight);
      setCalendarReduction(action.data.oldReduction);
      setCalendarReductionUserSet(action.data.oldReduction > 120);
      localStorage.setItem('calendarHeight', String(action.data.oldHeight));
      localStorage.setItem('calendarReduction', String(action.data.oldReduction));
    }
    toast({ title: "Undone", description: `Reverted ${action.data.resizeType === 'both' ? 'default reset' : actio...`
    break;
  }
}
};

const handleRedo = () => {
  if (redoStack.length === 0) return;
  const [action, ...rest] = redoStack;
  setRedoStack(rest);

  switch (action.type) {
    case 'complete': {
      const reverseAction: UndoAction = { type: 'complete', description: `Uncomplete "${action.data.taskTitle || 't...'`
      setUndoStack(prev => [reverseAction, ...prev]);
      completeMutation.mutate({ id: action.data.taskId, isCompleted: false, _skipUndo: true });
      setCompletedTaskHistory(prev => {
        const newHistory = prev.filter(id => id !== action.data.taskId);
        localStorage.setItem('completedTaskHistory', JSON.stringify(newHistory));
        return newHistory;
      });
      toast({ title: "Redone", description: `Unmarked "${action.data.taskTitle || 'task'}" });
      break;
    }
    case 'uncomplete': {
      const reverseAction: UndoAction = { type: 'uncomplete', description: `Re-complete "${action.data.taskTitle || '...'`
      setUndoStack(prev => [reverseAction, ...prev]);
      completeMutation.mutate({ id: action.data.taskId, isCompleted: true, _skipUndo: true });
      setCompletedTaskHistory(prev => {
        const newHistory = [action.data.taskId, ...prev];

```

```

        localStorage.setItem('completedTaskHistory', JSON.stringify(newHistory));
        return newHistory;
    });
    break;
}
case 'delete': {
    const taskToDelete = allTasks.find(t => t.title === action.data.title && t.courseName === action.data.courseN...
    if (taskToDelete) {
        const reverseAction: UndoAction = { type: 'delete', description: `Restore "${action.data.taskTitle} || 'task...'
        setUndoStack(prev => [reverseAction, ...prev]);
        deleteMutation.mutate(taskToDelete.id);
        toast({ title: "Redone", description: `Deleted "${action.data.taskTitle} || 'task'" again` });
    }
    break;
}
case 'edit':
case 'move': {
    const reverseAction: UndoAction = { type: action.type, description: `Undo edit on "${action.data.taskTitle} ||...
    setUndoStack(prev => [reverseAction, ...prev]);
    updateTaskFieldsMutation.mutate({ id: action.data.taskId, ...action.data.oldFields });
    toast({ title: "Redone", description: `Re-applied changes to "${action.data.taskTitle} || 'task'"` });
    break;
}
case 'settings': {
    const reverseAction: UndoAction = { type: 'settings', description: 'Undo settings change', data: { oldColor: ...
    setUndoStack(prev => [reverseAction, ...prev]);
    setColorSettings(action.data.oldColor);
    if (action.data.oldBlink) setBlinkSettings(action.data.oldBlink);
    localStorage.setItem('colorSettings', JSON.stringify(action.data.oldColor));
    if (action.data.oldBlink) localStorage.setItem('blinkSettings', JSON.stringify(action.data.oldBlink));
    setOriginalColorSettings({...action.data.oldColor});
    if (action.data.oldBlink) setOriginalBlinkSettings({...action.data.oldBlink});
    toast({ title: "Settings re-applied", description: "Your settings have been restored." });
    break;
}
case 'resize': {
    const reverseAction: UndoAction = { type: 'resize', description: `Undo ${action.data.resizeType} resize`, dat...
    ? { resizeType: 'both', oldHeight: action.data.newHeight, oldReduction: action.data.newReduction, newHeight...
      : { resizeType: action.data.resizeType, oldValue: action.data.newValue, newValue: action.data.oldValue } };
    setUndoStack(prev => [reverseAction, ...prev]);
    if (action.data.resizeType === 'width') {
        setCalendarReduction(action.data.oldValue);
        setCalendarReductionUserSet(true);
        localStorage.setItem('calendarReduction', String(action.data.oldValue));
    } else if (action.data.resizeType === 'height') {
        setCalendarHeight(action.data.oldValue);
        localStorage.setItem('calendarHeight', String(action.data.oldValue));
    } else if (action.data.resizeType === 'both') {
        setCalendarHeight(action.data.oldHeight);
        setCalendarReduction(action.data.oldReduction);
        setCalendarReductionUserSet(action.data.oldReduction > 120);
        localStorage.setItem('calendarHeight', String(action.data.oldHeight));
        localStorage.setItem('calendarReduction', String(action.data.oldReduction));
    }
    toast({ title: "Redone", description: `Re-applied ${action.data.resizeType} === 'both' ? 'default reset' : act...
    break;
}
}
});

const deleteMutation = useMutation({
    mutationFn: async (id: number) => {
        return apiRequest("DELETE", `/api/tasks/${id}`);
    },
    onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
        queryClient.invalidateQueries({ queryKey: ["/api/weeks"] });
    },
});

const isRecurringTask = (task: Task) => {
    return (task.repeatType && task.repeatType !== 'none') || !!task.parentTaskId;
};

const doDeleteSingleTask = (taskId: number) => {
    const task = allTasks.find(t => t.id === taskId);
    if (task) {
        pushUndo({
            type: 'delete',
            description: `Deleted "${task.title}"`,
            data: {
                taskId: task.id,
                taskTitle: task.title,
                title: task.title,
                description: task.description,
                type: task.type,
                courseName: task.courseName,
                dueDate: task.dueDate,
                startDate: task.startDate,
                weekNumber: task.weekNumber,
                isCompleted: task.isCompleted,
                eventStartTime: task.eventStartTime,
                eventEndTime: task.eventEndTime,
                priority: task.priority,
                referenceLink: task.referenceLink,
                attachments: task.attachments,
                repeatType: task.repeatType,
                repeatInterval: task.repeatInterval,
                repeatIntervalUnit: task.repeatIntervalUnit,
                repeatEndDate: task.repeatEndDate,
                reminder1: task.reminder1,
                reminder2: task.reminder2,
                reminder3: task.reminder3,
                reminder4: task.reminder4,
            }
        });
    }
});

deleteMutation.mutate(taskId);

```



```

});

const deleteFutureMutation = useMutation({
  mutationFn: async (taskId: number) => {
    return apiRequest("DELETE", `/api/tasks/${taskId}/future`);
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
  },
});

const deleteTaskWithUndo = (taskId: number, skipDialog?: boolean) => {
  const task = allTasks.find(t => t.id === taskId);
  if (!task) { deleteMutation.mutate(taskId); return; }
  if (!skipDialog && isRecurringTask(task)) {
    setRecurringDeleteTask({ id: taskId, title: task.title });
    return;
  }
  if (!skipDialog && !confirm(`Delete "${task.title}"?`)) return;
  doDeleteSingleTask(taskId);
};

const syncAllCalendarMutation = useMutation({
  mutationFn: async () => {
    return apiRequest("POST", "/api/tasks/sync-all-calendar", {});
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
  },
});

useEffect(() => {
  const ACTIVE_INTERVAL = 15 * 60 * 1000;
  const BACKGROUND_INTERVAL = 30 * 60 * 1000;
  let intervalId: ReturnType<typeof setInterval> | null = null;
  const lastSyncRef = { current: Date.now() };

  const doAutoSync = async () => {
    try {
      await Promise.all([
        fetch('/api/tasks/sync-all-calendar', { method: 'POST', headers: { 'Content-Type': 'application/json' }, bo...
        fetch('/api/calendar/pull', { method: 'POST' })
      ]);
      queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
      lastSyncRef.current = Date.now();
    } catch {}
  };

  const setupInterval = () => {
    if (intervalId) clearInterval(intervalId);
    const ms = document.visibilityState === 'visible' ? ACTIVE_INTERVAL : BACKGROUND_INTERVAL;
    intervalId = setInterval(doAutoSync, ms);
  };

  const handleVisibilityChange = () => {
    const now = Date.now();
    if (document.visibilityState === 'visible') {
      if (now - lastSyncRef.current >= ACTIVE_INTERVAL) {
        doAutoSync();
      }
      refreshFileCounts();
    }
    setupInterval();
  };

  setupInterval();
  document.addEventListener('visibilitychange', handleVisibilityChange);

  return () => {
    if (intervalId) clearInterval(intervalId);
    document.removeEventListener('visibilitychange', handleVisibilityChange);
  };
}, []);

// Mutation for updating task time when dragged to new slot
const updateTaskTimeMutation = useMutation({
  mutationFn: async ({ id, newDate, newHour, newMinutes = 0 }: { id: number; newDate: Date; newHour: number; newMin...
  const updatedDueDate = new Date(newDate);
  updatedDueDate.setHours(newHour, newMinutes, 0, 0);
  return apiRequest("PATCH", `/api/tasks/${id}`, {
    dueDate: updatedDueDate.toISOString(),
    eventStartTime: `${newHour.toString().padStart(2, '0')}:${newMinutes.toString().padStart(2, '0')}`
  });
},
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    queryClient.invalidateQueries({ queryKey: ["/api/weeks"] });
  },
});

// General task update mutation (for courseName, startDate, etc.)
const updateTaskFieldsMutation = useMutation({
  mutationFn: async ({ id, ...fields }: { id: number; [key: string]: any }) => {
    return apiRequest("PATCH", `/api/tasks/${id}`, fields);
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    queryClient.invalidateQueries({ queryKey: ["/api/weeks"] });
  },
});

// Mutation for creating new semester
const createSemesterMutation = useMutation({
  mutationFn: async (data: typeof newSemesterForm) => {
    const payload: Record<string, any> = {
      semesterName: data.semesterName,
      semesterType: data.semesterType,
      semesterStartDate: new Date(data.semesterStartDate).toISOString(),
    };
  },
});

```

```

semesterEndDate: data.semesterEndDate ? new Date(data.semesterEndDate).toISOString() : null,
course1Code: data.course1Code,
course1Name: data.course1Name,
course1Professor: data.course1Professor || null,
course1ProfessorEmail: data.course1ProfessorEmail || null,
course2Code: data.course2Code,
course2Name: data.course2Name,
course2Professor: data.course2Professor || null,
course2ProfessorEmail: data.course2ProfessorEmail || null,
course3Code: data.course3Code,
course3Name: data.course3Name,
course3Professor: data.course3Professor || null,
course3ProfessorEmail: data.course3ProfessorEmail || null,
});
['course1', 'course2', 'course3'].forEach(prefix => {
  payload['${prefix}DeliveryMode'] = (data as any)['${prefix}DeliveryMode'] || null;
  payload['${prefix}ClassDay'] = (data as any)['${prefix}ClassDay'] || null;
  payload['${prefix}ClassDay2'] = (data as any)['${prefix}ClassDay2'] || null;
  payload['${prefix}ClassTime'] = (data as any)['${prefix}ClassTime'] || null;
  payload['${prefix}ClassEndTime'] = (data as any)['${prefix}ClassEndTime'] || null;
  payload['${prefix}SpringSummerTerm'] = (data as any)['${prefix}SpringSummerTerm'] || null;
  const startVal = (data as any)['${prefix}StartDate'];
  const endVal = (data as any)['${prefix}EndDate'];
  payload['${prefix}StartDate'] = startVal ? new Date(startVal).toISOString() : null;
  payload['${prefix}EndDate'] = endVal ? new Date(endVal).toISOString() : null;
});
return apiRequest("POST", "/api/semester", payload);
},
onSuccess: async (response: any) => {
  queryClient.invalidateQueries({ queryKey: ["/api/semester"] });
  queryClient.invalidateQueries({ queryKey: ["/api/semesters"] });
  queryClient.invalidateQueries({ queryKey: ["/api/weeks"] });
  setIsNewSemesterDialogOpen(false);
  try {
    const created = await response.json();
    if (created?.id) {
      fetch('/api/onedrive/ensure-semester-folders', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        credentials: 'include',
        body: JSON.stringify({ semesterId: created.id }),
      }).then(() => {
        console.log('[Semester] OneDrive folders created for new semester');
      }).catch((e: any) => console.warn('[Semester] Failed to create OneDrive folders:', e.message));
    }
  } catch {}
},
});
// File upload hook for drag and drop
const handleCourseListUpload = async (file: File, courseName: string) => {
  setCourseListParsing(true);
  try {
    const text = await file.text();
    const resp = await apiRequest("POST", "/api/tasks/compare-course-list", {
      courseListText: text,
      courseName,
    });
    const data = await resp.json() as { changes: Array<{ id: string; changeType: 'new' | 'modified' | 'removed'; ca...
    setCourseListChanges((data.changes || []).map(c => ({ ...c, accepted: true })));
    setCourseListUploadCourse(courseName);
    setIsCourseListUploadOpen(true);
  } catch (err) {
    console.error("Error parsing course list:", err);
    toast({ title: "Error", description: "Failed to parse course list file", variant: "destructive" });
  } finally {
    setCourseListParsing(false);
  }
};

const handleApplyCourseChanges = async () => {
  const accepted = courseListChanges.filter(c => c.accepted);
  if (accepted.length === 0) {
    setIsCourseListUploadOpen(false);
    return;
  }
  setCourseListApplying(true);
  try {
    const resp = await apiRequest("POST", "/api/tasks/apply-course-changes", {
      changes: accepted,
      courseName: courseListUploadCourse,
    });
    const results = await resp.json() as { created: number; updated: number; deleted: number; errors: number };
    toast({
      title: "Changes Applied",
      description: `Created: ${results.created}, Updated: ${results.updated}, Removed: ${results.deleted}${results.errors ? `, Errors: ${results.errors}` : ''}`,
    });
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    setIsCourseListUploadOpen(false);
    setCourseListChanges([]);
  } catch (err) {
    console.error("Error applying changes:", err);
    toast({ title: "Error", description: "Failed to apply changes", variant: "destructive" });
  } finally {
    setCourseListApplying(false);
  }
};

const { uploadFile: uploadDroppedFile } = useUpload({
  onSuccess: () => {},
});

const createTaskFromFileMutation = useMutation({
  mutationFn: async ({ day, hour, attachmentPath, fileName }: { day: Date; hour: number; attachmentPath: string; fi...
    const dueDate = new Date(day);
    dueDate.setHours(hour, 0, 0, 0);
    const weekNum = getWeekNumber(dueDate, semStart, readingWeekStart);
    return apiRequest("POST", "/api/tasks", {

```

```

        title: fileName,
        type: "reading",
        dueDate: dueDate.toISOString(),
        eventStartTime: `${hour.toString().padStart(2, '0')}:00`,
        weekNumber: Math.max(2, Math.min(13, weekNum)),
        attachments: [attachmentPath],
        priority: "medium",
    });
},
onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    queryClient.invalidateQueries({ queryKey: ["/api/weeks"] });
},
});

// Mark file as completed (listened) and move to completed folder
const markFileCompletedMutation = useMutation({
    mutationFn: async ({ fileId }: { fileId: number }) => {
        return apiRequest("PATCH", `/api/files/${fileId}`, {
            listened: true,
            folder: "completed"
        });
    },
    onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: ["/api/files"] });
        refreshFileCounts();
        setPreviewFile(null);
        toast({ title: "File marked as completed and moved to Completed folder" });
        setShowCelebration(true);
    },
});

// Mutation to attach a file to an existing task
const attachFileToTaskMutation = useMutation({
    mutationFn: async ({ taskId, filePath, fileName }: { taskId: number; filePath: string; fileName: string }) => {
        // Get the current task first
        const currentTask = tasks?.find(t => t.id === taskId);
        const currentAttachments = currentTask?.attachments || [];
        const newAttachment = JSON.stringify({ url: filePath, name: fileName });
        const updatedAttachments = [...currentAttachments, newAttachment];
        return apiRequest("PATCH", `/api/tasks/${taskId}`, {
            attachments: updatedAttachments
        });
    },
    onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
        toast({ title: "File attached to task" });
    },
});

// Project mutations for flyout
const createProjectMutation = useMutation({
    mutationFn: async (data: { name: string; description?: string; color?: string; status?: string; courseName?: string }) => {
        return await apiRequest("POST", "/api/projects", data);
    },
    onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: ["/api/projects"] });
        toast({ title: "Project created successfully" });
    },
    onError: () => {
        toast({ title: "Failed to create project", variant: "destructive" });
    },
});

const updateProjectMutation = useMutation({
    mutationFn: async ({ id, data }: { id: number; data: { name?: string; description?: string; color?: string; status?: string } }) => {
        return await apiRequest("PATCH", `/api/projects/${id}`, data);
    },
    onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: ["/api/projects"] });
        toast({ title: "Project updated successfully" });
    },
    onError: () => {
        toast({ title: "Failed to update project", variant: "destructive" });
    },
});

const deleteProjectMutation = useMutation({
    mutationFn: async (id: number) => {
        return await apiRequest("DELETE", `/api/projects/${id}`);
    },
    onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: ["/api/projects"] });
        toast({ title: "Project deleted successfully" });
    },
    onError: () => {
        toast({ title: "Failed to delete project", variant: "destructive" });
    },
});

// Handle file drop on a task
const handleFileDropOnTask = (e: React.DragEvent, taskId: number) => {
    e.preventDefault();
    e.stopPropagation();
    try {
        const data = e.dataTransfer.getData('application/json');
        if (data) {
            const fileData = JSON.parse(data);
            attachFileToTaskMutation.mutate({
                taskId,
                filePath: fileData.url,
                fileName: fileData.name
            });
        }
    } catch (err) {
        console.error('Error handling file drop:', err);
    }
    setDraggedFile(null);
};

```

```

});

// Drag and drop handlers
const handleDragStart = (e: React.DragEvent, task: Task) => {
  setDraggedTask(task);
  e.dataTransfer.effectAllowed = 'move';
  e.dataTransfer.setData('text/plain', task.id.toString());
};

const handleDragEnd = () => {
  setDraggedTask(null);
  setDragOverSlot(null);
};

const handleDragOver = (e: React.DragEvent, day: Date, hour: number) => {
  e.preventDefault();
  // Allow both move (for tasks) and copy (for files)
  e.dataTransfer.dropEffect = e.dataTransfer.types.includes('Files') ? 'copy' : 'move';
  setDragOverSlot({ day, hour });
};

const handleDragLeave = () => {
  setDragOverSlot(null);
};

const handleDrop = async (e: React.DragEvent, day: Date, hour: number) => {
  e.preventDefault();

  // Detect if drop was in top or bottom half of cell for half-hour precision
  const rect = e.currentTarget.getBoundingClientRect();
  const dropY = e.clientY - rect.top;
  const isBottomHalf = dropY > rect.height / 2;
  const minutes = isBottomHalf ? 30 : 0;

  // Check if files are being dropped from external app
  if (e.dataTransfer.files && e.dataTransfer.files.length > 0) {
    const file = e.dataTransfer.files[0];
    try {
      // Upload the file
      const response = await uploadDroppedFile(file);
      if (response?.objectPath) {
        // Create a task with the file attached
        createTaskFromFileMutation.mutate({
          day,
          hour,
          attachmentPath: response.objectPath,
          fileName: file.name.replace(/\.([^.]*)$/, ''), // Remove file extension for title
        });
      }
    } catch (error) {
      console.error('Failed to upload dropped file:', error);
    }
  } else if (draggedTask) {
    const oldDueDate = draggedTask.dueDate;
    const oldStartTime = draggedTask.eventStartTime;
    pushUndo({
      type: 'move',
      description: `Moved "${draggedTask.title}"`,
      data: {
        taskId: draggedTask.id,
        taskTitle: draggedTask.title,
        oldFields: { dueDate: oldDueDate, eventStartTime: oldStartTime },
        newFields: { dueDate: day.toISOString(), eventStartTime: `${hour.toString().padStart(2, '0')}:${minutes.toS...`
      }
    });
    updateTaskTimeMutation.mutate({ id: draggedTask.id, newDate: day, newHour: hour, newMinutes: minutes });
  }

  setDraggedTask(null);
  setDragOverSlot(null);
};

// Handle dropping a task onto a course row
const handleCourseRowDrop = async (e: React.DragEvent, courseName: string, day: Date) => {
  e.preventDefault();
  e.stopPropagation();

  if (draggedTask) {
    // Find the full course info from coursesData
    // The course.name already contains the full name like "CPPA122 - Local Politics"
    const courseInfo = coursesData.courses.find(c => c.name === courseName);
    const fullCourseName = courseInfo?.name || courseName;

    // weekDays structure is always: [Sun, Mon, Tue, Wed, Thu, Fri, Sat]
    // Sunday is index 0, Friday is index 5
    const weekStartDay = new Date(weekDays[0]); // Sunday
    weekStartDay.setHours(0, 0, 0, 0);

    const weekEndDay = new Date(weekDays[5]); // Friday is always index 5
    weekEndDay.setHours(23, 59, 59, 0);

    pushUndo({
      type: 'move',
      description: `Moved "${draggedTask.title}" to ${fullCourseName}`,
      data: {
        taskId: draggedTask.id,
        taskTitle: draggedTask.title,
        oldFields: { courseName: draggedTask.courseName, startDate: draggedTask.startDate, dueDate: draggedTask.dueDate },
        newFields: { courseName: fullCourseName, startDate: weekStartDay.toISOString(), dueDate: weekEndDay.toISOString() }
      }
    });
    updateTaskFieldsMutation.mutate({
      id: draggedTask.id,
      courseName: fullCourseName,
      startDate: weekStartDay.toISOString(),
      dueDate: weekEndDay.toISOString(),
    });
  }
}

```

```

    setDraggedTask(null);
    setDragOverSlot(null);
};

// Filter tasks by selected date if a date is clicked
const displayTasks = selectedDate
  ? allTasks.filter(t => isSameDayET(new Date(t.dueDate), selectedDate))
  : tasks;

// Sort tasks so ones with attachments come first (for media control alignment)
const sortByAttachments = (tasks: Task[]) => {
  return [...tasks].sort((a, b) => {
    const aHasAttachments = a.attachments && a.attachments.length > 0;
    const bHasAttachments = b.attachments && b.attachments.length > 0;
    if (aHasAttachments && !bHasAttachments) return -1;
    if (!aHasAttachments && bHasAttachments) return 1;
    return 0;
  });
};

// Helper to check if a task is a CASL101 (ASL) class
const isCASL101Task = (task: Task) => {
  const courseName = task.courseName?.toUpperCase() || '';
  return courseName.startsWith("CASL101") || courseName.startsWith("CASL 101");
};

// Helper to check if a CASL101 task's time has passed (should be auto-hidden)
const isCASL101Finished = (task: Task) => {
  if (!isCASL101Task(task)) return false;
  const now = new Date();
  const dueDate = new Date(task.dueDate);
  if (task.eventEndTime) {
    // If task has an end time, check if it's passed
    const [endHour, endMin] = task.eventEndTime.split(':').map(Number);
    dueDate.setHours(endHour, endMin, 0, 0);
  }
  return now > dueDate;
};

const today = useMemo(() => new Date(), [Math.floor(Date.now() / 60000)]);
const todayETKey = useMemo(() => _etDateKey(today), [today]);
const missedTasks = useMemo(() => sortByAttachments(allTasks.filter(t => t.isMissed && !t.isCompleted)), [allTasks]);
const todayTasks = useMemo(() => sortByAttachments(allTasks.filter(t => {
  if (t.isMissed || t.isCompleted) return false;
  const isDueToday = t.dueDate && _etDateKey(new Date(t.dueDate)) === todayETKey;
  const isPrepToday = t.startDate && _etDateKey(new Date(t.startDate)) === todayETKey;
  return isDueToday || isPrepToday;
}))), [allTasks, todayETKey]);

// Update ref for jiggle effect
todayTaskCountRef.current = todayTasks.length;
// Be Prepared shows only prep tasks for the selected week that are not yet due
// Each task appears only once
const upcomingTasks = sortByAttachments(displayTasks.filter(t => {
  if (t.isMissed || t.isCompleted) return false;
  const dueDate = new Date(t.dueDate);
  // Must be a future task (not due today or in the past)
  if (dueDate <= today) return false;
  return true;
})));
const completedTasks = displayTasks.filter(t => t.isCompleted);

// New task filters for Due Today, Due Tomorrow, Due This Week boxes
const tomorrow = new Date(today);
tomorrow.setDate(tomorrow.getDate() + 1);

// Due Today: all tasks due today (not prep tasks, actual due dates)
// Also include full-week MODULE tasks that span today (startDate <= today <= dueDate)
const dueTodayTasks = useMemo(() => allTasks.filter(t => {
  if (t.isMissed || t.isCompleted) return false;
  if (isCASL101Finished(t)) return false;
  if (!t.dueDate) return false;
  if (/module/i.test(t.title || '')) return false;
  if (t.hideFromSummary) return false;

  if (isSameDayET(new Date(t.dueDate), today)) return true;

  if (t.startDate && (t.type === 'module' || t.type === 'reading' || t.type === 'Module' || t.type === 'Reading')) {
    const taskStartDate = startOfDayET(new Date(t.startDate));
    const taskDueDate = startOfDayET(new Date(t.dueDate));
    const todayStart = startOfDayET(today);

    if (taskStartDate <= todayStart && todayStart <= taskDueDate) {
      return true;
    }
  }

  return false;
}).sort((a, b) => new Date(a.dueDate).getTime() - new Date(b.dueDate).getTime()), [allTasks, todayETKey]);

// This Week: tasks due from tomorrow through end of Friday (school week ends Friday 11:59pm)
const todayDow = today.getDay();
const daysUntilFriday = todayDow <= 5 ? (5 - todayDow) : (5 + 7 - todayDow);
const thisWeekFriday = startOfDayET(addDays(today, daysUntilFriday));
const thisWeekFridayEnd = new Date(thisWeekFriday);
thisWeekFridayEnd.setHours(23, 59, 59, 999);
const daysUntilSaturday = todayDow <= 6 ? (6 - todayDow) : 0;
const nextSaturday = startOfDayET(addDays(today, daysUntilSaturday === 0 ? 7 : daysUntilSaturday));
const nextWeekEnd = (() => { const d = startOfDayET(addDays(nextSaturday, 6)); d.setHours(23, 59, 59, 999); return ...
const twoWeeksStart = startOfDayET(addDays(nextSaturday, 7));
const threeWeeksEnd = (() => { const d = startOfDayET(addDays(nextSaturday, 13)); d.setHours(23, 59, 59, 999); return ...
const thisWeekStart = nextSaturday;
const thisWeekEnd = threeWeeksEnd;
const threeWeeksPlusStart = startOfDayET(addDays(nextSaturday, 14));
const beyondStart = startOfDayET(addDays(nextSaturday, 7));

const { dueTomorrowTasks, dueNextWeekTasks, dueTwoWeeksTasks, dueBeyondTasks } = useMemo(() => {

```

```

const activeNotModule = allTasks.filter(t => {
  if (t.isMissed || t.isCompleted) return false;
  if (isCASL101Finished(t)) return false;
  if (!t.dueDate) return false;
  if (t.hideFromSummary) return false;
  return true;
});
const todayStart = startOfDayET(today);
const dueTomorrow = activeNotModule.filter(t => {
  if (/module/i.test(t.title) || '') return false;
  const dueDate = new Date(t.dueDate);
  const dueDateLocal = startOfDayET(dueDate);
  if (dueDateLocal <= todayStart) return false;
  if (dueDate > thisWeekFridayEnd) return false;
  if (isSameDayET(dueDate, today)) return false;
  return true;
}).sort((a, b) => new Date(a.dueDate).getTime() - new Date(b.dueDate).getTime());
const nextWeek = activeNotModule.filter(t => {
  const dueDate = new Date(t.dueDate);
  return dueDate >= nextSaturday && dueDate <= nextWeekEnd;
}).sort((a, b) => new Date(a.dueDate).getTime() - new Date(b.dueDate).getTime());
const twoWeeks = activeNotModule.filter(t => {
  const dueDate = new Date(t.dueDate);
  return dueDate >= twoWeeksStart && dueDate <= threeWeeksEnd;
}).sort((a, b) => new Date(a.dueDate).getTime() - new Date(b.dueDate).getTime());
const beyond = activeNotModule.filter(t => {
  const dueDate = new Date(t.dueDate);
  return dueDate > threeWeeksEnd;
}).sort((a, b) => new Date(a.dueDate).getTime() - new Date(b.dueDate).getTime());
return { dueTomorrowTasks: dueTomorrow, dueNextWeekTasks: nextWeek, dueTwoWeeksTasks: twoWeeks, dueBeyondTasks: b...
}, [allTasks, todayETKey]];
const dueThisWeekTasks = useMemo(() => [...dueNextWeekTasks, ...dueTwoWeeksTasks], [dueNextWeekTasks, dueTwoWeeksTa...

const hwWeeklyTimeline = useMemo(() => {
  const NUM_WEEKS = 13;
  const semDefs: Array<{ label: string; start: Date; end: Date }> = [];
  for (let y = 2025; y <= 2030; y++) {
    const winterStart = new Date(y, 0, 12);
    const winterW1Sat = startOfWeek(winterStart, { weekStartsOn: 6 });
    semDefs.push({ label: 'Winter ' + y, start: winterW1Sat, end: addDays(winterW1Sat, NUM_WEEKS * 7 - 1) });

    const springSumStart = new Date(y, 4, 5);
    const springSumW1Sat = startOfWeek(springSumStart, { weekStartsOn: 6 });
    semDefs.push({ label: 'Spring/Summer ' + y, start: springSumW1Sat, end: addDays(springSumW1Sat, NUM_WEEKS * 7 - ...

    const fallStart = new Date(y, 8, 8);
    const fallW1Sat = startOfWeek(fallStart, { weekStartsOn: 6 });
    semDefs.push({ label: 'Fall ' + y, start: fallW1Sat, end: addDays(fallW1Sat, NUM_WEEKS * 7 - 1) });
  }
  const todayStart = startOfDayET(new Date());
  const currentWeekSat = startOfWeek(todayStart, { weekStartsOn: 6 });
  const result: Array<{ weekStart: Date; weekEnd: Date; label: string; sublabel: string; semLabel: string | null; w...
  let cursor = currentWeekSat;
  const endLimit = new Date(2029, 11, 31);
  while (cursor < endLimit) {
    const wStart = new Date(cursor);
    const wEnd = addDays(wStart, 6);
    let matched: (typeof semDefs)[0] | null = null;
    let wNum: number | null = null;
    for (const sem of semDefs) {
      if (wStart >= sem.start && wStart <= sem.end) {
        matched = sem;
        const diffD = Math.floor((wStart.getTime() - sem.start.getTime()) / 86400000);
        wNum = Math.floor(diffD / 7) + 1;
        if (wNum > NUM_WEEKS) { matched = null; wNum = null; }
        break;
      }
    }
    const dr = wStart.getMonth() === wEnd.getMonth() ? format(wStart, 'MMM d') + '-' + format(wEnd, 'd') : format(w...
    if (matched && wNum !== null && wNum >= 1) {
      result.push({ weekStart: wStart, weekEnd: wEnd, label: 'Week ' + wNum, sublabel: dr, semLabel: matched.label, ...
    } else {
      result.push({ weekStart: wStart, weekEnd: wEnd, label: dr, sublabel: '', semLabel: null, weekNum: null });
    }
    cursor = addDays(cursor, 7);
  }
  return result;
}, []]);

const upcomingEventsToday = useMemo(() => {
  const now = new Date();
  return upcomingCalendarEvents.filter(e => {
    const eventStart = new Date(e.startDate);
    if (e.isAllDay) return isSameDayET(eventStart, today);
    return isSameDayET(eventStart, today) && eventStart >= now;
  }).sort((a, b) => new Date(a.startDate).getTime() - new Date(b.startDate).getTime());
}, [upcomingCalendarEvents, today]);

const upcomingEventsThisWeek = useMemo(() => {
  const tomorrowStart = startOfDayET(addDays(today, 1));
  return upcomingCalendarEvents.filter(e => {
    const eventStart = new Date(e.startDate);
    const eventDay = startOfDayET(eventStart);
    return eventDay >= tomorrowStart && eventStart <= thisWeekFridayEnd;
  }).sort((a, b) => new Date(a.startDate).getTime() - new Date(b.startDate).getTime());
}, [upcomingCalendarEvents, today, thisWeekFridayEnd]);

const upcomingEventsNextWeeks = useMemo(() => {
  return upcomingCalendarEvents.filter(e => {
    const eventStart = new Date(e.startDate);
    return eventStart >= nextSaturday && eventStart <= threeWeeksEnd;
  }).sort((a, b) => new Date(a.startDate).getTime() - new Date(b.startDate).getTime());
}, [upcomingCalendarEvents, nextSaturday, threeWeeksEnd]);

const maxDaysUntil = useMemo(() => {
  const allUpcoming = [...dueTomorrowTasks, ...dueThisWeekTasks];
  if (allUpcoming.length === 0) return 21;
  let max = 0;

```

```

for (const t of allUpcoming) {
  if (!t.dueDate) continue;
  const d = differenceInCalendarDays(new Date(t.dueDate), new Date());
  if (d > max) max = d;
}
return Math.max(max, 1);
}, [dueTodayTasks, dueTomorrowTasks, dueThisWeekTasks]);

// Measure first row positions after render for second row alignment
// IMPORTANT: Does NOT depend on dueThisWeekTasks - positions only update on resize handle changes
useEffect(() => {
  const measurePositions = () => {
    if (row1ContainerRef.current && row1TaskRef.current && row1CodeRef.current && row1CourseRef.current && row1DueR...
    const containerRect = row1ContainerRef.current.getBoundingClientRect();
    const containerLeft = containerRect.left;
    const containerTop = containerRect.top;
    setRow1Positions(prev => ({
      task: row1TaskRef.current!.getBoundingClientRect().left - containerLeft,
      code: row1CodeRef.current!.getBoundingClientRect().left - containerLeft,
      course: row1CourseRef.current!.getBoundingClientRect().left - containerLeft,
      due: row1DueRef.current ? row1DueRef.current.getBoundingClientRect().left - containerLeft : prev.due,
      days: row1DaysRef.current ? row1DaysRef.current.getBoundingClientRect().left - containerLeft : prev.days,
      progressBar: row1ProgressBarRef.current ? row1ProgressBarRef.current.getBoundingClientRect().left - contain...
      progressBarTop: row1ProgressBarRef.current ? row1ProgressBarRef.current.getBoundingClientRect().top - conta...
    }));
  }
});
const timer = setTimeout(measurePositions, 100);
window.addEventListener('resize', measurePositions);
return () => {
  clearTimeout(timer);
  window.removeEventListener('resize', measurePositions);
};
}, []);

// Track calendar wrapper and course rows positions for course button alignment
useEffect(() => {
  const updatePositions = () => {
    if (isResizingThisWeek || isResizingHomework) return;
    if (calendarWrapperRef.current) {
      const rect = calendarWrapperRef.current.getBoundingClientRect();
      const newTop = Math.round(rect.top + window.scrollY);
      const newBottom = Math.round(window.innerHeight - rect.bottom);
      const newRight = Math.round(window.innerWidth - rect.right);
      const newLeft = Math.round(rect.left);
      setCalendarTop(prev => prev === newTop ? prev : newTop);
      setCalendarBottom(prev => prev === newBottom ? prev : newBottom);
      setCalendarRight(prev => prev === newRight ? prev : newRight);
      setCalendarLeft(prev => prev === newLeft ? prev : newLeft);
      if (calendarBorderRef.current) {
        const borderRect = calendarBorderRef.current.getBoundingClientRect();
        const newBorderTop = Math.round(borderRect.top + window.scrollY);
        setCalendarBorderTop(prev => prev === newBorderTop ? prev : newBorderTop);
      }
      const curReduction = calendarReductionRef.current;
      const unreducedWidth = rect.width + curReduction;
      const frSpace = unreducedWidth - gridSizes.timeColumnWidth - gridSizes.moduleColumnWidth;
      const totalFr = gridSizes.dayColumnWidths.reduce((a: number, b: number) => a + b, 0) + gridSizes.progressColu...
      const tuesdayFr = gridSizes.dayColumnWidths[2] || 1;
      const tuesdayPixelWidth = (frSpace / totalFr) * tuesdayFr;
      const reduction = Math.round(1.5 * tuesdayPixelWidth) + 120;
      if (!calendarReductionUserSetRef.current) {
        setCalendarReduction(prev => prev === reduction ? prev : reduction);
      }
      const origLeft = Math.round(rect.left - Math.max(0, curReduction - 3));
      setOriginalCalendarLeft(prev => prev === origLeft ? prev : origLeft);
    }
    if (clockContainerRef.current) {
      const newW = clockContainerRef.current.offsetWidth;
      setClockWidth(prev => prev === newW ? prev : newW);
    }
    if (courseRowsRef.current) {
      const rect = courseRowsRef.current.getBoundingClientRect();
      const newRowsTop = Math.round(rect.top + window.scrollY);
      setCourseRowsTop(prev => prev === newRowsTop ? prev : newRowsTop);
      const newRects = courseRowRefs.current.filter(Boolean).map(el => {
        const r = el!.getBoundingClientRect();
        return { top: Math.round(r.top), height: Math.round(r.height) };
      });
      setCourseRowRects(prev => {
        if (prev.length !== newRects.length) return newRects;
        for (let i = 0; i < prev.length; i++) {
          if (prev[i].top !== newRects[i].top || prev[i].height !== newRects[i].height) return newRects;
        }
        return prev;
      });
    }
    if (allDayRowRef.current) {
      const actualHeight = allDayRowRef.current.offsetHeight;
      if (actualHeight > gridSizes.allDayRowHeight) {
        setGridSizes(prev => ({ ...prev, allDayRowHeight: actualHeight }));
      }
    }
  };
  updatePositions();
  let rafId: number;
  let lastRunTime = 0;
  const rafUpdatePositions = () => {
    if (isResizingThisWeek || isResizingHomework) return;
    const now = performance.now();
    if (now - lastRunTime < 500) return;
    cancelAnimationFrame(rafId);
    rafId = requestAnimationFrame(() => {
      lastRunTime = performance.now();
      updatePositions();
    });
  };
  window.addEventListener('resize', rafUpdatePositions);

```

```

const positionInterval = setInterval(updatePositions, 2000);
return () => {
  window.removeEventListener('resize', rafUpdatePositions);
  cancelAnimationFrame(rafId);
  clearInterval(positionInterval);
};
}, [dueTodayTasks.length, dueTomorrowTasks.length, dueThisWeekTasks.length, modulesHoneycombOpen, isResizingThisWee...

// Calculate shared row heights for consistent sizing between Urgent and Overdue boxes
const cppa122Height = 18 + Math.max(1, todayTasks.filter(t => t.courseName?.startsWith("CPPA122")).length, missedTa...
const cfnf400Height = 18 + Math.max(1, todayTasks.filter(t => t.courseName?.startsWith("CFNF400")).length, missedTa...
const casl101Height = 18 + Math.max(1, todayTasks.filter(t => t.courseName?.startsWith("CASL101")).length, missedTa...

const scrollHomeworkToWeek = (weekNum: number) => {
  if (!homeworkScrollRef.current) return;
  const currentWeekNum = semesterSettings?.semesterStartDate
    ? getWeekNumber(new Date(), new Date(semesterSettings.semesterStartDate), semesterSettings.readingWeekStart)
    : selectedWeek;
  const diff = weekNum - currentWeekNum;
  let sectionId = 'today';
  if (diff === 0) sectionId = 'today';
  else if (diff === 1) sectionId = 'thisweek';
  else if (diff === 2) sectionId = 'nextweek';
  else if (diff === 3) sectionId = 'twoweeks';
  else if (diff >= 4) sectionId = 'threeweeks';
  const el = homeworkScrollRef.current.querySelector(`[data-homework-section="${sectionId}"]`);
  if (el) {
    const scrollContainer = homeworkScrollRef.current;
    const elTop = (el as HTMLElement).offsetTop - scrollContainer.offsetTop;
    scrollContainer.scrollTo({ top: elTop, behavior: 'smooth' });
  }
};

// Weekly view - get the current selected week's days
const selectedWeekInfo = weeks.find(w => w.weekNumber === selectedWeek);
// Parse YYYY-MM-DD dates as local dates at noon to avoid any edge cases
const parseAsLocalDate = (dateStr: string) => {
  // Handle both YYYY-MM-DD and full ISO strings
  const datePart = dateStr.includes('T') ? dateStr.split('T')[0] : dateStr;
  const [year, month, day] = datePart.split('-').map(Number);
  return new Date(year, month - 1, day, 12, 0, 0); // noon local time
};
const computedWeekDates = useMemo(() => {
  if (selectedWeekInfo) return null;
  const semStart = semesterSettings?.semesterStartDate ? new Date(semesterSettings.semesterStartDate) : new Date(20...
  const readingWeek = semesterSettings?.readingWeekStart || null;
  const { start, end } = getWeekDates(selectedWeek, semStart, readingWeek);
  return { start, end };
}, [selectedWeek, selectedWeekInfo, semesterSettings]);
const weekStartDate = selectedWeekInfo ? parseAsLocalDate(selectedWeekInfo.startDate) : computedWeekDates ? new Dat...
const weekEndDate = selectedWeekInfo ? parseAsLocalDate(selectedWeekInfo.endDate) : computedWeekDates ? new Date(co...

// Generate weekdays for the weekly view
// School week runs Saturday to Friday, but we display Sunday-Saturday visually
// Logic:
// - On Sunday through Friday: Saturday column shows the upcoming Saturday (end of this school week)
// - On Saturday: Sunday-Friday columns show NEXT week's dates
const currentDate = new Date();
const currentDayOfWeek = currentDate.getDay(); // 0=Sun, 6=Sat
const isTodaySaturday = currentDayOfWeek === 6;

// Get the raw days from the school week (Saturday to Friday)
// IMPORTANT: eachDayOfInterval resets times to midnight, which can cause timezone issues
// (e.g., midnight UTC = previous day in EST). Normalize all dates to noon local time.
const rawWeekDays = eachDayOfInterval({ start: weekStartDate, end: weekEndDate }).map(d => {
  return new Date(d.getFullYear(), d.getMonth(), d.getDate(), 12, 0, 0);
});

let weekDays: Date[];
if (rawWeekDays.length >= 7) {
  // Ensure we only use first 7 days to avoid timezone edge cases
  const sevenDays = rawWeekDays.slice(0, 7);
  // rawWeekDays[0] = Saturday (start of school week), rawWeekDays[1-6] = Sun-Fri
  if (isTodaySaturday) {
    // On Saturday: Show the full school week starting today (Sat, Sun, Mon, Tue, Wed, Thu, Fri)
    // rawWeekDays already has [Sat, Sun, Mon, Tue, Wed, Thu, Fri] - just reorder to put Sat at end for display
    const saturdayDate = sevenDays[0]; // Current Saturday (today)
    const sunToFri = sevenDays.slice(1); // Sun through Fri of THIS week
    weekDays = [...sunToFri, saturdayDate];
  } else {
    // On Sunday-Friday: Show current week with UPCOMING Saturday at end
    // Saturday should be the UPCOMING Saturday (Friday + 1 day)
    const upcomingSaturday = addDays(sevenDays[6], 1); // Friday + 1 = Saturday
    weekDays = [...sevenDays.slice(1), upcomingSaturday];
  }
} else {
  weekDays = rawWeekDays.slice(0, 7); // Safety limit
}

{
  const todayStartLocal = startOfDayET(new Date());
  weekDays = weekDays.map(d => {
    if (startOfDayET(d) < todayStartLocal) {
      const nd = addDays(d, 7);
      return new Date(nd.getFullYear(), nd.getMonth(), nd.getDate(), 12, 0, 0);
    }
    return d;
  });
};
}

const thisWeekAnnouncements = (() => {
  if (!d2lAnnouncements || d2lAnnouncements.length === 0) return [];
  const todayLocal = startOfDayET(new Date());
  const dayOfWeek = todayLocal.getDay();
  const wkStart = new Date(todayLocal);
  wkStart.setDate(wkStart.getDate() - dayOfWeek);
  wkStart.setHours(0, 0, 0, 0);
  const wkEnd = new Date(wkStart);

```



```

    wkEnd.setDate(wkEnd.getDate() + 6);
    wkEnd.setHours(23, 59, 59, 999);
    return d2lAnnouncements.filter((a: any) => {
      const d = new Date(a.receivedAt || a.date);
      return d >= wkStart && d <= wkEnd;
    });
  }());

const todayCourseTasks = useMemo(() => {
  if (!allTasks || allTasks.length === 0) return [];
  const today = startOfDayET(new Date());
  const courseNames = (coursesData?.courses || []).map((c: any) => c.name?.split(' - ')[0]?.trim().toUpperCase()).f...
  return allTasks.filter(t => {
    if (t.isCompleted || t.isMissed) return false;
    if (t.hideFromSummary) return false;
    if (isCASL101Finished(t)) return false;
    if (t.type === 'class' || t.type === 'module' || t.type === 'other' || t.type === 'meeting' || t.type === 'remi...
    if (t.courseName) return false;
    const tCode = t.courseName.split(' - ')[0]?.trim().toUpperCase().replace(/\\s/g, '');
    if (!courseNames.some((cn: string) => cn.replace(/\\s/g, '') === tCode)) return false;
    return isSameDayET(new Date(t.dueDate), today);
  });
}, [allTasks, coursesData?.courses]);

const todayTaskTickerItems = useMemo(() => {
  if (todayCourseTasks.length === 0) return [];
  const byCourse: Record<string, typeof todayCourseTasks> = {};
  todayCourseTasks.forEach(t => {
    const code = t.courseName?.split(' - ')[0]?.trim().toUpperCase().replace(/\\s/g, '') || 'TASK';
    const taskId = `today-task-${t.id}`;
    const summaryId = `today-summary-${code}`;
    if (dismissedTodayTaskIds.has(taskId) || dismissedTodayTaskIds.has(summaryId)) return;
    if (!byCourse[code]) byCourse[code] = [];
    byCourse[code].push(t);
  });
  const items: Array<{ id: string; courseName: string; subject: string; body: string; snippet: string; receivedAt: ...
  Object.entries(byCourse).forEach([code, tasks] => {
    if (tasks.length === 0) return;
    const summaryId = `today-summary-${code}`;
    const fullCourseName = code;
    const summaryText = tasks.length === 1
      ? `${tasks[0].title} - due today`
      : `${tasks.length} tasks due today`;
    items.push({
      id: summaryId,
      courseName: fullCourseName,
      subject: summaryText,
      body: summaryText,
      snippet: summaryText,
      receivedAt: new Date().toISOString(),
      _isTodayTask: true,
    });
  });
  return items;
}, [todayCourseTasks, dismissedTodayTaskIds]);

const tickerWithTodayTasks = useMemo(() => {
  if (todayTaskTickerItems.length === 0) return thisWeekAnnouncements;
  return [...todayTaskTickerItems, ...thisWeekAnnouncements];
}, [thisWeekAnnouncements, todayTaskTickerItems]);

const d2lTickerHeight = 38;

// Time slots for the day view - always show all 24 hours
const isTravelMode = !(schoolData.isTravelling || profileData.travelTimezone);
const calStart = 0;
const timeSlots = Array.from({ length: 24 }, (_, i) => i);
const calendarScrollRef = useRef<HTMLDivElement>(null);
const calendarContentRef = useRef<HTMLDivElement>(null);

// Auto-scroll to current time by default
useEffect(() => {
  if (calendarView !== "week") return;

  const scrollToCurrentTime = () => {
    if (!calendarScrollRef.current) return;

    const now = new Date();
    const currentHour = now.getHours();
    const calStartHour = 0;

    // Calculate scroll position to show the entire current hour row at the top
    let scrollPosition = 0;
    for (let h = calStartHour; h < currentHour && h < 24; h++) {
      scrollPosition += getEffectiveRowHeight(h);
    }

    calendarScrollRef.current.scrollTop = Math.max(0, scrollPosition);
  };

  // Use requestAnimationFrame to ensure DOM is painted
  requestAnimationFrame(() => {
    scrollToCurrentTime();
  });
}, [calendarView, gridSizes]);

useEffect(() => {
  const timers = new Map<Element, ReturnType<typeof setTimeout>>();
  const handler = (e: Event) => {
    const el = e.target;
    if (!(el instanceof HTMLElement)) return;
    if (!el.classList.contains('scrollbar-hide') && !el.classList.contains('hide-scrollbar')) return;
    el.classList.add('is-scrolling');
    const prev = timers.get(el);
    if (prev) clearTimeout(prev);
    timers.set(el, setTimeout(() => { el.classList.remove('is-scrolling'); timers.delete(el); }, 1000));
  };
},

```

```

    document.addEventListener('scroll', handler, true);
    return () => { document.removeEventListener('scroll', handler, true); timers.forEach(t => clearTimeout(t)); };
}, []);

```

```

// Current week dates (Week 2 = Jan 17-23, 2026)
const currentWeekInfo = weeks.find(w => w.weekNumber === 2); // Current week is Week 2
const currentWeekStart = currentWeekInfo ? new Date(currentWeekInfo.startDate) : null;
const currentWeekEnd = currentWeekInfo ? new Date(currentWeekInfo.endDate) : null;

```

```

const isInCurrentWeek = (day: Date) => {
  if (!currentWeekStart || !currentWeekEnd) return false;
  const dayStart = startOfDayET(day);
  const weekStart = startOfDayET(currentWeekStart);
  const weekEnd = endOfDay(currentWeekEnd);
  return isWithinInterval(dayStart, { start: weekStart, end: weekEnd });
};

```

```

const tasksByDateKey = useMemo(() => {
  const map = new Map<string, Task[]>();
  for (const t of allTasks) {
    if (t.isCompleted) continue;
    if (isCASL101Finished(t)) continue;
    if (!t.dueDate) continue;
    const dueDate = new Date(t.dueDate);
    const isMidnightUTC = dueDate.getUTCHours() === 0 && dueDate.getUTCMinutes() === 0 && !t.eventStartTime;
    const key = isMidnightUTC
      ? (() => {
          const prev = new Date(dueDate.getTime() - 1);
          return _etDateKey(prev);
        })()
      : _etDateKey(dueDate);
    const arr = map.get(key);
    if (arr) arr.push(t); else map.set(key, [t]);
  }
  return map;
}, [allTasks]);

```

```

const getTasksForHour = (day: Date, hour: number) => {
  const dayKey = _etDateKey(day);
  const dayTasks = tasksByDateKey.get(dayKey);
  if (!dayTasks) return [];
  return dayTasks.filter(t => {
    if (t.eventStartTime) {
      const [startHour] = t.eventStartTime.split(':').map(Number);
      return startHour === hour;
    }
    const dueDate = new Date(t.dueDate);
    const isMidnightUTC = dueDate.getUTCHours() === 0 && dueDate.getUTCMinutes() === 0;
    if (isMidnightUTC) return false;
    return getETHours(dueDate) === hour;
  });
};

```

```

const getContinuingTasksForHour = (day: Date, hour: number) => {
  const dayKey = _etDateKey(day);
  const dayTasks = tasksByDateKey.get(dayKey);
  if (!dayTasks) return [];
  return dayTasks.filter(t => {
    if (!t.eventStartTime || !t.eventEndTime) return false;
    const [startHour] = t.eventStartTime.split(':').map(Number);
    const [endHour, endMin] = t.eventEndTime.split(':').map(Number);
    const startsBeforeThisHour = startHour < hour;
    const endsAtOrAfterThisHour = endHour > hour || (endHour === hour && endMin > 0);
    return startsBeforeThisHour && endsAtOrAfterThisHour;
  });
};

```

```

const getMultiHourTasksForWeek = () => {
  const weekDateKeys = new Set(weekDays.map(d => _etDateKey(d)));
  const weekTasks: Task[] = [];
  for (const [key, tasks] of tasksByDateKey) {
    if (weekDateKeys.has(key)) weekTasks.push(...tasks);
  }
  return weekTasks.filter(t => {
    if (!t.eventStartTime || !t.eventEndTime) return false;
    const [startHour] = t.eventStartTime.split(':').map(Number);
    const [endHour] = t.eventEndTime.split(':').map(Number);
    return endHour > startHour;
  }).map(t => {
    const dueDate = new Date(t.dueDate);
    const dueDateKey = _etDateKey(dueDate);
    const dayIdx = weekDays.findIndex(day => _etDateKey(day) === dueDateKey);
    const [startHour, startMin] = t.eventStartTime!.split(':').map(Number);
    const [endHour, endMin] = t.eventEndTime!.split(':').map(Number);

    // Return task data - actual position will be calculated at render time
    // to properly account for prep conflict heights
    return { task: t, dayIdx, startHour, startMin, endHour, endMin };
  });
};

```

```

// Check if a calendar event conflicts with any task
const eventConflictsWithTask = (event: CalendarEvent) => {
  // For all-day events, parse date part directly to avoid timezone issues
  let eventStart: Date;
  let eventEnd: Date;
  if (event.isAllDay) {
    const sp = event.startDate.includes('T') ? event.startDate.split('T')[0] : event.startDate;
    const [sy, sm, sd] = sp.split('-').map(Number);
    eventStart = new Date(sy, sm - 1, sd, 12, 0, 0);
    const ep = event.endDate.includes('T') ? event.endDate.split('T')[0] : event.endDate;
    const [ey, em, ed] = ep.split('-').map(Number);
    eventEnd = new Date(ey, em - 1, ed, 12, 0, 0);
  } else {
    eventStart = new Date(event.startDate);
    eventEnd = new Date(event.endDate);
  }
}

```

```

}

return allTasks.some(task => {
  const taskDue = new Date(task.dueDate);

  // For all-day events, check if any task is due on that day
  if (event.isAllDay) {
    return isSameDayET(eventStart, taskDue);
  }

  // For timed events, check time overlap
  // If task has eventStartTime and eventEndTime, use those
  if (task.eventStartTime && task.eventEndTime) {
    const taskStart = new Date(task.eventStartTime);
    const taskEnd = new Date(task.eventEndTime);
    // Check for overlap: event starts before task ends AND event ends after task starts
    return eventStart < taskEnd && eventEnd > taskStart;
  }

  // For tasks without specific times, check if event is on same day at same hour
  if (isSameDayET(eventStart, taskDue)) {
    const taskHour = getETHours(taskDue);
    const eventHour = getETHours(eventStart);
    // Consider conflict if within same hour or adjacent hours
    return Math.abs(taskHour - eventHour) <= 1;
  }
});

return false;
});
};

// Get Google Calendar events for a specific hour on a day (only conflicting events)
const getCalendarEventsForHour = (day: Date, hour: number) => {
  const now = new Date();
  return calendarEvents.filter(e => {
    if (e.isAllDay) return false;
    const eventDate = new Date(e.startDate);
    const eventEndDate = new Date(e.endDate);
    // Hide events that have already ended
    if (eventEndDate < now) return false;
    // Only show events that conflict with tasks
    if (!eventConflictsWithTask(e)) return false;
    return isSameDayET(eventDate, day) && getETHours(eventDate) === hour;
  });
};

const getEffectiveRowHeight = (h: number) => {
  return gridSizes.timeSlotHeights[h] || gridSizes.timeSlotHeight;
};

// Get all-day Google Calendar events for a day (only conflicting events)
const getAllDayCalendarEvents = (day: Date) => {
  const now = new Date();
  const todayStart = new Date(now.getFullYear(), now.getMonth(), now.getDate());
  return calendarEvents.filter(e => {
    if (!e.isAllDay) return false;
    // For all-day events, extract date part directly to avoid timezone parsing issues
    // startDate may be "2026-03-26T12:00:00" (no TZ) - parse the YYYY-MM-DD directly
    const datePart = e.startDate.includes('T') ? e.startDate.split('T')[0] : e.startDate;
    const [yr, mo, dy] = datePart.split('-').map(Number);
    const eventDate = new Date(yr, mo - 1, dy, 12, 0, 0);
    // Hide all-day events from days that have already passed
    const eventDayStart = new Date(yr, mo - 1, dy);
    if (eventDayStart < todayStart) return false;
    // Only show events that conflict with tasks
    if (!eventConflictsWithTask(e)) return false;
    return isSameDayET(eventDate, day);
  });
};

// Get all-day tasks (tasks without specific time - only midnight)
const getAllDayTasks = (day: Date) => {
  const dayKey = _etDateKey(day);
  const activeCourseNames = (coursesData?.courses || []).map(c => c.name?.split(' - ')[0]?.trim().toUpperCase()).filter(
    name => name !== 'Other'
  );
  return allTasks.filter(t => {
    if (t.isCompleted) return false;
    if (isCASL101Finished(t)) return false;
    if (t.eventStartTime) return false;
    if (t.type !== 'other') {
      const taskCourseCode = t.courseName?.split(' - ')[0]?.trim().toUpperCase();
      if (taskCourseCode && activeCourseNames.some(ac => taskCourseCode.startsWith(ac))) return false;
    }
    const dueDate = new Date(t.dueDate);
    const isMidnightET = getETHours(dueDate) === 0 && getETMinutes(dueDate) === 0;
    const isMidnightUTC = dueDate.getUTCHours() === 0 && dueDate.getUTCMinutes() === 0;
    if (!isMidnightET && !isMidnightUTC) return false;
    if (isMidnightUTC) {
      const prev = new Date(dueDate.getTime() - 1);
      return _etDateKey(prev) === dayKey;
    }
    return isSameDayET(dueDate, day);
  });
};

const weekHasAllDayItems = useMemo(() => {
  return weekDays.some(day => getAllDayTasks(day).length > 0 || getAllDayCalendarEvents(day).length > 0);
}, [allTasks, weekDays, calendarEvents]);

// Get paler version of a color for prep days extension
const getPalerColor = (colorClass: string) => {
  // Convert bg-green-200 style classes to paler versions
  if (colorClass.includes('green')) return 'bg-green-100';
  if (colorClass.includes('pink')) return 'bg-pink-100';
  if (colorClass.includes('indigo')) return 'bg-indigo-100';
  if (colorClass.includes('gray')) return 'bg-gray-100';
  return 'bg-gray-100';
};

```

```

// Get tasks with planning periods on a specific day (startDate <= day < dueDate)
const getPlanningTasksForDay = (day: Date) => {
  return allTasks.filter(t => {
    if (t.isCompleted) return false; // Completed tasks don't show on calendar
    if (!t.startDate) return false;
    const startDate = new Date(t.startDate);
    const dueDate = new Date(t.dueDate);
    const dayStart = new Date(day);
    dayStart.setHours(0, 0, 0, 0);
    const dayEnd = new Date(day);
    dayEnd.setHours(23, 59, 59, 999);

    // Day is in the planning period: startDate <= day < dueDate (not including due date itself)
    return startDate <= dayEnd && dayStart < dueDate && !isSameDayET(day, dueDate);
  });
};

// Get all planning tasks for the week and assign row slots
// Only includes tasks with explicit startDate (user-set prep days)
const getAllWeekPlanningTasks = () => {
  // Only show tasks with explicit start dates (no auto-generation)
  const tasksWithExplicitPlanningPeriods = allTasks.filter(t => t.startDate && !t.isCompleted && !t.isMissed);

  const allPlanningTasks = [...tasksWithExplicitPlanningPeriods];

  // Sort by start date to ensure consistent ordering
  return allPlanningTasks.sort((a, b) => {
    const aStart = new Date(a.startDate!).getTime();
    const bStart = new Date(b.startDate!).getTime();
    return aStart - bStart;
  });
};

// Check if a task's planning period includes a specific day
const isPlanningDayForTask = (task: Task, day: Date) => {
  if (!task.startDate) return false;
  const startDate = new Date(task.startDate);
  const dueDate = new Date(task.dueDate);
  const dayStart = new Date(day);
  dayStart.setHours(0, 0, 0, 0);
  const dayEnd = new Date(day);
  dayEnd.setHours(23, 59, 59, 999);
  return startDate <= dayEnd && dayStart < dueDate && !isSameDayET(day, dueDate);
};

const weekPlanningTasks = getAllWeekPlanningTasks();

const getTasksForDay = (day: Date) => {
  return allTasks.filter(t => isSameDayET(new Date(t.dueDate), day));
};

// Get projects with targetDate on a specific day
const getProjectsForDay = (day: Date) => {
  return allProjects.filter(p => p.targetDate && isSameDayET(new Date(p.targetDate), day));
};

// Get reminders for a day (tasks due 2 days after this day, and 24-hour urgent reminders)
const getRemindersForDay = (day: Date) => {
  const twoDayReminders = allTasks.filter(t => {
    const dueDate = new Date(t.dueDate);
    const reminderDate = subDays(dueDate, 2);
    return isSameDayET(reminderDate, day) && !t.isCompleted;
  }).map(t => ({ ...t, reminderType: '2day' as const }));

  const oneDayReminders = allTasks.filter(t => {
    const dueDate = new Date(t.dueDate);
    const reminderDate = subDays(dueDate, 1);
    return isSameDayET(reminderDate, day) && !t.isCompleted;
  }).map(t => ({ ...t, reminderType: '24hr' as const }));

  return [...oneDayReminders, ...twoDayReminders];
};

// Check if a task is urgent (within 24 hours of due date and not completed)
const isUrgentTask = (task: Task) => {
  if (task.isCompleted) return false;
  const now = new Date();
  const dueDate = new Date(task.dueDate);
  const hoursUntilDue = (dueDate.getTime() - now.getTime()) / (1000 * 60 * 60);
  return hoursUntilDue <= 24 && hoursUntilDue > 0;
};

const handlePrevMonth = () => {
  setCurrentMonth(prev => new Date(prev.getFullYear(), prev.getMonth() - 1, 1));
};

const handleNextMonth = () => {
  setCurrentMonth(prev => new Date(prev.getFullYear(), prev.getMonth() + 1, 1));
};

const handleDayClick = (day: Date) => {
  if (selectedDate && isSameDayET(selectedDate, day)) {
    setSelectedDate(null); // Deselect if clicking same day
  } else {
    setSelectedDate(day);
    // Find and switch to the week containing this day
    const weekForDay = weeks.find(w => {
      const start = parseISO(w.startDate);
      const end = parseISO(w.endDate);
      return isWithinInterval(day, { start: startOfDayET(start), end: endOfDayET(end) });
    });
    if (weekForDay) {
      setSelectedWeek(weekForDay.weekNumber);
    }
  }
};

return (

```

```

<div
  className="flex flex-col overflow-hidden relative"
  style={{
    minWidth: '1024px',
    height: '100%',
    minHeight: '100%',
    backgroundColor: colorSettings.mainBackgroundOverlay ? safeHex(colorSettings.mainBackground, '#3a8bbf') : '#0...
  }}
>
{false && hwTimelinePos && createPortal(
  <div style={{ position: 'fixed', top: `${hwTimelinePos.top}px`, left: `${hwTimelinePos.left}px`, width: '50px...
    <span style={{ fontSize: '8px', fontWeight: 700, color: '#ffffff', letterSpacing: '0.7px', lineHeight: 1 }}...
  </div>,
  document.body
)}
{bookAnimState && (
  <BookAnimation
    isOpen={bookAnimState.isOpen}
    onComplete={bookAnimState.onDone}
    bookColor={bookAnimState.color}
    title={bookAnimState.title}
    courseCode={bookAnimState.courseCode}
  >
  </>
)}
{bookReaderOverlay && createPortal(
  <div
    style={{
      position: 'fixed',
      inset: 0,
      zIndex: 999998,
      backgroundColor: 'rgba(0,0,0,0.88)',
      display: 'flex',
      alignItems: 'center',
      justifyContent: 'center',
      animation: 'bookOverlayFadeIn 0.4s ease forwards',
    }}
    data-testid="book-reader-overlay"
    onClick={(e) => { if (e.target === e.currentTarget) setBookReaderOverlay(null); }}
    onKeyDown={(e) => { if (e.key === 'Escape') setBookReaderOverlay(null); }}
    tabIndex={-1}
    ref={(el) => el?.focus()}
  >
    <style>{`
      @keyframes bookOverlayFadeIn {
        from { opacity: 0; }
        to { opacity: 1; }
      }
      @keyframes bookSlideIn {
        from { transform: scale(0.85) translateY(30px); opacity: 0; }
        to { transform: scale(1) translateY(0); opacity: 1; }
      }
    `}</style>
    <div
      style={{
        display: 'flex',
        width: '92vw',
        height: '88vh',
        maxWidth: '1800px',
        maxHeight: '680px',
        animation: 'bookSlideIn 0.5s ease 0.15s both',
        filter: 'drop-shadow(0 20px 60px rgba(0,0,0,0.6))',
      }}
    >
      <div style={{
        width: '32px',
        flexShrink: 0,
        background: `linear-gradient(90deg, ${bookReaderOverlay.color} 0%, rgba(0,0,0,0.5) 45%, ${bookReaderOve...
        borderRadius: '6px 0 0 6px',
        boxShadow: 'inset -4px 0 12px rgba(0,0,0,0.4), 4px 0 8px rgba(0,0,0,0.2)',
        position: 'relative',
      }}>
        <div style={{
          position: 'absolute',
          top: '50%',
          left: '50%',
          transform: 'translate(-50%, -50%) rotate(-90deg)',
          color: '#D4AF37',
          fontSize: '10px',
          fontWeight: 700,
          letterSpacing: '2px',
          textTransform: 'uppercase',
          whiteSpace: 'nowrap',
          textShadow: '0 1px 3px rgba(0,0,0,0.6)',
          opacity: 0.8,
        }}>
          {bookReaderOverlay.courseCode}
        </div>
      </div>

      <div style={{
        flex: 1,
        backgroundColor: '#fef3c7',
        borderRadius: '0 8px 8px 0',
        overflow: 'hidden',
        position: 'relative',
        boxShadow: '6px 6px 24px rgba(0,0,0,0.4)',
      }}>
        <div style={{
          position: 'absolute',
          top: 0,
          left: 0,
          right: 0,
          height: '36px',
          background: `linear-gradient(180deg, rgba(180,140,80,0.15) 0%, rgba(180,140,80,0.05) 100%)`,
          borderBottom: '1px solid rgba(180,140,80,0.2)',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'space-between',

```

```

padding: '0 12px 0 16px',
zIndex: 2,
}}>
<div style={{ display: 'flex', alignItems: 'center', gap: '8px', minWidth: 0, flex: 1 }}>
  <span style={{ fontSize: '11px', fontWeight: 700, color: bookReaderOverlay.color, textTransform: 'u...
    {bookReaderOverlay.courseCode}
  </span>
  <span style={{ fontSize: '10px', color: '#6b5c3e', fontWeight: 500, overflow: 'hidden', textOverflo...
    {bookReaderOverlay.title}
  </span>
</div>
<div style={{ display: 'flex', gap: '6px', flexShrink: 0 }}>
  <button
    onClick={() => { window.open(bookReaderOverlay.url, '_blank'); setBookReaderOverlay(null); }}
    style={{
      background: 'rgba(0,0,0,0.08)',
      border: 'none',
      borderRadius: '4px',
      padding: '4px 8px',
      cursor: 'pointer',
      fontSize: '9px',
      fontWeight: 600,
      color: '#5a4a2e',
    }}
    data-testid="book-reader-pop-out"
  >
    Pop Out
  </button>
  <button
    onClick={() => setBookReaderOverlay(null)}
    style={{
      background: 'rgba(0,0,0,0.08)',
      border: 'none',
      borderRadius: '4px',
      width: '24px',
      height: '24px',
      cursor: 'pointer',
      display: 'flex',
      alignItems: 'center',
      justifyContent: 'center',
      fontSize: '14px',
      fontWeight: 700,
      color: '#5a4a2e',
      lineHeight: 1,
    }}
    data-testid="book-reader-close"
  >
    x
  </button>
</div>
</div>

<iframe
  src={bookReaderOverlay.url}
  name="book-reader-iframe"
  style={{
    position: 'absolute',
    top: '36px',
    left: 0,
    right: 0,
    bottom: 0,
    width: '100%',
    height: 'calc(100% - 36px)',
    border: 'none',
    backgroundColor: 'fff',
  }}
  data-testid="book-reader-iframe"
/>

<div style={{
  position: 'absolute',
  top: '36px',
  left: 0,
  bottom: 0,
  width: '3px',
  background: 'linear-gradient(90deg, rgba(0,0,0,0.08) 0%, transparent 100%)',
  pointerEvents: 'none',
}} />
</div>
</div>
document.body
})
{/* Background photo layer - always shown when set, behind overlay */}
{colorSettings.backgroundPhoto && (
  <div
    className="absolute inset-0"
    style={{
      backgroundImage: `url(${colorSettings.backgroundPhoto})`,
      backgroundSize: 'cover',
      backgroundPosition: 'center',
      backgroundRepeat: 'no-repeat',
      zIndex: 0
    }}
  />
)}
{/* Default background image layer - only when no custom photo and overlay is off */}
{!colorSettings.backgroundPhoto && !colorSettings.mainBackgroundOverlay && (
  <div
    className="absolute inset-0"
    style={{
      backgroundImage: `url(${dashboardBg})`,
      backgroundSize: '100% 100%',
      backgroundPosition: 'center',
      backgroundRepeat: 'no-repeat',
      filter: 'brightness(1.8)',
      zIndex: 0
    }}
  )}

```



```

<DialogHeader>
  <DialogTitle className="flex items-center gap-2 text-base text-white">
    <CheckCircle2 className="h-5 w-5 text-green-400" />
    Files Uploaded & Player Ready
  </DialogTitle>
  <DialogDescription className="text-white/70 text-sm">
    New files have been synced from OneDrive and prepared for the TTS player.
  </DialogDescription>
</DialogHeader>
<div className="space-y-2 py-3 max-h-[300px] overflow-y-auto">
  {preparedFilesDialog.map((file) => {
    const folderParts = file.folder.match(/week-(\d+)-(\w+)-(module|reading)/);
    const weekNum = folderParts ? folderParts[1] : '?';
    const courseCode = folderParts ? folderParts[2].toUpperCase() : file.folder;
    const fileType = folderParts ? folderParts[3] : 'file';
    return (
      <div key={file.id} className="flex items-start gap-3 p-2 rounded-lg bg-white/5 border border-white/10">
        <div className="flex-1 min-w-0">
          <div className="text-sm font-medium truncate">{file.name}</div>
          <div className="text-xs text-white/50 mt-0.5">
            {courseCode} - Week {weekNum} - {fileType.charAt(0).toUpperCase() + fileType.slice(1)}
          </div>
          <div className="text-xs text-green-400/80 mt-0.5">
            {file.totalChunks} chunks ready ({Math.round(file.textLength / 1000)}k chars extracted)
          </div>
        </div>
        <div>
          </div>
      </div>
    );
  })};
</div>
<DialogFooter>
  <Button
    className="bg-green-600 hover:bg-green-700 text-white w-full"
    onClick={acknowledgePreparedFiles}
    data-testid="button-acknowledge-prepared"
  >
    OK
  </Button>
</DialogFooter>
</DialogContent>
</Dialog>

/* Readings Popup Dialog - OneDrive Files */
<Dialog open={!readingsPopupCourse} onOpenChange={{open} => !open && setReadingsPopupCourse(null)}>
  <DialogContent className="max-w-[420px] p-4 text-white [&_*]:text-white">
    <DialogHeader>
      <DialogTitle className="flex items-center gap-2 text-base text-white">
        <FolderOpen
          className={`h-4 w-4 ${readingsPopupCourse === 'cpa122' ? 'text-green-400 fill-green-200/50' : readin...}
          strokeWidth={1.5}
        />
        {readingsPopupCourse === 'cpa122' ? 'CPA122 Local Politics and Government' : readingsPopupCourse === ...}
      </DialogTitle>
    </DialogHeader>
    <div className="text-xs text-white/60 mb-2">Week {selectedWeek} Readings</div>
    <div className="flex flex-col gap-1 max-h-[250px] overflow-y-auto">
      {oneDriveReadingFiles.map(file => {
        const fullName = file.name || '';
        let cleanName = fullName
          .replace(/^\CPA\s*122[-_\s.]*\/i, '')
          .replace(/^\CFNF\s*400[-_\s.]*\/i, '')
          .replace(/^\CASL\s*101[-_\s.]*\/i, '')
          .replace(/Reading\s*\d+[-_\s.]*\/gi, '')
          .replace(/Local\s*Politics[-_\s.]*\/gi, '')
          .replace(/Human\s*Sexuality[-_\s.]*\/gi, '')
          .replace(/Sign\s*Language[-_\s.]*\/gi, '')
          .replace(/\.pdf$/i, '')
          .trim();
        while (cleanName.match(/^[.\\s\-\_:\.]*$/)) {
          cleanName = cleanName.replace(/^[.\\s\-\_:\.]*+/, '').trim();
        }
        const fileKey = file.path || file.id;
        const isListened = listenedOneDriveFiles.has(fileKey);

        return (
          <div
            key={file.id}
            className="flex items-center gap-2 px-2 py-1.5 rounded hover:bg-white/10 cursor-pointer"
            data-testid={`reading-file-onedrive-${file.id}`}
            onClick={() => {
              if (file.downloadUrl) {
                // Mark as listened and open in PDF reader
                const newListened = new Set(listenedOneDriveFiles);
                newListened.add(fileKey);
                setListenedOneDriveFiles(newListened);
                localStorage.setItem('listenedOneDriveFiles', JSON.stringify(Array.from(newListened)));
                setReadingsPopupCourse(null);
                window.open(`${pdf-reader/onedrive?url=${encodeURIComponent(file.downloadUrl)}&name=${encodeURIComponent...}
              }
            }
          >
          <Checkbox
            checked={isListened}
            onCheckedChange={{checked} => {
              const newListened = new Set(listenedOneDriveFiles);
              if (checked) {
                newListened.add(fileKey);
              } else {
                newListened.delete(fileKey);
              }
              setListenedOneDriveFiles(newListened);
              localStorage.setItem('listenedOneDriveFiles', JSON.stringify(Array.from(newListened)));
            }}
            onClick={(e) => e.stopPropagation()}
            className="h-3.5 w-3.5 border-white/50 data-[state=checked]:bg-green-500 data-[state=checked]:bor...
            data-testid={`checkbox-reading-onedrive-${file.id}`}
          />
          <img src={pdfIconPath} alt="PDF" className="h-3.5 w-3.5 shrink-0" />
        );
      })}
    </div>
  </DialogContent>
</Dialog>

```

```

        <span
          className={`text-[11px] ${isListened ? 'text-white/40 line-through' : 'text-white'}}`
        >
          {cleanName || fullName}
        </span>
      </div>
    );
  }}
  {oneDriveReadingFiles.length === 0 && (
    <div className="text-xs text-white/40 text-center py-4">No reading files for this week</div>
  )}
</div>
</DialogContent>
</Dialog>

<SemesterChecklistDialog
  open={showSemesterChecklist}
  onOpenChange={setShowSemesterChecklist}
  items={semesterChecklistItems}
  onItemsChange={setSemesterChecklistItems}
  semesterSettingsId={semesterSettings?.id}
/>

<Dialog open={showConfirmSemesterDialog} onOpenChange={setShowConfirmSemesterDialog}>
  <DialogContent className="max-w-md text-white" data-testid="confirm-semester-dialog">
    <DialogHeader>
      <DialogTitle className="text-white text-lg font-semibold flex items-center gap-2">
        <GraduationCap className="h-5 w-5" />
        Upcoming: {upcomingSemester?.label}
      </DialogTitle>
      <DialogDescription className="text-white/60 text-sm">
        Please confirm or update the dates for your upcoming semester. These are the dates we have on file:
      </DialogDescription>
    </DialogHeader>
    <div className="flex flex-col gap-3 mt-2">
      <div className="flex flex-col gap-1">
        <label className="text-xs font-medium text-white/70">Semester Start Date</label>
        <input
          type="date"
          className="bg-white/10 rounded px-3 py-1.5 text-sm text-white"
          value={confirmSemesterForm.startDate}
          onChange={(e) => { const v = e.target.value; startTransition(() => setConfirmSemesterForm(prev => ({ ...
            data-testid="input-confirm-start-date"
          })
        </div>
      <div className="flex flex-col gap-1">
        <label className="text-xs font-medium text-white/70">Semester End Date</label>
        <input
          type="date"
          className="bg-white/10 rounded px-3 py-1.5 text-sm text-white"
          value={confirmSemesterForm.endDate}
          onChange={(e) => { const v = e.target.value; startTransition(() => setConfirmSemesterForm(prev => ({ ...
            data-testid="input-confirm-end-date"
          })
        </div>
      <div className="flex flex-col gap-1">
        <label className="text-xs font-medium text-white/70">Reading Week Start</label>
        <input
          type="date"
          className="bg-white/10 rounded px-3 py-1.5 text-sm text-white"
          value={confirmSemesterForm.breakStart}
          onChange={(e) => { const v = e.target.value; startTransition(() => setConfirmSemesterForm(prev => ({ ...
            data-testid="input-confirm-break-start"
          })
        </div>
      <div className="flex flex-col gap-1">
        <label className="text-xs font-medium text-white/70">Reading Week End</label>
        <input
          type="date"
          className="bg-white/10 rounded px-3 py-1.5 text-sm text-white"
          value={confirmSemesterForm.breakEnd}
          onChange={(e) => { const v = e.target.value; startTransition(() => setConfirmSemesterForm(prev => ({ ...
            data-testid="input-confirm-break-end"
          })
        </div>
      </div>
    </DialogFooter className="mt-4 flex gap-2">
      <button
        className="px-4 py-2 text-sm font-medium text-white/70 hover:text-white transition-colors"
        onClick={() => setShowConfirmSemesterDialog(false)}
        data-testid="button-dismiss-confirm-semester"
      >
        Remind me later
      </button>
      <button
        className="px-4 py-2 text-sm font-semibold bg-white/20 hover:bg-white/30 rounded transition-colors text...
        onClick={() => {
          if (upcomingSemester) {
            const idx = FUTURE_SEMESTER_SCHEDULE.findIndex(s => s.label === upcomingSemester.label);
            if (idx !== -1) {
              FUTURE_SEMESTER_SCHEDULE[idx].startDate = confirmSemesterForm.startDate;
              FUTURE_SEMESTER_SCHEDULE[idx].endDate = confirmSemesterForm.endDate;
              FUTURE_SEMESTER_SCHEDULE[idx].breakStart = confirmSemesterForm.breakStart;
              FUTURE_SEMESTER_SCHEDULE[idx].breakEnd = confirmSemesterForm.breakEnd;
            }
            localStorage.setItem(`semConfirm_dismissed_${upcomingSemester.label}`, 'confirmed');
            localStorage.setItem(`semConfirm_dates_${upcomingSemester.label}`, JSON.stringify(confirmSemesterFo...
            toast({ title: "Dates confirmed", description: `${upcomingSemester.label} dates saved.` });
          }
          setShowConfirmSemesterDialog(false);
        }}
        data-testid="button-confirm-semester-dates"
      >
        Confirm dates
      </button>
    </DialogFooter>
  </DialogContent>
</Dialog>

```

```

<Dialog open={showPdfUploadDialog} onOpenChange={setShowPdfUploadDialog}>
  <DialogContent className="sm:max-w-[480px]">
    <DialogHeader>
      <DialogTitle className="text-lg font-bold text-center">
        Æ Update Course Offerings List
      </DialogTitle>
    </DialogHeader>
    <div className="space-y-3 text-sm">
      <p className="text-gray-700">
        Hey Bryn, the semester is ending soon! Time to upload the latest <strong>PAG Elective Course List Offer...
      </p>
      <p className="text-gray-600 text-xs">
        You can get the latest PDF from the PAG program office or your advisor. Upload it here and I'll update ...
      </p>
      <div className="border-2 border-dashed border-gray-400 rounded-lg p-6 text-center bg-white/50 hover:bg-wh...
        const input = document.createElement('input');
        input.type = 'file';
        input.accept = '.pdf';
        input.onChange = (e) => {
          const file = (e.target as HTMLInputElement).files?.[0];
          if (file) {
            alert('Received: ${file.name}\n\nIn a future update, I'll parse this PDF and update your elective c...
            setShowPdfUploadDialog(false);
          }
        };
        input.click();
      </div>
      <div data-testid="pdf-upload-dropzone">
        <Upload className="h-8 w-8 mx-auto mb-2 text-gray-400" />
        <p className="text-sm font-medium text-gray-600">Click to upload PDF</p>
        <p className="text-xs text-gray-400 mt-1">PAG Registration Package (.pdf)</p>
      </div>
    </div>
    <DialogFooter>
      <button
        className="px-4 py-2 text-sm bg-gray-200 rounded-md hover:bg-gray-300 text-gray-700"
        onClick={() => setShowPdfUploadDialog(false)}
        data-testid="btn-dismiss-pdf-upload"
      >
        Remind Me Later
      </button>
    </DialogFooter>
  </DialogContent>
</Dialog>

{/* New Semester Setup Dialog */}
<Dialog open={isNewSemesterDialogOpen} onOpenChange={setIsNewSemesterDialogOpen}>
  <DialogContent className="max-w-lg max-h-[90vh] overflow-y-auto text-white [&_*]:text-white [&_label]:text-wh...
    <DialogHeader>
      <DialogTitle className="flex items-center gap-2 text-white">
        <GraduationCap className="h-5 w-5" />
        Set Up New Semester
      </DialogTitle>
    </DialogHeader>
    <form
      className="space-y-4"
      onSubmit={(e) => {
        e.preventDefault();
        createSemesterMutation.mutate(newSemesterForm);
      }}
    >
      <div>
        <Label htmlFor="semesterName">Semester Name</Label>
        <Input
          id="semesterName"
          placeholder="e.g., Spring/Summer 2026 Semester"
          value={newSemesterForm.semesterName}
          onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...p...
            required
            data-testid="input-semester-name"
          </div>
        </div>
      <div>
        <Label htmlFor="semesterStartDate">Semester Start Date (Week 1 Saturday)</Label>
        <Input
          id="semesterStartDate"
          type="date"
          value={newSemesterForm.semesterStartDate}
          onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...p...
            required
            data-testid="input-semester-start"
          </div>
        </div>
      <div className="border rounded-lg p-3 space-y-3">
        <Label className="font-medium text-green-600">Course 1 (Green)</Label>
        <div className="grid grid-cols-2 gap-2">
          <div>
            <Label className="text-xs text-muted-foreground">Code</Label>
            <Input
              placeholder="e.g., CPPA122"
              value={newSemesterForm.course1Code}
              onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...
                required
                data-testid="input-course1-code"
            </div>
          </div>
          <div>
            <Label className="text-xs text-muted-foreground">Course Name</Label>
            <Input
              placeholder="e.g., Local Politics"
              value={newSemesterForm.course1Name}
              onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...
                required
                data-testid="input-course1-name"
            </div>
          </div>
        </div>
      </div>
    </DialogContent>
  </Dialog>

```

```

</div>
<div className="grid grid-cols-2 gap-2">
  <div>
    <Label className="text-xs text-muted-foreground">Professor Name (optional)</Label>
    <Input
      placeholder="e.g., Dr. Smith"
      value={newSemesterForm.course1Professor}
      onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...
        data-testid="input-course1-professor"
      />
    </div>
  </div>
  <div>
    <Label className="text-xs text-muted-foreground">Professor Email (optional)</Label>
    <Input
      type="email"
      placeholder="e.g., prof@university.edu"
      value={newSemesterForm.course1ProfessorEmail}
      onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...
        data-testid="input-course1-professor-email"
      />
    </div>
  </div>
</div>
</div>

<div className="border rounded-lg p-3 space-y-3">
  <Label className="font-medium text-pink-600">Course 2 (Pink)</Label>
  <div className="grid grid-cols-2 gap-2">
    <div>
      <Label className="text-xs text-muted-foreground">Code</Label>
      <Input
        placeholder="e.g., CFNF400"
        value={newSemesterForm.course2Code}
        onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...
          required
          data-testid="input-course2-code"
        />
      </div>
    </div>
    <div>
      <Label className="text-xs text-muted-foreground">Course Name</Label>
      <Input
        placeholder="e.g., Human Sexuality"
        value={newSemesterForm.course2Name}
        onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...
          required
          data-testid="input-course2-name"
        />
      </div>
    </div>
  </div>
  <div className="grid grid-cols-2 gap-2">
    <div>
      <Label className="text-xs text-muted-foreground">Professor Name (optional)</Label>
      <Input
        placeholder="e.g., Prof. Johnson"
        value={newSemesterForm.course2Professor}
        onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...
          data-testid="input-course2-professor"
        />
      </div>
    <div>
      <Label className="text-xs text-muted-foreground">Professor Email (optional)</Label>
      <Input
        type="email"
        placeholder="e.g., prof@university.edu"
        value={newSemesterForm.course2ProfessorEmail}
        onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...
          data-testid="input-course2-professor-email"
        />
      </div>
    </div>
  </div>
</div>

<div className="border rounded-lg p-3 space-y-3">
  <Label className="font-medium text-indigo-600">Course 3 (Indigo)</Label>
  <div className="grid grid-cols-2 gap-2">
    <div>
      <Label className="text-xs text-muted-foreground">Code</Label>
      <Input
        placeholder="e.g., CASL101"
        value={newSemesterForm.course3Code}
        onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...
          required
          data-testid="input-course3-code"
        />
      </div>
    </div>
    <div>
      <Label className="text-xs text-muted-foreground">Course Name</Label>
      <Input
        placeholder="e.g., Sign Language"
        value={newSemesterForm.course3Name}
        onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...
          required
          data-testid="input-course3-name"
        />
      </div>
    </div>
  </div>
  <div className="grid grid-cols-2 gap-2">
    <div>
      <Label className="text-xs text-muted-foreground">Professor Name (optional)</Label>
      <Input
        placeholder="e.g., Dr. Williams"
        value={newSemesterForm.course3Professor}
        onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...
          data-testid="input-course3-professor"
        />
      </div>
    </div>
    <div>
      <Label className="text-xs text-muted-foreground">Professor Email (optional)</Label>
      <Input

```

```

        type="email"
        placeholder="e.g., prof@university.edu"
        value={newSemesterForm.course3ProfessorEmail}
        onChange={(e) => { const v = e.target.value; startTransition(() => setNewSemesterForm(prev => ({ ...
        data-testid="input-course3-professor-email"
    />
  </div>
</div>
</div>

<Button
  type="submit"
  className="w-full"
  disabled={createSemesterMutation.isPending}
  data-testid="button-create-semester"
>
  {createSemesterMutation.isPending ? (
    <
      <Loader2 className="h-4 w-4 mr-2 animate-spin" />
      Creating New Semester...
    </>
  ) : (
    "Start New Semester"
  )}
</Button>
</form>
</DialogContent>
</Dialog>

/* Upload Files Dialog - Windows Explorer Style */
<Dialog open={isUploadDialogOpen} onOpenChange={setIsUploadDialogOpen}>
  <DialogContent className="max-w-4xl h-[70vh] p-0 overflow-hidden text-white shadow-2xl">
    <VisuallyHidden.Root><DialogTitle>Upload Files</DialogTitle></VisuallyHidden.Root>
    /* Title Bar */
    <div className="flex items-center gap-2 px-3 py-2 bg-[#323232] border-b border-[#3c3c3c]">
      <Upload className="h-4 w-4 text-blue-400" />
      <span className="text-sm font-medium">Upload Files</span>
    </div>

    /* Address Bar */
    <div className="flex items-center gap-2 px-3 py-1.5 border-b border-[#3c3c3c]">
      <div className="flex items-center gap-1 px-2 py-1 bg-[#3c3c3c] rounded text-xs flex-1">
        <Folder className="h-3 w-3 text-yellow-500" />
        <span className="text-white/80">{uploadTargetFolder} | 'Select a folder...</span>
      </div>
    </div>

    <div className="flex flex-1 overflow-hidden" style={{ height: 'calc(100% - 80px)' }}>
      /* Left Panel - Folder Tree */
      <div className="w-64 border-r border-[#3c3c3c] overflow-y-auto" style={{ scrollbarWidth: 'thin' }}>
        <div className="p-2">
          /* Root folder */
          <div className="flex items-center gap-1 py-1 px-1 text-white/90 text-sm font-medium">
            <Folder className="h-4 w-4 text-yellow-500 fill-yellow-400" />
            <span>Course Files</span>
          </div>

          /* Week folders */
          {Array.from({ length: 13 }, (_, i) => i + 1).map(weekNum => {
            const weekId = `upload-week-${weekNum}`;
            const isWeekExpanded = flyoutExpandedFolders.has(weekId);

            return (
              <div key={weekNum} className="ml-2">
                /* Week row */
                <div
                  className="flex items-center gap-1 py-0.5 px-1 hover:bg-[#2a2d2e] rounded cursor-pointer"
                  onClick={() => toggleFlyoutFolder(weekId)}
                  onContextMenu={(e) => handleFolderContextMenu(e, `week-${weekNum}`)}
                  data-testid={`folder-week-${weekNum}`}
                >
                  <div className="w-4 h-4 flex items-center justify-center">
                    {isWeekExpanded ? (
                      <ChevronDown className="h-3 w-3 text-white/60" />
                    ) : (
                      <ChevronRight className="h-3 w-3 text-white/60" />
                    )}
                  </div>
                  {isWeekExpanded ? (
                    <FolderOpen className="h-4 w-4 text-yellow-500 fill-yellow-400" />
                  ) : (
                    <Folder className="h-4 w-4 text-yellow-500 fill-yellow-400" />
                  )}
                  <span className="text-xs text-white/80">Week {weekNum}</span>
                </div>

                /* Course folders inside week */
                {isWeekExpanded && (
                  <div className="ml-4">
                    /* CPPA122 */
                    {(() => {
                      const courseId = `upload-week-${weekNum}-cpa122`;
                      const isCourseExpanded = flyoutExpandedFolders.has(courseId);
                      return (
                        <div>
                          <div
                            className="flex items-center gap-1 py-0.5 px-1 hover:bg-[#2a2d2e] rounded cursor-po...
                            onClick={() => toggleFlyoutFolder(courseId)}
                            onContextMenu={(e) => handleFolderContextMenu(e, `week-${weekNum}-cpa122`)}
                            data-testid={`folder-week-${weekNum}-cpa122`}
                          >
                            <div className="w-4 h-4 flex items-center justify-center">
                              {isCourseExpanded ? (
                                <ChevronDown className="h-3 w-3 text-white/60" />
                              ) : (
                                <ChevronRight className="h-3 w-3 text-white/60" />
                              )}
                            </div>
                          </div>
                        </div>
                      )}
                    )}
                  </div>
                )}
              </div>
            );
          })}
        </div>
      </div>
    </div>
  </DialogContent>
</Dialog>

```

```

{isCourseExpanded ? (
  <FolderOpen className="h-4 w-4 text-green-500 fill-green-400" />
) : (
  <Folder className="h-4 w-4 text-green-500 fill-green-400" />
)}
<span className="text-xs text-green-400">CPPA122</span>
</div>
{isCourseExpanded && (
  <div className="ml-4">
    <div
      className={`flex items-center gap-1 py-0.5 px-1 hover:bg-[#2a2d2e] rounded curs...
      onClick={() => setUploadTargetFolder(`week-${weekNum}-cppa122-reading`)}
    >
      <div className="w-4 h-4" />
      <Folder className="h-4 w-4 text-green-500 fill-green-400" />
      <span className="text-xs text-white/80">Reading</span>
    </div>
    <div
      className={`flex items-center gap-1 py-0.5 px-1 hover:bg-[#2a2d2e] rounded curs...
      onClick={() => setUploadTargetFolder(`week-${weekNum}-cppa122-module`)}
    >
      <div className="w-4 h-4" />
      <Folder className="h-4 w-4 text-green-500 fill-green-400" />
      <span className="text-xs text-white/80">Module</span>
    </div>
    {/* Custom folders for this course */}
    {customFolders.filter(f => f.parent === `week-${weekNum}-cppa122`).map(folder => (
      <div
        key={folder.id}
        className={`flex items-center gap-1 py-0.5 px-1 hover:bg-[#2a2d2e] rounded cu...
        onClick={() => setUploadTargetFolder(folder.id)}
        onContextMenu={(e) => handleFolderContextMenu(e, folder.id)}
        data-testid={`folder-custom-${folder.id}`}
      >
        <div className="w-4 h-4" />
        <Folder className="h-4 w-4 text-green-500 fill-green-400" />
        <span className="text-xs text-white/80">{folder.name}</span>
      </div>
    ))}
  </div>
)}
</div>
)}
)}()
)}()
{ /* CFNF400 */}
{(() => {
  const courseId = `upload-week-${weekNum}-cfnf400`;
  const isCourseExpanded = flyoutExpandedFolders.has(courseId);
  return (
    <div>
      <div
        className="flex items-center gap-1 py-0.5 px-1 hover:bg-[#2a2d2e] rounded cursor-po...
        onClick={() => toggleFlyoutFolder(courseId)}
        onContextMenu={(e) => handleFolderContextMenu(e, `week-${weekNum}-cfnf400`)}
        data-testid={`folder-week-${weekNum}-cfnf400`}
      >
        <div className="w-4 h-4 flex items-center justify-center">
          {isCourseExpanded ? (
            <ChevronDown className="h-3 w-3 text-white/60" />
          ) : (
            <ChevronRight className="h-3 w-3 text-white/60" />
          )}
        </div>
        {isCourseExpanded ? (
          <FolderOpen className="h-4 w-4 text-pink-500 fill-pink-400" />
        ) : (
          <Folder className="h-4 w-4 text-pink-500 fill-pink-400" />
        )}
        <span className="text-xs text-pink-400">CFNF400</span>
      </div>
      {isCourseExpanded && (
        <div className="ml-4">
          <div
            className={`flex items-center gap-1 py-0.5 px-1 hover:bg-[#2a2d2e] rounded curs...
            onClick={() => setUploadTargetFolder(`week-${weekNum}-cfnf400-reading`)}
          >
            <div className="w-4 h-4" />
            <Folder className="h-4 w-4 text-pink-500 fill-pink-400" />
            <span className="text-xs text-white/80">Reading</span>
          </div>
          <div
            className={`flex items-center gap-1 py-0.5 px-1 hover:bg-[#2a2d2e] rounded curs...
            onClick={() => setUploadTargetFolder(`week-${weekNum}-cfnf400-module`)}
          >
            <div className="w-4 h-4" />
            <Folder className="h-4 w-4 text-pink-500 fill-pink-400" />
            <span className="text-xs text-white/80">Module</span>
          </div>
          {/* Custom folders for this course */}
          {customFolders.filter(f => f.parent === `week-${weekNum}-cfnf400`).map(folder => (
            <div
              key={folder.id}
              className={`flex items-center gap-1 py-0.5 px-1 hover:bg-[#2a2d2e] rounded cu...
              onClick={() => setUploadTargetFolder(folder.id)}
              onContextMenu={(e) => handleFolderContextMenu(e, folder.id)}
              data-testid={`folder-custom-${folder.id}`}
            >
              <div className="w-4 h-4" />
              <Folder className="h-4 w-4 text-pink-500 fill-pink-400" />
              <span className="text-xs text-white/80">{folder.name}</span>
            </div>
          ))}
        </div>
      )}
    </div>
  )}
)}()

```



```

{/* New Folder Input */}
{isCreatingFolder && (
  <div className="flex items-center gap-2 px-3 py-2 bg-[#2d2d2d] border-b border-[#3c3c3c]">
    <input
      placeholder="New folder name..."
      value={newFolderName}
      onChange={(e) => { const v = e.target.value; startTransition(() => setNewFolderName(v)); }}
      className="flex-1 h-7 text-xs bg-[#3c3c3c] border-[#5c5c5c] text-white"
      data-testid="input-new-folder-name"
    />
    <Button
      size="sm"
      className="h-7 px-3 text-xs bg-blue-600 hover:bg-blue-500"
      onClick={() => {
        if (newFolderName.trim()) {
          setUploadTargetFolder(newFolderName.trim().toLowerCase().replace(/\s+/g, '-'));
          setNewFolderName('');
          setIsCreatingFolder(false);
        }
      }}
      data-testid="button-create-folder"
    />
    <Create
    />
    <Button
      size="sm"
      variant="ghost"
      className="h-7 px-2 text-xs text-white/60 hover:text-white"
      onClick={() => setIsCreatingFolder(false)}
    />
    <Cancel
    />
  </div>
)}

{/* Upload Drop Zone */}
<div className="flex-1 flex items-center justify-center p-8">
  <div className="w-full max-w-md border-2 border-dashed border-[#3c3c3c] rounded-lg p-8 text-center ho...
    <input
      type="file"
      id="file-upload"
      multiple
      accept=".pdf,.doc,.docx,.txt"
      className="hidden"
      onChange={async (e) => {
        const files = e.target.files;
        if (!files || files.length === 0) return;

        // Require folder selection
        if (!uploadTargetFolder) {
          toast({ title: "Please select a folder first", variant: "destructive" });
          e.target.value = '';
          return;
        }

        let successCount = 0;
        let errorCount = 0;

        for (const file of Array.from(files)) {
          try {
            // Step 1: Request a presigned upload URL (with folder)
            const requestRes = await fetch('/api/uploads/request-url', {
              method: 'POST',
              headers: { 'Content-Type': 'application/json' },
              body: JSON.stringify({
                name: file.name,
                size: file.size,
                contentType: file.type || 'application/octet-stream',
                folder: uploadTargetFolder,
              }),
            });

            if (!requestRes.ok) {
              throw new Error('Failed to get upload URL');
            }

            const { uploadURL } = await requestRes.json();

            // Step 2: Upload file directly to presigned URL
            const uploadRes = await fetch(uploadURL, {
              method: 'PUT',
              body: file,
              headers: {
                'Content-Type': file.type || 'application/octet-stream',
              },
            });

            if (!uploadRes.ok) {
              throw new Error('Failed to upload file');
            }

            successCount++;
          } catch (error) {
            console.error('Upload error:', error);
            errorCount++;
          }
        }

        // Show result toast
        if (successCount > 0) {
          toast({ title: `Uploaded ${successCount} file${successCount > 1 ? 's' : ''} to ${uploadTarget...`
        }
        if (errorCount > 0) {
          toast({ title: `Failed to upload ${errorCount} file${errorCount > 1 ? 's' : ''}`, variant: "d...`
        }

        queryClient.invalidateQueries({ queryKey: ['/api/files'] });
        e.target.value = '';
      }
    </div>
  </div>

```

```

        setIsUploadDialogOpen(false);
    })
  }
  />
  <label htmlFor="file-upload" className="cursor-pointer block">
    <div className="w-16 h-16 mx-auto mb-4 rounded-full bg-[#3c3c3c] flex items-center justify-center">
      <Upload className="h-8 w-8 text-blue-400" />
    </div>
    <p className="text-base text-white/90 font-medium mb-1">Drop files here</p>
    <p className="text-sm text-white/60 mb-3">or click to browse</p>
    <p className="text-xs text-white/40">Supports: PDF, DOC, DOCX, TXT</p>
  </label>
</div>
</div>

  {/ * Status Bar */}
  <div className="flex items-center justify-between px-3 py-1.5 bg-[#007acc] text-white text-xs">
    <span>{uploadTargetFolder ? `Uploading to: ${uploadTargetFolder}` : 'Select a folder from the tree'}</span>...
    <span>13 weeks available</span>
  </div>
</div>
</DialogContent>
</Dialog>

{/ * Rename File Dialog */}
<Dialog open={renameFileId !== null} onOpenChange={({open} => !open && setRenameFileId(null)}}>
  <DialogContent className="max-w-md text-white">
    <DialogHeader>
      <DialogTitle className="flex items-center gap-2 text-sm">
        <Pencil className="h-4 w-4" />
        Rename File
      </DialogTitle>
    </DialogHeader>
    <div className="space-y-4 py-4">
      <Input
        value={renameFileName}
        onChange={({e} => { const v = e.target.value; startTransition(() => setRenameFileName(v)); })}
        placeholder="Enter new name..."
        className="bg-[#3c3c3c] border-[#5c5c5c] text-white"
        data-testid="input-rename-file"
        onKeyDown={({e} => {
          if (e.key === 'Enter' && renameFileName.trim() && renameFileId) {
            fetch(`/api/files/${renameFileId}`, {
              method: 'PATCH',
              headers: { 'Content-Type': 'application/json' },
              body: JSON.stringify({ displayName: renameFileName.trim() })
            }).then(() => {
              queryClient.invalidateQueries({ queryKey: ['/api/files'] });
              setRenameFileId(null);
              toast({ title: "File renamed" });
            }).catch(() => {
              toast({ title: "Failed to rename file", variant: "destructive" });
            });
          }
        })
      </Input>
    </div>
    <div className="flex justify-end gap-2">
      <Button
        variant="ghost"
        onClick={() => setRenameFileId(null)}
        className="text-white/70 hover:text-white hover:bg-white/10"
      > Cancel
    </Button>
      <Button
        onClick={() => {
          if (renameFileName.trim() && renameFileId) {
            fetch(`/api/files/${renameFileId}`, {
              method: 'PATCH',
              headers: { 'Content-Type': 'application/json' },
              body: JSON.stringify({ displayName: renameFileName.trim() })
            }).then(() => {
              queryClient.invalidateQueries({ queryKey: ['/api/files'] });
              setRenameFileId(null);
              toast({ title: "File renamed" });
            }).catch(() => {
              toast({ title: "Failed to rename file", variant: "destructive" });
            });
          }
        })
        className="bg-blue-600 hover:bg-blue-500"
        data-testid="button-confirm-rename"
      > Rename
    </Button>
    </div>
  </DialogContent>
</Dialog>

{/ * Move File Dialog */}
<Dialog open={moveFileId !== null} onOpenChange={({open} => !open && setMoveFileId(null)}}>
  <DialogContent className="max-w-md text-white">
    <DialogHeader>
      <DialogTitle className="flex items-center gap-2 text-sm">
        <Folder className="h-4 w-4" />
        Move File to Folder
      </DialogTitle>
    </DialogHeader>
    <div className="space-y-2 py-4 max-h-[400px] overflow-y-auto">
      {(() => {
        const folders = allFiles
          .map(f => f.folder)
          .filter((f): f is string => !!f && f !== moveFileCurrentFolder)
          .filter((value, index, self) => self.indexOf(value) === index)
          .sort();

        return folders.map(folder => {

```

```

const parts = folder.split('-');
const weekNum = parts[1];
const course = parts[2]?.toUpperCase() || '';
const type = parts[3] || '';
const displayName = `Week ${weekNum} - ${course} - ${type.charAt(0).toUpperCase() + type.slice(1)}`;

return (
  <div
    key={folder}
    className="px-3 py-2 hover:bg-white/10 rounded cursor-pointer flex items-center gap-2"
    onClick={async () => {
      if (moveFileId) {
        try {
          await fetch(`/api/files/${moveFileId}`, {
            method: 'PATCH',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ folder })
          });
          queryClient.invalidateQueries({ queryKey: ['/api/files'] });
          setMoveFileId(null);
          toast({ title: "File moved successfully" });
        } catch (err) {
          toast({ title: "Failed to move file", variant: "destructive" });
        }
      }
    }}
  >
    <Folder className="h-4 w-4 text-yellow-500" />
    <span className="text-sm">{displayName}</span>
  </div>
);
});
})();
</div>
<div className="flex justify-end">
  <Button
    variant="ghost"
    onClick={() => setMoveFileId(null)}
    className="text-white/70 hover:text-white hover:bg-white/10"
  >
    Cancel
  </Button>
</div>
</DialogContent>
</Dialog>

/* Folder Context Menu */
{folderContextMenu && (
  <div
    className="fixed z-[9999] rounded-md shadow-lg py-1 min-w-[100px]"
    style={{ top: folderContextMenu.y, left: folderContextMenu.x }}
    onClick={(e) => e.stopPropagation()}
    data-testid="folder-context-menu"
  >
    <div className="px-3 py-1.5 text-xs text-white/50 border-b border-white/10">
      Create Course Folder
    </div>
    <button
      className="w-full px-3 py-2 text-left text-sm text-white/90 hover:bg-white/10 flex items-center gap-2"
      onClick={async () => {
        const parentFolder = folderContextMenu.parentFolder;
        try {
          const response = await fetch('/api/custom-folders', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ name: 'CPPA122', parentFolderId: parentFolder })
          });
          if (!response.ok) {
            console.error('Failed to save folder:', response.status);
            toast({ title: "Error", description: "Failed to save folder", variant: "destructive" });
            return;
          }
          const savedFolder = await response.json();
          console.log('Folder saved:', savedFolder);
          const folderId = `${parentFolder}-subfolder-cppa122-${savedFolder.id}`;
          setCustomFolders(prev => [...prev, { id: folderId, name: 'CPPA122', parent: parentFolder }]);
          toast({ title: "Folder created", description: "CPPA122 folder added" });
        } catch (err) {
          console.error('Failed to save folder to database:', err);
          toast({ title: "Error", description: "Failed to create folder", variant: "destructive" });
        }
        setFolderContextMenu(null);
      }}
      data-testid="button-new-folder-cppa122"
    >
      <Folder className="h-4 w-4 text-green-500" />
      CPA122
    </button>
    <button
      className="w-full px-3 py-2 text-left text-sm text-white/90 hover:bg-white/10 flex items-center gap-2"
      onClick={async () => {
        const parentFolder = folderContextMenu.parentFolder;
        try {
          const response = await fetch('/api/custom-folders', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ name: 'CFNF400', parentFolderId: parentFolder })
          });
          if (!response.ok) {
            console.error('Failed to save folder:', response.status);
            toast({ title: "Error", description: "Failed to save folder", variant: "destructive" });
            return;
          }
          const savedFolder = await response.json();
          console.log('Folder saved:', savedFolder);
          const folderId = `${parentFolder}-subfolder-cfnf400-${savedFolder.id}`;
          setCustomFolders(prev => [...prev, { id: folderId, name: 'CFNF400', parent: parentFolder }]);
          toast({ title: "Folder created", description: "CFNF400 folder added" });
        }

```

```

    } catch (err) {
      console.error('Failed to save folder to database:', err);
      toast({ title: "Error", description: "Failed to create folder", variant: "destructive" });
    }
    setFolderContextMenu(null);
  })
  data-testid="button-new-folder-cfnf400"
>
<Folder className="h-4 w-4 text-blue-500" />
CFNF400
</button>
<button
  className="w-full px-3 py-2 text-left text-sm text-white/90 hover:bg-white/10 flex items-center gap-2"
  onClick={async () => {
    const parentFolder = folderContextMenu.parentFolder;
    try {
      const response = await fetch('/api/custom-folders', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ name: 'CASL101', parentFolderId: parentFolder })
      });
      if (!response.ok) {
        console.error('Failed to save folder:', response.status);
        toast({ title: "Error", description: "Failed to save folder", variant: "destructive" });
        return;
      }
      const savedFolder = await response.json();
      console.log('Folder saved:', savedFolder);
      const folderId = `${parentFolder}-subfolder-casl101-${savedFolder.id}`;
      setCustomFolders(prev => [...prev, { id: folderId, name: 'CASL101', parent: parentFolder }]);
      toast({ title: "Folder created", description: "CASL101 folder added" });
    } catch (err) {
      console.error('Failed to save folder to database:', err);
      toast({ title: "Error", description: "Failed to create folder", variant: "destructive" });
    }
    setFolderContextMenu(null);
  }}
  data-testid="button-new-folder-casl101"
>
<Folder className="h-4 w-4 text-purple-500" />
CASL101
</button>
<div className="border-t border-white/10 my-1" />
<button
  className="w-full px-3 py-2 text-left text-sm text-white/90 hover:bg-white/10 flex items-center gap-2"
  onClick={() => {
    setNewFolderParent(folderContextMenu.parentFolder);
    setNewFolderName('');
    setNewFolderDialogOpen(true);
    setFolderContextMenu(null);
  }}
  data-testid="button-new-folder-custom"
>
<FolderPlus className="h-4 w-4" />
Custom Folder...
</button>
</div>
))

/* New Folder Dialog */
<Dialog open={newFolderDialogOpen} onOpenChange={setNewFolderDialogOpen}>
  <DialogContent className="text-white [&button]:text-white">
    <DialogHeader>
      <DialogTitle>Create New Folder</DialogTitle>
    </DialogHeader>
    <div className="space-y-4 py-2">
      <p className="text-sm text-white/60">
        Creating folder in: <span className="text-white/90 font-medium">{newFolderParent}</span>
      </p>
      <div>
        <Label htmlFor="folderName" className="text-white/80">Folder Name</Label>
        <Input
          id="folderName"
          value={newFolderName}
          onChange={(e) => { const v = e.target.value; startTransition(() => setNewFolderName(v)); }}
          placeholder="Enter folder name"
          className="bg-[#2a2d2e] text-white mt-1"
          onKeyDown={(e) => e.key === 'Enter' && handleCreateFolder()}
          autoFocus
          data-testid="input-new-folder-name"
        />
      </div>
    </div>
    <div className="flex justify-end gap-2">
      <Button
        variant="ghost"
        onClick={() => setNewFolderDialogOpen(false)}
        className="text-white/70 hover:text-white hover:bg-white/10"
      >
        Cancel
      </Button>
      <Button
        onClick={handleCreateFolder}
        disabled={!newFolderName.trim()}
        className="bg-blue-600 hover:bg-blue-700 text-white"
        data-testid="button-create-folder"
      >
        Create
      </Button>
    </div>
  </DialogContent>
</Dialog>

<div className="flex flex-1 overflow-visible relative z-10" style={{ backgroundColor: 'transparent' }}>
  /* Constant fade overlay - disabled for solid black background */
  <div
    className="absolute inset-0 pointer-events-none"
    style={{ background: 'transparent', zIndex: 0 }}
  >

```

```

/>
/* Night overlay - dims background based on Toronto sunrise/sunset - disabled when color overlay is active */
{!colorSettings.mainBackgroundOverlay && (
  <div
    className={`absolute inset-0 pointer-events-none transition-opacity duration-1000 ${isNighttime ? 'opacit...
    style={{ background: 'linear-gradient(to bottom, rgba(10, 15, 30, 0.6) 0%, rgba(5, 10, 25, 0.7) 100%)', z...
  />
)}
{isTodayExpanded && (
  <div
    className="today-backdrop"
    onClick={() => setIsTodayExpanded(false)}
  />
)}
)

/* School Logo - Fixed top left, customizable via school settings */
<div className="fixed flex items-start" data-tpo data-tpo-opacity="1" style={{ left: '23px', top: `${8 + d2lTic...
  <img src={schoolData.schoolLogo || changSchoolLogo} alt={schoolData.schoolName || "The Chang School"} style={...
  <div style={{ width: '1.5px', height: '28px', backgroundColor: 'rgba(255,255,255,0.45)', borderRadius: '1px',...
  <img src={profilePhotoUrl || profilePhoto} alt="Profile" style={{ width: '30px', height: '30px', borderRadius...
  <div className="flex flex-col">
    <span className="text-white font-bold text-[11.5px] leading-tight">Schedule for {profileData.firstName}{pro...
    <span className="text-white text-[10px] leading-tight" style={{ fontWeight: 300 }}>{schoolData.schoolName |...
  </div>
</div>

/* Coming Up label removed */

/* Next Task Countdown - center, where Chang School logo used to be */
{(() => {
  const now = new Date();
  const nowDate = new Date(now.getFullYear(), now.getMonth(), now.getDate());
  const upcoming = allTasks
    .filter(t => {
      if (!t.dueDate || t.isCompleted) return false;
      if (isCASL101Finished(t)) return false;
      const dd = new Date(t.dueDate);
      const ddOnly = new Date(dd.getFullYear(), dd.getMonth(), dd.getDate());
      if (ddOnly.getTime() < nowDate.getTime()) return false;
      return true;
    })
    .sort((a, b) => {
      if (a.flagged && !b.flagged) return -1;
      if (!a.flagged && b.flagged) return 1;
      return new Date(a.dueDate!).getTime() - new Date(b.dueDate!).getTime();
    });
  const overdue = upcoming.length === 0 ? allTasks
    .filter(t => {
      if (!t.dueDate || t.isCompleted) return false;
      if (isCASL101Finished(t)) return false;
      return true;
    })
    .sort((a, b) => new Date(b.dueDate!).getTime() - new Date(a.dueDate!).getTime()) : [];
  const next = upcoming[0] || overdue[0];
  if (!next) {
    return (
      <div
        className="font-raleway"
        data-tpo data-tpo-opacity="1"
        style={{
          position: 'fixed',
          top: `${23 + d2lTickerHeight}px`,
          zIndex: 101,
          opacity: isTopPillOpen ? 0 : 1,
          transition: 'opacity 0.4s ease-in-out',
          pointerEvents: 'none',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          gap: '6px',
          left: '-12px',
          right: '0px',
        }}
        data-testid="next-task-countdown"
      >
      <div style={{ position: 'relative', display: 'flex', flexDirection: 'row', alignItems: 'center' }}>
        <div style={{ display: 'flex', flexDirection: 'row', alignItems: 'center', background: 'linear-gradi...
          /* Digital clock inside pill */
          <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'center', justifyContent: 'cent...
            <span className="text-white" style={{ fontSize: '12px', fontWeight: '400', fontVariantNumeric: 't...
            <span className="text-white" style={{ fontSize: '9px', fontWeight: '400', textTransform: 'upperca...
          </div>
          /* Analogue clock in countdown pill */
          <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'center', marginRight: '10px', ...
            {(() => { const sz = 50; const center = sz / 2; const dotR = 1.3; const hourDotDist = 21; const h...
          </div>
          <div style={{ display: 'flex', flexDirection: 'column', gap: '6px', marginLeft: '2px' }}>
            <div style={{ display: 'flex', alignItems: 'center', gap: '4px', whiteSpace: 'nowrap' }} data-tes...
              <span style={{ color: '#ffffff', fontSize: '9.25px', fontWeight: 400, letterSpacing: '0.3px', t...
            </div>
          </div>
        </div>
      </div>
    );
  }
  const dueDate = new Date(next.dueDate!);
  const dueDateOnly = new Date(dueDate.getFullYear(), dueDate.getMonth(), dueDate.getDate());
  const diffDays = Math.round((dueDateOnly.getTime() - nowDate.getTime()) / (1000 * 60 * 60 * 24));
  const courseForNext = next.courseName ? coursesData.courses.find(c => next.courseName!.includes(c.name.split(...
  const nextPrep = upcoming
    .filter(t => t.startDate)
    .map(t => {
      const sd = new Date(t.startDate!);
      const sdOnly = new Date(sd.getFullYear(), sd.getMonth(), sd.getDate());
      return { ...t, startDateOnly: sdOnly, prepDaysLeft: Math.round((sdOnly.getTime() - nowDate.getTime()) / (...
    })
}

```



```

    {prepDaysText === 'today' ? (<><span data-countdown-bullet={`prep-${nextPrep.id}`}` style={{ color...
    </div>
  })
  }(() => {
    const prepDaysNum2 = prepDaysText === 'today' ? 0 : prepDaysText === 'now' ? -1 : Number(prepDaysTe...
    const showPrepFirst2 = prepDaysText && nextPrep && prepDaysNum2 < diffDays && diffDays > 0;
    const isSameTask2 = nextPrep && next.id === nextPrep.id;
    if (!showPrepFirst2 || isSameTask2) return null;
    const isDueZero2 = diffDays === 0;
    return (
      <div
        className=""
        style={{ display: 'flex', alignItems: 'center', gap: '4px', pointerEvents: 'auto', cursor: 'poi...
        onMouseEnter={() => setHoveredCountdownTaskIdDebounce(next.id)}
        onMouseLeave={() => setHoveredCountdownTaskIdDebounce(null)}
        data-testid="countdown-next-task-number-after"
      >
        {next.flagged ? (
          <Flag className="h-[10px] w-[10px] text-red-400 fill-red-400 flex-shrink-0" />
        ) : (
          <span data-countdown-bullet={next.id} style={{ color: isDueZero2 ? '#dc2626' : 'ffffff', fon...
        )}
        {isDueZero2 ? (
          <>
            <span style={{ color: 'ffffff', fontSize: '9.25px', fontWeight: 700, letterSpacing: '0.3px...
            <span style={{ color: 'ffffff', fontSize: '9.25px', fontWeight: next.type === 'class' ? 70...
          </>
        ) : (
          <>
            <span style={{ color: 'ffffff', fontSize: '9.25px', fontWeight: 400, letterSpacing: '0.3px...
            <span data-countdown-badger style={{ backgroundColor: diffDays >= 3 ? 'rgb(0, 180, 0)' : dif...
            <span style={{ color: 'ffffff', fontSize: '9.25px', fontWeight: 400, lineHeight: 1, letter...
            <span style={{ color: 'ffffff', fontSize: '9.25px', fontWeight: 400, letterSpacing: '0.3px...
          </>
        )}
      </div>
    );
  })()
  }flaggedSoon.map(ft => (
    <div
      key={`flag-${ft.id}`}
      style={{ display: 'flex', alignItems: 'center', gap: '4px', pointerEvents: 'auto', cursor: 'point...
      onMouseEnter={() => setHoveredCountdownTaskIdDebounce(ft.id)}
      onMouseLeave={() => setHoveredCountdownTaskIdDebounce(null)}
      data-testid="countdown-flagged-${ft.id}"
    >
      <Flag className="h-[10px] w-[10px] text-red-400 fill-red-400 flex-shrink-0" />
      {ft.flagDiffDays === 0 ? (
        <>
          <span style={{ color: 'ffffff', fontSize: '9.25px', fontWeight: 700, letterSpacing: '0.3px' ...
          <span style={{ color: 'ffffff', fontSize: '9.25px', fontWeight: ft.type === 'class' ? 700 : ...
        </>
      ) : (
        <>
          <span style={{ color: 'ffffff', fontSize: '9.25px', fontWeight: 400, letterSpacing: '0.3px' ...
          <span data-countdown-badger style={{ backgroundColor: ft.flagDiffDays >= 3 ? 'rgb(0, 180, 0)' ...
          <span style={{ color: 'ffffff', fontSize: '9.25px', fontWeight: 400, lineHeight: 1, letterSp...
          <span style={{ color: 'ffffff', fontSize: '9.25px', fontWeight: 400, letterSpacing: '0.3px' ...
        </>
      )}
    </div>
  )}
</div>
</div>
</div>
</div>
);
})();

```

/* Top Pill - Slide up/down container for toolbar buttons */

```

<div style={{ position: 'fixed', top: `${d2lTickerHeight}px`, left: 0, right: 0, bottom: 0, zIndex: editingTask...
  <div
    ref={topPillRef}
    id="top-pill-container"
    style={{
      position: 'absolute',
      zIndex: 110,
      left: '22px',
      right: '22px',
      pointerEvents: editingTask ? 'none' : 'auto',
      transform: `translateY(${isTopPillOpen ? '16px' : '-83px'})`,
      transition: topPillMounted ? 'transform 0.4s cubic-bezier(0.4, 0, 0.2, 1)' : 'none',
      animation: (!isTopPillOpen && topPillMounted) ? 'top-pill-container-nudge 6s ease-in-out 1s infinite' : 'no...
      top: '0px',
      height: '57px',
      padding: '1px',
      padding: '1px',
      padding: '1px',
      display: 'flex',
      align: 'center',
      justify: 'center',
      overflow: 'visible',
    }}
    onTouchStart={() => {
      if (topPillTimeoutRef.current) clearTimeout(topPillTimeoutRef.current);
    }}
    onTouchEnd={() => {
      if (isHamburgerOpenRef.current) return;
      if (topPillTimeoutRef.current) clearTimeout(topPillTimeoutRef.current);
      topPillTimeoutRef.current = setTimeout(() => {
        if (isHamburgerOpenRef.current) return;
        closeTopPill();
      }, 6000);
    }}
    onMouseEnter={() => {
      if (topPillTimeoutRef.current) clearTimeout(topPillTimeoutRef.current);
    }}
    onMouseLeave={() => {
      if (isHamburgerOpenRef.current) return;
    }}
  </div>

```

```

    if (topPillTimeoutRef.current) clearTimeout(topPillTimeoutRef.current);
    const delay = 'ontouchstart' in window ? 4000 : 1800;
    topPillTimeoutRef.current = setTimeout(() => {
      if (isHamburgerOpenRef.current) return;
      closeTopPill();
    }, delay);
  },
});
}
}

/* Glass pill background */
<div style={{
  position: 'absolute',
  inset: 0,
  background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',
  backdropFilter: 'blur(12px)',
  WebkitBackdropFilter: 'blur(12px)',
  borderRadius: '12px',
  border: '1.5px solid rgba(255,255,255,0.35)',
  borderTop: '1.5px solid rgba(255,255,255,0.55)',
  boxShadow: '0 4px 24px rgba(0,0,0,0.15), inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,0,0.05)',
  pointerEvents: 'none',
}} />

/* Down arrow placeholder - actual tab moved outside pill container */

/* Icon buttons and task buttons with adjustable spacing */
<div className="flex items-center flex-nowrap [&*>]:flex-shrink-0" style={{ gap: `${Math.max(0, (blinkSetting...
  /* Hamburger Menu */
  <DropdownMenu open={isHamburgerOpen} onOpenChange={open} => { if (hamburgerCloseTimer.current) { clearTime...
    <DropdownMenuTrigger asChild>
      <div
        className="pill-button-hover"
        style={{
          marginTop: '4px',
          width: '44px',
          height: '44px',
          borderRadius: '50%',
          background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',
          border: '1.5px solid rgba(255,255,255,0.35)',
          boxShadow: '0 4px 24px rgba(0,0,0,0.15), inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(...
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          cursor: 'pointer'
        }}
        data-testid="button-hamburger-menu"
        title="Menu"
        onClick={e} => { e.preventDefault(); e.stopPropagation(); startTransition(() => setIsHamburgerOpen(p...
        onMouseEnter={() => { if (!('ontouchstart' in window)) setIsHamburgerOpen(true); }}
      >
        <Menu className="h-[20px] w-[20px]" strokeWidth={3} style={{ color: '#ffffff', marginTop: '2px', filt...
      </div>
    </DropdownMenuTrigger>
    <DropdownMenuContent align="start" noAnimation onMouseLeave={() => { if (!('ontouchstart' in window)) { h...
      <DropdownMenuItem data-testid="menu-item-profile" className="text-xs" onClick={() => startTransition(()...
        <User className="h-3.5 w-3.5 mr-2" />
        Profile Settings
      </DropdownMenuItem>
      <DropdownMenuItem data-testid="menu-item-school" className="text-xs" onClick={() => startTransition(() ...
        <GraduationCap className="h-3.5 w-3.5 mr-2" />
        School Settings
      </DropdownMenuItem>
      <DropdownMenuItem data-testid="menu-item-settings" className="text-xs" onClick={() => {
        startTransition(() => {
          setIsQuickAddOpen(false);
          setOriginalColorSettings({...colorSettings});
          setOriginalBlinkSettings({...blinkSettings});
          setIsSettingsDialogOpen(true);
        });
      }}>
        <Settings className="h-3.5 w-3.5 mr-2" />
        Calendar Settings
      </DropdownMenuItem>
    </DropdownMenuContent>
  </DropdownMenu>

  /* Undo Button */
  {undoStack.length > 0 ? (
    <div
      style={{
        width: '44px', height: '44px', marginTop: '4px', zIndex: 100, borderRadius: '50%',
        background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',

        border: '1.5px solid rgba(255,255,255,0.35)',
        boxShadow: '0 4px 24px rgba(0,0,0,0.15), inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,...
        display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: 'pointer'
      }}
      className="pill-button-hover"
      onClick={() => { triggerButtonGlow('undo'); handleUndo(); }}
      data-testid="button-undo"
      title="Undo"
    >
      <Undo2 className="h-[18px] w-[18px] text-white" />
    </div>
  ) : (
    <div
      style={{
        marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%', zIndex: 100,
        background: 'linear-gradient(180deg, rgba(255,255,255,0.10) 0%, rgba(255,255,255,0.03) 100%)',

        border: '1.5px solid rgba(255,255,255,0.2)',
        boxShadow: '0 4px 24px rgba(0,0,0,0.15), inset 0 1px 0 rgba(255,255,255,0.15), inset 0 -1px 0 rgba(0,...
        display: 'flex', alignItems: 'center', justifyContent: 'center', opacity: 0.5
      }}
      data-testid="button-undo-disabled"
      title="Undo"
    >
      <Undo2 className="h-[18px] w-[18px] text-white" />
    </div>
  )
}

```



```

    })
    /* Redo Button */
    {redoStack.length > 0 ? (
      <div
        style={{
          width: '44px', height: '44px', marginTop: '4px', zIndex: 100, borderRadius: '50%',
          background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',

          border: '1.5px solid rgba(255,255,255,0.35)',
          boxShadow: '0 4px 24px rgba(0,0,0,0.15)', inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,...
          display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: 'pointer'
        }}
        className="pill-button-hover"
        onClick={() => { triggerButtonGlow('redo'); handleRedo(); }}
        data-testid="button-redo"
        title="Redo"
      >
        <Redo2 className="h-[18px] w-[18px] text-white" />
      </div>
    ) : (
      <div
        style={{
          marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%', zIndex: 100,
          background: 'linear-gradient(180deg, rgba(255,255,255,0.10) 0%, rgba(255,255,255,0.03) 100%)',

          border: '1.5px solid rgba(255,255,255,0.2)',
          boxShadow: '0 4px 24px rgba(0,0,0,0.15)', inset 0 1px 0 rgba(255,255,255,0.15), inset 0 -1px 0 rgba(0,...
          display: 'flex', alignItems: 'center', justifyContent: 'center', opacity: 0.5
        }}
        data-testid="button-redo-disabled"
        title="Redo"
      >
        <Redo2 className="h-[18px] w-[18px] text-white" />
      </div>
    )
  })

  /* Courses Button */
  <div className="pill-button-hover" style={{
    marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
    background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',
    border: '1.5px solid rgba(255,255,255,0.35)',
    boxShadow: '0 4px 24px rgba(0,0,0,0.15)', inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,0,...
    display: 'flex', alignItems: 'center', justifyContent: 'center'
  }}>
    <Button
      size="icon"
      variant="ghost"
      className="!h-[42px] !w-[42px] !min-h-[42px] !min-w-[42px] !p-0 aspect-square hover:opacity-80 rounded-...
      style={{ background: 'transparent' }}
      data-testid="button-courses-pill"
      title="Semesters and Courses"
      onClick={() => {
        startTransition(() => {
          setDraftPriorityBoth({ ...coursePlayPriority });
          setSchoolCoursesOpenSource('pill');
          startTransition(() => setIsSchoolCoursesDialogOpen(true));
        });
      }}
    >
      <BookOpen className="text-white" style={{ height: '20px', width: '20px' }} />
    </Button>
  </div>

  /* Graduation Hat - Swapped with Completed Tasks */
  <div className="pill-button-hover" style={{
    marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
    background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',

    border: '1.5px solid rgba(255,255,255,0.35)',
    boxShadow: '0 4px 24px rgba(0,0,0,0.15)', inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,0,...
    display: 'flex', alignItems: 'center', justifyContent: 'center'
  }}>
    <Button
      size="icon"
      variant="ghost"
      className="!h-[42px] !w-[42px] !min-h-[42px] !min-w-[42px] !p-0 aspect-square hover:opacity-80 rounded-...
      style={{ background: 'transparent' }}
      data-testid="button-settings-panel"
      title="Degree Tracking"
      onClick={() => {
        triggerButtonGlow('settings');
        startTransition(() => setIsSettingsPanelOpen(true));
        const currentSemCodes = semesterSettings ? [1,2,3].map(i => (semesterSettings as any)[`course${i}`]Code...
        const activeCourses = coursesData.courses.filter(c => {
          const code = c.name.split(' - ')[0];
          return c.name.trim() && currentSemCodes.includes(code);
        });
        const uncheckedAas = activeCourses.filter(c => {
          const code = c.name.split(' - ')[0];
          return !aasSentStatus[code];
        });
        if (uncheckedAas.length > 0) {
          const lastShown = localStorage.getItem('aasReminderLastShown');
          const now = Date.now();
          if (!lastShown || now - parseInt(lastShown) > 24 * 60 * 60 * 1000) {
            setShowAasReminder(true);
            localStorage.setItem('aasReminderLastShown', String(now));
          }
        }
      }}
    >
      <GraduationCap className="text-white" style={{ height: '22px', width: '22px' }} />
    </Button>
  </div>

  /* Scholarships Button */
  <div className="pill-button-hover" style={{

```

```

marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',

border: '1.5px solid rgba(255,255,255,0.35)',
boxShadow: '0 4px 24px rgba(0,0,0,0.15), inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,0,...
display: 'flex', alignItems: 'center', justifyContent: 'center'
})>
<Button
  size="icon"
  variant="ghost"
  className="!h-[42px] !w-[42px] !min-h-[42px] !min-w-[42px] !p-0 aspect-square hover:opacity-80 rounded-...
  style={{ background: 'transparent' }}
  data-testid="button-scholarships"
  title="Scholarships"
  onClick={() => {
    triggerButtonGlow('scholarships');
    setIsScholarshipsOpen(true);
  }}
>
  <Award className="text-white" style={{ height: '22px', width: '22px' }} />
</Button>
</div>

{/* Key Contacts Button */}
<div className="pill-button-hover" style={{
  marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
  background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',

  border: '1.5px solid rgba(255,255,255,0.35)',
  boxShadow: '0 4px 24px rgba(0,0,0,0.15), inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,0,...
  display: 'flex', alignItems: 'center', justifyContent: 'center'
})>
<Button
  size="icon"
  variant="ghost"
  className="!h-[42px] !w-[42px] !min-h-[42px] !min-w-[42px] !p-0 aspect-square hover:opacity-80 rounded-...
  style={{ background: 'transparent' }}
  data-testid="button-key-contacts"
  title="Contacts"
  onClick={() => {
    triggerButtonGlow('keycontacts');
    setIsKeyContactsOpen(true);
  }}
>
  <Contact className="text-white" style={{ height: '22px', width: '22px' }} />
</Button>
</div>

{/* Feedback Button */}
<div className="pill-button-hover" style={{
  marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
  background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',
  border: '1.5px solid rgba(255,255,255,0.35)',
  boxShadow: '0 4px 24px rgba(0,0,0,0.15), inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,0,...
  display: 'flex', alignItems: 'center', justifyContent: 'center'
})>
<Button
  size="icon"
  variant="ghost"
  className="!h-[42px] !w-[42px] !min-h-[42px] !min-w-[42px] !p-0 aspect-square hover:opacity-80 rounded-...
  style={{ background: 'transparent' }}
  data-testid="button-feedback"
  title="Send Feedback to Agent"
  onClick={() => setIsFeedbackOpen(true)}
>
  <MessageSquare className="text-white" style={{ height: '20px', width: '20px' }} />
</Button>
</div>

{/* Spotify Button */}
<div className="pill-button-hover" style={{
  marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
  background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',
  border: '1.5px solid rgba(255,255,255,0.35)',
  boxShadow: '0 4px 24px rgba(0,0,0,0.15), inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,0,...
  display: 'flex', alignItems: 'center', justifyContent: 'center'
})>
<Button
  size="icon"
  variant="ghost"
  className="!h-[42px] !w-[42px] !min-h-[42px] !min-w-[42px] !p-0 aspect-square hover:opacity-80 rounded-...
  style={{ background: 'transparent' }}
  data-testid="button-spotify-pill"
  title="Spotify Player"
  onClick={() => {
    triggerButtonGlow('spotify');
    const params = new URLSearchParams(window.location.search);
    const authParam = params.get('auth');
    window.location.href = '/spotify' + (authParam ? `?auth=${authParam}` : '');
  }}
>
  <Music2 className="text-white" style={{ height: '20px', width: '20px' }} />
</Button>
</div>

{/* Radio Dialog */}
<Dialog open={isRadioDialogOpen} onOpenChange={setIsRadioDialogOpen}>
  <DialogContent className="max-w-[260px] text-[10px] text-white [&_*]:text-white [&_label]:text-white [&_i...
  <div className="flex items-center justify-between px-4 py-3 border-b border-white/40 flex-shrink-0 roun...
    <div className="flex items-center gap-2">
      <Radio className="text-white" style={{ width: '15px', height: '15px' }} />
      <h2 className="font-normal text-white" style={{ fontFamily: 'Avenir, 'Avenir Next', -apple-system, ...
        RADIO CONTROLS
      </h2>
    </div>
    <button
      onClick={() => setIsRadioDialogOpen(false)}
      className="text-white hover:text-white/80 transition-colors p-1"
    >

```

```

    data-testid="button-close-radio"
  >
    <X className="h-4 w-4" />
  </button>
</div>
<div className="flex flex-col gap-3 p-4">
  { /* Speaker Selection */ }
  <div className="flex flex-col gap-1">
    <Label className="text-white text-[10px]">Speaker</Label>
    <Select value={selectedSpeaker} onChange={setSelectedSpeaker}>
      <SelectTrigger data-testid="select-speaker" className="h-8 text-[10px] bg-black/50 border-white/3...">
        <SelectValue />
      </SelectTrigger>
      <SelectContent className="bg-gray-900 max-h-[300px]">
        <SelectItem value="media_player.byhome">Apartment</SelectItem>
        <SelectItem value="media_player.cat_wash">Cat Wash</SelectItem>
        <SelectItem value="media_player.cat_wr">Cat Washroom Speakers</SelectItem>
        <SelectItem value="media_player.bathroom_speaker">Nest (Cat Washroom)</SelectItem>
        <SelectItem value="media_player.echo_cat_left_am">Cat Washroom Left</SelectItem>
        <SelectItem value="media_player.echo_cat_right_am">Cat Washroom Right</SelectItem>
        <SelectItem value="media_player.echo_cat_washroom_middle">Cat Washroom Middle</SelectItem>
        <SelectItem value="media_player.echo_closet_am">Closet</SelectItem>
        <SelectItem value="media_player.echo_lr_couch_r_am">Echo Corner</SelectItem>
        <SelectItem value="media_player.echo_hallway_entrance_am">Hallway Entrance</SelectItem>
        <SelectItem value="media_player.echo_king_l_am">King Left</SelectItem>
        <SelectItem value="media_player.echo_king_r_am">King Right</SelectItem>
        <SelectItem value="media_player.echo_king_tv_am">King TV</SelectItem>
        <SelectItem value="media_player.echo_kitchen_cupboards_left_am">Kitchen Cupboards Left</SelectItem>
        <SelectItem value="media_player.echo_kitchen_cupboards_r_am">Kitchen Cupboards Right</SelectItem>
        <SelectItem value="media_player.echo_kitchen_fridge_am">Kitchen Fridge</SelectItem>
        <SelectItem value="media_player.echo_kitchen_hutch_am">Kitchen Hutch</SelectItem>
        <SelectItem value="media_player.echo_kitchen_island_corner_am">Kitchen Island Corner</SelectItem>
        <SelectItem value="media_player.echo_kitchen_studio_black_am">Kitchen Studio Black</SelectItem>
        <SelectItem value="media_player.echo_lr_couch_l_am">Living Room Couch Left</SelectItem>
        <SelectItem value="media_player.echo_lr_hub_am">Living Room Hub</SelectItem>
        <SelectItem value="media_player.echo_lr_studio_white_am">Living Room Studio White</SelectItem>
        <SelectItem value="media_player.echo_lr_tv_shelf_am">Living Room TV Shelf</SelectItem>
        <SelectItem value="media_player.echo_queen_balcony_am">Queen Balcony</SelectItem>
        <SelectItem value="media_player.echo_queen_bed_l_am">Queen Bed Left</SelectItem>
        <SelectItem value="media_player.echo_queen_bed_r_am">Queen Bed Right</SelectItem>
        <SelectItem value="media_player.echo_show_pug_am">Echo Show Pug</SelectItem>
        <SelectItem value="media_player.everywhere_2">Everywhere</SelectItem>
        <SelectItem value="media_player.hallway">Hallway</SelectItem>
        <SelectItem value="media_player.king_bedroom">King Bedroom</SelectItem>
        <SelectItem value="media_player.queen_bedroom">Queen Bedroom</SelectItem>
      </SelectContent>
    </Select>
  </div>
  { /* Play/Stop Buttons */ }
  <div className="flex flex-col gap-1.5 items-center mt-2">
    <Button
      variant="outline"
      size="sm"
      className="h-6 w-[155px] text-[9px] border-blue-500 text-blue-400 hover:text-blue-300 hover:borde..."
      onClick={async () => {
        try {
          const response = await fetch('/api/media/play-radio', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
              stationId: 'CHUM FM',
              entityId: selectedSpeaker
            })
          });
          if (response.ok) {
            const speakerName = selectedSpeaker.replace('media_player.', '').replace(/_/g, ' ');
            toast({ title: "Playing CHUM FM", description: `on ${speakerName}` });
          } else {
            toast({ title: "Failed to play radio", variant: "destructive" });
          }
        } catch (error) {
          toast({ title: "Error playing radio", variant: "destructive" });
        }
      }}
      data-testid="button-play-chumfm"
    >
      <Play style={{ width: 12, height: 12 }} className="mr-1" />
      Play CHUM FM (select)
    </Button>
    <Button
      variant="outline"
      size="sm"
      className="h-6 w-[155px] text-[9px] border-blue-500 text-blue-400 hover:text-blue-300 hover:borde..."
      onClick={async () => {
        try {
          const response = await fetch('/api/media/play-radio-all', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ stationId: 'CHUM FM' })
          });
          if (response.ok) {
            toast({ title: "Playing CHUM FM", description: "on all devices" });
          } else {
            toast({ title: "Failed to play radio", variant: "destructive" });
          }
        } catch (error) {
          toast({ title: "Error playing radio", variant: "destructive" });
        }
      }}
      data-testid="button-play-chumfm-all"
    >
      <Play style={{ width: 12, height: 12 }} className="mr-1" />
      Play CHUM FM (on all)
    </Button>
    <Button
      variant="destructive"
      size="sm"

```

```

        className="h-6 w-[155px] text-[9px] bg-[rgb(255,0,0)] hover:bg-[rgb(220,0,0)] border-[rgb(255,0,0)]"
        onClick={async () => {
            try {
                const response = await fetch('/api/media/stop-radio', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                });
                if (response.ok) {
                    toast({ title: "Radio stopped" });
                }
            } catch (error) {
                toast({ title: "Error stopping radio", variant: "destructive" });
            }
        }}
        data-testid="button-stop-radio"
    >
    <Square style={{ width: 10, height: 10 }} className="mr-1 fill-current" />
    Stop
  </Button>
</div>

  { /* Volume Controls - All Speakers */ }
  <div className="flex flex-col gap-1.5 mt-1">
    <Label className="text-white text-[10px]">Volume (All Speakers):</Label>
    <div className="flex items-center gap-1">
      <button
        className="text-white hover:text-white/70 text-base px-1"
        onClick={async () => {
            try {
                const res = await fetch('/api/media/volume-all', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ direction: 'down' })
                });
                const data = await res.json();
                if (data.newVolume !== undefined) {
                    setRadioVolume(data.newVolume);
                    toast({ title: `Volume set to ${data.newVolume}% on all speakers` });
                }
            } catch (error) {
                toast({ title: "Error adjusting volume", variant: "destructive" });
            }
        }}
        data-testid="button-volume-down-all"
      >
        "
      </button>
      <div className="flex-1 h-1 bg-white/20 rounded-full overflow-hidden">
        <div
          className="h-full bg-green-500 rounded-full transition-opacity duration-200"
          style={{ width: `${radioVolume}%` }}
        />
      </div>
      <button
        className="text-white hover:text-white/70 text-base px-1"
        onClick={async () => {
            try {
                const res = await fetch('/api/media/volume-all', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ direction: 'up' })
                });
                const data = await res.json();
                if (data.newVolume !== undefined) {
                    setRadioVolume(data.newVolume);
                    toast({ title: `Volume set to ${data.newVolume}% on all speakers` });
                }
            } catch (error) {
                toast({ title: "Error adjusting volume", variant: "destructive" });
            }
        }}
        data-testid="button-volume-up-all"
      >
        +
      </button>
      <button
        className="text-white/70 hover:text-white text-[9px] px-1"
        onClick={async () => {
            try {
                const res = await fetch('/api/media/volume-all', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ level: 0 })
                });
                const data = await res.json();
                if (data.newVolume !== undefined) {
                    setRadioVolume(data.newVolume);
                    toast({ title: "All speakers muted" });
                }
            } catch (error) {
                toast({ title: "Error muting", variant: "destructive" });
            }
        }}
        data-testid="button-mute-all"
      >
        mute
      </button>
    </div>
  </div>
</DialogContent>
</Dialog>

  { /* Return-from-break reading prompt (dev only) */ }
  { isAdmin && window.location.hostname === 'localhost' && (
    <Dialog open={showReturnReadingPrompt && returnReadingFiles.length > 0} onOpenChange={setShowReturnReadin...
      <DialogContent className="max-w-[340px] text-[11px] text-white p-0 [>button.absolute]:hidden [&_*]:tex...
        <div className="flex items-center justify-between px-4 py-3 border-b border-white/40 rounded-t-lg" st...

```

```

<div className="flex items-center gap-2">
  <BookOpen className="h-3.5 w-3.5 text-blue-400" />
  <h2 className="text-xs font-normal" style={{ fontFamily: "Avenir, 'Avenir Next', -apple-system, s...
    WELCOME BACK
  </h2>
</div>
<button
  onClick={() => setShowReturnReadingPrompt(false)}
  className="text-white hover:text-white/80 transition-colors p-1"
  data-testid="button-close-return-prompt"
>
  <X className="h-3 w-3" />
</button>
</div>
<div className="px-4 py-3 space-y-3">
  <p className="text-[10px] text-white/80">
    {(() => {
      const anyPartial = returnReadingFiles.some(f => (f.lastChunkIndex != null && f.lastChunkIndex >...
      return anyPartial
        ? "Resume where you left off?"
        : "Play your weekly Module or Reading material?";
    })()}
  </p>
  <Select
    value={selectedReturnFile?.toString() || ''}
    onValueChange={(v) => setSelectedReturnFile(parseInt(v, 10))}
  >
    <SelectTrigger className="h-7 text-[10px] bg-black/30 border-white/20" data-testid="select-return...
      <SelectValue placeholder="Select a PDF..." />
    </SelectTrigger>
    <SelectContent className="bg-gray-900 text-white max-h-[200px]">
      {returnReadingFiles.map((f) => (
        <SelectItem key={f.id} value={f.id.toString()} className="text-[10px] text-white">
          {f.name}
        </SelectItem>
      ))}
    </SelectContent>
  </Select>
  <div className="flex gap-2 justify-end pt-1">
    <Button
      variant="outline"
      size="sm"
      className="h-6 text-[9px] hover:bg-white/10"
      onClick={() => setShowReturnReadingPrompt(false)}
      data-testid="button-return-no"
    >
      Not now
    </Button>
    {(() => {
      const file = selectedReturnFile ? returnReadingFiles.find(f => f.id === selectedReturnFile) : n...
      const hasProgress = file && file.lastChunkIndex && file.lastChunkIndex > 0;
      return (
        <>
          <Button
            size="sm"
            className="h-6 text-[9px] bg-blue-600 hover:bg-blue-500 border-0"
            onClick={() => {
              if (file) {
                setShowReturnReadingPrompt(false);
                window.location.href = `/pdf-reader/${file.id}?autoplay=true&resumeChunk=0`;
              }
            }}
            data-testid="button-return-play-start"
          >
            Play from Start
          </Button>
          {hasProgress && (
            <Button
              size="sm"
              className="h-6 text-[9px] bg-green-600 hover:bg-green-500 border-0"
              onClick={() => {
                if (file) {
                  const resumeIdx = Math.max(0, (file.lastChunkIndex || 1) - 1);
                  setShowReturnReadingPrompt(false);
                  window.location.href = `/pdf-reader/${file.id}?autoplay=true&resumeChunk=${resumeId...
                }
              }}
              data-testid="button-return-play-resume"
            >
              Resume ({file.lastChunkIndex}{file.totalChunks ? `/${file.totalChunks}` : ''})
            </Button>
          )}
        </>
      )}
    </div>
  </div>
</div>
</DialogContent>
</Dialog>
))}

{/* Share Link Dialog */}
<Dialog open={isShareDialogOpen} onOpenChange={setIsShareDialogOpen}>
  <DialogContent className="max-w-[360px] text-[11px] text-white p-0 [>button.absolute]:hidden">
    <div className="flex items-center justify-between px-4 py-3 bg-black/30 border-b border-white/20">
      <div className="flex items-center gap-2">
        <Share2 className="h-3 w-3 text-white" />
        <h2 className="text-xs font-normal text-white" style={{ fontFamily: "Avenir, 'Avenir Next', -apple-...
          TEMPORARY SHARE LINK
        </h2>
      </div>
      <button
        onClick={() => setIsShareDialogOpen(false)}
        className="text-white hover:text-white/80 transition-colors p-1"
        data-testid="button-close-share"
      >
        <X className="h-5 w-5" />
      </button>
    </div>
  </DialogContent>
</Dialog>

```

```

</div>
<div className="p-4 space-y-4">
  <p className="text-white/80 text-[10px]">
    This link will expire 1 hour after someone uses it. They can view but not make permanent changes.
  </p>
  <div className="flex gap-2">
    <input
      type="text"
      value={shareLink}
      readOnly
      className="flex-1 px-2 py-1.5 text-[10px] bg-black/50 border border-white/30 rounded text-white"
      data-testid="input-share-link"
    </>
    <Button
      size="sm"
      variant="outline"
      className="text-white border-white/30 hover:bg-white/10"
      onClick={copyShareLink}
      data-testid="button-copy-share-link"
    >
      <Copy className="h-3 w-3" />
    </Button>
  </div>
</div>
</DialogContent>
</Dialog>

/* Completed Tasks Button - Swapped with Graduation Hat */
<div className="pill-button-hover" style={{
  marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
  background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',

  border: '1.5px solid rgba(255,255,255,0.35)',
  boxShadow: '0 4px 24px rgba(0,0,0,0.15)', inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,0,...
  display: 'flex', alignItems: 'center', justifyContent: 'center'
}}>
  <Button
    size="icon"
    variant="ghost"
    className="!h-[42px] !w-[42px] !min-h-[42px] !min-w-[42px] !p-0 aspect-square hover:opacity-80 rounded-...
    style={{ background: 'transparent' }}
    data-testid="button-completed-tasks"
    title="Completed"
    onClick={() => { triggerButtonGlow('completed'); setIsCompletedTasksOpen(true); }}
  >
    <CheckSquare className="h-[18px] w-[18px] text-white" />
  </Button>
</div>

/* Todo Button (swapped from tall pill) */
<div
  style={{
    marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
    background: hasUnackedReminders
      ? 'linear-gradient(180deg, #FF9494 0%, #FF0000 100%)'
      : 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',

    border: hasUnackedReminders ? '1.5px solid rgba(220, 38, 38, 0.8)' : '1.5px solid rgba(255,255,255,0.35)',
    boxShadow: hasUnackedReminders
      ? '0 0 12px rgba(220, 38, 38, 0.6), inset 0 1px 2px rgba(255,255,255,0.3), inset 0 -1px 2px rgba(0,0,...
      : '0 4px 24px rgba(0,0,0,0.15), inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,0,0.05)',
    display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: 'pointer'
  }}
  className={`pill-button-hover ${hasUnackedReminders ? 'animate-pill-reminder' : ''}`}
  onClick={() => { if (!isTodoFlyoutOpen) bringFlyoutToFront('todo'); setIsTodoFlyoutOpen(!isTodoFlyoutOpen...
  data-testid="honeycomb-todo-header"
  title="Reminders"
>
  <ListChecks style={{ color: 'white', strokeWidth: 2, height: '18px', width: '18px' }} />
</div>

/* Bell Button (moved from bottom pill) */
<div
  style={{
    marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
    background: isMuted ? 'linear-gradient(180deg, #FF9494 0%, #FF0000 100%)' : 'linear-gradient(180deg, rg...

    border: '1.5px solid rgba(255,255,255,0.35)',
    boxShadow: isMuted ? 'inset 0 1px 2px rgba(255,255,255,0.3), inset 0 -1px 2px rgba(0,0,0,0.3), 0 2px 4p...
    display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: 'pointer'
  }}
  className="pill-button-hover"
  onClick={() => { triggerButtonGlow('bell'); toggleMute(); }}
  data-testid="button-mute-toggle"
  title={isMuted ? 'Muted for ${Math.ceil((muteUntil! - Date.now()) / 60000)} min' : "Mute for 30 min"}
>
  {isMuted ? <BellOff className="h-[18px] w-[18px] text-white" /> : <Bell className="h-[18px] w-[18px] text...
</div>

/* Sync Button - Push & Pull combined */
<div
  style={{
    marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
    background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',

    border: '1.5px solid rgba(255,255,255,0.35)',
    boxShadow: '0 4px 24px rgba(0,0,0,0.15), inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,...
    display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: 'pointer'
  }}
  className="pill-button-hover"
  onClick={async () => {
    triggerButtonGlow('sync');
    toast({ title: "Syncing...", description: "Pushing settings, tasks & pulling events" });
    try {
      const syncKeys = [
        'coursesData', 'schoolData', 'colorSettings', 'blinkSettings',
        'semesterCourseAssignments', 'pastCourseInfo', 'certCourseData',
        'coursePlayPriority', 'courseDisplayNames', 'profileData',

```

```

        'checkedCourses', 'inProgressCourses', 'courseGrades', 'openElectives',
        'gridSizes', 'calendarHeight', 'calendarReduction', 'showAllDayRow',
        'otherRowColors',
    ];
    const payload: Record<string, any> = {};
    for (const key of syncKeys) {
        const val = localStorage.getItem(key);
        if (val) {
            try { payload[key] = JSON.parse(val); } catch { payload[key] = val; }
        }
    }
    const [syncResult, pullResult, bulkResult] = await Promise.all([
        syncAllCalendarMutation.mutateAsync(),
        fetch('/api/calendar/pull', { method: 'POST' }),
        Object.keys(payload).length > 0
            ? fetch('/api/degree-tracking/bulk', { method: 'POST', headers: { 'Content-Type': 'application/js...
              : Promise.resolve({ ok: true, saved: 0 })),
    ]);
    const syncData = syncResult as any;
    const failCount = (syncData?.results?.dueEvents?.failed || 0) + (syncData?.results?.prepEvents?.faile...
    const settingsSaved = (bulkResult as any)?.saved || 0;
    if (failCount > 0) {
        toast({ title: "Sync partially complete", description: `${failCount} event(s) failed. ${settingsSav...
    } else {
        toast({ title: "Sync complete", description: `Push & pull finished. ${settingsSaved} settings sync...
    }
    queryClient.invalidateQueries({ queryKey: ['/api/tasks'] });
} catch (error) {
    toast({ title: "Sync failed", description: "Try again in a minute", variant: "destructive" });
}
}
data-testid="honeycomb-sync"
title="Sync"
>
<svg width="18" height="18" viewBox="0 0 24 24" fill="white" xmlns="http://www.w3.org/2000/svg"><path d="...
</div>

/* Refresh Button */
<div
  style={{
    marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
    background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',

    border: '1.5px solid rgba(255,255,255,0.35)',
    boxShadow: '0 4px 24px rgba(0,0,0,0.15)', inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,...
    display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: 'pointer'
  }}
  className="pill-button-hover"
  onClick={(e) => {
    triggerButtonGlow('refresh');
    const icon = (e.currentTarget as HTMLElement).querySelector('svg');
    if (icon) { icon.style.transition = 'transform 0.6s ease'; icon.style.transform = 'rotate(360deg)'; set...
    queryClient.invalidateQueries();
    queryClient.refetchQueries();
    refreshFileCounts();
    fetch('/api/google/calendar/events').catch(() => {});
    fetch('/api/tasks').catch(() => {});
    fetch('/api/announcements').catch(() => {});
  }}
  data-testid="button-refresh-data"
  title="Refresh"
>
  <RefreshCw className="h-[18px] w-[18px] text-white" />
</div>

/* Kitchen Stop Button (moved from bottom pill) */
<div
  style={{
    marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
    background: isKitchenPlaying ? 'linear-gradient(180deg, #8B0000 0%, #DC143C 50%, #FF4500 100%)' : 'line...

    border: '1.5px solid rgba(255,255,255,0.35)',
    boxShadow: isKitchenPlaying ? 'inset 0 1px 2px rgba(255,255,255,0.3), inset 0 -1px 2px rgba(0,0,0,0.3),...
    display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: 'pointer',
    animation: isKitchenPlaying ? 'pulse 2s infinite' : 'none'
  }}
  className="pill-button-hover"
  onClick={handleKitchenStop}
  data-testid="button-kitchen-stop"
  title="Stop"
>
  <Square className="text-white fill-white" style={{ height: '14px', width: '14px' }} />
</div>

/* Radio Button (moved from bottom pill) */
<div
  style={{
    marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
    background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',

    border: '1.5px solid rgba(255,255,255,0.35)',
    boxShadow: '0 4px 24px rgba(0,0,0,0.15)', inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,...
    display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: 'pointer'
  }}
  className="pill-button-hover"
  onClick={() => { triggerButtonGlow('radio'); setIsRadioDialogOpen(true); }}
  data-testid="button-radio-dialog"
  title="Radio Controls"
>
  <Radio className="text-white" style={{ height: '16px', width: '16px' }} />
</div>

/* Fullscreen Toggle (moved from bottom pill) */
<div
  style={{
    marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
    background: isFullscreen ? 'linear-gradient(180deg, #1a6b1a 0%, #2a8a2a 50%, #4aaa4a 100%)' : 'linear-g...

```

```

        border: '1.5px solid rgba(255,255,255,0.35)',
        boxShadow: isFullscreen ? 'inset 0 1px 2px rgba(255,255,255,0.3), inset 0 -1px 2px rgba(0,0,0,0.3), 0 2px 4px rgba(0,0,0,0.3)',
        display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: 'pointer'
    })
    className="pill-button-hover"
    onClick={toggleFullscreen}
    data-testid="button-fullscreen"
    title={isFullscreen ? "Exit Fullscreen" : "Enter Fullscreen"}
  >
    {isFullscreen ? <Minimize2 className="h-[18px] w-[18px] text-white" /> : <Maximize className="h-[18px] w-[18px] text-white" />}
  </div>

  /* Home Assistant (moved from bottom pill) */
  {isAdmin && (
    <div
      style={{
        marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
        background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',
        border: '1.5px solid rgba(255,255,255,0.35)',
        boxShadow: '0 4px 24px rgba(0,0,0,0.15), inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,0,0.15)',
        display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: 'pointer'
      }}
      className="pill-button-hover"
      onClick={() => {
        window.open('http://172.24.0.2:8123/config/dashboard', '_blank');
      }}
      data-testid="button-home-assistant"
      title="Open Home Assistant"
    >
      <svg viewBox="0 0 24 24" fill="white" style={{ height: '20px', width: '20px' }}>
        <path d="M12 0L1.5 6v12L12 24l10.5-6v6L12 0z" />
      </svg>
    </div>
  )}

  /* Quick Notes Button */
  <a
    href="/onenote"
    style={{
      marginTop: '4px', width: '44px', height: '44px', borderRadius: '50%',
      background: 'linear-gradient(135deg, #4B2D7F 0%, #7B5EA7 100%)',
      border: '1.5px solid rgba(255,255,255,0.5)',
      boxShadow: '0 4px 24px rgba(0,0,0,0.15), inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,0,0.15)',
      display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: 'pointer',
      textDecoration: 'none',
    }}
    className="pill-button-hover"
    title="Quick Notes"
    data-testid="honeycomb-quick-notes"
  >
    <StickyNote style={{ color: 'white', strokeWidth: 1.5, height: '18px', width: '18px' }} />
  </a>

  /* Projects Button */
  <Button
    variant="ghost"
    size="sm"
    className={`!h-[40px] !min-h-[40px] px-[16px] no-default-hover-elevate no-default-active-elevate text-white`}
    style={{ fontFamily: 'Avenir, Avenir Next, -apple-system, BlinkMacSystemFont, sans-serif', background: 'linear-gradient(135deg, #4B2D7F 0%, #7B5EA7 100%)' }}
    data-testid="button-projects"
    onClick={() => { triggerButtonGlow('projects'); setEditingProject(null); setProjectWizardStep(0); setProjectWizardStep(0); setProjectWizardStep(0); }}
  >
    + Project
  </Button>

  /* Quick Add Button */
  <Button variant="ghost" size="sm" className={`!h-[40px] !min-h-[40px] px-[16px] no-default-hover-elevate no-default-active-elevate text-white`} />
</div>
</div>
</div>

/* D2L Announcement Ticker - fixed at very top of page, matches bottom news ticker style */
<div className="fixed left-0 right-0 overflow-hidden flex" style={{ top: 0, height: '38px', zIndex: 111, background: 'linear-gradient(to right, #4B2D7F 0%, #7B5EA7 100%)' }}>
  <div className="flex-shrink-0 flex items-center justify-center cursor-pointer" style={{ height: '38px', width: '100%' }}>
    <img src={d2lTickerLabel} alt="D2L" style={{ height: '38px', width: 'auto', objectFit: 'contain' }} />
  </div>
</div>
<div className="flex-1 overflow-hidden relative h-full">
  {tickerWithTodayTasks.length > 0 ? (
    <div ref={el} => {
      if (!el) return;
      if ((el as any).__d2lTickerEl === el) return;
      (el as any).__d2lTickerEl = el;
      let isFirstRun = true;
      const startScroll = () => {
        if (!el || !el.parentElement) return;
        const parentWidth = el.parentElement.clientWidth || window.innerWidth;
        const contentWidth = el.scrollWidth;
        if (contentWidth <= 0) return;
        const startPos = isFirstRun ? Math.min(parentWidth, 100) : parentWidth;
        isFirstRun = false;
        const totalTravel = startPos + contentWidth;
        const speed = 65;
        const dur = totalTravel / speed;
        el.style.setProperty('--ticker-start', `${startPos}px`);
        el.style.setProperty('--ticker-end', `-${contentWidth}px`);
        el.style.animation = 'none';
        void el.offsetWidth;
        el.style.animation = `tickerScroll ${dur}s linear 1`;
      };
      const onEnd = () => {
        el.style.animation = 'none';
        requestAnimationFrame(() => requestAnimationFrame(startScroll));
      };
      el.removeEventListener('animationend', (el as any).__d2lTickerEndHandler);
      (el as any).__d2lTickerEndHandler = onEnd;
      el.addEventListener('animationend', onEnd);
    }
  ) : null}
</div>

```



```

    }}
    onTouchEnd={() => {
      if (tickerDragIdx !== null && tickerDragOverIdx !== null && tickerDragIdx !== tickerDragOverIdx) {
        const items = [...d2lAnnouncements];
        const [moved] = items.splice(tickerDragIdx, 1);
        items.splice(tickerDragOverIdx, 0, moved);
        reorderTickerMutation.mutate(items.map((i: any) => i.id));
      }
      setTickerDragIdx(null);
      setTickerDragOverIdx(null);
      tickerTouchDragRef.current = null;
    }}
    data-ticker-drag-item
    className="cursor-grab active:cursor-grabbing"
    style={{
      opacity: tickerDragIdx === idx ? 0.4 : 1,
      borderTop: tickerDragOverIdx === idx && tickerDragIdx !== null && tickerDragIdx !== idx ? '2px ...
      transition: 'opacity 0.15s',
    }}
    data-testid={`ticker-item-${a.id}`}
  >
  <div className="flex items-center gap-2 py-2">
    <GripVertical className="h-3.5 w-3.5 text-white/25 shrink-0" />
    <span className="text-[10px] px-1.5 py-0.5 rounded font-semibold shrink-0" style={{ minWidth: '...
      {a.courseName === 'Custom' ? '0=0i' : a.courseName}
    </span>
    <span className="text-white text-[12px] flex-1 min-w-0 truncate text-left">{a.body || a.snippet...
    <button
      onClick={() => {
        if (a._isSynthetic) {
          const newSet = new Set(dismissedTodayTaskIds);
          newSet.add(a.id);
          setDismissedTodayTaskIds(newSet);
          localStorage.setItem('dismissedTodayTaskIds', JSON.stringify([...newSet]));
        } else {
          deleteTickerMutation.mutate(a.id);
        }
      }}
      className="shrink-0 text-white/30 hover:text-red-400 transition-colors p-1"
      data-testid={`button-delete-ticker-${a.id}`}
    >
    <Trash2 className="h-3.5 w-3.5" />
  </button>
</div>
{idx < allDialogItems.length - 1 && (
  <div style={{ marginLeft: '22px', marginRight: '8px', borderBottom: '1px solid rgba(255,255,255...
)}}
</div>
));
}}())
</div>
<div className="px-4 py-3 flex items-center gap-2" style={{ borderTop: '1px solid rgba(255,255,255,0.15)'...
  <select
    value={newTickerTag}
    onChange={(e) => setNewTickerTag(e.target.value)}
    className="text-white text-[11px] px-2 py-2 rounded focus:outline-none"
    style={{ background: 'rgba(255,255,255,0.08)', border: '1px solid rgba(255,255,255,0.15)', minWidth: ...
    data-testid="select-ticker-tag"
  >
    <option value="Custom" style={{ background: '#1a1a2e' }}>0=0i Custom</option>
    {semesterSettings && [1, 2, 3].map(i => {
      const code = ((semesterSettings as any)[`course${i}Code`] || '').trim().replace(/\\/s/g, '').toUpperCase...
      return code ? <option key={code} value={code} style={{ background: '#1a1a2e' }}>{code}</option> : n...
    })}
    <option value="REMINDER" style={{ background: '#1a1a2e' }}>REMINDER</option>
    <option value="URGENT" style={{ background: '#1a1a2e' }}>URGENT</option>
  </select>
  <input
    type="text"
    value={newTickerText}
    onChange={(e) => setNewTickerText(e.target.value)}
    onKeyDown={(e) => { if (e.key === 'Enter' && newTickerText.trim()) { addTickerMutation.mutate({ body:...
    placeholder="Add ticker item..."
    className="flex-1 text-white text-[12px] px-3 py-2 rounded focus:outline-none placeholder:text-white/30"
    style={{ background: 'rgba(255,255,255,0.08)', border: '1px solid rgba(255,255,255,0.15)' }}
    data-testid="input-new-ticker"
  />
  <button
    onClick={() => { if (newTickerText.trim()) addTickerMutation.mutate({ body: newTickerText.trim(), tag...
    disabled={!newTickerText.trim() || addTickerMutation.isPending}
    className="shrink-0 disabled:opacity-40 text-white text-[12px] font-semibold px-3 py-2 rounded transi...
    style={{ background: 'linear-gradient(180deg, rgba(255,255,255,0.28) 0%, ${{colorSettings.headerBar}}cc...
    data-testid="button-add-ticker"
  >
  + Add
</button>
</div>
<div className="flex items-center justify-end gap-2 px-4 py-3" style={{ borderTop: '1px solid rgba(255,25...
  <Button
    variant="outline"
    className="border !border-white/30 text-white/70 hover:text-white hover:!border-white/50 hover:bg-tra...
    style={{ fontSize: '12px' }}
    onClick={() => setTickerDialogOpen(false)}
    data-testid="button-cancel-ticker-dialog"
  >
  Cancel
</Button>
  <Button
    variant="outline"
    className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-transpare...
    style={{ boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255...
    onClick={() => {
      toast({ title: "Saved", description: "Ticker items saved." });
      setTickerDialogOpen(false);
    }}
    data-testid="button-save-ticker-dialog"
  >
  Save

```

```

        </Button>
      </div>
    </div>
  </div>
})

/* Top pill tab - centered to page, outside pill container to avoid transform issues */
<div
  style={{
    position: 'fixed',
    left: '50%',
    transform: 'translateX(-50%)',
    top: `${d2lTickerViewHeight}px`,
    width: '120px',
    height: '35px',
    cursor: 'pointer',
    pointerEvents: isTopPillOpen ? 'none' : 'auto',
    zIndex: 110,
    touchAction: 'manipulation',
  }}
  onTouchStart={(e) => {
    e.preventDefault();
    openTopPill();
    if (topPillTimeoutRef.current) clearTimeout(topPillTimeoutRef.current);
    topPillTimeoutRef.current = setTimeout(() => {
      if (isHamburgerOpenRef.current) return;
      closeTopPill();
    }, 6000);
  }}
  onClick={() => {
    openTopPill();
    if (topPillTimeoutRef.current) clearTimeout(topPillTimeoutRef.current);
    const delay = 'ontouchstart' in window ? 6000 : 1800;
    topPillTimeoutRef.current = setTimeout(() => {
      if (isHamburgerOpenRef.current) return;
      closeTopPill();
    }, delay);
  }}
  onMouseEnter={() => {
    openTopPill();
    if (topPillTimeoutRef.current) clearTimeout(topPillTimeoutRef.current);
  }}
/>
<div
  style={{
    position: 'fixed',
    left: '50%',
    transform: 'translateX(-50%)',
    top: `${-9 + d2lTickerViewHeight}px`,
    width: '84px',
    height: '27px',
    cursor: 'pointer',
    pointerEvents: 'auto',
    opacity: 0.9,
    zIndex: 110,
    touchAction: 'manipulation',
    transition: 'opacity 0.3s ease-in-out',
  }}
  onTouchStart={(e) => {
    e.preventDefault();
    openTopPill();
    if (topPillTimeoutRef.current) clearTimeout(topPillTimeoutRef.current);
    topPillTimeoutRef.current = setTimeout(() => {
      if (isHamburgerOpenRef.current) return;
      closeTopPill();
    }, 6000);
  }}
  onClick={() => {
    openTopPill();
    if (topPillTimeoutRef.current) clearTimeout(topPillTimeoutRef.current);
    const delay = 'ontouchstart' in window ? 6000 : 1800;
    topPillTimeoutRef.current = setTimeout(() => {
      if (isHamburgerOpenRef.current) return;
      closeTopPill();
    }, delay);
  }}
  onMouseEnter={() => {
    openTopPill();
    if (topPillTimeoutRef.current) clearTimeout(topPillTimeoutRef.current);
  }}
  onMouseLeave={() => {
    if (isHamburgerOpenRef.current) return;
    if (topPillTimeoutRef.current) clearTimeout(topPillTimeoutRef.current);
    const delay = 'ontouchstart' in window ? 4000 : 1800;
    topPillTimeoutRef.current = setTimeout(() => {
      if (isHamburgerOpenRef.current) return;
      closeTopPill();
    }, delay);
  }}
/>
<svg width="84" height="27" viewBox="0 0 84 27" style={{ display: 'block' }}>
  <path d="M0,0 L84,0 L84,9 Q75,9 75,14 L75,13 Q75,25 63,25 L21,25 Q9,25 9,13 L9,14 Q9,9 0,9 Z" fill="rgba(255,255,255,0.8)" stroke="black" stroke-width="2.5" style={{ color: 'black', filter: 'drop-shadow(2px 2px 4px #000)' }}>
</svg>
<div style={{ position: 'absolute', left: '50%', top: '6px', transform: 'translateX(-50%)', display: 'flex', ...
  <LayoutGrid className="h-[13px] w-[13px]" stroke-width={2.5} style={{ color: 'black', filter: 'drop-shadow(2px 2px 4px #000)' }}>
</div>
</div>

/* Share Button - fixed position, bottom right beside copyright */
isAdmin && (
  <Share
    className="text-white/80 cursor-pointer hover:text-white"
    stroke-width={2.5}
    style={{ height: '18px', width: '18px', position: 'fixed', bottom: '43px', right: `${calendarRight - calend...
    onClick={generateShareLink}
    data-testid="button-share-main"
  />
)

```

```

/* Timer bar - controls left of timer */
<div style={{
  position: 'fixed',
  right: `${calendarRight - calendarReduction + 9}px`,
  top: `${3 + d2lTickerHeight}px`,
  zIndex: 100,
  display: 'flex',
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'flex-end',
  gap: '6px',
  paddingRight: '0px',
  opacity: isTopPillOpen ? 0 : 1,
  transition: isTopPillOpen ? 'opacity 0.3s ease-in-out' : 'opacity 0.1s ease-in-out',
  pointerEvents: isTopPillOpen ? 'none' : 'auto',
}} data-tpo data-tpo-opacity="1" data-testid="timer-bar">
  <div className="flex flex-row items-center gap-[25px]" style={{ marginTop: '2px', marginRight: '10px' }}>
    <button className="p-0 hover:bg-white/20 rounded transition-colors" onClick={() => { setPomodoroTime(prev =>...
      +1m
    }}>
      </button>
    <button className="p-0 hover:bg-white/20 rounded transition-colors" onClick={togglePomodoro} data-testid="b...
      {pomodoroRunning ? <Pause className="h-[16px] w-[16px] text-white" strokeWidth={2.5} /> : <Play className=...
    }}>
      </button>
    <button className="p-0 hover:bg-white/20 rounded transition-colors" onClick={resetPomodoro} data-testid="bu...
      <Square className="h-[14px] w-[14px] text-white" strokeWidth={2.5} />
    </button>
  </div>
  <div ref={clockContainerRef} data-testid="pomodoro-timer" style={{ position: 'relative', cursor: 'pointer' }}...
    {() => {
      const totalSec = pomodoroMode === 'work' ? 25 * 60 : pomodoroMode === 'shortBreak' ? 5 * 60 : 15 * 60;
      const elapsed = totalSec - pomodoroTime;
      const frac = totalSec > 0 ? elapsed / totalSec : 0;
      const r = 35, cx = 41, cy = 41, sw = 5;
      const startAng = Math.PI, endAng = 2 * Math.PI;
      const arcLen = endAng - startAng;
      const circumHalf = r * arcLen;
      const dash = frac * circumHalf;
      const arcColor = pomodoroMode === 'work' ? '#e74c8b' : pomodoroMode === 'shortBreak' ? '#22c55e' : '#3b82...
      return (
        <svg width="82" height="47" viewBox="0 0 82 47">
          <path d={`M ${cx - r} ${cy} A ${r} ${r} 0 1 1 ${cx + r} ${cy}` } fill="none" stroke="rgba(255,255,255,...
          <path d={`M ${cx - r} ${cy} A ${r} ${r} 0 1 1 ${cx + r} ${cy}` } fill="none" stroke={arcColor} strokeW...
          <text x={cx} y={cy - 1} textAnchor="middle" fill="#ffffff" fontSize="14" fontWeight="700" fontFamily=...
        </svg>
      );
    }}()>
  </div>
</div>

/* Clock removed - now inside countdown pill */

/* Settings Panel Popup - Degree Tracking */
{isSettingsPanelOpen && (
  <div className="fixed inset-0 z-[10001] bg-black/50" onClick={() => startTransition(() => setIsSettingsPanel0...
)}
{isSettingsPanelOpen && (
  <div
    className="fixed z-[10001] overflow-hidden flex flex-col text-[11px] p-0 sm:rounded-lg"
    style={{ left: '50%', transform: 'translateX(-50%)', top: 'calc(3vh - 6px)', width: 'calc(96vw + 24px)', ma...
  >
    <div className="flex items-center justify-between px-4 py-3 border-b border-white/40 flex-shrink-0 rounded-...
      <div className="flex items-center gap-2">
        <GraduationCap className="text-white" style={{ width: '15px', height: '15px' }} />
        <h2 className="font-normal text-white" style={{ fontFamily: 'Avenir, 'Avenir Next', -apple-system, Blin...
          DEGREE TRACKING
        </h2>
      </div>
    </div>
    <div
      className="flex items-center gap-1 cursor-pointer hover:opacity-80 transition-opacity"
      onClick={(e) => {
        e.stopPropagation();
        startTransition(() => {
          setDraftPriorityBoth({ ...coursePlayPriority });
          setSchoolCoursesOpenSource('degree');
          startTransition(() => setIsSchoolCoursesDialogOpen(true));
        });
      }}
      data-testid="button-degree-courses"
    >
      <span className="text-white text-[11px] font-medium" style={{ textShadow: '0 1px 2px rgba(0,0,0,0.2)' }}...
      <ChevronRight className="text-white/80" style={{ width: '14px', height: '14px' }} />
    </div>
  </div>
  <div className="flex-1 overflow-hidden p-2 pt-1 min-h-0 flex flex-col" style={{ color: '#000' }}>
    <div className="shrink-0 flex items-center justify-between pb-2" style={{ marginTop: '2px' }}>
      <div className="flex items-center">
        <input
          type="file"
          ref={courseListFileRef}
          accept=".txt,.csv,.md,.html,.htm"
          className="hidden"
          data-testid="input-course-list-file"
          onChange={(e) => {
            const file = e.target.files?.[0];
            if (file) {
              const activeCourses = coursesData.courses.filter((c: any) => c.name.trim());
              if (activeCourses.length === 1) {
                handleCourseListUpload(file, activeCourses[0].name);
              } else if (activeCourses.length > 1) {
                setCourseListPendingFile(file);
                setCourseListUploadCourse('');
                setIsCourseListUploadOpen(true);
                setCourseListChanges([]);
              }
            }
            e.target.value = '';
          }}
        </div>
      </div>
    </div>
  </div>

```

```

/>
<Button
  type="button"
  variant="outline"
  className="border !border-white/50 text-white hover:text-white hover:!border-white transition-opacity..."
  style={{ background: 'rgba(10,15,30,0.85)', boxShadow: '0 0 6px rgba(255,255,255,0.6)', width: 120px, height: 40px, ... }}
  onClick={() => courseListFileRef.current?.click()}
  disabled={courseListParsing}
  data-testid="button-upload-course-list"
  title="Upload course list to scan for task changes"
>
  <Upload className="w-3 h-3 mr-1" />
  {courseListParsing ? 'Scanning...' : 'Upload Course List'}
</Button>
</div>
{(() => {
  const semKeyOrder = ['ss2025', 'f2025', 'w2026', 'ss2026', 'f2026', 'w2027', 'ss2027', 'f2027', 'w2028', 'ss20...
  const now = new Date();
  const currentSemIdx = (() => {
    if (now >= new Date('2025-05-05') && now <= new Date('2025-08-08')) return semKeyOrder.indexOf('ss2...
    if (now >= new Date('2025-09-08') && now <= new Date('2025-12-12')) return semKeyOrder.indexOf('f20...
    if (now >= new Date('2026-01-12') && now <= new Date('2026-04-17')) return semKeyOrder.indexOf('w20...
    if (now >= new Date('2026-05-04') && now <= new Date('2026-08-07')) return semKeyOrder.indexOf('ss2...
    if (now >= new Date('2026-09-07') && now <= new Date('2026-12-11')) return semKeyOrder.indexOf('f20...
    if (now >= new Date('2027-01-11') && now <= new Date('2027-04-16')) return semKeyOrder.indexOf('w20...
    if (now >= new Date('2027-05-03') && now <= new Date('2027-08-06')) return semKeyOrder.indexOf('ss2...
    if (now >= new Date('2027-09-13') && now <= new Date('2027-12-17')) return semKeyOrder.indexOf('f20...
    if (now >= new Date('2028-01-10') && now <= new Date('2028-04-14')) return semKeyOrder.indexOf('w20...
    if (now >= new Date('2028-05-01') && now <= new Date('2028-08-04')) return semKeyOrder.indexOf('ss2...
    if (now >= new Date('2028-09-11') && now <= new Date('2028-12-15')) return semKeyOrder.indexOf('f20...
    if (now >= new Date('2029-01-08') && now <= new Date('2029-04-13')) return semKeyOrder.indexOf('w20...
    return -1;
  })();
  const relevantSemKeys = currentSemIdx >= 0 ? semKeyOrder.slice(0, currentSemIdx + 1) : [];
  const letterToGpa: Record<string, number> = { 'A+': 4.33, 'A': 4.0, 'A-': 3.67, 'B+': 3.33, 'B': 3.0, ...
  const pToGpa = (p: number) => p >= 90 ? 4.33 : p >= 85 ? 4.0 : p >= 80 ? 3.67 : p >= 77 ? 3.33 : p >= ...

  const semAllCourses: Record<string, { code: string }>[] = {};
  const allDefCodes = new Set<string>();
  for (const semKey of semKeyOrder) {
    semAllCourses[semKey] = [...(semesterCourseAssignments[semKey] || [])];
    semAllCourses[semKey].forEach(c => allDefCodes.add(c.code.toUpperCase().replace(/\s/g, '')));
  }
  const currentSemKeyForEnrich = relevantSemKeys[relevantSemKeys.length - 1] || '';
  if (currentSemKeyForEnrich && semAllCourses[currentSemKeyForEnrich]) {
    coursesData.courses.filter(c => c.name.trim()).forEach(c => {
      const code = c.name.split(' - ')[0]?.trim().toUpperCase().replace(/\s/g, '');
      if (code && !allDefCodes.has(code)) {
        semAllCourses[currentSemKeyForEnrich].push({ code });
        allDefCodes.add(code);
      }
    });
  }

  const allGpaVals: number[] = [];
  for (const semKey of relevantSemKeys) {
    const courses = semAllCourses[semKey] || [];
    const vals: number[] = [];
    for (const c of courses) {
      const code = c.code.replace(/\s/g, '');
      const codeNorm = code.toUpperCase();
      const codeNoC = codeNorm.replace(/^[C]?([A-Z]{2})$/, '');
      const info = pastCourseInfo[code] || pastCourseInfo[codeNoC];
      const certKey = Object.keys(certCourseMap).find(k => {
        const mc = certCourseMap[k].code.replace(/\s/g, '').toUpperCase();
        return mc === codeNorm || {'C' + mc} === codeNorm || mc === codeNoC;
      });
      const cg = certKey ? courseGrades[certKey] : null;
      const directGrade = courseGrades[codeNorm] || courseGrades[code] || courseGrades[codeNoC] || null;
      let electiveGrade: { grade: string; percent: string } | null = null;
      if (!cg?.percent && !cg?.grade && !directGrade?.percent && !directGrade?.grade) {
        for (const [slot, elVal] of Object.entries(openElectives)) {
          if (elVal && elVal.replace(/\s/g, '').toUpperCase().startsWith(codeNorm)) {
            if (courseGrades[slot]?.percent || courseGrades[slot]?.grade) {
              electiveGrade = courseGrades[slot];
              break;
            }
          }
        }
      }
      if (cg?.percent && cg.percent.trim()) { const p = parseFloat(cg.percent); if (!isNaN(p)) vals.push(p); }
      else if (cg?.grade && cg.grade.trim() && letterToGpa[cg.grade] !== undefined) { vals.push(letterToGpa[cg.grade]); }
      else if (directGrade?.percent && directGrade.percent.trim()) { const p = parseFloat(directGrade.percent); if (!isNaN(p)) vals.push(p); }
      else if (directGrade?.grade && directGrade.grade.trim() && letterToGpa[directGrade.grade] !== undefined) { vals.push(letterToGpa[directGrade.grade]); }
      else if (info?.grade && info.grade.trim() && letterToGpa[info.grade] !== undefined) { vals.push(letterToGpa[info.grade]); }
      else if (electiveGrade?.percent && electiveGrade.percent.trim()) { const p = parseFloat(electiveGrade.percent); if (!isNaN(p)) vals.push(p); }
      else if (electiveGrade?.grade && electiveGrade.grade.trim() && letterToGpa[electiveGrade.grade] !== undefined) { vals.push(letterToGpa[electiveGrade.grade]); }
    }
    for (const v of vals) allGpaVals.push(v);
  }
  const avgGpa = allGpaVals.length > 0 ? allGpaVals.reduce((a, b) => a + b, 0) / allGpaVals.length : null;
  const gpaToLetter = (gpa: number): string => {
    if (gpa >= 4.17) return 'A+';
    if (gpa >= 3.84) return 'A';
    if (gpa >= 3.5) return 'A-';
    if (gpa >= 3.17) return 'B+';
    if (gpa >= 2.84) return 'B';
    if (gpa >= 2.5) return 'B-';
    if (gpa >= 2.17) return 'C+';
    if (gpa >= 1.84) return 'C';
    if (gpa >= 1.34) return 'C-';
    if (gpa >= 0.5) return 'D';
    return 'F';
  };
  return (
    <div className="rounded-md flex items-center justify-center gap-1.5 px-3" style={{ backgroundColor: '#f0f0f0', padding: 10px, ... }}>
      <span className="text-[10px] font-semibold uppercase tracking-wider" style={{ color: '#555' }}>GPA: {avgGpa}
      {avgGpa === null ? (

```



```

<div className="flex border-b border-black" style={{ backgroundColor: '#000000', color: 'ffffff', ....
<div className="flex-1 px-1 py-0.5 text-[9px]">Professionally Related Elective: <span style={sectio...
</div>
[{'OPEN1', 'OPEN2'}].map((cid, i) => (
  <div key={cid} className={ `flex border-b border-black ${courseRowClass(cid)} ${isSectionAllGreen(3,...
    <div className="w-12 shrink-0 px-1 py-0.5 border-r border-black text-[9px] flex items-center curs...
    <div className="shrink-0 px-1 py-0.5 border-r border-black text-[9px] flex items-center course-co...
    <div className="flex-1 px-1 py-0.5 flex items-center overflow-hidden min-w-0">
      <select className={ `flex-1 min-w-0 mr-0 text-[9px] px-0.5 py-0.5 border border-black rounded-sm...
        <option value="">{'Select PR Elective ${i+1}...'}</option>
        {buildElectiveOptions(OPEN_ELECTIVE_COURSES, cid, ['OPEN1', 'OPEN2'])}.map(o => (
          <option key={o.key} value={o.value}>{o.label}</option>
        ))
      </select>
      <StrikethroughLabel id={cid} />
    </div>
    <div className="border-l border-black flex flex-col items-center justify-center gap-0.5 py-0.5" s...
      {renderGradeInput(cid)}
    </div>
    <div className="no-dim border-l border-black flex items-center justify-center" style={{ width: '3...
    <div className="no-dim border-l border-black flex items-center justify-center cursor-pointer hove...
      <Pencil className="w-3 h-3 text-gray-600 hover:text-gray-900" />
    </div>
  </div>
))
</div>
</div>
<div className="shrink-0 flex items-center gap-2 px-2 py-1.5 border-t-2 border-black">
  <span className="text-[9px] font-bold whitespace-nowrap">Progress</span>
  <div className="flex-1 bg-gray-200 rounded-full h-2.5">
    <div className="h-2.5 rounded-full transition-opacity duration-300" style={{ width: `${l1Progress....
  </div>
  <span className="text-[9px] font-bold whitespace-nowrap">{l1Progress.completed} / {l1Progress.total}<...
</div>
</div>
</div>
/* Level II */
<div className={ `flex-1 min-w-0 rounded-md p-2 text-[9px] flex flex-col ${allCoursesChecked ? 'bg-gray-...
<div className="border-2 border-black flex flex-col min-h-0 flex-1">
  <div className="shrink-0">
    <div className="flex border-b border-black transition-colors duration-300" style={{ backgroundColor...
      <div className="font-bold px-1 py-0.5 w-16 text-white" style={{ textShadow: '0 1px 2px rgba(0,0,0...
      <div className="font-bold px-1 py-0.5 flex-1 text-center text-white" style={{ textShadow: '0 1px ...
    </div>
    <div className="flex border-b border-black bg-gray-50">
      <div className="w-12 px-1 py-0.5 border-r border-black text-[9px] font-bold text-center">Type</div>
      <div className="px-1 py-0.5 border-r border-black text-[9px] font-bold text-center" style={{ widt...
      <div className="flex-1 px-1 py-0.5 text-[9px] font-bold">Title</div>
      <div className="px-1 py-0.5 border-l border-black text-[9px] font-bold text-center" style={{ widt...
      <div className="px-1 py-0.5 border-l border-black text-[9px] font-bold text-center" style={{ widt...
      <div className="px-1 py-0.5 border-l border-black text-[9px] font-bold text-center" style={{ widt...
    </div>
  </div>
  <div className="flex-1 overflow-y-auto l3-no-scrollbar">
    <div>
      <div className="flex border-b border-black" style={{ backgroundColor: '#000000', color: 'ffffff', ....
        <div className="flex-1 px-1 py-0.5 text-[9px]">Required Professional Course <span className="font-b...
      </div>
      <div className={ `flex border-b border-black ${courseRowClass('L2_PPA211')} ${isSectionAllGreen(0, 'L2...
        <div className="w-12 px-1 py-0.5 border-r border-black text-[9px]">Prof-Req'd</div>
        <div className="px-1 py-0.5 border-r border-black flex items-center course-code-mono" style={{ widt...
        <div className="flex-1 px-1 py-0.5 cursor-pointer hover:underline min-w-0 flex items-center" onClic...
        <div className="border-l border-black flex flex-col items-center justify-center gap-0.5 py-0.5" sty...
          {renderGradeInput('L2_PPA211')}
        </div>
        <div className="no-dim border-l border-black flex items-center justify-center" style={{ width: '38p...
        <div className="no-dim border-l border-black flex items-center justify-center cursor-pointer hove...
          <Pencil className="w-3 h-3 text-gray-600 hover:text-gray-900" />
        </div>
      </div>
    </div>
    <div>
      <div className="flex border-b border-black" style={{ backgroundColor: '#000000', color: 'ffffff', ....
        <div className="flex-1 px-1 py-0.5 text-[9px]">Required Professional Courses: <span style={sectionRemain...
      </div>
      [
        { id: 'L2_PPA120', code: 'CPPA 120', name: 'Canadian Politics and Government', type: 'SELECT' },
        { id: 'L2_PPA121', code: 'CPPA 121', name: 'Ontario Politics and Government', type: 'SELECT' },
        { id: 'L2_PPA122', code: 'CPPA 122', name: 'Local Politics and Government', type: 'SELECT' },
        { id: 'L2_PPA124', code: 'CPPA 124', name: 'Indigenous Politics and Governance', type: 'SELECT' },
        { id: 'L2_PPA235', code: 'CPPA 235', name: 'Theories of the State', type: 'SELECT' },
        { id: 'L2_PPA303', code: 'CPPA 303', name: 'Financial Management', type: 'SELECT' },
        { id: 'L2_PPA319', code: 'CPPA 319', name: 'Politics of Work and Labour', type: 'SELECT' },
      ].map((c) => {
        const disabled = isCheckDisabled(c.id);
        return (
          <div key={c.id} className={ `flex border-b border-black ${courseRowClass(c.id)} ${isSectionAllGreen(...
            <div className="w-12 px-1 py-0.5 border-r border-black text-[9px]">{c.type}</div>
            <div className="px-1 py-0.5 border-r border-black flex items-center course-code-mono" style={{ wi...
            <div className="flex-1 px-1 py-0.5 cursor-pointer hover:underline min-w-0 flex items-center" onCl...
            <div className="border-l border-black flex flex-col items-center justify-center gap-0.5 py-0.5" s...
              {renderGradeInput(c.id)}
            </div>
            <div className="no-dim border-l border-black flex items-center justify-center" style={{ width: '3...
            <div className="no-dim border-l border-black flex items-center justify-center cursor-pointer hove...
              <Pencil className="w-3 h-3 text-gray-600 hover:text-gray-900" />
            </div>
          </div>
        );
      });
    </div>
    <div>
      <div className="flex border-b border-black" style={{ backgroundColor: '#000000', color: 'ffffff', ....
        <div className="flex-1 px-1 py-0.5 text-[9px]">Liberal Studies Elective: <span style={sectionRemain...
      </div>
      <div className={ `flex border-b border-black ${courseRowClass('L2_LIBERAL')} ${isSectionAllGreen(2, 'L...
        <div className="w-12 shrink-0 px-1 py-0.5 border-r border-black text-[9px] flex items-center cursor...

```

```

<div className="shrink-0 px-1 py-0.5 border-r border-black text-[9px] flex items-center course-code...
<div className="flex-1 px-1 py-0.5 flex items-center overflow-hidden min-w-0">
  <select className={`flex-1 min-w-0 mr-0 text-[9px] px-0.5 py-0.5 border border-black rounded-sm $...
    <option value="">Select Lower Level Table A Course...</option>
    {buildElectiveOptions(getCoursesForLevel('LOWER', LIBERAL_STUDIES_COURSES), 'L2_LIBERAL').map(o...
      <option key={o.key} value={o.value}>{o.label}</option>
    )}
  </select>
  <StrikethroughLabel id="L2_LIBERAL" />
</div>
<div className="border-l border-black flex flex-col items-center justify-center gap-0.5 py-0.5" sty...
  {renderGradeInput('L2_LIBERAL')}
</div>
<div className="no-dim border-l border-black flex items-center justify-center" style={{ width: '38p...
<div className="no-dim border-l border-black flex items-center justify-center cursor-pointer hover:...
</div>
</div>
<div>
<div className="flex border-b border-black" style={{ backgroundColor: '#000000', color: '#ffffff', ....
  <div className="flex-1 px-1 py-0.5 text-[9px]">Professionally Related Required: <span style={sectio...
</div>
[[
  { id: 'L2_ECN1', code: 'CECN 101', name: 'Principles of Microeconomics **' },
  { id: 'L2_ECN2', code: 'CECN 104', name: 'Introductory Microeconomics **' },
  { id: 'L2_ECN3', code: 'CECN 110', name: 'The Economy and Society **' },
  { id: 'L2_ECN4', code: 'CECN 201', name: 'Principles of Macroeconomics **' },
  { id: 'L2_ECN5', code: 'CECN 204', name: 'Introductory Macroeconomics **' },
  { id: 'L2_ECN6', code: 'CECN 210', name: 'Understanding Economics **' },
  { id: 'L2_ECN7', code: 'CECN 220', name: 'Evolution of the Global Economy' },
  { id: 'L2_ECN8', code: 'CECN 320', name: 'Introduction to Financial Economics' },
]]
].map((ecn, i) => {
  const disabled = isCheckDisabled(ecn.id);
  return (
    <div key={ecn.id} className={`flex border-b border-black ${courseRowClass(ecn.id)} ${isSectionAllGr...
      <div className="w-12 shrink-0 px-1 py-0.5 border-r border-black text-[9px] flex items-center curs...
      <div className="px-1 py-0.5 border-r border-black flex items-center course-code-mono" style={{ wi...
      <div className="flex-1 px-1 py-0.5 cursor-pointer hover:underline min-w-0 flex items-center" onCl...
      <div className="border-l border-black flex flex-col items-center justify-center gap-0.5 py-0.5" s...
        {renderGradeInput(ecn.id)}
      </div>
      <div className="no-dim border-l border-black flex items-center justify-center" style={{ width: '3...
      <div className="no-dim border-l border-black flex items-center justify-center cursor-pointer hove...
    </div>
  );
});
</div>
<div>
<div className="flex border-b border-black" style={{ backgroundColor: '#000000', color: '#ffffff', ....
  <div className="flex-1 px-1 py-0.5 text-[9px]">Professionally-Related Elective: <span style={sectio...
</div>
[[['L2_OPEN1', 'L2_OPEN2']].map((cid, i) => (
  <div key={cid} className={`flex border-b border-black ${courseRowClass(cid)} ${isSectionAllGreen(4,...
    <div className="w-12 shrink-0 px-1 py-0.5 border-r border-black text-[9px] flex items-center curs...
    <div className="shrink-0 px-1 py-0.5 border-r border-black text-[9px] flex items-center course-co...
    <div className="flex-1 px-1 py-0.5 flex items-center overflow-hidden min-w-0">
      <select className={`flex-1 min-w-0 mr-0 text-[9px] px-0.5 py-0.5 border border-black rounded-sm...
        <option value="">{`Select PR Elective ${i+1}...`}</option>
        {buildElectiveOptions(OPEN_ELECTIVE_COURSES, cid, ['L2_OPEN1', 'L2_OPEN2']).map(o => (
          <option key={o.key} value={o.value}>{o.label}</option>
        ))}
      </select>
      <StrikethroughLabel id={cid} />
    </div>
    <div className="border-l border-black flex flex-col items-center justify-center gap-0.5 py-0.5" s...
      {renderGradeInput(cid)}
    </div>
    <div className="no-dim border-l border-black flex items-center justify-center" style={{ width: '3...
    <div className="no-dim border-l border-black flex items-center justify-center cursor-pointer hove...
  </div>
));
</div>
</div>
<div className="shrink-0 flex items-center gap-2 px-2 py-1.5 border-t-2 border-black">
  <span className="text-[9px] font-bold whitespace-nowrap">Progress</span>
  <div className="flex-1 bg-gray-200 rounded-full h-2.5">
    <div className="h-2.5 rounded-full transition-opacity duration-300" style={{ width: `${l2Progress....
  </div>
  <span className="text-[9px] font-bold whitespace-nowrap">{l2Progress.completed} / {l2Progress.total}<...
</div>
</div>
</div>
/* Level III */
<div className="flex-1 min-w-0 rounded-md p-2 text-[9px] bg-white text-black flex flex-col">
<div className="border-2 border-black flex flex-col min-h-0 flex-1">
  <div className="shrink-0">
    <div className="flex border-b border-black transition-colors duration-300" style={{ backgroundColor...
      <div className="font-bold px-1 py-0.5 w-16 text-white" style={{ textShadow: '0 1px 2px rgba(0,0,0...
      <div className="font-bold px-1 py-0.5 flex-1 text-center text-white" style={{ textShadow: '0 1px ...
    </div>
  </div>
  <div className="flex-1 overflow-y-auto l3-no-scrollbar">
    <table className="w-full" style={{ tableLayout: 'fixed', borderSpacing: 0 }}>
      <colgroup>
        <col style={{ width: '48px' }} />
        <col style={{ width: '53px' }} />
        <col />
        <col style={{ width: '54px' }} />
        <col style={{ width: '38px' }} />
        <col style={{ width: '30px' }} />
      </colgroup>
      <thead className="sticky top-0 z-10">
        <tr className="border-b border-black bg-gray-50">
          <th className="px-1 py-0.5 border-r border-black text-[9px] font-bold text-center">Type</th>
          <th className="px-1 py-0.5 border-r border-black text-[9px] font-bold text-center">Code</th>
          <th className="px-1 py-0.5 text-[9px] font-bold text-left">Title</th>
          <th className="px-1 py-0.5 border-l border-black text-[9px] font-bold text-center">Grade</th>
          <th className="px-1 py-0.5 border-l border-black text-[9px] font-bold text-center">Status</th>
        </tr>
      </thead>
    </table>
  </div>
</div>

```



```

    </div>
  </td>
  <td className="border-l border-black align-middle">
    {renderGradeInput(cid)}
  </td>
  <td className="no-dim border-l border-black text-center align-middle">{renderTriStateToggle(cid...
  <td className="no-dim border-l border-black cursor-pointer hover:bg-gray-100 text-center align-...
</tr>
))}
<tr className="border-b border-black" style={{isSectionAllGreen(4, 'L3')} ? { opacity: 0.45 } : und...
  <td colspan={6} className="px-1 py-0.5 text-[9px]" style={{ backgroundColor: '#000000', color: ...
</tr>
{{['L3_OPEN1', 'L3_OPEN2', 'L3_OPEN3', 'L3_OPEN4', 'L3_OPEN5', 'L3_OPEN6', 'L3_OPEN7'].map((cid, i) => {
  <tr key={cid} className={`border-b border-black ${courseRowClass(cid)} ${isSectionAllGreen(4, 'L3...
    <td className="px-1 py-0.5 border-r border-black text-[9px] align-middle cursor-pointer hover:b...
    <td className="px-1 py-0.5 border-r border-black text-[9px] align-middle course-code-mono">{get...
    <td className="px-1 py-0.5 align-middle text-[9px]">
      <div className="flex items-center overflow-hidden min-w-0">
        <select className={`flex-1 min-w-0 mr-0 text-[9px] px-0.5 py-0.5 border border-black rounde...
          <option value="">{`Select PR Elective ${i+1}...`}</option>
          {buildElectiveOptions(OPEN_ELECTIVE_COURSES, cid, ['L3_OPEN1', 'L3_OPEN2', 'L3_OPEN3', 'L3_0...
            <option key={o.key} value={o.value}>{o.label}</option>
          )}}
        </select>
        <StrikethroughLabel id={cid} />
      </div>
    </td>
    <td className="border-l border-black align-middle">
      {renderGradeInput(cid)}
    </td>
    <td className="no-dim border-l border-black text-center align-middle">{renderTriStateToggle(cid...
    <td className="no-dim border-l border-black cursor-pointer hover:bg-gray-100 text-center align-...
  </tr>
))}
</tbody>
</table>
</div>
<div className="shrink-0 flex items-center gap-2 px-2 py-1.5 border-t-2 border-black">
  <span className="text-[9px] font-bold whitespace-nowrap">Progress</span>
  <div className="flex-1 bg-gray-200 rounded-full h-2.5">
    <div className="h-2.5 rounded-full transition-opacity duration-300" style={{ width: `${l3Progress....
  </div>
  <span className="text-[9px] font-bold whitespace-nowrap">{l3Progress.completed} / {l3Progress.total}<...
</div>
</div>
</div>
</div>
<div className="px-2 pb-2 flex justify-end">
  <div className="flex items-center gap-2">
    <Button
      type="button"
      variant="outline"
      className="border !border-white/30 text-white/70 hover:text-white hover:!border-white/50 hover:bg-t...
      style={{ fontSize: '12px', marginRight: '5px' }}
      onClick={() => startTransition(() => setIsSettingsPanelOpen(false))}
      data-testid="button-cancel-settings-panel"
    >
      Cancel
    </Button>
    <Button
      type="button"
      variant="outline"
      className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-transpa...
      style={{
        boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255,255,...
        fontSize: '12px'
      }}
      onClick={() => {
        localStorage.setItem('checkedCourses', JSON.stringify(checkedCourses));
        localStorage.setItem('courseGrades', JSON.stringify(courseGrades));
        saveDegreeToServer('checkedCourses', checkedCourses);
        saveDegreeToServer('courseGrades', courseGrades);
        toast({ title: "Settings saved", description: "Your progress has been saved." });
        startTransition(() => setIsSettingsPanelOpen(false));
      }}
      data-testid="button-save-settings-panel"
    >
      Save
    </Button>
  </div>
</div>
</div>
))}

<Dialog open={icsImportOpen} onOpenChange={(open) => { if (!open) { setIcsImportOpen(false); setIcsImportEvents...
  <DialogContent className="max-w-[640px] max-h-[85vh] bg-gradient-to-br from-gray-900/98 via-black/95 to-gray-...
    <DialogHeader className="px-5 py-3 border-b border-white/15" style={{ background: 'rgba(139, 92, 246, 0.15)...
      <DialogTitle className="text-sm font-normal text-white flex items-center gap-2">
        <Upload className="h-4 w-4 text-violet-400" />
        Import ICS Events {icsImportEvents.filter(e => e.selected).length} of {icsImportEvents.length} selected
      </DialogTitle>
    </DialogHeader>
    <div className="overflow-y-auto px-4 py-3 flex-1" style={{ maxHeight: 'calc(85vh - 140px)', scrollbarWidth:...
      <div className="flex items-center justify-between mb-3">
        <button
          className="text-[10px] text-violet-400 hover:text-violet-300 transition-colors"
          onClick={() => {
            const allSelected = icsImportEvents.every(e => e.selected);
            setIcsImportEvents(prev => prev.map(e => ({ ...e, selected: !allSelected })));
          }}
          data-testid="button-ics-toggle-all"
        >
          {icsImportEvents.every(e => e.selected) ? 'Deselect All' : 'Select All'}
        </button>
      </div>
    </div>
    <div className="flex flex-col gap-1.5">
      {icsImportEvents.map((ev, idx) => {

```



```

initialEditMode={!selectedCertCourse?.openInEdit}
semesterSettings={perSemesterSettings as any}
courseRank=(({}) => {
  const cc = selectedCertCourse!.courseCode.replace(/\s/g, '');
  for (const sk of semesterKeyOrder) {
    const pk = `${sk}:${cc}`;
    if (coursePlayPriority[pk]) return coursePlayPriority[pk];
  }
  return 0;
})();
usedRanks=(({}) => {
  const cc = selectedCertCourse!.courseCode.replace(/\s/g, '');
  let semKey = '';
  for (const sk of semesterKeyOrder) {
    const courses = semesterCourseAssignments[sk] || [];
    if (courses.some(c => c.code.replace(/\s/g, '').toUpperCase() === cc.toUpperCase())) { semKey = sk; break; }
  }
  if (!semKey) return [];
  const used: number[] = [];
  const semCourses = semesterCourseAssignments[semKey] || [];
  for (const sc of semCourses) {
    const pk = `${semKey}:${sc.code.replace(/\s/g, '')}`;
    const v = coursePlayPriority[pk];
    if (v && v > 0 && sc.code.replace(/\s/g, '').toUpperCase() !== cc.toUpperCase()) used.push(v);
  }
  return used;
})();
isSpringSummer=(({}) => {
  const cc = selectedCertCourse!.courseCode.replace(/\s/g, '');
  for (const sk of semesterKeyOrder) {
    if (!sk.startsWith('ss')) continue;
    const courses = semesterCourseAssignments[sk] || [];
    if (courses.some(c => c.code.replace(/\s/g, '').toUpperCase() === cc.toUpperCase())) return true;
  }
  return false;
})();
courseSpSuTerm=(({}) => {
  const cc = selectedCertCourse!.courseCode.replace(/\s/g, '');
  const sems = allSemesterSettingsRef.current;
  if (sems) {
    for (const sem of sems) {
      for (let i = 1; i <= 3; i++) {
        const sc = (sem as any)[`course${i}Code`] || '').replace(/\s/g, '');
        if (sc.toUpperCase() === cc.toUpperCase()) {
          return ((sem as any)[`course${i}SpringSummerTerm`] || 'full').toLowerCase();
        }
      }
    }
  }
  return 'full';
})();
courseRankA=(({}) => {
  const cc = selectedCertCourse!.courseCode.replace(/\s/g, '');
  for (const sk of semesterKeyOrder) {
    if (!sk.startsWith('ss')) continue;
    const pkA = `${sk}:${cc}:A`;
    if (coursePlayPriority[pkA] && coursePlayPriority[pkA] > 0) return coursePlayPriority[pkA];
  }
  return 0;
})();
courseRankB=(({}) => {
  const cc = selectedCertCourse!.courseCode.replace(/\s/g, '');
  for (const sk of semesterKeyOrder) {
    if (!sk.startsWith('ss')) continue;
    const pkB = `${sk}:${cc}:B`;
    if (coursePlayPriority[pkB] && coursePlayPriority[pkB] > 0) return coursePlayPriority[pkB];
  }
  return 0;
})();
usedRanksA=(({}) => {
  const cc = selectedCertCourse!.courseCode.replace(/\s/g, '');
  let semKey = '';
  for (const sk of semesterKeyOrder) {
    if (!sk.startsWith('ss')) continue;
    const courses = semesterCourseAssignments[sk] || [];
    if (courses.some(c => c.code.replace(/\s/g, '').toUpperCase() === cc.toUpperCase())) { semKey = sk; break; }
  }
  if (!semKey) return [];
  const used: number[] = [];
  for (const sc of (semesterCourseAssignments[semKey] || [])) {
    if (sc.code.replace(/\s/g, '').toUpperCase() === cc.toUpperCase()) continue;
    const v = coursePlayPriority[`${semKey}:${sc.code.replace(/\s/g, '')}:A`];
    if (v && v > 0) used.push(v);
  }
  return used;
})();
usedRanksB=(({}) => {
  const cc = selectedCertCourse!.courseCode.replace(/\s/g, '');
  let semKey = '';
  for (const sk of semesterKeyOrder) {
    if (!sk.startsWith('ss')) continue;
    const courses = semesterCourseAssignments[sk] || [];
    if (courses.some(c => c.code.replace(/\s/g, '').toUpperCase() === cc.toUpperCase())) { semKey = sk; break; }
  }
  if (!semKey) return [];
  const used: number[] = [];
  for (const sc of (semesterCourseAssignments[semKey] || [])) {
    if (sc.code.replace(/\s/g, '').toUpperCase() === cc.toUpperCase()) continue;
    const v = coursePlayPriority[`${semKey}:${sc.code.replace(/\s/g, '')}:B`];
    if (v && v > 0) used.push(v);
  }
  return used;
})();
onRankChange=({suffix, val} => {
  const cc = selectedCertCourse!.courseCode.replace(/\s/g, '');
  let semKey = '';
  for (const sk of semesterKeyOrder) {
    if (!sk.startsWith('ss')) continue;

```

```

const courses = semesterCourseAssignments[sk] || [];
if (courses.some(c => c.code.replace(/\s/g, '').toUpperCase() === cc.toUpperCase())) { semKey = sk; b...
}
if (semKey) {
  const pk = `${semKey}:${cc}:${suffix}`;
  handlePriorityChange(pk, val);
}
}
onClose={() => {
  startTransition(() => setSelectedCertCourse(null));
  if (isSchoolCoursesDialogOpen) {
    // School Courses dialog is still open underneath, just go back to it
  } else if (openedCourseFromDegreeTracking) {
    setOpenedCourseFromDegreeTracking(false);
    startTransition(() => setIsSettingsPanelOpen(true));
  }
}
}
onLiveColorChange=(colorUpdates) => {
  const courseCode = selectedCertCourse!.courseCode;
  const updatedCourses = [...coursesData.courses];
  const matchIdx = updatedCourses.findIndex(c => {
    const cCode = c.name.split(' - ')[0]?.trim().replace(/\s/g, '');
    return cCode === courseCode.replace(/\s/g, '');
  });
  if (matchIdx >= 0) {
    updatedCourses[matchIdx] = {
      ...updatedCourses[matchIdx],
      ...(colorUpdates.color ? { color: colorUpdates.color } : {}),
      ...(colorUpdates.colorEnd ? { colorEnd: colorUpdates.colorEnd } : {}),
      ...(colorUpdates.colorStops !== undefined ? { colorStops: colorUpdates.colorStops } : {}),
      ...(colorUpdates.borderColor !== undefined ? { borderColor: colorUpdates.borderColor } : {}),
      ...(colorUpdates.courseRowColor !== undefined ? { courseRowColor: colorUpdates.courseRowColor } : {}),
      ...(colorUpdates.taskBgColor !== undefined ? { taskBgColor: colorUpdates.taskBgColor } : {}),
    };
    setCoursesData({ courses: updatedCourses });
  }
}
onSaveCourseInfo=(updates) => {
  const courseCode = selectedCertCourse!.courseCode;
  const certKey = selectedCertCourse!.certKey;
  const updatedCourses = [...coursesData.courses];
  const matchIdx = updatedCourses.findIndex(c => {
    const cCode = c.name.split(' - ')[0]?.trim().replace(/\s/g, '');
    return cCode === courseCode.replace(/\s/g, '');
  });
  if (matchIdx >= 0) {
    updatedCourses[matchIdx] = {
      ...updatedCourses[matchIdx],
      professor: updates.professor || updatedCourses[matchIdx].professor,
      professorEmail: updates.professorEmail || updatedCourses[matchIdx].professorEmail,
      ...(updates.color ? { color: updates.color } : {}),
      ...(updates.colorEnd ? { colorEnd: updates.colorEnd } : {}),
      ...((updates as any).colorStops !== undefined ? { colorStops: (updates as any).colorStops } : {}),
      ...((updates as any).borderColor !== undefined ? { borderColor: (updates as any).borderColor } : {}),
      ...(updates.courseRowColor !== undefined ? { courseRowColor: updates.courseRowColor } : {}),
      ...(updates.taskBgColor !== undefined ? { taskBgColor: updates.taskBgColor } : {}),
    };
    saveCourses({ courses: updatedCourses });
  }
}
const computeSemesterDates = (term: string, yr: string) => {
  const y = parseInt(yr);
  if (!y) return { startDate: '', endDate: '', semesterType: '', springSummerTerm: '' };
  switch (term) {
    case 'fall': return { startDate: `${y}-09-08`, endDate: `${y}-12-07`, semesterType: 'fall', springS...
    case 'winter': return { startDate: `${y}-01-13`, endDate: `${y}-04-17`, semesterType: 'winter', spr...
    case 'spring_summer_full': return { startDate: `${y}-05-05`, endDate: `${y}-08-04`, semesterType: '...
    case 'spring_summer_first': return { startDate: `${y}-05-05`, endDate: `${y}-06-20`, semesterType: '...
    case 'spring_summer_second': return { startDate: `${y}-06-23`, endDate: `${y}-08-04`, semesterType:...
    default: return { startDate: '', endDate: '', semesterType: '', springSummerTerm: '' };
  }
}
const currentSemSettings = allSemesterSettingsRef.current;
if (currentSemSettings) {
  const cc = courseCode.replace(/\s/g, '');
  let found = false;
  for (const sem of currentSemSettings) {
    for (let i = 1; i <= 3; i++) {
      const semCode = ((sem as any)['course${i}Code'] || '').replace(/\s/g, '');
      if (semCode === cc) {
        found = true;
        const prefix = `course${i}`;
        const payload: Record<string, any> = {};
        if (updates.deliveryMode !== undefined) payload[`${prefix}DeliveryMode`] = updates.deliveryMode;
        if (updates.classDay !== undefined) payload[`${prefix}ClassDay`] = updates.classDay;
        if (updates.classDay2 !== undefined) payload[`${prefix}ClassDay2`] = updates.classDay2;
        if (updates.classTime !== undefined) payload[`${prefix}ClassTime`] = updates.classTime;
        if (updates.classEndTime !== undefined) payload[`${prefix}ClassEndTime`] = updates.classEndTime;
        if (updates.classTime2 !== undefined) payload[`${prefix}ClassTime2`] = updates.classTime2;
        if (updates.classEndTime2 !== undefined) payload[`${prefix}ClassEndTime2`] = updates.classEndTi...
        if (updates.zoomLink !== undefined) payload[`${prefix}ZoomLink`] = updates.zoomLink;
        if (updates.professor !== undefined) payload[`${prefix}Professor`] = updates.professor;
        if (updates.professorEmail !== undefined) payload[`${prefix}ProfessorEmail`] = updates.proffesso...
        if (updates.color !== undefined) payload[`${prefix}Color`] = updates.color;
        if (updates.colorEnd !== undefined) payload[`${prefix}ColorEnd`] = updates.colorEnd;
        if ((updates as any).colorStops !== undefined) payload[`${prefix}ColorStops`] = (updates as any)...
        if ((updates as any).borderColor !== undefined) payload[`${prefix}BorderColor`] = (updates as a...
        if (updates.courseRowColor !== undefined) payload[`${prefix}CourseRowColor`] = updates.courseRo...
        if (updates.taskBgColor !== undefined) payload[`${prefix}TaskBgColor`] = updates.taskBgColor;
        if ((updates as any).startDate) payload[`${prefix}StartDate`] = new Date((updates as any).start...
        if ((updates as any).endDate) payload[`${prefix}EndDate`] = new Date((updates as any).endDate +...
        if ((updates as any).springSummerTerm !== undefined) payload[`${prefix}SpringSummerTerm`] = (up...
        if (updates.semesterTerm && updates.year && !(updates as any).startDate) {
          const dates = computeSemesterDates(updates.semesterTerm, updates.year);
          if (dates.startDate) {
            payload[`${prefix}StartDate`] = new Date(dates.startDate + 'T12:00:00').toISOString();
            payload[`${prefix}EndDate`] = new Date(dates.endDate + 'T12:00:00').toISOString();
            if (dates.springSummerTerm) payload[`${prefix}SpringSummerTerm`] = dates.springSummerTerm;
          }
        }
      }
    }
  }
}

```



```

        if (t.startsWith('spring_summer')) return `ss${y}`;
        if (t === 'fall') return `f${y}`;
        if (t === 'winter') return `w${y}`;
        return '';
    }
    if ((updates as any).semesterKey) {
        const targetKey = (updates as any).semesterKey;
        const codeNorm = courseCode.replace(/\s/g, '').toUpperCase();
        const updated = { ...semesterCourseAssignments };
        for (const sk of Object.keys(updated)) {
            updated[sk] = (updated[sk] || []).filter(c => c.code.replace(/\s/g, '').toUpperCase() !== codeNorm);
        }
        if (!updated[targetKey]) updated[targetKey] = [];
        const existingEntry = Object.values(semesterCourseAssignments).flat().find(c => c.code.replace(/\s/g, ...
        const courseName = existingEntry?.name || courseCode.replace(/([A-Z]+)(\d+)/, '$1 $2');
        const fullName = existingEntry?.fullName || updatedCourses[matchIdx >= 0 ? matchIdx : 0]?.name?.split...
        updated[targetKey].push({ code: courseCode.replace(/\s/g, ''), name: courseName, fullName, period: '...
        saveSemesterAssignments(updated);
    }
    const certData = localStorage.getItem('certCourseData');
    const savedData = certData ? JSON.parse(certData) : {};
    savedData[certKey] = { ...savedData[certKey], ...updates };
    const cc = courseCode.replace(/\s/g, '');
    if (certKey !== cc && updates.deliveryMode) {
        savedData[cc] = { ...(savedData[cc] || {}), deliveryMode: updates.deliveryMode };
    }
    localStorage.setItem('certCourseData', JSON.stringify(savedData));
    saveDegreeToServer('certCourseData', savedData);
    setDeliveryModeVersion(v => v + 1);
})
onGradeCalculated=({grade, percent} => {
    const certKey = selectedCertCourse!.certKey;
    const courseCode = selectedCertCourse!.courseCode;
    updateGrade(certKey, grade);
    updatePercent(certKey, percent);
    const codeNorm = courseCode.replace(/\s/g, '').toUpperCase();
    for (const [ck, info] of Object.entries(certCourseMap)) {
        if (ck === certKey) continue;
        if (shouldStrikethrough(ck)) continue;
        const mapCode = info.code.replace(/\s/g, '').toUpperCase();
        if (mapCode === codeNorm || codeNorm.endsWith(mapCode) || mapCode.endsWith(codeNorm.replace(/^\s/, ''))) {
            updateGrade(ck, grade);
            updatePercent(ck, percent);
        }
    }
    const electiveSlots = ['LIBERAL', 'OPEN1', 'OPEN2', 'L2_LIBERAL', 'L2_OPEN1', 'L2_OPEN2', 'L3_OPEN1', 'L3_OPEN...
    for (const slot of electiveSlots) {
        if (slot === certKey) continue;
        const elVal = openElectives[slot];
        if (elVal && elVal.replace(/\s/g, '').toUpperCase().startsWith(codeNorm)) {
            updateGrade(slot, grade);
            updatePercent(slot, percent);
        }
    }
})
onDeleteCourse=({}) => {
    const courseCode = selectedCertCourse!.courseCode;
    const updatedCourses = coursesData.courses.filter(c => {
        const cCode = c.name.split(' - ')[0]?.trim().replace(/\s/g, '');
        return cCode !== courseCode.replace(/\s/g, '');
    });
    saveCourses({ courses: updatedCourses });
    startTransition(() => setSelectedCertCourse(null));
    toast({ title: "Course deleted", description: `${courseCode} has been removed.` });
})
onOpenEditTask=({task} => {
    startTransition(() => setSelectedCertCourse(null));
    setTimeout(() => {
        setEditingTask(task);
    }, 150);
})
semesterStart=({}) => {
    const cc = selectedCertCourse!.courseCode.replace(/\s/g, '').toUpperCase();
    const semStartDatesForDialog: Record<string, string> = {
        'ss2025': '2025-05-05', 'f2025': '2025-09-08', 'w2026': '2026-01-12',
        'ss2026': '2026-05-04', 'f2026': '2026-09-07', 'w2027': '2027-01-11',
        'ss2027': '2027-05-03', 'f2027': '2027-09-13', 'w2028': '2028-01-10', 'ss2028': '2028-05-01', 'f2028'...
    };
    for (const sk of semesterKeyOrder) {
        const courses = semesterCourseAssignments[sk] || [];
        if (courses.some(c => c.code.replace(/\s/g, '').toUpperCase() === cc)) {
            const dateStr = semStartDatesForDialog[sk];
            if (dateStr) return new Date(dateStr + 'T00:00:00');
        }
    }
    if (info.startDate) return new Date(info.startDate + 'T00:00:00');
    return semStart || new Date();
})()
readingWeekStart={readingWeekStart}
onPushUndo=({action} => pushUndo(action as UndoAction))
/>
);
})();
{pendingSleepTask && (
    <div className="fixed inset-0 z-[9999] flex items-center justify-center bg-black/50" onClick={() => setPendin...
        <div
            className="rounded-xl w-[400px] overflow-hidden flex flex-col text-white shadow-2xl"
            style={{
                background: `linear-gradient(180deg, ${colorSettings.mainBackground} 0%, color-mix(in srgb, ${colorSett...
                border: `1px solid rgba(255, 255, 255, 0.2)`,
                fontFamily: "Avenir, 'Avenir Next', -apple-system, BlinkMacSystemFont, sans-serif"
            }}
            onClick={(e) => e.stopPropagation()}
            data-testid="sleep-confirm-dialog"
        >

```

```

<div className="flex items-center gap-3 px-5 py-4 bg-black/30 border-b border-white/20">
  <MoonIcon className="h-5 w-5 text-purple-300" />
  <h3 className="text-sm font-semibold">Sleep Hours</h3>
</div>
<div className="px-5 py-5">
  <p className="text-sm text-white/90 leading-relaxed">
    This time slot falls during scheduled sleep hours. Are you sure you want to add a task here?
  </p>
</div>
<div className="flex items-center justify-end gap-3 px-5 py-4 border-t border-white/20">
  <Button
    variant="outline"
    className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-white/10"
    onClick={() => setPendingSleepTask(null)}
    data-testid="sleep-confirm-cancel"
  >
    Cancel
  </Button>
  <Button
    className="bg-white/20 text-white hover:bg-white/30 border border-white/30"
    onClick={() => {
      const { day, hour, minutes } = pendingSleepTask;
      const dueDate = new Date(day);
      dueDate.setHours(hour, minutes, 0, 0);
      setSelectedDate(dueDate);
      setNewTaskType("other");
      setInitialStartTime(`${hour.toString().padStart(2, '0')}:${minutes.toString().padStart(2, '0')}`);
      setInitialEndTime(`${(hour + 1).toString().padStart(2, '0')}:${minutes.toString().padStart(2, '0')}`);
      bringFlyoutToFront('addTask');
      setIsAddDialogOpen(true);
      setPendingSleepTask(null);
    }}
    data-testid="sleep-confirm-proceed"
  >
    Add Task Anyway
  </Button>
</div>
</div>
</div>
)}

/* Copyright - Right side of page, right-aligned with homework box */
{!isTodoFlyoutOpen && {
  <div
    className="fixed text-white/60 text-[11px] font-medium z-[70] pointer-events-none"
    style={{ bottom: '40px', right: `${calendarRight - calendarReduction + 3 + 7 - 6 + 2 + 4 + 2 - 17 - 7 - 2 + ...`
  >
    © 2026
  </div>
)}

/* Navigation Arrows with week dates - now in top tab on glass box */

/* Voice Input Floating Button */
<div className="fixed z-[9999]" style={{ bottom: '43px', left: '5px' }}>
  <button
    onClick={() => isVoiceListening ? stopVoiceInput() : startVoiceInput()}
    className={`flex items-center gap-2 rounded-full shadow-lg transition-all duration-300 ${isVoiceListening ? ...`
    style={{ padding: isVoiceListening ? '10px 16px 10px 12px' : '9px', minWidth: isVoiceListening ? '120px' : ...`
    data-testid="button-voice-input"
  >
    {isVoiceListening ? (
      <>
        <MicOff className="h-4 w-4 text-white shrink-0" />
        <div className="flex flex-col items-start min-w-0">
          <span className="text-[10px] text-white font-medium">Listening...</span>
          {voiceTranscript && <span className="text-[9px] text-white/70 truncate max-w-[160px]">{voiceTranscrip...
        </div>
        <span className="w-2 h-2 rounded-full bg-white animate-pulse shrink-0" />
      </>
    ) : (
      <Mic className="text-white" style={{ height: '18px', width: '18px' }} />
    )}
  </button>
</div>

/* News Ticker */
<NewsTickerPortal headlines={{
  ...weatherAlerts.map(a => ({ title: `& p ${a.title}`, source: '_ALERT_', link: '' })),
  ...(weatherData ? (() => {
    const WMO_DESC: Record<number, string> = { 0: 'Clear', 1: 'Mainly Clear', 2: 'Partly Cloudy', 3: 'Overcast', 45: 'Fo...
    const desc = WMO_DESC[weatherData.code] || 'Mixed';
    const items: { title: string; source: string; link: string }[] = [];
    items.push({ title: = 3) {
      const dayNames = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'];
      const forecastParts = weatherData.daily.slice(0, 3).map(d => {
        const dt = new Date(d.date + 'T12:00:00');
        const dayName = dayNames[dt.getDay()];
        return `${dayName}: ${Math.round(d.high)}°/${Math.round(d.low)}°`;
      });
      items.push({ title:  ({ 0: 'clear', 1: 'mostly clear', 2: 'partly cloudy', 3: 'overcast', 45: 'foggy', 4...
      const todayDesc = wmoShort(todayD.weatherCode || weatherData.code);
      briefParts.push('Currently ${Math.round(weatherData.temp)}° and {todayDesc}. Today's high ${Math.round(t...
      if (tomorrowD) {
        const tmrDesc = wmoShort(tomorrowD.weatherCode || 0);
        const tmrName = dayNames[new Date(tomorrowD.date + 'T12:00:00').getDay()];
        briefParts.push(`{tmrName}: ${tmrDesc}, ${Math.round(tomorrowD.high)}°/${Math.round(tomorrowD.low)}°.`);
      }
      if (day3D) {
        const d3Desc = wmoShort(day3D.weatherCode || 0);
        const d3Name = dayNames[new Date(day3D.date + 'T12:00:00').getDay()];
        briefParts.push(`{d3Name}: ${d3Desc}, ${Math.round(day3D.high)}°/${Math.round(day3D.low)}°.`);
      }
    }
  )}
}

```



```

    }
    items.push({ title: `

```

```

style={{
  left: '-10px',
  top: '50vh',
  transform: 'translateY(-50%)',
  pointerEvents: 'auto',
  display: (isSettingsPanelOpen || isSchoolCoursesDialogOpen) ? 'none' : undefined,
}}
onClick={() => {
  setIsAddChooserOpen(true);
}}
data-testid="left-add-task-tab"
title="Add"
>
<svg width="25" height="84" viewBox="0 0 25 84" style={{ display: 'block' }}>
  <path d="M0,0 L0,84 L9,84 Q9,75 15,75 L16,75 Q25,75 25,63 L25,21 Q25,9 16,9 L15,9 Q9,9 9,0 Z" fill="rgba(255,255,255,0.8)" stroke="rgba(255,255,255,0.8)" stroke-width="1" />
  <text x="16" y="46" textAnchor="middle" fill="rgba(255,255,255,0.8)" fontSize="16" fontWeight="bold">+</text>
</svg>
</div>

{/* Right binder tab - Projects */}
<div
  className={
    'fixed z-[10002] cursor-pointer${tabBounceEnabled ? ' right-tab-bounce' : ''}
  }
  style={{
    right: '-10px',
    top: '50vh',
    transform: 'translateY(-50%)',
    pointerEvents: 'auto',
    display: (isSettingsPanelOpen || isSchoolCoursesDialogOpen) ? 'none' : undefined,
  }}
  onClick={() => {
    window.location.href = '/projects';
  }}
  data-testid="right-projects-tab"
  title="Projects"
>
  <svg width="25" height="84" viewBox="0 0 25 84" style={{ display: 'block' }}>
    <path d="M25,0 L25,84 L16,84 Q16,75 10,75 L9,75 Q0,75 0,63 L0,21 Q0,9 9,9 L10,9 Q16,9 16,0 Z" fill="rgba(255,255,255,0.8)" stroke="rgba(255,255,255,0.8)" stroke-width="1" />
    <g transform="translate(2, 36)">
      <rect x="1" y="1" width="4" height="4" rx="0.5" fill="rgba(255,255,255,0.8)" stroke="rgba(255,255,255,0.8)" stroke-width="1" />
      <rect x="5" y="1" width="4" height="4" rx="0.5" fill="rgba(255,255,255,0.8)" stroke="rgba(255,255,255,0.8)" stroke-width="1" />
      <rect x="9" y="1" width="4" height="4" rx="0.5" fill="rgba(255,255,255,0.8)" stroke="rgba(255,255,255,0.8)" stroke-width="1" />
      <rect x="13" y="1" width="4" height="4" rx="0.5" fill="rgba(255,255,255,0.8)" stroke="rgba(255,255,255,0.8)" stroke-width="1" />
      <rect x="17" y="1" width="4" height="4" rx="0.5" fill="rgba(255,255,255,0.8)" stroke="rgba(255,255,255,0.8)" stroke-width="1" />
      <rect x="21" y="1" width="4" height="4" rx="0.5" fill="rgba(255,255,255,0.8)" stroke="rgba(255,255,255,0.8)" stroke-width="1" />
    </g>
  </svg>
</div>

{/* Spring out honeycombs for course readings - moved outside files button */}
{/* CPPA122 */}
<div
  className={
    'absolute transition-opacity duration-500 ease-out z-50 ${decorativeHoneycombHover === 'left' ? 'o...
  }
  style={{
    width: decorativeHoneycombHover === 'left' ? gridSizes.courseRowHeight * 0.9 : gridSizes.courseRowHeight * ...
    height: decorativeHoneycombHover === 'left' ? gridSizes.courseRowHeight * 0.9 : gridSizes.courseRowHeight * ...
    top: decorativeHoneycombHover === 'left' ? `${432 + gridSizes.courseRowHeight * 1.2}px` : '455px',
    right: decorativeHoneycombHover === 'left' ? '-26px' : '30px',
    transformOrigin: 'center center'
  }}
>
  <div
    className="relative w-full h-full cursor-pointer"
    onClick={() => setReadingsPopupCourse(prev => prev === 'cpa122' ? null : 'cpa122')}
  >
    <img src={hexIcon} alt="" className="w-full h-full object-contain" style={{ filter: 'drop-shadow(2px 2px 1px...
    <FolderOpen className="absolute inset-0 m-auto h-4 w-4" style={{ color: 'white', strokeWidth: 3 }} />
  </div>
</div>
{/* CFNF400 */}
<div
  className={
    'absolute transition-opacity duration-500 ease-out z-50 ${decorativeHoneycombHover === 'left' ? 'o...
  }
  style={{
    width: decorativeHoneycombHover === 'left' ? gridSizes.courseRowHeight * 0.9 : gridSizes.courseRowHeight * ...
    height: decorativeHoneycombHover === 'left' ? gridSizes.courseRowHeight * 0.9 : gridSizes.courseRowHeight * ...
    top: decorativeHoneycombHover === 'left' ? `${432 + gridSizes.courseRowHeight * 2.2}px` : '455px',
    right: decorativeHoneycombHover === 'left' ? '-26px' : '30px',
    transformOrigin: 'center center',
    transitionDelay: '50ms'
  }}
>
  <div
    className="relative w-full h-full cursor-pointer"
    onClick={() => setReadingsPopupCourse(prev => prev === 'cfnf400' ? null : 'cfnf400')}
  >
    <img src={hexIcon} alt="" className="w-full h-full object-contain" style={{ filter: 'drop-shadow(2px 2px 1px...
    <FolderOpen className="absolute inset-0 m-auto h-4 w-4" style={{ color: 'white', strokeWidth: 3 }} />
  </div>
</div>
{/* CASL101 */}
<div
  className={
    'absolute transition-opacity duration-500 ease-out z-50 ${decorativeHoneycombHover === 'left' ? 'o...
  }
  style={{
    width: decorativeHoneycombHover === 'left' ? gridSizes.courseRowHeight * 0.9 : gridSizes.courseRowHeight * ...
    height: decorativeHoneycombHover === 'left' ? gridSizes.courseRowHeight * 0.9 : gridSizes.courseRowHeight * ...
    top: decorativeHoneycombHover === 'left' ? `${432 + gridSizes.courseRowHeight * 3.2}px` : '455px',
    right: decorativeHoneycombHover === 'left' ? '-26px' : '30px',
    transformOrigin: 'center center',
    transitionDelay: '100ms'
  }}
>
  <div
    className="relative w-full h-full cursor-pointer"
    onClick={() => setReadingsPopupCourse(prev => prev === 'casl101' ? null : 'casl101')}
  >
    <img src={hexIcon} alt="" className="w-full h-full object-contain" style={{ filter: 'drop-shadow(2px 2px 1px...
    <FolderOpen className="absolute inset-0 m-auto h-4 w-4" style={{ color: 'white', strokeWidth: 3 }} />
  </div>
</div>

```

```

/* Sticky Notes removed – functionality moved to Quick Notes (/onenote) */

/* Main Content - Full width, positioned below unified header */
<main className="flex-1 pt-2 pb-2 flex flex-col overflow-visible relative z-10 min-h-0" style={{ paddingLeft: '...

{
  /* Completed Tasks Popup */
  <Dialog open={isCompletedTasksOpen} onOpenChange={setIsCompletedTasksOpen}>
    <DialogContent className="max-w-md max-h-[80vh] overflow-hidden flex flex-col text-[11px] border-0 text-w...
      <div className="flex items-center justify-between px-4 py-3 border-b border-white/40 flex-shrink-0 roun...
        <div className="flex items-center gap-2">
          <CheckSquare className="text-white" style={{ width: '15px', height: '15px' }} />
          <h2 className="font-normal text-white" style={{ fontFamily: "Avenir, 'Avenir Next', -apple-system, ...
            COMPLETED TASKS
          </h2>
        </div>
      </div>
      {allTasks.filter(t => t.isCompleted).length > 0 && (
        <button
          className="text-xs text-white/70 hover:text-red-400 underline transition-colors font-normal"
          style={{ marginRight: '15px' }}
          onClick={() => {
            if (confirm('Permanently delete ALL completed tasks? This cannot be undone.')) {
              const completed = allTasks.filter(t => t.isCompleted);
              completed.forEach(t => deleteMutation.mutate(t.id));
            }
          }}
          data-testid="button-delete-all-completed"
        >
          Delete All
        </button>
      )}
    </div>
    <div className="flex-1 overflow-y-auto space-y-2 pr-2 px-3 pt-2">
      {(() => {
        const completedTasks = allTasks
          .filter(t => t.isCompleted)
          .sort((a, b) => {
            const aTime = a.completedAt ? new Date(a.completedAt).getTime() : new Date(a.dueDate).getTime();
            const bTime = b.completedAt ? new Date(b.completedAt).getTime() : new Date(b.dueDate).getTime();
            return bTime - aTime;
          });

        if (completedTasks.length === 0) {
          return <div className="text-muted-foreground text-sm py-4 text-center">No completed tasks yet</div>;
        }

        const getCourseColor = (courseName: string | null | undefined) => {
          if (!courseName) return '#888888';
          const course = coursesData.courses.find(c => c.name && courseName.includes(c.name.split(' - ')[0]));
          return course?.color || '#888888';
        };

        return completedTasks.map(task => (
          <div
            key={task.id}
            className="relative overflow-hidden rounded-md"
            data-testid={`completed-task-row-${task.id}`}
          >
            <div
              className="absolute inset-y-0 right-0 flex items-center justify-end px-4 bg-red-600"
              style={{ width: '80px' }}
            >
              <Trash2 className="h-4 w-4 text-white" />
            </div>
            <div
              className="flex items-center gap-3 p-2 border border-white/10 bg-inherit relative"
              style={{
                background: `linear-gradient(180deg, ${colorSettings.mainBackground} 0%, color-mix(in srgb,...
                touchAction: 'pan-y',
                transition: 'transform 0.2s ease-out',
                cursor: 'grab'
              }}
            >
              onTouchStart={(e) => {
                const el = e.currentTarget;
                const startX = e.touches[0].clientX;
                let currentX = 0;
                el.style.transition = 'none';
                const onMove = (ev: TouchEvent) => {
                  currentX = ev.touches[0].clientX - startX;
                  if (currentX > 0) currentX = 0;
                  el.style.transform = `translateX(${currentX}px)`;
                };
                const onEnd = () => {
                  el.style.transition = 'transform 0.2s ease-out';
                  if (currentX < -60) {
                    el.style.transform = 'translateX(-100%)';
                    setTimeout(() => deleteMutation.mutate(task.id), 200);
                  } else {
                    el.style.transform = 'translateX(0)';
                  }
                  document.removeEventListener('touchmove', onMove);
                  document.removeEventListener('touchend', onEnd);
                };
                document.addEventListener('touchmove', onMove, { passive: true });
                document.addEventListener('touchend', onEnd);
              }}
              onMouseDown={(e) => {
                const el = e.currentTarget;
                const startX = e.clientX;
                let currentX = 0;
                el.style.transition = 'none';
                el.style.cursor = 'grabbing';
                const onMove = (ev: MouseEvent) => {
                  currentX = ev.clientX - startX;
                  if (currentX > 0) currentX = 0;
                  el.style.transform = `translateX(${currentX}px)`;
                };
                const onUp = () => {

```

```

        el.style.transition = 'transform 0.2s ease-out';
        el.style.cursor = 'grab';
        if (currentX < -60) {
            el.style.transform = 'translateX(-100%)';
            setTimeout(() => deleteMutation.mutate(task.id), 200);
        } else {
            el.style.transform = 'translateX(0)';
        }
        document.removeEventListener('mousemove', onMove);
        document.removeEventListener('mouseup', onUp);
    });
    document.addEventListener('mousemove', onMove);
    document.addEventListener('mouseup', onUp);
    })
}
>
<input
  type="checkbox"
  checked={true}
  onChange={(e) => completeMutation.mutate({ id: task.id, isCompleted: e.target.checked })}
  className="h-4 w-4 rounded-sm cursor-pointer flex-shrink-0"
  style={{ accentColor: getCourseColor(task.courseName) }}
  data-testid={`completed-checkbox-${task.id}`}
/>
<div className="flex-1 min-w-0">
  <div className="text-sm font-medium truncate">{task.title}</div>
  <div className="text-xs text-muted-foreground flex items-center gap-2">
    <span style={{ color: getCourseColor(task.courseName) }}>{task.courseName?.split(' - ')[0...
    <span>*</span>
    <span>{format(new Date(task.dueDate), 'MMM d, yyyy')}</span>
  </div>
</div>
</div>
</div>
));
})();
</div>
<div className="px-4 py-3 border-t border-white/20 flex items-center justify-end flex-shrink-0" style={...
  <Button
    variant="outline"
    onClick={() => setIsCompletedTasksOpen(false)}
    className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-transpa...
    style={{
      boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255,255,...
      fontSize: '12px'
    }}
    data-testid="button-done-completed-tasks"
  >
    Done
  </Button>
</div>
</DialogContent>
</Dialog>
{/* Add Task Flyout - Burst from Top */}
<div
  className={`fixed transition-[opacity,transform] ease-out ${isAddDialogOpen ? 'opacity-100 scale-100' : '...'
  style={{
    width: '900px',
    maxWidth: '95vw',
    height: '85vh',
    top: '50%',
    left: '50%',
    transform: isAddDialogOpen ? 'translate(-50%, -50%) scale(1)' : 'translate(-50%, -50%) scale(0)',
    transformOrigin: '50% calc(-50vh + 42.5vh)',
    transitionDuration: '400ms',
    zIndex: getFlyoutZIndex('addTask')}
  }}
  onClick={() => bringFlyoutToFront('addTask')}
>
  {/* Flyout content */}
  <section
    className="h-full overflow-hidden flex flex-col rounded-xl text-white [&_label]:font-normal"
    style={{
      background: `linear-gradient(180deg, ${colorSettings.mainBackground} 0%, color-mix(in srgb, ${colorSe...
      border: '1px solid rgba(255, 255, 255, 0.2)',
      boxShadow: '0 10px 40px rgba(0,0,0,0.3)'
    }}
    data-testid="section-add-task"
  >
    <div className="flex items-center justify-between px-4 py-3 border-b border-white/40 rounded-t-xl styl...
      <div className="flex items-center gap-2">
        <CheckSquare className="text-white" style={{ width: '15px', height: '15px' }} />
        <h2 className="font-normal text-white" style={{ fontFamily: 'Avenir, 'Avenir Next', -apple-system, ...
          ADD TASK
        </h2>
      </div>
      <div className="flex items-center gap-3">
        <Button
          variant="outline"
          className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-trans...
          style={{
            boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255,25...
            fontSize: '12px'
          }}
          onMouseEnter={(e) => {
            e.currentTarget.style.boxShadow = '0 0 8px rgba(255,255,255,0.8), 0 0 16px rgba(255,255,255,0.6...
          }}
          onMouseLeave={(e) => {
            e.currentTarget.style.boxShadow = '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4...
          }}
          onClick={() => {
            const form = document.querySelector('[data-task-form]') as HTMLFormElement;
            if (form) form.requestSubmit();
          }}
          data-testid="button-submit-task-header"
        >
          <Plus className="h-4 w-4 mr-2" />
          Add Task
        </Button>
      </div>
    </div>
  </div>

```

```

</Button>
<Button
  variant="outline"
  className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-trans...
  style={{
    fontSize: '12px',
    boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255,25...
  }}
  onMouseEnter={(e) => {
    e.currentTarget.style.boxShadow = '0 0 8px rgba(255,255,255,0.8), 0 0 16px rgba(255,255,255,0.6...
  }}
  onMouseLeave={(e) => {
    e.currentTarget.style.boxShadow = '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4...
  }}
  onClick={() => {
    setIsAddDialogOpen(false);
    setInitialStartTime("");
    setInitialEndTime("");
    setNewTaskType("module");
  }}
  data-testid="button-close-add-task"
>
  Cancel
</Button>
<button
  onClick={() => {
    setIsAddDialogOpen(false);
    setInitialStartTime("");
    setInitialEndTime("");
    setNewTaskType("module");
  }}
  className="text-white hover:text-white/80 transition-colors p-1"
  data-testid="button-x-close-add-task"
>
  <X className="h-4 w-4" />
</button>
</div>
</div>
<div className="p-4 flex-1 overflow-y-auto [&_label]:text-white [&_label]:font-normal [&_input]:font-no...
  <TaskForm
    keys={['add-task-form-${selectedDate?.getTime() || 0}-${initialStartTime}-${initialEndTime}-${newTask...
    weekNumber={selectedWeek}
    initialDate={selectedDate}
    initialType={newTaskType}
    initialStartTime={initialStartTime}
    initialEndTime={initialEndTime}
    hideSubmitButton={true}
    onSuccess={() => {
      setIsAddDialogOpen(false);
      setInitialStartTime("");
      setInitialEndTime("");
    }}
  />
</div>
</section>
</div>

```

```

/* Add Chooser Dialog */
{isAddChooserOpen && (
  <div className="fixed inset-0 z-[10003] flex items-center justify-center bg-black/30" onClick={() => setI...
    <div
      className="rounded-xl w-[320px] overflow-hidden flex flex-col text-white shadow-2xl"
      style={{
        fontFamily: "Avenir, 'Avenir Next', -apple-system, BlinkMacSystemFont, sans-serif",
        background: `linear-gradient(180deg, ${colorSettings.mainBackground} 0%, color-mix(in srgb, ${color...
        border: '1px solid rgba(255, 255, 255, 0.2)',
        boxShadow: '0 10px 40px rgba(0,0,0,0.3)',
      }}
      onClick={(e) => e.stopPropagation()}
    >
      <div className="flex items-center justify-between px-4 py-3 border-b border-white/40 rounded-t-xl" st...
        <span className="text-[12px] font-normal text-white" style={{ textShadow: '0 1px 2px rgba(0,0,0,0.2...
          <button
            onClick={() => setIsAddChooserOpen(false)}
            className="text-white hover:text-white/80 transition-colors p-1"
            data-testid="button-close-chooser"
          >
            <X className="h-4 w-4" />
          </button>
        </div>
      <div className="flex flex-col gap-3 p-6">
        <Button
          variant="outline"
          className="w-full border !border-white/40 text-white hover:text-white hover:!border-white hover:b...
          style={{ boxShadow: '0 0 6px rgba(255,255,255,0.4), 0 0 12px rgba(255,255,255,0.2)' }}
          onClick={() => {
            setIsAddChooserOpen(false);
            setQuickAddStep(0);
            setQuickAddData({ type: '', title: '', courseName: '', dueDate: '', dueDateHour: '18', dueDateM...
            setIsQuickAddOpen(true);
          }}
          data-testid="button-chooser-add-task"
        >
          + Add Task
        </Button>
        <Button
          variant="outline"
          className="w-full border !border-white/40 text-white hover:text-white hover:!border-white hover:b...
          style={{ boxShadow: '0 0 6px rgba(255,255,255,0.4), 0 0 12px rgba(255,255,255,0.2)' }}
          onClick={() => {
            setIsAddChooserOpen(false);
            setEditingProject(null);
            setProjectWizardStep(0);
            setProjectWizardData({ name: '', description: '', color: '#6366F1', status: 'planning', targetD...
            setProjectDialogOpen(true);
          }}
          data-testid="button-chooser-add-project"
        >

```



```

        Course
      </button>
    </div>
    <div className="border-t border-white/15 mt-3 pt-3">
      <input
        type="file"
        accept=".ics"
        className="hidden"
        ref={icsFileInputRef}
        onChange={handleIcsFileUpload}
        data-testid="input-ics-file"
      />
      <button
        className="w-full px-3 py-2.5 rounded-lg text-[12px] text-left transition-opacity duration-200"
        onClick={() => icsFileInputRef.current?.click()}
        data-testid="button-import-ics"
      >
        <Upload className="h-3.5 w-3.5 text-violet-400" />
        Import from .ICS File
        <span className="text-[9px] text-white/40 ml-auto">Calendar events</span>
      </button>
    </div>
  </div>
)}

{/* Step 1: Title */}
{quickAddStep === 1 && (
  <div className="flex flex-col gap-3 flex-1">
    <div className="text-center mb-2">
      <FileText className="h-8 w-8 text-sky-400 mx-auto mb-2" />
      <h3 className="text-sm font-medium text-white">Task Name</h3>
      <p className="text-[9px] text-white/50 mt-1">Enter the {quickAddData.type} name</p>
    </div>
    <FastInput
      type="text"
      value={quickAddData.title}
      onChange={(v) => setQuickAddData(p => ({ ...p, title: v })))
      placeholder="e.g. Chapter 5 {quickAddData.type}"
      className="w-full bg-white/10 border border-white/15 rounded-lg px-4 py-3 text-white text-[13px] font-medium"
      autoFocus
      data-testid="quick-add-title"
      onKeyDown={(e) => { if (e.key === 'Enter' && quickAddData.title.trim()) setQuickAddStep(2); }}
    />
  </div>
)}

{/* Step 2: Course */}
{quickAddStep === 2 && (
  <div className="flex flex-col gap-2">
    <div className="text-center mb-2">
      <GraduationCap className="h-8 w-8 text-emerald-400 mx-auto mb-2" />
      <h3 className="text-sm font-medium text-white">Course</h3>
      <p className="text-[9px] text-white/50 mt-1">Select the course</p>
    </div>
    <button
      className={`px-3 py-2.5 rounded-lg text-[12px] text-left transition-opacity duration-200 ${quickAddData.courseName ? 'text-white' : 'text-white/50'} `}
      onClick={() => { setQuickAddData(p => ({ ...p, courseName: '' })); setQuickAddStep(3); }}
      data-testid="quick-add-course-none"
    >
      No course
    </button>
    {COURSES.map(course => (
      <button
        key={course.code}
        className={`px-3 py-2.5 rounded-lg text-[12px] text-left transition-opacity duration-200 ${quickAddData.courseName === course.code ? 'text-white' : 'text-white/50'} `}
        onClick={() => { setQuickAddData(p => ({ ...p, courseName: course.code })); setQuickAddStep(3); }}
        data-testid={`quick-add-course-${course.code}`}
      >
        {course.code} - {course.name}
      </button>
    ))}
  </div>
)}

{/* Step 3: Due Date + Start/End Time */}
{quickAddStep === 3 && (
  <div className="flex flex-col gap-3">
    <div className="text-center mb-2">
      <Calendar className="h-8 w-8 text-amber-400 mx-auto mb-2" />
      <h3 className="text-sm font-medium text-white">Date & Time</h3>
      <p className="text-[9px] text-white/50 mt-1">When is it due?</p>
    </div>
    <input
      type="date"
      value={quickAddData.dueDate}
      onChange={(e) => { const v = e.target.value; startTransition(() => setQuickAddData(p => ({ ...p, dueDate: v })); }}
      className="w-full bg-white/10 border border-white/15 rounded-lg px-4 py-3 text-white text-[13px] font-medium"
      data-testid="quick-add-due-date"
    />
    <div className="flex gap-2 items-center">
      <span className="text-white/50 text-[11px]">Due time:</span>
      <select
        value={quickAddData.dueDateHour}
        onChange={(e) => { const v = e.target.value; startTransition(() => setQuickAddData(p => ({ ...p, dueDateHour: v })); }}
        className="bg-white/10 border border-white/15 rounded-lg px-2 py-2 text-white text-[12px] font-medium"
        data-testid="quick-add-hour"
      >
        {Array.from({ length: 24 }, (_, i) => (
          <option key={i} value={i.toString().padStart(2, '0')} style={{ color: 'white', background: 'linear-gradient(to right, transparent 49%, #000000 49% 51%, #000000 51% 53%, transparent 53%)" }}>
            {i === 0 ? '12 AM' : i < 12 ? `${i} AM` : i === 12 ? '12 PM' : `${i - 12} PM`}
          </option>
        ))}
      </select>
      <span className="text-white/40"></span>
      <select
        value={quickAddData.dueDateMinute}
        onChange={(e) => { const v = e.target.value; startTransition(() => setQuickAddData(p => ({ ...p, dueDateMinute: v })); }}
        className="bg-white/10 border border-white/15 rounded-lg px-2 py-2 text-white text-[12px] font-medium"
      >
        {Array.from({ length: 60 }, (_, i) => (
          <option key={i} value={i.toString().padStart(2, '0')} style={{ color: 'white', background: 'linear-gradient(to right, transparent 49%, #000000 49% 51%, #000000 51% 53%, transparent 53%)" }}>
            {i === 0 ? '00' : i < 10 ? `0${i}` : i}
          </option>
        ))}
      </select>
    </div>
  </div>
)}

```



```

        {REMINDER_OPTIONS.map(opt => {
          <option key={opt.value} value={opt.value} style={{ color: 'black' }}>{opt.label}</option>
        })}
      </select>
    </div>
  )}
</div>
))
</div>

{/* Step 7: Attachments */}
{quickAddStep === 7 && (
  <div className="flex flex-col gap-3">
    <div className="text-center mb-2">
      <Paperclip className="h-8 w-8 text-cyan-400 mx-auto mb-2" />
      <h3 className="text-sm font-medium text-white">Attachments</h3>
      <p className="text-[9px] text-white/50 mt-1">Add attachments</p>
    </div>
    <label className="flex items-center justify-center gap-2 px-4 py-3 rounded-lg border border-das...>
      <upload className="h-4 w-4 text-white/50" />
      <span className="text-white/50 text-[12px]">Upload file</span>
      <input
        type="file"
        className="hidden"
        multiple
        onChange={async (e) => {
          const files = e.target.files;
          if (!files) return;
          for (const file of Array.from(files)) {
            try {
              const response = await uploadDroppedFile(file);
              if (response?.objectPath) {
                setQuickAddData(p => ({ ...p, attachments: [...p.attachments, response.objectPath] ...
              })
            } catch (err) {
              toast({ title: "Upload failed", description: file.name, variant: "destructive" });
            }
          }
        }}
        data-testid="quick-add-file-upload"
      />
    </label>
    <div className="flex gap-2">
      <input
        type="url"
        value={quickAddData.pasteUrl}
        onChange={(e) => { const v = e.target.value; startTransition(() => setQuickAddData(p => ({ ...
          placeholder="Paste URL to attach..."
          className="flex-1 bg-white/10 border border-white/15 rounded-lg px-3 py-2 text-white text-[...
          data-testid="quick-add-paste-url"
          onKeyDown={(e) => {
            if (e.key === 'Enter' && quickAddData.pasteUrl.trim()) {
              setQuickAddData(p => ({ ...p, attachments: [...p.attachments, p.pasteUrl.trim()], paste...
            }
          }}
        />
      <button
        onClick={() => {
          if (quickAddData.pasteUrl.trim()) {
            setQuickAddData(p => ({ ...p, attachments: [...p.attachments, p.pasteUrl.trim()], paste...
          })
        }}
        className="px-3 py-2 rounded-lg text-[11px] bg-white/15 text-white hover:bg-white/25 transi...
        data-testid="quick-add-paste-url-add"
      >
        Add
      </button>
    </div>
    {quickAddData.attachments.length > 0 && (
      <div className="flex flex-col gap-1.5">
        {quickAddData.attachments.map((att, idx) => (
          <div key={idx} className="flex items-center gap-2 bg-white/5 rounded-lg px-3 py-2 border ...>
            <Paperclip className="h-3 w-3 text-white/40 flex-shrink-0" />
            <span className="text-white/70 text-[11px] truncate flex-1">{att.split('/').pop()} || at...
            <button
              onClick={() => setQuickAddData(p => ({ ...p, attachments: p.attachments.filter((_, i)...
              className="text-white/30 hover:text-red-400 transition-colors"
              data-testid={`quick-add-remove-attachment-${idx}`}
            >
              <X className="h-3 w-3" />
            </button>
          </div>
        ))}
      </div>
    ))
    {quickAddData.attachments.length === 0 && (
      <p className="text-white/30 text-[10px] text-center">No attachments added yet</p>
    ))
  </div>
)}

{/* Step 8: Notes & Reference Links */}
{quickAddStep === 8 && (
  <div className="flex flex-col gap-3">
    <div className="text-center mb-2">
      <StickyNote className="h-8 w-8 text-yellow-400 mx-auto mb-2" />
      <h3 className="text-sm font-medium text-white">Notes & Links</h3>
      <p className="text-[9px] text-white/50 mt-1">Add notes and reference links</p>
    </div>
    <div>
      <span className="text-white/50 text-[11px]">Notes</span>
      <textarea
        value={quickAddData.notes}
        onChange={(e) => { const v = e.target.value; startTransition(() => setQuickAddData(p => ({ ...
          placeholder="Add any notes..."
          rows={3}
          className="w-full mt-1 bg-white/10 border border-white/15 rounded-lg px-3 py-2 text-white t...
          data-testid="quick-add-notes"
        />
    </div>
  </div>
)}

```

```

    </div>
    <div>
      <span className="text-white/50 text-[11px]">Reference link</span>
      <input
        type="url"
        value={quickAddData.referenceLink}
        onChange={(e) => { const v = e.target.value; startTransition(() => setQuickAddData(p => ({ ...
          placeholder="https://..."
          className="w-full mt-1 bg-white/10 border border-white/15 rounded-lg px-3 py-2 text-white t...
          data-testid="quick-add-reference-link"
        />
      </div>
    </div>
  )}

  { /* Step 9: Subtasks & Project */ }
  {quickAddStep === 9 && (
    <div className="flex flex-col gap-3">
      <div className="text-center mb-2">
        <ListChecks className="h-8 w-8 text-indigo-400 mx-auto mb-2" />
        <h3 className="text-sm font-medium text-white">Subtasks & Project</h3>
        <p className="text-[9px] text-white/50 mt-1">Add subtasks and link to a project</p>
      </div>
      <div>
        <span className="text-white/50 text-[11px]">Subtasks</span>
        <div className="flex gap-2 mt-1">
          <input
            type="text"
            value={quickAddData.subtaskInput}
            onChange={(e) => { const v = e.target.value; startTransition(() => setQuickAddData(p => ({ ...
              placeholder="Add a subtask..."
              className="flex-1 bg-white/10 border border-white/15 rounded-lg px-3 py-2 text-white text...
              data-testid="quick-add-subtask-input"
            onKeyDown={(e) => {
              if (e.key === 'Enter' && quickAddData.subtaskInput.trim()) {
                setQuickAddData(p => ({ ...p, subtasks: [...p.subtasks, { title: p.subtaskInput.trim(...
              }
            }
          />
          <button
            onClick={() => {
              if (quickAddData.subtaskInput.trim()) {
                setQuickAddData(p => ({ ...p, subtasks: [...p.subtasks, { title: p.subtaskInput.trim(...
              }
            }
            className="px-3 py-2 rounded-lg text-[11px] bg-white/15 text-white hover:bg-white/25 tran...
            data-testid="quick-add-subtask-add"
          >
            Add
          </button>
        </div>
      </div>
      {quickAddData.subtasks.length > 0 && (
        <div className="flex flex-col gap-1.5 mt-2">
          {quickAddData.subtasks.map((st, idx) => (
            <div key={idx} className="flex items-center gap-2 bg-white/5 rounded-lg px-3 py-2 borde...
              <div className="w-3 h-3 rounded-sm border border-white/30 flex-shrink-0" />
              <span className="text-white/70 text-[11px] flex-1">{st.title}</span>
              <button
                onClick={() => setQuickAddData(p => ({ ...p, subtasks: p.subtasks.filter((_, i) => ...
                  className="text-white/30 hover:text-red-400 transition-colors"
                  data-testid={`quick-add-remove-subtask-${idx}`}
                >
                  <X className="h-3 w-3" />
                </button>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
    <div className="border-t border-white/10 pt-3">
      <span className="text-white/50 text-[11px]">Link to project</span>
      <select
        value={quickAddData.projectId ?? ''}
        onChange={(e) => { const v = e.target.value; startTransition(() => setQuickAddData(p => ({ ...
          className="w-full mt-1 bg-white/10 border border-white/15 rounded-lg px-3 py-2 text-white t...
          data-testid="quick-add-project"
        >
          <option value="" style={{ color: 'black' }}>No project</option>
          {allProjects.map(proj) => (
            <option key={proj.id} value={proj.id} style={{ color: 'black' }}>{proj.name}</option>
          </div>
        </select>
      </div>
    </div>
  )}

  { /* Step 10: Repeat */ }
  {quickAddStep === 10 && (
    <div className="flex flex-col gap-3">
      <div className="text-center mb-2">
        <Repeat2 className="h-8 w-8 text-purple-400 mx-auto mb-2" />
        <h3 className="text-sm font-medium text-white">Repeat</h3>
        <p className="text-[9px] text-white/50 mt-1">Set task repetition</p>
      </div>
      <div className="flex flex-col gap-2">
        {REPEAT_TYPES.map(rt => (
          <button
            key={rt}
            className={`px-3 py-2.5 rounded-lg text-[12px] text-left transition-opacity duration-200 ...
            onClick={() => setQuickAddData(p => ({ ...p, repeatType: rt })))}
            data-testid={`quick-add-repeat-${rt}`}
          >
            {rt.charAt(0).toUpperCase() + rt.slice(1)}
          </button>
        </div>
      </div>
      {quickAddData.repeatType === 'custom' && (
        <div className="flex gap-2 items-center mt-1">

```

```

<span className="text-white/50 text-[11px]">Every</span>


```

```

    {quickAddData.projectId && (
      <div className="flex justify-between text-[11px]">
        <span className="text-white/50">Project</span>
        <span className="text-white">{allProjects.find(p => p.id === quickAddData.projectId)?.nam...
      </div>
    )}
    {quickAddData.repeatType !== 'none' && (
      <div className="flex justify-between text-[11px]">
        <span className="text-white/50">Repeat</span>
        <span className="text-white">
          {quickAddData.repeatType === 'custom'
            ? `Every ${quickAddData.repeatInterval || 1} ${quickAddData.repeatIntervalUnit || 'da...
              : quickAddData.repeatType.charAt(0).toUpperCase() + quickAddData.repeatType.slice(1)}
            {quickAddData.repeatEndDate ? ` until ${format(new Date(quickAddData.repeatEndDate + 'T...
          </span>
        </div>
      )}
    </div>
  </div>
)}
</div>

{/* Footer with navigation */}
<div className="flex items-center justify-between px-4 py-3 border-t border-white/10 bg-black/20 flex...
  <button
    onClick={() => { if (quickAddStep > 0) setQuickAddStep(s => s - 1); else handleQuickAddClose(); }}
    className="inline-flex items-center justify-center rounded-md px-3 py-1.5 text-white transition-o...
    style={{ fontSize: "11px", border: '1.5px solid rgba(255,255,255,0.6)', background: 'linear-gradi...
    onMouseEnter={(e) => { e.currentTarget.style.background = 'linear-gradient(180deg, rgba(255,255,2...
    onMouseLeave={(e) => { e.currentTarget.style.background = 'linear-gradient(180deg, rgba(255,255,2...
    data-testid="quick-add-back"
  >
    {quickAddStep === 0 ? 'Cancel' : <<ChevronLeft className="h-3 w-3 mr-1" /> Back</> }
  </button>

  {quickAddStep < 11 && quickAddStep > 0 && (() => {
    const canNext = !((quickAddStep === 1 && !quickAddData.title.trim()) || (quickAddStep === 3 && !q...
    return (
      <Button
        variant="outline"
        onClick={() => setQuickAddStep(s => s + 1)}
        disabled={!canNext}
        className="border !border-white/50 text-white transition-opacity duration-200 disabled:opacit...
        style={{
          boxShadow: canNext ? "0 0 6px rgba(255,255,255,0.4), 0 0 12px rgba(255,255,255,0.2)" : "none",
          fontSize: "11px",
        }}
        data-testid="quick-add-next"
      >
        Next <ChevronRight className="h-3 w-3 ml-1" />
      </Button>
    );
  })();

  {quickAddStep === 11 && (
    <Button
      variant="outline"
      className="border !border-white/50 text-white transition-opacity duration-200"
      style={{
        boxShadow: "0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255,...
        fontSize: "11px",
      }}
    >
      {onClick={async () => {
        try {
          const dueDate = new Date(quickAddData.dueDate + 'T' + quickAddData.dueDateHour + ':' + quic...
          let startDate: string | null = null;
          if (quickAddData.prepDays > 0) {
            const sd = new Date(dueDate);
            sd.setDate(sd.getDate() - quickAddData.prepDays);
            startDate = sd.toISOString();
          }
          const res = await apiRequest("POST", "/api/tasks", {
            title: quickAddData.title,
            type: quickAddData.type,
            courseName: quickAddData.courseName || null,
            dueDate: dueDate.toISOString(),
            startDate,
            priority: quickAddData.priority,
            weekNumber: selectedWeek,
            description: quickAddData.description || "",
            reminder1: quickAddData.reminder1,
            reminder2: quickAddData.reminder2,
            reminder3: quickAddData.reminder3,
            reminder4: quickAddData.reminder4,
            attachments: quickAddData.attachments.length > 0 ? quickAddData.attachments : [],
            referenceLink: quickAddData.referenceLink || "",
            repeatType: quickAddData.repeatType,
            repeatInterval: quickAddData.repeatType === 'custom' ? quickAddData.repeatInterval : null,
            repeatIntervalUnit: quickAddData.repeatType === 'custom' ? quickAddData.repeatIntervalUni...
            repeatEndDate: quickAddData.repeatEndDate ? new Date(quickAddData.repeatEndDate + 'T00:00...
            eventStartTime: quickAddData.eventStartTime || null,
            eventEndTime: quickAddData.eventEndTime || null,
            notes: quickAddData.notes || null,
            projectId: quickAddData.projectId,
            flagged: (quickAddData as any).flagged ?? false,
          });
          const newTask = await res.json();
          if (quickAddData.subtasks.length > 0 && newTask?.id) {
            for (let i = 0; i < quickAddData.subtasks.length; i++) {
              try {
                await apiRequest("POST", "/api/subtasks", {
                  taskId: newTask.id,
                  title: quickAddData.subtasks[i].title,
                  isCompleted: false,
                  position: i,
                });
              } catch (e) {
                console.error('Failed to create subtask:', e);
              }
            }
          }
        } catch (e) {
          console.error('Failed to create task:', e);
        }
      }}
    }
  >

```

```

    }
  }
}
queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
queryClient.invalidateQueries({ queryKey: ["/api/weeks"] });
queryClient.invalidateQueries({ queryKey: ["/api/subtasks"] });
setIsQuickAddOpen(false);
toast({ title: "Task added", description: `${quickAddData.title} has been added to your cal...` });
catch (err) {
  toast({ title: "Error", description: "Failed to add task. Please try again.", variant: "des...` });
}
}}
data-testid="quick-add-submit"
>
<Plus className="h-3 w-3 mr-1" />
Add Task
</Button>
}}
</div>
</div>
{showQuickAddCloseConfirm && (
  <div className="absolute inset-0 z-10 flex items-center justify-center" style={{ background: 'rgba(0,...`
    <div className="flex flex-col items-center gap-4 px-6 py-5 rounded-lg" style={{ background: 'linear...`
      <p className="text-white text-[12px] text-center">You have unsaved changes.<br />Discard this tas...`
      <div className="flex gap-3">
        <button
          onClick={() => setShowQuickAddCloseConfirm(false)}
          className="px-4 py-2 rounded-lg text-[11px] text-white/70 hover:text-white bg-white/10 hover:...`
          data-testid="quick-add-cancel-discard"
        >
          Go Back
        </button>
        <button
          onClick={() => { setShowQuickAddCloseConfirm(false); setIsQuickAddOpen(false); }}
          className="px-4 py-2 rounded-lg text-[11px] text-white bg-red-600/80 hover:bg-red-600 border ...`
          data-testid="quick-add-confirm-discard"
        >
          Discard
        </button>
      </div>
    </div>
  </div>
)}
</div>
)}
</div>
))
}

/* Scholarships Dialog */
{(() => {
  const [scholarshipsList, setScholarshipsList] = useState<Array<{ id: number; name: string; organization: ...`
  const [scholarshipWizardOpen, setScholarshipWizardOpen] = useState(false);
  const [scholarshipWizardStep, setScholarshipWizardStep] = useState(0);
  const [scholarshipForm, setScholarshipForm] = useState({ name: '', organization: '', amount: '', applicat...`
  const [editingScholarshipId, setEditingScholarshipId] = useState<number | null>(null);

  useEffect(() => {
    if (isScholarshipsOpen) {
      fetch('/api/scholarships').then(r => r.json()).then(d => setScholarshipsList(d)).catch(() => {});
    }
  }, [isScholarshipsOpen]);

  const saveScholarship = async () => {
    const body = { ...scholarshipForm, status: 'not_started' };
    if (editingScholarshipId) {
      await fetch(`/api/scholarships/${editingScholarshipId}`, { method: 'PATCH', headers: { 'Content-Type':...`
    } else {
      await fetch('/api/scholarships', { method: 'POST', headers: { 'Content-Type': 'application/json' }, b...`
      const calDescription = [
        scholarshipForm.organization ? `Organization: ${scholarshipForm.organization}` : '',
        scholarshipForm.amount ? `Amount: ${scholarshipForm.amount}` : '',
        scholarshipForm.applicationUrl ? `Apply: ${scholarshipForm.applicationUrl}` : '',
        scholarshipForm.contactInfo ? `Contact: ${scholarshipForm.contactInfo}` : '',
        scholarshipForm.additionalInfo || '',
      ].filter(Boolean).join('\n');
      if (scholarshipForm.deadline) {
        try {
          const dueDate = new Date(scholarshipForm.deadline + 'T18:00:00');
          const weekNum = getWeekNumber(dueDate, semStart, readingWeekStart);
          await apiRequest("POST", "/api/tasks", {
            title: `Scholarship: ${scholarshipForm.name}`,
            type: "other",
            dueDate: dueDate.toISOString(),
            weekNumber: Math.max(2, Math.min(13, weekNum)),
            priority: "high",
            description: calDescription,
            referenceLink: scholarshipForm.applicationUrl || '',
          });
          queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
        } catch (e) {
          console.error('Failed to create scholarship task:', e);
        }
      }
    }
  }

  if (scholarshipForm.applicationsOpen || scholarshipForm.deadline) {
    try {
      await fetch('/api/scholarships/calendar-events', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          name: scholarshipForm.name,
          applicationsOpen: scholarshipForm.applicationsOpen,
          deadline: scholarshipForm.deadline,
          description: calDescription,
          recurring: scholarshipForm.recurring,
        }),
      });
    } catch (e) {
      console.error('Failed to create scholarship calendar events:', e);
    }
  }
})
}

```



```

        <button onClick={(e) => { e.stopPropagation(); deleteScholarship(s.id); }} className="tex...
        <Trash2 className="h-3.5 w-3.5" />
      </button>
    </td>
  </tr>
</tbody>
</table>
))
</div>
</DialogContent>
</Dialog>

/* Scholarship Add/Edit Wizard */
<Dialog open={scholarshipWizardOpen} onOpenChange={(open) => { if (!open) { setScholarshipWizardOpen(false)...
<DialogContent className="max-w-sm text-[11px] text-white [&_*]:text-white p-0 [&>button.absolute]:hidden...
  <div className="flex items-center justify-between px-4 py-3 border-b border-white/20 rounded-t-xl" styl...
    <div className="flex items-center gap-2">
      <Award className="h-3.5 w-3.5 text-white" />
      <h2 className="text-xs font-normal text-white" style={{ fontFamily: 'Avenir, 'Avenir Next', -apple-...
        {editingScholarshipId ? 'EDIT SCHOLARSHIP' : 'ADD SCHOLARSHIP'} - Step {scholarshipWizardStep + 1...
      </h2>
    </div>
    <button onClick={() => { setScholarshipWizardOpen(false); setEditingScholarshipId(null); }} className...
    <X className="h-5 w-5" />
  </button>
</div>
<div className="p-4 space-y-3">
  <div className="flex items-center gap-1 mb-2">
    {wizardSteps.map((_, i) => {
      <div key={i} className="h-1 flex-1 rounded-full transition-opacity duration-300" style={{ backgro...
    })}
  </div>
  <p className="text-[12px] font-medium text-white">{wizardSteps[scholarshipWizardStep].title}{wizardSt...
  {wizardSteps[scholarshipWizardStep].isMulti ? (
    <div className="space-y-2">
      <input
        type="url"
        value={scholarshipForm.applicationUrl}
        onChange={(e) => { const v = e.target.value; startTransition(() => setScholarshipForm(p => ({ ...
        placeholder="Application URL (https://...)"
        className="w-full px-3 py-2 rounded border border-white/20 bg-white/5 text-white text-[11px] fo...
        data-testid="input-scholarship-url"
      </>
      <input
        type="text"
        value={scholarshipForm.contactInfo}
        onChange={(e) => { const v = e.target.value; startTransition(() => setScholarshipForm(p => ({ ...
        placeholder="Contact info (email, phone, etc.)"
        className="w-full px-3 py-2 rounded border border-white/20 bg-white/5 text-white text-[11px] fo...
        data-testid="input-scholarship-contact"
      </>
    </div>
  ) : wizardSteps[scholarshipWizardStep].isTextarea ? (
    <div className="space-y-3">
      <textarea
        value={scholarshipForm[wizardSteps[scholarshipWizardStep].field]}
        onChange={(e) => { const v = e.target.value; const field = wizardSteps[scholarshipWizardStep].f...
        placeholder={wizardSteps[scholarshipWizardStep].placeholder}
        rows={4}
        className="w-full px-3 py-2 rounded border border-white/20 bg-white/5 text-white text-[11px] fo...
        data-testid={`input-scholarship-${wizardSteps[scholarshipWizardStep].field}`}
      </>
      <label className="flex items-center gap-2 cursor-pointer select-none" data-testid="checkbox-schol...
      <input
        type="checkbox"
        checked={scholarshipForm.recurring}
        onChange={(e) => setScholarshipForm(p => ({ ...p, recurring: e.target.checked })})
        className="h-4 w-4 rounded border border-white/30 bg-white/10 accent-green-400 cursor-pointer"
        data-testid="input-scholarship-recurring"
      </>
      <span className="text-[11px] text-white/80">Recurring - add to calendar every year</span>
    </label>
  </div>
  ) : wizardSteps[scholarshipWizardStep].isDate ? (
    <input
      type="date"
      value={scholarshipForm[wizardSteps[scholarshipWizardStep].field]}
      onChange={(e) => {
        const val = e.target.value;
        setScholarshipForm(p => ({ ...p, [wizardSteps[scholarshipWizardStep].field]: val }));
        if (val && /\d{4}-\d{2}-\d{2}$/.test(val)) {
          setTimeout(() => {
            if (scholarshipWizardStep < wizardSteps.length - 1) setScholarshipWizardStep(s => s + 1);
          }, 300);
        }
      }}
    </div>
    <div className="w-full px-3 py-2 rounded border border-white/20 bg-white/5 text-white text-[11px] focu...
    data-testid={`input-scholarship-${wizardSteps[scholarshipWizardStep].field}`}
  </>
  ) : (wizardSteps[scholarshipWizardStep] as any).isCurrency ? (
    <div className="relative">
      <span className="absolute left-3 top-1/2 -translate-y-1/2 text-white/60 text-[11px] pointer-event...
      <input
        type="text"
        value={scholarshipForm[wizardSteps[scholarshipWizardStep].field]}
        onChange={(e) => { const v = e.target.value.replace(/\$/g, ''); const field = wizardSteps[schol...
        placeholder={wizardSteps[scholarshipWizardStep].placeholder}
        className="w-full pl-6 pr-3 py-2 rounded border border-white/20 bg-white/5 text-white text-[11p...
        autoFocus
        data-testid={`input-scholarship-${wizardSteps[scholarshipWizardStep].field}`}
        onKeyDown={(e) => {
          if (e.key === 'Enter') {
            if (scholarshipWizardStep < wizardSteps.length - 1) setScholarshipWizardStep(s => s + 1);
            else saveScholarship();
          }
        }}
    </div>
  )
}

```

```

    />
  </div>
) : (
  <input
    type="text"
    value={scholarshipForm[wizardSteps[scholarshipWizardStep].field]}
    onChange={(e) => { const v = e.target.value; const field = wizardSteps[scholarshipWizardStep].fie...
    placeholder={wizardSteps[scholarshipWizardStep].placeholder}
    className="w-full px-3 py-2 rounded border border-white/20 bg-white/5 text-white text-[11px] focu...
    autoFocus
    data-testid={`input-scholarship-${wizardSteps[scholarshipWizardStep].field}`}
    onKeyDown={(e) => {
      if (e.key === 'Enter') {
        if (scholarshipWizardStep < wizardSteps.length - 1) setScholarshipWizardStep(s => s + 1);
        else saveScholarship();
      }
    }}
  />
)}
<div className="flex items-center justify-between pt-2">
  <button
    onClick={() => scholarshipWizardStep > 0 ? setScholarshipWizardStep(s => s - 1) : setScholarshipW...
    className="px-3 py-1.5 rounded text-[10px] font-medium bg-white/5 hover:bg-white/10 transition-co...
    data-testid="button-scholarship-back"
  >
    {scholarshipWizardStep > 0 ? 'Back' : 'Cancel'}
  </button>
  <div className="flex gap-2">
    {scholarshipWizardStep > 1 && scholarshipForm.name && scholarshipForm.organization && (
      <button
        onClick={saveScholarship}
        className="px-3 py-1.5 rounded text-[10px] font-medium bg-green-500/20 hover:bg-green-500/30 ...
        data-testid="button-scholarship-save-early"
      >
        Save Now
      </button>
    )}
    {scholarshipWizardStep < wizardSteps.length - 1 ? (
      <button
        onClick={() => setScholarshipWizardStep(s => s + 1)}
        disabled={!scholarshipWizardStep.required && !scholarshipForm[wizardSteps[scholar...
        className="px-3 py-1.5 rounded text-[10px] font-medium bg-white/10 hover:bg-white/20 transiti...
        data-testid="button-scholarship-next"
      >
        Next
      </button>
    ) : (
      <button
        onClick={saveScholarship}
        disabled={!scholarshipForm.name || !scholarshipForm.organization}
        className="px-3 py-1.5 rounded text-[10px] font-medium bg-blue-500/20 hover:bg-blue-500/30 tr...
        data-testid="button-scholarship-save"
      >
        {editingScholarshipId ? 'Update' : 'Save'}
      </button>
    )}
  </div>
</div>
</DialogContent>
</Dialog>
</>
))())}

/* Key Contacts Dialog */
{(() => {
  const [contactsList, setContactsList] = useState<Array<{ id: number; name: string; title: string | null; ...
  const [contactFormOpen, setContactFormOpen] = useState(false);
  const [editingContactId, setEditingContactId] = useState<number | null>(null);
  const [contactForm, setContactForm] = useState({ name: '', title: '', organization: '', department: '', e...
  const [contactFilter, setContactFilter] = useState('all');

  useEffect(() => {
    if (isKeyContactsOpen) {
      fetch('/api/key-contacts').then(r => r.json()).then(d => setContactsList(d)).catch(() => {});
    }
  }, [isKeyContactsOpen]);

  const saveContact = async () => {
    if (editingContactId) {
      await fetch('/api/key-contacts/${editingContactId}', { method: 'PATCH', headers: { 'Content-Type': 'a...
    } else {
      await fetch('/api/key-contacts', { method: 'POST', headers: { 'Content-Type': 'application/json' }, b...
    }
    const updated = await fetch('/api/key-contacts').then(r => r.json());
    setContactsList(updated);
    setContactFormOpen(false);
    setEditingContactId(null);
    setContactForm({ name: '', title: '', organization: '', department: '', email: '', phone: '', office: '...
  };

  const deleteContact = async (id: number) => {
    await fetch('/api/key-contacts/${id}', { method: 'DELETE' });
    setContactsList(prev => prev.filter(c => c.id !== id));
  };

  const categories = [
    { value: 'professor', label: 'Professors' },
    { value: 'advisor', label: 'Academic Advisors' },
    { value: 'ta', label: 'Teaching Assistants' },
    { value: 'admin', label: 'Administration' },
    { value: 'accessibility', label: 'Accessibility Services' },
    { value: 'financial', label: 'Financial Aid' },
    { value: 'classmate', label: 'Classmates' },
    { value: 'other', label: 'Other' },
  ];
}

```



```

const filteredContacts = contactFilter === 'all' ? contactsList : contactsList.filter(c => c.category ===...)

const getCategoryColor = (cat: string | null) => {
  switch (cat) {
    case 'professor': return '#ef4444';
    case 'advisor': return '#3b82f6';
    case 'ta': return '#f59e0b';
    case 'admin': return '#8b5cf6';
    case 'accessibility': return '#10b981';
    case 'financial': return '#ec4899';
    case 'classmate': return '#06b6d4';
    default: return '#6b7280';
  }
};

return (
  <
<Dialog open={isFeedbackOpen} onOpenChange={setIsFeedbackOpen}>
  <DialogContent className="max-w-[320px] text-[11px] text-white [&_*]:text-white [&_i...
  <DialogTitle className="sr-only">Send Feedback</DialogTitle>
  <div className="flex items-center justify-between px-4 py-2 border-b border-white/40 flex-shrink-0 roun...
    <div className="flex items-center gap-2">
      <MessageSquare className="text-white" style={{ width: '14px', height: '14px' }} />
      <h2 className="font-normal text-white" style={{ fontFamily: 'Avenir', 'Avenir Next', -apple-system, ...
    </div>
    <button onClick={() => setIsFeedbackOpen(false)} className="text-white hover:text-white/80 transition...
  </div>
  <div className="flex flex-col gap-3 p-4">
    <textarea
      value={feedbackText}
      onChange={(e) => setFeedbackText(e.target.value)}
      placeholder="Describe a change, bug, or feature request..."
      className="w-full h-[120px] bg-black/40 border border-white/30 rounded-lg p-3 text-[11px] text-whit...
      data-testid="input-feedback-note"
    />
    <Button
      disabled={!feedbackText.trim() || feedbackSending}
      data-testid="button-send-feedback"
      className="w-full h-8 text-[11px] font-medium"
      style={{ background: 'linear-gradient(180deg, rgba(255,255,255,0.25) 0%, rgba(255,255,255,0.08) 100...
      onClick={async () => {
        if (!feedbackText.trim()) return;
        setFeedbackSending(true);
        try {
          const resp = await fetch('/api/feedback-notes', { method: 'POST', headers: { 'Content-Type': 'a...
          if (resp.ok) {
            toast({ title: "Note Saved", description: "Your feedback has been saved for the agent." });
            setFeedbackText('');
            setIsFeedbackOpen(false);
          }
        } catch {} finally { setFeedbackSending(false); }
      }}
    >
      {feedbackSending ? 'Sending...' : 'Send Note'}
    </Button>
  </div>
</DialogContent>
</Dialog>

<Dialog open={isKeyContactsOpen} onOpenChange={setIsKeyContactsOpen}>
  <DialogContent className="overflow-hidden flex flex-col text-[11px] text-white [&_*]:text-white [&_label]...
  <DialogTitle className="sr-only">Key Contacts</DialogTitle>
  <div className="flex items-center justify-between px-4 py-1.5 border-b border-white/40 flex-shrink-0 ro...
    <div className="flex items-center gap-2">
      <Contact className="h-3.5 w-3.5 text-white" />
      <h2 className="text-xs font-normal text-white" style={{ fontFamily: 'Avenir', 'Avenir Next', -apple-...
    </div>
    <div className="flex items-center gap-2">
      <button
        onClick={() => { window.open('/api/key-contacts/export.vcf', '_blank'); }}
        className="flex items-center gap-1 px-2 py-1 rounded text-[10px] font-medium bg-white/10 hover:...
        data-testid="button-export-contacts-vcf"
      >
        <Download className="h-3 w-3" /> iPhone
      </button>
      <button
        onClick={() => { setContactFormOpen(true); setEditingContactId(null); setContactForm({ name: '...'
        className="flex items-center gap-1 px-2 py-1 rounded text-[10px] font-medium bg-white/10 hover:...
        data-testid="button-add-contact"
      >
        <Plus className="h-3 w-3" /> Add Contact
      </button>
    </div>
  </div>
  <div className="flex-1 overflow-hidden flex" style={{ minHeight: 0 }}>
    {(() => {
      const grouped = categories
        .map(cat => ({
          ...cat,
          contacts: contactsList.filter(c => c.category === cat.value),
        }));
      const ungrouped = contactsList.filter(c => !categories.some(cat => cat.value === c.category));
      if (ungrouped.length > 0) grouped.push({ value: 'uncategorized', label: 'Other', contacts: ungrouped...
      const activeGroup = grouped.find(g => g.value === contactFilter) || grouped[0];
      const sectionRef = useRef<Record<string, HTMLDivElement | null>>({});
      return (
        <
        <div className="flex flex-col py-2 overflow-y-auto flex-shrink-0" style={{ width: '130px', bord...
          <button
            onClick={() => startTransition(() => setContactFilter('all'))}
            className="text-left px-3 py-2 text-[10px] font-medium transition-colors"
            style={{ color: contactFilter === 'all' ? 'white' : 'rgba(255,255,255,0.5)', background: co...
            data-testid="tab-contacts-all"
          >
            All ({contactsList.length})
          </button>
          {grouped.map(g => (
            <button

```



```

        <X className="h-5 w-5" />
      </button>
    </div>
  </div>
  <div className="p-4 space-y-2.5 overflow-y-auto" style={{ maxHeight: 'calc(80vh - 60px)', scrollbarWidt...
    <div>
      <label className="text-[10px] text-white/60 mb-1 block">Name </label>
      <FastInput type="text" value={contactForm.name} onValueChange={(v) => setContactForm(p => ({ ...p, ...
    </div>
    <div>
      <label className="text-[10px] text-white/60 mb-1 block">Title / Role</label>
      <FastInput type="text" value={contactForm.title} onValueChange={(v) => setContactForm(p => ({ ...p,...
    </div>
    <div className="grid grid-cols-2 gap-2">
      <div>
        <label className="text-[10px] text-white/60 mb-1 block">Organization</label>
        <FastInput type="text" value={contactForm.organization} onValueChange={(v) => setContactForm(p =>...
      </div>
      <div>
        <label className="text-[10px] text-white/60 mb-1 block">Department</label>
        <FastInput type="text" value={contactForm.department} onValueChange={(v) => setContactForm(p => (...
      </div>
    </div>
    <div>
      <label className="text-[10px] text-white/60 mb-1 block">Email</label>
      <FastInput type="email" value={contactForm.email} onValueChange={(v) => setContactForm(p => ({ ...p...
    </div>
    <div className="grid grid-cols-2 gap-2">
      <div>
        <label className="text-[10px] text-white/60 mb-1 block">Phone</label>
        <FastInput type="tel" value={contactForm.phone} onValueChange={(v) => setContactForm(p => ({ ...p...
      </div>
      <div>
        <label className="text-[10px] text-white/60 mb-1 block">Office Location</label>
        <FastInput type="text" value={contactForm.office} onValueChange={(v) => setContactForm(p => ({ ....
      </div>
    </div>
    <div>
      <label className="text-[10px] text-white/60 mb-1 block">Category</label>
      <select value={contactForm.category} onChange={(e) => { const v = e.target.value; startTransition(...
        {categories.map(c => <option key={c.value} value={c.value}>{c.label}</option>}}
      </select>
    </div>
    <div>
      <label className="text-[10px] text-white/60 mb-1 block">Notes</label>
      <FastTextarea value={contactForm.notes} onValueChange={(v) => setContactForm(p => ({ ...p, notes: v...
    </div>
    <div className="flex justify-end gap-2 pt-1">
      <button onClick={() => { setContactFormOpen(false); setEditingContactId(null); }} className="px-3 p...
      <button onClick={saveContact} disabled={!contactForm.name} className="px-3 py-1.5 rounded text-[10p...
    </div>
  </div>
</DialogContent>
</Dialog>
</>
);
})();

/* AAS Reminder Popup */
<Dialog open={showAasReminder} onOpenChange={setShowAasReminder}>
  <DialogContent className="max-w-sm text-[11px] text-white [&_*]:text-white p-0 [&button.absolute]:hidden...
    <div className="flex items-center justify-between px-4 py-3 border-b border-white/20" style={{ backgrou...
      <div className="flex items-center gap-2">
        <AlertCircle className="h-3.5 w-3.5 text-amber-400" />
        <h2 className="text-xs font-normal text-white" style={{ fontFamily: 'Avenir, 'Avenir Next', -apple-...
          AAS REMINDER
        </h2>
      </div>
      <button
        onClick={() => setShowAasReminder(false)}
        className="text-white hover:text-white/80 transition-colors p-1"
        data-testid="button-close-aas-reminder"
      >
        <X className="h-5 w-5" />
      </button>
    </div>
    <div className="p-4 space-y-3">
      <p className="text-[12px] text-white/90">
        You haven't confirmed sending your AAS letter via the portal for the following course(s):
      </p>
      <div className="space-y-2">
        {(() => {
          const currentCodes = semesterSettings ? [1,2,3].map(i => (semesterSettings as any)[`course${i}Cod...
          return coursesData.courses.filter(c => {
            const code = c.name.split(' - ')[0];
            return c.name.trim() && currentCodes.includes(code) && !aasSentStatus[code];
          });
        })().map((course, idx) => {
          const code = course.name.split(' - ')[0];
          const name = course.name.split(' - ').slice(1).join(' - ');
          return (
            <div key={idx} className="flex items-center gap-2 bg-amber-500/10 border border-amber-500/30 ro...
              <div className="w-3 h-2 rounded-full" style={{ background: course.colorEnd ? `linear-gradient...
                <span className="text-[11px] font-medium">{code}</span>
                {name} && <span className="text-[11px] text-white/70">- {name}</span>
                {course.professor} && <span className="text-[10px] text-white ml-auto">({course.professor})</s...
              </div>
            );
          });
        })()
      </div>
      <p className="text-[11px] text-white/60">
        Please send your Academic Accommodation Support letter to each professor via the portal. Check off ...
      </p>
      <Button
        variant="outline"
        className="w-full border-white/30 text-white hover:text-white hover:bg-white/10"
        onClick={() => setShowAasReminder(false)}
        data-testid="button-dismiss-aas-reminder"
      >

```

```

        Got it
      </Button>
    </div>
  </DialogContent>
</Dialog>

{/* Rank Courses Reminder */}
<Dialog open={showRankCoursesReminder} onOpenChange={setShowRankCoursesReminder}>
  <DialogContent className="max-w-sm text-[11px] text-white [&_*]:text-white p-0 [&button.absolute]:hidden..."
    <div className="flex items-center justify-between px-4 py-3 border-b border-white/20" style={{ backgrou...
      <div className="flex items-center gap-2">
        <AlertCircle className="h-3.5 w-3.5 text-amber-400" />
        <h2 className="text-xs font-normal text-white" style={{ fontFamily: "Avenir, 'Avenir Next', -apple-...
          RANK YOUR COURSES
        </h2>
      </div>
    </div>
    <button
      onClick={() => setShowRankCoursesReminder(false)}
      className="text-white hover:text-white/80 transition-colors p-1"
      data-testid="button-close-rank-courses-reminder"
    >
      <X className="h-5 w-5" />
    </button>
  </div>
  <div className="p-4 space-y-3">
    <p className="text-[12px] text-white/90">
      The semester has started but you haven't ranked all your courses in order of importance yet.
    </p>
    <p>
      {{{() => {
        const now = new Date();
        const activeSem = SEMESTER_COURSE_DEFS.find(s => now >= new Date(s.start) && now <= new Date(s.end));
        if (!activeSem) return null;
        const isSSActive = activeSem.key.startsWith('ss');
        const unranked = activeSem.codes.filter(code => {
          if (isSSActive) {
            const kA = `${activeSem.key}:${code}:A`;
            const kB = `${activeSem.key}:${code}:B`;
            return (!coursePlayPriority[kA] || coursePlayPriority[kA] <= 0) && (!coursePlayPriority[kB] || ...
          }
          const key = `${activeSem.key}:${code}`;
          return !coursePlayPriority[key] || coursePlayPriority[key] <= 0;
        });
        if (unranked.length === 0) return null;
        return (
          <div className="space-y-1.5">
            {unranked.map(code => (
              <div key={code} className="flex items-center gap-2 bg-amber-500/10 border border-amber-500/30...
                <span className="text-[11px] font-medium">{code}</span>
                <span className="text-[10px] text-white/50 ml-auto">Not ranked</span>
              </div>
            ))}
          </div>
        );
      }}}()
    </p>
    <p className="text-[11px] text-white/60">
      Open School Courses and assign a priority ranking to each course for this semester.
    </p>
    <div className="flex gap-2">
      <Button
        variant="outline"
        className="flex-1 border-white/30 text-white hover:text-white hover:bg-white/10"
        onClick={() => {
          const now = new Date();
          const activeSem = SEMESTER_COURSE_DEFS.find(s => now >= new Date(s.start) && now <= new Date(s....
            if (activeSem) localStorage.setItem('rankCoursesReminderDismissed_${activeSem.key}', 'true');
            setShowRankCoursesReminder(false);
          }}
          data-testid="button-dismiss-rank-courses-reminder"
        >
          Dismiss
        </Button>
      <Button
        className="flex-1 bg-blue-500 hover:bg-blue-600 text-white"
        onClick={() => {
          startTransition(() => {
            setShowRankCoursesReminder(false);
            setDraftPriorityBoth({ ...coursePlayPriority });
            startTransition(() => setIsSchoolCoursesDialogOpen(true));
          });
        }}
        data-testid="button-rank-courses-now"
      >
        Rank Now
      </Button>
    </div>
  </DialogContent>
</Dialog>

{/* Profile Dialog */}
<Dialog open={isProfileDialogOpen} onOpenChange={setIsProfileDialogOpen}>
  <DialogContent className="max-w-md text-[11px] text-white [&_*:not(input)]:text-white [&_label]:text-whit...
    <div className="flex items-center justify-between px-4 py-3 border-b border-white/40 rounded-t-lg" styl...
      <div className="flex items-center gap-2">
        <User className="h-3 w-3 text-white" />
        <h2 className="font-normal text-white" style={{ fontFamily: "Avenir, 'Avenir Next', -apple-system, ...
          PROFILE SETTINGS
        </h2>
      </div>
    </div>
    <button
      onClick={() => setIsProfileDialogOpen(false)}
      className="text-white hover:text-white/80 transition-colors p-1"
      data-testid="button-close-profile"
    >
      <X className="h-5 w-5" />
    </button>
  </div>

```

```

<div className="p-4">
  <ProfileForm
    key={isProfileDialogOpen ? 'open' : 'closed'}
    profileData={profileData}
    timezones={timezones}
    onSave={saveProfile}
    onCancel={() => setIsProfileDialogOpen(false)}
    profilePhotoUrl={profilePhotoUrl || profilePhoto}
    onProfilePhotoChange={(url: string | null) => {
      setProfilePhotoUrl(url);
      if (url) {
        localStorage.setItem('profilePhotoUrl', url);
      } else {
        localStorage.removeItem('profilePhotoUrl');
      }
    }}
  />
</div>
</DialogContent>
</Dialog>

{/* School Dialog */}
<Dialog open={isSchoolDialogOpen} onOpenChange={(open) => { if (!isNewCourseDialogOpen && !newCourseDialogC...
  <DialogContent
    data-settings-dialog
    className="overflow-hidden flex flex-col text-[11px] text-white [&_*]:text-white [&_label]:text-white [...
    style={{ position: 'fixed', top: `${(calendarBorderTop || (calendarTop + 15)) - 16 - 30 - 40}px`, left:...
    onInteractOutside={(e) => { if (isNewCourseDialogOpen || newCourseDialogClosingRef.current || isNewCour...
    onEscapeKeyDown={(e) => { if (isNewCourseDialogOpen || newCourseDialogClosingRef.current || isNewCourse...
    onPointerDownOutside={(e) => { if (isNewCourseDialogOpen || newCourseDialogClosingRef.current || isNewC...
    onFocusOutside={(e) => { if (isNewCourseDialogOpen || newCourseDialogClosingRef.current || isNewCourseW...
  >
    {/* Header bar matching flyouts */}
    <div className="flex items-center justify-between px-4 py-3 border-b border-white/40 flex-shrink-0 roun...
      <div className="flex items-center gap-2">
        <GraduationCap className="text-white" style={{ width: '15px', height: '15px' }} />
        <h2 className="font-normal text-white" style={{ fontFamily: 'Avenir', 'Avenir Next', '-apple-system, ...
          SCHOOL SETTINGS
        </h2>
      </div>
    </div>
    <div className="flex-1 overflow-y-auto px-4 pb-4 pt-0" style={{ scrollbarWidth: 'none', msOverflowStyle...
  <div className="grid grid-cols-2 gap-4">
    /* Left Column - School Settings */
    <div className="flex flex-col gap-4" style={{ padding: '0px', margin: '6px' }}>
      <SchoolForm
        key={isSchoolDialogOpen ? 'open' : 'closed'}
        schoolData={schoolData}
        semesterSettings={semesterSettings}
        onSave={saveSchool}
        onCancel={() => setIsSchoolDialogOpen(false)}
      />
    </div>

    /* Right Column - Courses & Weeks */
    <div className="flex flex-col gap-4" style={{ padding: '0px', margin: '-9px' }}>
      /* Course Legend */
      <div className="border rounded-lg p-3 space-y-3" style={{ margin: '10px' }}>
        <div className="flex items-center justify-between">
          <Label className="text-[10px] font-medium">Courses</Label>
          <div className="flex items-center gap-3">
            <div
              className="flex items-center gap-1 cursor-pointer hover:opacity-80 transition-opacity"
              onClick={() => { setIsSchoolDialogOpen(false); setTimeout(() => setIsRemainingCoursesDialogOp...
              data-testid="button-remaining-courses"
            >
              <span className="text-[10px] text-white/70 font-medium">Remaining</span>
              <span className="text-[8px] font-bold text-white/60 bg-white/15 px-1.5 py-0.5 rounded-full">{...
              <ChevronRight className="text-white/70" style={{ width: '12px', height: '12px' }} />
            </div>
            <div
              className="flex items-center gap-1 cursor-pointer hover:opacity-80 transition-opacity"
              onClick={() => { setIsSchoolDialogOpen(false); setTimeout(() => { setDraftPriorityBoth({ ...c...
              data-testid="button-past-courses"
            >
              <span className="text-[10px] text-white/70 font-bold">ALL COURSES</span>
              <ChevronRight className="text-white/70" style={{ width: '12px', height: '12px' }} />
            </div>
          </div>
        </div>
      </div>
    <div>
      {coursesData.courses.filter(course => course.name.trim()).map((course, index) => {
        const courseCode = course.name.split(' - ')[0];
        const courseName = course.name.split(' - ').slice(1).join(' - ') || course.name;
        const tomorrow = addDays(startOfDayET(new Date()), 1);
        const hasDueTomorrow = allTasks.some(task =>
          task.courseName?.includes(courseCode) &&
          !task.isCompleted &&
          isSameDayET(new Date(task.dueDate), tomorrow)
        );
        const professorEmail = course.professorEmail;
        const semCourse = semesterSettings ? (()) => {
          const idx = index + 1;
          const s = semesterSettings as any;
          return {
            delivery: s[`course${idx}DeliveryMode`],
            day: s[`course${idx}ClassDay`],
            day2: s[`course${idx}ClassDay2`],
            time: s[`course${idx}ClassTime`],
            endTime: s[`course${idx}ClassEndTime`],
          };
        }(): null;
        const courseTasks = allTasks.filter(t => t.courseName?.includes(courseCode) && t.gradeWeight);
        const gradedCompleted = courseTasks.filter(t => t.isCompleted && t.gradeValue !== null && t.grade...
        const totalWeightGraded = gradedCompleted.reduce((sum, t) => sum + (t.gradeWeight || 0), 0);
        const weightedScore = gradedCompleted.reduce((sum, t) => {
          const pct = (t.gradeTotal && t.gradeTotal > 0) ? ((t.gradeValue || 0) / t.gradeTotal) * 100 : (...
          return sum + (pct * (t.gradeWeight || 0) / 100);
        }, 0);
      }
    </div>
  </div>
}

```

```

const currentGrade = totalWeightGraded > 0 ? (weightedScore / totalWeightGraded) * 100 : null;
const totalWeight = courseTasks.reduce((sum, t) => sum + (t.gradeWeight || 0), 0);
return (
  <div key={index} className="space-y-0.5">
    <div className="flex items-center gap-1.5">
      <label className="flex items-center gap-1 cursor-pointer" data-testid={`checkbox-school-aas...`>
        <div className={`w-3 h-3 rounded-sm border flex items-center justify-center flex-shrink-0...`>
          {(aasSentStatus[courseCode] || aasSentStatus[courseCode.replace(/\s/g, '')]) && (
            <Check className="w-2.5 h-2.5 text-white" />
          )}
        </div>
      </label>
      <div className="relative flex-shrink-0 flex items-center gap-0.5">
        <div className="relative">
          <div
            className={`w-3 h-3 rounded-sm cursor-pointer ${hasDueTomorrow ? "animate-blink" : ""}...`
            style={{ backgroundColor: course.color }}
            onClick={() => document.getElementById(`school-course-color-${index}`)?.click()}
            title="Gradient start color"
          />
          <input
            id={`school-course-color-${index}`}
            type="color"
            value={course.color}
            onChange={(e) => {
              const updatedCourses = [...coursesData.courses];
              updatedCourses[index] = { ...updatedCourses[index], color: e.target.value };
              setCoursesData({ courses: updatedCourses });
              localStorage.setItem('coursesData', JSON.stringify({ courses: updatedCourses }));
              saveCourses({ courses: updatedCourses });
            }}
            className="absolute inset-0 w-0 h-0 opacity-0"
            data-testid={`input-school-course-color-${index}`}
          />
        </div>
        <div className="w-4 h-2.5 rounded-sm" style={{ background: `linear-gradient(to right, ${c...`
      </div>
      <div className="relative">
        <div
          className="w-3 h-3 rounded-sm cursor-pointer"
          style={{ backgroundColor: course.colorEnd || course.color }}
          onClick={() => document.getElementById(`school-course-color-end-${index}`)?.click()}
          title="Gradient end color"
        />
        <input
          id={`school-course-color-end-${index}`}
          type="color"
          value={course.colorEnd || course.color}
          onChange={(e) => {
            const updatedCourses = [...coursesData.courses];
            updatedCourses[index] = { ...updatedCourses[index], colorEnd: e.target.value };
            setCoursesData({ courses: updatedCourses });
            localStorage.setItem('coursesData', JSON.stringify({ courses: updatedCourses }));
            saveCourses({ courses: updatedCourses });
          }}
          className="absolute inset-0 w-0 h-0 opacity-0"
          data-testid={`input-school-course-color-end-${index}`}
        />
      </div>
    </div>
    <span className="text-[10px] text-white">
      <span className="font-medium">{courseCode}</span>
      {courseName !== courseCode && <span className="text-white/80"> {courseName}</span>}
    </span>
    {courseCertificateTypes[courseCode] && (
      <span className="text-[8px] px-1.5 py-0.5 rounded-full bg-white/10 text-white/60 whitespa...`
    )}
    {course.professor && (
      professorEmail ? (
        <a
          href={`https://mail.google.com/mail/?view=cm&fs=1&to=${encodeURIComponent(professorEm...`
          target="_blank"
          rel="noopener noreferrer"
          className="text-[10px] !text-blue-400 underline hover:!text-blue-300 cursor-pointer"
          data-testid={`link-school-email-professor-${index + 1}`}
          onClick={(e) => {
            e.stopPropagation();
          }}
        >
          {course.professor}
        </a>
      ) : (
        <span className="text-[10px] text-white">{course.professor}</span>
      )
    )
  </button>
  <button
    className="flex-shrink-0 p-0.5 rounded hover:bg-white/10 cursor-pointer"
    onClick={(e) => {
      e.stopPropagation();
      startTransition(() => setSelectedCertCourse({ courseCode, courseName: courseName !== co...`
    })
    data-testid={`button-edit-course-${index}`}
  >
    <Pencil className="w-2.5 h-2.5 text-white/50 hover:text-white/80" strokeWidth={2.5} />
  </button>
  <div className="flex items-center gap-2 ml-auto text-[9px]" data-testid={`grade-display-${c...`
    {courseTasks.length > 0 ? (
      <div>
        {currentGrade !== null ? (
          <span className={`font-medium ${currentGrade >= 80 ? 'text-green-400' : currentGrad...`>
            {currentGrade.toFixed(1)}%
          </span>
        ) : (
          <span className="text-white/30">--</span>
        )}
      </div>
      <span className="text-white/30">{totalWeightGraded}/{totalWeight}%</span>
    </div>
  ) : (
    <span className="text-white/30">--</span>
  )
)

```

```

    })
  </div>
</div>
<div>
  {semCourse && (semCourse.delivery || semCourse.day || semCourse.time) && (
    <div className="flex items-center gap-2 pl-4 text-[10px] text-white/50">
      {semCourse.delivery && <span>{semCourse.delivery}</span>}
      {semCourse.day && <span>{semCourse.day}</span>}
      {semCourse.day2 ? `/${semCourse.day2} : ` : ''}</span>}
      {semCourse.time && <span>{semCourse.time}</span>}
    </div>
  )}
</div>
);
}}
</div>

/* Weeks */
<div className="border rounded-lg p-3 flex flex-col flex-1" style={{ marginTop: '-4px' }}>
  <Label className="text-[10px] font-medium mb-1 block">Weeks</Label>
  <div className="flex flex-col flex-1 justify-evenly gap-1">
    {[...weeks].sort((a, b) => {
      const today = startOfDayET(new Date());
      const aEndDay = startOfDayET(parseISO(a.endDate));
      const bEndDay = startOfDayET(parseISO(b.endDate));
      const aFinished = aEndDay < today;
      const bFinished = bEndDay < today;
      if (aFinished && !bFinished) return 1;
      if (!aFinished && bFinished) return -1;
      return a.weekNumber - b.weekNumber;
    })}.map((week) => {
      const weekEnd = parseISO(week.endDate);
      const isWeekFinished = startOfDayET(weekEnd) < startOfDayET(new Date());
      const isSelected = selectedWeek === week.weekNumber && !selectedDate;
      return (
        <div key={week.weekNumber} className={`flex items-center gap-0.5 rounded-md`} style={isSelected...}
        <Button
          variant="ghost"
          className={
            { justify-start gap-1 h-auto !py-0 !min-h-0 px-1 w-full }
            { isWeekFinished ? "opacity-50" : "" }
          size="sm"
          onClick={() => {
            setSelectedWeek(week.weekNumber);
            setSelectedDate(null);
            setIsSchoolDialogOpen(false);
          }}
          data-testid={`button-week-school-${week.weekNumber}`}
        >
          <div className={`flex items-center gap-1 ${isWeekFinished ? "line-through" : ""}`}>
            <Calendar className="h-3 w-3 text-white" />
            <span className="text-[10px] text-white">Week {week.weekNumber}</span>
            <span className={`text-[10px] font-bold ${isSelected ? 'text-white' : 'text-white/70'}`}>
              {format(parseISO(week.startDate), "MMM d")} - {format(parseISO(week.endDate), "MMM d")}
            </span>
          </div>
          {week.taskCount > 0 && (
            <Badge variant="outline" className="text-[10px] px-1 py-0 min-w-5 text-center justify-cen...>
              {week.taskCount}
            </Badge>
          )}
        </Button>
      </div>
    )};
  ]}
</div>
</div>
</div>
</div>
</div>
<div className="px-2 flex justify-end" style={{ marginTop: '-14px' }}>
  <div className="flex items-center gap-2">
    <Button
      type="button"
      variant="outline"
      className="border !border-white/30 text-white/70 hover:text-white hover:!border-white/50 hover:bg-t...
      style={{ fontSize: '12px', marginRight: '5px' }}
      onClick={() => setIsSchoolDialogOpen(false)}
      data-testid="button-cancel-school"
    >
      Cancel
    </Button>
    <Button
      type="submit"
      form="school-settings-form"
      variant="outline"
      className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-transpa...
      style={{
        boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255,255,...
        fontSize: '12px'
      }}
      data-testid="button-save-school"
    >
      Save
    </Button>
  </div>
</div>
</DialogContent>
</Dialog>

/* Courses Dialog */
<Dialog open={isCoursesDialogOpen} onOpenChange={setIsCoursesDialogOpen}>
  <DialogContent className="max-w-xl text-[11px] text-white [&_*]:text-white [&_label]:text-white [&_input]...
  <div className="flex items-center justify-between px-4 py-3 bg-black/30 border-b border-white/20">
    <div className="flex items-center gap-2">
      <GraduationCap className="h-3 w-3 text-white" />
      <h2 className="text-xs font-normal text-white" style={{ fontFamily: 'Avenir, 'Avenir Next', -apple-...
        COURSES & SCHEDULE
      </h2>
    </div>
  </div>
</DialogContent>
</Dialog>

```

```

</div>
<button
  onClick={() => setIsCoursesDialogOpen(false)}
  className="text-white hover:text-white/80 transition-colors p-1"
  data-testid="button-close-courses"
>
  <X className="h-5 w-5" />
</button>
</div>
<div className="p-4">
  <CoursesForm
    coursesData={coursesData}
    semesterSettings={semesterSettings}
    onSave={saveCourses}
    onSaveSemesterSchedule={(data) => saveSemesterScheduleMutation.mutate(data)}
    onGenerateClassTasks={() => generateClassTasksMutation.mutate()}
    isGenerating={generateClassTasksMutation.isPending}
    onCancel={() => setIsCoursesDialogOpen(false)}
  />
</div>
</DialogContent>
</Dialog>

{/* Courses Dialog - All past + current courses organized by semester */}
{isSchoolCoursesDialogOpen && createPortal(
  <div>
    <div className={`fixed inset-0 z-[10002] ${schoolCoursesOpenSource === 'pill' ? 'bg-black/50' : ''}`} onC...
    <div
      className="fixed left-[50%] translate-x-[-50%] z-[10002] overflow-hidden flex flex-col text-[11px] text...
      style={schoolCoursesOpenSource === 'pill' ? { top: 'calc(3vh - 6px)', width: 'calc(96vw + 24px)', maxWi...
    >
      <div className="flex items-center justify-between px-4 py-3 border-b border-white/40 flex-shrink-0 roun...
        <div className="flex items-center gap-2">
          <GraduationCap className="text-white" style={{ width: '15px', height: '15px' }} />
          <h2 className="font-normal text-white" style={{ fontFamily: 'Avenir, 'Avenir Next', -apple-system, ...
            SEMESTERS AND COURSES
          </h2>
        </div>
        <div
          className="flex items-center gap-1 cursor-pointer hover:opacity-80 transition-opacity"
          onClick={(e) => {
            e.stopPropagation();
            startTransition(() => setIsSchoolCoursesDialogOpen(false));
            setTimeout(() => startTransition(() => setIsSettingsPanelOpen(true)), 100);
          }}
          data-testid="button-courses-degree-tracking"
        >
          <span className="text-white text-[11px] font-medium" style={{ textShadow: '0 1px 2px rgba(0,0,0,0.2...
            <ChevronRight className="text-white/80" style={{ width: '14px', height: '14px' }} />
          </div>
        </div>
        <div ref={coursesScrollRef} className="flex-1 overflow-y-auto px-4 py-3" style={{ scrollbarWidth: 'thin...
          <div className="shrink-0 flex items-center pb-2" style={{ marginTop: '2px' }}>
            <Button
              type="button"
              variant="outline"
              className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-trans...
              style={{ background: 'rgba(10,15,30,0.85)', boxShadow: '0 0 6px rgba(255,255,255,0.6)', 0 0 12px r...
              onClick={() => { startTransition(() => setIsSchoolCoursesDialogOpen(false)); setTimeout(() => set...
              data-testid="button-new-course-school-courses"
            >
              <Plus className="h-3 w-3 mr-1" />
              New Course
            </Button>
          </div>
          {(() => {
            const semesterMeta = [
              { key: 'ss2025', year: 2025, label: 'Spring/Summer 2025', dates: 'May 5 - August 8, 2025' },
              { key: 'f2025', year: 2025, label: 'Fall 2025', dates: 'September 8 - December 12, 2025' },
              { key: 'w2026', year: 2026, label: 'Winter 2026', dates: 'January 12 - April 17, 2026' },
              { key: 'ss2026', year: 2026, label: 'Spring/Summer 2026', dates: 'May 4 - August 7, 2026' },
              { key: 'f2026', year: 2026, label: 'Fall 2026', dates: 'September 7 - December 11, 2026' },
              { key: 'w2027', year: 2027, label: 'Winter 2027', dates: 'January 11 - April 16, 2027' },
              { key: 'ss2027', year: 2027, label: 'Spring/Summer 2027', dates: 'May 3 - August 6, 2027' },
              { key: 'f2027', year: 2027, label: 'Fall 2027', dates: 'September 13 - December 17, 2027' },
              { key: 'w2028', year: 2028, label: 'Winter 2028', dates: 'January 10 - April 14, 2028' },
              { key: 'ss2028', year: 2028, label: 'Spring/Summer 2028', dates: 'May 1 - August 4, 2028' },
              { key: 'f2028', year: 2028, label: 'Fall 2028', dates: 'September 11 - December 15, 2028' },
              { key: 'w2029', year: 2029, label: 'Winter 2029', dates: 'January 8 - April 13, 2029' },
            ];
            const semesterDefs = semesterMeta.map(m => ({ ...m, courses: semesterCourseAssignments[m.key] || [] }...
            const allDefCodes = new Set(semesterDefs.flatMap(s => s.courses.map(c => c.code.toUpperCase()).repla...
            const currentSemKey = (() => {
              const now = new Date();
              if (now >= new Date('2025-05-05') && now <= new Date('2025-08-08')) return 'ss2025';
              if (now >= new Date('2025-09-08') && now <= new Date('2025-12-12')) return 'f2025';
              if (now >= new Date('2026-01-12') && now <= new Date('2026-04-17')) return 'w2026';
              if (now >= new Date('2026-05-04') && now <= new Date('2026-08-07')) return 'ss2026';
              if (now >= new Date('2026-09-07') && now <= new Date('2026-12-11')) return 'f2026';
              if (now >= new Date('2027-01-11') && now <= new Date('2027-04-16')) return 'w2027';
              if (now >= new Date('2027-05-03') && now <= new Date('2027-08-06')) return 'ss2027';
              if (now >= new Date('2027-09-13') && now <= new Date('2027-12-17')) return 'f2027';
              if (now >= new Date('2028-01-10') && now <= new Date('2028-04-14')) return 'w2028';
              if (now >= new Date('2028-05-01') && now <= new Date('2028-08-04')) return 'ss2028';
              if (now >= new Date('2028-09-11') && now <= new Date('2028-12-15')) return 'f2028';
              if (now >= new Date('2029-01-08') && now <= new Date('2029-04-13')) return 'w2029';
              return '';
            })();
            if (currentSemKey) {
              const currentSem = semesterDefs.find(s => s.key === currentSemKey);
              if (currentSem) {
                coursesData.courses.filter(c => c.name.trim()).forEach(c => {
                  const code = c.name.split(' - ')[0]?.trim().toUpperCase().replace(/\s/g, '');
                  const name = c.name.split(' - ').slice(1).join(' - ').trim();
                  if (code && !allDefCodes.has(code)) {
                    currentSem.courses.push({ code, name: code, fullName: name, period: '' });
                    allDefCodes.add(code);
                  }
                });
              }
            }
          })
        </div>
      </div>
    </div>
  )
}

```



```

    }
  });
}
}

const semStartDates: Record<string, string> = {
  'ss2025': '2025-05-05', 'f2025': '2025-09-08', 'w2026': '2026-01-12',
  'ss2026': '2026-05-04', 'f2026': '2026-09-07', 'w2027': '2027-01-11',
  'ss2027': '2027-05-03', 'f2027': '2027-09-13', 'w2028': '2028-01-10', 'ss2028': '2028-05-01', 'f2...
};
const hasSemStarted = (key: string) => {
  const start = semStartDates[key];
  return start ? new Date() >= new Date(start) : false;
};

const years = [...new Set(semesterDefs.map(s => s.year))];
const currentCoursesMap = new Map(coursesData.courses.filter(c => c.name.trim()).map(c => {
  const code = c.name.split(' - ')[0]?.trim().toUpperCase().replace(/\s/g, '');
  return [code, c];
}));

const allPastEntries = new Map<string, { certKey: string; code: string; name: string }>();
Object.entries(checkedCourses).forEach(([certKey, isChecked]) => {
  if (!isChecked) return;
  const info = certCourseMap[certKey];
  if (!info) return;
  const elective = openElectives[certKey]?.trim();
  let actualCode = info.code;
  let actualName = info.name;
  if (elective) {
    const codeMatch = elective.match(/^[A-Z]{2,5}\s?\d{3}[A-Z]?\\s*(.*)/i);
    if (codeMatch) {
      actualCode = codeMatch[1].replace(/\s/g, '');
      actualName = codeMatch[2] || actualCode;
    } else {
      actualCode = elective.split(' ')[0];
      actualName = elective.split(' ').slice(1).join(' ');
    }
  }
  if (!allPastEntries.has(actualCode.toUpperCase().replace(/\s/g, ''))) {
    allPastEntries.set(actualCode.toUpperCase().replace(/\s/g, ''), { certKey, code: actualCode, na...
  }
});

const isCurrent = (code: string) => {
  const now = new Date();
  const w2026End = new Date('2026-04-17');
  return now <= w2026End && ['CPA122', 'CFNF400', 'CASL101'].includes(code.toUpperCase().replace(/...
});

const renderCourseRow = (semCourse: { code: string; name: string; fullName?: string; period: string...
const codeNorm = semCourse.code.toUpperCase().replace(/\s/g, '');
const currentCourse = currentCoursesMap.get(codeNorm);
const pastEntry = allPastEntries.get(codeNorm);
const profInfo = pastCourseInfo[semCourse.code] || pastCourseInfo[codeNorm] || { professor: '', e...
const rowCertKey = pastEntry?.certKey || Object.keys(certCourseMap).find(k => { const mc = certCo...
const grade = courseGrades[rowCertKey]?.percent || courseGrades[codeNorm]?.percent || courseGrade...
const isCurrentCourse = !currentCourse || isCurrent(codeNorm);
const certType = courseCertificateTypes[semCourse.code] || courseCertificateTypes[codeNorm] || '';
const displayName = semCourse.name;
const subtitle = semCourse.fullName || '';
const dotColor = (() => {
  if (currentCourse) {
    return currentCourse.colorEnd ? `linear-gradient(to right, ${currentCourse.color}, ${currentC...
  }
} try {
  const cd = localStorage.getItem('certCourseData');
  if (cd) {
    const sd = JSON.parse(cd);
    const certKey = pastEntry?.certKey || semCourse.code;
    const saved = sd[certKey] || sd[codeNorm] || sd[semCourse.code];
    if (saved?.color) {
      return saved.colorEnd ? `linear-gradient(to right, ${saved.color}, ${saved.colorEnd})` : ...
    }
  }
} catch {}
return isCurrentCourse ? '#22c55e' : '#1e40af';
})();
const isSS = semKey.startsWith('ss');
const courseSSTerm = (() => {
  if (!isSS) return '';
  const sems = allSemesterSettingsRef.current;
  if (sems) {
    const cc = semCourse.code.replace(/\s/g, '');
    for (const sem of sems) {
      for (let i = 1; i <= 3; i++) {
        const sc = ((sem as any)['course${i}Code'] || '').replace(/\s/g, '');
        if (sc.toUpperCase() === cc.toUpperCase()) {
          return ((sem as any)['course${i}SpringSummerTerm'] || 'full').toLowerCase();
        }
      }
    }
  }
  return 'full';
})();
const ssNeedsA = isSS && (courseSSTerm === 'first_half' || courseSSTerm === 'full');
const ssNeedsB = isSS && (courseSSTerm === 'second_half' || courseSSTerm === 'full');
const priorityKeyA = isSS ? `${semKey}:${semCourse.code}:A` : `${semKey}:${semCourse.code}`;
const priorityKeyB = `${semKey}:${semCourse.code}:B`;
const currentPriorityA = draftCoursePlayPriority[priorityKeyA] ?? 0;
const currentPriorityB = draftCoursePlayPriority[priorityKeyB] ?? 0;
const getUsedForSuffix = (sfx: 'A' | 'B' | '') => {
  const used: number[] = [];
  const semCourses = semesterCourseAssignments[semKey] || [];
  for (const sc of semCourses) {
    if (sc.code === semCourse.code) continue;
    const pk = sfx ? `${semKey}:${sc.code}:${sfx}` : `${semKey}:${sc.code}`;
    const v = draftCoursePlayPriority[pk] ?? coursePlayPriority[pk];

```

```

        if (v && v > 0) used.push(v);
    }
    return used;
};

return (
    <div
      key={semCourse.code}
      className="items-center px-2 py-1.5 rounded bg-white/5 border border-white/10 hover:border-wh...
      style={{ display: 'flex', alignItems: 'center', gap: '6px' }}
      draggable
      onDragStart={(e) => {
        dragCourseRef.current = { code: semCourse.code, fromSemKey: semKey };
        e.dataTransfer.effectAllowed = 'move';
        if (e.currentTarget instanceof HTMLElement) e.currentTarget.style.opacity = '0.5';
      }}
      onDragEnd={(e) => {
        if (e.currentTarget instanceof HTMLElement) e.currentTarget.style.opacity = '1';
        dragCourseRef.current = null;
      }}
      onClick={() => {
        const certKey = pastEntry?.certKey || semCourse.code;
        startTransition(() => setSelectedCertCourse({ courseCode: semCourse.code, courseName: subti...
      }}
      data-testid={`school-course-${semCourse.code}`}
    >
      <div className="w-2.5 h-2.5 rounded-full flex-shrink-0" style={{ background: dotColor }} />
      {isSS ? (
        <div style={{ display: 'flex', alignItems: 'center', width: '70px', minWidth: '70px', flexS...
          {ssNeedsA && (
            <PrioritySelect
              priorityKey={priorityKeyA}
              initialValue={currentPriorityA}
              totalInSem={totalInSem}
              draftRef={draftCoursePlayPriorityRef}
              courseCode={semCourse.code}
              usedValues={getUsedForSuffix('A')}
              onPriorityChange={handlePriorityChange}
              suffix="A"
            />
          )}
          {ssNeedsB && (
            <PrioritySelect
              priorityKey={priorityKeyB}
              initialValue={currentPriorityB}
              totalInSem={totalInSem}
              draftRef={draftCoursePlayPriorityRef}
              courseCode={semCourse.code}
              usedValues={getUsedForSuffix('B')}
              onPriorityChange={handlePriorityChange}
              suffix="B"
            />
          )}
        </div>
      ) : (
        <PrioritySelect
          priorityKey={priorityKeyA}
          initialValue={currentPriorityA}
          totalInSem={totalInSem}
          draftRef={draftCoursePlayPriorityRef}
          courseCode={semCourse.code}
          usedValues={getUsedForSuffix('')}
          onPriorityChange={handlePriorityChange}
        />
      )}
    </div>
    <div
      className={`flex flex-col items-center cursor-pointer hover:opacity-80 transition-opacity f...
      style={{ width: '24px', marginTop: '2px', marginLeft: '5px' }}
      data-testid={`aas-status-${semCourse.code}`}
      onClick={(e) => { e.stopPropagation(); toggleAasSent(semCourse.code); }}
    >
      <div className="flex items-center justify-center border border-white rounded-sm" style={{ w...
        {(aasSentStatus[semCourse.code] || aasSentStatus[semCourse.code.replace(/^[A-Z+](\d)/, ...
      </div>
      <span className="text-[7px] text-white font-medium" style={{ lineHeight: 1.1, marginTop: '1...
    </div>
    <button
      className="flex-shrink-0 text-white/40 hover:text-yellow-300 transition-colors p-0.5"
      style={{ marginLeft: '0px' }}
      title="Course comments"
      onClick={(e) => { e.stopPropagation(); setDashboardCommentTarget(dashboardCommentTarget?.ty...
      data-testid={`button-comment-course-${semCourse.code}`}
    >
      <MessageSquare className="w-3.5 h-3.5 strokeWidth={2.5} />
    </button>
    <span className="text-[10px] truncate min-w-0 flex-1 cursor-pointer hover:underline" style={{...
    <div className="flex items-center gap-1 flex-shrink-0" style={{ marginRight: '-3px' }}>
      {(() => {
        const profName = currentCourse?.professor || profInfo.professor;
        const profEmail = currentCourse?.professorEmail || profInfo.email;
        if (!profName) return null;
        return profEmail ? (
          <a
            className="text-[9px] text-white truncate underline cursor-pointer hover:text-white/80"
            style={{ width: '80px', display: 'inline-block', textAlign: 'right' }}
            href={`https://mail.google.com/mail/?view=cm&to=${encodeURIComponent(profEmail)}`}
            target="blank"
            rel="noopener noreferrer"
            onClick={(e) => e.stopPropagation()}
            data-testid={`link-professor-${semCourse.code}`}
          >{profName}</a>
        ) : (
          <span className="text-[9px] text-white truncate" style={{ width: '80px', display: 'inli...
        );
      })()}
    </div>
    <button
      className="flex-shrink-0 p-0.5 rounded hover:bg-white/10"

```



```

    >
    <ChevronUp className="text-white" style={{ width: '18px', height: '18px' }} />
  </button>
</div>
))

<div className="flex items-center justify-end px-4 py-3 border-t border-white/20">
  <div className="flex gap-2">
    <Button variant="outline" className="border !border-white/30 text-white/70 hover:text-white hover:!bor...
    <Button variant="outline" className="border !border-white/50 text-white hover:text-white hover:!bor...
    const draft = draftCoursePlayPriorityRef.current;
    setCoursePlayPriority(draft);
    localStorage.setItem('coursePlayPriority', JSON.stringify(draft));
    fetch('/api/course-play-priority', { method: 'POST', headers: { 'Content-Type': 'application/json...
    startTransition(() => setIsSchoolCoursesDialogOpen(false));
  }} data-testid="button-save-school-courses">Save</Button>
  </div>
</div>
</div>
</div>,
document.body
))

{semSettingsDialogKey && createPortal(
  <div>
    <div className="fixed inset-0 z-[10003] bg-black/50" onClick={() => setSemSettingsDialogKey(null)} />
    <div
      className="fixed left-[50%] top-[50%] translate-x-[-50%] translate-y-[-50%] z-[10003] overflow-hidden f...
      style={{ width: '500px', maxWidth: '94vw', maxHeight: '80vh', background: 'linear-gradient(180deg, ${co...
    >
    <div className="flex items-center justify-between px-4 py-3 border-b border-white/40 flex-shrink-0 roun...
    <div className="flex items-center gap-2">
      <Settings className="text-white" style={{ width: '14px', height: '14px' }} />
      <h2 className="font-normal text-white" style={{ fontFamily: 'Avenir, 'Avenir Next', -apple-system, ...
        SEMESTER SETTINGS
      </h2>
    </div>
    <span className="text-[11px] font-medium text-white" style={{ textShadow: '0 1px 2px rgba(0,0,0,0.2)'}}...
    ({() => { const semesterMeta: Record<string, string> = { 'ss2025': 'Spring/Summer 2025', 'f2025': '...
    </span>
  </div>
  <SemesterSettingsFormBody
    semKey={semSettingsDialogKey}
    existing={perSemesterSettings[semSettingsDialogKey] || {}}
    onCancel={() => setSemSettingsDialogKey(null)}
    onSave={(data) => {
      const semKey = semSettingsDialogKey!;
      const updated = { ...perSemesterSettings, [semKey]: data };
      setPerSemesterSettings(updated);
      localStorage.setItem('perSemesterSettings', JSON.stringify(updated));
      if (semKey === 'w2026') {
        saveSchool({ ...schoolData, numberOfWeeks: data.numberOfWeeks, week1StartDate: data.week1StartDat...
        if (data.readingWeekDate) { apiRequest("PATCH", "/api/semester", { readingWeekStart: new Date(dat...
        else { apiRequest("PATCH", "/api/semester", { readingWeekStart: null }); }
      }
      if (semKey.startsWith('ss') && (data.springStartDate || data.springEndDate || data.summerStartDate ...
        apiRequest("POST", "/api/semester-halves", { semKey, springStartDate: data.springStartDate, sprin...
      }
      setSemSettingsDialogKey(null);
    })
  </div>
</div>,
document.body
))

{ /* Remaining Courses Dialog */ }
<Dialog open={isRemainingCoursesDialogOpen} onOpenChange={setIsRemainingCoursesDialogOpen}>
  <DialogContent
    className="overflow-hidden flex flex-col max-w-3xl max-h-[85vh] text-[11px] text-white [&_*]:text-white...
  >
    <div className="flex items-center justify-between px-4 py-1.5 border-b border-white/40 flex-shrink-0 ro...
    <div className="flex items-center gap-2">
      <GraduationCap className="h-3 w-3 text-white" />
      <h2 className="font-normal text-white" style={{ fontFamily: 'Avenir, 'Avenir Next', -apple-system, ...
        REMAINING COURSES
      </h2>
    </div>
    <button onClick={() => setIsRemainingCoursesDialogOpen(false)} className="text-white hover:text-white...
    <X className="h-5 w-5" />
  </button>
</div>
  {() => {
    const remainingItems = { level: string; category: string; items: string[]; needed: number }[] = [];
    const addRemaining = (level: string, category: string, section: { required: number; members: string[]...
    const checked = section.members.filter(m => checkedCourses[m]);
    const needed = section.required - checked.length;
    if (needed <= 0) return;
    const unchecked = section.members.filter(m => !checkedCourses[m]);
    const names = unchecked.map(m => {
      const info = certCourseMap[m];
      const elective = openElectives[m];
      if (elective?.trim()) return elective;
      return info ? `${info.code} - ${info.name}` : m;
    });
    remainingItems.push({ level, category, items: names, needed });
  };
  addRemaining('Certificate', 'Core Requirements', certSections.L1[0]);
  addRemaining('Certificate', 'Core Electives (pick 2 of 4)', certSections.L1[1]);
  addRemaining('Certificate', 'Liberal Studies', certSections.L1[2]);
  addRemaining('Certificate', 'PR Electives', certSections.L1[3]);
  addRemaining('Adv. Certificate', 'Core Requirement', certSections.L2[0]);
  addRemaining('Adv. Certificate', 'Core Electives (pick 3 of 7)', certSections.L2[1]);
  addRemaining('Adv. Certificate', 'Liberal Studies', certSections.L2[2]);
  addRemaining('Adv. Certificate', 'Economics (pick 1 of 8)', certSections.L2[3]);
  addRemaining('Adv. Certificate', 'PR Electives', certSections.L2[4]);
  addRemaining('Degree', 'Required Professional (CPPA 333 + CPPA 51)', certSections.L3[0]);
  addRemaining('Degree', 'Required Professional (pick 8 of 14)', certSections.L3[1]);

```



```

const targetIdx = emptyIdx !== -1 ? emptyIdx : updatedCourses.length;
if (emptyIdx !== -1) {
  updatedCourses[emptyIdx] = {
    name: fullName,
    color: courseData.color,
    professor: courseData.professorName,
    professorEmail: courseData.professorEmail,
  };
} else {
  updatedCourses.push({
    name: fullName,
    color: courseData.color,
    professor: courseData.professorName,
    professorEmail: courseData.professorEmail,
  });
}
setCoursesData({ courses: updatedCourses });
localStorage.setItem('coursesData', JSON.stringify({ courses: updatedCourses }));
saveDegreeToServer('coursesData', { courses: updatedCourses });

const prefix = `course${targetIdx + 1}`;
if (targetIdx < 3) {
  const schedulePayload: Record<string, any> = {
    semesterType: courseData.semesterType,
    [`${prefix}DeliveryMode`]: courseData.deliveryMode || null,
    [`${prefix}ClassDay`]: courseData.classDay || null,
    [`${prefix}ClassDay2`]: courseData.classDay2 || null,
    [`${prefix}ClassTime`]: courseData.classTime || null,
    [`${prefix}ClassEndTime`]: courseData.classEndTime || null,
    [`${prefix}ClassTime2`]: (courseData as any).classTime2 || null,
    [`${prefix}ClassEndTime2`]: (courseData as any).classEndTime2 || null,
    [`${prefix}SpringSummerTerm`]: courseData.springSummerTerm || null,
    [`${prefix}StartDate`]: courseData.startDate ? new Date(courseData.startDate).toISOString() : null,
    [`${prefix}EndDate`]: courseData.endDate ? new Date(courseData.endDate).toISOString() : null,
    [`${prefix}Color`]: courseData.color || null,
    [`${prefix}ColorEnd`]: courseData.colorEnd || null,
  };
  saveSemesterScheduleMutation.mutate(schedulePayload);
}

if (courseData.reminders && courseData.reminders.length > 0) {
  const courseReminders = JSON.parse(localStorage.getItem('courseReminders') || '{}');
  courseReminders[fullName] = courseData.reminders;
  localStorage.setItem('courseReminders', JSON.stringify(courseReminders));
}

newCourseDialogClosingRef.current = true;
setIsNewCourseDialogOpen(false);
setTimeout(() => { newCourseDialogClosingRef.current = false; }, 300);
toast({ title: "Course added", description: `${fullName} has been added.` });

if (courseData.deadlines.length > 0) {
  (async () => {
    for (const deadline of courseData.deadlines) {
      if (deadline.title && deadline.dueDate) {
        try {
          const dueDate = new Date(deadline.dueDate);
          dueDate.setHours(23, 59, 0, 0);
          await apiRequest("POST", "/api/tasks", {
            title: deadline.title,
            description: deadline.description || '',
            type: deadline.type || 'assignment',
            courseName: fullName,
            dueDate: dueDate.toISOString(),
            priority: deadline.type === 'exam' || deadline.type === 'quiz' ? 'high' : 'medium',
            weekNumber: getWeekNumber(dueDate, semStart, readingWeekStart),
            reminder1: DEFAULT_REMINDER_1,
            reminder2: DEFAULT_REMINDER_2,
          });
        } catch (err) {
          console.error("Failed to create deadline task:", err);
        }
      }
    }
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    toast({ title: "Deadlines created", description: `${courseData.deadlines.length} deadline(s) added.` });
  })();
}
}
onClose={() => {
  newCourseDialogClosingRef.current = true;
  setIsNewCourseDialogOpen(false);
  setTimeout(() => { newCourseDialogClosingRef.current = false; }, 300);
}}
</DialogContent>
</Dialog>

{isNewCourseWizardOpen && (
  <NewCourseWizard
    existingSemesterType={newSemesterForm.semesterType || "winter"}
    colorSettings={{ mainBackground: colorSettings.mainBackground, mainBackgroundGradientEnd: colorSettings...
    onClose={() => setIsNewCourseWizardOpen(false)}
    onSave={({wizardData}) => {
      const fullName = `${wizardData.courseCode} - ${wizardData.courseName}`;
      const updatedCourses = [...coursesData.courses];
      const emptyIdx = updatedCourses.findIndex(c => !c.name.trim());
      const targetIdx = emptyIdx !== -1 ? emptyIdx : updatedCourses.length;
      if (emptyIdx !== -1) {
        updatedCourses[emptyIdx] = {
          name: fullName,
          color: wizardData.color,
          colorEnd: wizardData.colorEnd,
          professor: wizardData.professorName,
          professorEmail: wizardData.professorEmail,
        };
      } else {
        updatedCourses.push({

```



```

}}>
<DialogContent data-settings-dialog className="max-w-4xl max-h-[98vh] text-[9px] text-white [&_*]:text-wh...
  {/* Header bar matching flyouts */}
  <div className="flex items-center justify-between px-4 py-3 border-b border-white/40 flex-shrink-0 roun...
    <div className="flex items-center gap-2">
      <Settings className="h-3 w-3 text-white" />
      <h2 className="font-normal text-white" style={{ fontFamily: 'Avenir', 'Avenir Next', '-apple-system, ...
        CALENDAR SETTINGS
      </h2>
    </div>
  </div>
</div>
<div className="px-4 pb-3 pt-0 overflow-y-auto flex-1" style={{ marginTop: '-4px', scrollbarWidth: 'thi...
<div className="grid grid-cols-2 gap-3">
  {/* Left Column */}
  <div className="flex flex-col gap-3 justify-between">
    <div className="flex flex-col gap-3">
      <div className="border rounded-lg p-3 space-y-3" style={{ marginTop: '-1px' }}>
        <div className="border-b border-primary inline-block -mt-1 pb-0">
          <Label className="text-sm font-medium">Colour Settings</Label>
          <span className="text-sm" style={{ color: '#3b82f6' }}>&nbsp;</span><span className="text-xs tex...
        </div>

      <div className="space-y-4">
        {/* Box Background Colour */}
        <div className="flex items-center gap-3">
          <Label className="text-xs whitespace-nowrap flex-1">Task Boxes Background</Label>
          <input
            type="text"
            className="text-xs font-mono w-[60px] text-right flex-shrink-0 bg-white/10 border border-whit...
            value={colorSettings.boxBackground}
            onChange={(e) => {
              let val = e.target.value;
              if (!val.startsWith('#')) val = '#' + val;
              if (/^[0-9a-fA-F]{0,6}$/i.test(val)) {
                setColorSettings(prev => ({ ...prev, boxBackground: val }));
              }
            }}
            data-testid="input-box-background-hex"
          />
          <div className="w-14 flex justify-end">
            <div className="relative">
              <div
                className="w-5 h-5 rounded cursor-pointer border border-white/30"
                style={{
                  backgroundColor: colorSettings.boxGlassEffect
                    ? `rgba(${parseInt(colorSettings.boxBackground.slice(1,3), 16)}, ${parseInt(colorSett...
                    : colorSettings.boxBackground
                }}
                onClick={() => document.getElementById('color-box-background-input')?.click()}
                data-testid="color-box-background"
              />
              <input
                id="color-box-background-input"
                type="color"
                value={colorSettings.boxBackground}
                onChange={(e) => setColorSettings(prev => ({ ...prev, boxBackground: e.target.value })))
                className="absolute opacity-0 w-0 h-0"
              />
            </div>
          </div>
        </div>
      </div>

      {/* Glass Effect Toggle */}
      <div className="flex items-center justify-between">
        <Label className="text-xs">Header Glass Effect</Label>
        <div
          className={`w-5 h-2.5 rounded-full cursor-pointer transition-colors flex items-center ${color...
          onClick={() => setColorGlassSettings(prev => ({ ...prev, boxGlassEffect: !prev.boxGlassEffect })))
          data-testid="toggle-glass-effect"
        >
          <div className={`w-1.5 h-1.5 bg-white rounded-full transition-transform ${colorSettings.boxGl...
        </div>
      </div>

      {/* Transparency Slider */}
      {colorSettings.boxGlassEffect && (
        <div className="flex items-center gap-3">
          <Label className="text-xs whitespace-nowrap flex-1">Transparency</Label>
          <input
            type="range"
            min="10"
            max="90"
            value={colorSettings.boxTransparency}
            onChange={(e) => setColorSettings(prev => ({ ...prev, boxTransparency: parseInt(e.target.va...
            className="w-1/3 h-0.5 bg-muted rounded-lg appearance-none cursor-pointer [&::-webkit-slide...
            data-testid="slider-transparency"
          />
          <span className="text-xs text-muted-foreground w-14 text-right">{colorSettings.boxTransparenc...
        </div>
      )}

    </div>
  </div>

  {/* Header Bar Colour */}
  <div className="flex items-center gap-3">
    <Label className="text-xs whitespace-nowrap flex-1">Upcoming Header, Date, and Time Label Backg...
    <input
      type="text"
      className="text-xs font-mono w-[60px] text-right flex-shrink-0 bg-white/10 border border-whit...
      value={colorSettings.headerBar}
      onChange={(e) => {
        let val = e.target.value;
        if (!val.startsWith('#')) val = '#' + val;
        if (/^[0-9a-fA-F]{0,6}$/i.test(val)) {
          setColorSettings(prev => ({ ...prev, headerBar: val }));
        }
      }}
      data-testid="input-header-bar-hex"
    />
  </div>
  <div className="w-14 flex justify-end">

```

```

<div className="relative">
  <div
    className="w-5 h-5 rounded cursor-pointer border border-white/30"
    style={{ backgroundColor: colorSettings.headerBar }}
    onClick={() => document.getElementById('color-header-bar-input')?.click()}
    data-testid="color-header-bar"
  />
  <input
    id="color-header-bar-input"
    type="color"
    value={colorSettings.headerBar}
    onChange={(e) => setColorSettings(prev => ({ ...prev, headerBar: e.target.value })))
    className="absolute opacity-0 w-0 h-0"
  />
</div>
</div>
</div>
</div>

{/* Today Column Cell Background */}
<div className="flex items-center gap-3">
  <Label className="text-xs whitespace-nowrap flex-1">Today Column</Label>
  <input
    type="text"
    className="text-xs font-mono w-[60px] text-right flex-shrink-0 bg-white/10 border border-whit..."
    value={colorSettings.todayCellBackground}
    onChange={(e) => {
      let val = e.target.value;
      if (!val.startsWith('#')) val = '#' + val;
      if (/^[0-9a-fA-F]{0,6}$/i.test(val)) {
        setColorSettings(prev => ({ ...prev, todayCellBackground: val }));
      }
    }}
    data-testid="input-today-cell-hex"
  />
  <div className="w-14 flex justify-end">
    <input
      type="color"
      value={colorSettings.todayCellBackground}
      onChange={(e) => setColorSettings(prev => ({ ...prev, todayCellBackground: e.target.value })))
      className="w-5 h-5 cursor-pointer border-0 p-0 bg-transparent"
      style={{ WebkitAppearance: 'none' }}
      data-testid="color-today-cell"
    />
  </div>
</div>

{/* Current Time Row Background */}
<div className="flex items-center gap-3">
  <Label className="text-xs whitespace-nowrap flex-1">Current Time Row</Label>
  <input
    type="text"
    className="text-xs font-mono w-[60px] text-right flex-shrink-0 bg-white/10 border border-whit..."
    value={colorSettings.currentHourRowBackground}
    onChange={(e) => {
      let val = e.target.value;
      if (!val.startsWith('#')) val = '#' + val;
      if (/^[0-9a-fA-F]{0,6}$/i.test(val)) {
        setColorSettings(prev => ({ ...prev, currentHourRowBackground: val }));
      }
    }}
    data-testid="input-current-hour-row-hex"
  />
  <div className="w-14 flex justify-end">
    <input
      type="color"
      value={colorSettings.currentHourRowBackground}
      onChange={(e) => setColorSettings(prev => ({ ...prev, currentHourRowBackground: e.target.va...
      className="w-5 h-5 cursor-pointer border-0 p-0 bg-transparent"
      style={{ WebkitAppearance: 'none' }}
      data-testid="color-current-hour-row"
    />
  </div>
</div>

{/* Today Date & Current Hour Cell Background */}
<div className="flex items-center gap-3">
  <Label className="text-xs whitespace-nowrap flex-1">Today Date & Current Hour</Label>
  <input
    type="text"
    className="text-xs font-mono w-[60px] text-right flex-shrink-0 bg-white/10 border border-whit..."
    value={colorSettings.todayCurrentHourCellBackground}
    onChange={(e) => {
      let val = e.target.value;
      if (!val.startsWith('#')) val = '#' + val;
      if (/^[0-9a-fA-F]{0,6}$/i.test(val)) {
        setColorSettings(prev => ({ ...prev, todayCurrentHourCellBackground: val }));
      }
    }}
    data-testid="input-today-current-hour-hex"
  />
  <div className="w-14 flex justify-end">
    <input
      type="color"
      value={colorSettings.todayCurrentHourCellBackground}
      onChange={(e) => setColorSettings(prev => ({ ...prev, todayCurrentHourCellBackground: e.tar...
      className="w-5 h-5 cursor-pointer border-0 p-0 bg-transparent"
      style={{ WebkitAppearance: 'none' }}
      data-testid="color-today-current-hour-cell"
    />
  </div>
</div>

{/* Main Background Overlay Toggle */}
<div className="flex items-center justify-between">
  <Label className="text-xs">Main Background Colour Overlay</Label>
  <div className="flex items-center gap-2">
    <div
      className={`w-5 h-2.5 rounded-full cursor-pointer transition-colors flex items-center ${col...

```



```

        className="absolute opacity-0 w-0 h-0"
      />
    </div>
  </div>
</div>

/* Gradient End Colour - only visible when gradient is on */
{colorSettings.mainBackgroundGradient && colorSettings.mainBackgroundOverlay && (
  <div className="flex items-center gap-3">
    <Label className="text-xs whitespace-nowrap flex-1">Gradient Bottom (Darkest)</Label>
    <input
      type="text"
      className="text-xs font-mono w-[60px] text-right flex-shrink-0 bg-white/10 border border-wh...
      value={colorSettings.mainBackgroundGradientEnd}
      onChange={(e) => {
        let val = e.target.value;
        if (!val.startsWith('#')) val = '#' + val;
        if (/^[0-9a-fA-F]{0,6}$/.test(val)) {
          setColorSettings(prev => ({ ...prev, mainBackgroundGradientEnd: val }));
        }
      }}
      data-testid="input-gradient-bottom-hex"
    />
    <div className="w-14 flex justify-end flex-shrink-0">
      <div className="relative w-5 h-5">
        <div
          className="w-5 h-5 rounded border border-white/30 cursor-pointer"
          style={{ backgroundColor: colorSettings.mainBackgroundGradientEnd }}
          onClick={() => document.getElementById('color-main-background-gradient-end-input').cli...
          data-testid="color-main-background-gradient-end"
        />
        <input
          id="color-main-background-gradient-end-input"
          type="color"
          value={colorSettings.mainBackgroundGradientEnd}
          onChange={(e) => setColorSettings(prev => ({ ...prev, mainBackgroundGradientEnd: e.targ...
          className="absolute opacity-0 w-0 h-0"
        />
      </div>
    </div>
  </div>
)}

/* Gradient Preview */
{colorSettings.mainBackgroundGradient && colorSettings.mainBackgroundOverlay && (
  <div className="flex items-center gap-3">
    <Label className="text-xs whitespace-nowrap flex-1">Preview</Label>
    <div
      className="w-full h-4 rounded"
      style={{ background: `linear-gradient(90deg, ${colorSettings.mainBackground} 0%, ${colorSet...
      data-testid="gradient-preview"
    />
  </div>
)}
</div>
</div>

/* Layout Settings - in left column */
<div className="border rounded-lg p-3 space-y-3" style={{ marginTop: '-5px' }}>
  <div className="border-b border-primary inline-block -mt-1 pb-0">
    <Label className="text-sm font-medium">Layout Settings</Label>
    <span className="text-sm" style={{ color: '#3b82f6' }}>&nbsp;</span><span className="text-xs tex...
  </div>

  <div className="space-y-3">
    <div className="flex items-center gap-3">
      <Label className="text-xs whitespace-nowrap flex-1">Header Button Spacing</Label>
      <input
        type="range"
        min="0"
        max="40"
        step="2"
        value={blinkSettings.buttonSpacing}
        onChange={(e) => setBlinkSettings(prev => ({ ...prev, buttonSpacing: Number(e.target.value) })})
        className="w-1/3 h-0.5 bg-muted rounded-lg appearance-none cursor-pointer [&::-webkit-slider-...
        style={{ touchAction: 'none' }}
        data-testid="slider-button-spacing"
      />
      <span className="text-xs text-muted-foreground w-14 text-right">{blinkSettings.buttonSpacing}px...
    </div>

    <div className="flex items-center gap-3">
      <Label className="text-xs whitespace-nowrap flex-1">Media Controls Spacing</Label>
      <input
        type="range"
        min="4"
        max="40"
        step="2"
        value={blinkSettings.mediaControlSpacing}
        onChange={(e) => setBlinkSettings(prev => ({ ...prev, mediaControlSpacing: Number(e.target.va...
        className="w-1/3 h-0.5 bg-muted rounded-lg appearance-none cursor-pointer [&::-webkit-slider-...
        data-testid="slider-media-control-spacing"
      />
      <span className="text-xs text-muted-foreground w-14 text-right">{blinkSettings.mediaControlSpac...
    </div>

    <div className="flex items-center gap-3">
      <Label className="text-xs whitespace-nowrap flex-1">Menu Background Button Spacing</Label>
      <input
        type="range"
        min="20"
        max="40"
        step="1"
        value={blinkSettings.tallPillButtonSpacing}
        onChange={(e) => setBlinkSettings(prev => ({ ...prev, tallPillButtonSpacing: Number(e.target....
        className="w-1/3 h-0.5 bg-muted rounded-lg appearance-none cursor-pointer [&::-webkit-slider-...
        data-testid="slider-tall-pill-button-spacing"
      />
    </div>
  </div>

```

```

        <span className="text-xs text-muted-foreground w-14 text-right">{blinkSettings.tallPillButtonSp...
    </div>

    <div className="flex items-center gap-3">
        <Label className="text-xs whitespace-nowrap flex-1">Menu Background Height</Label>
        <input
            type="range"
            min="-50"
            max="100"
            step="1"
            value={blinkSettings.tallPillHeight}
            onChange={(e) => setBlinkSettings(prev => ({ ...prev, tallPillHeight: Number(e.target.value) ...
            className="w-1/3 h-0.5 bg-muted rounded-lg appearance-none cursor-pointer [&::-webkit-slider-...
            data-testid="slider-tall-pill-height"
        />
        <span className="text-xs text-muted-foreground w-14 text-right">{blinkSettings.tallPillHeight}p...
    </div>

</div>
</div>

</div>

</div>

/* Right Column */
<div className="flex flex-col gap-3 justify-between" style={{ marginTop: '-2px' }}>
<div className="flex flex-col gap-3">
    /* Blinking & Spacing Settings */
    <div className="border rounded-lg p-3 space-y-3" style={{ marginTop: '0px' }}>
        <div className="border-b border-primary inline-block -mt-1 pb-0">
            <Label className="text-sm font-medium">Blinking & Spacing</Label>
            <span className="text-sm" style={{ color: '#3b82f6' }}>&nbsp;</span><span className="text-xs tex...
        </div>

        <div className="space-y-4">
            /* Today Column Blink */
            <div className="space-y-2">
                <div className="flex items-center justify-between">
                    <Label className="text-xs font-medium">Today Column Blink</Label>
                    <input
                        type="checkbox"
                        checked={blinkSettings.todayColumnBlink}
                        onChange={(e) => setBlinkSettings(prev => ({ ...prev, todayColumnBlink: e.target.checked })))
                        className="h-1.5 w-1.5 rounded border-gray-300 accent-[#3b82f6]"
                        data-testid="toggle-today-column-blink"
                    />
                </div>
                {blinkSettings.todayColumnBlink && (
                    <div className="flex items-center gap-3">
                        <Label className="text-[10px] text-muted-foreground flex-1">Speed</Label>
                        <input
                            type="range"
                            min="0.2"
                            max="2"
                            step="0.1"
                            value={blinkSettings.todayColumnBlinkSpeed}
                            onChange={(e) => setBlinkSettings(prev => ({ ...prev, todayColumnBlinkSpeed: Number(e.tar...
                            className="w-1/3 h-0.5 bg-muted rounded-lg appearance-none cursor-pointer [&::-webkit-sli...
                            data-testid="slider-today-column-speed"
                        />
                        <span className="text-[10px] text-muted-foreground w-14 text-right">{blinkSettings.todayCol...
                    </div>
                )}
            </div>

            /* All Day Files Blink */
            <div className="space-y-2">
                <div className="flex items-center justify-between">
                    <Label className="text-xs font-medium">All Day Files Blink</Label>
                    <input
                        type="checkbox"
                        checked={blinkSettings.allDayFilesBlink}
                        onChange={(e) => setBlinkSettings(prev => ({ ...prev, allDayFilesBlink: e.target.checked })))
                        className="h-1.5 w-1.5 rounded border-gray-300 accent-[#3b82f6]"
                        data-testid="toggle-allday-files-blink"
                    />
                </div>
                {blinkSettings.allDayFilesBlink && (
                    <div className="flex items-center gap-3">
                        <Label className="text-[10px] text-muted-foreground flex-1">Speed</Label>
                        <input
                            type="range"
                            min="0.2"
                            max="2"
                            step="0.1"
                            value={blinkSettings.allDayFilesBlinkSpeed}
                            onChange={(e) => setBlinkSettings(prev => ({ ...prev, allDayFilesBlinkSpeed: Number(e.tar...
                            className="w-1/3 h-0.5 bg-muted rounded-lg appearance-none cursor-pointer [&::-webkit-sli...
                            data-testid="slider-allday-files-speed"
                        />
                        <span className="text-[10px] text-muted-foreground w-14 text-right">{blinkSettings.allDayFi...
                    </div>
                )}
            </div>
        )}
    </div>

    /* Show Arrows Toggle */

</div>
</div>

/* TTS Highlighting Settings */
<div className="border rounded-lg p-3 space-y-3" style={{ marginTop: '-4px' }}>
    <div className="border-b border-primary inline-block -mt-1 pb-0">
        <Label className="text-sm font-medium">Text-to-Speech Highlighting</Label>
        <span className="text-sm" style={{ color: '#3b82f6' }}>&nbsp;</span><span className="text-xs tex...
    </div>

```

```

<div className="space-y-4">
  <div className="space-y-1">
    <div className="flex items-center gap-3">
      <Label className="text-xs whitespace-nowrap flex-1">Start Delay</Label>
      <input
        type="range"
        min="5"
        max="60"
        step="1"
        value={ttsSettings.startDelay}
        onChange={(e) => setTtsSettings(prev => ({ ...prev, startDelay: Number(e.target.value) })))}
        className="w-1/3 h-0.5 bg-muted rounded-lg appearance-none cursor-pointer [&::-webkit-slide...
        data-testid="input-tts-start-delay"
      />
      <span className="text-xs text-muted-foreground w-14 text-right">{ttsSettings.startDelay}s</span>
    </div>
    <p className="text-xs text-muted-foreground">
      Time before highlighting begins (network + TTS processing)
    </p>
  </div>

  <div className="space-y-1">
    <div className="flex items-center gap-3">
      <Label className="text-xs whitespace-nowrap flex-1">Speech Rate</Label>
      <input
        type="range"
        min="60"
        max="200"
        step="5"
        value={ttsSettings.wordsPerMinute}
        onChange={(e) => setTtsSettings(prev => ({ ...prev, wordsPerMinute: Number(e.target.value) ...
        className="w-1/3 h-0.5 bg-muted rounded-lg appearance-none cursor-pointer [&::-webkit-slide...
        data-testid="input-tts-wpm"
      />
      <span className="text-xs text-muted-foreground w-14 text-right">{ttsSettings.wordsPerMinute} ...
    </div>
    <p className="text-xs text-muted-foreground">
      Words per minute - match your TTS voice speed
    </p>
  </div>

  <div className="flex items-center justify-between">
    <div className="space-y-0.5">
      <Label className="text-xs">Smart Timing</Label>
      <p className="text-xs text-muted-foreground">
        Adjust for word length (longer words = longer highlight)
      </p>
    </div>
    <input
      type="checkbox"
      checked={ttsSettings.useSmartTiming}
      onChange={(e) => setTtsSettings(prev => ({ ...prev, useSmartTiming: e.target.checked })))}
      className="h-1.5 w-1.5 rounded border-gray-300 accent-[#3b82f6]"
      data-testid="input-tts-smart-timing"
    />
  </div>

  <Button
    variant="outline"
    size="sm"
    className="h-6 text-xs px-2"
    onClick={() => saveTtsSettings(ttsSettings)}
    data-testid="button-save-tts-settings"
  >
    Save TTS
  </Button>
</div>
</div>

/* Data Sync Section */
<div className="border rounded-lg p-3 space-y-3" style={{ marginTop: '-4px' }}>
  <div className="border-b border-primary inline-block -mt-1 pb-0">
    <Label className="text-sm font-medium">Data Sync</Label>
    <span className="text-sm" style={{ color: '#3b82f6' }}>&nbsp;&nbsp;&nbsp;</span><span className="text-xs tex...
  </div>
  <div className="flex gap-2">
    <Button
      variant="outline"
      size="sm"
      className="h-6 text-xs px-2"
      onClick={async () => {
        try {
          toast({ title: "Pushing...", description: "Sending data to production." });

          const exportRes = await fetch("/api/export");
          const exportResData = await exportRes.json();

          const importRes = await fetch("https://home-view--bkh416.replit.app/api/import", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(exportData),
          });
          const result = await importRes.json();

          if (result.success) {
            // Clean up duplicates and sync file names
            try {
              await fetch("https://home-view--bkh416.replit.app/api/cleanup-duplicates", { method: "P...
            } catch (e) {
              // Cleanup is optional
            }

            toast({
              title: "Push complete!",
              description: `Pushed ${result.imported.tasks} tasks, ${result.imported.files} files.`
            });
          } else {

```

```

        toast({ title: "Push failed", description: result.error, variant: "destructive" });
    }
    catch (err: any) {
        console.error("Push error:", err);
        toast({ title: "Push failed", description: err?.message || "Could not connect to production..." });
    }
    })
    data-testid="button-push-production"
  >
  <Upload className="h-3 w-3 mr-1" />
  Push
</Button>
<Button
  variant="outline"
  size="sm"
  className="h-6 text-xs px-2"
  onClick={async () => {
    try {
      toast({ title: "Pulling...", description: "Getting data from production." });

      const exportRes = await fetch("https://home-view--bkh416.replit.app/api/export");
      if (!exportRes.ok) {
        toast({ title: "Pull failed", description: `Export failed: ${exportRes.status}`, variant: "destructive" });
        return;
      }
      const exportData = await exportRes.json();
      console.log("Export data:", exportData);

      const importRes = await fetch("/api/import", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(exportData),
      });
      if (!importRes.ok) {
        toast({ title: "Pull failed", description: `Import failed: ${importRes.status}`, variant: "destructive" });
        return;
      }
      const result = await importRes.json();
      console.log("Import result:", result);

      if (result.success) {
        toast({
          title: "Pull complete!",
          description: `Pulled ${result.imported.tasks} tasks, ${result.imported.files} files.`
        });
        queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
        queryClient.invalidateQueries({ queryKey: ["/api/files"] });
        queryClient.invalidateQueries({ queryKey: ["/api/semester"] });
        refreshFileCounts();
      } else {
        toast({ title: "Pull failed", description: result.error || "Unknown error", variant: "destructive" });
      }
    } catch (err: any) {
      console.error("Pull error:", err);
      toast({ title: "Pull failed", description: err?.message || "Could not connect to production..." });
    }
  }}
  data-testid="button-pull-production"
  >
  <Download className="h-3 w-3 mr-1" />
  Pull
</Button>
</div>
</div>

/* Column Spacing */
<div className="border rounded-lg p-3 space-y-2" style={{ marginTop: '-5px' }}>
  <div className="border-b border-primary inline-block -mt-1 pb-0">
    <Label className="text-sm font-medium">Column Spacing</Label>
    <span className="text-sm" style={{ color: '#3b82f6' }}>&nbsp;</span><span className="text-xs tex...
  </div>
  <Button
    variant="outline"
    size="sm"
    className="h-6 text-xs px-2"
    onClick={() => {
      const defaultWidths = {
        taskGap: 0,
        taskName: 48,
        courseCode: 100,
        courseName: 145,
        dueDate: 55
      };
      setTaskColumnWidths(defaultWidths);
      localStorage.setItem('taskColumnWidths', JSON.stringify(defaultWidths));

      toast({ title: "Column widths reset", description: "All summary box columns have been reset to ..." });
    }}
    data-testid="button-reset-column-widths"
  >
  <RotateCcw className="h-3 w-3 mr-1" />
  Reset Column Widths
</Button>
</div>

/* Partner Shift Schedule - moved from secondary dialog */
<div className="border rounded-lg p-3 space-y-2" style={{ marginTop: '-7px' }} data-shift-schedule-se...
  <div className="flex items-center justify-between cursor-pointer" onClick={() => { const opening = ...
    <div>
      <div className="border-b border-primary inline-block -mt-1 pb-0">
        <Label className="text-sm font-medium cursor-pointer">Partner Shift Schedule</Label>
        <span className="text-sm" style={{ color: '#3b82f6' }}>&nbsp;</span><span className="text-xs...
      </div>
    </div>
    <span className="text-xs">{shiftScheduleOpen ? '%%' : '%'}</span>
  </div>
  {shiftScheduleOpen && () => {
    const MONTHS = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];

```

```

const DAY_LABELS = ['S', 'M', 'T', 'W', 'T', 'F', 'S'];

const cycleShift = (dateStr: string) => {
  const current = localShiftMap[dateStr] || 'off';
  const next = current === 'off' ? 'day' : current === 'day' ? 'night' : 'off';
  setLocalShiftMap(prev => {
    const updated = { ...prev };
    if (next === 'off') delete updated[dateStr];
    else updated[dateStr] = next;
    return updated;
  });
  setShiftDirty(true);
};

const getShiftColor = (type: string | undefined) => {
  if (type === 'day') return { bg: '#fcfdc9', border: 'rgba(200, 200, 100, 0.9)', text: '#000' };
  if (type === 'night') return { bg: 'rgba(139, 92, 246, 0.7)', border: 'rgba(139, 92, 246, 0.9)' };
  return { bg: 'transparent', border: 'rgba(255,255,255,0.15)', text: 'rgba(255,255,255,0.4)' };
};

const getDaysInMonth = (year: number, month: number) => new Date(year, month + 1, 0).getDate();
const getFirstDayOfMonth = (year: number, month: number) => new Date(year, month, 1).getDay();

const dayCounts = { day: 0, night: 0 };
Object.entries(localShiftMap).forEach(([date, type]) => {
  if (date.startsWith(String(shiftScheduleYear)) && (type === 'day' || type === 'night')) {
    dayCounts[type]++;
  }
});

return (
  <div className="space-y-2">
    <div className="flex items-center justify-between gap-2">
      <div className="flex items-center gap-1">
        <Button variant="ghost" size="sm" className="h-6 w-6 p-0 text-white hover:bg-white/10" on...
          <span className="text-sm font-medium w-12 text-center">{shiftScheduleYear}</span>
        <Button variant="ghost" size="sm" className="h-6 w-6 p-0 text-white hover:bg-white/10" on...
      </div>
      <div className="flex items-center gap-2 text-xs">
        <span className="flex items-center gap-1"><span className="w-3 h-3 rounded-sm" style={{ b...
          <span className="flex items-center gap-1"><span className="w-3 h-3 rounded-sm" style={{ b...
        </div>
      </div>
    </div>
    <div className="grid grid-cols-4 gap-2">
      {MONTHS.map((monthName, monthIdx) => {
        const daysInMonth = getDaysInMonth(shiftScheduleYear, monthIdx);
        const firstDay = getFirstDayOfMonth(shiftScheduleYear, monthIdx);
        const cells = [];
        for (let i = 0; i < firstDay; i++) {
          cells.push(<div key={`empty-${i}`} className="w-4 h-4" />);
        }
        for (let day = 1; day <= daysInMonth; day++) {
          const dateStr = `${shiftScheduleYear}-${String(monthIdx + 1).padStart(2, '0')}-${String...
          const shift = localShiftMap[dateStr];
          const colors = getShiftColor(shift);
          cells.push(
            <button
              key={day}
              className="w-4 h-4 rounded-sm text-[7px] flex items-center justify-center cursor-po...
              style={{ background: colors.bg, border: `1px solid ${colors.border}`, color: colors...
              onClick={() => cycleShift(dateStr)}
              title={`{monthName} ${day}: ${shift || 'off'}`}
              data-testid={`shift-cell-${dateStr}`}
            >
              {day}
              {shift === 'day' && <SunIcon className="absolute -top-1 -right-1 h-3 w-3 text-orang...
              {shift === 'night' && <MoonIcon className="absolute -top-0.5 -right-0.5 h-2 w-2 tex...
            </button>
          );
        }
      })
    </div>
    <div key={monthName} className="space-y-0.5">
      <div className="text-[9px] font-medium text-center">{monthName}</div>
      <div className="grid grid-cols-7 gap-px">
        {DAY_LABELS.map((d, i) => {
          <div key={i} className="w-4 h-3 text-[6px] text-center text-muted-foreground">{d}...
        })}
        {cells}
      </div>
    </div>
  </div>
);
  </div>
</div>
<div className="flex items-center justify-between pt-1">
  <div className="flex gap-1">
    <Button
      variant="outline"
      size="sm"
      className="h-7 text-xs"
      onClick={() => {
        setLocalShiftMap(prev => {
          const updated = { ...prev };
          Object.keys(updated).forEach(k => {
            if (k.startsWith(String(shiftScheduleYear))) delete updated[k];
          });
          return updated;
        });
        setShiftDirty(true);
      }}
      data-testid="button-clear-shifts"
    >
      Clear Year
    </Button>
  </div>
  <Button
    size="sm"
    className="h-7 text-xs"
    disabled={!shiftDirty || saveShiftScheduleMutation.isPending}
  >
    Save Shift Schedule
  </Button>
</div>

```



```

        onClick={() => saveShiftScheduleMutation.mutate({ shiftMap: localShiftMap, previousData: ...
        data-testid="button-save-shifts"
      }
    }
    {saveShiftScheduleMutation.isPending ? 'Saving...' : 'Save Schedule'}
  </Button>
</div>
<div>
  <div className="text-xs text-muted-foreground space-y-0.5 pt-1 border-t border-white/10">
    <p><strong>Day shift</strong> (7:30a-7:30p): Quiet hours 10pm-5am</p>
    <p><strong>Night shift</strong> (7:30p-7:30a): Quiet hours 10am-5pm</p>
    <p className="italic">Calendar task reminders always announce, even during quiet hours.</p>
  </div>
</div>
  );
})();
</div>

/* Save/Cancel Settings Buttons */
</div>
<div className="flex justify-end gap-2" style={{ marginTop: '-3px' }}>
  <Button
    variant="outline"
    className="border !border-white/30 text-white/70 hover:text-white hover:!border-white/50 hover:bg...
    style={{ fontSize: '12px' }}
    onClick={() => {
      setColorSettings(originalColorSettings);
      setBlinkSettings(originalBlinkSettings);
      setIsSettingsDialogOpen(false);
      setTimeout(() => { document.body.style.pointerEvents = ''; }, 100);
    }}
    data-testid="button-cancel-settings"
  >
    Cancel
  </Button>
  <Button
    variant="outline"
    className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-trans...
    style={{
      boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255,25...
      fontSize: '12px'
    }}
    onClick={() => {
      pushUndo({
        type: 'settings',
        description: 'Changed settings',
        data: { oldColor: {...originalColorSettings}, newColor: {...colorSettings}, oldBlink: {...ori...
      });
      localStorage.setItem('colorSettings', JSON.stringify(colorSettings));
      localStorage.setItem('blinkSettings', JSON.stringify(blinkSettings));
      saveDegreeToServer('colorSettings', colorSettings);
      saveDegreeToServer('blinkSettings', blinkSettings);
      setOriginalColorSettings({...colorSettings});
      setOriginalBlinkSettings({...blinkSettings});
      toast({ title: "Settings saved", description: "Your settings have been applied." });
      setIsSettingsDialogOpen(false);
      setTimeout(() => { document.body.style.pointerEvents = ''; }, 100);
    }}
    data-testid="button-save-settings"
  >
    Save
  </Button>
</div>
</div>
</div>
</div>
</DialogContent>
</Dialog>

/* Calendar Settings Dialog */
<Dialog open={isCalendarSettingsOpen} onOpenChange={setIsCalendarSettingsOpen}>
  <DialogContent className={` ${shiftScheduleOpen ? 'max-w-2xl' : 'max-w-md'} text-white [&_]:text-white [&...
  <DialogHeader>
    <DialogTitle className="text-white">Calendar Settings</DialogTitle>
  </DialogHeader>
  <div className="space-y-4">
    /* Second Google Account Connection */
    <div className="border rounded-lg p-3 space-y-2">
      <Label className="text-[10px] font-medium">Second Google Account</Label>
      {secondAccountStatus?.connected ? (
        <div className="flex items-center justify-between gap-2">
          <span className="text-xs text-green-600 dark:text-green-400 font-medium">
            Connected: {secondAccountStatus.email}
          </span>
          <Button
            variant="destructive"
            size="sm"
            onClick={() => disconnectSecondAccountMutation.mutate()}
            disabled={disconnectSecondAccountMutation.isPending}
            data-testid="button-disconnect-second-account"
          >
            {disconnectSecondAccountMutation.isPending ? "Disconnecting..." : "Disconnect"}
          </Button>
        </div>
      ) : (
        <Button
          variant="outline"
          size="sm"
          onClick={async () => {
            try {
              const res = await fetch("/api/google/second-account/auth");
              const data = await res.json();
              if (data.authUrl) {
                window.open(data.authUrl, "_blank", "width=600,height=700");
              } else {
                toast({ title: "Error", description: data.error || "Failed to get auth URL", variant: "de...
              }
            } catch (err) {
              toast({ title: "Error", description: "Failed to start OAuth flow", variant: "destructive" });
            }
          }}
        </Button>
      )
    }
  </div>
</DialogContent>
</Dialog>

```

```

    }}
    data-testid="button-connect-second-account"
  >
    <Connect Second Google Account
  </Button>
  </div>
  </div>
  <div className="border rounded-lg p-3 space-y-2">
    <Label className="text-[10px] font-medium">Secondary Calendar</Label>
    <Select
      value={selectedSecondaryCalendar}
      onValueChange={(value) => {
        setSelectedSecondaryCalendar(value);
        updateSecondaryCalendarMutation.mutate(value === "none" ? "" : value);
      }}
    >
      <SelectTrigger data-testid="select-secondary-calendar">
        <SelectValue placeholder="Select a calendar..." />
      </SelectTrigger>
      <SelectContent>
        <SelectItem value="none">No secondary calendar</SelectItem>
        {availableCalendars?.filter(cal => !cal.primary).map(cal => (
          <SelectItem key={cal.id} value={cal.id}>
            {cal.summary}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
  </div>
  <div className="border rounded-lg p-3 space-y-1">
    <Label className="text-[10px] font-medium">Connection Status</Label>
    <div className="text-[10px] text-white/60 space-y-0.5">
      <p><strong>Primary Account:</strong> {availableCalendars?.find(c => c.primary)?.summary || "Not c...}</p>
      <p><strong>Secondary Calendar:</strong> {selectedSecondaryCalendar && selectedSecondaryCalendar ? ...}</p>
      <p><strong>Second Account:</strong> {secondAccountStatus?.connected ? secondAccountStatus.email : ...}</p>
    </div>
  </div>
  </div>
  <div className="border rounded-lg p-3">
    <div className="flex items-center justify-between">
      <div>
        <Label className="text-[10px] font-medium">All Day Row</Label>
      </div>
      <input
        type="checkbox"
        checked={showAllDayRow}
        onChange={(e) => {
          setShowAllDayRow(e.target.checked);
          localStorage.setItem('showAllDayRow', JSON.stringify(e.target.checked));
        }}
        className="h-4 w-4 accent-blue-500"
        data-testid="toggle-show-all-day-row"
      />
    </div>
  </div>
  </div>
  <div className="border rounded-lg p-3">
    <div className="flex items-center justify-between">
      <div>
        <Label className="text-[10px] font-medium">Tab Bounce Animation</Label>
      </div>
      <input
        type="checkbox"
        checked={tabBounceEnabled}
        onChange={(e) => {
          setTabBounceEnabled(e.target.checked);
          localStorage.setItem('tabBounceEnabled', JSON.stringify(e.target.checked));
        }}
        className="h-4 w-4 accent-blue-500"
        data-testid="toggle-tab-bounce"
      />
    </div>
  </div>
  </div>
  <div className="border rounded-lg p-3">
    <div className="flex items-center justify-between cursor-pointer" onClick={() => { const opening = ...}}>
      <div>
        <Label className="text-[10px] font-medium">Partner Shift Schedule</Label>
      </div>
      <div>
        <span>{shiftScheduleOpen ? '%4' : '%'}</span>
      </div>
    </div>
    {shiftScheduleOpen && (() => {
      const MONTHS = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
      const DAY_LABELS = ['S', 'M', 'T', 'W', 'T', 'F', 'S'];

      const cycleShift = (dateStr: string) => {
        const current = localShiftMap[dateStr] || 'off';
        const next = current === 'off' ? 'day' : current === 'day' ? 'night' : 'off';
        setLocalShiftMap(prev => {
          const updated = { ...prev };
          if (next === 'off') delete updated[dateStr];
          else updated[dateStr] = next;
          return updated;
        });
        setShiftDirty(true);
      };

      const getShiftColor = (type: string | undefined) => {
        if (type === 'day') return { bg: '#fcfdc9', border: 'rgba(200, 200, 100, 0.9)', text: '#000' };
        if (type === 'night') return { bg: 'rgba(139, 92, 246, 0.7)', border: 'rgba(139, 92, 246, 0.9)' };
        return { bg: 'transparent', border: 'rgba(255, 255, 255, 0.15)', text: 'rgba(255, 255, 255, 0.4)' };
      };

      const getDaysInMonth = (year: number, month: number) => new Date(year, month + 1, 0).getDate();
      const getFirstDayOfMonth = (year: number, month: number) => new Date(year, month, 1).getDay();

      const dayCounts = { day: 0, night: 0 };
    })}
  </div>
  </div>

```

```

Object.entries(localShiftMap).forEach(([date, type]) => {
  if (date.startsWith(String(shiftScheduleYear)) && (type === 'day' || type === 'night')) {
    dayCounts[type]++;
  }
});

return (
  <div className="space-y-2">
    <div className="flex items-center justify-between gap-2">
      <div className="flex items-center gap-1">
        <Button variant="ghost" size="sm" className="h-6 w-6 p-0 text-white hover:bg-white/10" on...
          <span className="text-sm font-medium w-12 text-center">{shiftScheduleYear}</span>
        <Button variant="ghost" size="sm" className="h-6 w-6 p-0 text-white hover:bg-white/10" on...
      </div>
      <div className="flex items-center gap-2 text-xs">
        <span className="flex items-center gap-1"><span className="w-3 h-3 rounded-sm" style={{ b...
          <span className="flex items-center gap-1"><span className="w-3 h-3 rounded-sm" style={{ b...
        </div>
      </div>
    <div className="text-xs text-muted-foreground mb-1">
      Click a day to cycle: off !' <span style={{ color: '#b8a800' }}>day</span> !' <span style={{ ...
    </div>
    <div className="grid grid-cols-4 gap-2" style={{ maxHeight: '400px', overflowY: 'auto' }}>
      {MONTHS.map((monthName, monthIdx) => {
        const daysInMonth = getDaysInMonth(shiftScheduleYear, monthIdx);
        const firstDay = getFirstDayOfMonth(shiftScheduleYear, monthIdx);
        const cells: JSX.Element[] = [];
        for (let i = 0; i < firstDay; i++) {
          cells.push(<div key={`empty-${i}`} className="w-full" style={{ aspectRatio: '1' }}/>);
        }
        for (let d = 1; d <= daysInMonth; d++) {
          const dateStr = `${shiftScheduleYear}-${String(monthIdx + 1).padStart(2, '0')}-${String...
          const shiftType = localShiftMap[dateStr];
          const colors = getShiftColor(shiftType);
          cells.push(
            <div
              key={d}
              onClick={() => cycleShift(dateStr)}
              className="w-full flex items-center justify-center cursor-pointer rounded-sm text-[...
              style={{ aspectRatio: '1', background: colors.bg, border: '1px solid ${colors.borde...
              title={` ${monthName} ${d}: ${shiftType} || 'off' }}
              data-testid={`shift-day-${dateStr}`}
            >
              {d}
              {shiftType === 'day' && <SunIcon className="absolute -top-1 -right-1 h-3 w-3 text-o...
              {shiftType === 'night' && <MoonIcon className="absolute -top-0.5 -right-0.5 h-2 w-2...
            </div>
          );
        }
      })
      return (
        <div key={monthIdx} className="space-y-0.5">
          <div className="text-[10px] font-semibold text-center" style={{ color: 'rgba(255,255,...
          <div className="grid grid-cols-7 gap-px">
            {DAY_LABELS.map((dl, i) => {
              <div key={i} className="text-[7px] text-center" style={{ color: 'rgba(255,255,255...
            })}
          <div>{cells}</div>
        </div>
      </div>
    );
  });
</div>
<div className="flex items-center justify-between pt-1">
  <div className="flex gap-1">
    <Button
      variant="outline"
      size="sm"
      className="h-7 text-xs"
      onClick={() => {
        setLocalShiftMap(prev => {
          const updated = { ...prev };
          Object.keys(updated).forEach(k => {
            if (k.startsWith(String(shiftScheduleYear))) delete updated[k];
          });
          return updated;
        });
        setShiftDirty(true);
      }}
      data-testid="button-clear-shifts"
    >
      Clear Year
    </Button>
  </div>
  <Button
    size="sm"
    className="h-7 text-xs"
    disabled={!shiftDirty || saveShiftScheduleMutation.isPending}
    onClick={() => saveShiftScheduleMutation.mutate({ shiftMap: localShiftMap, previousData: ...
      data-testid="button-save-shifts"
    >
      {saveShiftScheduleMutation.isPending ? 'Saving...' : 'Save Schedule'}
    </Button>
  </div>
  <div className="text-xs text-muted-foreground space-y-0.5 pt-1 border-t border-white/10">
    <p><strong>Day shift</strong> (7:30a-7:30p): Quiet hours 10pm-5am</p>
    <p><strong>Night shift</strong> (7:30p-7:30a): Quiet hours 10am-5pm</p>
    <p className="italic">Calendar task reminders always announce, even during quiet hours.</p>
  </div>
</div>
);
})();
</div>
</DialogContent>
</Dialog>

<div className="flex-1 overflow-visible scrollbar-hidden flex flex-col" style={{ marginTop: '0px', marginLeft...

```

```

    { /* Calendar Views */
    {calendarView === "week" ? (
    <div className="mb-[0px] mt-[0px] relative flex gap-4 transition-opacity duration-300" style={{ height: calen...

    { /* Module Media Controls Dialog */
    <Dialog open={moduleMediaControlCourse !== null} onOpenChange={(open) => !open && setModuleMediaControlCour...
    <DialogContent className="max-w-[320px] p-4 text-white [&_]:text-white">
    <DialogHeader>
    <DialogTitle className="text-sm font-medium">
    {moduleMediaControlCourse === 'cpa122' ? 'CPA122' : moduleMediaControlCourse === 'cfnf400' ? 'CFN...
    </DialogTitle>
    </DialogHeader>
    <div className="flex flex-col gap-3 mt-2">
    <div className="flex items-center justify-between bg-white/10 rounded-lg p-3">
    <span className="text-xs">Week {selectedWeek} Module</span>
    <div className="flex items-center gap-2">
    <Button variant="ghost" size="icon" className="h-8 w-8 hover:bg-white/20">
    <SkipBack className="h-4 w-4" />
    </Button>
    <Button variant="ghost" size="icon" className="h-8 w-8 hover:bg-white/20">
    <Play className="h-4 w-4" />
    </Button>
    <Button variant="ghost" size="icon" className="h-8 w-8 hover:bg-white/20">
    <SkipForward className="h-4 w-4" />
    </Button>
    </div>
    </div>
    <div className="flex items-center gap-2">
    <Volume2 className="h-4 w-4 text-white/70" />
    <Slider defaultValue={[50]} max={100} step={1} className="flex-1" />
    </div>
    </div>
    </DialogContent>
    </Dialog>

    { /* Calendar wrapper - leaves space for honeycombs on right */
    <div ref={calendarWrapperRef} style={{ width: `calc(100% - 60px${calendarReduction > 0 ? ` - ${calendarRedu...

    { /* Glass effect backing box - resizes with calendar */
    <div
    className="absolute pointer-events-none"
    style={{
    top: '-4px',
    left: '-15px',
    right: `${-(calendarReduction - 3) - 15 + 6 + 6 + 2}px`,
    bottom: '-16px',
    background: 'linear-gradient(180deg, rgba(255,255,255,0.18) 0%, rgba(255,255,255,0.06) 100%)',
    borderRadius: '12px',
    border: '1px solid rgba(255,255,255,0.5)',
    borderTop: '1px solid rgba(255,255,255,0.7)',
    borderBottom: '1px solid rgba(255,255,255,0.15)',
    boxShadow: '0 4px 24px rgba(0,0,0,0.15), inset 0 1px 0 rgba(255,255,255,0.25), inset 0 -1px 0 rgba(0,0,...
    }}
    />

    { /* Month View button - below glass backing, left-aligned */
    <Button
    variant="ghost"
    className="h-5 !min-h-0 px-2 text-[11px] hover:bg-white/20 rounded font-medium text-white/60 border-0 le...
    data-tpo data-tpo-opacity="1"
    style={{ bottom: `${calendarBottom - 18}px`, left: `${calendarLeft + 9}px`, zIndex: 60, display: (isSetti...
    onClick={() => {
    if (calendarView === "week") {
    setCurrentMonth(new Date());
    }
    setCalendarView(calendarView === "month" ? "week" : "month");
    }}
    data-testid="button-month-view"
    >
    {calendarView === "month" ? "Week View" : "Month View"}
    </Button>
    <div
    className="fixed"
    data-tpo data-tpo-opacity="1"
    style={{ bottom: `${calendarBottom - 18}px`, left: `${calendarLeft + (calendarView === "month" ? 78 : 86)...
    >
    <div style={{ width: '1px', height: '12px', background: 'rgba(255,255,255,0.5)', marginLeft: '2px', margi...
    <Button
    variant="ghost"
    className="h-5 !min-h-0 px-2 text-[11px] hover:bg-white/20 rounded font-medium text-white/60 border-0 ...
    onClick={() => {
    if (calendarView === "month") {
    setCurrentMonth(new Date());
    } else {
    const cw = semesterSettings?.semesterStartDate ? getWeekNumber(new Date(), new Date(semesterSetting...
    setSelectedWeek(cw);
    scrollHomeworkToWeek(cw);
    }
    }}
    data-testid="button-current-view"
    >
    Current
    </Button>
    </div>

    {weatherData?.daily && (
    <div className="grid w-full flex-shrink-0" style={{ gridTemplateColumns: getGridTemplateColumns(), height...
    <div style={{ minWidth: 0 }} />
    {gridSizes.moduleColumnWidth > 0 && <div style={{ minWidth: 0 }} />
    {weekDays.map((day, idx) => {
    const dateStr = format(day, 'yyyy-MM-dd');
    const dayForecast = weatherData.daily?.find(d => d.date === dateStr);
    const isPast = day < startOfDayET(new Date());
    const WMO_SHORT: Record<number, string> = { 0: 'Clear', 1: 'Clear', 2: 'Cloudy', 3: 'Overcast', 45: 'Fog', 48: '...
    const wIcon = ((wc: number | undefined) => {
    if (wc === undefined) return '';
    if (wc === 0) return '&#x2600'; if (wc === 1) return '&#x2600'; if (wc === 2) return '&#x2600'; if (wc === 3) retu...

```



```

        if (visibleStops.length === 0) visibleStops.push(skyStops[0]);
        const lastVisible = visibleStops[visibleStops.length - 1];
        const scaledStops = visibleStops.map(s => `${s.c} ${pct > 0 ? (s.p / pct * pct).toFixed(1) : 0}...
        scaledStops.push( `${lastVisible.c} ${pct}%`);
        scaledStops.push( `#ffffff ${pct}%`);
        scaledStops.push( `#ffffff 100%`);
        return { background: `linear-gradient(to right, ${scaledStops.join(', ')});` };
    })() : { backgroundColor: colorSettings.headerBar }}
    data-testid={ `day-header-${format(day, 'yyyy-MM-dd')}` }`
  >
  {!isSameDayET(day, subDays(new Date(), 1)) && shiftForDay && (
    <div
      className="absolute inset-0 cursor-pointer z-10"
      style={{ opacity: sleepDisabledDays.has(shiftDateStr) ? 0.3 : 0 }}
      onClick={(e) => { e.stopPropagation(); updateSleepDisabledDays(shiftDateStr); }}
      data-testid={ `toggle-sleep-${shiftDateStr}` }
    />
  )}
  {!isToday && !isSameDayET(day, subDays(new Date(), 1)) && shiftForDay && (shiftForDay === 'day' |...
    <div className="absolute bottom-0 left-0 right-0" style={{ height: '5px', backgroundColor: shif...
  )}
  {(() => {
    return (
      <>
        {isToday && (
          <div className="absolute top-0 left-0 right-0 text-center z-20" style={{ padding: '0' }} ...
            <span style={{ display: 'block', fontSize: '8px', fontWeight: 700, color: '#ffffff', ba...
              {format(day, 'EEEE, MMMM d')}
            </span>
          </div>
        )}
        {day.getDay() === 6 && !isToday && (
          <div className="absolute top-0 left-0 right-0 text-center" style={{ fontSize: '7.5px', fo...
        )}
        {!isToday && (() => {
          const now = new Date();
          const todayIdx = weekDays.findIndex(d => isSameDayET(d, now));
          return todayIdx >= 0 && idx < todayIdx && day.getDay() !== 6;
        })() && (
          <div className="absolute top-0 left-0 right-0 text-center" style={{ fontSize: '6.5px', fo...
        )}
        <div className="flex items-center gap-1.5">
          <div className="text-[10px] font-medium tracking-wide" style={{ color: isToday ? '#fff' : ...
            <div style={{ fontSize: isToday ? '25px' : '24px', fontWeight: isToday ? 600 : 700, color...
          </div>
        </div>
      </>
    );
  })()}
  {idx < 6 && (
    <div
      className="absolute right-0 top-0 bottom-0 w-[2px] cursor-col-resize bg-white/20 hover:bg-whi...
      style={{ zIndex: 9999 }}
      onMouseDown={(e) => { e.stopPropagation(); handleColumnResizeStart(e, idx); }}
      onTouchStart={(e) => { e.stopPropagation(); handleColumnResizeStart(e, idx); }}
      data-testid={ `day-column-resize-handle-${idx}` }
    />
  )}
</div>
);
}}
</div>

```

```

/* Course Rows - Dynamic based on selected week */
/* Pre-compute course row height: minimum = 3 tasks height, expand only if any course has >3 tasks */
{(() => {
  const springSummerCourses = [
    { name: 'CECN210 - Understanding Economics', color: '#059669', colorEnd: '#34D399', professor: '', ...
    { name: 'CPHL110 - Philosophy of Religion', color: '#2563EB', colorEnd: '#60A5FA', professor: '', p...
    { name: 'CHIS105 - Inventing Popular Culture', color: '#DC2626', colorEnd: '#F87171', professor: ''...
  ];
  const currentWeekStart = weekDays[0];
  const currentWeekEnd = weekDays[6];
  const winterCourseDefs = [
    { code: 'CPA122', endDate: '2026-04-17' },
    { code: 'CFNF400', endDate: '2026-04-17' },
    { code: 'CASL101', endDate: '2026-04-17' },
  ];
  const winterCourses = coursesData.courses.filter(c => {
    if (!c.name) return false;
    const code = c.name.split(' - ')[0]?.trim().toUpperCase();
    const def = winterCourseDefs.find(d => code === d.code);
    if (!def) return false;
    const courseEnd = new Date(def.endDate + 'T23:59:59');
    return currentWeekStart <= courseEnd;
  });
  const springSummerSemesterStart = new Date('2026-05-04T00:00:00');
  const springSummerSemesterEnd = new Date('2026-08-14T23:59:59');
  const activeSpringSummerCourses = (currentWeekEnd >= springSummerSemesterStart && currentWeekStart <=...
    ? springSummerCourses
    : [];
  const allDisplayCourses = [...winterCourses];
  const defaultWinterCourses = [
    { name: 'CPA122 - Local Politics and Government', color: '#0F5004', colorEnd: '#47B045', professor...
    { name: 'CFNF400 - Human Sexuality', color: '#DE1864', colorEnd: '#FA67B3', professor: 'Alex McKay'...
    { name: 'CASL101 - ASL', color: '#974B8A', colorEnd: '#B045A2', professor: 'Christina Moreau', prof...
  ];
  for (const dwc of defaultWinterCourses) {
    const dwcCode = dwc.name.split(' - ')[0]?.trim().toUpperCase();
    const dwcDef = winterCourseDefs.find(d => d.code === dwcCode);
    if (dwcDef && currentWeekStart <= new Date(dwcDef.endDate + 'T23:59:59')) {
      const alreadyHas = allDisplayCourses.some(c => c.name.split(' - ')[0]?.trim().toUpperCase() === d...
      if (!alreadyHas) allDisplayCourses.push(dwc);
    }
  }
  for (const sc of activeSpringSummerCourses) {
    const alreadyExists = allDisplayCourses.some(c => c.name.split(' - ')[0]?.trim().toUpperCase() === ...

```

```

    if (!alreadyExists) allDisplayCourses.push(sc);
  }
  const filteredCourses = allDisplayCourses;
  const maxCourseRowHeight = 3 * 20 + 2 * 2 + 4;
  return (
    <div ref={courseRowsRef} data-testid="course-rows-container" style={{ borderTop: '1px solid black', mar...
    {filteredCourses.map((courseData, courseIdx) => {
      const courseName = courseData.name.split(' - ')[0].toUpperCase();
      const rgb = hexToRgb(courseData.color);
      const course = {
        name: courseName,
        bg: () => {
          if ((courseData as any).courseRowColor) return (courseData as any).courseRowColor;
          const end = courseData.colorEnd ? hexToRgb(courseData.colorEnd) : rgb;
          return `rgb(${end.r}, ${end.g}, ${end.b})`;
        }(),
        label: () => {
          const startColor = courseData.color || `rgb(${Math.max(0, rgb.r - 40)}, ${Math.max(0, rgb.g - 40)}...
          const endColor = courseData.colorEnd || `rgb(${Math.min(255, rgb.r + 100)}, ${Math.min(255, rgb.g...
          if (courseData.colorStops) {
            try {
              const stops: Array<{position: number; color: string}> = JSON.parse(courseData.colorStops);
              if (stops.length > 0) {
                const allParts = [`${startColor} 0%`, ...stops.map(s => `${s.color} ${s.position}%`), `${en...
                return `linear-gradient(180deg, ${allParts.join(', ')});`;
              }
            } catch {}
          }
          return `linear-gradient(180deg, ${startColor} 0%, ${endColor} 100%)`;
        }(),
        darkColor: courseData.borderColor || courseData.color || () => {
          const dR = Math.max(0, rgb.r - 40), dG = Math.max(0, rgb.g - 40), dB = Math.max(0, rgb.b - 40);
          return `rgb(${dR}, ${dG}, ${dB})`;
        }(),
        colors: dynamicCourseColors[courseName]
      };
      // Get full-week tasks for this course (tasks that span from visible start to Friday)
      // Exclude completed tasks so they are removed from view
      const fullWeekTasks = weekPlanningTasks.filter(task => {
        if (!task.startDate || !task.courseName?.startsWith(course.name)) return false;
        if (task.isCompleted) return false; // Remove completed module tasks from view

        // Use format to compare dates consistently (avoiding timezone issues)
        const taskStartDateStr = format(new Date(task.startDate), 'yyyy-MM-dd');
        const taskDueDateStr = format(new Date(task.dueDate), 'yyyy-MM-dd');

        // weekDays structure is always: [Sun, Mon, Tue, Wed, Thu, Fri, Sat]
        // Sunday is index 0, Friday is index 5
        const weekStartStr = format(weekDays[0], 'yyyy-MM-dd');
        const weekFridayStr = format(weekDays[5], 'yyyy-MM-dd');

        // Check if task spans the full visible week (from Sunday to Friday)
        const startsOnSunday = taskStartDateStr === weekStartStr;
        const endsOnFriday = taskDueDateStr === weekFridayStr;

        return startsOnSunday && endsOnFriday;
      });

      // Calculate position for full-week tasks
      const todayDate = new Date();
      const isSaturdayToday = todayDate.getDay() === 6;
      // Get Saturday column width (Saturday is at index 6 when not Saturday, index 6 when Saturday)
      // When not Saturday: weekDays = [Sun, Mon, Tue, Wed, Thu, Fri, Sat] - Sat at end
      // When Saturday: weekDays = [Sun, Mon, Tue, Wed, Thu, Fri, Sat] - same order
      const saturdayColumnWidth = gridSizes.dayColumnWidths[6] || 100;

      // Check if this course has any full-week tasks
      const hasFullWeekTasks = fullWeekTasks.length > 0;

      // If there are full-week tasks, render them using grid column spanning
      if (hasFullWeekTasks) {
        return (
          <div key={course.name} className="w-full flex-shrink-0 flex" style={{ borderBottom: '1.5px dotted...
          <div className="px-1 py-0.5 text-[10px] font-bold tracking-wide flex items-center justify-cen...
            {course.name}
            <div style={{ position: 'absolute', top: '1px', right: '1px', zIndex: 2 }} onClick={{e}} => { ...
          </div>
          <div className="flex-1 min-w-0">
            {fullWeekTasks.map((task, taskIdx) => {
              const today = startOfDayET(new Date());
              const tomorrow = addDays(today, 1);
              const taskDueDate = startOfDayET(new Date(task.dueDate));
              const isDueToday = !task.isCompleted && isSameDayET(taskDueDate, today);
              const isDueTomorrow = !task.isCompleted && isSameDayET(taskDueDate, tomorrow);

              const currentDayOfWeek = today.getDay();
              const isWednesdayOrLater = currentDayOfWeek >= 3 && currentDayOfWeek <= 5;
              const shouldBlink = !task.isCompleted && isWednesdayOrLater;

              const remainingColumns = getGridTemplateColumns().split(' ').slice(1).join(' ');
              return (
                <div
                  key={`fullweek-row-${task.id}`}
                  className="grid w-full"
                  style={{
                    gridTemplateColumns: remainingColumns,
                    minHeight: `${gridSizes.courseRowHeight}px`
                  }}
                >
                  /* Static MODULE column task - only when module column is visible */
                  {gridSizes.moduleColumnWidth > 0 && {
                    <div style={{ backgroundColor: course.bg }}>
                      <div
                        className={`flex items-center gap-1 text-[8px] px-1 py-0.5 rounded m-0.5 border ${tas...
                        style={{
                          backgroundColor: task.isCompleted ? '#e5e7eb' : () => { const cMatch = coursesData...
                          borderColor: task.isCompleted ? '#d1d5db' : course.darkColor

```



```

        </span>
      </div>
    </div>
  );
}

// Continuation bar for days after today - no left margin to connect with previous
const contBorderClass = isFriday
  ? 'border-t border-b border-r ${borderColor} rounded-r'
  : 'border-t border-b ${borderColor}';
return (
  <div key={dayIdx} className="flex items-center" style={{ backgroundColor: cellBg }}>
    <div
      className={`w-full ${bgOnly} ${contBorderClass} ${
        shouldBlink ? "animate-blink" : ""
      } ${isFriday ? 'mr-0.5' : ''}`}
      style={{ height: 'calc(100% - 4px)', marginTop: '2px', marginBottom: '2px' }}
    </div>
  </div>
);
  }}
</div>
);
  }}
</div>
);
}

// Calculate how many tasks this course has in the current week to set dynamic height
const courseTaskCount = tasks?.filter(task => {
  const courseCode = course.name.split(' - ')[0]?.trim().toUpperCase() || '';
  if (!courseCode || !task.courseName?.toUpperCase().startsWith(courseCode)) return false;
  if (task.isCompleted) return false;
  if (task.type === 'other') return false;
  if (task.hideFromSummary) return false;
  const taskDueDate = startOfDayET(new Date(task.dueDate));
  const weekStart = startOfDayET(weekDays[0]);
  const weekEnd = startOfDayET(addDays(weekDays[6], 1));
  if (taskDueDate >= weekStart && taskDueDate < weekEnd) return true;
  if (task.startDate) {
    const taskStartDate = startOfDayET(new Date(task.startDate));
    if (taskStartDate < weekEnd && taskDueDate > weekStart) return true;
  }
  return false;
}).length || 0;

// Skip rendering course row if it has no tasks or projects this week
const courseHasProjects = allProjects.some(proj => {
  if (!proj.courseName) return false;
  const projCourseCode = proj.courseName.split(' - ')[0]?.toUpperCase();
  if (projCourseCode !== courseName) return false;
  if (proj.status === 'completed' || proj.status === 'cancelled') return false;
  if (!proj.targetDate) return false;
  const projTargetDate = startOfDayET(new Date(proj.targetDate));
  const weekStart = startOfDayET(weekDays[0]);
  const weekEnd = startOfDayET(addDays(weekDays[6], 1));
  if (projTargetDate >= weekStart && projTargetDate < weekEnd) return true;
  if (proj.startDate) {
    const projStartDate = startOfDayET(new Date(proj.startDate));
    if (projStartDate < weekEnd && projTargetDate > weekStart) return true;
  }
  return false;
});
const mainCourseNames = ['CPPA122', 'CFNF400', 'CASL101'];
const isMainCourse = mainCourseNames.includes(courseName);
if (courseTaskCount === 0 && !courseHasProjects && !isMainCourse) {
  return null;
}

return (
  <div key={course.name} ref={el => { courseRowRefs.current[courseIdx] = el; }} className="grid w-full ...
    <div className="px-1 py-0.5 text-[8px] font-medium tracking-wide flex flex-col items-center justify...
      {(() => {
        const code = course.name.split(' - ')[0];
        const fullName = course.name.split(' - ').slice(1).join(' - ');
        if (code === 'CPPA122') {
          return <span className="text-center"><span className="font-[785]">CPPA122</span> Local Politi...
        }
        if (code === 'CFNF400') {
          return (
            <
              <span className="font-[785]">CFNF400</span>
              <span>Human</span>
              <span>Sexuality</span>
            </>
          );
        }
        if (code === 'CASL101') {
          return (
            <
              <span className="text-center font-[785]">CASL101</span>
              <span className="text-center">American Sign</span>
              <span className="text-center">Language</span>
            </>
          );
        }
      })()
      const words = fullName.split(' ');
      return (
        <
          <span className="font-[785]">{code}</span>
          {words.map((word, i) => <span key={i}>{word}</span>)}
        </>
      );
    })()
  <div style={{ position: 'absolute', top: '1px', right: '1px', zIndex: 2 }} onClick={(e) => { e.st...
</div>

```

```

{gridSizes.moduleColumnWidth > 0 && <div style={{ minWidth: 0, backgroundColor: course.bg, borderBo...
{() => {
  const cwWeekStart = startOfDayET(weekDays[0]);
  const cwWeekEnd = startOfDayET(addDays(weekDays[5], 1));
  const courseCodeUpper = course.name.split(' - ')[0]?.trim().toUpperCase() || '';
  const courseWeekTasks = (allTasks?.filter(task => {
    if (!courseCodeUpper || !task.courseName?.toUpperCase().startsWith(courseCodeUpper)) return false;
    if (task.isCompleted) return false;
    if (task.type === 'other') return false;
    if (task.hideFromSummary) return false;
    const taskDueDate = startOfDayET(new Date(task.dueDate));
    if (taskDueDate >= cwWeekStart && taskDueDate < cwWeekEnd) return true;
    if (task.startDate) {
      const taskStartDate = startOfDayET(new Date(task.startDate));
      if (taskStartDate < cwWeekEnd && taskDueDate >= cwWeekStart) return true;
    }
  }) || []).sort((a, b) => new Date(a.dueDate).getTime() - new Date(b.dueDate).getTime());
  const taskSlotMap = new Map<number, number>();
  const recurringGroups = new Map<string, typeof courseWeekTasks>();
  courseWeekTasks.forEach(t => {
    const key = t.title;
    if (!recurringGroups.has(key)) recurringGroups.set(key, []);
    recurringGroups.get(key)!.push(t);
  });
  const taskIntervals: { id: number; start: number; end: number; span: number; groupIds?: number[] ...
  const groupedTaskIds = new Set<number>();
  recurringGroups.forEach((tasks, title) => {
    if (tasks.length >= 2 && tasks.every(t => !t.startDate || isSameDayET(startOfDayET(new Date(t.s...
      const days = tasks.map(t => Math.max(0, Math.floor((startOfDayET(new Date(t.dueDate)).getTime...
      const minDay = Math.min(...days);
      const maxDay = Math.max(...days);
      taskIntervals.push({ id: tasks[0].id, start: minDay, end: maxDay, span: maxDay - minDay, grou...
      tasks.forEach(t => groupedTaskIds.add(t.id));
    });
  });
  courseWeekTasks.forEach(t => {
    if (groupedTaskIds.has(t.id)) return;
    const dueDay = Math.floor((startOfDayET(new Date(t.dueDate)).getTime() - cwWeekStart.getTime())...
    const startDate = t.startDate
      ? Math.floor((startOfDayET(new Date(t.startDate)).getTime() - cwWeekStart.getTime()) / (1000 ...
      : dueDay;
    taskIntervals.push({ id: t.id, start: Math.max(0, startDate), end: Math.max(0, dueDay), span: Ma...
  });
  taskIntervals.sort((a, b) => b.span - a.span || a.start - b.start);
  const slotOccupancy: Set<number>[] = [];
  for (const interval of taskIntervals) {
    let assignedSlot = -1;
    for (let s = 0; s < slotOccupancy.length; s++) {
      let conflict = false;
      for (let d = interval.start; d <= interval.end; d++) {
        if (slotOccupancy[s].has(d)) { conflict = true; break; }
      }
      if (!conflict) { assignedSlot = s; break; }
    }
    if (assignedSlot === -1) {
      assignedSlot = slotOccupancy.length;
      slotOccupancy.push(new Set());
    }
    for (let d = interval.start; d <= interval.end; d++) {
      slotOccupancy[assignedSlot].add(d);
    }
    taskSlotMap.set(interval.id, assignedSlot);
    if (interval.groupIds) {
      interval.groupIds.forEach(gid => taskSlotMap.set(gid, assignedSlot));
    }
  }
  const totalSlots = slotOccupancy.length || courseWeekTasks.length;

  const recurringSpans = new Map<number, { firstDayIdx: number; lastDayIdx: number; title: string }...
  const titleSlotGroups = new Map<string, number[]>();
  courseWeekTasks.forEach(t => {
    const slot = taskSlotMap.get(t.id);
    if (slot === undefined) return;
    const key = `${t.title}::${slot}`;
    if (!titleSlotGroups.has(key)) titleSlotGroups.set(key, []);
    titleSlotGroups.get(key)!.push(t.id);
  });
  titleSlotGroups.forEach((taskIds, key) => {
    if (taskIds.length < 2) return;
    const tasks = taskIds.map(id => courseWeekTasks.find(t => t.id === id)!).filter(Boolean);
    const dayIndices = tasks.map(t => {
      const dueDay = startOfDayET(new Date(t.dueDate));
      return weekDays.findIndex(wd => isSameDayET(wd, dueDay));
    }).filter(d => d >= 0).sort((a, b) => a - b);
    if (dayIndices.length < 2) return;
    const firstDay = dayIndices[0];
    const lastDay = dayIndices[dayIndices.length - 1];
    tasks.forEach(t => {
      recurringSpans.set(t.id, { firstDayIdx: firstDay, lastDayIdx: lastDay, title: t.title });
    });
  });
});

return weekDays.map((day, dayIdx) => {
  const isDayToday = isSameDayET(day, new Date());
  const isDayAfterToday = day > new Date() && !isDayToday;
  const isDayBeforeToday = !isDayToday && day < new Date();
  const cellBgColor = isDayToday ? '#e4ecf5' : isDayBeforeToday ? dimColor(course.bg) : course.bg;
  const cellDate = startOfDayET(day);

  const dueTasks = allTasks?.filter(task => {
    if (!courseCodeUpper || !task.courseName?.toUpperCase().startsWith(courseCodeUpper)) return false;
    if (task.isCompleted) return false;
    if (task.type === 'other') return false;
    const taskDueDate = startOfDayET(new Date(task.dueDate));
    return isSameDayET(taskDueDate, cellDate);
  }) || [];

```

```

const prepTasks = allTasks?.filter(task => {
  if (!courseCodeUpper || !task.courseName?.toUpperCase().startsWith(courseCodeUpper)) return false;
  if (task.isCompleted) return false;
  if (task.type === 'other') return false;
  if (!task.startDate) return false;
  if (!task.prepDays || task.prepDays <= 0) return false;
  const taskDueDate = startOfDayET(new Date(task.dueDate));
  const taskStartDate = startOfDayET(new Date(task.startDate));
  return cellDate >= taskStartDate && cellDate < taskDueDate;
}) || [];

const rgb = hexToRgb(course.label);
const borderColor = `rgba(${rgb.r}, ${rgb.g}, ${rgb.b}, 0.5)`;

const slottedItems: ({ task: typeof dueTasks[0], isPrep: boolean } | null)[] = Array(totalSlots)....
dueTasks.forEach(t => { const slot = taskSlotMap.get(t.id); if (slot !== undefined) slottedItems[...
prepTasks.forEach(t => { const slot = taskSlotMap.get(t.id); if (slot !== undefined && slottedIte...

const cellBgActual = cellBgColor;
const cellTaskCount = slottedItems.filter(s => s !== null).length;
const cellHasScroll = cellTaskCount > 3;

return (
  <div
    key={dayIdx}
    className="relative pt-0.5"
    style={{ backgroundColor: cellBgColor, padding: '2px 1px 2px 1px', borderBottom: isDayToday ?...
    data-testid={`course-row-${course.name}-${format(day, "yyyy-MM-dd")}`}
    onDragOver={(e) => {
      e.preventDefault();
      e.currentTarget.style.backgroundColor = '#8B8070';
    }}
    onDragLeave={(e) => {
      e.currentTarget.style.backgroundColor = cellBgColor;
    }}
    onDrop={(e) => {
      e.currentTarget.style.backgroundColor = cellBgColor;
      handleCourseRowDrop(e, course.name, day);
    }}
  >
    <div style={{ position: 'absolute', left: 0, top: 0, bottom: 0, width: '0px', borderLeft: '1p...
    {isDayToday && (() => {
      const cCode2 = course.name;
      const nextTask = (allTasks || []).filter(t => {
        const tc = t.courseName?.split(' ')[0]?.toUpperCase() || '';
        return tc === cCode2 && !t.isCompleted && new Date(t.dueDate) >= startOfDayET(new Date());
      }).sort((a, b) => new Date(a.dueDate).getTime() - new Date(b.dueDate).getTime())[0];
      if (!nextTask) return null;
      const daysUntil = differenceInCalendarDays(new Date(nextTask.dueDate), new Date());
      const badgeGrad = getCourseGradientColors(cCode2);
      const badgeStartRgb = hexToRgb(badgeGrad.start);
      const badgeEndRgb = hexToRgb(badgeGrad.end);
      const badgeBg = `rgb(${Math.round((badgeStartRgb.r + badgeEndRgb.r) / 2)}, ${Math.round((ba...
      return (
        <div style={{ position: 'absolute', top: 0, left: 0, right: 0, height: '15px', background...
        <span style={{ fontSize: '9px', color: '#ffffff', whiteSpace: 'nowrap', fontWeight: 500...
      </div>
    );
  })());
  {isDayAfterToday && <div style={{ height: '15px', flexShrink: 0 }} />}
  <div className={`flex flex-col gap-0.5${cellHasScroll ? ' course-cell-scroll' : ''}`} style={...
  /* Course-associated projects */
  {allProjects.filter(proj => {
    if (!proj.courseName) return false;
    const projCourseCode = proj.courseName.split(' - ')[0]?.toUpperCase();
    if (projCourseCode !== course.name) return false;
    if (proj.status === 'completed' || proj.status === 'cancelled') return false;
    if (!proj.targetDate) return false;
    const projTargetDate = startOfDayET(new Date(proj.targetDate));
    if (isSameDayET(projTargetDate, cellDate)) return true;
    if (proj.startDate) {
      const projStartDate = startOfDayET(new Date(proj.startDate));
      return cellDate >= projStartDate && cellDate < projTargetDate;
    }
    return false;
  }).map(proj => {
    const today = startOfDayET(new Date());
    const tomorrow = addDays(today, 1);
    const projTarget = startOfDayET(new Date(proj.targetDate!));
    const isDueToday = isSameDayET(projTarget, today);
    const isDueTomorrow = isSameDayET(projTarget, tomorrow);
    const isOnTargetDay = isSameDayET(projTarget, cellDate);
    return (
      <div
        key={`proj-${proj.id}`}
        className={`flex items-center gap-0.5 text-[9px] pl-1 pr-0.5 py-0.5 rounded border curs...
        style={{
          backgroundColor: isOnTargetDay ? 'white' : 'white',
          borderColor: proj.color || course.darkColor,
          borderStyle: 'dashed',
        }}
        onClick={() => { setEditingProject(proj); setProjectDialogOpen(true); }}
        title={proj.name}
        data-testid={`course-project-${proj.id}`}
      >
        <FolderKanban className="h-3 w-3 shrink-0" style={{ color: proj.color || course.darkCol...
        <span className="truncate text-gray-700 pl-0.5 flex-1 min-w-0">{proj.name}</span>
      </div>
    );
  });
  {}
  {(() => {
    const firstFilledIdx = slottedItems.findIndex(item => item !== null);
    const lastFilledIdx = (() => { for (let i = slottedItems.length - 1; i >= 0; i--) { if (slo...
    return slottedItems.map((item, itemIdx) => {
      if (!item) {
        if (itemIdx < firstFilledIdx || itemIdx > lastFilledIdx) return null;
        return <div key={`empty-${itemIdx}`} style={{ height: '2px' }} />;
      }
    }
  })
}

```

```

const task = item.task;
const today = startOfDayET(new Date());
const tomorrow = addDays(today, 1);
const isDueToday = !task.isCompleted && isSameDayET(new Date(task.dueDate), today);
const isDueTomorrow = !task.isCompleted && isSameDayET(new Date(task.dueDate), tomorrow);

if (item.isPrep) {
  const prepStartDate = startOfDayET(new Date(task.startDate));
  const prepDueDate = startOfDayET(new Date(task.dueDate));
  const cellDate = startOfDayET(day);
  if (cellDate < today) return null;
  const isFirstPrepDay = isSameDayET(cellDate, prepStartDate);
  const isLastPrepDay = isSameDayET(addDays(cellDate, 1), prepDueDate);
  const taskCourseCode = task.courseName?.split(' - ')[0]?.toUpperCase() || '';
  const taskCourseCodeLower = taskCourseCode.toLowerCase();
  const moduleFolderName = `week-${task.weekNumber}-${taskCourseCodeLower}-module`;
  const readingFolderName = `week-${task.weekNumber}-${taskCourseCodeLower}-reading`;
  const moduleFile = weeklyFiles.find(f => f.folder === moduleFolderName && f.objectPath) || ...
  const moduleAttachmentPdf = !moduleFile && task.attachments?.length ? (() => { for (const ...
  const moduleRefLinkPdf = !moduleFile && !moduleAttachmentPdf && task.referenceLink ? task...
  const modulePdfUrl = moduleFile?.objectPath || moduleAttachmentPdf || moduleRefLinkPdf || ...
  return (
    <div
      key={`prep-${task.id}`}
      className="relative w-full"
    >
      {!isFirstPrepDay && (
        <div style={{ position: 'absolute', left: '-5px', top: '50%', transform: 'translate...
      )}
      {isLastPrepDay && (
        <div style={{ position: 'absolute', right: '-3px', top: '50%', transform: 'translat...
      )}
      <div
        className="flex items-center text-[9px] rounded border cursor-pointer relative w-full"
        style={{
          backgroundColor: (() => { const cMatch = coursesData.courses.find(c => c.name?.sp...
          borderColor: course.darkColor,
          zIndex: hoveredCountdownTaskId === task.id ? 55 : 1,
          transform: hoveredCountdownTaskId === task.id ? 'scale(1.12)' : undefined,
          boxShadow: hoveredCountdownTaskId === task.id ? '0 4px 16px rgba(0,0,0,0.25)' : u...
          transition: 'transform 0.2s ease, box-shadow 0.2s ease',
          overflow: 'hidden',
          position: 'relative',
        }}
        onClick={() => setEditingTask(task)}
        title={`Prep Day - ${task.title}`}
      >
        {(() => { const totalPrepDays = differenceInCalendarDays(prepDueDate, prepStartDate...
        <span className="truncate pl-[3px] py-0.5 flex-1 min-w-0" style={{ fontSize: '9px',...
      </div>
    </div>
  );
}

const dueTaskCourseCode = task.courseName?.split(' - ')[0]?.toUpperCase() || '';
const dueTaskCourseCodeLower = dueTaskCourseCode.toLowerCase();
const dueModuleFolderName = `week-${task.weekNumber}-${dueTaskCourseCodeLower}-module`;
const dueReadingFolderName = `week-${task.weekNumber}-${dueTaskCourseCodeLower}-reading`;
const dueModuleFile = (task.type === 'discussion' || task.type === 'module' || task.type === ...
const dueAttachmentUrl = task.attachments?.length ? (() => { for (const att of task.attachm...
const dueAttachmentLink = !dueAttachmentUrl && task.attachments?.length ? (() => { for (con...
const dueRefLinkPdf = !dueAttachmentUrl && task.referenceLink ? task.referenceLink : null;
const dueModulePdfUrl = dueAttachmentUrl || dueRefLinkPdf || dueModuleFile?.objectPath || n...
const hasPrepDays = task.startDate && !isSameDayET(startOfDayET(new Date(task.startDate)), ...
const spanInfo = recurringSpans.get(task.id);
const isSpanFirst = spanInfo && dayIdx === spanInfo.firstDayIdx;
const isSpanLast = spanInfo && dayIdx === spanInfo.lastDayIdx;
const isSpanMiddle = spanInfo && dayIdx > spanInfo.firstDayIdx && dayIdx < spanInfo.lastDay...
const hasLeftConnector = hasPrepDays || (spanInfo && !isSpanFirst);
const hasRightConnector = spanInfo && !isSpanLast;
return (
  <div key={task.id} className="relative w-full min-w-0" style={{ zIndex: hoveredCountdownT...
  {hasLeftConnector && (
    <div style={{ position: 'absolute', left: '-5px', top: '50%', transform: 'translateY(-5...
  )}
  {hasRightConnector && (
    <div style={{ position: 'absolute', right: '-3px', top: '50%', transform: 'translateY(-...
  )}
  <div
    className={`flex flex-col gap-0 text-[9px] pl-1 pr-0.5 py-0.5 rounded border cursor-poi...
    style={{
      backgroundColor: (() => { const cMatch = coursesData.courses.find(c => c.name?.split(...
      borderColor: course.darkColor,
      transform: hoveredCountdownTaskId === task.id ? 'scale(1.12)' : undefined,
      boxShadow: hoveredCountdownTaskId === task.id ? '0 4px 16px rgba(0,0,0,0.25)' : undef...
      transition: 'transform 0.2s ease, box-shadow 0.2s ease',
      paddingRight: dueModulePdfUrl && courseDeliveryModes[task.courseName?.split(' - ')[0]...
      position: 'relative',
    }}
    title={task.title}
  >
    <div className="flex items-center gap-0.5">
      <Checkbox
        checked={task.isCompleted || false}
        onChange={checked => completeMutation.mutate({ id: task.id, isCompleted: !!(c...
        onClick={(e) => e.stopPropagation()}
        className="h-3 w-3 shrink-0 border-black data-[state=checked]:bg-black data-[state=ch...
        data-testid={`checkbox-course-row-${task.id}`}
      />
      <span
        className="truncate cursor-pointer hover:opacity-80 pl-1 flex-1 min-w-0"
        onClick={() => setEditingTask(task)}
      >
        {task.title}
      </span>
      {dueAttachmentLink && !dueModulePdfUrl && (
        <a href={dueAttachmentLink} target="_blank" rel="noopener noreferrer" onClick={(e) =>...
        <Externallink className="h-3 w-3 text-black/60 hover:text-black" />
      )}
    </div>
  </div>
)

```

```

    </a>
  })
  {task.referenceLink && !dueModulePdfUrl && (
    <a href={task.referenceLink} target="_blank" rel="noopener noreferrer" onClick={(e) => ...
    <ExternalLink className="h-3 w-3 text-black/60 hover:text-black" />
    </a>
  )}
</div>
<div style={{ position: 'absolute', right: '2px', top: '50%', transform: 'translateY(-50%
  {() => { const cc = task.courseName?.split(' - ')[0]?.trim().replace(/s/g, '') || '...
  {dueModulePdfUrl && (
    <img
      src={pdfIconPath}
      alt="Open PDF"
      style={{ width: '28px', height: '28px', objectFit: 'contain', cursor: 'pointer', ...
      onClick={(e) => { e.stopPropagation(); e.preventDefault(); const url = dueModuleP...
      data-testid={`pdf-icon-task-${task.id}`}
    </>
  )}
</div>
</div>
</div>
  );
  });
  })();
</div>
</div>
);
});
})();
{ /* Progress column - half-width black with M/R/O bars */
{() => {
  const courseCode = course.name?.split(' - ')[0]?.toUpperCase() || '';
  const courseCodeLower = courseCode.toLowerCase();
  const courseTasks = allTasks.filter(t => {
    const taskCourse = t.courseName?.split(' - ')[0]?.toUpperCase() || '';
    return taskCourse === courseCode && t.weekNumber === selectedWeek;
  });
  const moduleFiles = weeklyFiles.filter(f => f.folder === `week-${selectedWeek}-${courseCodeLower}`...
  const readingFiles = weeklyFiles.filter(f => f.folder === `week-${selectedWeek}-${courseCodeLower}`...
  const calcFileProgress = (files: WeeklyFile[], folderKey: string) => {
    const fc = fileCounts[folderKey];
    const fcPct = (fc && fc.total > 0) ? (fc.partialProgress != null ? Math.min(100, Math.round(fc....
    const fcHasFiles = fc && fc.total > 0;
    if (files.length === 0) {
      if (fcHasFiles) {
        return { percent: Math.max(0, fcPct), hasFiles: true };
      }
      return { percent: 0, hasFiles: false };
    }
    const preparedFiles = files.filter(f => f.listened || (f.totalChunks && f.totalChunks > 0));
    if (preparedFiles.length === 0 && fcPct <= 0) {
      return { percent: 0, hasFiles: false };
    }
    let totalProgress = 0;
    const countableFiles = preparedFiles.length > 0 ? preparedFiles : files;
    for (const f of countableFiles) {
      if (f.listened) {
        totalProgress += 100;
      } else {
        const isCurrentFile = previewFile && f.id === previewFile.id;
        const liveChecked = isCurrentFile ? checkedChunksRef.current : null;
        const liveTotal = isCurrentFile ? (ttsChunksRef.current.length || totalChunks) : 0;
        if (liveChecked && liveChecked.size > 0 && liveTotal > 0) {
          totalProgress += Math.round((liveChecked.size / liveTotal) * 100);
        } else if (f.checkedChunks && f.checkedChunks !== 'null' && f.checkedChunks !== '[]' && f.t...
          try {
            const checked = JSON.parse(f.checkedChunks);
            if (Array.isArray(checked) && checked.length > 0) {
              totalProgress += Math.round((checked.length / f.totalChunks) * 100);
            } else if (f.lastChunkIndex != null && f.lastChunkIndex > 0) {
              totalProgress += Math.round((f.lastChunkIndex / f.totalChunks) * 100);
            }
          } catch {
            if (f.lastChunkIndex != null && f.lastChunkIndex > 0) {
              totalProgress += Math.round((f.lastChunkIndex / f.totalChunks) * 100);
            }
          }
        } else if (isCurrentFile && isPlaying && totalChunks > 0) {
          totalProgress += Math.round(((currentChunkIndex + 1) / totalChunks) * 100);
        } else if (f.totalChunks && f.totalChunks > 0 && f.lastChunkIndex != null && f.lastChunkInd...
          totalProgress += Math.round((f.lastChunkIndex / f.totalChunks) * 100);
        }
      }
    }
    const filesPct = Math.min(100, Math.round(totalProgress / countableFiles.length));
    return { percent: Math.max(filesPct, fcPct >= 0 ? fcPct : 0), hasFiles: preparedFiles.length > ...
  };
  const getProgressColor = (percent: number) => {
    if (percent === 100) return '#22c55e';
    if (percent > 0) return '#f97316';
    return '#ef4444';
  };
  const moduleFolderKey = `week-${selectedWeek}-${courseCodeLower}-module`;
  const readingFolderKey = `week-${selectedWeek}-${courseCodeLower}-reading`;
  const otherFolderKey = `week-${selectedWeek}-${courseCodeLower}-other`;
  const moduleP = calcFileProgress(moduleFiles, moduleFolderKey);
  const readingP = calcFileProgress(readingFiles, readingFolderKey);
  const otherFiles = weeklyFiles.filter(f => f.folder === otherFolderKey);
  const otherP = calcFileProgress(otherFiles, otherFolderKey);
  const hasNoData = false;
  const courseMatch = coursesData.courses.find(c => c.name?.split(' - ')[0]?.toUpperCase() === cour...
  const courseHexColor = courseMatch?.color || '#6b7280';
  const courseHexColorEnd = courseMatch?.colorEnd;
  const moduleFolderCount = fileCounts[moduleFolderKey];
  const readingFolderCount = fileCounts[readingFolderKey];
  const moduleUnread = moduleFolderCount?.unlistened || 0;
  const readingUnread = readingFolderCount?.unlistened || 0;

```

```

const progressBg = () => {
  const startColor = courseHexColor;
  const endColor = courseHexColorEnd || courseHexColor;
  if (courseMatch?.colorStops) {
    try {
      const stops: Array<{position: number; color: string}> = JSON.parse(courseMatch.colorStops);
      if (stops.length > 0) {
        const allParts = ['${startColor} 0%', ...stops.map(s => `${s.color} ${s.position}%`), `${...
          return `linear-gradient(180deg, ${allParts.join(', ')}`;
        }
      } catch {}
    }
    return `linear-gradient(180deg, ${startColor} 0%, ${endColor} 100%)`;
  }
})();
const handlePlayFiles = (fileType: 'module' | 'reading') => {
  const weekFiles = allFiles.filter(f => {
    const weekMatch = f.folder?.match(/week-(\d+)/i);
    return weekMatch && parseInt(weekMatch[1], 10) === selectedWeek;
  });
  const isModuleFile = (f: any) => f.folder?.toLowerCase().includes('module') || f.originalName?....
  const hasUnlistenedModules = weekFiles.some(f => isModuleFile(f) && !f.listened);

  if (fileType === 'reading' && hasUnlistenedModules) {
    const nextModule = weekFiles.filter(f => isModuleFile(f) && !f.listened)[0];
    if (nextModule) {
      toast({ title: `Modules not complete yet – playing next module instead` });
      const readerUrl = nextModule.objectPath?.startsWith('http')
        ? `/pdf-reader/onedrive?oneDriveUrl=${encodeURIComponent(nextModule.objectPath || '')}&na...
          : `/pdf-reader/${nextModule.id}?autoplay=1`;
      const fileName = nextModule.displayName || nextModule.originalName || 'Module';
      setBookAnimState({
        isOpen: true,
        courseCode,
        title: fileName,
        color: courseHexColor,
        onDone: () => {
          setBookAnimState(null);
          setBookReaderOverlay({ url: readerUrl, courseCode, title: fileName, color: courseHexCol...
        });
      });
    }
    return;
  }

  const courseCodeLower = courseCode.toLowerCase();
  const folderKey = `week-${selectedWeek}-${courseCodeLower}-${fileType}`;
  const filesForType = allFiles.filter(f => f.folder === folderKey);
  const unreadFile = filesForType.find(f => !f.listened);
  const targetFile = unreadFile || filesForType[0];
  if (targetFile) {
    const readerUrl = targetFile.objectPath?.startsWith('http')
      ? `/pdf-reader/onedrive?oneDriveUrl=${encodeURIComponent(targetFile.objectPath || '')}&name...
        : `/pdf-reader/${targetFile.id}?autoplay=1`;
    const fileName = targetFile.displayName || targetFile.originalName || fileType;
    setBookAnimState({
      isOpen: true,
      courseCode,
      title: fileName,
      color: courseHexColor,
      onDone: () => {
        setBookAnimState(null);
        setBookReaderOverlay({ url: readerUrl, courseCode, title: fileName, color: courseHexColor...
      });
    });
  } else {
    toast({ title: `No ${fileType} files found for week ${selectedWeek}`, variant: 'destructive' });
  }
};
if (courseProgressDataRef.current.length <= courseIdx) {
  courseProgressDataRef.current.length = courseIdx + 1;
}
courseProgressDataRef.current[courseIdx] = {
  courseCode,
  progressBg,
  progressStartColor: courseHexColor,
  progressEndColor: courseHexColorEnd || courseHexColor,
  courseRowColor: courseMatch?.courseRowColor || courseHexColor,
  taskBgColor: courseMatch?.taskBgColor,
  moduleP,
  readingP,
  moduleUnread,
  readingUnread,
  hasNoData,
  handlePlayModule: () => handlePlayFiles('module'),
  handlePlayReading: () => handlePlayFiles('reading'),
  handleUpload: (uploadType: 'module' | 'reading') => {
    const input = document.createElement('input');
    input.type = 'file';
    input.accept = '.pdf,.doc,.docx,.ppt,.pptx,.txt';
    input.onchange = async (ev) => {
      const file = (ev.target as HTMLInputElement).files?.[0];
      if (!file) return;
      await doHwUpload(file, uploadType, courseCode, course?.name || '', selectedWeek, courseHexC...
    };
    input.click();
  },
  handleFileDrop: (uploadType: 'module' | 'reading', file: File) => {
    doHwUpload(file, uploadType, courseCode, course?.name || '', selectedWeek, courseHexColor);
  },
};
return null;
})();
}
/* Course row resize handle */
<div
  className="absolute bottom-0 left-0 right-0 h-[3px] cursor-row-resize z-[50] opacity-0 group-hove...
  onMouseDown={(e) => handleRowResizeStart(e, 'course')}
  onTouchStart={(e) => handleRowResizeStart(e, 'course')}
  data-testid="course-row-resize-handle-${course.name}"

```

```

    />
  </div>
  });
  })}

  /* Other Tasks Summary Row - black background, only shows tasks with type "other" */
  (() => {
    const otherTasks = allTasks?.filter(task => {
      if (task.type !== 'other' && task.type !== 'meeting' && task.type !== 'reminder') return false;
      if (task.type === 'meeting' && task.courseName) return false;
      if (task.isCompleted) return false;
      if (task.hideFromSummary) return false;
      const taskDueDate = startOfDayET(new Date(task.dueDate));
      const weekStart = startOfDayET(weekDays[0]);
      const weekEnd = startOfDayET(addDays(weekDays[6], 1));
      if (taskDueDate >= weekStart && taskDueDate < weekEnd) return true;
      if (task.startDate) {
        const taskStartDate = startOfDayET(new Date(task.startDate));
        if (taskStartDate < weekEnd && taskDueDate > weekStart) return true;
      }
      return false;
    }) || [];
    const activeCourseNames = filteredCourses.map(c => c.name.split(' - ')[0]?.toUpperCase()).filter(Bool...
    const otherProjects = allProjects.filter(proj => {
      if (proj.status === 'completed' || proj.status === 'cancelled') return false;
      if (!proj.targetDate) return false;
      const hasCourseMatch = proj.courseName && activeCourseNames.includes(proj.courseName.split(' - ')[0]...
      if (hasCourseMatch) return false;
      const projTargetDate = startOfDayET(new Date(proj.targetDate));
      const weekStart = startOfDayET(weekDays[0]);
      const weekEnd = startOfDayET(addDays(weekDays[6], 1));
      if (projTargetDate >= weekStart && projTargetDate < weekEnd) return true;
      if (proj.startDate) {
        const projStartDate = startOfDayET(new Date(proj.startDate));
        if (projStartDate < weekEnd && projTargetDate > weekStart) return true;
      }
      return false;
    });
    const otherRowHeight = Math.max(57, gridSizes.otherRowHeight || 57);
    return (
      <div className="grid w-full flex-shrink-0 relative z-[43] group/otherrow" style={{ gridTemplateColu...
        <div className="px-1 py-0.5 text-[8px] font-[785] tracking-wide flex items-center justify-center ...
          OTHER
          <div style={{ position: 'absolute', top: '1px', right: '1px', zIndex: 2 }} onClick={{e}} => { e....
          </div>
          {gridSizes.moduleColumnWidth > 0 && (
            <div style={{ backgroundColor: otherRowColors.courseRowColor || otherRowColors.cellBg }} />
          )}
          {weekDays.map((day, dayIdx) => {
            const cellDate = startOfDayET(day);
            const isOtherToday = isSameDayET(day, new Date());
            const dayOtherTasks = otherTasks.filter(task => {
              const taskDueDate = startOfDayET(new Date(task.dueDate));
              if (isSameDayET(taskDueDate, cellDate)) return true;
              if (task.startDate) {
                const taskStartDate = startOfDayET(new Date(task.startDate));
                return cellDate >= taskStartDate && cellDate < taskDueDate;
              }
            })
            return false;
          });
          const dayOtherProjs = otherProjects.filter(proj => {
            const projTargetDate = startOfDayET(new Date(proj.targetDate!));
            if (isSameDayET(projTargetDate, cellDate)) return true;
            if (proj.startDate) {
              const projStartDate = startOfDayET(new Date(proj.startDate));
              return cellDate >= projStartDate && cellDate < projTargetDate;
            }
            return false;
          });
          const otherCellCount = dayOtherTasks.length + dayOtherProjs.length;
          const otherHasScroll = otherCellCount > 3;
          const isOtherBeforeToday = !isOtherToday && day < new Date();
          const otherBaseBg = otherRowColors.courseRowColor || otherRowColors.cellBg;
          const otherCellBg = isOtherToday ? '#e4ecf5' : isOtherBeforeToday ? dimColor(otherBaseBg) : oth...
          return (
            <div
              key={dayIdx}
              className={`relative flex flex-col gap-0.5 pt-0.5${otherHasScroll ? ' course-cell-scroll' : ''} ...
              style={{ backgroundColor: otherCellBg, padding: '2px 1px 2px 1px', borderBottom: isOtherTod...
              data-testid={`other-row-${format(day, "yyyy-MM-dd")}`}
            >
              <div style={{ position: 'absolute', left: 0, top: 0, bottom: 0, width: '0px', borderLeft: '...
                {dayOtherTasks.map(task => {
                  const today = startOfDayET(new Date());
                  const tomorrow = addDays(today, 1);
                  const isDueToday = !task.isCompleted && isSameDayET(new Date(task.dueDate), today);
                  const isDueTomorrow = !task.isCompleted && isSameDayET(new Date(task.dueDate), tomorrow);
                  const isUnackedReminder = task.type === 'reminder' && !(task as any).isAcknowledged;
                  return (
                    <div
                      key={task.id}
                      className={`flex items-center gap-0.5 text-[9px] pl-1 pr-0.5 py-0.5 truncate rounded ...
                      style={{
                        backgroundColor: isUnackedReminder ? 'rgba(220, 38, 38, 0.25)' : otherRowColors.tas...
                        borderColor: isUnackedReminder ? 'rgba(220, 38, 38, 0.6)' : otherRowColors.borderCo...
                      }}
                      title={task.title}
                      data-testid={`other-task-${task.id}`}
                    >
                      <Checkbox
                        checked={task.isCompleted || false}
                        onChange={{checked}} => completeMutation.mutate({ id: task.id, isCompleted: !...
                        onClick={{e}} => e.stopPropagation()}
                        className="h-3 w-3 shrink-0 border-black data-[state=checked]:bg-black data-[state=...
                        data-testid={`checkbox-other-${task.id}`}
                      />
                      <span
                        onClick={{}} => setEditingTask(task)}
                    </div>
                  </div>
                })
              </div>
            </div>
          )
        </div>
      )
    )
  )

```



```

        taskTitle: task.title
    });
  }}
  onTouchStart={(e) => handleTouchStart(e, task.id, task.title)}
  onTouchEnd={handleTouchEnd}
  onTouchMove={handleTouchMove}
>
{
  <Checkbox
    checked={task.isCompleted || false}
    onCheckedChange={(checked) => completeMutation.mutate({ id: task.id, isCompleted: !!checked })}
    onClick={(e) => e.stopPropagation()}
    className="h-3 w-3 shrink-0 border-black data-[state=checked]:bg-black data-[state=checked]:border-white"
    data-testid={`checkbox-all-day-${task.id}`}
  />
}
<span
  onClick={() => setEditingTask(task)}
  className={`cursor-pointer hover:opacity-80 truncate flex-1 font-bold ${task.isCompleted ? 'text-gray-400' : 'text-gray-600'}`}
>
  {task.title}
</span>
{task.referenceLink && (
  <a href={task.referenceLink} target="_blank" rel="noopener noreferrer" onClick={(e) => ...}
    <ExternalLink className="h-2.5 w-2.5 text-black/60 hover:text-black" />
  </a>
)}
{adPdfUrl && (
  <img
    src={pdfIconPath}
    alt="Open PDF"
    className="shrink-0"
    style={{ width: '14px', height: '14px', objectFit: 'contain', marginRight: '1px', cursor: 'pointer' }}
    onClick={(e) => { e.stopPropagation(); window.open(adPdfFileId ? `/pdf-reader/${adPdfFileId}` : null); }}
    data-testid={`pdf-icon-all-day-${task.id}`}
  />
)}
{ /* Delete button - always visible */ }
<button
  onClick={(e) => {
    e.stopPropagation();
    deleteTaskWithUndo(task.id);
  }}
  className="ml-auto shrink-0 p-0.5 rounded hover:bg-red-500/20 text-red-500"
  title="Delete task"
  data-testid={`button-delete-all-day-${task.id}`}
>
  <X className="h-3 w-3" />
</button>
</div>
</div>
);
}}
{ /* All-day Google Calendar events */ }
{allDayEvents.map(event => {
  const isPrepEvent = event.title.startsWith('PREP');
  const displayTitle = isPrepEvent ? event.title.replace(/^[PREP]\s*/, '') : event.title;
  return (
    <a
      key={event.id}
      href={event.htmlLink}
      target="_blank"
      rel="noopener noreferrer"
      className={`flex items-center text-[8px] py-0.5 rounded border cursor-pointer hover:opacity-80`}
      data-testid={`all-day-gcal-${event.id}`}
    >
      {isPrepEvent ? (
        <span className="flex items-center whitespace-nowrap font-bold shrink-0 rounded-l" style={{ padding: 0 5px }}>PREP</span>
        <span className="truncate font-bold flex-1 min-w-0 pl-1">{displayTitle}</span>
      ) : (
        <CalendarDays className="h-3 w-3 shrink-0 text-gray-600 dark:text-gray-300" />
        <span className="truncate font-bold flex-1 min-w-0">{displayTitle}</span>
      )
    </a>
  );
});
};
{ /* Projects with target date on this day */ }
{getProjectsForDay(day).map(project => {
  const isCompleted = project.status === 'completed';
  return (
    <RouterLink
      key={`project-${project.id}`}
      href="/projects"
      className={`flex items-center gap-1 text-[8px] px-1 py-0.5 rounded truncate border cursor-pointer`}
      data-testid={`calendar-project-${project.id}`}
    >
      <FolderOpen className="h-3 w-3 shrink-0" />
      <span className={`truncate font-bold flex-1 min-w-0 ${isCompleted ? "line-through" : ""}`}>
        {project.name}
      </span>
    </RouterLink>
  );
});
};
</div>
);
}}
{ /* ALL DAY row resize handle */ }
<div

```

```

        className="absolute bottom-0 left-0 right-0 h-[3px] cursor-row-resize z-[50] opacity-0 group-hover/al...
        onMouseDown={(e) => handleRowResizeStart(e, 'allDay')}
        onTouchStart={(e) => handleRowResizeStart(e, 'allDay')}
        data-testid="allDay-row-resize-handle"
      />
    </div>}}

    /* Time Slots - Scrollable area */
    <div ref={calendarScrollRef} className="flex-1 overflow-y-auto relative" style={{ borderBottomLeftRadius:...
    <div style={{ backgroundColor: '#faf8f5', borderBottomLeftRadius: '8px', borderBottomRightRadius: '8px'...
      {timeSlots.map((hour, hourIdx) => {
        const currentHour = new Date().getHours();
        const isCurrentHour = hour === currentHour;
        const isNightHour = hour >= 21 || hour <= 6;
        const nightHourHasTasks = isNightHour && weekDays.some(day => {
          const ht = getTasksForHour(day, hour);
          const ct = getContinuingTasksForHour(day, hour);
          const ce = getCalendarEventsForHour(day, hour);
          return ht.length > 0 || ct.length > 0 || ce.length > 0;
        });
        const compressedHours = [0,1,2,3,4,5,6,21,22,23];
        const isCompressedHour = compressedHours.includes(hour);
        const eightAMH = gridSizes.timeSlotHeights[0] || gridSizes.timeSlotHeight;
        const baseRowHeight = gridSizes.timeSlotHeights[hour] || gridSizes.timeSlotHeight;
        const rowHeight = baseRowHeight;
        return (
          <div
            key={hour}
            className={`grid relative group/row flex-shrink-0`}
            style={{ gridTemplateColumns: getGridTemplateColumns(), height: `${rowHeight}px`, minHeight: `${r...
          >
          <div className={`text-[10px] font-medium tracking-wide flex items-center justify-center relative ...
            {hour === 0 || hour === 24 ? '12 AM' : hour === 12 ? '12 PM' : hour > 12 ? `${hour - 12} PM` : ...
          </div>
          {gridSizes.moduleColumnWidth > 0 && <div style={{ minWidth: 0, backgroundColor: isCurrentHour ? '...
          {weekDays.map((day, dayIdx) => {
            const hourTasks = getTasksForHour(day, hour);
            const continuingTasks = getContinuingTasksForHour(day, hour);
            const hourCalendarEvents = getCalendarEventsForHour(day, hour);
            const visibleCalendarEvents = hourCalendarEvents.filter(e => !dismissedCalendarEvents.has(e.id));
            const isFriday = day.getDay() === 5;
            const isToday = isSameDayET(day, new Date());
            const totalItems = hourTasks.length + visibleCalendarEvents.length;
            const hasAnyTasks = totalItems > 0 || continuingTasks.length > 0;
            const columnWidth = totalItems > 0 ? 100 / totalItems : 100;
            const cellDateStr = format(day, "yyyy-MM-dd");
            const isYesterday = isSameDayET(day, subDays(new Date(), 1));
            const cellShift = isYesterday ? undefined : localShiftMap[cellDateStr];
            const sleepDisabledForDay = sleepDisabledDays.has(cellDateStr);
            const prevDayStr = format(addDays(day, -1), "yyyy-MM-dd");
            const prevDayShift = localShiftMap[prevDayStr];
            const sleepDisabledForPrevDay = sleepDisabledDays.has(prevDayStr);
            const isNightShiftSleepHour = !sleepDisabledForDay && cellShift === 'night' && hour >= 10 && ho...
            const nightSleepColor = 'rgba(180, 180, 180, 0.22)';
            const isDayShiftSleepHour = (!sleepDisabledForDay && cellShift === 'day' && hour >= 22) || (!sl...
            const isSleepHour = isNightShiftSleepHour || isDayShiftSleepHour;
            const isFirstNightSleep = isNightShiftSleepHour && hour === 10;
            const nightSleepCellCount = 7;
            const isFirstDayShiftEvening = !sleepDisabledForDay && cellShift === 'day' && hour === 22;
            const dayShiftEveningCellCount = 2;
            const isFirstDayShiftMorning = !sleepDisabledForPrevDay && prevDayShift === 'day' && hour === 0;
            const dayShiftMorningCellCount = 5;
            const sleepLabelStart = isFirstNightSleep || isFirstDayShiftEvening || isFirstDayShiftMorning;
            const sleepCellCount = isFirstNightSleep ? nightSleepCellCount : isFirstDayShiftEvening ? daySh...
            return (
              <div
                key={dayIdx}
                className={`border-l relative p-0.5 ${dragOverSlot && isSameDayET(dragOverSlot.day, day) &&...
                style={{
                  borderLeftColor: isCurrentHour ? 'rgba(0,0,0,0.15)' : 'hsl(var(--border) / 0.5)',
                  borderBottomRightRadius: hourIdx === timeSlots.length - 1 && dayIdx === 6 ? '16px' : unde...
                  backgroundColor: (isToday || isCurrentHour) ? '#e4ecf5' : isNightShiftSleepHour ? nightSl...
                  borderTop: hour === 12 ? '2.5px solid rgba(150,150,150,0.5)' : undefined,
                  paddingLeft: `${2 + DAY_COL_LEFT_REDUCTION}px`,
                }}
                data-testid={`time-slot-${format(day, "yyyy-MM-dd")}-${hour}`}
                onDragOver={(e) => handleDragOver(e, day, hour)}
                onDragLeave={handleDragLeave}
                onDrop={(e) => handleDrop(e, day, hour)}
                onDoubleClick={(e) => {
                  const rect = e.currentTarget.getBoundingClientRect();
                  const clickY = e.clientY - rect.top;
                  const isBottomHalf = clickY > rect.height / 2;
                  const minutes = isBottomHalf ? 30 : 0;

                  if (isSleepHour) {
                    setPendingSleepTask({ day: new Date(day), hour, minutes });
                    return;
                  }

                  const dueDate = new Date(day);
                  dueDate.setHours(hour, minutes, 0, 0);

                  setSelectedDate(dueDate);
                  setNewTaskType("other");
                  setInitialStartTime(`${hour.toString().padStart(2, '0')}:${minutes.toString().padStart(2,...
                  setInitialEndTime(`${(hour + 1).toString().padStart(2, '0')}:${minutes.toString().padStar...
                  bringFlyoutToFront('addTask');
                  setIsAddDialogOpen(true);
                }}
              >
            /* Background layer - sits below tasks so they don't get covered */
            <div
              className={`absolute inset-0 z-0 ${hasAnyTasks && !isToday && !isCurrentHour ? "bg-blue-5...
              style={{
                borderBottomRightRadius: hourIdx === timeSlots.length - 1 && dayIdx === 6 ? '16px' : un...
            }}
          </div>
        )
      />
    }
  }
}

```

```

<div className="absolute top-0 bottom-0 z-[1] pointer-events-none" style={{ left: `${gridSi...
{sleepLabelStart && !isToday && (
  <div style={{
    position: 'absolute',
    top: 0,
    left: 0,
    right: 0,
    height: `${sleepCellCount * 100}%`,
    display: 'flex',
    alignItems: 'center',
    justifyContent: 'center',
    zIndex: 1,
    pointerEvents: 'none',
    overflow: 'hidden'
  }}>
    {isFirstDayShiftEvening ? (
      <span style={{
        fontSize: '18px',
        fontWeight: 900,
        letterSpacing: '4px',
        color: 'rgba(150, 150, 150, 0.07)',
        userSelect: 'none',
        fontFamily: "Avenir, 'Avenir Next', -apple-system, BlinkMacSystemFont, sans-serif",
        textTransform: 'uppercase',
        whiteSpace: 'nowrap'
      }}>SLEEP</span>
    ) : (
      <span style={{
        writingMode: 'vertical-rl',
        textOrientation: 'mixed',
        transform: 'rotate(180deg)',
        fontSize: isFirstDayShiftMorning ? '32px' : '48px',
        fontWeight: 900,
        letterSpacing: isFirstDayShiftMorning ? '4px' : '8px',
        color: 'rgba(150, 150, 150, 0.07)',
        userSelect: 'none',
        fontFamily: "Avenir, 'Avenir Next', -apple-system, BlinkMacSystemFont, sans-serif",
        textTransform: 'uppercase',
        whiteSpace: 'nowrap'
      }}>SLEEP</span>
    )
  }
</div>
)}
{isToday && isCurrentHour && (() => {
  const now = new Date();
  const currentHourNow = now.getHours();
  const isNightTime = currentHourNow >= 21 || currentHourNow < 6;
  if (isNightTime) return null;
  const narrowColWidth = gridSizes.timeSlotHeight;
  const getTaskTime = (t: any) => {
    if (t.eventStartTime) {
      const [h, m] = t.eventStartTime.split(':').map(Number);
      const d = new Date(t.dueDate);
      const local = new Date(d.getFullYear(), d.getMonth(), d.getDate(), h, m);
      return local.getTime();
    }
    return new Date(t.dueDate).getTime();
  };
  const todayStart = startOfDayET(now);
  const missedSchoolTasks = allTasks.filter(t => {
    if (!t.courseName) return false;
    if (!t.isCompleted) return false;
    if (!t.eventStartTime) {
      const d = new Date(t.dueDate);
      if (d.getUTCHours() === 0 && d.getUTCMinutes() === 0) return false;
      if (getETHours(d) === 0 && getETMinutes(d) === 0) return false;
    }
    const taskDueDate = startOfDayET(new Date(t.dueDate));
    if (isSameDayET(taskDueDate, todayStart)) return true;
    if (taskDueDate < todayStart) return false;
    const taskTime = getTaskTime(t);
    if (taskTime > now.getTime()) return false;
    return true;
  });
  const missedCount = missedSchoolTasks.length;
  const nextSchoolTask = allTasks
    .filter(t => !t.isCompleted && t.courseName && getTaskTime(t) > now.getTime())
    .sort((a, b) => getTaskTime(a) - getTaskTime(b))[0];
  const hoursUntil = nextSchoolTask ? (getTaskTime(nextSchoolTask) - now.getTime()) / (1000...
  const displayTime = hoursUntil !== null ? (hoursUntil < 1 ? `${Math.max(1, Math.round(hou...
  return (
    <>
      {missedCount > 0 && (
        <div className="absolute top-0 bottom-0 z-[3] pointer-events-none animate-due-box-b...
        <div style={{ width: '100%', height: '100%', position: 'relative', backgroundColo...
          <span style={{ fontSize: '20px', color: 'white', fontWeight: 900, lineHeight: 1...
          <span style={{ fontSize: '9px', color: 'white', fontWeight: 600, lineHeight: 1...
        </div>
      )
    }
    <div className="absolute top-0 bottom-0 z-[3] pointer-events-none" style={{ left: (ho...
      {(() => {
        const courseCode = nextSchoolTask?.courseName?.split(' - ')[0]?.trim() || '';
        const gradColors = getCourseGradientColors(courseCode);
        return (
          <div style={{ flex: 1, position: 'relative', borderRadius: '5px', overflow: 'hi...
            <span style={{ zIndex: 1, fontSize: '6px', color: 'rgba(255,255,255,0.0)', fo...
            <span style={{ zIndex: 1, fontSize: '20px', color: 'white', fontWeight: 900, ...
          </div>
        );
      })()
    </div>
  </>
)}
)}
)}
{hour !== 12 && <div
  className={absolute left-0 right-0 border-t border-dotted z-[1] border-gray-400 dark:bor...

```

```

    style={{ top: '0' }}
  />
  /* Half-hour dotted line - hidden for compact night hours and current time intersection */
  {(isNightHour && !nightHourHasTasks) && !(isToday && isCurrentHour) && <div
    className={ absolute left-0 right-0 border-t border-dotted z-[1] border-gray-300 dark:bor...
    style={{ top: '50%' }}
  />
  /* Multi-hour tasks are now rendered at scroll container level as single elements */
  hourTasks.filter(task => {
    // Skip multi-hour tasks - they're rendered at scroll container level
    if (task.eventStartTime && task.eventEndTime) {
      const [startHour] = task.eventStartTime.split(':').map(Number);
      const [endHour] = task.eventEndTime.split(':').map(Number);
      if (endHour > startHour) return false;
    }
    return true;
  }).map((task, taskIdx) => {
    const courseCode = task.courseName?.split(" ")[0]?.toUpperCase() || "";
    const colors = dynamicCourseColors[courseCode];
    const today = startOfDayET(new Date());
    const tomorrow = addDays(today, 1);
    const isDueToday = !task.isCompleted && isSameDayET(new Date(task.dueDate), today);
    const isDueTomorrow = !task.isCompleted && isSameDayET(new Date(task.dueDate), tomorrow);

    // Calculate height based on duration for events with start/end times
    let taskHeight = Math.min(40, rowHeight - 4); // Same height for all tasks, clamped to row
    let topOffset = 2; // Default top offset

    if (task.eventStartTime && task.eventEndTime) {
      const [startHour, startMin] = task.eventStartTime.split(':').map(Number);
      const [endHour, endMin] = task.eventEndTime.split(':').map(Number);
      const startMinutes = startHour * 60 + startMin;
      let endMinutes = endHour * 60 + endMin;
      if (endMinutes <= startMinutes) endMinutes = startMinutes + 60;
      const calendarEndMinutes = 24 * 60;
      endMinutes = Math.min(endMinutes, calendarEndMinutes);
      const durationMinutes = endMinutes - startMinutes;
      taskHeight = Math.max(40, (durationMinutes / 60) * 44 - 4);
      topOffset = (startMin / 60) * 44;
      const slotHeight = rowHeight;
      const maxTaskHeight = slotHeight - topOffset - 2;
      if (taskHeight > maxTaskHeight) {
        taskHeight = Math.max(20, maxTaskHeight);
      }
    } else if (task.eventStartTime) {
      const [, startMin] = task.eventStartTime.split(':').map(Number);
      if (startMin > 0) {
        topOffset = (startMin / 60) * rowHeight;
        taskHeight = Math.min(taskHeight, rowHeight - topOffset - 2);
        taskHeight = Math.max(20, taskHeight);
      }
    } else {
      const dueMin = getETMinutes(new Date(task.dueDate));
      if (dueMin > 0) {
        topOffset = (dueMin / 60) * rowHeight;
        taskHeight = Math.min(taskHeight, rowHeight - topOffset - 2);
        taskHeight = Math.max(20, taskHeight);
      }
    }
  })

  return (
    <div
      key={task.id}
      draggable
      tabIndex={0}
      onDragStart={(e) => handleDragStart(e, task)}
      onDragEnd={handleDragEnd}
      onClick={(e) => {
        e.stopPropagation();
        setSelectedTaskId(task.id);
      }}
      onDoubleClick={(e) => {
        e.stopPropagation();
        setEditingTask(task);
      }}
      onKeyDown={(e) => {
        if (e.key === 'Delete' || e.key === 'Backspace') {
          e.preventDefault();
          deleteTaskWithUndo(task.id);
          setSelectedTaskId(null);
        }
      }}
      onContextMenu={(e) => {
        e.preventDefault();
        e.stopPropagation();
        setContextMenu({
          x: e.clientX,
          y: e.clientY,
          taskId: task.id,
          taskTitle: task.title
        });
      }}
      onTouchStart={(e) => handleTouchStart(e, task.id, task.title)}
      onTouchEnd={handleTouchEnd}
      onTouchMove={handleTouchMove}
      onMouseEnter={() => { if (totalItems > 1) setHoveredCalTaskId(task.id); }}
      onMouseLeave={() => { if (hoveredCalTaskId === task.id) setHoveredCalTaskId(null); }}
      className={ absolute shadow-sm cursor-grab active:cursor-grabbing rounded overflow-hi...
      draggedTask?.id === task.id ? "opacity-50" : ""
    }
    <div
      selectedTaskId === task.id ? "ring-2 ring-red-500 ring-offset-1" : ""
    >
    <div
      style={{
        top: `${topOffset}px`,
        left: hoveredCalTaskId === task.id ? '2px' : (() => { const currentHourNow = new Da...
        width: hoveredCalTaskId === task.id ? 'calc(100% - 4px)' : (() => { const currentHo...

```



```

topPx += (startMin / 60) * startHourHeight;

let heightPx = 0;
heightPx += ((60 - startMin) / 60) * startHourHeight;
for (let h = startHour + 1; h < endHour; h++) {
  heightPx += getEffectiveRowHeight(h);
}
if (endHour > startHour) {
  const endHourHeight = getEffectiveRowHeight(endHour);
  heightPx += (endMin / 60) * endHourHeight;
}

const courseCode = task.courseName?.split(" ")[0]?.toUpperCase() || "";
const colors = dynamicCourseColors[courseCode];
const today = startOfDayET(new Date());
const tomorrow = addDays(today, 1);
const isDueToday = !task.isCompleted && isSameDayET(new Date(task.dueDate), today);
const isDueTomorrow = !task.isCompleted && isSameDayET(new Date(task.dueDate), tomorrow);

const taskDay = weekDays[dayIdx];

return (
  <div
    key={`multi-${task.id}`}
    draggable
    onDragStart={(e) => handleDragStart(e, task)}
    onDragEnd={handleDragEnd}
    onClick={(e) => {
      e.stopPropagation();
      setSelectedTaskId(task.id);
    }}
    onDoubleClick={(e) => {
      e.stopPropagation();
      setEditingTask(task);
    }}
    onContextMenu={(e) => {
      e.preventDefault();
      e.stopPropagation();
      setContextMenu({
        x: e.clientX,
        y: e.clientY,
        taskId: task.id,
        taskTitle: task.title
      });
    }}
    className={`absolute hover:opacity-90 shadow-sm cursor-grab active:cursor-grabbing rounded over...
      draggedTask?.id === task.id ? "opacity-50" : ""
    } ${
      selectedTaskId === task.id ? "ring-2 ring-red-500 ring-offset-1" : ""
    } ${
      ""
    }`
    style={{
      top: `${topPx}px`,
      left: `calc(${gridSizes.timeColumnWidth + gridSizes.moduleColumnWidth}px + (${gridSizes.dayCo...
      width: (() => { const now = new Date(); const currentHourNow = now.getHours(); const taskDay ...
      height: `${heightPx}px`,
      zIndex: hoveredCountdownTaskId === task.id ? 55 : (selectedTaskId === task.id ? 50 : (dragged...
      backgroundColor: task.isCompleted ? '#e5e7eb' : (colors?.bg || (task.type === 'other' ? other...
      borderColor: task.isCompleted ? '#d1d5db' : (colors?.hex || colors?.border || (task.type === ...
      transform: hoveredCountdownTaskId === task.id ? 'scale(1.08)' : undefined,
      boxShadow: hoveredCountdownTaskId === task.id ? '0 4px 16px rgba(0,0,0,0.25)' : undefined,
      transition: 'transform 0.2s ease, box-shadow 0.2s ease',
      transformOrigin: 'center center',
    }}
    data-testid={`multi-hour-task-${task.id}`}
    data-cal-task-id={task.id}
    data-cal-date={format(taskDay, 'yyyy-MM-dd')}}
  >
    /* Red shimmer swipe for due today tasks */
    <div className={`flex items-center gap-1.5 px-0.5 py-1 ${isDueToday ? "silver-shimmer-header" : ...
      <Checkbox
        checked={task.isCompleted || false}
        onCheckedChange={(checked) => completeMutation.mutate({ id: task.id, isCompleted: !!check...
        className="h-3 w-3 shrink-0 border-black data-[state=checked]:bg-black data-[state=checke...
        onClick={(e) => e.stopPropagation()}
      />
      <span
        onClick={() => setEditingTask(task)}
        className={`text-[9px] leading-tight font-normal line-clamp-2 cursor-pointer flex-1 ${task....
      >
        {{{() => { const t = task.title || ''; const isModRead = /^(module|reading|module\s*(?:&|and...
      </span>
        {{{() => { const cc = task.courseName?.split(' - ')[0]?.trim().replace(/\\s/g, ' ') || ''; const...
        {{{() => { const pdfUrl = task.attachments?.length ? (() => { for (const att of task.attachmen...
      </div>
      {task.eventStartTime && task.eventEndTime && (
        <div
          className="text-[8px] font-normal text-muted-foreground ml-[18px] px-0.5"
          style={{ animation: 'none' }}
        >
          {formatTimeTo12Hour(task.eventStartTime)} - {formatTimeTo12Hour(task.eventEndTime)}
        </div>
      )}
    </div>
  );
}})
</div>
{
  /* Current time indicator line - rendered last to appear on top */
  {{{() => {
    const now = new Date();
    const currentHour = now.getHours();
    const currentMinutes = now.getMinutes();
    const calStartHour = calStart;
    const calEndHour = isTravelMode ? 23 : 21;

```

```

    if (currentHour < calStartHour || currentHour > calEndHour) return null;

    let topPosition = 0;
    for (let h = calStartHour; h < currentHour; h++) {
      topPosition += getEffectiveRowHeight(h);
    }
    const currentRowHeight = getEffectiveRowHeight(currentHour);
    topPosition += (currentMinutes / 60) * currentRowHeight;

    const isTodayInView = weekDays.some(d => isSameDayET(d, now));
    const todayDayIdx = weekDays.findIndex(d => isSameDayET(d, now));
    const isTodaySat = todayDayIdx === 6;
    const totalFrUnits = gridSizes.dayColumnWidths.reduce((a: number, b: number) => a + b, 0);
    const satFr = gridSizes.dayColumnWidths[6];

    return (
      <div
        className="absolute left-0 z-[5] pointer-events-none"
        style={{
          top: `${topPosition}px`,
          right: isTodaySat
            ? '0px'
            : `calc((${satFr} / ${totalFrUnits}) * (100% - ${gridSizes.timeColumnWidth + gridSizes.modu...`
        }}
      >
        <div
          className="w-full border-t border-dashed"
          style={{ borderColor: 'rgba(0, 0, 0, 0.3)' }}
        />
      </div>
    );
  })()
</div>
</div>
</div>
</div>
<div
  style={{ position: 'absolute', left: '50%', transform: 'translateX(-50%)', bottom: '-12px', width: '191px...
  data-testid="calendar-bottom-resize-handle"
>
  <div style={{ width: '191px', height: '14px', borderRadius: '0 0 6px 6px', background: 'rgba(255,255,255,...
    <div className="cursor-pointer" style={{ flex: 1, display: 'flex', alignItems: 'center', justifyContent...
      <span style={{ width: '1px', height: '6px', background: 'rgba(120,120,120,0.3)', flexShrink: 0 }} />
      <div className="cursor-grab active:cursor-grabbing select-none" style={{ flex: 1, display: 'flex', align...
        <span style={{ width: '1px', height: '6px', background: 'rgba(120,120,120,0.3)', flexShrink: 0 }} />
        <div className="cursor-pointer" style={{ flex: 1, display: 'flex', alignItems: 'center', justifyContent...
      </div>
    </div>
  </div>
  {/} Set Default checkbox – below calendar {/}
  <button
    className="cursor-pointer select-none hover:opacity-80 transition-opacity"
    style={{ position: 'absolute', right: '0px', bottom: '-15px', display: 'flex', alignItems: 'center', gap:...
    onClick={() => {
      localStorage.setItem('calendarHeight', String(calendarHeight));
      localStorage.setItem('calendarReduction', String(calendarReduction));
      localStorage.setItem('gridSizes', JSON.stringify(gridSizes));
      setCalendarReductionUserSet(true);
      const screenWidth = window.screen.width;
      const screenHeight = window.screen.height;
      const pixelRatio = window.devicePixelRatio || 1;
      const deviceId = `device_${screenWidth}x${screenHeight}@${pixelRatio}`;
      localStorage.setItem(`calendarHeight_${deviceId}`, String(calendarHeight));
      localStorage.setItem(`calendarReduction_${deviceId}`, String(calendarReduction));
      localStorage.setItem(`gridSizes_${deviceId}`, JSON.stringify(gridSizes));
      fetch('/api/degree-tracking/bulk', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ calendarHeight, calendarReduction, gridSizes }),
      }).catch(() => {});
      toast({ title: "Saved", description: "Calendar size saved as default" });
    }}
    data-testid="button-set-default-size"
  >
    <span style={{ fontSize: '11px', fontWeight: 500, color: 'rgba(255,255,255,0.6)', whiteSpace: 'nowrap' }}...
  </button>
</div>

{/} Weeks Flyout - centered panel for week folders {/}
<div
  className={`fixed ${isResizingWeeksFlyout ? '' : 'transition-opacity duration-400 ease-out'} overflow-hid...
  style={{ width: isMobile ? '95vw' : '900px', maxWidth: '900px', height: isMobile ? '90vh' : '85vh', top: ...
  onClick={() => bringFlyoutToFront('files')}
>
  {/} Resize Handle {/}
  <div
    className="absolute left-0 top-0 bottom-0 w-2 cursor-ew-resize z-50 hover:bg-white/20 active:bg-white/30"
    style={{ touchAction: 'none' }}
    onMouseDown={(e: React.MouseEvent) => {
      e.preventDefault();
      setIsResizingWeeksFlyout(true);
      const startX = e.clientX;
      const startWidth = Math.max(flyoutWidth, flyout2Width);

      const handleMove = (moveEvent: MouseEvent | TouchEvent) => {
        const { clientX } = getPointerXY(moveEvent);
        const delta = startX - clientX;
        const newWidth = Math.max(150, Math.min(400, startWidth + delta));
        setFlyoutWidth(newWidth);
        setFlyout2Width(newWidth);
        setWeeksFlyoutWidth(newWidth);
      };

      const handleEnd = () => {
        setIsResizingWeeksFlyout(false);
        document.removeEventListener('mousemove', handleMove);
        document.removeEventListener('mouseup', handleEnd);
        document.removeEventListener('touchmove', handleMove);
      };
    }}
  >

```

```

    document.removeEventListener('touchend', handleEnd);
  };

  document.addEventListener('mousemove', handleMove);
  document.addEventListener('mouseup', handleEnd);
  document.addEventListener('touchmove', handleMove, { passive: false });
  document.addEventListener('touchend', handleEnd);
})
onTouchStart=(e: React.TouchEvent) => {
  e.preventDefault();
  setIsResizingWeeksFlyout(true);
  const { clientX: startX } = getPointerXY(e);
  const startWidth = Math.max(flyoutWidth, flyout2Width);

  const handleMove = (moveEvent: MouseEvent | TouchEvent) => {
    const { clientX } = getPointerXY(moveEvent);
    const delta = startX - clientX;
    const newWidth = Math.max(150, Math.min(400, startWidth + delta));
    setFlyoutWidth(newWidth);
    setFlyout2Width(newWidth);
    setWeeksFlyoutWidth(newWidth);
  };

  const handleEnd = () => {
    setIsResizingWeeksFlyout(false);
    document.removeEventListener('mousemove', handleMove);
    document.removeEventListener('mouseup', handleEnd);
    document.removeEventListener('touchmove', handleMove);
    document.removeEventListener('touchend', handleEnd);
  };

  document.addEventListener('mousemove', handleMove);
  document.addEventListener('mouseup', handleEnd);
  document.addEventListener('touchmove', handleMove, { passive: false });
  document.addEventListener('touchend', handleEnd);
})
data-testid="weeks-flyout-resize-handle"
/>
<div className="h-full flex flex-col text-white relative rounded-xl" style={{ background: `linear-gradien...
  /* Header */
  <div className="flex items-center justify-between px-4 py-3 bg-black/30 border-b border-white/20">
    <div className="flex items-center gap-3">
      <Button
        variant="outline"
        className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-trans...
        style={{
          boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255,25...
        }}
        onMouseEnter={(e) => {
          e.currentTarget.style.boxShadow = '0 0 8px rgba(255,255,255,0.8), 0 0 16px rgba(255,255,255,0.6...
        }}
        onMouseLeave={(e) => {
          e.currentTarget.style.boxShadow = '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4...
        }}
        onClick={() => setIsUploadDialogOpen(true)}
        data-testid="button-upload-file"
      >
      <Upload className="h-4 w-4 mr-2" />
      Upload
    </Button>
    <div className="flex items-center gap-1">
      <Button
        variant="ghost"
        size="icon"
        className="h-7 w-7 hover:bg-white/20 rounded-md"
        onClick={() => setSelectedWeek(selectedWeek - 1)}
        data-testid="button-weeks-flyout-prev-week"
      >
      <ChevronLeft className="h-4 w-4 text-white" strokeWidth={2.5} />
    </Button>
    <div className="flex items-center gap-0.5 bg-white/10 rounded-md px-1.5 py-0.5 backdrop-blur-sm w...
      <span className="text-[10px] font-medium text-white">{format(weekStartDate, "MMMM d")}</span>
      <span className="text-[10px] text-white/50"></span>
      <span className="text-[10px] font-medium text-white">{format(weekEndDate, "MMMM d")}</span>
    </div>
    <Button
      variant="ghost"
      size="icon"
      className="h-7 w-7 hover:bg-white/20 rounded-md"
      onClick={() => setSelectedWeek(selectedWeek + 1)}
      data-testid="button-weeks-flyout-next-week"
    >
    <ChevronRight className="h-4 w-4 text-white" strokeWidth={2.5} />
  </Button>
</div>
</div>
<div className="flex items-center gap-3">
  <div className="flex items-center gap-2">
    <Folder className="h-3 w-3 text-white" />
    <h2 className="text-xs font-normal text-white" style={{ fontFamily: "Avenir, 'Avenir Next', -appl...
      FILES ({allFiles.length})
    </h2>
  </div>
  <button
    onClick={() => setIsWeeksFlyoutOpen(false)}
    className="text-white hover:text-white/80 transition-colors p-1"
    data-testid="button-close-weeks-flyout"
  >
  <X className="h-5 w-5" />
</button>
</div>
</div>
/* OneDrive Files */
<div
  className="flex-1 overflow-y-auto py-2 px-2"
  style={{ scrollbarWidth: 'none' }}
>

```



```

/* Read-only mode header - show simplified navigation for share link users */
{!isReadOnly && (() => {
  const defaultPath = "/School/1. TMU/Courses/2026/Winter";
  const isAtDefaultPath = oneDrivePath === defaultPath;
  const canGoBack = oneDrivePathHistory.length > 0 && !isAtDefaultPath;

  return (
    <div className="flex items-center gap-2 mb-3 pb-2 border-b border-white/20">
      <Button
        variant="ghost"
        size="icon"
        className="h-6 w-6 hover:bg-white/20"
        onClick={() => {
          if (canGoBack) {
            const previousPath = oneDrivePathHistory[oneDrivePathHistory.length - 1];
            // Don't go above the default path
            if (previousPath.startsWith(defaultPath) || previousPath === defaultPath) {
              setOneDrivePathHistory(oneDrivePathHistory.slice(0, -1));
              setOneDrivePath(previousPath);
            }
          }
        }}
        disabled={!canGoBack}
        data-testid="button-onedrive-back-readonly"
      />
      <ArrowLeft className="h-3 w-3 text-white" />
    </Button>
    <Eye className="h-3 w-3 text-white/50" />
    <span className="text-[11px] text-white/60">View Only</span>
    <span className="text-white/30 mx-1">|</span>
    <span className="text-[11px] text-white/80 truncate flex-1">
      {oneDrivePath.split('/').filter(Boolean).pop() || 'Course Files'}
    </span>
  </div>
);
})();

/* OneDrive Navigation - hidden for read-only share link users */
{!isReadOnly && (
  <div className="flex items-center gap-2 mb-3 pb-2 border-b border-white/20">
    <Button
      variant="ghost"
      size="icon"
      className="h-7 w-7 hover:bg-white/20"
      onClick={() => {
        if (oneDrivePathHistory.length > 0) {
          const previousPath = oneDrivePathHistory[oneDrivePathHistory.length - 1];
          setOneDrivePathHistory(oneDrivePathHistory.slice(0, -1));
          setOneDrivePath(previousPath);
        }
      }}
      disabled={oneDrivePathHistory.length === 0}
      data-testid="button-onedrive-back"
    />
    <ArrowLeft className="h-4 w-4 text-white" />
  </Button>
  <Button
    variant="ghost"
    size="icon"
    className="h-7 w-7 hover:bg-white/20"
    onClick={() => {
      setOneDrivePathHistory([]);
      setOneDrivePath("/");
    }}
    data-testid="button-onedrive-home"
  />
    <Home className="h-4 w-4 text-white" />
  </Button>
  <Button
    variant="ghost"
    size="icon"
    className="h-7 w-7 hover:bg-white/20"
    onClick={() => {
      queryClient.invalidateQueries({ queryKey: ["/api/onedrive/files", oneDrivePath] });
    }}
    data-testid="button-onedrive-sync"
  />
    <RefreshCw className={`h-4 w-4 text-white ${oneDriveLoading ? 'animate-spin' : ''}`} />
  </Button>
  <div className="flex items-center gap-1 text-[11px] text-white/60 flex-1 truncate">
    <Cloud className="h-3 w-3" />
    <span className="truncate">{oneDrivePath === "/" ? "OneDrive" : oneDrivePath}</span>
  </div>
</div>
);

{oneDriveLoading ? (
  <div className="flex items-center justify-center py-8">
    <Loader2 className="h-6 w-6 animate-spin text-white/50" />
  </div>
) : (
  <div className="space-y-1">
    /* Folders - show all folders for read-only users (they can navigate freely) */
    {oneDriveFolders.map(({folder}) => {
      let folderColor: string | undefined;
      for (const c of coursesData.courses) {
        if (!c.name.trim()) continue;
        const code = c.name.split('-')[0]?.trim().toUpperCase();
        if (oneDrivePath.includes(code)) { folderColor = getCourseGradientColors(code).end; break; }
      }
      if (!folderColor) {
        for (const c of coursesData.courses) {
          if (!c.name.trim()) continue;
          const code = c.name.split('-')[0]?.trim().toUpperCase();
          if (folder.name.includes(code)) { folderColor = getCourseGradientColors(code).end; break; }
        }
      }
    })
  </div>
);

```

```

return (
  <div
    key={folder.id}
    className="flex items-center gap-2 px-2 py-1.5 hover:bg-white/10 cursor-pointer rounded tra...
    onClick={() => {
      setOneDrivePathHistory([...oneDrivePathHistory, oneDrivePath]);
      setOneDrivePath(folder.path);
    }}
    data-testid={`onedrive-folder-${folder.id}`}
  >
    <Folder
      className={folderColor ? "h-4 w-4" : "h-4 w-4 text-yellow-500 fill-yellow-400"}
      style={folderColor ? { color: folderColor, fill: folderColor } : undefined}
    />
    <span className="text-[13px] text-white/90 truncate flex-1">{folder.name}</span>
    <ChevronRight className="h-3 w-3 text-white/40" />
  </div>
);
});

/* PDF Files - hidden for read-only share link users */
{!isReadOnly && oneDrivePdfFiles.map((file) => (
  <div
    key={file.id}
    className="flex items-center gap-2 px-2 py-1.5 hover:bg-white/10 cursor-pointer rounded trans...
    onClick={() => {
      if (file.downloadUrl) {
        const encodedUrl = encodeURIComponent(file.downloadUrl);
        const encodedName = encodeURIComponent(file.name);
        window.open(`/pdf-reader/onedrive?url=${encodedUrl}&name=${encodedName}`, '_blank');
      }
    }}
    data-testid={`onedrive-file-${file.id}`}
  >
    <FileText className="h-4 w-4 text-red-400" />
    <span className="text-[13px] text-white/90 truncate flex-1">{file.name}</span>
    <Play className="h-3 w-3 text-white/40 opacity-0 group-hover:opacity-100 transition-opacity" />
  </div>
))

/* Other Files - hidden for read-only share link users */
{!isReadOnly && oneDriveFiles.filter(f => !f.mimeType?.includes("pdf")).map((file) => (
  <div
    key={file.id}
    className="flex items-center gap-2 px-2 py-1.5 hover:bg-white/10 cursor-pointer rounded trans...
    onClick={() => {
      if (file.downloadUrl) {
        window.open(file.downloadUrl, "_blank");
      }
    }}
    data-testid={`onedrive-file-${file.id}`}
  >
    <FileText className="h-4 w-4 text-white/60" />
    <span className="text-[13px] text-white/70 truncate flex-1">{file.name}</span>
  </div>
))

{oneDriveItems.length === 0 && (
  <div className="flex flex-col items-center justify-center py-8 text-white/40">
    <Folder className="h-8 w-8 mb-2 opacity-50" />
    <p className="text-[12px]">No files found</p>
  </div>
)}

</div>
))

/* Legacy week folders hidden - using OneDrive now */
{false && (() => {
  // Get current week number based on today's date
  const today = new Date();
  const currentWeekData = weeks.find(w => {
    const start = new Date(w.startDate);
    const end = new Date(w.endDate);
    return today >= start && today <= end;
  });
  const currentWeekNum = currentWeekData?.weekNumber || selectedWeek;

  // Sort weeks: current/future weeks first (by number), past weeks at the bottom
  const sortedWeeks = [...FLYOUT_WEEKS].sort((a, b) => {
    const aNum = parseInt(a.id.replace('week-', ''));
    const bNum = parseInt(b.id.replace('week-', ''));

    // Check if weeks are past (ended)
    const aWeekData = weeks.find(w => w.weekNumber === aNum);
    const bWeekData = weeks.find(w => w.weekNumber === bNum);
    const aPast = aWeekData ? new Date(aWeekData.endDate) < today : aNum < currentWeekNum;
    const bPast = bWeekData ? new Date(bWeekData.endDate) < today : bNum < currentWeekNum;

    // Past weeks go to the bottom
    if (aPast && !bPast) return 1;
    if (!aPast && bPast) return -1;

    // Within same status, sort by week number
    return aNum - bNum;
  });

  const renderWeekFolder = (week: typeof sortedWeeks[0]) => {
    const weekNum = parseInt(week.id.replace('week-', ''));
    const isPastWeek = weekNum < currentWeekNum;
    const weekFiles = getFilesInFlyoutWeek(week.id);
    const isWeekExpanded = flyoutExpandedFolders.has(week.id);
    const hasFiles = weekFiles.length > 0;
    const allFilesCompleted = hasFiles && weekFiles.every(f => f.listened);
    const shouldBlink = isPastWeek && hasFiles && !allFilesCompleted;
    const shouldStrikethrough = isPastWeek && allFilesCompleted;

    if (!hasFiles) return null;

```

```

return (
  <div key={week.id} className="mb-1">
    {/* Week folder row */}
    <div
      className={flex items-center gap-1.5 pr-2 py-1 hover:bg-white/10 cursor-pointer rounded tr...
      onClick={() => toggleFlyoutFolder(week.id)}
      onContextMenu={(e) => handleFolderContextMenu(e, week.id)}
      data-testid={`flyout-${week.id}`}
      onDragOver={(e) => {
        e.preventDefault();
        e.stopPropagation();
        e.dataTransfer.dropEffect = 'move';
        if (draggedFileForMove) {
          e.currentTarget.style.backgroundColor = 'rgba(59, 130, 246, 0.3)';
        }
      }}
      onDragLeave={(e) => {
        e.currentTarget.style.backgroundColor = '';
      }}
      onDrop={async (e) => {
        e.preventDefault();
        e.stopPropagation();
        e.currentTarget.style.backgroundColor = '';
        const fileId = e.dataTransfer.getData('text/plain');
        // Default to first course's reading folder
        const targetFolder = `${week.id}-cppa122-reading`;
        if (fileId) {
          try {
            const res = await fetch(`/api/files/${fileId}`, {
              method: 'PATCH',
              headers: { 'Content-Type': 'application/json' },
              body: JSON.stringify({ folder: targetFolder })
            });
            if (res.ok) {
              queryClient.invalidateQueries({ queryKey: ['/api/files'] });
              refreshFileCounts();
              toast({ title: "File moved to CPPA122 Reading" });
            } else {
              toast({ title: "Failed to move file", variant: "destructive" });
            }
          } catch (err) {
            toast({ title: "Failed to move file", variant: "destructive" });
          }
        }
      }}
      setDraggedFileForMove(null);
    </div>
    >
    <span style={{ display: 'inline-flex', alignItems: 'center', justifyContent: 'center', width: ...
      {isWeekExpanded ? <FolderOpen className="h-3.5 w-3.5 text-yellow-500 fill-yellow-400" /> : ...
      <span className="text-[13px] truncate ${shouldStrikethrough ? 'text-white/50' : 'text-whit...
      <span className="text-[11px] text-white/40 ml-auto">{weekFiles.length}</span>
    </div>

    {/* Course folders inside week */}
    {isWeekExpanded && (
      <div className="ml-3">
        {FLYOUT_COURSES.map((course) => {
          const courseFiles = getFilesInFlyoutCourse(week.id, course.id);
          const courseFolderId = `${week.id}-${course.id}`;
          const isCourseExpanded = flyoutExpandedFolders.has(courseFolderId);

          if (courseFiles.length === 0) return null;

          return (
            <div key={courseFolderId} className="mb-0.5">
              <div
                className={`flex items-center gap-1.5 pr-2 py-1 hover:bg-white/10 cursor-pointer ...
                onClick={() => toggleFlyoutFolder(courseFolderId)}
                onContextMenu={(e) => handleFolderContextMenu(e, courseFolderId)}
                data-testid={`flyout-folder-${courseFolderId}`}
                onDragOver={(e) => {
                  e.preventDefault();
                  e.stopPropagation();
                  e.dataTransfer.dropEffect = 'move';
                  if (draggedFileForMove) {
                    e.currentTarget.style.backgroundColor = 'rgba(59, 130, 246, 0.3)';
                  }
                }}
                onDragLeave={(e) => {
                  e.currentTarget.style.backgroundColor = '';
                }}
                onDrop={async (e) => {
                  e.preventDefault();
                  e.stopPropagation();
                  e.currentTarget.style.backgroundColor = '';
                  const fileId = e.dataTransfer.getData('text/plain');
                  // Default to 'reading' content folder when dropping on course
                  const targetFolder = `${week.id}-${course.id}-reading`;
                  if (fileId) {
                    try {
                      const res = await fetch(`/api/files/${fileId}`, {
                        method: 'PATCH',
                        headers: { 'Content-Type': 'application/json' },
                        body: JSON.stringify({ folder: targetFolder })
                      });
                      if (res.ok) {
                        queryClient.invalidateQueries({ queryKey: ['/api/files'] });
                        toast({ title: "File moved to Reading folder" });
                      } else {
                        toast({ title: "Failed to move file", variant: "destructive" });
                      }
                    } catch (err) {
                      toast({ title: "Failed to move file", variant: "destructive" });
                    }
                  }
                }}
                setDraggedFileForMove(null);
              </div>
            </div>
          );
        })}
      </div>
    )}
  )

```

```

>
<span style={{ display: 'inline-flex', alignItems: 'center', justifyContent: 'cen...
  {isCourseExpanded ? <ChevronDown className="h-3.5 w-3.5 text-black" /> : <Chevr...
  </span>
  {isCourseExpanded ? <FolderOpen className="h-3.5 w-3.5 text-yellow-500 fill-yello...
  <span className={ `text-[12px] truncate flex-1 ${course.color} `}>{course.name}</span>
  <span className="text-[11px] text-white/40">{courseFiles.length}</span>
</div>

{/* Content folders inside course */}
{isCourseExpanded && (
  <div className="ml-3">
    {FLYOUT_CONTENT.map((content) => {
      const contentFolderId = `${week.id}-${course.id}-${content.id}`;
      const contentFiles = getFilesInFlyoutFolder(contentFolderId);
      const isContentExpanded = flyoutExpandedFolders.has(contentFolderId);

      if (contentFiles.length === 0) return null;

      return (
        <div key={contentFolderId} className="mb-0.5">
          <div
            className={ `flex items-center gap-1.5 pr-2 py-1 hover:bg-white/10 curso...
            onClick={() => toggleFlyoutFolder(contentFolderId)}
            onDragOver={(e) => {
              e.preventDefault();
              e.stopPropagation();
              e.dataTransfer.dropEffect = 'move';
              if (draggedFileForMove && draggedFileForMove.folder !== contentFolder...
                e.currentTarget.style.backgroundColor = 'rgba(59, 130, 246, 0.3)';
            }}
            onDragLeave={(e) => {
              e.currentTarget.style.backgroundColor = '';
            }}
            onDrop={async (e) => {
              e.preventDefault();
              e.stopPropagation();
              e.currentTarget.style.backgroundColor = '';
              const fileId = e.dataTransfer.getData('text/plain');
              if (fileId) {
                try {
                  const res = await fetch(`/api/files/${fileId}`, {
                    method: 'PATCH',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ folder: contentFolderId })
                  });
                  if (res.ok) {
                    queryClient.invalidateQueries({ queryKey: ['/api/files'] });
                    toast({ title: "File moved successfully" });
                  } else {
                    toast({ title: "Failed to move file", variant: "destructive" });
                  }
                } catch (err) {
                  toast({ title: "Failed to move file", variant: "destructive" });
                }
              }
              setDraggedFileForMove(null);
            }}
          </div>
          <span style={{ display: 'inline-flex', alignItems: 'center', justifyCon...
            {isContentExpanded ? <FolderOpen className="h-3.5 w-3.5 text-yellow-500...
            <span className="text-[12px] text-white/90 truncate flex-1">{content.na...
            <span className="text-[11px] text-white/40">{contentFiles.length}</span>
          </div>

          {/* Files inside content folder */}
          {isContentExpanded && (
            <div className="ml-3 space-y-0.5">
              {contentFiles.map((file) => (
                <div
                  key={file.id}
                  draggable
                  onDragStart={(e) => {
                    e.dataTransfer.setData('text/plain', file.id.toString());
                    e.dataTransfer.effectAllowed = 'move';
                    setDraggedFileForMove({ id: file.id, folder: file.folder || '' });
                  }}
                  onDragEnd={() => setDraggedFileForMove(null)}
                  className={ `flex items-center gap-1.5 pr-2 py-1 hover:bg-white/10...
                >
                <Checkbox
                  checked={file.listened || false}
                  onCheckedChange={async (checked) => {
                    try {
                      const resp = await fetch(`/api/files/${file.id}`, {
                        method: 'PATCH',
                        headers: { 'Content-Type': 'application/json' },
                        credentials: 'include',
                        body: JSON.stringify({ listened: checked === true })
                      });
                      if (!resp.ok) throw new Error(`HTTP ${resp.status}`);
                      queryClient.invalidateQueries({ queryKey: ['/api/files'] });
                      queryClient.invalidateQueries({ queryKey: ['/api/files/coun...
                      queryClient.invalidateQueries({ queryKey: ['/api/onedrive/w...
                    } catch (err) {
                      console.error('Failed to update listened status:', err);
                    }
                  }}
                  onClick={(e) => e.stopPropagation()}
                  className="h-3.5 w-3.5 border border-white/40 data-[state=check...
                />
                <FileText className="h-3.5 w-3.5 text-white/50 shrink-0" />
                <span
                  className={ `text-[11px] truncate flex-1 hover:underline cursor-...
                  onClick={() => { const url = file.objectPath?.startsWith('http'...
                  onContextMenu={(e) => {
                    e.preventDefault();

```



```

const swipeableRow = (rowEl: HTMLDivElement, contentEl: HTMLDivElement, deleteEl: HTMLDivElement, reschedul...
let startX = 0;
let currentX = 0;
let offsetX = 0;
let swiping = false;
let moved = false;
const getMax = () => (rowEl.offsetWidth || 150) / 3;
const update = () => {
  const mx = getMax();
  const clamped = Math.max(-mx, Math.min(mx, currentX));
  currentX = clamped;
  contentEl.style.transform = `translateX(${clamped}px)`;
  contentEl.style.transition = swiping ? 'none' : 'transform 0.2s ease-out';
  deleteEl.style.width = clamped > 0 ? `${clamped}px` : '0px';
  deleteEl.style.display = clamped > 0 ? 'flex' : 'none';
  rescheduleEl.style.width = clamped < 0 ? `${Math.abs(clamped)}px` : '0px';
  rescheduleEl.style.display = clamped < 0 ? 'flex' : 'none';
};
const reset = () => { currentX = 0; offsetX = 0; swiping = false; moved = false; update(); };
const end = () => {
  if (!swiping) return;
  swiping = false;
  const mx = getMax();
  if (!moved || Math.abs(currentX) < mx * 0.4) { currentX = 0; offsetX = 0; }
  else { currentX = currentX > 0 ? mx : -mx; offsetX = currentX; }
  moved = false;
  update();
};
const onStart = (x: number) => { startX = x; swiping = true; moved = false; };
const onMove = (x: number) => { if (!swiping) return; const delta = x - startX; if (Math.abs(delta) > 2) ...
contentEl.addEventListener('touchstart', (e) => { onStart(e.touches[0].clientX); }, { passive: true });
contentEl.addEventListener('touchmove', (e) => { onMove(e.touches[0].clientX); }, { passive: true });
contentEl.addEventListener('touchend', (e) => { if (!moved && offsetX !== 0) { reset(); e.preventDefault(...
contentEl.addEventListener('mousedown', (e) => { e.preventDefault(); onStart(e.clientX); });
contentEl.addEventListener('mousemove', (e) => { if (!swiping) return; e.preventDefault(); onMove(e.clie...
contentEl.addEventListener('mouseup', (e) => { if (!moved && offsetX !== 0) { reset(); e.preventDefault()}...
contentEl.addEventListener('mouseleave', end);
return { reset };
};

const calcCourseFileProgress = (courseCode: string) => {
  const courseCodeLower = courseCode.toLowerCase();
  const moduleFolderKey = `week-${selectedWeek}-${courseCodeLower}-module`;
  const readingFolderKey = `week-${selectedWeek}-${courseCodeLower}-reading`;
  const mFiles = weeklyFiles.filter(f => f.folder === moduleFolderKey);
  const rFiles = weeklyFiles.filter(f => f.folder === readingFolderKey);
  const calc = (files: typeof weeklyFiles, folderKey: string) => {
    const fc = fileCounts[folderKey];
    const fcPct = (fc && fc.total > 0) ? (fc.partialProgress !== null ? Math.min(100, Math.round(fc.partialP...
    const fcHasFiles = fc && fc.total > 0;
    if (files.length === 0) {
      if (fcHasFiles) return { percent: Math.max(0, fcPct), hasFiles: true };
      return { percent: 0, hasFiles: false };
    }
    var prepared = files.filter(function(f) { return f.listened || (f.totalChunks && f.totalChunks > 0); });
    if (prepared.length === 0 && fcPct <= 0) {
      return { percent: 0, hasFiles: false };
    }
    var countable = prepared.length > 0 ? prepared : files;
    var totalProgress = 0;
    for (var fi = 0; fi < countable.length; fi++) {
      var f = countable[fi];
      if (f.listened) { totalProgress += 100; }
      else if (f.checkedChunks && f.checkedChunks !== 'null' && f.checkedChunks !== '[]' && f.totalChunks &...
        try { var checked = JSON.parse(f.checkedChunks); if (Array.isArray(checked) && checked.length > 0) ...
      } else if (f.totalChunks && f.totalChunks > 0 && f.lastChunkIndex !== null && f.lastChunkIndex > 0) {
        totalProgress += Math.round((f.lastChunkIndex / f.totalChunks) * 100);
      }
    }
    var filesPct = Math.min(100, Math.round(totalProgress / countable.length));
    return { percent: Math.max(filesPct, fcPct >= 0 ? fcPct : 0), hasFiles: prepared.length > 0 };
  };
const getFileProgressColor = (percent: number) => percent === 100 ? '#22c55e' : percent > 0 ? '#f97316' : ...
const moduleP = calc(mFiles, moduleFolderKey);
const readingP = calc(rFiles, readingFolderKey);
return { moduleP, readingP, getFileProgressColor };
};

// Helper function to parse task attachments
const parseAttachments = (attachments: any[] | null | undefined) => {
  if (!attachments || attachments.length === 0) return [];
  return attachments.map(att => {
    if (typeof att === 'string') {
      try {
        const parsed = JSON.parse(att);
        return { name: parsed.name || parsed.url?.split('/').pop() || 'File', url: parsed.url || att };
      } catch {
        return { name: att.split('/').pop() || 'File', url: att };
      }
    }
  });
  return { name: att.name || att.url?.split('/').pop() || 'File', url: att.url || '' };
};

// Helper to find matching file from allFiles
const findFileByUrl = (url: string) => {
  return allFiles.find(f => f.objectPath === url || f.objectPath.includes(url) || url.includes(f.objectPath));
};

// Group tasks by course
const groupByCourse = (tasks: typeof dueTodayTasks) => {
  const grouped: Record<string, typeof tasks> = {};
  const courseOrder = ['CPPA122', 'CFNF400', 'CASL101', 'CECN210', 'CPHL110', 'CHIS105', 'CPPA235'];
  tasks.forEach(task => {
    const courseCode = task.courseName?.split(' - ')[0] || task.courseName?.split(' ')[0] || 'OTHER';
    if (!grouped[courseCode]) grouped[courseCode] = [];
    grouped[courseCode].push(task);
  });
};

```

```

return Object.entries(grouped).sort(([a], [b]) => {
  const aIdx = courseOrder.indexOf(a);
  const bIdx = courseOrder.indexOf(b);
  if (aIdx === -1 && bIdx === -1) return a.localeCompare(b);
  if (aIdx === -1) return 1;
  if (bIdx === -1) return -1;
  return aIdx - bIdx;
});
};

// Render column header with resize handles on both sides of progress bar
const renderTaskColumnHeader = () => {
  const handleStyle = { width: '3px', minHeight: '12px', backgroundColor: 'transparent' };
  return (
    <div style={{ display: 'flex', alignItems: 'center', marginLeft: '-25px', marginBottom: '4px', gap: '0p...
      { /* Col 1: Checkbox placeholder */ }
      <div style={{ width: '16px', flexShrink: 0 }} />
      { /* Handle 1 */ }
      <div className="cursor-col-resize hover:bg-white/50" style={handleStyle} onMouseDown={(e) => handleTa...
      { /* Col 2: Progress bar placeholder */ }
      <div style={{ width: '44px', flexShrink: 0 }} />
      { /* Handle 2 */ }
      <div className="cursor-col-resize hover:bg-white/50" style={handleStyle} onMouseDown={(e) => handleTa...
      { /* Col 3: Task */ }
      <div style={{ width: `${taskColumnWidths.taskName}px`, flexShrink: 0 }} className="text-[8px] text-wh...
      { /* Handle 3 */ }
      <div className="cursor-col-resize hover:bg-white/50" style={handleStyle} onMouseDown={(e) => handleTa...
      { /* Col 4: Code */ }
      <div style={{ width: `${taskColumnWidths.courseCode}px`, flexShrink: 0 }} className="text-[8px] text-...
      { /* Handle 4 */ }
      <div className="cursor-col-resize hover:bg-white/50" style={handleStyle} onMouseDown={(e) => handleTa...
      { /* Col 5: Course */ }
      <div style={{ width: `${taskColumnWidths.courseName}px`, flexShrink: 0 }} className="text-[8px] text-...
      { /* Handle 5 */ }
      <div className="cursor-col-resize hover:bg-white/50" style={handleStyle} onMouseDown={(e) => handleTa...
      { /* Col 6: Due */ }
      <div style={{ width: `${taskColumnWidths.dueDate}px`, flexShrink: 0 }} className="text-[8px] text-whi...
      { /* Handle 6 */ }
      <div className="cursor-col-resize hover:bg-white/50" style={handleStyle} onMouseDown={(e) => handleTa...
      { /* Col 7: Days */ }
      <div className="text-[8px] text-white font-normal">Days</div>
    </div>
  );
};

// Helper to calculate progress bar width based on time until due
// Shows time remaining - more days = longer bar
const getProgressBarWidth = (task: typeof dueTodayTasks[0] | undefined): number => {
  if (!task) return 0;
  if (task.isCompleted) return 44;

  // Always use time-based progress (days until due)
  // More days until due = longer bar (more time remaining)
  const daysUntil = differenceInDays(startOfDayET(new Date(task.dueDate)), startOfDayET(new Date()));
  const maxDays = 7;
  // More days = more fill (capped at 7 days)
  const progressPercent = Math.max(0, Math.min(100, (daysUntil / maxDays) * 100));
  return Math.round((progressPercent / 100) * 44);
};

// Helper to get progress bar color based on box type and days until due
const getProgressColor = (task: typeof dueTodayTasks[0] | undefined, boxType: 'today' | 'tomorrow' | 'thisw...
  if (!task) return '#22c55e';
  const daysUntil = differenceInDays(startOfDayET(new Date(task.dueDate)), startOfDayET(new Date()));
  if (boxType === 'today') return '#ef4444';
  if (boxType === 'tomorrow') return '#eab308';
  return daysUntil < 3 ? '#eab308' : '#22c55e';
};

// Helper to format days display for Tomorrow box
// Shows "1(X)d" for prep tasks where X is the actual due date days
// Returns JSX with proper coloring: "1" and "d" in progress bar color, "(X)" in green if X >= 4
const getTomorrowDaysDisplay = (task: typeof dueTodayTasks[0] | undefined, progressColor?: string): React.R...
  if (!task?.dueDate) return '';
  const daysUntilDue = Math.ceil((new Date(task.dueDate).getTime() - new Date().getTime()) / (1000 * 60 * 6...
  const barColor = progressColor || getProgressColor(task, 'tomorrow');

  // If task is actually due tomorrow (1 day), just show "1d" in progress bar color
  if (daysUntilDue <= 1) {
    return <span style={{ color: barColor }}>1d</span>;
  }

  // If task has a startDate and is in prep period, show "1(X)d" format
  if (task.startDate) {
    // Green color for (X) if X >= 4, otherwise use a medium green
    const bracketColor = daysUntilDue >= 4 ? '#22c55e' : '#86efac';
    return (
      <div>
        <span style={{ color: barColor }}>1</span>
        <span style={{ color: bracketColor }}>{daysUntilDue}</span>
        <span style={{ color: barColor }}>d</span>
      </div>
    );
  }

  // Fallback to regular days display in progress bar color
  return <span style={{ color: barColor }}>{daysUntilDue}d</span>;
};

// Render a single task row
const renderTask = (task: typeof dueTodayTasks[0], showDaysUntil = false, boxType: 'today' | 'tomorrow' | '...
  const attachments = parseAttachments(task.attachments);
  const daysUntil = differenceInDays(startOfDayET(new Date(task.dueDate)), startOfDayET(new Date()));

  // Check if this is a module task (has startDate spanning multiple days)
  // and if it's Wednesday or later (should blink)
  const today = new Date();
  const currentDayOfWeek = today.getDay();

```



```

const isWednesdayOrLater = currentDayOfWeek >= 3 && currentDayOfWeek <= 5;
const isModuleTask = task.startDate && task.startDate !== task.dueDate;
const shouldBlinkInTodayBox = isModuleTask && isWednesdayOrLater && !task.isCompleted;
const isAssignment = task.type && ![ 'discussion', 'poll' ].includes(task.type);
const shouldBlinkAssignment = isAssignment && daysUntil === 2 && !task.isCompleted;
const isZeroDaysTask = daysUntil === 0 && !task.isCompleted;

// Calculate progress using the helper function (subtasks or time-based)
const progressBarWidth = getProgressBarWidth(task);

// Color based on box type: Today=red, Tomorrow=yellow, This Week=based on days
const progressColor = getProgressColor(task, boxType);

// Parse course code and name (format: "CPPA122" or "CPPA122 - Full Name")
const courseCode = (task.courseName?.split(' - ')[0] || '').trim();
const courseFullName = (task.courseName?.includes(' - ') ? task.courseName.split(' - ').slice(1).join(' - ...') : task.courseName);
return (
  <div
    key={task.id}
    className={`mb-1.5 rounded transition-colors ${draggedFile ? 'hover:bg-white/20 hover:ring-2 hover:ring-white' : ''}`}
    data-box-task-id={task.id}
    style={{ fontFamily: "Avenir, 'Avenir Next', -apple-system, BlinkMacSystemFont, sans-serif" }}
    onMouseEnter={() => setHoveredCountdownTaskIdDebounce(task.id)}
    onMouseLeave={() => setHoveredCountdownTaskIdDebounce(null)}
    onDragOver={(e) => { if (draggedFile) { e.preventDefault(); e.stopPropagation(); } }}
    onDrop={(e) => handleFileDropOnTask(e, task.id)}
    data-testid={`draggable-task-${task.id}`}
  >
    <div style={{ display: 'flex', alignItems: 'center', marginLeft: '-7px', gap: '0px', width: 'calc(100% - 14px)' }} >
      <div style={{ width: '16px', flexShrink: 0 }} >
        <input
          type="checkbox"
          checked={task.isCompleted ?? false}
          onChange={(e) => completeMutation.mutate({ id: task.id, isCompleted: e.target.checked })}
          onTouchEnd={(e) => {
            e.preventDefault();
            e.stopPropagation();
            const newVal = !task.isCompleted ?? false;
            completeMutation.mutate({ id: task.id, isCompleted: newVal });
          }}
          className="h-3.5 w-3.5 rounded-sm cursor-pointer"
          style={{ accentColor: getCourseColor(task.courseName), outline: '1.5px solid white', outlineOffset: '-1px' }}
          data-testid={`checkbox-task-${task.id}`}
          {...(boxType === 'today' ? { 'data-today-checkbox': task.id } : {})}
          {...(boxType === 'tomorrow' ? { 'data-tomorrow-checkbox': task.id } : {})}
        </input>
      </div>
      <div style={{ width: '3px', flexShrink: 0 }} </div>
      <div style={{ flex: '1 1 0', minWidth: 0, display: 'flex', flexDirection: 'column', gap: '0px' }} >
        <div className="flex items-center gap-1">
          <button
            className="text-[12px] text-white font-bold truncate hover:underline cursor-pointer leading-normal"
            onClick={() => setEditingTask(task)}
            data-testid={`task-link-${task.id}`}
            style={{ textAlign: 'left', flex: '1 1 0', minWidth: 0 }}
          >
            {task.title}
          </button>
          {task.referenceLink && (
            <a href={task.referenceLink} target="_blank" rel="noopener noreferrer" onClick={(e) => e.stopPropagation()}>
              <ExternalLink className="h-3 w-3 text-white/60 hover:text-white" />
            </a>
          )}
        </div>
        <span className="text-[9px] text-white/50 font-normal truncate leading-tight">{courseCode}{courseName}</span>
        {task.type === 'module' || task.type === 'reading' || task.type === 'Module' || task.type === 'R...' ? (
          const cc = courseCode.toLowerCase();
          const wk = task.weekNumber ?? selectedWeek;
          const mKey = `week-${wk}-${cc}-module`;
          const rKey = `week-${wk}-${cc}-reading`;
          const mFiles = weeklyFiles.filter(f => f.folder === mKey);
          const rFiles = weeklyFiles.filter(f => f.folder === rKey);
          const calcPct = (files: typeof weeklyFiles, folderKey: string) => {
            const fc = fileCounts[folderKey];
            const fcPct = (fc && fc.total > 0) ? (fc.partialProgress !== null ? Math.min(100, Math.round(fc.partialProgress / fc.total * 100)) : 0) : 0;
            if (files.length === 0) return fcPct >= 0 ? fcPct : 0;
            let tp = 0;
            for (const f of files) {
              if (f.listened) { tp += 100; }
              else if (f.checkedChunks && f.checkedChunks !== 'null' && f.checkedChunks !== '[]' && f.totalChunks > 0) {
                try { const ck = JSON.parse(f.checkedChunks); if (Array.isArray(ck) && ck.length > 0) tp += ck.length; }
                else if (f.totalChunks && f.totalChunks > 0 && f.lastChunkIndex && f.lastChunkIndex > 0) ...
              }
            }
            const fp = Math.min(100, Math.round(tp / files.length));
            return Math.max(fp, fcPct >= 0 ? fcPct : 0);
          };
          const mPct = calcPct(mFiles, mKey);
          const rPct = calcPct(rFiles, rKey);
          const pColor = (p: number) => p === 100 ? '#22c55e' : p > 0 ? '#f97316' : '#ef4444';
          const hasMFiles = mFiles.length > 0 || (fileCounts[mKey] && fileCounts[mKey].total > 0);
          const hasRFiles = rFiles.length > 0 || (fileCounts[rKey] && fileCounts[rKey].total > 0);
          if (!hasMFiles && !hasRFiles) return null;
          return (
            <div className="flex items-center gap-2 mt-0.5">
              {hasMFiles && (
                <div className="flex items-center gap-1">
                  <span className="text-[7px] font-medium uppercase" style={{ color: '#000000' }}>M</span>
                  <div style={{ width: '32px', height: '3px', backgroundColor: 'rgba(255,255,255,0.15)', position: 'relative' }} >
                    <div style={{ width: `${mPct}%`, height: '3px', backgroundColor: pColor(mPct), border: '1px solid #000000' }} </div>
                  <span className="text-[7px] font-bold" style={{ color: '#000000' }}>{mPct}%</span>
                </div>
              )}
              {hasRFiles && (
                <div className="flex items-center gap-1">
                  <span className="text-[7px] font-medium uppercase" style={{ color: '#000000' }}>R</span>
                </div>
              )}
            </div>
          )
        ) : null
      )
    </div>
  </div>
)

```

```

        <div style={{ width: '32px', height: '3px', backgroundColor: 'rgba(255,255,255,0.15)', ...
        <div style={{ width: `${rPct}%`, height: '3px', backgroundColor: pColor(rPct), border...
        </div>
        <span className="text-[7px] font-bold" style={{ color: '#000000' }}>{rPct}%</span>
    </div>
    )}
  </div>
  );
}
})();
</div>
<div style={{ width: '5px', flexShrink: 0 }} />
{ /* Col 3: Days count + progress bar */ }
<div style={{ width: '36px', flexShrink: 0, display: 'flex', flexDirection: 'column', alignItems: '...
  <span className="text-[9px] font-bold" style={{ color: progressColor, lineHeight: 1 }}>{daysUntil...

<Popover>
  <PopoverTrigger asChild>
    <button className="text-[9px] font-bold hover:underline cursor-pointer" style={{ color: progr...
  </PopoverTrigger>
  <PopoverContent className="w-auto p-0" align="end" side="top" style={{ zIndex: 10010 }} onOpenA...
    <div className="p-2">
      <CalendarPicker
        mode="single"
        selected={new Date(task.dueDate)}
        onSelect={(date) => {
          if (date) {
            const old = new Date(task.dueDate);
            date.setHours(old.getHours(), old.getMinutes(), old.getSeconds());
            apiRequest("PATCH", `/api/tasks/${task.id}`, { dueDate: date.toISOString() }).then(()...
              queryClient.invalidateQueries({ queryKey: ['/api/tasks'] });
            });
          }
        }}
        defaultMonth={new Date(task.dueDate)}
      />
    </div>
  </PopoverContent>
</Popover>
</div>
<div style={{ width: '3px', flexShrink: 0 }} />
</div>
{attachments.length > 0 && (
  <div className="mt-0.5 space-y-0.5">
    {attachments.map((file, idx) => {
      const matchingFile = findFileByUrl(file.url);
      const displayName = matchingFile?.displayName || file.name || file.url.split('/').pop() || 'File';
      return (
        <button
          key={idx}
          className={`flex items-center gap-1.5 text-[10px] text-white cursor-pointer w-full pl-6 ${b...
          onClick={() => {
            if (matchingFile) {
              const url = matchingFile.objectPath?.startsWith('http')
                ? `/pdf-reader/onedrive?oneDriveUrl=${encodeURIComponent(matchingFile.objectPath || '...
                  : `/pdf-reader/${matchingFile.id}?autoplay=1`;
              window.open(url, '_blank');
            } else {
              window.open(file.url, '_blank');
            }
          }}
          data-testid={`file-link-${task.id}-${idx}`}
        >
          <FileText className="h-3 w-3" />
          <span className="truncate">{displayName}</span>
        </button>
      );
    })}
  </div>
);
</div>
);
};
// Render grouped tasks with course headers
const renderGroupedTasks = (tasks: typeof dueTodayTasks, showDaysUntil = false) => {
  const grouped = groupByCourse(tasks);
  return grouped.map(([courseCode, courseTasks]) => {
    const courseName = courseTasks[0]?.courseName || courseCode;
    return (
      <div key={courseCode} className="mb-3 last:mb-0">
        <div
          className="text-[10px] text-black font-normal mb-1 pb-0.5 border-b border-white/30"
        >
          {courseName}
        </div>
        <div className="space-y-0.5">
          {courseTasks.map(task => renderTask(task, showDaysUntil, 'thisweek'))}
        </div>
      </div>
    );
  });
};
};
return (
  <div style={{ order: 3, height: '0px', position: 'relative', flexShrink: 0 }}>
    { /* Coming Up box - positioned to the right of the calendar in the reduction gap */ }
    <section
      ref={homeworkSectionRef}
      className="rounded-[12px] flex flex-col fixed"
      style={{
        zIndex: 35,
        overflow: 'visible',
        right: `${calendarRight - calendarReduction + 3 + 7 - 6 + 2 + 4 + 3 - 2 + 4 + 3 + 2 - 3 + 2 + 1}px`,
        width: `${calendarReduction + 10 - 20 - 2 - 5 - 1 - 2 - 1 - 3 + 1 + 1 - 1 - 3 - 4 - 1 - 1 + 1 - 5 - 2 - 1...
        top: `${(calendarBorderTop || (calendarTop + 15)) - 1}px`,
        height: `${window.innerHeight - (calendarBorderTop || (calendarTop + 15)) + 1 - calendarBottom}px`,
        background: `linear-gradient(180deg, ${colorSettings.mainBackground} 0%, color-mix(in srgb, ${colorSettin...
        boxShadow: `0 8px 32px rgba(0,0,0,0.08), 0 2px 8px rgba(0,0,0,0.04), inset 0 1px 0 rgba(255,255,255,0.48)`.
      }}
    </section>
  </div>
);

```

```

border: '1px solid white',
opacity: (isPillMenuOpen && !sidePillIdle) ? 0 : 1,
pointerEvents: (isPillMenuOpen && !sidePillIdle) ? 'none' : 'auto',
transition: 'opacity 0.2s ease'
}}
data-testid="section-coming-up"
>
{ /* Date navigation tab above glass box */
<div
  className="absolute z-[60]"
  style={{ top: '-29px', left: 'calc(50% + 65px)', transform: 'translateX(-50%)', height: '15px', display: ...
  data-testid="date-nav-tab"
  >
  <div style={{ width: '191px', height: '15px', borderRadius: '6px 6px 0 0', background: 'rgba(255,255,255,...
    <div className="cursor-pointer hover:bg-white/20 rounded" data-date-nav style={{ display: 'flex', align...
    <span data-testid="text-week-dates" style={{ fontSize: '9px', fontWeight: 500, color: '#000', whiteSpac...
      {format(weekStartDate, 'EEE, MMM d')} - {format(weekEndDate, 'EEE, MMM d')}}
    </span>
    <div className="cursor-pointer hover:bg-white/20 rounded" data-date-nav style={{ display: 'flex', align...
  </div>
</div>
{!hwFloating.detached ? (
  <button
    onClick={hwFloatingHandlers.onDetach}
    onTouchEnd={() => { e.preventDefault(); hwFloatingHandlers.onDetach(); }}
    className="absolute z-[70] w-5 h-5 rounded-tl-[11px] rounded-br-[4px] rounded-tr-[2px] rounded-bl-[2px]...
    style={{ top: '1px', left: '1px', background: 'rgba(255,255,255,0.15)', border: '1px solid rgba(255,255...
    data-testid="hw-detach-button"
    title="Pop out progress as floating window"
  >
    <Maximize2 className="h-2.5 w-2.5 text-white" />
  </button>
) : (
  <button
    onClick={hwFloatingHandlers.onDock}
    onTouchEnd={() => { e.preventDefault(); hwFloatingHandlers.onDock(); }}
    className="absolute z-[70] w-5 h-5 rounded-tl-[11px] rounded-br-[4px] rounded-tr-[2px] rounded-bl-[2px]...
    style={{ top: '1px', left: '1px', background: 'rgba(255,255,255,0.15)', border: '1px solid rgba(255,255...
    data-testid="hw-dock-button"
    title="Dock progress bars back"
  >
    <Minimize2 className="h-2.5 w-2.5 text-white" />
  </button>
)}
{ /* Joint Resize Handle – controls both calendar+homework width and calendar height */
{!hwFloating.detached && <div
  className="absolute z-[60]"
  style={{ left: '-18px', top: '50%', transform: 'translateY(-50%)', width: '17px', height: '181px', touchA...
  data-testid="resize-handle-homework"
  >
  <div style={{ width: '17px', height: '181px', borderRadius: '0', background: 'rgba(255,255,255,0.55)', bo...
    <div className="cursor-pointer" style={{ flex: 1, display: 'flex', alignItems: 'center', justifyContent...
    <span style={{ width: '6px', height: '1px', background: 'rgba(120,120,120,0.3)', flexShrink: 0 }} />
    <div className="cursor-grab active:cursor-grabbing select-none" style={{ flex: 1, display: 'flex', alig...
    <span style={{ width: '6px', height: '1px', background: 'rgba(120,120,120,0.3)', flexShrink: 0 }} />
    <div className="cursor-pointer" style={{ flex: 1, display: 'flex', alignItems: 'center', justifyContent...
  </div>
</div>
{(() => {
  const allSemDefs: Array<{ letter: string; year: string; semLabel: string; endDate: Date }> = [];
  for (let y = 2026; y <= 2029; y++) {
    allSemDefs.push({ letter: 'W', year: String(y).slice(2), semLabel: 'Winter ' + y, endDate: new Date(y, ...
    if (y < 2029) {
      allSemDefs.push({ letter: 'S', year: String(y).slice(2), semLabel: 'Spring/Summer ' + y, endDate: new...
      allSemDefs.push({ letter: 'F', year: String(y).slice(2), semLabel: 'Fall ' + y, endDate: new Date(y, ...
    }
  }
  const todayDate = new Date();
  const semTabs = allSemDefs.filter(s => s.endDate >= todayDate);
  const currentSemLabel = hwWeeklyTimeline[0]?.semLabel || null;
  const scrollActiveSem = hwVisibleSemLabel || currentSemLabel;
  return (
    <div
      className="absolute"
      style={{() => {
        const boxEl = document.querySelector('[data-testid="section-coming-up"]') as HTMLElement | null;
        const boxH = boxEl ? boxEl.offsetHeight : (window.innerHeight - (calendarBorderTop || (calendarTop ...
        const availH = Math.max(60, boxH - 20);
        return { right: '-16px', top: '10px', pointerEvents: 'auto' as const, zIndex: 0, display: (isSettin...
      }}()
    >
    {(() => {
      const boxEl = document.querySelector('[data-testid="section-coming-up"]') as HTMLElement | null;
      const boxH = boxEl ? boxEl.offsetHeight : (window.innerHeight - (calendarBorderTop || (calendarTop ...
      const availH = Math.max(60, boxH - 20);
      const n = semTabs.length;
      const overlapRatio = 0.4;
      const tabH = availH / (1 + (n - 1) * (1 - overlapRatio));
      const stepPx = tabH * (1 - overlapRatio);
      return semTabs.map((tab, tabIndex) => {
        const isActive = scrollActiveSem === tab.semLabel;
        const reversedIdx = semTabs.length - 1 - tabIndex;
        return (
          <div
            key={tab.semLabel}
            className={`cursor-pointer ${isActive && tabBounceEnabled ? ' semester-tab-bounce' : ''}`}
            style={{ position: 'absolute', bottom: `${reversedIdx * stepPx}px`, width: '18px', height: `${t...
            onClick={() => {
              let idx = -1;
              if (tab.letter === 'W' && tab.year === '26') {
                idx = hwWeeklyTimeline.findIndex(w => w.semLabel === 'Winter 2026' && w.weekNum === 10);
              } else if (tab.letter === 'S' && tab.year === '26') {
                idx = hwWeeklyTimeline.findIndex(w => w.semLabel === 'Winter 2026' && w.weekNum === 11);
              } else if (tab.letter === 'F' && tab.year === '26') {
                idx = hwWeeklyTimeline.findIndex(w => w.semLabel === 'Winter 2026' && w.weekNum === 12);
              } else if (tab.letter === 'W' && tab.year === '27') {
                idx = hwWeeklyTimeline.findIndex(w => w.weekStart.getMonth() === 4 && w.weekStart.getFullYe...
              } else if (tab.letter === 'S' && tab.year === '27') {

```



```

const isFog = wc >= 45 && wc <= 48;
const isOvercast = wc === 3;
const isCloudy = wc >= 2 && wc <= 3;
const isPartlyCloudy = wc === 1 || wc === 2;
const isClear = wc === 0;
const isFreezingRain = wc >= 66 && wc <= 67;
const isSleet = wc >= 85 && wc <= 86;
const particleCount = isSnow ? (wc >= 75 ? 55 : wc >= 73 ? 35 : 20)
  : isRain ? (wc >= 65 || wc >= 82 ? 60 : isDrizzle ? 18 : 35)
  : isSleet ? 30 : isFreezingRain ? 40 : 0;
const showParticles = particleCount > 0;

return (
  <div className="absolute overflow-hidden pointer-events-none" style={{ zIndex: 45, borderRadius: '12px ...
    {isFog && (
      <div className="absolute inset-0" style={{ background: 'rgba(200,200,210,0.15)', backdropFilter: 'b...
    })
    {isThunder && (
      <div className="absolute inset-0 weather-lightning" />
    })
    {showParticles && weatherParticles.slice(0, particleCount).map((p, i) => {
      const size = isSnow ? (2 + p.sizeFactor * 3) : isSleet ? (1.5 + p.sizeFactor * 2) : (1 + p.sizeFact...
      const duration = isSnow ? (3 + p.durationFactor * 4) : isRain ? (0.6 + p.durationFactor * 0.8) : (1...
      const drift = isSnow ? (p.driftFactor * 30 - 15) : 0;
      const opacity = isSnow ? (0.5 + p.opacityFactor * 0.5) : isRain ? (0.3 + p.opacityFactor * 0.4) : 0.5;
      const windAngle = isRain ? 8 : isSleet ? 5 : 0;

      return (
        <div
          key={i}
          style={{
            position: 'absolute',
            left: `${p.left}%`,
            top: '-8px',
            width: `${isSnow ? size : Math.max(1, size * 0.5)}px`,
            height: `${isSnow ? size : size * (isRain ? 8 : 4)}px`,
            borderRadius: isSnow ? '50%' : '1px',
            background: isSnow ? 'white' : isFreezingRain ? 'rgba(180,210,255,0.7)' : 'rgba(170,195,220,0...
            opacity,
            animation: `weatherFall ${duration}s linear ${p.delay}s infinite`,
            transform: `translateX(${drift}px) rotate(${windAngle}deg)`,
          }}
        />
      );
    })
  )
  {isClear || isPartlyCloudy} && weatherData.isDay && (
    <div className="absolute" style={{
      top: '-2px', right: '8px', width: '18px', height: '18px',
      borderRadius: '50%', background: 'radial-gradient(circle, rgba(255,220,80,0.9) 0%, rgba(255,200,5...
      boxShadow: '0 0 12px rgba(255,200,50,0.4)',
      zIndex: 1,
    }} />
  )
  {isOvercast && (
    <div className="absolute inset-0" style={{ background: 'linear-gradient(180deg, rgba(120,130,150,0....
  })
  {isCloudy || isPartlyCloudy} && (
    <
      <div className="weather-cloud" style={{ top: '4px', left: '5%', width: isOvercast ? '65px' : '45p...
      <div className="weather-cloud" style={{ top: '16px', left: '40%', width: isOvercast ? '55px' : '3...
      <div className="weather-cloud" style={{ top: '8px', left: '68%', width: isOvercast ? '50px' : '35...
      {isOvercast && (
        <
          <div className="weather-cloud" style={{ top: '26px', left: '20%', width: '58px', height: '17p...
          <div className="weather-cloud" style={{ top: '34px', left: '55%', width: '48px', height: '15p...
          <div className="weather-cloud" style={{ top: '42px', left: '8%', width: '52px', height: '16px...
          <div className="weather-cloud" style={{ top: '50px', left: '75%', width: '40px', height: '13p...
        </>
      )
    </>
  )
  </div>
);
})();
<div style={{ padding: '0 8px', height: courseRowRects.length > 0 ? `${courseRowRects[0].top - (calendarBor...
  <div style={{ position: 'absolute', top: '15px', left: '15px', right: '22px', height: '0.5px', backgroundColo...
  <span className="text-[10px] font-medium text-white" style={{ position: 'absolute', left: '6px', bottom: ...
  <span className="text-[10px] font-medium text-white" style={{ position: 'absolute', left: `${effectiveDiv...
  {weatherData && (
    <div style={{ position: 'absolute', left: '28px', top: '1px', display: 'flex', alignItems: 'center', ga...
    <span className="text-[9.5px] font-medium" style={{ color: 'rgba(255,255,255,1)' }} data-testid="home...
    <span className="text-[9.5px]" style={{ color: 'rgba(255,255,255,1)' }} data-testid="homework-weather...
  </div>
  )
  {weatherData && (weatherData.sunrise || weatherData.sunset) && (
    <div style={{ position: 'absolute', right: '15px', top: '2px', display: 'flex', alignItems: 'center', g...
    {weatherData.sunrise && <span className="text-[9.5px]" data-testid="homework-sunrise"><span style={{ ...
    {weatherData.sunset && <span className="text-[9.5px]" style={{ marginLeft: '14px' }} data-testid="hom...
  </div>
  )
  <div ref={homeworkSpacerRef} style={{ position: 'absolute', right: '4px', top: 0, width: '0px', height: '...'
</div>
{courseRowRects.length > 0 && courseProgressDataRef.current.length > 0 && (() => {
  const upcomingTop = calendarBorderTop || (calendarTop + 15);
  const firstRowOffset = 0;
  const leftWidth = (() => {
    if (homeworkSpacerRef.current && homeworkSectionRef.current) {
      const spacerRect = homeworkSpacerRef.current.getBoundingClientRect();
      const sectionRect = homeworkSectionRef.current.getBoundingClientRect();
      return `${spacerRect.left - sectionRect.left}px`;
    }
  }
  return '70px';
})();
const rightWidth = (() => {
  if (homeworkSpacerRef.current && homeworkSectionRef.current) {
    const spacerRect = homeworkSpacerRef.current.getBoundingClientRect();
    const sectionRect = homeworkSectionRef.current.getBoundingClientRect();
    return `${sectionRect.right - spacerRect.left}px`;
  }
})();

```



```

const isToday = daysUntil <= 0;
const isThroughFriday = !isToday && tDue <= thisWeekFridayEnd;
const isNextWeek = tDue >= nextSaturday && tDue <= nextWeekEnd;
const is2Weeks = tDue >= twoWeeksStart && tDue <= threeWeeksEnd;
const barColor = isToday ? '#ef4444' : isThroughFriday ? '#eab300' : isNextWeek ? (daysUn...
return (
  <div className="flex-shrink-0 flex items-center justify-end gap-1" style={{ marginLeft:...
    <div style={{ width: '56px', position: 'relative', height: '5px', flexShrink: 0 }}>
      <div style={{ position: 'absolute', top: 0, right: 0, width: '56px', height: '5px',...
        <div className="rounded-full" style={{ position: 'absolute', top: 0, right: 0, widt...
      </div>
      <span style={{ fontSize: '8px', color: 'ffffff', fontWeight: 400, minWidth: '16px', ...
      <span style={{ fontSize: '9px', color: dateColor, fontWeight: 400, minWidth: '42px', ...
    </div>
  );
  })()
</div>
);
return (
  <div data-scroll-tasks style={{ flex: 1, minHeight: 0, overflowY: 'auto', scrollbarWidth: 'none...
    {courseTasks.map((t, tIdx) => renderCourseTask(t, tIdx))}
  </div>
);
  })()
</div>,
);
});
const hwBlackBg = () => {
  const validRects = courseRowRects.filter((_, i) => courseProgressDataRef.current[i]);
  if (validRects.length === 0) return null;
  const firstTop = validRects[0].top - upcomingTop - firstRowOffset;
  const lastRect = validRects[validRects.length - 1];
  const lastBottom = lastRect.top + lastRect.height - upcomingTop - firstRowOffset;
  return (
    <div key="hw-black-bg" style={{
      position: 'absolute',
      top: `${firstTop}px`,
      left: 0,
      width: `${effectiveDividerPct}%`,
      height: `${lastBottom - firstTop}px`,
      background: 'ffffff',
      zIndex: 39,
      borderRadius: '6px',
    }} />
  );
  })();
const courseRows = courseProgressDataRef.current.map((pd, idx) => {
  if (!pd || !courseRowRects[idx]) return null;
  const rowTop = courseRowRects[idx].top;
  const rowHeight = courseRowRects[idx].height;
  const offsetFromUpcoming = rowTop - upcomingTop - firstRowOffset;
  const getProgressColor = (percent: number) => {
    if (percent === 100) return '#22c55e';
    if (percent > 0) return '#f97316';
    return '#ef4444';
  };
  return (
    <div
      key={pd.courseCode}
      style={{
        position: 'absolute',
        top: `${offsetFromUpcoming}px`,
        left: 0,
        width: `${effectiveDividerPct}%`,
        height: `${rowHeight}px`,
        background: 'transparent',
        zIndex: 40,
        display: 'flex',
        flexDirection: 'column',
        overflow: 'visible',
        borderBottom: '0.5px solid rgba(255,255,255,0.2)',
      }}
    >
      {pd.hasNoData ? (
        <div className="flex-1 flex items-center justify-center">
          <span className="text-[9px] font-bold text-black/60 text-center" style={{ lineHeight: '1.6' }}>...
        </div>
      ) : (
        <div style={{ display: 'flex', flexDirection: 'row', alignItems: 'stretch', flex: 1, borderRadius...
          {() => { const cGrad = getCourseGradientColors(pd.courseCode); return [
            { type: 'module' as const, label: 'Module', p: pd.moduleP, unread: pd.moduleUnread, play: pd....
            { type: 'reading' as const, label: 'Reading', p: pd.readingP, unread: pd.readingUnread, play:...
          ]}().map(item => {
            const circleSize = 40;
            const strokeWidth = 3.5;
            const radius = (circleSize - strokeWidth) / 2;
            const circumference = 2 * Math.PI * radius;
            const offset = circumference - (item.p.percent / 100) * circumference;
            const isModule = item.type === 'module';
            const textColor = item.dark ? 'fff' : '#000';
            const dragKey = `${pd.courseCode}-${item.type}`;
            const isDragOver = hwDragOverTarget === dragKey;
            return (
              <div key={item.type}
                onDragOver={(e) => { e.preventDefault(); e.stopPropagation(); e.dataTransfer.dropEffect = ...
                onDragEnter={(e) => { e.preventDefault(); e.stopPropagation(); setHwDragOverTarget(dragKe...
                onDragLeave={(e) => { e.preventDefault(); e.stopPropagation(); if (hwDragOverTarget === d...
                onDrop={(e) => { e.preventDefault(); e.stopPropagation(); setHwDragOverTarget(null); cons...
                style={{ display: 'flex', flexDirection: 'column', alignItems: 'center', justifyContent: ...
                data-testid={ `drop-${item.type}-${pd.courseCode.toLowerCase()}` }>
                  <span style={{ position: 'absolute', top: '2px', left: 0, right: 0, textAlign: 'center', ...
                <div style={{ display: 'flex', alignItems: 'center', gap: '3px', marginTop: '8px' }}>
                  <div style={{ position: 'relative', width: circleSize, height: circleSize, marginLeft: ...
                    <svg width={circleSize} height={circleSize} style={{ transform: 'rotate(-90deg)' }}>
                      <circle cx={circleSize/2} cy={circleSize/2} r={radius} fill="none" stroke="rgba(255...
                    {item.p.hasFiles && item.p.percent > 0 && (

```



```

        <span className="truncate" style={{ fontSize: '9px', color: '#000000', fontWeight: 400, fle...
        <div className="flex-shrink-0 flex items-center justify-end gap-1" style={{ marginLeft: 'au...
        <div style={{ width: '56px', flexShrink: 0 }} />
        <span style={{ fontSize: '8px', color: 'transparent', fontWeight: 400, minWidth: '16px', ...
        <span style={{ fontSize: '9px', color: '#000000', fontWeight: 400, minWidth: '42px', text...
      </div>
    </div>
  );
});
</div>
</div>
);
}
}

return rows;
})();
<div className="flex-1 flex flex-col" style={{ paddingLeft: '0px', paddingRight: '0px', marginTop: (() => {
  const upcomingTop = calendarBorderTop || (calendarTop + 15);
  if (courseRowReacts.length > 0) {
    const firstOffset = 0;
    const lastRect = courseRowReacts[courseRowReacts.length - 1];
    if (lastRect) {
      const lastBottom = lastRect.top + lastRect.height - upcomingTop - firstOffset;
      const otherRowH = gridSizes.otherRowHeight || 57;
      return `${lastBottom + otherRowH + 2 - 49}px`;
    }
  }
  return '12px';
}}), flex: 1, paddingBottom: '0px', overflowY: 'auto', scrollbarWidth: 'none', position: 'relative', zInde...
<div style={{ width: '100%', height: '15px', backgroundColor: colorSettings.headerBar, borderRadius: '2px',...
  <span style={{ fontSize: '8px', fontWeight: 700, color: '#ffffff', letterSpacing: '0.7px', lineHeight: 1 ...
</div>
<div style={{ flex: 1, overflow: 'auto', scrollbarWidth: 'none', position: 'relative' }} ref={homeworkScro...
  {isLoading ? (
    <div className="flex-1 flex items-center justify-center text-white/60 text-xs">Loading...</div>
  ) : (
    <div ref={hwTimelineDividerRef} style={{ display: 'flex', flexDirection: 'column', minHeight: 0, positi...
      <div
        style={{ position: 'absolute', top: 0, bottom: 0, left: `${1 + hwGroupBarWidth}px`, width: '8px', m...
        data-testid="hw-group-bar-handle"
      />
      <div data-homework-section="today" style={{ display: 'flex', flexDirection: 'column', gap: '0px', pad...
        <div style={{ display: 'flex', alignItems: 'center', gap: '4px' }}>
          <span className="text-[12px]" style={{ color: '#FFFF00' }}>Today</span>
          <span className="text-[9px] font-normal" style={{ color: 'rgba(255,255,255,0.5)' , lineHeight: '14...
        </div>
      </div>
      {dueTodayTasks.length === 0 ? (
        <div className="text-[10px] text-white/50 text-center" style={{ padding: '4px 0' }}>No tasks due to...
      ) : (
        <div data-hw-group-row style={{ display: 'flex', alignItems: 'flex-start', marginTop: '4px' }}>
          <div data-hw-group-bar style={{ width: `${hwGroupBarWidth}px`, flexShrink: 0, display: 'flex', fl...
            <span className="text-[9px] font-medium" style={{ color: '#ffffff', marginBottom: '2px' }}>
              {format(new Date(), 'MMMM d')}
            </span>
            <div style={{ display: 'flex', flexDirection: 'column', gap: '2px', width: '100%' }}>
              <div style={{ display: 'flex', gap: '2px', padding: '0 2px', width: '100%', justifyContent: '...
                {[ 'Su', 'Mo', 'Tu', 'We', 'Th', 'Fr', 'Sa' ].map((dl, dli) => {
                  <div key={dli} style={{ flex: 1, minWidth: 0, fontSize: '8px', fontWeight: 600, textAlig...
                })
              </div>
            </div>
          <div>
            {(() => {
              const todayDate = startOfDayET(new Date());
              const weekStart = startOfWeek(todayDate, { weekStartsOn: 0 });
              const days = eachDayOfInterval({ start: weekStart, end: addDays(weekStart, 6) });
              const dueDates = dueTodayTasks.map(t => ({ date: startOfDayET(new Date(t.dueDate)), courseC...
              return (
                <div style={{ display: 'flex', gap: '2px', padding: '1px 2px', width: '100%', justifyCont...
                  {days.map((d, di) => {
                    const isToday = isSameDayET(d, todayDate);
                    const dueMatch = dueDates.find(dd => isSameDayET(d, dd.date));
                    const isDue = !!dueMatch;
                    const dueColor = dueMatch ? getCourseGradientColors(dueMatch.courseCode).end : '';
                    return (
                      <div key={di} onClick={() => { const wk = weeks.find(w => { const s = parseISO(w.st...
                        flex: 1, minWidth: 0, height: '11px', borderRadius: '2px', fontSize: '8px', fontW...
                        display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: 'pointer',
                        color: (isToday || isDue) ? '#fff' : 'rgba(255,255,255,0.85)',
                        background: isToday ? todayCellBg : isDue ? dueColor : 'rgba(255,255,255,0.15)',
                        border: isDue && !isToday ? '1px solid ${dueColor}' : 'none',
                      >> data-testid={ `mini-cal-today-${format(d, 'yyyy-MM-dd')}` }>
                        {d.getDate()}
                      </div>
                    )
                  })
                </div>
              </div>
            </div>
          </div>
        </div>
      <div style={{ display: 'flex', alignItems: 'stretch', flexShrink: 0, width: '10px', marginLeft: '...'
        <div style={{ width: '3px', height: '100%', borderRadius: '2px', background: 'linear-gradient(1...
      </div>
    <div style={{ flex: 1, minWidth: 0, marginRight: '-7px' }}><div style={{ maxHeight: '80px', overf...
    <div className="flex flex-col gap-0.5">
      {dueTodayTasks.map((task, tIdx) => {
        const progressBarWidth = getProgressBarWidth(task);
        const progressColor = getProgressColor(task, 'today');
        const daysUntil = differenceInCalendarDays(new Date(task.dueDate), new Date());
        const courseName = task.courseName?.split(' - ').slice(1).join(' - ') || task.courseName?.split...
        const taskCourseCode = task.courseName?.split(' - ')[0]?.toUpperCase() || '';
        const cfp = taskCourseCode ? calcCourseFileProgress(taskCourseCode) : null;
        const hwPdfUrl = task.attachments?.length ? (() => { for (const att of task.attachments) { cons...
        return (
          <div key={task.id} className="" style={{ position: 'relative', overflow: 'visible', borderBot...
            onMouseEnter={() => setHoveredCountdownTaskIdDebounce(task.id)}
          </div>
        )
      })
    </div>
  </div>
);
}
}

```



```

const weeks: Date[] = [];
let ws = todayWeekStart;
while (ws <= dueWeekStart) { weeks.push(ws); ws = addDays(ws, 7); }
if (weeks.length > 4) weeks.splice(0, weeks.length - 4);
groupMap.set(key, groups.length);
groups.push({ key, tasks: [task], weeks });
}
});
return groups.map((group, groupIdx) => {
const dueDates = group.tasks.map(t => ({ date: startOfDayET(new Date(t.dueDate)), courseCode:...
return (
<div key={group.key} data-hw-group-row style={{ display: 'flex', alignItems: 'flex-start', ...
<div data-hw-group-bar style={{ width: `${hwGroupBarWidth}px`, flexShrink: 0, display: 'f...
<div style={{ flex: 1, display: 'flex', flexDirection: 'column', justifyContent: 'cente...
<div style={{ display: 'flex', gap: '2px', padding: '0 2px', marginBottom: '3px', wid...
[[ 'Su', 'Mo', 'Tu', 'We', 'Th', 'Fr', 'Sa' ].map((dl, dli) => {
<div key={dli} style={{ flex: 1, minWidth: 0, fontSize: '8px', fontWeight: 600, t...
}))
</div>
</div>
{group.weeks.map((weekStart: Date, wi: number) => {
const isLastWeek = wi === group.weeks.length - 1;
const days = eachDayOfInterval({ start: weekStart, end: addDays(weekStart, 6) });
return (
<div key={wi} style={{ display: 'flex', gap: '2px', padding: '1px 2px', width: '1...
{days.map((d, di) => {
const isToday = isSameDayET(d, today);
const dueMatch = dueDates.find(dd => isSameDayET(d, dd.date));
const isDue = !!dueMatch;
const dueColor = dueMatch ? (dynamicCourseColors[dueMatch.courseCode]??.hex ||...
return (
<div key={di} onClick={() => { const wk = weeks.find(w => { const s = parse...
flex: 1, minWidth: 0, height: '11px', borderRadius: '2px', fontSize: '8px...
display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: ...
color: (isToday || isDue) ? '#fff' : 'rgba(255,255,255,0.85)',
background: isToday ? todayCellBg : isDue ? dueColor : 'rgba(255,255,255,...
border: isDue && !isToday ? '1px solid ${dueColor}` : 'none',
}} data-testid={`mini-cal-w10-date-${format(d, 'yyyy-MM-dd')}`}>
{d.getDate()}
</div>
);
}))
</div>
);
});
</div>
);
</div>
<div style={{ display: 'flex', alignItems: 'stretch', flexShrink: 0, width: '10px', margi...
<div style={{ width: '3px', height: '100%', borderRadius: '2px', background: 'linear-gr...
</div>
<div style={{ flex: 1, minWidth: 0, marginRight: '-7px' }}>
<div className="flex flex-col gap-0.5">
{group.tasks.filter((t, i, arr) => t.type !== "class" || arr.findIndex(x => x.type === ...
const progressColor = getProgressColor(task, 'tomorrow');
const daysUntil = differenceInCalendarDays(new Date(task.dueDate), new Date());
const courseName = task.courseName?.split(' - ').slice(1).join(' - ') || task.courseN...
const taskCourseCode = task.courseName?.split(' - ')[0]?.toUpperCase() || '';
const cfp = taskCourseCode ? calcCourseFileProgress(taskCourseCode) : null;
const hwPdfUrl = task.attachments?.length ? (() => { for (const att of task.attachmen...
const isLastTask = taskIdx === group.tasks.length - 1;
return (
<div key={task.id} className="" style={{ position: 'relative', overflow: 'visible',...
onMouseEnter={() => setHoveredCountdownTaskIdDebounce(task.id)}
onMouseLeave={() => setHoveredCountdownTaskIdDebounce(null)}
ref={rowEl} => {
if (!rowEl || rowEl.dataset.swipeInit) return;
rowEl.dataset.swipeInit = '1';
const delEl = rowEl.querySelector('[data-swipe-delete]') as HTMLDivElement;
const resEl = rowEl.querySelector('[data-swipe-reschedule]') as HTMLDivElement;
const contentEl = rowEl.querySelector('[data-swipe-content]') as HTMLDivElement;
if (delEl && resEl && contentEl) {
const ctrl = swipeableRow(rowEl, contentEl, delEl, resEl);
delEl.addEventListener('click', (e) => { e.stopPropagation(); if (window.conf...
resEl.addEventListener('click', (e) => { e.stopPropagation(); ctrl.reset(); s...
}
})
>
<div data-swipe-delete data-testid={`swipe-delete-${task.id}`} style={{ position:...
<span className="text-white text-[9px] font-bold">Delete</span>
</div>
<div data-swipe-reschedule data-testid={`swipe-reschedule-${task.id}`} style={{ p...
<span className="text-white text-[9px] font-bold">Reschedule</span>
</div>
<div data-swipe-content style={{ position: 'relative', zIndex: 2, background: 'tr...
<div data-box-task-id={task.id} style={{ display: 'flex', gap: '2px', alignItem...
<div ref={centerCircleOnGroupBar} style={{ position: 'absolute', left: '86px'...
<span style={{ fontSize: '7px', fontWeight: 700, color: '#1a1a2e', lineHeig...
<span style={{ fontSize: '9px', fontWeight: 700, color: '#1a1a2e', lineHeig...
</div>
<div style={{ minWidth: 0, flex: 1, display: 'flex', flexDirection: 'column',...
<div>
<button
className="text-[10px] truncate hover:underline cursor-pointer leading-...
onClick={() => setEditingTask(task)}
data-testid={`task-link-tomorrow-${task.id}`}
data-upcoming-task-name
style={{ textAlign: 'left', fontWeight: task.type === 'class' ? 700 : 4...
>
{task.type === 'discussion' || /discussion/i.test(task.title)} ? `${((...
</button>
<div className="text-[9px] text-white" style={{ display: 'flex', alignIte...
<span style={{ overflow: 'hidden', textOverflow: 'ellipsis' }}>{courseN...
</div>
</div>
<div>
{cfp && (cfp.moduleP.hasFiles || cfp.readingP.hasFiles) && (task.type === '...
<div style={{ display: 'flex', gap: '6px', padding: '2px', flexWrap: '...
{cfp.moduleP.hasFiles && (
<div style={{ display: 'flex', alignItems: 'center', gap: '2px', flex...
<span className="text-[11px] text-white font-medium" style={{ margi...

```



```

return showYearSep ? (
  <div style={{ display: 'flex', alignItems: 'center', gap: '6px', margin: '18px 0 4px 0'...
    <div style={{ flex: 1, height: '1px', background: 'linear-gradient(90deg, transparent...
      <span className="text-[10px] font-bold tracking-widest" style={{ color: 'rgba(255,255...
      <div style={{ flex: 1, height: '1px', background: 'linear-gradient(90deg, transparent...
    </div>
  ) : null;
})();
){
  <div data-semester-label={tlEntry.semLabel || ''} style={{ display: 'flex', alignItems: '...
    <div style={{ display: 'flex', alignItems: 'center', gap: '4px' }}>
      <span className="text-[12px] font-semibold" style={{ color: 'ffffff' }}>{groupWeekLa...
      <span className="text-[9px] font-normal" style={{ color: 'rgba(255,255,255,0.5)', lin...
    </div>
  </div>
)}
<div data-hw-group-row style={{ display: 'flex', alignItems: 'flex-start', marginBottom: '2...
  <div data-hw-group-bar style={{ width: `${hwGroupBarWidth}px`, flexShrink: 0, display: 'f...
    {/Week\s+\d/i.test(groupWeekLabel) && {
      <div style={{ display: 'flex', alignItems: 'center', gap: '2px', flexShrink: 0, justify...
        <div style={{ width: '16px', height: '18px', borderRadius: '2px', border: '1.5px soli...
          <div style={{ background: 'rgb(100, 100, 100)', textAlign: 'center', fontSize: '5px'...
          <div style={{ flex: 1, background: 'white', textAlign: 'center', fontSize: '9px', f...
        </div>
        <span style={{ fontSize: '6px', color: 'white', lineHeight: 1 }}>&#9654;</span>
        <div style={{ width: '16px', height: '18px', borderRadius: '2px', border: '1.5px soli...
          <div style={{ background: 'rgb(100, 100, 100)', textAlign: 'center', fontSize: '5px'...
          <div style={{ flex: 1, background: 'white', textAlign: 'center', fontSize: '9px', f...
        </div>
      </div>
    )}
    {groupSublabel && !/Week\s+\d/i.test(groupWeekLabel) && {
      <span className="text-[9px] font-medium" style={{ color: 'ffffff', marginBottom: '2px'...
        {groupSublabel}
      </span>
    )}
  <div style={{ flex: 1, display: 'flex', flexDirection: 'column', justifyContent: 'cente...
    <div style={{ display: 'flex', gap: '2px', padding: '0 2px', marginBottom: '3px', wid...
      {[ 'Su', 'Mo', 'Tu', 'We', 'Th', 'Fr', 'Sa' ].map((dl, dli) => {
        <div key={dli} style={{ flex: 1, minWidth: 0, fontSize: '8px', fontWeight: 600, t...
      })}
    </div>
    {group.weeks.map((weekStart: Date, wi: number) => {
      const isLastWeek = wi === group.weeks.length - 1;
      const days = eachDayOfInterval({ start: weekStart, end: addDays(weekStart, 6) });
      return (
        <div key={wi} style={{ display: 'flex', gap: '2px', padding: '1px 2px', width: '1...
          {days.map((d, di) => {
            const isToday = isSameDayET(d, today);
            const dueMatch = dueDates.find(dd => isSameDayET(d, dd.date));
            const isDue = !!dueMatch;
            const dueColor = dueMatch ? (dynamicCourseColors[dueMatch.courseCode]?.hex ||...
            return (
              <div key={di} onClick={() => { const wk = weeks.find(w => { const s = parse...
                flex: 1, minWidth: 0, height: '11px', borderRadius: '2px', fontSize: '8px...
                display: 'flex', alignItems: 'center', justifyContent: 'center', cursor: ...
                color: (isToday || isDue) ? '#fff' : 'rgba(255,255,255,0.85)',
                background: isToday ? todayCellBg : isDue ? dueColor : 'rgba(255,255,255,...
                border: isDue && !isToday ? '1px solid ${dueColor}' : 'none',
              )} data-testid={`mini-cal-date-${format(d, 'yyyy-MM-dd')}`}>
                {d.getDate()}
              </div>
            );
          })}
        </div>
      );
    })}
  </div>
);
</div>
)}
</div>
<div style={{ display: 'flex', alignItems: 'stretch', flexShrink: 0, width: '10px', margi...
  <div style={{ width: '3px', height: '100%', borderRadius: '2px', background: 'linear-gr...
  </div>
<div style={{ flex: 1, minWidth: 0, marginRight: '-7px' }}>
  {group.tasks.filter((t, i, arr) => t.type !== "class" || arr.findIndex(x => x.type === ...
    const daysUntil = differenceInCalendarDays(new Date(task.dueDate), new Date());
    const progressColor = daysUntil <= 1 ? '#ef4444' : daysUntil <= 3 ? '#f97316' : daysU...
    const courseName = task.courseName?.split(' - ').slice(1).join(' - ') || task.courseN...
    const taskCourseCode = task.courseName?.split(' - ')[0]??.toUpperCase() || '';

    const hwPdfUrl = task.attachments?.length ? (() => { for (const att of task.attachmen...
    const isLastTask = taskIdx === group.tasks.length - 1;
    return (
      <div key={task.id} className="" style={{ position: 'relative', overflow: 'visible',...
        onMouseEnter={() => setHoveredCountdownTaskIdDebounce(task.id)}
        onMouseLeave={() => setHoveredCountdownTaskIdDebounce(null)}
        ref={rowEl} => {
          if (!rowEl || rowEl.dataset.swipeInit) return;
          rowEl.dataset.swipeInit = '1';
          const delEl = rowEl.querySelector('[data-swipe-delete]') as HTMLDivElement;
          const resEl = rowEl.querySelector('[data-swipe-reschedule]') as HTMLDivElement;
          const contentEl = rowEl.querySelector('[data-swipe-content]') as HTMLDivElement;
          if (delEl && resEl && contentEl) {
            const ctrl = swipeableRow(rowEl, contentEl, delEl, resEl);
            delEl.addEventListener('click', (e) => { e.stopPropagation(); if (window.conf...
            resEl.addEventListener('click', (e) => { e.stopPropagation(); ctrl.reset(); s...
          }
        })
      </div>
    )
  >
  <div data-swipe-delete data-testid={`swipe-delete-3w-${task.id}`} style={{ positi...
    <span className="text-white text-[9px] font-bold">Delete</span>
  </div>
  <div data-swipe-reschedule data-testid={`swipe-reschedule-3w-${task.id}`} style={...
    <span className="text-white text-[9px] font-bold">Reschedule</span>
  </div>
  <div data-swipe-content style={{ position: 'relative', zIndex: 2, background: 'tr...
    <div data-box-task-id={task.id} style={{ display: 'flex', gap: '2px', alignItem...
      <div ref={centerCircleOnGroupBar} style={{ position: 'absolute', left: '86px'...
        <span style={{ fontSize: '7px', fontWeight: 700, color: '#1a1a2e', lineHeig...

```



```

    >
    {dashboardCommentSaving ? 'Saving...' : 'Save'}
  </button>
</div>
</div>,
document.body
))
<div style={{ display: 'none' }} data-task-boxes-container="true" />
</div>
);
})();
}/* To Do Bottom Flyout - Burst from bottom */}
<div
  className={`fixed transition-[opacity,transform] ease-out ${isTodoFlyoutOpen ? 'opacity-100 scale-100' : 'o...
  style={{
    width: '900px',
    height: '85vh',
    top: '50%',
    left: '50%',
    transform: isTodoFlyoutOpen ? 'translate(-50%, -50%) scale(1)' : 'translate(-50%, -50%) scale(0)',
    transformOrigin: '50% calc(50vh + 42.5vh)',
    transitionDuration: '400ms',
    zIndex: getFlyoutZIndex('todo')
  }}
  onClick={() => bringFlyoutToFront('todo')}
>
  {/* Flyout content */}
  <section
    className="h-full overflow-hidden flex flex-col rounded-xl text-white"
    style={{
      background: `linear-gradient(180deg, ${colorSettings.mainBackground} 0%, color-mix(in srgb, ${colorSett...
      border: '1px solid rgba(255, 255, 255, 0.2)',
      boxShadow: '0 -10px 40px rgba(0,0,0,0.3)'
    }}
    data-testid="section-todo"
  >
    <div className="flex items-center justify-between px-4 py-3 border-b border-white/40 flex-shrink-0 rounde...
      <div className="flex items-center gap-2">
        <Bell className="text-white" style={{ width: '15px', height: '15px' }} />
        <h2 className="font-normal text-white" style={{ fontFamily: 'Avenir', 'Avenir Next', -apple-system, Bl...
          REMINDERS ((allTasks || []).filter(t => t.type === 'reminder' && !t.isCompleted).length)
        </h2>
      </div>
    </div>
    </div>
    <div className="flex-1 p-4 overflow-y-auto" style={{ scrollbarWidth: 'thin' }}>
      <div className="mb-3 flex items-end gap-2">
        <div className="flex-1">
          <input
            type="text"
            className="w-full text-xs px-2 py-1.5 rounded bg-white/10 text-white placeholder:text-white/40 fo...
            placeholder="New reminder..."
            value={newReminderTitle}
            onChange={(e) => setNewReminderTitle(e.target.value)}
            onKeyDown={(e) => {
              if (e.key === 'Enter' && newReminderTitle.trim()) {
                const tomorrow = new Date();
                tomorrow.setDate(tomorrow.getDate() + 1);
                tomorrow.setHours(9, 0, 0, 0);
                fetch('/api/tasks', {
                  method: 'POST',
                  headers: { 'Content-Type': 'application/json' },
                  credentials: 'include',
                  body: JSON.stringify({
                    title: newReminderTitle.trim(),
                    type: 'reminder',
                    dueDate: newReminderDueDate || tomorrow.toISOString(),
                    weekNumber: getWeekNumber(newReminderDueDate ? new Date(newReminderDueDate) : tomorrow),
                    isAcknowledged: false,
                    priority: 'medium',
                  }),
                }).then(() => {
                  queryClient.invalidateQueries({ queryKey: ['/api/tasks'] });
                  setNewReminderTitle('');
                  setNewReminderDueDate('');
                });
              }
            })
          </div>
          data-testid="input-new-reminder"
        </div>
        <div>
          <input
            type="datetime-local"
            className="text-xs px-2 py-1.5 rounded bg-white/10 text-white focus:outline-none focus:ring-1 foc...
            style={{ colorScheme: 'dark', fontSize: '10px' }}
            value={newReminderDueDate}
            onChange={(e) => setNewReminderDueDate(e.target.value)}
            data-testid="input-reminder-due-date"
          </div>
        </div>
        <Button
          type="button"
          variant="outline"
          className="border !border-white/50 text-white hover:text-white hover:!border-white transition-opaci...
          style={{ background: 'rgba(10,15,30,0.85)', boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgb...
          disabled={!newReminderTitle.trim()}
          onClick={() => {
            if (!newReminderTitle.trim()) return;
            const tomorrow = new Date();
            tomorrow.setDate(tomorrow.getDate() + 1);
            tomorrow.setHours(9, 0, 0, 0);
            fetch('/api/tasks', {
              method: 'POST',
              headers: { 'Content-Type': 'application/json' },
              credentials: 'include',
              body: JSON.stringify({
                title: newReminderTitle.trim(),
                type: 'reminder',

```

```

        dueDate: newReminderDueDate || tomorrow.toISOString(),
        weekNumber: getWeekNumber(newReminderDueDate ? new Date(newReminderDueDate) : tomorrow),
        isAcknowledged: false,
        priority: 'medium',
    })),
    }).then(() => {
        queryClient.invalidateQueries({ queryKey: ['/api/tasks'] });
        setNewReminderTitle('');
        setNewReminderDueDate('');
    });
    })
    data-testid="button-add-reminder"
  >
  <Plus className="w-3 h-3 mr-1" />
  Add
</Button>
</div>
{(() => {
    const reminderTasks = (allTasks || []).filter(t => t.type === 'reminder').sort((a, b) => {
        if (a.isCompleted !== b.isCompleted) return a.isCompleted ? 1 : -1;
        return new Date(a.dueDate).getTime() - new Date(b.dueDate).getTime();
    });
    if (reminderTasks.length === 0) {
        return (
            <div className="flex items-center justify-center h-32 text-white/40 text-sm">
                No reminders yet – add one above
            </div>
        );
    }
    return (
        <div className="space-y-2">
            {reminderTasks.map(reminder => {
                const isOverdue = !reminder.isCompleted && new Date(reminder.dueDate) < new Date();
                const isUnacked = !(reminder as any).isAcknowledged;
                return (
                    <div
                        key={reminder.id}
                        className={flex items-center gap-2 p-2 rounded-lg transition-all ${reminder.isCompleted ? ...
                        style={{ background: isOverdue ? 'rgba(220, 38, 38, 0.15)' : 'rgba(255, 255, 255, 0.08)' }}
                        data-testid={`reminder-item-${reminder.id}`}
                    >
                        <input
                            type="checkbox"
                            className="w-4 h-4 rounded border-white/40 bg-transparent accent-green-500 flex-shrink-0"
                            checked={!!reminder.isCompleted}
                            onChange={(e) => {
                                fetch(`/api/tasks/${reminder.id}/complete`, {
                                    method: 'PATCH',
                                    headers: { 'Content-Type': 'application/json' },
                                    credentials: 'include',
                                    body: JSON.stringify({ isCompleted: e.target.checked }),
                                }).then(() => {
                                    queryClient.invalidateQueries({ queryKey: ['/api/tasks'] });
                                });
                            }}
                        >
                        data-testid={`checkbox-reminder-${reminder.id}`}
                    </div>
                    <div className="flex-1 min-w-0">
                        <div className={text-xs ${reminder.isCompleted ? 'line-through text-white/50' : 'text-wh...
                        {reminder.title}
                    </div>
                    <div className={text-[10px] flex items-center gap-1 ${isOverdue ? 'text-red-400' : 'text...
                        <Clock className="w-2.5 h-2.5" />
                        {format(new Date(reminder.dueDate), 'MMM d, yyyy h:mm a')}
                        {isOverdue && <span className="text-red-400 font-medium">OVERDUE</span>}
                    </div>
                    </div>
                    <div className="flex items-center gap-1 flex-shrink-0">
                        {isUnacked && !reminder.isCompleted && (
                            <button
                                className="text-red-400 hover:text-red-300 transition-colors p-1"
                                title="Acknowledge"
                                onClick={() => {
                                    fetch(`/api/tasks/${reminder.id}`, {
                                        method: 'PATCH',
                                        headers: { 'Content-Type': 'application/json' },
                                        credentials: 'include',
                                        body: JSON.stringify({ isAcknowledged: true }),
                                    }).then(() => {
                                        queryClient.invalidateQueries({ queryKey: ['/api/tasks'] });
                                    });
                                }}
                            >
                            data-testid={`button-ack-reminder-${reminder.id}`}
                        >
                        <BellOff className="h-3 w-3" />
                    </button>
                )}
            {!reminder.isCompleted && (
                <button
                    className="text-white/40 hover:text-teal-300 transition-colors p-1"
                    title="Send email reminder"
                    onClick={() => {
                        fetch('/api/email/reminder', {
                            method: 'POST',
                            headers: { 'Content-Type': 'application/json' },
                            credentials: 'include',
                            body: JSON.stringify({ taskId: reminder.id }),
                        }).then(res => {
                            if (res.ok) {
                                toast({ title: 'Reminder sent', description: 'Email reminder sent for "${remind...
                                } else {
                                    toast({ title: 'Failed', description: 'Could not send reminder email', variant:...
                                }
                            }
                        });
                    }}
                >
                data-testid={`button-email-reminder-${reminder.id}`}
            >
            <Mail className="h-3 w-3" />

```

```

        </button>
      ))
      <button
        className="text-white/40 hover:text-white/80 transition-colors p-1"
        title="Edit"
        onClick={() => {
          setEditingTask(reminder);
          setIsTodoFlyoutOpen(false);
        }}
        data-testid={`button-edit-reminder-${reminder.id}`}
      >
        <Pencil className="h-3 w-3" />
      </button>
      <button
        className="text-white/40 hover:text-red-400 transition-colors p-1"
        title="Delete"
        onClick={() => {
          deleteTaskWithUndo(reminder.id);
        }}
        data-testid={`button-delete-reminder-${reminder.id}`}
      >
        <Trash2 className="h-3 w-3" />
      </button>
    </div>
  </div>
);
  })
}
</div>
);
  })
}
</div>
<div className="px-4 py-3 border-t border-white/20 flex items-center justify-end flex-shrink-0" style={{ ...
  <Button
    variant="outline"
    onClick={() => setIsTodoFlyoutOpen(false)}
    className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-transpare...
    style={{
      boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255,255,25...
      fontSize: '12px'
    }}
    data-testid="button-close-reminders"
  >
    Close
  </Button>
</div>
</section>
</div>
{ /* Projects Flyout - Burst from Left */ }
<div
  className={`fixed transition-[opacity,transform] ease-out ${isProjectsFlyoutOpen ? 'opacity-100 scale-100' ...
  style={{
    width: '900px',
    height: '85vh',
    top: '50%',
    left: '50%',
    transform: isProjectsFlyoutOpen ? 'translate(-50%, -50%) scale(1)' : 'translate(-50%, -50%) scale(0)',
    transformOrigin: 'calc(-50vw + 450px) 50%',
    transitionDuration: '400ms',
    zIndex: getFlyoutZIndex('projects')
  }}
  onClick={() => bringFlyoutToFront('projects')}
>
  { /* Flyout Panel */ }
  <div
    className="h-full overflow-hidden flex flex-col rounded-xl text-white"
    style={{
      fontFamily: "Avenir, 'Avenir Next', -apple-system, BlinkMacSystemFont, sans-serif",
      background: `linear-gradient(180deg, ${colorSettings.mainBackground} 0%, color-mix(in srgb, ${colorSett...
      border: '1px solid rgba(255, 255, 255, 0.2)',
      boxShadow: '10px 0 40px rgba(0,0,0,0.3)'
    }}
  >
    <div className="flex items-center justify-between px-4 py-3 border-b border-white/40 flex-shrink-0 roun...
      <div className="flex items-center gap-2">
        <FolderOpen className="text-white" style={{ width: '15px', height: '15px' }} />
        <h2 className="font-normal text-white" style={{ fontFamily: "Avenir, 'Avenir Next', -apple-system, ...
          PROJECTS {allProjects.length}
        </h2>
        <span className="text-white/70" style={{ fontSize: '10px' }}>
          {allTasks.filter(t => t.projectId).length} tasks assigned
        </span>
      </div>
    </div>
  </div>
  { /* Content */ }
  <div className="flex-1 overflow-y-auto p-6" style={{ scrollbarWidth: 'thin' }}>
    <div className="mb-4">
      <Button
        type="button"
        variant="outline"
        className="border !border-white/50 text-white hover:text-white hover:!border-white transition-opa...
        style={{ background: 'rgba(10,15,30,0.85)', boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px r...
        onClick={() => {
          setEditingProject(null);
          setProjectWizardStep(0);
          setProjectWizardData({ name: '', description: '', color: '#6366F1', status: 'planning', targetD...
          setProjectDialogOpen(true);
        }}
        data-testid="button-new-project-flyout"
      >
        <Plus className="w-3 h-3 mr-1" />
        New Project
      </Button>
    </div>
    { /* Status Filter Cards */ }
    <div className="grid grid-cols-2 md:grid-cols-5 gap-4 mb-6">
      {

```

```

    { key: 'all', label: 'All Projects', count: allProjects.length, color: 'text-white' },
    { key: 'in_progress', label: 'In Progress', count: allProjects.filter(p => p.status === 'in_progr...
    { key: 'planning', label: 'Planning', count: allProjects.filter(p => p.status === 'planning').len...
    { key: 'completed', label: 'Completed', count: allProjects.filter(p => p.status === 'completed')....
    { key: 'on_hold', label: 'On Hold', count: allProjects.filter(p => p.status === 'on_hold').length...
  ].map(filter => (
    <div
      key={filter.key}
      className={`cursor-pointer rounded-xl p-4 text-center transition-all ${projectStatusFilter === ...
      onClick={() => setProjectStatusFilter(filter.key)}
      style={{
        background: projectStatusFilter === filter.key ? 'rgba(255, 255, 255, 0.2)' : 'rgba(255, 255,...
        border: '1px solid rgba(255, 255, 255, 0.2)'
      }}
    >
      <div className={`text-2xl font-bold ${filter.color}`}>{filter.count}</div>
      <div className="text-sm text-white/70">{filter.label}</div>
    </div>
  ))
</div>

/* Projects Grid */
{() => {
  const filteredProjects = projectStatusFilter === 'all'
    ? allProjects
    : allProjects.filter(p => p.status === projectStatusFilter);

  if (filteredProjects.length === 0) {
    return (
      <div className="text-center py-12 text-white">
        <FolderOpen className="h-16 w-16 mx-auto mb-4 opacity-60" />
        <h3 className="text-lg font-medium mb-2">
          {projectStatusFilter === 'all' ? 'No projects yet' : `No ${projectStatusFilter.replace('_', ...
        </h3>
        <p className="text-sm mb-4 text-white/70">Create a project to organize your tasks and track p...
        <Button
          variant="outline"
          className="border-white/30 text-white hover:bg-white/10 hover:text-white"
          onClick={() => { setEditingProject(null); setProjectWizardStep(0); setProjectWizardData({ n...
        >
        <Plus className="w-4 h-4 mr-2" />
        Create Your First Project
      </Button>
    </div>
  );
}

return (
  <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
    {filteredProjects.map(project => {
      const projectTasks = allTasks.filter(t => t.projectId === project.id);
      const completedTasks = projectTasks.filter(t => t.isCompleted);
      const progress = projectTasks.length > 0 ? Math.round((completedTasks.length / projectTasks.l...
      const isExpanded = expandedProjects.has(project.id);

      return (
        <div
          key={project.id}
          className="rounded-xl overflow-hidden flex flex-col"
          style={{
            background: 'rgba(255, 255, 255, 0.2)',
            border: '1px solid rgba(255, 255, 255, 0.2)',
            height: '240px'
          }}
        >
          /* Project Header with Brown */
          <div
            className="px-2 py-0.5"
            style={{
              background: colorSettings.headerBar
            }}
          >
            <div className="flex items-center justify-between gap-2">
              <div className="flex items-center gap-2 flex-1 min-w-0">
                <div
                  className="w-3 h-3 rounded-full flex-shrink-0 border border-white/30"
                  style={{ backgroundColor: project.color || "#6366F1" }}
                />
                <span
                  className="text-sm font-medium text-white truncate"
                  style={{ textShadow: '0 1px 2px rgba(0,0,0,0.2)' }}
                >
                  {project.name}
                </span>
              </div>
              <div className="flex items-center gap-1 flex-shrink-0">
                <Button
                  size="icon"
                  variant="ghost"
                  onClick={() => {
                    setEditingProject(project);
                    setProjectWizardStep(0);
                    setProjectWizardData({ name: project.name, description: project.description || ...
                    setProjectDialogOpen(true);
                  }}
                  className="h-7 w-7 text-white hover:text-white hover:bg-white/20"
                  data-testid={`button-edit-project-${project.id}`}
                >
                <Pencil className="w-3.5 h-3.5 !text-black strokeWidth={1.5} />
              </div>
            </div>
            <Button
              size="icon"
              variant="ghost"
              onClick={() => {
                if (confirm("Delete this project?")) {
                  deleteProjectMutation.mutate(project.id);
                }
              }}
            >

```

```

        className="h-7 w-7 text-white hover:text-white hover:bg-white/20"
        data-testid={`button-delete-project-${project.id}`}
      >
        <Trash2 className="w-3.5 h-3.5 !text-black" strokeWidth={1.5} />
      </Button>
    </div>
  </div>
</div>

{/* Project Body */}
<div className="flex-1 p-4 text-white flex flex-col">
  <div className="flex flex-wrap items-center gap-1.5 mb-2" style={{ marginTop: '-4px' }}>
    <Badge className={`text-[9px] px-1.5 py-0 h-4 ${
      project.status === 'completed' ? 'bg-green-500/30 text-green-300 border-green-500/5...' :
      project.status === 'in_progress' ? 'bg-yellow-500/30 text-yellow-300 border-yellow-...' :
      project.status === 'planning' ? 'bg-blue-500/30 text-blue-300 border-blue-500/50' :
      project.status === 'on_hold' ? 'bg-gray-500/30 text-gray-300 border-gray-500/50' :
      'bg-red-500/30 text-red-300 border-red-500/50'
    }`}>
      {(project.status || 'planning').replace('_', ' ')}
    </Badge>
    <Badge className={`text-[9px] px-1.5 py-0 h-4 ${
      project.priority === 'high' ? 'bg-red-500/30 text-red-300 border-red-500/50' :
      project.priority === 'medium' ? 'bg-orange-500/30 text-orange-300 border-orange-500...' :
      'bg-green-500/30 text-green-300 border-green-500/50'
    }`}>
      {(project.priority || 'medium').toUpperCase()}
    </Badge>
    {project.courseName && (
      <Badge
        className="text-[9px] px-1.5 py-0 h-4"
        style={{
          backgroundColor: project.courseName === 'CPPA122' ? '#22C55E' :
            project.courseName === 'CFNF400' ? '#EC4899' :
            project.courseName === 'CASL101' ? '#6366F1' : '#6366F1',
          color: 'white'
        }}
      >
        {project.courseName}
      </Badge>
    )}
  </div>

  {project.description && (
    <p className="text-[10px] text-white/70 mb-3 line-clamp-2">
      {project.description}
    </p>
  )}

  <div className="flex items-center gap-4 text-xs text-white/60 mb-3">
    {project.startDate && (
      <div className="flex items-center gap-1">
        <Calendar className="w-3 h-3" />
        <span>Start: {format(new Date(project.startDate), "MMM d")}</span>
      </div>
    )}
    {project.targetDate && (
      <div className="flex items-center gap-1">
        <Calendar className="w-3 h-3" />
        <span>Due: {format(new Date(project.targetDate), "MMM d")}</span>
      </div>
    )}
  </div>

  <div className="mt-auto space-y-1">
    <div className="flex items-center justify-between text-[10px] text-white/70">
      <span>Remaining</span>
      <span className="font-medium">{completedTasks.length}/{projectTasks.length} tasks</span>
    </div>
    <div className="h-2 bg-white/20 rounded-full overflow-hidden">
      <div
        className="h-full bg-green-400 transition-opacity duration-300"
        style={{ width: `${progress}%` }}
      />
    </div>
  </div>

  <Button
    variant="ghost"
    size="sm"
    onClick={() => {
      setExpandedProjects(prev => {
        const next = new Set(prev);
        if (next.has(project.id)) {
          next.delete(project.id);
        } else {
          next.add(project.id);
        }
        return next;
      });
    }}
    className="w-full justify-center gap-1 text-white/70 hover:text-white hover:bg-white/...
    style={{ marginTop: '12px', marginBottom: '6px' }}
  >
    {isExpanded ? 'Hide Tasks' : `Show Tasks (${projectTasks.length})`}
  </Button>

  {isExpanded && projectTasks.length > 0 && (
    <div className="space-y-1 pt-2 mt-2 border-t border-white/20">
      {projectTasks.map((task) => (
        <div
          key={task.id}
          className={`flex items-center gap-2 p-2 rounded-md text-xs ${
            task.isCompleted
              ? 'bg-green-500/20 line-through text-white/50'
              : 'bg-white/10 text-white'
          }`}
        >

```



```

        className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-t...
        style={{ boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4)' }}
        disabled={!projectWizardData.name.trim()}
        onClick={() => setProjectWizardStep(1)}
        data-testid="button-project-wizard-next-0"
      >
        Next
      </Button>
    </div>
  </div>
)}

{projectWizardStep === 1 && (
  <div className="flex flex-col gap-3">
    <div className="text-center mb-2">
      <FileText className="h-8 w-8 text-blue-400 mx-auto mb-2" />
      <h3 className="text-sm font-medium text-white">Description</h3>
      <p className="text-[9px] text-white/50 mt-1">Describe what this project is about (optional)</p>
    </div>
    <Textarea
      value={projectWizardData.description}
      onChange={(e) => setProjectWizardData(p => ({ ...p, description: e.target.value })))
      placeholder="Describe your project..."
      rows={4}
      className="text-[12px]"
      autoFocus
      data-testid="input-project-description-wizard"
    />
    <div className="flex justify-between mt-2">
      <Button variant="outline" className="border !border-white/30 text-white/70 hover:text-white hov...
        <Button
          variant="outline"
          className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-t...
          style={{ boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4)' }}
          onClick={() => setProjectWizardStep(2)}
          data-testid="button-project-wizard-next-1"
        >
          Next
        </Button>
      </div>
    </div>
  </div>
)}

{projectWizardStep === 2 && (
  <div className="flex flex-col gap-3">
    <div className="text-center mb-2">
      <Palette className="h-8 w-8 text-pink-400 mx-auto mb-2" />
      <h3 className="text-sm font-medium text-white">Color & Status</h3>
      <p className="text-[9px] text-white/50 mt-1">Choose a color and set the project status</p>
    </div>
    <div className="grid grid-cols-2 gap-4">
      <div className="space-y-2">
        <label className="text-[11px] font-medium text-white">Color</label>
        <div className="flex flex-wrap gap-2">
          {[ '#6366F1', '#EC4899', '#22C55E', '#F59E0B', '#3B82F6', '#EF4444', '#8B5CF6', '#14B8A6', '...'
            <button
              key={c}
              className={`w-8 h-8 rounded-full border-2 transition-all ${projectWizardData.color === ...
                style={{ backgroundColor: c }}
                onClick={() => setProjectWizardData(p => ({ ...p, color: c })))
                data-testid={`project-color-${c}`}
            />
          ])}
        </div>
      </div>
      <div>
        <input
          type="color"
          value={projectWizardData.color}
          onChange={(e) => setProjectWizardData(p => ({ ...p, color: e.target.value })))
          className="h-8 w-full rounded-md border cursor-pointer mt-1"
          data-testid="input-project-color-custom"
        />
      </div>
    </div>
    <div className="space-y-2">
      <label className="text-[11px] font-medium text-white">Status</label>
      <div className="flex flex-col gap-1.5">
        {[
          { value: 'planning', label: 'Planning', icon: '0=0É' },
          { value: 'in progress', label: 'In Progress', icon: '0=Y{' },
          { value: 'on hold', label: 'On Hold', icon: '#0p' },
          { value: 'completed', label: 'Completed', icon: '' },
        ].map(s => (
          <button
            key={s.value}
            className={`px-3 py-2 rounded-lg text-[11px] text-left transition-all flex items-center...
              onClick={() => setProjectWizardData(p => ({ ...p, status: s.value })))
              data-testid={`project-status-${s.value}`}
            >
              <span>{s.icon}</span> {s.label}
            </button>
          )
        )
      </div>
    </div>
    <div>
      <div className="flex justify-between mt-2">
        <Button variant="outline" className="border !border-white/30 text-white/70 hover:text-white hov...
          <Button
            variant="outline"
            className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-t...
            style={{ boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4)' }}
            onClick={() => setProjectWizardStep(3)}
            data-testid="button-project-wizard-next-2"
          >
            Next
          </Button>
        </div>
      </div>
    </div>
  </div>
)}

```

```

{projectWizardStep === 3 && (
  <div className="flex flex-col gap-3">
    <div className="text-center mb-2">
      <Calendar className="h-8 w-8 text-green-400 mx-auto mb-2" />
      <h3 className="text-sm font-medium text-white">Date & Priority</h3>
      <p className="text-[9px] text-white/50 mt-1">Set a target date and priority level</p>
    </div>
    <div className="grid grid-cols-2 gap-4">
      <div className="space-y-2">
        <label className="text-[11px] font-medium text-white">Target Date</label>
        <input
          type="date"
          value={projectWizardData.targetDate}
          onChange={(e) => setProjectWizardData(p => ({ ...p, targetDate: e.target.value })}
          className="text-[11px]"
          data-testid="input-project-targetdate-wizard"
        />
      </div>
      <div className="space-y-2">
        <label className="text-[11px] font-medium text-white">Priority</label>
        <div className="flex flex-col gap-1.5">
          {[
            { value: 'low', label: 'Low', color: 'text-gray-300' },
            { value: 'medium', label: 'Medium', color: 'text-yellow-300' },
            { value: 'high', label: 'High', color: 'text-red-300' },
          ].map(p => (
            <button
              key={p.value}
              className={`px-3 py-2 rounded-lg text-[11px] text-left transition-all flex items-center...`}
              onClick={() => setProjectWizardData(prev => ({ ...prev, priority: p.value })}
              data-testid={`project-priority-${p.value}`}
            >
              <span className={` ${p.color} font-medium`}>{p.label}</span>
            </button>
          ))}
        </div>
      </div>
    </div>
    <div className="flex justify-between mt-2">
      <button variant="outline" className="border !border-white/30 text-white/70 hover:text-white hov...
        <button
          variant="outline"
          className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-t...
          style={{ boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4)' }}
          onClick={() => setProjectWizardStep(4)}
          data-testid="button-project-wizard-next-3"
        >
          Next
        </button>
      </div>
    </div>
  ))
)

{projectWizardStep === 4 && (
  <div className="flex flex-col gap-3">
    <div className="text-center mb-2">
      <CheckCircle2 className="h-8 w-8 text-emerald-400 mx-auto mb-2" />
      <h3 className="text-sm font-medium text-white">Review</h3>
      <p className="text-[9px] text-white/50 mt-1">Confirm your project details</p>
    </div>
    <div className="space-y-2 bg-white/5 rounded-lg p-4 border border-white/10">
      <div className="flex items-center gap-2">
        <div className="w-4 h-4 rounded-full" style={{ backgroundColor: projectWizardData.color }} />
        <span className="text-[13px] font-medium">{projectWizardData.name}</span>
      </div>
      <div>
        {projectWizardData.description && (
          <p className="text-[10px] text-white/60 mt-1">{projectWizardData.description}</p>
        )}
      </div>
      <div className="grid grid-cols-3 gap-2 mt-3 text-[10px]">
        <div className="bg-white/5 rounded p-2">
          <div className="text-white/40 mb-0.5">Status</div>
          <div className="capitalize">{projectWizardData.status.replace('_', ' ')}</div>
        </div>
        <div className="bg-white/5 rounded p-2">
          <div className="text-white/40 mb-0.5">Priority</div>
          <div className="capitalize">{projectWizardData.priority}</div>
        </div>
        <div className="bg-white/5 rounded p-2">
          <div className="text-white/40 mb-0.5">Target Date</div>
          <div>{projectWizardData.targetDate || 'None'}</div>
        </div>
      </div>
    </div>
    <div>
      <div className="flex justify-between mt-2">
        <button variant="outline" className="border !border-white/30 text-white/70 hover:text-white hov...
          <button
            variant="outline"
            className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-t...
            style={{ boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px ...
            onClick={() => {
              const data = {
                name: projectWizardData.name,
                description: projectWizardData.description,
                color: projectWizardData.color,
                status: projectWizardData.status,
                targetDate: projectWizardData.targetDate || undefined,
                priority: projectWizardData.priority,
              };
              if (editingProject) {
                updateProjectMutation.mutate({ id: editingProject.id, data });
              } else {
                createProjectMutation.mutate(data);
              }
              setProjectDialogOpen(false);
              setEditingProject(null);
            }}
            data-testid="button-save-project-wizard"
          >
            Save
          </button>
        </div>
      </div>
    </div>
  )
)

```

```

        {editingProject ? "Save Changes" : "Create Project"}
      </Button>
    </div>
  </div>
))
</div>
</div>
</div>
</div>
))
{
  /* Task Context Menu for right-click delete */
  {contextMenu && (
    <div
      className="fixed bg-gray-900/95 rounded-lg shadow-xl py-1 z-[9999]"
      style={{
        left: contextMenu.x,
        top: contextMenu.y,
        minWidth: '120px'
      }}
      onClick={(e) => e.stopPropagation()}
    >
      <button
        className="w-full px-3 py-2 text-left text-sm text-red-400 hover:bg-red-500/20 flex items-center gap-2"
        onClick={() => {
          deleteTaskWithUndo(contextMenu.taskId);
          setContextMenu(null);
        }}
      >
        <Trash2 className="h-4 w-4" />
        Delete Task
      </button>
    </div>
  ))
}

{
  /* Recurring Delete Dialog */
  <Dialog open={!recurringDeleteTask} onOpenChange={(open) => !open && setRecurringDeleteTask(null)}>
    <DialogContent className="max-w-sm text-white">
      <DialogHeader>
        <DialogTitle className="text-white text-sm">Delete Recurring Task</DialogTitle>
      </DialogHeader>
      <p className="text-white/80 text-xs">
        "{recurringDeleteTask?.title}" is a recurring task. What would you like to do?
      </p>
      <div className="flex flex-col gap-2 pt-2">
        <button
          className="inline-flex items-center justify-center rounded-md px-4 py-2 text-white transition-opacity..."
          style={{
            border: '1.5px solid rgba(255,255,255,0.6)',
            background: 'linear-gradient(180deg, rgba(255,255,255,0.38) 0%, rgba(255,255,255,0.15) 48%, rgba(255,255,255,0.1) 48%, rgba(255,255,255,0.1) 100%)',
            boxShadow: 'inset 0 1px 0 rgba(255,255,255,0.6), inset 0 -1px 0 rgba(255,255,255,0.1)'
          }}
          onMouseEnter={(e) => { e.currentTarget.style.background = 'linear-gradient(180deg, rgba(255,255,255,0.38) 0%, rgba(255,255,255,0.15) 48%, rgba(255,255,255,0.1) 48%, rgba(255,255,255,0.1) 100%)'; }}
          onMouseLeave={(e) => { e.currentTarget.style.background = 'linear-gradient(180deg, rgba(255,255,255,0.38) 0%, rgba(255,255,255,0.15) 48%, rgba(255,255,255,0.1) 48%, rgba(255,255,255,0.1) 100%)'; }}
          onClick={() => {
            if (recurringDeleteTask) {
              doDeleteSingleTask(recurringDeleteTask.id);
              setRecurringDeleteTask(null);
              setEditingTask(null);
              setContextMenu(null);
            }
          }}
          data-testid="button-delete-this-only"
        >
          Delete this task only
        </button>
        <button
          className="inline-flex items-center justify-center rounded-md px-4 py-2 text-red-300 transition-opacity..."
          style={{
            border: '1.5px solid rgba(239,68,68,0.6)',
            background: 'linear-gradient(180deg, rgba(239,68,68,0.38) 0%, rgba(239,68,68,0.15) 48%, rgba(239,68,68,0.1) 48%, rgba(239,68,68,0.1) 100%)',
            boxShadow: 'inset 0 1px 0 rgba(239,68,68,0.4), inset 0 -1px 0 rgba(239,68,68,0.1)'
          }}
          onMouseEnter={(e) => { e.currentTarget.style.background = 'linear-gradient(180deg, rgba(239,68,68,0.38) 0%, rgba(239,68,68,0.15) 48%, rgba(239,68,68,0.1) 48%, rgba(239,68,68,0.1) 100%)'; }}
          onMouseLeave={(e) => { e.currentTarget.style.background = 'linear-gradient(180deg, rgba(239,68,68,0.38) 0%, rgba(239,68,68,0.15) 48%, rgba(239,68,68,0.1) 48%, rgba(239,68,68,0.1) 100%)'; }}
          onClick={() => {
            if (recurringDeleteTask) {
              deleteFutureMutation.mutate(recurringDeleteTask.id);
              setRecurringDeleteTask(null);
              setEditingTask(null);
              setContextMenu(null);
            }
          }}
          data-testid="button-delete-all-future"
        >
          Delete this and all future events
        </button>
        <button
          className="inline-flex items-center justify-center rounded-md px-4 py-2 text-white/60 transition-opacity..."
          onClick={() => setRecurringDeleteTask(null)}
          data-testid="button-cancel-recurring-delete"
        >
          Cancel
        </button>
      </div>
    </DialogContent>
  </Dialog>
}

{
  /* Recurring Edit Confirmation Dialog */
  <Dialog open={!recurringEditPending} onOpenChange={(open) => !open && setRecurringEditPending(null)}>
    <DialogContent className="max-w-sm text-white">
      <DialogHeader>
        <DialogTitle className="text-white text-sm">Edit Recurring Task</DialogTitle>
      </DialogHeader>
      <p className="text-white/80 text-xs">
        "{recurringEditPending?.title}" is a recurring task. Apply changes to:
      </p>
      <div className="flex flex-col gap-2 pt-2">
        <button

```

```

        className="inline-flex items-center justify-center rounded-md px-4 py-2 text-white transition-opacity...
        style={{
          border: '1.5px solid rgba(255,255,255,0.6)',
          background: 'linear-gradient(180deg, rgba(255,255,255,0.38) 0%, rgba(255,255,255,0.15) 48%, rgba(25...
          boxShadow: 'inset 0 1px 0 rgba(255,255,255,0.6), inset 0 -1px 0 rgba(255,255,255,0.1)'
        }}
        onMouseEnter={(e) => { e.currentTarget.style.background = 'linear-gradient(180deg, rgba(255,255,255,0...
        onMouseLeave={(e) => { e.currentTarget.style.background = 'linear-gradient(180deg, rgba(255,255,255,0...
        onClick={async () => {
          if (recurringEditPending) {
            const { originalTitle, ...singlePayload } = recurringEditPending.payload as Record<string, unknown>;
            const oldTask = allTasks.find(t => t.id === recurringEditPending.taskId);
            if (oldTask) {
              const oldFields: Record<string, any> = {};
              const newFields: Record<string, any> = {};
              for (const key of Object.keys(singlePayload)) {
                const oldVal = (oldTask as any)[key];
                const newVal = singlePayload[key];
                if (String(oldVal ?? '') !== String(newVal ?? '')) {
                  oldFields[key] = oldVal;
                  newFields[key] = newVal;
                }
              }
              if (Object.keys(oldFields).length > 0) {
                pushUndo({ type: 'edit', description: `Edited "${oldTask.title}"`, data: { taskId: oldTask.id...
              }
            }
            await apiRequest("PATCH", `/api/tasks/${recurringEditPending.taskId}`, singlePayload);
            queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
            queryClient.invalidateQueries({ queryKey: ["/api/weeks"] });
            recurringEditPending.onSuccess();
            setRecurringEditPending(null);
            setEditingTask(null);
          }
        }}
        data-testid="button-edit-this-only"
      >
        This task only
      </button>
      <button
        className="inline-flex items-center justify-center rounded-md px-4 py-2 text-white transition-opacity...
        style={{
          border: '1.5px solid rgba(59,130,246,0.6)',
          background: 'linear-gradient(180deg, rgba(59,130,246,0.38) 0%, rgba(59,130,246,0.15) 48%, rgba(59,1...
          boxShadow: 'inset 0 1px 0 rgba(59,130,246,0.4), inset 0 -1px 0 rgba(59,130,246,0.1)'
        }}
        onMouseEnter={(e) => { e.currentTarget.style.background = 'linear-gradient(180deg, rgba(59,130,246,0...
        onMouseLeave={(e) => { e.currentTarget.style.background = 'linear-gradient(180deg, rgba(59,130,246,0...
        onClick={async () => {
          if (recurringEditPending) {
            const { weekNumber, ...recurringFields } = recurringEditPending.payload as Record<string, unknown>;
            await apiRequest("PATCH", `/api/tasks/${recurringEditPending.taskId}/update-recurring`, recurring...
            queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
            queryClient.invalidateQueries({ queryKey: ["/api/weeks"] });
            recurringEditPending.onSuccess();
            setRecurringEditPending(null);
            setEditingTask(null);
          }
        }}
        data-testid="button-edit-all-recurring"
      >
        All recurring tasks
      </button>
      <button
        className="inline-flex items-center justify-center rounded-md px-4 py-2 text-white/60 transition-opac...
        onClick={() => setRecurringEditPending(null)}
        data-testid="button-cancel-recurring-edit"
      >
        Cancel
      </button>
    </div>
  </DialogContent>
</Dialog>

{/* Reschedule Dialog */}
<Dialog open={!rescheduleTask} onOpenChange={(open) => !open && setRescheduleTask(null)}>
  <DialogContent className="max-w-[95vw] sm:max-w-md max-h-[90vh] overflow-y-auto text-white [&_*]:text-white...
    <DialogHeader>
      <DialogTitle className="text-white">Reschedule Task</DialogTitle>
    </DialogHeader>
    {rescheduleTask && (
      <RescheduleForm
        task={rescheduleTask}
        onSuccess={() => setRescheduleTask(null)}
        onUndoPush={(action) => pushUndo(action as UndoAction)}
      />
    )}
  </DialogContent>
</Dialog>

{/* Edit Dialog */}
<Dialog open={!editingTask} onOpenChange={(open) => { if (!open) { const active = document.activeElement; if...
  <DialogContent onInteractOutside={(e) => e.preventDefault()} onPointerDownOutside={(e) => e.preventDefault(...
    <DialogHeader className="flex flex-row items-center justify-between px-4 py-3 -mx-6 -mt-6 rounded-t-lg bo...
      <div className="flex items-center gap-2">
        <Pencil className="h-3 w-3 text-white" />
        <DialogTitle className="font-normal text-white" style={{ fontFamily: "Avenir, 'Avenir Next', -apple-s...
      </div>
      <div className="flex items-center gap-3">
        <Button
          variant="outline"
          className="border !border-white/30 text-white/70 hover:text-white hover:!border-white/50 hover:bg-t...
          style={{ fontSize: '12px' }}
          onClick={() => setEditingTask(null)}
          data-testid="button-cancel-edit-task"
        >
          Cancel
        </Button>
      </div>
    </DialogContent>
  </Dialog>

```

```

{editingTask && editingTask.type === 'reminder' && !(editingTask as any).isAcknowledged && (
  <Button
    variant="outline"
    className="border !border-red-400 text-white hover:text-white hover:!border-red-300 hover:bg-red-...
    style={{
      fontSize: '12px',
      boxShadow: '0 0 6px rgba(220, 38, 38, 0.6), 0 0 12px rgba(220, 38, 38, 0.4)',
      background: 'rgba(220, 38, 38, 0.15)',
    }}
    onClick={() => {
      fetch(`/api/tasks/${editingTask.id}`, {
        method: 'PATCH',
        headers: { 'Content-Type': 'application/json' },
        credentials: 'include',
        body: JSON.stringify({ isAcknowledged: true }),
      }).then(() => {
        queryClient.invalidateQueries({ queryKey: ['/api/tasks'] });
        setEditingTask(null);
      });
    }}
    data-testid="button-acknowledge-reminder"
  > Acknowledge
</Button>
)}
{editingTask && (
  <Button
    variant="outline"
    className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-trans...
    style={{
      fontSize: '12px',
      boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255,25...
    }}
    onMouseEnter={(e) => { e.currentTarget.style.boxShadow = '0 0 8px rgba(255,255,255,0.8), 0 0 16px...
    onMouseLeave={(e) => { e.currentTarget.style.boxShadow = '0 0 6px rgba(255,255,255,0.6), 0 0 12px...
    onPointerDown={(e) => {
      e.stopPropagation();
      e.preventDefault();
      const form = document.querySelector('[data-edit-task-form]') as HTMLFormElement;
      if (form) form.requestSubmit();
    }}
    data-testid="button-update-task-header"
  > Update Task
</Button>
)}
</div>
</DialogHeader>
{editingTask && (
  <div className="relative">
    <Button
      size="icon"
      variant="ghost"
      className="absolute top-0 right-0 z-10 text-white/60 hover:text-white hover:bg-transparent transiti...
      onClick={() => {
        if (!confirm('Delete "${editingTask.title}"?')) return;
        const taskId = editingTask.id;
        setEditingTask(null);
        doDeleteSingleTask(taskId);
      }}
      data-testid="button-delete-task-dialog"
    >
    <Trash2 className="h-3.5 w-3.5" />
  </Button>
</div>
)}
{editingTask && (
  <TaskForm
    key={`edit-task-${editingTask.id}`}
    task={editingTask}
    weekNumber={editingTask.weekNumber}
    hideSubmitButton
    onSuccess={() => setEditingTask(null)}
    onRecurringEdit={({taskId, title, payload, onSuccessCb}) => {
      setRecurringEditPending({ taskId, title, payload, onSuccess: onSuccessCb });
    }}
    onUndoPush={(action) => pushUndo(action as UndoAction)}
  />
)}
</DialogContent>
</Dialog>

<Dialog open={isCourseListUploadOpen} onOpenChange={setIsCourseListUploadOpen}>
  <DialogContent className="max-w-2xl max-h-[80vh] overflow-y-auto" data-testid="dialog-course-list-changes">
    <DialogHeader>
      <DialogTitle className="text-base">Course List Changes</DialogTitle>
    </DialogHeader>
    {courseListChanges.length === 0 && !courseListUploadCourse ? (
      <div className="space-y-3">
        <p className="text-sm text-gray-600">Select which course this list belongs to:</p>
        <div className="grid gap-2">
          {coursesData.courses.filter(c => c.name.trim()).map((course, idx) => (
            <button
              key={idx}
              className="text-left px-3 py-2 border rounded hover:bg-gray-50 text-sm"
              data-testid={`button-select-course-${idx}`}
              onClick={() => {
                if (courseListPendingFile) {
                  handleCourseListUpload(courseListPendingFile, course.name);
                  setCourseListPendingFile(null);
                }
              }}
            >
            <span className="inline-block w-3 h-3 rounded-full mr-2" style={{ background: course.color }} />
            {course.name}
          </button>
          )
        )}
      </div>
    )
  )}
</DialogContent>
</Dialog>

```

```

    </div>
  </div>
) : courseListChanges.length === 0 ? (
  <div className="py-8 text-center">
    <p className="text-sm text-gray-500">No changes detected between the uploaded list and existing tasks...
  </div>
) : (
  <div className="space-y-3">
    <p className="text-xs text-gray-500">
      {courseListUploadCourse} - {courseListChanges.length} change{courseListChanges.length !== 1 ? 's' : ''} ...
    </p>
    <div className="flex gap-2 mb-2">
      <button
        className="text-[10px] text-blue-600 hover:underline"
        onClick={() => setCourseListChanges(prev => prev.map(c => ({ ...c, accepted: true })))}
        data-testid="button-accept-all-changes"
      >
        Accept All
      </button>
      <button
        className="text-[10px] text-gray-500 hover:underline"
        onClick={() => setCourseListChanges(prev => prev.map(c => ({ ...c, accepted: false })))}
        data-testid="button-decline-all-changes"
      >
        Decline All
      </button>
    </div>
    {[ 'new', 'modified', 'removed' ].map(changeType => {
      const items = courseListChanges.filter(c => c.changeType === changeType);
      if (items.length === 0) return null;
      const label = changeType === 'new' ? 'New Tasks' : changeType === 'modified' ? 'Modified Tasks' : '...';
      const color = changeType === 'new' ? '#16a34a' : changeType === 'modified' ? '#ca8a04' : '#dc2626';
      return (
        <div key={changeType} className="space-y-1">
          <div className="flex items-center gap-2">
            <span className="inline-block w-2 h-2 rounded-full" style={{ background: color }} />
            <span className="text-[11px] font-semibold uppercase tracking-wider" style={{ color }}>{label}...
          </div>
          {items.map(change => (
            <label
              key={change.id}
              className="flex items-start gap-2 px-2 py-1.5 rounded border cursor-pointer hover:bg-gray-50"
              style={{ borderColor: change.accepted ? color : '#e5e7eb', opacity: change.accepted ? 1 : 0.5 }}
              data-testid={`change-item-${change.id}`}
            >
              <input
                type="checkbox"
                checked={change.accepted}
                onChange={() => setCourseListChanges(prev => prev.map(c => c.id === change.id ? { ...c, a...
                className="mt-0.5"
                data-testid={`checkbox-${change.id}`}
              />
              <div className="flex-1 min-w-0">
                <p className="text-xs font-medium">{change.description}</p>
                {Object.entries(change.details).length > 0 && (
                  <div className="mt-0.5 space-y-0.5">
                    {Object.entries(change.details).map(([field, diff]) => (
                      <p key={field} className="text-[10px] text-gray-500">
                        {field}: <span className="line-through text-red-400">{(diff as { old: string; new...
                      </p>
                    ))}
                  </div>
                )}
              </div>
            </label>
          ))}
        </div>
      );
    })}
  </div>
  <div className="flex justify-end gap-2 pt-2 border-t">
    <Button
      variant="outline"
      size="sm"
      onClick={() => { setIsCourseListUploadOpen(false); setCourseListChanges([]); }}
      data-testid="button-cancel-course-changes"
    >
      Cancel
    </Button>
    <Button
      size="sm"
      onClick={handleApplyCourseChanges}
      disabled={courseListApplying || courseListChanges.every(c => !c.accepted)}
      data-testid="button-apply-course-changes"
    >
      {courseListApplying ? 'Applying...' : `Apply ${courseListChanges.filter(c => c.accepted).length} ...
    </Button>
  </div>
</div>
)}
</DialogContent>
</Dialog>

/* Celebration Popup */
{showCelebration && (
  <div
    className="fixed inset-0 z-[100] flex items-center justify-center pointer-events-none"
    onClick={() => setShowCelebration(false)}
  >
    <div className="celebration-container flex flex-col items-center pointer-events-auto">
      {/* Hooray text in arch shape */}
      <div className="relative mb-2">
        <svg viewBox="0 0 200 60" className="w-72 h-24">
          <defs>
            <path id="arch" d="M 10,55 Q 100,-5 190,55" fill="transparent" />
            <linearGradient id="shimmerGradient" x1="0%" y1="0%" x2="100%" y2="0%">
              <stop offset="0%" stopColor="#ffd700">
            </stop>
            <animate attributeName="stop-color" values="#ffd700;#ff6b6b;#ffd700" dur="1s" repeatCount="in...
          </stop>
        </svg>
      </div>
    </div>
  </div>
)

```

```

        <stop offset="50%" stopColor="#ff6b6b">
          <animate attributeName="stop-color" values="#ff6b6b;#ffd700;#ff6b6b" dur="1s" repeatCount="in...
        </stop>
        <stop offset="100%" stopColor="#ffd700">
          <animate attributeName="stop-color" values="#ffd700;#ff6b6b;#ffd700" dur="1s" repeatCount="in...
        </stop>
      </linearGradient>
      <filter id="glow">
        <feGaussianBlur stdDeviation="2" result="coloredBlur"/>
        <feMerge>
          <feMergeNode in="coloredBlur"/>
          <feMergeNode in="SourceGraphic"/>
        </feMerge>
      </filter>
    </defs>
    <text
      fill="url(#shimmerGradient)"
      filter="url(#glow)"
      style={{
        fontFamily: 'Avenir, 'Avenir Next', sans-serif',
        fontWeight: 800,
        fontSize: '28px',
        letterSpacing: '0.1em'
      }}
    >
    <textPath href="#arch" startOffset="50%" textAnchor="middle">
      HOORAY!
    </textPath>
    </text>
  </svg>
</div>
{ /* Animoji with clapping hands */ }
<div className="relative">
  <img
    src={celebrationAnimoji}
    alt="Celebration"
    className="w-52 h-52 object-contain drop-shadow-2xl"
  />
  { /* Clapping hands */ }
  <div className="absolute -left-8 bottom-12 text-4xl animate-clap-left">0=00</div>
  <div className="absolute -right-8 bottom-12 text-4xl animate-clap-right">0=00</div>
</div>
</div>
</div>
)}
</main>
</div>
</div>
);
}

```

```

function TaskCard({
  task,
  onComplete,
  onReschedule,
  onEdit,
  onDelete,
  onFileClick,
  cardBgClass,
  compact = false,
  overdueBlink = false,
  urgentBlink = false,
  blinkSyncDelay,
  colorSettings,
  courseColors,
}): {
  task: Task;
  onComplete: (isCompleted: boolean) => void;
  onReschedule: () => void;
  onEdit: () => void;
  onDelete: () => void;
  onFileClick?: (attachment: string) => void;
  cardBgClass?: string;
  compact?: boolean;
  overdueBlink?: boolean;
  blinkSyncDelay?: string;
  urgentBlink?: boolean;
  colorSettings?: { headerBar: string; pomodoroButton: string; icons: string };
  courseColors?: Record<string, { bg: string; border: string; text: string; dot: string; prepBg: string; prepBorder: ...
}) {
  const Icon = iconMap[task.type] || ClipboardCheck;
  const isMissed = task.isMissed && !task.isCompleted;

  // Get course color
  const courseCode = task.courseName?.split(" ")[0] || "";
  const colors = courseColors?.[courseCode];

  const handleExportCalendar = () => {
    window.open(`/api/tasks/${task.id}/ics`, '_blank');
  };

  const [isSendingTTS, setIsSendingTTS] = useState(false);
  const [isControlling, setIsControlling] = useState(false);

  const handlePlayTTS = async () => {
    setIsSendingTTS(true);
    try {
      const attachments = task.attachments || [];

      if (attachments.length > 0) {
        // Read PDF/file content aloud
        const mediaUrl = attachments[0];
        const response = await fetch('/api/media/play', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ mediaUrl, entityId: 'media_player.echo_lr_studio_white_am' }),
        });
      }
    }
  };
}

```



```

        if (!response.ok) {
            console.error('PDF TTS failed');
        }
    } catch (error) {
        console.error('TTS error:', error);
    } finally {
        setIsSendingTTS(false);
    }
};

const handleStop = async () => {
    setIsControlling(true);
    try {
        await fetch('/api/media/stop', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ entityId: 'media_player.echo_lr_studio_white_am' }),
        });
    } catch (error) {
        console.error('Stop error:', error);
    } finally {
        setIsControlling(false);
    }
};

const handleResume = async () => {
    setIsControlling(true);
    try {
        await fetch('/api/media/resume', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ entityId: 'media_player.echo_lr_studio_white_am' }),
        });
    } catch (error) {
        console.error('Resume error:', error);
    } finally {
        setIsControlling(false);
    }
};

const handleVolume = async (action: "up" | "down") => {
    setIsControlling(true);
    try {
        await fetch('/api/media/volume', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ action, entityId: 'media_player.echo_lr_studio_white_am' }),
        });
    } catch (error) {
        console.error('Volume error:', error);
    } finally {
        setIsControlling(false);
    }
};

const [isSyncingCalendar, setIsSyncingCalendar] = useState(false);

const handleGoogleCalendarSync = async () => {
    setIsSyncingCalendar(true);
    try {
        if (task.calendarEventId) {
            // Remove from calendar
            await fetch(`/api/tasks/${task.id}/calendar`, { method: 'DELETE' });
        } else {
            // Add to calendar
            await fetch(`/api/tasks/${task.id}/calendar`, { method: 'POST' });
        }
        // Refresh tasks to get updated calendar status
        queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    } catch (error) {
        console.error('Google Calendar sync error:', error);
    } finally {
        setIsSyncingCalendar(false);
    }
};

const [showInviteInput, setShowInviteInput] = useState(false);
const [inviteEmailInput, setInviteEmailInput] = useState('');
const [isSendingInvite, setIsSendingInvite] = useState(false);

const handleSendInvite = async () => {
    const emails = inviteEmailInput.split(/[,;\s]+/).map(e => e.trim()).filter(e => e.includes('@'));
    if (emails.length === 0) return;
    setIsSendingInvite(true);
    try {
        const res = await fetch(`/api/tasks/${task.id}/invite`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ emails }),
        });
        if (res.ok) {
            setShowInviteInput(false);
            setInviteEmailInput('');
        }
    } catch (e) { console.error('[Invite] send error:', e); }
    setIsSendingInvite(false);
};

const hasAttachments = task.attachments && task.attachments.length > 0;

const cardElement = (
    <Card
        className={`transition-all rounded-md shadow-sm border ${
            compact ? "h-[60px] flex flex-col" : "flex-1"
        } ${isMissed && !cardBgClass ? "border-destructive bg-destructive/5" : ""} ${
            task.isCompleted ? "opacity-60" : ""
        }`
    >

```

```

} ${overdueBlink ? "animate-urgent-blink" : ""} ${urgentBlink ? "animate-shimmer" : ""}}
style={{
  ...(overdueBlink && blinkSyncDelay ? { animationDelay: blinkSyncDelay } : {}),
  backgroundColor: cardBgClass ? undefined : (colors?.bg || undefined),
  borderColor: colors?.border || '#9ca3af'
}}
data-testid={`card-task-${task.id}`}
>
<CardHeader className={`flex flex-row items-start justify-between gap-1 space-y-0 ${compact ? "pb-0 pt-1.5 px-2..."`
  <div className="flex items-start gap-1.5">
    <Checkbox
      checked={task.isCompleted || false}
      onCheckedChange={(checked) => onComplete(!checked)}
      data-testid={`checkbox-task-${task.id}`}
      className={` ${compact ? "h-3.5 w-3.5" : ""} border-black data-[state=checked]:bg-black data-[state=checked]...`
    />
    <div>
      <CardTitle className={`font-normal ${task.isCompleted ? "line-through" : ""} ${compact ? "text-[10px] leading..."`
        {task.title}
      />
      </CardTitle>
      {task.courseName && (
        <p
          className={`font-normal ${compact ? "text-[8px]" : "text-[10px]"}`
          style={{ color: colors?.text || undefined }}
        >
          {compact ? (task.courseName.split(" - ")[1] || task.courseName) : task.courseName.split(" - ")[0]}
        />
      )}
    </div>
  </div>
  </div>
  {!compact && (
    <Badge
      className={colors ? "" : typeColors[task.type]}
      style={colors ? { backgroundColor: colors.bg, borderColor: colors.border, color: colors.text } : undefined}
    >
      <Icon className="h-3 w-3 mr-1" />
      {task.type}
    </Badge>
  )}
</CardHeader>
<CardContent className={`space-y-1 ${compact ? "px-2 pb-1 pt-0 mt-auto" : "px-3 pb-3 pt-0"}`>
  {!compact && task.description && (
    <p className="text-xs text-muted-foreground line-clamp-2">
      {task.description}
    </p>
  )}
  {compact ? (
    <div className="flex items-center">
      <Badge
        className={` ${colors ? "" : typeColors[task.type]} text-[8px] px-1.5 py-0.5`
        style={colors ? { backgroundColor: colors.bg, borderColor: colors.border, color: colors.text } : undefi...`
      >
        <Icon className="h-2.5 w-2.5 mr-0.5" />
        {task.type}
      </Badge>
      {hasAttachments && (
        <Paperclip className="h-3 w-3 ml-1.5 text-muted-foreground" data-testid={`icon-attachment-${task.id}`} />
      )}
      <div className="flex-1 flex items-center justify-end gap-1 text-black dark:text-white text-[10px]">
        <Clock className="h-2.5 w-2.5" />
        <span className="font-bold">DUE</span> {format(new Date(task.dueDate), "MMM d")}
      </div>
    </div>
  ) : (
    <div>
      <div className="flex items-center gap-1 text-muted-foreground text-[10px]">
        <Clock className="h-3 w-3" />
        {format(new Date(task.dueDate), "MMM d, h:mm a")}
      </div>
      <div className="flex items-center gap-1 text-[10px] text-muted-foreground">
        <Bell className="h-3 w-3" />
        <span>Reminders: 12h, 6h, 2h, 30min before</span>
      </div>
    </div>
  )}
  {task.referenceLink && (
    <div className="flex items-center gap-1 text-[10px]">
      <Link className="h-3 w-3 text-primary" />
      <a
        href={task.referenceLink}
        target="_blank"
        rel="noopener noreferrer"
        className="text-primary hover:underline truncate"
        data-testid={`link-reference-${task.id}`}
      >
        {task.referenceLink}
      </a>
    </div>
  )}
  {task.attachments && task.attachments.length > 0 && (
    <div className="space-y-1">
      <div className="flex items-center gap-1 text-[10px] text-muted-foreground">
        <Paperclip className="h-3 w-3" />
        <span>{task.attachments.length} attachment{task.attachments.length > 1 ? "s" : ""}</span>
      </div>
      <div className="flex flex-wrap gap-1">
        {task.attachments.map((attachment, idx) => {
          const attachmentName = attachment.split('/').pop() || attachment;
          return (
            <div key={idx} className="flex items-center gap-1">
              <button
                onClick={() => onFileClick?.(attachment)}
                className="text-[10px] text-primary hover:underline truncate max-w-[150px] cursor-pointer"
                data-testid={`link-attachment-${task.id}-${idx}`}
              >
                {attachmentName}
            </div>
          )
        })}
      </div>
    </div>
  )}

```

```

        </button>
      </div>
    });
  });
</div>
</div>
))

<div className="flex items-center gap-2 pt-1 flex-wrap">
  {isMissed && (
    <Button size="sm" variant="destructive" onClick={onReschedule} data-testid={`button-reschedule-${task...}`}>
      <RefreshCw className="h-3 w-3 mr-1" />
      Reschedule
    </Button>
  )}
  <Button size="sm" variant="outline" onClick={handleExportCalendar} data-testid={`button-export-${task.i...}`}>
    <Download className="h-3 w-3 mr-1" />
    .ics
  </Button>
  <Button
    size="sm"
    variant="outline"
    onClick={() => setShowInviteInput(!showInviteInput)}
    data-testid={`button-invite-${task.id}`}
  >
    <Mail className="h-3 w-3 mr-1" />
    Invite
  </Button>
  <Button
    size="sm"
    variant={task.calendarEventId ? "default" : "outline"}
    className={task.calendarEventId ? "bg-[#5979CC] hover:bg-[#4a68b3]" : ""}
    onClick={handleGoogleCalendarSync}
    disabled={isSyncingCalendar}
    data-testid={`button-gcal-${task.id}`}
  >
    {isSyncingCalendar ? (
      <Loader2 className="h-3 w-3 mr-1 animate-spin" />
    ) : (
      <CalendarDays className="h-3 w-3 mr-1" />
    )}
    {task.calendarEventId ? "On Calendar" : "Google Cal"}
  </Button>
  <Button size="sm" variant="ghost" onClick={onEdit} data-testid={`button-edit-${task.id}`}>
    Edit
  </Button>
</div>
{showInviteInput && (
  <div className="flex items-center gap-2 mt-2" data-testid={`invite-input-${task.id}`}>
    <Input
      placeholder="Email addresses (comma separated)"
      value={inviteEmailInput}
      onChange={(e) => setInviteEmailInput(e.target.value)}
      className="flex-1 h-7 text-xs"
      onKeyDown={(e) => { if (e.key === 'Enter') handleSendInvite(); }}
      data-testid={`input-invite-email-${task.id}`}
    />
    <Button
      size="sm"
      className="h-7 text-xs bg-blue-600 hover:bg-blue-700"
      onClick={handleSendInvite}
      disabled={isSendingInvite || !inviteEmailInput.trim()}
      data-testid={`button-send-invite-${task.id}`}
    >
      {isSendingInvite ? <Loader2 className="h-3 w-3 animate-spin" /> : <Share2 className="h-3 w-3 mr-1" />}
      Send
    </Button>
  </div>
)}
</>
)}
</CardContent>
</Card>
);

// For compact mode with attachments, add media controls
if (compact) {
  if (hasAttachments) {
    return (
      <div className="relative pt-1 h-full flex flex-col cursor-pointer pb-2" onClick={onEdit}>
        {/* Mini Media Controls for compact cards */}
        <div className="h-5 flex items-center justify-around rounded-md px-1 text-white border-[0.1px] border-white...">
          <div
            className="cursor-pointer hover:opacity-70"
            onClick={(e) => { e.stopPropagation(); handlePlayTTS(); }}
            data-testid={`button-play-${task.id}`}
            title="Play"
          >
            {isSendingTTS ? (
              <Loader2 className="h-3 w-3 animate-spin" />
            ) : (
              <Play className="h-3 w-3 fill-current" />
            )}
          </div>
          <div
            className="cursor-pointer hover:opacity-70"
            onClick={(e) => { e.stopPropagation(); handleStop(); }}
            data-testid={`button-stop-${task.id}`}
            title="Stop"
          >
            <Square className="h-3 w-3 fill-current" />
          </div>
          <div
            className="cursor-pointer hover:opacity-70"
            onClick={(e) => { e.stopPropagation(); handleResume(); }}
            data-testid={`button-resume-${task.id}`}
            title="Resume"
          >
            <Square className="h-3 w-3 fill-current" />
          </div>
        </div>
      </div>
    );
  }
}

```

```

        <RefreshCw className="h-3 w-3" />
    </div>
    <div>
        className="cursor-pointer hover:opacity-70"
        onClick={(e) => { e.stopPropagation(); handleVolume("down"); }}
        data-testid={`button-voldown-${task.id}`}
        title="Volume Down"
    >
        <MinusCircle className="h-3 w-3" />
    </div>
    <div>
        className="cursor-pointer hover:opacity-70"
        onClick={(e) => { e.stopPropagation(); handleVolume("up"); }}
        data-testid={`button-volup-${task.id}`}
        title="Volume Up"
    >
        <PlusCircle className="h-3 w-3" />
    </div>
    </div>
    {cardElement}
</div>
);
}
// Compact without attachments - add pt-1 to align with cards that have media controls
return (
    <div className="pt-1 cursor-pointer" onClick={onEdit}>
        {cardElement}
    </div>
);
}

if (hasAttachments) {
    return (
        <div className="relative pt-1 h-full flex flex-col">
            /* Media Controls - positioned absolutely at top, half height */
            <div className="absolute -top-5 left-0 right-0 h-5 flex items-center justify-around rounded-md px-1 bg-[#5979...]
                <div>
                    className="cursor-pointer hover:opacity-70"
                    onClick={handlePlayTTS}
                    data-testid={`button-play-${task.id}`}
                    title="Play"
                >
                    {isSendingTTS ? (
                        <Loader2 className="h-3.5 w-3.5 animate-spin" />
                    ) : (
                        <Play className="h-3.5 w-3.5 fill-current" />
                    )}
                </div>
                <div>
                    className="cursor-pointer hover:opacity-70"
                    onClick={handleStop}
                    data-testid={`button-stop-${task.id}`}
                    title="Stop"
                >
                    <Square className="h-3.5 w-3.5 fill-current" />
                </div>
                <div>
                    className="cursor-pointer hover:opacity-70"
                    onClick={handleResume}
                    data-testid={`button-resume-${task.id}`}
                    title="Resume"
                >
                    <RefreshCw className="h-3.5 w-3.5" />
                </div>
                <div>
                    className="cursor-pointer hover:opacity-70"
                    onClick={() => handleVolume("down")}
                    data-testid={`button-voldown-${task.id}`}
                    title="Volume Down"
                >
                    <MinusCircle className="h-3.5 w-3.5" />
                </div>
                <div>
                    className="cursor-pointer hover:opacity-70"
                    onClick={() => handleVolume("up")}
                    data-testid={`button-volup-${task.id}`}
                    title="Volume Up"
                >
                    <PlusCircle className="h-3.5 w-3.5" />
                </div>
            </div>
            {cardElement}
        </div>
    );
}

// For cards without attachments, add padding to align with cards that have media controls
return (
    <div className="pt-5 h-full flex flex-col">
        {cardElement}
    </div>
);
}

interface FileRecord {
    id: number;
    originalName: string;
    displayName: string;
    objectPath: string;
    contentType: string;
    size: number;
    folder: string | null;
}

function FileSelector({
    onSelect,
    excludePaths
}): {

```

```

onSelect: (objectPath: string) => void;
excludePaths: string[];
}) {
  const { data: files = [] } = useQuery<FileRecord[]>({
    queryKey: ["/api/files"],
  });

  const availableFiles = files
    .filter(f => !excludePaths.includes(f.objectPath))
    .sort((a, b) => a.displayName.localeCompare(b.displayName, undefined, { numeric: true, sensitivity: 'base' }));

  if (availableFiles.length === 0) {
    return (
      <Button type="button" variant="outline" disabled className="flex-1 !text-black bg-white h-8" style={{ fontSize: ...
        <FolderOpen className="h-3 w-3 mr-1" />
        No Files
      </Button>
    );
  }

  return (
    <Select onChange={onSelect}>
      <SelectTrigger className="flex-1 !text-black [&_*]:!text-black bg-white h-8" style={{ color: 'black', fontSize: ...
        <FolderOpen className="h-3 w-3 mr-1" />
        <SelectValue placeholder="Select File" />
      </SelectTrigger>
      <SelectContent>
        {availableFiles.map(file => (
          <SelectItem key={file.id} value={file.objectPath}>
            {file.displayName}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
  );
}

function ProfileForm({
  profileData,
  timezones,
  onSave,
  onCancel,
  profilePhotoUrl,
  onProfilePhotoChange
}): {
  profileData: { firstName: string; lastName: string; birthdate: string; timezone: string; travelTimezone: string | n...
  timezones: { value: string; label: string }[];
  onSave: (data: { firstName: string; lastName: string; birthdate: string; timezone: string; travelTimezone: string | ...
  onCancel: () => void;
  profilePhotoUrl: string | null;
  onProfilePhotoChange: (url: string | null) => void;
}) {
  const [firstName, setFirstName] = useState(profileData.firstName);
  const [lastName, setLastName] = useState(profileData.lastName);
  const [birthdate, setBirthdate] = useState(profileData.birthdate);
  const [timezone, setTimezone] = useState(profileData.timezone);
  const [travelTimezone, setTravelTimezone] = useState<string | null>(profileData.travelTimezone);
  const [isUploadingPhoto, setIsUploadingPhoto] = useState(false);
  const [cropImageSrc, setCropImageSrc] = useState<string | null>(null);
  const [crop, setCrop] = useState({ x: 0, y: 0 });
  const [zoom, setZoom] = useState(1);
  const [croppedAreaPixels, setCroppedAreaPixels] = useState<{ x: number; y: number; width: number; height: number } ...
  const photoInputRef = useRef<HTMLInputElement>(null);
  const [isTraveling, setIsTraveling] = useState(!profileData.travelTimezone);
  const [postalCode, setPostalCode] = useState(profileData.postalCode || '');
  const [location, setLocation] = useState(profileData.location || '');
  const [phoneNumber, setPhoneNumber] = useState(profileData.phoneNumber || '');
  const [email, setEmail] = useState(profileData.email || '');
  const [address, setAddress] = useState(profileData.address || '');
  const [country, setCountry] = useState(profileData.country || '');
  const [provinceState, setProvinceState] = useState(profileData.provinceState || '');
  const [emergencyContactName, setEmergencyContactName] = useState(profileData.emergencyContactName || '');
  const [emergencyContactPhone, setEmergencyContactPhone] = useState(profileData.emergencyContactPhone || '');
  const [allergies, setAllergies] = useState(profileData.allergies || '');

  const COUNTRIES = [
    { value: 'CA', label: 'Canada' },
    { value: 'US', label: 'United States' },
    { value: 'UK', label: 'United Kingdom' },
    { value: 'AU', label: 'Australia' },
    { value: 'FR', label: 'France' },
    { value: 'DE', label: 'Germany' },
    { value: 'IN', label: 'India' },
    { value: 'JP', label: 'Japan' },
    { value: 'MX', label: 'Mexico' },
    { value: 'BR', label: 'Brazil' },
    { value: 'other', label: 'Other' },
  ];

  const PROVINCES_STATES: Record<string, { value: string; label: string }[]> = {
    CA: [
      { value: 'AB', label: 'Alberta' }, { value: 'BC', label: 'British Columbia' }, { value: 'MB', label: 'Manitoba' },
      { value: 'NB', label: 'New Brunswick' }, { value: 'NL', label: 'Newfoundland and Labrador' }, { value: 'NS', la...
      { value: 'NT', label: 'Northwest Territories' }, { value: 'NU', label: 'Nunavut' }, { value: 'ON', label: 'Onta...
      { value: 'PE', label: 'Prince Edward Island' }, { value: 'QC', label: 'Quebec' }, { value: 'SK', label: 'Saskat...
    ],
    US: [
      { value: 'AL', label: 'Alabama' }, { value: 'AK', label: 'Alaska' }, { value: 'AZ', label: 'Arizona' }, { value: ...
      { value: 'CA', label: 'California' }, { value: 'CO', label: 'Colorado' }, { value: 'CT', label: 'Connecticut' }...
      { value: 'FL', label: 'Florida' }, { value: 'GA', label: 'Georgia' }, { value: 'HI', label: 'Hawaii' }, { value: ...
      { value: 'IL', label: 'Illinois' }, { value: 'IN', label: 'Indiana' }, { value: 'IA', label: 'Iowa' }, { value: ...
      { value: 'KY', label: 'Kentucky' }, { value: 'LA', label: 'Louisiana' }, { value: 'ME', label: 'Maine' }, { value: ...
      { value: 'MA', label: 'Massachusetts' }, { value: 'MI', label: 'Michigan' }, { value: 'MN', label: 'Minnesota' }...
      { value: 'MO', label: 'Missouri' }, { value: 'MT', label: 'Montana' }, { value: 'NE', label: 'Nebraska' }, { va...
      { value: 'NH', label: 'New Hampshire' }, { value: 'NJ', label: 'New Jersey' }, { value: 'NM', label: 'New Mexic...
      { value: 'NC', label: 'North Carolina' }, { value: 'ND', label: 'North Dakota' }, { value: 'OH', label: 'Ohio' }...
      { value: 'OR', label: 'Oregon' }, { value: 'PA', label: 'Pennsylvania' }, { value: 'RI', label: 'Rhode Island' }...
      { value: 'SD', label: 'South Dakota' }, { value: 'TN', label: 'Tennessee' }, { value: 'TX', label: 'Texas' }, { ...

```

```

    { value: 'VT', label: 'Vermont' }, { value: 'VA', label: 'Virginia' }, { value: 'WA', label: 'Washington' }, { ...
  }, { value: 'WI', label: 'Wisconsin' }, { value: 'WY', label: 'Wyoming' }, { value: 'DC', label: 'District of Colu...
  },
  AU: [
    { value: 'ACT', label: 'Australian Capital Territory' }, { value: 'NSW', label: 'New South Wales' }, { value: '...'
    { value: 'QLD', label: 'Queensland' }, { value: 'SA', label: 'South Australia' }, { value: 'TAS', label: 'Tasma...'
    { value: 'VIC', label: 'Victoria' }, { value: 'WA', label: 'Western Australia' },
  ],
};

const regionLabel = country === 'CA' ? 'Province' : country === 'US' ? 'State' : country === 'AU' ? 'State/Territor...
const regionOptions = PROVINCES_STATES[country] || [];

const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  onSave({ firstName, lastName, birthdate, timezone, travelTimezone: isTraveling ? travelTimezone : null, postalCod...
});

const handlePhotoSelect = (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (!file || !file.type.startsWith('image/')) return;
  const reader = new FileReader();
  reader.onload = () => {
    setCropImageSrc(reader.result as string);
    setCrop({ x: 0, y: 0 });
    setZoom(1);
  };
  reader.readAsDataURL(file);
  if (photoInputRef.current) photoInputRef.current.value = '';
};

const getCroppedBlob = async (): Promise<Blob> => {
  const image = new Image();
  image.src = cropImageSrc!;
  await new Promise((resolve) => { image.onload = resolve; });
  const canvas = document.createElement('canvas');
  const ctx = canvas.getContext('2d')!;
  const area = croppedAreaPixels!;
  canvas.width = area.width;
  canvas.height = area.height;
  ctx.drawImage(image, area.x, area.y, area.width, area.height, 0, 0, area.width, area.height);
  return new Promise((resolve) => canvas.toBlob((blob) => resolve(blob!), 'image/jpeg', 0.9));
};

const handleCropConfirm = async () => {
  if (!croppedAreaPixels) return;
  setIsUploadingPhoto(true);
  try {
    const blob = await getCroppedBlob();
    const resp = await fetch('/api/profile-photo/upload', { method: 'POST', headers: { 'Content-Type': 'application...
    const { uploadURL, objectPath } = await resp.json();
    await fetch(uploadURL, { method: 'PUT', headers: { 'Content-Type': 'image/jpeg' }, body: blob });
    const servedUrl = `/objects/${objectPath.split('/').slice(1).join('/')}`;
    onProfilePhotoChange(servedUrl);
    setCropImageSrc(null);
  } catch (err) {
    console.error('Photo upload failed:', err);
  } finally {
    setIsUploadingPhoto(false);
  }
};

return (
  <form onSubmit={handleSubmit} className="space-y-3 text-[10px]">
    {cropImageSrc && (
      <div className="rounded-lg overflow-hidden bg-black/50 p-3 space-y-3">
        <span className="text-[10px] text-white/70">Move and zoom to crop your photo</span>
        <div className="relative w-full style={{ height: '220px' }}>
          <Cropper
            image={cropImageSrc}
            crop={crop}
            zoom={zoom}
            aspect={1}
            cropShape="round"
            showGrid={false}
            onCropChange={setCrop}
            onZoomChange={setZoom}
            onCropComplete={(_: any, croppedPixels: { x: number; y: number; width: number; height: number }) => set...
          />
        </div>
        <div className="flex items-center gap-2">
          <span className="text-[9px] text-white/50">Zoom</span>
          <input type="range" min={1} max={3} step={0.1} value={zoom} onChange={(e) => setZoom(Number(e.target.valu...
        </div>
        <div className="flex gap-2 justify-end">
          <button type="button" className="text-[10px] px-3 py-1.5 rounded border border-white/30 text-white/70 hov...
          <button type="button" className="text-[10px] px-3 py-1.5 rounded bg-white/20 text-white hover:bg-white/30...
            {isUploadingPhoto ? <Loader2 className="w-3 h-3 animate-spin inline mr-1" /> : null}
            {isUploadingPhoto ? 'Saving...' : 'Save Photo'}
          </button>
        </div>
      </div>
    )}
    <div className="flex items-center gap-3 pb-2 border-b border-white/20" style={{ marginTop: '-8px' }}>
      <div
        className="relative cursor-pointer group"
        style={{ marginTop: '-15px' }}
        onClick={() => photoInputRef.current?.click()}
        data-testid="button-upload-profile-photo"
      >
        <div className="w-[84px] h-[84px] rounded-full overflow-hidden border-2 border-white/30 group-hover:border-...
          {profilePhotoUrl ? (
            <img src={profilePhotoUrl} alt="Profile" className="w-full h-full object-cover" />
          ) : (
            <div className="w-full h-full bg-white/10 flex items-center justify-center">
              <Camera className="w-5 h-5 text-white/50" />
            </div>
          )}
        </div>
      </div>
    </div>
  )
);

```

```

</div>
<div className="absolute inset-0 rounded-full flex items-center justify-center bg-black/40 opacity-0 group-...
  {isUploadingPhoto ? <Loader2 className="w-4 h-4 text-white animate-spin" /> : <Camera className="w-4 h-4 ...
</div>
<input ref={photoInputRef} type="file" accept="image/*" className="hidden" onChange={handlePhotoSelect} dat...
</div>
<div className="flex flex-col gap-1">
  <span className="text-[10px] text-white/70">Profile Photo</span>
  <span className="text-[9px] text-white/40">{isUploadingPhoto ? 'Uploading...' : 'Click to change'}</span>
  {profilePhotoUrl && (
    <button type="button" className="text-[9px] text-red-400 hover:text-red-300 text-left" onClick={{e} => { ...
  )}}
</div>
{((firstName || lastName) && (
  <span className="ml-auto text-[16px] font-semibold text-black self-start" style={{ marginRight: '60px' }} d...
))}
</div>
<div className="flex gap-[6px]" style={{ marginTop: '-1px' }}>
  <div className="space-y-0 w-1/2">
    <Label htmlFor="firstName" className="text-[10px]">First Name</Label>
    <Input
      id="firstName"
      value={firstName}
      onChange={{e} => setFirstName(e.target.value)}
      placeholder="First name"
      className="!text-black !text-[10px] h-8"
      style={{ fontSize: '10px', color: 'black' }}
      data-testid="input-profile-firstname"
    />
  </div>
  <div className="space-y-0 w-1/2">
    <Label htmlFor="lastName" className="text-[10px]">Last Name</Label>
    <Input
      id="lastName"
      value={lastName}
      onChange={{e} => setLastName(e.target.value)}
      placeholder="Last name"
      className="!text-black !text-[10px] h-8"
      style={{ fontSize: '10px', color: 'black' }}
      data-testid="input-profile-lastname"
    />
  </div>
</div>
<div className="space-y-0">
  <Label htmlFor="address" className="text-[10px]">Address</Label>
  <Input
    id="address"
    value={address}
    onChange={{e} => setAddress(e.target.value)}
    placeholder="e.g. 123 Main Street, Apt 4"
    className="bg-white !text-black !text-[10px] h-8"
    style={{ fontSize: '10px', color: 'black' }}
    data-testid="input-profile-address"
  />
</div>
<div className="flex gap-[6px]">
  <div className="space-y-0 w-1/2">
    <Label htmlFor="location" className="text-[10px]">City</Label>
    <select
      value={location}
      onChange={{e} => setLocation(e.target.value)}
      className="w-full h-8 px-2 text-[10px] rounded-md bg-white !text-black focus:outline-none focus:ring-2 fo...
      data-testid="select-profile-location"
    >
      <option value="">Select your city</option>
      {TRAVEL_CITIES.map(c => (
        <option key={c.label} value={c.label}>{c.label}</option>
      ))}
    </select>
  </div>
  <div className="space-y-0 w-1/2">
    <Label htmlFor="provinceState" className="text-[10px]">{regionLabel}</Label>
    <select
      value={provinceState}
      onChange={{e} => setProvinceState(e.target.value)}
      className="w-full h-8 px-2 text-[10px] rounded-md bg-white !text-black focus:outline-none focus:ring-2 fo...
      data-testid="select-profile-province-state"
    >
      <option value="">Select {regionLabel.toLowerCase()}</option>
      {regionOptions.map(r => (
        <option key={r.value} value={r.value}>{r.label}</option>
      ))}
    </select>
  </div>
</div>
<div className="flex gap-[6px]">
  <div className="space-y-0" style={{ width: 'calc(30% - 6px)', flexShrink: 0 }}>
    <Label htmlFor="country" className="text-[10px]">Country</Label>
    <select
      value={country}
      onChange={{e} => { setCountry(e.target.value); setProvinceState(''); }}
      className="w-full h-8 px-2 text-[10px] rounded-md bg-white !text-black focus:outline-none focus:ring-2 fo...
      data-testid="select-profile-country"
    >
      <option value="">Select country</option>
      {COUNTRIES.map(c => (
        <option key={c.value} value={c.value}>{c.label}</option>
      ))}
    </select>
  </div>
  <div className="space-y-0" style={{ width: 'calc(20% - 3px)', flexShrink: 0 }}>
    <Label htmlFor="postalCode" className="text-[10px]">Postal Code</Label>
    <Input
      id="postalCode"
      value={postalCode}
      onChange={{e} => setPostalCode(e.target.value.toUpperCase())}
      placeholder="M5V 2T6"
      className="bg-white !text-black !text-[10px] h-8"
    />
  </div>
</div>

```

```

        style={{ fontSize: '10px', color: 'black' }}
        data-testid="input-profile-postalcode"
      />
    </div>
    <div className="space-y-0 flex-1">
      <Label htmlFor="timezone" className="text-[10px]">Home Time Zone</Label>
      <select
        value={timezone}
        onChange={(e) => setTimezone(e.target.value)}
        className="w-full h-8 px-2 text-[10px] rounded-md bg-white !text-black focus:outline-none focus:ring-2 fo..."
        data-testid="select-profile-timezone"
      >
        {timezones.map(tz => (
          <option key={tz.value} value={tz.value} className="text-black bg-white">{tz.label}</option>
        ))}
      </select>
    </div>
  </div>
  <div className="flex gap-[6px]">
    <div className="space-y-0 w-1/2">
      <Label htmlFor="phoneNumber" className="text-[10px]">Phone Number</Label>
      <Input
        id="phoneNumber"
        type="tel"
        value={phoneNumber}
        onChange={(e) => setPhoneNumber(e.target.value)}
        placeholder="+1 (416) 555-0123"
        className="bg-white !text-black !text-[10px] h-8"
        style={{ fontSize: '10px', color: 'black' }}
        data-testid="input-profile-phone"
      />
    </div>
    <div className="space-y-0 w-1/2">
      <Label htmlFor="email" className="text-[10px]">Email</Label>
      <Input
        id="email"
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="e.g. name@example.com"
        className="bg-white !text-black !text-[10px] h-8"
        style={{ fontSize: '10px', color: 'black' }}
        data-testid="input-profile-email"
      />
    </div>
  </div>
  <div className="flex gap-[6px]">
    <div className="space-y-0 w-1/2">
      <Label htmlFor="birthdate" className="text-[10px]">Birthdate</Label>
      <Input
        id="birthdate"
        type="date"
        value={birthdate}
        onChange={(e) => setBirthdate(e.target.value)}
        className="bg-white !text-black !text-[10px] h-8"
        style={{ fontSize: '10px', color: 'black' }}
        data-testid="input-profile-birthdate"
      />
    </div>
    <div className="space-y-0 w-1/2">
      <Label htmlFor="allergies" className="text-[10px]">Allergies</Label>
      <Input
        id="allergies"
        value={allergies}
        onChange={(e) => setAllergies(e.target.value)}
        placeholder="e.g. Peanuts, Penicillin"
        className="bg-white !text-black !text-[10px] h-8"
        style={{ fontSize: '10px', color: 'black' }}
        data-testid="input-profile-allergies"
      />
    </div>
  </div>
  <div className="flex gap-[6px]">
    <div className="space-y-0 w-1/2">
      <Label htmlFor="emergencyContactName" className="text-[10px]">Emergency Contact Name</Label>
      <Input
        id="emergencyContactName"
        value={emergencyContactName}
        onChange={(e) => setEmergencyContactName(e.target.value)}
        placeholder="Contact name"
        className="bg-white !text-black !text-[10px] h-8"
        style={{ fontSize: '10px', color: 'black' }}
        data-testid="input-profile-emergency-name"
      />
    </div>
    <div className="space-y-0 w-1/2">
      <Label htmlFor="emergencyContactPhone" className="text-[10px]">Emergency Contact Phone</Label>
      <Input
        id="emergencyContactPhone"
        type="tel"
        value={emergencyContactPhone}
        onChange={(e) => setEmergencyContactPhone(e.target.value)}
        placeholder="+1 (416) 555-0123"
        className="bg-white !text-black !text-[10px] h-8"
        style={{ fontSize: '10px', color: 'black' }}
        data-testid="input-profile-emergency-phone"
      />
    </div>
  </div>
  <div className="flex justify-end" style={{ marginTop: '22px' }}>
    <Button
      type="submit"
      variant="outline"
      className="border !border-white/50 text-white hover:text-white hover:!border-white hover:bg-transparent tra..."
      style={{
        boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255,255,255,0.3)',
        fontSize: '12px'
      }}
    >

```



```

const [localTravel, setLocalTravel] = useState(existing.isTravelling || false);
const [localTravelTz, setLocalTravelTz] = useState(existing.travelTimezone || '');
const [localTravelStart, setLocalTravelStart] = useState(existing.travelStartDate || '');
const [localTravelEnd, setLocalTravelEnd] = useState(existing.travelEndDate || '');
const [springStart, setSpringStart] = useState(existing.springStartDate || '');
const [springEnd, setSpringEnd] = useState(existing.springEndDate || '');
const [summerStart, setSummerStart] = useState(existing.summerStartDate || '');
const [summerEnd, setSummerEnd] = useState(existing.summerEndDate || '');
const isSSType = localType === 'spring_summer';

return (
  <>
    <div className="flex-1 overflow-y-auto px-4 py-3 space-y-3" style={{ scrollbarWidth: 'thin', scrollbarColor: 'r...
      <div className="grid grid-cols-2 gap-3">
        <div className="space-y-1">
          <Label className="text-[10px] text-white/70">Week 1, Day 1 Date</Label>
          <input
            type="date"
            value={localW1}
            onChange={(e) => setLocalW1(e.target.value)}
            onClick={(e) => (e.target as HTMLInputElement).showPicker?.()}
            className="w-full h-8 px-2 text-[10px] rounded-md bg-white border border-white/20 focus:outline-none fo...
            style={{ fontSize: '10px', color: 'black', colorScheme: 'light' }}
            data-testid="input-sem-settings-week1"
          />
        </div>
        <div className="space-y-1">
          <Label className="text-[10px] text-white/70">Semester Type</Label>
          <select
            value={localType}
            onChange={(e) => setLocalType(e.target.value)}
            className="w-full h-8 px-2 text-[10px] rounded-md bg-white !text-black focus:outline-none focus:ring-2 ...
            data-testid="select-sem-settings-type"
          >
            <option value="fall" className="text-black bg-white">Fall</option>
            <option value="winter" className="text-black bg-white">Winter</option>
            <option value="spring_summer" className="text-black bg-white">Spring/Summer</option>
          </select>
        </div>
      </div>
      <div className="grid grid-cols-2 gap-3">
        <div className="space-y-1">
          <Label className="text-[10px] text-white/70">School Weeks</Label>
          <select
            value={String(localWeeks)}
            onChange={(e) => setLocalWeeks(Number(e.target.value))}
            className="w-full h-8 px-2 text-[10px] rounded-md bg-white !text-black focus:outline-none focus:ring-2 ...
            data-testid="select-sem-settings-weeks"
          >
            {[10, 11, 12, 13, 14, 15, 16].map(w => (
              <option key={w} value={String(w)} className="text-black bg-white">{w} weeks</option>
            ))}
          </select>
        </div>
        <div className="space-y-1">
          <Label className="text-[10px] text-white/70">Time Zone</Label>
          <select
            value={localTz}
            onChange={(e) => setLocalTz(e.target.value)}
            className="w-full h-8 px-2 text-[10px] rounded-md bg-white !text-black focus:outline-none focus:ring-2 ...
            data-testid="select-sem-settings-timezone"
          >
            [[
              { value: 'America/Toronto', label: 'Eastern (Toronto)' },
              { value: 'America/New_York', label: 'Eastern (New York)' },
              { value: 'America/Chicago', label: 'Central (Chicago)' },
              { value: 'America/Denver', label: 'Mountain (Denver)' },
              { value: 'America/Los_Angeles', label: 'Pacific (LA)' },
              { value: 'America/Vancouver', label: 'Pacific (Vancouver)' },
              { value: 'America/Edmonton', label: 'Mountain (Edmonton)' },
              { value: 'America/Winnipeg', label: 'Central (Winnipeg)' },
              { value: 'America/Halifax', label: 'Atlantic (Halifax)' },
              { value: 'America/St_Johns', label: 'Newfoundland (St. John's)' },
              { value: 'America/Regina', label: 'Central - No DST (Regina)' },
              { value: 'Pacific/Honolulu', label: 'Hawaii' },
              { value: 'America/Anchorage', label: 'Alaska' },
              { value: 'Europe/London', label: 'GMT (London)' },
              { value: 'Europe/Paris', label: 'CET (Paris)' },
              { value: 'Europe/Berlin', label: 'CET (Berlin)' },
              { value: 'Asia/Tokyo', label: 'JST (Tokyo)' },
              { value: 'Asia/Shanghai', label: 'CST (Shanghai)' },
              { value: 'Australia/Sydney', label: 'AEST (Sydney)' },
              { value: 'UTC', label: 'UTC' },
            ]}.map(tz => (
              <option key={tz.value} value={tz.value} className="text-black bg-white">{tz.label}</option>
            ))}
          </select>
        </div>
      </div>
      {isSSType && (
        <div className="space-y-2 p-2.5 rounded-md border border-white/15 bg-white/5">
          <div className="grid grid-cols-2 gap-3">
            <div className="space-y-2">
              <Label className="text-[10px] font-semibold text-white/90">Spring Half</Label>
              <div className="space-y-1">
                <Label className="text-[9px] text-white/60">Start Date</Label>
                <input
                  type="date"
                  value={springStart}
                  onChange={(e) => setSpringStart(e.target.value)}
                  onClick={(e) => (e.target as HTMLInputElement).showPicker?.()}
                  className="w-full h-7 px-2 text-[10px] rounded-md bg-white border border-white/20 focus:outline-n...
                  style={{ fontSize: '10px', color: 'black', colorScheme: 'light' }}
                  data-testid="input-sem-spring-start"
                />
              </div>
            </div>
            <div className="space-y-1">
              <Label className="text-[9px] text-white/60">End Date</Label>

```



```

    </div>
  </div>
</>
});
}

function WeekVariantsSection({ semesterSettings, week1StartDate }: { semesterSettings: SemesterSettings | null | unde...
const { data: mappings = [] } = useQuery<CourseWeekMapping[]>({
  queryKey: ["/api/course-week-mappings"],
});

const variantsByWeek = useMemo(() => {
  const result: Record<number, { courseCode: string; label: string }[]> = {};
  for (const m of mappings) {
    if (m.courseWeekLabel && m.courseWeekLabel.trim()) {
      if (!result[m.weekNumber]) result[m.weekNumber] = [];
      result[m.weekNumber].push({ courseCode: m.courseCode, label: m.courseWeekLabel });
    }
  }
  return result;
}, [mappings]);

const weeksWithVariants = Object.keys(variantsByWeek).map(Number).sort((a, b) => a - b);
if (weeksWithVariants.length === 0) return null;

const semStart = week1StartDate ? new Date(week1StartDate) : (semesterSettings?.semesterStartDate ? new Date(semesterSettings?.semesterStartDate) : null);
const readingWeek = semesterSettings?.readingWeekStart || null;
const currentWeek = semStart ? getWeekNumber(new Date(), semStart, readingWeek) : null;

return (
  <div className="border rounded-lg p-3 space-y-2" style={{ marginTop: '12px' }} data-testid="week-variants-section">
    <Label className="text-[10px] font-medium">Course Week Variants</Label>
    <span className="text-[9px] text-white/50 block">Courses using different week numbering than the TMU standard</span>
    <div className="space-y-1 text-[10px]">
      {weeksWithVariants.map(weekNum => {
        const variants = variantsByWeek[weekNum];
        const isCurrent = weekNum === currentWeek;
        return (
          <div key={weekNum} className={`flex items-center justify-between py-1.5 px-2 rounded ${isCurrent ? 'bg-blue-500' : 'bg-white'}>
            <div className="flex items-center gap-2">
              <span className={`text-white font-medium ${isCurrent ? 'text-blue-300' : 'text-blue-300'}>Week {weekNum}</span>
              {isCurrent && <span className="text-[7px] px-1 py-0.5 bg-blue-500/20 text-blue-300 rounded">Current</span>}
            </div>
            <div className="flex items-center gap-2">
              {variants.map((v, i) => (
                <span key={i} className="text-[8px] px-1.5 py-0.5 bg-amber-500/15 text-amber-300 rounded border border-amber-500">
                  {v.courseCode}: Wk {v.label}
                </span>
              ))}
            </div>
          </div>
        );
      })}
    </div>
  </div>
);
}

function SchoolForm({
  schoolData,
  semesterSettings,
  onSave,
  onCancel
}): {
  schoolData: { schoolLogo: string | null; schoolName: string; numberOfWeeks: number; week1StartDate: string; firstDayOfSchoolWeek: string;
  semesterSettings: SemesterSettings | null | undefined;
  onSave: (data: { schoolLogo: string | null; schoolName: string; numberOfWeeks: number; week1StartDate: string; firstDayOfSchoolWeek: string;
  onCancel: () => void;
}
) {
  const [schoolName, setSchoolName] = useState(schoolData.schoolName || 'Toronto Metropolitan University');
  const [customSchoolName, setCustomSchoolName] = useState('');
  const [numberOfWeeks, setNumberOfWeeks] = useState(schoolData.numberOfWeeks);
  const [week1StartDate, setWeek1StartDate] = useState(schoolData.week1StartDate);
  const [firstDayOfSchoolWeek, setFirstDayOfSchoolWeek] = useState(schoolData.firstDayOfSchoolWeek);
  const [lastDayOfSchoolWeek, setLastDayOfSchoolWeek] = useState(schoolData.lastDayOfSchoolWeek || 'friday');
  const [timeZone, setTimezone] = useState(schoolData.timeZone || Intl.DateTimeFormat().resolvedOptions().timeZone);
  const [isTravelling, setIsTravelling] = useState(schoolData.isTravelling || false);
  const [travelTimezone, setTravelTimezone] = useState(schoolData.travelTimezone || '');
  const [travelStartDate, setTravelStartDate] = useState(schoolData.travelStartDate || '');
  const [travelEndDate, setTravelEndDate] = useState(schoolData.travelEndDate || '');
  const [semesterType, setSemesterType] = useState(semesterSettings?.semesterType || 'winter');
  const [readingWeekDate, setReadingWeekDate] = useState(() => {
    if (semesterSettings?.readingWeekStart) {
      const d = new Date(semesterSettings.readingWeekStart);
      return `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, '0')}-${String(d.getDate()).padStart(2, '0')}`;
    }
    return '';
  });
  const [logoPreview, setLogoPreview] = useState<string | null>(schoolData.schoolLogo);
  const logoInputRef = useRef<HTMLInputElement>(null);

  const handleLogoUpload = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0];
    if (!file) return;
    if (file.size > 2 * 1024 * 1024) {
      return;
    }
    const reader = new FileReader();
    reader.onload = () => {
      setLogoPreview(reader.result as string);
    };
    reader.readAsDataURL(file);
  };

  const daysOfWeek = [
    { value: 'sunday', label: 'Sunday' },
    { value: 'monday', label: 'Monday' },
    { value: 'tuesday', label: 'Tuesday' },
  ];

```

```

    { value: 'wednesday', label: 'Wednesday' },
    { value: 'thursday', label: 'Thursday' },
    { value: 'friday', label: 'Friday' },
    { value: 'saturday', label: 'Saturday' },
  ];

const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  const finalSchoolName = schoolName === 'Other' ? customSchoolName : schoolName;
  onSave({ schoolLogo: logoPreview, schoolName: finalSchoolName, numberOfWeeks, week1StartDate, firstDayOfWeek, las...
  if (readingWeekDate) {
    apiRequest("PATCH", "/api/semester", { readingWeekStart: new Date(readingWeekDate).toISOString() });
  } else {
    apiRequest("PATCH", "/api/semester", { readingWeekStart: null });
  }
};

const semesterEnd = week1StartDate
  ? format(addWeeks(new Date(week1StartDate), numberOfWeeks), 'MMMM d, yyyy')
  : 'Not set';

return (
  <form id="school-settings-form" onSubmit={handleSubmit} className="space-y-4 text-[10px]">
    <div className="border rounded-lg p-3 space-y-3" style={{ marginTop: '-2px' }}>
      <Label className="text-[10px] font-medium" style={{ fontSize: '11px', letterSpacing: '0.05em' }}>School Setti...
      <div>
        <Label className="text-[10px] font-medium">School Logo</Label>
      </div>
      <div className="space-y-3">
        <div className="space-y-1">
          <div className="flex items-center gap-3">
            <div className="h-10 w-auto flex items-center justify-center rounded overflow-hidden bg-white/10 px-2">
              {logoPreview ? (
                <img src={logoPreview} alt="School logo" className="h-8 object-contain" />
              ) : (
                <img src={changSchoolLogo} alt="Default logo" className="h-8 object-contain" />
              )}
            </div>
            <div className="flex gap-2">
              <button
                type="button"
                onClick={() => logoInputRef.current?.click()}
                className="px-2 py-1 text-[9px] bg-white/20 hover:bg-white/30 rounded text-white transition-colors"
                data-testid="button-upload-logo"
              >
                Upload New Logo
              </button>
              {logoPreview && (
                <button
                  type="button"
                  onClick={() => setLogoPreview(null)}
                  className="px-2 py-1 text-[9px] bg-red-500/30 hover:bg-red-500/50 rounded text-white transition-c...
                  data-testid="button-remove-logo"
                >
                  Reset to Default
                </button>
              )}
            </div>
          </div>
          <input
            ref={logoInputRef}
            type="file"
            accept="image/*"
            onChange={handleLogoUpload}
            className="hidden"
            data-testid="input-logo-upload"
          />
        </div>
      </div>
      <div className="space-y-1">
        <Label htmlFor="schoolName" className="text-[10px]">School Name</Label>
        <select
          value={NORTH_AMERICAN_SCHOOLS.includes(schoolName) ? schoolName : 'Other'}
          onChange={(e) => { const v = e.target.value; setSchoolName(v); if (v !== 'Other') setCustomSchoolName('...
          style={{ color: '#000000', backgroundColor: 'ffffff' }}
          className="w-full h-8 px-2 text-[10px] rounded-md focus:outline-none focus:ring-2 focus:ring-blue-400"
          data-testid="select-school-name"
        >
          {NORTH_AMERICAN_SCHOOLS.map(s => (
            <option key={s} value={s} style={{ color: '#000000', backgroundColor: 'ffffff' }}>{s}</option>
          ))}
        </select>
        {(schoolName === 'Other' || !NORTH_AMERICAN_SCHOOLS.includes(schoolName)) && (
          <input
            value={customSchoolName || (NORTH_AMERICAN_SCHOOLS.includes(schoolName) ? '' : schoolName)}
            onChange={(e) => { setCustomSchoolName(e.target.value); setSchoolName('Other'); }}
            placeholder="Enter your school name"
            className="!text-black !text-[10px] h-8 mt-1"
            style={{ fontSize: '10px' }}
            data-testid="input-custom-school-name"
          />
        )}
      </div>
      <div className="space-y-1">
        <Label htmlFor="firstDayOfWeek" className="text-[10px]">First Day of School Week</Label>
        <select
          value={firstDayOfWeek}
          onChange={(e) => setFirstDayOfWeek(e.target.value)}
          className="w-full h-8 px-2 text-[10px] rounded-md bg-white !text-black focus:outline-none focus:ring-2 ...
          data-testid="select-first-day-of-week"
        >
          {daysOfWeek.map(day => (
            <option key={day.value} value={day.value} className="text-black bg-white">{day.label}</option>
          ))}
        </select>
      </div>
      <div className="space-y-1">
        <Label htmlFor="lastDayOfSchoolWeek" className="text-[10px]">Last Day of School Week</Label>
        <select

```

```

        value={lastDayOfSchoolWeek}
        onChange={(e) => setLastDayOfSchoolWeek(e.target.value)}
        className="w-full h-8 px-2 text-[10px] rounded-md bg-white !text-black focus:outline-none focus:ring-2 ..."
        data-testid="select-last-day-of-school-week"
      >
        {daysOfWeek.map(day => (
          <option key={day.value} value={day.value} className="text-black bg-white">{day.label}</option>
        ))}
      </select>
    </div>
    <div className="text-[10px] text-muted-foreground pt-1">
      Current Semester Ends: {semesterEnd}
    </div>
  </div>
</div>

<div className="border rounded-lg p-3 space-y-2" style={{ marginTop: '12px' }}>
  <Label className="text-[10px] font-medium">Semesters</Label>
  <div className="space-y-1 text-[10px]">
    {[
      { key: 'ss2025', label: 'Spring/Summer 2025', dates: 'May 5 - August 8, 2025' },
      { key: 'f2025', label: 'Fall 2025', dates: 'September 8 - December 12, 2025' },
      { key: 'w2026', label: 'Winter 2026', dates: 'January 12 - April 17, 2026' },
      { key: 'ss2026', label: 'Spring/Summer 2026', dates: 'May 4 - August 7, 2026' },
      { key: 'f2026', label: 'Fall 2026', dates: 'September 7 - December 11, 2026' },
      { key: 'w2027', label: 'Winter 2027', dates: 'January 11 - April 16, 2027' },
      { key: 'ss2027', label: 'Spring/Summer 2027', dates: 'May 3 - August 6, 2027' },
      { key: 'f2027', label: 'Fall 2027', dates: 'September 13 - December 17, 2027' },
      { key: 'w2028', label: 'Winter 2028', dates: 'January 10 - April 14, 2028' },
      { key: 'ss2028', label: 'Spring/Summer 2028', dates: 'May 1 - August 4, 2028' },
      { key: 'f2028', label: 'Fall 2028', dates: 'September 11 - December 15, 2028' },
      { key: 'w2029', label: 'Winter 2029', dates: 'January 8 - April 13, 2029' },
    ]}.map(sem => {
      const isCurrent = (() => {
        const now = new Date();
        if (sem.key === 'ss2025' && now >= new Date('2025-05-05') && now <= new Date('2025-08-08')) return true;
        if (sem.key === 'f2025' && now >= new Date('2025-09-08') && now <= new Date('2025-12-12')) return true;
        if (sem.key === 'w2026' && now >= new Date('2026-01-12') && now <= new Date('2026-04-17')) return true;
        if (sem.key === 'ss2026' && now >= new Date('2026-05-04') && now <= new Date('2026-08-07')) return true;
        if (sem.key === 'f2026' && now >= new Date('2026-09-07') && now <= new Date('2026-12-11')) return true;
        if (sem.key === 'w2027' && now >= new Date('2027-01-11') && now <= new Date('2027-04-16')) return true;
        if (sem.key === 'ss2027' && now >= new Date('2027-05-03') && now <= new Date('2027-08-06')) return true;
        if (sem.key === 'f2027' && now >= new Date('2027-09-13') && now <= new Date('2027-12-17')) return true;
        if (sem.key === 'w2028' && now >= new Date('2028-01-10') && now <= new Date('2028-04-14')) return true;
        if (sem.key === 'ss2028' && now >= new Date('2028-05-01') && now <= new Date('2028-08-04')) return true;
        if (sem.key === 'f2028' && now >= new Date('2028-09-11') && now <= new Date('2028-12-15')) return true;
        if (sem.key === 'w2029' && now >= new Date('2029-01-08') && now <= new Date('2029-04-13')) return true;
        return false;
      })();
      return (
        <div key={sem.key} className="flex items-center justify-between py-1 px-1.5 rounded" style={{ background...
          <div className="flex items-center gap-1.5">
            {isCurrent && <div className="w-1.5 h-1.5 rounded-full bg-emerald-400 flex-shrink-0" />}
            <span className={`text-white ${isCurrent ? 'font-bold' : 'font-normal'}`}>{sem.label}</span>
          </div>
          <span className="text-white/50">{sem.dates}</span>
        </div>
      );
    })}
  </div>
</div>

<WeekVariantsSection semesterSettings={semesterSettings} week1StartDate={week1StartDate} />

</form>
);
}

function CoursesForm({
  coursesData,
  semesterSettings,
  onSave,
  onSaveSemesterSchedule,
  onGenerateClassTasks,
  isGenerating,
  onCancel
}) {
  coursesData: { courses: Array<{ name: string; color: string; professor: string; professorEmail?: string }> };
  semesterSettings: SemesterSettings | null | undefined;
  onSave: (data: { courses: Array<{ name: string; color: string; professor: string; professorEmail?: string }> }) => ...
  onSaveSemesterSchedule: (data: Record<string, any>) => void;
  onGenerateClassTasks: () => void;
  isGenerating?: boolean;
  onCancel: () => void;
}) {
  const [courses, setCourses] = useState(coursesData.courses);
  const [isNewCourseOpen, setIsNewCourseOpen] = useState(false);
  const [editingCourseIndex, setEditingCourseIndex] = useState<number | null>(null);

  const handleSaveNewCourse = (courseData: {
    courseCode: string;
    courseName: string;
    professorName: string;
    professorEmail: string;
    color: string;
    semesterType: string;
    deliveryMode: string;
    classDay: string;
    classDay2: string;
    classTime: string;
    classEndTime: string;
    startDate: string;
    endDate: string;
    springSummerTerm: string;
    deadlines: Array<{ title: string; type: string; dueDate: string; description: string }>;
    reminders: number[];
  }) => {
    const fullName = `${courseData.courseCode} - ${courseData.courseName}`;

```

```

const updatedCourses = [...courses];

if (editingCourseIndex !== null) {
  updatedCourses[editingCourseIndex] = {
    name: fullName,
    color: courseData.color,
    professor: courseData.professorName,
    professorEmail: courseData.professorEmail,
  };
} else {
  const emptyIdx = updatedCourses.findIndex(c => !c.name.trim());
  if (emptyIdx !== -1) {
    updatedCourses[emptyIdx] = {
      name: fullName,
      color: courseData.color,
      professor: courseData.professorName,
      professorEmail: courseData.professorEmail,
    };
  } else {
    updatedCourses.push({
      name: fullName,
      color: courseData.color,
      professor: courseData.professorName,
      professorEmail: courseData.professorEmail,
    });
  }
}

setCourses(updatedCourses);
onSave({ courses: updatedCourses });

const courseIndex = editingCourseIndex !== null ? editingCourseIndex : updatedCourses.findIndex(c => c.name === f...
const prefix = `course${courseIndex + 1}` as const;

if (courseIndex >= 0 && courseIndex < 3) {
  const schedulePayload: Record<string, any> = {
    semesterType: courseData.semesterType,
    [`${prefix}Code`]: courseData.courseCode || null,
    [`${prefix}Name`]: fullName || null,
    [`${prefix}DeliveryMode`]: courseData.deliveryMode || null,
    [`${prefix}ClassDay`]: courseData.classDay || null,
    [`${prefix}ClassDay2`]: courseData.classDay2 || null,
    [`${prefix}ClassTime`]: courseData.classTime || null,
    [`${prefix}ClassEndTime`]: courseData.classEndTime || null,
    [`${prefix}SpringSummerTerm`]: courseData.springSummerTerm || null,
    [`${prefix}StartDate`]: courseData.startDate ? new Date(courseData.startDate).toISOString() : null,
    [`${prefix}EndDate`]: courseData.endDate ? new Date(courseData.endDate).toISOString() : null,
  };
  onSaveSemesterSchedule(schedulePayload);

  fetch('/api/onedrive/rename-course-folder', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include',
    body: JSON.stringify({
      courseIndex: courseIndex + 1,
      oldCode: courseData.courseCode,
      oldName: '',
      newCode: courseData.courseCode,
      newName: courseData.courseName,
    })),
  }).catch(() => {});
}

if (courseData.reminders && courseData.reminders.length > 0) {
  const courseReminders = JSON.parse(localStorage.getItem('courseReminders') || '{}');
  courseReminders[fullName] = courseData.reminders;
  localStorage.setItem('courseReminders', JSON.stringify(courseReminders));
}

setIsNewCourseOpen(false);
setEditingCourseIndex(null);

if (courseData.deadlines.length > 0) {
  (async () => {
    let created = 0;
    for (const deadline of courseData.deadlines) {
      if (deadline.title && deadline.dueDate) {
        try {
          const dueDate = new Date(deadline.dueDate);
          dueDate.setHours(23, 59, 0, 0);
          await apiRequest("POST", "/api/tasks", {
            title: deadline.title,
            description: deadline.description || '',
            type: deadline.type || 'assignment',
            courseName: fullName,
            dueDate: dueDate.toISOString(),
            priority: deadline.type === 'exam' || deadline.type === 'quiz' ? 'high' : 'medium',
            weekNumber: getWeekNumber(dueDate, semesterSettings?.semesterStartDate ? new Date(semesterSettings.se...) : null),
            reminder1: DEFAULT_REMINDER_1,
            reminder2: DEFAULT_REMINDER_2,
          });
          created++;
        } catch (err) {
          console.error("Failed to create deadline task:", err);
        }
      }
    }
  })();
  queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
}

const handleEditCourse = (index: number) => {
  setEditingCourseIndex(index);
  setIsNewCourseOpen(true);
};

```



```

        disabled={!canAddMore}
        className="border border-green-400/50 text-green-300 transition-opacity duration-200"
        style={{ fontSize: '11px' }}
        data-testid="button-new-course"
      >
        <Plus className="h-3.5 w-3.5 mr-1" />
        {canAddMore ? 'New Course' : 'Max 3 Courses'}
      </Button>
    <div className="flex items-center gap-2">
      {activeCoursesWithIndex.some(({ index }) => {
        return (semesterSettings as any)?.['course${index + 1}DeliveryMode`] === 'virtual';
      })} && (
        <Button
          type="button"
          variant="outline"
          onClick={onGenerateClassTasks}
          disabled={isGenerating}
          className="border border-blue-400/50 text-blue-300 transition-opacity duration-200"
          style={{ fontSize: '11px' }}
          data-testid="button-generate-class-tasks"
        >
          {isGenerating ? 'Generating...' : 'Generate Class Tasks'}
        </Button>
      )
    </div>
  </div>
</div>

{isNewCourseOpen && (
  <NewCourseDialog
    existingCourse={editingCourseIndex !== null ? {
      courseCode: courses[editingCourseIndex]?.name?.split(' - ')[0] || '',
      courseName: courses[editingCourseIndex]?.name?.split(' - ').slice(1).join(' - ') || '',
      professorName: courses[editingCourseIndex]?.professor || '',
      professorEmail: (courses[editingCourseIndex] as any)?.professorEmail || '',
      color: courses[editingCourseIndex]?.color || '#6b7280',
      semesterType: semesterSettings?.semesterType || 'winter',
      deliveryMode: (semesterSettings as any)?.['course${editingCourseIndex + 1}DeliveryMode`] || '',
      classDay: (semesterSettings as any)?.['course${editingCourseIndex + 1}ClassDay`] || '',
      classDay2: (semesterSettings as any)?.['course${editingCourseIndex + 1}ClassDay2`] || '',
      classTime: (semesterSettings as any)?.['course${editingCourseIndex + 1}ClassTime`] || '',
      classEndTime: (semesterSettings as any)?.['course${editingCourseIndex + 1}ClassEndTime`] || '',
      startDate: (semesterSettings as any)?.['course${editingCourseIndex + 1}StartDate`] ? new Date((semesterSe...
      endDate: (semesterSettings as any)?.['course${editingCourseIndex + 1}EndDate`] ? new Date((semesterSettin...
      springSummerTerm: (semesterSettings as any)?.['course${editingCourseIndex + 1}SpringSummerTerm`] || 'full',
    } : undefined)
    onSave={handleSaveNewCourse}
    onClose={() => { setIsNewCourseOpen(false); setEditingCourseIndex(null); }}
  />
)}
</div>
);
}

type NewCourseDialogProps = {
  existingCourse?: {
    courseCode: string;
    courseName: string;
    professorName: string;
    professorEmail: string;
    color: string;
    semesterType: string;
    deliveryMode: string;
    classDay: string;
    classDay2: string;
    classTime: string;
    classEndTime: string;
    startDate: string;
    endDate: string;
    springSummerTerm: string;
  };
  onSave: (data: {
    courseCode: string;
    courseName: string;
    professorName: string;
    professorEmail: string;
    color: string;
    semesterType: string;
    deliveryMode: string;
    classDay: string;
    classDay2: string;
    classTime: string;
    classEndTime: string;
    startDate: string;
    endDate: string;
    springSummerTerm: string;
    deadlines: Array<{ title: string; type: string; dueDate: string; description: string }>;
    reminders: number[];
  }) => void;
  onClose: () => void;
};

function NewCourseDialogInner({ existingCourse, onSave, onClose }: NewCourseDialogProps) {
  const [courseCode, setCourseCode] = useState(existingCourse?.courseCode || '');
  const [courseName, setCourseName] = useState(existingCourse?.courseName || '');
  const [professorName, setProfessorName] = useState(existingCourse?.professorName || '');
  const [professorEmail, setProfessorEmail] = useState(existingCourse?.professorEmail || '');
  const [color, setColor] = useState(existingCourse?.color || '#6366f1');
  const [semesterType, setSemesterType] = useState(existingCourse?.semesterType || 'winter');
  const [deliveryMode, setDeliveryMode] = useState(existingCourse?.deliveryMode || '');
  const [classDay, setClassDay] = useState(existingCourse?.classDay || '');
  const [classDay2, setClassDay2] = useState(existingCourse?.classDay2 || '');
  const [classTime, setClassTime] = useState(existingCourse?.classTime || '');
  const [classEndTime, setClassEndTime] = useState(existingCourse?.classEndTime || '');
  const [startDate, setStartDate] = useState(existingCourse?.startDate || '');
  const [endDate, setEndDate] = useState(existingCourse?.endDate || '');
  const [springSummerTerm, setSpringSummerTerm] = useState(existingCourse?.springSummerTerm || 'full');
  const [reminder1, setReminder1] = useState(15);
  const [reminder2, setReminder2] = useState(60);

```

```

const [reminder3, setReminder3] = useState(0);
const [deadlines, setDeadlines] = useState<Array<{ title: string; type: string; dueDate: string; description: strin...

const addDeadline = () => {
  setDeadlines(prev => [...prev, { title: '', type: 'assignment', dueDate: '', description: '' }]);
};

const updateDeadline = (index: number, field: string, value: string) => {
  setDeadlines(prev => {
    const updated = [...prev];
    updated[index] = { ...updated[index], [field]: value };
    return updated;
  });
};

const removeDeadline = (index: number) => {
  setDeadlines(prev => prev.filter((_, i) => i !== index));
};

const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  if (!courseCode.trim() || !courseName.trim()) return;
  onSave({
    courseCode: courseCode.trim(),
    courseName: courseName.trim(),
    professorName: professorName.trim(),
    professorEmail: professorEmail.trim(),
    color,
    semesterType,
    deliveryMode,
    classDay,
    classDay2,
    classTime,
    classEndTime,
    startDate,
    endDate,
    springSummerTerm,
    deadlines: deadlines.filter(d => d.title.trim() && d.dueDate),
    reminders: [15, ...(reminder2 > 0 ? [reminder2] : []), ...(reminder3 > 0 ? [reminder3] : [])],
  });
};

const dayOptions = [
  { value: '', label: 'None' },
  { value: 'monday', label: 'Monday' },
  { value: 'tuesday', label: 'Tuesday' },
  { value: 'wednesday', label: 'Wednesday' },
  { value: 'thursday', label: 'Thursday' },
  { value: 'friday', label: 'Friday' },
  { value: 'saturday', label: 'Saturday' },
  { value: 'sunday', label: 'Sunday' },
];

const deadlineTypes = [
  { value: 'assignment', label: 'Assignment' },
  { value: 'exam', label: 'Exam' },
  { value: 'quiz', label: 'Quiz' },
  { value: 'essay', label: 'Essay' },
  { value: 'project', label: 'Project' },
  { value: 'discussion', label: 'Discussion' },
  { value: 'reading', label: 'Reading' },
  { value: 'other', label: 'Other' },
];

return (
  <>
    <div className="flex items-center justify-between px-4 py-3 bg-black/30 border-b flex-shrink-0">
      <div className="flex items-center gap-2">
        <GraduationCap className="h-3.5 w-3.5 text-white" />
        <h2 className="text-xs font-normal text-white" style={{ fontFamily: 'Avenir, 'Avenir Next', -apple-system, ...
          {existingCourse ? 'EDIT COURSE' : 'NEW COURSE'}} />
      </div>
      <button onClick={(e) => { e.preventDefault(); e.stopPropagation(); onClose(); }} className="text-white hover:...
        <X className="h-5 w-5" />
      </button>
    </div>

    <form onSubmit={handleSubmit} className="flex-1 overflow-y-auto p-4 space-y-4">
      <div className="grid grid-cols-[1fr_1fr_auto] gap-2">
        <div>
          <Label className="text-[9px] text-white/60 mb-1 block">Course Number</Label>
          <Input
            value={courseCode}
            onChange={(e) => setCourseCode(e.target.value)}
            placeholder="e.g. CSOC103"
            className="h-8 !text-[10px] !text-black"
            style={{ fontSize: '10px' }}
            required
            data-testid="input-new-course-code"
          />
        </div>
        <div>
          <Label className="text-[9px] text-white/60 mb-1 block">Course Name</Label>
          <Input
            value={courseName}
            onChange={(e) => setCourseName(e.target.value)}
            placeholder="e.g. How Society Works"
            className="h-8 !text-[10px] !text-black"
            style={{ fontSize: '10px' }}
            required
            data-testid="input-new-course-name"
          />
        </div>
        <div>
          <Label className="text-[9px] text-white/60 mb-1 block">Color</Label>
          <input
            type="color"

```

```

        value={color}
        onChange={(e) => setColor(e.target.value)}
        className="w-8 h-8 rounded cursor-pointer p-0"
        data-testid="input-new-course-color"
      />
    </div>
  </div>

  <div className="grid grid-cols-2 gap-2">
    <div>
      <Label className="text-[9px] text-white/60 mb-1 block">Professor Name</Label>
      <Input
        value={professorName}
        onChange={(e) => setProfessorName(e.target.value)}
        placeholder="e.g. Dr. Smith"
        className="h-8 !text-[10px] !text-black"
        style={{ fontSize: '10px' }}
        data-testid="input-new-professor-name"
      />
    </div>
    <div>
      <Label className="text-[9px] text-white/60 mb-1 block">Professor Email</Label>
      <Input
        value={professorEmail}
        onChange={(e) => setProfessorEmail(e.target.value)}
        placeholder="e.g. prof@university.ca"
        className="h-8 !text-[10px] !text-black"
        style={{ fontSize: '10px' }}
        data-testid="input-new-professor-email"
      />
    </div>
  </div>

  <div className="border-t border-white/10 pt-3">
    <Label className="text-[10px] font-medium mb-2 block">Semester & Schedule</Label>
    <div className="grid grid-cols-2 gap-2">
      <div>
        <Label className="text-[9px] text-white/60 mb-1 block">Semester</Label>
        <select
          value={semesterType}
          onChange={(e) => setSemesterType(e.target.value)}
          className="w-full h-8 rounded bg-white/10 text-white text-[10px] px-2"
          data-testid="select-new-course-semester"
        >
          <option value="fall" className="bg-gray-800">Fall</option>
          <option value="winter" className="bg-gray-800">Winter</option>
          <option value="spring_summer" className="bg-gray-800">Spring/Summer</option>
        </select>
      </div>
      <div>
        <Label className="text-[9px] text-white/60 mb-1 block">Delivery Mode</Label>
        <select
          value={deliveryMode}
          onChange={(e) => setDeliveryMode(e.target.value)}
          className="w-full h-8 rounded bg-white/10 text-white text-[10px] px-2"
          data-testid="select-new-course-delivery"
        >
          <option value="" className="bg-gray-800">Select...</option>
          <option value="virtual" className="bg-gray-800">Virtual (live class)</option>
          <option value="online" className="bg-gray-800">Online (async)</option>
        </select>
      </div>
    </div>

    {semesterType === 'spring_summer' && (
      <div className="mt-2">
        <Label className="text-[9px] text-white/60 mb-1 block">Spring/Summer Term</Label>
        <select
          value={springSummerTerm}
          onChange={(e) => setSpringSummerTerm(e.target.value)}
          className="w-full h-8 rounded bg-white/10 text-white text-[10px] px-2"
          data-testid="select-new-course-term"
        >
          <option value="full" className="bg-gray-800">Full Length (May-Aug)</option>
          <option value="first_half" className="bg-gray-800">First Half (May-Jun)</option>
          <option value="second_half" className="bg-gray-800">Second Half (Jun-Aug)</option>
        </select>
      </div>
    )}
  </div>

  {deliveryMode === 'virtual' && (
    <div className="grid grid-cols-4 gap-2 mt-2">
      <div>
        <Label className="text-[9px] text-white/60 mb-1 block">Day 1</Label>
        <select
          value={classDay}
          onChange={(e) => setClassDay(e.target.value)}
          className="w-full h-8 rounded bg-white/10 text-white text-[10px] px-2"
          data-testid="select-new-course-day1"
        >
          {dayOptions.map(d => (
            <option key={d.value} value={d.value} className="bg-gray-800">{d.label}</option>
          ))}
        </select>
      </div>
      <div>
        <Label className="text-[9px] text-white/60 mb-1 block">Day 2</Label>
        <select
          value={classDay2}
          onChange={(e) => setClassDay2(e.target.value)}
          className="w-full h-8 rounded bg-white/10 text-white text-[10px] px-2"
          data-testid="select-new-course-day2"
        >
          {dayOptions.map(d => (
            <option key={d.value} value={d.value} className="bg-gray-800">{d.label}</option>
          ))}
        </select>
      </div>
    </div>
  )}

```

```

<div>
  <Label className="text-[9px] text-white/60 mb-1 block">Start Time</Label>
  <Input
    type="time"
    value={classTime}
    onChange={(e) => setClassTime(e.target.value)}
    className="h-8 !text-[10px] !text-black"
    data-testid="input-new-course-start-time"
  />
</div>
<div>
  <Label className="text-[9px] text-white/60 mb-1 block">End Time</Label>
  <Input
    type="time"
    value={classEndTime}
    onChange={(e) => setClassEndTime(e.target.value)}
    className="h-8 !text-[10px] !text-black"
    data-testid="input-new-course-end-time"
  />
</div>
</div>
))

<div className="grid grid-cols-2 gap-2 mt-2">
  <div>
    <Label className="text-[9px] text-white/60 mb-1 block">Course Start Date</Label>
    <Input
      type="date"
      value={startDate}
      onChange={(e) => setStartDate(e.target.value)}
      className="h-8 !text-[10px] !text-black"
      data-testid="input-new-course-start-date"
    />
  </div>
  <div>
    <Label className="text-[9px] text-white/60 mb-1 block">Course End Date</Label>
    <Input
      type="date"
      value={endDate}
      onChange={(e) => setEndDate(e.target.value)}
      className="h-8 !text-[10px] !text-black"
      data-testid="input-new-course-end-date"
    />
  </div>
</div>
</div>

<div className="border-t border-white/10 pt-3">
  <div className="flex items-center justify-between mb-2">
    <Label className="text-[10px] font-medium flex items-center gap-1.5">
      <Bell className="h-3.5 w-3.5 text-amber-400" />
      Class Reminders
    </Label>
  </div>
  <p className="text-[9px] text-white/40 mb-2">Popup reminders before class starts. 15-minute reminder is a...

  <div className="space-y-2">
    <div className="flex items-center gap-2 px-2 py-1.5 rounded bg-white/5 border border-white/10">
      <div className="w-2 h-2 rounded-full bg-amber-400 shrink-0" />
      <span className="text-[10px] text-white/80 flex-1">15 minutes before</span>
      <span className="text-[9px] text-amber-400/80 italic">Always on</span>
    </div>

    <div className="flex items-center gap-2 px-2 py-1.5 rounded bg-white/5 border border-white/10">
      <div className={`w-2 h-2 rounded-full shrink-0 ${reminder2 > 0 ? 'bg-blue-400' : 'bg-white/20'}`} />
      <span className="text-[10px] text-white/80 flex-1">Reminder 2</span>
      <select
        value={reminder2}
        onChange={(e) => setReminder2(Number(e.target.value))}
        className="h-6 rounded bg-white/10 text-white text-[10px] px-1.5"
        data-testid="select-reminder-2"
      >
        <option value={0} className="bg-gray-800">Off</option>
        <option value={5} className="bg-gray-800">5 min before</option>
        <option value={10} className="bg-gray-800">10 min before</option>
        <option value={30} className="bg-gray-800">30 min before</option>
        <option value={45} className="bg-gray-800">45 min before</option>
        <option value={60} className="bg-gray-800">1 hour before</option>
        <option value={90} className="bg-gray-800">1.5 hours before</option>
        <option value={120} className="bg-gray-800">2 hours before</option>
        <option value={180} className="bg-gray-800">3 hours before</option>
        <option value={1440} className="bg-gray-800">1 day before</option>
      </select>
    </div>

    <div className="flex items-center gap-2 px-2 py-1.5 rounded bg-white/5 border border-white/10">
      <div className={`w-2 h-2 rounded-full shrink-0 ${reminder3 > 0 ? 'bg-blue-400' : 'bg-white/20'}`} />
      <span className="text-[10px] text-white/80 flex-1">Reminder 3</span>
      <select
        value={reminder3}
        onChange={(e) => setReminder3(Number(e.target.value))}
        className="h-6 rounded bg-white/10 text-white text-[10px] px-1.5"
        data-testid="select-reminder-3"
      >
        <option value={0} className="bg-gray-800">Off</option>
        <option value={5} className="bg-gray-800">5 min before</option>
        <option value={10} className="bg-gray-800">10 min before</option>
        <option value={30} className="bg-gray-800">30 min before</option>
        <option value={45} className="bg-gray-800">45 min before</option>
        <option value={60} className="bg-gray-800">1 hour before</option>
        <option value={90} className="bg-gray-800">1.5 hours before</option>
        <option value={120} className="bg-gray-800">2 hours before</option>
        <option value={180} className="bg-gray-800">3 hours before</option>
        <option value={1440} className="bg-gray-800">1 day before</option>
      </select>
    </div>
  </div>
</div>

```

```

<div className="border-t border-white/10 pt-3">
  <div className="flex items-center justify-between mb-2">
    <Label className="text-[10px] font-medium">Known Deadlines</Label>
    <button
      type="button"
      onClick={addDeadline}
      className="flex items-center gap-1 text-[9px] text-blue-300 transition-colors"
      data-testid="button-add-deadline"
    >
      <Plus className="h-3 w-3" />
      Add Deadline
    </button>
  </div>
  <p className="text-[9px] text-white/40 mb-2">Add tests, exams, assignments, and other deadlines you alrea...

  {deadlines.length === 0 ? (
    <div className="text-center py-3 border border-dashed border-white/15 rounded">
      <p className="text-[9px] text-white/30">No deadlines added yet</p>
    </div>
  ) : (
    <div className="space-y-2 max-h-[200px] overflow-y-auto pr-1">
      {deadlines.map((deadline, idx) => (
        <div key={idx} className="border border-white/15 rounded p-2 space-y-2">
          <div className="flex items-center gap-2">
            <div className="flex-1 grid grid-cols-[1fr_auto_auto] gap-2">
              <Input
                value={deadline.title}
                onChange={(e) => updateDeadline(idx, 'title', e.target.value)}
                placeholder="Deadline title (e.g. Midterm Exam)"
                className="h-7 !text-[10px] !text-black"
                style={{ fontSize: '10px' }}
                data-testid={`input-deadline-title-${idx}`}
              />
              <select
                value={deadline.type}
                onChange={(e) => updateDeadline(idx, 'type', e.target.value)}
                className="h-7 rounded bg-white/10 text-white text-[10px] px-2 w-28"
                data-testid={`select-deadline-type-${idx}`}
              >
                {deadlineTypes.map(t => (
                  <option key={t.value} value={t.value} className="bg-gray-800">{t.label}</option>
                ))}
              </select>
              <button
                type="button"
                onClick={() => removeDeadline(idx)}
                className="p-1 text-white/40 transition-colors"
                data-testid={`button-remove-deadline-${idx}`}
              >
                <Minus className="h-3 w-3" />
              </button>
            </div>
            <div>
              <div>
                <div className="grid grid-cols-[1fr_1fr] gap-2">
                  <Input
                    type="date"
                    value={deadline.dueDate}
                    onChange={(e) => updateDeadline(idx, 'dueDate', e.target.value)}
                    className="h-7 !text-[10px] !text-black"
                    data-testid={`input-deadline-date-${idx}`}
                  />
                  <Input
                    value={deadline.description}
                    onChange={(e) => updateDeadline(idx, 'description', e.target.value)}
                    placeholder="Notes (optional)"
                    className="h-7 !text-[10px] !text-black"
                    style={{ fontSize: '10px' }}
                    data-testid={`input-deadline-description-${idx}`}
                  />
                </div>
              </div>
            </div>
          </div>
        </div>
      ))}
    </div>
  )}
</div>

<div className="flex justify-end gap-2 pt-2 border-t border-white/10">
  <Button
    type="button"
    variant="outline"
    onClick={(e) => { e.preventDefault(); e.stopPropagation(); onClose(); }}
    className="border !border-white/30 text-white/60 transition-opacity duration-200"
    style={{ fontSize: '11px' }}
    data-testid="button-cancel-new-course"
  >
    Cancel
  </Button>
  <Button
    type="submit"
    variant="outline"
    className="border !border-white/50 text-white transition-opacity duration-200"
    style={{
      boxShadow: '0 0 6px rgba(255,255,255,0.6), 0 0 12px rgba(255,255,255,0.4), 0 0 18px rgba(255,255,255,...',
      fontSize: '11px'
    }}
    data-testid="button-save-new-course"
  >
    {existingCourse ? 'Update Course' : 'Save Course'}
  </Button>
</div>
</form>

</>
});
}

function NewCourseDialog(props: NewCourseDialogProps) {
  return createPortal(

```

```

<div className="fixed inset-0 z-[200] flex items-center justify-center bg-black/50" onClick={(e) => { if (e.targe...
  <div className="bg-gradient-to-br from-gray-800 via-[#111] to-gray-900 rounded-lg w-[520px] max-h-[85vh] overfl...
    <NewCourseDialogInner {...props} />
  </div>
</div>,
document.body
);
}

function TaskForm({
  task,
  weekNumber,
  initialDate,
  initialType,
  initialStartTime,
  initialEndTime,
  hideSubmitButton,
  onSuccess,
  onRecurringEdit,
  onUndoPush
}): {
  task?: Task;
  weekNumber: number;
  initialDate?: Date | null;
  initialType?: string;
  initialStartTime?: string;
  initialEndTime?: string;
  hideSubmitButton?: boolean;
  onSuccess: () => void;
  onRecurringEdit?: (taskId: number, title: string, payload: Record<string, unknown>, onSuccess: () => void) => void;
  onUndoPush?: (action: { type: 'edit'; description: string; data: { taskId: number; taskTitle: string; oldFields: Re...
}) {
  const { data: allTasks = [] } = useQuery<Task[]>({
    queryKey: ["/api/tasks"],
  });

  const getDefaultDate = () => {
    if (task?.dueDate) return format(new Date(task.dueDate), "yyyy-MM-dd'T'HH:mm");
    if (initialDate) {
      // Always set default time to 6 PM for new tasks
      const date = new Date(initialDate);
      date.setHours(18, 0, 0, 0);
      return format(date, "yyyy-MM-dd'T'HH:mm");
    }
    return "";
  };

  const getDefaultPrepDays = () => {
    if (task?.startDate && task?.dueDate) {
      const start = new Date(task.startDate);
      const due = new Date(task.dueDate);
      const diffTime = due.getTime() - start.getTime();
      const diffDays = Math.ceil(diffTime / (1000 * 60 * 60 * 24));
      return diffDays > 0 ? diffDays : 0;
    }
    return 0;
  };

  const [formData, setFormData] = useState({
    title: task?.title || "",
    description: task?.description || "",
    type: task?.type || initialType || "reading",
    courseName: task?.courseName || "",
    prepDays: getDefaultPrepDays(),
    dueDate: getDefaultDate(),
    eventStartTime: task?.eventStartTime || initialStartTime || "",
    eventEndTime: task?.eventEndTime || initialEndTime || "",
    reminder1: task?.reminder1 ?? DEFAULT_REMINDER_1,
    reminder2: task?.reminder2 ?? DEFAULT_REMINDER_2,
    reminder3: task?.reminder3 ?? 0,
    reminder4: task?.reminder4 ?? 0,
    priority: task?.priority || "medium",
    weekNumber: task?.weekNumber || weekNumber,
    referenceLink: task?.referenceLink || "",
    attachments: task?.attachments || [] as string[],
    repeatType: (task?.repeatType as typeof REPEAT_TYPES[number]) || "none",
    repeatInterval: task?.repeatInterval || 1,
    repeatIntervalUnit: (task?.repeatIntervalUnit as typeof REPEAT_INTERVAL_UNITS[number]) || "weeks",
    repeatEndDate: task?.repeatEndDate ? format(new Date(task.repeatEndDate), "yyyy-MM-dd") : "",
    hideFromSummary: task?.hideFromSummary ?? false,
    flagged: task?.flagged ?? false,
  });
  const [newAttachment, setNewAttachment] = useState("");
  const fileInputRef = useRef<HTMLInputElement>(null);

  // Pending subtasks for new task creation
  const [pendingSubtasks, setPendingSubtasks] = useState<string[]>([]);
  const [newPendingSubtask, setNewPendingSubtask] = useState("");

  // Query files to look up display names for attachments
  const { data: allFiles = [] } = useQuery<FileRecord[]>({
    queryKey: ["/api/files"],
  });

  // Helper to get display name for an attachment
  const getAttachmentDisplayName = (attachment: string) => {
    const file = allFiles.find(f => f.objectPath === attachment);
    return file?.displayName || attachment.split('/').pop() || 'File';
  };

  // Date picker popover state
  const [isDatePickerOpen, setIsDatePickerOpen] = useState(false);
  const [tempDate, setTempDate] = useState<Date | undefined>(() => {
    if (formData.dueDate) {
      return new Date(formData.dueDate);
    }
    return undefined;
  });
};

```

```

const [tempHour, setTempHour] = useState(() => {
  if (formData.dueDate) {
    return getETHours(new Date(formData.dueDate)).toString().padStart(2, '0');
  }
  return "18";
});
const [tempMinute, setTempMinute] = useState(() => {
  if (formData.dueDate) {
    return getETMinutes(new Date(formData.dueDate)).toString().padStart(2, '0');
  }
  return "00";
});

const { uploadFile, isUploading } = useUpload({
  onSuccess: (response) => {
    // Add the object path to attachments
    setFormData(prev => ({
      ...prev,
      attachments: [...prev.attachments, response.objectPath]
    }));
  },
});

const handleFileUpload = async (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (file) {
    await uploadFile(file);
    // Reset file input
    if (fileInputRef.current) {
      fileInputRef.current.value = "";
    }
  }
};

const createMutation = useMutation({
  mutationFn: async (data: typeof formData) => {
    // For REMINDER tasks, auto-set due date to tomorrow 9am if not provided
    let effectiveDueDate = data.dueDate;
    if (!effectiveDueDate && data.type === 'reminder') {
      const tomorrow = new Date();
      tomorrow.setDate(tomorrow.getDate() + 1);
      tomorrow.setHours(9, 0, 0, 0);
      effectiveDueDate = tomorrow.toISOString();
    }
    // For MODULE tasks, automatically set startDate to Sunday and dueDate to Friday of current week
    let finalDueDate = new Date(effectiveDueDate);
    let finalStartDate: Date | null = null;

    if (data.type === "module" && !task) {
      // Get current date
      const today = new Date();
      const currentDayOfWeek = today.getDay(); // 0 = Sunday, 6 = Saturday

      // Calculate Sunday of the current week
      const sunday = new Date(today);
      sunday.setDate(today.getDate() - currentDayOfWeek);
      sunday.setHours(0, 0, 0, 0);

      // Calculate Friday of the current week
      const friday = new Date(sunday);
      friday.setDate(sunday.getDate() + 5);
      friday.setHours(18, 0, 0, 0); // 6 PM on Friday

      finalStartDate = sunday;
      finalDueDate = friday;
    } else if (data.prepDays > 0) {
      // Calculate startDate from prepDays if set
      const dueDate = new Date(data.dueDate);
      finalStartDate = new Date(dueDate);
      finalStartDate.setDate(finalStartDate.getDate() - data.prepDays);
    }

    // Build payload explicitly
    const payload: Record<string, unknown> = {
      title: data.title,
      description: data.description,
      type: data.type,
      courseName: data.courseName,
      dueDate: finalDueDate.toISOString(),
      eventStartTime: data.eventStartTime || null,
      eventEndTime: data.eventEndTime || null,
      reminder1: data.reminder1 || null,
      reminder2: data.reminder2 || null,
      reminder3: data.reminder3 || null,
      reminder4: data.reminder4 || null,
      priority: data.priority,
      weekNumber: data.weekNumber,
      referenceLink: data.referenceLink,
      attachments: data.attachments,
      repeatType: data.repeatType,
      repeatInterval: data.repeatType === "custom" ? data.repeatInterval : null,
      repeatIntervalUnit: data.repeatType === "custom" ? data.repeatIntervalUnit : null,
      repeatEndDate: data.repeatEndDate ? new Date(data.repeatEndDate).toISOString() : null,
      startDate: finalStartDate ? finalStartDate.toISOString() : null,
      hideFromSummary: data.hideFromSummary ?? false,
      flagged: data.flagged ?? false,
    };
  },
});
if (task) {
  if (onUndoPush) {
    const oldFields: Record<string, any> = {};
    const newFields: Record<string, any> = {};
    for (const key of Object.keys(payload)) {
      const oldVal = (task as any)[key];
      const newVal = payload[key];
      const oldStr = oldVal instanceof Date ? oldVal.toISOString() : String(oldVal ?? '');
      const newStr = String(newVal ?? '');
      if (oldStr !== newStr) {
        oldFields[key] = oldVal instanceof Date ? oldVal.toISOString() : oldVal;
      }
    }
  }
}

```



```

        newFields[key] = newVal;
    }
}
if (Object.keys(oldFields).length > 0) {
    onUndoPush({
        type: 'edit',
        description: `Edited "${task.title}"`,
        data: { taskId: task.id, taskTitle: task.title, oldFields, newFields }
    });
}
}
return apiRequest("PATCH", `/api/tasks/${task.id}`, payload);
}
// Create the task and return the response to get the new task ID
const response = await apiRequest("POST", "/api/tasks", payload);
const newTask = await response.json();

// If there are pending subtasks, create them for the new task
if (pendingSubtasks.length > 0 && newTask?.id) {
    for (const subtaskTitle of pendingSubtasks) {
        await apiRequest("POST", `/api/tasks/${newTask.id}/subtasks`, { title: subtaskTitle });
    }
}

return newTask;
},
onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    queryClient.invalidateQueries({ queryKey: ["/api/weeks"] });
    setPendingSubtasks([]); // Clear pending subtasks after successful creation
    onSuccess();
},
});

const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (!formData.dueDate && formData.type === 'reminder') {
        const tomorrow = new Date();
        tomorrow.setDate(tomorrow.getDate() + 1);
        tomorrow.setHours(9, 0, 0);
        formData.dueDate = format(tomorrow, "yyyy-MM-dd'T'HH:mm");
    }
    const isLinkedRecurring = task && ((task.repeatType && task.repeatType !== 'none') || !!task.parentTaskId);
    const hasSameTitleSiblings = task && !isLinkedRecurring && allTasks.filter(t => t.id !== task.id && t.title === t...
    if (task && onRecurringEdit && (isLinkedRecurring || (hasSameTitleSiblings && formData.title !== task.title))) {
        const data = formData;
        const finalDueDate = new Date(data.dueDate);
        let finalStartDate: Date | null = null;
        if (data.startDate) {
            finalStartDate = new Date(data.startDate);
        } else if (data.prepDays > 0) {
            finalStartDate = new Date(finalDueDate);
            finalStartDate.setDate(finalStartDate.getDate() - data.prepDays);
        }
        const payload: Record<string, unknown> = {
            title: data.title,
            description: data.description,
            type: data.type,
            courseName: data.courseName,
            dueDate: finalDueDate.toISOString(),
            eventStartTime: data.eventStartTime || null,
            eventEndTime: data.eventEndTime || null,
            reminder1: data.reminder1 || null,
            reminder2: data.reminder2 || null,
            reminder3: data.reminder3 || null,
            reminder4: data.reminder4 || null,
            priority: data.priority,
            weekNumber: data.weekNumber,
            referenceLink: data.referenceLink,
            attachments: data.attachments,
            repeatType: data.repeatType,
            repeatInterval: data.repeatType === "custom" ? data.repeatInterval : null,
            repeatIntervalUnit: data.repeatType === "custom" ? data.repeatIntervalUnit : null,
            repeatEndDate: data.repeatEndDate ? new Date(data.repeatEndDate).toISOString() : null,
            startDate: finalStartDate ? finalStartDate.toISOString() : null,
            hideFromSummary: data.hideFromSummary ?? false,
            flagged: data.flagged ?? false,
        };
        if (hasSameTitleSiblings && !isLinkedRecurring) {
            payload.originalTitle = task.title;
        }
        onRecurringEdit(task.id, task.title, payload, onSuccess);
        return;
    }
    createMutation.mutate(formData);
};

return (
    <form onSubmit={handleSubmit} className="space-y-3" data-task-form {...(task ? { 'data-edit-task-form': true } : ...
    <div className="grid grid-cols-2 gap-5">
        { /* Left Column */ }
    <div className="space-y-3">
        <div>
            <Label htmlFor="title" className="text-[11px] text-white">Title</Label>
            <Input
                id="title"
                value={formData.title}
                onChange={(e) => setFormData(prev => ({ ...prev, title: e.target.value })}
                placeholder="Assignment title"
                required
                data-testid="input-title"
                className="bg-white h-8 font-normal"
                style={{ color: 'black', fontSize: '11px' }}
            />
        </div>
        <div className="grid grid-cols-2 gap-3">
            <div>

```

```

<Label htmlFor="courseName" className="text-[11px] text-white">Course</Label>
<select
  value={formData.courseName}
  onChange={(e) => setFormData(prev => ({ ...prev, courseName: e.target.value })))}
  data-testid="select-course"
  className="flex h-8 w-full rounded-md border border-input bg-white px-2 py-1 font-normal ring-offset-...
  style={{ color: 'black', fontSize: '11px' }}
>
  <option value="">Select course</option>
  {COURSES.map(course => (
    <option key={course.code} value={` ${course.code} - ${course.name}`}>
      {course.code} - {course.name}
    </option>
  ))}
</select>
</div>

<div>
<Label htmlFor="type" className="text-[11px] text-white">Type</Label>
<select
  value={formData.type}
  onChange={(e) => setFormData(prev => ({ ...prev, type: e.target.value })))}
  data-testid="select-type"
  className="flex h-8 w-full rounded-md border border-input bg-white px-2 py-1 font-normal ring-offset-...
  style={{ color: 'black', fontSize: '11px' }}
>
  {TASK_TYPES.map(type => (
    <option key={type} value={type}>
      {type.charAt(0).toUpperCase() + type.slice(1)}
    </option>
  ))}
</select>
</div>
</div>

<div>
<Label htmlFor="dueDate" className="text-[11px] text-white">Due Date</Label>
<Popover open={isDatePickerOpen} onOpenChange={setIsDatePickerOpen}>
  <PopoverTrigger asChild>
    <Button
      variant="outline"
      className="w-full justify-start text-left font-normal h-8 bg-white"
      style={{ color: 'black', fontSize: '11px' }}
      data-testid="input-duedate"
    >
      <CalendarDays className="mr-2 h-4 w-4" />
      {formData.dueDate ? format(new Date(formData.dueDate), "MMM d, yyyy 'at' h:mm a") : "Pick a date"}
    </Button>
  </PopoverTrigger>
  <PopoverContent className="w-auto p-0" align="start" side="top" style={{ zIndex: 10010, maxHeight: '50v...
    <div className="p-3">
      {(() => {
        const now = new Date();
        const dueDate = tempDate || (formData.dueDate ? new Date(formData.dueDate) : null);
        const currentWeekStart = startOfWeek(now, { weekStartsOn: 0 });
        const dueWeekEnd = dueDate ? endOfWeek(dueDate, { weekStartsOn: 0 }) : null;
        const nowMonth = now.getFullYear() * 12 + now.getMonth();
        const dueMonth = dueDate ? dueDate.getFullYear() * 12 + dueDate.getMonth() : nowMonth;
        const numMonths = Math.min(Math.max(1, dueMonth - nowMonth + 1), 2);
        const weekRangeDays: Date[] = [];
        if (dueDate && dueWeekEnd) {
          let d = new Date(currentWeekStart);
          while (d <= dueWeekEnd) {
            weekRangeDays.push(new Date(d));
            d = addDays(d, 1);
          }
        }
        return (
          <CalendarPicker
            mode="single"
            selected={tempDate}
            onSelect={(date) => {
              if (date) {
                setTempDate(date);
              }
            }}
            defaultMonth={now}
            numberOfMonths={numMonths}
            modifiers={{ weekRange: weekRangeDays }}
            modifiersStyles={{ weekRange: { backgroundColor: 'rgba(59, 130, 246, 0.12)', borderRadius: 0 ...
              classNames={{ months: "flex flex-col space-y-4" }}
          </>
        );
      })()
    </div>
    <div className="border-t pt-3 mt-3">
      <Label className="text-sm font-medium">Time</Label>
      <div className="flex items-center gap-2 mt-2">
        <select
          value={tempHour}
          onChange={(e) => setTempHour(e.target.value)}
          className="w-16 h-8 rounded-md border border-input bg-white px-2 font-normal"
          style={{ color: 'black', fontSize: '11px' }}
        >
          {Array.from({ length: 24 }, (_, i) => (
            <option key={i} value={i.toString().padStart(2, '0')}>
              {i.toString().padStart(2, '0')}
            </option>
          ))}
        </select>
        <span>:</span>
        <select
          value={tempMinute}
          onChange={(e) => setTempMinute(e.target.value)}
          className="w-16 h-8 rounded-md border border-input bg-white px-2 font-normal"
          style={{ color: 'black', fontSize: '11px' }}
        >
          {['00', '15', '30', '45'].map((min) => (
            <option key={min} value={min}>{min}</option>

```

```

    ))}
  </select>
</div>
</div>
<div className="flex justify-end gap-2 mt-4 pt-3 border-t">
  <Button
    variant="outline"
    size="sm"
    onClick={() => setIsDatePickerOpen(false)}
  >
    Cancel
  </Button>
  <Button
    size="sm"
    onClick={() => {
      if (tempDate) {
        const newDate = new Date(tempDate);
        newDate.setHours(parseInt(tempHour), parseInt(tempMinute), 0, 0);
        setFormData(prev => ({ ...prev, dueDate: format(newDate, "yyyy-MM-dd'T'HH:mm") }));
      }
      setIsDatePickerOpen(false);
    }}
    data-testid="button-apply-date"
  >
    Apply
  </Button>
</div>
</PopoverContent>
</Popover>
</div>

<div className="grid grid-cols-3 gap-3">
  <div>
    <Label htmlFor="eventStartTime" className="text-[11px] text-white">Start</Label>
    <div className="flex gap-1">
      <Input
        id="eventStartTime"
        type="text"
        placeholder="HH:MM"
        value={formData.eventStartTime ? () => {
          const [h, m] = formData.eventStartTime.split(':');
          const hour = parseInt(h);
          const hour12 = hour === 0 ? 12 : hour > 12 ? hour - 12 : hour;
          return `${hour12}:${m}`;
        }}() : ''}
        onChange={(e) => {
          const val = e.target.value;
          if (/^[0-9:]*$/.test(val) && val.length <= 5) {
            const isPM = formData.eventStartTime ? parseInt(formData.eventStartTime.split(':')[0]) >= 12 : ...
            const [h, m] = val.split(':');
            if (h && m) {
              let hour24 = parseInt(h);
              if (isPM && hour24 < 12) hour24 += 12;
              if (!isPM && hour24 === 12) hour24 = 0;
              setFormData(prev => ({ ...prev, eventStartTime: `${hour24.toString().padStart(2, '0')}:${m}` ...
            } else {
              setFormData(prev => ({ ...prev, eventStartTime: val }));
            }
          }
        }}
        data-testid="input-start-time"
        className="bg-white h-8 flex-1 font-normal"
        style={{ color: 'black', fontSize: '11px' }}
      />
      <select
        value={formData.eventStartTime ? (parseInt(formData.eventStartTime.split(':')[0]) >= 12 ? 'PM' : 'A...
        onChange={(e) => {
          if (!formData.eventStartTime) return;
          const [h, m] = formData.eventStartTime.split(':');
          let hour = parseInt(h);
          if (e.target.value === 'PM' && hour < 12) hour += 12;
          if (e.target.value === 'AM' && hour >= 12) hour -= 12;
          setFormData(prev => ({ ...prev, eventStartTime: `${hour.toString().padStart(2, '0')}:${m}` }));
        }}
        className="h-8 rounded-md border border-input bg-white px-1 font-normal"
        style={{ color: 'black', fontSize: '11px' }}
      >
        <option value="AM">AM</option>
        <option value="PM">PM</option>
      </select>
    </div>
  </div>
  <div>
    <Label htmlFor="eventEndTime" className="text-[11px] text-white">End</Label>
    <div className="flex gap-1">
      <Input
        id="eventEndTime"
        type="text"
        placeholder="HH:MM"
        value={formData.eventEndTime ? () => {
          const [h, m] = formData.eventEndTime.split(':');
          const hour = parseInt(h);
          const hour12 = hour === 0 ? 12 : hour > 12 ? hour - 12 : hour;
          return `${hour12}:${m}`;
        }}() : ''}
        onChange={(e) => {
          const val = e.target.value;
          if (/^[0-9:]*$/.test(val) && val.length <= 5) {
            const isPM = formData.eventEndTime ? parseInt(formData.eventEndTime.split(':')[0]) >= 12 : false;
            const [h, m] = val.split(':');
            if (h && m) {
              let hour24 = parseInt(h);
              if (isPM && hour24 < 12) hour24 += 12;
              if (!isPM && hour24 === 12) hour24 = 0;
              setFormData(prev => ({ ...prev, eventEndTime: `${hour24.toString().padStart(2, '0')}:${m}` }));
            } else {
              setFormData(prev => ({ ...prev, eventEndTime: val }));
            }
          }
        }}
      />
    </div>
  </div>

```

```

    }
  }}
  data-testid="input-end-time"
  className="bg-white h-8 flex-1 font-normal"
  style={{ color: 'black', fontSize: '11px' }}
/>
<select
  value={formData.eventEndTime ? (parseInt(formData.eventEndTime.split(':')[0]) >= 12 ? 'PM' : 'AM') ...
  onChange={(e) => {
    if (!formData.eventEndTime) return;
    const [h, m] = formData.eventEndTime.split(':');
    let hour = parseInt(h);
    if (e.target.value === 'PM' && hour < 12) hour += 12;
    if (e.target.value === 'AM' && hour >= 12) hour -= 12;
    setFormData(prev => ({ ...prev, eventEndTime: `${hour.toString().padStart(2, '0')}:${m}` }));
  }}
  className="h-8 rounded-md border border-input bg-white px-1 font-normal"
  style={{ color: 'black', fontSize: '11px' }}
>
  <option value="AM">AM</option>
  <option value="PM">PM</option>
</select>
</div>
</div>
<div>
  <Label htmlFor="prepDays" className="text-[11px] text-white">Prep Days</Label>
  <Input
    id="prepDays"
    type="number"
    min={0}
    max={30}
    value={formData.prepDays}
    onChange={(e) => setFormData(prev => ({ ...prev, prepDays: parseInt(e.target.value) || 0 })))
    placeholder="0"
    data-testid="input-prepdays"
    className="bg-white h-8 font-normal"
    style={{ color: 'black', fontSize: '11px' }}
  />
</div>
</div>
<div>
  <Label className="text-[11px] text-white">Reminders</Label>
  <div className="grid grid-cols-4 gap-2">
    <select
      value={String(formData.reminder1)}
      onChange={(e) => setFormData(prev => ({ ...prev, reminder1: parseInt(e.target.value) })))
      data-testid="select-reminder1"
      className="flex h-8 w-full rounded-md border border-input bg-white px-1 py-1 ring-offset-background f...
      style={{ color: 'black', fontSize: '11px' }}
    >
      {REMINDER_OPTIONS.map(opt => (
        <option key={opt.value} value={String(opt.value)}>{opt.label}</option>
      ))}
    </select>
    <select
      value={String(formData.reminder2)}
      onChange={(e) => setFormData(prev => ({ ...prev, reminder2: parseInt(e.target.value) })))
      data-testid="select-reminder2"
      className="flex h-8 w-full rounded-md border border-input bg-white px-1 py-1 ring-offset-background f...
      style={{ color: 'black', fontSize: '11px' }}
    >
      {REMINDER_OPTIONS.map(opt => (
        <option key={opt.value} value={String(opt.value)}>{opt.label}</option>
      ))}
    </select>
    <select
      value={String(formData.reminder3)}
      onChange={(e) => setFormData(prev => ({ ...prev, reminder3: parseInt(e.target.value) })))
      data-testid="select-reminder3"
      className="flex h-8 w-full rounded-md border border-input bg-white px-1 py-1 ring-offset-background f...
      style={{ color: 'black', fontSize: '11px' }}
    >
      {REMINDER_OPTIONS.map(opt => (
        <option key={opt.value} value={String(opt.value)}>{opt.label}</option>
      ))}
    </select>
    <select
      value={String(formData.reminder4)}
      onChange={(e) => setFormData(prev => ({ ...prev, reminder4: parseInt(e.target.value) })))
      data-testid="select-reminder4"
      className="flex h-8 w-full rounded-md border border-input bg-white px-1 py-1 ring-offset-background f...
      style={{ color: 'black', fontSize: '11px' }}
    >
      {REMINDER_OPTIONS.map(opt => (
        <option key={opt.value} value={String(opt.value)}>{opt.label}</option>
      ))}
    </select>
  </div>
</div>
</div>
</div>
{ /* Right Column */ }
<div className="space-y-3">
  <div className="grid grid-cols-2 gap-4">
    <div>
      <Label htmlFor="priority" className="text-[11px] text-white">Priority</Label>
      <div className="flex items-center gap-2">
        <select
          value={formData.priority}
          onChange={(e) => setFormData(prev => ({ ...prev, priority: e.target.value })))
          data-testid="select-priority"
          className="flex h-8 w-full rounded-md border border-input bg-white px-2 py-1 ring-offset-background...
          style={{ color: 'black', fontSize: '11px' }}
        >
          <option value="low">Low</option>
          <option value="medium">Medium</option>

```

```

        <option value="high">High</option>
      </select>
      <button
        type="button"
        onClick={() => setFormData(prev => ({ ...prev, flagged: !prev.flagged, priority: !prev.flagged ? 'h...
        className={h-8 w-8 flex items-center justify-center rounded-md border transition-all flex-shrink-0...
        title={formData.flagged ? 'Remove flag' : 'Flag as priority'}
        data-testid="button-flag-task"
      >
        <Flag className={h-4 w-4 ${{formData.flagged ? 'fill-red-400' : ''}} />
      </button>
    </div>
  </div>
  <div>
    <Label className="text-[11px] text-white">Repeat</Label>
    <select
      value={formData.repeatType}
      onChange={(e) => setFormData(prev => ({ ...prev, repeatType: e.target.value as typeof REPEAT_TYPES[nu...
      data-testid="select-repeat-type"
      className="flex h-8 w-full rounded-md border border-input bg-white px-2 py-1 ring-offset-background f...
      style={{ color: 'black', fontSize: '11px' }}
    >
      <option value="none">No repeat</option>
      <option value="daily">Daily</option>
      <option value="weekly">Weekly</option>
      <option value="monthly">Monthly</option>
      <option value="custom">Custom...</option>
    </select>
  </div>
</div>

{formData.repeatType === "custom" && (
  <div className="grid grid-cols-2 gap-2">
    <div>
      <Label className="text-[11px] text-white">Every</Label>
      <input
        type="number"
        min="1"
        max="52"
        value={formData.repeatInterval}
        onChange={(e) => setFormData(prev => ({ ...prev, repeatInterval: parseInt(e.target.value) || 1 })))}
        data-testid="input-repeat-interval"
        className="flex h-8 w-full rounded-md border border-input bg-white px-2 py-1 ring-offset-background...
        style={{ color: 'black', fontSize: '11px' }}
      />
    </div>
    <div>
      <Label className="text-[11px] text-white">Unit</Label>
      <select
        value={formData.repeatIntervalUnit}
        onChange={(e) => setFormData(prev => ({ ...prev, repeatIntervalUnit: e.target.value as typeof REPEA...
        data-testid="select-repeat-unit"
        className="flex h-8 w-full rounded-md border border-input bg-white px-2 py-1 ring-offset-background...
        style={{ color: 'black', fontSize: '11px' }}
      >
        <option value="days">Days</option>
        <option value="weeks">Weeks</option>
      </select>
    </div>
  </div>
)}

{formData.repeatType !== "none" && (
  <div>
    <Label className="text-[11px] text-white">End Repeat (optional)</Label>
    <input
      type="date"
      value={formData.repeatEndDate}
      onChange={(e) => setFormData(prev => ({ ...prev, repeatEndDate: e.target.value })))}
      data-testid="input-repeat-end-date"
      className="flex h-8 w-full rounded-md border border-input bg-white px-2 py-1 ring-offset-background f...
      style={{ color: 'black', fontSize: '11px' }}
    />
  </div>
)}

<div>
  <Label htmlFor="description" className="text-[11px] text-white">Description</Label>
  <textarea
    id="description"
    value={formData.description}
    onChange={(e) => setFormData(prev => ({ ...prev, description: e.target.value })))}
    placeholder="Add notes or details..."
    rows={3}
    data-testid="input-description"
    className="flex w-full rounded-md border border-input bg-white px-2 py-1 ring-offset-background focus-v...
    style={{ color: 'black', fontSize: '11px' }}
  />
</div>

<div className="flex items-center gap-2">
  <label className="flex items-center gap-2 cursor-pointer select-none" data-testid="toggle-hide-from-summa...
    <input
      type="checkbox"
      checked={formData.hideFromSummary}
      onChange={(e) => setFormData(prev => ({ ...prev, hideFromSummary: e.target.checked })))}
      className="w-3.5 h-3.5 rounded accent-blue-500"
    />
    <span className="text-[10px] text-white/70">Hide from summary rows</span>
  </label>
</div>

{task?.attachments && task.attachments.length > 0 && (
  <div>
    <Label className="text-[11px] text-white">Attachments</Label>
    <div className="flex flex-col gap-1 mt-1">
      {task.attachments.map((att: string, idx: number) => {
        const fileName = att.split('/').pop() || 'Attachment';

```

```

        return (
          <a
            key={idx}
            href={att}
            target="_blank"
            rel="noopener noreferrer"
            className="flex items-center gap-2 px-2 py-1.5 bg-blue-500/10 border border-blue-400/20 rounded...
            data-testid={`attachment-link-${idx}`}
          >
            <FileText className="h-3.5 w-3.5 text-blue-400 flex-shrink-0" />
            <span className="text-[10px] text-blue-300 truncate">{task.title ? `${task.title}.pdf` : fileName...
            <ExternalLink className="h-3 w-3 text-blue-400/50 flex-shrink-0 ml-auto" />
          </a>
        );
      });
    }
  </div>
</div>
))}

<div>
  <Label htmlFor="referenceLink" className="text-[11px] text-white">Reference Link</Label>
  <input
    id="referenceLink"
    type="url"
    value={formData.referenceLink}
    onChange={(e) => setFormData(prev => ({ ...prev, referenceLink: e.target.value })))
    placeholder="https://example.com"
    data-testid="input-reference-link"
    className="flex h-8 w-full rounded-md border border-input bg-white px-2 py-1 ring-offset-background foc...
    style={{ color: 'black', fontSize: '11px' }}
  />
</div>
</div>
</div>

<div>
  <Label className="text-[11px] text-white">Attachments (optional)</Label>
  <div className="space-y-2">
    {formData.attachments.map((attachment, idx) => (
      <div key={idx} className="flex items-center gap-2" style={{ fontSize: '11px' }}>
        <Paperclip className="h-3 w-3 flex-shrink-0 text-muted-foreground" />
        <a href={attachment.startsWith('/') ? attachment : attachment} target="_blank" rel="noopener no...
          {getAttachmentDisplayName(attachment)}
        </a>
        <Button
          type="button"
          variant="ghost"
          size="icon"
          onClick={() => setFormData(prev => ({
            ...prev,
            attachments: prev.attachments.filter((_, i) => i !== idx)
          })))
          data-testid={`button-remove-attachment-${idx}`}
        >
          <X className="h-4 w-4" />
        </Button>
      </div>
    ))}
  </div>

  <div className="flex gap-2">
    <input
      ref={fileInputRef}
      type="file"
      onChange={handleFileUpload}
      className="hidden"
      data-testid="input-file-upload"
    />
    <Button
      type="button"
      onClick={() => fileInputRef.current?.click()}
      disabled={isUploading}
      className="flex-1 bg-transparent hover:bg-[#5979CC]/10 text-[#5979CC] border-2 border-[#5979CC] shadow...
      style={{ fontSize: '11px' }}
      data-testid="button-upload-file"
    >
      {isUploading ? (
        <div>
          <Loader2 className="h-3 w-3 mr-1 animate-spin" />
          Uploading...
        </div>
      ) : (
        <div>
          <Upload className="h-3 w-3 mr-1" />
          Upload New
        </div>
      )}
    </Button>
    <FileSelector
      onSelect={(objectPath) => {
        if (!formData.attachments.includes(objectPath)) {
          setFormData(prev => ({
            ...prev,
            attachments: [...prev.attachments, objectPath]
          }));
        }
      }}
      excludePaths={formData.attachments}
    />
  </div>

  <div className="flex gap-2 mt-4">
    <input
      value={newAttachment}
      onChange={(e) => setNewAttachment(e.target.value)}
      placeholder="Or paste URL..."
      data-testid="input-new-attachment"
      className="flex h-8 w-full rounded-md border border-input bg-white px-2 py-1 ring-offset-background foc...
      style={{ color: 'black', fontSize: '11px' }}
    />
  </div>

```

```

    />
    <Button
      type="button"
      variant="outline"
      className="h-8"
      style={{ fontSize: '11px' }}
      onClick={() => {
        if (newAttachment.trim()) {
          setFormData(prev => ({
            ...prev,
            attachments: [...prev.attachments, newAttachment.trim()]
          }));
          setNewAttachment("");
        }
      }}
      data-testid="button-add-attachment"
    >
    Add
  </Button>
</div>

{/* Subtasks Section - Only show when editing existing task */}
{task && (
  <SubtasksSection taskId={task.id} />
)}

{/* Dependencies Section - Only show when editing existing task */}
{task && (
  <TaskDependencies taskId={task.id} taskTitle={task.title} />
)}
</div>
</div>

{/* Subtasks Section for NEW tasks - Pending subtasks */}
{!task && (
  <div className="border rounded-lg p-3 space-y-3">
    <div className="flex items-center justify-between">
      <Label className="text-xs font-medium">Subtasks (optional)</Label>
      <span className="text-[10px] text-white/60">{pendingSubtasks.length} subtask{pendingSubtasks.length !== 1...}</span>
    </div>

    {pendingSubtasks.length > 0 && (
      <div className="space-y-1">
        {pendingSubtasks.map((subtask, index) => (
          <div key={index} className="flex items-center gap-2 bg-white/5 rounded px-2 py-1.5">
            <div className="w-3 h-3 rounded-full border border-white/40" />
            <span className="flex-1 text-xs">{subtask}</span>
            <Button
              type="button"
              size="icon"
              variant="ghost"
              className="h-5 w-5 hover:bg-red-500/20"
              onClick={() => setPendingSubtasks(prev => prev.filter((_, i) => i !== index))}
            >
            <X className="h-3 w-3" />
          </Button>
        </div>
      </div>
    )
  )}
</div>
)}

<div className="flex gap-2">
  <Input
    placeholder="Add a subtask..."
    value={newPendingSubtask}
    onChange={(e) => setNewPendingSubtask(e.target.value)}
    className="flex-1 h-8 text-xs bg-black/20 border-white/20"
    onKeyDown={(e) => {
      if (e.key === 'Enter') {
        e.preventDefault();
        if (newPendingSubtask.trim()) {
          setPendingSubtasks(prev => [...prev, newPendingSubtask.trim()]);
          setNewPendingSubtask("");
        }
      }
    }}
  />
  <Button
    type="button"
    size="sm"
    variant="outline"
    className="h-8 text-xs hover:bg-white/10"
    onClick={() => {
      if (newPendingSubtask.trim()) {
        setPendingSubtasks(prev => [...prev, newPendingSubtask.trim()]);
        setNewPendingSubtask("");
      }
    }}
    data-testid="button-add-subtask"
  >
  <Plus className="h-3 w-3 mr-1" />
  Add
</Button>
</div>
</div>
)}

{!hideSubmitButton && (
  <div className="flex justify-end pt-2">
    <button
      type="submit"
      disabled={createMutation.isPending}
      className="inline-flex items-center justify-center rounded-md px-4 py-1.5 text-white transition-opacity d..."
      style={{
        fontSize: '11px',
        border: '1.5px solid rgba(255,255,255,0.6)',
        background: 'linear-gradient(180deg, rgba(255,255,255,0.38) 0%, rgba(255,255,255,0.15) 48%, rgba(255,25...'
      }}
    >
    Submit
  </button>
</div>
)
}

```

```

        boxShadow: 'inset 0 1px 0 rgba(255,255,255,0.6), inset 0 -1px 0 rgba(255,255,255,0.1)'
    })
    onMouseEnter={(e) => { e.currentTarget.style.background = 'linear-gradient(180deg, rgba(255,255,255,0.5) ...
    onMouseLeave={(e) => { e.currentTarget.style.background = 'linear-gradient(180deg, rgba(255,255,255,0.38)...
    data-testid="button-submit-task"
  }
  {createMutation.isPending ? "Saving..." : task ? "Update Task" : "Add Task"}
</button>
</div>
  )}
</form>
  );
}
}

// Subtasks Section Component
function SubtasksSection({ taskId }: { taskId: number }) {
  const [newSubtaskTitle, setNewSubtaskTitle] = useState("");
  const [showAddForm, setShowAddForm] = useState(false);

  // Fetch subtasks for this task - use array queryKey for proper cache invalidation
  const { data: subtasks = [], isLoading, isError } = useQuery<Subtask[]>({
    queryKey: [`/api/tasks/${taskId}/subtasks`],
  });

  // Create subtask mutation using apiRequest
  const createSubtaskMutation = useMutation({
    mutationFn: async (title: string) => {
      return apiRequest("POST", `/api/tasks/${taskId}/subtasks`, { title });
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: [`/api/tasks/${taskId}/subtasks`] });
      setNewSubtaskTitle("");
      setShowAddForm(false);
    },
  });

  // Toggle subtask completion using apiRequest
  const toggleSubtaskMutation = useMutation({
    mutationFn: async ({ id, isCompleted }: { id: number; isCompleted: boolean }) => {
      return apiRequest("PATCH", `/api/subtasks/${id}`, { isCompleted });
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: [`/api/tasks/${taskId}/subtasks`] });
    },
  });

  // Delete subtask mutation using apiRequest
  const deleteSubtaskMutation = useMutation({
    mutationFn: async (id: number) => {
      return apiRequest("DELETE", `/api/subtasks/${id}`);
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: [`/api/tasks/${taskId}/subtasks`] });
    },
  });

  const handleAddSubtask = () => {
    if (newSubtaskTitle.trim()) {
      createSubtaskMutation.mutate(newSubtaskTitle.trim());
    }
  };

  const completedCount = subtasks.filter(s => s.isCompleted).length;
  const totalCount = subtasks.length;

  return (
    <div className="border-t pt-4 mt-4">
      <div className="flex items-center justify-between mb-2">
        <div className="flex items-center gap-2">
          <ListChecks className="h-4 w-4 text-white/70" />
          <Label className="text-[11px] text-white">Subtasks</Label>
          {totalCount > 0 && (
            <span className="text-[10px] text-white/50">
              ({completedCount}/{totalCount} done)
            </span>
          )}
        </div>
        {!showAddForm && (
          <Button
            type="button"
            size="sm"
            variant="ghost"
            onClick={() => setShowAddForm(true)}
            className="h-6 px-2 text-[10px] text-white/70 hover:text-white hover:bg-white/10"
            data-testid="button-add-subtask"
          >
            <Plus className="h-3 w-3 mr-1" />
            Add
          </Button>
        )}
      </div>

      {isLoading ? (
        <div className="text-[10px] text-white/50">Loading subtasks...</div>
      ) : isError ? (
        <div className="text-[10px] text-red-400">Failed to load subtasks</div>
      ) : (
        <div className="space-y-1 max-h-40 overflow-y-auto">
          {subtasks.map((subtask) => (
            <div
              key={subtask.id}
              className="flex items-center gap-2 group p-1 rounded hover:bg-white/5"
              data-testid={`subtask-item-${subtask.id}`}
            >
              <button
                type="button"
                onClick={() => toggleSubtaskMutation.mutate({
                  id: subtask.id,

```



```

        isCompleted: !subtask.isCompleted
      ))}
      className={`w-4 h-4 rounded border flex-shrink-0 flex items-center justify-center transition-colors ${
        subtask.isCompleted
          ? 'bg-green-500 border-green-500'
          : 'border-white/40 hover:border-white/60'
      }`}
      data-testid={`button-toggle-subtask-${subtask.id}`}
    >
      {subtask.isCompleted && <Check className="h-3 w-3 text-white" />}
    </button>
    <span
      className={`flex-1 text-[11px] ${
        subtask.isCompleted ? 'text-white/40 line-through' : 'text-white/90'
      }`}
    >
      {subtask.title}
    </span>
    <Button
      type="button"
      size="icon"
      variant="ghost"
      onClick={() => deleteSubtaskMutation.mutate(subtask.id)}
      className="h-5 w-5 opacity-0 group-hover:opacity-100 text-red-400 hover:text-red-300 hover:bg-red-500..."
      data-testid={`button-delete-subtask-${subtask.id}`}
    >
      <X className="h-3 w-3" />
    </Button>
  </div>
))}

{subtasks.length === 0 && !showAddForm && (
  <div className="text-[10px] text-white/40 py-2">
    No subtasks yet. Click "Add" to create one.
  </div>
)}
</div>
)}

{/* Add subtask form */}
{showAddForm && (
  <div className="flex gap-2 mt-2">
    <input
      type="text"
      value={newSubtaskTitle}
      onChange={(e) => setNewSubtaskTitle(e.target.value)}
      placeholder="Subtask title..."
      onKeyDown={(e) => {
        if (e.key === "Enter") {
          e.preventDefault();
          handleAddSubtask();
        } else if (e.key === "Escape") {
          setShowAddForm(false);
          setNewSubtaskTitle("");
        }
      }}
      autoFocus
      className="flex h-7 flex-1 rounded-md bg-white/10 px-2 py-1 ring-offset-background focus-visible:outline-..."
      style={{ fontSize: '11px' }}
      data-testid="input-new-subtask"
    />
    <Button
      type="button"
      size="sm"
      onClick={handleAddSubtask}
      disabled={!newSubtaskTitle.trim() || createSubtaskMutation.isPending}
      className="h-7 px-2 text-[10px] bg-green-600 hover:bg-green-500 text-white"
      data-testid="button-save-subtask"
    >
      {createSubtaskMutation.isPending ? "...": "Add"}
    </Button>
    <Button
      type="button"
      size="sm"
      variant="ghost"
      onClick={() => {
        setShowAddForm(false);
        setNewSubtaskTitle("");
      }}
      className="h-7 px-2 text-[10px] text-white/70 hover:text-white hover:bg-white/10"
      data-testid="button-cancel-subtask"
    >
      Cancel
    </Button>
  </div>
)}
</div>
);
}

function TaskDependencies({ taskId, taskTitle }: { taskId: number; taskTitle: string }) {
  const [showAddForm, setShowAddForm] = useState(false);
  const [searchQuery, setSearchQuery] = useState("");
  const [selectedLinkType, setSelectedLinkType] = useState<string>("blocks");

  const { data: allLinks = [] } = useQuery<TaskLink[]>({
    queryKey: ['/api/links'],
  });

  const { data: allTasks = [] } = useQuery<Task[]>({
    queryKey: ['/api/tasks'],
  });

  const taskLinks = allLinks.filter(
    l => (l.sourceType === 'task' && l.sourceId === taskId) ||
      (l.targetType === 'task' && l.targetId === taskId)
  );
}

```

```

const createLinkMutation = useMutation({
  mutationFn: async (data: { targetId: number; linkType: string }) => {
    return apiRequest("POST", "/api/links", {
      sourceType: "task",
      sourceId: taskId,
      targetType: "task",
      targetId: data.targetId,
      linkType: data.linkType,
    });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['/api/links'] });
    setShowAddForm(false);
    setSearchQuery("");
  },
});

const deleteLinkMutation = useMutation({
  mutationFn: async (linkId: number) => {
    return apiRequest("DELETE", `/api/links/${linkId}`);
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['/api/links'] });
  },
});

const linkTypeLabels: Record<string, string> = {
  blocks: "Blocks",
  blocked_by: "Blocked by",
  relates_to: "Related to",
};

const linkTypeColors: Record<string, string> = {
  blocks: "text-orange-400",
  blocked_by: "text-red-400",
  relates_to: "text-blue-400",
};

const filteredTasks = allTasks.filter(t => {
  if (t.id === taskId) return false;
  const alreadyLinked = taskLinks.some(
    l => (l.sourceId === t.id && l.sourceType === 'task') ||
      (l.targetId === t.id && l.targetType === 'task')
  );
  if (alreadyLinked) return false;
  if (!searchQuery.trim()) return false;
  const q = searchQuery.toLowerCase();
  return t.title.toLowerCase().includes(q) || (t.courseName || '').toLowerCase().includes(q);
});

const getLinkedTaskInfo = (link: TaskLink) => {
  const linkedId = link.sourceId === taskId && link.sourceType === 'task' ? link.targetId : link.sourceId;
  const linkedTask = allTasks.find(t => t.id === linkedId);
  let displayType = link.linkType;
  if (link.targetId === taskId && link.targetType === 'task') {
    if (link.linkType === 'blocks') displayType = 'blocked_by';
    else if (link.linkType === 'blocked_by') displayType = 'blocks';
  }
  return { linkedTask, displayType };
};

return (
  <div className="space-y-2 pt-2 border-t border-white/10">
    <div className="flex items-center justify-between">
      <label className="text-xs text-white/60 font-medium">Dependencies</label>
      <Button
        type="button"
        variant="ghost"
        size="sm"
        className="h-6 px-2 text-xs text-white/60 hover:text-white hover:bg-white/10"
        onClick={() => setShowAddForm(!showAddForm)}
        data-testid="button-toggle-dependencies"
      >
        {showAddForm ? <X className="h-3 w-3" /> : <Plus className="h-3 w-3 mr-1" />}
        {showAddForm ? "" : "Add"}
      </Button>
    </div>

    {taskLinks.length === 0 && !showAddForm && (
      <p className="text-xs text-white/30 italic">No dependencies</p>
    )}

    {taskLinks.map(link => {
      const { linkedTask, displayType } = getLinkedTaskInfo(link);
      if (!linkedTask) return null;
      return (
        <div key={link.id} className="flex items-center gap-2 rounded px-2 py-1.5 bg-white/5" data-testid={`depende...`}
        <span className={`text-[10px] font-medium uppercase tracking-wider ${linkTypeColors[displayType]} || 'text...'`}
          {linkTypeLabels[displayType]} || displayType
        </span>
        <span className="text-xs text-white/80 truncate flex-1">{linkedTask.title}</span>
        {linkedTask.courseName && (
          <span className="text-[10px] text-white/40 truncate max-w-[120px]">{linkedTask.courseName.split(' - ')[...]}
        </span>
        <button
          type="button"
          onClick={() => deleteLinkMutation.mutate(link.id)}
          className="shrink-0 p-0.5 rounded hover:bg-red-500/20 text-white/30 hover:text-red-400"
          data-testid={`button-delete-dependency-${link.id}`}
        >
          <X className="h-3 w-3" />
        </button>
        </div>
      );
    })}

    {showAddForm && (
      <div className="space-y-2 p-2 rounded bg-white/5 border border-white/10">

```

```

<div className="flex gap-2">
  <select
    value={selectedLinkType}
    onChange={(e) => setSelectedLinkType(e.target.value)}
    className="h-7 text-xs rounded bg-white/10 text-white px-2"
    data-testid="select-link-type"
  >
    <option value="blocks">Blocks</option>
    <option value="blocked_by">Blocked by</option>
    <option value="relates_to">Related to</option>
  </select>
  <input
    type="text"
    value={searchQuery}
    onChange={(e) => setSearchQuery(e.target.value)}
    placeholder="Search tasks..."
    className="flex-1 h-7 text-xs rounded bg-white/10 text-white px-2 placeholder:text-white/30"
    autoFocus
    data-testid="input-search-dependency"
  />
</div>

{searchQuery.trim() && (
  <div className="max-h-[150px] overflow-y-auto space-y-0.5">
    {filteredTasks.length === 0 ? (
      <p className="text-xs text-white/30 py-1 px-1">No matching tasks</p>
    ) : (
      filteredTasks.slice(0, 10).map(t => (
        <button
          key={t.id}
          type="button"
          className="w-full flex items-center gap-2 px-2 py-1.5 rounded text-left hover:bg-white/10 transit..."
          onClick={() => createLinkMutation.mutate({ targetId: t.id, linkType: selectedLinkType })}
          data-testid={`button-link-task-${t.id}`}
        >
          <span className="text-xs text-white/80 truncate flex-1">{t.title}</span>
          {t.courseName && (
            <span className="text-[10px] text-white/40 truncate max-w-[100px]">{t.courseName.split(' - ')[0...
            )}
            {t.dueDate && (
              <span className="text-[10px] text-white/30">{format(new Date(t.dueDate), 'MMM d')}</span>
            )}
          </button>
        )}
      )}
    </div>
  )}
</div>
)}
</div>
);
}

function RescheduleForm({
  task,
  onSuccess,
  onUndoPush
}): {
  task: Task;
  onSuccess: () => void;
  onUndoPush?: (action: { type: 'edit'; description: string; data: { taskId: number; taskTitle: string; oldFields: Re...
}) {
  const [newDate, setNewDate] = useState("");
  const [newWeek, setNewWeek] = useState(task.weekNumber);

  const rescheduleMutation = useMutation({
    mutationFn: async () => {
      if (onUndoPush) {
        onUndoPush({
          type: 'edit',
          description: `Rescheduled "${task.title}"`,
          data: {
            taskId: task.id,
            taskTitle: task.title,
            oldFields: { dueDate: task.dueDate instanceof Date ? task.dueDate.toISOString() : task.dueDate, weekNumbe...
            newFields: { dueDate: newDate, weekNumber: newWeek }
          }
        });
      }
    }
  });

  return apiRequest("PATCH", `/api/tasks/${task.id}/reschedule`, {
    dueDate: newDate,
    weekNumber: newWeek,
  });
},
onSuccess: () => {
  queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
  queryClient.invalidateQueries({ queryKey: ["/api/weeks"] });
  onSuccess();
},
});

return (
  <div className="space-y-4">
    <p className="text-sm text-muted-foreground">
      Reschedule "${task.title}" to a new date and week.
    </p>

    <div>
      <Label htmlFor="newDate">New Due Date & Time</Label>
      <Input
        id="newDate"
        type="datetime-local"
        value={newDate}
        onChange={(e) => setNewDate(e.target.value)}
        required
        data-testid="input-reschedule-date"
      />
    </div>
  </div>
)

```

```

<div>
  <Label htmlFor="newWeek">Week Number</Label>
  <Select value={String(newWeek)} onChange={(v) => setNewWeek(Number(v))}>
    <SelectTrigger data-testid="select-reschedule-week">
      <SelectValue />
    </SelectTrigger>
    <SelectContent>
      {Array.from({ length: 12 }, (_, i) => i + 2).map(w => (
        <SelectItem key={w} value={String(w)}>Week {w}</SelectItem>
      ))}
    </SelectContent>
  </Select>
</div>

<Button
  onClick={() => rescheduleMutation.mutate()}
  disabled={!newDate || rescheduleMutation.isPending}
  data-testid="button-confirm-reschedule"
>
  {rescheduleMutation.isPending ? "Saving..." : "Reschedule"}
</Button>
</div>
);
}

```

`client/src/pages/files.tsx`

```

import { useState, DragEvent, useRef, useCallback, useEffect, useMemo } from "react";
import { useQuery, useMutation } from "@tanstack/react-query";
import { queryClient, apiRequest } from "@lib/queryClient";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { parseISO } from "date-fns";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Badge } from "@components/ui/badge";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@components/ui/dialog";
import { ContextMenu, ContextMenuContent, ContextMenuItem, ContextMenuTrigger } from "@components/ui/context-menu";
import { useToast } from "@hooks/use-toast";
import { useUpload } from "@hooks/use-upload";
import { ObjectUploader } from "@components/ObjectUploader";
import { Link } from "wouter";
import {
  FileText,
  File,
  Image,
  Video,
  Music,
  Archive,
  Edit2,
  Link2,
  Trash2,
  ArrowLeft,
  Download,
  Clock,
  CheckCircle2,
  Upload,
  Play,
  Square,
  Volume2,
  VolumeX,
  Folder,
  FolderOpen,
  Loader2,
  ChevronRight,
  ChevronDown,
  ChevronLeft,
  Plus,
  FolderPlus,
  SkipBack,
  SkipForward,
  RotateCcw,
  Minus,
  BookOpen,
  Home,
  Cloud,
  RefreshCw
} from "lucide-react";
import { Checkbox } from "@components/ui/checkbox";
import type { FileRecord } from "@shared/schema";

interface Task {
  id: number;
  title: string;
  courseName: string | null;
  dueDate: string;
  isCompleted: boolean;
  attachments: string[] | null;
}

const WEEKS = [
  { id: "week-1", name: "Week 1" },
  { id: "week-2", name: "Week 2" },
  { id: "week-3", name: "Week 3" },
  { id: "week-4", name: "Week 4" },
  { id: "week-5", name: "Week 5" },
  { id: "week-6", name: "Week 6" },
  { id: "week-7", name: "Week 7" },
  { id: "week-8", name: "Week 8" },
  { id: "week-9", name: "Week 9" },
  { id: "week-10", name: "Week 10" },
  { id: "week-11", name: "Week 11" },
  { id: "week-12", name: "Week 12" },

```

```

    { id: "week-13", name: "Week 13" },
    { id: "other", name: "Other" },
  ];

const COURSE_FOLDERS = [
  { id: "cas101", name: "CASL101 - ASL", color: "text-purple-500" },
  { id: "cfnf400", name: "CFNF400 - Sexuality", color: "text-pink-500" },
  { id: "cppa122", name: "CPPA122 - Local Politics", color: "text-green-500" },
];

const CONTENT_FOLDERS = [
  { id: "module", name: "Module" },
  { id: "reading", name: "Reading" },
];

function getFileIcon(contentType: string | null) {
  if (!contentType) return File;
  if (contentType.startsWith("image/")) return Image;
  if (contentType.startsWith("video/")) return Video;
  if (contentType.startsWith("audio/")) return Music;
  if (contentType.includes("pdf") || contentType.includes("document") || contentType.includes("text")) return FileText;
  if (contentType.includes("zip") || contentType.includes("archive")) return Archive;
  return File;
}

function formatFileSize(bytes: number | null) {
  if (!bytes) return "Unknown size";
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`;
  return `${(bytes / (1024 * 1024)).toFixed(1)} MB`;
}

function formatDate(dateValue: string | Date | null) {
  if (!dateValue) return "Unknown date";
  const date = typeof dateValue === "string" ? new Date(dateValue) : dateValue;
  return date.toLocaleDateString("en-US", {
    month: "short",
    day: "numeric",
    year: "numeric",
    hour: "2-digit",
    minute: "2-digit",
  });
}

type SortOption = "name-asc" | "name-desc" | "date-newest" | "date-oldest" | "size-largest" | "size-smallest";

const SPEAKERS = [
  { id: "media_player.byhome", name: "Apartment" },
  { id: "media_player.cat_wr", name: "Cat Washroom Speakers" },
  { id: "media_player.echo_cat_left_am", name: "Cat Washroom Left" },
  { id: "media_player.echo_cat_right_am", name: "Cat Washroom Right" },
  { id: "media_player.echo_cat_washroom_middle", name: "Cat Washroom Middle" },
  { id: "media_player.echo_closet_am", name: "Closet" },
  { id: "media_player.echo_lr_couch_r_am", name: "Echo Corner" },
  { id: "media_player.echo_hallway_entrance_am", name: "Hallway Entrance" },
  { id: "media_player.echo_king_l_am", name: "King Left" },
  { id: "media_player.echo_king_r_am", name: "King Right" },
  { id: "media_player.echo_king_tv_am", name: "King TV" },
  { id: "media_player.echo_kitchen_cupboards_left_am", name: "Kitchen Cupboards Left" },
  { id: "media_player.echo_kitchen_cupboards_r_am", name: "Kitchen Cupboards Right" },
  { id: "media_player.echo_kitchen_fridge_am", name: "Kitchen Fridge" },
  { id: "media_player.echo_kitchen_hutch_am", name: "Kitchen Hutch" },
  { id: "media_player.echo_kitchen_island_corner_am", name: "Kitchen Island Corner" },
  { id: "media_player.echo_kitchen_studio_black_am", name: "Kitchen Studio Black" },
  { id: "media_player.echo_lr_couch_l_am", name: "Living Room Couch Left" },
  { id: "media_player.echo_lr_hub_am", name: "Living Room Hub" },
  { id: "media_player.echo_lr_studio_white_am", name: "Living Room Studio White" },
  { id: "media_player.echo_lr_tv_shelf_am", name: "Living Room TV Shelf" },
  { id: "media_player.echo_queen_balcony_am", name: "Queen Balcony" },
  { id: "media_player.echo_queen_bed_l_am", name: "Queen Bed Left" },
  { id: "media_player.echo_queen_bed_r_am", name: "Queen Bed Right" },
  { id: "media_player.echo_show_pug_am", name: "Echo Show Pug" },
  { id: "media_player.everywhere_2", name: "Everywhere" },
  { id: "media_player.hallway", name: "Hallway" },
  { id: "media_player.king_bedroom", name: "King Bedroom" },
  { id: "media_player.queen_bedroom", name: "Queen Bedroom" },
];

interface OneDriveItem {
  id: string;
  name: string;
  type: "folder" | "file";
  mimeType?: string;
  size?: number;
  lastModified?: string;
  downloadUrl?: string;
  path: string;
}

type ViewMode = "all" | "week";

export default function FilesPage() {
  const { toast } = useToast();
  const [viewMode, setViewMode] = useState<ViewMode>("all");
  const [oneDrivePath, setOneDrivePath] = useState("/School/1. TMU/Courses/2026/Winter");
  const [oneDrivePathHistory, setOneDrivePathHistory] = useState<string[]>([]);
  const [editingFile, setEditingFile] = useState<FileRecord | null>(null);
  const [newName, setNewName] = useState("");
  const [assigningFile, setAssigningFile] = useState<FileRecord | null>(null);
  const [selectedTaskId, setSelectedTaskId] = useState<string>("");
  const [showAssignAfterUpload, setShowAssignAfterUpload] = useState(false);
  const [lastUploadedObjectPath, setLastUploadedObjectPath] = useState<string>("");
  const [sortBy, setSortBy] = useState<SortOption>("date-newest");
  const [fileSpeakers, setFileSpeakers] = useState<Record<number, string>>({});
  const [expandedFolders, setExpandedFolders] = useState<Set<string>>(new Set());
  const [draggedFileId, setDraggedFileId] = useState<number | null>(null);
  const [dragOverFolder, setDragOverFolder] = useState<string | null>(null);
  const [isExternalDragOver, setIsExternalDragOver] = useState(false);

```

```

const [uploadingCount, setUploadingCount] = useState(0);
const [selectedFolder, setSelectedFolder] = useState<string | null>(null);
const [showDeleteFolderConfirm, setShowDeleteFolderConfirm] = useState(false);
const [showAddFolderDialog, setShowAddFolderDialog] = useState(false);
const [newFolderName, setNewFolderName] = useState("");
const [addFolderParentId, setAddFolderParentId] = useState<string | null>(null);
const [editingFolderId, setEditingFolderId] = useState<number | null>(null);
const [editingFolderName, setEditingFolderName] = useState("");
const [editingBuiltinFolderId, setEditingBuiltinFolderId] = useState<string | null>(null);
const [editingBuiltinFolderName, setEditingBuiltinFolderName] = useState("");
const [folderDisplayNames, setFolderDisplayNames] = useState<Record<string, string>>({}) => {
  const saved = localStorage.getItem("folderDisplayNames");
  return saved ? JSON.parse(saved) : {};
};
const [foldersPanelWidth, setFoldersPanelWidth] = useState(224); // 224px = w-56
const isResizingPanelRef = useRef(false);

// Column widths state for draggable resizing
const [columnWidths, setColumnWidths] = useState({
  name: 300,
  actions: 96,
  status: 80,
  date: 144,
  type: 112,
  size: 80,
});
const resizingColumn = useRef<string | null>(null);
const startX = useRef<number>(0);
const startWidth = useRef<number>(0);

const handleMouseDown = useCallback((e: React.MouseEvent, column: string) => {
  e.preventDefault();
  resizingColumn.current = column;
  startX.current = e.clientX;
  startWidth.current = columnWidths[column as keyof typeof columnWidths];
  document.addEventListener('mousemove', handleMouseMove);
  document.addEventListener('mouseup', handleMouseUp);
}, [columnWidths]);

const handleMouseMove = useCallback((e: MouseEvent) => {
  if (!resizingColumn.current) return;
  const diff = e.clientX - startX.current;
  const newWidth = Math.max(50, startWidth.current + diff);
  setColumnWidths(prev => ({
    ...prev,
    [resizingColumn.current!]: newWidth,
  }));
}, []);

const handleMouseUp = useCallback(() => {
  resizingColumn.current = null;
  document.removeEventListener('mousemove', handleMouseMove);
  document.removeEventListener('mouseup', handleMouseUp);
}, [handleMouseMove]);

useEffect(() => {
  return () => {
    document.removeEventListener('mousemove', handleMouseMove);
    document.removeEventListener('mouseup', handleMouseUp);
  };
}, [handleMouseMove, handleMouseUp]);

// Panel resize handlers
const panelStartX = useRef<number>(0);
const panelStartWidth = useRef<number>(0);

const handlePanelMouseMove = useCallback((e: MouseEvent) => {
  if (!isResizingPanelRef.current) return;
  const diff = e.clientX - panelStartX.current;
  const newWidth = Math.max(150, Math.min(500, panelStartWidth.current + diff));
  setFoldersPanelWidth(newWidth);
}, []);

const handlePanelMouseUp = useCallback(() => {
  isResizingPanelRef.current = false;
  document.removeEventListener('mousemove', handlePanelMouseMove);
  document.removeEventListener('mouseup', handlePanelMouseUp);
}, [handlePanelMouseMove]);

const handlePanelMouseDown = useCallback((e: React.MouseEvent) => {
  e.preventDefault();
  isResizingPanelRef.current = true;
  panelStartX.current = e.clientX;
  panelStartWidth.current = foldersPanelWidth;
  document.addEventListener('mousemove', handlePanelMouseMove);
  document.addEventListener('mouseup', handlePanelMouseUp);
}, [foldersPanelWidth, handlePanelMouseMove, handlePanelMouseUp]);

useEffect(() => {
  return () => {
    document.removeEventListener('mousemove', handlePanelMouseMove);
    document.removeEventListener('mouseup', handlePanelMouseUp);
  };
}, [handlePanelMouseMove, handlePanelMouseUp]);

const { getUploadParameters, uploadFile } = useUpload({
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/files"] });
  },
});

const { data: files = [], isLoading: filesLoading } = useQuery<FileRecord[]>({
  queryKey: ["/api/files"],
});

const { data: tasks = [], isLoading: tasksLoading } = useQuery<Task[]>({
  queryKey: ["/api/tasks"],
});

```

```

const { data: deletedFoldersData = [] } = useQuery<{ id: number; folderId: string }[]>({
  queryKey: ["/api/deleted-folders"],
});

// Fetch weeks to determine current week for sorting
interface Week {
  weekNumber: number;
  startDate: string;
  endDate: string;
}
const { data: weeks = [] } = useQuery<Week[]>({
  queryKey: ["/api/weeks"],
  queryFn: () => fetch("/api/weeks").then(r => { if (!r.ok) throw new Error(r.statusText); return r.json(); }),
});

const { data: oneDriveItems = [], isLoading: oneDriveLoading } = useQuery<OneDriveItem[]>({
  queryKey: ["/api/onedrive/files", oneDrivePath],
  queryFn: async () => {
    const response = await fetch(`/api/onedrive/files?path=${encodeURIComponent(oneDrivePath)}`);
    if (!response.ok) {
      const err = await response.json();
      throw new Error(err.error || "Failed to load files");
    }
    return response.json();
  },
  enabled: viewMode === "week",
});

const oneDriveFolders = oneDriveItems
  .filter(item => item.type === "folder")
  .sort((a, b) => {
    const courseOrder = ['CPPA122', 'CFNF400', 'CASL101'];
    const getCourseIndex = (name: string) => {
      for (let i = 0; i < courseOrder.length; i++) {
        if (name.includes(courseOrder[i])) return i;
      }
      return -1;
    };
    const courseIdxA = getCourseIndex(a.name);
    const courseIdxB = getCourseIndex(b.name);
    if (courseIdxA !== -1 && courseIdxB !== -1) return courseIdxA - courseIdxB;
    if (courseIdxA !== -1) return -1;
    if (courseIdxB !== -1) return 1;
    const weekMatch = (name: string) => {
      const match = name.match(/week\s*(\d+)/i);
      return match ? parseInt(match[1], 10) : null;
    };
    const weekA = weekMatch(a.name);
    const weekB = weekMatch(b.name);
    if (weekA !== null && weekB !== null) {
      return weekA - weekB;
    }
    if (weekA !== null) return -1;
    if (weekB !== null) return 1;
    return a.name.localeCompare(b.name);
  });
const oneDriveFiles = oneDriveItems.filter(item => item.type === "file");
const oneDrivePdfFiles = oneDriveFiles.filter(item => item.mimeType?.includes("pdf"));

const courseColorMap: Record<string, string> = {
  'CPPA122': '#47B045',
  'CFNF400': '#FA67B3',
  'CASL101': '#6366f1',
};

const getOneDriveFolderColor = (folderName: string): string | undefined => {
  for (const [courseCode, color] of Object.entries(courseColorMap)) {
    if (oneDrivePath.includes(courseCode)) return color;
  }
  for (const [courseCode, color] of Object.entries(courseColorMap)) {
    if (folderName.includes(courseCode)) return color;
  }
  return undefined;
};

// Helper to get week number from week id
const getWeekNumberFromId = (weekId: string): number | null => {
  if (weekId === "other") return null;
  const match = weekId.match(/^week-(\d+)$/);
  return match ? parseInt(match[1], 10) : null;
};

// Check if a week is past based on its end date
const isWeekPast = useCallback((weekId: string): boolean => {
  const weekNum = getWeekNumberFromId(weekId);
  if (weekNum === null) return false; // "other" folder is never past

  // Find the week data from API
  const weekData = weeks.find(w => w.weekNumber === weekNum);
  if (!weekData) return false;

  const today = new Date();
  const todayDateOnly = new Date(today.getFullYear(), today.getMonth(), today.getDate());
  const endDate = parseISO(weekData.endDate);
  const endDateOnly = new Date(endDate.getFullYear(), endDate.getMonth(), endDate.getDate());

  // Week is past if today is after the week's end date
  return todayDateOnly > endDateOnly;
}, [weeks]);

// Check if a week is the current week (today falls between start and end)
const isCurrentWeek = useCallback((weekId: string): boolean => {
  const weekNum = getWeekNumberFromId(weekId);
  if (weekNum === null) return false;

  const weekData = weeks.find(w => w.weekNumber === weekNum);
  if (!weekData) return false;

```

```

const today = new Date();
const todayDateOnly = new Date(today.getFullYear(), today.getMonth(), today.getDate());
const startDate = parseISO(weekData.startDate);
const startDateOnly = new Date(startDate.getFullYear(), startDate.getMonth(), startDate.getDate());
const endDate = parseISO(weekData.endDate);
const endDateOnly = new Date(endDate.getFullYear(), endDate.getMonth(), endDate.getDate());

return todayDateOnly >= startDateOnly && todayDateOnly <= endDateOnly;
}, [weeks]);

// Sort weeks: current and future weeks first (in order), then past weeks (in order)
const sortedWeeks = useMemo(() => {
  return [...WEEKS].sort((a, b) => {
    const aNum = getWeekNumberFromId(a.id);
    const bNum = getWeekNumberFromId(b.id);

    // "Other" folder always at the end
    if (a.id === "other") return 1;
    if (b.id === "other") return -1;

    if (aNum === null || bNum === null) return 0;

    const aIsPast = isWeekPast(a.id);
    const bIsPast = isWeekPast(b.id);

    // If one is past and one is not, the past one goes to the bottom
    if (aIsPast && !bIsPast) return 1;
    if (!aIsPast && bIsPast) return -1;

    // Both past or both current/future: sort by week number
    return aNum - bNum;
  });
}, [weeks, isWeekPast]);

const deletedFolderIds = new Set(deletedFoldersData.map(f => f.folderId));

const renameMutation = useMutation({
  mutationFn: async ({ id, displayName }: { id: number; displayName: string }) => {
    return await apiRequest("PATCH", `/api/files/${id}`, { displayName });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["api/files"] });
    setEditingFile(null);
    toast({ title: "File renamed successfully" });
  },
  onError: (err) => {
    toast({ title: "Failed to rename file", description: String(err), variant: "destructive" });
  },
});

const listenedMutation = useMutation({
  mutationFn: async ({ id, listened }: { id: number; listened: boolean }) => {
    return await apiRequest("PATCH", `/api/files/${id}`, { listened });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["api/files"] });
  },
  onError: (err) => {
    toast({ title: "Failed to update file", description: String(err), variant: "destructive" });
  },
});

const moveFolderMutation = useMutation({
  mutationFn: async ({ id, folder }: { id: number; folder: string | null }) => {
    // If moving to a folder that was deleted, restore it first
    if (folder && deletedFolderIds.has(folder)) {
      await apiRequest("DELETE", `/api/deleted-folders/${encodeURIComponent(folder)}`);
    }
    return await apiRequest("PATCH", `/api/files/${id}`, { folder });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["api/files"] });
    queryClient.invalidateQueries({ queryKey: ["api/deleted-folders"] });
    toast({ title: "File moved to folder" });
  },
  onError: (err) => {
    toast({ title: "Failed to move file", description: String(err), variant: "destructive" });
  },
});

const assignMutation = useMutation({
  mutationFn: async ({ fileId, taskId }: { fileId: number; taskId: number }) => {
    return await apiRequest("POST", `/api/files/${fileId}/assign`, { taskId });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["api/files"] });
    queryClient.invalidateQueries({ queryKey: ["api/tasks"] });
    setAssigningFile(null);
    setSelectedTaskId("");
    toast({ title: "File assigned to task" });
  },
  onError: (err) => {
    toast({ title: "Failed to assign file", description: String(err), variant: "destructive" });
  },
});

const deleteMutation = useMutation({
  mutationFn: async (id: number) => {
    return await apiRequest("DELETE", `/api/files/${id}`);
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["api/files"] });
    toast({ title: "File deleted" });
  },
  onError: (err) => {
    toast({ title: "Failed to delete file", description: String(err), variant: "destructive" });
  },
});

```



```

});

const addDeletedFolderMutation = useMutation({
  mutationFn: async (folderId: string) => {
    return await apiRequest("POST", "/api/deleted-folders", { folderId });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/deleted-folders"] });
  },
  onError: (err) => {
    toast({ title: "Failed to delete folder", description: String(err), variant: "destructive" });
  },
});

const restoreFolderMutation = useMutation({
  mutationFn: async (folderId: string) => {
    return await apiRequest("DELETE", `/api/deleted-folders/${encodeURIComponent(folderId)}`);
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/deleted-folders"] });
  },
});

const { data: customFoldersData = [] } = useQuery<{ id: number; parentFolderId: string; name: string }[]>({
  queryKey: ["/api/custom-folders"],
});

const createCustomFolderMutation = useMutation({
  mutationFn: async ({ parentFolderId, name }: { parentFolderId: string; name: string }) => {
    return await apiRequest("POST", "/api/custom-folders", { parentFolderId, name });
  },
  onSuccess: (_data, variables) => {
    queryClient.invalidateQueries({ queryKey: ["/api/custom-folders"] });
    // Expand the parent folder and all ancestors so the new folder is visible
    const parentId = variables.parentFolderId;
    setExpandedFolders(prev => {
      const next = new Set(prev);
      next.add(parentId);
      // Expand ancestor folders (parse the hierarchy)
      const parts = parentId.split("-");
      // e.g. "week-1-cppa122-module" -> expand "week-1", "week-1-cppa122"
      for (let i = 2; i <= parts.length; i++) {
        next.add(parts.slice(0, i).join("-"));
      }
      return next;
    });
    setShowAddFolderDialog(false);
    setNewFolderName("");
    setAddFolderParentId(null);
    toast({ title: "Folder created successfully" });
  },
  onError: (err) => {
    toast({ title: "Failed to create folder", description: String(err), variant: "destructive" });
  },
});

const deleteCustomFolderMutation = useMutation({
  mutationFn: async (id: number) => {
    return await apiRequest("DELETE", `/api/custom-folders/${id}`);
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/custom-folders"] });
    toast({ title: "Folder deleted" });
  },
  onError: (err) => {
    toast({ title: "Failed to delete folder", description: String(err), variant: "destructive" });
  },
});

const renameCustomFolderMutation = useMutation({
  mutationFn: async ({ id, name }: { id: number; name: string }) => {
    return await apiRequest("PATCH", `/api/custom-folders/${id}`, { name });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/custom-folders"] });
    setEditingFolderId(null);
    setEditingFolderName("");
    toast({ title: "Folder renamed" });
  },
  onError: (err) => {
    toast({ title: "Failed to rename folder", description: String(err), variant: "destructive" });
  },
});

const getFilesToDeleteInFolder = (folderId: string) => {
  // Check if this is a week folder (e.g., "week-1")
  if (folderId.match(/^week-\d+$/)) {
    return files.filter(f => f.folder?.startsWith(folderId + "-"));
  }
  // Check if this is a course folder (e.g., "week-1-cppa122")
  const parts = folderId.split("-");
  if (parts.length === 3 && parts[0] === "week") {
    return files.filter(f => f.folder?.startsWith(folderId + "-"));
  }
  // Content folder - exact match
  return files.filter(f => f.folder === folderId);
};

// Keyboard event listener for Delete key to delete selected folder
useEffect(() => {
  const handleKeyDown = (e: KeyboardEvent) => {
    if (e.key === "Delete" && selectedFolder && !editingFile && !assigningFile) {
      e.preventDefault();
      // Check if folder has contents - if empty, delete immediately without confirmation
      const folderFiles = getFilesToDeleteInFolder(selectedFolder);
      if (folderFiles.length === 0) {
        // Empty folder - delete immediately
        addDeletedFolderMutation.mutate(selectedFolder);
      }
    }
  };
  document.addEventListener("keydown", handleKeyDown);
  return () => {
    document.removeEventListener("keydown", handleKeyDown);
  };
}, [selectedFolder, editingFile, assigningFile]);

```

```

        toast({ title: "Folder deleted" });
        setSelectedFolder(null);
    } else {
        // Folder has contents - show confirmation
        setShowDeleteFolderConfirm(true);
    }
}
};

document.addEventListener("keydown", handleKeyDown);
return () => document.removeEventListener("keydown", handleKeyDown);
}, [selectedFolder, editingFile, assigningFile, files, addDeletedFolderMutation, toast]);

const saveFolderDisplayName = useCallback((folderId: string, name: string) => {
    const newNames = { ...folderDisplayNames, [folderId]: name };
    setFolderDisplayNames(newNames);
    localStorage.setItem("folderDisplayNames", JSON.stringify(newNames));
    setEditingBuiltinFolderId(null);
    setEditingBuiltinFolderName("");
    toast({ title: "Folder renamed" });
}, [folderDisplayNames, toast]);

const handleDeleteFolder = async () => {
    if (!selectedFolder) return;

    const folderFiles = getFilesToDeleteInFolder(selectedFolder);

    try {
        // Delete all files in the folder
        for (const file of folderFiles) {
            await apiRequest("DELETE", `/api/files/${file.id}`);
        }
        // Add folder to deleted list
        await apiRequest("POST", "/api/deleted-folders", { folderId: selectedFolder });
        queryClient.invalidateQueries({ queryKey: ["/api/files"] });
        queryClient.invalidateQueries({ queryKey: ["/api/deleted-folders"] });
        toast({ title: folderFiles.length > 0 ? `Deleted folder and ${folderFiles.length} file(s) : "Folder deleted" });
        setSelectedFolder(null);
    } catch (err) {
        toast({ title: "Failed to delete folder contents", description: String(err), variant: "destructive" });
    }
    setShowDeleteFolderConfirm(false);
};

const getSpeakerForFile = (fileId: number) => fileSpeakers[fileId] || "media_player.echo_cat_left_am";

const setSpeakerForFile = (fileId: number, speakerId: string) => {
    setFileSpeakers(prev => ({ ...prev, [fileId]: speakerId }));
};

const handlePlayFile = async (fileId: number, fileUrl: string, fileName: string) => {
    const speaker = getSpeakerForFile(fileId);
    try {
        const response = await fetch("/api/media/play", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ mediaUrl: fileUrl, entityId: speaker }),
        });
        if (response.ok) {
            const speakerName = SPEAKERS.find(s => s.id === speaker)?.name || speaker;
            toast({ title: `Playing on ${speakerName}: ${fileName}` });
        } else {
            toast({ title: "Failed to play file", variant: "destructive" });
        }
    } catch (error) {
        console.error("Play error:", error);
        toast({ title: "Failed to play file", variant: "destructive" });
    }
};

const handleStop = async (fileId: number) => {
    const speaker = getSpeakerForFile(fileId);
    try {
        await fetch("/api/media/stop", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ entityId: speaker }),
        });
    } catch (error) {
        console.error("Stop error:", error);
    }
};

const handleVolume = async (fileId: number, action: "up" | "down") => {
    const speaker = getSpeakerForFile(fileId);
    try {
        await fetch("/api/media/volume", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ action, entityId: speaker }),
        });
    } catch (error) {
        console.error("Volume error:", error);
    }
};

const handleRestart = async (fileId: number, fileUrl: string, fileName: string) => {
    const speaker = getSpeakerForFile(fileId);
    try {
        // First clear the session/stop
        await fetch("/api/media/restart", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ mediaUrl: fileUrl, entityId: speaker }),
        });
        // Then start playback from beginning
        await handlePlayFile(fileId, fileUrl, fileName);
    } catch (error) {

```

```

        console.error("Restart error:", error);
      }
      toast({ title: "Failed to restart", variant: "destructive" });
    }
  };

const handleSkipChunk = async (fileId: number, direction: "forward" | "backward") => {
  const speaker = getSpeakerForFile(fileId);
  try {
    await fetch("/api/media/skip-chunk", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ direction, entityId: speaker }),
    });
  } catch (error) {
    console.error("Skip chunk error:", error);
  }
};

const handleRename = () => {
  if (editingFile && newName.trim()) {
    renameMutation.mutate({ id: editingFile.id, displayName: newName.trim() });
  }
};

const handleAssign = () => {
  if (assigningFile && selectedTaskId) {
    assignMutation.mutate({ fileId: assigningFile.id, taskId: parseInt(selectedTaskId) });
  }
};

const assignByPathMutation = useMutation({
  mutationFn: async ({ objectPath, taskId }: { objectPath: string; taskId: number }) => {
    return await apiRequest("POST", `/api/files/assign-by-path`, { objectPath, taskId });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/files"] });
    queryClient.invalidateQueries({ queryKey: ["/api/tasks"] });
    setShowAssignAfterUpload(false);
    setSelectedTaskId("");
    setLastUploadedObjectPath("");
    toast({ title: "File assigned to task" });
  },
  onError: (err) => {
    toast({ title: "Failed to assign file", description: String(err), variant: "destructive" });
  },
});

const handleAssignAfterUpload = () => {
  if (lastUploadedObjectPath && selectedTaskId) {
    assignByPathMutation.mutate({ objectPath: lastUploadedObjectPath, taskId: parseInt(selectedTaskId) });
  }
};

const handleUploadComplete = () => {
  queryClient.invalidateQueries({ queryKey: ["/api/files"] });
  toast({ title: "File uploaded successfully" });
};

const getTasksForFile = (file: FileRecord): Task[] => {
  return tasks.filter(task =>
    task.attachments && task.attachments.includes(file.objectPath)
  );
};

const toggleFolder = (folderId: string) => {
  setExpandedFolders(prev => {
    const newSet = new Set(prev);
    if (newSet.has(folderId)) {
      newSet.delete(folderId);
    } else {
      newSet.add(folderId);
    }
    return newSet;
  });
};

const handleDragStart = (e: DragEvent<HTMLDivElement>, fileId: number) => {
  e.dataTransfer.effectAllowed = "move";
  e.dataTransfer.setData("text/plain", fileId.toString());
  setDraggedFileId(fileId);
};

const handleDragEnd = () => {
  setDraggedFileId(null);
  setDragOverFolder(null);
};

const handleDragOver = (e: DragEvent<HTMLDivElement>, folderId: string) => {
  e.preventDefault();
  e.stopPropagation();
  // Check if this is an external file drop
  if (e.dataTransfer.types.includes("Files")) {
    e.dataTransfer.dropEffect = "copy";
    setIsExternalDragOver(true);
  } else {
    e.dataTransfer.dropEffect = "move";
  }
  if (dragOverFolder !== folderId) {
    setDragOverFolder(folderId);
  }
};

const handleDragLeave = (e: DragEvent<HTMLDivElement>) => {
  e.stopPropagation();
  // Only clear if we're leaving to a non-child element
  const relatedTarget = e.relatedTarget as HTMLElement;
  if (!relatedTarget || !relatedTarget.contains(e.currentTarget)) {
    setDragOverFolder(null);
  }
};

```

```

    setIsExternalDragOver(false);
  }
};

const handleExternalFileDrop = async (files: FileList, targetFolder: string | null) => {
  const fileArray = Array.from(files);
  setUploadingCount(fileArray.length);

  for (const file of fileArray) {
    try {
      const response = await uploadFile(file);
      if (response && targetFolder) {
        // Move the uploaded file to the target folder
        // We need to find the file by objectPath after it's created
        const filesResponse = await fetch("/api/files");
        const allFiles = await filesResponse.json();
        const uploadedFile = allFiles.find((f: FileRecord) => f.objectPath === response.objectPath);
        if (uploadedFile) {
          await apiRequest("PATCH", `/api/files/${uploadedFile.id}`, { folder: targetFolder });
        }
      }
      queryClient.invalidateQueries({ queryKey: ["/api/files"] });
    } catch (error) {
      console.error("Upload error:", error);
      toast({ title: "Failed to upload file", variant: "destructive" });
    }
  }

  setUploadingCount(0);
  toast({ title: `${fileArray.length} file(s) uploaded successfully` });
};

const handleDrop = (e: DragEvent<HTMLDivElement>, folderId: string) => {
  e.preventDefault();
  e.stopPropagation();

  let targetFolder = folderId;
  const parts = folderId.split("-");

  // If dropping on a week folder (e.g., "week-1"), redirect to first course's "other" subfolder
  if (parts.length === 2 && parts[0] === "week") {
    // Week folder - redirect to first available course's "other" folder
    targetFolder = `${folderId}-${COURSE_FOLDERS[0].id}-other`;
  }
  // If dropping on a course folder (e.g., "week-1-cppa122"), redirect to "other" subfolder
  else if (parts.length === 3 && parts[0] === "week") {
    // This is a course folder, redirect to "other" content folder
    targetFolder = `${folderId}-other`;
  }

  // Check if this is an external file drop
  if (e.dataTransfer.files && e.dataTransfer.files.length > 0) {
    handleExternalFileDrop(e.dataTransfer.files, targetFolder);
    setDragOverFolder(null);
    setIsExternalDragOver(false);
    return;
  }

  // Internal drag-drop (moving existing files)
  const fileId = parseInt(e.dataTransfer.getData("text/plain"));
  if (fileId) {
    moveFolderMutation.mutate({ id: fileId, folder: targetFolder });
  }
  setDraggedFileId(null);
  setDragOverFolder(null);
};

const handleDropOnUnfiled = (e: DragEvent<HTMLDivElement>) => {
  e.preventDefault();
  e.stopPropagation();

  // Check if this is an external file drop
  if (e.dataTransfer.files && e.dataTransfer.files.length > 0) {
    handleExternalFileDrop(e.dataTransfer.files, null);
    setDragOverFolder(null);
    setIsExternalDragOver(false);
    return;
  }

  const fileId = parseInt(e.dataTransfer.getData("text/plain"));
  if (fileId) {
    moveFolderMutation.mutate({ id: fileId, folder: null });
  }
  setDraggedFileId(null);
  setDragOverFolder(null);
};

const getFilesInFolder = (folderId: string) => {
  return files.filter(f => f.folder === folderId);
};

const getFilesInWeek = (weekId: string) => {
  // Include files directly in the week folder OR in any subfolder
  return files.filter(f => f.folder === weekId || f.folder?.startsWith(weekId + "-"));
};

const getFilesInCourse = (weekId: string, courseId: string) => {
  return files.filter(f => f.folder?.startsWith(`${weekId}-${courseId}-`));
};

const unfiledFiles = files.filter(f => !f.folder);

const sortedFiles = (fileList: FileRecord[]) => {
  return [...fileList].sort((a, b) => {
    switch (sortBy) {
      case "name-asc":
        return (a.displayName || "").localeCompare(b.displayName || "");
      case "name-desc":

```

```

    return (b.displayName || "").localeCompare(a.displayName || "");
  case "date-newest":
    return new Date(b.createdAt || 0).getTime() - new Date(a.createdAt || 0).getTime();
  case "date-oldest":
    return new Date(a.createdAt || 0).getTime() - new Date(b.createdAt || 0).getTime();
  case "size-largest":
    return (b.size || 0) - (a.size || 0);
  case "size-smallest":
    return (a.size || 0) - (b.size || 0);
  default:
    return 0;
  }
});
});

const renderFileRow = (file: FileRecord) => {
  const FileIcon = getFileIcon(file.contentType);
  const assignedTasks = getTasksForFile(file);
  const isDragging = draggedFileId === file.id;

  return (
    <div
      key={file.id}
      draggable
      onDragStart={(e) => handleDragStart(e, file.id)}
      onDragEnd={handleDragEnd}
      className={`flex flex-col gap-1 p-2 bg-[#ffd251] border-2 border-black dark:border-white rounded-[8px] hover-...
        isDragging ? "opacity-50" : ""
      }`}
      data-testid={`file-row-${file.id}`}
    >
      <div className="flex items-center justify-between gap-2">
        <Checkbox
          checked={file.listened || false}
          onCheckedChange={(checked) => {
            listenedMutation.mutate({ id: file.id, listened: checked === true });
          }}
          onClick={(e) => e.stopPropagation()}
          className="h-4 w-4 border-2 border-black data-[state=checked]:bg-green-600 data-[state=checked]:border-gr...
          data-testid={`checkbox-listened-${file.id}`}
        />
        <a
          href={file.objectPath}
          target="_blank"
          rel="noopener noreferrer"
          className={`font-medium text-[10px] truncate hover:underline cursor-pointer text-black flex-1 ${file.list...
          onClick={(e) => e.stopPropagation()}
          data-testid={`text-filename-${file.id}`}
        >
          {file.displayName}
          <div className="text-[8px] text-black/50 truncate font-normal" data-testid={`text-filepath-${file.id}`}>{...
        </a>
        {assignedTasks.length > 0 && (
          <Badge variant="secondary" className="text-[8px] py-0 px-1">
            {assignedTasks.length}
          </Badge>
        )}
        <Select
          value={getSpeakerForFile(file.id)}
          onValueChange={(value) => setSpeakerForFile(file.id, value)}
        >
          <SelectTrigger className="w-[75px] h-5 text-xs font-medium bg-black hover:bg-gray-800 text-white border b...
          <SelectValue placeholder="Spkr" />
          </SelectTrigger>
          <SelectContent>
            {SPEAKERS.map(speaker => (
              <SelectItem key={speaker.id} value={speaker.id}>
                {speaker.name}
              </SelectItem>
            ))}
          </SelectContent>
        </Select>
      </div>

      <div className="flex flex-col w-[calc(100%-16px)] bg-[#c9a033] dark:bg-[#c9a033] rounded-b-[6px] -ml-2 -mr-2 ...
        /* Main playback controls - larger */
        <div className="flex items-center justify-center gap-4">
          <SkipBack
            className="h-5 w-5 text-black cursor-pointer hover:opacity-70"
            onClick={(e) => { e.stopPropagation(); handleSkipChunk(file.id, "backward"); }}
            data-testid={`button-skip-back-${file.id}`}
          />
          <Play
            className="h-6 w-6 fill-black text-black cursor-pointer hover:opacity-70"
            onClick={() => handlePlayFile(file.id, file.objectPath, file.displayName)}
            data-testid={`button-play-${file.id}`}
          />
          <Square
            className="h-5 w-5 fill-black text-black cursor-pointer hover:opacity-70"
            onClick={() => handleStop(file.id)}
            data-testid={`button-stop-${file.id}`}
          />
          <SkipForward
            className="h-5 w-5 text-black cursor-pointer hover:opacity-70"
            onClick={(e) => { e.stopPropagation(); handleSkipChunk(file.id, "forward"); }}
            data-testid={`button-skip-forward-${file.id}`}
          />
          <RotateCcw
            className="h-5 w-5 text-black cursor-pointer hover:opacity-70"
            onClick={() => handleRestart(file.id, file.objectPath, file.displayName)}
            data-testid={`button-restart-${file.id}`}
          />
        </div>
        /* Volume controls */
        <div className="flex items-center justify-center gap-3">
          <Minus
            className="h-4 w-4 text-black cursor-pointer hover:opacity-70"
            onClick={(e) => { e.stopPropagation(); handleVolume(file.id, "down"); }}

```

```

        data-testid={`button-vol-down-${file.id}`}
      />
<Volume2 className="h-4 w-4 text-black" />
<Plus
  className="h-4 w-4 text-black cursor-pointer hover:opacity-70"
  onClick={(e) => { e.stopPropagation(); handleVolume(file.id, "up"); }}
  data-testid={`button-vol-up-${file.id}`}
/>
</div>
{ /* Action buttons */ }
<div className="flex items-center justify-center gap-4">
  {file.contentType?.includes("pdf") && (
    <Link href={`/pdf-reader/${file.id}`}>
      <BookOpen
        className="h-4 w-4 text-amber-600 cursor-pointer hover:opacity-70"
        data-testid={`button-read-pdf-${file.id}`}
      />
    </Link>
  )}
  <Edit2
    className="h-4 w-4 text-black cursor-pointer hover:opacity-70"
    onClick={() => {
      setEditingFile(file);
      setNewName(file.displayName);
    }}
    data-testid={`button-rename-${file.id}`}
  />
  <Link2
    className="h-4 w-4 text-black cursor-pointer hover:opacity-70"
    onClick={() => {
      setAssigningFile(file);
      setSelectedTaskId("");
    }}
    data-testid={`button-assign-${file.id}`}
  />
  <Trash2
    className="h-4 w-4 text-black cursor-pointer hover:opacity-70"
    onClick={() => {
      if (confirm("Are you sure you want to delete this file?")) {
        deleteMutation.mutate(file.id);
      }
    }}
    data-testid={`button-delete-${file.id}`}
  />
</div>
</div>
</div>
);
};

if (filesLoading || tasksLoading) {
  return (
    <div className="min-h-screen bg-background p-6">
      <div className="max-w-6xl mx-auto">
        <div className="animate-pulse space-y-4">
          <div className="h-8 bg-muted rounded w-48" />
          <div className="grid gap-4">
            {[1, 2, 3].map(i => (
              <div key={i} className="h-24 bg-muted rounded" />
            ))}
          </div>
        </div>
      </div>
    </div>
  );
}

const handlePageDragOver = (e: DragEvent<HTMLDivElement>) => {
  if (e.dataTransfer.types.includes("Files")) {
    e.preventDefault();
    e.dataTransfer.dropEffect = "copy";
    setIsExternalDragOver(true);
  }
};

const handlePageDragLeave = (e: DragEvent<HTMLDivElement>) => {
  const relatedTarget = e.relatedTarget as HTMLElement;
  if (!e.currentTarget.contains(relatedTarget)) {
    setIsExternalDragOver(false);
  }
};

const handlePageDrop = (e: DragEvent<HTMLDivElement>) => {
  e.preventDefault();
  if (e.dataTransfer.files && e.dataTransfer.files.length > 0) {
    handleExternalFileDrop(e.dataTransfer.files, null);
  }
  setIsExternalDragOver(false);
};

// Get files for the currently selected folder
const getCurrentFolderFiles = () => {
  if (!selectedFolder) return unfiledFiles;
  return getFilesInFolder(selectedFolder);
};

// Get breadcrumb path for selected folder
const getBreadcrumb = () => {
  if (!selectedFolder) return ["/files"];

  // Handle custom folders
  if (selectedFolder.startsWith("custom-")) {
    const customId = parseInt(selectedFolder.replace("custom-", ""));
    const customFolder = customFoldersData.find(cf => cf.id === customId);
    if (customFolder) {
      return ["/files/${customFolder.parentFolderId}/${customFolder.name}"];
    }
  }
}

```

```

const folderPath = selectedFolder.replace(/^(week-\d+)-/, '$1/');
return [`/files/${folderPath}`];
};

const navigateBack = () => {
  if (!selectedFolder) return;
  if (selectedFolder.startsWith("custom-")) {
    const customId = parseInt(selectedFolder.replace("custom-", ""));
    const customFolder = customFoldersData.find(cf => cf.id === customId);
    if (customFolder) {
      if (customFolder.parentFolderId === "root") {
        setSelectedFolder(null);
      } else if (customFolder.parentFolderId.startsWith("custom-")) {
        setSelectedFolder(customFolder.parentFolderId);
      } else {
        setSelectedFolder(customFolder.parentFolderId);
      }
    }
    return;
  }
}

const parts = selectedFolder.split("-");
if (parts.length <= 1) {
  setSelectedFolder(null);
} else {
  parts.pop();
  setSelectedFolder(parts.join("-"));
}
};

const navigateHome = () => {
  setSelectedFolder(null);
  setExpandedFolders(new Set());
};

return (
  <div
    className={`h-screen flex flex-col bg-[#191919] text-white transition-all ${isExternalDragOver ? "ring-4 ring-p...
    onDragOver={handlePageDragOver}
    onDragLeave={handlePageDragLeave}
    onDrop={handlePageDrop}
  >
    {uploadingCount > 0 && (
      <div className="fixed top-4 right-4 bg-primary text-primary-foreground px-4 py-2 rounded-md shadow-lg flex it...
        <Loader2 className="h-4 w-4 animate-spin" />
        Uploading {uploadingCount} file(s)...
      </div>
    )}
    {isExternalDragOver && (
      <div className="fixed inset-0 bg-primary/10 pointer-events-none z-40 flex items-center justify-center">
        <div className="bg-[#2d2d2d] border-2 border-dashed border-blue-500 rounded-lg p-8 text-center">
          <Upload className="h-12 w-12 mx-auto text-blue-400 mb-2" />
          <p className="text-lg font-medium">Drop files here to upload</p>
        </div>
      </div>
    )}
  </div>

  { /* Toolbar */ }
  <div className="flex items-center gap-1 px-3 py-2 border-b border-[#3d3d3d] bg-[#202020]">
    { /* Back Arrow + Home - always at very left */ }
    {viewMode === 'all' ? (
      <Button
        variant="ghost"
        size="icon"
        className="h-8 w-8 hover:bg-[#3d3d3d]"
        data-testid="button-back"
        onClick={navigateBack}
        disabled={!selectedFolder}
      >
        <ArrowLeft className="h-4 w-4" />
      </Button>
    ) : (
      <Button
        variant="ghost"
        size="icon"
        className="h-8 w-8 hover:bg-[#3d3d3d]"
        onClick={() => {
          if (oneDrivePathHistory.length > 0) {
            const previousPath = oneDrivePathHistory[oneDrivePathHistory.length - 1];
            setOneDrivePathHistory(oneDrivePathHistory.slice(0, -1));
            setOneDrivePath(previousPath);
          }
        }}
        disabled={oneDrivePathHistory.length === 0}
        data-testid="button-onedrive-back"
      >
        <ArrowLeft className="h-4 w-4" />
      </Button>
    )}
    <a
      href="/"
      className="inline-flex items-center justify-center h-8 w-8 rounded-md hover:bg-[#3d3d3d]"
      data-testid="button-home"
      title="Back to Dashboard"
    >
      <Home className="h-4 w-4" />
    </a>

    {viewMode === 'all' ? (
      <div
        className="flex items-center gap-1 text-sm text-gray-400 flex-1 min-w-0 truncate ml-2">
        <span className="truncate text-white">{getBreadcrumb()[0]}</span>
      </div>
      <ObjectUploader
        maxNumberOfFiles={5}
        onGetUploadParameters={getUploadParameters}
        onComplete={handleUploadComplete}
        buttonClassName="h-7 text-xs bg-transparent hover:bg-white/10 text-white border border-white/40 sh...

```



```

    }
  })
  data-testid={`onedrive-file-${file.id}`}
  >
    <FileText className="h-4 w-4 text-red-400" />
    <span className="text-[12px] text-white/80 truncate flex-1">{file.name}</span>
    {file.size && (
      <span className="text-[10px] text-white/30">{formatFileSize(file.size)}</span>
    )}
  </div>
))}

{oneDriveFiles.filter(f => !f.mimeType?.includes("pdf")).map((file) => (
  <div
    key={file.id}
    className="flex items-center gap-2 px-2 py-1.5 hover:bg-white/10 rounded transition-colors"
    data-testid={`onedrive-file-${file.id}`}
  >
    <File className="h-4 w-4 text-white/50" />
    <span className="text-[12px] text-white/60 truncate flex-1">{file.name}</span>
    {file.size && (
      <span className="text-[10px] text-white/30">{formatFileSize(file.size)}</span>
    )}
  </div>
))}

{oneDriveFolders.length === 0 && oneDriveFiles.length === 0 && (
  <div className="flex flex-col items-center justify-center py-8 text-white/40">
    <Folder className="h-8 w-8 mb-2 opacity-50" />
    <p className="text-[12px]">No files found</p>
  </div>
)}
</div>
))
</div>
): (
<div className="py-2">
  /* Root-level custom folders (at week level) */
  (() => {
    const rootCustomFolders = customFoldersData.filter(cf => cf.parentFolderId === "root");
    return (
      <ContextMenu>
        <ContextMenuTrigger asChild>
          <div
            className="flex items-center gap-1 px-2 py-1 cursor-pointer hover:bg-[#2d2d2d] text-gray-400 te...
            onClick={() => toggleFolder("root")}
            data-testid="root-folder-header"
          >
            {rootCustomFolders.length > 0 ? (
              expandedFolders.has("root") ? (
                <ChevronDown className="h-3 w-3 text-gray-500" />
              ) : (
                <ChevronRight className="h-3 w-3 text-gray-500" />
              )
            ) : (
              <div className="w-3 h-3" />
            )}
            <FolderPlus className="h-4 w-4 text-gray-500" />
            <span className="flex-1">My Folders</span>
            <span className="text-xs text-gray-600">{rootCustomFolders.length}</span>
          </div>
        </ContextMenuTrigger>
        <ContextMenuContent className="w-48">
          <ContextMenuitem>
            onClick={() => {
              setAddFolderParentId("root");
              setShowAddFolderDialog(true);
            }}
            data-testid="add-root-folder"
          >
            <FolderPlus className="h-4 w-4 mr-2" />
            Add Folder
          </ContextMenuitem>
        </ContextMenuContent>
      </ContextMenu>
    );
  })()
}

/* Root-level custom folders children */
{expandedFolders.has("root") && customFoldersData.filter(cf => cf.parentFolderId === "root").map(folder => {
  const folderFiles = files.filter(f => f.folder === `custom-${folder.id}`);
  const isSelected = selectedFolder === `custom-${folder.id}`;
  return (
    <div key={folder.id} className="ml-3">
      <ContextMenu>
        <ContextMenuTrigger asChild>
          <div
            className={`flex items-center gap-1 px-2 py-1 cursor-pointer hover:bg-[#2d2d2d] ${isSelected ? ...
            onClick={() => setSelectedFolder(`custom-${folder.id}`)}
            onDragOver={(e) => handleDragOver(e, `custom-${folder.id}`)}
            onDragLeave={(e) => handleDragLeave(e)}
            onDrop={(e) => handleDrop(e, `custom-${folder.id}`)}
            data-testid={`custom-folder-${folder.id}`}
          >
            <div className="w-3 h-3" />
            <Folder className="h-4 w-4 text-blue-400" />
            {editingFolderId === folder.id ? (
              <input
                type="text"
                value={editingFolderName}
                onChange={(e) => setEditingFolderName(e.target.value)}
                onKeyDown={(e) => {
                  if (e.key === "Enter" && editingFolderName.trim()) {
                    renameCustomFolderMutation.mutate({ id: folder.id, name: editingFolderName.trim() });
                  } else if (e.key === "Escape") {
                    setEditingFolderId(null);
                    setEditingFolderName("");
                  }
                }}
              />
            ) : (

```



```

        className="text-sm flex-1 bg-[#1e1e1e] border border-[#0078d4] rounded px-1 py-0 text-whi...
        data-testid={`input-rename-week-folder-${week.id}`}
      />
    ) : (
      <span className={`text-sm flex-1 ${shouldStrikethrough ? 'line-through text-gray-500' : isC...
    )}
    <span className={`text-xs ${isCurrent && viewMode === 'week' ? 'text-white/70' : 'text-gray-5...
  </div>
</ContextMenuTrigger>
<ContextMenuContent className="w-48">
  <ContextMenu
    onClick={() => {
      setEditingBuiltinFolderId(week.id);
      setEditingBuiltinFolderName(folderDisplayNames[week.id] || week.name);
    }}
    data-testid={`rename-week-folder-${week.id}`}
  >
    <Edit2 className="h-4 w-4 mr-2" />
    Rename Folder
  </ContextMenu>
  <ContextMenu
    onClick={() => {
      setAddFolderParentId(week.id);
      setShowAddFolderDialog(true);
    }}
    data-testid={`add-folder-${week.id}`}
  >
    <FolderPlus className="h-4 w-4 mr-2" />
    Add Folder
  </ContextMenu>
</ContextMenuContent>
</ContextMenu>

{isWeekExpanded && (
  <div className="ml-3">
    {COURSE_FOLDERS.map((course) => {
      const courseFolderId = `${week.id}-${course.id}`;
      // Skip deleted folders
      if (deletedFolderIds.has(courseFolderId)) return null;
      const courseFiles = getFilesInCourse(week.id, course.id);
      const isCourseExpanded = expandedFolders.has(courseFolderId);

      return (
        <div key={courseFolderId}>
          <ContextMenu>
            <ContextMenuTrigger asChild>
              <div
                className={`flex items-center gap-1 px-2 py-1 cursor-pointer hover:bg-[#2d2d2d]} ${s...
                onClick={(e) => {
                  setSelectedFolder(courseFolderId);
                  if (!(e.ctrlKey || e.metaKey) && (courseFiles.length > 0 || CONTENT_FOLDERS.some(...
                    toggleFolder(courseFolderId);
                }}
                onDragOver={(e) => handleDragOver(e, courseFolderId)}
                onDragLeave={(e) => handleDragLeave(e)}
                onDrop={(e) => handleDrop(e, courseFolderId)}
                data-testid={`course-folder-${courseFolderId}`}
              >
                {courseFiles.length > 0 || CONTENT_FOLDERS.some(cf => !deletedFolderIds.has(`${cour...
                  isCourseExpanded ? (
                    <ChevronDown className="h-3 w-3 text-gray-500" />
                  ) : (
                    <ChevronRight className="h-3 w-3 text-gray-500" />
                  )
                ) : (
                  <div className="w-3 h-3">
                )}
              {isCourseExpanded ? (
                <FolderOpen className="h-4 w-4 text-yellow-500 fill-yellow-400" />
              ) : (
                <Folder className="h-4 w-4 text-yellow-600 fill-yellow-400" />
              )}
              {editingBuiltinFolderId === courseFolderId ? (
                <input
                  type="text"
                  value={editingBuiltinFolderName}
                  onChange={(e) => setEditingBuiltinFolderName(e.target.value)}
                  onKeyDown={(e) => {
                    if (e.key === "Enter" && editingBuiltinFolderName.trim()) {
                      saveFolderDisplayName(courseFolderId, editingBuiltinFolderName.trim());
                    } else if (e.key === "Escape") {
                      setEditingBuiltinFolderId(null);
                      setEditingBuiltinFolderName("");
                    }
                  }}
                >
                <div
                  onBlur={() => {
                    if (editingBuiltinFolderName.trim()) {
                      saveFolderDisplayName(courseFolderId, editingBuiltinFolderName.trim());
                    } else {
                      setEditingBuiltinFolderId(null);
                      setEditingBuiltinFolderName("");
                    }
                  }}
                >
                <div
                  onClick={(e) => e.stopPropagation()}
                  autoFocus
                  className={`text-sm flex-1 bg-[#1e1e1e] border border-[#0078d4] rounded px-1 py...
                  data-testid={`input-rename-course-folder-${courseFolderId}`}
                />
              ) : (
                <span className={`text-sm flex-1 ${course.color}`}>{folderDisplayNames[courseFold...
              )}
            <span className="text-xs text-gray-500">{courseFiles.length}</span>
          </div>
        </ContextMenuTrigger>
      <ContextMenuContent className="w-48">
        <ContextMenu
          onClick={() => {

```

```

        setEditingBuiltinFolderId(courseFolderId);
        setEditingBuiltinFolderName(folderDisplayNames[courseFolderId] || course.name);
    })
    data-testid={`rename-course-folder-${courseFolderId}`}
  >
  <Edit2 className="h-4 w-4 mr-2" />
  Rename Folder
</ContextMenuItem>
<ContextMenuItem
  onClick={() => {
    setAddFolderParentId(courseFolderId);
    setShowAddFolderDialog(true);
  }}
  data-testid={`add-folder-${courseFolderId}`}
>
  <FolderPlus className="h-4 w-4 mr-2" />
  Add Folder
</ContextMenuItem>
</ContextMenuContent>
</ContextMenu>

{isCourseExpanded && (
  <div className="ml-4">
    {CONTENT_FOLDERS.map((content) => {
      const contentFolderId = `${week.id}-${course.id}-${content.id}`;
      // Skip deleted folders
      if (deletedFolderIds.has(contentFolderId)) return null;
      const contentFiles = getFilesInFolder(contentFolderId);
      const isSelected = selectedFolder === contentFolderId;
      const contentCustomFolders = customFoldersData.filter(cf => cf.parentFolderId === c...
      const isContentExpanded = expandedFolders.has(contentFolderId);
      const hasSubfolders = contentCustomFolders.length > 0;

      return (
        <div key={contentFolderId}>
          <ContextMenu>
            <ContextMenuTrigger asChild>
              <div
                className={`flex items-center gap-1 px-2 py-1 cursor-pointer hover:bg-[#2...
                onClick={() => {
                  if (hasSubfolders) {
                    toggleFolder(contentFolderId);
                  }
                  setSelectedFolder(contentFolderId);
                }}
                onDragOver={(e) => handleDragOver(e, contentFolderId)}
                onDragLeave={(e) => handleDragLeave(e)}
                onDrop={(e) => handleDrop(e, contentFolderId)}
                data-testid={`content-folder-${contentFolderId}`}
              >
                {hasSubfolders ? (
                  isContentExpanded ? (
                    <ChevronDown className="h-3 w-3 text-gray-500" />
                  ) : (
                    <ChevronRight className="h-3 w-3 text-gray-500" />
                  )
                ) : (
                  <div className="w-3 h-3" />
                )}
                <Folder className="h-3.5 w-3.5 text-yellow-600 fill-yellow-400" />
                {editingBuiltinFolderId === contentFolderId ? (
                  <input
                    type="text"
                    value={editingBuiltinFolderName}
                    onChange={(e) => setEditingBuiltinFolderName(e.target.value)}
                    onKeyDown={(e) => {
                      if (e.key === "Enter" && editingBuiltinFolderName.trim()) {
                        saveFolderDisplayName(contentFolderId, editingBuiltinFolderName.t...
                      } else if (e.key === "Escape") {
                        setEditingBuiltinFolderId(null);
                        setEditingBuiltinFolderName("");
                      }
                    }}
                    onBlur={() => {
                      if (editingBuiltinFolderName.trim()) {
                        saveFolderDisplayName(contentFolderId, editingBuiltinFolderName.t...
                      } else {
                        setEditingBuiltinFolderId(null);
                        setEditingBuiltinFolderName("");
                      }
                    }}
                    onClick={(e) => e.stopPropagation()}
                    autoFocus
                    className="text-sm flex-1 bg-[#1e1e1e] border border-[#0078d4] rounde...
                    data-testid={`input-rename-builtin-folder-${contentFolderId}`}
                  >
                </input>
                <span className="text-sm flex-1">{folderDisplayNames[contentFolderId]} ...
              </span>
            </div>
          </ContextMenuTrigger>
          <ContextMenuContent className="w-48">
            <ContextMenuItem
              onClick={() => {
                setEditingBuiltinFolderId(contentFolderId);
                setEditingBuiltinFolderName(folderDisplayNames[contentFolderId] || cont...
              }}
              data-testid={`rename-builtin-folder-${contentFolderId}`}
            >
              <Edit2 className="h-4 w-4 mr-2" />
              Rename Folder
            </ContextMenuItem>
            <ContextMenuItem
              onClick={() => {
                setAddFolderParentId(contentFolderId);
                setShowAddFolderDialog(true);
              }}
            >

```

```

    data-testid={`add-folder-${contentFolderId}`}
  >
    <FolderPlus className="h-4 w-4 mr-2" />
    Add Folder
  </ContextMenuItem>
</ContextMenuContent>
</ContextMenu>
{isContentExpanded && hasSubfolders && (
  <div className="ml-4">
    {contentCustomFolders.map((customFolder) => {
      const customFolderId = `custom-${customFolder.id}`;
      const customFiles = files.filter(f => f.folder === customFolderId);
      const isCustomSelected = selectedFolder === customFolderId;

      return (
        <ContextMenu key={customFolderId}>
          <ContextMenuTrigger asChild>
            <div
              className={`flex items-center gap-1 px-2 py-1 cursor-pointer hover...
              onClick={() => setSelectedFolder(customFolderId)}
              onDragOver={(e) => handleDragOver(e, customFolderId)}
              onDragLeave={(e) => handleDragLeave(e)}
              onDrop={(e) => handleDrop(e, customFolderId)}
              data-testid={`custom-folder-${customFolderId}`}
            >
              <div className="w-3 h-3" />
              <Folder className="h-3.5 w-3.5 text-blue-500 fill-blue-400" />
              {editingFolderId === customFolder.id ? (
                <input
                  type="text"
                  value={editingFolderName}
                  onChange={(e) => setEditingFolderName(e.target.value)}
                  onKeyDown={(e) => {
                    if (e.key === "Enter" && editingFolderName.trim()) {
                      renameCustomFolderMutation.mutate({ id: customFolder.id, ...
                    } else if (e.key === "Escape") {
                      setEditingFolderId(null);
                      setEditingFolderName("");
                    }
                  }}
                  onBlur={() => {
                    if (editingFolderName.trim() && editingFolderName !== custo...
                      renameCustomFolderMutation.mutate({ id: customFolder.id, ...
                    } else {
                      setEditingFolderId(null);
                      setEditingFolderName("");
                    }
                  }}
                >
              ) : (
                <span
                  className="text-sm flex-1 hover:underline"
                  onClick={(e) => {
                    e.stopPropagation();
                    setEditingFolderId(customFolder.id);
                    setEditingFolderName(customFolder.name);
                  }}
                  data-testid={`folder-name-${customFolder.id}`}
                >
              {customFolder.name}
            </span>
          )}
        <span className="text-xs text-gray-500">{customFiles.length}</span>
      </div>
    </ContextMenuTrigger>
    <ContextMenuContent className="w-48">
      <ContextMenuItem
        onClick={() => {
          setEditingFolderId(customFolder.id);
          setEditingFolderName(customFolder.name);
        }}
        data-testid={`rename-folder-${customFolderId}`}
      >
        <Edit2 className="h-4 w-4 mr-2" />
        Rename Folder
      </ContextMenuItem>
      <ContextMenuItem
        onClick={() => {
          setAddFolderParentId(customFolderId);
          setShowAddFolderDialog(true);
        }}
        data-testid={`add-folder-${customFolderId}`}
      >
        <FolderPlus className="h-4 w-4 mr-2" />
        Add Folder
      </ContextMenuItem>
      <ContextMenuItem
        onClick={() => deleteCustomFolderMutation.mutate(customFolder.id)}
        className="text-red-500"
        data-testid={`delete-folder-${customFolderId}`}
      >
        <Trash2 className="h-4 w-4 mr-2" />
        Delete Folder
      </ContextMenuItem>
    </ContextMenuContent>
  </ContextMenu>
);
  }}
</div>
)}
</div>

```



```

    data-testid={`file-row-${file.id}`}
  >
    { /* Checkbox */ }
    <div className="w-8 flex-shrink-0">
      <Checkbox
        checked={file.listened || false}
        onCheckedChange={(checked) => {
          listenedMutation.mutate({ id: file.id, listened: checked === true });
        }}
        onClick={(e) => e.stopPropagation()}
        className="h-4 w-4 border-gray-500 data-[state=checked]:bg-green-600 data-[state=checked]:bor...
        data-testid={`checkbox-listened-${file.id}`}
      />
    </div>

    { /* Name */ }
    <div className="flex items-center gap-2 min-w-0" style={{ width: columnWidths.name }}>
      <FileIcon className="h-4 w-4 text-gray-400 flex-shrink-0" />
      <a
        href={file.objectPath}
        target="_blank"
        rel="noopener noreferrer"
        className={text-sm truncate hover:underline ${file.listened ? 'line-through text-gray-500' : ...}
        onClick={(e) => e.stopPropagation()}
        data-testid={`text-filename-${file.id}`}
      >
        {file.displayName}
      </a>
    </div>

    { /* Actions */ }
    <div className="flex items-center gap-3" style={{ width: columnWidths.actions }}>
      <Button
        variant="ghost"
        size="icon"
        className="h-6 w-6 text-gray-400 hover:text-white"
        onClick={(e) => {
          e.stopPropagation();
          setEditingFile(file);
          setNewName(file.displayName);
        }}
        data-testid={`button-rename-${file.id}`}
      >
        <Edit2 className="h-3 w-3" />
      </Button>
      <Button
        variant="ghost"
        size="icon"
        className="h-6 w-6 text-gray-400 hover:text-white"
        onClick={(e) => {
          e.stopPropagation();
          navigator.clipboard.writeText(file.objectPath);
          toast({ title: "Link copied to clipboard" });
        }}
        data-testid={`button-link-${file.id}`}
      >
        <Link2 className="h-3 w-3" />
      </Button>
      <Button
        variant="ghost"
        size="icon"
        className="h-6 w-6 text-gray-400 hover:text-red-500"
        onClick={(e) => {
          e.stopPropagation();
          deleteMutation.mutate(file.id);
        }}
        data-testid={`button-delete-${file.id}`}
      >
        <Trash2 className="h-3 w-3" />
      </Button>
    </div>

    { /* Status */ }
    <div className="flex justify-center" style={{ width: columnWidths.status }}>
      {file.listened ? (
        <CheckCircle2 className="h-4 w-4 text-green-500" />
      ) : assignedTasks.length > 0 ? (
        <Badge variant="secondary" className="text-[10px] py-0 px-1.5 bg-blue-600">
          {assignedTasks.length}
        </Badge>
      ) : null}
    </div>

    { /* Date modified */ }
    <div className="text-xs text-gray-400" style={{ width: columnWidths.date }}>
      {formatDate(file.createdAt)}
    </div>

    { /* Type */ }
    <div className="text-xs text-gray-400 truncate" style={{ width: columnWidths.type }}>
      {file.contentType?.split("/")[1]??.toUpperCase() || "File"}
    </div>

    { /* Size */ }
    <div className="text-xs text-gray-400 text-right" style={{ width: columnWidths.size }}>
      {formatFileSize(file.size)}
    </div>
  </div>
);
  }}
</div>

{ /* Bottom Action Bar */ }
{sortedFiles(getCurrentFolderFiles()).length > 0 && (
  <div className="border-t border-[#3d3d3d] bg-[#202020] px-4 py-2">
    <div className="text-xs text-gray-400">
      {sortedFiles(getCurrentFolderFiles()).length} items
    </div>
  </div>
)}

```

```

        </div>
      </div>
    )}
  </div>
</div>

/* Dialogs */
<Dialog open={!editingFile} onOpenChange={() => setEditingFile(null)}>
  <DialogContent>
    <DialogHeader>
      <DialogTitle>Rename File</DialogTitle>
    </DialogHeader>
    <div className="py-4">
      <Input
        value={newName}
        onChange={(e) => setNewName(e.target.value)}
        placeholder="Enter new file name"
        data-testid="input-rename"
      />
    </div>
    <DialogFooter>
      <Button variant="outline" onClick={() => setEditingFile(null)}>
        Cancel
      </Button>
      <Button
        onClick={handleRename}
        disabled={renameMutation.isPending}
        data-testid="button-confirm-rename"
      >
        {renameMutation.isPending ? "Renaming..." : "Rename"}
      </Button>
    </DialogFooter>
  </DialogContent>
</Dialog>

<Dialog open={!assigningFile} onOpenChange={() => setAssigningFile(null)}>
  <DialogContent>
    <DialogHeader>
      <DialogTitle>Assign to Task</DialogTitle>
    </DialogHeader>
    <div className="py-4">
      <Select value={selectedTaskId} onValueChange={setSelectedTaskId}>
        <SelectTrigger data-testid="select-task">
          <SelectValue placeholder="Select a task..." />
        </SelectTrigger>
        <SelectContent>
          {tasks.map(task => (
            <SelectItem key={task.id} value={task.id.toString()}>
              {task.title}
              {task.courseName && ` (${task.courseName})`}
            </SelectItem>
          ))}
        </SelectContent>
      </Select>
    </div>
    <DialogFooter>
      <Button variant="outline" onClick={() => setAssigningFile(null)}>
        Cancel
      </Button>
      <Button
        onClick={handleAssign}
        disabled={!selectedTaskId || assignMutation.isPending}
        data-testid="button-confirm-assign"
      >
        {assignMutation.isPending ? "Assigning..." : "Assign"}
      </Button>
    </DialogFooter>
  </DialogContent>
</Dialog>

<Dialog open={showAssignAfterUpload} onOpenChange={setShowAssignAfterUpload}>
  <DialogContent>
    <DialogHeader>
      <DialogTitle>Assign Uploaded File to Task</DialogTitle>
    </DialogHeader>
    <div className="py-4">
      <Select value={selectedTaskId} onValueChange={setSelectedTaskId}>
        <SelectTrigger data-testid="select-task-after-upload">
          <SelectValue placeholder="Select a task..." />
        </SelectTrigger>
        <SelectContent>
          {tasks.map(task => (
            <SelectItem key={task.id} value={task.id.toString()}>
              {task.title}
              {task.courseName && ` (${task.courseName})`}
            </SelectItem>
          ))}
        </SelectContent>
      </Select>
    </div>
    <DialogFooter>
      <Button variant="outline" onClick={() => setShowAssignAfterUpload(false)}>
        Skip
      </Button>
      <Button
        onClick={handleAssignAfterUpload}
        disabled={!selectedTaskId || assignByPathMutation.isPending}
        data-testid="button-confirm-assign-after-upload"
      >
        {assignByPathMutation.isPending ? "Assigning..." : "Assign"}
      </Button>
    </DialogFooter>
  </DialogContent>
</Dialog>

<Dialog open={showDeleteFolderConfirm} onOpenChange={setShowDeleteFolderConfirm}>
  <DialogContent>

```



```

<DialogHeader>
  <DialogTitle>Delete Folder Contents</DialogTitle>
</DialogHeader>
<div className="py-4">
  <p className="text-sm text-muted-foreground">
    Are you sure you want to delete all files in this folder? This action cannot be undone.
  </p>
  {selectedFolder && (
    <p className="text-sm font-medium mt-2">
      Files to delete: {getFilesToDeleteInFolder(selectedFolder).length}
    </p>
  )}
</div>
<DialogFooter>
  <Button variant="outline" onClick={() => setShowDeleteFolderConfirm(false)}>
    Cancel
  </Button>
  <Button
    variant="destructive"
    onClick={handleDeleteFolder}
    data-testid="button-confirm-delete-folder"
  >
    Delete All Files
  </Button>
</DialogFooter>
</DialogContent>
</Dialog>

<Dialog open={showAddFolderDialog} onOpenChange={setShowAddFolderDialog}>
  <DialogContent>
    <DialogHeader>
      <DialogTitle>Add New Folder</DialogTitle>
    </DialogHeader>
    <div className="py-4">
      <Input
        placeholder="Folder name"
        value={newFolderName}
        onChange={(e) => setNewFolderName(e.target.value)}
        onKeyDown={(e) => {
          if (e.key === "Enter" && newFolderName.trim() && addFolderParentId) {
            createCustomFolderMutation.mutate({
              parentFolderId: addFolderParentId,
              name: newFolderName.trim()
            });
          }
        }}
        data-testid="input-new-folder-name"
      </div>
    </div>
    <DialogFooter>
      <Button variant="outline" onClick={() => {
        setShowAddFolderDialog(false);
        setNewFolderName("");
        setAddFolderParentId(null);
      }}>
        Cancel
      </Button>
      <Button
        onClick={() => {
          if (newFolderName.trim() && addFolderParentId) {
            createCustomFolderMutation.mutate({
              parentFolderId: addFolderParentId,
              name: newFolderName.trim()
            });
          }
        }}
        disabled={!newFolderName.trim() || createCustomFolderMutation.isPending}
        data-testid="button-create-folder"
      >
        {createCustomFolderMutation.isPending ? "Creating..." : "Create Folder"}
      </Button>
    </DialogFooter>
  </DialogContent>
</Dialog>

</div>
);
}

```

`client/src/pages/mobile-notes.tsx`

```

import { useState, useEffect, useRef } from "react";
import { useQuery, useMutation } from "@tanstack/react-query";
import { useToast } from "@hooks/use-toast";
import { queryClient, apiRequest } from "@lib/queryClient";
import {
  Plus,
  ArrowLeft,
  Trash2,
  Loader2,
  FileText,
  RefreshCw,
  Check,
  Pencil,
  BookOpen,
  StickyNote,
  ChevronRight,
} from "lucide-react";

interface QuickNoteFile {
  id: string;
  name: string;
  type: string;
  size?: number;
}

```

```

    lastModified?: string;
    path: string;
}

interface OneNoteNotebook {
    name: string;
    path: string;
    sections: { name: string; id: string }[];
}

interface OneNotePage {
    title: string;
    content: string;
    position: number;
}

type Tab = "quicknotes" | "notebooks";
type View = "list" | "editor" | "notebook-sections" | "notebook-pages" | "onenote-page";

function timeAgo(dateStr: string) {
    const now = new Date();
    const d = new Date(dateStr);
    const diffMs = now.getTime() - d.getTime();
    const secs = Math.floor(diffMs / 1000);
    if (secs < 10) return "just now";
    if (secs < 60) return `${secs}s ago`;
    const mins = Math.floor(secs / 60);
    if (mins < 60) return `${mins}m ago`;
    const hrs = Math.floor(mins / 60);
    if (hrs < 24) return `${hrs}h ago`;
    const days = Math.floor(hrs / 24);
    if (days < 7) return `${days}d ago`;
    return d.toLocaleDateString("en-US", { month: "short", day: "numeric" });
}

export default function MobileNotesPage() {
    const { toast } = useToast();
    const [tab, setTab] = useState<Tab>("quicknotes");
    const [view, setView] = useState<View>("list");
    const [selectedFile, setSelectedFile] = useState<QuickNoteFile | null>(null);
    const [editorContent, setEditorContent] = useState("");
    const [editingTitle, setEditingTitle] = useState("");
    const [isDirty, setIsDirty] = useState(false);
    const [isCreating, setIsCreating] = useState(false);
    const [newNoteName, setNewNoteName] = useState("");
    const [confirmDelete, setConfirmDelete] = useState<string | null>(null);
    const saveTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
    const textareaRef = useRef<HTMLTextAreaElement>(null);

    const [selectedNotebook, setSelectedNotebook] = useState<OneNoteNotebook | null>(null);
    const [selectedSection, setSelectedSection] = useState<{ name: string; id: string } | null>(null);
    const [selectedOneNotePage, setSelectedOneNotePage] = useState<OneNotePage | null>(null);

    const filesQuery = useQuery<QuickNoteFile[]>({
        queryKey: ["/api/quicknotes/files"],
        staleTime: 15000,
    });

    const contentQuery = useQuery<{ content: string }>({
        queryKey: ["/api/quicknotes/file", selectedFile?.id, "content"],
        enabled: !!selectedFile,
        staleTime: 30000,
        refetchOnWindowFocus: false,
    });

    const notebooksQuery = useQuery<OneNoteNotebook[]>({
        queryKey: ["/api/onenote/notebooks"],
        enabled: tab === "notebooks",
        staleTime: 60000,
    });

    const pagesQuery = useQuery<OneNotePage[]>({
        queryKey: ["/api/onenote/pages", selectedNotebook?.path, selectedSection?.name],
        enabled: !!selectedNotebook && !!selectedSection,
        staleTime: 30000,
        queryFn: async () => {
            const params = new URLSearchParams({
                notebook: selectedNotebook!.path,
                section: selectedSection!.name,
            });
            const res = await fetch(`/api/onenote/pages?${params}`);
            if (!res.ok) throw new Error("Failed to load pages");
            return res.json();
        },
    });

    useEffect(() => {
        if (contentQuery.data?.content !== undefined && !isDirty) {
            setEditorContent(contentQuery.data.content);
        }
    }, [contentQuery.data]);

    useEffect(() => {
        if (selectedFile) {
            setEditingTitle(selectedFile.name.replace(/\.txt$/i, ""));
        }
    }, [selectedFile]);

    const saveMutation = useMutation({
        mutationFn: async ({ id, content }: { id: string; content: string }) => {
            const res = await apiRequest("PUT", `/api/quicknotes/file/${id}/content`, { content });
            return res.json();
        },
        onSuccess: (_, variables) => {
            setIsDirty(false);
            queryClient.setQueryData(
                ["/api/quicknotes/file", variables.id, "content"],
                { content: variables.content }
            );
        }
    });

```

```

    });
    queryClient.invalidateQueries({ queryKey: ["/api/quicknotes/files"] });
  },
});

const renameMutation = useMutation({
  mutationFn: async ({ id, name }: { id: string; name: string }) => {
    const res = await apiRequest("PATCH", `/api/quicknotes/file/${id}/rename`, { name });
    return res.json();
  },
  onSuccess: (data) => {
    queryClient.invalidateQueries({ queryKey: ["/api/quicknotes/files"] });
    if (selectedFile && data.file) {
      setSelectedFile(data.file);
    }
  },
});

const createMutation = useMutation({
  mutationFn: async ({ name, content }: { name: string; content: string }) => {
    const res = await apiRequest("POST", "/api/quicknotes/files", { name, content });
    return res.json();
  },
  onSuccess: (data) => {
    queryClient.invalidateQueries({ queryKey: ["/api/quicknotes/files"] });
    setIsCreating(false);
    setNewNoteName("");
    if (data.file) {
      setSelectedFile(data.file);
      setEditorContent("");
      setIsDirty(false);
      setView("editor");
    }
    toast({ title: "Note created" });
  },
});

const deleteMutation = useMutation({
  mutationFn: async (id: string) => {
    const res = await apiRequest("DELETE", `/api/quicknotes/file/${id}`);
    return res.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/quicknotes/files"] });
    setSelectedFile(null);
    setConfirmDelete(null);
    setView("list");
    toast({ title: "Note deleted" });
  },
});

const handleContentChange = (value: string) => {
  setEditorContent(value);
  setIsDirty(true);
  if (saveTimeoutRef.current) clearTimeout(saveTimeoutRef.current);
  if (selectedFile) {
    saveTimeoutRef.current = setTimeout(() => {
      saveMutation.mutate({ id: selectedFile.id, content: value });
    }, 1500);
  }
};

const handleTitleBlur = () => {
  if (!selectedFile) return;
  const currentName = selectedFile.name.replace(/\.txt$/i, "");
  if (editingTitle.trim() && editingTitle.trim() !== currentName) {
    renameMutation.mutate({ id: selectedFile.id, name: editingTitle.trim() });
  }
};

const handleBack = () => {
  if (view === "editor") {
    if (isDirty && selectedFile) {
      saveMutation.mutate({ id: selectedFile.id, content: editorContent });
    }
    setSelectedFile(null);
    setIsDirty(false);
    setConfirmDelete(null);
    setView("list");
  } else if (view === "onenote-page") {
    setSelectedOneNotePage(null);
    setView("notebook-pages");
  } else if (view === "notebook-pages") {
    setSelectedSection(null);
    setView("notebook-sections");
  } else if (view === "notebook-sections") {
    setSelectedNotebook(null);
    setView("list");
  }
};

const files = filesQuery.data || [];
const notebooks = notebooksQuery.data || [];

const headerStyle: React.CSSProperties = {
  display: "flex",
  alignItems: "center",
  gap: 8,
  padding: "12px 16px",
  borderBottom: "1px solid rgba(255,255,255,0.1)",
  background: "#1a1a1a",
};

const backBtnStyle: React.CSSProperties = {
  background: "none",
  border: "none",
  color: "#fff",
  padding: 8,
};

```

```

    cursor: "pointer",
  };
}

if (view === "onenote-page" && selectedOneNotePage) {
  return (
    <div data-testid="mobile-onenote-page-view" style={{ minHeight: "100dvh", background: "#111", color: "#fff", di...
      <div style={headerStyle}>
        <button data-testid="button-back" onClick={handleBack} style={backBtnStyle}>
          <ArrowLeft size={22} />
        </button>
        <div style={{ flex: 1 }}>
          <div style={{ fontSize: 18, fontWeight: 600 }}>{selectedOneNotePage.title}</div>
          <div style={{ fontSize: 12, color: "#888" }}>
            {selectedNotebook?.name} &rsquo; {selectedSection?.name}
          </div>
        </div>
      </div>
      <div style={{ flex: 1, overflowY: "auto", padding: "16px", whiteSpace: "pre-wrap", fontSize: 15, lineHeight: ...
        {selectedOneNotePage.content || "(empty page)"}
      </div>
    </div>
  );
}

if (view === "notebook-pages" && selectedSection) {
  return (
    <div data-testid="mobile-onenote-pages" style={{ minHeight: "100dvh", background: "#111", color: "#fff", displa...
      <div style={headerStyle}>
        <button data-testid="button-back" onClick={handleBack} style={backBtnStyle}>
          <ArrowLeft size={22} />
        </button>
        <div style={{ flex: 1 }}>
          <div style={{ fontSize: 18, fontWeight: 600 }}>{selectedSection.name}</div>
          <div style={{ fontSize: 12, color: "#888" }}>{selectedNotebook?.name}</div>
        </div>
      </div>
      <div style={{ fontSize: 13, color: "#9b8ec4", padding: "12px 16px 4px", fontWeight: 500 }}>Pages</div>
      <div style={{ flex: 1, overflowY: "auto" }}>
        {pagesQuery.isLoading ? (
          <div style={{ display: "flex", alignItems: "center", justifyContent: "center", padding: 40 }}>
            <Loader2 size={28} className="animate-spin" style={{ color: "#888" }} />
          </div>
        ) : (pagesQuery.data || []).length === 0 ? (
          <div style={{ padding: 40, textAlign: "center", color: "#888" }}>No pages found</div>
        ) : (
          (pagesQuery.data || []).map((page, idx) => (
            <button
              key={idx}
              data-testid={`onenote-page-${idx}`}
              onClick={() => { setSelectedOneNotePage(page); setView("onenote-page"); }}
              style={{
                width: "100%",
                textAlign: "left",
                background: "none",
                border: "none",
                borderBottom: "1px solid rgba(255,255,255,0.06)",
                color: "#fff",
                padding: "14px 16px",
                cursor: "pointer",
              }}
            >
              <div style={{ fontSize: 16, fontWeight: 500 }}>{page.title}</div>
              <div style={{ fontSize: 13, color: "#888", marginTop: 4, overflow: "hidden", textOverflow: "ellipsis"...
                {page.content.substring(0, 80) || "(empty)"}
              </div>
            </button>
          ))
        )
      </div>
      <BottomTabs tab={tab} setTab={(t) => { setTab(t); setView("list"); setSelectedNotebook(null); setSelectedSect...
    </div>
  );
}

if (view === "notebook-sections" && selectedNotebook) {
  return (
    <div data-testid="mobile-onenote-sections" style={{ minHeight: "100dvh", background: "#111", color: "#fff", dis...
      <div style={headerStyle}>
        <button data-testid="button-back" onClick={handleBack} style={backBtnStyle}>
          <ArrowLeft size={22} />
        </button>
        <div style={{ flex: 1, fontSize: 18, fontWeight: 600 }}>{selectedNotebook.name}</div>
      </div>
      <div style={{ fontSize: 13, color: "#9b8ec4", padding: "12px 16px 4px", fontWeight: 500 }}>Sections</div>
      <div style={{ flex: 1, overflowY: "auto" }}>
        {selectedNotebook.sections.map((section) => (
          <button
            key={section.id}
            data-testid={`onenote-section-${section.id}`}
            onClick={() => { setSelectedSection(section); setView("notebook-pages"); }}
            style={{
              width: "100%",
              textAlign: "left",
              background: "none",
              border: "none",
              borderBottom: "1px solid rgba(255,255,255,0.06)",
              color: "#fff",
              padding: "14px 16px",
              cursor: "pointer",
              display: "flex",
              alignItems: "center",
              gap: 12,
            }}
          >
            <div style={{ width: 4, height: 28, borderRadius: 2, background: "#5b5fc7", flexShrink: 0 }} />
            <div style={{ flex: 1, fontSize: 16, fontWeight: 500 }}>{section.name}</div>
            <ChevronRight size={18} style={{ color: "#666" }} />
          </button>
        ))
      </div>
    </div>
  );
}

```

```

    ))}
  </div>
  <BottomTabs tab={tab} setTab={() => { setTab(t); setView("list"); setSelectedNotebook(null); }} />
</div>
);
}

if (view === "editor" && selectedFile) {
  return (
    <div data-testid="mobile-notes-editor" style={{ minHeight: "100dvh", background: "#111", color: "#fff", display: ...
      <div style={headerStyle}>
        <button data-testid="button-back" onClick={handleBack} style={backBtnStyle}>
          <ArrowLeft size={22} />
        </button>
        <input
          data-testid="input-note-title"
          value={editingTitle}
          onChange={(e) => setEditingTitle(e.target.value)}
          onBlur={handleTitleBlur}
          style={{ flex: 1, background: "none", border: "none", color: "#fff", fontSize: 18, fontWeight: 600, outli...
        />
        <div style={{ display: "flex", alignItems: "center", gap: 8 }}>
          {saveMutation.isPending ? (
            <Loader2 size={18} className="animate-spin" style={{ color: "#888" }} />
          ) : isDirty ? (
            <Pencil size={16} style={{ color: "#f59e0b" }} />
          ) : (
            <Check size={16} style={{ color: "#4ade80" }} />
          )}
          <button data-testid="button-delete-note" onClick={() => setConfirmDelete(selectedFile.id)} style={{ backg...
            <Trash2 size={20} />
          </button>
        </div>
      </div>
    </div>
    {confirmDelete && (
      <div style={{ padding: "12px 16px", background: "rgba(239,68,68,0.15)", borderBottom: "1px solid rgba(239,6...
        <span style={{ fontSize: 14, color: "#fca5a5" }}>Delete this note?</span>
        <div style={{ display: "flex", gap: 12 }}>
          <button data-testid="button-cancel-delete" onClick={() => setConfirmDelete(null)} style={{ background: ...
            <button data-testid="button-confirm-delete" onClick={() => deleteMutation.mutate(selectedFile.id)} styl...
              {deleteMutation.isPending ? "Deleting..." : "Delete"}
            </button>
          </div>
        </div>
      </div>
    )}
    {contentQuery.isLoading ? (
      <div style={{ flex: 1, display: "flex", alignItems: "center", justifyContent: "center" }}>
        <Loader2 size={28} className="animate-spin" style={{ color: "#888" }} />
      </div>
    ) : (
      <textarea
        ref={textareaRef}
        data-testid="textarea-note-content"
        value={editorContent}
        onChange={(e) => handleContentChange(e.target.value)}
        placeholder="Start typing..."
        style={{ flex: 1, background: "transparent", border: "none", color: "#e0e0e0", fontSize: 16, lineHeight: ...
      </div>
    )}
  </div>
);
}

return (
  <div data-testid="mobile-notes-list" style={{ minHeight: "100dvh", background: "#111", color: "#fff", display: "f...
    <div style={{ display: "flex", alignItems: "center", justifyContent: "space-between", padding: "16px", borderBo...
      <h1 style={{ fontSize: 22, fontWeight: 700, margin: 0 }}>
        {tab === "quicknotes" ? "QuickNotes" : "Notebooks"}
      </h1>
      <div style={{ display: "flex", gap: 8 }}>
        {tab === "quicknotes" && (
          <button
            data-testid="button-refresh-notes"
            onClick={() => filesQuery.refetch()}
            style={{ background: "rgba(255,255,255,0.08)", border: "none", color: "#fff", width: 40, height: 40, ...
          >
            <RefreshCw size={18} className={filesQuery.isFetching ? "animate-spin" : ""} />
          </button>
          <button
            data-testid="button-create-note"
            onClick={() => setIsCreating(true)}
            style={{ background: "#7c5cbf", border: "none", color: "#fff", width: 40, height: 40, borderRadius: 1...
          >
            <Plus size={20} />
          </button>
        </div>
      </div>
    )}
    {tab === "notebooks" && (
      <button
        data-testid="button-refresh-notebooks"
        onClick={() => notebooksQuery.refetch()}
        style={{ background: "rgba(255,255,255,0.08)", border: "none", color: "#fff", width: 40, height: 40, bo...
      >
        <RefreshCw size={18} className={notebooksQuery.isFetching ? "animate-spin" : ""} />
      </button>
    )}
  </div>
</div>
</div>

{isCreating && tab === "quicknotes" && (
  <div style={{ padding: "12px 16px", borderBottom: "1px solid rgba(255,255,255,0.1)", background: "rgba(124,92...
    <input
      data-testid="input-new-note-name"
      autoFocus
      value={newNoteName}
      onChange={(e) => setNewNoteName(e.target.value)}
      placeholder="Note name..."
    />
  </div>
)}

```

```

    onKeyDown={(e) => {
      if (e.key === "Enter" && newNoteName.trim()) createMutation.mutate({ name: newNoteName.trim(), content:...
      if (e.key === "Escape") { setIsCreating(false); setNewNoteName(""); }
    }}
    style={{ width: "100%", background: "rgba(255,255,255,0.06)", border: "1px solid rgba(255,255,255,0.15)",...
  />
  <div style={{ display: "flex", gap: 8, justifyContent: "flex-end" }}>
    <button data-testid="button-cancel-create" onClick={() => { setIsCreating(false); setNewNoteName(""); }} ...
    <button
      data-testid="button-save-new-note"
      onClick={() => { if (newNoteName.trim()) createMutation.mutate({ name: newNoteName.trim(), content: "" ...
      disabled={!newNoteName.trim() || createMutation.isPending}
      style={{ background: "#7c5cbf", border: "none", color: "#fff", padding: "8px 16px", borderRadius: 8, fo...
    >
      {createMutation.isPending ? "Creating..." : "Create"}
    </button>
  </div>
</div>
))
<div style={{ flex: 1, overflowY: "auto" }}>
  {tab === "quicknotes" && (
    <>
      {filesQuery.isLoading ? (
        <div style={{ display: "flex", alignItems: "center", justifyContent: "center", padding: 40 }}>
          <Loader2 size={28} className="animate-spin" style={{ color: "#888" }} />
        </div>
      ) : files.length === 0 ? (
        <div style={{ display: "flex", flexDirection: "column", alignItems: "center", justifyContent: "center",...
          <FileText size={40} />
          <p style={{ fontSize: 16 }}>No notes yet</p>
          <button data-testid="button-create-first-note" onClick={() => setIsCreating(true)} style={{ background...
            Create your first note
          </button>
        </div>
      ) : (
        files.map((file) => (
          <button
            key={file.id}
            data-testid={`note-item-${file.id}`}
            onClick={() => { setSelectedFile(file); setIsDirty(false); setView("editor"); }}
            style={{ width: "100%", textAlign: "left", background: "none", border: "none", borderBottom: "1px s...
          >
            <FileText size={20} style={{ color: "#888", flexShrink: 0 }} />
            <div style={{ flex: 1, minWidth: 0 }}>
              <div style={{ fontSize: 16, fontWeight: 500, whiteSpace: "nowrap", overflow: "hidden", textOverfl...
                {file.name.replace(/\.txt$/i, "")}
              </div>
              <div style={{ fontSize: 13, color: "#888", marginTop: 2 }}>
                {file.lastModified ? timeAgo(file.lastModified) : ""}
                {file.size !== undefined && ` · ${file.size} bytes`}
              </div>
            </div>
          </button>
        ))
      )}
    </>
  )}
  {tab === "notebooks" && (
    <>
      {notebooksQuery.isLoading ? (
        <div style={{ display: "flex", alignItems: "center", justifyContent: "center", padding: 40 }}>
          <Loader2 size={28} className="animate-spin" style={{ color: "#888" }} />
        </div>
      ) : notebooks.length === 0 ? (
        <div style={{ padding: 40, textAlign: "center", color: "#888" }}>
          <BookOpen size={40} style={{ margin: "0 auto 12px" }} />
          <p>No notebooks found</p>
        </div>
      ) : (
        notebooks.map((nb) => (
          <button
            key={nb.path}
            data-testid={`notebook-${nb.name}`}
            onClick={() => { setSelectedNotebook(nb); setView("notebook-sections"); }}
            style={{ width: "100%", textAlign: "left", background: "none", border: "none", borderBottom: "1px s...
          >
            <div style={{ width: 28, height: 28, borderRadius: 4, background: "#7c5cbf", display: "flex", align...
              <BookOpen size={16} />
            </div>
            <div style={{ flex: 1 }}>
              <div style={{ fontSize: 16, fontWeight: 500 }}>{nb.name}</div>
              <div style={{ fontSize: 13, color: "#888", marginTop: 2 }}>
                {nb.sections.length} section{nb.sections.length !== 1 ? "s" : ""}
              </div>
            </div>
            <ChevronRight size={18} style={{ color: "#666" }} />
          </button>
        ))
      )}
    </>
  )}
</div>
<BottomTabs tab={tab} setTab={(t) => { setTab(t); setView("list"); }} />
</div>
);
}

function BottomTabs({ tab, setTab }: { tab: Tab; setTab: (t: Tab) => void }) {
  return (
    <div
      style={{
        display: "flex",
        borderTop: "1px solid rgba(255,255,255,0.1)",
        background: "#1a1a1a",
        padding: "0 16px 16px 16px",
      }}
    >

```

```

    }}
  >
  <button
    data-testid="tab-quicknotes"
    onClick={() => setTab("quicknotes")}
    style={{
      flex: 1,
      background: "none",
      border: "none",
      color: tab === "quicknotes" ? "#9b8ec4" : "#666",
      padding: "10px 0",
      cursor: "pointer",
      display: "flex",
      flexDirection: "column",
      alignItems: "center",
      gap: 4,
    }}
  >
  <StickyNote size={20} />
  <span style={{ fontSize: 11, fontWeight: 500 }}>QuickNotes</span>
</button>
<button
  data-testid="tab-notebooks"
  onClick={() => setTab("notebooks")}
  style={{
    flex: 1,
    background: "none",
    border: "none",
    color: tab === "notebooks" ? "#9b8ec4" : "#666",
    padding: "10px 0",
    cursor: "pointer",
    display: "flex",
    flexDirection: "column",
    alignItems: "center",
    gap: 4,
  }}
  >
  <BookOpen size={20} />
  <span style={{ fontSize: 11, fontWeight: 500 }}>Notebooks</span>
</button>
</div>
);
}

```

`client/src/pages/not-found.tsx`

```

import { Card, CardContent } from "@components/ui/card";
import { AlertCircle } from "lucide-react";

export default function NotFound() {
  return (
    <div className="min-h-screen w-full flex items-center justify-center bg-gray-50">
      <Card className="w-full max-w-md mx-4">
        <CardContent className="pt-6">
          <div className="flex mb-4 gap-2">
            <AlertCircle className="h-8 w-8 text-red-500" />
            <h1 className="text-2xl font-bold text-gray-900">404 Page Not Found</h1>
          </div>

          <p className="mt-4 text-sm text-gray-600">
            Did you forget to add the page to the router?
          </p>
        </CardContent>
      </Card>
    </div>
  );
}

```

`client/src/pages/onedrive.tsx`

```

import { useState, useEffect } from "react";
import { useQuery } from "@tanstack/react-query";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { useToast } from "@hooks/use-toast";
import { Link, useLocation } from "wouter";
import {
  FileText,
  File,
  Folder,
  FolderOpen,
  ChevronRight,
  ArrowLeft,
  Search,
  Loader2,
  Home,
  Cloud,
  Play
} from "lucide-react";

interface OneDriveItem {
  id: string;
  name: string;
  type: "folder" | "file";
  mimeType?: string;
  size?: number;
  lastModified?: string;
  downloadUrl?: string;
  path: string;
}

```

```

}

function getFileIcon(mimeType?: string) {
  if (!mimeType) return File;
  if (mimeType.includes("pdf")) return FileText;
  return File;
}

function formatFileSize(bytes?: number) {
  if (!bytes) return "";
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`;
  return `${(bytes / (1024 * 1024)).toFixed(1)} MB`;
}

function formatDate(dateString?: string) {
  if (!dateString) return "";
  const date = new Date(dateString);
  return date.toLocaleDateString("en-US", {
    month: "short",
    day: "numeric",
    year: "numeric",
  });
}

export default function OneDrivePage() {
  const { toast } = useToast();
  const [, setLocation] = useLocation();
  const [currentPath, setCurrentPath] = useState("/");
  const [pathHistory, setPathHistory] = useState<string[]>([]);
  const [searchQuery, setSearchQuery] = useState("");
  const [isSearching, setIsSearching] = useState(false);

  const { data: items, isLoading, error, refetch } = useQuery<OneDriveItem[]>({
    queryKey: ["/api/onedrive/files", currentPath],
    queryFn: async () => {
      const response = await fetch(`/api/onedrive/files?path=${encodeURIComponent(currentPath)}`);
      if (!response.ok) {
        const err = await response.json();
        throw new Error(err.error || "Failed to load files");
      }
      return response.json();
    },
  });

  const { data: searchResults, isLoading: searchLoading } = useQuery<OneDriveItem[]>({
    queryKey: ["/api/onedrive/search", searchQuery],
    queryFn: async () => {
      if (!searchQuery.trim()) return [];
      const response = await fetch(`/api/onedrive/search?q=${encodeURIComponent(searchQuery)}`);
      if (!response.ok) {
        const err = await response.json();
        throw new Error(err.error || "Search failed");
      }
      return response.json();
    },
  });
  enabled: isSearching && searchQuery.trim().length > 0,
});

const handleFolderClick = (item: OneDriveItem) => {
  if (item.type === "folder") {
    setPathHistory([...pathHistory, currentPath]);
    setCurrentPath(item.path);
    setIsSearching(false);
    setSearchQuery("");
  }
};

const handleFileClick = (item: OneDriveItem) => {
  if (item.type === "file" && item.mimeType?.includes("pdf")) {
    const encodedUrl = encodeURIComponent(item.downloadUrl || "");
    const encodedName = encodeURIComponent(item.name);
    setLocation(`/pdf-reader/onedrive?url=${encodedUrl}&name=${encodedName}`);
  } else if (item.downloadUrl) {
    window.open(item.downloadUrl, "_blank");
  }
};

const handleGoBack = () => {
  if (pathHistory.length > 0) {
    const previousPath = pathHistory[pathHistory.length - 1];
    setPathHistory(pathHistory.slice(0, -1));
    setCurrentPath(previousPath);
  }
};

const handleGoHome = () => {
  setPathHistory([]);
  setCurrentPath("/");
  setIsSearching(false);
  setSearchQuery("");
};

const handleSearch = () => {
  if (searchQuery.trim()) {
    setIsSearching(true);
  }
};

const handleClearSearch = () => {
  setIsSearching(false);
  setSearchQuery("");
};

const displayItems = isSearching && searchResults ? searchResults : items;
const pdfFiles = displayItems?.filter(item =>
  item.type === "file" && item.mimeType?.includes("pdf")
) || [];

```



```

const folders = displayItems?.filter(item => item.type === "folder") || [];
const otherFiles = displayItems?.filter(item =>
  item.type === "file" && item.mimeType?.includes("pdf")
) || [];

const pathParts = currentPath === "/" ? [] : currentPath.split("/").filter(Boolean);

if (error) {
  return (
    <div className="flex items-center justify-center h-full p-8">
      <Card className="max-w-md w-full">
        <CardHeader>
          <CardTitle className="text-destructive flex items-center gap-2">
            <Cloud className="w-5 h-5" />
            OneDrive Error
          </CardTitle>
        </CardHeader>
        <CardContent>
          <p className="text-muted-foreground mb-4">
            {(error as Error).message || "Failed to connect to OneDrive"}
          </p>
          <Button onClick={() => refetch()}>Try Again</Button>
        </CardContent>
      </Card>
    </div>
  );
}

return (
  <div className="flex flex-col h-full">
    <div className="border-b p-4">
      <div className="flex items-center gap-4 mb-4">
        <Button
          variant="ghost"
          size="icon"
          onClick={handleGoBack}
          disabled={pathHistory.length === 0 && !isSearching}
          data-testid="button-go-back"
        >
          <ArrowLeft className="w-4 h-4" />
        </Button>
        <Button
          variant="ghost"
          size="icon"
          onClick={handleGoHome}
          data-testid="button-go-home"
        >
          <Home className="w-4 h-4" />
        </Button>
      </div>
      <div className="flex items-center gap-1 text-sm text-muted-foreground flex-1">
        <Cloud className="w-4 h-4" />
        <span className="font-medium">OneDrive</span>
        {pathParts.map((part, i) => (
          <span key={i} className="flex items-center">
            <ChevronRight className="w-4 h-4 mx-1" />
            <span>{part}</span>
          </span>
        ))}
      </div>
    </div>
    <div className="flex gap-2">
      <Input
        placeholder="Search files..."
        value={searchQuery}
        onChange={(e) => setSearchQuery(e.target.value)}
        onKeyDown={(e) => e.key === "Enter" && handleSearch()}
        className="flex-1"
        data-testid="input-search"
      />
      <Button onClick={handleSearch} disabled={!searchQuery.trim()} data-testid="button-search">
        <Search className="w-4 h-4" />
      </Button>
      {isSearching && (
        <Button variant="outline" onClick={handleClearSearch} data-testid="button-clear-search">
          <Clear />
        </Button>
      )}
    </div>
    <div className="flex-1 overflow-auto p-4">
      {isLoading || searchLoading ? (
        <div className="flex items-center justify-center h-full">
          <Loader2 className="w-8 h-8 animate-spin text-muted-foreground" />
        </div>
      ) : (
        <div className="space-y-6">
          {folders.length > 0 && (
            <div>
              <h3 className="text-sm font-medium text-muted-foreground mb-2">Folders</h3>
              <div className="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4 lg:grid-cols-5 gap-3">
                {folders.map((item) => (
                  <button
                    key={item.id}
                    onClick={() => handleFolderClick(item)}
                    className="flex flex-col items-center p-4 rounded-lg border bg-card hover-elevate transition-al..."
                    data-testid={`folder-${item.id}`}
                  >
                    <Folder className="w-10 h-10 text-yellow-500 mb-2" />
                    <span className="text-sm font-medium truncate w-full">{item.name}</span>
                  </button>
                ))}
              </div>
            </div>
          )}
          {pdfFiles.length > 0 && (
            <div>

```

```

<h3 className="text-sm font-medium text-muted-foreground mb-2">PDF Files</h3>
<div className="space-y-2">
  {pdfFiles.map((item) => {
    <div
      key={item.id}
      className="flex items-center gap-4 p-3 rounded-lg border bg-card hover-elevate cursor-pointer"
      onClick={() => handleFileClick(item)}
      data-testid={`file-${item.id}`}
    >
      <FileText className="w-8 h-8 text-red-500 flex-shrink-0" />
      <div className="flex-1 min-w-0">
        <div className="font-medium truncate">{item.name}</div>
        <div className="text-sm text-muted-foreground flex items-center gap-3">
          <span>{formatFileSize(item.size)}</span>
          <span>{formatDate(item.lastModified)}</span>
        </div>
      </div>
      <Button size="sm" variant="ghost" data-testid={`button-open-${item.id}`}>
        <Play className="w-4 h-4" />
        Open
      </Button>
    </div>
  )})
</div>
)}

{otherFiles.length > 0 && (
  <div>
    <h3 className="text-sm font-medium text-muted-foreground mb-2">Other Files</h3>
    <div className="space-y-2">
      {otherFiles.map((item) => {
        const Icon = getFileIcon(item.mimeType);
        return (
          <div
            key={item.id}
            className="flex items-center gap-4 p-3 rounded-lg border bg-card hover-elevate cursor-pointer"
            onClick={() => handleFileClick(item)}
            data-testid={`file-${item.id}`}
          >
            <Icon className="w-8 h-8 text-muted-foreground flex-shrink-0" />
            <div className="flex-1 min-w-0">
              <div className="font-medium truncate">{item.name}</div>
              <div className="text-sm text-muted-foreground flex items-center gap-3">
                <span>{formatFileSize(item.size)}</span>
                <span>{formatDate(item.lastModified)}</span>
              </div>
            </div>
          </div>
        );
      })}
    </div>
  </div>
)}

{displayItems?.length === 0 && (
  <div className="flex flex-col items-center justify-center h-64 text-muted-foreground">
    <Folder className="w-16 h-16 mb-4 opacity-50" />
    <p>No files found</p>
  </div>
)}
</div>
)}
</div>
</div>
);
}

```

`client/src/pages/onenote.tsx`

```

import { useState, useEffect, useRef, useCallback } from "react";
import { useQuery, useMutation } from "@tanstack/react-query";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { useToast } from "@/hooks/use-toast";
import { Link } from "wouter";
import {
  ArrowLeft,
  Loader2,
  Home,
  FileText,
  StickyNote,
  Plus,
  Save,
  Search,
  X,
  BookOpen,
  ChevronRight,
  ChevronDown,
  RefreshCw,
  Trash2,
  NotebookPen,
  Camera,
  CheckSquare,
  Mic,
  Filter,
} from "lucide-react";
import { queryClient, apiRequest } from "@/lib/queryClient";

interface QuickNoteFile {
  id: string;
  name: string;
  type: string;
  size?: number;
}

```

```

    lastModified?: string;
    path: string;
  }

  interface OneNoteNotebook {
    name: string;
    path: string;
    sections: { name: string; id: string }[];
  }

  interface OneNotePage {
    title: string;
    content: string;
    position: number;
  }

  interface PageCard {
    notebookName: string;
    sectionName: string;
    title: string;
    preview: string;
    content: string;
    position: number;
  }

  type TopTab = "recent" | "notebooks";
  type BottomTab = "notebooks" | "stickynotes" | "search";
  type View = "grid" | "page" | "quicknote-editor";

  export default function OneNotePage() {
    const { toast } = useToast();
    const [topTab, setTopTab] = useState<TopTab>("recent");
    const [bottomTab, setBottomTab] = useState<BottomTab>("notebooks");
    const [view, setView] = useState<View>("grid");
    const [selectedPageCard, setSelectedPageCard] = useState<PageCard | null>(null);
    const [selectedQuickNote, setSelectedQuickNote] = useState<QuickNoteFile | null>(null);
    const [editorContent, setEditorContent] = useState('');
    const [isDirty, setIsDirty] = useState(false);
    const [isCreating, setIsCreating] = useState(false);
    const [newNoteName, setNewNoteName] = useState('');
    const [searchQuery, setSearchQuery] = useState('');
    const [expandedNotebooks, setExpandedNotebooks] = useState<Set<string>>(new Set());
    const [allPageCards, setAllPageCards] = useState<PageCard[]>([]);
    const [loadingPages, setLoadingPages] = useState(false);
    const [filterNotebook, setFilterNotebook] = useState<string>('');
    const [showFilterMenu, setShowFilterMenu] = useState(false);
    const [isRecording, setIsRecording] = useState(false);
    const [isChecklistCreating, setIsChecklistCreating] = useState(false);
    const [checklistItems, setChecklistItems] = useState<string[]>(['']);
    const [checklistName, setChecklistName] = useState('');
    const [isCreatingPage, setIsCreatingPage] = useState(false);
    const [newPageTitle, setNewPageTitle] = useState('');
    const [newPageNotebook, setNewPageNotebook] = useState('');
    const [newPageSection, setNewPageSection] = useState('');
    const saveTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
    const fileInputRef = useRef<HTMLInputElement>(null);
    const speechRecRef = useRef<any>(null);
    const filterMenuRef = useRef<HTMLDivElement>(null);

    const notebooksQuery = useQuery<OneNoteNotebook[]>({
      queryKey: ["/api/onenote/notebooks"],
      staleTime: 60000,
    });

    const filesQuery = useQuery<QuickNoteFile[]>({
      queryKey: ["/api/quicknotes/files"],
      staleTime: 15000,
    });

    const contentQuery = useQuery<{ content: string }>({
      queryKey: ["/api/quicknotes/file", selectedQuickNote?.id, "content"],
      enabled: !!selectedQuickNote && view === 'quicknote-editor',
      staleTime: 30000,
      refetchOnWindowFocus: false,
    });

    const saveMutation = useMutation({
      mutationFn: async ({ id, content }: { id: string; content: string }) => {
        const res = await apiRequest('PUT', `/api/quicknotes/file/${id}/content`, { content });
        return res.json();
      },
      onSuccess: (_data, variables) => {
        setIsDirty(false);
        queryClient.setQueryData(["/api/quicknotes/file", variables.id, "content"], { content: variables.content });
        queryClient.invalidateQueries({ queryKey: ["/api/quicknotes/files"] });
        toast({ title: "Saved to OneDrive" });
      },
      onError: (err: any) => {
        toast({ title: "Save failed", description: err.message, variant: "destructive" });
      },
    });

    const createMutation = useMutation({
      mutationFn: async ({ name, content }: { name: string; content: string }) => {
        const res = await apiRequest('POST', '/api/quicknotes/files', { name, content });
        return res.json();
      },
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey: ["/api/quicknotes/files"] });
        setIsCreating(false);
        setNewNoteName('');
        toast({ title: "Note created" });
      },
      onError: (err: any) => {
        toast({ title: "Create failed", description: err.message, variant: "destructive" });
      },
    });
  }

```

```

const deleteMutation = useMutation({
  mutationFn: async (id: string) => {
    const res = await apiRequest('DELETE', `/api/quicknotes/file/${id}`);
    return res.json();
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/quicknotes/files"] });
    setSelectedQuickNote(null);
    setView('grid');
    toast({ title: "Note deleted" });
  },
  onError: (err: any) => {
    toast({ title: "Delete failed", description: err.message, variant: "destructive" });
  },
});

const deletePageMutation = useMutation({
  mutationFn: async (card: PageCard) => {
    const res = await apiRequest('DELETE', '/api/onenote/page', {
      notebook: card.notebookName,
      section: card.sectionName,
      title: card.title,
    });
    return res.json();
  },
  onSuccess: (_data, card) => {
    setAllPageCards(prev => prev.filter(c => !(c.notebookName === card.notebookName && c.sectionName === card.secti...
    toast({ title: "Page deleted from OneNote" });
  },
  onError: (err: any) => {
    toast({ title: "Delete failed", description: err.message, variant: "destructive" });
  },
});

const createPageMutation = useMutation({
  mutationFn: async ({ notebook, section, title, content }: { notebook: string; section: string; title: string; con...
    const res = await apiRequest('POST', '/api/onenote/pages', { notebook, section, title, content });
    return res.json();
  },
  onSuccess: (_data, vars) => {
    setAllPageCards(prev => [{
      notebookName: vars.notebook,
      sectionName: vars.section,
      title: vars.title,
      preview: vars.content.substring(0, 100) || 'No additional text',
      content: vars.content,
      position: 0,
    }, ...prev]);
    setIsCreatingPage(false);
    setNewPageTitle('');
    toast({ title: "Page created in OneNote" });
  },
  onError: (err: any) => {
    toast({ title: "Create failed", description: err.message, variant: "destructive" });
  },
});

useEffect(() => {
  if (contentQuery.data && !isDirty) {
    setEditorContent(contentQuery.data.content);
  }
}, [contentQuery.data, isDirty]);

useEffect(() => {
  if (notebooksQuery.data && notebooksQuery.data.length > 0) {
    loadAllPages(notebooksQuery.data);
  }
}, [notebooksQuery.data]);

useEffect(() => {
  return () => {
    if (speechRecRef.current) {
      speechRecRef.current.stop();
      speechRecRef.current = null;
    }
  };
}, []);

useEffect(() => {
  if (!showFilterMenu) return;
  function handleClickOutside(e: MouseEvent) {
    if (filterMenuRef.current && !filterMenuRef.current.contains(e.target as Node)) {
      setShowFilterMenu(false);
    }
  }
  document.addEventListener('mousedown', handleClickOutside);
  return () => document.removeEventListener('mousedown', handleClickOutside);
}, [showFilterMenu]);

async function loadAllPages(notebooks: OneNoteNotebook[]) {
  setLoadingPages(true);
  const cards: PageCard[] = [];
  for (const nb of notebooks) {
    for (const sec of nb.sections) {
      try {
        const params = new URLSearchParams({ notebook: nb.path, section: sec.name, notebookName: nb.name });
        const res = await fetch(`/api/onenote/pages?${params}`);
        if (res.ok) {
          const pages: OneNotePage[] = await res.json();
          for (const page of pages) {
            const preview = page.content.substring(0, 100).replace(/\n/g, ' ').trim();
            cards.push({
              notebookName: nb.name,
              sectionName: sec.name,
              title: page.title,
              preview: preview || 'No additional text',
              content: page.content,
              position: page.position,
            });
          }
        }
      } catch {
        // ignore
      }
    }
  }
}

```

```

    });
  }
} catch (e) {
  console.error(`Failed to load pages for ${nb.name}/${sec.name}:`, e);
}
}
}
setAllPageCards(cards);
setLoadingPages(false);
}

const handleEditorChange = useCallback((value: string) => {
  setEditorContent(value);
  setIsDirty(true);
  if (saveTimeoutRef.current) clearTimeout(saveTimeoutRef.current);
  if (selectedQuickNote) {
    saveTimeoutRef.current = setTimeout(() => {
      saveMutation.mutate({ id: selectedQuickNote.id, content: value });
    }, 2000);
  }
}, [selectedQuickNote]);

function saveNow() {
  if (saveTimeoutRef.current) clearTimeout(saveTimeoutRef.current);
  if (selectedQuickNote && isDirty) {
    saveMutation.mutate({ id: selectedQuickNote.id, content: editorContent });
  }
}

function openPageCard(card: PageCard) {
  if (isDirty && selectedQuickNote) saveNow();
  setSelectedPageCard(card);
  setSelectedQuickNote(null);
  setView('page');
}

function openQuickNote(file: QuickNoteFile) {
  if (isDirty && selectedQuickNote) saveNow();
  setSelectedQuickNote(file);
  setSelectedPageCard(null);
  setView('quicknote-editor');
}

function handleCameraCapture() {
  fileInputRef.current?.click();
}

async function handleFileSelected(e: React.ChangeEvent<HTMLInputElement>) {
  const file = e.target.files?.[0];
  if (!file) return;
  const timestamp = new Date().toLocaleString('en-US', { month: 'short', day: 'numeric', hour: '2-digit', minute: '...' });
  const name = `Photo Note ${timestamp}.txt`;
  const content = `0+ Photo captured: ${file.name}\nDate: ${timestamp}\nSize: ${((file.size / 1024).toFixed(1))} KB\n...`;
  createMutation.mutate({ name, content });
  e.target.value = '';
}

function handleStartRecording() {
  const SpeechRecognition = (window as any).SpeechRecognition || (window as any).webkitSpeechRecognition;
  if (!SpeechRecognition) {
    toast({ title: "Not supported", description: "Speech recognition not available in this browser", variant: "destructive" });
    return;
  }
  const recognition = new SpeechRecognition();
  recognition.continuous = true;
  recognition.interimResults = false;
  recognition.lang = 'en-US';
  let transcript = '';

  recognition.onresult = (event: any) => {
    for (let i = event.resultIndex; i < event.results.length; i++) {
      if (event.results[i].isFinal) {
        transcript += event.results[i][0].transcript + ' ';
      }
    }
  };

  recognition.onend = () => {
    setIsRecording(false);
    if (transcript.trim()) {
      const timestamp = new Date().toLocaleString('en-US', { month: 'short', day: 'numeric', hour: '2-digit', minute: '...' });
      const name = `Voice Note ${timestamp}.txt`;
      createMutation.mutate({ name, content: transcript.trim() });
      toast({ title: "Voice note saved", description: `${transcript.trim().split(' ').length} words captured` });
    } else {
      toast({ title: "No speech detected", description: "Try again and speak clearly" });
    }
  };

  recognition.onerror = (event: any) => {
    setIsRecording(false);
    if (event.error === 'not-allowed') {
      toast({ title: "Microphone blocked", description: "Allow microphone access in your browser", variant: "destructive" });
    } else {
      toast({ title: "Recording error", description: event.error, variant: "destructive" });
    }
  };

  setIsRecording(true);
  recognition.start();
  toast({ title: "Listening...", description: "Speak now. Tap mic again to stop." });

  speechRecRef.current = recognition;
}

function handleStopRecording() {
  if (speechRecRef.current) {

```

```

        speechRecRef.current.stop();
        speechRecRef.current = null;
    }
    setIsRecording(false);
}

function handleCreateChecklist() {
    const items = checklistItems.filter(item => item.trim());
    if (items.length === 0) {
        toast({ title: "Add at least one item" });
        return;
    }
    const name = (checklistName.trim() || 'Checklist') + '.txt';
    const content = items.map(item => `& ${item}`).join('\n');
    createMutation.mutate({ name, content });
    setIsChecklistCreating(false);
    setChecklistItems(['']);
    setChecklistName('');
}

function goBack() {
    if (isDirty && selectedQuickNote) saveNow();
    setSelectedPageCard(null);
    setSelectedQuickNote(null);
    setView('grid');
}

const notebooks = notebooksQuery.data || [];
const quickNoteFiles = filesQuery.data || [];

const filteredByNotebook = filterNotebook
    ? allPageCards.filter(c => c.notebookName === filterNotebook)
    : allPageCards;

const filteredCards = searchQuery.trim()
    ? filteredByNotebook.filter(c =>
        c.title.toLowerCase().includes(searchQuery.toLowerCase()) ||
        c.preview.toLowerCase().includes(searchQuery.toLowerCase()) ||
        c.sectionName.toLowerCase().includes(searchQuery.toLowerCase())
    )
    : filteredByNotebook;

const uniqueNotebookNames = [...new Set(allPageCards.map(c => c.notebookName))];

const truncatePath = (nbName: string, secName: string) => {
    const full = `${nbName} » ${secName}`;
    return full.length > 24 ? full.substring(0, 22) + '...' : full;
};

if (view === 'page' && selectedPageCard) {
    return (
        <div className="h-screen flex flex-col" style={{ background: '#1a1a1a', color: '#e0e0e0' }}>
            <div className="h-12 flex items-center px-4 gap-3 shrink-0" style={{ background: '#2d2d30', borderBottom: '1p...' }}>
                <button onClick={goBack} className="flex items-center gap-1.5 text-white/70 hover:text-white transition-col...">
                    <ArrowLeft className="h-4 w-4" />
                    <span className="text-sm">Back</span>
                </button>
                <div className="flex-1" />
                <span className="text-xs text-white/30">{selectedPageCard.notebookName} » {selectedPageCard.sectionName}</span>
                <Link href="/">
                    <Button variant="ghost" size="icon" className="h-7 w-7 text-white/50 hover:text-white hover:bg-white/10" ...>
                        <Home className="h-3.5 w-3.5" />
                    </Button>
                </Link>
            </div>
            <div className="flex-1 overflow-y-auto">
                <OneNotePageContent page={selectedPageCard} />
            </div>
        </div>
    );
}

if (view === 'quicknote-editor' && selectedQuickNote) {
    return (
        <div className="h-screen flex flex-col" style={{ background: '#1a1a1a', color: '#e0e0e0' }}>
            <div className="h-12 flex items-center px-4 gap-3 shrink-0" style={{ background: '#2d2d30', borderBottom: '1p...' }}>
                <button onClick={goBack} className="flex items-center gap-1.5 text-white/70 hover:text-white transition-col...">
                    <ArrowLeft className="h-4 w-4" />
                    <span className="text-sm">Back</span>
                </button>
                <div className="flex-1" />
                <Button
                    variant="ghost" size="sm" onClick={saveNow}
                    disabled={saveMutation.isPending || !isDirty}
                    className={`h-7 gap-1 text-xs ${isDirty ? 'text-orange-400' : 'text-white/30'}`}
                    data-testid="button-save"
                >
                    {saveMutation.isPending ? <Loader2 className="h-3 w-3 animate-spin" /> : <Save className="h-3 w-3" />}
                    {saveMutation.isPending ? 'Saving...' : isDirty ? 'Save' : 'Saved'}
                </Button>
                <Button
                    variant="ghost" size="icon" onClick={() => { if (confirm('Delete this note from OneDrive?')) deleteMutati...
                    className="h-7 w-7 text-white/30 hover:text-red-400"
                    data-testid="button-delete-note"
                >
                    <Trash2 className="h-3.5 w-3.5" />
                </Button>
                <Link href="/">
                    <Button variant="ghost" size="icon" className="h-7 w-7 text-white/50 hover:text-white hover:bg-white/10" ...>
                        <Home className="h-3.5 w-3.5" />
                    </Button>
                </Link>
            </div>
            <div className="flex-1 overflow-y-auto">
                <QuickNoteEditor
                    file={selectedQuickNote}
                    content={editorContent}
                    onChange={handleEditorChange}
                />
            </div>
        </div>
    );
}

```



```

        <StickyNotesView
          files={quickNoteFiles}
          isLoading={filesQuery.isLoading}
          onOpenNote={openQuickNote}
          onStartCreate={() => setIsCreating(true)}
        />
      )}
    {bottomTab === 'search' && (
      <SearchView
        searchQuery={searchQuery}
        onSearchChange={setSearchQuery}
        cards={filteredCards}
        quickNotes={quickNoteFiles}
        onOpenCard={openPageCard}
        onDeleteCard={(card) => deletePageMutation.mutate(card)}
        deletingCard={deletePageMutation.isPending ? deletePageMutation.variables : null}
        onOpenNote={openQuickNote}
        truncatePath={truncatePath}
      />
    )}
  </div>

  {/} Hidden file input for camera */}
  <input
    ref={fileInputRef}
    type="file"
    accept="image/*"
    capture="environment"
    className="hidden"
    onChange={handleFileSelected}
  />

  {/} Filter indicator */}
  {filterNotebook && (
    <div className="shrink-0 flex items-center justify-center gap-2 py-1.5 px-4" style={{ background: '#1a1a1a' }}>
      <span className="text-[11px] text-white/40">Filtered:</span>
      <span className="text-[11px] font-medium" style={{ color: '#c586c0' }}><filterNotebook></span>
      <button onClick={() => setFilterNotebook('')} className="text-white/30 hover:text-white/60">
        <X className="h-3 w-3" />
      </button>
    </div>
  )}

  {/} Floating action bar */}
  <div className="shrink-0 flex items-center justify-center gap-1 py-2 px-4" style={{ background: '#1a1a1a' }}>
    <div className="flex items-center gap-1 px-2 py-1.5 rounded-full relative" style={{ background: '#2d2d30', bo...
      <button
        className="h-9 w-9 rounded-full flex items-center justify-center text-white/50 hover:text-white hover:bg-...
        onClick={handleCameraCapture}
        data-testid="action-camera"
        title="Photo note"
      >
        <Camera className="h-4.5 w-4.5" />
      </button>
      <button
        className="h-9 w-9 rounded-full flex items-center justify-center text-white/50 hover:text-white hover:bg-...
        onClick={() => { setIsChecklistCreating(true); setChecklistItems(['']); setChecklistName(''); }}
        data-testid="action-checklist"
        title="New checklist"
      >
        <CheckSquare className="h-4.5 w-4.5" />
      </button>
      <button
        className={`h-9 w-9 rounded-full flex items-center justify-center transition-colors ${isRecording ? 'text-...
        onClick={isRecording ? handleStopRecording : handleStartRecording}
        data-testid="action-voice"
        title={isRecording ? "Stop recording" : "Voice note"}
      >
        <Mic className="h-4.5 w-4.5" />
      </button>
      <div className="relative" ref={filterMenuRef}>
        <button
          className={`h-9 w-9 rounded-full flex items-center justify-center transition-colors ${filterNotebook ? ...
          onClick={() => setShowFilterMenu(!showFilterMenu)}
          data-testid="action-filter"
          title="Filter by notebook"
        >
          <Filter className="h-4.5 w-4.5" />
        </button>
        {showFilterMenu && (
          <div className="absolute bottom-12 left-1/2 -translate-x-1/2 rounded-lg py-1 min-w-[180px] z-50 shadow-...
            <button
              className={`w-full text-left px-3 py-2 text-xs transition-colors ${!filterNotebook ? 'text-purple-4...
              onClick={() => { setFilterNotebook(''); setShowFilterMenu(false); }}
            >
              All notebooks
            </button>
            {uniqueNotebookNames.map(name => (
              <button
                key={name}
                className={`w-full text-left px-3 py-2 text-xs transition-colors ${filterNotebook === name ? 'tex...
                onClick={() => { setFilterNotebook(name); setShowFilterMenu(false); }}
              >
                {name}
              </button>
            )}
          </div>
        )}
      </div>
    </div>
  </div>
  <div>
    <button
      className="h-11 w-11 rounded-full flex items-center justify-center text-white ml-2 shadow-lg hover:brightne...
      style={{ background: '#7b2d8e' }}
      onClick={() => setIsCreating(true)}
      data-testid="action-create"
      title="New page"
    >
    <Plus className="h-5 w-5" />
  </div>

```



```

    </button>
  </div>

  /* Bottom navigation bar */
  <div className="shrink-0 flex items-center justify-around py-2 px-4" style={{ background: '#2d2d30', borderTop:...
    <button
      className={`flex flex-col items-center gap-0.5 px-6 py-1 rounded transition-colors ${bottomTab === 'noteboo...
      onClick={() => { setBottomTab('notebooks'); setView('grid'); }}
      data-testid="nav-notebooks"
    >
      <BookOpen className="h-5 w-5" />
      <span className="text-[10px]">Notebooks</span>
    </button>
    <button
      className={`flex flex-col items-center gap-0.5 px-6 py-1 rounded transition-colors ${bottomTab === 'stickyn...
      onClick={() => { setBottomTab('stickynotes'); setView('grid'); }}
      data-testid="nav-stickynotes"
    >
      <StickyNote className="h-5 w-5" />
      <span className="text-[10px]">Sticky Notes</span>
    </button>
    <button
      className={`flex flex-col items-center gap-0.5 px-6 py-1 rounded transition-colors ${bottomTab === 'search'...
      onClick={() => { setBottomTab('search'); setView('grid'); }}
      data-testid="nav-search"
    >
      <Search className="h-5 w-5" />
      <span className="text-[10px]">Search</span>
    </button>
  </div>

  /* Create note modal */
  {isCreating && (
    <div className="fixed inset-0 bg-black/60 backdrop-blur-sm z-50 flex items-center justify-center p-4" onClick...
    <div className="rounded-xl p-5 w-full max-w-sm" style={{ background: '#2d2d30', border: '1px solid #3e3e42'...
      <h3 className="text-sm font-semibold mb-3 text-white">New Sticky Note</h3>
      <input
        value={newNoteName}
        onChange={e => setNewNoteName(e.target.value)}
        placeholder="Note name (e.g. Meeting Notes.txt)"
        className="bg-white/5 border-white/15 text-white placeholder:text-white/30 mb-3 text-sm"
        autoFocus
        data-testid="input-new-note-name"
        onKeyDown={e => {
          if (e.key === 'Enter' && newNoteName.trim()) {
            const name = newNoteName.trim().endsWith('.txt') || newNoteName.trim().endsWith('.md') ? newNoteNam...
              createMutation.mutate({ name, content: '' });
          }
        }}
      />
    <div className="flex gap-2 justify-end">
      <button variant="ghost" size="sm" onClick={() => { setIsCreating(false); setNewNoteName(''); }} classNa...
      <button size="sm" disabled={!newNoteName.trim()} createMutation.isPending
        onClick={() => {
          const name = newNoteName.trim().endsWith('.txt') || newNoteName.trim().endsWith('.md') ? newNoteNam...
            createMutation.mutate({ name, content: '' });
        }}
        className="text-xs text-white"
        style={{ background: '#7b2d8e' }}
        data-testid="button-confirm-create"
      >
        {createMutation.isPending ? <Loader2 className="h-3 w-3 animate-spin mr-1" /> : <Plus className="h-3 ...
        Create
      </button>
    </div>
  </div>
  </div>
  </div>
)}

/* Checklist creation modal */
{isChecklistCreating && (
  <div className="fixed inset-0 bg-black/60 backdrop-blur-sm z-50 flex items-center justify-center p-4" onClick...
  <div className="rounded-xl p-5 w-full max-w-sm" style={{ background: '#2d2d30', border: '1px solid #3e3e42'...
    <h3 className="text-sm font-semibold mb-3 text-white flex items-center gap-2">
      <CheckSquare className="h-4 w-4" style={{ color: '#c586c0' }} />
      New Checklist
    </h3>
    <input
      value={checklistName}
      onChange={e => setChecklistName(e.target.value)}
      placeholder="Checklist title (optional)"
      className="bg-white/5 border-white/15 text-white placeholder:text-white/30 mb-3 text-sm"
      autoFocus
      data-testid="input-checklist-name"
    />
    <div className="space-y-1.5 mb-3 max-h-[200px] overflow-y-auto">
      {checklistItems.map((item, i) => (
        <div key={i} className="flex items-center gap-2">
          <span className="text-white/30 text-sm">& </span>
          <input
            value={item}
            onChange={e => {
              const updated = [...checklistItems];
              updated[i] = e.target.value;
              setChecklistItems(updated);
            }}
            placeholder={`Item ${i + 1}`}
            className="bg-white/5 border-white/15 text-white placeholder:text-white/30 text-sm flex-1"
            data-testid={`input-checklist-item-${i}`}
            onKeyDown={e => {
              if (e.key === 'Enter') {
                e.preventDefault();
                setChecklistItems([...checklistItems, '']);
              }
            }}
          />
        </div>
      ))
    </div>
    {checklistItems.length > 1 && (
      <button onClick={() => setChecklistItems(checklistItems.filter((_, j) => j !== i))} className="te...

```



```

        <span className="ml-3 text-sm text-white/30">Loading pages...</span>
      </div>
    );
  }

  if (cards.length === 0) {
    return (
      <div className="text-center py-16 text-white/30">
        <NotebookPen className="h-10 w-10 mx-auto mb-3 opacity-30" />
        <p className="text-sm">No pages found</p>
      </div>
    );
  }

  return (
    <div className="grid gap-3" style={{ gridTemplateColumns: 'repeat(auto-fill, minmax(200px, 1fr))' }}>
      {cards.map((card, i) => (
        <PageCardComponent key={`_${card.title}-${i}`} card={card} onClick={() => onOpenCard(card)} onDelete={() => on...
      ))}
    </div>
  );
}

function PageCardComponent({ card, onClick, onDelete, isDeleting, truncatePath, index }: {
  card: PageCard;
  onClick: () => void;
  onDelete?: () => void;
  isDeleting?: boolean;
  truncatePath: (nb: string, sec: string) => string;
  index: number;
}) {
  const [swipeX, setSwipeX] = useState(0);
  const [swiping, setSwiping] = useState(false);
  const touchStartRef = useRef<{ x: number; y: number; time: number } | null>(null);
  const cardRef = useRef<HTMLDivElement>(null);
  const swipeXRef = useRef(0);
  const cleanupRef = useRef<() => void> | null>(null);
  const deleteThreshold = -80;

  useEffect(() => () => { cleanupRef.current?.(); }, []);

  const updateSwipeX = (val: number) => { swipeXRef.current = val; setSwipeX(val); };

  const handleTouchStart = (e: React.TouchEvent) => {
    touchStartRef.current = { x: e.touches[0].clientX, y: e.touches[0].clientY, time: Date.now() };
  };

  const handleTouchMove = (e: React.TouchEvent) => {
    if (!touchStartRef.current) return;
    const dx = e.touches[0].clientX - touchStartRef.current.x;
    const dy = e.touches[0].clientY - touchStartRef.current.y;
    if (Math.abs(dy) > Math.abs(dx) && !swiping) return;
    if (dx < -10) {
      setSwiping(true);
      updateSwipeX(Math.max(dx, -120));
      e.preventDefault();
    }
  };

  const handleTouchEnd = () => {
    if (swipeXRef.current < deleteThreshold) {
      updateSwipeX(-120);
    } else {
      updateSwipeX(0);
      setSwiping(false);
    }
    touchStartRef.current = null;
  };

  const handleMouseDown = (e: React.MouseEvent) => {
    cleanupRef.current?.();
    touchStartRef.current = { x: e.clientX, y: e.clientY, time: Date.now() };
    const handleMouseMove = (ev: MouseEvent) => {
      if (!touchStartRef.current) return;
      const dx = ev.clientX - touchStartRef.current.x;
      if (dx < -10) {
        setSwiping(true);
        updateSwipeX(Math.max(dx, -120));
      }
    };
  };

  const handleMouseUp = () => {
    if (swipeXRef.current < deleteThreshold) {
      updateSwipeX(-120);
    } else {
      updateSwipeX(0);
      setSwiping(false);
    }
    touchStartRef.current = null;
    cleanup();
  };

  const cleanup = () => {
    window.removeEventListener('mousemove', handleMouseMove);
    window.removeEventListener('mouseup', handleMouseUp);
    cleanupRef.current = null;
  };

  cleanupRef.current = cleanup;
  window.addEventListener('mousemove', handleMouseMove);
  window.addEventListener('mouseup', handleMouseUp);
};

return (
  <div ref={cardRef} className="relative overflow-hidden rounded-lg" style={{ minHeight: '140px' }} data-testid={`p...
    <div
      className="absolute inset-0 flex items-center justify-end pr-4 rounded-lg"
      style={{ background: '#dc2626' }}
    >
      <button
        className="flex flex-col items-center gap-1 text-white"

```

```

        onClick={() => {
          e.stopPropagation();
          if (onDelete && !isDeleting) onDelete();
        }}
        data-testid={`button-delete-page-${index}`}
      >
      {isDeleting ? (
        <Loader2 className="h-5 w-5 animate-spin" />
      ) : (
        <Trash2 className="h-5 w-5" />
      )}
      <span className="text-[10px] font-medium">Delete</span>
    </button>
  </div>
  <div
    className="relative text-left rounded-lg p-3 flex flex-col justify-between cursor-pointer"
    style={{
      background: '#2d2d30',
      border: '1px solid #3e3e42',
      minHeight: '140px',
      transform: `translateX(${swipeX}px)`,
      transition: swiping ? 'none' : 'transform 0.3s ease',
    }}
    onClick={() => { if (!swiping && swipeX === 0) onClick(); else { updateSwipeX(0); setSwiping(false); } }}
    onTouchStart={handleTouchStart}
    onTouchMove={handleTouchMove}
    onTouchEnd={handleTouchEnd}
    onMouseDown={handleMouseDown}
  >
    <div>
      <div className="text-[10px] text-white/35 mb-1.5 truncate">
        {truncatePath(card.notebookName, card.sectionName)}
      </div>
      <div className="text-sm font-semibold text-white mb-1 line-clamp-2">
        {card.title || 'Untitled Page'}
      </div>
      <div className="text-xs text-white/40 line-clamp-2">
        {card.preview || 'No additional text'}
      </div>
    </div>
    <div className="flex items-center justify-between mt-3">
      <FileText className="h-4 w-4" style={{ color: '#7b2d8e' }} />
      <span className="text-[10px] text-white/25">
        {new Date().toLocaleDateString('en-GB', { day: '2-digit', month: '2-digit' })}
      </span>
    </div>
  </div>
</div>
);
}

function NotebookListView({ notebooks, isLoading, expandedNotebooks, onToggleNotebook, allCards, onOpenCard, onDelete...
notebooks: OneNoteNotebook[];
isLoading: boolean;
expandedNotebooks: Set<string>;
onToggleNotebook: (name: string) => void;
allCards: PageCard[];
onOpenCard: (c: PageCard) => void;
onDeleteCard: (c: PageCard) => void;
deletingCard: PageCard | null | undefined;
truncatePath: (nb: string, sec: string) => string;
}) {
  if (isLoading) {
    return (
      <div className="flex items-center justify-center py-16">
        <Loader2 className="h-6 w-6 animate-spin text-white/30" />
      </div>
    );
  }

  return (
    <div className="space-y-2">
      {notebooks.map(nb => (
        <div key={nb.name} className="rounded-lg overflow-hidden" style={{ background: '#2d2d30', border: '1px solid ...
          <button
            className="w-full flex items-center gap-3 px-4 py-3 text-left hover:bg-white/5 transition-colors"
            onClick={() => onToggleNotebook(nb.name)}
            data-testid={`notebook-${nb.name}`}
          >
            <div className="h-8 w-8 rounded flex items-center justify-center shrink-0" style={{ background: '#7b2d8e'...
              <BookOpen className="h-4 w-4 text-white" />
            </div>
            <div className="flex-1 min-w-0">
              <div className="text-sm font-medium text-white truncate">{nb.name}</div>
              <div className="text-[11px] text-white/35">{nb.sections.length} section{nb.sections.length !== 1 ? 's' ...
            </div>
            {expandedNotebooks.has(nb.name) ? (
              <ChevronDown className="h-4 w-4 text-white/30 shrink-0" />
            ) : (
              <ChevronRight className="h-4 w-4 text-white/30 shrink-0" />
            )}
          </button>

          {expandedNotebooks.has(nb.name) && (
            <div className="px-4 pb-3 space-y-1" style={{ borderTop: '1px solid #3e3e42' }}>
              {nb.sections.map(sec => {
                const sectionCards = allCards.filter(c => c.notebookName === nb.name && c.sectionName === sec.name);
                return (
                  <div key={sec.id} className="mt-2">
                    <div className="flex items-center gap-2 px-2 py-1.5">
                      <FileText className="h-3.5 w-3.5" style={{ color: '#7b2d8e' }} />
                      <span className="text-xs font-medium text-white/60">{sec.name}</span>
                      <span className="text-[10px] text-white/25">{sectionCards.length} pages</span>
                    </div>
                    {sectionCards.length > 0 && (
                      <div className="grid gap-2 ml-5" style={{ gridTemplateColumns: 'repeat(auto-fill, minmax(180px,...
                        {sectionCards.map((card, i) => (
                          <PageCardComponent

```

```

        key={`${card.title}-${i}`}
        card={card}
        onClick={() => onOpenCard(card)}
        onDelete={() => onDeleteCard(card)}
        isDeleting={deletingCard?.title === card.title && deletingCard?.sectionName === card.sect...
        truncatePath={truncatePath}
        index={i}
      )/>
    ))}
  </div>
)
</div>
);
}}
</div>
))
</div>
);
}
}

function StickyNotesView({ files, isLoading, onOpenNote, onStartCreate }: {
  files: QuickNoteFile[];
  isLoading: boolean;
  onOpenNote: (f: QuickNoteFile) => void;
  onStartCreate: () => void;
}) {
  if (isLoading) {
    return (
      <div className="flex items-center justify-center py-16">
        <Loader2 className="h-6 w-6 animate-spin text-white/30" />
      </div>
    );
  }

  return (
    <div>
      <div className="flex items-center justify-between mb-4">
        <h2 className="text-sm font-medium text-white/60">Sticky Notes</h2>
        <button
          className="flex items-center gap-1 px-3 py-1.5 rounded-full text-xs text-white/60 hover:text-white transiti...
          style={{ background: 'rgba(255,255,255,0.08)', border: '1px solid rgba(255,255,255,0.1)' }}
          onClick={onStartCreate}
          data-testid="button-new-quicknote"
        >
          <Plus className="h-3 w-3" />
          New
        </button>
      </div>
      {files.length === 0 ? (
        <div className="text-center py-12 text-white/30">
          <StickyNote className="h-8 w-8 mx-auto mb-3 opacity-30" />
          <p className="text-sm">No sticky notes yet</p>
        </div>
      ) : (
        <div className="space-y-1.5">
          {files.map(f => (
            <button
              key={f.id}
              className="w-full text-left flex items-center gap-3 px-4 py-3 rounded-lg transition-colors hover:bg-whi...
              style={{ background: '#2d2d30', border: '1px solid #3e3e42' }}
              onClick={() => onOpenNote(f)}
              data-testid={`quicknote-${f.id}`}
            >
              <StickyNote className="h-5 w-5 shrink-0" style={{ color: '#e8b230' }} />
              <div className="flex-1 min-w-0">
                <div className="text-sm text-white/80 truncate">{f.name}</div>
                {f.lastModified && (
                  <div className="text-[10px] text-white/30 mt-0.5">
                    {new Date(f.lastModified).toLocaleDateString('en-GB', { day: '2-digit', month: '2-digit', year: '...
                  </div>
                )}
              </div>
              <ChevronRight className="h-4 w-4 text-white/20 shrink-0" />
            </button>
          ))}
        </div>
      )}
    </div>
  );
}

function SearchView({ searchQuery, onSearchChange, cards, quickNotes, onOpenCard, onDeleteCard, deletingCard, onOpenN...
  searchQuery: string;
  onSearchChange: (q: string) => void;
  cards: PageCard[];
  quickNotes: QuickNoteFile[];
  onOpenCard: (c: PageCard) => void;
  onDeleteCard: (c: PageCard) => void;
  deletingCard: PageCard | null | undefined;
  onOpenNote: (f: QuickNoteFile) => void;
  truncatePath: (nb: string, sec: string) => string;
}) {
  const filteredNotes = searchQuery.trim()
    ? quickNotes.filter(f => f.name.toLowerCase().includes(searchQuery.toLowerCase()))
    : [];

  return (
    <div>
      <div className="relative mb-4">
        <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-white/25" />
        <input
          type="text"
          value={searchQuery}
          onChange={e => onSearchChange(e.target.value)}
          placeholder="Search all notes..."
          className="w-full h-10 pl-10 pr-8 text-sm rounded-lg text-white placeholder:text-white/25 focus:outline-none"
        />
      </div>
    </div>
  );
}

```

```

        style={{ background: '#2d2d30', border: '1px solid #3e3e42' }}
        autoFocus
        data-testid="input-search"
      />
    {searchQuery && (
      <button onClick={() => onSearchChange('')} className="absolute right-3 top-1/2 -translate-y-1/2" data-testi...
      <X className="h-4 w-4 text-white/30" />
    </button>
    )}
  </div>

  {!searchQuery.trim() ? (
    <div className="text-center py-12 text-white/25">
      <Search className="h-8 w-8 mx-auto mb-3 opacity-30" />
      <p className="text-sm">Type to search your notes</p>
    </div>
  ) : (
    <div className="space-y-4">
      {cards.length > 0 && (
        <div>
          <h3 className="text-xs font-medium text-white/40 mb-2">Notebook Pages ({cards.length})</h3>
          <div className="grid gap-2" style={{ gridTemplateColumns: 'repeat(auto-fill, minmax(200px, 1fr))' }}>
            {cards.map((card, i) => (
              <PageCardComponent key={`_${card.title}-${i}`} card={card} onClick={() => onOpenCard(card)} onDelete...
            ))}
          </div>
        </div>
      )}
      {filteredNotes.length > 0 && (
        <div>
          <h3 className="text-xs font-medium text-white/40 mb-2">Sticky Notes ({filteredNotes.length})</h3>
          <div className="space-y-1">
            {filteredNotes.map(f => (
              <button
                key={f.id}
                className="w-full text-left flex items-center gap-3 px-3 py-2 rounded-lg hover:bg-white/5 transit...
                onClick={() => onOpenNote(f)}
              >
                <StickyNote className="h-4 w-4 shrink-0" style={{ color: '#e8b230' }} />
                <span className="text-sm text-white/70 truncate">{f.name}</span>
              </button>
            ))}
          </div>
        </div>
      )}
      {cards.length === 0 && filteredNotes.length === 0 && (
        <div className="text-center py-8 text-white/25 text-sm">
          No results for "{searchQuery}"
        </div>
      )}
    </div>
  )}
};
</div>
</div>
}

function OneNotePageContent({ page }: { page: PageCard }) {
  const lines = page.content.split('\n');

  return (
    <div className="max-w-3xl mx-auto px-8 py-6">
      <h1 className="text-2xl font-bold text-white mb-1" data-testid="text-onenote-page-title">{page.title}</h1>
      <div className="text-[11px] text-white/30 mb-2">{page.notebookName} » {page.sectionName}</div>
      <div className="h-px mb-5" style={{ background: '#7b2d8e' }} />
      <div className="space-y-1.5">
        {lines.map((line, i) => {
          if (!line.trim()) return <div key={i} className="h-3" />;
          const isHeading = line.startsWith('# ') || line.startsWith('## ');
          const isBullet = line.trimStart().startsWith('- ') || line.trimStart().startsWith('• ');
          const isNumbered = /\^d+\.\s\/.test(line.trimStart());
          const isLink = line.includes('http');

          if (isHeading) {
            const level = line.startsWith('## ') ? 2 : 1;
            const text = line.replace(/\^#{1,2}\s\/, '');
            return (
              <h2 key={i} className={`font-semibold text-white ${level === 1 ? 'text-lg mt-4' : 'text-base mt-3'}`>
                {text}
              </h2>
            );
          }

          if (isBullet) {
            const indent = line.length - line.trimStart().length;
            const text = line.trimStart().replace(/\^[-\s\/, ' ']/, '');
            return (
              <div key={i} className="flex gap-2 text-sm text-white/80" style={{ paddingLeft: indent * 8 + 'px' }}>
                <span className="text-white/30 mt-0.5 shrink-0">•</span>
                <span>{text}</span>
              </div>
            );
          }

          if (isNumbered) {
            const match = line.trimStart().match(/\^(\d+)\.\s(.*)/);
            if (match) {
              return (
                <div key={i} className="flex gap-2 text-sm text-white/80">
                  <span className="text-white/40 shrink-0 w-5 text-right">{match[1]}</span>
                  <span>{match[2]}</span>
                </div>
              );
            }
          }

          if (isLink) {
            const parts = line.split(/\^([^\s]+)\s\/g);
            return (

```

```

        <p key={i} className="text-sm text-white/80">
          {parts.map((part, pi) =>
            part.startsWith('http') ? (
              <a key={pi} href={part} target="_blank" rel="noopener noreferrer" className="text-blue-400 hover:...
            ) : (
              <span key={pi}>{part}</span>
            )
          )}
        </p>
      );
    }
  }

  return <p key={i} className="text-sm text-white/80 leading-relaxed">{line}</p>;
  }}}
</div>
</div>
);
}

function QuickNoteEditor({ file, content, onChange, isLoading }: {
  file: QuickNoteFile;
  content: string;
  onChange: (val: string) => void;
  isLoading: boolean;
}) {
  const textareaRef = useRef<HTMLTextAreaElement>(null);

  useEffect(() => {
    if (textareaRef.current) {
      textareaRef.current.style.height = 'auto';
      textareaRef.current.style.height = Math.max(400, textareaRef.current.scrollHeight) + 'px';
    }
  }, [content]);

  if (isLoading) {
    return (
      <div className="flex items-center justify-center py-16">
        <Loader2 className="h-5 w-5 animate-spin text-white/30" />
      </div>
    );
  }

  const displayName = file.name.replace(/\.(\.txt|md)$/, '');

  return (
    <div className="max-w-3xl mx-auto px-8 py-6">
      <h1 className="text-2xl font-bold text-white mb-1" data-testid="text-quicknote-title">{displayName}</h1>
      <div className="text-[10px] text-white/25 mb-4 flex items-center gap-2">
        <StickyNote className="h-3 w-3" />
        <span>Auto-saves to OneDrive</span>
      </div>
      <div className="h-px mb-4" style={{ background: '#3e3e42' }} />
      <textarea
        ref={textareaRef}
        value={content}
        onChange={e => onChange(e.target.value)}
        className="w-full bg-transparent text-white/85 text-sm leading-relaxed resize-none focus:outline-none min-h-[...
        style={{ fontFamily: "'Segoe UI', sans-serif" }}
        placeholder="Start typing..."
        spellCheck
        data-testid="textarea-editor"
      />
    </div>
  );
}

```

`client/src/pages/pdf-reader.tsx`

```

import { useState, useEffect, useRef } from "react";
import { useQuery, useQueryClient } from "@tanstack/react-query";
import { Link, useRoute } from "wouter";
import { Document, Page, pdfjs } from "react-pdf";
import { Button } from "@/components/ui/button";
import { Slider } from "@/components/ui/slider";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { useToast } from "@/hooks/use-toast";
import {
  ArrowLeft,
  Play,
  Pause,
  Square,
  SkipBack,
  SkipForward,
  Volume1,
  Volume2,
  VolumeX,
  ChevronLeft,
  ChevronRight,
  FileText,
  Loader2,
  RefreshCw,
  RotateCcw,
  RotateCw,
  Pencil,
  Check,
  X,
  Cast,
  Monitor,
  Speaker,
  Headphones,
  Search,
  Replace,
  ChevronUp,
  ChevronDown,
}

```

```

Mic,
Maximize2,
Minimize2,
Minus,
GripHorizontal,
ArrowDownToLine
} from "lucide-react";
import type { FileRecord } from "@shared/schema";
import { getWeekNumber } from "@shared/schema";
import tmuBgPath from "@assets/TMU2_1772842397746.png";
import dragTabPath from "@assets/drag-tab.svg";

pdfjs.GlobalWorkerOptions.workerSrc = `//unpkg.com/pdfjs-dist@${pdfjs.version}/build/pdf.worker.min.mjs`;

const SPEAKERS = [
  { id: "browser_tts", name: "Bluetooth" },
  { id: "media_player.byhome", name: "Apartment" },
  { id: "media_player.cat_wash", name: "Cat Wash" },
  { id: "media_player.cat_wr", name: "Cat Washroom Speakers" },
  { id: "media_player.echo_cat_left_am", name: "Cat Washroom Left" },
  { id: "media_player.echo_cat_right_am", name: "Cat Washroom Right" },
  { id: "media_player.echo_cat_washroom_middle", name: "Cat Washroom Middle" },
  { id: "media_player.echo_closet_am", name: "Closet" },
  { id: "media_player.echo_lr_couch_r_am", name: "Echo Corner" },
  { id: "media_player.echo_hallway_entrance_am", name: "Hallway Entrance" },
  { id: "media_player.echo_king_l_am", name: "King Left" },
  { id: "media_player.echo_king_r_am", name: "King Right" },
  { id: "media_player.echo_king_tv_am", name: "King TV" },
  { id: "media_player.echo_kitchen_cupboards_left_am", name: "Kitchen Cupboards Left" },
  { id: "media_player.echo_kitchen_cupboards_r_am", name: "Kitchen Cupboards Right" },
  { id: "media_player.echo_kitchen_fridge_am", name: "Kitchen Fridge" },
  { id: "media_player.echo_kitchen_hutch_am", name: "Kitchen Hutch" },
  { id: "media_player.echo_kitchen_island_corner_am", name: "Kitchen Island Corner" },
  { id: "media_player.echo_kitchen_studio_black_am", name: "Kitchen Studio Black" },
  { id: "media_player.echo_lr_couch_l_am", name: "Living Room Couch Left" },
  { id: "media_player.echo_lr_hub_am", name: "Living Room Hub" },
  { id: "media_player.echo_lr_studio_white_am", name: "Living Room Studio White" },
  { id: "media_player.echo_lr_tv_shelf_am", name: "Living Room TV Shelf" },
  { id: "media_player.echo_queen_balcony_am", name: "Queen Balcony" },
  { id: "media_player.echo_queen_bed_l_am", name: "Queen Bed Left" },
  { id: "media_player.echo_queen_bed_r_am", name: "Queen Bed Right" },
  { id: "media_player.echo_show_pug_am", name: "Echo Show Pug" },
  { id: "media_player.everywhere_2", name: "Everywhere" },
  { id: "media_player.hallway", name: "Hallway" },
  { id: "media_player.king_bedroom", name: "King Bedroom" },
  { id: "media_player.queen_bedroom", name: "Queen Bedroom" },
];

type Voice = "alloy" | "ash" | "echo" | "fable" | "onyx";

const VOICE_LABELS: Record<Voice, string> = {
  alloy: "Alloy - English (Neutral)",
  ash: "Ash - English (Male, Warm)",
  echo: "Echo - English (Male, Clear)",
  fable: "Fable - English (Male, British)",
  onyx: "Onyx - English (Male, Deep)",
};

export default function PDFReaderPage() {
  const [, params] = useRoute("/pdf-reader/:fileId");
  const [isOneDriveRoute] = useRoute("/pdf-reader/onedrive");
  const urlParams = new URLSearchParams(window.location.search);
  const queryFileId = urlParams.get("fileId") ? parseInt(urlParams.get("fileId")!) : null;
  const fileId = params?.fileId && params.fileId !== "onedrive" ? parseInt(params.fileId) : (queryFileId || null);

  const oneDriveUrl = urlParams.get("oneDriveUrl") || urlParams.get("url");
  const oneDriveName = urlParams.get("name");
  const filesParam = urlParams.get("files");
  const courseParam = urlParams.get("course");
  const autoplayParam = urlParams.get("autoplay") === "true" || urlParams.get("autoplay") === "1";
  const speakerParam = urlParams.get("speaker");
  const resumeChunkParam = urlParams.get("resumeChunk") ? parseInt(urlParams.get("resumeChunk")!) : null;
  const catWashFollow = urlParams.get("catWashFollow") === "true";
  const isInIframe = (() => { try { return window.self !== window.top; } catch { return true; } })();
  const followOnly = urlParams.get("followOnly") === "true" || (isInIframe && !urlParams.get("autoplay") && window.na...
  const voiceParam = urlParams.get("voice");
  const fullscreenParam = urlParams.get("fullscreen") === "true";
  const autoplayTriggeredRef = useRef(
    !autoplayParam ? false :
    catWashFollow ? false :
    sessionStorage.getItem(`autoplay_consumed_${fileId}_${resumeChunkParam}`) === 'true'
  );

  useEffect(() => {
    const myId = Date.now().toString() + Math.random().toString(36).slice(2);
    try { localStorage.setItem('pdf-reader-instance', myId); } catch {}
    const handleStorage = (e: StorageEvent) => {
      if (e.key === 'pdf-reader-instance' && e.newValue && e.newValue !== myId) {
        try { window.location.href = 'about:blank'; } catch {}
      }
    };
    window.addEventListener('storage', handleStorage);
    return () => window.removeEventListener('storage', handleStorage);
  }, []);

  useEffect(() => {
    if (!fullscreenParam && !catWashFollow && !autoplayParam) return;
    const enterFullscreen = () => {
      if (!document.fullscreenElement) {
        const el = document.documentElement as any;
        const req = el.requestFullscreen || el.webkitRequestFullscreen || el.mozRequestFull...
        if (req) req.call(el).catch(() => {});
      }
    };
    enterFullscreen();
    setTimeout(() => enterFullscreen(), 500);
    setTimeout(() => enterFullscreen(), 1500);
    setTimeout(() => {

```



```

    document.body.click();
    enterFullscreen();
}, 2000);
const onInteraction = () => {
    enterFullscreen();
};
document.addEventListener('touchstart', onInteraction);
document.addEventListener('click', onInteraction);
const recheckInterval = setInterval(() => {
    if (!document.fullscreenElement) enterFullscreen();
}, 2000);
let wakeLock: any = null;
const requestWakeLock = async () => {
    try {
        if ('wakeLock' in navigator) {
            wakeLock = await (navigator as any).wakeLock.request('screen');
        }
    } catch {}
};
requestWakeLock();
const handleVisChange = () => {
    if (!document.hidden) {
        enterFullscreen();
        requestWakeLock();
    }
};
document.addEventListener('visibilitychange', handleVisChange);
return () => {
    clearInterval(recheckInterval);
    document.removeEventListener('touchstart', onInteraction);
    document.removeEventListener('click', onInteraction);
    document.removeEventListener('visibilitychange', handleVisChange);
    if (wakeLock) wakeLock.release().catch(() => {});
};
}, [fullscreenParam, catWashFollow, autoplayParam]);

useEffect(() => {
    if (fileId || isOneDriveRoute || oneDriveUrl) return;
    Promise.all([
        fetch("/api/files").then(r => r.json()),
        fetch("/api/semester", { credentials: 'include' }).then(r => r.ok ? r.json() : null),
    ]).then(([files, semSettings]: [any[], any]) => {
        let weekNum = 1;
        if (semSettings?.semesterStartDate) {
            weekNum = getWeekNumber(new Date(), new Date(semSettings.semesterStartDate), semSettings.readingWeekStart);
        }
        const isPartial = (f: any) => {
            if (f.listened) return false;
            const hasChunks = (() => { if (!f.checkedChunks) return false; try { const a = JSON.parse(f.checkedChunks); r...
            const hasLast = f.lastChunkIndex != null && f.lastChunkIndex > 0;
            if (!hasChunks && !hasLast) return false;
            if (f.totalChunks && f.totalChunks > 0) {
                if (hasChunks) { try { if (JSON.parse(f.checkedChunks).length >= f.totalChunks) return false; } catch {} }
                if (hasLast && f.lastChunkIndex >= f.totalChunks) return false;
            }
            return true;
        };
        const partials = files.filter(isPartial);
        const weekFiles = files.filter((f: any) => {
            if (f.listened) return false;
            if (partials.some((p: any) => p.id === f.id)) return false;
            const m = f.folder?.match(/week-(\d+)/1);
            return m && parseInt(m[1], 10) === weekNum;
        });
        const best = [...partials, ...weekFiles][0];
        if (best) {
            window.location.replace(`/pdf-reader/${best.id}`);
        }
    }).catch(() => {});
}, []);

const [followState, setFollowState] = useState<{
    active: boolean;
    chunkIndex: number;
    totalChunks: number;
    chunkText: string;
    words: string[];
    estimatedWordIndex: number;
    progress: number;
    fileName: string;
} | null>(null);

// Parse files list for dropdown
const [allFiles, setAllFiles] = useState<Array<{name: string; downloadUrl: string; path: string}>>([]);
const [currentFileUrl, setCurrentFileUrl] = useState<string>(oneDriveUrl ? decodeURIComponent(oneDriveUrl) : '');
const [currentFileName, setCurrentFileName] = useState<string>(oneDriveName ? decodeURIComponent(oneDriveName) : '');
const [listenedFiles, setListenedFiles] = useState<Set<string>>(() => {
    const saved = localStorage.getItem('listenedOneDriveFiles');
    return saved ? new Set(JSON.parse(saved)) : new Set();
});

useEffect(() => {
    if (filesParam) {
        try {
            const parsed = JSON.parse(decodeURIComponent(filesParam));
            setAllFiles(parsed);
        } catch (e) {
            console.error('Failed to parse files param:', e);
        }
    }
}, [filesParam]);

const [numPages, setNumPages] = useState<number>(0);
const numPagesRef = useRef<number>(0);
const [currentPage, setCurrentPage] = useState<number>(1);
const [pdfContainerHeight, setPdfContainerHeight] = useState<number>(0);
const [extractedText, setExtractedText] = useState<string>("");
const extractedTextRef = useRef<string>("");

```

```

useEffect(() => { extractedTextRef.current = extractedText; }, [extractedText]);
const pdfUrlRef = useRef<string | undefined | null>(undefined);
const [isPlaying, setIsPlaying] = useState(false);
const [isPaused, setIsPaused] = useState(false);
const [isLoading, setIsLoading] = useState(false);
useEffect(() => {
  if (window.parent !== window) {
    window.parent.postMessage({ type: 'tts-state-update', isPlaying }, '*');
  }
}, [isPlaying]);

useEffect(() => {
  if (window.parent === window) return;
  const handler = (e: MessageEvent) => {
    if (!e.data?.type) return;
    if (e.data.type === 'tts-reset') {
      if (audioRef.current) { audioRef.current.pause(); audioRef.current.src = ''; }
      setIsPlaying(false);
      setIsPaused(false);
      setCurrentChunk(0);
      setCheckedChunks(new Set());
      const fileIdMatch = window.location.pathname.match(/\/pdf-reader\/(\d+)/);
      if (fileIdMatch) {
        const fid = fileIdMatch[1];
        fetch(`/api/files/${fid}`, { method: 'PATCH', headers: { 'Content-Type': 'application/json' }, body: JSON.s...
        localStorage.removeItem('checkedChunks_file_${fid}');
        localStorage.removeItem('tts-progress-${fid}');
      }
      setTimeout(() => {
        const el = document.querySelector(`[data-testid="chunk-row-0"]`);
        if (el) el.scrollIntoView({ behavior: 'smooth', block: 'center' });
      }, 100);
    }
  };
  window.addEventListener('message', handler);
  return () => window.removeEventListener('message', handler);
}, []);

const [isPreloading, setIsPreloading] = useState(false);
const [currentChunk, setCurrentChunk] = useState(0);
const [totalChunks, setTotalChunks] = useState(0);
const [voice, setVoice] = useState<Voice>(() => {
  if (voiceParam && ["alloy", "ash", "echo", "fable", "onyx"].includes(voiceParam)) return voiceParam as Voice;
  const saved = localStorage.getItem('pdf-reader-voice');
  return (saved && ["alloy", "ash", "echo", "fable", "onyx"].includes(saved) ? saved : "echo") as Voice;
});
const [isFullPage, setIsFullPage] = useState(false);
const [isFullScreen, setIsFullScreen] = useState(!document.fullscreenElement);
const [playbackSpeed, setPlaybackSpeed] = useState(1);
const [volume, setVolume] = useState(1);
const [isMobile, setIsMobile] = useState(false);
const [currentWordIndex, setCurrentWordIndex] = useState(0);
const [chunkWords, setChunkWords] = useState<string[]>([]);
const [checkedChunks, setCheckedChunks] = useState<Set<number>>(new Set());
const [chunksList, setChunksList] = useState<string[]>([]);
const [isEditingText, setIsEditingText] = useState(false);
const [catWashPaused, setCatWashPaused] = useState(false);
const [editableText, setEditableText] = useState("");
const [editingChunkIndex, setEditingChunkIndex] = useState<number | null>(null);
const [editableChunkText, setEditableChunkText] = useState("");
const [searchOpen, setSearchOpen] = useState(false);
const [searchQuery, setSearchQuery] = useState("");
const [searchMatchIndex, setSearchMatchIndex] = useState(0);
const [showReplace, setShowReplace] = useState(false);
const [replaceText, setReplaceText] = useState("");
const searchInputRef = useRef<HTMLInputElement>(null);
const ttsPreloadCache = useRef<Record<number, string>>({});
const voiceRef = useRef<string>(voice);
const [showFlickMenu, setShowFlickMenu] = useState(false);
const [flickDeviceGroups, setFlickDeviceGroups] = useState<Array<{ room: string; icon: string; devices: Array<{ id: s...
const [isFlicking, setIsFlicking] = useState(false);
const [selectedSpeaker, setSelectedSpeaker] = useState(speakerParam || "browser_tts");
const selectedSpeakerRef = useRef(selectedSpeaker);
const [volumeOverlay, setVolumeOverlay] = useState<{ volume: number; direction: string } | null>(null);
const volumeOverlayTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
const lastVolumeTimestampRef = useRef<number>(0);

const [ctrlFloating, setCtrlFloating] = useState<{ detached: boolean; minimized: boolean; x: number; y: number }>({...
  try {
    const saved = localStorage.getItem('pdfReaderCtrlFloating');
    if (saved) return JSON.parse(saved);
  } catch {}
  return { detached: false, minimized: false, x: 100, y: 100 };
});
const ctrlDragRef = useRef<{ startX: number; startY: number; origX: number; origY: number } | null>(null);

useEffect(() => {
  localStorage.setItem('pdfReaderCtrlFloating', JSON.stringify(ctrlFloating));
}, [ctrlFloating]);

useEffect(() => {
  const onMove = (e: MouseEvent | TouchEvent) => {
    const d = ctrlDragRef.current;
    if (!d) return;
    const clientX = 'touches' in e ? e.touches[0].clientX : e.clientX;
    const clientY = 'touches' in e ? e.touches[0].clientY : e.clientY;
    const newX = Math.max(0, Math.min(window.innerWidth - 200, d.origX + (clientX - d.startX)));
    const newY = Math.max(0, Math.min(window.innerHeight - 50, d.origY + (clientY - d.startY)));
    setCtrlFloating(prev => ({ ...prev, x: newX, y: newY }));
  };
  const onUp = () => { ctrlDragRef.current = null; };
  window.addEventListener('mousemove', onMove);
  window.addEventListener('mouseup', onUp);
  window.addEventListener('touchmove', onMove, { passive: false });
  window.addEventListener('touchend', onUp);
  return () => {
    window.removeEventListener('mousemove', onMove);
    window.removeEventListener('mouseup', onUp);
  };

```

```

        window.removeEventListener('touchmove', onMove);
        window.removeEventListener('touchend', onUp);
    };
}, []);

const ctrlDragStart = (e: React.MouseEvent | React.TouchEvent) => {
    const clientX = 'touches' in e ? e.touches[0].clientX : e.clientX;
    const clientY = 'touches' in e ? e.touches[0].clientY : e.clientY;
    ctrlDragRef.current = { startX: clientX, startY: clientY, origX: ctrlFloating.x, origY: ctrlFloating.y };
};

const audioRef = useRef<HTMLAudioElement | null>(null);
const chunksRef = useRef<string[]>([]);
const pdfDocRef = useRef<any>(null);
const isExtractingRef = useRef<boolean>(false);
const audioDurationRef = useRef<number>(0);
const currentChunkRef = useRef<number>(0);
const isPlayingRef = useRef<boolean>(false);
const isPausedRef = useRef<boolean>(false);
const playingAttentionPromptRef = useRef<boolean>(false);
const attentionPromptBlobUrlRef = useRef<string | null>(null);
const playbackSpeedRef = useRef<number>(1);
const volumeRef = useRef<number>(1);
const canvasRef = useRef<HTMLCanvasElement | null>(null);
const analyserRef = useRef<AnalyserNode | null>(null);
const audioContextRef = useRef<AudioContext | null>(null);
const sourceRef = useRef<MediaElementAudioSourceNode | null>(null);
const animFrameRef = useRef<number>(0);
const previewAudioRef = useRef<HTMLAudioElement | null>(null);
const [isPreviewing, setIsPreviewing] = useState(false);
const queryClient = useQueryClient();
const { toast } = useToast();

const previewVoice = async () => {
    if (isPreviewing) {
        if (previewAudioRef.current) { previewAudioRef.current.pause(); previewAudioRef.current = null; }
        setIsPreviewing(false);
        return;
    }
    setIsPreviewing(true);
    try {
        const res = await fetch('/api/tts', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ text: 'Hello, this is a preview of the selected voice.', voice }),
        });
        if (!res.ok) throw new Error('TTS failed!');
        const blob = await res.blob();
        const url = URL.createObjectURL(blob);
        const audio = new Audio(url);
        audio.volume = volumeRef.current;
        audio.playbackRate = playbackSpeedRef.current;
        previewAudioRef.current = audio;
        audio.onended = () => { setIsPreviewing(false); previewAudioRef.current = null; };
        await audio.play();
    } catch {
        setIsPreviewing(false);
        toast({ title: 'Preview failed', variant: 'destructive' });
    }
};

useEffect(() => {
    const checkMobile = () => setIsMobile(window.innerWidth < 640);
    checkMobile();
    window.addEventListener("resize", checkMobile);
    return () => window.removeEventListener("resize", checkMobile);
}, []);

useEffect(() => {
    const onFsChange = () => setIsFullScreen(!document.fullscreenElement);
    document.addEventListener("fullscreenchange", onFsChange);
    return () => document.removeEventListener("fullscreenchange", onFsChange);
}, []);

useEffect(() => {
    if (!audioRef.current) return;
    const audio = audioRef.current;

    const initVisualizer = () => {
        if (audioContextRef.current) return;
        try {
            const ctx = new AudioContext();
            const analyser = ctx.createAnalyser();
            analyser.fftSize = 256;
            analyser.smoothingTimeConstant = 0.8;
            const source = ctx.createMediaElementSource(audio);
            source.connect(analyser);
            analyser.connect(ctx.destination);
            audioContextRef.current = ctx;
            analyserRef.current = analyser;
            sourceRef.current = source;
        } catch (e) {
            console.warn('[Visualizer] Init failed:', e);
        }
    };

    let canvasInit = false;
    let cw = 0;
    let ch = 0;
    const barCount = 24;
    const gap = 2;

    const drawWaveform = () => {
        const canvas = canvasRef.current;
        const analyser = analyserRef.current;
        if (!canvas || !analyser) {
            animFrameRef.current = requestAnimationFrame(drawWaveform);
            return;
        }
    };

```

```

}

const canvasCtx = canvas.getContext('2d');
if (!canvasCtx) return;

if (!canvasInitd || canvas.offsetWidth !== cW || canvas.offsetHeight !== cH) {
  const dpr = window.devicePixelRatio || 1;
  cW = canvas.offsetWidth;
  cH = canvas.offsetHeight;
  canvas.width = cW * dpr;
  canvas.height = cH * dpr;
  canvasCtx.setTransform(dpr, 0, 0, dpr, 0, 0);
  canvasInitd = true;
}

const bufLen = analyser.frequencyBinCount;
const dataArray = new Uint8Array(bufLen);
analyser.getByteFrequencyData(dataArray);

canvasCtx.clearRect(0, 0, cW, cH);

const barWidth = Math.max(2, (cW - (barCount - 1) * gap) / barCount);
const centerY = cH / 2;

for (let i = 0; i < barCount; i++) {
  const dataIdx = Math.floor((i / barCount) * bufLen);
  const val = dataArray[dataIdx] / 255;
  const barH = Math.max(2, val * centerY * 0.85);
  const x = i * (barWidth + gap);
  const alpha = 0.3 + val * 0.7;
  const lightness = 70 + val * 30;

  canvasCtx.fillStyle = `hsla(200, 90%, ${Math.round(lightness)}%, ${alpha.toFixed(2)})`;
  canvasCtx.shadowColor = `hsla(200, 100%, 80%, ${val * 0.6}.toFixed(2))`;
  canvasCtx.shadowBlur = val * 6;

  canvasCtx.fillRect(Math.round(x), Math.round(centerY - barH), Math.round(barWidth), Math.round(barH));
  canvasCtx.fillRect(Math.round(x), Math.round(centerY + 1), Math.round(barWidth), Math.round(barH));

  canvasCtx.shadowBlur = 0;
}

animFrameRef.current = requestAnimationFrame(drawWaveform);
};

const onPlay = () => {
  initVisualizer();
  if (audioContextRef.current?.state === 'suspended') {
    audioContextRef.current.resume();
  }
  cancelAnimationFrame(animFrameRef.current);
  animFrameRef.current = requestAnimationFrame(drawWaveform);
};

const onPauseEnd = () => {
  setTimeout(() => {
    cancelAnimationFrame(animFrameRef.current);
    const canvas = canvasRef.current;
    if (canvas) {
      const ctx = canvas.getContext('2d');
      if (ctx) ctx.clearRect(0, 0, canvas.width, canvas.height);
    }
  }, 100);
};

audio.addEventListener('play', onPlay);
audio.addEventListener('pause', onPauseEnd);
audio.addEventListener('ended', onPauseEnd);

return () => {
  audio.removeEventListener('play', onPlay);
  audio.removeEventListener('pause', onPauseEnd);
  audio.removeEventListener('ended', onPauseEnd);
  cancelAnimationFrame(animFrameRef.current);
};
}, []);

useEffect(() => {
  fetch("/api/flick/rooms")
    .then(r => r.json())
    .then(setFlickDeviceGroups)
    .catch(() => {});
}, []);

const unlockAudioRef = useRef(false);
const audioUnlockedRef = useRef(false);
const unlockAudio = async () => {
  if (audioUnlockedRef.current) return;
  const tryUnlock = async () => {
    try {
      const AudioCtx = window.AudioContext || (window as any).webkitAudioContext;
      if (AudioCtx) {
        const ctx = new AudioCtx();
        const buf = ctx.createBuffer(1, 1, 22050);
        const src = ctx.createBufferSource();
        src.buffer = buf;
        src.connect(ctx.destination);
        src.start(0);
        if (ctx.state === 'suspended') await ctx.resume();
        console.log("[Autoplay] AudioContext state:", ctx.state);
      }
    }
    if (audioRef.current) {
      audioRef.current.muted = true;
      audioRef.current.src = "data:audio/wav;base64,UklGRiQAABXQVZFZm10IBAAAAAABAEARKwAAIhYACABAAZGF0YQAAAAA=";
      await audioRef.current.play();
      audioRef.current.pause();
      audioRef.current.muted = false;
      audioRef.current.src = "";
    }
  };
  tryUnlock();
};

```

```

        audioUnlockedRef.current = true;
        console.log("[Autoplay] Audio element unlocked");
    }
} catch (e) {
    console.log("[Autoplay] Audio unlock attempt:", e);
}
};
await tryUnlock();
if (!audioUnlockedRef.current) {
    document.dispatchEvent(new MouseEvent('click', { bubbles: true }));
    document.dispatchEvent(new TouchEvent('touchend', { bubbles: true }));
    await new Promise(r => setTimeout(r, 100));
    await tryUnlock();
}
};

useEffect(() => {
    if (!autoplayParam) return;
    const tapToUnlock = async () => {
        await unlockAudio();
        if (audioUnlockedRef.current) {
            document.removeEventListener('click', tapToUnlock);
            document.removeEventListener('touchend', tapToUnlock);
        }
    };
    document.addEventListener('click', tapToUnlock, { once: true });
    document.addEventListener('touchend', tapToUnlock, { once: true });

    const retryInterval = setInterval(async () => {
        if (audioUnlockedRef.current) { clearInterval(retryInterval); return; }
        await unlockAudio();
    }, 1000);
    setTimeout(() => clearInterval(retryInterval), 15000);
    return () => {
        clearInterval(retryInterval);
        document.removeEventListener('click', tapToUnlock);
        document.removeEventListener('touchend', tapToUnlock);
    };
}, [autoplayParam]);

useEffect(() => {
    if (!followOnly && !catWashFollow && !autoplayParam) return;
    console.log("[FollowOnly] Starting progress polling for display");
    const poll = async () => {
        try {
            const resp = await fetch("/api/cat-wash/progress");
            const data = await resp.json();
            if (data.active) {
                setFollowState({
                    active: true,
                    chunkIndex: data.chunkIndex || 0,
                    totalChunks: data.totalChunks || 1,
                    chunkText: data.chunkText || '',
                    words: data.words || [],
                    estimatedWordIndex: data.wordIndex || 0,
                    progress: 0,
                    fileName: data.fileName || '',
                });
                if (data.volumeChange && data.volumeChange.timestamp > lastVolumeTimestampRef.current) {
                    lastVolumeTimestampRef.current = data.volumeChange.timestamp;
                    setVolumeOverlay({ volume: data.volumeChange.volume, direction: data.volumeChange.direction });
                    if (volumeOverlayTimerRef.current) clearTimeout(volumeOverlayTimerRef.current);
                    volumeOverlayTimerRef.current = setTimeout(() => setVolumeOverlay(null), 2500);
                }
            } else {
                setFollowState(null);
            }
        } catch {}
    };
    poll();
    const interval = setInterval(poll, 500);
    return () => clearInterval(interval);
}, [followOnly, catWashFollow, autoplayParam]);

const lastFollowChunkRef = useRef<number>(-1);
useEffect(() => {
    if (!catWashFollow || !followOnly || !followState?.active) return;
    if (followState.chunkIndex !== lastFollowChunkRef.current) {
        lastFollowChunkRef.current = followState.chunkIndex;
        const el = document.querySelector(`[data-testid="chunk-row-${followState.chunkIndex}"]`);
        if (el) el.scrollIntoView({ behavior: 'smooth', block: 'center' });
    }
}, [catWashFollow, followOnly, followState?.active, followState?.chunkIndex]);

useEffect(() => {
    if (!catWashFollow || !followOnly || !followState?.active) return;
    const wordIdx = followState.estimatedWordIndex || 0;
    if (wordIdx > 0 && wordIdx % 5 === 0) {
        const wordEl = document.querySelector(`[data-word-index="${wordIdx}"]`);
        if (wordEl) wordEl.scrollIntoView({ behavior: 'smooth', block: 'center' });
    }
}, [followState?.estimatedWordIndex]);

useEffect(() => {
    if (!catWashFollow || !autoplayParam) return;
    const pollVolume = async () => {
        try {
            const resp = await fetch("/api/cat-wash/progress");
            const data = await resp.json();
            if (data.volumeChange && data.volumeChange.timestamp > lastVolumeTimestampRef.current) {
                lastVolumeTimestampRef.current = data.volumeChange.timestamp;
                setVolumeOverlay({ volume: data.volumeChange.volume, direction: data.volumeChange.direction });
                if (volumeOverlayTimerRef.current) clearTimeout(volumeOverlayTimerRef.current);
                volumeOverlayTimerRef.current = setTimeout(() => setVolumeOverlay(null), 2500);
            }
        } catch {}
    };
    const interval = setInterval(pollVolume, 800);

```

```

    return () => clearInterval(interval);
  }, [catWashFollow, autoplayParam]);

const lastNavTimestamp = useRef(() => {
  try { return Number(localStorage.getItem('lastNavTimestamp')) || '0'; } catch { return 0; }
});
useEffect(() => {
  const isFireDevice = /\bSilk\b/i.test(navigator.userAgent) || /\bKF[A-Z]{2,4}\b/.test(navigator.userAgent) || /\b...
  if (isFireDevice) return;
  const isFireTV = /\bFireTV\b/i.test(navigator.userAgent) || /\bAFT[A-Z]\b/.test(navigator.userAgent);
  const deviceRole = (() => { try { return localStorage.getItem('tabletDeviceRole'); } catch { return null; } })() ...
  const saved = lastNavTimestamp.current;
  lastNavTimestamp.current = typeof saved === 'function' ? saved() : saved;
  const interval = setInterval(async () => {
    try {
      const resp = await fetch(`/api/tablet-nav?device=${deviceRole}`);
      const data = await resp.json();
      if (data.timestamp <= lastNavTimestamp.current) return;
      if (Date.now() - data.timestamp > 120000) return;
      if (data.action === 'navigate' && data.url) {
        lastNavTimestamp.current = data.timestamp;
        try { localStorage.setItem('lastNavTimestamp', String(data.timestamp)); } catch {}
        fetch('/api/tablet-nav/ack', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON....
        window.location.href = data.url;
      } else if (data.action === 'stop_playback') {
        lastNavTimestamp.current = data.timestamp;
        try { localStorage.setItem('lastNavTimestamp', String(data.timestamp)); } catch {}
        fetch('/api/tablet-nav/ack', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON....
        console.log('[TabletNav] Received stop_playback command');
        if (audioRef.current) {
          audioRef.current.pause();
          audioRef.current.src = '';
        }
        isPlayingRef.current = false;
        setIsPlaying(false);
        if (data.goodbyeText) {
          try {
            const goodbyeAudio = new Audio();
            const ttsResp = await fetch('/api/tts', { method: 'POST', headers: { 'Content-Type': 'application/json'...
            if (ttsResp.ok) {
              const blob = await ttsResp.blob();
              const url = URL.createObjectURL(blob);
              goodbyeAudio.src = url;
              goodbyeAudio.volume = 1;
              goodbyeAudio.playbackRate = 1;
              console.log('[TabletNav] Playing goodbye TTS...');
              await goodbyeAudio.play();
              await new Promise<void>(resolve => {
                goodbyeAudio.onended = () => { URL.revokeObjectURL(url); resolve(); };
                setTimeout(() => { URL.revokeObjectURL(url); resolve(); }, 15000);
              });
              console.log('[TabletNav] Goodbye TTS finished');
            }
          } catch (e) {
            console.log('[TabletNav] Goodbye TTS failed:', e);
          }
        }
        if (!data.keepOpen) {
          window.location.href = '/';
        }
      } else if (data.action === 'go_home') {
        lastNavTimestamp.current = data.timestamp;
        try { localStorage.setItem('lastNavTimestamp', String(data.timestamp)); } catch {}
        fetch('/api/tablet-nav/ack', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON....
        window.location.href = '/';
      }
    } catch {}
  }, 3000);
  return () => clearInterval(interval);
}, []);

const handleFlick = async (deviceId: string) => {
  if (!fileId) {
    toast({ title: "Can't flick", description: "Flick only works with stored files, not OneDrive links." });
    return;
  }
  setIsFlicking(true);
  try {
    if (isPlaying && currentChunkRef.current > 0) {
      await fetch(`/api/files/${fileId}`, {
        method: 'PATCH',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          lastChunkIndex: currentChunkRef.current,
          totalChunks
        })
      });
    }
    if (audioRef.current) {
      audioRef.current.pause();
    }
    setIsPlaying(false);
    isPlayingRef.current = false;
    setIsPaused(false);
    isPausedRef.current = false;

    const resp = await fetch("/api/flick", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        deviceId,
        fileId,
        currentChunkIndex: currentChunkRef.current,
        totalChunks
      })
    });
    const data = await resp.json();
    if (data.success) {

```

```

        toast({
            title: `Flicked to ${data.device}`,
            description: data.canDisplay
                ? `Opening on ${data.device} in ${data.room}`
                : `Playing on ${data.device} in ${data.room}`
        });
    } else {
        toast({ title: "Flick failed", description: data.error || "Unknown error" });
    }
} catch (e: any) {
    toast({ title: "Flick failed", description: e.message });
} finally {
    setIsFlicking(false);
    setShowFlickMenu(false);
}
};

const [moduleFiles, setModuleFiles] = useState<FileRecord[]>([]);
const [readingFiles, setReadingFiles] = useState<FileRecord[]>([]);

const { data: file, isLoading: fileLoading } = useQuery<FileRecord>({
    queryKey: ["/api/files", fileId],
    queryFn: async () => {
        const response = await fetch(`/api/files/${fileId}`);
        if (!response.ok) throw new Error("Failed to fetch file");
        return response.json();
    },
    enabled: !!fileId,
});

useEffect(() => {
    if (file?.extractedText && !extractedText) {
        console.log(`[TTS] Loading pre-extracted text from DB: ${file.extractedText.length} chars`);
        setExtractedText(file.extractedText);
    }
}, [file?.extractedText]);

useEffect(() => {
    if (!file?.folder) return;
    const parts = file.folder.split('-');
    if (parts.length < 3) return;
    const weekNum = parts[1];
    const courseCode = parts[2];
    fetch('/api/files')
        .then(r => r.json())
        .then(allDbFiles: FileRecord[] => {
            const mods = allDbFiles.filter(f => f.folder?.includes(`week-${weekNum}-${courseCode}`)) && f.folder?.includes...
            const reads = allDbFiles.filter(f => f.folder?.includes(`week-${weekNum}-${courseCode}`)) && f.folder?.include...
            setModuleFiles(mods);
            setReadingFiles(reads);
        })
        .catch(() => {});
}, [file?.folder]);

const isOneDrive = isOneDriveRoute && oneDriveUrl;
const pdfUrl = isOneDrive
    ? (currentFileUrl || decodeURIComponent(oneDriveUrl))
    : fileId
    ? `/api/files/${fileId}/download`
    : file?.objectPath;
pdfUrlRef.current = pdfUrl;
const rawFileName = isOneDrive ? (currentFileName || (oneDriveName ? decodeURIComponent(oneDriveName) : "OneDrive P...
const fileName = (() => {
    if (!rawFileName) return rawFileName;
    let clean = rawFileName
        .replace(/^CPPA\s*122[-\s.]*\/i, '')
        .replace(/^CFNF\s*400[-\s.]*\/i, '')
        .replace(/^CASL\s*101[-\s.]*\/i, '')
        .replace(/^CSOC\s*103[-\s.]*\/i, '')
        .replace(/^CPHL\s*110[-\s.]*\/i, '')
        .replace(/^CASL\s*201[-\s.]*\/i, '')
        .replace(/Reading\s*d*[-\s.]*\/gi, '')
        .replace(/\.pdf$/i, '')
        .trim();
    while (clean.match(/^[.\s_-:~]*$/)) {
        clean = clean.replace(/^[.\s_-:~]*$/, '').trim();
    }
    return clean || rawFileName;
})();

// Mark current file as listened when playing
const markCurrentFileListened = () => {
    if (currentFileUrl || oneDriveUrl) {
        const fileKey = currentFileUrl || (oneDriveUrl ? decodeURIComponent(oneDriveUrl) : '');
        const currentFile = allFiles.find(f => f.downloadUrl === fileKey);
        if (currentFile?.path) {
            const newListened = new Set(listenedFiles);
            newListened.add(currentFile.path);
            setListenedFiles(newListened);
            localStorage.setItem('listenedOneDriveFiles', JSON.stringify(Array.from(newListened)));
        }
    }
};

const getFileKey = () => {
    if (fileId) return `file_${fileId}`;
    const url = currentFileUrl || (oneDriveUrl ? decodeURIComponent(oneDriveUrl) : '');
    return `onedrive_${btoa(url).slice(0, 40)}`;
};

const beacon = (step: string, data?: any) => {
    if (!catWashFollow) return;
    fetch("/api/debug-beacon", { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringif...
});

const catWashChunksLoadedRef = useRef(false);
useEffect(() => {
    if (catWashFollow && !autoplayParam && !catWashChunksLoadedRef.current && pdfUrl && numPages > 0) {

```

```

catWashChunksLoadedRef.current = true;
beacon("catWashFollow-loading-chunks", { fileId, numPages });
const loadChunks = async () => {
  let textToRead = extractedTextRef.current || extractedText;
  if (!textToRead) {
    textToRead = await extractAllText();
  }
  if (!textToRead && fileId) {
    try {
      const resp = await fetch(`/api/files/${fileId}/text`);
      if (resp.ok) {
        const data = await resp.json();
        if (data.text) {
          textToRead = data.text;
          setExtractedText(data.text);
        }
      }
    } catch {}
  }
  if (textToRead) {
    const newChunks = chunkText(textToRead);
    chunksRef.current = newChunks;
    setChunksList(newChunks);
    setTotalChunks(newChunks.length);
    beacon("catWashFollow-chunks-loaded", { numChunks: newChunks.length });
    const key = getFileKey();
    let serverChecked = new Set<number>();
    if (file?.checkedChunks) {
      try { const arr = JSON.parse(file.checkedChunks); if (Array.isArray(arr)) serverChecked = new Set(arr); }...
    }
    if (serverChecked.size > 0) {
      setCheckedChunks(serverChecked);
      saveCheckedChunks(key, serverChecked, newChunks.length);
    }
    if (resumeChunkParam !== null && resumeChunkParam < newChunks.length) {
      setCurrentChunk(resumeChunkParam);
      currentChunkRef.current = resumeChunkParam;
      setTimeout(() => {
        const el = document.querySelector(`[data-testid="chunk-row-${resumeChunkParam}"]`);
        if (el) el.scrollIntoView({ behavior: 'smooth', block: 'center' });
      }, 300);
    }
  }
};
loadChunks();
}, [catWashFollow, autoplayParam, pdfUrl, numPages]);

useEffect(() => {
  if (autoplayParam && !autoplayTriggeredRef.current && pdfUrl && numPages > 0) {
    autoplayTriggeredRef.current = true;
    beacon("autoplay-triggered", { catWashFollow, numPages, pdfUrl: !!pdfUrl, fileId });
    const key = `autoplay_consumed_${fileId}_${resumeChunkParam}`;
    try { sessionStorage.setItem(key, 'true'); } catch {}
    const doAutoplay = async () => {
      beacon("autoplay-unlocking-audio");
      await unlockAudio();
      beacon("autoplay-calling-startReading");
      startReading();
    };
    doAutoplay();
  }
}, [autoplayParam, pdfUrl, numPages]);

const loadCheckedChunks = (key: string): Set<number> => {
  const saved = localStorage.getItem(`checkedChunks_${key}`);
  return saved ? new Set(JSON.parse(saved)) : new Set();
};

const saveCheckedChunks = (key: string, checked: Set<number>, total: number) => {
  localStorage.setItem(`checkedChunks_${key}`, JSON.stringify(Array.from(checked)));
  localStorage.setItem(`chunkProgress_${key}`, JSON.stringify({ checked: checked.size, total }));
  const allProgress = JSON.parse(localStorage.getItem('allChunkProgress') || '{}');
  allProgress[key] = { checked: checked.size, total };
  localStorage.setItem('allChunkProgress', JSON.stringify(allProgress));
};

const toggleChunkChecked = (idx: number) => {
  const key = getFileKey();
  const newChecked = new Set(checkedChunks);
  if (newChecked.has(idx)) {
    newChecked.delete(idx);
  } else {
    for (let i = 0; i <= idx; i++) {
      newChecked.add(i);
    }
  }
  setCheckedChunks(newChecked);
  saveCheckedChunks(key, newChecked, totalChunks);
  if (fileId) {
    fetch(`/api/files/${fileId}`, {
      method: 'PATCH',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        checkedChunks: JSON.stringify(Array.from(newChecked)),
        totalChunks,
        lastChunkIndex: Math.max(...Array.from(newChecked), 0),
      }),
    }).then(() => {
      queryClient.invalidateQueries({ queryKey: ['/api/files'] });
    }).catch(() => {});
  }
};

const jumpToUnlisted = () => {
  const firstUnchecked = chunksList.findIndex((_, idx) => !checkedChunks.has(idx));
  if (firstUnchecked >= 0) {
    const el = document.querySelector(`[data-testid="chunk-row-${firstUnchecked}"]`);
  }
};

```



```

    if (el) el.scrollIntoView({ behavior: 'smooth', block: 'center' });
  }
};

const chunkProgress = totalChunks > 0 ? Math.round((checkedChunks.size / totalChunks) * 100) : 0;

const searchMatchCount = (() => {
  if (searchQuery.trim().length < 2) return 0;
  const q = searchQuery.toLowerCase().trim();
  let count = 0;
  for (const chunk of chunksList) {
    let pos = 0;
    const lower = chunk.toLowerCase();
    while ((pos = lower.indexOf(q, pos)) !== -1) { count++; pos += 1; }
  }
  return count;
})();

// Switch to a different file
const switchToFile = (file: { name: string; downloadUrl: string; path: string }) => {
  setCurrentFileUrl(file.downloadUrl);
  setCurrentFileName(file.name);
  setCurrentPage(1);
  setNumPages(0);
  numPagesRef.current = 0;
  setExtractedText('');
  setIsPlaying(false);
  setIsPaused(false);
  setIsPreloading(false);
  setTotalChunks(0);
  setCheckedChunks(new Set());
  setChunksList([]);
  chunksRef.current = [];
  pdfDocRef.current = null;
  isExtractingRef.current = false;
  if (audioRef.current) {
    audioRef.current.pause();
    audioRef.current.removeAttribute('src');
    audioRef.current.load();
  }
};

const onDocumentLoadSuccess = async ({ numPages: pages }: { numPages: number }) => {
  setNumPages(pages);
  numPagesRef.current = pages;
  setCurrentPage(1);

  const key = getFileKey();
  const TTS_CACHE_VERSION = "v2";
  const savedText = localStorage.getItem(`tts_edited_${TTS_CACHE_VERSION}_${key}`);
  if (savedText) {
    setExtractedText(savedText);
    console.log("Loaded saved TTS text for", key, savedText.length, "chars");
    return;
  }
  const oldSaved = localStorage.getItem(`tts_edited_${key}`);
  if (oldSaved) {
    localStorage.removeItem(`tts_edited_${key}`);
    console.log("Removed stale unfiltered TTS cache for", key);
  }

  if (pdfUrl && pages > 0 && !extractedText && !isExtractingRef.current) {
    isExtractingRef.current = true;
    extractTextInBackground(pdfUrl, pages);
  }
};

// Extract text in background without blocking UI
const cleanTextViaServer = async (rawText: string): Promise<string> => {
  try {
    const response = await fetch("/api/tts/clean-text", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ text: rawText }),
    });
    if (response.ok) {
      const data = await response.json();
      console.log("[TTS] Server cleaned text: ${rawText.length} -> ${data.text.length} chars");
      return data.text;
    }
  } catch (err) {
    console.error("[TTS] Server text cleaning failed, using raw:", err);
  }
  return rawText;
};

const extractPageTextWithParagraphs = async (pdf: any, pageNum: number): Promise<string> => {
  const page = await pdf.getPage(pageNum);
  const textContent = await page.getTextContent();
  const items = textContent.items as any[];
  if (items.length === 0) return "";

  let result = "";
  let lastY: number | null = null;
  let lastHeight = 12;

  for (const item of items) {
    if (!item.str || item.str.trim() === "") continue;
    const y = item.transform ? item.transform[5] : null;
    const height = item.height || lastHeight;

    if (lastY !== null && y !== null) {
      const gap = Math.abs(lastY - y);
      if (gap > height * 1.3) {
        result += "\n\n";
      } else if (gap > height * 0.3) {
        result += " ";
      } else {
        result += "\n";
      }
    }
    result += item.str;
    lastY = y;
    lastHeight = height;
  }
  return result;
};

```

```

        result += " ";
    }
}

result += item.str;
if (y !== null) lastY = y;
if (height > 0) lastHeight = height;
}
return result;
};

const extractTextInBackground = async (url: string, pages: number) => {
    setIsPreloading(true);
    try {
        const loadingTask = pdfjs.getDocument(url);
        const pdf = await loadingTask.promise;
        pdfDocRef.current = pdf;

        let fullText = "";
        for (let i = 1; i <= pages; i++) {
            const pageText = await extractPageTextWithParagraphs(pdf, i);
            fullText += pageText + "\n\n";
        }

        const cleanedText = await cleanTextViaServer(fullText);
        setExtractedText(cleanedText);
        console.log("PDF text pre-extracted and cleaned:", cleanedText.length, "chars");
        if (chunksRef.current.length === 0) {
            const preChunks = chunkText(cleanedText);
            chunksRef.current = preChunks;
            setChunksList(preChunks);
            setTotalChunks(preChunks.length);
            const key = getFileKey();
            let finalChecked = new Set<number>();
            if (!skipCheckedReloadRef.current) {
                let serverChecked = new Set<number>();
                if (file?.checkedChunks) {
                    try { const arr = JSON.parse(file.checkedChunks); if (Array.isArray(arr)) serverChecked = new Set(arr); }...
                }
                finalChecked = serverChecked.size > 0 ? serverChecked : new Set<number>();
                setCheckedChunks(finalChecked);
                if (finalChecked.size > 0) saveCheckedChunks(key, finalChecked, preChunks.length);
            }
            console.log("Pre-populated chunks:", preChunks.length);

            const firstUnlisted = preChunks.findIndex((_, idx) => !finalChecked.has(idx));
            const preloadIdx = firstUnlisted >= 0 ? firstUnlisted : 0;
            if (preChunks[preloadIdx] && !ttsPreloadCache.current[preloadIdx]) {
                console.log(`[TTS] Pre-fetching audio for chunk ${preloadIdx + 1}...`);
                fetch("/api/tts", {
                    method: "POST",
                    headers: { "Content-Type": "application/json" },
                    body: JSON.stringify({ text: preChunks[preloadIdx], voice: voiceRef.current || 'echo' }),
                }).then(r => r.ok ? r.blob() : null).then(blob => {
                    if (blob) {
                        ttsPreloadCache.current[preloadIdx] = URL.createObjectURL(blob);
                        console.log(`[TTS] Pre-fetched chunk ${preloadIdx + 1} audio ready`);
                    }
                }).catch(() => {});
            }
        }
    } catch (error) {
        console.error("Background text extraction failed:", error);
    } finally {
        isExtractingRef.current = false;
        setIsPreloading(false);
    }
};

const extractAllText = async (): Promise<string> => {
    const currentNumPages = numPagesRef.current || numPages;
    if (!pdfUrl || currentNumPages === 0) return "";

    if (extractedTextRef.current) return extractedTextRef.current;
    if (extractedText) return extractedText;

    if (isExtractingRef.current) {
        setIsLoading(true);
        while (isExtractingRef.current) {
            await new Promise(r => setTimeout(r, 100));
            if (extractedTextRef.current) {
                setIsLoading(false);
                return extractedTextRef.current;
            }
        }
        setIsLoading(false);
        return extractedTextRef.current || extractedText || "";
    }

    setIsLoading(true);
    let fullText = "";

    try {
        let pdf = pdfDocRef.current;
        if (!pdf) {
            const loadingTask = pdfjs.getDocument(pdfUrl);
            pdf = await loadingTask.promise;
            pdfDocRef.current = pdf;
        }

        for (let i = 1; i <= currentNumPages; i++) {
            const pageText = await extractPageTextWithParagraphs(pdf, i);
            fullText += pageText + "\n\n";
        }

        const cleanedText = await cleanTextViaServer(fullText);
        setExtractedText(cleanedText);
        return cleanedText;
    }

```

```

} catch (error) {
  console.error("Error extracting text:", error);
  toast({
    title: "Error",
    description: "Failed to extract text from PDF",
    variant: "destructive",
  });
  return "";
} finally {
  setIsLoading(false);
}
};

const chunkText = (text: string, maxLength: number = 1000): string[] => {
  const paragraphs = text.split(/\n\s*\n/).map(p => p.trim()).filter(p => p.length > 0);

  if (paragraphs.length <= 1) {
    const sentences = text.match(/^[^!?]+[.!?]+/g) || [text];
    const chunks: string[] = [];
    let currentChunk = "";
    for (const sentence of sentences) {
      if ((currentChunk + sentence).length > maxLength) {
        if (currentChunk) chunks.push(currentChunk.trim());
        currentChunk = sentence;
      } else {
        currentChunk += sentence;
      }
    }
    if (currentChunk) chunks.push(currentChunk.trim());
    return chunks;
  }

  const chunks: string[] = [];
  let currentChunk = "";

  for (const para of paragraphs) {
    if (para.length > maxLength) {
      if (currentChunk) { chunks.push(currentChunk.trim()); currentChunk = ""; }
      const sentences = para.match(/^[^!?]+[.!?]+/g) || [para];
      let subChunk = "";
      for (const sentence of sentences) {
        if ((subChunk + sentence).length > maxLength) {
          if (subChunk) chunks.push(subChunk.trim());
          subChunk = sentence;
        } else {
          subChunk += sentence;
        }
      }
      if (subChunk) chunks.push(subChunk.trim());
    } else if ((currentChunk + "\n\n" + para).length > maxLength) {
      if (currentChunk) chunks.push(currentChunk.trim());
      currentChunk = para;
    } else {
      currentChunk = currentChunk ? currentChunk + "\n\n" + para : para;
    }
  }
  if (currentChunk) chunks.push(currentChunk.trim());
  return chunks;
};

const speakerTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);

const playTTS = async (text: string, retryCount = 0): Promise<boolean> => {
  beacon("playTTS-enter", { textLen: text.length || 0, retry: retryCount, isPlaying: isPlayingRef.current });
  if (!isPlayingRef.current) {
    beacon("playTTS-aborted-not-playing");
    console.log('[TTS] playTTS aborted – not playing');
    return false;
  }
  try {
    const words = text.split(/\s+/).filter(w => w.length > 0);
    setChunkWords(words);
    setCurrentWordIndex(0);

    if (speakerParam) {
      console.log(`[TTS] Playing on external speaker: ${speakerParam}`);
      const response = await fetch("/api/tts/speaker", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ text, voice, entityId: speakerParam }),
      });
      if (!response.ok) throw new Error(`Speaker TTS failed: ${response.status}`);
      if (!isPlayingRef.current) return false;
      const estimatedDuration = Math.max(10, (words.length / 2.5));
      if (speakerTimerRef.current) clearTimeout(speakerTimerRef.current);
      speakerTimerRef.current = setTimeout(() => {
        if (isPlayingRef.current && audioRef.current) {
          audioRef.current.dispatchEvent(new Event('ended'));
        }
      }, estimatedDuration * 1000);
      return true;
    }

    const chunkIdx = currentChunkRef.current;
    const cachedUrl = ttsPreloadCache.current[chunkIdx];
    let audioUrl: string;

    if (cachedUrl) {
      console.log(`[TTS] Using preloaded audio for chunk ${chunkIdx + 1}`);
      audioUrl = cachedUrl;
      delete ttsPreloadCache.current[chunkIdx];
    } else {
      console.log(`[TTS] Fetching audio for ${words.length} words, voice=${voice}`);
      const response = await fetch("/api/tts", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ text, voice }),
      });
    }
  }

```

```

    if (!isPlayingRef.current) {
      console.log('[TTS] Stopped during fetch - aborting playback');
      return false;
    }

    if (!response.ok) {
      const errText = await response.text();
      console.error('[TTS] Request failed: ${response.status} ${errText}');
      throw new Error('TTS request failed: ${response.status}');
    }

    const audioBlob = await response.blob();
    if (!isPlayingRef.current) return false;
    beacon("playTTS-blob-received", { size: audioBlob.size });
    console.log('[TTS] Audio blob received: ${audioBlob.size} bytes');
    audioUrl = URL.createObjectURL(audioBlob);
  }

  if (audioRef.current) {
    audioRef.current.src = audioUrl;
    audioRef.current.volume = volumeRef.current;
    audioRef.current.playbackRate = playbackSpeedRef.current;

    audioRef.current.onloadedmetadata = () => {
      if (audioRef.current) {
        audioDurationRef.current = audioRef.current.duration;
        audioRef.current.playbackRate = playbackSpeedRef.current;
        audioRef.current.volume = volumeRef.current;
        console.log('[TTS] Metadata: dur=${audioRef.current.duration}s, speed=${playbackSpeedRef.current}x, vol=$...');
      }
    };

    if (!isPlayingRef.current) {
      console.log('[TTS] Stopped before play - aborting');
      return false;
    }
    beacon("playTTS-calling-play");
    if (audioContextRef.current?.state === 'suspended') {
      await audioContextRef.current.resume();
      console.log('[TTS] Resumed visualizer AudioContext');
    }

    audioRef.current.onerror = (e) => {
      console.error('[TTS] Audio element error:', e, audioRef.current?.error);
    };

    await audioRef.current.play();
    audioRef.current.playbackRate = playbackSpeedRef.current;
    audioRef.current.volume = volumeRef.current;
    beacon("playTTS-playing-success");
    console.log('[TTS] Playing: speed=${audioRef.current.playbackRate}, vol=${audioRef.current.volume}');

    await new Promise<void>((resolve) => {
      if (!audioRef.current) { resolve(); return; }
      const onEnd = () => {
        audioRef.current?.removeEventListener('ended', onEnd);
        resolve();
      };
      audioRef.current.addEventListener('ended', onEnd);
    }));
  }
  return true;
} catch (error: any) {
  beacon("playTTS-ERROR", { msg: error?.message || String(error), retry: retryCount });
  if (!isPlayingRef.current) return false;
  console.error('[TTS] Error (attempt ${retryCount + 1}):', error);
  if (retryCount < 5 && isPlayingRef.current) {
    const delay = retryCount < 2 ? 2000 : 3000;
    console.log('[TTS] Retrying in ${delay/1000}s (attempt ${retryCount + 2}/6)...');
    await new Promise(resolve => setTimeout(resolve, delay));
    if (!isPlayingRef.current) return false;
    audioUnlockedRef.current = false;
    await unlockAudio();
    return playTTS(text, retryCount + 1);
  }
  return false;
}

const lastReportedWordRef = useRef(-1);
const highlightRafRef = useRef<number | null>(null);
const pollWordHighlight = () => {
  handleTimeUpdate();
  highlightRafRef.current = requestAnimationFrame(pollWordHighlight);
};

useEffect(() => {
  if (isPlaying && !isPaused) {
    highlightRafRef.current = requestAnimationFrame(pollWordHighlight);
  } else {
    if (highlightRafRef.current) { cancelAnimationFrame(highlightRafRef.current); highlightRafRef.current = null; }
  }
  return () => { if (highlightRafRef.current) { cancelAnimationFrame(highlightRafRef.current); highlightRafRef.curr... }
}, [isPlaying, isPaused]);
const wordWeightsRef = useRef<number[]>([]);
const wordCumulativeRef = useRef<number[]>([]);

useEffect(() => {
  if (chunkWords.length === 0) return;
  const weights = chunkWords.map((w: string) => {
    const base = Math.max(w.replace(/^[^a-zA-Z]/g, '').length, 1);
    const hasPause = /[.!?,:;]$/g.test(w) ? 2.5 : /[.,]$/g.test(w) ? 1.3 : 0;
    return base + hasPause;
  });
  const total = weights.reduce((a: number, b: number) => a + b, 0);
  const cumulative: number[] = [];
  let sum = 0;
  for (const w of weights) {

```

```

    sum += w / total;
    cumulative.push(sum);
  }
  wordWeightsRef.current = weights;
  wordCumulativeRef.current = cumulative;
}, [chunkWords]);

const handleTimeUpdate = () => {
  if (!audioRef.current || chunkWords.length === 0 || audioDurationRef.current === 0) return;

  var currentTime = audioRef.current.currentTime;
  var duration = audioDurationRef.current;
  var progress = Math.max(0, (currentTime / duration) - (1 / 195));

  var estimatedWordIndex: number;
  const cumulative = wordCumulativeRef.current;
  if (cumulative.length === chunkWords.length) {
    estimatedWordIndex = cumulative.findIndex((c: number) => c >= progress);
    if (estimatedWordIndex === -1) estimatedWordIndex = chunkWords.length - 1;
  } else {
    estimatedWordIndex = Math.floor(progress * chunkWords.length);
  }
  var clampedIndex = Math.min(estimatedWordIndex, chunkWords.length - 1);

  if (clampedIndex !== currentWordIndex) {
    setCurrentWordIndex(clampedIndex);
    if (clampedIndex % 3 === 0 || clampedIndex === chunkWords.length - 1) {
      const activeWord = document.querySelector(`[data-word-index="${clampedIndex}"]`);
      if (activeWord) activeWord.scrollIntoView({ behavior: 'smooth', block: 'center' });
    }
  }
  if (catWashFollow && fileId && clampedIndex !== lastReportedWordRef.current) {
    lastReportedWordRef.current = clampedIndex;
    const currentChunkWords = chunksRef.current[currentChunkRef.current]?.split(/\s+/).filter((w: string) => w.le...
    fetch("/api/cat-wash/update-progress", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ fileId, chunkIndex: currentChunkRef.current, totalChunks: chunksRef.current.length, ...
    }).catch(() => {});
  }
}
};

const skipCheckedReloadRef = useRef(false);
const resetStartRef = useRef(false);
const startReading = async () => {
  beacon("startReading-begin", { fileId });
  await unlockAudio();
  markCurrentFileListened();

  let textToRead = extractedTextRef.current || extractedText;
  beacon("startReading-cached-text", { hasText: !!textToRead, len: textToRead?.length || 0 });

  if (!textToRead) {
    beacon("startReading-extracting");
    textToRead = await extractAllText();
    beacon("startReading-extracted", { len: textToRead?.length || 0 });
  }

  if (!textToRead && fileId) {
    beacon("startReading-server-fallback");
    console.log("[TTS] Client-side extraction failed, trying server-side extraction for file", fileId);
    try {
      const resp = await fetch(`/api/files/${fileId}/text`);
      if (resp.ok) {
        const data = await resp.json();
        if (data.text) {
          textToRead = data.text;
          setExtractedText(data.text);
          console.log(`[TTS] Server-side extraction succeeded: ${data.text.length} chars`);
        }
      }
    } catch (err) {
      console.error("[TTS] Server-side extraction also failed:", err);
    }
  }

  if (!textToRead) {
    beacon("startReading-NO-TEXT");
    toast({
      title: "No text found",
      description: "Could not extract text from this PDF. Make sure the PDF is loaded.",
      variant: "destructive",
    });
    return;
  }

  const newChunks = chunkText(textToRead);
  beacon("startReading-chunked", { numChunks: newChunks.length, resumeChunkParam });
  chunksRef.current = newChunks;
  setChunksList(newChunks);
  setTotalChunks(newChunks.length);
  const key = getFileKey();
  if (skipCheckedReloadRef.current) {
    skipCheckedReloadRef.current = false;
  } else {
    let serverChecked = new Set<number>();
    if (file?.checkedChunks) {
      try { const arr = JSON.parse(file.checkedChunks); if (Array.isArray(arr)) serverChecked = new Set(arr); } cat...
    }
    const finalChecked = serverChecked.size > 0 ? serverChecked : new Set<number>();
    setCheckedChunks(finalChecked);
    if (finalChecked.size > 0) saveCheckedChunks(key, finalChecked, newChunks.length);
  }
  const mergedForStart = resetStartRef.current ? new Set<number>() : checkedChunks;
  resetStartRef.current = false;
  let startChunk = (resumeChunkParam !== null && resumeChunkParam < newChunks.length) ? resumeChunkParam : 0;
  if (mergedForStart.size > 0 && startChunk === 0) {
    const firstUnchecked = newChunks.findIndex((_, idx) => !mergedForStart.has(idx));

```

```

    if (firstUnchecked >= 0) {
      startChunk = firstUnchecked;
      console.log(`[TTS] Skipping ${mergedForStart.size} checked chunks, starting at chunk ${startChunk}`);
    }
  }
  if (startChunk > 0) {
    console.log(`[TTS] Resuming from chunk ${startChunk}`);
  }
  setCurrentChunk(startChunk);
  setIsPlaying(true);
  isPlayingRef.current = true;
  setIsPaused(false);
  isPausedRef.current = false;

  if (!attentionPromptBlobUrlRef.current && !catWashFollow && !speakerParam) {
    fetch("/api/tts", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ text: "Bryn, are you paying attention?", voice }),
    }).then(r => r.ok ? r.blob() : null).then(blob => {
      if (blob) {
        attentionPromptBlobUrlRef.current = URL.createObjectURL(blob);
        console.log(`[TTS] Pre-cached attention prompt (${blob.size} bytes)`);
      }
    }).catch(() => {});
  }
}

beacon("startReading-calling-playNextChunk", { startChunk, isPlaying: isPlayingRef.current, checkedCount: mergedC...

if (catWashFollow && file) {
  const hour = new Date().getHours();
  const greeting = hour < 12 ? "Good morning" : hour < 17 ? "Good afternoon" : "Good evening";
  const folder = file.folder || '';
  const weekMatch = folder.match(/week-(\d+)/i);
  const weekNum = weekMatch ? weekMatch[1] : null;
  const isModule = folder.toLowerCase().includes('module');
  const courseMatch = folder.match(/(?:cpga|cfnf|casl|csoc|cphl)\d*/i);
  const courseCode = courseMatch ? courseMatch[0].toUpperCase() : '';
  const raw = file.displayName || file.originalName || '';
  const cleanName = raw.replace(/\.pdf$/i, '').replace(/,\\s*(Module|Reading)\\s*\d+[-_\\s]*/i, ' ').replace(/[-_...

  let introText: string;
  if (isModule && weekNum) {
    introText = `${greeting} Bryn. I will now read the ${courseCode} module for week ${weekNum}.`;
  } else if (weekNum) {
    introText = `${greeting} Bryn. I will now read the ${courseCode} reading file: ${cleanName}.`;
  } else {
    introText = `${greeting} Bryn. I will now read: ${cleanName}.`;
  }

  if (startChunk > 0) {
    introText += ` Resuming from chunk ${startChunk + 1}.`;
  }

  console.log(`[TTS] Playing intro: ${introText}`);
  await playTTS(introText);
  if (!isPlayingRef.current) return;
}

playNextChunk(startChunk);
};

const markChunkChecked = (idx: number) => {
  const key = getFileKey();
  setCheckedChunks(prev => {
    if (prev.has(idx)) return prev;
    const newChecked = new Set(prev);
    newChecked.add(idx);
    saveCheckedChunks(key, newChecked, totalChunks);
    if (fileId) {
      fetch(`/api/files/${fileId}`, {
        method: 'PATCH',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          checkedChunks: JSON.stringify(Array.from(newChecked)),
          totalChunks,
          lastChunkIndex: Math.max(...Array.from(newChecked), 0),
        }),
      }).then(() => {
        queryClient.invalidateQueries({ queryKey: ['/api/files'] });
      }).catch(() => {});
    }
  })
  return newChecked;
};

const playNextChunk = async (index: number) => {
  beacon("playNextChunk-enter", { index, chunksLen: chunksRef.current.length, isPlaying: isPlayingRef.current });
  if (index > 0) {
    markChunkChecked(index - 1);
  }
  if (!isPlayingRef.current) {
    beacon("playNextChunk-aborted-not-playing");
    console.log(`[TTS] playNextChunk aborted - not playing`);
    return;
  }
  console.log(`[TTS] playNextChunk called: index=${index}, totalChunks=${chunksRef.current.length}`);
  if (index >= chunksRef.current.length) {
    console.log(`[TTS] All chunks finished`);
    setIsPlaying(false);
    isPlayingRef.current = false;
    setCurrentChunk(0);
    currentChunkRef.current = 0;

    if (catWashFollow && fileId) {
      try {
        const resp = await fetch("/api/cat-wash/update-progress", {
          method: "POST",

```

```

        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ fileId, completed: true })),
    });
    const data = await resp.json();
    if (data.nextFile) {
        console.log(`[Cat Wash] Auto-continuing to: ${data.nextFile.name}`);
        toast({ title: "Next reading", description: `Loading: ${data.nextFile.name}` });
        window.location.href = data.nextFile.readerUrl;
        return;
    } else if (data.allComplete) {
        toast({ title: "All done!", description: "All readings for this week are complete." });
    }
} catch (e) {
    console.error("[Cat Wash] Failed to report completion:", e);
}
} else {
    if (fileId) {
        try {
            await fetch(`/api/files/${fileId}`, {
                method: 'PATCH',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ listened: true, lastChunkIndex: chunksRef.current.length, totalChunks: chunksRef...
            });
            queryClient.invalidateQueries({ queryKey: ['/api/files'] });
        } catch (e) {
            console.error('Failed to mark file as listened:', e);
        }
    }
    toast({ title: "Finished", description: "Finished reading the document" });
}
return;
}

if (catWashFollow && fileId) {
    const words = chunksRef.current[index]?.split(/\s+/).filter((w: string) => w.length > 0) || [];
    fetch("/api/cat-wash/update-progress", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ fileId, chunkIndex: index, totalChunks: chunksRef.current.length, words, chunkText: ch...
    }).catch(() => {});
}

setCurrentChunk(index);
currentChunkRef.current = index;
playingAttentionPromptRef.current = false;
console.log(`[TTS] Playing chunk ${index + 1}/${chunksRef.current.length}`);

const scrollToChunk = () => {
    const chunkRow = document.querySelector(`[data-testid="chunk-row-${index}]`);
    if (chunkRow) chunkRow.scrollIntoView({ behavior: 'smooth', block: 'center' });
};
setTimeout(scrollToChunk, 100);
setTimeout(scrollToChunk, 400);
setTimeout(scrollToChunk, 800);

if (index + 1 < chunksRef.current.length && !ttsPreloadCache.current[index + 1]) {
    const nextText = chunksRef.current[index + 1];
    const nextVoice = voiceRef.current || voice;
    fetch("/api/tts", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ text: nextText, voice: nextVoice }),
    }).then(r => r.ok ? r.blob() : null).then(blob => {
        if (blob) {
            ttsPreloadCache.current[index + 1] = URL.createObjectURL(blob);
            console.log(`[TTS] Preloaded chunk ${index + 2}`);
        }
    }).catch(() => {});
}

const success = await playTTS(chunksRef.current[index]);
if (!isPlayingRef.current) return;
if (!success) {
    console.log(`[TTS] Chunk ${index + 1} failed after retries, skipping to next chunk`);
    toast({ title: "Skipped chunk", description: `Chunk ${index + 1} failed, moving to next` });
}
if (index >= 2 && index < chunksRef.current.length - 1 && (index + 1) % 3 === 0 && !playingAttentionPromptRef.cur...
    playingAttentionPromptRef.current = true;
    console.log(`[TTS] Playing attention prompt after chunk ${index + 1}`);
    if (catWashFollow || speakerParam) {
        try {
            await fetch("/api/media/attention-prompt", { method: "POST" });
            await new Promise(r => setTimeout(r, 4000));
        } catch (e) {
            console.log(`[TTS] Attention prompt server call failed:`, e);
        }
    } else {
        if (!attentionPromptBlobUrlRef.current) {
            try {
                console.log(`[TTS] Generating attention prompt audio (first time)`);
                const resp = await fetch("/api/tts", {
                    method: "POST",
                    headers: { "Content-Type": "application/json" },
                    body: JSON.stringify({ text: "Bryn, are you paying attention?", voice }),
                });
                if (resp.ok) {
                    const blob = await resp.blob();
                    attentionPromptBlobUrlRef.current = URL.createObjectURL(blob);
                    console.log(`[TTS] Attention prompt cached (${blob.size} bytes)`);
                }
            } catch (e) {
                console.log(`[TTS] Failed to pre-generate attention prompt:`, e);
            }
        }
        if (attentionPromptBlobUrlRef.current) {
            console.log(`[TTS] Playing cached attention prompt (separate audio)`);
            await new Promise<void>((resolve) => {
                const tempAudio = new Audio(attentionPromptBlobUrlRef.current!);

```

```

        tempAudio.volume = volumeRef.current;
        tempAudio.playbackRate = 1;
        tempAudio.onended = () => { tempAudio.remove(); resolve(); };
        tempAudio.onerror = () => { tempAudio.remove(); resolve(); };
        tempAudio.play().catch(() => { tempAudio.remove(); resolve(); });
    });
} else {
    console.log('[TTS] Playing attention prompt via separate audio fetch');
    await new Promise<void>(async (resolve) => {
        try {
            const resp = await fetch("/api/tts", {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({ text: "Bryn, are you paying attention?", voice }),
            });
            if (resp.ok) {
                const blob = await resp.blob();
                const url = URL.createObjectURL(blob);
                attentionPromptBlobUrlRef.current = url;
                const tempAudio = new Audio(url);
                tempAudio.volume = volumeRef.current;
                tempAudio.onended = () => { tempAudio.remove(); resolve(); };
                tempAudio.onerror = () => { tempAudio.remove(); resolve(); };
                tempAudio.play().catch(() => { tempAudio.remove(); resolve(); });
            } else { resolve(); }
        } catch { resolve(); }
    });
}
}
playingAttentionPromptRef.current = false;
}
if (isPlayingRef.current) {
    playNextChunk(index + 1);
}
};

const handleAudioEnded = async () => {
};

const pauseReading = () => {
    console.log('[TTS] Pausing');
    if (audioRef.current) {
        audioRef.current.pause();
    }
    setIsPaused(true);
    isPausedRef.current = true;
};

const resumeReading = () => {
    console.log('[TTS] Resuming');
    if (audioRef.current) {
        audioRef.current.play();
    }
    setIsPaused(false);
    isPausedRef.current = false;
};

const stopReading = async () => {
    console.log('[TTS] Stopping');
    setIsPlaying(false);
    isPlayingRef.current = false;
    setIsPaused(false);
    isPausedRef.current = false;
    if (speakerTimerRef.current) {
        clearTimeout(speakerTimerRef.current);
        speakerTimerRef.current = null;
    }
    if (audioRef.current) {
        audioRef.current.pause();
        audioRef.current.onended = null;
        audioRef.current.removeAttribute('src');
        audioRef.current.load();
    }
    try {
        const utterance = new SpeechSynthesisUtterance('Stop received');
        utterance.rate = 1.1;
        utterance.volume = 0.8;
        speechSynthesis.speak(utterance);
    } catch {}
};

const savedChunk = currentChunkRef.current;

if (fileId && savedChunk > 0) {
    fetch(`/api/files/${fileId}`, {
        method: 'PATCH',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ lastChunkIndex: savedChunk, totalChunks: totalChunks })
    }).then(() => {
        queryClient.invalidateQueries({ queryKey: ['/api/files'] });
    }).catch(e => console.error('Failed to save progress:', e));
}

if (catWashFollow || speakerParam) {
    fetch("/api/webhook/voice-command", { method: "POST", headers: { 'Content-Type': 'application/json' }, body: JS...
    }).then(() => { setFollowState(null); setCatWashPaused(false); })
    .catch(e => console.error('Failed to stop server-side playback:', e));
}
};

const resumeFromLast = async () => {
    if (!file?.lastChunkIndex || file.lastChunkIndex === 0) {
        startReading();
        return;
    }
    setIsLoading(true);

```



```

setIsPlaying(true);
isPlayingRef.current = true;
setIsPaused(false);
isPausedRef.current = false;

// Extract text if not already done - use startReading which handles extraction
if (chunksRef.current.length === 0) {
  // Text not extracted yet, let startReading handle it then we'll skip to saved position
  await new Promise<void>((resolve) => {
    const checkChunks = setInterval(() => {
      if (chunksRef.current.length > 0) {
        clearInterval(checkChunks);
        resolve();
      }
    }, 100);
    // Timeout after 30 seconds
    setTimeout(() => {
      clearInterval(checkChunks);
      resolve();
    }, 30000);
  });
}

// Start from saved position
if (chunksRef.current.length > 0 && file.lastChunkIndex < chunksRef.current.length) {
  setCurrentChunk(file.lastChunkIndex);
  await playTTS(chunksRef.current[file.lastChunkIndex]);
} else {
  startReading();
}

setIsLoading(false);
};

const restartFromBeginning = async () => {
  skipCheckedReloadRef.current = true;
  resetStartRef.current = true;
  const key = getFileKey();
  const emptySet = new Set<number>();
  setCheckedChunks(emptySet);
  setCurrentChunk(0);
  currentChunkRef.current = 0;
  localStorage.removeItem(`checkedChunks_${key}`);
  localStorage.removeItem(`chunkProgress_${key}`);
  const allProgress = JSON.parse(localStorage.getItem('allChunkProgress') || '{}');
  delete allProgress[key];
  localStorage.setItem('allChunkProgress', JSON.stringify(allProgress));
  if (fileId) {
    try {
      await fetch(`/api/files/${fileId}`, {
        method: 'PATCH',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ checkedChunks: '[]', lastChunkIndex: 0, totalChunks, listened: false }),
      });
      console.log(`[Reset] Server progress cleared for file ${fileId}`);
      queryClient.setQueryData(['/api/files', fileId], (old: any) => old ? { ...old, checkedChunks: '[]', lastChunk...
        queryClient.setQueryData(['/api/files'], (old: any) => {
          if (!Array.isArray(old)) return old;
          return old.map((f: any) => f.id === Number(fileId) ? { ...f, checkedChunks: '[]', lastChunkIndex: 0, listen...
        });
      queryClient.invalidateQueries({ queryKey: ['/api/files'] });
      queryClient.invalidateQueries({ queryKey: ['/api/files', fileId] });
    } catch (e) {
      console.error(`[Reset] Failed to clear server progress:`, e);
    }
  }
  skipCheckedReloadRef.current = true;
  startReading();
};

const restartCurrentChunk = () => {
  if (audioRef.current) {
    audioRef.current.pause();
  }
  const idx = currentChunkRef.current;
  console.log(`[TTS] Restart current chunk ${idx}`);
  setIsPlaying(true);
  isPlayingRef.current = true;
  setIsPaused(false);
  isPausedRef.current = false;
  playNextChunk(idx);
};

const skipBack = () => {
  if (currentChunkRef.current > 0) {
    try {
      const utterance = new SpeechSynthesisUtterance('previous chunk');
      utterance.rate = 1.1;
      utterance.volume = 0.8;
      speechSynthesis.speak(utterance);
    } catch {}

    if (catWashFollow || speakerParam) {
      fetch("/api/media/skip-chunk", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ direction: "backward" }),
      }).catch(e => console.error('[TTS] Server skip back failed:', e));
    }

    if (audioRef.current) {
      audioRef.current.pause();
    }
    const prevChunk = currentChunkRef.current - 1;
    console.log(`[TTS] Skip back to chunk ${prevChunk}`);
    setCurrentChunk(prevChunk);
    currentChunkRef.current = prevChunk;
    setIsPlaying(true);
  }
};

```

```

        isPlayingRef.current = true;
        setIsPaused(false);
        isPausedRef.current = false;
        playNextChunk(prevChunk);
    }
};

const skipForward = () => {
    if (currentChunkRef.current < totalChunks - 1) {
        try {
            const utterance = new SpeechSynthesisUtterance('next chunk');
            utterance.rate = 1.1;
            utterance.volume = 0.8;
            speechSynthesis.speak(utterance);
        } catch {}

        if (catWashFollow || speakerParam) {
            fetch("/api/media/skip-chunk", {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({ direction: "forward" }),
            }).catch(e => console.error('[TTS] Server skip forward failed:', e));
        }

        if (audioRef.current) {
            audioRef.current.pause();
        }
        const nextChunk = currentChunkRef.current + 1;
        console.log(`[TTS] Skip forward to chunk ${nextChunk}`);
        setCurrentChunk(nextChunk);
        currentChunkRef.current = nextChunk;
        setIsPlaying(true);
        isPlayingRef.current = true;
        setIsPaused(false);
        isPausedRef.current = false;
        playNextChunk(nextChunk);
    }
};

const goToPreviousPage = () => {
    if (currentPage > 1) {
        setCurrentPage(currentPage - 1);
    }
};

const goToNextPage = () => {
    if (currentPage < numPages) {
        setCurrentPage(currentPage + 1);
    }
};

useEffect(() => {
    if (extractedText && chunksRef.current.length === 0) {
        const newChunks = chunkText(extractedText);
        chunksRef.current = newChunks;
        setChunksList(newChunks);
        setTotalChunks(newChunks.length);
        let loaded = new Set<number>();
        if (!skipCheckedReloadRef.current) {
            let serverChecked = new Set<number>();
            if (file?.checkedChunks) {
                try { const arr = JSON.parse(file.checkedChunks); if (Array.isArray(arr)) serverChecked = new Set(arr); } c...
            }
            loaded = serverChecked.size > 0 ? serverChecked : new Set<number>();
            setCheckedChunks(loaded);
        }
    }

    const firstUnlistened = newChunks.findIndex((_, idx) => !loaded.has(idx));
    const preloadIdx = firstUnlistened >= 0 ? firstUnlistened : 0;
    if (newChunks[preloadIdx] && !ttsPreloadCache.current[preloadIdx]) {
        console.log(`[TTS] Auto-preloading chunk ${preloadIdx + 1} audio...`);
        fetch("/api/tts", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ text: newChunks[preloadIdx], voice: voiceRef.current || 'echo' }),
        }).then(r => r.ok ? r.blob() : null).then(blob => {
            if (blob) {
                ttsPreloadCache.current[preloadIdx] = URL.createObjectURL(blob);
                console.log(`[TTS] Chunk ${preloadIdx + 1} audio pre-cached and ready`);
            }
        }).catch(() => {});
    }
}, [extractedText]);

useEffect(() => {
    playbackSpeedRef.current = playbackSpeed;
    if (audioRef.current) {
        audioRef.current.playbackRate = playbackSpeed;
        console.log(`[TTS] Speed changed: ${playbackSpeed}x, actual=${audioRef.current.playbackRate}`);
    }
}, [playbackSpeed]);

useEffect(() => {
    volumeRef.current = volume;
    if (audioRef.current) {
        audioRef.current.volume = volume;
        console.log(`[TTS] Volume changed: ${volume}`);
    }
}, [volume]);

const getTimeEstimate = () => {
    if (totalChunks === 0 || !chunksRef.current.length) return null;
    const uncheckedChunks = Array.from({ length: totalChunks }, (_, i) => i).filter(i => !checkedChunks.has(i));
    if (uncheckedChunks.length === 0) return { remaining: '0m', total: '0m', remainingChunks: 0 };
    let totalWords = 0;
    let remainingWords = 0;
    for (let i = 0; i < chunksRef.current.length; i++) {

```



```

>
<ArrowLeft className="h-3.5 w-3.5" />
Exit
</button>
{(() => {
  const isDbFile = !fileId && !isOneDrive;
  const isModule = file?.folder?.includes('module');
  const isReading = file?.folder?.includes('reading');
  const relatedFiles = isReading ? readingFiles : moduleFiles;
  const currentIndex = relatedFiles.findIndex(f => f.id === fileId);
  const canGoPrev = currentIndex > 0;
  const canGoNext = currentIndex < relatedFiles.length - 1;

  if (isDbFile && (moduleFiles.length > 0 || readingFiles.length > 0)) {
    return (
      <div className="flex items-center gap-1.5 min-w-0 shrink" style={{ marginLeft: '-3px' }}>
        <span className="text-[13px] text-white shrink-0">Module:</span>
        <Select
          value={isModule && fileId ? fileId.toString() : (moduleFiles[0]?.id?.toString() || 'none')}
          onChange={(val) => {
            if (val === 'none') return;
            window.location.href = '/pdf-reader/${val}';
          }}
        >
          <SelectTrigger className="h-8 text-[11px] px-2.5 bg-white/10 border !border-white focus:ring-0 ...
            <span className="truncate block" style={{ maxWidth: '300px', minWidth: '60px' }}>
              {moduleFiles.length === 0 ? 'No modules' : (() => {
                const f = isModule ? file : moduleFiles[0];
                return f ? (f.displayName || f.originalName || '').replace(/\.pdf$/i, '') : 'No modules';
              })()}
            </span>
          </SelectTrigger>
          <SelectContent className="max-h-[300px] min-w-[350px]">
            {moduleFiles.length === 0 && <SelectItem value="none" disabled className="text-[10px] text-gr...
              {moduleFiles.map(f => (
                <SelectItem key={f.id} value={f.id.toString()} className={`text-[10px] ${f.listened ? 'text...
                  {(f.displayName || f.originalName || '').replace(/\.pdf$/i, '')}
                </SelectItem>
              ))}
            </SelectContent>
          </Select>
        </div>

        <div className="flex items-center gap-1.5 min-w-0 shrink" style={{ marginLeft: '45px' }}>
          <span className="text-[13px] text-white shrink-0">Reading:</span>
          <Select
            value={isReading && fileId ? fileId.toString() : (readingFiles[0]?.id?.toString() || 'none')}
            onChange={(val) => {
              if (val === 'none') return;
              window.location.href = '/pdf-reader/${val}';
            }}
          >
            <SelectTrigger className="h-8 text-[11px] px-2.5 bg-white/10 border !border-white focus:ring-0 ...
              <span className="truncate block" style={{ maxWidth: '300px', minWidth: '60px' }}>
                {readingFiles.length === 0 ? 'No readings' : (() => {
                  const f = isReading ? file : readingFiles[0];
                  return f ? (f.displayName || f.originalName || '').replace(/\.pdf$/i, '') : 'No readings';
                })()}
              </span>
            </SelectTrigger>
            <SelectContent className="max-h-[300px] min-w-[350px]">
              {readingFiles.length === 0 ? <SelectItem value="none" disabled className="text-[10px] text-gr...
                <SelectItem key={f.id} value={f.id.toString()} className={`text-[10px] ${f.listened ? 'text...
                  {(f.displayName || f.originalName || '').replace(/\.pdf$/i, '')}
                </SelectItem>
              ))}
            </SelectContent>
          </Select>
        </div>

        <div className="flex items-center gap-1" style={{ marginLeft: '40px' }}>
          <Button size="icon" variant="ghost" className="h-6 w-6 text-white hover:bg-white/20 disabled:opac...
            if (canGoPrev) window.location.href = '/pdf-reader/${relatedFiles[currentIndex - 1].id}';
          </Button>
          <Button size="icon" variant="ghost" className="h-6 w-6 text-white hover:bg-white/20 disabled:opac...
            if (canGoNext) window.location.href = '/pdf-reader/${relatedFiles[currentIndex + 1].id}';
          </Button>
        </div>
      </div>
    );
  }
}

if (allFiles.length > 1) {
  return (
    <div className="flex items-center gap-1.5 min-w-0 shrink">
      <span className="text-[13px] text-white shrink-0">Module:</span>
      <Select
        value={currentFileUrl || (oneDriveUrl ? decodeURIComponent(oneDriveUrl) : '')}
        onChange={(url) => {
          const f = allFiles.find(af => af.downloadUrl === url);
          if (f) switchToFile(f);
        }}
      >
        <SelectTrigger className="h-8 text-[11px] px-2.5 bg-white/10 border !border-white focus:ring-0 ...
          <span className="truncate block" style={{ maxWidth: '300px', minWidth: '60px' }}>
            {(() => {
              const current = allFiles.find(af => af.downloadUrl === (currentFileUrl || (oneDriveUrl ? ...
                return current ? current.name.replace(/\.pdf$/i, '') : 'Select...';
            })}
          </span>
        </SelectTrigger>
      </div>
    );
  );
}

```



```

<div className="flex-1 flex relative overflow-hidden min-h-0" style={{ zIndex: 2 }}>
  <div className={` ${isFullPage ? 'w-full' : 'flex-1 lg:w-1/2'} overflow-auto pdf-reader-scrollbar`} style={{ b...
    {!followOnly && <div className="sticky top-0 z-20" style={{ background: 'rgba(0,10,30,0.8)', backdropFilter...
      <div className="flex items-center justify-between px-4 py-1.5 border-b border-white/10">
        <span className="text-[10px] uppercase tracking-wider text-white/50 font-semibold">Filtered Text</span>
        <div className="flex items-center gap-2">
          <span className="text-[14px] font-semibold text-white">{checkedChunks.size}/{totalChunks} chunks</span>
        </div>
      </div>
    </div>
    {searchOpen && (
      <div className="px-3 py-2 border-b border-white/10 space-y-1.5">
        <div className="flex items-center gap-2">
          <button className="flex-shrink-0" onClick={() => setShowReplace(prev => !prev)} data-testid="button...
            {showReplace ? <ChevronDown className="h-3.5 w-3.5 text-white/40" /> : <ChevronRight className="h...
          </button>
          <Search className="h-3.5 w-3.5 text-white/40 flex-shrink-0" />
          <input
            ref={searchInputRef}
            type="text"
            value={searchQuery}
            onChange={(e) => { setSearchQuery(e.target.value); setSearchMatchIndex(0); }}
            onKeyDown={(e) => {
              if (e.key === 'Enter') {
                if (e.shiftKey) setSearchMatchIndex(prev => Math.max(0, prev - 1));
                else setSearchMatchIndex(prev => prev < searchMatchCount - 1 ? prev + 1 : 0);
              }
              if (e.key === 'Escape') { setSearchOpen(false); setSearchQuery(""); setSearchMatchIndex(0); set...
            }}
            placeholder="Find..."
            className="flex-1 bg-transparent text-white text-xs outline-none placeholder:text-white/30"
            data-testid="input-search"
          />
          {searchQuery.trim().length >= 2 && (
            <span className="text-[10px] text-white/40 whitespace-nowrap">
              {searchMatchCount > 0 ? `${Math.min(searchMatchIndex + 1, searchMatchCount)}/${searchMatchCount}...
            </span>
          )}
          <Button size="icon" variant="ghost" className="h-5 w-5 text-white/40" onClick={() => setSearchMatch...
            <ChevronUp className="h-3 w-3" />
          </Button>
          <Button size="icon" variant="ghost" className="h-5 w-5 text-white/40" onClick={() => setSearchMatch...
            <ChevronDown className="h-3 w-3" />
          </Button>
          <Button size="icon" variant="ghost" className="h-5 w-5 text-white/40" onClick={() => { setSearchOpe...
            <X className="h-3 w-3" />
          </Button>
        </div>
      </div>
    {showReplace && (
      <div className="flex items-center gap-2 pl-[22px]">
        <Replace className="h-3.5 w-3.5 text-white/40 flex-shrink-0" />
        <input
          type="text"
          value={replaceText}
          onChange={(e) => setReplaceText(e.target.value)}
          onKeyDown={(e) => { if (e.key === 'Escape') { setShowReplace(false); setReplaceText(""); } }}
          placeholder="Replace with..."
          className="flex-1 bg-transparent text-white text-xs outline-none placeholder:text-white/30"
          data-testid="input-replace"
        />
        <Button size="sm" variant="ghost" className="h-5 px-2 text-[10px] text-blue-400 hover:text-blue-3...
          disabled={searchMatchCount === 0 || searchQuery.trim().length < 2}
          onClick={() => {
            if (searchQuery.trim().length < 2 || searchMatchCount === 0) return;
            const q = searchQuery.trim();
            const qLower = q.toLowerCase();
            let globalIdx = 0;
            const newChunks = chunksList.map(chunk => {
              let result = '';
              let lower = chunk.toLowerCase();
              let lastPos = 0;
              let pos = lower.indexOf(qLower, lastPos);
              while (pos !== -1) {
                if (globalIdx === searchMatchIndex) {
                  result += chunk.slice(lastPos, pos) + replaceText;
                  lastPos = pos + q.length;
                  const newText = result + chunk.slice(lastPos);
                  const remaining = chunksList.map((c, i) => i === chunksList.indexOf(chunk) ? newText : c);
                  const fullText = remaining.join('\n\n');
                  setExtractedText(fullText);
                  const rechunked = chunkText(fullText);
                  chunksRef.current = rechunked;
                  setChunksList(rechunked);
                  setTotalChunks(rechunked.length);
                  if (searchMatchIndex >= searchMatchCount - 1) setSearchMatchIndex(Math.max(0, searchMat...
                    return;
                }
                result += chunk.slice(lastPos, pos + q.length);
                lastPos = pos + q.length;
                globalIdx++;
                pos = lower.indexOf(qLower, lastPos);
              }
              return undefined;
            });
          }}
        >Replace</Button>
        <Button size="sm" variant="ghost" className="h-5 px-2 text-[10px] text-amber-400 hover:text-amber...
          disabled={searchMatchCount === 0 || searchQuery.trim().length < 2}
          onClick={() => {
            if (searchQuery.trim().length < 2 || searchMatchCount === 0) return;
            const q = searchQuery.trim();
            const fullText = extractedText.split(q).join(replaceText);
            setExtractedText(fullText);
            const rechunked = chunkText(fullText);
            chunksRef.current = rechunked;
            setChunksList(rechunked);
            setTotalChunks(rechunked.length);
            setSearchMatchIndex(0);
          }}
        </div>
      </div>
    )}
  </div>

```



```

    </span>
  ))
</p>
) : followState.chunkText ? (
  <p className="text-2xl leading-relaxed text-white/90">{followState.chunkText}</p>
) : (
  <div className="flex items-center justify-center h-full">
    <span className="text-lg text-white/40 animate-pulse">Loading text...</span>
  </div>
  ))
</div>
</div>
);
})();

{followOnly && !followState?.active && (
  <div className="fixed inset-0 z-50 flex items-center justify-center" style={{ background: '#0a0a1a' }}>
    <span className="text-lg text-white/40">Waiting for playback...</span>
  </div>
)}

{catWashFollow && !followOnly && followState?.active && (
  <div className="mx-4 mt-4 mb-2 p-3 rounded-lg border border-blue-400/30 flex items-center justify-between...
    <div className="flex items-center gap-3">
      <span className="text-sm font-medium text-blue-300">Following Cat Wash Playback</span>
      <span className="text-xs bg-blue-500/20 text-blue-300 px-2 py-1 rounded-full">
        Chunk {followState.chunkIndex + 1} of {followState.totalChunks} ({Math.round(((followState.chunkInd...
      </span>
    </div>
    <button className="text-xs text-red-400 hover:text-red-300 underline" onClick={() => fetch("/api/webhoo...
      Stop Playback
    </button>
  </div>
)}

{catWashFollow && !followOnly && !autoplayParam && !followState?.active && (
  <div className="mx-4 mt-4 mb-2 p-3 rounded-lg border border-white/10 text-center" style={{ background: 'r...
    <span className="text-sm text-white/50">{catWashPaused ? 'Paused. Tap play to resume.' : 'Waiting for p...
  </div>
)}

{!isEditingText && !followOnly && (
  <div className="p-4 space-y-3">
    {chunksList.map((chunk, idx) => {
      const isFollowActive = catWashFollow && !followOnly && followState?.active && followState.chunkIndex ...
      const isActive = (currentChunk === idx && isPlaying) || !isFollowActive;
      const isChecked = checkedChunks.has(idx);
      return (
        <div
          key={idx}
          className={`flex gap-0 rounded-xl transition-colors border ${isActive ? 'bg-white/10 border-white...
          data-testid={`chunk-row-${idx}`}
        >
          <div
            className="flex flex-col items-center shrink-0 pt-4 px-3"
            style={{
              background: isChecked
                ? `linear-gradient(180deg, ${waveColor} 0%, ${waveColor}BB 100%)`
                : `linear-gradient(180deg, ${waveColor} 0%, ${waveColor}99 100%)`,
            }}
          >
            <div className="sticky flex flex-col items-center pt-2" style={{ top: '12px', zIndex: 3 }}>
              <div
                onClick={() => toggleChunkChecked(idx)}
                className="cursor-pointer flex items-center justify-center"
                style={{ width: '20px', height: '20px', minWidth: '20px', minHeight: '20px', border: '1px s...
                data-testid={`checkbox-chunk-${idx}`}
              >
                {isChecked && <Check className="h-4 w-4 text-gray-900" strokeWidth={3} />}
              </div>
              <button
                className="transition-colors mt-[23px] hover:bg-white/25 flex items-center justify-center"
                style={{ width: '20px', height: '20px', minWidth: '20px', minHeight: '20px', padding: 0 }}
                onClick={() => {
                  const chunkEl = document.querySelector(`[data-testid="chunk-row-${idx}"]`);
                  if (chunkEl) chunkEl.scrollIntoView({ behavior: 'smooth', block: 'center' });
                  setSearchOpen(true);
                  setTimeout(() => searchInputRef.current?.focus(), 100);
                }}
                data-testid={`button-chunk-search-${idx}`}
                title="Search in text"
              >
                <Search className="text-white" style={{ width: '20px', height: '20px' }} />
              </button>
              <button
                className={`rounded-full transition-colors mt-[23px] border border-white ${isActive ? 'bg-w...
                style={{ width: '20px', height: '20px', minWidth: '20px', minHeight: '20px', padding: 0 }}
                onClick={() => {
                  if (isActive && !isPaused) {
                    pauseReading();
                  } else if (isActive && isPaused) {
                    resumeReading();
                  } else {
                    if (audioRef.current) audioRef.current.pause();
                    setIsPlaying(true);
                    isPlayingRef.current = true;
                    setIsPaused(false);
                    isPausedRef.current = false;
                    playNextChunk(idx);
                  }
                }}
                data-testid={`button-chunk-play-${idx}`}
              >
                {isActive && !isPaused ? (
                  <Pause className="h-3 w-3 text-white" />
                ) : (
                  <Play className="h-3 w-3 text-white ml-px" />
                )
              }
            </div>
          </div>
        )
      )
    })
  </div>
)

```



```

    })
  </button>
</div>
<div style={{ marginTop: '380px' }}>
  <button
    className="transition-colors hover:bg-white/25 flex items-center justify-center"
    style={{ width: '20px', height: '20px', minWidth: '20px', minHeight: '20px', padding: 0 }}
    onClick={() => {
      setEditingChunkIndex(idx);
      setEditableChunkText(chunk);
    }}
    data-testid={`button-chunk-edit-${idx}`}
    title="Edit chunk"
  >
    <Pencil className="h-4 w-4 text-white" strokeWidth={3} />
  </button>
</div>
<div className="flex-1" />
<div className="flex-1 min-w-0 p-4 pl-3 relative">
  {editingChunkIndex !== idx && (
    <div className="absolute right-2" style={{ top: '-2px', zIndex: 2 }}>
      <button
        className="transition-colors hover:bg-white/25 flex items-center justify-center rounded"
        style={{ width: '20px', height: '20px', padding: 0 }}
        onClick={() => setIsFullPage(!isFullPage)}
        data-testid={`button-chunk-fullpage-${idx}`}
        title={isFullPage ? "Exit full page" : "Full page reader"}
      >
        {isFullPage ? <Minimize2 className="h-4 w-4 text-white/60 hover:text-white" strokeWidth={...} /> : null}
      </button>
    </div>
  )}
  {editingChunkIndex === idx ? (
    <div className="relative">
      <textarea
        value={editableChunkText}
        onChange={(e) => setEditableChunkText(e.target.value)}
        className="w-full bg-white/5 text-white/90 text-[22px] leading-[2.1] p-2 rounded-lg border"
        style={{ minHeight: '80px' }}
        autoFocus
        data-testid={`textarea-chunk-edit-${idx}`}
      />
      <div className="flex gap-2 mt-2">
        <button
          className="px-3 py-1 bg-white/20 text-white text-xs rounded hover:bg-white/30 transition-colors"
          onClick={() => {
            const newChunks = [...chunksList];
            newChunks[idx] = editableChunkText;
            setChunksList(newChunks);
            setEditingChunkIndex(null);
          }}
          data-testid={`button-chunk-save-${idx}`}
        >Save</button>
        <button
          className="px-3 py-1 bg-white/10 text-white/60 text-xs rounded hover:bg-white/15 transition-colors"
          onClick={() => setEditingChunkIndex(null)}
          data-testid={`button-chunk-cancel-${idx}`}
        >Cancel</button>
      </div>
    </div>
  ) : (
    <p className={`${text[22px] leading-[2.1] cursor-pointer ${isChecked ? 'text-white/30 line-through' : ''}}`}>
      {isFollowActive && followState?.words && followState?.words.length > 0 ? (
        followState?.words.map((word: string, wIdx: number) => (
          <span
            key={wIdx}
            data-word-index={wIdx}
            className={` ${
              wIdx === (followState?.estimatedWordIndex || 0)
                ? "bg-red-500 text-white font-semibold px-0.5 rounded"
                : wIdx < (followState?.estimatedWordIndex || 0)
                ? "text-white/40"
                : ""
            } transition-colors duration-100`}
          >
            {word}{" "}
          </span>
        ))
      ) : isActive && !isFollowActive && chunkWords.length > 0 ? (
        chunkWords.map((word, wIdx) => (
          <span
            key={wIdx}
            data-word-index={wIdx}
            className={` ${
              wIdx === currentWordIndex
                ? "bg-red-500 text-white font-semibold px-0.5 rounded"
                : wIdx < currentWordIndex
                ? "text-white/40"
                : ""
            } transition-colors duration-100`}
          >
            {word}{" "}
          </span>
        ))
      ) : searchQuery.trim().length >= 2 ? (
        (() => {
          const q = searchQuery.trim();
          const qLower = q.toLowerCase();
          const parts: Array<{text: string; isMatch: boolean}> = [];
          let remaining = chunk;
          while (remaining.length > 0) {

```

```

const idx = remaining.toLowerCase().indexOf(qLower);
if (idx === -1) { parts.push({text: remaining, isMatch: false}); break; }
if (idx > 0) parts.push({text: remaining.slice(0, idx), isMatch: false});
parts.push({text: remaining.slice(idx, idx + q.length), isMatch: true});
remaining = remaining.slice(idx + q.length);
}
return parts.map((p, pi) => p.isMatch
? <mark key={pi} className="bg-yellow-400/80 text-black rounded px-0.5">{p.text}</mark>
: <span key={pi}>{p.text}</span>
);
})();
): {
  chunk
}
</p>
)
</div>
</div>
);
})
{chunksList.length === 0 && !isPreloading && extractedText && (
  <p className="text-center text-white/40 py-8 text-sm">Press play to extract and chunk the text</p>
)}
{chunksList.length === 0 && !extractedText && !isPreloading && (
  <p className="text-center text-white/40 py-8 text-sm">Loading PDF text...</p>
)}
{isPreloading && (
  <div className="flex items-center justify-center py-8 gap-2">
    <loader2 className="h-5 w-5 animate-spin text-white/40" />
    <span className="text-sm text-white/40">Preparing text...</span>
  </div>
)}
</div>
)}
</div>
</div>
<div className={` ${isFullPage ? 'hidden' : 'hidden lg:flex'} lg:w-[45%] flex-col border-l border-white/10`} s...
  <div className="sticky top-0 z-10 flex items-center justify-between px-4 py-1.5 border-b border-white/10" s...
    <span className="text-[10px] uppercase tracking-wider text-white/50 font-semibold">Original PDF</span>
    <div className="flex items-center gap-1" style={{ marginRight: '13px' }}>
      <Button variant="ghost" size="icon" className="h-6 w-6 text-white/60 hover:text-white" onClick={goToPre...
        <ChevronLeft className="h-3.5 w-3.5" />
      </Button>
      <span className="text-[10px] text-white/60 min-w-[40px] text-center">{currentPage} / {numPages}</span>
      <Button variant="ghost" size="icon" className="h-6 w-6 text-white/60 hover:text-white" onClick={goToNex...
        <ChevronRight className="h-3.5 w-3.5" />
      </Button>
    </div>
  </div>
</div>
<div className="flex-1 overflow-auto flex justify-center items-start p-2" ref={(el) => { if (el && !el.data...
  {pdfUrl && (
    <Document
      file={pdfUrl}
      onLoadSuccess={onDocumentLoadSuccess}
      loading={
        <div className="flex items-center justify-center h-full">
          <loader2 className="h-8 w-8 animate-spin text-white/40" />
        </div>
      }
    >
      <Page
        pageNumber={currentPage}
        height={pdfContainerHeight ? pdfContainerHeight - 16 : undefined}
        width={pdfContainerHeight ? (isMobile ? window.innerWidth - 32 : 480) : undefined}
        renderTextLayer={true}
        renderAnnotationLayer={true}
      </Page>
    </Document>
  )}
</div>
</div>
</div>
</div>
{totalChunks > 0 && !(catWashFollow && followState?.active) && (
  <div className="relative flex-shrink-0" style={{ zIndex: 10 }}>
    <div className="w-full px-0" data-testid="progress-gradient-bar">
      <div className="relative w-full h-8 flex items-center" style={{ background: `linear-gradient(90deg, ${wav...
        <div className="flex items-center gap-3 px-4 w-full">
          <input
            type="checkbox"
            checked={chunkProgress === 100}
            readOnly
            className="h-4 w-4 rounded border-2 border-white/60 cursor-default appearance-none checked:bg-white...
            style={{ WebkitAppearance: 'none', MozAppearance: 'none' }}
            data-testid="checkbox-progress-all"
          />
          <div className="flex-1 flex items-center justify-between" style={{ marginTop: '4px' }}>
            <div className="flex items-center gap-2">
              <span className="text-[11px] text-white font-medium">{checkedChunks.size} / {totalChunks} Chunks ...
              <div style={{ width: '130px', marginLeft: '10px' }}>
                <div className="h-2 rounded-full" style={{ background: 'rgba(255,255,255,0.15)' }}>
                  <div className="h-full rounded-full transition-all duration-500" style={{ width: `${chunkProg...
                </div>
              </div>
            </div>
            <div className="flex items-center gap-1 shrink-0">
              <Button variant="ghost" size="icon" className="h-7 w-7 text-white/80 hover:text-white hover:bg-wh...
                <ChevronLeft className="h-5 w-5" strokeWidth={3} />
              </Button>
              <Button variant="ghost" size="icon" className="h-7 w-7 text-white/80 hover:text-white hover:bg-wh...
                <ChevronRight className="h-5 w-5" strokeWidth={3} />
              </Button>
            </div>
          </div>
          {checkedChunks.size < totalChunks && (
            <button
              className="text-[13px] text-white/70 hover:text-white underline font-semibold shrink-0"
              style={{ marginRight: '27px' }}
              onClick={() => {

```



```

    const fileKey = getFileKey();
    const allProgress = JSON.parse(localStorage.getItem('allChunkProgress')) || '{}';
    if (allProgress[fileKey]) { allProgress[fileKey] = { total: allProgress[fileKey].total, checked: 0 ...
    if (file) { fetch(`/api/files/${file.id}`, { method: 'PATCH', headers: { 'Content-Type': 'applicati...
  }) disabled={checkedChunks.size === 0} title="Reset all progress" data-testid="button-reset-progress">
    <X className="h-5 w-5 text-white" />
    <span className="text-[8px] text-white/70 leading-none mt-0.5">Reset</span>
  </button>

  <button className="w-12 h-12 flex items-center justify-center rounded-full hover:bg-white/10 disabled...
    <RotateCcw className="h-5 w-5 text-white" />
    <span className="text-[9px] text-white font-medium ml-0.5">15s</span>
  </button>

  <button className="w-12 h-12 flex items-center justify-center rounded-full hover:bg-white/10 disabled...
    <SkipBack className="h-6 w-6 text-white" />
  </button>
</div>

<div className="shrink-0 flex items-center justify-center" style={{ width: '80px' }}>
  {file && file.lastChunkIndex > 0 && !isPlaying && !isPaused ? (
    <button
      className="w-16 h-16 flex flex-col items-center justify-center rounded-full shrink-0"
      style={{ outline: '2px solid ${waveColor}66', outlineOffset: '3px', background: `${waveColor}22` }}
      onClick={resumeFromLast}
      disabled={isLoading || numPages === 0}
      title={`Resume from chunk ${file.lastChunkIndex + 1} of ${file.totalChunks}...`}
      data-testid="button-resume"
    >
      {isLoading ? <Loader2 className="h-6 w-6 text-white animate-spin" /> : <RotateCcw className="h-6 ...
      <span className="text-[8px] text-white/70 leading-none mt-0.5">Resume</span>
    </button>
  ) : catWashFollow && followState?.active ? (
    <button
      className="w-16 h-16 flex items-center justify-center rounded-full bg-yellow-600 hover:bg-yellow-...
      style={{ outline: '2px solid rgba(255,200,50,0.5)', outlineOffset: '3px' }}
      onClick={() => {
        fetch("/api/webhook/voice-command", { method: "POST", headers: { 'Content-Type': 'applicati...
        .then(() => setCatWashPaused(true))
        .catch(e => console.error('Failed to pause:', e));
      }}
      data-testid="button-pause-catwash"
    >
      <Pause className="h-7 w-7 text-white fill-white" />
    </button>
  ) : catWashFollow && catWashPaused ? (
    <button
      className="w-16 h-16 flex items-center justify-center rounded-full bg-green-600 hover:bg-green-50...
      style={{ outline: '2px solid rgba(100,255,100,0.5)', outlineOffset: '3px' }}
      onClick={() => {
        fetch("/api/webhook/voice-command", { method: "POST", headers: { 'Content-Type': 'applicati...
        .then(() => setCatWashPaused(false))
        .catch(e => console.error('Failed to resume:', e));
      }}
      data-testid="button-resume-catwash"
    >
      <Play className="h-7 w-7 text-white fill-white ml-0.5" />
    </button>
  ) : isPlaying && !isPaused ? (
    <button className="w-16 h-16 flex items-center justify-center rounded-full bg-red-600 hover:bg-red-...
    <Square className="h-7 w-7 text-white fill-white" />
  </button>
  ) : isPaused ? (
    <button className="w-16 h-16 flex items-center justify-center rounded-full shrink-0" style={{ outli...
    <Play className="h-7 w-7 text-white fill-white ml-0.5" />
  </button>
  ) : (
    <button
      className="w-16 h-16 flex items-center justify-center rounded-full shrink-0"
      style={{ outline: '2px solid ${waveColor}88', outlineOffset: '3px', background: waveColor, boxSha...
      onClick={startReading}
      disabled={isLoading || numPages === 0}
      data-testid="button-play"
    >
      {isLoading ? <Loader2 className="h-7 w-7 text-white animate-spin" /> : <Play className="h-7 w-7 t...
    </button>
  )}
</div>

<div className="flex-1 flex items-center justify-evenly">
  <button className="w-12 h-12 flex items-center justify-center rounded-full hover:bg-white/10 disabled...
    <SkipForward className="h-6 w-6 text-white" />
  </button>

  <button className="w-12 h-12 flex items-center justify-center rounded-full hover:bg-white/10 disabled...
    <span className="text-[9px] text-white font-medium mr-0.5">15s</span>
    <RotateCw className="h-5 w-5 text-white" />
  </button>

  <button className="w-12 h-12 flex flex-col items-center justify-center rounded-full hover:bg-white/10...
    if (catWashFollow && (followState?.active || catWashPaused)) {
      fetch("/api/webhook/voice-command", { method: "POST", headers: { 'Content-Type': 'applicati...
      .then(() => { setFollowState(null); setCatWashPaused(false); })
      .catch(e => console.error('Failed to stop:', e));
    } else {
      stopReading();
    }
  } disabled={!isPlaying && !(catWashFollow && followState?.active) && !catWashPaused} data-testid="bu...
  <Square className="h-5 w-5 text-white fill-white" />
  <span className="text-[8px] text-white/70 leading-none mt-0.5">Stop</span>
</button>
</div>
</div>
</div>

<div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', padding: '6px 12px', ...
  <div className="flex items-center gap-1.5 flex-shrink-0" data-testid="voice-selector">
    <span style={{ fontSize: '9px', color: 'rgba(255,255,255,0.6)', textTransform: 'uppercase', letterSpaci...

```

```

<select
  value={voice}
  onChange={(e) => { const v = e.target.value as Voice; setVoice(v); voiceRef.current = v; localStorage...
  style={{ background: 'rgba(255,255,255,0.1)', color: 'white', fontSize: '11px', borderRadius: '4px', ...
  data-testid="select-voice"
>
  {[["alloy", "ash", "echo", "fable", "onyx"] as Voice[]].map(v => (
    <option key={v} value={v} className="bg-gray-900 text-white">{VOICE_LABELS[v]}</option>
  ))}
</select>
<button
  style={{ padding: '3px', borderRadius: '50%', border: '1px solid rgba(255,255,255,0.2)', background: ...
  onClick={previewVoice}
  data-testid="button-preview-voice"
  title="Preview voice"
>
  {isPreviewing ? <Square className="h-3 w-3 text-white fill-white" /> : <Volume2 className="h-3 w-3 te...
</button>
</div>

<div style={{ width: '1px', height: '20px', background: 'rgba(255,255,255,0.15)', flexShrink: 0 }} />

<div className="flex items-center gap-1.5 flex-shrink-0" data-testid="speed-control">
  <span style={{ fontSize: '9px', color: 'rgba(255,255,255,0.6)', textTransform: 'uppercase', letterSpaci...
  <button
    style={{ width: '18px', height: '18px', display: 'flex', alignItems: 'center', justifyContent: 'cente...
    onClick={() => setPlaybackSpeed(Math.max(0.5, +(playbackSpeed - 0.1).toFixed(2)))}
    data-testid="button-speed-down"
  > </button>
  <input
    type="range" min="0.5" max="3" step="0.1" value={playbackSpeed}
    onChange={(e) => setPlaybackSpeed(parseFloat(e.target.value))}
    style={{ width: '60px', height: '3px', cursor: 'pointer', accentColor: 'white' }}
    data-testid="slider-speed"
  />
  <button
    style={{ width: '18px', height: '18px', display: 'flex', alignItems: 'center', justifyContent: 'cente...
    onClick={() => setPlaybackSpeed(Math.min(3, +(playbackSpeed + 0.1).toFixed(2)))}
    data-testid="button-speed-up"
  > </button>
  <span style={{ fontSize: '11px', color: 'white', fontWeight: 600, minWidth: '28px', textAlign: 'center'...
</div>

<div style={{ width: '1px', height: '20px', background: 'rgba(255,255,255,0.15)', flexShrink: 0 }} />

<div className="flex items-center gap-1.5 flex-shrink-0" data-testid="volume-control">
  <span style={{ fontSize: '9px', color: 'rgba(255,255,255,0.6)', textTransform: 'uppercase', letterSpaci...
  <button
    style={{ width: '18px', height: '18px', display: 'flex', alignItems: 'center', justifyContent: 'cente...
    onClick={() => setVolume(Math.max(0, +(volume - 0.1).toFixed(2)))}
    data-testid="button-volume-down"
  > </button>
  <input
    type="range" min="0" max="1" step="0.05" value={volume}
    onChange={(e) => setVolume(parseFloat(e.target.value))}
    style={{ width: '60px', height: '3px', cursor: 'pointer', accentColor: 'white' }}
    data-testid="slider-volume"
  />
  <button
    style={{ width: '18px', height: '18px', display: 'flex', alignItems: 'center', justifyContent: 'cente...
    onClick={() => setVolume(Math.min(1, +(volume + 0.1).toFixed(2)))}
    data-testid="button-volume-up"
  > </button>
  <Volume2 className="h-3.5 w-3.5 text-white" />
</div>

<div style={{ width: '1px', height: '20px', background: 'rgba(255,255,255,0.15)', flexShrink: 0 }} />

<button
  style={{ padding: '3px 10px', borderRadius: '4px', background: '#059669', fontSize: '10px', color: 'whi...
  onClick={async () => {
    try {
      const testBtn = document.querySelector('[data-testid="button-test-audio"]');
      if (testBtn) testBtn.textContent = 'Playing...';
      const res = await fetch('/api/tts', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ text: 'Audio test successful. If you can hear this through the Echo speake...
      });
      if (!res.ok) throw new Error('TTS failed');
      const blob = await res.blob();
      const url = URL.createObjectURL(blob);
      const testAudio = new Audio(url);
      testAudio.volume = volumeRef.current;
      testAudio.onended = () => {
        URL.revokeObjectURL(url);
        const btn = document.querySelector('[data-testid="button-test-audio"]');
        if (btn) btn.textContent = 'Test';
      };
    } catch {
      const btn = document.querySelector('[data-testid="button-test-audio"]');
      if (btn) btn.textContent = 'Failed';
      setTimeout(() => { if (btn) btn.textContent = 'Test'; }, 2000);
    }
  }}
  data-testid="button-test-audio"
>Test</button>
</div>

{allFiles.length > 1 && (() => {
  const allChunkProgress = JSON.parse(localStorage.getItem('allChunkProgress') || '{}');
  let totalFilesComplete = 0;
  let totalChunksAll = 0;
  let totalCheckedAll = 0;
  for (const f of allFiles) {
    const fileKey = `onedrive_${btoa(f.downloadUrl).slice(0, 40)}`;
    const progress = allChunkProgress[fileKey];
  }
})

```

```
`client/src/pages/pdf-viewer.tsx`
```

```

const handlePageInputSubmit = useCallback(() => {
  const p = parseInt(pageInput);
  if (!isNaN(p)) goToPage(p);
  else setPageInput(String(currentPage));
}, [pageInput, currentPage, goToPage]);

const handleSearch = useCallback(() => {
  if (!searchQuery.trim()) {
    setSearchResults([]);
    return;
  }
  const query = searchQuery.toLowerCase();
  const results: { page: number; text: string }[] = [];
  pdfText.forEach((text, page) => {
    if (text.toLowerCase().includes(query)) {
      const idx = text.toLowerCase().indexOf(query);
      const start = Math.max(0, idx - 40);
      const end = Math.min(text.length, idx + query.length + 40);
      const snippet = (start > 0 ? "... " : "") + text.slice(start, end) + (end < text.length ? "..." : "");
      results.push({ page, text: snippet });
    }
  });
  results.sort((a, b) => a.page - b.page);
  setSearchResults(results);
  setCurrentSearchIndex(0);
  if (results.length > 0) goToPage(results[0].page);
}, [searchQuery, pdfText, goToPage]);

const navigateSearchResult = useCallback((direction: 1 | -1) => {
  if (searchResults.length === 0) return;
  const next = (currentSearchIndex + direction + searchResults.length) % searchResults.length;
  setCurrentSearchIndex(next);
  goToPage(searchResults[next].page);
}, [searchResults, currentSearchIndex, goToPage]);

useEffect(() => {
  const handleKeyDown = (e: KeyboardEvent) => {
    if (e.target === searchInputRef.current || e.target === pageInputRef.current) {
      if (e.key === "Enter") {
        if (e.target === searchInputRef.current) handleSearch();
        if (e.target === pageInputRef.current) handlePageInputSubmit();
      }
      if (e.key === "Escape") {
        setSearchOpen(false);
        setSearchResults([]);
      }
    }
    return;
  }
  if (e.key === "ArrowLeft" || e.key === "ArrowUp") {
    e.preventDefault();
    goToPage(currentPage - 1);
  }
  else if (e.key === "ArrowRight" || e.key === "ArrowDown" || e.key === " ") {
    e.preventDefault();
    goToPage(currentPage + 1);
  }
  else if ((e.ctrlKey || e.metaKey) && e.key === "f") {
    e.preventDefault();
    setSearchOpen(true);
    setTimeout(() => searchInputRef.current?.focus(), 100);
  }
};
window.addEventListener("keydown", handleKeyDown);
return () => window.removeEventListener("keydown", handleKeyDown);
}, [currentPage, goToPage, handleSearch, handlePageInputSubmit]);

useEffect(() => {
  if (searchOpen) setTimeout(() => searchInputRef.current?.focus(), 100);
}, [searchOpen]);

if (!filePath) {
  return (
    <div style={{ display: "flex", alignItems: "center", justifyContent: "center", height: "100vh", background: "#1...
      No file specified
    </div>
  );
}

return (
  <div style={{ display: "flex", flexDirection: "column", height: "100vh", background: "#2d2d2d", color: "white", o...
    <div style={{ display: "flex", alignItems: "center", gap: "8px", padding: "6px 12px", background: "#1a1a1a", bo...
      <button
        onClick={() => window.history.back()}
        style={{ background: "none", border: "none", color: "#ccc", cursor: "pointer", padding: "4px", display: "fl...
        title="Go back"
        data-testid="btn-back"
      >
      <ArrowLeft size={18} />
    </button>

    <div style={{ width: "1px", height: "20px", background: "#555" }} />

    <button
      onClick={() => goToPage(currentPage - 1)}
      disabled={currentPage <= 1}
      style={{ background: "none", border: "none", color: currentPage <= 1 ? "#555" : "#ccc", cursor: currentPage...
      title="Previous page"
      data-testid="btn-prev-page"
    >
    <ChevronLeft size={20} />
  </button>

  <div style={{ display: "flex", alignItems: "center", gap: "4px", fontSize: "13px" }}>
    <input
      ref={pageInputRef}
      type="text"
      value={pageInput}
      onChange={(e) => setPageInput(e.target.value)}
      onBlur={handlePageInputSubmit}
      onKeyDown={(e) => { if (e.key === "Enter") handlePageInputSubmit(); }}

```

```

        style={{ width: "40px", textAlign: "center", background: "#333", border: "1px solid #555", borderRadius: ...
        data-testid="input-page-number"
    } />
    <span style={{ color: "#999" }}>of {numPages}</span>
</div>

<button
  onClick={() => goToPage(currentPage + 1)}
  disabled={currentPage >= numPages}
  style={{ background: "none", border: "none", color: currentPage >= numPages ? "#555" : "#ccc", cursor: curr...
  title="Next page"
  data-testid="btn-next-page"
>
  <ChevronRight size={20} />
</button>

<div style={{ width: "1px", height: "20px", background: "#555" }} />

<button
  onClick={() => { setSearchOpen(!searchOpen); if (!searchOpen) setTimeout(() => searchInputRef.current?.focu...
  style={{ background: searchOpen ? "#444" : "none", border: "none", color: "#ccc", cursor: "pointer", paddin...
  title="Search (Ctrl+F)"
  data-testid="btn-search-toggle"
>
  <Search size={16} />
</button>

{searchOpen && (
  <div style={{ display: "flex", alignItems: "center", gap: "4px", flex: 1, maxWidth: "400px" }}>
    <input
      ref={searchInputRef}
      type="text"
      value={searchQuery}
      onChange={(e) => setSearchQuery(e.target.value)}
      onKeyDown={(e) => {
        if (e.key === "Enter") {
          e.preventDefault();
          if (e.shiftKey) navigateSearchResult(-1);
          else if (searchResults.length > 0 && searchQuery === searchQuery) navigateSearchResult(1);
          else handleSearch();
        }
      }}
      placeholder="Search in document..."
      style={{ flex: 1, background: "#333", border: "1px solid #555", borderRadius: "3px", color: "white", pa...
      data-testid="input-search"
    />
    <button
      onClick={handleSearch}
      style={{ background: "#4a90d9", border: "none", color: "white", cursor: "pointer", padding: "3px 10px",...
      data-testid="btn-search-submit"
    >
      Find
    </button>
    {searchResults.length > 0 && (
      <div>
        <span style={{ fontSize: "12px", color: "#999", whiteSpace: "nowrap" }}>{currentSearchIndex + 1}/{sea...
        <button onClick={() => navigateSearchResult(-1)} style={{ background: "none", border: "none", color: ...
        <button onClick={() => navigateSearchResult(1)} style={{ background: "none", border: "none", color: "...
      </div>
    )}
    {searchResults.length === 0 && searchQuery && pdfText.size > 0 && (
      <span style={{ fontSize: "12px", color: "#f87171" }}>No results</span>
    )}
    <button
      onClick={() => { setSearchOpen(false); setSearchQuery(""); setSearchResults([]); }}
      style={{ background: "none", border: "none", color: "#999", cursor: "pointer", padding: "2px" }}
      data-testid="btn-search-close"
    >
      <X size={14} />
    </button>
  </div>
))

<div style={{ flex: 1 }} />

<div style={{ display: "flex", alignItems: "center", gap: "6px", fontSize: "12px" }}>
  <button onClick={() => setScale(s => Math.max(0.5, s - 0.2))} style={{ background: "#333", border: "1px sol...
  <span style={{ color: "#999", minWidth: "40px", textAlign: "center" }}>{Math.round(scale * 100)}%</span>
  <button onClick={() => setScale(s => Math.min(3, s + 0.2))} style={{ background: "#333", border: "1px solid...
</div>
</div>

{searchResults.length > 0 && (
  <div style={{ background: "#2a2a3a", borderBottom: "1px solid #444", padding: "4px 12px", maxHeight: "100px",...
    {searchResults.map((r, i) => (
      <div
        key={i}
        onClick={() => { setCurrentSearchIndex(i); goToPage(r.page); }}
        style={{
          padding: "3px 8px",
          fontSize: "11px",
          cursor: "pointer",
          borderRadius: "3px",
          background: i === currentSearchIndex ? "#4a90d9" : "transparent",
          color: i === currentSearchIndex ? "white" : "#aaa",
        }}
        data-testid={`search-result-${i}`}
      >
        <strong>Page {r.page}</strong> {r.text}
      </div>
    ))}
  </div>
))
</div>

<div ref={containerRef} style={{ flex: 1, overflow: "auto", display: "flex", justifyContent: "center", padding:...
  {loading && (
    <div style={{ display: "flex", alignItems: "center", justifyContent: "center", height: "100%" }}>
      <div style={{ color: "#999", fontSize: "16px" }}>Loading PDF...</div>
    </div>
  )}
</div>

```



```

    </div>
  })
  {error && (
    <div style={{ display: "flex", alignItems: "center", justifyContent: "center", height: "100%" }}>
      <div style={{ color: "#f87171", fontSize: "14px", textAlign: "center", padding: "20px" }}>
        Failed to load PDF<br /><span style={{ fontSize: "12px", color: "#999" }}>{error}</span>
      </div>
    </div>
  })
  {pdfUrl && (
    <Document
      file={pdfUrl}
      onLoadSuccess={onDocumentLoadSuccess}
      onLoadError={onDocumentLoadError}
      loading=""
    >
      <div style={{ position: "relative" }}>
        <div style={{
          position: "absolute", top: "8px", right: "8px", background: "rgba(0,0,0,0.6)",
          color: "white", padding: "2px 8px", borderRadius: "4px", fontSize: "11px", zIndex: 10
        }}>
          Page {currentPage} of {numPages}
        </div>
        <Page
          pageNumber={currentPage}
          scale={scale}
          renderTextLayer={true}
          renderAnnotationLayer={true}
        />
      </div>
    </Document>
  )}
</div>
</div>
);
}

```

`client/src/pages/projects.tsx`

```

import { useState, useMemo, useEffect } from "react";
import { useQuery, useMutation } from "@tanstack/react-query";
import { Link } from "wouter";
import { queryClient, apiRequest } from "@lib/queryClient";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Textarea } from "@components/ui/textarea";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger, DialogFooter } from "@components/ui/dialog";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
import { Badge } from "@components/ui/badge";
import { Progress } from "@components/ui/progress";
import { useToast } from "@hooks/use-toast";
import {
  Plus,
  FolderOpen,
  Calendar,
  CheckCircle2,
  Clock,
  Trash2,
  Pencil,
  ArrowLeft,
  Target,
  PauseCircle,
  XCircle,
  Lightbulb,
  LayoutGrid,
  LayoutList,
  ChevronDown,
  ChevronUp,
  GitBranch,
  ArrowRight,
  Lock,
  Unlock
} from "lucide-react";
import { format } from "date-fns";
import type { Project, Task, TaskLink } from "@shared/schema";

const PROJECT_STATUSES = ["planning", "in_progress", "on_hold", "completed", "cancelled"] as const;
const PRIORITIES = ["low", "medium", "high"] as const;
const COURSE_COLORS: Record<string, string> = {
  CPPA122: "#22C55E",
  CFNF400: "#EC4899",
  CASL101: "#6366F1",
};

function getStatusIcon(status: string) {
  switch (status) {
    case "planning": return <Lightbulb className="w-4 h-4" />;
    case "in_progress": return <Clock className="w-4 h-4" />;
    case "on_hold": return <PauseCircle className="w-4 h-4" />;
    case "completed": return <CheckCircle2 className="w-4 h-4" />;
    case "cancelled": return <XCircle className="w-4 h-4" />;
    default: return <FolderOpen className="w-4 h-4" />;
  }
}

function getStatusColor(status: string) {
  switch (status) {
    case "planning": return "bg-blue-100 text-blue-800 dark:bg-blue-900/40 dark:text-blue-200";
    case "in_progress": return "bg-yellow-100 text-yellow-800 dark:bg-yellow-900/40 dark:text-yellow-200";
    case "on_hold": return "bg-gray-100 text-gray-800 dark:bg-gray-800/40 dark:text-gray-200";
    case "completed": return "bg-green-100 text-green-800 dark:bg-green-900/40 dark:text-green-200";
    case "cancelled": return "bg-red-100 text-red-800 dark:bg-red-900/40 dark:text-red-200";
    default: return "bg-gray-100 text-gray-800 dark:bg-gray-800 dark:text-gray-200";
  }
}

```

```

    }
  }

function getPriorityColor(priority: string) {
  switch (priority) {
    case "high": return "bg-red-100 text-red-800 dark:bg-red-900/40 dark:text-red-200";
    case "medium": return "bg-orange-100 text-orange-800 dark:bg-orange-900/40 dark:text-orange-200";
    case "low": return "bg-green-100 text-green-800 dark:bg-green-900/40 dark:text-green-200";
    default: return "bg-gray-100 text-gray-800";
  }
}

interface ProjectFormData {
  name: string;
  description: string;
  color: string;
  status: string;
  courseName: string;
  startDate: string;
  targetDate: string;
  priority: string;
  notes: string;
}

function ProjectDialog({
  project,
  open,
  onOpenChange,
  onSave
}): {
  project?: Project | null;
  open: boolean;
  onOpenChange: (open: boolean) => void;
  onSave: (data: ProjectFormData) => void;
} {
  const [formData, setFormData] = useState<ProjectFormData>({
    name: project?.name || "",
    description: project?.description || "",
    color: project?.color || "#6366F1",
    status: project?.status || "planning",
    courseName: project?.courseName || "",
    startDate: project?.startDate ? format(new Date(project.startDate), "yyyy-MM-dd") : "",
    targetDate: project?.targetDate ? format(new Date(project.targetDate), "yyyy-MM-dd") : "",
    priority: project?.priority || "medium",
    notes: project?.notes || ""
  });

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    onSave(formData);
    onOpenChange(false);
  };

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="max-w-lg text-[11px] text-white [&_label]:text-white [&_input]:text-black [&_input]:b...
        <DialogHeader>
          <DialogTitle className="text-white text-sm">{project ? "Edit Project" : "Create New Project"}</DialogTitle>
        </DialogHeader>
        <form onSubmit={handleSubmit} className="space-y-3">
          <div className="space-y-2">
            <label className="text-[11px] font-medium text-white">Project Name</label>
            <Input
              data-testid="input-project-name"
              value={formData.name}
              onChange={(e) => setFormData({ ...formData, name: e.target.value })}
              placeholder="Enter project name"
              required
            />
          </div>

          <div className="space-y-2">
            <label className="text-[11px] font-medium">Description</label>
            <Textarea
              data-testid="input-project-description"
              value={formData.description}
              onChange={(e) => setFormData({ ...formData, description: e.target.value })}
              placeholder="Describe your project..."
              rows={3}
            />
          </div>

          <div className="grid grid-cols-2 gap-4">
            <div className="space-y-2">
              <label className="text-[11px] font-medium">Color</label>
              <div className="flex gap-2 items-center">
                <input
                  type="color"
                  data-testid="input-project-color"
                  value={formData.color}
                  onChange={(e) => setFormData({ ...formData, color: e.target.value })}
                  className="h-9 w-14 rounded-md border cursor-pointer"
                />
                <Input
                  value={formData.color}
                  onChange={(e) => setFormData({ ...formData, color: e.target.value })}
                  className="flex-1"
                />
              </div>
            </div>

            <div className="space-y-2">
              <label className="text-[11px] font-medium">Status</label>
              <Select
                value={formData.status}
                onValueChange={(v) => setFormData({ ...formData, status: v })}
              >
                <SelectTrigger data-testid="select-project-status" className="bg-white text-black text-[11px] [&span...

```

```

        <SelectValue />
      </SelectTrigger>
      <SelectContent>
        {PROJECT_STATUSES.map((s) => (
          <SelectItem key={s} value={s}>
            {s.replace("_", " ").replace(/\b\w/g, l => l.toUpperCase())}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
  </div>
</div>

<div className="grid grid-cols-2 gap-4">
  <div className="space-y-2">
    <label className="text-[11px] font-medium">Course (optional)</label>
    <Select
      value={formData.courseName || "none"}
      onChange={(v) => setFormData({ ...formData, courseName: v === "none" ? "" : v })}
    >
      <SelectTrigger data-testid="select-project-course" className="bg-white text-black text-[11px] [&span...
        <SelectValue placeholder="Select course" />
      </SelectTrigger>
      <SelectContent>
        <SelectItem value="none">No Course</SelectItem>
        <SelectItem value="CPPI122">CPPI122</SelectItem>
        <SelectItem value="CFNF400">CFNF400</SelectItem>
        <SelectItem value="CASL101">CASL101</SelectItem>
      </SelectContent>
    </Select>
  </div>

  <div className="space-y-2">
    <label className="text-[11px] font-medium">Priority</label>
    <Select
      value={formData.priority}
      onChange={(v) => setFormData({ ...formData, priority: v })}
    >
      <SelectTrigger data-testid="select-project-priority" className="bg-white text-black text-[11px] [&sp...
        <SelectValue />
      </SelectTrigger>
      <SelectContent>
        {PRIORITIES.map((p) => (
          <SelectItem key={p} value={p}>
            {p.charAt(0).toUpperCase() + p.slice(1)}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
  </div>
</div>

<div className="grid grid-cols-2 gap-4">
  <div className="space-y-2">
    <label className="text-[11px] font-medium">Start Date</label>
    <Input
      type="date"
      data-testid="input-project-start-date"
      value={formData.startDate}
      onChange={(e) => setFormData({ ...formData, startDate: e.target.value })}
    />
  </div>
  <div className="space-y-2">
    <label className="text-[11px] font-medium">Target Date</label>
    <Input
      type="date"
      data-testid="input-project-target-date"
      value={formData.targetDate}
      onChange={(e) => setFormData({ ...formData, targetDate: e.target.value })}
    />
  </div>
</div>

<div className="space-y-2">
  <label className="text-[11px] font-medium">Notes</label>
  <Textarea
    data-testid="input-project-notes"
    value={formData.notes}
    onChange={(e) => setFormData({ ...formData, notes: e.target.value })}
    placeholder="Additional notes..."
    rows={2}
  />
</div>

<DialogFooter>
  <Button type="button" variant="outline" onClick={() => onOpenChange(false)} className="border-white/30 te...
    Cancel
  </Button>
  <Button
    type="submit"
    variant="outline"
    data-testid="button-save-project"
    className="border !border-blue-500 text-white hover:text-white hover:!border-blue-400 hover:bg-transpar...
  >
    {project ? "Save Changes" : "Create Project"}
  </Button>
</DialogFooter>
</form>
</DialogContent>
</Dialog>
);
}

function ProjectCard({
  project,
  tasks,
  onEdit,
  onDelete,

```

```

expanded,
onToggleExpand,
headerBarColor,
onRename
}): {
  project: Project;
  tasks: Task[];
  onEdit: () => void;
  onDelete: () => void;
  expanded: boolean;
  onToggleExpand: () => void;
  headerBarColor: string;
  onRename: (name: string) => void;
}) {
  const completedTasks = tasks.filter(t => t.isCompleted);
  const progress = tasks.length > 0 ? Math.round((completedTasks.length / tasks.length) * 100) : 0;
  const [isEditingName, setIsEditingName] = useState(false);
  const [editName, setEditName] = useState(project.name);

  return (
    <div
      data-testid={`card-project-${project.id}`}
      className="rounded-[12px] overflow-hidden flex flex-col hover-elevate transition-all"
      style={{
        background: 'linear-gradient(180deg, rgba(255,255,255,0.45) 0%, rgba(255,255,255,0.25) 50%, rgba(255,255,255,...
        backdropFilter: 'blur(40px)',
        WebkitBackdropFilter: 'blur(40px)',
        border: '1px solid rgba(255,255,255,0.5)',
        borderTop: '1px solid rgba(255,255,255,0.7)',
        boxShadow: '0 8px 32px rgba(0,0,0,0.08), 0 2px 8px rgba(0,0,0,0.04), inset 0 1px 0 rgba(255,255,255,0.6), ins...
      }}
    >
      { /* Header - matching Today box header */ }
      <div
        style={{
          backgroundColor: headerBarColor,
          padding: '8px 12px'
        }}
      >
        <div className="flex items-center justify-between gap-2">
          <div className="flex items-center gap-2 flex-1 min-w-0">
            <div
              className="w-3 h-3 rounded-full flex-shrink-0 border border-white/30"
              style={{ backgroundColor: project.color || "#6366F1" }}
            />
            {isEditingName ? (
              <input
                className="text-xs font-medium text-white bg-white/20 border border-white/40 rounded px-1 py-0.5 outl...
                style={{ fontFamily: "Avenir, 'Avenir Next', -apple-system, BlinkMacSystemFont, sans-serif" }}
                value={editName}
                onChange={(e) => setEditName(e.target.value)}
                onBlur={() => {
                  if (editName.trim() && editName.trim() !== project.name) {
                    onRename(editName.trim());
                  } else {
                    setEditName(project.name);
                  }
                  setIsEditingName(false);
                }}
                onKeyDown={(e) => {
                  if (e.key === 'Enter') {
                    (e.target as HTMLInputElement).blur();
                  } else if (e.key === 'Escape') {
                    setEditName(project.name);
                    setIsEditingName(false);
                  }
                }}
                autoFocus
                data-testid={`input-rename-project-${project.id}`}
              />
            ) : (
              <span
                className="text-xs font-medium text-white truncate cursor-text"
                style={{ fontFamily: "Avenir, 'Avenir Next', -apple-system, BlinkMacSystemFont, sans-serif", textShad...
                onClick={(e) => { e.stopPropagation(); setEditName(project.name); setIsEditingName(true); }}
                data-testid={`text-project-name-${project.id}`}
              >
                {project.name}
              </span>
            )
          </div>
          <div className="flex items-center gap-0.5 flex-shrink-0">
            <Button
              size="icon"
              variant="ghost"
              onClick={onEdit}
              className="h-7 w-7 text-white hover:text-white hover:bg-white/20"
              data-testid={`button-edit-project-${project.id}`}
            >
              <Pencil className="w-3.5 h-3.5" />
            </Button>
            <Button
              size="icon"
              variant="ghost"
              onClick={onDelete}
              className="h-7 w-7 text-white hover:text-white hover:bg-white/20"
              data-testid={`button-delete-project-${project.id}`}
            >
              <Trash2 className="w-3.5 h-3.5" />
            </Button>
          </div>
        </div>
      </div>
      { /* Body */ }
      <div className="flex-1 p-3">
        {project.description && (
          <p className="text-xs text-white/80 mb-3 line-clamp-2">

```

```

    {project.description}
  </p>
})
<div className="flex flex-wrap items-center gap-2 mb-3">
  <Badge className={` ${getStatusColor(project.status || "planning")} text-xs`} >
    {getStatusIcon(project.status || "planning")}
    <span className="ml-1">{(project.status || "planning").replace("_", " ")}</span>
  </Badge>
  <Badge className={` ${getPriorityColor(project.priority || "medium")} text-xs`} >
    {(project.priority || "medium").toUpperCase()}
  </Badge>
  {project.courseName && (
    <Badge
      className="text-xs"
      style={{
        backgroundColor: COURSE_COLORS[project.courseName] || "#6366F1",
        color: "white"
      }}
    >
      {project.courseName}
    </Badge>
  )}
</div>

<div className="flex items-center gap-4 text-xs text-white/70 mb-3">
  {project.startDate && (
    <div className="flex items-center gap-1">
      <Calendar className="w-3 h-3" />
      <span>Start: {format(new Date(project.startDate), "MMM d")}</span>
    </div>
  )}
  {project.targetDate && (
    <div className="flex items-center gap-1">
      <Target className="w-3 h-3" />
      <span>Due: {format(new Date(project.targetDate), "MMM d")}</span>
    </div>
  )}
</div>

<div className="space-y-1 mb-3">
  <div className="flex items-center justify-between text-xs text-white/80">
    <span>Progress</span>
    <span className="font-medium">{completedTasks.length}/{tasks.length} tasks</span>
  </div>
  <Progress value={progress} className="h-2" />
</div>

<Button
  variant="ghost"
  size="sm"
  onClick={onToggleExpand}
  className="w-full justify-center gap-1 text-white hover:text-white hover:bg-white/20 text-xs"
  data-testid={`button-toggle-tasks-${project.id}`}
>
  {expanded ? (
    <>
      <ChevronUp className="w-3.5 h-3.5" />
      Hide Tasks
    </>
  ) : (
    <>
      <ChevronDown className="w-3.5 h-3.5" />
      Show Tasks {tasks.length}
    </>
  )}
</Button>

{expanded && tasks.length > 0 && (
  <div className="space-y-1 pt-2 mt-2 border-t border-white/30">
    {tasks.map((task) => (
      <div
        key={task.id}
        className={`flex items-center gap-2 p-2 rounded-md text-xs ${
          task.isCompleted
            ? "bg-green-500/20 line-through text-white/60"
            : "bg-white/20 text-white"
        }`}
        data-testid={`task-item-${task.id}`}
      >
        {task.isCompleted ? (
          <CheckCircle2 className="w-3.5 h-3.5 text-green-400 flex-shrink-0" />
        ) : (
          <div className="w-3.5 h-3.5 rounded-full border-2 border-white/50 flex-shrink-0" />
        )}
        <span className="flex-1 truncate">{task.title}</span>
        {task.dueDate && (
          <span className="text-[10px] text-white/60">
            {format(new Date(task.dueDate), "MMM d")}
          </span>
        )}
      </div>
    )}
  </div>
)}
</div>
);
}

function WorkflowView({
  project,
  tasks,
  taskLinks
}): {
  project: Project;
  tasks: Task[];
  taskLinks: TaskLink[];

```



```

        {task.courseName}
      </Badge>
    ))
  </div>

  {blockedByTasks.length > 0 && (
    <div className="mt-2 flex flex-wrap items-center gap-1 text-xs">
      <span className="text-orange-600 dark:text-orange-400 flex items-center gap-1">
        <Lock className="w-3 h-3" /> Blocked by:
      </span>
      {blockedByTasks.map(t => (
        <Badge
          key={t!.id}
          variant="outline"
          className={`text-[10px] ${t!.isCompleted ? 'line-through opacity-50' : 'border-orange-300...`}
        >
          {t!.title}
        </Badge>
      ))}
    </div>
  )}

  {blocksTasks.length > 0 && (
    <div className="mt-1 flex flex-wrap items-center gap-1 text-xs">
      <span className="text-blue-600 dark:text-blue-400 flex items-center gap-1">
        <ArrowRight className="w-3 h-3" /> Blocks:
      </span>
      {blocksTasks.map(t => (
        <Badge
          key={t!.id}
          variant="outline"
          className={`text-[10px] ${t!.isCompleted ? 'line-through opacity-50' : 'border-blue-300'}`}
        >
          {t!.title}
        </Badge>
      ))}
    </div>
  )}
</div>
</div>
</CardContent>
</Card>
</div>
);
}}
</div>
}
);
}

```

```

export default function ProjectsPage() {
  const { toast } = useToast();
  const savedColors = useMemo(() => {
    try {
      const saved = localStorage.getItem('colorSettings');
      if (saved) {
        const parsed = JSON.parse(saved);
        return {
          headerBar: parsed.headerBar || '#160502',
          mainBackground: parsed.mainBackground || '#3a8bbf',
          mainBackgroundGradient: parsed.mainBackgroundGradient ?? true,
          mainBackgroundGradientEnd: parsed.mainBackgroundGradientEnd || '#164a72',
        };
      }
    } catch {}
    return { headerBar: '#160502', mainBackground: '#3a8bbf', mainBackgroundGradient: true, mainBackgroundGradientEnd...
  }, []);
  const headerBarColor = savedColors.headerBar;
  const [dialogOpen, setDialogOpen] = useState(false);
  const [editingProject, setEditingProject] = useState<Project | null>(null);
  const [expandedProjects, setExpandedProjects] = useState<Set<number>>(new Set());
  const [viewMode, setViewMode] = useState<"grid" | "list" | "workflow">("grid");
  const [statusFilter, setStatusFilter] = useState<string>("all");
  const [selectedProjectId, setSelectedProjectId] = useState<number | null>(null);

  const { data: projects = [], isLoading: projectsLoading } = useQuery<Project[]>({
    queryKey: ["/api/projects"],
  });

  const { data: allTasks = [] } = useQuery<Task[]>({
    queryKey: ["/api/tasks"],
  });

  const { data: allTaskLinks = [] } = useQuery<TaskLink[]>({
    queryKey: ["/api/links"],
    enabled: viewMode === "workflow",
  });

  const tasksByProject = useMemo(() => {
    const map = new Map<number, Task[]>();
    allTasks.forEach(task => {
      if (task.projectId) {
        const existing = map.get(task.projectId) || [];
        existing.push(task);
        map.set(task.projectId, existing);
      }
    });
    return map;
  }, [allTasks]);

  const filteredProjects = useMemo(() => {
    if (statusFilter === "all") return projects;
    return projects.filter(p => p.status === statusFilter);
  }, [projects, statusFilter]);

  const createProjectMutation = useMutation({
    mutationFn: async (data: ProjectFormData) => {
      return await apiRequest("POST", "/api/projects", data);
    },
  },

```

```

    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["/api/projects"] });
      toast({ title: "Project created successfully" });
    },
    onError: () => {
      toast({ title: "Failed to create project", variant: "destructive" });
    },
  });

const updateProjectMutation = useMutation({
  mutationFn: async ({ id, data }: { id: number; data: ProjectFormData }) => {
    return await apiRequest("PATCH", `/api/projects/${id}`, data);
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/projects"] });
    toast({ title: "Project updated successfully" });
  },
  onError: () => {
    toast({ title: "Failed to update project", variant: "destructive" });
  },
});

const deleteProjectMutation = useMutation({
  mutationFn: async (id: number) => {
    return await apiRequest("DELETE", `/api/projects/${id}`);
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/projects"] });
    toast({ title: "Project deleted successfully" });
  },
  onError: () => {
    toast({ title: "Failed to delete project", variant: "destructive" });
  },
});

const handleSaveProject = (data: ProjectFormData) => {
  if (editingProject) {
    updateProjectMutation.mutate({ id: editingProject.id, data });
  } else {
    createProjectMutation.mutate(data);
  }
  setEditingProject(null);
  setDialogOpen(false);
};

const handleEditProject = (project: Project) => {
  setEditingProject(project);
  setDialogOpen(true);
};

const handleDeleteProject = (id: number) => {
  if (window.confirm("Are you sure you want to delete this project? Tasks will be unlinked but not deleted. ")) {
    deleteProjectMutation.mutate(id);
  }
};

const toggleProjectExpanded = (id: number) => {
  setExpandedProjects(prev => {
    const next = new Set(prev);
    if (next.has(id)) {
      next.delete(id);
    } else {
      next.add(id);
    }
    return next;
  });
};

const projectStats = useMemo(() => {
  const stats = {
    total: projects.length,
    planning: 0,
    in_progress: 0,
    on_hold: 0,
    completed: 0,
    cancelled: 0,
  };
  projects.forEach(p => {
    const status = p.status as keyof typeof stats;
    if (status in stats) stats[status]++;
  });
  return stats;
}, [projects]);

const overallProgress = useMemo(() => {
  const tasksWithProjects = allTasks.filter(t => t.projectId);
  const completedTasks = tasksWithProjects.filter(t => t.isCompleted);
  const totalTasks = tasksWithProjects.length;
  const completionRate = totalTasks > 0 ? Math.round((completedTasks.length / totalTasks) * 100) : 0;

  const byPriority = {
    high: { total: 0, completed: 0 },
    medium: { total: 0, completed: 0 },
    low: { total: 0, completed: 0 },
  };

  projects.forEach(p => {
    const tasks = tasksByProject.get(p.id) || [];
    const priority = (p.priority || 'medium') as keyof typeof byPriority;
    if (priority in byPriority) {
      byPriority[priority].total += tasks.length;
      byPriority[priority].completed += tasks.filter(t => t.isCompleted).length;
    }
  });

  return {
    totalTasks,
    completedTasks: completedTasks.length,
  };
});

```



```

        completionRate,
        pendingTasks: totalTasks - completedTasks.length,
        byPriority,
    });
}, [allTasks, projects, tasksByProject]);

return (
  <div
    className="min-h-screen"
    style={{
      background: savedColors.mainBackgroundGradient
        ? `linear-gradient(180deg, ${savedColors.mainBackground} 0%, ${savedColors.mainBackgroundGradientEnd} 100%)`
        : savedColors.mainBackground,
      backgroundAttachment: 'fixed',
      fontFamily: 'Avenir, 'Avenir Next', -apple-system, BlinkMacSystemFont, sans-serif"
    }}
  >
    <header className="border-b border-white/20 top-0 z-10 backdrop-blur-sm" style={{ backgroundColor: saved...
      <div className="container mx-auto px-4 py-4">
        <div className="flex items-center justify-between gap-4 flex-wrap">
          <div className="flex items-center gap-4">
            <Link href="/">
              <div
                style={{
                  width: '44px',
                  height: '44px',
                  borderRadius: '50%',
                  background: 'linear-gradient(0deg, #042550 0%, #4578B0 100%)',
                  padding: '1px',
                  cursor: 'pointer'
                }}
                data-testid="button-back-home"
              >
                <Button variant="ghost" size="icon" className="!h-[42px] !w-[42px] !min-h-[42px] !min-w-[42px] !p-0...
                  background: 'linear-gradient(180deg, #042550 0%, #4578B0 100%)',
                  boxShadow: 'inset 0 1px 2px rgba(255,255,255,0.3), inset 0 -1px 2px rgba(0,0,0,0.3), 0 2px 4px rg...
                >>
                <ArrowLeft className="w-5 h-5 text-white" />
              </Button>
            </div>
          </Link>
        </div>
        <h1 className="text-base font-bold flex items-center gap-1.5 text-white">
          <FolderOpen className="w-3.5 h-3.5" />
          Projects
        </h1>
        <p className="text-sm text-white/70">
          {projects.length} projects, {allTasks.filter(t => t.projectId).length} tasks assigned
        </p>
      </div>
    </div>

    <div className="flex items-center gap-2">
      <div className="flex rounded-md">
        <Button
          variant={viewMode === "grid" ? "secondary" : "ghost"}
          size="sm"
          onClick={() => setViewMode("grid")}
          data-testid="button-view-grid"
          style={{
            background: 'linear-gradient(to bottom, #042550, #4578B0)',
            boxShadow: '0 0 8px rgba(59, 130, 246, 0.5), 0 0 16px rgba(59, 130, 246, 0.3), inset 0 1px 0 rgba...
          }}
          className="text-white hover:text-white border border-blue-400/50 rounded-md"
        >
          <LayoutGrid className="w-4 h-4" />
        </Button>
        <Button
          variant={viewMode === "list" ? "secondary" : "ghost"}
          size="sm"
          onClick={() => setViewMode("list")}
          data-testid="button-view-list"
          className="text-white hover:text-white hover:bg-white/10"
        >
          <LayoutList className="w-4 h-4" />
        </Button>
        <Button
          variant={viewMode === "workflow" ? "secondary" : "ghost"}
          size="sm"
          onClick={() => setViewMode("workflow")}
          data-testid="button-view-workflow"
          className="text-white hover:text-white hover:bg-white/10"
        >
          <GitBranch className="w-4 h-4" />
        </Button>
      </div>

      <Button
        variant="outline"
        onClick={() => { setEditingProject(null); setDialogOpen(true); }}
        data-testid="button-create-project"
        className="border !border-blue-500 text-white hover:text-white hover:!border-blue-400 hover:bg-transp...
      >
        <Plus className="w-4 h-4 mr-2" />
        New Project
      </Button>
    </div>
  </div>
</div>
</header>

<main className="container mx-auto px-4 py-6">
  <div className="grid grid-cols-2 md:grid-cols-5 gap-6 mb-6">
    <div
      className={cursor-pointer hover:elevate rounded-[12px] p-4 text-center ${statusFilter === "all" ? "bg-wh...
      onClick={() => setStatusFilter("all")}
      data-testid="filter-all"
      style={{

```

```

        background: 'rgba(255, 255, 255, 0.35)',
        border: '1px solid rgba(255, 255, 255, 0.4)',
        backdropFilter: 'blur(8px)',
        WebkitBackdropFilter: 'blur(8px)'
    })
}
>
<div className="text-2xl font-bold text-white" data-testid="stat-total">{projectStats.total}</div>
<div className="text-sm text-white/70">All Projects</div>
</div>
<div
  className={`cursor-pointer hover-elevate rounded-[12px] p-4 text-center ${statusFilter === "in_progress" ...
  onClick={() => setStatusFilter("in_progress")}}
  data-testid="filter-in-progress"
  style={{
    background: 'rgba(255, 255, 255, 0.35)',
    border: '1px solid rgba(255, 255, 255, 0.4)',
    backdropFilter: 'blur(8px)',
    WebkitBackdropFilter: 'blur(8px)'
  }}
>
<div className="text-2xl font-bold text-yellow-300" data-testid="stat-in-progress">{projectStats.in_progr...
<div className="text-sm text-white/70">In Progress</div>
</div>
<div
  className={`cursor-pointer hover-elevate rounded-[12px] p-4 text-center ${statusFilter === "planning" ? "...
  onClick={() => setStatusFilter("planning")}}
  data-testid="filter-planning"
  style={{
    background: 'rgba(255, 255, 255, 0.35)',
    border: '1px solid rgba(255, 255, 255, 0.4)',
    backdropFilter: 'blur(8px)',
    WebkitBackdropFilter: 'blur(8px)'
  }}
>
<div className="text-2xl font-bold text-blue-300" data-testid="stat-planning">{projectStats.planning}</div>
<div className="text-sm text-white/70">Planning</div>
</div>
<div
  className={`cursor-pointer hover-elevate rounded-[12px] p-4 text-center ${statusFilter === "completed" ? ...
  onClick={() => setStatusFilter("completed")}}
  data-testid="filter-completed"
  style={{
    background: 'rgba(255, 255, 255, 0.35)',
    border: '1px solid rgba(255, 255, 255, 0.4)',
    backdropFilter: 'blur(8px)',
    WebkitBackdropFilter: 'blur(8px)'
  }}
>
<div className="text-2xl font-bold text-green-300" data-testid="stat-completed">{projectStats.completed}<...
<div className="text-sm text-white/70">Completed</div>
</div>
<div
  className={`cursor-pointer hover-elevate rounded-[12px] p-4 text-center ${statusFilter === "on_hold" ? "b...
  onClick={() => setStatusFilter("on_hold")}}
  data-testid="filter-on-hold"
  style={{
    background: 'rgba(255, 255, 255, 0.35)',
    border: '1px solid rgba(255, 255, 255, 0.4)',
    backdropFilter: 'blur(8px)',
    WebkitBackdropFilter: 'blur(8px)'
  }}
>
<div className="text-2xl font-bold text-gray-300" data-testid="stat-on-hold">{projectStats.on_hold}</div>
<div className="text-sm text-white/70">On Hold</div>
</div>
</div>
{overallProgress.totalTasks > 0 && (
  <Card className="mb-6">
    <CardHeader className="pb-2">
      <CardTitle className="text-base flex items-center gap-2">
        <Target className="w-4 h-4" />
        Overall Progress
      </CardTitle>
    </CardHeader>
    <CardContent>
      <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
        <div className="text-center">
          <div className="text-3xl font-bold text-primary">{overallProgress.completionRate}</div>
          <div className="text-sm text-muted-foreground">Completion Rate</div>
          <Progress value={overallProgress.completionRate} className="h-2 mt-2" />
        </div>
        <div className="text-center">
          <div className="text-3xl font-bold text-green-600">{overallProgress.completedTasks}</div>
          <div className="text-sm text-muted-foreground">Completed Tasks</div>
        </div>
        <div className="text-center">
          <div className="text-3xl font-bold text-orange-600">{overallProgress.pendingTasks}</div>
          <div className="text-sm text-muted-foreground">Pending Tasks</div>
        </div>
        <div className="text-center">
          <div className="text-3xl font-bold">{overallProgress.totalTasks}</div>
          <div className="text-sm text-muted-foreground">Total Tasks</div>
        </div>
      </div>
      <div className="grid grid-cols-3 gap-3 mt-4 pt-4 border-t">
        <div className="text-center">
          <div className="flex items-center justify-center gap-1 text-sm font-medium text-red-600">
            High Priority
          </div>
          <div className="text-lg font-bold">
            {overallProgress.byPriority.high.completed}/{overallProgress.byPriority.high.total}
          </div>
          <Progress
            value={overallProgress.byPriority.high.total > 0
              ? (overallProgress.byPriority.high.completed / overallProgress.byPriority.high.total) * 100
              : 0}
          />
        </div>
      </div>
    </CardContent>
  </Card>
)

```

```

        className="h-1.5 mt-1"
      />
    </div>
    <div className="text-center">
      <div className="flex items-center justify-center gap-1 text-sm font-medium text-orange-600">
        Medium Priority
      </div>
      <div className="text-lg font-bold">
        {overallProgress.byPriority.medium.completed}/{overallProgress.byPriority.medium.total}
      </div>
      <Progress
        value={overallProgress.byPriority.medium.total > 0
          ? (overallProgress.byPriority.medium.completed / overallProgress.byPriority.medium.total) * 100
          : 0}
        className="h-1.5 mt-1"
      />
    </div>
    <div className="text-center">
      <div className="flex items-center justify-center gap-1 text-sm font-medium text-green-600">
        Low Priority
      </div>
      <div className="text-lg font-bold">
        {overallProgress.byPriority.low.completed}/{overallProgress.byPriority.low.total}
      </div>
      <Progress
        value={overallProgress.byPriority.low.total > 0
          ? (overallProgress.byPriority.low.completed / overallProgress.byPriority.low.total) * 100
          : 0}
        className="h-1.5 mt-1"
      />
    </div>
  </div>
</CardContent>
</Card>
})

{projectsLoading ? (
  <div className="flex items-center justify-center py-12">
    <div className="animate-spin rounded-full h-8 w-8 border-b-2 border-primary"></div>
  </div>
) : filteredProjects.length === 0 ? (
  <div className="text-center py-12">
    <FolderOpen className="w-16 h-16 mx-auto text-muted-foreground/30 mb-4" />
    <h3 className="text-lg font-medium mb-2">
      {statusFilter === "all" ? "No projects yet" : `No ${statusFilter.replace("_", " ")} projects`}
    </h3>
    <p className="text-muted-foreground mb-4">
      Create a project to organize your tasks and track progress.
    </p>
    <Button onClick={() => { setEditingProject(null); setDialogOpen(true); }}>
      <Plus className="w-4 h-4 mr-2" />
      Create Your First Project
    </Button>
  </div>
) : viewModel === "workflow" ? (
  <div className="space-y-6">
    <div className="flex items-center gap-3 mb-4">
      <Select
        value={selectedProjectId?.toString() || ""}
        onChange={(v) => setSelectedProjectId(v ? Number(v) : null)}
      >
        <SelectTrigger className="w-[280px]" data-testid="select-workflow-project">
          <SelectValue placeholder="Select a project to view workflow" />
        </SelectTrigger>
        <SelectContent>
          {projects.map((p) => (
            <SelectItem key={p.id} value={p.id.toString()}>
              <div className="flex items-center gap-2">
                <div
                  className="w-2.5 h-2.5 rounded-full"
                  style={{ backgroundColor: p.color || "#6366F1" }}
                />
                {p.name}
              </div>
            </SelectItem>
          ))}
        </SelectContent>
      </Select>
      <Badge className={getStatusColor(projects.find(p => p.id === selectedProjectId)?.status || "planning")}>
        {(projects.find(p => p.id === selectedProjectId)?.status || "planning").replace("_", " ")}
      </Badge>
    </div>
    <div>
      {selectedProjectId ? (
        <WorkflowView
          project={projects.find(p => p.id === selectedProjectId)!}
          tasks={tasksByProject.get(selectedProjectId) || []}
          taskLinks={allTaskLinks}
        />
      ) : (
        <div className="text-center py-12 text-muted-foreground">
          <GitBranch className="w-16 h-16 mx-auto mb-4 opacity-30" />
          <p>Select a project above to view its task workflow and dependencies.</p>
        </div>
      )}
    </div>
  </div>
) : (
  <div className={viewModel === "grid"
    ? "grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8"
    : "space-y-8"}
  >
    {filteredProjects.map((project) => (
      <ProjectCard
        key={project.id}
        project={project}
        tasks={tasksByProject.get(project.id) || []}
      >

```

```

        onEdit={() => handleEditProject(project)}
        onDelete={() => handleDeleteProject(project.id)}
        expanded={expandedProjects.has(project.id)}
        onToggleExpand={() => toggleProjectExpanded(project.id)}
        headerBarColor={headerBarColor}
        onRename={(name) => updateProjectMutation.mutate({ id: project.id, data: { name } })}
      />
    ))}
  </div>
)
</main>

<ProjectDialog
  project={editingProject}
  open={dialogOpen}
  onOpenChange={() => {
    setDialogOpen(open);
    if (!open) setEditingProject(null);
  }}
  onSave={handleSaveProject}
/>
</div>
);
}

```

`client/src/pages/spotify-player.tsx`

```

import { useState, useEffect, useRef, useCallback } from "react";
import {
  Play, Pause, SkipForward, SkipBack, Music2, Loader2,
  ChevronLeft, ChevronDown, Shuffle, Repeat, Volume2, VolumeX, Speaker,
  Search, Radio, Menu, X, Home, Zap, Wifi, Star, Volume1,
  Bed, Bath, DoorOpen, CookingPot, Sofa, Crown, ShirtIcon, Globe2,
  Sun, Cast, Monitor, Square,
} from "lucide-react";
import floorplanImg from "@assets/Floorplan11_1774005505273.png";
import massBg from "@assets/mass-background2_1774005959332.png";
import musicBg from "@assets/Music_BG20_1774006032495.png";
import hallwayNight from "@assets/Hallway_Night_1774008796945.png";
import kingNight from "@assets/King_Night_1774008796946.png";
import kitchenNight from "@assets/Kitchen_Night_1774008796947.png";
import livingRoomNight from "@assets/Living_Room_Night_1774008796948.png";
import pugNight from "@assets/Pug_Night_1774008796949.png";
import queenNight from "@assets/Queen_Night_1774008796950.png";
import catNight from "@assets/Cat_Night_1774008796951.png";
import closetNight from "@assets/Closet_Night_1774008796953.png";
import echoSpeakerImg from "@assets/Echo_1774213902054.png";
import chumfmLogo from "@assets/chumfm_logo.svg";
import yasuPhoto from "@assets/Yasu_1774341165517.png";
import brynPhoto from "@assets/Bryn_1774341392117.png";
import jalLogo from "@assets/jal_logo.png";

interface NowPlaying {
  playing: boolean; name?: string; artist?: string; album?: string;
  albumArt?: string; albumArtSmall?: string; progress?: number; duration?: number;
}

interface RoomGroup {
  room: string; icon: string;
  speakers: { id: string; name: string; entityId: string; type: string; room: string }[];
}

interface ProfileArtist {
  name: string; uri: string; image?: string; searchQuery: string;
}

type ProfileKey = "bryn" | "yasu" | "guest";

interface StationShortcut {
  name: string; command: string; uri?: string; icon?: string; image?: string;
}

const STATION_SHORTCUTS: StationShortcut[] = [
  { name: "Gay FM", command: "play Gay FM Radio on tunein on the everywhere group", icon: "0=00" },
  { name: "Vibe of Vegas", command: "play the Vibe of Vegas Radio on tunein on the everywhere group", icon: "0<B=" },
  { name: "Dinner Jazz", command: "play Dinner Jazz music on Spotify on the everywhere group", icon: "0<B." },
  { name: "Chill Electronic", command: "play Chill Electronic station on Spotify on the everywhere group", icon: "0<B$" },
  { name: "CHUM FM", command: "play 104.5 Chum FM", icon: "0=Uá" },
  { name: "Spring Cleaning", command: "play spring cleaning music on Spotify on the everywhere group", icon: "0<B8" },
  { name: "Pink", command: "", uri: "spotify:playlist:37i9dQZF1DZ06ev00YT088", icon: "0=0-" },
  { name: "Easy Listening", command: "play easy listening on Spotify on the everywhere group", icon: "0<Bµ" },
  { name: "Katy Perry", command: "", uri: "spotify:playlist:37i9dQZF1DZ06ev03Jefw4", icon: "0<Bα" },
  { name: "Disney", command: "play Disney songs on Spotify on the everywhere group", icon: "0<Bð" },
  { name: "Club Riva", command: "play Club Riva Lounge Radio on tunein on the everywhere group", icon: "0<Bx" },
  { name: "Samui Island", command: "play Samui Island Radio on tunein", icon: "0<B4" },
  { name: "Calm My Cat", command: "Enable Calm My Cat everywhere", icon: "0=01" },
];

const PROFILES: Record<ProfileKey, {
  label: string; artists: ProfileArtist[]; theme: string; accent: string; glow: string;
}> = {
  bryn: {
    label: "Bryn",
    theme: "neon",
    accent: "#3b82f6",
    glow: "rgba(59,130,246,0.3)",
    artists: [
      { name: "Katy Perry", uri: "spotify:artist:6jJ0s89eD6GaHleKKya26X", searchQuery: "Katy Perry" },
      { name: "Pink", uri: "spotify:artist:1KCSPY1glIKqW2TotWuX0R", searchQuery: "Pink singer" },
      { name: "Lady Gaga", uri: "spotify:artist:1HY2Jd0NmPumShAr6KMms", searchQuery: "Lady Gaga" },
      { name: "Cher", uri: "spotify:artist:720aDtakiy6yFqkt4TsiFt", searchQuery: "Cher" },
      { name: "This Is Me", uri: "spotify:track:2MYDnXBdJkFRuWgyOjpdth", searchQuery: "This Is Me Greatest Showman" },
      { name: "CHUM FM", uri: "", searchQuery: "104.5 Chum FM", image: chumfmLogo },
      { name: "Disney", uri: "spotify:playlist:37i9dQZF1DX8C585qnMYHP", searchQuery: "Disney hits" },
      { name: "Chill Electro", uri: "spotify:playlist:37i9dQZF1DX4E3UdUs7fUx", searchQuery: "Chill electronic" },
    ],
  },
};

```

```

    { name: "Dinner Jazz", uri: "spotify:playlist:37i9dQZF1DX4wta20PHgwo", searchQuery: "Dinner jazz" },
  ],
},
yasu: {
  label: "Yasu",
  theme: "sakura",
  accent: "#38bdf8",
  glow: "rgba(56,189,248,0.35)",
  artists: [
    { name: "N-\\00*0†0M", uri: "spotify:artist:7IKFMPUXJDZkXFGY0awBo", searchQuery: "Miyuki Nakajima" },
    { name: "YQASOBI", uri: "spotify:artist:64tJ2EAv1R6UaZqc4i0Cyj", searchQuery: "YQASOBI" },
    { name: "Kenshi Yonezu", uri: "spotify:artist:1snhtMLeb2DY0M0cVkiKnR", searchQuery: "Kenshi Yonezu" },
    { name: "Aimyon", uri: "spotify:artist:5Lak6GhYbSghRimRYhE0dP", searchQuery: "Aimyon" },
    { name: "ONE OK ROCK", uri: "spotify:artist:7q4KJIqziJOKnsTaFKpMII", searchQuery: "ONE OK ROCK" },
    { name: "Official HIGE DANDISM", uri: "spotify:artist:3YmVszTadghiHjP0YaG3PM", searchQuery: "Official HIGE DAND..." },
    { name: "Hikaru Utada", uri: "spotify:artist:7lbSsjYACZHN1MNXOk2fAz", searchQuery: "Hikaru Utada" },
    { name: "LiSA", uri: "spotify:artist:0blbVefux0GltDBa00hcmS", searchQuery: "LiSA" },
    { name: "Mrs. GREEN APPLE", uri: "spotify:artist:4qujR4RMTEMz5E5hPjHvTk", searchQuery: "Mrs. GREEN APPLE" },
    { name: "Pink", uri: "spotify:artist:1KCSpy1gLIKqW2TotWuXOR", searchQuery: "Pink" },
    { name: "Katy Perry", uri: "spotify:artist:7n2Ycct7Beij7Dj7meI4X0", searchQuery: "Katy Perry" },
    { name: "JAL Boarding Music", uri: "spotify:track:5cIqGKfJqhH0BnAoCp3xqj", searchQuery: "JAL boarding music I w..." },
  ],
},
guest: {
  label: "Guest",
  theme: "neon",
  accent: "#a78bfa",
  glow: "rgba(167,139,250,0.3)",
  artists: [
    { name: "Dua Lipa", uri: "spotify:artist:6M2wZ9GZgrQXHCffjv46we", searchQuery: "Dua Lipa" },
    { name: "The Weeknd", uri: "spotify:artist:1Xyo4u8uXc1ZmMpatF05Pj", searchQuery: "The Weeknd" },
    { name: "Taylor Swift", uri: "spotify:artist:06HL4z0CvFAXyc27GxpF02", searchQuery: "Taylor Swift" },
    { name: "Ed Sheeran", uri: "spotify:artist:6eUKZxaKkcviH0Ku9w2n3v", searchQuery: "Ed Sheeran" },
    { name: "Billie Eilish", uri: "spotify:artist:6qqNVTkY8uBg9cP3Jd7DAH", searchQuery: "Billie Eilish" },
    { name: "Harry Styles", uri: "spotify:artist:6KImCVD70vtIoJWnq6nGn3", searchQuery: "Harry Styles" },
    { name: "Doja Cat", uri: "spotify:artist:5cj0LLjcoR7Y0SnhnX0Po5", searchQuery: "Doja Cat" },
    { name: "SZA", uri: "spotify:artist:7tYKF4w9nC0nq9CsP2zThyP", searchQuery: "SZA" },
  ],
},
};

const ROOM_JP: Record<string, string> = {
  "Balcony": "0000000000",
  "Queen Bedroom": "00000000000000",
  "Pug Washroom": "000000000000",
  "Hallway": "0000000000",
  "Kitchen": "0000000000",
  "Living Room": "0000000000",
  "King Bedroom": "00000000000000",
  "Cat Washroom": "s+m-bb00",
  "Closet": "000000000000",
  "Everywhere": "Qh00",
};

const ROOM_HOTSPOTS: { room: string; x: number; y: number; w: number; h: number; entityId: string; groupEntityId: str...
  { room: "Balcony", x: 2, y: 72, w: 18, h: 25, entityId: "media_player.balcony_speaker", groupEntityId: "media_playe...
  { room: "Queen Bedroom", x: 2, y: 38, w: 18, h: 33, entityId: "media_player.queen_bedroom", groupEntityId: "media_p...
  { room: "Pug Washroom", x: 2, y: 5, w: 16, h: 32, entityId: "media_player.echo_show_pug_am", groupEntityId: "media_...
  { room: "Hallway", x: 19, y: 5, w: 16, h: 32, entityId: "media_player.hallway_2", groupEntityId: "media_player.hall...
  { room: "Kitchen", x: 36, y: 5, w: 28, h: 45, entityId: "media_player.kitchen_lr", groupEntityId: "media_player.kit...
  { room: "Living Room", x: 36, y: 52, w: 28, h: 45, entityId: "media_player.kitchen_lr", groupEntityId: "media_playe...
  { room: "King Bedroom", x: 65, y: 30, w: 33, h: 50, entityId: "media_player.king_bedroom", groupEntityId: "media_pl...
  { room: "Cat Washroom", x: 84, y: 3, w: 14, h: 26, entityId: "media_player.cat_speakers", groupEntityId: "media_pla...
  { room: "Closet", x: 65, y: 3, w: 18, entityId: "media_player.echo_closet_am", groupEntityId: "media_player....
  { room: "Everywhere", x: 84, y: 78, w: 14, h: 18, entityId: "media_player.byhome", groupEntityId: "media_player.byh...
};

type ViewMode = "floor" | "stations" | "rooms";

function formatMs(ms: number) {
  const s = Math.floor(ms / 1000), m = Math.floor(s / 60);
  return `${m}:${s % 60}.toString().padStart(2, "0")`;
}

const ROOM_SLIDER_COLORS: Record<string, string> = {
  "Queen Bedroom": "#e84393",
  "Pug Washroom": "#00b4d8",
  "Hallway": "#6c5ce7",
  "Kitchen": "#fdbc6e",
  "Living Room": "#0984e3",
  "King Bedroom": "#e17055",
  "Cat Washroom": "#00cece9",
  "Closet": "#a29bfe",
  "Everywhere": "#ff6b6b",
  "Balcony": "#55efc4",
};

function VolumeKnob({ value, onChange, size = 54, accent = "#382f6", glow = "rgba(59,130,246,0.3)", roomName }: { va...
  const trackRef = useRef<HTMLDivElement>(null);
  const dragging = useRef(false);
  const sliderColor = roomName ? (ROOM_SLIDER_COLORS[roomName] || accent) : accent;
  const trackWidth = 90;
  const thumbSize = 16;

  const valueFromEvent = (e: { clientX: number }) => {
    if (!trackRef.current) return value;
    const rect = trackRef.current.getBoundingClientRect();
    const x = e.clientX - rect.left;
    const pct = Math.round(Math.max(0, Math.min(100, (x / rect.width) * 100)));
    return pct;
  };

  const onPointerDown = (e: React.PointerEvent) => {
    e.stopPropagation(); e.preventDefault();
    dragging.current = true;
    (e.target as HTMLElement).setPointerCapture(e.pointerId);
    onChange(valueFromEvent(e));
  };
};

```

```

const onPointerMove = (e: React.PointerEvent) => {
  if (!dragging.current) return;
  e.stopPropagation();
  onChange(valueFromEvent(e));
};
const onPointerUp = (e: React.PointerEvent) => {
  dragging.current = false;
  (e.target as HTMLElement).releasePointerCapture(e.pointerId);
};

return (
  <div className="flex flex-col items-center gap-1" style={{ touchAction: 'none' }}>
    <div className="flex items-center gap-1.5">
      <button onClick={(e) => { e.stopPropagation(); onChange(Math.max(0, value - 3)); }}
        className="text-white font-bold text-[11px] w-4 h-4 flex items-center justify-center rounded-full transiti...
        style={{ textShadow: '0 0 4px rgba(255,255,255,0.5)' }}
        data-testid="volume-minus"></button>
      <div ref={trackRef} className="relative cursor-pointer"
        style={{ width: trackWidth, height: 3 }}
        onPointerDown={onPointerDown} onPointerMove={onPointerMove} onPointerUp={onPointerUp}>
        <div className="absolute inset-0 rounded-full"
          style={{
            background: 'rgba(0,0,0,0.6)',
            boxShadow: 'inset 0 2px 4px rgba(0,0,0,0.8), inset 0 -1px 2px rgba(255,255,255,0.05)',
            border: '1px solid rgba(255,255,255,0.06)',
          }} />
        <div className="absolute top-0 left-0 h-full rounded-full"
          style={{
            width: `${value}%`,
            background: `linear-gradient(90deg, ${sliderColor}cc, ${sliderColor})`,
            boxShadow: `0 0 8px ${sliderColor}60, 0 0 2px ${sliderColor}40`,
            transition: dragging.current ? 'none' : 'width 0.1s ease',
          }} />
        <div className="absolute top-1/2"
          style={{
            left: `calc(${value}% - ${thumbSize} / 2)px`,
            transform: 'translateY(-50%)',
            width: thumbSize,
            height: thumbSize,
            borderRadius: '50%',
            background: `radial-gradient(circle at 40% 35%, rgba(60,60,70,1), rgba(25,25,35,1))`,
            boxShadow: `0 2px 8px rgba(0,0,0,0.7), 0 0 12px ${sliderColor}40, inset 0 1px 1px rgba(255,255,255,0.15)`,
            border: '1px solid rgba(255,255,255,0.1)',
            transition: dragging.current ? 'none' : 'left 0.1s ease',
          }}>
          <div style={{
            position: 'absolute', top: '50%', left: '50%',
            transform: 'translate(-50%, -50%)',
            width: 5, height: 5, borderRadius: '50%',
            background: sliderColor,
            boxShadow: `0 0 6px ${sliderColor}`,
          }} />
        </div>
      </div>
      <button onClick={(e) => { e.stopPropagation(); onChange(Math.min(100, value + 3)); }}
        className="text-white font-bold text-[11px] w-4 h-4 flex items-center justify-center rounded-full transiti...
        style={{ textShadow: '0 0 4px rgba(255,255,255,0.5)' }}
        data-testid="volume-plus">+</button>
    </div>
    <span className="text-[9px] font-bold" style={{ color: 'rgba(255,255,255,0.85)', textShadow: '0 0 4px rgba(0,0,...
  </div>
);
}

```

```

function CherryBlossoms() {
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const animRef = useRef<number>();

  useEffect(() => {
    const canvas = canvasRef.current;
    if (!canvas) return;
    const ctx = canvas.getContext("2d");
    if (!ctx) return;
    const W = canvas.offsetWidth, H = canvas.offsetHeight;
    canvas.width = W * 2; canvas.height = H * 2;
    ctx.scale(2, 2);

    const petalColors = [
      [255, 183, 197], [255, 192, 210], [255, 175, 195],
      [255, 200, 215], [249, 170, 190], [255, 210, 220],
      [255, 218, 228], [252, 165, 185], [255, 205, 218],
    ];

    interface Petal {
      x: number; y: number; r: number; vx: number; vy: number; vr: number;
      alpha: number; size: number; color: number[]; delay: number;
      wobbleSpeed: number; wobbleAmp: number; flutter: number;
      curl: number; tilt3d: number; tiltSpeed: number;
      shape: number;
    }
  }

```

```

const petals: Petal[] = [];
for (let i = 0; i < 70; i++) {
  petals.push({
    x: Math.random() * W, y: Math.random() * H * 2 - H,
    r: Math.random() * Math.PI * 2,
    vx: (Math.random() - 0.3) * 0.5, vy: 0.15 + Math.random() * 0.55,
    vr: (Math.random() - 0.5) * 0.018,
    alpha: 0.45 + Math.random() * 0.45,
    size: 4 + Math.random() * 10,
    color: petalColors[Math.floor(Math.random() * petalColors.length)],
    delay: Math.random() * 6000,
    wobbleSpeed: 2000 + Math.random() * 2500,
    wobbleAmp: 0.4 + Math.random() * 0.8,
    flutter: Math.random() * 0.015,
    curl: 0.2 + Math.random() * 0.6,
    tilt3d: Math.random() * Math.PI * 2,
    tiltSpeed: 800 + Math.random() * 1200,
  });
}

```

```

    shape: Math.floor(Math.random() * 3),
  });
}

const drawSinglePetal = (ctx: CanvasRenderingContext2D, s: number, cr: number, cg: number, cb: number, curl: numb...
  ctx.beginPath();
  ctx.moveTo(0, 0);
  ctx.bezierCurveTo(s * 0.4, -s * 0.6, s * 0.9, -s * 0.5, s * 0.5, -s * 0.05);
  ctx.bezierCurveTo(s * 0.9, s * (0.4 + curl * 0.2), s * 0.4, s * 0.55, 0, 0);
  const grad = ctx.createLinearGradient(0, -s * 0.5, s * 0.3, s * 0.3);
  grad.addColorStop(0, `rgba(${cr},${cg},${cb},0.92)`);
  grad.addColorStop(0.5, `rgba(${Math.min(cr+15,255)},${Math.min(cg+10,255)},${Math.min(cb+10,255)},0.85)`);
  grad.addColorStop(1, `rgba(${cr-15},${cg-10},${cb-8},0.75)`);
  ctx.fillStyle = grad;
  ctx.fill();
  ctx.beginPath();
  ctx.moveTo(0, 0);
  ctx.quadraticCurveTo(s * 0.35, -s * 0.15, s * 0.5, -s * 0.05);
  ctx.strokeStyle = `rgba(${cr-30},${cg-25},${cb-20},0.25)`;
  ctx.lineWidth = 0.4;
  ctx.stroke();
};

const drawPetal = (ctx: CanvasRenderingContext2D, p: Petal, t: number) => {
  ctx.save();
  ctx.translate(p.x, p.y);
  ctx.rotate(p.r);
  const tilt = Math.sin((t + p.delay) / p.tiltSpeed) * 0.5;
  ctx.scale(1, 0.4 + Math.abs(Math.cos(tilt)) * 0.6);
  ctx.globalAlpha = p.alpha * (0.7 + Math.abs(Math.cos(tilt)) * 0.3);

  const [cr, cg, cb] = p.color;
  const s = p.size;

  if (p.shape === 0) {
    drawSinglePetal(ctx, s, cr, cg, cb, p.curl);
  } else if (p.shape === 1) {
    ctx.save();
    drawSinglePetal(ctx, s, cr, cg, cb, p.curl);
    ctx.rotate(Math.PI * 0.4);
    drawSinglePetal(ctx, s * 0.85, cr, cg, cb, p.curl);
    ctx.restore();
  } else {
    for (let i = 0; i < 5; i++) {
      ctx.save();
      ctx.rotate((i / 5) * Math.PI * 2);
      drawSinglePetal(ctx, s * 0.7, cr, cg, cb, p.curl);
      ctx.restore();
    }
    ctx.beginPath();
    ctx.arc(0, 0, s * 0.1, 0, Math.PI * 2);
    ctx.fillStyle = `rgba(255,225,180,0.9)`;
    ctx.fill();
    ctx.beginPath();
    ctx.arc(0, 0, s * 0.06, 0, Math.PI * 2);
    ctx.fillStyle = `rgba(240,180,120,0.7)`;
    ctx.fill();
  }

  ctx.globalAlpha = p.alpha * 0.15;
  ctx.beginPath();
  ctx.arc(0, 0, s * 1.5, 0, Math.PI * 2);
  const glow = ctx.createRadialGradient(0, 0, 0, 0, 0, 0, s * 1.5);
  glow.addColorStop(0, `rgba(${cr},${cg},${cb},0.2)`);
  glow.addColorStop(1, `rgba(${cr},${cg},${cb},0)`);
  ctx.fillStyle = glow;
  ctx.fill();

  ctx.restore();
};

const animate = () => {
  const t = Date.now();
  ctx.clearRect(0, 0, W, H);
  for (const p of petals) {
    const windGust = Math.sin(t / 8000) * 0.3 + Math.sin(t / 3000 + p.delay) * 0.15;
    p.x += p.vx + Math.sin((t + p.delay) / p.wobbleSpeed) * p.wobbleAmp + windGust;
    p.y += p.vy + Math.sin((t + p.delay) / 2500) * 0.1;
    p.r += p.vr + Math.sin((t + p.delay) / 1800) * p.flutter;
    if (p.y > H + 30) { p.y = -30; p.x = Math.random() * W; }
    if (p.x > W + 40) p.x = -40;
    if (p.x < -40) p.x = W + 40;
    drawPetal(ctx, p, t);
  }
  animRef.current = requestAnimationFrame(animate);
};
animate();
return () => { if (animRef.current) cancelAnimationFrame(animRef.current); };
}, []);

return <canvas ref={canvasRef} className="absolute inset-0 w-full h-full pointer-events-none" style={{ zIndex: 40 }}...
}

function JapaneseWaves({ accent }: { accent: string }) {
  return (
    <div className="absolute bottom-0 left-0 right-0 pointer-events-none overflow-hidden" style={{ height: 80, zIndex...
      <svg viewBox="0 0 1920 80" preserveAspectRatio="none" className="w-full h-full">
        <defs>
          <pattern id="seigaiha" x="0" y="0" width="60" height="30" patternUnits="userSpaceOnUse">
            <path d="M30 30 Q30 0 0 0" fill="none" stroke={accent} strokeWidth="0.8" opacity="0.6" />
            <path d="M30 30 Q30 0 60 0" fill="none" stroke={accent} strokeWidth="0.8" opacity="0.6" />
            <path d="M30 30 Q30 5 5 5" fill="none" stroke={accent} strokeWidth="0.5" opacity="0.4" />
            <path d="M30 30 Q30 5 55 5" fill="none" stroke={accent} strokeWidth="0.5" opacity="0.4" />
            <path d="M30 30 Q30 10 10 10" fill="none" stroke={accent} strokeWidth="0.4" opacity="0.3" />
            <path d="M30 30 Q30 10 50 10" fill="none" stroke={accent} strokeWidth="0.4" opacity="0.3" />
          </pattern>
        </defs>
        <rect width="1920" height="80" fill="url(#seigaiha)" />
      </svg>
    )
  )
}

```

```

    </svg>
  </div>
});
}

function HoloCircuitLines({ accent, sakura = false }: { accent: string; sakura?: boolean }) {
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const animRef = useRef<number>();

  useEffect(() => {
    const canvas = canvasRef.current;
    if (!canvas) return;
    const ctx = canvas.getContext("2d");
    if (!ctx) return;
    const W = canvas.offsetWidth, H = canvas.offsetHeight;
    canvas.width = W * 2; canvas.height = H * 2;
    ctx.scale(2, 2);

    interface CircuitPath { points: { x: number; y: number }[]; speed: number; offset: number }
    const paths: CircuitPath[] = [];
    for (let i = 0; i < 30; i++) {
      const pts: { x: number; y: number }[] = [];
      let cx = Math.random() * W, cy = Math.random() * H;
      pts.push({ x: cx, y: cy });
      const segs = 3 + Math.floor(Math.random() * 5);
      for (let s = 0; s < segs; s++) {
        if (Math.random() > 0.5) cx += (Math.random() - 0.3) * 200;
        else cy += (Math.random() - 0.3) * 150;
        cx = Math.max(0, Math.min(W, cx));
        cy = Math.max(0, Math.min(H, cy));
        pts.push({ x: cx, y: cy });
      }
      paths.push({ points: pts, speed: 0.3 + Math.random() * 1.2, offset: Math.random() * Math.PI * 2 });
    }

    const nodes: { x: number; y: number; pulseOffset: number; size: number }[] = [];
    for (let i = 0; i < 40; i++) {
      nodes.push({ x: Math.random() * W, y: Math.random() * H, pulseOffset: Math.random() * Math.PI * 2, size: 1 + Ma...
    }

    const junctions: { x: number; y: number; size: number; pulseOffset: number }[] = [];
    for (let i = 0; i < 15; i++) {
      junctions.push({ x: Math.random() * W, y: Math.random() * H, size: 3 + Math.random() * 4, pulseOffset: Math.ran...
    }

    const animate = () => {
      ctx.clearRect(0, 0, W, H);
      const t = Date.now() / 1000;

      for (const path of paths) {
        const alpha = 0.1 + Math.sin(t * path.speed + path.offset) * 0.06;
        ctx.strokeStyle = `${accent}${Math.round(alpha * 255).toString(16).padStart(2, '0')}`;
        ctx.lineWidth = 0.8;
        ctx.beginPath();
        for (let p = 0; p < path.points.length; p++) {
          if (p === 0) ctx.moveTo(path.points[p].x, path.points[p].y);
          else ctx.lineTo(path.points[p].x, path.points[p].y);
        }
        ctx.stroke();

        let totalLen = 0;
        for (let p = 1; p < path.points.length; p++) {
          totalLen += Math.hypot(path.points[p].x - path.points[p-1].x, path.points[p].y - path.points[p-1].y);
        }
        const travelDist = ((t * path.speed * 40 + path.offset * 100) % totalLen);
        let accum = 0;
        for (let p = 1; p < path.points.length; p++) {
          const segLen = Math.hypot(path.points[p].x - path.points[p-1].x, path.points[p].y - path.points[p-1].y);
          if (accum + segLen >= travelDist) {
            const frac = (travelDist - accum) / segLen;
            const px = path.points[p-1].x + (path.points[p].x - path.points[p-1].x) * frac;
            const py = path.points[p-1].y + (path.points[p].y - path.points[p-1].y) * frac;
            ctx.beginPath();
            ctx.arc(px, py, 2.5, 0, Math.PI * 2);
            ctx.fillStyle = `${accent}60`;
            ctx.fill();
            const grd = ctx.createRadialGradient(px, py, 0, px, py, 8);
            grd.addColorStop(0, `${accent}30`);
            grd.addColorStop(1, 'transparent');
            ctx.beginPath();
            ctx.arc(px, py, 8, 0, Math.PI * 2);
            ctx.fillStyle = grd;
            ctx.fill();
            break;
          }
          accum += segLen;
        }

        for (const pt of path.points) {
          ctx.beginPath();
          ctx.arc(pt.x, pt.y, 1.5, 0, Math.PI * 2);
          ctx.fillStyle = `${accent}25`;
          ctx.fill();
        }
      }

      for (const junc of junctions) {
        const pulse = Math.sin(t * 1.5 + junc.pulseOffset) * 0.5 + 0.5;
        const s = junc.size;
        ctx.strokeStyle = `${accent}${Math.round((0.08 + pulse * 0.12) * 255).toString(16).padStart(2, '0')}`;
        ctx.lineWidth = 0.8;
        ctx.strokeRect(junc.x - s / 2, junc.y - s / 2, s, s);
        ctx.beginPath();
        ctx.arc(junc.x, junc.y, 1, 0, Math.PI * 2);
        ctx.fillStyle = `${accent}${Math.round((0.15 + pulse * 0.2) * 255).toString(16).padStart(2, '0')}`;
        ctx.fill();
      }
    }
  });
}

```



```

    for (const node of nodes) {
      const pulse = Math.sin(t * 2 + node.pulseOffset) * 0.5 + 0.5;
      ctx.beginPath();
      ctx.arc(node.x, node.y, node.size + pulse * 1.5, 0, Math.PI * 2);
      ctx.fillStyle = `${accent}${Math.round((0.08 + pulse * 0.14) * 255).toString(16).padStart(2, '0')}`;
      ctx.fill();
    }

    animRef.current = requestAnimationFrame(animate);
  };
  animate();
  return () => { if (animRef.current) cancelAnimationFrame(animRef.current); };
}, [accent]];

return <canvas ref={canvasRef} className="absolute inset-0 w-full h-full pointer-events-none" style={{ zIndex: 0, o...
}

function HoloScanLine() {
  return (
    <div className="absolute inset-0 pointer-events-none overflow-hidden" style={{ zIndex: 2 }}>
      <div style={{
        position: 'absolute', left: 0, right: 0, height: '2px',
        background: 'linear-gradient(90deg, transparent, rgba(0,210,255,0.15), transparent)',
        animation: 'holoScan 4s linear infinite',
      }} />
    </div>
  );
}

function HoloVinyl({ albumArt, playing, accent, size = 220 }: { albumArt?: string; playing: boolean; accent: string; ...
const rotRef = useRef(0);
const wasPlayingRef = useRef(false);
const canvasRef = useRef<HTMLCanvasElement>(null);
const animRef = useRef<number>();

useEffect(() => {
  if (playing && !wasPlayingRef.current) {
    rotRef.current = 0;
  }
  wasPlayingRef.current = playing;
}, [playing]);

useEffect(() => {
  const canvas = canvasRef.current;
  if (!canvas) return;
  const ctx = canvas.getContext("2d");
  if (!ctx) return;
  canvas.width = size * 2; canvas.height = size * 2;
  ctx.scale(2, 2);
  const cx = size / 2, cy = size / 2;

  let imgEl: HTMLImageElement | null = null;
  if (albumArt) { imgEl = new Image(); imgEl.crossOrigin = "anonymous"; imgEl.src = albumArt; }

  const animate = () => {
    ctx.clearRect(0, 0, size, size);
    if (playing) rotRef.current += 0.015;
    const t = Date.now() / 1000;

    ctx.save(); ctx.translate(cx, cy); ctx.rotate(rotRef.current);
    const R = size * 0.42;

    const outerGlow = ctx.createRadialGradient(0, 0, R * 0.9, 0, 0, R * 1.1);
    outerGlow.addColorStop(0, 'transparent');
    outerGlow.addColorStop(0.5, playing ? `${accent}15` : 'transparent');
    outerGlow.addColorStop(1, 'transparent');
    ctx.beginPath(); ctx.arc(0, 0, R * 1.1, 0, Math.PI * 2);
    ctx.fillStyle = outerGlow; ctx.fill();

    const vg = ctx.createRadialGradient(0, 0, R * 0.35, 0, 0, R);
    vg.addColorStop(0, "#142540"); vg.addColorStop(0.3, "#182d48"); vg.addColorStop(0.6, "#122240"); vg.addColorSto...
    ctx.beginPath(); ctx.arc(0, 0, R, 0, Math.PI * 2); ctx.fillStyle = vg; ctx.fill();

    ctx.beginPath(); ctx.arc(0, 0, R, 0, Math.PI * 2);
    ctx.strokeStyle = playing ? `${accent}40` : 'rgba(0,180,255,0.15)';
    ctx.lineWidth = 1.5; ctx.stroke();

    for (let g = 0; g < 24; g++) {
      const r = R * 0.35 + g * (R * 0.028);
      if (r > R) break;
      ctx.beginPath(); ctx.arc(0, 0, r, 0, Math.PI * 2);
      const baseHue = playing ? 200 : 210;
      const hue = (baseHue + g * 8 + (playing ? t * 30 : 0)) % 360;
      const alpha = playing
        ? 0.08 + Math.sin(t * 3 + g * 0.5) * 0.05
        : 0.03 + Math.sin(t * 0.5 + g * 0.3) * 0.015;
      ctx.strokeStyle = `hsla(${hue},80%,60%,${alpha})`;
      ctx.lineWidth = 0.5; ctx.stroke();
    }

    for (let i = 0; i < 6; i++) {
      const angle = (Math.PI * 2 / 6) * i + t * 0.3;
      ctx.beginPath();
      ctx.moveTo(Math.cos(angle) * R * 0.36, Math.sin(angle) * R * 0.36);
      ctx.lineTo(Math.cos(angle) * R * 0.95, Math.sin(angle) * R * 0.95);
      ctx.strokeStyle = playing ? `${accent}10` : 'rgba(0,180,255,0.04)';
      ctx.lineWidth = 0.3; ctx.stroke();
    }

    const lr = R * 0.33;
    if (imgEl && imgEl.complete && imgEl.naturalWidth > 0) {
      ctx.save(); ctx.beginPath(); ctx.arc(0, 0, lr, 0, Math.PI * 2); ctx.clip();
      ctx.drawImage(imgEl, -lr, -lr, lr * 2, lr * 2);
      ctx.restore();
      ctx.beginPath(); ctx.arc(0, 0, lr, 0, Math.PI * 2);
      ctx.strokeStyle = playing ? `${accent}50` : 'rgba(0,180,255,0.2)';
      ctx.lineWidth = 1; ctx.stroke();
    } else {

```

```

const labelGrad = ctx.createRadialGradient(0, 0, 0, 0, 0, lr);
labelGrad.addColorStop(0, playing ? accent : '#1a3560');
labelGrad.addColorStop(1, playing ? `${accent}80` : '#122850');
ctx.beginPath(); ctx.arc(0, 0, lr, 0, Math.PI * 2);
ctx.fillStyle = labelGrad; ctx.fill();
ctx.strokeStyle = 'rgba(0,180,255,0.3)'; ctx.lineWidth = 1; ctx.stroke();
}

ctx.beginPath(); ctx.arc(0, 0, 4, 0, Math.PI * 2);
ctx.fillStyle = "rgba(0,200,255,0.4)"; ctx.fill();
ctx.beginPath(); ctx.arc(0, 0, 2, 0, Math.PI * 2);
ctx.fillStyle = "rgba(255,255,255,0.6)"; ctx.fill();

ctx.restore();

if (playing) {
  for (let i = 0; i < 3; i++) {
    const ringR = R + 6 + i * 10;
    const alpha = 0.12 + Math.sin(t * 2 + i * 1.2) * 0.08;
    ctx.beginPath(); ctx.arc(cx, cy, ringR, 0, Math.PI * 2);
    ctx.strokeStyle = `${accent}${Math.round(alpha * 255).toString(16).padStart(2, '0')}`;
    ctx.lineWidth = 1; ctx.stroke();
  }

  for (let i = 0; i < 8; i++) {
    const angle = (Math.PI * 2 / 8) * i + t * 0.5;
    const eqH = 4 + Math.sin(t * 4 + i * 0.8) * 8;
    const eqR = R + 20;
    const px = cx + Math.cos(angle) * eqR;
    const py = cy + Math.sin(angle) * eqR;
    ctx.save();
    ctx.translate(px, py);
    ctx.rotate(angle + Math.PI / 2);
    ctx.fillStyle = `${accent}${Math.round((0.3 + Math.sin(t * 3 + i) * 0.2) * 255).toString(16).padStart(2, '0')}`;
    ctx.fillRect(-1, -eqH / 2, 2, eqH);
    ctx.restore();
  }
}

animRef.current = requestAnimationFrame(animate);
};
animate();
return () => { if (animRef.current) cancelAnimationFrame(animRef.current); };
}, [albumArt, playing, accent, size]);

return <canvas ref={canvasRef} style={{ width: size * 0.85, height: size * 0.85 }} data-testid="spinning-vinyl" />;
}

function RoomIcon({ icon, size = 18, color } : { icon: string; size?: number; color: string }) {
  const props = { className: "flex-shrink-0", style: { width: size, height: size, color, filter: `drop-shadow(0 0 4px...`
  switch (icon) {
    case "bed": return <Bed {...props} />;
    case "bath": return <Bath {...props} />;
    case "hallway": return <DoorOpen {...props} />;
    case "kitchen": return <CookingPot {...props} />;
    case "sofa": return <Sofa {...props} />;
    case "crown": return <Crown {...props} />;
    case "closet": return <ShirtIcon {...props} />;
    case "everywhere": return <Globe2 {...props} />;
    case "balcony": return <Sun {...props} />;
    default: return <Speaker {...props} />;
  }
}

function HoloPanel({ children, className = "", accent, glow = false, style = {}, sakura = false, ...rest } : {
  children: React.ReactNode; className?: string; accent: string; glow?: boolean; style?: React.CSSProperties; sakura?...
} & Omit<React.HTMLAttributes<HTMLDivElement>, 'style'>) {
  const bg = sakura ? 'rgba(22,50,82,0.7)' : 'rgba(25,50,95,0.75)';
  const borderIdle = sakura ? 'rgba(56,189,248,0.35)' : 'rgba(80,160,255,0.3)';
  const innerGlowIdle = sakura ? 'rgba(40,100,160,0.25)' : 'rgba(30,70,140,0.2)';
  const topLineIdle = sakura ? 'rgba(56,189,248,0.5)' : 'rgba(80,180,255,0.4)';
  const gradTop = sakura ? 'rgba(50,170,245,0.12)' : 'rgba(60,140,255,0.08)';
  const gradBot = sakura ? 'rgba(45,160,235,0.08)' : 'rgba(50,120,240,0.06)';
  return (
    <div {...rest} className={`relative rounded-xl overflow-hidden ${className}` ` style={{
      background: bg,
      backdropFilter: 'blur(30px)',
      border: `1px solid ${glow ? accent + '60' : borderIdle}`,
      boxShadow: glow
        ? `0 0 30px ${accent}30, inset 0 1px 0 ${sakura ? 'rgba(56,189,248,0.22)' : 'rgba(80,180,255,0.2)'}`, inset 0 ...
        : `inset 0 1px 0 ${sakura ? 'rgba(56,189,248,0.18)' : 'rgba(80,180,255,0.15)'}`, inset 0 0 40px ${innerGlowIdl...
      ...style,
    }}>
      <div className="absolute inset-0 pointer-events-none" style={{
        background: `linear-gradient(180deg, ${gradTop} 0%, transparent 30%, transparent 70%, ${gradBot} 100%)`,
      }} />
      <div className="absolute top-0 left-0 right-0 h-[1px] pointer-events-none" style={{
        background: `linear-gradient(90deg, transparent, ${glow ? accent + '70' : topLineIdle}, transparent)`,
      }} />
      <div className="relative z-10">{children}</div>
    </div>
  );
}

export default function SpotifyPlayerPage() {
  const [nowPlaying, setNowPlaying] = useState<NowPlaying | null>(null);
  const [loading, setLoading] = useState(true);
  const [actionPending, setActionPending] = useState(false);
  const [localProgress, setLocalProgress] = useState(0);
  const [volume, setVolume] = useState(50);
  const [shuffleOn, setShuffleOn] = useState(false);
  const [repeatMode, setRepeatMode] = useState("off");
  const [rooms, setRooms] = useState<RoomGroup[]>([]);
  const [activeProfile, setActiveProfile] = useState<ProfileKey>(() => {
    const p = new URLSearchParams(window.location.search).get("auth");
    if (p && (p === "bryn" || p === "yasu" || p === "guest")) return p;
    return "bryn";
  });
}

```

```

const [artistImages, setArtistImages] = useState<Record<string, string>>({});
const [dragItem, setDragItem] = useState<{ type: "artist" | "room" | "speaker"; data: any } | null>(null);
const [dropTarget, setDropTarget] = useState<string | null>(null);
const [activeSpeakers, setActiveSpeakers] = useState<{ name: string; entityId: string; room: string; type: string }...
const [selectedArtist, setSelectedArtist] = useState<ProfileArtist | null>(null);
const [notification, setNotification] = useState<string | null>(null);
const [profileSpinning, setProfileSpinning] = useState(false);
const [activeRooms, setActiveRooms] = useState<Set<string>>(new Set());
const [roomVolumes, setRoomVolumes] = useState<Record<string, number>>({});
const volumeTimerRef = useRef<Record<string, ReturnType<typeof setTimeout>>>({});
const [viewMode, setViewMode] = useState<ViewMode>("floor");
const [viewSpinning, setViewSpinning] = useState(false);
const [expandedRoom, setExpandedRoom] = useState<string | null>(null);
const [expandedSpeaker, setExpandedSpeaker] = useState<string | null>(null);
const [floorSpeakerPopup, setFloorSpeakerPopup] = useState<string | null>(null);
const [everywhereExcluded, setEverywhereExcluded] = useState<Set<string>>(() => {
  try { const s = localStorage.getItem("holomusic-everywhere-excluded"); return s ? new Set(JSON.parse(s)) : new Se...
});
const [recommendations, setRecommendations] = useState<{ name: string; image: string; uri: string; id: string }[]>(...
const [recoIndex, setRecoIndex] = useState(0);
const [voiceConfirm, setVoiceConfirm] = useState(() => { const v = localStorage.getItem("holomusic-voice"); return ...
const [menuOpen, setMenuOpen] = useState(false);
const [searchQuery, setSearchQuery] = useState("");
const [searchResults, setSearchResults] = useState<any[]>([]);
const [searching, setSearching] = useState(false);
const [showSearch, setShowSearch] = useState(false);
const [showFlickMenu, setShowFlickMenu] = useState(false);
const [flickDeviceGroups, setFlickDeviceGroups] = useState<Array<{room: string; icon: string; devices: Array<{id: s...
const [isFlicking, setIsFlicking] = useState(false);
const [jaTranslations, setJaTranslations] = useState<Record<string, string>>({});
const jaTranslationCache = useRef<Record<string, string>>({});
const jaTranslationPending = useRef<Set<string>>(new Set());

const searchParams = new URLSearchParams(window.location.search);
const authParam = searchParams.get("auth");
const authQuery = authParam ? `?auth=${authParam}` : "";
const isEmbedded = searchParams.get("embed") === "true";
const lightEntity = searchParams.get("lightEntity");

useEffect(() => {
  const checkAuth = () => {
    const p = new URLSearchParams(window.location.search).get("auth");
    if (p && (p === "bryn" || p === "yasu" || p === "guest") && p !== activeProfile) {
      setActiveProfile(p);
      setSelectedArtist(null);
    }
  };
  window.addEventListener("focus", checkAuth);
  document.addEventListener("visibilitychange", checkAuth);
  return () => { window.removeEventListener("focus", checkAuth); document.removeEventListener("visibilitychange", c...
}, [activeProfile]);

useEffect(() => {
  if (!lightEntity) return;
  let active = true;
  const check = async () => {
    try {
      const res = await fetch(`/api/ha-entity-state?entity=${encodeURIComponent(lightEntity)}`);
      if (!res.ok) return;
      const data = await res.json();
      if (data.state === "off" && active) {
        fetch("/api/spotify/go-home", { method: "POST", headers: { "Content-Type": "application/json" } }).catch(()...
      }
    } catch {}
  };
  check();
  const interval = setInterval(check, 3000);
  return () => { active = false; clearInterval(interval); };
}, [lightEntity]);

const profile = PROFILES[activeProfile];
const notifTimeout = useRef<any>(null);
const searchTimeout = useRef<any>(null);

const showNotif = (msg: string) => {
  setNotification(msg);
  if (notifTimeout.current) clearTimeout(notifTimeout.current);
  notifTimeout.current = setTimeout(() => setNotification(null), 3000);
};

const announceTrack = useCallback((trackName: string, artistName: string, roomName?: string) => {
  if (!voiceConfirm || !window.speechSynthesis) return;
  window.speechSynthesis.cancel();
  const lang = activeProfile === "yasu" ? "ja-JP" : "en-US";
  const text = activeProfile === "yasu"
    ? roomName
      ? `${artistName} on ${trackName} of ${roomName} on Q•u 0W0-0Y`
      : `${artistName} on ${trackName} of Q•u 0W0-0Y`
    : roomName
      ? `Now playing ${trackName} by ${artistName} on the ${roomName}`
      : `Now playing ${trackName} by ${artistName}`;
  const utter = new SpeechSynthesisUtterance(text);
  utter.lang = lang;
  utter.rate = 1.05;
  utter.pitch = 1.0;
  utter.volume = 0.7;
  window.speechSynthesis.speak(utter);
}, [voiceConfirm, activeProfile]);

const toggleVoiceConfirm = () => {
  const next = !voiceConfirm;
  setVoiceConfirm(next);
  localStorage.setItem("holomusic-voice", String(next));
  const isJp = activeProfile === "yasu";
  showNotif(next ? (isJp ? "おしゃべり中" : "Voice confirmations on") : (isJp ? "おしゃべり終了" : "Voice confirmations off"));
};

const fetchNowPlaying = useCallback(async () => {

```

```

    try {
      const res = await fetch(`/api/spotify/now-playing${authQuery}`);
      if (res.ok) { const d = await res.json(); setNowPlaying(d); if (d.progress) setLocalProgress(d.progress); }
    } catch {} finally { setLoading(false); }
  }, [authQuery]);

const fetchRooms = useCallback(async () => {
  try { const res = await fetch(`/api/spotify/rooms${authQuery}`); if (res.ok) setRooms(await res.json()); } catch {}
}, [authQuery]);

const fetchPlaybackState = useCallback(async () => {
  try {
    const res = await fetch(`/api/spotify/playback-state${authQuery}`);
    if (res.ok) {
      const d = await res.json();
      if (d.active) {
        setVolume(d.volume); setShuffleOn(d.shuffle); setRepeatMode(d.repeat);
        const dn = (d.deviceName || "").toLowerCase();
        if (dn === "byhome" || dn.includes("everywhere")) {
          setActiveRooms(prev => { const n = new Set(prev); n.add("Everywhere"); return n; });
        } else {
          const matchedRoom = ROOM_HOTSPOTS.find(spot => {
            const eid = spot.entityId.toLowerCase();
            return eid.includes(dn.replace(/^[a-z0-9]/g, ''));
          });
          if (matchedRoom) {
            setActiveRooms(prev => { const n = new Set(prev); n.add(matchedRoom.room); return n; });
          }
        }
      }
    }
  } catch {}
}, [authQuery]);

const artistIdsRef = useRef<Record<string, string>>({});

const fetchArtistImages = useCallback(async () => {
  const profileArtists = [...PROFILES.bryn.artists, ...PROFILES.yasu.artists, ...PROFILES.guest.artists];
  const stationItems = STATION_SHORTCUTS.filter(s => s.uri).map(s => ({ name: s.name, uri: s.uri, searchQuery: s.n...
  const all = [...profileArtists, ...stationItems];
  const uniqueItems = all.filter((a, i, arr) => arr.findIndex(x => x.name === a.name) === i);
  try {
    const res = await fetch(`/api/spotify/bulk-images${authQuery}`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ items: uniqueItems.map(a => ({ name: a.name, uri: a.uri, searchQuery: a.searchQuery || ...
    });
    if (res.ok) {
      const data = await res.json();
      setArtistImages(data.images || {});
      artistIdsRef.current = data.ids || {};
      fetchRecommendations(data.ids || {});
      return;
    }
  } catch {}
  const imgs: Record<string, string> = {};
  const ids: Record<string, string> = {};
  for (const a of uniqueItems) {
    try {
      const res = await fetch(`/api/spotify/search?q=${encodeURIComponent(a.searchQuery)}&${authQuery.slice(1)}`);
      if (res.ok) {
        const data = await res.json();
        const artist = data.artists?.[0];
        const img = artist?.image || data.tracks?.[0]?.image || "";
        if (img) imgs[a.name] = img;
        if (artist?.id) ids[a.name] = artist.id;
      }
    } catch {}
  }
  setArtistImages(imgs);
  artistIdsRef.current = ids;
  fetchRecommendations(ids);
}, [authQuery]);

const fetchRecommendations = useCallback(async (ids: Record<string, string>) => {
  const currentArtists = activeProfile === "bryn" ? PROFILES.bryn.artists : PROFILES.yasu.artists;
  const seedArtist = currentArtists[Math.floor(Math.random() * currentArtists.length)];
  const artistId = ids[seedArtist.name];
  if (!artistId) return;
  try {
    const res = await fetch(`/api/spotify/related-artists?artistId=${artistId}&${authQuery.slice(1)}`);
    if (res.ok) {
      const data = await res.json();
      if (data.artists?.length) setRecommendations(data.artists);
    }
  } catch {}
}, [authQuery, activeProfile]);

const doSearch = useCallback(async (q: string) => {
  if (!q.trim()) { setSearchResults([]); return; }
  setSearching(true);
  try {
    const res = await fetch(`/api/spotify/search?q=${encodeURIComponent(q)}&${authQuery.slice(1)}`);
    if (res.ok) {
      const data = await res.json();
      setSearchResults([...(data.tracks || []).slice(0, 6), ...(data.artists || []).slice(0, 4)]);
    }
  } catch {} finally { setSearching(false); }
}, [authQuery]);

useEffect(() => {
  fetchNowPlaying(); fetchRooms(); fetchPlaybackState(); fetchArtistImages();
  fetch("/api/flick-devices").then(r => r.ok ? r.json() : []).then(groups => setFlickDeviceGroups(groups)).catch(()...
  const interval = setInterval(fetchNowPlaying, 4000);
  return () => clearInterval(interval);
}, []);

const handleSpotifyFlick = async (deviceId: string) => {

```

```

setIsFlicking(true);
try {
  const resp = await fetch(`/api/spotify/flick${authQuery}`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ deviceId }),
  });
  const data = await resp.json();
  if (data.success) {
    showNotif(isSakura ? `${data.device}0kZâ 0W0-0W0_` : `Sent to ${data.device} in ${data.room}`);
  } else {
    showNotif(data.error || "Flick failed");
  }
} catch (e: any) {
  showNotif("Flick failed");
} finally {
  setIsFlicking(false);
  setShowFlickMenu(false);
}
};

const handleStopAll = async () => {
  setActionPending(true);
  try {
    await fetch(`/api/spotify/stop-all${authQuery}`, { method: "POST" });
    showNotif(isSakura ? "QhP\kb" : "All playback stopped");
    setTimeout(fetchNowPlaying, 600);
  } catch {} finally { setActionPending(false); }
};

useEffect(() => {
  if (recommendations.length <= 1) return;
  const timer = setInterval(() => {
    setRecoIndex(prev => (prev + 1) % recommendations.length);
  }, 5000);
  return () => clearInterval(timer);
}, [recommendations.length]);

useEffect(() => {
  if (nowPlaying?.playing) {
    const tick = setInterval(() => {
      setLocalProgress(p => { const n = p + 500; return nowPlaying.duration && n > nowPlaying.duration ? nowPlaying...
    }, 500);
    return () => clearInterval(tick);
  }
}, [nowPlaying?.playing, nowPlaying?.duration]);

const setRoomVolume = useCallback((room: string, vol: number) => {
  setRoomVolumes(prev => ({ ...prev, [room]: vol }));
  const spot = ROOM_HOTSPOTS.find(h => h.room === room);
  if (!spot) return;
  if (volumeTimerRef.current[room]) clearTimeout(volumeTimerRef.current[room]);
  volumeTimerRef.current[room] = setTimeout(() => {
    const payload: any = { entityId: spot.groupEntityId, level: vol };
    if (room === "Everywhere") {
      try { const s = localStorage.getItem("holomusic-everywhere-excluded"); if (s) payload.excludeEntityIds = JSON...
    }
    fetch(`/api/media/volume`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload),
    });
  }, 200);
}, []);

const doAction = async (action: string, method = "POST", body?: any) => {
  setActionPending(true);
  try {
    await fetch(`/api/spotify/${action}${authQuery}`, { method, headers: body ? { "Content-Type": "application/json..."
  } catch {} finally { setActionPending(false); }
};

const playOnRoom = async (roomName: string, artistData: ProfileArtist) => {
  const hotspot = ROOM_HOTSPOTS.find(h => h.room === roomName);
  const roomGroup = rooms.find(r => r.room === roomName);
  const groupSpeaker = roomGroup?.speakers.find(s => s.type === "group") || roomGroup?.speakers[0];
  const entityId = groupSpeaker?.entityId || hotspot?.groupEntityId || hotspot?.entityId;
  const deviceType = hotspot?.deviceType || "echo";
  if (!entityId) { showNotif(`No speakers in ${roomName}`); return; }
  const isJp = activeProfile === "yasu";
  const announceMessage = isJp
    ? `${artistData.name}0'${ROOM_JP[roomName] || roomName}${roomName === "Everywhere" ? "0°0000x" : ""}0gQ•u 0W0-0Y`
    : `Now playing ${artistData.name} on the ${roomName}${roomName === "Everywhere" ? " group" : ""}`;
  console.log(`[PlayOnRoom] ${roomName}: entity=${entityId}, device=${deviceType}, artist=${artistData.name}, uri=$...
  try {
    const resp = await fetch(`/api/spotify/play-on-speaker${authQuery}`, {
      method: "POST", headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ entityId, spotifyUri: artistData.uri, artistName: artistData.name, searchQuery: artist...
    });
    const data = await resp.json();
    console.log(`[PlayOnRoom] Response: ${resp.status}`, data);
    setActiveRooms(prev => new Set(prev).add(roomName));
    showNotif(`Playing ${artistData.name} in ${roomName}`);
    setTimeout(fetchNowPlaying, 1500);
  } catch (err) { console.error(`[PlayOnRoom] Error:`, err); showNotif("Failed to play"); }
};

const playStation = async (station: StationShortcut) => {
  if (station.uri) {
    try {
      await fetch(`/api/spotify/play-context${authQuery}`, {
        method: "POST", headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ contextUri: station.uri, shuffle: true }),
      });
      announceTrack(station.name, "");
      showNotif(`Playing ${station.name}`);
      setTimeout(fetchNowPlaying, 1000);
    }
  }
};

```



```

const cleanupTouchDrag = useCallback(() => {
  if (touchDragRef.current?.ghostEl) {
    touchDragRef.current.ghostEl.remove();
  }
  touchDragRef.current = null;
  setDropTarget(null);
  setDragItem(null);
}, []);

const handleTouchStart = (type: "artist" | "room", data: any) => (e: React.TouchEvent) => {
  const touch = e.touches[0];
  touchDragRef.current = {
    type,
    data,
    startX: touch.clientX,
    startY: touch.clientY,
    isDragging: false,
    ghostEl: null,
    movedEnough: false,
    currentDropRoom: null,
  };
};

useEffect(() => {
  const onTouchMove = (e: TouchEvent) => {
    const td = touchDragRef.current;
    if (!td) return;
    const touch = e.touches[0];
    const dx = touch.clientX - td.startX;
    const dy = touch.clientY - td.startY;
    const dist = Math.sqrt(dx * dx + dy * dy);

    if (!td.movedEnough && dist > 15) {
      td.movedEnough = true;
      td.isDragging = true;
      setDragItem({ type: td.type, data: td.data });
      if (td.type === "artist" && viewMode !== "floor") {
        switchView("floor");
      }
      const ghost = document.createElement("div");
      ghost.style.cssText = `
        position:fixed; z-index:9999; pointer-events:none;
        padding:6px 14px; border-radius:20px; font-size:13px; font-weight:600;
        color:#fff; background:rgba(59,130,246,0.85); backdrop-filter:blur(8px);
        box-shadow:0 0 20px rgba(59,130,246,0.5); white-space:nowrap;
        transform:translate(-50%,-50%);
      `;
      ghost.textContent = td.data?.name || td.data?.room || "?";
      document.body.appendChild(ghost);
      td.ghostEl = ghost;
      console.log(`[DnD-Touch] Drag started: ${td.type} = ${td.data?.name || td.data?.room}`);
    }

    if (td.isDragging && td.ghostEl) {
      e.preventDefault();
      td.ghostEl.style.left = `${touch.clientX}px`;
      td.ghostEl.style.top = `${touch.clientY}px`;

      const els = document.elementsFromPoint(touch.clientX, touch.clientY);
      let foundRoom: string | null = null;
      let foundPlayer = false;
      for (const el of els) {
        const hotspotEl = (el as HTMLElement).closest("[data-room-hotspot]") as HTMLElement | null;
        if (hotspotEl) {
          foundRoom = hotspotEl.dataset.roomHotspot || null;
          break;
        }
        const playerEl = (el as HTMLElement).closest("[data-player-drop]") as HTMLElement | null;
        if (playerEl) {
          foundPlayer = true;
          break;
        }
      }
      td.currentDropRoom = foundRoom;
      setDropTarget(foundRoom);
      setPlayerDropHighlight(foundPlayer && td.type === "speaker");
    }
  };

  const onTouchEnd = () => {
    const td = touchDragRef.current;
    if (!td) return;

    if (td.isDragging && td.movedEnough) {
      const room = td.currentDropRoom;
      if (room) {
        console.log(`[DnD-Touch] Dropped on room: ${room}`);
        if (td.type === "artist") playOnRoom(room, td.data);
        else if (td.type === "room" && td.data.room !== room) groupRooms(td.data.room, room);
      } else {
        console.log(`[DnD-Touch] Dropped outside any room`);
      }
    }
    cleanupTouchDrag();
  };

  document.addEventListener("touchmove", onTouchMove, { passive: false });
  document.addEventListener("touchend", onTouchEnd);
  document.addEventListener("touchcancel", onTouchEnd);
  return () => {
    document.removeEventListener("touchmove", onTouchMove);
    document.removeEventListener("touchend", onTouchEnd);
    document.removeEventListener("touchcancel", onTouchEnd);
  };
}, [viewMode, cleanupTouchDrag]);

const switchProfile = (p: ProfileKey) => {
  if (p === activeProfile) return;

```

```

    setProfileSpinning(true);
    setTimeout(() => { setActiveProfile(p); setSelectedArtist(null); setTimeout(() => setProfileSpinning(false), 50);...
});

const switchView = (v: ViewMode) => {
  if (v === ViewMode) return;
  setViewSpinning(true);
  setShowSearch(false);
  setSearchQuery("");
  setSearchResults([]);
  setTimeout(() => { setViewMode(v); setTimeout(() => setViewSpinning(false), 50); }, 300);
};

const isPlaying = !!nowPlaying?.playing;
const progressPct = nowPlaying?.duration ? (localProgress / nowPlaying.duration) * 100 : 0;
const isSakura = activeProfile === "yasu";

useEffect(() => {
  if (!isSakura || !nowPlaying) return;
  const textsToTranslate: string[] = [];
  const addIfNeeded = (t?: string) => {
    if (t && !jaTranslationCache.current[t] && !jaTranslationPending.current.has(t)) {
      textsToTranslate.push(t);
    }
  };
  addIfNeeded(nowPlaying.name);
  addIfNeeded(nowPlaying.album);
  if (textsToTranslate.length === 0) return;
  textsToTranslate.forEach(t => jaTranslationPending.current.add(t));
  const timer = setTimeout(() => {
    fetch("/api/translate-ja", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ texts: textsToTranslate }),
    })
      .then(r => r.json())
      .then(data => {
        if (data.translations) {
          jaTranslationCache.current = { ...jaTranslationCache.current, ...data.translations };
          setJaTranslations(prev => ({ ...prev, ...data.translations }));
        }
        textsToTranslate.forEach(t => jaTranslationPending.current.delete(t));
      })
      .catch(() => {
        textsToTranslate.forEach(t => jaTranslationPending.current.delete(t));
      });
  }, 300);
  return () => clearTimeout(timer);
}, [isSakura, nowPlaying?.name, nowPlaying?.album]);

const ja = (text?: string) => {
  if (!text || !isSakura) return text;
  return jaTranslations[text] || text;
};

const tc = {
  textMuted: isSakura ? 'rgba(150,210,248,0.8)' : 'rgba(100,180,255,0.65)',
  textSoft: isSakura ? 'rgba(160,220,250,0.6)' : 'rgba(100,180,255,0.45)',
  textMid: isSakura ? 'rgba(140,210,248,0.9)' : 'rgba(140,200,255,0.8)',
  textBright: isSakura ? 'rgba(170,225,255,0.95)' : 'rgba(140,200,255,0.85)',
  navIdle: isSakura ? 'rgba(120,210,250,0.8)' : 'rgba(120,200,255,0.65)',
  menuToggle: isSakura ? 'rgba(100,200,250,0.8)' : 'rgba(90,200,255,0.7)',
  divider: isSakura ? 'rgba(56,189,248,0.2)' : 'rgba(70,160,255,0.2)',
  sideEdge: isSakura
    ? 'linear-gradient(180deg, rgba(56,189,248,0.4), rgba(56,189,248,0.1), rgba(56,189,248,0.3))'
    : 'linear-gradient(180deg, rgba(60,180,255,0.4), rgba(60,180,255,0.1), rgba(60,180,255,0.3))',
  homeBtnBg: isSakura ? 'rgba(56,189,248,0.18)' : 'rgba(50,130,255,0.12)',
  homeBtnBorder: isSakura ? 'rgba(56,189,248,0.3)' : 'rgba(80,170,255,0.2)',
  homeBtnText: isSakura ? 'rgba(160,225,255,0.9)' : 'rgba(140,215,255,0.8)',
  btnBg: isSakura ? 'rgba(56,189,248,0.15)' : 'rgba(60,140,255,0.1)',
  btnBorder: isSakura ? 'rgba(56,189,248,0.3)' : 'rgba(60,140,255,0.2)',
  btnText: isSakura ? 'rgba(150,215,250,0.85)' : 'rgba(100,180,255,0.7)',
  cardBorder: isSakura ? 'rgba(56,189,248,0.22)' : 'rgba(80,160,255,0.25)',
  progressBg: isSakura ? 'rgba(56,189,248,0.15)' : 'rgba(60,160,255,0.12)',
  progressGrad: isSakura ? 'rgba(30,180,240,0.45)' : 'rgba(0,180,255,0.4)',
  dotIdle: isSakura ? 'rgba(56,189,248,0.45)' : 'rgba(80,160,255,0.4)',
  roomBorder: isSakura ? 'rgba(56,189,248,0.18)' : 'rgba(60,140,255,0.15)',
  speakerBorder: isSakura ? 'rgba(56,189,248,0.2)' : 'rgba(80,160,255,0.18)',
  speakerIcon: isSakura ? 'rgba(100,195,245,0.65)' : 'rgba(100,180,255,0.6)',
  headerBorder: isSakura ? 'rgba(56,189,248,0.28)' : 'rgba(80,160,255,0.25)',
  voiceOff: isSakura ? 'rgba(100,195,240,0.55)' : 'rgba(100,180,255,0.5)',
};

return (
  <div className={` ${window.self !== window.top ? 'absolute' : 'fixed'} inset-0 flex flex-col overflow-hidden selec...
    <img
      src={isPlaying ? massBg : musicBg}
      alt=""
      className="absolute inset-0 w-full h-full object-cover transition-opacity duration-1000"
      style={{ opacity: isPlaying ? 0.35 : 0.25, filter: isSakura ? "brightness(1.3) saturate(0.5) hue-rotate(200de...
    />

    <div className="absolute inset-0" style={{
      background: isSakura
        ?
          radial-gradient(ellipse at 15% 30%, rgba(255,183,197,0.4) 0%, transparent 45%),
          radial-gradient(ellipse at 85% 20%, rgba(56,189,248,0.45) 0%, transparent 40%),
          radial-gradient(ellipse at 50% 70%, rgba(255,192,203,0.3) 0%, transparent 50%),
          radial-gradient(ellipse at 70% 80%, rgba(56,189,248,0.4) 0%, transparent 45%),
          radial-gradient(ellipse at 30% 50%, rgba(100,200,255,0.25) 0%, transparent 50%),
          linear-gradient(180deg, rgba(25,60,100,0.12) 0%, rgba(20,48,80,0.15) 100%)
        :
          radial-gradient(ellipse at 20% 50%, rgba(50,130,240,0.3) 0%, transparent 50%),
          radial-gradient(ellipse at 80% 30%, rgba(70,150,255,0.25) 0%, transparent 40%),
          radial-gradient(ellipse at 50% 80%, rgba(40,100,200,0.25) 0%, transparent 60%),
          linear-gradient(180deg, rgba(20,50,100,0.2) 0%, rgba(25,55,110,0.35) 100%)
        ,
    }}
  />

```



```

}} />

{isSakura && {
  <div className="absolute inset-0 pointer-events-none" style={{
    background:
      radial-gradient(circle at 10% 15%, rgba(255,182,193,0.12) 0%, transparent 25%),
      radial-gradient(circle at 90% 75%, rgba(255,192,203,0.1) 0%, transparent 25%),
      radial-gradient(circle at 40% 90%, rgba(255,175,185,0.08) 0%, transparent 20%)
    ,
    zIndex: 0,
  }} />
}}
})

<HoloCircuitLines accent={profile.accent} sakura={isSakura} />
{isSakura && <CherryBlossoms />}
{isSakura && <JapaneseWaves accent={profile.accent} />}
<HoloScanLine />

{notification && {
  <div className="absolute top-4 left-1/2 -translate-x-1/2 z-50 px-6 py-2.5 rounded-lg text-xs font-medium text...
    style={{
      background: isSakura ? 'rgba(20,50,80,0.9)' : 'rgba(25,55,105,0.92)',
      border: '1px solid ${profile.accent}60',
      backdropFilter: "blur(20px)",
      boxShadow: '0 0 30px ${profile.glow}, 0 0 60px rgba(0,0,0,0.3)',
      animation: "fadeInUp 0.3s ease",
    }}
    data-testid="notification">
    <Zap className="inline h-3 w-3 mr-1.5" style={{ color: profile.accent }} />
    {notification}
  </div>
})

(({() => {
  const h = new Date().getHours();
  if (isSakura) {
    const greeting = h < 12 ? "0J0o0`0F0T0V0D0-0Y" : h < 18 ? "0S0`0k0a0o" : "0S0`0p0`0o";
    return (
      <div className="absolute top-5 z-50 pointer-events-none text-right flex items-start gap-3" style={{ right:...
        <p className="text-3xl font-bold tracking-wide" style={{
          color: 'rgba(200,230,255,0.85)',
          textShadow: '0 0 20px rgba(56,189,248,0.6), 0 0 40px rgba(255,183,197,0.3)',
          fontFamily: "'Noto Sans JP', 'Hiragino Sans', sans-serif",
          marginTop: '6px',
        }}>
          {greeting}0 0,Y
        </p>
        <img src={yasuPhoto} alt="Yasu" data-testid="img-yasu-photo" style={{ width: '60px', height: '60px', bo...
        </div>
      );
    }
    const greeting = h < 12 ? "Good Morning" : h < 18 ? "Good Afternoon" : "Good Evening";
    if (activeProfile === "bryn") {
      return (
        <div className="absolute top-5 z-50 pointer-events-none text-right flex items-start gap-3" style={{ right:...
          <p className="text-3xl font-bold tracking-wide" style={{
            color: 'rgba(200,230,255,0.85)',
            textShadow: '0 0 20px ${profile.glow}, 0 0 40px rgba(80,160,255,0.3)',
            fontFamily: "'Inter', system-ui, sans-serif",
            letterSpacing: '0.04em',
            marginTop: '6px',
          }}>
            {greeting}, Bryn
          </p>
          <img src={brynPhoto} alt="Bryn" data-testid="img-bryn-photo" style={{ width: '60px', height: '60px', bo...
          </div>
        );
      }
    }
    if (activeProfile === "guest") {
      return (
        <div className="absolute top-5 z-50 pointer-events-none text-right" style={{ right: '2%' }} data-testid="...
          <p className="text-3xl font-bold tracking-wide" style={{
            color: 'rgba(200,230,255,0.85)',
            textShadow: '0 0 20px ${profile.glow}, 0 0 40px rgba(160,160,255,0.3)',
            fontFamily: "'Inter', system-ui, sans-serif",
            letterSpacing: '0.04em',
          }}>
            {greeting}
          </p>
        </div>
      );
    }
    return null;
  })())

<div className="relative z-10 flex-1 flex overflow-hidden" style={{
  transform: profileSpinning ? "perspective(1200px) rotateY(-90deg)" : "perspective(1200px) rotateY(0deg)",
  opacity: profileSpinning ? 0 : 1,
  transition: "transform 0.35s cubic-bezier(0.4, 0, 0.2, 1), opacity 0.35s ease",
}}>

  <div className="flex flex-col flex-shrink-0 relative" style={{ width: menuOpen ? 200 : 48, transition: "width...
    <div className="absolute inset-0" style={{
      background: isSakura ? 'rgba(20,50,80,0.75)' : 'rgba(22,48,90,0.8)',
      backdropFilter: "blur(30px)",
      borderRight: isSakura ? '1px solid rgba(56,189,248,0.22)' : '1px solid rgba(70,160,255,0.25)',
    }} />
    <div className="absolute top-0 right-0 bottom-0 w-[1px]" style={{
      background: tc.sideEdge,
    }} />

    <div className="relative z-10 flex flex-col h-full py-2">
      <button onClick={() => setMenuOpen(!menuOpen)}
        className="w-full flex items-center justify-center py-2.5 mb-1 transition-colors"
        style={{ color: tc.menuToggle }}
        data-testid="menu-toggle">
        {menuOpen ? <X className="h-4 w-4" /> : <Menu className="h-4 w-4" />}
      </button>

```

```

<div className="flex-1 flex flex-col gap-0.5 mt-2 px-1">
  {[
    { mode: "floor" as ViewMode, icon: <Home className="h-3.5 w-3.5" />, label: isSakura ? "⌚S005" : "Floor...
    { mode: "stations" as ViewMode, icon: <Radio className="h-3.5 w-3.5" />, label: isSakura ? "0-0#000-0c00" : ...
    { mode: "rooms" as ViewMode, icon: <Speaker className="h-3.5 w-3.5" />, label: isSakura ? "e\K" : "Roo...
  ]).map(item => (
    <button key={item.mode}
      onClick={() => switchView(item.mode)}
      className="w-full flex items-center gap-3 px-2.5 py-2 rounded-lg transition-all"
      style={{
        background: viewMode === item.mode ? `${profile.accent}18` : "transparent",
        color: viewMode === item.mode ? profile.accent : tc.navIdle,
        borderLeft: viewMode === item.mode ? `2px solid ${profile.accent}` : `2px solid transparent`,
      }}
      data-testid={`nav-${item.mode}`}>
      <div className="flex-shrink-0">{item.icon}</div>
      {menuOpen && <span className="text-[12px] whitespace-nowrap font-medium">{item.label}</span>}
    </button>
  ))

  <div className="my-2 mx-2 h-[1px]" style={{ background: tc.divider }} />

  <button onClick={() => setShowSearch(!showSearch)}
    className="w-full flex items-center gap-3 px-2.5 py-2 rounded-lg transition-all"
    style={{ color: showSearch ? profile.accent : tc.navIdle }}
    data-testid="nav-search">
    <Search className="h-3.5 w-3.5 flex-shrink-0" />
    {menuOpen && <span className="text-[12px] whitespace-nowrap font-medium">Search</span>}
  </button>

  <div className="my-2 mx-2 h-[1px]" style={{ background: tc.divider }} />

  {flickDeviceGroups.length > 0 && (
    <div className="relative">
      <button onClick={() => setShowFlickMenu(!showFlickMenu)}
        className="w-full flex items-center gap-3 px-2.5 py-2 rounded-lg transition-all ${showFlickMenu ...
        style={{ color: showFlickMenu ? profile.accent : tc.navIdle }}
        data-testid="nav-flick-cast"
        disabled={isFlicking}>
        {isFlicking ? <Loader2 className="h-3.5 w-3.5 flex-shrink-0 animate-spin" /> : <Cast className="h...
        {menuOpen && <span className="text-[12px] whitespace-nowrap font-medium">{isSakura ? "Ža" : "Cast...
      </button>
      {showFlickMenu && (
        <div className="absolute left-full top-0 ml-1 w-52 rounded-lg shadow-2xl overflow-hidden z-50"
          style={{ background: isSakura ? `rgba(20,50,80,0.95)` : `rgba(10,25,50,0.95)`, border: `1px sol...
          <div className="px-2.5 py-1.5 flex items-center justify-between" style={{ borderBottom: `1px so...
            <span className="text-[12px] font-semibold" style={{ color: profile.accent }}>{isSakura ? "Ža" ...
            <button onClick={() => setShowFlickMenu(false)} style={{ color: tc.navIdle }} data-testid="bu...
          </div>
          <div className="max-h-[400px] overflow-y-auto">
            {flickDeviceGroups.map((group) => (
              <div key={group.room}>
                <div className="px-2.5 py-1 flex items-center gap-1.5 sticky top-0" style={{ background: ...
                  <span className="text-[11px]">{group.icon}</span>
                  <span className="text-[11px] font-semibold uppercase tracking-wider" style={{ color: 'r...
                </div>
                {group.devices.filter(d => d.canDisplay).map((device) => (
                  <button key={device.id} data-testid={`button-flick-${device.id}`}
                    className="w-full px-2.5 py-1.5 pl-5 flex items-center gap-2 transition-colors text-l...
                    style={{ color: `rgba(200,220,255,0.85)` }}
                    onClick={() => handleSpotifyFlick(device.id)} disabled={isFlicking}>
                    <Monitor className="h-3 w-3 flex-shrink-0" style={{ color: profile.accent }} />
                    <span className="text-[12px] truncate">{device.name}</span>
                  </button>
                ))}
              </div>
            ))}
          </div>
        </div>
      ))
    </div>
  )

  <button onClick={handleStopAll}
    className="w-full flex items-center gap-3 px-2.5 py-2 rounded-lg transition-all hover:scale-105"
    style={{ color: `rgba(255,100,100,0.7)` }}
    data-testid="nav-stop-all">
    <Square className="h-3.5 w-3.5 flex-shrink-0" fill="currentColor" />
    {menuOpen && <span className="text-[12px] whitespace-nowrap font-medium">{isSakura ? "QhP\kb" : "Stop Al...
  </button>
</div>

<div className="mt-auto flex flex-col gap-0.5 px-1 pb-1">
  <button onClick={() => {
    fetch("/api/spotify/go-home", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
    }).catch(() => {});
    const haUrl = "https://ec8ebfanqrqlsnmnggrdl4yzq2i8koah.ui.nabu.casa/lovelace/test-home";
    try { window.top!.location.href = haUrl; } catch(e) { window.location.href = haUrl; }
  }}
    className="w-full flex items-center gap-3 px-2.5 py-2 rounded-lg transition-all hover:scale-105 mb-1"
    style={{
      color: tc.homeBtnText,
      background: tc.homeBtnBg,
      border: `1px solid ${tc.homeBtnBorder}`,
    }}
    data-testid="back-to-dashboard">
    <Home className="h-4 w-4 flex-shrink-0" />
    {menuOpen && <span className="text-[12px] whitespace-nowrap font-medium">{isSakura ? "00000a" : "Home"}<...
  </button>

  <div className="mx-2 mb-1 h-[1px]" style={{ background: tc.divider }} />
  {[Object.keys(PROFILES) as ProfileKey[]].map(k => (
    <button key={k} onClick={() => switchProfile(k)}
      className="w-full flex items-center gap-2.5 px-2 py-1.5 rounded-lg transition-all"
      style={{
        background: activeProfile === k ? `${PROFILES[k].accent}15` : "transparent",

```



```

    })
  </div>
</HoloPanel>

<HoloPanel accent={profile.accent} sakura={isSakura} className="flex-1 overflow-hidden p-3">
  <div className="text-[12px] uppercase tracking-[0.2em] font-bold mb-2 px-1 flex items-center gap-1.5"
    style={{ color: `${profile.accent}90` }}>
    <Wifi className="h-3 w-3" />
    {isSakura ? "0Jl 0kQe0S" : "Favorites"}
  </div>
  <div className="overflow-y-auto flex-1" style={{ scrollbarWidth: "none", maxHeight: 'calc(100% - 24px)' }}...
    <div className="grid grid-cols-2 gap-1.5">
      {profile.artists.map((artist, i) => {
        const isSelected = selectedArtist?.name === artist.name;
        return (
          <div key={artist.name} draggable
            onDragStart={handleDragStart("artist", artist)}
            onDragEnd={() => setDragItem(null)}
            onTouchStart={handleTouchStart("artist", artist)}
            onClick={() => {
              if (selectedArtist?.name === artist.name) { setSelectedArtist(null); }
              else { setSelectedArtist(artist); if (viewModel !== "floor") { switchView("floor"); } showNoti...
            }}
            className="flex flex-col items-center gap-1 p-2 rounded-lg cursor-pointer transition-all group ..."
            style={{
              touchAction: "none",
              background: isSelected
                ? `${profile.accent}25`
                : isSakura ? `rgba(30,65,100,0.5)` : `rgba(30,60,110,0.6)`,
              border: `1px solid ${isSelected ? profile.accent : tc.cardBorder}`,
              boxShadow: isSelected ? `0 0 20px ${profile.glow}`, inset 0 0 15px ${profile.glow}` : 'none',
              animation: `fadeInUp 0.4s ease ${i * 60}ms both`,
            }}
            data-testid={`artist-card-${artist.name.toLowerCase().replace(/\s/g, "-")}`}>
            <div className="w-12 h-12 rounded-full overflow-hidden flex-shrink-0 transition-all"
              style={{
                border: `2px solid ${isSelected ? profile.accent : `${profile.accent}50`}`,
                boxShadow: isSelected ? `0 0 25px ${profile.accent}` : `0 0 18px ${profile.glow}`,
                background: (artistImages[artist.name] || artist.image)
                  ? `url(${artistImages[artist.name] || artist.image}) center/cover`
                  : `linear-gradient(135deg, ${profile.accent}40, rgba(30,60,115,0.7))`,
              }}
            <div>
              {artistImages[artist.name] && !artist.image && <div className="w-full h-full flex items-cent...
            </div>
            <span className="text-[11px] truncate w-full text-center font-medium transition-colors"
              style={{ color: isSelected ? profile.accent : tc.textBright }}>{artist.name}</span>
            </div>
          </div>
        );
      })}
    </div>

    {recommendations.length > 0 && (
      <div className="mt-3 pt-3" style={{ borderTop: `1px solid ${profile.accent}15` }}>
        <div className="text-[10px] uppercase tracking-[0.2em] font-bold mb-2 px-1 flex items-center gap-...
          style={{ color: `${profile.accent}60` }}>
          <Star className="h-3 w-3" />
          {isSakura ? "0J0Y0Y0•" : "You Might Like"}
        </div>
        <div className="relative overflow-hidden rounded-lg" style={{ background: `rgba(20,45,85,0.5)` }, b...
          {recommendations.map((reco, i) => {
            <div key={reco.id}
              className="flex items-center gap-2 p-2 cursor-pointer transition-all"
              style={{
                display: i === recoIndex ? 'flex' : 'none',
                animation: i === recoIndex ? 'fadeInUp 0.5s ease' : 'none',
              }}
              onClick={() => {
                doAction("play-context", "POST", { contextUri: reco.uri });
                announceTrack(reco.name, reco.name);
                showNotif(`Playing ${reco.name}`);
              }}
              data-testid={`reco-artist-${i}`}>
              {reco.image ? (
                <img src={reco.image} alt={reco.name} className="w-10 h-10 rounded-full object-cover flex...
                  style={{ border: `1.5px solid ${profile.accent}30`, boxShadow: `0 0 12px ${profile.glow...
                ) : (
                  <div className="w-10 h-10 rounded-full flex items-center justify-center" style={{ backgro...
                    <Music2 className="h-4 w-4" style={{ color: profile.accent }} />
                  </div>
                )
              )}
              <div className="flex-1 min-w-0">
                <p className="text-[11px] font-semibold truncate" style={{ color: `rgba(180,220,255,0.9)` }}...
                <p className="text-[9px]" style={{ color: `${profile.accent}50` }}>
                  {isSakura ? "0L0A0x0W0fQ•u" : "Tap to play"}
                </p>
              </div>
              <Play className="h-3 w-3 flex-shrink-0" style={{ color: `${profile.accent}60` }} />
            </div>
          )}
        </div>
        {recommendations.length > 1 && (
          <div className="flex justify-center gap-1 pb-1.5">
            {recommendations.map((_, i) => {
              <div key={i} className="w-1 h-1 rounded-full transition-all"
                style={{ background: i === recoIndex ? profile.accent : `${profile.accent}25` }} />
            )}
          </div>
        )}
      </div>
    )}
  </div>
</HoloPanel>
</div>

<div className="flex-1 relative rounded-xl overflow-hidden" style={{
  border: `1px solid ${dragItem ? `${profile.accent}50` : (isSakura ? `rgba(56,189,248,0.15)` : `rgba(50,14...
  transition: "border-color 0.3s ease, box-shadow 0.3s ease",
  boxShadow: dragItem

```

```

    ? `0 0 40px ${profile.glow}, inset 0 0 30px ${profile.glow}`
    : `inset 0 1px 0 ${isSakura ? 'rgba(56,189,248,0.12)' : 'rgba(50,140,255,0.1)'}`,
    background: isSakura ? 'rgba(14,38,62,0.55)' : 'rgba(25,50,90,0.55)',
    backdropFilter: "blur(20px)",
    perspective: '1200px',
  }}>
<div className="absolute inset-0" style={{
  transform: viewSpinning ? 'rotateY(90deg)' : 'rotateY(0deg)',
  opacity: viewSpinning ? 0 : 1,
  transition: 'transform 0.3s cubic-bezier(0.4, 0, 0.2, 1), opacity 0.25s ease',
  transformStyle: 'preserve-3d',
}}>
<div className="absolute top-0 left-0 right-0 h-[1px]" style={{
  background: isSakura ? 'linear-gradient(90deg, transparent, rgba(56,189,248,0.15), transparent)' : 'lin...
}} />

{showSearch && (
  <div className="absolute inset-0 z-20 p-4 overflow-y-auto" style={{ background: isSakura ? 'rgba(18,45,...
    <div className="flex items-center gap-2 mb-3">
      <div className="flex-1 flex items-center gap-2 px-3 py-2 rounded-lg" style={{
        background: isSakura ? 'rgba(30,65,100,0.5)' : 'rgba(28,55,100,0.6)', border: '1px solid ${tc.car...
      }}>
        <Search className="h-3.5 w-3.5" style={{ color: tc.voiceOff }} />
        <input
          type="text" value={searchQuery} placeholder="Search songs, artists..."
          onChange={e => {
            setSearchQuery(e.target.value);
            if (searchTimeout.current) clearTimeout(searchTimeout.current);
            searchTimeout.current = setTimeout(() => doSearch(e.target.value), 500);
          }}
          className="flex-1 bg-transparent text-xs outline-none placeholder:text-blue-400/40"
          style={{ color: 'rgba(200,230,255,0.8)' }}
          data-testid="search-input" autoFocus
        />
      </div>
      <button onClick={() => { setShowSearch(false); setSearchQuery(""); setSearchResults([]); }}
        className="p-2 rounded-lg transition-colors" style={{ color: tc.voiceOff }}>
        <X className="h-4 w-4" />
      </button>
    </div>
    {searching && <div className="flex items-center justify-center py-8"><Loader2 className="h-5 w-5 anim...
    <div className="grid grid-cols-2 gap-2">
      {searchResults.map((r: any, i: number) => (
        <button key={i} onClick={() => {
          if (r.uri) {
            doAction("play-context", "POST", { contextUri: r.uri });
            announceTrack(r.name, r.artist || r.name);
            showNotif('Playing ${r.name}');
            setShowSearch(false);
          }
        }}
        className="flex items-center gap-2 p-2.5 rounded-lg transition-all text-left group"
        style={{ background: isSakura ? 'rgba(30,65,100,0.45)' : 'rgba(28,55,100,0.55)', border: '1px s...
        data-testid={`search-result-${i}`}>
          {r.image ? (
            <img src={r.image} alt="" className="w-9 h-9 rounded-md object-cover flex-shrink-0"
              style={{ border: '1px solid rgba(0,180,255,0.15)' }} />
          ) : (
            <div className="w-9 h-9 rounded-md flex items-center justify-center flex-shrink-0"
              style={{ background: `${profile.accent}18`, border: '1px solid rgba(60,150,255,0.15)' }}>
              <Music2 className="h-3.5 w-3.5" style={{ color: `${profile.accent}60` }} />
            </div>
          )}
          <div className="flex-1 min-w-0">
            <p className="text-[12px] font-medium truncate" style={{ color: 'rgba(200,230,255,0.8)' }}>{r...
            <p className="text-[11px] truncate" style={{ color: 'rgba(80,170,255,0.5)' }}>{r.artist || r....
          </div>
        </button>
      )}
    </div>
  ))
)

{viewMode === "floor" && (
  <div className="absolute" style={{ top: 70, bottom: 70, left: 70, right: 70 }}>
    <img src={floorPlanImg} alt="Apartment floor plan" className="absolute inset-0 w-full h-full object-c...
    style={{ filter: "brightness(0.9) contrast(1.1) saturate(0.7) hue-rotate(190deg)", opacity: 0.9 }} />
    <div className="absolute inset-0" onClick={() => setFloorSpeakerPopup(null)} style={{
      background: `radial-gradient(ellipse at center, transparent 30%, rgba(3,8,20,0.6) 100%)`,
    }} />
    {ROOM_HOTSPOTS.map(spot => {
      const isActive = activeRooms.has(spot.room);
      const isDrop = dropTarget === spot.room;
      const roomData = rooms.find(r => r.room === spot.room);
      const speakerCount = roomData?.speakers?.length || 0;
      return (
        <div key={spot.room}
          data-room-hotspot={spot.room}
          draggable onDragStart={handleDragStart("room", spot)}
          onDrop={handleRoomDrop(spot.room)}
          onDragOver={handleDragOver(spot.room)}
          onDragLeave={() => setDropTarget(null)}
          onTouchStart={handleTouchStart("room", spot)}
          onClick={() => {
            if (selectedArtist) {
              playOnRoom(spot.room, selectedArtist);
              setSelectedArtist(null);
            }
          }}
        >
          <div className="absolute cursor-pointer transition-all"
            style={{
              left: `${spot.x}%`, top: `${spot.y}%`, width: `${spot.w}%`, height: `${spot.h}%`,
              background: isActive
                ? `${profile.accent}12`
                : (isDrop || selectedArtist)
                  ? `${profile.accent}10`
                  : 'transparent',
              boxShadow: isActive

```

```

? `0 0 35px ${profile.glow}, inset 0 0 20px ${profile.glow}`
: (isDrop || selectedArtist)
? `0 0 25px ${profile.glow}`
: "none",
border: selectedArtist && isActive ? `1px dashed ${profile.accent}50` : 'none',
animation: selectedArtist && !isActive ? 'holoPulse 2s ease-in-out infinite' : 'none',
}}
data-testid={`room-${spot.room.toLowerCase().replace(/\s/g, "-")}`}>
{!spot.hideLabel && <div className="absolute inset-0 flex flex-col items-center justify-end gap...
<div className="flex items-center gap-1">
  <span className="text-[9px] font-bold uppercase tracking-wider text-center leading-tight px-1"
    style={{
      color: isActive ? profile.accent : 'rgba(255,255,255,0.95)',
      textShadow: isActive ? `0 0 10px ${profile.glow}` : `0 1px 4px rgba(0,0,0,0.7)`,
      marginRight: -2,
    }}>
    {isSakura ? (ROOM_JP[spot.room] || spot.room) : spot.room}
  </span>
  {speakerCount > 0 && (
    <div className="relative" style={{ marginLeft: 2 }}>
      <button className="flex items-center rounded-md cursor-pointer transition-all hover:sca...
        onClick={(e) => { e.stopPropagation(); e.preventDefault(); setFloorSpeakerPopup(floor...
        data-testid={`floor-speakers-${spot.room.toLowerCase().replace(/\s/g, "-")}`}>
        <img src={echoSpeakerImg} alt="Speaker" className="rounded-md object-cover"
          style={{
            width: 18, height: 18,
            filter: isActive ? `drop-shadow(0 0 4px ${profile.accent}) brightness(1.1)` : 'no...
          }} />
        </button>
      {floorSpeakerPopup === spot.room && (
        <div className="absolute rounded-lg shadow-2xl overflow-hidden"
          onClick={(e) => e.stopPropagation()}
          style={{
            zIndex: 9999,
            width: spot.room === "Everywhere" ? 220 : 192,
            maxHeight: spot.room === "Everywhere" ? 340 : undefined,
            ...(spot.x < 30
              ? { bottom: '100%', left: '0', marginBottom: 4 }
              : { bottom: '100%', left: '50%', transform: 'translateX(-50%)', marginBottom: 4...
            background: isSakura ? 'rgba(20,50,80,0.95)' : 'rgba(10,25,50,0.95)',
            border: '1px solid ${profile.accent}30',
            backdropFilter: 'blur(20px)',
          }}>
          <div className="px-2.5 py-1.5 flex items-center justify-between" style={{ borderBot...
            <span className="text-[11px] font-semibold" style={{ color: profile.accent }}>
              {isSakura ? (ROOM_JP[spot.room] || spot.room) : spot.room}
            </span>
            <button onClick={() => setFloorSpeakerPopup(null)} style={{ color: 'rgba(255,255,...
              <X className="h-3 w-3" />
            </button>
          </div>
          {spot.room === "Everywhere" ? (
            <div className="px-2 py-1.5 flex flex-col gap-0.5 overflow-y-auto" style={{ maxHe...
              {rooms.filter(r => r.room !== "Everywhere" && r.room !== "Balcony").map(roomGro...
              const roomSpeakers = roomGroup.speakers.filter(sp => spk.type !== "group");
              if (roomSpeakers.length === 0) return null;
              return (
                <div key={roomGroup.room}>
                  <div className="text-[9px] font-bold uppercase tracking-wider px-1 pt-1 p...
                    style={{ color: `${profile.accent}90` }}>
                      {isSakura ? (ROOM_JP[roomGroup.room] || roomGroup.room) : roomGroup.room}
                    </div>
                    {roomSpeakers.map(sp => {
                      const excluded = everywhereExcluded.has(spk.entityId);
                      return (
                        <button key={spk.entityId}
                          onClick={() => {
                            setEverywhereExcluded(prev => {
                              const next = new Set(prev);
                              if (next.has(spk.entityId)) next.delete(spk.entityId);
                              else next.add(spk.entityId);
                              localStorage.setItem("holomusic-everywhere-excluded", JSON.stri...
                              return next;
                            });
                          }}
                        <div className="flex items-center gap-2 w-full text-left px-2 py-1 round...
                          style={{
                            background: excluded ? 'rgba(100,100,100,0.1)' : `${profile.accen...
                            border: '1px solid ${excluded ? 'rgba(100,100,100,0.15)' : `${pro...
                            opacity: excluded ? 0.5 : 1,
                          }}
                          data-testid={`everywhere-toggle-${spk.name.toLowerCase().replace(/\...
                        <div className="w-4 h-4 rounded flex items-center justify-center fl...
                          style={{
                            background: excluded ? 'rgba(100,100,100,0.3)' : `${profile.acce...
                            border: '1px solid ${excluded ? 'rgba(100,100,100,0.3)' : profi...
                          }}>
                          {!excluded && <span className="text-[10px]" style={{ color: profi...
                        </div>
                        <img src={echoSpeakerImg} alt="Echo" className="rounded-md object-c...
                          style={{
                            width: 18, height: 18,
                            filter: excluded ? 'brightness(0.4) saturate(0)' : 'drop-shadow...
                          }} />
                        <span className="text-[10px] font-medium truncate flex-1"
                          style={{ color: excluded ? 'rgba(140,170,200,0.4)' : 'rgba(200,22...
                        </button>
                      </div>
                    </div>
                  </div>
                </div>
              </div>
            </div>
          </div>
        ) : (
          <div className="px-2 py-1.5 flex flex-col gap-1">
            {rooms.find(r => r.room === spot.room)?.speakers || []}.filter(sp => spk.type !...
            const isEcho = spk.type === "echo" || spk.type === "echo_show";
            const isSpeakerActive = isActive || activeRooms.has("Everywhere");
          </div>
        ) : (

```



```

    >>
    <div className="absolute right-0 top-1/2 -translate-y-1/2 w-3 h-3 rounded-full opacity-0 group-hover:...
      style={{ background: profile.accent, boxShadow: `0 0 8px ${profile.accent}, 0 0 16px ${profile.glow...
    </div>
  </div>
  <span className="text-[12px] tabular-nums w-10" style={{ color: tc.textMuted }}>{formatMs(nowPlaying?.dur...
</div>

<div className="flex items-center justify-center gap-6 pb-1">
  <button onClick={() => doAction("shuffle", "POST")} className="transition-all hover:scale-110"
    style={{ color: shuffleOn ? profile.accent : "rgba(120,200,255,0.6)", filter: shuffleOn ? `drop-shadow(...
    data-testid="btn-shuffle">
    <Shuffle className="h-4 w-4" />
  </button>
  <button onClick={() => doAction("previous")} className="hover:scale-110 transition-all"
    style={{ color: tc.navIdle }} data-testid="btn-prev">
    <SkipBack className="h-5 w-5" />
  </button>
  <button onClick={() => doAction(isPlaying ? "pause" : "play", "PUT")}
    className="w-12 h-12 rounded-full flex items-center justify-center transition-all hover:scale-105 activ...
    style={{
      background: `linear-gradient(135deg, ${profile.accent}cc, ${profile.accent})`,
      boxShadow: `0 0 30px ${profile.glow}, 0 0 60px ${profile.glow}, inset 0 1px 0 rgba(255,255,255,0.15)`,
      border: `1px solid ${profile.accent}60`,
    }}
    data-testid="btn-play-pause">
    {actionPending ? <Loader2 className="h-5 w-5 text-white animate-spin" /> :
      isPlaying ? <Pause className="h-5 w-5 text-white" /> : <Play className="h-5 w-5 text-white ml-0.5" />}
  </button>
  <button onClick={() => doAction("next")} className="hover:scale-110 transition-all"
    style={{ color: tc.navIdle }} data-testid="btn-next">
    <SkipForward className="h-5 w-5" />
  </button>
  <button onClick={() => doAction("repeat", "POST")} className="transition-all hover:scale-110 relative"
    style={{ color: repeatMode !== "off" ? profile.accent : tc.navIdle, filter: repeatMode !== "off" ? `dro...
    data-testid="btn-repeat">
    <Repeat className="h-4 w-4" />
    {repeatMode === "track" && <span className="absolute -top-1 -right-1 text-[6px] font-bold" style={{ col...
  </button>

  <div className="ml-8 flex items-center gap-2">
    <button onClick={() => doAction("volume", "POST", { volume: volume > 0 ? 0 : 30 })}
      className="transition-colors" style={{ color: tc.speakerIcon }} data-testid="btn-mute">
      {volume === 0 ? <VolumeX className="h-4 w-4" /> : <Volume2 className="h-4 w-4" />}
    </button>
    <input type="range" min={0} max={100} value={volume}
      onChange={e => { setVolume(+e.target.value); doAction("volume", "POST", { volume: +e.target.value }); }}
      className="w-24 holo-range" style={{ height: 3 }}
      data-testid="volume-slider" />
  </div>

  <button onClick={toggleVoiceConfirm}
    className="ml-4 flex items-center gap-1 px-2 py-1 rounded-md transition-all hover:scale-105"
    style={{
      background: voiceConfirm ? `${profile.accent}20` : (isSakura ? `rgba(56,189,248,0.1)` : `rgba(40,120,...
      border: `1px solid ${voiceConfirm ? profile.accent + '50' : (isSakura ? `rgba(56,189,248,0.2)` : `rgb...
      color: voiceConfirm ? profile.accent : tc.voiceOff,
      boxShadow: voiceConfirm ? `0 0 10px ${profile.glow}` : 'none',
    }}
    data-testid="btn-voice-confirm"
    title={isSakura ? "🔊🔊🔊" : "Voice confirmations"}>
    <Volume1 className="h-3.5 w-3.5" />
    <span className="text-[10px] font-medium">{isSakura ? "🔊🔊" : "Voice"}</span>
  </button>
</div>
</div>
</div>

<style>`
@keyframes fadeInUp { from { opacity: 0; transform: translateY(8px); } to { opacity: 1; transform: translateY...
@keyframes eqBounce { from { height: 3px; } to { height: 12px; } }
@keyframes albumPulse { 0%, 100% { box-shadow: 0 0 20px rgba(59,130,246,0.2), 0 4px 15px rgba(0,0,0,0.4); } 5...
@keyframes holoScan {
  0% { top: -2px; opacity: 0; }
  10% { opacity: 1; }
  90% { opacity: 1; }
  100% { top: 100%; opacity: 0; }
}
@keyframes holoPulse {
  0%, 100% { opacity: 0.3; }
  50% { opacity: 0.8; }
}
.holo-range {
  -webkit-appearance: none;
  background: ${isSakura ? `rgba(56,189,248,0.15)` : `rgba(60,160,255,0.12)`};
  border-radius: 4px;
  outline: none;
}
.holo-range::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 10px;
  height: 10px;
  border-radius: 50%;
  background: ${profile.accent};
  cursor: pointer;
  box-shadow: 0 0 8px ${profile.glow}, 0 0 16px ${profile.glow};
}
.holo-range::-moz-range-thumb {
  width: 10px;
  height: 10px;
  border-radius: 50%;
  background: ${profile.accent};
  cursor: pointer;
  box-shadow: 0 0 8px ${profile.glow}, 0 0 16px ${profile.glow};
  border: none;
}
`</style>
</div>

```

```
});  
}
```

`client/src/pages/ticker.tsx`

```
import { useState, useEffect, useRef, useMemo } from "react";  
  
import newsTickerLabel from "@assets/News_1773894837015.png";  
import ctvLogoPath from "@assets/CTV2_1773545440801.png";  
import cnnLogoPath from "@assets/CNN_1773536484180.png";  
import globalLogoPath from "@assets/Global_White_1773536754594.png";  
import cbcLogoPath from "@assets/cbc-news-logo-black-and-white_1773536865600.png";  
import msnbcLogoPath from "@assets/MSNBC_1773536950584.png";  
import politicoLogoPath from "@assets/Politico_1773537080711.png";  
import rawStoryLogoPath from "@assets/Raw_Story_1773607642361.png";  
import abcNewsLogoPath from "@assets/ABC_1773609250051.png";  
import weatherAlertLogoPath from "@assets/Weather_Alert_1773608511887.png";  
import bbcNewsLogoPath from "@assets/BBC_1773609711103.png";  
import forecastIconPath from "@assets/Forecast2_1773897989398.png";  
import cnTowerPath from "@assets/CN2_1773897525570.png";  
import newspaperIconPath from "@assets/Newspaper2_1773898792176.png";  
import foxNewsLogoPath from "@assets/Fox_News_1773610204651.png";  
  
const TICKER_LOGO_MAP: Record<string, { src: string; height: number }> = {  
  CNN: { src: cnnLogoPath, height: 33 },  
  CBC: { src: cbcLogoPath, height: 78 },  
  CTV: { src: ctvLogoPath, height: 42 },  
  Global: { src: globalLogoPath, height: 42 },  
  MSNBC: { src: msnbcLogoPath, height: 68 },  
  Politico: { src: politicoLogoPath, height: 57 },  
  'Raw Story': { src: rawStoryLogoPath, height: 64 },  
  'ABC News': { src: abcNewsLogoPath, height: 56 },  
  'BBC': { src: bbcNewsLogoPath, height: 64 },  
  'Fox News': { src: foxNewsLogoPath, height: 72 },  
};  
  
const WMO_DESC: Record<number, string> = { 0: 'Clear', 1: 'Mainly Clear', 2: 'Partly Cloudy', 3: 'Overcast', 45: 'Fog', 48: 'Rim...  
  
const wmoShort = (c: number) => ({ 0: 'clear', 1: 'mostly clear', 2: 'partly cloudy', 3: 'overcast', 45: 'foggy', 48: 'foggy', 51...  
  
type Headline = { title: string; source: string; link: string; publishedAt?: string };  
type WeatherData = { code: number; temp: number; windSpeed: number; isDay: boolean; daily?: { date: string; high: num...  
type PollenData = { tree: { value: number; level: string }; grass: { value: number; level: string }; weed: { value: n...  
  
function buildTickerItems(newsHeadlines: Headline[], weatherData: WeatherData | null, pollenData: PollenData | null, ...  
  const items: Headline[] = [];  
  
  for (const a of weatherAlerts) {  
    items.push({ title: `& p ${a.title}`, source: '_ALERT_', link: '' });  
  }  
  
  if (weatherData) {  
    const desc = WMO_DESC[weatherData.code] || 'Mixed';  
    items.push({ title: `= 3) {  
      const dayNames = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'];  
      const forecastParts = weatherData.daily.slice(0, 3).map(d => {  
        const dt = new Date(d.date + 'T12:00:00');  
        const dayName = dayNames[dt.getDay()];  
        return `${dayName}: ${Math.round(d.high)}°/${Math.round(d.low)}°`;  
      });  
      items.push({ title: `0<?</spa...  
  }  
  
  const US_SOURCES = ['CNN', 'Politico', 'Raw Story', 'MSNBC', 'ABC News', 'Fox News'];  
  const ca = newsHeadlines.filter(h => !US_SOURCES.includes(h.source) && h.source !== 'BBC');  
  const us = newsHeadlines.filter(h => US_SOURCES.includes(h.source));  
  const bbc = newsHeadlines.filter(h => h.source === 'BBC');  
  const max = Math.max(ca.length, us.length, bbc.length);  
  for (let i = 0; i < max; i++) {  
    if (i < ca.length) items.push(ca[i]);  
    if (i < us.length) items.push(us[i]);  
    if (i < bbc.length) items.push(bbc[i]);  
  }  
  
  return items;  
}  
  
function renderTickerHtml(headlines: Headline[]): string {  
  return headlines.map((item, i) => {  
    if (item.source === '_ALERT_') {  
      const safeTitle = item.title.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;').replace(/"/g, '&quo...  
    }  
  });  
}
```

```

    return `<span class="inline-flex items-center gap-1.5 mx-4" style="animation:tickerAlertBlink 1s ease-in-out in...
  }
  if (item.source === '_FORECAST_' || item.source === '_FORECAST_NOSEP_') {
    const forecastHtml = item.title.replace(/<b>[^<*>\b>:?(?:(Toronto Forecast|3-Day Forecast|Forecast Brief|P...
    const sep = item.source === '_FORECAST_' ? `<span class="text-white/20 mx-2">|</span>` : ` `;
    return `<span class="inline-flex items-center gap-1.5 mx-4" data-testid="weather-forecast-${i}"><span class="te...
  }
  const logoInfo = TICKER_LOGO_MAP[item.source];
  const logoHtml = logoInfo
    ? `${item.source}</span>`;
  const safeTitle = item.title.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;').replace(/"/g, '&quot;');
  let timeAgoHtml = ` `;
  if (item.publishedAt) {
    const diff = Date.now() - new Date(item.publishedAt).getTime();
    const mins = Math.floor(diff / 60000);
    if (mins >= 0 && mins <= 4320) {
      const ago = mins < 60 ? `${mins}m` : mins < 1440 ? `${Math.floor(mins / 60)}h` : `${Math.floor(mins / 1440)}d`;
      timeAgoHtml = `<span class="text-[11px]" style="color:rgba(255,255,255,0.6);margin-left:4px">${ago}</span>`;
    }
  }
  return `<a href="${item.link}" target="_blank" rel="noopener noreferrer" class="inline-flex items-center gap-1.5 ...
  }).join('');
}

export default function TickerPage() {
  const [newsHeadlines, setNewsHeadlines] = useState<Headline[]>([]);
  const [weatherData, setWeatherData] = useState<WeatherData | null>(null);
  const [pollenData, setPollenData] = useState<PollenData | null>(null);
  const [weatherAlerts, setWeatherAlerts] = useState<{ title: string; summary: string; type: string }[]>([]);
  const tickerRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    const fetchNews = async () => {
      try {
        const res = await fetch('/api/news');
        if (res.ok) setNewsHeadlines(await res.json());
      } catch (e) { console.error('[Ticker] news fetch failed:', e); }
    };
    fetchNews();
    const interval = setInterval(fetchNews, 10 * 60 * 1000);
    return () => clearInterval(interval);
  }, []);

  useEffect(() => {
    const fetchWeather = async () => {
      try {
        const res = await fetch('/api/weather');
        if (res.ok) setWeatherData(await res.json());
      } catch (e) { console.error('[Ticker] weather fetch failed:', e); }
    };
    fetchWeather();
    const interval = setInterval(fetchWeather, 15 * 60 * 1000);
    return () => clearInterval(interval);
  }, []);

  useEffect(() => {
    const fetchPollen = async () => {
      try {
        const res = await fetch('/api/pollen');
        if (res.ok) setPollenData(await res.json());
      } catch (e) { console.error('[Ticker] pollen fetch failed:', e); }
    };
    fetchPollen();
    const interval = setInterval(fetchPollen, 30 * 60 * 1000);
    return () => clearInterval(interval);
  }, []);

  useEffect(() => {
    const fetchAlerts = async () => {
      try {
        const res = await fetch('/api/weather-alerts');
        if (res.ok) setWeatherAlerts(await res.json());
      } catch (e) { console.error('[Ticker] alerts fetch failed:', e); }
    };
    fetchAlerts();
    const interval = setInterval(fetchAlerts, 10 * 60 * 1000);
    return () => clearInterval(interval);
  }, []);

  const allHeadlines = useMemo(() =>
    buildTickerItems(newsHeadlines, weatherData, pollenData, weatherAlerts),
    [newsHeadlines, weatherData, pollenData, weatherAlerts]
  );

  useEffect(() => {
    if (!tickerRef.current || allHeadlines.length === 0) return;
    const html = renderTickerHtml(allHeadlines);
    const scrollContainer = tickerRef.current.querySelector('.news-ticker-scroll') as HTMLElement;
    if (scrollContainer) {
      scrollContainer.innerHTML = html;
      const applyAnim = () => {
        const parentWidth = scrollContainer.parentElement?.clientWidth || window.innerWidth;
        const startScroll = () => {
          scrollContainer.style.transform = ` `;
          const contentWidth = scrollContainer.scrollWidth;
          const totalTravel = parentWidth + contentWidth;
          const speed = 65;
          const dur = totalTravel / speed;
          scrollContainer.style.setProperty('--ticker-start', `${parentWidth}px`);
          scrollContainer.style.setProperty('--ticker-end', `-${contentWidth}px`);
          scrollContainer.style.animation = `tickerScroll ${dur}s linear infinite `;
        };
        startScroll();
      };
      const imgs = scrollContainer.querySelectorAll('img');
      if (imgs.length > 0) {
        let loaded = 0;

```

```

        const onLoad = () => { loaded++; if (loaded >= imgs.length) requestAnimationFrame(applyAnim); };
        imgs.forEach(img => { if (img.complete) { loaded++; } else { img.addEventListener('load', onLoad); img.addEve...
        if (loaded >= imgs.length) requestAnimationFrame(applyAnim);
        setTimeout(applyAnim, 500);
    } else {
        requestAnimationFrame(applyAnim);
    }
    }, [allHeadlines]);

return (
    <div
        style={{ width: '100vw', height: '38px', overflow: 'hidden', background: 'transparent', margin: 0, padding: 0 }}
        data-testid="ticker-page"
    >
        <div
            ref={tickerRef}
            className="flex overflow-hidden"
            style={{
                height: '38px',
                background: 'linear-gradient(90deg,#000000 0%,#14141e 50%,#000000 100%)',
                borderTop: '1px solid rgba(255,255,255,0.15)',
            }}
            data-testid="news-ticker"
        >
            <div className="flex-shrink-0 flex items-center justify-center" style={{ height: '38px', width: 'auto' }}>
                <img src={newsTickerLabel} alt="NEWS" style={{ height: '38px', width: 'auto', objectFit: 'contain' }} />
            </div>
            <div className="flex-1 overflow-hidden relative h-full">
                <div className="flex items-center h-full whitespace-nowrap news-ticker-scroll" style={{ position: 'relative...
            </div>
        </div>
    </div>
);
}

```

`shared/electiveCourses.ts`

```

export interface ElectiveCourse {
    code: string;
    name: string;
    level: 'LOWER' | 'UPPER' | 'BOTH';
    category: string;
    prereq?: boolean;
    antireq?: string;
    semesters: {
        fall2025?: string;
        winter2026?: string;
        spring2026?: string;
        summer2026?: string;
    };
}

export const LIBERAL_STUDIES_COURSES: ElectiveCourse[] = [
    // ARABIC
    { code: 'CARB 101', name: 'Introductory Arabic I', level: 'LOWER', category: 'Arabic', semesters: { fall2025: 'T: V...' },
    { code: 'CARB 201', name: 'Introductory Arabic II', level: 'LOWER', category: 'Arabic', prereq: true, semesters: { ...
    { code: 'CARB 401', name: 'Intermediate Arabic II', level: 'UPPER', category: 'Arabic', prereq: true, semesters: { ...
    { code: 'CARB 501', name: 'Advanced Arabic I', level: 'UPPER', category: 'Arabic', prereq: true, semesters: { fall2...
    { code: 'CARB 601', name: 'Advanced Arabic II', level: 'UPPER', category: 'Arabic', prereq: true, semesters: { wint...

    // AMERICAN SIGN LANGUAGE
    { code: 'CASL 101', name: 'Introductory American Sign Language I', level: 'LOWER', category: 'ASL', semesters: { fa...
    { code: 'CASL 201', name: 'Introductory American Sign Language II', level: 'LOWER', category: 'ASL', prereq: true, ...
    { code: 'CASL 301', name: 'Intermediate American Sign Language I', level: 'UPPER', category: 'ASL', prereq: true, s...

    // BIOLOGY
    { code: 'BMS 150', name: 'Introduction to the Human Genome', level: 'LOWER', category: 'Biology', semesters: { spri...
    { code: 'CBLG 599', name: 'Biology Facts in Pop Media Sci-Fiction', level: 'UPPER', category: 'Biology', semesters:...
    { code: 'CBLG 655', name: 'Viruses Among Us', level: 'UPPER', category: 'Biology', semesters: { fall2025: 'T: Virtu...
    { code: 'CBLG 699', name: 'Social Factors in Drug Development', level: 'UPPER', category: 'Biology', semesters: { s...
    { code: 'CBLG 850', name: 'What is Cancer?', level: 'UPPER', category: 'Biology', semesters: { fall2025: 'M/W/R: Vi...

    // CARIBBEAN STUDIES
    { code: 'CCRB 100', name: 'Introduction to the Caribbean', level: 'LOWER', category: 'Caribbean Studies', semesters...
    { code: 'CCRB 500', name: 'Families in the Caribbean', level: 'UPPER', category: 'Caribbean Studies', semesters: { ...
    { code: 'CCRB 501', name: 'Racism and Caribbean Peoples in Canada', level: 'UPPER', category: 'Caribbean Studies', ...
    { code: 'CCRB 502', name: 'Cultural Traditions in the Caribbean', level: 'UPPER', category: 'Caribbean Studies', se...

    // CHEMISTRY
    { code: 'CHY 183', name: 'Introduction to Forensic Science', level: 'LOWER', category: 'Chemistry', semesters: { f...
    { code: 'CHY 583', name: 'Alternative Energies', level: 'UPPER', category: 'Chemistry', semesters: { fall2025: 'M/...
    { code: 'CHY 599', name: 'The Business of Chemistry and Biology', level: 'UPPER', category: 'Chemistry', semesters...

    // CHINESE
    { code: 'CCHN 101', name: 'Introductory Chinese I', level: 'LOWER', category: 'Chinese', semesters: { fall2025: 'W:...
    { code: 'CCHN 201', name: 'Introductory Chinese II', level: 'LOWER', category: 'Chinese', prereq: true, semesters: {...
    { code: 'CCHN 301', name: 'Intermediate Chinese I', level: 'UPPER', category: 'Chinese', prereq: true, semesters: {...

    // CRIMINOLOGY
    { code: 'CCRM 101', name: 'Understanding Crime in Canada', level: 'LOWER', category: 'Criminology', semesters: { fa...
    { code: 'CRM 601', name: 'Violence and Society', level: 'UPPER', category: 'Criminology', semesters: { spring2026: ...

    // ECONOMICS
    { code: 'CECN 210', name: 'Understanding Economics', level: 'LOWER', category: 'Economics', antireq: 'ECN104, ECN20...
    { code: 'CECN 340', name: 'The Economics of Human Behavior', level: 'LOWER', category: 'Economics', semesters: { wi...
    { code: 'CECN 503', name: 'Economic Development', level: 'UPPER', category: 'Economics', antireq: 'ECN726', semeste...
    { code: 'CECN 511', name: 'Economy and the Environment', level: 'UPPER', category: 'Economics', antireq: 'ECN510', ...
    { code: 'CECN 512', name: 'Economics of Sex', level: 'UPPER', category: 'Economics', semesters: { fall2025: 'R: Vir...
    { code: 'CECN 603', name: 'Economic Issues in Globalization', level: 'UPPER', category: 'Economics', semesters: { s...
    { code: 'CECN 722', name: 'Economic Issues in Professional Sports', level: 'UPPER', category: 'Economics', semester...

    // ENGLISH
    { code: 'CENG 101', name: 'Laughter and Tears: Comedy and Tragedy', level: 'LOWER', category: 'English', semesters:...

```

```

{ code: 'CENG 104', name: 'The Short Story', level: 'LOWER', category: 'English', semesters: { fall2025: 'W: Virtua...
{ code: 'CENG 112', name: 'Zap, Pow, Bang: Pop Literature', level: 'LOWER', category: 'English', semesters: { fall2...
{ code: 'CENG 203', name: 'The Literature of Indigenous Peoples', level: 'LOWER', category: 'English', semesters: { ...
{ code: 'CENG 212', name: 'Cultures in Crisis', level: 'LOWER', category: 'English', semesters: { spring2026: 'TR: ...
{ code: 'CENG 201', name: 'Myth and Literature', level: 'LOWER', category: 'English', semesters: { summer2026: 'MW:...
{ code: 'CENG 503', name: 'Science Fiction', level: 'UPPER', category: 'English', semesters: { spring2026: 'MW: Vir...
{ code: 'CENG 505', name: 'Creative Writing', level: 'BOTH', category: 'English', semesters: { fall2025: 'M/R: Virt...
{ code: 'CENG 510', name: 'Gothic Horror', level: 'UPPER', category: 'English', semesters: { winter2026: 'W: Virtua...
{ code: 'CENG 511', name: 'The Art of Writing Life', level: 'UPPER', category: 'English', semesters: { fall2025: 'M:...
{ code: 'CENG 602', name: 'Women's Writing', level: 'UPPER', category: 'English', semesters: { fall2025: 'Online', ...

// FRENCH
{ code: 'CFRE 101', name: 'Introductory French I', level: 'LOWER', category: 'French', semesters: { spring2026: 'TR...
{ code: 'CFRE 201', name: 'Introductory French II', level: 'LOWER', category: 'French', prereq: true, semesters: { ...
{ code: 'CFRE 301', name: 'Intermediate French I', level: 'UPPER', category: 'French', prereq: true, semesters: { s...
{ code: 'CFRE 302', name: 'French Food, Wine and Hospitality', level: 'LOWER', category: 'French', semesters: { spr...
{ code: 'CFRE 401', name: 'Intermediate French II', level: 'UPPER', category: 'French', prereq: true, semesters: { ...
{ code: 'CFRE 501', name: 'Speaking and Writing French', level: 'UPPER', category: 'French', prereq: true, semester...

// GEOGRAPHY
{ code: 'CGEO 106', name: 'Geography of Everyday Life', level: 'LOWER', category: 'Geography', semesters: { fall202...
{ code: 'CGEO 108', name: 'Geography of the Global Village', level: 'LOWER', category: 'Geography', semesters: { fa...
{ code: 'CGEO 110', name: 'The Physical Environment', level: 'LOWER', category: 'Geography', semesters: { fall2025:...
{ code: 'CGEO 208', name: 'Geography of the Global Economy', level: 'LOWER', category: 'Geography', semesters: { wi...
{ code: 'CGEO 605', name: 'The Geography of the Canadian North', level: 'UPPER', category: 'Geography', semesters: ...
{ code: 'CGEO 702', name: 'Technology and the Contemporary Environment', level: 'UPPER', category: 'Geography', sem...
{ code: 'CGEO 793', name: 'The Geography of Toronto', level: 'UPPER', category: 'Geography', semesters: { fall2025:...
{ code: 'CGEO 802', name: 'The Geography of Recreation', level: 'UPPER', category: 'Geography', semesters: { spring...
{ code: 'CGEO 820', name: 'The Outer Landscape of Culture', level: 'UPPER', category: 'Geography', semesters: { fal...

// HISTORY
{ code: 'CHST 118', name: 'The City in History', level: 'LOWER', category: 'History', semesters: { winter2026: 'W: ...
{ code: 'CHST 119', name: 'Rise of Empires: History Through Film', level: 'LOWER', category: 'History', semesters: ...
{ code: 'CHST 207', name: 'Introduction to Ancient Greece and Rome', level: 'LOWER', category: 'History', semesters...
{ code: 'CHST 219', name: 'Decolonization: History Through Film', level: 'LOWER', category: 'History', semesters: {...
{ code: 'CHST 222', name: 'The History of the Caribbean', level: 'LOWER', category: 'History', semesters: { spring2...
{ code: 'CHST 501', name: 'The American Civil War', level: 'UPPER', category: 'History', semesters: { summer2026: '...'
{ code: 'CHST 503', name: 'Crime and Punishment in Modern Canada', level: 'UPPER', category: 'History', semesters: ...
{ code: 'CHST 506', name: 'The Ancient Egyptian World', level: 'UPPER', category: 'History', semesters: { winter202...
{ code: 'CHST 527', name: 'Toronto: Wilderness to Metropolis', level: 'UPPER', category: 'History', semesters: { fa...
{ code: 'CHST 533', name: 'Africa Before 1850', level: 'UPPER', category: 'History', semesters: { spring2026: 'Onli...
{ code: 'CHST 602', name: 'Propaganda', level: 'UPPER', category: 'History', semesters: { fall2025: 'R: Virtual', w...
{ code: 'CHST 603', name: 'The Third Reich', level: 'UPPER', category: 'History', semesters: { spring2026: 'MW: Vir...
{ code: 'CHST 604', name: 'The Uneasy Peace: The Cold War', level: 'UPPER', category: 'History', semesters: { fall2...
{ code: 'CHST 633', name: 'Modern Africa', level: 'UPPER', category: 'History', semesters: { spring2026: 'Online' }},
{ code: 'CHST 658', name: 'Sex in the City', level: 'UPPER', category: 'History', semesters: { fall2025: 'M: Virtua...
{ code: 'CHST 701', name: 'Scientific Technology and Society', level: 'UPPER', category: 'History', semesters: { su...
{ code: 'CHST 702', name: 'First World War', level: 'UPPER', category: 'History', semesters: { spring2026: 'MW: Vir...
{ code: 'CHST 711', name: 'Canada and the United States', level: 'UPPER', category: 'History', semesters: { fall202...
{ code: 'CHST 787', name: 'Astronomy vs Astrology', level: 'UPPER', category: 'History', semesters: { fall2025: 'T:...
{ code: 'CHST 802', name: 'Second World War', level: 'UPPER', category: 'History', semesters: { summer2026: 'MW: Vi...
{ code: 'CHST 811', name: 'The Holocaust', level: 'UPPER', category: 'History', semesters: { summer2026: 'TR: Virtu...

// INTERIOR DESIGN / ART
{ code: 'CIRL 100', name: 'Introduction to World Art I: Pictorial Arts', level: 'LOWER', category: 'Art', semesters...
{ code: 'CIRL 500', name: 'Modern and Contemporary Art, Design', level: 'UPPER', category: 'Art', semesters: { spr...

// MUSIC
{ code: 'CMUS 101', name: 'Intro to World and Early European Music', level: 'LOWER', category: 'Music', semesters: ...
{ code: 'CMUS 106', name: 'The Architecture of Music', level: 'LOWER', category: 'Music', semesters: { spring2026: ...
{ code: 'CMUS 201', name: 'Introduction to Classical Music', level: 'LOWER', category: 'Music', semesters: { fall20...
{ code: 'CMUS 220', name: 'Global Popular Music', level: 'LOWER', category: 'Music', semesters: { winter2026: 'R: V...
{ code: 'CMUS 501', name: 'Music of World Cultures', level: 'UPPER', category: 'Music', semesters: { fall2025: 'W: ...
{ code: 'CMUS 503', name: 'Social Issues in Popular Music', level: 'UPPER', category: 'Music', semesters: { winter2...
{ code: 'CMUS 505', name: 'The History of Pop Music', level: 'UPPER', category: 'Music', semesters: { winter2026: '...'
{ code: 'CMUS 520', name: 'Women in Popular Music', level: 'UPPER', category: 'Music', semesters: { spring2026: 'TR...

// PHILOSOPHY
{ code: 'CPHL 101', name: 'Plato and the Roots of Western Philosophy', level: 'LOWER', category: 'Philosophy', seme...
{ code: 'CPHL 110', name: 'Philosophy of Religion', level: 'LOWER', category: 'Philosophy', semesters: { fall2025: ...
{ code: 'CPHL 187', name: 'Ancient Greek Philosophy', level: 'LOWER', category: 'Philosophy', semesters: { summer20...
{ code: 'CPHL 201', name: 'Problems in Philosophy', level: 'LOWER', category: 'Philosophy', semesters: { fall2025: ...
{ code: 'CPHL 214', name: 'Critical Thinking', level: 'LOWER', category: 'Philosophy', antireq: 'SSH105', semesters...
{ code: 'CPHL 306', name: 'Freedom, Equality, Limits of Authority', level: 'LOWER', category: 'Philosophy', semeste...
{ code: 'CPHL 333', name: 'Philosophy of Human Nature', level: 'LOWER', category: 'Philosophy', semesters: { fall20...
{ code: 'CPHL 366', name: 'Existentialism and Art and Culture', level: 'LOWER', category: 'Philosophy', semesters: ...
{ code: 'CPHL 406', name: 'Issues of Life, Death and Poverty', level: 'LOWER', category: 'Philosophy', semesters: {...
{ code: 'CPHL 500', name: 'Philosophy of the Natural Environment', level: 'UPPER', category: 'Philosophy', semester...
{ code: 'CPHL 504', name: 'Philosophy of Art', level: 'UPPER', category: 'Philosophy', semesters: { winter2026: 'R:...
{ code: 'CPHL 509', name: 'Bioethics', level: 'UPPER', category: 'Philosophy', semesters: { fall2025: 'Online', win...
{ code: 'CPHL 550', name: 'Knowledge, Truth and Belief', level: 'UPPER', category: 'Philosophy', semesters: { fall2...
{ code: 'CPHL 551', name: 'Metaphysics', level: 'UPPER', category: 'Philosophy', semesters: { fall2025: 'R: Virtual...
{ code: 'CPHL 603', name: 'The Nature of Ethics', level: 'UPPER', category: 'Philosophy', semesters: { spring2026: ...
{ code: 'CPHL 605', name: 'Existentialism', level: 'UPPER', category: 'Philosophy', semesters: { spring2026: 'TR: V...
{ code: 'CPHL 606', name: 'Philosophy of Love and Sex', level: 'UPPER', category: 'Philosophy', semesters: { fall20...
{ code: 'CPHL 611', name: 'Philosophy of Mind', level: 'UPPER', category: 'Philosophy', semesters: { fall2025: 'W: ...
{ code: 'CPHL 612', name: 'Philosophy of Law', level: 'UPPER', category: 'Philosophy', semesters: { fall2025: 'Onli...
{ code: 'CPHL 708', name: 'Early Modern Philosophy', level: 'UPPER', category: 'Philosophy', semesters: { winter202...
{ code: 'CPHL 709', name: 'Religion, Science and Philosophy', level: 'UPPER', category: 'Philosophy', semesters: { ...
{ code: 'CPHL 710', name: 'Philosophy and Film', level: 'UPPER', category: 'Philosophy', semesters: { winter2026: '...'

// PHYSICS
{ code: 'CPCS 181', name: 'Introduction to Astronomy', level: 'LOWER', category: 'Physics', semesters: { fall2025: ...
{ code: 'CPCS 182', name: 'Life in the Milky Way Galaxy', level: 'LOWER', category: 'Physics', semesters: { fall202...
{ code: 'CPCS 581', name: 'Advanced Topics in Astronomy', level: 'UPPER', category: 'Physics', semesters: { fall202...

// PSYCHOLOGY (Liberal)
{ code: 'CPSY 105', name: 'Perspectives in Psychology', level: 'LOWER', category: 'Psychology', antireq: 'PSY102', ...
{ code: 'CPSY 308', name: 'Psychology of Thinking', level: 'LOWER', category: 'Psychology', antireq: 'PSY108', seme...
{ code: 'CPSY 504', name: 'Social Psychology', level: 'UPPER', category: 'Psychology', prereq: true, semesters: { f...
{ code: 'CPSY 505', name: 'Personality Theory', level: 'UPPER', category: 'Psychology', prereq: true, semesters: {...
{ code: 'CPSY 606', name: 'Abnormal Psychology', level: 'UPPER', category: 'Psychology', prereq: true, semesters: {...
{ code: 'CPSY 607', name: 'Drugs and Human Behaviour', level: 'UPPER', category: 'Psychology', prereq: true, semest...
{ code: 'CPSY 614', name: 'Psychology of Sport', level: 'UPPER', category: 'Psychology', prereq: true, semesters: {...
{ code: 'CPSY 621', name: 'Psychology of Human Sexuality', level: 'UPPER', category: 'Psychology', prereq: true, se...
{ code: 'CPSY 706', name: 'Positive Psychology', level: 'UPPER', category: 'Psychology', prereq: true, semesters: {...
{ code: 'CPSY 707', name: 'Models of Stress and Adaptation', level: 'UPPER', category: 'Psychology', prereq: true, ...
{ code: 'CPSY 807', name: 'Psychology of Prejudice', level: 'UPPER', category: 'Psychology', prereq: true, semester...

```

```

// MEDIA
{ code: 'CRTA 180', name: 'Music and Film', level: 'LOWER', category: 'Media', semesters: { fall2025: 'T/W/R: Virtu...

// RELIGION
{ code: 'CREL 101', name: 'Introduction to World Religion', level: 'LOWER', category: 'Religion', semesters: { fall...

// SOCIOLOGY (Liberal)
{ code: 'CSOC 103', name: 'How Society Works', level: 'LOWER', category: 'Sociology', antireq: 'SOC104', semesters:...
{ code: 'CSOC 202', name: 'Popular Culture', level: 'LOWER', category: 'Sociology', semesters: { fall2025: 'Online'...
{ code: 'CSOC 203', name: 'Social Class and Inequality', level: 'LOWER', category: 'Sociology', semesters: { winter...
{ code: 'CSOC 506', name: 'Health and Society', level: 'UPPER', category: 'Sociology', prereq: true, semesters: { f...
{ code: 'CSOC 507', name: 'Race and Ethnicity in Canadian Society', level: 'UPPER', category: 'Sociology', semester...
{ code: 'CSOC 603', name: 'Sociology of Gender', level: 'UPPER', category: 'Sociology', prereq: true, semesters: { ...
{ code: 'CSOC 633', name: 'Sexualities, Identities and Society', level: 'UPPER', category: 'Sociology', semesters: ...
{ code: 'CSOC 808', name: 'Sociology of Food and Eating', level: 'BOTH', category: 'Sociology', semesters: { fall20...
{ code: 'CSOC 880', name: 'Information Technology and Society', level: 'UPPER', category: 'Sociology', semesters: {...
{ code: 'CSOC 885', name: 'Women and Islam', level: 'UPPER', category: 'Sociology', semesters: { fall2025: 'Online'...

// SPANISH
{ code: 'CSPN 101', name: 'Introductory Spanish I', level: 'LOWER', category: 'Spanish', semesters: { fall2025: 'M/...
{ code: 'CSPN 201', name: 'Introductory Spanish II', level: 'LOWER', category: 'Spanish', prereq: true, semesters: ...
{ code: 'CSPN 301', name: 'Intermediate Spanish I', level: 'BOTH', category: 'Spanish', prereq: true, semesters: { ...
};

export const OPEN_ELECTIVE_COURSES: ElectiveCourse[] = [
// ACCOUNTING
{ code: 'CACC 100', name: 'Introductory Financial Accounting', level: 'LOWER', category: 'Accounting', antireq: 'AC...
{ code: 'CACC 110', name: 'Financial Accounting', level: 'LOWER', category: 'Accounting', antireq: 'ACC100', semest...
{ code: 'CACC 333', name: 'Core Concepts of Accounting', level: 'UPPER', category: 'Accounting', prereq: true, seme...
{ code: 'CACC 406', name: 'Introductory Management Accounting I', level: 'UPPER', category: 'Accounting', prereq: t...
{ code: 'CACC 410', name: 'Management Accounting', level: 'UPPER', category: 'Accounting', prereq: true, semesters:...
{ code: 'CACC 450', name: 'Intermediate Financial Accounting', level: 'UPPER', category: 'Accounting', prereq: true...
{ code: 'CACC 521', name: 'Auditing', level: 'UPPER', category: 'Accounting', prereq: true, semesters: { winter2026...
{ code: 'CACC 522', name: 'Taxation for Managers and Financial Planners', level: 'UPPER', category: 'Accounting', p...
{ code: 'CACC 550', name: 'Intermediate Financial Accounting II', level: 'UPPER', category: 'Accounting', prereq: t...
{ code: 'CACC 696', name: 'Information Systems Audit and Data Analytics', level: 'UPPER', category: 'Accounting', p...
{ code: 'CACC 703', name: 'Advanced Financial Accounting', level: 'UPPER', category: 'Accounting', prereq: true, se...
{ code: 'CACC 750', name: 'Taxation', level: 'UPPER', category: 'Accounting', prereq: true, semesters: { winter2026...
{ code: 'CACC 801', name: 'Intermediate Cost and Management Accounting', level: 'UPPER', category: 'Accounting', pr...
{ code: 'CACC 842', name: 'Canadian Business Taxation II', level: 'UPPER', category: 'Accounting', prereq: true, se...

// BUSINESS COMMUNICATIONS
{ code: 'CCMN 114', name: 'Short Management Reports', level: 'LOWER', category: 'Communications', semesters: { fall...
{ code: 'CCMN 279', name: 'Introduction to Professional Communication', level: 'LOWER', category: 'Communications',...
{ code: 'CCMN 288', name: 'Communities and Social Media', level: 'LOWER', category: 'Communications', semesters: { ...
{ code: 'CCMN 304', name: 'Career Advancement Communications', level: 'UPPER', category: 'Communications', semester...
{ code: 'CCMN 313', name: 'Organizational Problem Solving and Report Writing', level: 'UPPER', category: 'Communica...
{ code: 'CCMN 314', name: 'Professional Presentations', level: 'UPPER', category: 'Communications', prereq: true, s...
{ code: 'CCMN 315', name: 'Issues in Organizational Communications', level: 'UPPER', category: 'Communications', pr...
{ code: 'CCMN 408', name: 'Proposal and Grant Writing', level: 'UPPER', category: 'Communications', semesters: { sp...
{ code: 'CCMN 413', name: 'Corporate Communications', level: 'UPPER', category: 'Communications', prereq: true, sem...
{ code: 'CCMN 414', name: 'Interpersonal Communications', level: 'UPPER', category: 'Communications', prereq: true,...
{ code: 'CCMN 443', name: 'Contemporary Intercultural Communications', level: 'UPPER', category: 'Communications', ...

// CRIMINOLOGY (OE)
{ code: 'CCRM 100', name: 'Introduction to Canadian Criminal Justice', level: 'LOWER', category: 'Criminology', ant...
{ code: 'CCRM 102', name: 'Introduction to Criminology', level: 'LOWER', category: 'Criminology', antireq: 'CRM101'...
{ code: 'CCRM 200', name: 'Criminal Law', level: 'UPPER', category: 'Criminology', prereq: true, semesters: { fall2...
{ code: 'CCRM 300', name: 'Policing in Canada', level: 'UPPER', category: 'Criminology', prereq: true, semesters: {...
{ code: 'CCRM 302', name: 'Criminological Theories', level: 'UPPER', category: 'Criminology', prereq: true, semeste...
{ code: 'CCRM 318', name: 'Violence and Communities', level: 'UPPER', category: 'Criminology', semesters: { fall202...
{ code: 'CCRM 402', name: 'Criminal Justice and Social Inequality', level: 'UPPER', category: 'Criminology', semest...

// ECONOMICS (OE)
{ code: 'CECN 104', name: 'Introductory Microeconomics', level: 'LOWER', category: 'Economics', antireq: 'ECN110', ...
{ code: 'CECN 204', name: 'Introductory Macroeconomics', level: 'LOWER', category: 'Economics', antireq: 'ECN210', ...
{ code: 'CECN 301', name: 'Intermediate Macroeconomics I', level: 'UPPER', category: 'Economics', prereq: true, sem...
{ code: 'CECN 321', name: 'Introduction to Law and Economics', level: 'UPPER', category: 'Economics', prereq: true,...
{ code: 'CECN 504', name: 'Intermediate Microeconomics I', level: 'UPPER', category: 'Economics', prereq: true, sem...
{ code: 'CECN 506', name: 'Money and Banking', level: 'UPPER', category: 'Economics', prereq: true, semesters: { sp...
{ code: 'CECN 600', name: 'Intermediate Macroeconomics II', level: 'UPPER', category: 'Economics', prereq: true, se...
{ code: 'CECN 606', name: 'International Monetary Economics', level: 'UPPER', category: 'Economics', prereq: true, ...
{ code: 'CECN 620', name: 'Applied Economic Analysis', level: 'UPPER', category: 'Economics', prereq: true, semeste...
{ code: 'CECN 627', name: 'Econometrics', level: 'UPPER', category: 'Economics', prereq: true, semesters: { spring2...
{ code: 'CECN 640', name: 'Economics of Immigration', level: 'UPPER', category: 'Economics', prereq: true, semester...
{ code: 'CECN 700', name: 'Intermediate Microeconomics II', level: 'UPPER', category: 'Economics', prereq: true, se...
{ code: 'CECN 702', name: 'Econometrics II', level: 'UPPER', category: 'Economics', prereq: true, semesters: { summ...
{ code: 'CECN 707', name: 'Economics of International Trade', level: 'UPPER', category: 'Economics', prereq: true, ...

// ENGLISH (OE)
{ code: 'CENG 110', name: 'Literatures Across Borders', level: 'LOWER', category: 'English', semesters: { fall2025:...
{ code: 'CENG 222', name: 'Fairy Tales and Fantasies', level: 'LOWER', category: 'English', semesters: { fall2025: ...
{ code: 'CENG 224', name: 'Children's Literature', level: 'LOWER', category: 'English', semesters: { winter2026: 'R...
{ code: 'CENG 230', name: 'Creativity, Writing and Everyday Life', level: 'LOWER', category: 'English', semesters: ...
{ code: 'CENG 706', name: 'Shakespeare and Performance', level: 'UPPER', category: 'English', prereq: true, semeste...
{ code: 'CSSH 102', name: 'Learning and Development Strategies', level: 'LOWER', category: 'Social Sciences', semes...
{ code: 'CSSH 205', name: 'Academic Writing and Research', level: 'LOWER', category: 'Social Sciences', semesters: ...

// DISABILITY STUDIES
{ code: 'CDST 504', name: 'Mad People's History', level: 'UPPER', category: 'Disability Studies', semesters: { fall...
{ code: 'CDST 506', name: 'Making Ontario Accessible', level: 'UPPER', category: 'Disability Studies', semesters: {...

// FAMILY STUDIES
{ code: 'CFNF 400', name: 'The Social Content of Human Sexuality', level: 'UPPER', category: 'Family Studies', seme...

// FINANCE
{ code: 'CFIN 300', name: 'Managerial Finance I', level: 'UPPER', category: 'Finance', prereq: true, semesters: { f...
{ code: 'CFIN 401', name: 'Managerial Finance II', level: 'UPPER', category: 'Finance', prereq: true, semesters: { ...
{ code: 'CFIN 501', name: 'Investment Analysis I', level: 'UPPER', category: 'Finance', prereq: true, semesters: {...
{ code: 'CFIN 502', name: 'Personal Financial Planning', level: 'UPPER', category: 'Finance', prereq: true, semeste...
{ code: 'CFIN 512', name: 'Risk Management and Insurance', level: 'UPPER', category: 'Finance', prereq: true, semes...
{ code: 'CFIN 612', name: 'Retirement and Estate Planning', level: 'UPPER', category: 'Finance', prereq: true, seme...
{ code: 'CFIN 621', name: 'International Finance', level: 'UPPER', category: 'Finance', prereq: true, semesters: { ...

// FOOD SECURITY
{ code: 'CFNY 403', name: 'Food Security Concepts and Principles', level: 'UPPER', category: 'Food Security', semes...
{ code: 'CFNY 404', name: 'Food Policy and Programs for Food Security', level: 'UPPER', category: 'Food Security', ...

```

```

// GEOGRAPHY (OE)
{ code: 'CGEO 151', name: 'Location, Location, Location', level: 'LOWER', category: 'Geography', antireq: 'PLG300',...
{ code: 'CGEO 301', name: 'Marketing Geography', level: 'UPPER', category: 'Geography', semesters: { spring2026: 'T...
{ code: 'CGEO 691', name: 'Canadian Immigration: Patterns and Place', level: 'UPPER', category: 'Geography', semest...
{ code: 'CGEO 785', name: 'Environment and Society in the Caribbean', level: 'UPPER', category: 'Geography', semest...

// GRAPHIC COMMUNICATIONS
{ code: 'CGCM 110', name: 'Intro to Graphic Communications', level: 'LOWER', category: 'Graphic Comm', semesters: {...
{ code: 'CGCM 111', name: 'Graphic Communication Technologies', level: 'LOWER', category: 'Graphic Comm', semesters...
{ code: 'CGCM 130', name: 'Design and Layout', level: 'LOWER', category: 'Graphic Comm', semesters: { spring2026: '...
{ code: 'CGCM 210', name: 'Introduction to Packaging', level: 'LOWER', category: 'Graphic Comm', semesters: { fall2...
{ code: 'CGCM 230', name: 'Typography', level: 'UPPER', category: 'Graphic Comm', prereq: true, semesters: { winter...
{ code: 'CGCM 720', name: 'Magazine Production and Publishing', level: 'UPPER', category: 'Graphic Comm', prereq: t...
{ code: 'CGCM 738', name: 'Photoshopped! The Art of Image Retouching', level: 'UPPER', category: 'Graphic Comm', pr...
{ code: 'CGCM 740', name: 'Accessibility for Graphic Communications', level: 'UPPER', category: 'Graphic Comm', sem...
{ code: 'CGCM 746', name: 'Sustainability in Print and Packaging', level: 'UPPER', category: 'Graphic Comm', semest...

// GLOBAL MANAGEMENT
{ code: 'CGMS 200', name: 'Introduction to Global Management', level: 'LOWER', category: 'Global Management', semes...
{ code: 'CGMS 401', name: 'Operations Management', level: 'UPPER', category: 'Global Management', prereq: true, sem...
{ code: 'CGMS 402', name: 'Introduction to Management Economics', level: 'UPPER', category: 'Global Management', pr...
{ code: 'CGMS 522', name: 'International Marketing', level: 'UPPER', category: 'Global Management', prereq: true, s...
{ code: 'CGMS 690', name: 'The North American Business Environment', level: 'UPPER', category: 'Global Management',...
{ code: 'CGMS 724', name: 'Management of International Enterprise', level: 'UPPER', category: 'Global Management', ...

// HEALTHCARE
{ code: 'CHIM 301', name: 'Healthcare Information Analysis', level: 'UPPER', category: 'Healthcare', prereq: true, ...
{ code: 'CHIM 305', name: 'Introduction to Health Informatics', level: 'UPPER', category: 'Healthcare', semesters: ...
{ code: 'CHSM 301', name: 'The Healthcare System', level: 'UPPER', category: 'Healthcare', semesters: { fall2025: '...
{ code: 'CHSM 305', name: 'The Management Cycle', level: 'UPPER', category: 'Healthcare', prereq: true, semesters: ...
{ code: 'CHSM 437', name: 'Human Resources Management in Healthcare', level: 'UPPER', category: 'Healthcare', semes...

// HISTORY (OE)
{ code: 'CHIS 105', name: 'Inventing Popular Culture', level: 'LOWER', category: 'History', semesters: { summer2026...
{ code: 'CHIS 106', name: 'Technology, Warfare and Social Change', level: 'LOWER', category: 'History', semesters: ...
{ code: 'CHIS 590', name: 'Modern International Relations', level: 'UPPER', category: 'History', semesters: { sprin...

// HR MANAGEMENT
{ code: 'CMHR 523', name: 'Human Resources Management', level: 'UPPER', category: 'HR Management', semesters: { spr...

// IT MANAGEMENT
{ code: 'CITM 102', name: 'Business Information Systems', level: 'LOWER', category: 'IT Management', antireq: 'ITM1...
{ code: 'CITM 107', name: 'Managerial Decision Making', level: 'LOWER', category: 'IT Management', semesters: { fal...
{ code: 'CITM 200', name: 'Fundamentals of Programming', level: 'LOWER', category: 'IT Management', prereq: true, s...
{ code: 'CITM 207', name: 'Computer-Enabled Problem Solving', level: 'LOWER', category: 'IT Management', semesters:...
{ code: 'CITM 305', name: 'Systems Analysis and Design', level: 'UPPER', category: 'IT Management', prereq: true, s...
{ code: 'CITM 350', name: 'Concepts of e-Business', level: 'UPPER', category: 'IT Management', semesters: { spring2...
{ code: 'CITM 410', name: 'Business Process Design', level: 'UPPER', category: 'IT Management', prereq: true, semes...
{ code: 'CITM 500', name: 'Data and Information Management', level: 'UPPER', category: 'IT Management', prereq: tru...
{ code: 'CITM 750', name: 'IS Project Management', level: 'UPPER', category: 'IT Management', prereq: true, semeste...
{ code: 'CITM 820', name: 'Information Systems Security and Privacy', level: 'UPPER', category: 'IT Management', pr...

// LAW
{ code: 'CLAW 122', name: 'Business Law', level: 'LOWER', category: 'Law', semesters: { fall2025: 'M; Online', wint...
{ code: 'CLAW 525', name: 'The Law of the Marketplace', level: 'UPPER', category: 'Law', prereq: true, semesters: {...
{ code: 'CLAW 529', name: 'Employment and Labour Law', level: 'UPPER', category: 'Law', prereq: true, semesters: { ...
{ code: 'CLAW 533', name: 'Corporate Social Responsibility and the Law', level: 'UPPER', category: 'Law', prereq: t...
{ code: 'CLAW 603', name: 'Advanced Business Law', level: 'UPPER', category: 'Law', prereq: true, semesters: { wint...
{ code: 'CLAW 723', name: 'Issues in Information Technology Law', level: 'UPPER', category: 'Law', prereq: true, se...
{ code: 'CLAW 724', name: 'Legal Aspects of International Business', level: 'UPPER', category: 'Law', prereq: true,...

// NON-PROFIT MANAGEMENT
{ code: 'CINP 900', name: 'Understanding the Non-Profit Sector', level: 'UPPER', category: 'Non-Profit', semesters:...
{ code: 'CINP 901', name: 'Effective Non-profit Organizations', level: 'UPPER', category: 'Non-Profit', semesters: ...
{ code: 'CINP 902', name: 'Evaluation for Non-profits', level: 'UPPER', category: 'Non-Profit', antireq: 'PPA402', ...
{ code: 'CINP 910', name: 'Strategic Planning for Nonprofits', level: 'UPPER', category: 'Non-Profit', semesters: {...
{ code: 'CINP 911', name: 'Advocacy and Government Relations', level: 'UPPER', category: 'Non-Profit', semesters: {...
{ code: 'CINP 912', name: 'Marketing for Nonprofits', level: 'UPPER', category: 'Non-Profit', semesters: { spring20...
{ code: 'CINP 913', name: 'Leadership in Nonprofit', level: 'UPPER', category: 'Non-Profit', semesters: { winter202...
{ code: 'CINP 915', name: 'Financial Management for Non-profits', level: 'UPPER', category: 'Non-Profit', antireq: ...
{ code: 'CINP 920', name: 'Critical Issues in the Non-profit', level: 'UPPER', category: 'Non-Profit', prereq: true...

// OCC HEALTH & SAFETY
{ code: 'COHS 208', name: 'Occupational Health and Safety Law', level: 'LOWER', category: 'OHS', semesters: { fall2...
{ code: 'COHS 508', name: 'Occupational Health', level: 'UPPER', category: 'OHS', semesters: { fall2025: 'Online', ...
{ code: 'COHS 608', name: 'Hazard Recognition and Control', level: 'UPPER', category: 'OHS', semesters: { winter202...
{ code: 'COHS 811', name: 'OHSE Management Systems', level: 'UPPER', category: 'OHS', semesters: { fall2025: 'Onlin...

// PHILOSOPHY (OE)
{ code: 'CPHL 302', name: 'Ethics in Health Care', level: 'UPPER', category: 'Philosophy', semesters: { fall2025: '...'
{ code: 'CPHL 307', name: 'Business Ethics', level: 'UPPER', category: 'Philosophy', semesters: { fall2025: 'Online...

// PSYCHOLOGY (OE)
{ code: 'CPSY 102', name: 'Introduction to Psychology I', level: 'LOWER', category: 'Psychology', antireq: 'PSY105'...
{ code: 'CPSY 202', name: 'Introduction to Psychology II', level: 'LOWER', category: 'Psychology', prereq: true, se...
{ code: 'CPSY 215', name: 'Psychology of Addictions', level: 'UPPER', category: 'Psychology', prereq: true, semeste...
{ code: 'CPSY 300', name: 'Psychology of Law', level: 'UPPER', category: 'Psychology', prereq: true, semesters: { f...
{ code: 'CPSY 302', name: 'Child Development', level: 'UPPER', category: 'Psychology', prereq: true, semesters: { f...
{ code: 'CPSY 324', name: 'Biological Psychology', level: 'UPPER', category: 'Psychology', prereq: true, semesters:...
{ code: 'CPSY 325', name: 'Psychological Disorders', level: 'UPPER', category: 'Psychology', prereq: true, antireq:...
{ code: 'CPSY 335', name: 'Clinical Psychology', level: 'UPPER', category: 'Psychology', prereq: true, semesters: {...
{ code: 'CPSY 682', name: 'Sleep', level: 'UPPER', category: 'Psychology', prereq: true, semesters: { spring2026: '...'
{ code: 'CPSY 808', name: 'Community Psychology', level: 'UPPER', category: 'Psychology', prereq: true, semesters: ...

// SOCIOLOGY (OE)
{ code: 'CSOC 107', name: 'Sociology of the Everyday', level: 'LOWER', category: 'Sociology', semesters: { spring20...
{ code: 'CSOC 502', name: 'Violence and the Family', level: 'UPPER', category: 'Sociology', prereq: true, semesters...
{ code: 'CSOC 605', name: 'Canadian Families: Myth and Legal Reality', level: 'UPPER', category: 'Sociology', prere...
{ code: 'CSOC 606', name: 'Work and Families in the 21st Century', level: 'UPPER', category: 'Sociology', prereq: t...
{ code: 'CSOC 608', name: 'Women, Power and Change', level: 'UPPER', category: 'Sociology', prereq: true, semesters...

// CARIBBEAN (OE)
{ code: 'CCRB 605', name: 'Caribbean Tourism: Impacts and Resistance', level: 'UPPER', category: 'Caribbean Studies...
];

export const POG_COURSES: ElectiveCourse[] = [
  { code: 'POG 316', name: 'Social Policy', level: 'UPPER', category: 'Politics & Governance', semesters: {} },
];

```



```

{ code: 'POG 317', name: 'Education Politics and Policy', level: 'UPPER', category: 'Politics & Governance', semest...
{ code: 'POG 411', name: 'Canadian Foreign Policy', level: 'UPPER', category: 'Politics & Governance', semesters: {...
{ code: 'POG 412', name: 'Government and the Economy', level: 'UPPER', category: 'Politics & Governance', semesters: {...
{ code: 'POG 415', name: 'Environmental Politics and Policy', level: 'UPPER', category: 'Politics & Governance', se...
{ code: 'POG 443', name: 'Global Cities', level: 'UPPER', category: 'Politics & Governance', semesters: {} },
{ code: 'CPOG 417', name: 'Canadian-American Relations', level: 'UPPER', category: 'Politics & Governance', semeste...
{ code: 'CPOG 444', name: 'Politics, Media and Technology', level: 'UPPER', category: 'Politics & Governance', seme...
};

export const getCoursesForLevel = (level: 'LOWER' | 'UPPER' | 'ALL', list: ElectiveCourse[]): ElectiveCourse[] => {
  if (level === 'ALL') return list;
  return list.filter(c => c.level === level || c.level === 'BOTH');
};

```

`shared/models/chat.ts`

```

import { pgTable, serial, integer, text, timestamp } from "drizzle-orm/pg-core";
import { createInsertSchema } from "drizzle-zod";
import { z } from "zod";
import { sql } from "drizzle-orm";

export const conversations = pgTable("conversations", {
  id: serial("id").primaryKey(),
  title: text("title").notNull(),
  createdAt: timestamp("created_at").default(sql`CURRENT_TIMESTAMP`).notNull(),
});

export const messages = pgTable("messages", {
  id: serial("id").primaryKey(),
  conversationId: integer("conversation_id").notNull().references(() => conversations.id, { onDelete: "cascade" }),
  role: text("role").notNull(),
  content: text("content").notNull(),
  createdAt: timestamp("created_at").default(sql`CURRENT_TIMESTAMP`).notNull(),
});

export const insertConversationSchema = createInsertSchema(conversations).omit({
  id: true,
  createdAt: true,
});

export const insertMessageSchema = createInsertSchema(messages).omit({
  id: true,
  createdAt: true,
});

export type Conversation = typeof conversations.$inferSelect;
export type InsertConversation = z.infer<typeof insertConversationSchema>;
export type Message = typeof messages.$inferSelect;
export type InsertMessage = z.infer<typeof insertMessageSchema>;

```

`shared/routes.ts`

```

import { z } from 'zod';
import { insertTaskSchema, tasks, insertSemesterSettingsSchema, semesterSettings } from './schema';

export const errorSchemas = {
  validation: z.object({
    message: z.string(),
    field: z.string().optional(),
  }),
  notFound: z.object({
    message: z.string(),
  }),
  internal: z.object({
    message: z.string(),
  }),
};

export const api = {
  tasks: {
    list: {
      method: 'GET' as const,
      path: '/api/tasks',
      input: z.object({
        weekNumber: z.coerce.number().optional(),
        type: z.string().optional(),
        showCompleted: z.string().optional(),
      }).optional(),
      responses: {
        200: z.array(z.custom<typeof tasks.$inferSelect>()),
      },
    },
    get: {
      method: 'GET' as const,
      path: '/api/tasks/:id',
      responses: {
        200: z.custom<typeof tasks.$inferSelect>(),
        404: errorSchemas.notFound,
      },
    },
    create: {
      method: 'POST' as const,
      path: '/api/tasks',
      input: insertTaskSchema,
      responses: {
        201: z.custom<typeof tasks.$inferSelect>(),
        400: errorSchemas.validation,
      },
    },
  },
};

```

```

update: {
  method: 'PATCH' as const,
  path: '/api/tasks/:id',
  input: insertTaskSchema.partial(),
  responses: {
    200: z.custom<typeof tasks.$inferSelect>(),
    400: errorSchemas.validation,
    404: errorSchemas.notFound,
  },
},
delete: {
  method: 'DELETE' as const,
  path: '/api/tasks/:id',
  responses: {
    204: z.void(),
    404: errorSchemas.notFound,
  },
},
complete: {
  method: 'PATCH' as const,
  path: '/api/tasks/:id/complete',
  input: z.object({ isCompleted: z.boolean() }),
  responses: {
    200: z.custom<typeof tasks.$inferSelect>(),
    404: errorSchemas.notFound,
  },
},
reschedule: {
  method: 'PATCH' as const,
  path: '/api/tasks/:id/reschedule',
  input: z.object({
    dueDate: z.string(),
    weekNumber: z.coerce.number(),
  }),
  responses: {
    200: z.custom<typeof tasks.$inferSelect>(),
    404: errorSchemas.notFound,
  },
},
exportCalendar: {
  method: 'GET' as const,
  path: '/api/tasks/:id/ics',
  responses: {
    200: z.string(),
    404: errorSchemas.notFound,
  },
},
},
weeks: {
  current: {
    method: 'GET' as const,
    path: '/api/weeks/current',
    responses: {
      200: z.object({
        weekNumber: z.number(),
        startDate: z.string(),
        endDate: z.string(),
      }),
    },
  },
  list: {
    method: 'GET' as const,
    path: '/api/weeks',
    responses: {
      200: z.array(z.object({
        weekNumber: z.number(),
        startDate: z.string(),
        endDate: z.string(),
        taskCount: z.number(),
      })),
    },
  },
},
semester: {
  get: {
    method: 'GET' as const,
    path: '/api/semester',
    responses: {
      200: z.custom<typeof semesterSettings.$inferSelect>().nullable(),
    },
  },
  create: {
    method: 'POST' as const,
    path: '/api/semester',
    input: insertSemesterSettingsSchema,
    responses: {
      201: z.custom<typeof semesterSettings.$inferSelect>(),
      400: errorSchemas.validation,
    },
  },
},
};

export function buildUrl(path: string, params?: Record<string, string | number>): string {
  let url = path;
  if (params) {
    Object.entries(params).forEach(([key, value]) => {
      if (url.includes(`:${key}`)) {
        url = url.replace(`:${key}`, String(value));
      }
    });
  }
  return url;
}

```

shared/schema.ts

```

import { pgTable, text, serial, boolean, integer, timestamp, real, doublePrecision } from "drizzle-orm/pg-core";
import { createInsertSchema } from "drizzle-zod";
import { z } from "zod";

export const TASK_TYPES = [
  "class",
  "reading",
  "module",
  "essay",
  "project",
  "discussion",
  "poll",
  "exam",
  "quiz",
  "reminder",
  "meeting",
  "other"
] as const;

export const REPEAT_TYPES = [
  "none",
  "daily",
  "weekly",
  "monthly",
  "custom"
] as const;

export const REPEAT_INTERVAL_UNITS = [
  "days",
  "weeks"
] as const;

export type RepeatType = typeof REPEAT_TYPES[number];
export type RepeatIntervalUnit = typeof REPEAT_INTERVAL_UNITS[number];

export const COURSES = [
  { code: "CPPA101", name: "Cdn Public Admin I: Institutions", color: "blue" },
  { code: "CPPA102", name: "Cdn Public Admin II: Processes", color: "blue" },
  { code: "CPPA120", name: "Canadian Politics and Government", color: "blue" },
  { code: "CPPA121", name: "Ontario Politics and Government", color: "blue" },
  { code: "CPPA122", name: "Local Politics and Government", color: "green" },
  { code: "CPPA125", name: "Rights, Equity and the State", color: "blue" },
  { code: "CPPA235", name: "Issues in Canadian Politics", color: "yellow" },
  { code: "CNF400", name: "Human Sexuality", color: "pink" },
  { code: "CASL101", name: "American Sign Language", color: "purple" },
  { code: "CGCM738", name: "Photoshopped!", color: "orange" },
  { code: "CECN210", name: "Understanding Economics", color: "emerald" },
  { code: "CPHL110", name: "Philosophy of Religion", color: "blue" },
  { code: "CHIS105", name: "Inventing Popular Culture", color: "red" },
] as const;

export type Course = typeof COURSES[number];

export type TaskType = typeof TASK_TYPES[number];

// Reminder options in minutes - lots of choices
export const REMINDER_OPTIONS = [
  { value: 0, label: "None" },
  { value: 5, label: "5 minutes before" },
  { value: 10, label: "10 minutes before" },
  { value: 15, label: "15 minutes before" },
  { value: 30, label: "30 minutes before" },
  { value: 45, label: "45 minutes before" },
  { value: 60, label: "1 hour before" },
  { value: 90, label: "1.5 hours before" },
  { value: 120, label: "2 hours before" },
  { value: 180, label: "3 hours before" },
  { value: 240, label: "4 hours before" },
  { value: 360, label: "6 hours before" },
  { value: 480, label: "8 hours before" },
  { value: 720, label: "12 hours before" },
  { value: 1440, label: "1 day before" },
  { value: 2880, label: "2 days before" },
  { value: 4320, label: "3 days before" },
  { value: 10080, label: "1 week before" },
] as const;

// Default reminders: 30 min and 2 hours
export const DEFAULT_REMINDER_1 = 30; // 30 minutes
export const DEFAULT_REMINDER_2 = 120; // 2 hours

// Semester type enums
export const SEMESTER_TYPES = ["fall", "winter", "spring_summer"] as const;
export type SemesterType = typeof SEMESTER_TYPES[number];

export const DELIVERY_MODES = ["virtual", "online"] as const;
export type DeliveryMode = typeof DELIVERY_MODES[number];

export const SPRING_SUMMER_TERMS = ["full", "first_half", "second_half"] as const;
export type SpringSummerTerm = typeof SPRING_SUMMER_TERMS[number];

export const DAYS_OF_WEEK = ["monday", "tuesday", "wednesday", "thursday", "friday", "saturday", "sunday"] as const;
export type DayOfWeek = typeof DAYS_OF_WEEK[number];

// Semester settings table to store dynamic semester configuration
export const semesterSettings = pgTable("semester_settings", {
  id: serial("id").primaryKey(),
  semesterName: text("semester_name").notNull().default("Winter 2026 Semester"),
  semesterStartDate: timestamp("semester_start_date").notNull(),
  semesterEndDate: timestamp("semester_end_date"),
  semesterType: text("semester_type").default("winter"),
  course1Code: text("course1_code").notNull(),
  course1Name: text("course1_name").notNull(),
  course1Professor: text("course1_professor"),
  course1ProfessorEmail: text("course1_professor_email"),
  course1SemesterType: text("course1_semester_type"),
  course1DeliveryMode: text("course1_delivery_mode"),
});

```

```

course1ClassDay: text("course1_class_day"),
course1ClassDay2: text("course1_class_day2"),
course1ClassTime: text("course1_class_time"),
course1ClassEndTime: text("course1_class_end_time"),
course1ClassTime2: text("course1_class_time2"),
course1ClassEndTime2: text("course1_class_end_time2"),
course1StartDate: timestamp("course1_start_date"),
course1EndDate: timestamp("course1_end_date"),
course1SpringSummerTerm: text("course1_spring_summer_term"),
course1CourseType: text("course1_course_type"),
course1ZoomLink: text("course1_zoom_link"),
course1Color: text("course1_color"),
course1ColorEnd: text("course1_color_end"),
course1ColorStops: text("course1_color_stops"),
course1BorderColor: text("course1_border_color"),
course1CourseRowColor: text("course1_course_row_color"),
course1TaskBgColor: text("course1_task_bg_color"),
course1FinalGrade: integer("course1_final_grade"),
course1Completed: boolean("course1_completed").default(false),
course2Code: text("course2_code").notNull(),
course2Name: text("course2_name").notNull(),
course2Professor: text("course2_professor"),
course2ProfessorEmail: text("course2_professor_email"),
course2SemesterType: text("course2_semester_type"),
course2DeliveryMode: text("course2_delivery_mode"),
course2ClassDay: text("course2_class_day"),
course2ClassDay2: text("course2_class_day2"),
course2ClassTime: text("course2_class_time"),
course2ClassEndTime: text("course2_class_end_time"),
course2ClassTime2: text("course2_class_time2"),
course2ClassEndTime2: text("course2_class_end_time2"),
course2StartDate: timestamp("course2_start_date"),
course2EndDate: timestamp("course2_end_date"),
course2SpringSummerTerm: text("course2_spring_summer_term"),
course2CourseType: text("course2_course_type"),
course2ZoomLink: text("course2_zoom_link"),
course2Color: text("course2_color"),
course2ColorEnd: text("course2_color_end"),
course2ColorStops: text("course2_color_stops"),
course2BorderColor: text("course2_border_color"),
course2CourseRowColor: text("course2_course_row_color"),
course2TaskBgColor: text("course2_task_bg_color"),
course2FinalGrade: integer("course2_final_grade"),
course2Completed: boolean("course2_completed").default(false),
course3Code: text("course3_code").notNull(),
course3Name: text("course3_name").notNull(),
course3Professor: text("course3_professor"),
course3ProfessorEmail: text("course3_professor_email"),
course3SemesterType: text("course3_semester_type"),
course3DeliveryMode: text("course3_delivery_mode"),
course3ClassDay: text("course3_class_day"),
course3ClassDay2: text("course3_class_day2"),
course3ClassTime: text("course3_class_time"),
course3ClassEndTime: text("course3_class_end_time"),
course3ClassTime2: text("course3_class_time2"),
course3ClassEndTime2: text("course3_class_end_time2"),
course3StartDate: timestamp("course3_start_date"),
course3EndDate: timestamp("course3_end_date"),
course3SpringSummerTerm: text("course3_spring_summer_term"),
course3CourseType: text("course3_course_type"),
course3ZoomLink: text("course3_zoom_link"),
course3Color: text("course3_color"),
course3ColorEnd: text("course3_color_end"),
course3ColorStops: text("course3_color_stops"),
course3BorderColor: text("course3_border_color"),
course3CourseRowColor: text("course3_course_row_color"),
course3TaskBgColor: text("course3_task_bg_color"),
course3FinalGrade: integer("course3_final_grade"),
course3Completed: boolean("course3_completed").default(false),
secondaryCalendarId: text("secondary_calendar_id"),
readingWeekStart: timestamp("reading_week_start"),
isActive: boolean("is_active").default(true),
createdAt: timestamp("created_at").defaultNow(),
});

export const insertSemesterSettingsSchema = createInsertSchema(semesterSettings).omit({ id: true, createdAt: true });
export type SemesterSettings = typeof semesterSettings.$inferSelect;
export type InsertSemesterSettings = z.infer<typeof insertSemesterSettingsSchema>;

// Store OAuth tokens for second Google account
export const secondGoogleAccount = pgTable("second_google_account", {
  id: serial("id").primaryKey(),
  email: text("email").notNull(),
  accessToken: text("access_token").notNull(),
  refreshToken: text("refresh_token").notNull(),
  expiresAt: timestamp("expires_at").notNull(),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

export const insertSecondGoogleAccountSchema = createInsertSchema(secondGoogleAccount).omit({ id: true, createdAt: tr...
export type SecondGoogleAccount = typeof secondGoogleAccount.$inferSelect;
export type InsertSecondGoogleAccount = z.infer<typeof insertSecondGoogleAccountSchema>;

export const thirdGoogleAccount = pgTable("third_google_account", {
  id: serial("id").primaryKey(),
  email: text("email").notNull(),
  accessToken: text("access_token").notNull(),
  refreshToken: text("refresh_token").notNull(),
  expiresAt: timestamp("expires_at").notNull(),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

export const insertThirdGoogleAccountSchema = createInsertSchema(thirdGoogleAccount).omit({ id: true, createdAt: true...
export type ThirdGoogleAccount = typeof thirdGoogleAccount.$inferSelect;
export type InsertThirdGoogleAccount = z.infer<typeof insertThirdGoogleAccountSchema>;

```

```

export const files = pgTable("files", {
  id: serial("id").primaryKey(),
  originalName: text("original_name").notNull(),
  displayName: text("display_name").notNull(),
  objectPath: text("object_path").notNull().unique(),
  contentType: text("content_type"),
  size: integer("size"),
  folder: text("folder"),
  listened: boolean("listened").default(false),
  lastChunkIndex: integer("last_chunk_index").default(0),
  totalChunks: integer("total_chunks").default(0),
  checkedChunks: text("checked_chunks"),
  ttsAudioUrl: text("tts_audio_url"),
  ttsGeneratedAt: timestamp("tts_generated_at"),
  extractedText: text("extracted_text"),
  preparedAudioPaths: text("prepared_audio_paths"),
  preparedAt: timestamp("prepared_at"),
  createdAt: timestamp("created_at").defaultNow(),
});

export const insertFileSchema = createInsertSchema(files).omit({ id: true, createdAt: true });
export type FileRecord = typeof files.$inferSelect;
export type InsertFile = z.infer<typeof insertFileSchema>;

export const deletedFolders = pgTable("deleted_folders", {
  id: serial("id").primaryKey(),
  folderId: text("folder_id").notNull().unique(),
  deletedAt: timestamp("deleted_at").defaultNow(),
});

export const insertDeletedFolderSchema = createInsertSchema(deletedFolders).omit({ id: true, deletedAt: true });
export type DeletedFolder = typeof deletedFolders.$inferSelect;
export type InsertDeletedFolder = z.infer<typeof insertDeletedFolderSchema>;

export const customFolders = pgTable("custom_folders", {
  id: serial("id").primaryKey(),
  parentFolderId: text("parent_folder_id").notNull(),
  name: text("name").notNull(),
  createdAt: timestamp("created_at").defaultNow(),
});

export const insertCustomFolderSchema = createInsertSchema(customFolders).omit({ id: true, createdAt: true });
export type CustomFolder = typeof customFolders.$inferSelect;
export type InsertCustomFolder = z.infer<typeof insertCustomFolderSchema>;

// Projects table - containers for grouping related tasks and workflows
export const PROJECT_STATUSES = ["planning", "in_progress", "on_hold", "completed", "cancelled"] as const;
export type ProjectStatus = typeof PROJECT_STATUSES[number];

export const projects = pgTable("projects", {
  id: serial("id").primaryKey(),
  name: text("name").notNull(),
  description: text("description"),
  color: text("color").default("#6366F1"), // Default indigo
  status: text("status").default("planning"), // planning, in_progress, on_hold, completed, cancelled
  courseName: text("course_name"), // Optional: associate with a course
  startDate: timestamp("start_date"),
  targetDate: timestamp("target_date"), // Target completion date
  completedAt: timestamp("completed_at"),
  priority: text("priority").default("medium"), // low, medium, high
  notes: text("notes"),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

export const insertProjectSchema = createInsertSchema(projects).omit({ id: true, createdAt: true, updatedAt: true });
export type Project = typeof projects.$inferSelect;
export type InsertProject = z.infer<typeof insertProjectSchema>;

// Subtasks table - child items of tasks with full scheduling support
export const subtasks = pgTable("subtasks", {
  id: serial("id").primaryKey(),
  parentTaskId: integer("parent_task_id").notNull(), // Links to tasks.id
  parentSubtaskId: integer("parent_subtask_id"), // Optional: for nested subtasks (hierarchy)
  title: text("title").notNull(),
  description: text("description"),
  startDate: timestamp("start_date"), // When to start working on subtask
  dueDate: timestamp("due_date"), // When subtask is due
  eventStartTime: text("event_start_time"), // Time slot start
  eventEndTime: text("event_end_time"), // Time slot end
  weekNumber: integer("week_number"), // Derived from dueDate
  isCompleted: boolean("is_completed").default(false),
  completedAt: timestamp("completed_at"),
  priority: text("priority").default("medium"), // low, medium, high
  notes: text("notes"),
  referenceLink: text("reference_link"),
  attachments: text("attachments").array(),
  position: integer("position").default(0), // For ordering subtasks
  createdAt: timestamp("created_at").defaultNow(),
});

export const insertSubtaskSchema = createInsertSchema(subtasks).omit({ id: true, createdAt: true });
export type Subtask = typeof subtasks.$inferSelect;
export type InsertSubtask = z.infer<typeof insertSubtaskSchema>;

// Task links table - dependencies and relationships between tasks/subtasks
export const LINK_TYPES = ["blocks", "blocked_by", "relates_to", "parent_of", "child_of"] as const;
export type LinkType = typeof LINK_TYPES[number];

export const taskLinks = pgTable("task_links", {
  id: serial("id").primaryKey(),
  sourceType: text("source_type").notNull(), // "task" or "subtask"
  sourceId: integer("source_id").notNull(),
  targetType: text("target_type").notNull(), // "task" or "subtask"
  targetId: integer("target_id").notNull(),
  linkType: text("link_type").notNull().default("relates_to"), // blocks, blocked_by, relates_to, etc.
  createdAt: timestamp("created_at").defaultNow(),
});

```

```

export const insertTaskLinkSchema = createInsertSchema(taskLinks).omit({ id: true, createdAt: true });
export type TaskLink = typeof taskLinks.$inferSelect;
export type InsertTaskLink = z.infer<typeof insertTaskLinkSchema>;

export const tasks = pgTable("tasks", {
  id: serial("id").primaryKey(),
  title: text("title").notNull(),
  description: text("description"),
  type: text("type").notNull(), // reading, module, essay, project, discussion, poll, exam, quiz
  courseName: text("course_name"),
  projectId: integer("project_id"), // Optional: link task to a project
  startDate: timestamp("start_date"), // Optional: when to start working on the task (planning/prep period)
  dueDate: timestamp("due_date").notNull(),
  eventStartTime: text("event_start_time"), // Time the task/event starts (e.g., "09:00")
  eventEndTime: text("event_end_time"), // Time the task/event ends (e.g., "10:00")
  reminder1: integer("reminder_1").default(30), // Default: 30 minutes before
  reminder2: integer("reminder_2").default(120), // Default: 2 hours before
  reminder3: integer("reminder_3"), // Optional additional reminder
  reminder4: integer("reminder_4"), // Optional additional reminder
  weekNumber: integer("week_number").notNull(), // 2-13
  isCompleted: boolean("is_completed").default(false),
  completedAt: timestamp("completed_at"), // When the task was marked as completed
  isMissed: boolean("is_missed").default(false),
  priority: text("priority").default("medium"), // low, medium, high
  notes: text("notes"),
  referenceLink: text("reference_link"), // URL reference for the task
  attachments: text("attachments").array(), // Array of attachment URLs/paths
  calendarEventId: text("calendar_event_id"), // For synced calendar events (due date)
  calendarProvider: text("calendar_provider"), // 'google' or 'outlook'
  prepCalendarEventId: text("prep_calendar_event_id"), // For synced prep/start date events
  secondaryCalendarEventId: text("secondary_calendar_event_id"), // For synced secondary calendar events
  secondAccountCalendarEventId: text("second_account_calendar_event_id"), // For second Google account sync
  secondAccountPrepEventId: text("second_account_prep_event_id"), // For second Google account prep events
  repeatType: text("repeat_type").default("none"), // none, daily, weekly, monthly, custom
  repeatInterval: integer("repeat_interval"), // For custom: every X units
  repeatIntervalUnit: text("repeat_interval_unit"), // days or weeks (for custom)
  repeatEndDate: timestamp("repeat_end_date"), // Optional: when to stop repeating
  parentTaskId: integer("parent_task_id"), // Links repeated instances to original task
  gradeWeight: doublePrecision("grade_weight"), // Percentage of final grade (e.g., 25 = 25%, 0.4 = 0.4%)
  gradeValue: doublePrecision("grade_value"), // Score achieved (e.g., 4.45 out of 7.5)
  gradeTotal: doublePrecision("grade_total"), // Total possible points (e.g., 7.5)
  assignmentGroup: text("assignment_group"), // Group name for organizing assignments
  sortOrder: integer("sort_order").default(0), // Order within group or list
  isAcknowledged: boolean("is_acknowledged").default(true), // For reminders: false until user acknowledges
  excludeFromGpa: boolean("exclude_from_gpa").default(true),
  inviteEmails: text("invite_emails").array(),
  hideFromSummary: boolean("hide_from_summary").default(false),
  flagged: boolean("flagged").default(false),
});

// Base schema from drizzle, then override date fields to accept ISO strings
const baseInsertSchema = createInsertSchema(tasks).omit({ id: true });

// Helper to validate and transform date strings
const dateStringToDate = z.string().min(1).transform((s, ctx) => {
  const date = new Date(s);
  if (isNaN(date.getTime())) {
    ctx.addIssue({
      code: z.ZodIssueCode.custom,
      message: "Invalid date format",
    });
    return z.NEVER;
  }
  return date;
});

export const insertTaskSchema = baseInsertSchema.extend({
  dueDate: z.union([z.date(), dateStringToDate]),
  startDate: z.union([z.date(), dateStringToDate]).optional().nullable(),
  repeatEndDate: z.union([z.date(), dateStringToDate]).optional().nullable(),
  repeatType: z.enum(REPEAT_TYPES).optional().default("none"),
  repeatIntervalUnit: z.enum(REPEAT_INTERVAL_UNITS).optional().nullable(),
});

export type Task = typeof tasks.$inferSelect;
export type InsertTask = z.infer<typeof insertTaskSchema>;
export type UpdateTaskRequest = Partial<InsertTask>;

// Sticky notes table for post-it notes that can be placed anywhere on screen
export const stickyNotes = pgTable("sticky_notes", {
  id: serial("id").primaryKey(),
  title: text("title").notNull().default("Note"),
  content: text("content").notNull().default(""),
  color: text("color").notNull().default("yellow"),
  customColor: text("custom_color"),
  positionX: integer("position_x").notNull().default(100),
  positionY: integer("position_y").notNull().default(100),
  width: integer("width").notNull().default(200),
  height: integer("height").notNull().default(150),
  zIndex: integer("z_index").notNull().default(100),
  isMinimized: boolean("is_minimized").notNull().default(false),
  taskId: integer("task_id"),
  projectId: integer("project_id"),
  reminderTime: timestamp("reminder_time"),
  reminderAlarm: boolean("reminder_alarm").notNull().default(false),
  reminderEmail: boolean("reminder_email").notNull().default(false),
  reminderPush: boolean("reminder_push").notNull().default(false),
  homePositionX: integer("home_position_x"),
  homePositionY: integer("home_position_y"),
  lastMovedAt: timestamp("last_moved_at"),
  isDeleted: boolean("is_deleted").notNull().default(false),
  deletedAt: timestamp("deleted_at"),
  createdAt: timestamp("created_at").defaultNow(),
});

export const insertStickyNoteSchema = createInsertSchema(stickyNotes).omit({ id: true, createdAt: true });
export type InsertStickyNote = z.infer<typeof insertStickyNoteSchema>;

```

```

export type StickyNote = typeof stickyNotes.$inferSelect;

// Temporary access tokens for time-limited sharing
export const accessTokens = pgTable("access_tokens", {
  id: serial("id").primaryKey(),
  token: text("token").notNull().unique(),
  name: text("name"), // Optional name/description for the token
  firstUsedAt: timestamp("first_used_at"), // When someone first accessed with this token
  expiresAt: timestamp("expires_at"), // Set to 1 hour after first use
  isRevoked: boolean("is_revoked").default(false),
  createdAt: timestamp("created_at").defaultNow(),
});

export const insertAccessTokenSchema = createInsertSchema(accessTokens).omit({ id: true, createdAt: true });
export type InsertAccessToken = z.infer<typeof insertAccessTokenSchema>;
export type AccessToken = typeof accessTokens.$inferSelect;

// Week calculation helpers
// Week 2 starts Saturday Jan 11, 2025 (today is Jan 17, 2025 - Friday of Week 2)
// Weeks run Saturday to Friday
export const SEMESTER_START = new Date("2026-01-10T12:00:00"); // Week 1 Saturday (noon to avoid timezone shifts)
export const FIRST_WEEK = 1;
export const LAST_WEEK = 13;

export function alignToSaturday(d: Date): Date {
  const date = new Date(d);
  date.setHours(12, 0, 0, 0);
  const day = date.getDay();
  const diff = day === 6 ? 0 : -(day + 1);
  date.setDate(date.getDate() + diff);
  date.setHours(12, 0, 0, 0);
  return date;
}

export function getWeekNumber(date: Date, customSemesterStart?: Date, readingWeekStart?: Date | null): number {
  const rawStart = new Date(customSemesterStart || SEMESTER_START);
  const startOfSemester = alignToSaturday(rawStart);
  const normalizedDate = new Date(date);
  normalizedDate.setHours(12, 0, 0, 0);
  const diffTime = normalizedDate.getTime() - startOfSemester.getTime();
  const diffDays = Math.round(diffTime / (1000 * 60 * 60 * 24));
  let weekNum = Math.floor(diffDays / 7) + 1;
  if (readingWeekStart) {
    const rwStart = new Date(readingWeekStart);
    rwStart.setHours(12, 0, 0, 0);
    const rwDiff = rwStart.getTime() - startOfSemester.getTime();
    const rwWeek = Math.floor(Math.round(rwDiff / (1000 * 60 * 60 * 24)) / 7) + 1;
    if (weekNum >= rwWeek) {
      const rwEnd = new Date(rwStart);
      rwEnd.setDate(rwEnd.getDate() + 7);
      if (normalizedDate.getTime() >= rwStart.getTime() && normalizedDate.getTime() < rwEnd.getTime()) {
        return -1;
      }
    }
    if (weekNum > rwWeek) {
      weekNum -= 1;
    }
  }
  return weekNum;
}

export function getWeekDates(weekNum: number, customSemesterStart?: Date, readingWeekStart?: Date | null): { start: D...
  const startOfSemester = alignToSaturday(new Date(customSemesterStart || SEMESTER_START));
  const msPerDay = 24 * 60 * 60 * 1000;
  let calendarWeek = weekNum;
  if (readingWeekStart) {
    const rwStart = new Date(readingWeekStart);
    rwStart.setHours(12, 0, 0, 0);
    const rwDiff = rwStart.getTime() - startOfSemester.getTime();
    const rwWeek = Math.floor(Math.round(rwDiff / (1000 * 60 * 60 * 24)) / 7) + 1;
    if (weekNum >= rwWeek) {
      calendarWeek = weekNum + 1;
    }
  }
  const weekStartMs = startOfSemester.getTime() + (calendarWeek - 1) * 7 * msPerDay;
  const weekStart = new Date(weekStartMs);
  const weekEnd = new Date(weekStartMs + 6 * msPerDay);
  return { start: weekStart, end: weekEnd };
}

export const appState = pgTable("app_state", {
  key: text("key").primaryKey(),
  value: text("value").notNull(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

export const SHIFT_TYPES = ["day", "night", "off"] as const;
export type ShiftType = typeof SHIFT_TYPES[number];

export const shiftSchedule = pgTable("shift_schedule", {
  id: serial("id").primaryKey(),
  date: text("date").notNull().unique(),
  shiftType: text("shift_type").notNull().default("off"),
});

export const insertShiftScheduleSchema = createInsertSchema(shiftSchedule).omit({ id: true });
export type InsertShiftScheduleEntry = typeof shiftSchedule.$inferSelect;
export type ShiftScheduleEntry = z.infer<typeof insertShiftScheduleSchema>;

export const semesterChecklist = pgTable("semester_checklist", {
  id: serial("id").primaryKey(),
  semesterSettingsId: integer("semester_settings_id").notNull(),
  courseCode: text("course_code").notNull(),
  itemType: text("item_type").notNull(),
  isChecked: boolean("is_checked").default(false),
  checkedAt: timestamp("checked_at"),
});

```

```

export const insertSemesterChecklistSchema = createInsertSchema(semesterChecklist).omit({ id: true, checkedAt: true });
export type SemesterChecklistItem = typeof semesterChecklist.$inferSelect;
export type InsertSemesterChecklistItem = z.infer<typeof insertSemesterChecklistSchema>;

export const courseWeekMappings = pgTable("course_week_mappings", {
  id: serial("id").primaryKey(),
  courseCode: text("course_code").notNull(),
  semesterSettingsId: integer("semester_settings_id").notNull(),
  weekNumber: integer("week_number").notNull(),
  confirmed: boolean("confirmed").default(false),
  courseWeekLabel: text("course_week_label"),
  notes: text("notes"),
});

export const insertCourseWeekMappingSchema = createInsertSchema(courseWeekMappings).omit({ id: true });
export type CourseWeekMapping = typeof courseWeekMappings.$inferSelect;
export type InsertCourseWeekMapping = z.infer<typeof insertCourseWeekMappingSchema>;

export const scholarships = pgTable("scholarships", {
  id: serial("id").primaryKey(),
  name: text("name").notNull(),
  organization: text("organization").notNull(),
  amount: text("amount"),
  applicationsOpen: text("applications_open"),
  deadline: text("deadline"),
  winnersAnnounced: text("winners_announced"),
  applicationUrl: text("application_url"),
  contactInfo: text("contact_info"),
  additionalInfo: text("additional_info"),
  status: text("status").default("not_started"),
});

export const insertScholarshipSchema = createInsertSchema(scholarships).omit({ id: true });
export type Scholarship = typeof scholarships.$inferSelect;
export type InsertScholarship = z.infer<typeof insertScholarshipSchema>;

export const keyContacts = pgTable("key_contacts", {
  id: serial("id").primaryKey(),
  name: text("name").notNull(),
  title: text("title"),
  organization: text("organization"),
  department: text("department"),
  email: text("email"),
  phone: text("phone"),
  office: text("office"),
  category: text("category").default("other"),
  notes: text("notes"),
});

export const insertKeyContactSchema = createInsertSchema(keyContacts).omit({ id: true });
export type KeyContact = typeof keyContacts.$inferSelect;
export type InsertKeyContact = z.infer<typeof insertKeyContactSchema>;

export const degreeTrackingData = pgTable("degree_tracking_data", {
  id: serial("id").primaryKey(),
  key: text("key").notNull().unique(),
  value: text("value").notNull(),
});

export const feedbackNotes = pgTable("feedback_notes", {
  id: serial("id").primaryKey(),
  note: text("note").notNull(),
  status: text("status").default("pending"),
  createdAt: timestamp("created_at").defaultNow(),
});

export const insertFeedbackNoteSchema = createInsertSchema(feedbackNotes).omit({ id: true, status: true, createdAt: true });
export type FeedbackNote = typeof feedbackNotes.$inferSelect;
export type InsertFeedbackNote = z.infer<typeof insertFeedbackNoteSchema>;

export const entityComments = pgTable("entity_comments", {
  id: serial("id").primaryKey(),
  entityType: text("entity_type").notNull(),
  entityId: text("entity_id").notNull(),
  content: text("content").notNull(),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

export const insertEntityCommentSchema = createInsertSchema(entityComments).omit({ id: true, createdAt: true, updatedAt: true });
export type EntityComment = typeof entityComments.$inferSelect;
export type InsertEntityComment = z.infer<typeof insertEntityCommentSchema>;

export const announcements = pgTable("announcements", {
  id: serial("id").primaryKey(),
  emailId: text("email_id").notNull(),
  subject: text("subject").notNull(),
  body: text("body").notNull(),
  snippet: text("snippet"),
  courseName: text("course_name"),
  receivedAt: timestamp("received_at").defaultNow(),
  sortOrder: integer("sort_order").default(0),
});

export const insertAnnouncementSchema = createInsertSchema(announcements).omit({ id: true });
export type Announcement = typeof announcements.$inferSelect;
export type InsertAnnouncement = z.infer<typeof insertAnnouncementSchema>;

export const PENDING_REVIEW_SOURCES = ["outlook_calendar", "gmail_task"] as const;
export type PendingReviewSource = typeof PENDING_REVIEW_SOURCES[number];

export const PENDING_REVIEW_STATUSES = ["pending", "accepted", "rejected"] as const;
export type PendingReviewStatus = typeof PENDING_REVIEW_STATUSES[number];

export const pendingReviewItems = pgTable("pending_review_items", {
  id: serial("id").primaryKey(),
  source: text("source").notNull(),
  sourceEmail: text("source_email"),
});

```



```

externalId: text("external_id"),
title: text("title").notNull(),
description: text("description"),
startDate: timestamp("start_date"),
endDate: timestamp("end_date"),
eventStartTime: text("event_start_time"),
eventEndTime: text("event_end_time"),
location: text("location"),
rawData: text("raw_data"),
status: text("status").notNull().default("pending"),
courseName: text("course_name"),
taskType: text("task_type").default("meeting"),
createdAt: timestamp("created_at").defaultNow(),
});

export const insertPendingReviewItemSchema = createInsertSchema(pendingReviewItems).omit({ id: true, createdAt: true });
export type PendingReviewItem = typeof pendingReviewItems.$inferSelect;
export type InsertPendingReviewItem = z.infer<typeof insertPendingReviewItemSchema>;

// Export chat models for AI integrations
export * from "./models/chat";

```

`shared/semesterUtils.ts`

```

// Semester date calculation utilities
// These compute standard academic semester date ranges

export type SemesterType = "fall" | "winter" | "spring_summer";
export type SpringSummerTerm = "full" | "first_half" | "second_half";

function getNthWeekdayOfMonth(year: number, month: number, weekday: number, n: number): Date {
  const firstDay = new Date(year, month, 1);
  let day = firstDay.getDay();
  let diff = weekday - day;
  if (diff < 0) diff += 7;
  const firstOccurrence = 1 + diff;
  const nthOccurrence = firstOccurrence + (n - 1) * 7;
  return new Date(year, month, nthOccurrence);
}

function getLastWeekdayOfMonth(year: number, month: number, weekday: number): Date {
  const lastDay = new Date(year, month + 1, 0);
  let day = lastDay.getDay();
  let diff = day - weekday;
  if (diff < 0) diff += 7;
  return new Date(year, month, lastDay.getDate() - diff);
}

function getSecondToLastWeekdayOfMonth(year: number, month: number, weekday: number): Date {
  const last = getLastWeekdayOfMonth(year, month, weekday);
  return new Date(year, month, last.getDate() - 7);
}

// Fall: Second Monday in September to second Friday in December
export function getFallDates(year: number): { start: Date; end: Date } {
  return {
    start: getNthWeekdayOfMonth(year, 8, 1, 2), // 2nd Monday of September (month 8)
    end: getNthWeekdayOfMonth(year, 11, 5, 2), // 2nd Friday of December (month 11)
  };
}

// Winter: Second Monday in January to second Monday in April
export function getWinterDates(year: number): { start: Date; end: Date } {
  return {
    start: getNthWeekdayOfMonth(year, 0, 1, 2), // 2nd Monday of January
    end: getNthWeekdayOfMonth(year, 3, 1, 2), // 2nd Monday of April
  };
}

// Spring/Summer: First Monday in May to first Friday in August (full semester)
export function getSpringSummerDates(year: number, term: SpringSummerTerm = "full"): { start: Date; end: Date } {
  const fullStart = getNthWeekdayOfMonth(year, 4, 1, 1); // 1st Monday of May
  const fullEnd = getNthWeekdayOfMonth(year, 7, 5, 1); // 1st Friday of August

  if (term === "full") {
    return { start: fullStart, end: fullEnd };
  }

  if (term === "first_half") {
    // First Monday in May to second-to-last Friday in June
    return {
      start: fullStart,
      end: getSecondToLastWeekdayOfMonth(year, 5, 5), // 2nd-to-last Friday of June
    };
  }

  // second_half: Last Monday in June to first Friday in August
  return {
    start: getLastWeekdayOfMonth(year, 5, 1), // Last Monday of June
    end: fullEnd,
  };
}

export function getSemesterDates(type: SemesterType, year: number, springSummerTerm?: SpringSummerTerm): { start: Dat...
  switch (type) {
    case "fall":
      return getFallDates(year);
    case "winter":
      return getWinterDates(year);
    case "spring_summer":
      return getSpringSummerDates(year, springSummerTerm || "full");
  }
}

```

```

export function parseSemesterLabel(label: string): { type: SemesterType; year: number } | null {
  const match = label.match(/^(winter|spring|summer|fall)\s+(\d{4})$/);
  if (!match) return null;
  const typeMap: Record<string, SemesterType> = {
    "winter": "winter",
    "spring/summer": "spring_summer",
    "fall": "fall",
  };
  return { type: typeMap[match[1]], year: parseInt(match[2]) };
}

export function generateSemesterOptions(startYear: number = 2026, endYear: number = 2029): string[] {
  const options: string[] = [];
  for (let year = startYear; year <= endYear; year++) {
    options.push(`winter ${year}`);
    options.push(`spring/summer ${year}`);
    options.push(`fall ${year}`);
  }
  return options;
}

export interface FutureSemesterDates {
  label: string;
  type: SemesterType;
  startDate: string;
  endDate: string;
  breakStart: string;
  breakEnd: string;
  confirmMonth: string;
}

export const FUTURE_SEMESTER_SCHEDULE: FutureSemesterDates[] = [
  { label: "Fall 2026", type: "fall", startDate: "2026-09-14", endDate: "2026-12-07", breakStart: "2026-10-12", break...
  { label: "Winter 2027", type: "winter", startDate: "2027-01-11", endDate: "2027-04-09", breakStart: "2027-02-15", b...
  { label: "Spring/Summer 2027", type: "spring_summer", startDate: "2027-05-03", endDate: "2027-08-13", breakStart: "...
  { label: "Fall 2027", type: "fall", startDate: "2027-09-13", endDate: "2027-12-06", breakStart: "2027-10-11", break...
  { label: "Winter 2028", type: "winter", startDate: "2028-01-10", endDate: "2028-04-07", breakStart: "2028-02-14", b...
  { label: "Spring/Summer 2028", type: "spring_summer", startDate: "2028-05-01", endDate: "2028-08-04", breakStart: "...
  { label: "Fall 2028", type: "fall", startDate: "2028-09-11", endDate: "2028-12-04", breakStart: "2028-10-09", break...
  { label: "Winter 2029", type: "winter", startDate: "2029-01-15", endDate: "2029-04-13", breakStart: "2029-02-12", b...
];

export function getUpcomingSemesterToConfirm(): FutureSemesterDates | null {
  const now = new Date(new Date().toLocaleString('en-US', { timeZone: 'America/Toronto' }));
  const currentYM = `${now.getFullYear()}-${String(now.getMonth() + 1).padStart(2, '0')}`;
  return FUTURE_SEMESTER_SCHEDULE.find(s => s.confirmMonth === currentYM) || null;
}

export function getNextSemesterByStartDate(): FutureSemesterDates | null {
  const now = new Date(new Date().toLocaleString('en-US', { timeZone: 'America/Toronto' }));
  const todayStr = `${now.getFullYear()}-${String(now.getMonth() + 1).padStart(2, '0')}-${String(now.getDate()).padSt...
  return FUTURE_SEMESTER_SCHEDULE.find(s => s.startDate === todayStr) || null;
}

export function formatDate(date: Date): string {
  return date.toISOString().split('T')[0];
}

export function dayOfWeekToNumber(day: string): number {
  const map: Record<string, number> = {
    sunday: 0, monday: 1, tuesday: 2, wednesday: 3,
    thursday: 4, friday: 5, saturday: 6,
  };
  return map[day.toLowerCase()] ?? 1;
}

```

`package.json`

```

{
  "name": "rest-express",
  "version": "1.0.0",
  "type": "module",
  "license": "MIT",
  "scripts": {
    "dev": "NODE_ENV=development tsx server/index.ts",
    "build": "tsx script/build.ts",
    "start": "NODE_ENV=production node dist/index.cjs",
    "check": "tsc",
    "db:push": "drizzle-kit push"
  },
  "dependencies": {
    "@google-cloud/storage": "^7.18.0",
    "@hookform/resolvers": "^3.10.0",
    "@jridgewell/trace-mapping": "^0.3.25",
    "@microsoft/microsoft-graph-client": "^3.0.7",
    "@radix-ui/react-accordion": "^1.2.4",
    "@radix-ui/react-alert-dialog": "^1.1.7",
    "@radix-ui/react-aspect-ratio": "^1.1.3",
    "@radix-ui/react-avatar": "^1.1.4",
    "@radix-ui/react-checkbox": "^1.1.5",
    "@radix-ui/react-collapsible": "^1.1.4",
    "@radix-ui/react-context-menu": "^2.2.7",
    "@radix-ui/react-dialog": "^1.1.7",
    "@radix-ui/react-dropdown-menu": "^2.1.7",
    "@radix-ui/react-hover-card": "^1.1.7",
    "@radix-ui/react-label": "^2.1.3",
    "@radix-ui/react-menubar": "^1.1.7",
    "@radix-ui/react-navigation-menu": "^1.2.6",
    "@radix-ui/react-popover": "^1.1.7",
    "@radix-ui/react-progress": "^1.1.3",
    "@radix-ui/react-radio-group": "^1.2.4",
    "@radix-ui/react-scroll-area": "^1.2.4",
    "@radix-ui/react-select": "^2.1.7",
    "@radix-ui/react-separator": "^1.1.3",
  }
}

```

```

"@radix-ui/react-slider": "^1.2.4",
"@radix-ui/react-slot": "^1.2.0",
"@radix-ui/react-switch": "^1.1.4",
"@radix-ui/react-tabs": "^1.1.4",
"@radix-ui/react-toast": "^1.2.7",
"@radix-ui/react-toggle": "^1.1.3",
"@radix-ui/react-toggle-group": "^1.1.3",
"@radix-ui/react-tooltip": "^1.2.0",
"@react-pdf/renderer": "^4.3.2",
"@spotify/web-api-ts-sdk": "^1.2.0",
"@tanstack/react-query": "^5.60.5",
"@types/cookie-parser": "^1.4.10",
"@uppy/aws-s3": "^5.1.0",
"@uppy/core": "^5.2.0",
"@uppy/dashboard": "^5.1.0",
"@uppy/react": "^5.1.1",
"@vitejs/plugin-legacy": "^7.2.1",
"class-variance-authority": "^0.7.1",
"clsx": "^2.1.1",
"cmdk": "^1.1.1",
"connect-pg-simple": "^10.0.0",
"cookie-parser": "^1.4.7",
"date-fns": "^3.6.0",
"date-fns-tz": "^3.2.0",
"drizzle-orm": "^0.39.3",
"drizzle-zod": "^0.7.1",
"edge-tts": "^1.0.1",
"embla-carousel-react": "^8.6.0",
"express": "^5.0.1",
"express-session": "^1.18.1",
"fast-xml-parser": "^4.5.4",
"framer-motion": "^11.18.2",
"google-auth-library": "^10.5.0",
"googleapis": "^148.0.0",
"input-otp": "^1.4.2",
"lodash": "^4.17.23",
"lucide-react": "^0.453.0",
"memorystore": "^1.6.7",
"minimatch": "^9.0.7",
"next-themes": "^0.4.6",
"node-edge-tts": "^1.2.10",
"openai": "^6.17.0",
"p-limit": "^7.2.0",
"p-retry": "^7.1.1",
"passport": "^0.7.0",
"passport-local": "^1.0.0",
"pdf-lib": "^1.17.1",
"pdf-parse": "^2.4.5",
"pdfkit": "^0.18.0",
"pg": "^8.16.3",
"qs": "^6.15.0",
"react": "^18.3.1",
"react-day-picker": "^8.10.1",
"react-dom": "^18.3.1",
"react-easy-crop": "^5.5.6",
"react-hook-form": "^7.55.0",
"react-icons": "^5.4.0",
"react-pdf": "^10.3.0",
"react-resizable-panels": "^2.1.7",
"recharts": "^2.15.2",
"resend": "^6.9.4",
"rollup": "^4.59.0",
"tailwind-merge": "^2.6.0",
"tailwindcss-animate": "^1.0.7",
"tw-animate-css": "^1.2.5",
"vaul": "^1.1.2",
"wouter": "^3.3.5",
"ws": "^8.18.0",
"zod": "^3.25.76",
"zod-validation-error": "^3.5.4"
},
"devDependencies": {
  "@replit/vite-plugin-cartographer": "^0.4.4",
  "@replit/vite-plugin-dev-banner": "^0.1.1",
  "@replit/vite-plugin-runtime-error-modal": "^0.0.3",
  "@tailwindcss/typography": "^0.5.15",
  "@tailwindcss/vite": "^4.1.18",
  "@types/connect-pg-simple": "^7.0.3",
  "@types/express": "^5.0.0",
  "@types/express-session": "^1.18.0",
  "@types/node": "^20.19.27",
  "@types/passport": "^1.0.16",
  "@types/passport-local": "^1.0.38",
  "@types/react": "^18.3.11",
  "@types/react-dom": "^18.3.1",
  "@types/ws": "^8.5.13",
  "@vitejs/plugin-react": "^4.7.0",
  "autoprefixer": "^10.4.20",
  "drizzle-kit": "^0.31.8",
  "esbuild": "^0.25.0",
  "postcss": "^8.4.47",
  "tailwindcss": "^3.4.17",
  "tsx": "^4.20.5",
  "typescript": "5.6.3",
  "vite": "^7.3.0"
},
"overrides": {
  "drizzle-kit": {
    "@esbuild-kit/esm-loader": "npm:tsx@^4.20.4"
  }
},
"optionalDependencies": {
  "bufferutil": "^4.0.8"
}
}

```

tsconfig.json

```
{
  "include": ["client/src/**/*", "shared/**/*", "server/**/*"],
  "exclude": ["node_modules", "build", "dist", "**/*.test.ts"],
  "compilerOptions": {
    "incremental": true,
    "tsBuildInfoFile": "./node_modules/typescript/tsbuildinfo",
    "noEmit": true,
    "module": "ESNext",
    "strict": true,
    "lib": ["esnext", "dom", "dom.iterable"],
    "jsx": "preserve",
    "esModuleInterop": true,
    "skipLibCheck": true,
    "allowImportingTsExtensions": true,
    "moduleResolution": "bundler",
    "baseUrl": ".",
    "types": ["node", "vite/client"],
    "paths": {
      "@/*": [". /client/src/*"],
      "@shared/*": [". /shared/*"]
    }
  }
}
```

`vite.config.ts`

```
import { defineConfig, Plugin } from "vite";
import react from "@vitejs/plugin-react";
import path from "path";
import runtimeErrorOverlay from "@replit/vite-plugin-runtime-error-modal";

function babelLetConstToVar(): Plugin {
  return {
    name: 'babel-let-const-to-var',
    apply: 'build',
    enforce: 'post',
    async renderChunk(code) {
      const babel = await import("@babel/core");
      const result = babel.transformSync(code, {
        configFile: false,
        babelrc: false,
        compact: true,
        plugins: ['@babel/plugin-transform-block-scoping'],
        sourceType: 'module',
      });
      if (result?.code) {
        return { code: result.code.replace(/blet /g, 'var ').replace(/bconst /g, 'var '), map: null };
      }
      return null;
    },
  };
}

export default defineConfig({
  plugins: [
    react(),
    babelLetConstToVar(),
    ...(process.env.NODE_ENV !== "production" &&
    process.env.REPL_ID !== undefined
    ? [
      await import("@replit/vite-plugin-cartographer").then((m) =>
        m.cartographer(),
      ),
      await import("@replit/vite-plugin-dev-banner").then((m) =>
        m.devBanner(),
      ),
    ]
    : []),
  ],
  resolve: {
    alias: {
      "@": path.resolve(import.meta.dirname, "client", "src"),
      "@shared": path.resolve(import.meta.dirname, "shared"),
      "@assets": path.resolve(import.meta.dirname, "attached_assets"),
      "react": "preact/compat",
      "react-dom": "preact/compat",
      "react-dom/client": "preact/compat",
      "react/jsx-runtime": "preact/jsx-runtime",
    },
  },
  root: path.resolve(import.meta.dirname, "client"),
  build: {
    outDir: path.resolve(import.meta.dirname, "dist/public"),
    emptyOutDir: true,
    target: false as any,
    minify: 'terser',
    terserOptions: {
      ecma: 5,
      compress: { ecma: 5 },
      format: { ecma: 5 },
    },
  },
  server: {
    fs: {
      strict: true,
      deny: ["**/*.ts"],
    },
  },
});
```

`drizzle.config.ts`

```
import { defineConfig } from "drizzle-kit";

if (!process.env.DATABASE_URL) {
  throw new Error("DATABASE_URL, ensure the database is provisioned");
}

export default defineConfig({
  out: "./migrations",
  schema: "./shared/schema.ts",
  dialect: "postgres",
  dbCredentials: {
    url: process.env.DATABASE_URL,
  },
});
```

`tailwind.config.ts`

```
import type { Config } from "tailwindcss";

export default {
  darkMode: ["class"],
  content: [
    "./client/index.html",
    "./client/src/**/*.{js,jsx,ts,tsx}",
  ],
  theme: {
    extend: {
      borderRadius: {
        lg: "5625rem", /* 9px */
        md: "375rem", /* 6px */
        sm: "1875rem", /* 3px */
      },
      colors: {
        // Flat / base colors (regular buttons)
        background: "hsl(var(--background) / <alpha-value>)",
        foreground: "hsl(var(--foreground) / <alpha-value>)",
        border: "hsl(var(--border) / <alpha-value>)",
        input: "hsl(var(--input) / <alpha-value>)",
        card: {
          DEFAULT: "hsl(var(--card) / <alpha-value>)",
          foreground: "hsl(var(--card-foreground) / <alpha-value>)",
          border: "hsl(var(--card-border) / <alpha-value>)",
        },
        popover: {
          DEFAULT: "hsl(var(--popover) / <alpha-value>)",
          foreground: "hsl(var(--popover-foreground) / <alpha-value>)",
          border: "hsl(var(--popover-border) / <alpha-value>)",
        },
        primary: {
          DEFAULT: "hsl(var(--primary) / <alpha-value>)",
          foreground: "hsl(var(--primary-foreground) / <alpha-value>)",
          border: "var(--primary-border)",
        },
        secondary: {
          DEFAULT: "hsl(var(--secondary) / <alpha-value>)",
          foreground: "hsl(var(--secondary-foreground) / <alpha-value>)",
          border: "var(--secondary-border)",
        },
        muted: {
          DEFAULT: "hsl(var(--muted) / <alpha-value>)",
          foreground: "hsl(var(--muted-foreground) / <alpha-value>)",
          border: "var(--muted-border)",
        },
        accent: {
          DEFAULT: "hsl(var(--accent) / <alpha-value>)",
          foreground: "hsl(var(--accent-foreground) / <alpha-value>)",
          border: "var(--accent-border)",
        },
        destructive: {
          DEFAULT: "hsl(var(--destructive) / <alpha-value>)",
          foreground: "hsl(var(--destructive-foreground) / <alpha-value>)",
          border: "var(--destructive-border)",
        },
        ring: "hsl(var(--ring) / <alpha-value>)",
        chart: {
          "1": "hsl(var(--chart-1) / <alpha-value>)",
          "2": "hsl(var(--chart-2) / <alpha-value>)",
          "3": "hsl(var(--chart-3) / <alpha-value>)",
          "4": "hsl(var(--chart-4) / <alpha-value>)",
          "5": "hsl(var(--chart-5) / <alpha-value>)",
        },
        sidebar: {
          ring: "hsl(var(--sidebar-ring) / <alpha-value>)",
          DEFAULT: "hsl(var(--sidebar) / <alpha-value>)",
          foreground: "hsl(var(--sidebar-foreground) / <alpha-value>)",
          border: "hsl(var(--sidebar-border) / <alpha-value>)",
        },
        "sidebar-primary": {
          DEFAULT: "hsl(var(--sidebar-primary) / <alpha-value>)",
          foreground: "hsl(var(--sidebar-primary-foreground) / <alpha-value>)",
          border: "var(--sidebar-primary-border)",
        },
        "sidebar-accent": {
          DEFAULT: "hsl(var(--sidebar-accent) / <alpha-value>)",
          foreground: "hsl(var(--sidebar-accent-foreground) / <alpha-value>)",
          border: "var(--sidebar-accent-border)",
        },
        status: {
          online: "rgb(34 197 94)",
          away: "rgb(245 158 11)",
          busy: "rgb(239 68 68)",
          offline: "rgb(156 163 175)",
        },
      },
    },
  },
  fontFamily: {
```

```
display: ['"Space Grotesk"', "sans-serif"],
sans: ['"Inter"', "sans-serif"],
serif: ["var(--font-serif)"],
mono: ["var(--font-mono)"],
},
keyframes: {
  "accordion-down": {
    from: { height: "0" },
    to: { height: "var(--radix-accordion-content-height)" },
  },
  "accordion-up": {
    from: { height: "var(--radix-accordion-content-height)" },
    to: { height: "0" },
  },
},
animation: {
  "accordion-down": "accordion-down 0.2s ease-out",
  "accordion-up": "accordion-up 0.2s ease-out",
},
},
},
plugins: [require("tailwindcss-animate"), require("@tailwindcss/typography")],
} satisfies Config;
```

`postcss.config.js`

```
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
}
```
